

Designing Data-Intensive Applications

Bản dịch song ngữ Anh–Việt · The Big Ideas Behind Reliable, Scalable, and
Maintainable Systems

Martin Kleppmann (nguyên tác)

2026

Mục lục

Chapter 1. Reliable, Scalable, and Maintainable Applications — Ứng dụng tin cậy, khả mở rộng và dễ bảo trì	7
Thinking About Data Systems — Suy nghĩ về các hệ thống dữ liệu	9
Reliability — Độ tin cậy	12
Hardware Faults — Lỗi phần cứng	13
Software Errors — Lỗi phần mềm	15
Human Errors — Lỗi do con người	16
How Important Is Reliability? — Độ tin cậy quan trọng đến mức nào?	17
Scalability — Khả năng mở rộng	19
Describing Load — Mô tả tải	19
Describing Performance — Mô tả hiệu năng	22
Latency and response time — Độ trễ và thời gian phản hồi	23
Approaches for Coping with Load — Các cách tiếp cận để ứng phó với tải	27
Maintainability — Khả năng bảo trì	30
Operability: Making Life Easy for Operations — Khả năng vận hành: Làm cho cuộc sống của bộ phận vận hành dễ thở hơn	31
Simplicity: Managing Complexity — Tính đơn giản: Quản lý độ phức tạp	32
Evolvability: Making Change Easy — Khả năng tiến hóa: Làm cho việc thay đổi trở nên dễ dàng	34
Summary — Tóm tắt	36
Chapter 2. Data Models and Query Languages — Mô hình dữ liệu và ngôn ngữ truy vấn	40
Relational Model Versus Document Model — Mô hình quan hệ so với mô hình tài liệu	43
The Birth of NoSQL — Sự ra đời của NoSQL	44
The Object-Relational Mismatch — Lệnh pha đối tượng–quan hệ	45
Many-to-One and Many-to-Many Relationships — Quan hệ nhiều-một và nhiều-nhiều	48
Are Document Databases Repeating History? — Phải chăng cơ sở dữ liệu tài liệu đang lặp lại lịch sử?	50
Relational Versus Document Databases Today — Cơ sở dữ liệu quan hệ so với cơ sở dữ liệu tài liệu ngày nay	54
Query Languages for Data — Ngôn ngữ truy vấn cho dữ liệu	61
Declarative Queries on the Web — Truy vấn khai báo trên Web	63

MapReduce Querying — Truy vấn bằng MapReduce	66
Graph-Like Data Models — Các mô hình dữ liệu dạng đồ thị	70
Property Graphs — Đồ thị thuộc tính	71
The Cypher Query Language — Ngôn ngữ truy vấn Cypher	74
Graph Queries in SQL — Truy vấn đồ thị bằng SQL	76
Triple-Stores and SPARQL — Kho bộ ba và SPARQL	78
The Foundation: Datalog — Nền tảng: Datalog	84
Summary — Tóm tắt	88
Chapter 3. Storage and Retrieval — Chương 3. Lưu trữ và Truy xuất	93
Data Structures That Power Your Database — Các cấu trúc dữ liệu tạo sức mạnh cho cơ sở dữ liệu của bạn	95
Hash Indexes — Chỉ mục băm	97
SSTables and LSM-Trees — SSTable và LSM-tree	101
B-Trees — B-tree	106
Comparing B-Trees and LSM-Trees — So sánh B-tree và LSM-tree	110
Other Indexing Structures — Các cấu trúc lập chỉ mục khác	113
Transaction Processing or Analytics? — Xử lý giao dịch hay Phân tích?	121
Data Warehousing — Kho dữ liệu	123
Stars and Snowflakes: Schemas for Analytics — Sao và Bông tuyết: Các lược đồ cho phân tích	125
Column-Oriented Storage — Lưu trữ theo cột	128
Column Compression — Nén cột	130
Column-oriented storage and column families — Lưu trữ hướng cột và họ cột	132
Sort Order in Column Storage — Thứ tự sắp xếp trong lưu trữ cột	133
Writing to Column-Oriented Storage — Ghi vào lưu trữ hướng cột	135
Aggregation: Data Cubes and Materialized Views — Tổng hợp: Khối dữ liệu và Khung nhìn cụ thể hóa	136
Summary — Tóm tắt	138
Chapter 4. Encoding and Evolution — Chương 4. Mã hóa và Tiến hóa	144
Formats for Encoding Data — Các định dạng mã hóa dữ liệu	147
Terminology clash — Xung đột thuật ngữ	148
Language-Specific Formats — Các định dạng đặc thù theo ngôn ngữ	148
JSON, XML, and Binary Variants — JSON, XML và các biến thể nhị phân	149
Thrift and Protocol Buffers — Thrift và Protocol Buffers	153
Avro — Avro	157
The Merits of Schemas — Những ưu điểm của lược đồ	164
Modes of Dataflow — Các phương thức luồng dữ liệu	166
Dataflow Through Databases — Luồng dữ liệu qua cơ sở dữ liệu	167
Dataflow Through Services: REST and RPC — Luồng dữ liệu qua dịch vụ: REST và RPC . . .	169
Message-Passing Dataflow — Luồng dữ liệu truyền thông điệp	177
Summary — Tóm tắt	182

Chapter 5. Replication — Chương 5. Sao chép dữ liệu	187
Leaders and Followers — Leader và follower	189
Synchronous Versus Asynchronous Replication — Sao chép đồng bộ so với bất đồng bộ . . .	190
Setting Up New Followers — Thiết lập follower mới	192
Handling Node Outages — Xử lý gián đoạn nút	194
Implementation of Replication Logs — Hiện thực của nhật ký sao chép	197
Problems with Replication Lag — Các vấn đề với độ trễ sao chép	202
Reading Your Own Writes — Đọc thấy bản ghi của chính mình	203
Monotonic Reads — Đọc đơn điệu	206
Consistent Prefix Reads — Đọc tiền tố nhất quán	207
Solutions for Replication Lag — Giải pháp cho độ trễ sao chép	208
Multi-Leader Replication — Sao chép nhiều-leader	210
Use Cases for Multi-Leader Replication — Các tình huống dùng sao chép nhiều-leader . . .	210
Handling Write Conflicts — Xử lý xung đột ghi	214
Multi-Leader Replication Topologies — Các tô-pô sao chép nhiều-leader	219
Leaderless Replication — Sao chép không-leader	222
Writing to the Database When a Node Is Down — Ghi vào cơ sở dữ liệu khi một nút đang ngừng hoạt động	223
Limitations of Quorum Consistency — Hạn chế của tính nhất quán nhóm tức số	226
Sloppy Quorums and Hinted Handoff — Nhóm tức số luộm thuộm và bàn giao gợi ý	229
Detecting Concurrent Writes — Phát hiện các thao tác ghi tương tranh	232
Version vectors and vector clocks — Vector phiên bản và đồng hồ vector	241
Summary — Tóm tắt	242
Chapter 6. Partitioning — Chương 6. Phân vùng	248
Terminological confusion — Sự rối rắm về thuật ngữ	249
Partitioning and Replication — Phân vùng và nhân bản	251
Partitioning of Key-Value Data — Phân vùng dữ liệu khóa-giá trị	252
Partitioning by Key Range — Phân vùng theo khoảng khóa	253
Partitioning by Hash of Key — Phân vùng theo hàm băm của khóa	254
Skewed Workloads and Relieving Hot Spots — Tải bị lệch và làm giảm điểm nóng	257
Partitioning and Secondary Indexes — Phân vùng và chỉ mục phụ	259
Partitioning Secondary Indexes by Document — Phân vùng chỉ mục phụ theo tài liệu	260
Partitioning Secondary Indexes by Term — Phân vùng chỉ mục phụ theo thuật ngữ	261
Rebalancing Partitions — Tái cân bằng các phân vùng	264
Strategies for Rebalancing — Các chiến lược tái cân bằng	265
Operations: Automatic or Manual Rebalancing — Vận hành: Tái cân bằng tự động hay thủ công	269
Request Routing — Định tuyến yêu cầu	271
Parallel Query Execution — Thực thi truy vấn song song	273

Summary — Tóm tắt	275
Chapter 7. Transactions — Chương 7. Giao dịch	279
The Slippery Concept of a Transaction — Khái niệm trơn trượt về giao dịch	282
The Meaning of ACID — Ý nghĩa của ACID	283
Single-Object and Multi-Object Operations — Thao tác đơn đối tượng và đa đối tượng	289
Weak Isolation Levels — Các mức cô lập yếu	295
Read Committed — Read Committed	296
Snapshot Isolation and Repeatable Read — Snapshot Isolation và Repeatable Read	300
Preventing Lost Updates — Ngăn chặn cập nhật bị mất	307
Write Skew and Phantoms — Lệnh khi ghi và Bóng ma	312
Serializability — Tính khả tuần tự hóa	319
Actual Serial Execution — Thực thi tuần tự thực sự	320
Two-Phase Locking (2PL) — Khóa hai pha (2PL)	326
2PL is not 2PC — 2PL không phải là 2PC	327
Serializable Snapshot Isolation (SSI) — Cô lập ảnh chụp khả tuần tự hóa (SSI)	333
Summary — Tóm tắt	340
Chapter 8. The Trouble with Distributed Systems — Những rắc rối với hệ thống phân tán	347
Faults and Partial Failures — Lỗi và hỏng hóc cục bộ	349
Cloud Computing and Supercomputing — Điện toán đám mây và siêu máy tính	350
Unreliable Networks — Mạng không đáng tin cậy	355
Network Faults in Practice — Sự cố mạng trong thực tế	356
Network partitions — Phân vùng mạng	358
Detecting Faults — Phát hiện lỗi	359
Timeouts and Unbounded Delays — Thời gian chờ và độ trễ không có giới hạn	360
Synchronous Versus Asynchronous Networks — Mạng đồng bộ so với mạng bất đồng bộ . .	364
Unreliable Clocks — Đồng hồ không đáng tin cậy	369
Monotonic Versus Time-of-Day Clocks — Đồng hồ đơn điệu so với đồng hồ thời gian trong ngày	370
Clock Synchronization and Accuracy — Đồng bộ đồng hồ và độ chính xác	372
Relying on Synchronized Clocks — Trông cậy vào các đồng hồ đã đồng bộ	374
Process Pauses — Tạm dừng tiến trình	381
Is real-time really real? — Thời gian thực có thực sự là thực không?	386
Knowledge, Truth, and Lies — Tri thức, sự thật và lời dối	389
The Truth Is Defined by the Majority — Sự thật được định nghĩa bởi đa số	390
Byzantine Faults — Lỗi Byzantine	395
System Model and Reality — Mô hình hệ thống và hiện thực	399
Summary — Tóm tắt	405
Chapter 9. Consistency and Consensus — Chương 9. Tính nhất quán và Đồng thuận	413

Consistency Guarantees — Các bảo đảm về tính nhất quán	416
Linearizability — Tính tuyến tính hóa	419
What Makes a System Linearizable? — Điều gì làm cho một hệ thống tuyến tính hóa?	420
Relying on Linearizability — Dựa vào tính tuyến tính hóa	425
Implementing Linearizable Systems — Hiện thực các hệ thống tuyến tính hóa	429
The Cost of Linearizability — Cái giá của tính tuyến tính hóa	432
Ordering Guarantees — Các bảo đảm về sắp thứ tự	437
Ordering and Causality — Sắp thứ tự và tính nhân quả	438
Sequence Number Ordering — Sắp thứ tự bằng số thứ tự	444
Total Order Broadcast — Phát toàn-thứ-tự	450
Scope of ordering guarantee — Phạm vi của bảo đảm sắp thứ tự	451
Distributed Transactions and Consensus — Giao dịch phân tán và Đồng thuận	457
Atomic Commit and Two-Phase Commit (2PC) — Commit nguyên tử và Commit hai pha (2PC)	459
Don't confuse 2PC and 2PL — Đừng nhầm 2PC với 2PL	463
Distributed Transactions in Practice — Giao dịch phân tán trong thực tế	468
Fault-Tolerant Consensus — Đồng thuận chịu lỗi	475
Membership and Coordination Services — Các dịch vụ thành viên và điều phối	483
Summary — Tóm tắt	489
Chapter 10. Batch Processing — Chương 10. Xử lý theo lô	500
Batch Processing with Unix Tools — Xử lý theo lô với các công cụ Unix	503
Simple Log Analysis — Phân tích nhật ký đơn giản	504
The Unix Philosophy — Triết lý Unix	507
MapReduce and Distributed Filesystems — MapReduce và hệ thống tệp phân tán	513
MapReduce Job Execution — Thực thi tác vụ MapReduce	515
Reduce-Side Joins and Grouping — Phép join và phép gom nhóm phía reduce	520
Map-Side Joins — Phép join phía map	527
The Output of Batch Workflows — Đầu ra của các luồng công việc theo lô	530
Comparing Hadoop to Distributed Databases — So sánh Hadoop với các cơ sở dữ liệu phân tán	537
Beyond MapReduce — Vượt ra ngoài MapReduce	544
Materialization of Intermediate State — Vật chất hóa trạng thái trung gian	545
Graphs and Iterative Processing — Đồ thị và xử lý lặp	551
High-Level APIs and Languages — Các API và ngôn ngữ ở mức cao	556
Summary — Tóm tắt	560
Chapter 11. Stream Processing — Chương 11. Xử lý luồng	568
Transmitting Event Streams — Truyền tải các luồng sự kiện	570
Messaging Systems — Các hệ thống thông điệp	571
Partitioned Logs — Nhật ký được phân mảnh	578
Databases and Streams — Cơ sở dữ liệu và luồng	586

Keeping Systems in Sync — Giữ các hệ thống đồng bộ	587
Change Data Capture — Nắm bắt thay đổi dữ liệu	589
Event Sourcing — Lấy nguồn từ sự kiện	593
State, Streams, and Immutability — Trạng thái, luồng, và tính bất biến	597
Processing Streams — Xử lý các luồng	605
Uses of Stream Processing — Các ứng dụng của xử lý luồng	606
Reasoning About Time — Lập luận về thời gian	612
Stream Joins — Các phép join luồng	617
Fault Tolerance — Khả năng chịu lỗi	623
Summary — Tóm tắt	628
Chapter 12. The Future of Data Systems — Chương 12. Tương lai của các hệ thống dữ liệu	636
Data Integration — Tích hợp dữ liệu	638
Combining Specialized Tools by Deriving Data — Kết hợp các công cụ chuyên biệt bằng cách dẫn xuất dữ liệu	639
Batch and Stream Processing — Xử lý theo lô và xử lý luồng	645
Unbundling Databases — Tháo gói cơ sở dữ liệu	652
Composing Data Storage Technologies — Kết cấu các công nghệ lưu trữ dữ liệu	653
Designing Applications Around Dataflow — Thiết kế ứng dụng xoay quanh luồng dữ liệu	660
Observing Derived State — Quan sát trạng thái dẫn xuất	667
Aiming for Correctness — Hướng tới sự đúng đắn	677
The End-to-End Argument for Databases — Lập luận đầu-cuối cho cơ sở dữ liệu	679
Enforcing Constraints — Thực thi các ràng buộc	685
Timeliness and Integrity — Tính kịp thời và tính toàn vẹn	690
Trust, but Verify — Hãy tin, nhưng phải kiểm chứng	697
Doing the Right Thing — Làm điều đúng đắn	705
Predictive Analytics — Phân tích dự báo	706
Privacy and Tracking — Quyền riêng tư và theo dõi	711
Summary — Tóm tắt	722

Chapter 1. Reliable, Scalable, and Maintainable Applications — Ứng dụng tin cậy, khả mở rộng và dễ bảo trì

The Internet was done so well that most people think of it as a natural resource like the Pacific Ocean, rather than something that was man-made. When was the last time a technology with a scale like that was so error-free?

Internet được làm tốt đến mức hầu hết mọi người xem nó như một tài nguyên thiên nhiên, giống như Thái Bình Dương, chứ không phải thứ do con người tạo ra. Lần gần nhất một công nghệ có quy mô như vậy mà gần như không có lỗi là khi nào?

Alan Kay, in interview with Dr Dobb's Journal (2012)

— Alan Kay, trong cuộc phỏng vấn với Dr Dobb's Journal (2012)

Many applications today are **data-intensive**, as opposed to **compute-intensive**. Raw CPU power is rarely a limiting factor for these applications—bigger problems are usually the amount of data, the complexity of data, and the speed at which it is changing.

Ngày nay, nhiều ứng dụng thiên về dữ liệu thay vì thiên về tính toán. Sức mạnh CPU thô hiếm khi là yếu tố giới hạn đối với những ứng dụng này — vấn đề lớn hơn thường là lượng dữ liệu, độ phức tạp của dữ liệu, và tốc độ thay đổi của nó.

A data-intensive application is typically built from standard building blocks that provide commonly needed functionality. For example, many applications need to:

Một ứng dụng thiên về dữ liệu thường được xây dựng từ những khối tiêu chuẩn cung cấp các chức năng thường dùng. Ví dụ, nhiều ứng dụng cần:

- Store data so that they, or another application, can find it again later (databases)
- Remember the result of an expensive operation, to speed up reads (caches)
- Allow users to search data by keyword or filter it in various ways (search indexes)
- Send a message to another process, to be handled asynchronously (**stream processing**)
- Periodically crunch a large amount of accumulated data (**batch processing**)
 - Lưu trữ dữ liệu để sau này chính nó, hoặc một ứng dụng khác, có thể tìm lại (cơ sở dữ liệu).
 - Ghi nhớ kết quả của một thao tác tốn kém, nhằm tăng tốc các lần đọc (bộ nhớ đệm).

- Cho phép người dùng tìm kiếm dữ liệu theo từ khóa hoặc lọc theo nhiều cách (chỉ mục tìm kiếm).
- Gửi một thông điệp tới tiến trình khác để xử lý bất đồng bộ (xử lý luồng).
- Định kỳ xử lý một lượng lớn dữ liệu đã tích lũy (xử lý theo lô).

If that sounds painfully obvious, that's just because these data systems are such a successful **abstraction**: we use them all the time without thinking too much. When building an application, most engineers wouldn't dream of writing a new data storage **engine** from scratch, because databases are a perfectly good tool for the job.

Nếu điều này nghe có vẻ hiển nhiên đến phát chán, đó chỉ vì các hệ thống dữ liệu này là một sự trừu tượng hóa quá thành công: ta dùng chúng suốt mà chẳng cần nghĩ ngợi nhiều. Khi xây dựng một ứng dụng, hầu hết kỹ sư sẽ không bao giờ nghĩ đến chuyện tự viết một engine lưu trữ dữ liệu mới từ đầu, bởi cơ sở dữ liệu vốn đã là công cụ quá tốt cho việc đó.

But reality is not that simple. There are many **database** systems with different characteristics, because different applications have different requirements. There are various approaches to caching, several ways of building search indexes, and so on. When building an application, we still need to figure out which tools and which approaches are the most appropriate for the task at hand. And it can be hard to combine tools when you need to do something that a single tool cannot do alone.

Nhưng thực tế không đơn giản như vậy. Có rất nhiều hệ cơ sở dữ liệu với những đặc tính khác nhau, bởi các ứng dụng khác nhau có những yêu cầu khác nhau. Có nhiều cách tiếp cận việc đệm, vài cách xây dựng chỉ mục tìm kiếm, v.v. Khi xây dựng một ứng dụng, ta vẫn phải tìm ra công cụ nào và cách tiếp cận nào là phù hợp nhất cho nhiệm vụ trước mắt. Và việc kết hợp nhiều công cụ có thể khó khăn khi bạn cần làm điều mà một công cụ đơn lẻ không thể tự làm được.

This book is a journey through both the principles and the practicalities of data systems, and how you can use them to build data-intensive applications. We will explore what different tools have in common, what distinguishes them, and how they achieve their characteristics.

Cuốn sách này là một hành trình đi qua cả nguyên lý lẫn thực tiễn của các hệ thống dữ liệu, và cách bạn dùng chúng để xây dựng những ứng dụng thiên về dữ liệu. Ta sẽ khám phá điểm chung giữa các công cụ khác nhau, điều gì phân biệt chúng, và bằng cách nào chúng đạt được những đặc tính của mình.

In this chapter, we will start by exploring the fundamentals of what we are trying to achieve: reliable, scalable, and maintainable data systems. We'll clarify what those things **mean**, outline some ways of thinking about them, and go over the basics that we will need for later chapters. In the following chapters we will continue layer by layer, looking at different design decisions that need to be considered when working on a data-intensive application.

Trong chương này, ta sẽ bắt đầu bằng việc tìm hiểu những nền tảng cơ bản của điều ta đang cố đạt được: các hệ thống dữ liệu tin cậy, khả mở rộng và dễ bảo trì. Ta sẽ làm rõ những khái niệm đó nghĩa là gì, phác họa vài cách tư duy về chúng, và đi qua những kiến thức nền tảng cần cho các chương sau. Trong những chương tiếp theo, ta sẽ tiếp tục đi từng lớp một, xem xét các quyết định thiết kế khác nhau cần cân nhắc khi làm việc với một ứng dụng thiên về dữ liệu.

Thinking About Data Systems — Suy nghĩ về các hệ thống dữ liệu

We typically think of databases, queues, caches, etc. as being very different categories of tools. Although a database and a **message queue** have some superficial similarity—both store data for some time—they have very different access patterns, which means different performance characteristics, and thus very different implementations.

Ta thường nghĩ về cơ sở dữ liệu, hàng đợi, bộ nhớ đệm, v.v. như những loại công cụ rất khác nhau. Mặc dù một cơ sở dữ liệu và một hàng đợi thông điệp có đôi chút tương đồng bề ngoài — cả hai đều lưu dữ liệu trong một khoảng thời gian — nhưng chúng có những mẫu truy cập rất khác nhau, dẫn đến đặc tính hiệu năng khác nhau, và do đó cách hiện thực cũng rất khác nhau.

So why should we lump them all together under an umbrella term like data systems?

Vậy tại sao ta lại gộp tất cả chúng lại dưới một thuật ngữ chung là “hệ thống dữ liệu”?

Many new tools for data storage and processing have emerged in recent years. They are optimized for a variety of different use cases, and they no longer neatly fit into traditional categories [1]. For example, there are datastores that are also used as message queues (Redis), and there are message queues with database-like **durability** guarantees (Apache Kafka). The boundaries between the categories are becoming blurred.

Những năm gần đây xuất hiện nhiều công cụ mới để lưu trữ và xử lý dữ liệu. Chúng được tối ưu cho nhiều tình huống sử dụng khác nhau, và không còn nằm gọn trong các phân loại truyền thống nữa [1]. Ví dụ, có những kho dữ liệu cũng được dùng làm hàng đợi thông điệp (Redis), và có những hàng đợi thông điệp với đảm bảo về độ bền giống cơ sở dữ liệu (Apache Kafka). Ranh giới giữa các phân loại đang dần mờ đi.

Secondly, increasingly many applications now have such demanding or wide-ranging requirements that a single tool can no longer meet all of its data processing and storage needs. Instead, the work is broken down into tasks that can be performed efficiently on a single tool, and those different tools are stitched together using **application code**.

Thứ hai, ngày càng nhiều ứng dụng có những yêu cầu khắt khe hoặc đa dạng đến mức một công cụ đơn lẻ không còn đáp ứng nổi mọi nhu cầu lưu trữ và xử lý dữ liệu. Thay vào đó, công việc được chia nhỏ thành các tác vụ mà từng công cụ có thể thực hiện hiệu quả, rồi những công cụ khác nhau đó được ghép lại với nhau bằng mã ứng dụng.

For example, if you have an application-managed **caching layer** (using Memcached or similar), or a **full-text search** server (such as Elasticsearch or Solr) separate from your main database, it is normally the application code’s responsibility to keep those caches and indexes in sync with the

main database. Figure 1-1 gives a glimpse of what this may look like (we will go into detail in later chapters).

Ví dụ, nếu bạn có một tầng đệm do ứng dụng quản lý (dùng Memcached hoặc tương tự), hay một máy chủ tìm kiếm toàn văn (như Elasticsearch hoặc Solr) tách biệt khỏi cơ sở dữ liệu chính, thì thường mã ứng dụng phải chịu trách nhiệm giữ cho các bộ đệm và chỉ mục đó đồng bộ với cơ sở dữ liệu chính. Hình 1-1 cho ta một hình dung sơ bộ về điều này (ta sẽ đi sâu vào chi tiết ở các chương sau).

Figure 1-1. One possible architecture for a data system that combines several components.

— **Hình 1-1. Một kiến trúc khả dĩ cho hệ thống dữ liệu kết hợp nhiều thành phần.** When you combine several tools in order to provide a service, the service’s interface or application programming interface (**API**) usually hides those implementation details from clients. Now you have essentially created a new, special-purpose **data system** from smaller, general-purpose components. Your composite data system may provide certain guarantees: e.g., that the **cache** will be correctly invalidated or updated on writes so that outside clients see consistent results. You are now not only an application developer, but also a data system designer.

Khi bạn kết hợp nhiều công cụ để cung cấp một dịch vụ, giao diện của dịch vụ — hay giao diện lập trình ứng dụng (**API**) — thường che giấu những chi tiết hiện thực đó khỏi phía client. Giờ đây bạn về cơ bản đã tạo ra một hệ thống dữ liệu mới, chuyên dụng, từ những thành phần nhỏ hơn, đa dụng. Hệ thống dữ liệu tổng hợp của bạn có thể cung cấp những đảm bảo nhất định: ví dụ, bộ đệm sẽ được vô hiệu hóa hoặc cập nhật đúng cách khi có thao tác ghi, để các client bên ngoài nhìn thấy kết quả nhất quán. Lúc này bạn không chỉ là một lập trình viên ứng dụng, mà còn là một người thiết kế hệ thống dữ liệu.

If you are designing a data system or service, a lot of tricky questions arise. How do you ensure that the data remains correct and complete, even when things go wrong internally? How do you provide consistently good performance to clients, even when parts of your system are degraded? How do you scale to handle an increase in **load**? What does a good API for the service look like?

Nếu bạn đang thiết kế một hệ thống dữ liệu hay dịch vụ, sẽ nảy sinh rất nhiều câu hỏi hóc búa. Làm sao đảm bảo dữ liệu vẫn đúng và đầy đủ, ngay cả khi có sự cố nội bộ? Làm sao cung cấp hiệu năng tốt một cách ổn định cho client, ngay cả khi một số phần của hệ thống đang suy giảm? Làm sao mở rộng để xử lý lượng tải tăng thêm? Một API tốt cho dịch vụ trông như thế nào?

There are many factors that may influence the design of a data system, including the skills and experience of the people involved, **legacy** system dependencies, the timescale for delivery, your organization’s tolerance of different kinds of risk, regulatory constraints, etc. Those factors depend very much on the situation.

Có nhiều yếu tố ảnh hưởng đến thiết kế của một hệ thống dữ liệu, bao gồm kỹ năng và kinh nghiệm của những người liên quan, sự phụ thuộc vào các hệ thống cũ, thời hạn bàn giao, mức độ chấp nhận rủi ro của tổ chức, các ràng buộc pháp lý, v.v. Những yếu tố này phụ thuộc rất nhiều vào hoàn cảnh cụ thể.

In this book, we focus on three concerns that are important in most software systems:

Trong cuốn sách này, ta tập trung vào ba mối quan tâm quan trọng trong hầu hết các hệ thống phần mềm:

The system should continue to work correctly (performing the correct function at the desired level of performance) even in the face of adversity (hardware or software faults, and even human error). See “**Reliability**”.

Hệ thống cần tiếp tục hoạt động đúng (thực hiện đúng chức năng ở mức hiệu năng mong muốn) ngay cả khi gặp nghịch cảnh (lỗi phần cứng hoặc phần mềm, thậm chí lỗi do con người). Xem mục “Độ tin cậy”.

As the system grows (in data volume, traffic volume, or complexity), there should be reasonable ways of dealing with that growth. See “**Scalability**”.

Khi hệ thống lớn lên (về khối lượng dữ liệu, lưu lượng truy cập, hoặc độ phức tạp), cần có những cách hợp lý để xử lý sự tăng trưởng đó. Xem mục “Khả năng mở rộng”.

Over time, many different people will work on the system (engineering and operations, both maintaining current behavior and adapting the system to new use cases), and they should all be able to work on it productively. See “**Maintainability**”.

Theo thời gian, nhiều người khác nhau sẽ làm việc trên hệ thống (kỹ sư phát triển và vận hành, vừa duy trì hành vi hiện tại vừa điều chỉnh hệ thống cho các tình huống dùng mới), và tất cả họ đều cần làm việc hiệu quả trên đó. Xem mục “Khả năng bảo trì”.

These words are often cast around without a clear understanding of what they mean. In the interest of thoughtful engineering, we will spend the rest of this chapter exploring ways of thinking about reliability, scalability, and maintainability. Then, in the following chapters, we will look at various techniques, architectures, and algorithms that are used in order to achieve those goals.

Những từ này thường được dùng tùy tiện mà không thực sự hiểu rõ ý nghĩa. Vì mục tiêu kỹ thuật thấu đáo, ta sẽ dành phần còn lại của chương để khám phá các cách tư duy về độ tin cậy, khả năng mở rộng và khả năng bảo trì. Sau đó, ở các chương tiếp theo, ta sẽ xem xét nhiều kỹ thuật, kiến trúc và thuật toán được dùng để đạt được những mục tiêu này.

Reliability — Độ tin cậy

Everybody has an intuitive idea of what it means for something to be reliable or unreliable. For software, typical expectations include:

Ai cũng có một ý niệm trực giác về việc một thứ gì đó tin cậy hay không tin cậy nghĩa là gì. Với phần mềm, những kỳ vọng điển hình bao gồm:

- The application performs the function that the user expected.
- It can tolerate the user making mistakes or using the software in unexpected ways.
- Its performance is good enough for the required use case, under the expected load and data volume.
- The system prevents any unauthorized access and abuse.
 - Ứng dụng thực hiện đúng chức năng mà người dùng mong đợi.
 - Nó chịu được việc người dùng phạm sai lầm hoặc dùng phần mềm theo những cách bất ngờ.
 - Hiệu năng của nó đủ tốt cho tình huống sử dụng đặt ra, dưới mức tải và khối lượng dữ liệu dự kiến.
 - Hệ thống ngăn chặn mọi truy cập trái phép và lạm dụng.

If all those things together mean “working correctly,” then we can understand reliability as meaning, roughly, “continuing to work correctly, even when things go wrong.”

Nếu gộp tất cả những điều đó lại mà gọi là “hoạt động đúng”, thì ta có thể hiểu độ tin cậy đại khái nghĩa là “tiếp tục hoạt động đúng, ngay cả khi có trục trặc”.

The things that can go wrong are called faults, and systems that anticipate faults and can cope with them are called **fault-tolerant** or **resilient**. The former term is slightly misleading: it suggests that we could make a system tolerant of every possible kind of **fault**, which in reality is not feasible. If the entire planet Earth (and all servers on it) were swallowed by a black hole, tolerance of that fault would require web hosting in space—good luck getting that budget item approved. So it only makes sense to talk about tolerating certain types of faults.

Những thứ có thể trục trặc được gọi là lỗi, và những hệ thống lường trước được lỗi và ứng phó được với chúng được gọi là chịu lỗi hay kiên cường. Thuật ngữ đầu hơi gây hiểu lầm: nó gợi ý rằng ta có thể làm cho hệ thống chịu được mọi loại lỗi có thể, điều mà trên thực tế là bất khả thi. Nếu cả hành tinh Trái Đất (và mọi máy chủ trên đó) bị một hố đen nuốt chửng, thì để chịu được lỗi đó ta sẽ cần lưu trữ web ngoài vũ trụ — chúc may mắn xin duyệt khoản ngân sách ấy. Vậy nên chỉ hợp lý khi nói về việc chịu được một số loại lỗi nhất định.

Note that a fault is not the same as a **failure** [2]. A fault is usually defined as one component of the system deviating from its spec, whereas a failure is when the system as a whole stops providing the required service to the user. It is impossible to reduce the probability of a fault to zero; therefore it is

usually best to design fault-tolerance mechanisms that prevent faults from causing failures. In this book we cover several techniques for building reliable systems from unreliable parts.

Lưu ý rằng lỗi (fault) không giống với hỏng hóc, hay sự cố hệ thống (failure) [2]. Một lỗi thường được định nghĩa là một thành phần của hệ thống đi chệch khỏi đặc tả của nó, trong khi một sự cố hệ thống là khi cả hệ thống nói chung ngừng cung cấp dịch vụ cần thiết cho người dùng. Không thể giảm xác suất xảy ra lỗi xuống bằng không; do đó thường tốt nhất là thiết kế các cơ chế chịu lỗi để ngăn lỗi gây ra sự cố hệ thống. Trong cuốn sách này, ta đề cập đến nhiều kỹ thuật xây dựng hệ thống tin cậy từ những thành phần không đáng tin cậy.

Counterintuitively, in such fault-tolerant systems, it can make sense to increase the rate of faults by triggering them deliberately—for example, by randomly killing individual processes without warning. Many critical bugs are actually due to poor error handling [3]; by deliberately inducing faults, you ensure that the fault-tolerance machinery is continually exercised and tested, which can increase your confidence that faults will be handled correctly when they occur naturally. The Netflix Chaos Monkey [4] is an example of this approach.

Trái với trực giác, trong những hệ thống chịu lỗi như vậy, đôi khi việc tăng tần suất lỗi lại hợp lý — bằng cách cố tình kích hoạt chúng, ví dụ ngẫu nhiên “giết” từng tiến trình mà không báo trước. Nhiều bug nghiêm trọng thực ra bắt nguồn từ việc xử lý lỗi kém [3]; bằng cách cố tình gây ra lỗi, bạn đảm bảo rằng bộ máy chịu lỗi luôn được vận hành và kiểm thử liên tục, qua đó tăng độ tự tin rằng lỗi sẽ được xử lý đúng khi chúng xảy ra một cách tự nhiên. Netflix Chaos Monkey [4] là một ví dụ cho cách tiếp cận này.

Although we generally prefer tolerating faults over preventing faults, there are cases where prevention is better than cure (e.g., because no cure exists). This is the case with security matters, for example: if an attacker has compromised a system and gained access to sensitive data, that event cannot be undone. However, this book mostly deals with the kinds of faults that can be cured, as described in the following sections.

Mặc dù nhìn chung ta ưu tiên chịu lỗi hơn là ngăn lỗi, vẫn có những trường hợp phòng bệnh hơn chữa bệnh (ví dụ vì chẳng có “thuốc chữa” nào cả). An ninh chẳng hạn: nếu kẻ tấn công đã xâm nhập hệ thống và truy cập được dữ liệu nhạy cảm, sự kiện đó không thể đảo ngược. Tuy nhiên, cuốn sách này chủ yếu bàn về những loại lỗi có thể “chữa được”, như mô tả trong các mục sau.

Hardware Faults — Lỗi phần cứng

When we think of causes of system failure, hardware faults quickly come to mind. Hard disks crash, RAM becomes faulty, the power grid has a blackout, someone unplugs the wrong network cable. Anyone who has worked with large datacenters can tell you that these things happen all the time when you have a lot of machines.

Khi nghĩ về nguyên nhân gây sự cố hệ thống, lỗi phần cứng nhanh chóng hiện lên trong đầu. Đĩa cứng hỏng, RAM lỗi, lưới điện mất điện, ai đó rút nhầm dây mạng. Bất cứ ai từng làm việc với các trung tâm dữ liệu lớn đều có thể nói với bạn rằng những chuyện này xảy ra suốt khi bạn có nhiều máy.

Hard disks are reported as having a mean time to failure (**MTTF**) of about 10 to 50 years [5, 6]. Thus, on a storage cluster with 10,000 disks, we should expect on **average** one disk to die per day.

Đĩa cứng được báo cáo có thời gian trung bình đến khi hỏng (MTTF) khoảng 10 đến 50 năm [5, 6]. Vì vậy, trên một cụm lưu trữ với 10.000 đĩa, ta nên kỳ vọng trung bình mỗi ngày có

một đĩa “chết”.

Our first response is usually to add **redundancy** to the individual hardware components in order to reduce the failure rate of the system. Disks may be set up in a **RAID** configuration, servers may have dual power supplies and **hot-swappable** CPUs, and datacenters may have batteries and diesel generators for **backup** power. When one component dies, the redundant component can take its place while the broken component is replaced. This approach cannot completely prevent hardware problems from causing failures, but it is well understood and can often keep a machine running uninterrupted for years.

Phản ứng đầu tiên của ta thường là thêm dự phòng cho từng thành phần phần cứng nhằm giảm tỷ lệ sự cố của hệ thống. Đĩa có thể được thiết lập theo cấu hình RAID, máy chủ có thể có hai nguồn điện và CPU thay nóng, còn trung tâm dữ liệu có thể có pin và máy phát điện diesel để dự phòng. Khi một thành phần “chết”, thành phần dự phòng có thể thế chỗ trong lúc thành phần hỏng được thay thế. Cách này không thể ngăn chặn hoàn toàn việc sự cố phần cứng dẫn tới hỏng hóc, nhưng nó được hiểu rõ và thường có thể giữ một máy chạy liên tục trong nhiều năm.

Until recently, redundancy of hardware components was sufficient for most applications, since it makes total failure of a single machine fairly rare. As long as you can restore a backup onto a new machine fairly quickly, the **downtime** in case of failure is not catastrophic in most applications. Thus, **multi-machine redundancy** was only required by a small number of applications for which **high availability** was absolutely essential.

Cho đến gần đây, việc dự phòng các thành phần phần cứng là đủ cho hầu hết ứng dụng, vì nó khiến việc hỏng toàn bộ một máy đơn lẻ trở nên khá hiếm. Miễn là bạn có thể khôi phục bản sao lưu lên một máy mới khá nhanh, thì thời gian ngừng hoạt động khi có sự cố không quá thảm khốc với hầu hết ứng dụng. Vì vậy, dự phòng nhiều máy chỉ cần thiết cho một số ít ứng dụng mà tính sẵn sàng cao là điều tuyệt đối thiết yếu.

However, as data volumes and applications’ computing demands have increased, more applications have begun using larger numbers of machines, which proportionally increases the rate of hardware faults. Moreover, in some cloud platforms such as Amazon Web Services (AWS) it is fairly common for **virtual machine** instances to become unavailable without warning [7], as the platforms are designed to prioritize flexibility and **elasticity** over single-machine reliability.

Tuy nhiên, khi khối lượng dữ liệu và nhu cầu tính toán của ứng dụng tăng lên, ngày càng nhiều ứng dụng bắt đầu dùng số lượng máy lớn hơn, làm tăng tương ứng tỷ lệ lỗi phần cứng. Hơn nữa, trên một số nền tảng đám mây như Amazon Web Services (AWS), việc các máy ảo trở nên không khả dụng mà không báo trước là khá phổ biến [7], bởi các nền tảng này được thiết kế để ưu tiên tính linh hoạt và co giãn hơn là độ tin cậy của từng máy đơn lẻ.

Hence there is a move toward systems that can tolerate the loss of entire machines, by using software fault-tolerance techniques in preference or in addition to hardware redundancy. Such systems also have operational advantages: a single-server system requires planned downtime if you need to reboot the machine (to apply operating system security patches, for example), whereas a system that can tolerate machine failure can be patched one **node** at a time, without downtime of the entire system (a **rolling upgrade**; see Chapter 4).

Do đó có một xu hướng chuyển sang những hệ thống có thể chịu được việc mất nguyên cả máy, bằng cách dùng các kỹ thuật chịu lỗi bằng phần mềm — ưu tiên hơn, hoặc bổ sung cho, việc dự phòng phần cứng. Những hệ thống như vậy còn có lợi thế về vận hành: một hệ thống chạy trên máy chủ đơn lẻ đòi hỏi phải có thời gian ngừng theo kế hoạch nếu cần

khởi động lại máy (chẳng hạn để vá bảo mật cho hệ điều hành), trong khi một hệ thống chịu được lỗi máy thì có thể vá từng nút một, không cần ngừng toàn bộ hệ thống (nâng cấp cuốn chiếu; xem Chương 4).

Software Errors — Lỗi phần mềm

We usually think of hardware faults as being random and independent from each other: one machine's disk failing does not imply that another machine's disk is going to fail. There may be weak correlations (for example due to a common cause, such as the temperature in the server rack), but otherwise it is unlikely that a large number of hardware components will fail at the same time.

Ta thường nghĩ về lỗi phần cứng như những sự kiện ngẫu nhiên và độc lập với nhau: đĩa của một máy hỏng không hàm ý đĩa của một máy khác sắp hỏng. Có thể có những mối tương quan yếu (ví dụ do một nguyên nhân chung, như nhiệt độ trong tủ rack máy chủ), nhưng ngoài ra thì khó có chuyện một số lượng lớn thành phần phần cứng cùng hỏng một lúc.

Another class of fault is a **systematic error** within the system [8]. Such faults are harder to anticipate, and because they are correlated across nodes, they tend to cause many more system failures than uncorrelated hardware faults [5]. Examples include:

Một loại lỗi khác là lỗi mang tính hệ thống bên trong hệ thống [8]. Những lỗi này khó lường trước hơn, và vì chúng tương quan với nhau giữa các nút, chúng có xu hướng gây ra nhiều sự cố hệ thống hơn hẳn so với các lỗi phần cứng không tương quan [5]. Ví dụ:

- A software **bug** that causes every **instance** of an application server to crash when given a particular bad input. For example, consider the **leap second** on June 30, 2012, that caused many applications to hang simultaneously due to a bug in the Linux kernel [9].
- A runaway process that uses up some shared resource—CPU time, memory, disk space, or network bandwidth.
- A service that the system depends on that slows down, becomes unresponsive, or starts returning corrupted responses.
- Cascading failures, where a small fault in one component triggers a fault in another component, which in turn triggers further faults [10].
 - Một bug phần mềm khiến mọi phiên bản của một máy chủ ứng dụng đều sập khi nhận một đầu vào xấu cụ thể. Chẳng hạn, hãy nhớ lại giây nhuận ngày 30 tháng 6 năm 2012 đã khiến nhiều ứng dụng treo đồng loạt do một bug trong nhân Linux [9].
 - Một tiến trình “chạy loạn” ngốn hết một tài nguyên dùng chung nào đó — thời gian CPU, bộ nhớ, dung lượng đĩa, hay băng thông mạng.
 - Một dịch vụ mà hệ thống phụ thuộc vào bỗng chậm đi, ngừng phản hồi, hoặc bắt đầu trả về các phản hồi bị hỏng.
 - Sự cố dây chuyền, khi một lỗi nhỏ ở một thành phần kích hoạt lỗi ở thành phần khác, rồi lại kích hoạt thêm nhiều lỗi nữa [10].

The bugs that cause these kinds of software faults often lie dormant for a long time until they are triggered by an unusual set of circumstances. In those circumstances, it is revealed that the software is making some kind of assumption about its environment—and while that assumption is usually true, it eventually stops being true for some reason [11].

Những bug gây ra các loại lỗi phần mềm này thường nằm im trong thời gian dài cho đến khi bị kích hoạt bởi một tập hợp hoàn cảnh bất thường. Trong những hoàn cảnh đó, hóa

ra phần mềm đang đặt ra một giả định nào đó về môi trường của nó — và dù giả định ấy thường đúng, đến một lúc nào đó nó thôi không còn đúng nữa vì một lý do nào đó [11].

There is no quick solution to the problem of systematic faults in software. Lots of small things can help: carefully thinking about assumptions and interactions in the system; thorough testing; **process isolation**; allowing processes to crash and restart; measuring, monitoring, and analyzing system behavior in **production**. If a system is expected to provide some guarantee (for example, in a message **queue**, that the number of incoming messages equals the number of outgoing messages), it can constantly check itself while it is running and raise an alert if a discrepancy is found [12].

Không có giải pháp nhanh gọn cho vấn đề lỗi hệ thống trong phần mềm. Nhiều việc nhỏ có thể giúp ích: suy nghĩ cẩn thận về các giả định và tương tác trong hệ thống; kiểm thử kỹ lưỡng; cô lập tiến trình; cho phép các tiến trình sập rồi khởi động lại; đo lường, giám sát và phân tích hành vi hệ thống trong môi trường production. Nếu một hệ thống được kỳ vọng cung cấp một đảm bảo nào đó (ví dụ, trong một hàng đợi thông điệp, số thông điệp đi vào bằng số thông điệp đi ra), nó có thể liên tục tự kiểm tra trong khi chạy và phát cảnh báo nếu phát hiện chênh lệch [12].

Human Errors — Lỗi do con người

Humans design and build software systems, and the operators who keep the systems running are also human. Even when they have the best intentions, humans are known to be unreliable. For example, one study of large internet services found that configuration errors by operators were the leading cause of outages, whereas hardware faults (servers or network) played a role in only 10–25% of outages [13].

Con người thiết kế và xây dựng hệ thống phần mềm, và những người vận hành giữ cho hệ thống chạy cũng là con người. Ngay cả với thiện chí tốt nhất, con người vẫn nổi tiếng là không đáng tin cậy. Ví dụ, một nghiên cứu về các dịch vụ internet lớn cho thấy lỗi cấu hình do người vận hành là nguyên nhân hàng đầu gây gián đoạn dịch vụ, trong khi lỗi phần cứng (máy chủ hoặc mạng) chỉ đóng vai trò trong 10–25% các vụ gián đoạn [13].

How do we make our systems reliable, in spite of unreliable humans? The best systems combine several approaches:

Làm sao để hệ thống của ta tin cậy, bất chấp con người không đáng tin? Những hệ thống tốt nhất kết hợp nhiều cách:

- Design systems in a way that minimizes opportunities for error. For example, well-designed abstractions, APIs, and admin interfaces make it easy to do “the right thing” and discourage “the wrong thing.” However, if the interfaces are too restrictive people will work around them, negating their benefit, so this is a tricky balance to get right.
- Decouple the places where people make the most mistakes from the places where they can cause failures. In particular, provide fully featured non-production **sandbox** environments where people can explore and experiment safely, using real data, without affecting real users.
- Test thoroughly at all levels, from unit tests to whole-system integration tests and manual tests [3]. Automated testing is widely used, well understood, and especially valuable for covering corner cases that rarely arise in normal operation.
- Allow quick and easy recovery from human errors, to minimize the impact in the case of a failure. For example, make it fast to **roll back** configuration changes, roll out new code gradually (so that any unexpected bugs affect only a small subset of users), and provide tools to recompute data (in case it turns out that the old computation was incorrect).

- Set up detailed and clear monitoring, such as performance metrics and error rates. In other engineering disciplines this is referred to as **telemetry**. (Once a rocket has left the ground, telemetry is essential for tracking what is happening, and for understanding failures [14].) Monitoring can show us early warning signals and allow us to check whether any assumptions or constraints are being violated. When a problem occurs, metrics can be invaluable in diagnosing the issue.
- Implement good management practices and training—a complex and important aspect, and beyond the scope of this book.
 - Thiết kế hệ thống sao cho giảm thiểu cơ hội phạm lỗi. Ví dụ, những trừu tượng hóa, API và giao diện quản trị được thiết kế tốt sẽ giúp dễ làm “điều đúng” và cản trở “điều sai”. Tuy nhiên, nếu giao diện quá hạn chế, người ta sẽ tìm cách lách qua, triệt tiêu lợi ích của nó — nên đây là một thể cân bằng khó đạt cho đúng.
 - Tách rời những nơi con người dễ phạm lỗi nhất khỏi những nơi mà lỗi có thể gây ra sự cố. Cụ thể, hãy cung cấp các môi trường thử nghiệm phi-production đầy đủ tính năng, nơi người ta có thể khám phá và thử nghiệm an toàn bằng dữ liệu thật mà không ảnh hưởng đến người dùng thật.
 - Kiểm thử kỹ lưỡng ở mọi cấp độ, từ kiểm thử đơn vị đến kiểm thử tích hợp toàn hệ thống và kiểm thử thủ công [3]. Kiểm thử tự động được dùng rộng rãi, được hiểu rõ, và đặc biệt giá trị để bao phủ các trường hợp biên hiểm khi xảy ra trong vận hành bình thường.
 - Cho phép phục hồi nhanh và dễ dàng từ lỗi của con người, để giảm thiểu tác động khi xảy ra sự cố. Ví dụ, làm sao để hoàn tác các thay đổi cấu hình thật nhanh, triển khai mã mới một cách từ từ (để mọi bug bất ngờ chỉ ảnh hưởng đến một nhóm nhỏ người dùng), và cung cấp công cụ để tính toán lại dữ liệu (phòng khi hóa ra phép tính cũ bị sai).
 - Thiết lập giám sát chi tiết và rõ ràng, chẳng hạn các chỉ số hiệu năng và tỷ lệ lỗi. Trong các ngành kỹ thuật khác, điều này được gọi là đo từ xa. (Một khi tên lửa đã rời mặt đất, đo từ xa là thiết yếu để theo dõi điều gì đang diễn ra và để hiểu các sự cố [14].) Giám sát có thể cho ta những tín hiệu cảnh báo sớm và cho phép ta kiểm tra xem có giả định hay ràng buộc nào đang bị vi phạm hay không. Khi sự cố xảy ra, các chỉ số có thể là vô giá trong việc chẩn đoán vấn đề.
 - Triển khai các thực hành quản lý tốt và đào tạo — một khía cạnh phức tạp và quan trọng, nằm ngoài phạm vi cuốn sách này.

How Important Is Reliability? — Độ tin cậy quan trọng đến mức nào?

Reliability is not just for nuclear power stations and air traffic control software—more mundane applications are also expected to work reliably. Bugs in business applications cause lost productivity (and legal risks if figures are reported incorrectly), and outages of ecommerce sites can have huge costs in terms of lost revenue and damage to reputation.

Độ tin cậy không chỉ dành cho nhà máy điện hạt nhân và phần mềm điều khiển không lưu — những ứng dụng đòi hỏi thường hơn cũng được kỳ vọng hoạt động một cách tin cậy. Bug trong các ứng dụng nghiệp vụ gây mất năng suất (và rủi ro pháp lý nếu số liệu bị báo cáo sai), còn việc các trang thương mại điện tử bị gián đoạn có thể gây thiệt hại khổng lồ về doanh thu và uy tín.

Even in “noncritical” applications we have a responsibility to our users. Consider a parent who stores

all their pictures and videos of their children in your photo application [15]. How would they feel if that database was suddenly corrupted? Would they know how to restore it from a backup?

Ngay cả trong những ứng dụng “không trọng yếu”, ta vẫn có trách nhiệm với người dùng của mình. Hãy nghĩ đến một người cha hay người mẹ lưu toàn bộ ảnh và video về con cái họ trong ứng dụng ảnh của bạn [15]. Họ sẽ cảm thấy thế nào nếu cơ sở dữ liệu đó bỗng nhiên hỏng? Liệu họ có biết cách khôi phục nó từ bản sao lưu?

There are situations in which we may choose to sacrifice reliability in order to reduce development cost (e.g., when developing a prototype product for an unproven market) or operational cost (e.g., for a service with a very narrow profit margin)—but we should be very conscious of when we are cutting corners.

Có những tình huống ta có thể chọn hy sinh độ tin cậy để giảm chi phí phát triển (ví dụ, khi làm một sản phẩm nguyên mẫu cho một thị trường chưa được kiểm chứng) hoặc chi phí vận hành (ví dụ, cho một dịch vụ có biên lợi nhuận rất hẹp) — nhưng ta nên hết sức ý thức được khi nào mình đang làm tắt, làm ẩu.

Scalability — Khả năng mở rộng

Even if a system is working reliably today, that doesn't mean it will necessarily work reliably in the future. One common reason for degradation is increased load: perhaps the system has grown from 10,000 concurrent users to 100,000 concurrent users, or from 1 million to 10 million. Perhaps it is processing much larger volumes of data than it did before.

Ngay cả khi một hệ thống đang hoạt động tin cậy ở hiện tại, điều đó không có nghĩa nó sẽ nhất thiết hoạt động tin cậy trong tương lai. Một lý do phổ biến gây suy giảm là tải tăng lên: có thể hệ thống đã tăng từ 10.000 lên 100.000 người dùng đồng thời, hoặc từ 1 triệu lên 10 triệu. Có thể nó đang xử lý lượng dữ liệu lớn hơn nhiều so với trước.

Scalability is the term we use to describe a system's ability to cope with increased load. Note, however, that it is not a one-dimensional label that we can attach to a system: it is meaningless to say “X is scalable” or “Y doesn't scale.” Rather, discussing scalability means considering questions like “If the system grows in a particular way, what are our options for coping with the growth?” and “How can we add computing resources to handle the additional load?”

Khả năng mở rộng là thuật ngữ ta dùng để mô tả khả năng của một hệ thống trong việc ứng phó với tải tăng thêm. Tuy nhiên, lưu ý rằng đây không phải một nhãn một chiều mà ta có thể gán cho một hệ thống: nói “X có khả năng mở rộng” hay “Y không mở rộng được” là vô nghĩa. Thay vào đó, bàn về khả năng mở rộng nghĩa là xem xét những câu hỏi như “Nếu hệ thống lớn lên theo một cách cụ thể nào đó, ta có những lựa chọn nào để ứng phó với sự tăng trưởng?” và “Làm sao ta có thể bổ sung tài nguyên tính toán để xử lý lượng tải tăng thêm?”

Describing Load — Mô tả tải

First, we need to succinctly describe the current load on the system; only then can we discuss growth questions (what happens if our load doubles?). Load can be described with a few numbers which we call load parameters. The best choice of parameters depends on the architecture of your system: it may be requests per second to a web server, the ratio of reads to writes in a database, the number of simultaneously active users in a chat room, the **hit rate** on a cache, or something else. Perhaps the average case is what matters for you, or perhaps your bottleneck is dominated by a small number of extreme cases.

Trước hết, ta cần mô tả ngắn gọn lượng tải hiện tại của hệ thống; chỉ khi đó ta mới có thể bàn về các câu hỏi tăng trưởng (điều gì xảy ra nếu tải của ta tăng gấp đôi?). Tải có thể được mô tả bằng vài con số mà ta gọi là tham số tải. Cách chọn tham số tốt nhất tùy thuộc vào kiến trúc hệ thống của bạn: có thể là số yêu cầu mỗi giây tới một máy chủ web, tỷ lệ giữa đọc và ghi trong một cơ sở dữ liệu, số người dùng đang hoạt động đồng thời trong một phòng chat, tỷ lệ trúng của một bộ nhớ đệm, hoặc thứ gì khác. Có thể trường hợp trung

binh là điều bạn quan tâm, hoặc có thể nút thắt cổ chai của bạn bị chi phối bởi một số ít trường hợp cực đoan.

To make this idea more concrete, let's consider Twitter as an example, using data published in November 2012 [16]. Two of Twitter's main operations are:

Để cụ thể hóa ý này, hãy xét Twitter làm ví dụ, dùng dữ liệu công bố hồi tháng 11 năm 2012 [16]. Hai thao tác chính của Twitter là:

A user can publish a new message to their followers (4.6k requests/sec on average, over 12k requests/sec at peak).

Đăng tweet: người dùng có thể đăng một thông điệp mới cho những người theo dõi của mình (trung bình 4,6k yêu cầu/giây, lúc cao điểm trên 12k yêu cầu/giây).

A user can view tweets posted by the people they follow (300k requests/sec).

Dòng thời gian chính: người dùng có thể xem các tweet do những người họ theo dõi đăng (300k yêu cầu/giây).

Simply handling 12,000 writes per second (the peak rate for posting tweets) would be fairly easy. However, Twitter's scaling challenge is not primarily due to tweet volume, but due to **fan-out**—each user follows many people, and each user is followed by many people. There are broadly two ways of implementing these two operations:

Việc chỉ xử lý 12.000 lượt ghi mỗi giây (tốc độ cao điểm khi đăng tweet) thì khá dễ. Tuy nhiên, thách thức mở rộng của Twitter không chủ yếu nằm ở khối lượng tweet, mà ở hiện tượng fan-out — mỗi người dùng theo dõi nhiều người, và mỗi người dùng cũng được nhiều người theo dõi. Nhìn chung có hai cách hiện thực hai thao tác này:

- Posting a tweet simply inserts the new tweet into a global collection of tweets. When a user requests their **home timeline**, look up all the people they follow, find all the tweets for each of those users, and merge them (sorted by time). In a relational database like in Figure 1-2, you could write a query such as: `SELECT tweets., users. FROM tweets JOIN users ON tweets.sender_id = users.id JOIN follows ON follows.followee_id = users.id WHERE follows.follower_id = current_user`
- Maintain a cache for each user's home timeline—like a mailbox of tweets for each recipient user (see Figure 1-3). When a user posts a tweet, look up all the people who follow that user, and insert the new tweet into each of their home timeline caches. The request to read the home timeline is then cheap, because its result has been computed ahead of time.
 - Đăng một tweet đơn giản là chèn tweet mới vào một tập hợp tweet toàn cục. Khi một người dùng yêu cầu dòng thời gian chính của mình, tra cứu tất cả những người họ theo dõi, tìm mọi tweet của từng người đó, rồi trộn chúng lại (sắp xếp theo thời gian). Trong một cơ sở dữ liệu quan hệ như ở Hình 1-2, bạn có thể viết một truy vấn SQL như ở bản tiếng Anh phía trên.
 - Duy trì một bộ nhớ đệm cho dòng thời gian chính của mỗi người dùng — giống như một hộp thư chứa tweet cho mỗi người nhận (xem Hình 1-3). Khi một người dùng đăng một tweet, tra cứu tất cả những người theo dõi người đó, và chèn tweet mới vào từng bộ đệm dòng thời gian chính của họ. Khi đó, yêu cầu đọc dòng thời gian chính trở nên rẻ, vì kết quả của nó đã được tính toán sẵn từ trước.

Figure 1-2. Simple relational schema for implementing a Twitter home timeline. — Hình 1-2. Lược đồ quan hệ đơn giản để hiện thực dòng thời gian chính của Twitter.

Figure 1-3. Twitter’s data pipeline for delivering tweets to followers, with load parameters as of November 2012 [16]. — Hình 1-3. Đường ống dữ liệu của Twitter để phân phối tweet tới người theo dõi, với các tham số tải tính đến tháng 11 năm 2012 [16]. The first version of Twitter used approach 1, but the systems struggled to keep up with the load of home timeline queries, so the company switched to approach 2. This works better because the average rate of published tweets is almost two orders of magnitude lower than the rate of home timeline reads, and so in this case it’s preferable to do more work at write time and less at read time.

Phiên bản đầu tiên của Twitter dùng cách 1, nhưng các hệ thống chật vật theo kịp tải của các truy vấn dòng thời gian chính, nên công ty chuyển sang cách 2. Cách này hiệu quả hơn vì tốc độ trung bình của các tweet được đăng thấp hơn gần hai bậc độ lớn so với tốc độ đọc dòng thời gian chính, và do đó trong trường hợp này, làm nhiều việc hơn ở lúc ghi và ít việc hơn ở lúc đọc là điều đáng ưu tiên.

However, the downside of approach 2 is that posting a tweet now requires a lot of extra work. On average, a tweet is delivered to about 75 followers, so 4.6k tweets per second become 345k writes per second to the home timeline caches. But this average hides the fact that the number of followers per user varies wildly, and some users have over 30 million followers. This means that a single tweet may result in over 30 million writes to home timelines! Doing this in a timely manner—Twitter tries to deliver tweets to followers within five seconds—is a significant challenge.

Tuy nhiên, nhược điểm của cách 2 là giờ đây việc đăng một tweet đòi hỏi rất nhiều công việc phát sinh thêm. Trung bình, một tweet được phân phối tới khoảng 75 người theo dõi, nên 4,6k tweet mỗi giây trở thành 345k lượt ghi mỗi giây vào các bộ đệm dòng thời gian chính. Nhưng con số trung bình này che giấu một sự thật là số người theo dõi của mỗi người dùng dao động rất lớn, và một số người dùng có trên 30 triệu người theo dõi. Điều này nghĩa là một tweet đơn lẻ có thể dẫn đến trên 30 triệu lượt ghi vào các dòng thời gian chính! Làm việc này một cách kịp thời — Twitter cố gắng phân phối tweet tới người theo dõi trong vòng năm giây — là một thách thức đáng kể.

In the example of Twitter, the distribution of followers per user (maybe weighted by how often those users tweet) is a key **load parameter** for discussing scalability, since it determines the fan-out load. Your application may have very different characteristics, but you can apply similar principles to reasoning about its load.

Trong ví dụ Twitter, phân bố số người theo dõi trên mỗi người dùng (có thể có trọng số theo tần suất những người dùng đó đăng tweet) là một tham số tải then chốt để bàn về khả năng mở rộng, vì nó quyết định tải fan-out. Ứng dụng của bạn có thể có những đặc tính rất khác, nhưng bạn có thể áp dụng các nguyên lý tương tự để suy luận về tải của nó.

The final twist of the Twitter anecdote: now that approach 2 is robustly implemented, Twitter is moving to a hybrid of both approaches. Most users’ tweets continue to be fanned out to home timelines at the time when they are posted, but a small number of users with a very large number of followers (i.e., celebrities) are excepted from this fan-out. Tweets from any celebrities that a user may follow are fetched separately and merged with that user’s home timeline when it is read, like in approach 1. This hybrid approach is able to deliver consistently good performance. We will revisit this example in Chapter 12 after we have covered some more technical ground.

Khúc ngoặt cuối của giai thoại Twitter: giờ đây khi cách 2 đã được hiện thực vững vàng, Twitter đang chuyển sang một cách lai kết hợp cả hai. Tweet của hầu hết người dùng vẫn tiếp tục được fan-out tới các dòng thời gian chính vào lúc chúng được đăng, nhưng một số ít người dùng có rất nhiều người theo dõi (tức là những người nổi tiếng) được miễn khỏi việc fan-out này. Tweet từ bất kỳ người nổi tiếng nào mà một người dùng theo dõi sẽ được lấy riêng và trộn vào dòng thời gian chính của người dùng đó khi nó được đọc, giống như

ở cách 1. Cách lại này có thể mang lại hiệu năng tốt một cách ổn định. Ta sẽ quay lại ví dụ này ở Chương 12 sau khi đã đề cập thêm một số nền tảng kỹ thuật.

Describing Performance — Mô tả hiệu năng

Once you have described the load on your system, you can investigate what happens when the load increases. You can look at it in two ways:

Một khi đã mô tả được tải của hệ thống, bạn có thể khảo sát điều gì xảy ra khi tải tăng lên. Có thể nhìn theo hai cách:

- When you increase a load parameter and keep the system resources (CPU, memory, network bandwidth, etc.) unchanged, how is the performance of your system affected?
- When you increase a load parameter, how much do you need to increase the resources if you want to keep performance unchanged?
 - Khi bạn tăng một tham số tải mà giữ nguyên tài nguyên hệ thống (CPU, bộ nhớ, băng thông mạng, v.v.), hiệu năng của hệ thống bị ảnh hưởng thế nào?
 - Khi bạn tăng một tham số tải, bạn cần tăng tài nguyên lên bao nhiêu nếu muốn giữ nguyên hiệu năng?

Both questions require performance numbers, so let's look briefly at describing the performance of a system.

Cả hai câu hỏi đều cần các con số hiệu năng, nên hãy cùng xem qua cách mô tả hiệu năng của một hệ thống.

In a batch processing system such as Hadoop, we usually care about **throughput**—the number of records we can process per second, or the total time it takes to run a job on a dataset of a certain size. In online systems, what's usually more important is the service's **response time**—that is, the time between a **client** sending a request and receiving a response.

Trong một hệ thống xử lý theo lô như Hadoop, ta thường quan tâm đến thông lượng — số bản ghi ta có thể xử lý mỗi giây, hoặc tổng thời gian cần để chạy một công việc trên một tập dữ liệu có kích thước nhất định. Trong các hệ thống trực tuyến, điều thường quan trọng hơn là thời gian phản hồi của dịch vụ — tức là khoảng thời gian giữa lúc client gửi một yêu cầu và lúc nhận được phản hồi.

Latency and response time — Độ trễ và thời gian phản hồi

Latency and response time are often used synonymously, but they are not the same. The response time is what the client sees: besides the actual time to process the request (the **service time**), it includes network delays and queueing delays. Latency is the duration that a request is waiting to be handled—during which it is latent, awaiting service [17].

Độ trễ và thời gian phản hồi thường được dùng như từ đồng nghĩa, nhưng chúng không giống nhau. Thời gian phản hồi là cái mà client nhìn thấy: ngoài thời gian thực sự để xử lý yêu cầu (thời gian phục vụ), nó còn bao gồm độ trễ mạng và độ trễ xếp hàng. Độ trễ là khoảng thời gian mà một yêu cầu phải chờ để được xử lý — trong lúc đó nó đang “tiềm ẩn”, chờ được phục vụ [17].

Even if you only make the same request over and over again, you’ll get a slightly different response time on every try. In practice, in a system handling a variety of requests, the response time can vary a lot. We therefore need to think of response time not as a single number, but as a distribution of values that you can measure.

Ngay cả khi bạn chỉ thực hiện đi thực hiện lại đúng một yêu cầu, mỗi lần thử bạn vẫn sẽ nhận được một thời gian phản hồi hơi khác nhau. Trên thực tế, trong một hệ thống xử lý nhiều loại yêu cầu khác nhau, thời gian phản hồi có thể dao động rất nhiều. Vì vậy, ta cần nghĩ về thời gian phản hồi không phải như một con số đơn lẻ, mà như một phân bố các giá trị mà bạn có thể đo được.

In Figure 1-4, each gray bar represents a request to a service, and its height shows how long that request took. Most requests are reasonably fast, but there are occasional outliers that take much longer. Perhaps the slow requests are intrinsically more expensive, e.g., because they process more data. But even in a scenario where you’d think all requests should take the same time, you get variation: random additional latency could be introduced by a **context switch** to a background process, the loss of a network packet and TCP retransmission, a **garbage collection** pause, a **page fault** forcing a read from disk, mechanical vibrations in the server rack [18], or many other causes.

Trong Hình 1-4, mỗi thanh xám đại diện cho một yêu cầu tới một dịch vụ, và chiều cao của nó cho biết yêu cầu đó mất bao lâu. Hầu hết yêu cầu đều khá nhanh, nhưng thỉnh thoảng có những giá trị ngoại lai mất nhiều thời gian hơn hẳn. Có lẽ những yêu cầu chậm về bản chất là tốn kém hơn, ví dụ vì chúng xử lý nhiều dữ liệu hơn. Nhưng ngay cả trong một tình huống mà bạn nghĩ rằng mọi yêu cầu đều phải mất cùng một khoảng thời gian, bạn vẫn gặp sự biến thiên: độ trễ ngẫu nhiên phát sinh thêm có thể đến từ một lần chuyển ngữ cảnh sang một tiến trình nền, việc mất một gói tin mạng và phải truyền lại TCP, một lần tạm dừng do thu gom rác, một lỗi trang buộc phải đọc từ đĩa, rung động cơ học trong tủ rack máy chủ [18], hay vô số nguyên nhân khác.

Figure 1-4. Illustrating mean and percentiles: response times for a sample of 100 requests to a service. — Hình 1-4. Minh họa trung bình và các phân vị: thời gian phản hồi cho một mẫu gồm 100 yêu cầu tới một dịch vụ.

It's common to see the average response time of a service reported. (Strictly speaking, the term “average” doesn't refer to any particular formula, but in practice it is usually understood as the arithmetic mean: given n values, add up all the values, and divide by n .) However, the mean is not a very good metric if you want to know your “typical” response time, because it doesn't tell you how many users actually experienced that delay.

Người ta thường báo cáo thời gian phản hồi trung bình của một dịch vụ. (Nói chính xác, từ “trung bình” không ám chỉ một công thức cụ thể nào, nhưng trên thực tế nó thường được hiểu là trung bình cộng: cho n giá trị, cộng tất cả lại rồi chia cho n .) Tuy nhiên, trung bình cộng không phải là một thước đo tốt nếu bạn muốn biết thời gian phản hồi “điển hình” của mình, vì nó không cho bạn biết thực sự có bao nhiêu người dùng đã trải qua độ trễ đó.

Usually it is better to use percentiles. If you take your list of response times and sort it from fastest to slowest, then the **median** is the halfway point: for example, if your median response time is 200 ms, that means half your requests return in less than 200 ms, and half your requests take longer than that.

Thường thì dùng các phân vị sẽ tốt hơn. Nếu bạn lấy danh sách thời gian phản hồi của mình và sắp xếp từ nhanh nhất đến chậm nhất, thì trung vị là điểm giữa: ví dụ, nếu thời gian phản hồi trung vị của bạn là 200 ms, điều đó nghĩa là một nửa số yêu cầu trả về trong dưới 200 ms, và một nửa còn lại mất lâu hơn.

This makes the median a good metric if you want to know how long users typically have to wait: half of user requests are served in less than the median response time, and the other half take longer than the median. The median is also known as the 50th **percentile**, and sometimes abbreviated as p50. Note that the median refers to a single request; if the user makes several requests (over the course of a session, or because several resources are included in a single page), the probability that at least one of them is slower than the median is much greater than 50%.

Điều này khiến trung vị trở thành một thước đo tốt nếu bạn muốn biết người dùng thường phải chờ bao lâu: một nửa số yêu cầu của người dùng được phục vụ trong thời gian dưới mức trung vị, và nửa còn lại mất lâu hơn mức trung vị. Trung vị còn được gọi là phân vị thứ 50, đôi khi viết tắt là p50. Lưu ý rằng trung vị ám chỉ một yêu cầu đơn lẻ; nếu người dùng thực hiện nhiều yêu cầu (trong suốt một phiên, hoặc vì một trang gồm nhiều tài nguyên), xác suất để ít nhất một trong số đó chậm hơn trung vị cao hơn 50% rất nhiều.

In order to figure out how bad your outliers are, you can look at higher percentiles: the 95th, 99th, and 99.9th percentiles are common (abbreviated p95, p99, and p999). They are the response time thresholds at which 95%, 99%, or 99.9% of requests are faster than that particular threshold. For example, if the 95th percentile response time is 1.5 seconds, that means 95 out of 100 requests take less than 1.5 seconds, and 5 out of 100 requests take 1.5 seconds or more. This is illustrated in Figure 1-4.

Để biết các giá trị ngoại lai của bạn tệ đến đâu, bạn có thể nhìn vào các phân vị cao hơn: phân vị thứ 95, 99 và 99,9 là phổ biến (viết tắt p95, p99 và p999). Chúng là những ngưỡng thời gian phản hồi mà tại đó tương ứng 95%, 99% hoặc 99,9% số yêu cầu nhanh hơn ngưỡng cụ thể đó. Ví dụ, nếu thời gian phản hồi ở phân vị thứ 95 là 1,5 giây, điều đó nghĩa là 95 trên 100 yêu cầu mất dưới 1,5 giây, và 5 trên 100 yêu cầu mất từ 1,5 giây trở lên. Điều này được minh họa trong Hình 1-4.

High percentiles of response times, also known as tail latencies, are important because they directly affect users' experience of the service. For example, Amazon describes response time requirements

for internal services in terms of the 99.9th percentile, even though it only affects 1 in 1,000 requests. This is because the customers with the slowest requests are often those who have the most data on their accounts because they have made many purchases—that is, they’re the most valuable customers [19]. It’s important to keep those customers happy by ensuring the website is fast for them: Amazon has also observed that a 100 ms increase in response time reduces sales by 1% [20], and others report that a 1-second slowdown reduces a customer satisfaction metric by 16% [21, 22].

Các phân vị cao của thời gian phản hồi, còn gọi là độ trễ đuôi, rất quan trọng vì chúng ảnh hưởng trực tiếp đến trải nghiệm của người dùng đối với dịch vụ. Ví dụ, Amazon mô tả yêu cầu về thời gian phản hồi cho các dịch vụ nội bộ theo phân vị thứ 99,9, mặc dù nó chỉ ảnh hưởng đến 1 trên 1.000 yêu cầu. Đó là vì những khách hàng có yêu cầu chậm nhất thường là những người có nhiều dữ liệu nhất trên tài khoản do họ đã mua sắm nhiều — tức là những khách hàng giá trị nhất [19]. Việc giữ cho những khách hàng đó hài lòng bằng cách đảm bảo website nhanh với họ là điều quan trọng: Amazon cũng nhận thấy rằng thời gian phản hồi tăng thêm 100 ms làm giảm doanh số 1% [20], và một số nơi khác báo cáo rằng việc chậm đi 1 giây làm giảm một chỉ số hài lòng của khách hàng tới 16% [21, 22].

On the other hand, optimizing the 99.9th percentile (the slowest 1 in 10,000 requests) was deemed too expensive and to not yield enough benefit for Amazon’s purposes. Reducing response times at very high percentiles is difficult because they are easily affected by random events outside of your control, and the benefits are diminishing.

Mặt khác, việc tối ưu phân vị thứ 99,99 (1 trên 10.000 yêu cầu chậm nhất) bị cho là quá tốn kém và không mang lại đủ lợi ích cho mục đích của Amazon. Việc giảm thời gian phản hồi ở các phân vị rất cao là khó, vì chúng dễ bị ảnh hưởng bởi các sự kiện ngẫu nhiên nằm ngoài tầm kiểm soát của bạn, và lợi ích thì giảm dần.

For example, percentiles are often used in service level objectives (SLOs) and service level agreements (SLAs), contracts that define the expected performance and availability of a service. An **SLA** may state that the service is considered to be up if it has a median response time of less than 200 ms and a 99th percentile under 1 s (if the response time is longer, it might as well be down), and the service may be required to be up at least 99.9% of the time. These metrics set expectations for clients of the service and allow customers to demand a refund if the SLA is not met.

Chẳng hạn, các phân vị thường được dùng trong các mục tiêu mức dịch vụ (SLO) và cam kết mức dịch vụ (SLA), những hợp đồng định nghĩa hiệu năng và tính sẵn sàng kỳ vọng của một dịch vụ. Một SLA có thể quy định rằng dịch vụ được xem là “đang hoạt động” nếu nó có thời gian phản hồi trung vị dưới 200 ms và phân vị thứ 99 dưới 1 giây (nếu thời gian phản hồi dài hơn, thì coi như nó “ngừng hoạt động” cũng được), và dịch vụ có thể được yêu cầu phải hoạt động ít nhất 99,9% thời gian. Những chỉ số này đặt ra kỳ vọng cho các client của dịch vụ và cho phép khách hàng đòi hoàn tiền nếu SLA không được đáp ứng.

Queueing delays often account for a large part of the response time at high percentiles. As a server can only process a small number of things in parallel (limited, for example, by its number of CPU cores), it only takes a small number of slow requests to hold up the processing of subsequent requests—an effect sometimes known as **head-of-line blocking**. Even if those subsequent requests are fast to process on the server, the client will see a slow overall response time due to the time waiting for the prior request to complete. Due to this effect, it is important to measure response times on the client side.

Độ trễ xếp hàng thường chiếm một phần lớn trong thời gian phản hồi ở các phân vị cao. Vì một máy chủ chỉ có thể xử lý song song một số ít việc (bị giới hạn, ví dụ, bởi số lõi CPU của nó), chỉ cần một số ít yêu cầu chậm là đủ để cản trở việc xử lý các yêu cầu tiếp theo — một hiệu ứng đôi khi được gọi là tắc nghẽn đầu hàng. Ngay cả khi những yêu cầu tiếp theo

đó được xử lý nhanh trên máy chủ, client vẫn sẽ thấy thời gian phản hồi tổng thể chậm do thời gian chờ yêu cầu trước hoàn thành. Vì hiệu ứng này, việc đo thời gian phản hồi ở phía client là rất quan trọng.

When generating load artificially in order to test the scalability of a system, the load-generating client needs to keep sending requests independently of the response time. If the client waits for the previous request to complete before sending the next one, that behavior has the effect of artificially keeping the queues shorter in the test than they would be in reality, which skews the measurements [23].

Khi tạo tải một cách nhân tạo để kiểm thử khả năng mở rộng của một hệ thống, client tạo tải cần tiếp tục gửi yêu cầu một cách độc lập với thời gian phản hồi. Nếu client chờ yêu cầu trước hoàn thành rồi mới gửi yêu cầu kế tiếp, hành vi đó có tác dụng giữ cho các hàng đợi trong bài kiểm thử ngắn hơn một cách giả tạo so với thực tế, làm sai lệch các phép đo [23].

Percentiles in Practice — Phân vị trong thực tế High percentiles become especially important in backend services that are called multiple times as part of serving a single end-user request. Even if you make the calls in parallel, the end-user request still needs to wait for the slowest of the parallel calls to complete. It takes just one slow call to make the entire end-user request slow, as illustrated in Figure 1-5. Even if only a small percentage of backend calls are slow, the chance of getting a slow call increases if an end-user request requires multiple backend calls, and so a higher proportion of end-user requests end up being slow (an effect known as **tail latency amplification** [24]).

Các phân vị cao trở nên đặc biệt quan trọng trong các dịch vụ backend được gọi nhiều lần như một phần của việc phục vụ một yêu cầu đơn lẻ từ người dùng cuối. Ngay cả khi bạn thực hiện các lời gọi song song, yêu cầu của người dùng cuối vẫn phải chờ lời gọi song song chậm nhất hoàn thành. Chỉ cần một lời gọi chậm là đủ làm chậm toàn bộ yêu cầu của người dùng cuối, như minh họa trong Hình 1-5. Ngay cả khi chỉ một tỷ lệ nhỏ các lời gọi backend bị chậm, khả năng gặp một lời gọi chậm vẫn tăng lên nếu một yêu cầu của người dùng cuối đòi hỏi nhiều lời gọi backend, và do đó một tỷ lệ cao hơn các yêu cầu của người dùng cuối rốt cuộc bị chậm (một hiệu ứng gọi là khuếch đại độ trễ đuôi [24]).

If you want to add response time percentiles to the monitoring dashboards for your services, you need to efficiently calculate them on an ongoing basis. For example, you may want to keep a **rolling window** of response times of requests in the last 10 minutes. Every minute, you calculate the median and various percentiles over the values in that window and plot those metrics on a graph.

Nếu bạn muốn thêm các phân vị thời gian phản hồi vào bảng điều khiển giám sát cho các dịch vụ của mình, bạn cần tính toán chúng một cách hiệu quả và liên tục. Ví dụ, bạn có thể muốn giữ một cửa sổ trượt chứa thời gian phản hồi của các yêu cầu trong 10 phút gần nhất. Mỗi phút, bạn tính trung vị và các phân vị khác nhau trên các giá trị trong cửa sổ đó rồi vẽ những chỉ số này lên một biểu đồ.

The naïve implementation is to keep a list of response times for all requests within the time window and to sort that list every minute. If that is too inefficient for you, there are algorithms that can calculate a good approximation of percentiles at minimal CPU and memory cost, such as forward decay [25], t-digest [26], or HdrHistogram [27]. Beware that averaging percentiles, e.g., to reduce the time resolution or to combine data from several machines, is mathematically meaningless—the right way of aggregating response time data is to add the histograms [28].

Cách hiện thực ngây thơ là giữ một danh sách thời gian phản hồi cho mọi yêu cầu trong cửa sổ thời gian và sắp xếp danh sách đó mỗi phút. Nếu cách đó quá kém hiệu quả với bạn, có những thuật toán có thể tính một xấp xỉ tốt cho các phân vị với chi phí CPU và bộ nhớ tối thiểu, chẳng hạn forward decay [25], t-digest [26], hay HdrHistogram [27]. Hãy cẩn

thận rằng việc lấy trung bình của các phân vị — ví dụ để giảm độ phân giải thời gian hoặc để kết hợp dữ liệu từ nhiều máy — là vô nghĩa về mặt toán học; cách đúng để tổng hợp dữ liệu thời gian phản hồi là cộng các biểu đồ tần suất lại [28].

Figure 1-5. When several backend calls are needed to serve a request, it takes just a single slow backend request to slow down the entire end-user request. — Hình 1-5. Khi cần nhiều lời gọi backend để phục vụ một yêu cầu, chỉ cần một yêu cầu backend chậm là đủ làm chậm toàn bộ yêu cầu của người dùng cuối.

Approaches for Coping with Load — Các cách tiếp cận để ứng phó với tải

Now that we have discussed the parameters for describing load and metrics for measuring performance, we can start discussing scalability in earnest: how do we maintain good performance even when our load parameters increase by some amount?

Giờ đây khi đã bàn về các tham số để mô tả tải và các chỉ số để đo hiệu năng, ta có thể bắt đầu bàn về khả năng mở rộng một cách nghiêm túc: làm sao ta duy trì hiệu năng tốt ngay cả khi các tham số tải tăng lên một lượng nào đó?

An architecture that is appropriate for one level of load is unlikely to cope with 10 times that load. If you are working on a fast-growing service, it is therefore likely that you will need to rethink your architecture on every order of magnitude load increase—or perhaps even more often than that.

Một kiến trúc phù hợp cho một mức tải nào đó khó lòng ứng phó nổi với mức tải gấp 10 lần. Nếu bạn đang làm việc với một dịch vụ tăng trưởng nhanh, nhiều khả năng bạn sẽ phải nghĩ lại kiến trúc của mình mỗi khi tải tăng thêm một bậc độ lớn — hoặc thậm chí thường xuyên hơn thế.

People often talk of a dichotomy between scaling up (**vertical scaling**, moving to a more powerful machine) and scaling out (**horizontal scaling**, distributing the load across multiple smaller machines). Distributing load across multiple machines is also known as a **shared-nothing architecture**. A system that can run on a single machine is often simpler, but high-end machines can become very expensive, so very intensive workloads often can't avoid scaling out. In reality, good architectures usually involve a pragmatic mixture of approaches: for example, using several fairly powerful machines can still be simpler and cheaper than a large number of small virtual machines.

Người ta thường nói về sự lưỡng phân giữa mở rộng theo chiều dọc (chuyển sang một máy mạnh hơn) và mở rộng theo chiều ngang (phân tán tải ra nhiều máy nhỏ hơn). Việc phân tán tải ra nhiều máy còn được gọi là kiến trúc shared-nothing. Một hệ thống có thể chạy trên một máy đơn lẻ thường đơn giản hơn, nhưng các máy cao cấp có thể trở nên rất đắt đỏ, nên những khối lượng công việc rất nặng thường không thể tránh khỏi việc mở rộng theo chiều ngang. Trên thực tế, các kiến trúc tốt thường bao gồm một sự pha trộn thực dụng giữa các cách tiếp cận: chẳng hạn, dùng vài máy khá mạnh vẫn có thể đơn giản và rẻ hơn so với một số lượng lớn máy ảo nhỏ.

Some systems are **elastic**, meaning that they can automatically add computing resources when they detect a load increase, whereas other systems are scaled manually (a human analyzes the capacity and decides to add more machines to the system). An elastic system can be useful if load is highly unpredictable, but manually scaled systems are simpler and may have fewer operational surprises (see “Rebalancing Partitions”).

Một số hệ thống có tính co giãn, nghĩa là chúng có thể tự động bổ sung tài nguyên tính toán khi phát hiện tải tăng lên, trong khi các hệ thống khác được mở rộng thủ công (một con người phân tích dung lượng và quyết định thêm máy vào hệ thống). Một hệ thống co giãn có thể hữu ích nếu tải rất khó dự đoán, nhưng các hệ thống mở rộng thủ công thì đơn giản hơn và có thể ít gây bất ngờ trong vận hành hơn (xem mục “Tái cân bằng phân vùng”).

While distributing **stateless** services across multiple machines is fairly straightforward, taking **stateful** data systems from a single node to a distributed setup can introduce a lot of additional complexity. For this reason, common wisdom until recently was to keep your database on a single node (**scale up**) until scaling cost or high-availability requirements forced you to make it distributed.

Trong khi việc phân tán các dịch vụ không trạng thái ra nhiều máy là khá đơn giản, thì việc đưa các hệ thống dữ liệu có trạng thái từ một nút đơn sang một thiết lập phân tán có thể tạo ra rất nhiều độ phức tạp phát sinh. Vì lý do này, hiểu biết phổ thông cho đến gần đây là giữ cơ sở dữ liệu của bạn trên một nút đơn (mở rộng theo chiều dọc) cho đến khi chi phí mở rộng hoặc các yêu cầu về tính sẵn sàng cao buộc bạn phải làm cho nó trở thành phân tán.

As the tools and abstractions for distributed systems get better, this common wisdom may change, at least for some kinds of applications. It is conceivable that distributed data systems will become the default in the future, even for use cases that don’t handle large volumes of data or traffic. Over the course of the rest of this book we will cover many kinds of distributed data systems, and discuss how they fare not just in terms of scalability, but also ease of use and maintainability.

Khi các công cụ và trừu tượng hóa cho hệ thống phân tán ngày càng tốt hơn, hiểu biết phổ thông này có thể thay đổi, ít nhất là với một số loại ứng dụng. Có thể hình dung rằng trong tương lai các hệ thống dữ liệu phân tán sẽ trở thành mặc định, ngay cả với những tình huống không xử lý lượng dữ liệu hay lưu lượng lớn. Trong suốt phần còn lại của cuốn sách, ta sẽ đề cập đến nhiều loại hệ thống dữ liệu phân tán, và bàn xem chúng xoay sở thế nào không chỉ về khả năng mở rộng, mà còn về tính dễ dùng và khả năng bảo trì.

The architecture of systems that operate at large scale is usually highly specific to the application—there is no such thing as a generic, one-size-fits-all scalable architecture (informally known as **magic scaling sauce**). The problem may be the volume of reads, the volume of writes, the volume of data to store, the complexity of the data, the response time requirements, the access patterns, or (usually) some mixture of all of these plus many more issues.

Kiến trúc của các hệ thống vận hành ở quy mô lớn thường rất đặc thù cho từng ứng dụng — không có thứ gì gọi là một kiến trúc khả mở rộng kiểu tổng quát, “một cỡ vừa cho tất cả” (mà dân trong nghề gọi đùa là “nước sốt mở rộng thần kỳ”). Vấn đề có thể là khối lượng đọc, khối lượng ghi, khối lượng dữ liệu cần lưu, độ phức tạp của dữ liệu, các yêu cầu về thời gian phản hồi, các mẫu truy cập, hoặc (thường là) một sự pha trộn của tất cả những điều này cộng thêm nhiều vấn đề khác.

For example, a system that is designed to handle 100,000 requests per second, each 1 kB in size, looks very different from a system that is designed for 3 requests per minute, each 2 GB in size—even though the two systems have the same data throughput.

Ví dụ, một hệ thống được thiết kế để xử lý 100.000 yêu cầu mỗi giây, mỗi yêu cầu 1 kB, trông rất khác so với một hệ thống được thiết kế cho 3 yêu cầu mỗi phút, mỗi yêu cầu 2 GB — dù hai hệ thống có cùng thông lượng dữ liệu.

An architecture that scales well for a particular application is built around assumptions of which operations will be common and which will be rare—the load parameters. If those assumptions turn out to be wrong, the engineering effort for scaling is at best wasted, and at worst counterproductive.

In an early-stage startup or an unproven product it's usually more important to be able to iterate quickly on product features than it is to scale to some hypothetical future load.

Một kiến trúc mở rộng tốt cho một ứng dụng cụ thể được xây dựng xoay quanh các giả định về thao tác nào sẽ phổ biến và thao tác nào sẽ hiếm — tức là các tham số tải. Nếu những giả định đó hóa ra sai, thì nỗ lực kỹ thuật để mở rộng tốt nhất là lãng phí, và tệ nhất là phản tác dụng. Ở một startup giai đoạn đầu hoặc một sản phẩm chưa được kiểm chứng, thường thì khả năng lặp nhanh trên các tính năng sản phẩm quan trọng hơn khả năng mở rộng tới một mức tải tương lai mang tính giả định.

Even though they are specific to a particular application, scalable architectures are nevertheless usually built from general-purpose building blocks, arranged in familiar patterns. In this book we discuss those building blocks and patterns.

Mặc dù đặc thù cho từng ứng dụng cụ thể, các kiến trúc khả mở rộng dù sao vẫn thường được xây dựng từ các khối dựng đa dụng, sắp xếp theo những mẫu quen thuộc. Trong cuốn sách này, ta bàn về những khối dựng và mẫu đó.

Maintainability — Khả năng bảo trì

It is well known that the majority of the cost of software is not in its initial development, but in its ongoing maintenance—fixing bugs, keeping its systems operational, investigating failures, adapting it to new platforms, modifying it for new use cases, repaying **technical debt**, and adding new features.

Ai cũng biết rằng phần lớn chi phí của phần mềm không nằm ở giai đoạn phát triển ban đầu, mà ở việc bảo trì liên tục — sửa bug, giữ cho các hệ thống vận hành được, điều tra sự cố, thích ứng với nền tảng mới, sửa đổi cho các tình huống dùng mới, trả nợ kỹ thuật, và thêm tính năng mới.

Yet, unfortunately, many people working on software systems dislike maintenance of so-called legacy systems—perhaps it involves fixing other people’s mistakes, or working with platforms that are now outdated, or systems that were forced to do things they were never intended for. Every legacy system is unpleasant in its own way, and so it is difficult to give general recommendations for dealing with them.

Thế nhưng, đáng tiếc là nhiều người làm việc với hệ thống phần mềm lại ghét việc bảo trì những hệ thống cũ — có lẽ vì nó liên quan đến việc sửa lỗi của người khác, hoặc làm việc với các nền tảng nay đã lỗi thời, hoặc với những hệ thống bị ép làm những việc chúng vốn không được thiết kế để làm. Mỗi hệ thống cũ lại khó chịu theo một kiểu riêng, nên rất khó đưa ra những khuyến nghị chung để xử lý chúng.

However, we can and should design software in such a way that it will hopefully minimize pain during maintenance, and thus avoid creating legacy software ourselves. To this end, we will pay particular attention to three design principles for software systems:

Tuy nhiên, ta có thể và nên thiết kế phần mềm theo cách mà hy vọng sẽ giảm thiểu nỗi đau trong quá trình bảo trì, và nhờ đó tránh việc chính ta tạo ra phần mềm cũ kỹ. Vì mục tiêu này, ta sẽ đặc biệt chú ý đến ba nguyên tắc thiết kế cho hệ thống phần mềm:

Make it easy for operations teams to keep the system running smoothly.

Khả năng vận hành: giúp các đội vận hành dễ dàng giữ cho hệ thống chạy trơn tru.

Make it easy for new engineers to understand the system, by removing as much complexity as possible from the system. (Note this is not the same as **simplicity** of the user interface.)

Tính đơn giản: giúp các kỹ sư mới dễ hiểu hệ thống, bằng cách loại bỏ càng nhiều độ phức tạp khỏi hệ thống càng tốt. (Lưu ý điều này không giống với sự đơn giản của giao diện người dùng.)

Make it easy for engineers to make changes to the system in the future, adapting it for unanticipated use cases as requirements change. Also known as **extensibility**, modifiability, or plasticity.

Khả năng tiến hóa: giúp các kỹ sư dễ dàng thay đổi hệ thống trong tương lai, thích ứng nó với những tình huống dùng không lường trước khi yêu cầu thay đổi. Còn được gọi là khả năng mở rộng tính năng, khả năng sửa đổi, hay tính dẻo.

As previously with reliability and scalability, there are no easy solutions for achieving these goals. Rather, we will try to think about systems with **operability**, simplicity, and **evolvability** in mind.

Như trước đây với độ tin cậy và khả năng mở rộng, không có giải pháp dễ dàng nào để đạt được những mục tiêu này. Thay vào đó, ta sẽ cố gắng tư duy về hệ thống với khả năng vận hành, tính đơn giản và khả năng tiến hóa trong đầu.

Operability: Making Life Easy for Operations — Khả năng vận hành: Làm cho cuộc sống của bộ phận vận hành dễ thở hơn

It has been suggested that “good operations can often work around the limitations of bad (or incomplete) software, but good software cannot run reliably with bad operations” [12]. While some aspects of operations can and should be automated, it is still up to humans to set up that automation in the first place and to make sure it’s working correctly.

Có ý kiến cho rằng “vận hành tốt thường có thể khắc phục những hạn chế của phần mềm tồi (hoặc chưa hoàn chỉnh), nhưng phần mềm tốt không thể chạy tin cậy với vận hành tồi” [12]. Mặc dù một số khía cạnh của vận hành có thể và nên được tự động hóa, thì việc thiết lập sự tự động hóa đó ngay từ đầu và đảm bảo nó hoạt động đúng vẫn phụ thuộc vào con người.

Operations teams are vital to keeping a software system running smoothly. A good operations team typically is responsible for the following, and more [29]:

Các đội vận hành đóng vai trò thiết yếu trong việc giữ cho một hệ thống phần mềm chạy trơn tru. Một đội vận hành tốt thường chịu trách nhiệm những việc sau, và còn nhiều hơn nữa [29]:

- Monitoring the health of the system and quickly restoring service if it goes into a bad state
- Tracking down the cause of problems, such as system failures or degraded performance
- Keeping software and platforms up to date, including security patches
- Keeping tabs on how different systems affect each other, so that a problematic change can be avoided before it causes damage
- Anticipating future problems and solving them before they occur (e.g., **capacity planning**)
- Establishing good practices and tools for **deployment**, configuration management, and more
- Performing complex maintenance tasks, such as moving an application from one platform to another
- Maintaining the security of the system as configuration changes are made
- Defining processes that make operations predictable and help keep the production environment stable
- Preserving the organization’s knowledge about the system, even as individual people come and go
 - Giám sát tình trạng của hệ thống và nhanh chóng khôi phục dịch vụ nếu nó rơi vào trạng thái xấu.
 - Truy tìm nguyên nhân của các vấn đề, chẳng hạn sự cố hệ thống hoặc hiệu năng suy giảm.
 - Cập nhật phần mềm và nền tảng, bao gồm các bản vá bảo mật.

- Theo dõi cách các hệ thống khác nhau ảnh hưởng lẫn nhau, để có thể tránh một thay đổi gây vấn đề trước khi nó gây thiệt hại.
- Lường trước các vấn đề tương lai và giải quyết chúng trước khi chúng xảy ra (ví dụ, hoạch định dung lượng).
- Thiết lập các thực hành và công cụ tốt cho triển khai, quản lý cấu hình, và hơn thế nữa.
- Thực hiện các tác vụ bảo trì phức tạp, chẳng hạn chuyển một ứng dụng từ nền tảng này sang nền tảng khác.
- Duy trì an ninh của hệ thống khi có các thay đổi cấu hình.
- Định nghĩa các quy trình giúp việc vận hành có thể dự đoán được và giữ cho môi trường production ổn định.
- Bảo tồn tri thức của tổ chức về hệ thống, ngay cả khi từng cá nhân đến rồi đi.

Good operability means making routine tasks easy, allowing the operations team to focus their efforts on high-value activities. Data systems can do various things to make routine tasks easy, including:

Khả năng vận hành tốt nghĩa là làm cho các tác vụ thường lệ trở nên dễ dàng, cho phép đội vận hành tập trung công sức vào những hoạt động giá trị cao. Các hệ thống dữ liệu có thể làm nhiều việc để khiến các tác vụ thường lệ dễ dàng hơn, bao gồm:

- Providing **visibility** into the runtime behavior and internals of the system, with good monitoring
- Providing good support for automation and integration with standard tools
- Avoiding dependency on individual machines (allowing machines to be taken down for maintenance while the system as a whole continues running uninterrupted)
- Providing good documentation and an easy-to-understand operational model (“If I do X, Y will happen”)
- Providing good default behavior, but also giving administrators the freedom to override defaults when needed
- **Self-healing** where appropriate, but also giving administrators manual control over the system state when needed
- Exhibiting predictable behavior, minimizing surprises
 - Cung cấp khả năng quan sát hành vi lúc chạy và nội bộ của hệ thống, với việc giám sát tốt.
 - Hỗ trợ tốt cho việc tự động hóa và tích hợp với các công cụ tiêu chuẩn.
 - Tránh phụ thuộc vào từng máy riêng lẻ (cho phép tắt máy để bảo trì trong khi cả hệ thống vẫn tiếp tục chạy không gián đoạn).
 - Cung cấp tài liệu tốt và một mô hình vận hành dễ hiểu (“Nếu tôi làm X, thì Y sẽ xảy ra”).
 - Cung cấp hành vi mặc định tốt, nhưng cũng cho quản trị viên quyền tự do ghi đè mặc định khi cần.
 - Tự phục hồi khi thích hợp, nhưng cũng cho quản trị viên quyền kiểm soát thủ công trạng thái hệ thống khi cần.
 - Thể hiện hành vi có thể dự đoán được, giảm thiểu bất ngờ.

Simplicity: Managing Complexity — Tính đơn giản: Quản lý độ phức tạp

Small software projects can have delightfully simple and expressive code, but as projects get larger, they often become very complex and difficult to understand. This complexity slows down everyone

who needs to work on the system, further increasing the cost of maintenance. A software project mired in complexity is sometimes described as a **big ball of mud** [30].

Các dự án phần mềm nhỏ có thể có mã nguồn đơn giản và giàu tính biểu đạt một cách thú vị, nhưng khi dự án lớn lên, chúng thường trở nên rất phức tạp và khó hiểu. Độ phức tạp này làm chậm tất cả những ai cần làm việc trên hệ thống, càng làm tăng chi phí bảo trì. Một dự án phần mềm sa lầy trong độ phức tạp đôi khi được mô tả là một “quả cầu bùn khổng lồ” [30].

There are various possible symptoms of complexity: explosion of the state space, **tight coupling** of modules, tangled dependencies, inconsistent naming and terminology, hacks aimed at solving performance problems, special-casing to work around issues elsewhere, and many more. Much has been said on this topic already [31, 32, 33].

Có nhiều triệu chứng khả dĩ của độ phức tạp: sự bùng nổ của không gian trạng thái, sự gắn kết chặt giữa các mô-đun, các phụ thuộc rối rắm, cách đặt tên và thuật ngữ thiếu nhất quán, những “mánh” nhằm giải quyết vấn đề hiệu năng, việc xử lý đặc biệt để né tránh các vấn đề ở nơi khác, và còn nhiều nữa. Đã có rất nhiều điều được bàn về chủ đề này [31, 32, 33].

When complexity makes maintenance hard, budgets and schedules are often overrun. In complex software, there is also a greater risk of introducing bugs when making a change: when the system is harder for developers to understand and reason about, hidden assumptions, unintended consequences, and unexpected interactions are more easily overlooked. Conversely, reducing complexity greatly improves the maintainability of software, and thus simplicity should be a key goal for the systems we build.

Khi độ phức tạp khiến việc bảo trì trở nên khó khăn, ngân sách và lịch trình thường bị vượt. Trong phần mềm phức tạp, cũng có nguy cơ cao hơn về việc tạo ra bug khi thực hiện một thay đổi: khi hệ thống khó hiểu và khó suy luận hơn đối với lập trình viên, thì những giả định ẩn, những hệ quả ngoài ý muốn và những tương tác bất ngờ dễ bị bỏ sót hơn. Ngược lại, việc giảm độ phức tạp cải thiện đáng kể khả năng bảo trì của phần mềm, và do đó tính đơn giản nên là một mục tiêu then chốt của những hệ thống ta xây dựng.

Making a system simpler does not necessarily mean reducing its functionality; it can also mean removing **accidental complexity**. Moseley and Marks [32] define complexity as accidental if it is not inherent in the problem that the software solves (as seen by the users) but arises only from the implementation.

Làm cho một hệ thống đơn giản hơn không nhất thiết có nghĩa là giảm chức năng của nó; nó cũng có thể nghĩa là loại bỏ độ phức tạp ngẫu sinh. Moseley và Marks [32] định nghĩa độ phức tạp là “ngẫu sinh” nếu nó không vốn có trong bài toán mà phần mềm giải quyết (theo cách người dùng nhìn nhận) mà chỉ phát sinh từ cách hiện thực.

One of the best tools we have for removing accidental complexity is abstraction. A good abstraction can hide a great deal of implementation detail behind a clean, simple-to-understand façade. A good abstraction can also be used for a wide range of different applications. Not only is this reuse more efficient than reimplementing a similar thing multiple times, but it also leads to higher-quality software, as quality improvements in the abstracted component benefit all applications that use it.

Một trong những công cụ tốt nhất ta có để loại bỏ độ phức tạp ngẫu sinh là trừu tượng hóa. Một sự trừu tượng hóa tốt có thể che giấu vô số chi tiết hiện thực sau một bề mặt sạch sẽ, dễ hiểu. Một sự trừu tượng hóa tốt cũng có thể được dùng cho nhiều ứng dụng khác nhau. Việc tái sử dụng này không chỉ hiệu quả hơn so với việc hiện thực lại một thứ tương tự nhiều lần, mà còn dẫn đến phần mềm chất lượng cao hơn, vì những cải thiện về chất

lượng trong thành phần được trừu tượng hóa sẽ mang lại lợi ích cho mọi ứng dụng dùng nó.

For example, high-level programming languages are abstractions that hide **machine code**, CPU registers, and syscalls. SQL is an abstraction that hides complex on-disk and in-memory data structures, concurrent requests from other clients, and inconsistencies after crashes. Of course, when programming in a high-level language, we are still using machine code; we are just not using it directly, because the programming language abstraction saves us from having to think about it.

Ví dụ, các ngôn ngữ lập trình bậc cao là những trừu tượng hóa che giấu mã máy, thanh ghi CPU và các lời gọi hệ thống. SQL là một trừu tượng hóa che giấu các cấu trúc dữ liệu phức tạp trên đĩa và trong bộ nhớ, các yêu cầu đồng thời từ những client khác, và các tình trạng thiếu nhất quán sau khi sập. Tất nhiên, khi lập trình bằng một ngôn ngữ bậc cao, ta vẫn đang dùng mã máy; ta chỉ là không dùng nó trực tiếp, vì sự trừu tượng hóa của ngôn ngữ lập trình giúp ta khỏi phải nghĩ về nó.

However, finding good abstractions is very hard. In the field of distributed systems, although there are many good algorithms, it is much less clear how we should be packaging them into abstractions that help us keep the complexity of the system at a manageable level.

Tuy nhiên, tìm ra những trừu tượng hóa tốt là rất khó. Trong lĩnh vực hệ thống phân tán, mặc dù có nhiều thuật toán tốt, vẫn chưa rõ ràng lắm về việc ta nên đóng gói chúng thành những trừu tượng hóa thế nào để giúp giữ độ phức tạp của hệ thống ở mức quản lý được.

Throughout this book, we will keep our eyes open for good abstractions that allow us to extract parts of a large system into well-defined, reusable components.

Xuyên suốt cuốn sách này, ta sẽ luôn để mắt tìm những trừu tượng hóa tốt cho phép ta tách các phần của một hệ thống lớn thành những thành phần được định nghĩa rõ ràng, có thể tái sử dụng.

Evolvability: Making Change Easy — Khả năng tiến hóa: Làm cho việc thay đổi trở nên dễ dàng

It's extremely unlikely that your system's requirements will remain unchanged forever. They are much more likely to be in constant flux: you learn new facts, previously unanticipated use cases emerge, business priorities change, users request new features, new platforms replace old platforms, legal or regulatory requirements change, growth of the system forces architectural changes, etc.

Cực kỳ khó có chuyện các yêu cầu của hệ thống của bạn sẽ giữ nguyên không đổi mãi mãi. Nhiều khả năng hơn là chúng sẽ luôn biến động: bạn học được những sự thật mới, những tình huống dùng trước đây không lường trước nay xuất hiện, các ưu tiên kinh doanh thay đổi, người dùng yêu cầu tính năng mới, các nền tảng mới thay thế nền tảng cũ, các yêu cầu pháp lý hay quy định thay đổi, sự tăng trưởng của hệ thống buộc phải thay đổi kiến trúc, v.v.

In terms of organizational processes, **Agile** working patterns provide a framework for adapting to change. The Agile community has also developed technical tools and patterns that are helpful when developing software in a frequently changing environment, such as **test-driven development** (TDD) and **refactoring**.

Về mặt quy trình tổ chức, các mô hình làm việc Agile cung cấp một khung để thích ứng với thay đổi. Cộng đồng Agile cũng đã phát triển những công cụ và mẫu kỹ thuật hữu ích khi

phát triển phần mềm trong một môi trường thay đổi thường xuyên, chẳng hạn phát triển hướng kiểm thử (TDD) và tái cấu trúc.

Most discussions of these Agile techniques focus on a fairly small, local scale (a couple of source code files within the same application). In this book, we search for ways of increasing agility on the level of a larger data system, perhaps consisting of several different applications or services with different characteristics. For example, how would you “refactor” Twitter’s architecture for assembling home timelines (“Describing Load”) from approach 1 to approach 2?

Hầu hết các thảo luận về những kỹ thuật Agile này tập trung vào một quy mô khá nhỏ, cục bộ (một vài file mã nguồn trong cùng một ứng dụng). Trong cuốn sách này, ta tìm những cách tăng tính linh hoạt ở cấp độ của một hệ thống dữ liệu lớn hơn, có thể gồm nhiều ứng dụng hoặc dịch vụ khác nhau với những đặc tính khác nhau. Ví dụ, bạn sẽ “tái cấu trúc” kiến trúc lắp ráp dòng thời gian chính của Twitter (mục “Mô tả tải”) từ cách 1 sang cách 2 như thế nào?

The ease with which you can modify a data system, and adapt it to changing requirements, is closely linked to its simplicity and its abstractions: simple and easy-to-understand systems are usually easier to modify than complex ones. But since this is such an important idea, we will use a different word to refer to agility on a data system level: *evolvability* [34].

Mức độ dễ dàng mà bạn có thể sửa đổi một hệ thống dữ liệu, và thích ứng nó với những yêu cầu thay đổi, gắn liền chặt chẽ với tính đơn giản và các trừu tượng hóa của nó: những hệ thống đơn giản và dễ hiểu thường dễ sửa đổi hơn những hệ thống phức tạp. Nhưng vì đây là một ý tưởng quan trọng đến vậy, ta sẽ dùng một từ khác để chỉ tính linh hoạt ở cấp độ hệ thống dữ liệu: khả năng tiến hóa [34].

Summary — Tóm tắt

In this chapter, we have explored some fundamental ways of thinking about data-intensive applications. These principles will guide us through the rest of the book, where we dive into deep technical detail.

Trong chương này, ta đã khám phá một số cách tư duy nền tảng về các ứng dụng thiên về dữ liệu. Những nguyên lý này sẽ dẫn dắt ta xuyên suốt phần còn lại của cuốn sách, nơi ta đi sâu vào chi tiết kỹ thuật.

An application has to meet various requirements in order to be useful. There are functional requirements (what it should do, such as allowing data to be stored, retrieved, searched, and processed in various ways), and some nonfunctional requirements (general properties like security, reliability, compliance, scalability, compatibility, and maintainability). In this chapter we discussed reliability, scalability, and maintainability in detail.

Một ứng dụng phải đáp ứng nhiều yêu cầu khác nhau để trở nên hữu ích. Có những yêu cầu chức năng (nó nên làm gì, chẳng hạn cho phép dữ liệu được lưu, truy xuất, tìm kiếm và xử lý theo nhiều cách), và một số yêu cầu phi chức năng (những thuộc tính chung như an ninh, độ tin cậy, sự tuân thủ, khả năng mở rộng, khả năng tương thích và khả năng bảo trì). Trong chương này, ta đã bàn chi tiết về độ tin cậy, khả năng mở rộng và khả năng bảo trì.

Reliability means making systems work correctly, even when faults occur. Faults can be in hardware (typically random and uncorrelated), software (bugs are typically systematic and hard to deal with), and humans (who inevitably make mistakes from time to time). Fault-tolerance techniques can hide certain types of faults from the end user.

Độ tin cậy nghĩa là làm cho hệ thống hoạt động đúng, ngay cả khi lỗi xảy ra. Lỗi có thể nằm ở phần cứng (thường ngẫu nhiên và không tương quan), phần mềm (bug thường mang tính hệ thống và khó xử lý), và con người (những người chắc chắn thỉnh thoảng vẫn phạm sai lầm). Các kỹ thuật chịu lỗi có thể che giấu một số loại lỗi nhất định khỏi người dùng cuối.

Scalability means having strategies for keeping performance good, even when load increases. In order to discuss scalability, we first need ways of describing load and performance quantitatively. We briefly looked at Twitter's home timelines as an example of describing load, and response time percentiles as a way of measuring performance. In a scalable system, you can add processing capacity in order to remain reliable under high load.

Khả năng mở rộng nghĩa là có các chiến lược để giữ hiệu năng tốt, ngay cả khi tải tăng lên. Để bàn về khả năng mở rộng, trước tiên ta cần những cách mô tả tải và hiệu năng một cách định lượng. Ta đã xem qua dòng thời gian chính của Twitter như một ví dụ về mô tả tải, và các phân vị thời gian phản hồi như một cách đo hiệu năng. Trong một hệ thống khả

mở rộng, bạn có thể bổ sung năng lực xử lý để vẫn giữ được độ tin cậy dưới tải cao.

Maintainability has many facets, but in essence it's about making life better for the engineering and operations teams who need to work with the system. Good abstractions can help reduce complexity and make the system easier to modify and adapt for new use cases. Good operability means having good visibility into the system's health, and having effective ways of managing it.

Khả năng bảo trì có nhiều khía cạnh, nhưng về bản chất là làm cho cuộc sống tốt đẹp hơn cho các đội kỹ thuật và vận hành cần làm việc với hệ thống. Những trừu tượng hóa tốt có thể giúp giảm độ phức tạp và làm cho hệ thống dễ sửa đổi và thích ứng cho các tình huống dùng mới. Khả năng vận hành tốt nghĩa là có khả năng quan sát tốt tình trạng của hệ thống, và có những cách hiệu quả để quản lý nó.

There is unfortunately no easy fix for making applications reliable, scalable, or maintainable. However, there are certain patterns and techniques that keep reappearing in different kinds of applications. In the next few chapters we will take a look at some examples of data systems and analyze how they work toward those goals.

Đáng tiếc là không có cách khắc phục dễ dàng nào để làm cho ứng dụng trở nên tin cậy, khả mở rộng hay dễ bảo trì. Tuy nhiên, có những mẫu và kỹ thuật nhất định cứ tái xuất hiện trong nhiều loại ứng dụng khác nhau. Trong vài chương tới, ta sẽ xem một số ví dụ về các hệ thống dữ liệu và phân tích cách chúng hướng tới những mục tiêu đó.

Later in the book, in Part III, we will look at patterns for systems that consist of several components working together, such as the one in Figure 1-1.

Về sau trong cuốn sách, ở Phần III, ta sẽ xem xét các mẫu cho những hệ thống gồm nhiều thành phần phối hợp với nhau, chẳng hạn như cái ở Hình 1-1.

Footnotes Defined in “Approaches for Coping with Load”.

A term borrowed from electronic engineering, where it describes the number of logic gate inputs that are attached to another gate's output. The output needs to supply enough current to drive all the attached inputs. In transaction processing systems, we use it to describe the number of requests to other services that we need to make in order to serve one incoming request.

In an ideal world, the running time of a batch job is the size of the dataset divided by the throughput. In practice, the running time is often longer, due to skew (data not being spread evenly across worker processes) and needing to wait for the slowest task to complete.

References [1] Michael Stonebraker and Uğur Çetintemel: “‘One Size Fits All’: An Idea Whose Time Has Come and Gone,” at 21st International Conference on Data Engineering (ICDE), April 2005.

[2] Walter L. Heimerdinger and Charles B. Weinstock: “A Conceptual Framework for System Fault Tolerance,” Technical Report CMU/SEI-92-TR-033, Software Engineering Institute, Carnegie Mellon University, October 1992.

[3] Ding Yuan, Yu Luo, Xin Zhuang, et al.: “Simple Testing Can Prevent Most Critical Failures: An Analysis of Production Failures in Distributed Data-Intensive Systems,” at 11th USENIX Symposium on Operating Systems Design and Implementation (OSDI), October 2014.

[4] Yury Izrailevsky and Ariel Tseitlin: “The Netflix Simian Army,” techblog.netflix.com, July 19, 2011.

[5] Daniel Ford, François Labelle, Florentina I. Popovici, et al.: “Availability in Globally Distributed Storage Systems,” at 9th USENIX Symposium on Operating Systems Design and Implementation (OSDI), October 2010.

- [6] Brian Beach: “Hard Drive Reliability Update – Sep 2014,” backblaze.com, September 23, 2014.
- [7] Laurie Voss: “AWS: The Good, the Bad and the Ugly,” blog.awe.sm, December 18, 2012.
- [8] Haryadi S. Gunawi, Mingzhe Hao, Tanakorn Leesatapornwongsa, et al.: “What Bugs Live in the Cloud?,” at 5th ACM Symposium on Cloud Computing (SoCC), November 2014. doi:10.1145/2670979.2670986
- [9] Nelson Minar: “Leap Second Crashes Half the Internet,” somebits.com, July 3, 2012.
- [10] Amazon Web Services: “Summary of the Amazon EC2 and Amazon RDS Service Disruption in the US East Region,” aws.amazon.com, April 29, 2011.
- [11] Richard I. Cook: “How Complex Systems Fail,” Cognitive Technologies Laboratory, April 2000.
- [12] Jay Kreps: “Getting Real About Distributed System Reliability,” blog.empathybox.com, March 19, 2012.
- [13] David Oppenheimer, Archana Ganapathi, and David A. Patterson: “Why Do Internet Services Fail, and What Can Be Done About It?,” at 4th USENIX Symposium on Internet Technologies and Systems (USITS), March 2003.
- [14] Nathan Marz: “Principles of Software Engineering, Part 1,” nathanmarz.com, April 2, 2013.
- [15] Michael Jurewitz: “The Human Impact of Bugs,” jury.me, March 15, 2013.
- [16] Raffi Krikorian: “Timelines at Scale,” at QCon San Francisco, November 2012.
- [17] Martin Fowler: Patterns of Enterprise Application Architecture. Addison Wesley, 2002. ISBN: 978-0-321-12742-6
- [18] Kelly Sommers: “After all that run around, what caused 500ms disk latency even when we replaced physical server?” twitter.com, November 13, 2014.
- [19] Giuseppe DeCandia, Deniz Hastorun, Madan Jampani, et al.: “Dynamo: Amazon’s Highly Available Key-Value Store,” at 21st ACM Symposium on Operating Systems Principles (SOSP), October 2007.
- [20] Greg Linden: “Make Data Useful,” slides from presentation at Stanford University Data Mining class (CS345), December 2006.
- [21] Tammy Everts: “The Real Cost of Slow Time vs Downtime,” webperformancetoday.com, November 12, 2014.
- [22] Jake Brutlag: “Speed Matters for Google Web Search,” googleresearch.blogspot.co.uk, June 22, 2009.
- [23] Tyler Treat: “Everything You Know About Latency Is Wrong,” bravenewgeek.com, December 12, 2015.
- [24] Jeffrey Dean and Luiz André Barroso: “The Tail at Scale,” Communications of the ACM, volume 56, number 2, pages 74–80, February 2013. doi:10.1145/2408776.2408794
- [25] Graham Cormode, Vladislav Shkapenyuk, Divesh Srivastava, and Bojian Xu: “Forward Decay: A Practical Time Decay Model for Streaming Systems,” at 25th IEEE International Conference on Data Engineering (ICDE), March 2009.
- [26] Ted Dunning and Otmar Ertl: “Computing Extremely Accurate Quantiles Using t-Digests,” github.com, March 2014.
- [27] Gil Tene: “HdrHistogram,” hdrhistogram.org.

- [28] Baron Schwartz: “Why Percentiles Don’t Work the Way You Think,” vividcortex.com, December 7, 2015.
- [29] James Hamilton: “On Designing and Deploying Internet-Scale Services,” at 21st Large Installation System Administration Conference (LISA), November 2007.
- [30] Brian Foote and Joseph Yoder: “Big Ball of Mud,” at 4th Conference on Pattern Languages of Programs (PloP), September 1997.
- [31] Frederick P Brooks: “No Silver Bullet – Essence and Accident in Software Engineering,” in The Mythical Man-Month, Anniversary edition, Addison-Wesley, 1995. ISBN: 978-0-201-83595-3
- [32] Ben Moseley and Peter Marks: “Out of the Tar Pit,” at BCS Software Practice Advancement (SPA), 2006.
- [33] Rich Hickey: “Simple Made Easy,” at Strange Loop, September 2011.
- [34] Hongyu Pei Breivold, Ivica Crnkovic, and Peter J. Eriksson: “Analyzing Software Evolvability,” at 32nd Annual IEEE International Computer Software and Applications Conference (COMPSAC), July 2008. doi:10.1109/COMPSAC.2008.50

Chapter 2. Data Models and Query Languages — Mô hình dữ liệu và ngôn ngữ truy vấn

The limits of my language **mean** the limits of my world.

Giới hạn ngôn ngữ của tôi cũng chính là giới hạn thế giới của tôi.

Ludwig Wittgenstein, *Tractatus Logico-Philosophicus* (1922)

— Ludwig Wittgenstein, *Tractatus Logico-Philosophicus* (1922)

Data models are perhaps the most important part of developing software, because they have such a profound effect: not only on how the software is written, but also on how we think about the problem that we are solving.

Mô hình dữ liệu có lẽ là phần quan trọng nhất khi phát triển phần mềm, bởi chúng có ảnh hưởng sâu sắc: không chỉ tới cách phần mềm được viết ra, mà còn tới cách ta suy nghĩ về chính bài toán đang giải.

Most applications are built by layering one **data model** on top of another. For each layer, the key question is: how is it represented in terms of the next-lower layer? For example:

Hầu hết ứng dụng được xây dựng bằng cách xếp chồng mô hình dữ liệu này lên mô hình dữ liệu khác. Với mỗi lớp, câu hỏi mấu chốt là: nó được biểu diễn như thế nào dựa trên lớp ngay bên dưới? Ví dụ:

- As an application developer, you look at the real world (in which there are people, organizations, goods, actions, money flows, sensors, etc.) and model it in terms of objects or data structures, and APIs that manipulate those data structures. Those structures are often specific to your application.
- When you want to store those data structures, you express them in terms of a general-purpose data model, such as **JSON** or **XML** documents, tables in a **relational database**, or a **graph** model.
- The engineers who built your **database** software decided on a way of representing that JSON/XML/relational/graph data in terms of bytes in memory, on disk, or on a network. The representation may allow the data to be queried, searched, manipulated, and processed in various ways.
- On yet lower levels, hardware engineers have figured out how to represent bytes in terms of electrical currents, pulses of light, magnetic fields, and more.
- Là lập trình viên ứng dụng, bạn nhìn vào thế giới thực (gồm con người, tổ chức, hàng

hóa, hành động, dòng tiền, cảm biến, v.v.) và mô hình hóa nó thành các đối tượng hoặc cấu trúc dữ liệu, cùng những API thao tác trên các cấu trúc đó. Những cấu trúc này thường đặc thù cho ứng dụng của bạn.

- Khi muốn lưu trữ các cấu trúc dữ liệu đó, bạn biểu diễn chúng qua một mô hình dữ liệu đa dụng, chẳng hạn tài liệu JSON hoặc XML, các bảng trong cơ sở dữ liệu quan hệ, hay một mô hình đồ thị.
- Các kỹ sư xây dựng phần mềm cơ sở dữ liệu của bạn đã quyết định cách biểu diễn dữ liệu JSON/XML/quan hệ/đồ thị đó dưới dạng các byte trong bộ nhớ, trên đĩa, hoặc trên mạng. Cách biểu diễn này có thể cho phép truy vấn, tìm kiếm, thao tác và xử lý dữ liệu theo nhiều cách.
- Ở những lớp thấp hơn nữa, các kỹ sư phần cứng đã tìm ra cách biểu diễn các byte qua dòng điện, xung ánh sáng, từ trường, và hơn thế nữa.

In a complex application there may be more intermediary levels, such as APIs built upon APIs, but the basic idea is still the same: each layer hides the complexity of the layers below it by providing a clean data model. These abstractions allow different groups of people—for example, the engineers at the database vendor and the application developers using their database—to work together effectively.

Trong một ứng dụng phức tạp có thể có thêm nhiều lớp trung gian, chẳng hạn các API được dựng trên các API khác, nhưng ý tưởng cơ bản vẫn vậy: mỗi lớp che giấu sự phức tạp của các lớp bên dưới bằng cách cung cấp một mô hình dữ liệu gọn gàng. Những trừu tượng hóa này cho phép các nhóm người khác nhau — ví dụ các kỹ sư ở nhà cung cấp cơ sở dữ liệu và các lập trình viên ứng dụng dùng cơ sở dữ liệu của họ — phối hợp với nhau một cách hiệu quả.

There are many different kinds of data models, and every data model embodies assumptions about how it is going to be used. Some kinds of usage are easy and some are not supported; some operations are fast and some perform badly; some data transformations feel natural and some are awkward.

Có rất nhiều loại mô hình dữ liệu khác nhau, và mỗi mô hình hàm chứa những giả định về cách nó sẽ được sử dụng. Một số kiểu sử dụng thì dễ dàng, số khác lại không được hỗ trợ; một số thao tác thì nhanh, số khác lại chậm tệ hại; một số phép biến đổi dữ liệu thì tự nhiên, số khác lại gượng gạo.

It can take a lot of effort to master just one data model (think how many books there are on relational data modeling). Building software is hard enough, even when working with just one data model and without worrying about its inner workings. But since the data model has such a profound effect on what the software above it can and can't do, it's important to choose one that is appropriate to the application.

Có thể tốn rất nhiều công sức chỉ để thành thạo một mô hình dữ liệu (hãy nghĩ xem có bao nhiêu cuốn sách viết về mô hình hóa dữ liệu quan hệ). Xây dựng phần mềm vốn đã đủ khó, kể cả khi chỉ làm việc với một mô hình dữ liệu duy nhất và không cần bận tâm tới cách nó vận hành bên trong. Nhưng vì mô hình dữ liệu có ảnh hưởng sâu sắc tới những gì phần mềm bên trên nó có thể và không thể làm, nên việc chọn một mô hình phù hợp với ứng dụng là rất quan trọng.

In this chapter we will look at a range of general-purpose data models for data storage and querying (point 2 in the preceding list). In particular, we will compare the **relational model**, the **document model**, and a few graph-based data models. We will also look at various query languages and compare their use cases. In Chapter 3 we will discuss how storage engines work; that is, how these data models are actually implemented (point 3 in the list).

Trong chương này, ta sẽ xem xét một loạt mô hình dữ liệu đa dụng dùng cho lưu trữ và

truy vấn (điểm 2 trong danh sách phía trên). Cụ thể, ta sẽ so sánh mô hình quan hệ, mô hình tài liệu, và một vài mô hình dữ liệu dựa trên đồ thị. Ta cũng sẽ xem xét nhiều ngôn ngữ truy vấn khác nhau và so sánh các tình huống sử dụng của chúng. Ở Chương 3 ta sẽ bàn về cách các engine lưu trữ hoạt động; tức là các mô hình dữ liệu này thực sự được hiện thực ra sao (điểm 3 trong danh sách).

Relational Model Versus Document Model — Mô hình quan hệ so với mô hình tài liệu

The best-known data model today is probably that of **SQL**, based on the relational model proposed by Edgar Codd in 1970 [1]: data is organized into relations (called tables in SQL), where each **relation** is an unordered collection of tuples (rows in SQL).

Mô hình dữ liệu nổi tiếng nhất hiện nay có lẽ là mô hình của SQL, dựa trên mô hình quan hệ do Edgar Codd đề xuất năm 1970 [1]: dữ liệu được tổ chức thành các quan hệ (gọi là bảng trong SQL), trong đó mỗi quan hệ là một tập hợp không có thứ tự các bộ (hàng trong SQL).

The relational model was a theoretical proposal, and many people at the time doubted whether it could be implemented efficiently. However, by the mid-1980s, relational database management systems (RDBMSes) and SQL had become the tools of choice for most people who needed to store and query data with some kind of regular structure. The dominance of relational databases has lasted around 25–30 years—an eternity in computing history.

Mô hình quan hệ là một đề xuất mang tính lý thuyết, và nhiều người thời đó nghi ngờ liệu nó có thể được hiện thực một cách hiệu quả hay không. Tuy nhiên, đến giữa thập niên 1980, các hệ quản trị cơ sở dữ liệu quan hệ (RDBMS) và SQL đã trở thành công cụ được chọn lựa cho hầu hết những ai cần lưu trữ và truy vấn dữ liệu có cấu trúc tương đối quy củ. Sự thống trị của cơ sở dữ liệu quan hệ đã kéo dài khoảng 25–30 năm — một quãng thời gian dài đẳng đẳng trong lịch sử ngành máy tính.

The roots of relational databases lie in business data processing, which was performed on mainframe computers in the 1960s and '70s. The use cases appear mundane from today's perspective: typically **transaction processing** (entering sales or banking transactions, airline reservations, stock-keeping in warehouses) and **batch processing** (customer invoicing, payroll, reporting).

Cội nguồn của cơ sở dữ liệu quan hệ nằm ở xử lý dữ liệu nghiệp vụ, vốn được thực hiện trên các máy mainframe trong thập niên 1960 và 1970. Các tình huống sử dụng nhìn từ hôm nay có vẻ tầm thường: thường là xử lý giao dịch (nhập giao dịch bán hàng hoặc ngân hàng, đặt vé máy bay, quản lý tồn kho trong nhà kho) và xử lý theo lô (lập hóa đơn khách hàng, tính lương, làm báo cáo).

Other databases at that time forced application developers to think a lot about the internal representation of the data in the database. The goal of the relational model was to hide that implementation detail behind a cleaner interface.

Các cơ sở dữ liệu khác ở thời đó buộc lập trình viên ứng dụng phải suy nghĩ rất nhiều về cách dữ liệu được biểu diễn nội bộ bên trong cơ sở dữ liệu. Mục tiêu của mô hình quan hệ là giấu chi tiết hiện thực đó đằng sau một giao diện gọn gàng hơn.

Over the years, there have been many competing approaches to data storage and querying. In the 1970s and early 1980s, the **network model** and the **hierarchical model** were the main alternatives, but the relational model came to dominate them. Object databases came and went again in the late 1980s and early 1990s. XML databases appeared in the early 2000s, but have only seen niche adoption. Each competitor to the relational model generated a lot of hype in its time, but it never lasted [2].

Qua nhiều năm, đã có nhiều cách tiếp cận cạnh tranh nhau trong việc lưu trữ và truy vấn dữ liệu. Trong thập niên 1970 và đầu thập niên 1980, mô hình mạng và mô hình phân cấp là những lựa chọn thay thế chính, nhưng mô hình quan hệ rốt cuộc đã thống trị chúng. Cơ sở dữ liệu đối tượng xuất hiện rồi lại biến mất vào cuối thập niên 1980 và đầu thập niên 1990. Cơ sở dữ liệu XML xuất hiện vào đầu những năm 2000, nhưng chỉ được áp dụng trong những ngách hẹp. Mỗi đối thủ của mô hình quan hệ đều tạo ra rất nhiều ồn ào vào thời của nó, nhưng chưa bao giờ trụ vững được [2].

As computers became vastly more powerful and networked, they started being used for increasingly diverse purposes. And remarkably, relational databases turned out to generalize very well, beyond their original scope of business data processing, to a broad variety of use cases. Much of what you see on the web today is still powered by relational databases, be it online publishing, discussion, social networking, ecommerce, games, software-as-a-service productivity applications, or much more.

Khi máy tính trở nên mạnh mẽ hơn nhiều và được kết nối mạng, chúng bắt đầu được dùng cho những mục đích ngày càng đa dạng. Và đáng chú ý là, cơ sở dữ liệu quan hệ hóa ra lại tổng quát hóa rất tốt, vượt ra ngoài phạm vi ban đầu là xử lý dữ liệu nghiệp vụ, sang vô số tình huống sử dụng khác. Phần lớn những gì bạn thấy trên web ngày nay vẫn được vận hành bởi cơ sở dữ liệu quan hệ, dù đó là xuất bản trực tuyến, thảo luận, mạng xã hội, thương mại điện tử, trò chơi, các ứng dụng năng suất dạng phần-mềm-như-dịch-vụ, hay nhiều thứ khác nữa.

The Birth of NoSQL — Sự ra đời của NoSQL

Now, in the 2010s, **NoSQL** is the latest attempt to overthrow the relational model's dominance. The name “NoSQL” is unfortunate, since it doesn't actually refer to any particular technology—it was originally intended simply as a catchy Twitter hashtag for a meetup on open source, distributed, nonrelational databases in 2009 [3]. Nevertheless, the term struck a nerve and quickly spread through the web startup community and beyond. A number of interesting database systems are now associated with the #NoSQL hashtag, and it has been retroactively reinterpreted as Not Only SQL [4].

Giờ đây, ở thập niên 2010, NoSQL là nỗ lực mới nhất nhằm lật đổ sự thống trị của mô hình quan hệ. Cái tên “NoSQL” thật đáng tiếc, vì nó thực ra không chỉ tới một công nghệ cụ thể nào — ban đầu nó chỉ đơn giản là một hashtag Twitter bắt tai cho một buổi gặp mặt về cơ sở dữ liệu phi quan hệ, phân tán, mã nguồn mở vào năm 2009 [3]. Tuy vậy, thuật ngữ này đã chạm đúng mạch và nhanh chóng lan rộng trong cộng đồng startup web và xa hơn nữa. Một số hệ cơ sở dữ liệu thú vị hiện được gắn với hashtag #NoSQL, và nó đã được diễn giải lại về sau thành Not Only SQL (Không chỉ SQL) [4].

There are several driving forces behind the adoption of NoSQL databases, including:

Có vài động lực thúc đẩy việc áp dụng cơ sở dữ liệu NoSQL, bao gồm:

- A need for greater **scalability** than relational databases can easily achieve, including very large datasets or very high write **throughput**
- A widespread preference for free and open source software over commercial database products
- Specialized query operations that are not well supported by the relational model
- Frustration with the restrictiveness of relational schemas, and a desire for a more dynamic and expressive data model [5]
 - Nhu cầu về khả năng mở rộng lớn hơn so với mức mà cơ sở dữ liệu quan hệ dễ dàng đạt được, bao gồm các tập dữ liệu cực lớn hoặc thông lượng ghi rất cao.
 - Sự ưa chuộng rộng rãi dành cho phần mềm miễn phí và mã nguồn mở thay vì các sản phẩm cơ sở dữ liệu thương mại.
 - Các thao tác truy vấn chuyên biệt mà mô hình quan hệ hỗ trợ không tốt.
 - Sự bức bối với tính ràng buộc của các lược đồ quan hệ, cùng mong muốn có một mô hình dữ liệu linh hoạt và giàu sức biểu đạt hơn [5].

Different applications have different requirements, and the best choice of technology for one use case may well be different from the best choice for another use case. It therefore seems likely that in the foreseeable future, relational databases will continue to be used alongside a broad variety of nonrelational datastores—an idea that is sometimes called **polyglot persistence** [3].

Các ứng dụng khác nhau có những yêu cầu khác nhau, và lựa chọn công nghệ tốt nhất cho một tình huống sử dụng này rất có thể khác với lựa chọn tốt nhất cho một tình huống khác. Do đó, có vẻ như trong tương lai gần, cơ sở dữ liệu quan hệ sẽ tiếp tục được dùng song song với vô số kho dữ liệu phi quan hệ khác nhau — một ý tưởng đôi khi được gọi là lưu trữ đa hệ (polyglot persistence) [3].

The Object-Relational Mismatch — Lệch pha đối tượng–quan hệ

Most application development today is done in object-oriented programming languages, which leads to a common criticism of the SQL data model: if data is stored in relational tables, an awkward translation layer is required between the objects in the **application code** and the database model of tables, rows, and columns. The disconnect between the models is sometimes called an **impedance mismatch**.

Phần lớn việc phát triển ứng dụng ngày nay được thực hiện bằng các ngôn ngữ lập trình hướng đối tượng, điều này dẫn tới một lời phê bình phổ biến dành cho mô hình dữ liệu SQL: nếu dữ liệu được lưu trong các bảng quan hệ, thì cần một lớp chuyển đổi gượng gạo giữa các đối tượng trong mã ứng dụng và mô hình bảng, hàng, cột của cơ sở dữ liệu. Sự lệch lạc giữa hai mô hình này đôi khi được gọi là lệch trở kháng (impedance mismatch).

Object-relational mapping (**ORM**) frameworks like ActiveRecord and Hibernate reduce the amount of **boilerplate** code required for this translation layer, but they can't completely hide the differences between the two models.

Các framework ánh xạ đối tượng–quan hệ (ORM) như ActiveRecord và Hibernate giúp giảm lượng mã khuôn mẫu (boilerplate) cần cho lớp chuyển đổi này, nhưng chúng không thể che giấu hoàn toàn những khác biệt giữa hai mô hình.

For example, Figure 2-1 illustrates how a résumé (a LinkedIn profile) could be expressed in a relational **schema**. The profile as a whole can be identified by a unique identifier, `user_id`. Fields like `first_name` and `last_name` appear exactly once per user, so they can be modeled as columns on the `users` **table**. However, most people have had more than one job in their career (positions), and people may have varying numbers of periods of education and any number of pieces of contact

information. There is a **one-to-many** relationship from the user to these items, which can be represented in various ways:

Ví dụ, Hình 2-1 minh họa cách một sơ yếu lý lịch (một hồ sơ LinkedIn) có thể được biểu diễn trong một lược đồ quan hệ. Toàn bộ hồ sơ có thể được nhận diện bằng một định danh duy nhất, `user_id`. Các trường như `first_name` và `last_name` xuất hiện đúng một lần cho mỗi người dùng, nên chúng có thể được mô hình hóa thành các cột trên bảng `users`. Tuy nhiên, hầu hết mọi người đã trải qua nhiều hơn một công việc trong sự nghiệp (positions), và mỗi người có thể có số giai đoạn học tập khác nhau và số lượng thông tin liên hệ bất kỳ. Có một quan hệ một-nhiều từ người dùng tới những mục này, và quan hệ đó có thể được biểu diễn theo nhiều cách:

- In the traditional SQL model (prior to SQL:1999), the most common normalized representation is to put positions, education, and contact information in separate tables, with a **foreign key** reference to the users table, as in Figure 2-1.
- Later versions of the SQL standard added support for structured datatypes and XML data; this allowed multi-valued data to be stored within a single **row**, with support for querying and indexing inside those documents. These features are supported to varying degrees by Oracle, IBM DB2, MS SQL Server, and PostgreSQL [6, 7]. A JSON datatype is also supported by several databases, including IBM DB2, MySQL, and PostgreSQL [8].
- A third option is to encode jobs, education, and contact info as a JSON or XML **document**, store it on a text **column** in the database, and let the application interpret its structure and content. In this setup, you typically cannot use the database to query for values inside that encoded column.
 - Trong mô hình SQL truyền thống (trước SQL:1999), cách biểu diễn chuẩn hóa phổ biến nhất là đặt positions, education, và thông tin liên hệ vào các bảng riêng, với một khóa ngoại tham chiếu tới bảng users, như trong Hình 2-1.
 - Các phiên bản sau của chuẩn SQL bổ sung hỗ trợ cho các kiểu dữ liệu có cấu trúc và dữ liệu XML; điều này cho phép lưu dữ liệu đa giá trị bên trong một hàng duy nhất, kèm khả năng truy vấn và lập chỉ mục bên trong những tài liệu đó. Những tính năng này được Oracle, IBM DB2, MS SQL Server, và PostgreSQL hỗ trợ ở các mức độ khác nhau [6, 7]. Một kiểu dữ liệu JSON cũng được vài cơ sở dữ liệu hỗ trợ, bao gồm IBM DB2, MySQL, và PostgreSQL [8].
 - Một lựa chọn thứ ba là mã hóa công việc, học vấn, và thông tin liên hệ thành một tài liệu JSON hoặc XML, lưu nó vào một cột văn bản trong cơ sở dữ liệu, và để ứng dụng tự diễn giải cấu trúc cùng nội dung của nó. Trong cách bố trí này, bạn thường không thể dùng cơ sở dữ liệu để truy vấn các giá trị bên trong cột đã mã hóa đó.

Figure 2-1. Representing a LinkedIn profile using a relational schema. Photo of Bill Gates courtesy of Wikimedia Commons, Ricardo Stuckert, Agência Brasil. — Hình 2-1. Biểu diễn một hồ sơ LinkedIn bằng lược đồ quan hệ. Ảnh Bill Gates do Wikimedia Commons, Ricardo Stuckert, Agência Brasil cung cấp. For a data structure like a résumé, which is mostly a self-contained document, a JSON representation can be quite appropriate: see Example 2-1. JSON has the appeal of being much simpler than XML. Document-oriented databases like MongoDB [9], RethinkDB [10], CouchDB [11], and Espresso [12] support this data model.

Với một cấu trúc dữ liệu như sơ yếu lý lịch, vốn phần lớn là một tài liệu tự chứa, cách biểu diễn bằng JSON có thể khá phù hợp: xem Ví dụ 2-1. JSON có sức hút ở chỗ nó đơn giản hơn nhiều so với XML. Các cơ sở dữ liệu hướng tài liệu như MongoDB [9], RethinkDB [10], CouchDB [11], và Espresso [12] đều hỗ trợ mô hình dữ liệu này.

Example 2-1. Representing a LinkedIn profile as a JSON document — Ví dụ 2-1. Biểu diễn một hồ sơ LinkedIn dưới dạng tài liệu JSON

```
{
  "user_id":      251,
  "first_name":   "Bill",
  "last_name":    "Gates",
  "summary":      "Co-chair of the Bill & Melinda Gates... Active blogger.",
  "region_id":    "us:91",
  "industry_id":  131,
  "photo_url":    "/p/7/000/253/05b/308dd6e.jpg",
  "positions": [
    {"job_title": "Co-chair", "organization": "Bill & Melinda Gates Foundation"},
    {"job_title": "Co-founder, Chairman", "organization": "Microsoft"}
  ],
  "education": [
    {"school_name": "Harvard University", "start": 1973, "end": 1975},
    {"school_name": "Lakeside School, Seattle", "start": null, "end": null}
  ],
  "contact_info": {
    "blog": "http://thegatesnotes.com",
    "twitter": "http://twitter.com/BillGates"
  }
}
```

Some developers feel that the JSON model reduces the impedance mismatch between the application code and the storage layer. However, as we shall see in Chapter 4, there are also problems with JSON as a data encoding format. The lack of a schema is often cited as an advantage; we will discuss this in “**Schema flexibility** in the document model”.

Một số lập trình viên cảm thấy mô hình JSON giúp giảm sự lệch trở kháng giữa mã ứng dụng và tầng lưu trữ. Tuy nhiên, như ta sẽ thấy ở Chương 4, JSON với tư cách là một định dạng mã hóa dữ liệu cũng có những vấn đề riêng. Việc thiếu một lược đồ thường được nêu ra như một ưu điểm; ta sẽ bàn về điều này ở phần “Tính linh hoạt lược đồ trong mô hình tài liệu”.

The JSON representation has better **locality** than the multi-table schema in Figure 2-1. If you want to fetch a profile in the relational example, you need to either perform multiple queries (query each table by user_id) or perform a messy multi-way **join** between the users table and its subordinate tables. In the JSON representation, all the relevant information is in one place, and one query is sufficient.

Cách biểu diễn JSON có tính cục bộ (locality) tốt hơn so với lược đồ nhiều bảng trong Hình 2-1. Nếu muốn lấy một hồ sơ trong ví dụ quan hệ, bạn phải hoặc thực hiện nhiều truy vấn (truy vấn từng bảng theo user_id), hoặc thực hiện một phép join nhiều chiều rối rắm giữa bảng users và các bảng phụ thuộc của nó. Trong cách biểu diễn JSON, mọi thông tin liên quan đều nằm cùng một chỗ, và một truy vấn là đủ.

The one-to-many relationships from the user profile to the user’s positions, educational history, and contact information imply a **tree structure** in the data, and the JSON representation makes this tree structure explicit (see Figure 2-2).

Các quan hệ một-nhiều từ hồ sơ người dùng tới các vị trí công việc, lịch sử học tập, và thông tin liên hệ ngụ ý một cấu trúc cây trong dữ liệu, và cách biểu diễn JSON làm cho cấu

trúc cây này trở nên tường minh (xem Hình 2-2).

Figure 2-2. One-to-many relationships forming a tree structure. — Hình 2-2. Các quan hệ một-nhiều tạo thành một cấu trúc cây.

Many-to-One and Many-to-Many Relationships — Quan hệ nhiều-một và nhiều-nhiều

In Example 2-1 in the preceding section, `region_id` and `industry_id` are given as IDs, not as plain-text strings “Greater Seattle Area” and “Philanthropy”. Why?

Trong Ví dụ 2-1 ở phần trước, `region_id` và `industry_id` được cho dưới dạng ID, chứ không phải các chuỗi văn bản thuần “Greater Seattle Area” và “Philanthropy”. Vì sao?

If the user interface has free-text fields for entering the region and the industry, it makes sense to store them as plain-text strings. But there are advantages to having standardized lists of geographic regions and industries, and letting users choose from a drop-down list or autocompleter:

Nếu giao diện người dùng có các trường nhập văn bản tự do để nhập khu vực và ngành nghề, thì việc lưu chúng dưới dạng chuỗi văn bản thuần là hợp lý. Nhưng có những lợi ích khi dùng các danh sách chuẩn hóa về khu vực địa lý và ngành nghề, rồi để người dùng chọn từ một danh sách thả xuống hoặc một bộ gợi ý tự động hoàn thành:

- Consistent style and spelling across profiles
- Avoiding ambiguity (e.g., if there are several cities with the same name)
- Ease of updating—the name is stored in only one place, so it is easy to update across the board if it ever needs to be changed (e.g., change of a city name due to political events)
- Localization support—when the site is translated into other languages, the standardized lists can be localized, so the region and industry can be displayed in the viewer’s language
- Better search—e.g., a search for philanthropists in the state of Washington can match this profile, because the list of regions can encode the fact that Seattle is in Washington (which is not apparent from the string “Greater Seattle Area”)
 - Cách viết và chính tả nhất quán giữa các hồ sơ.
 - Tránh nhập nhằng (ví dụ, khi có nhiều thành phố trùng tên).
 - Dễ cập nhật — tên chỉ được lưu ở một chỗ, nên dễ dàng cập nhật đồng loạt nếu một lúc nào đó cần thay đổi (ví dụ, đổi tên một thành phố do biến cố chính trị).
 - Hỗ trợ bản địa hóa — khi trang web được dịch sang ngôn ngữ khác, các danh sách chuẩn hóa có thể được bản địa hóa, nhờ đó khu vực và ngành nghề có thể hiển thị bằng ngôn ngữ của người xem.
 - Tìm kiếm tốt hơn — ví dụ, một truy vấn tìm các nhà từ thiện ở bang Washington có thể khớp với hồ sơ này, vì danh sách khu vực có thể ghi nhận được việc Seattle nằm ở Washington (điều không hiển hiện từ chuỗi “Greater Seattle Area”).

Whether you store an ID or a text string is a question of duplication. When you use an ID, the information that is meaningful to humans (such as the word *Philanthropy*) is stored in only one place, and everything that refers to it uses an ID (which only has meaning within the database). When you store the text directly, you are duplicating the human-meaningful information in every record that uses it.

Việc bạn lưu một ID hay một chuỗi văn bản là câu hỏi về sự trùng lặp. Khi dùng một ID, thông tin có ý nghĩa với con người (chẳng hạn từ *Philanthropy*) chỉ được lưu ở một chỗ, và

mọi thứ tham chiếu tới nó đều dùng một ID (vốn chỉ có ý nghĩa bên trong cơ sở dữ liệu). Khi lưu trực tiếp văn bản, bạn đang nhân bản thông tin có ý nghĩa với con người vào mọi bản ghi sử dụng nó.

The advantage of using an ID is that because it has no meaning to humans, it never needs to change: the ID can remain the same, even if the information it identifies changes. Anything that is meaningful to humans may need to change sometime in the future—and if that information is duplicated, all the redundant copies need to be updated. That incurs write overheads, and risks inconsistencies (where some copies of the information are updated but others aren't). Removing such duplication is the key idea behind **normalization** in databases.

Ưu điểm của việc dùng ID là vì nó không mang ý nghĩa gì với con người, nó không bao giờ cần thay đổi: ID có thể giữ nguyên, ngay cả khi thông tin mà nó định danh thay đổi. Bất cứ thứ gì có ý nghĩa với con người đều có thể cần thay đổi vào một lúc nào đó trong tương lai — và nếu thông tin đó bị nhân bản, thì mọi bản sao dư thừa đều phải được cập nhật. Điều đó gây ra chi phí ghi, và rủi ro bất nhất (khi một số bản sao của thông tin được cập nhật còn số khác thì không). Loại bỏ sự trùng lặp như vậy là ý tưởng cốt lõi đằng sau việc chuẩn hóa trong cơ sở dữ liệu.

Note — Ghi chú Database administrators and developers love to argue about normalization and **denormalization**, but we will suspend judgment for now. In Part III of this book we will return to this topic and explore systematic ways of dealing with caching, denormalization, and **derived data**.

Các quản trị viên cơ sở dữ liệu và lập trình viên rất thích tranh cãi về chuẩn hóa và phi chuẩn hóa, nhưng ta sẽ tạm gác phán xét lại lúc này. Ở Phần III của cuốn sách, ta sẽ quay lại chủ đề này và khám phá những cách tiếp cận có hệ thống để xử lý việc caching, phi chuẩn hóa, và dữ liệu dẫn xuất.

Unfortunately, normalizing this data requires **many-to-one** relationships (many people live in one particular region, many people work in one particular industry), which don't fit nicely into the document model. In relational databases, it's normal to refer to rows in other tables by ID, because joins are easy. In document databases, joins are not needed for one-to-many tree structures, and support for joins is often weak.

Đáng tiếc, việc chuẩn hóa dữ liệu này lại đòi hỏi các quan hệ nhiều-một (nhiều người sống ở cùng một khu vực, nhiều người làm việc trong cùng một ngành), vốn không khớp gọn gàng với mô hình tài liệu. Trong cơ sở dữ liệu quan hệ, việc tham chiếu các hàng ở bảng khác bằng ID là bình thường, vì phép join rất dễ. Trong cơ sở dữ liệu tài liệu, phép join không cần thiết cho các cấu trúc cây một-nhiều, và sự hỗ trợ cho join thường yếu.

If the database itself does not support joins, you have to emulate a join in application code by making multiple queries to the database. (In this case, the lists of regions and industries are probably small and slow-changing enough that the application can simply keep them in memory. But nevertheless, the work of making the join is shifted from the database to the application code.)

Nếu bản thân cơ sở dữ liệu không hỗ trợ join, bạn phải mô phỏng một phép join trong mã ứng dụng bằng cách thực hiện nhiều truy vấn tới cơ sở dữ liệu. (Trong trường hợp này, các danh sách khu vực và ngành nghề có lẽ đủ nhỏ và đủ ít thay đổi để ứng dụng có thể đơn giản giữ chúng trong bộ nhớ. Nhưng dù vậy, công việc thực hiện phép join vẫn bị dịch chuyển từ cơ sở dữ liệu sang mã ứng dụng.)

Moreover, even if the initial version of an application fits well in a join-free document model, data has a tendency of becoming more interconnected as features are added to applications. For example, consider some changes we could make to the résumé example:

Hơn nữa, ngay cả khi phiên bản ban đầu của một ứng dụng khớp tốt với một mô hình tài liệu không-cần-join, dữ liệu vẫn có xu hướng trở nên liên kết với nhau nhiều hơn khi các tính năng được thêm vào ứng dụng. Ví dụ, hãy xét vài thay đổi mà ta có thể thực hiện với ví dụ sơ yếu lý lịch:

In the previous description, organization (the company where the user worked) and school_name (where they studied) are just strings. Perhaps they should be references to entities instead? Then each organization, school, or university could have its own web page (with logo, news feed, etc.); each résumé could link to the organizations and schools that it mentions, and include their logos and other information (see Figure 2-3 for an example from LinkedIn).

Trong mô tả trước đó, organization (công ty nơi người dùng từng làm việc) và school_name (nơi họ từng học) chỉ là các chuỗi. Có lẽ chúng nên là tham chiếu tới các thực thể thì hơn? Khi đó mỗi tổ chức, trường học, hoặc đại học có thể có trang web riêng (với logo, dòng tin tức, v.v.); mỗi sơ yếu lý lịch có thể liên kết tới các tổ chức và trường học mà nó nhắc tới, và đính kèm logo cùng thông tin khác của chúng (xem Hình 2-3 cho một ví dụ từ LinkedIn).

Say you want to add a new feature: one user can write a recommendation for another user. The recommendation is shown on the résumé of the user who was recommended, together with the name and photo of the user making the recommendation. If the recommender updates their photo, any recommendations they have written need to reflect the new photo. Therefore, the recommendation should have a reference to the author's profile.

Giả sử bạn muốn thêm một tính năng mới: một người dùng có thể viết một lời giới thiệu cho người dùng khác. Lời giới thiệu được hiển thị trên sơ yếu lý lịch của người được giới thiệu, kèm tên và ảnh của người viết lời giới thiệu. Nếu người viết cập nhật ảnh của mình, mọi lời giới thiệu mà họ đã viết đều cần phản ánh ảnh mới. Do đó, lời giới thiệu nên có một tham chiếu tới hồ sơ của tác giả.

Figure 2-3. The company name is not just a string, but a link to a company entity. Screenshot of linkedin.com. — Hình 2-3. Tên công ty không chỉ là một chuỗi, mà là một liên kết tới một thực thể công ty. Ảnh chụp màn hình từ linkedin.com. Figure 2-4 illustrates how these new features require **many-to-many** relationships. The data within each dotted rectangle can be grouped into one document, but the references to organizations, schools, and other users need to be represented as references, and require joins when queried.

Hình 2-4 minh họa các tính năng mới này đòi hỏi những quan hệ nhiều-nhiều như thế nào. Dữ liệu bên trong mỗi khung nét đứt có thể được gom thành một tài liệu, nhưng các tham chiếu tới tổ chức, trường học, và những người dùng khác cần được biểu diễn dưới dạng tham chiếu, và đòi hỏi phép join khi truy vấn.

Figure 2-4. Extending résumés with many-to-many relationships. — Hình 2-4. Mở rộng sơ yếu lý lịch với các quan hệ nhiều-nhiều.

Are Document Databases Repeating History? — Phải chăng cơ sở dữ liệu tài liệu đang lặp lại lịch sử?

While many-to-many relationships and joins are routinely used in relational databases, document databases and NoSQL reopened the debate on how best to represent such relationships in a database. This debate is much older than NoSQL—in fact, it goes back to the very earliest computerized database systems.

Trong khi các quan hệ nhiều-nhiều và phép join được dùng thường xuyên trong cơ sở dữ liệu quan hệ, thì cơ sở dữ liệu tài liệu và NoSQL lại khơi lại cuộc tranh luận về cách tốt nhất để biểu diễn những quan hệ như vậy trong một cơ sở dữ liệu. Cuộc tranh luận này còn xưa hơn NoSQL nhiều — thực ra nó quay về tận những hệ cơ sở dữ liệu tin học hóa đầu tiên.

The most popular database for business data processing in the 1970s was IBM's Information Management System (IMS), originally developed for stock-keeping in the Apollo space program and first commercially released in 1968 [13]. It is still in use and maintained today, running on OS/390 on IBM mainframes [14].

Cơ sở dữ liệu phổ biến nhất cho xử lý dữ liệu nghiệp vụ trong thập niên 1970 là Information Management System (IMS) của IBM, ban đầu được phát triển để quản lý tồn kho trong chương trình không gian Apollo và lần đầu được thương mại hóa năm 1968 [13]. Nó vẫn còn được dùng và bảo trì cho đến ngày nay, chạy trên OS/390 trên các máy mainframe của IBM [14].

The design of IMS used a fairly simple data model called the hierarchical model, which has some remarkable similarities to the JSON model used by document databases [2]. It represented all data as a tree of records nested within records, much like the JSON structure of Figure 2-2.

Thiết kế của IMS dùng một mô hình dữ liệu khá đơn giản gọi là mô hình phân cấp, vốn có những điểm tương đồng đáng kể với mô hình JSON mà cơ sở dữ liệu tài liệu dùng [2]. Nó biểu diễn mọi dữ liệu dưới dạng một cây các bản ghi lồng trong bản ghi, rất giống cấu trúc JSON của Hình 2-2.

Like document databases, IMS worked well for one-to-many relationships, but it made many-to-many relationships difficult, and it didn't support joins. Developers had to decide whether to duplicate (**denormalize**) data or to manually resolve references from one record to another. These problems of the 1960s and '70s were very much like the problems that developers are running into with document databases today [15].

Giống cơ sở dữ liệu tài liệu, IMS làm tốt với các quan hệ một-nhiều, nhưng lại khiến các quan hệ nhiều-nhiều trở nên khó khăn, và nó không hỗ trợ phép join. Lập trình viên phải quyết định hoặc nhân bản (phi chuẩn hóa) dữ liệu, hoặc tự tay phân giải các tham chiếu từ bản ghi này sang bản ghi khác. Những vấn đề của thập niên 1960 và 1970 này rất giống những vấn đề mà lập trình viên đang gặp phải với cơ sở dữ liệu tài liệu ngày nay [15].

Various solutions were proposed to solve the limitations of the hierarchical model. The two most prominent were the relational model (which became SQL, and took over the world) and the network model (which initially had a large following but eventually faded into obscurity). The “great debate” between these two camps lasted for much of the 1970s [2].

Nhiều giải pháp đã được đề xuất để khắc phục những hạn chế của mô hình phân cấp. Hai giải pháp nổi bật nhất là mô hình quan hệ (về sau trở thành SQL, và chiếm lĩnh cả thế giới) và mô hình mạng (ban đầu có lượng người theo đông đảo nhưng cuối cùng chìm vào quên lãng). “Cuộc đại tranh luận” giữa hai phe này kéo dài gần như suốt thập niên 1970 [2].

Since the problem that the two models were solving is still so relevant today, it's worth briefly revisiting this debate in today's light.

Vì vấn đề mà hai mô hình này nhắm tới giải quyết vẫn còn rất thời sự cho đến hôm nay, nên cũng đáng để ôn lại cuộc tranh luận đó một cách vắn tắt dưới ánh sáng ngày nay.

The network model — Mô hình mạng

The network model was standardized by a committee called the Conference on Data Systems Languages (CODASYL) and implemented by several different database vendors; it is also known as the CODASYL model [16].

Mô hình mạng được chuẩn hóa bởi một ủy ban tên là Conference on Data Systems Languages (CODASYL) và được hiện thực bởi nhiều nhà cung cấp cơ sở dữ liệu khác nhau; nó cũng được biết đến với tên mô hình CODASYL [16].

The CODASYL model was a generalization of the hierarchical model. In the tree structure of the hierarchical model, every record has exactly one parent; in the network model, a record could have multiple parents. For example, there could be one record for the “Greater Seattle Area” region, and every user who lived in that region could be linked to it. This allowed many-to-one and many-to-many relationships to be modeled.

Mô hình CODASYL là một sự tổng quát hóa của mô hình phân cấp. Trong cấu trúc cây của mô hình phân cấp, mỗi bản ghi có đúng một cha; trong mô hình mạng, một bản ghi có thể có nhiều cha. Ví dụ, có thể có một bản ghi cho khu vực “Greater Seattle Area”, và mọi người dùng sống ở khu vực đó đều có thể được liên kết tới nó. Điều này cho phép mô hình hóa các quan hệ nhiều-một và nhiều-nhiều.

The links between records in the network model were not foreign keys, but more like pointers in a programming language (while still being stored on disk). The only way of accessing a record was to follow a path from a root record along these chains of links. This was called an **access path**.

Các liên kết giữa các bản ghi trong mô hình mạng không phải là khóa ngoại, mà giống con trỏ trong một ngôn ngữ lập trình hơn (dù vẫn được lưu trên đĩa). Cách duy nhất để truy cập một bản ghi là đi theo một đường từ một bản ghi gốc dọc theo các chuỗi liên kết này. Điều này được gọi là một đường truy cập (access path).

In the simplest case, an access path could be like the traversal of a linked list: start at the head of the list, and look at one record at a time until you find the one you want. But in a world of many-to-many relationships, several different paths can lead to the same record, and a programmer working with the network model had to keep track of these different access paths in their head.

Trong trường hợp đơn giản nhất, một đường truy cập có thể giống việc duyệt một danh sách liên kết: bắt đầu từ đầu danh sách, và lần lượt xem từng bản ghi một cho đến khi tìm thấy bản ghi bạn cần. Nhưng trong một thế giới của các quan hệ nhiều-nhiều, nhiều đường khác nhau có thể dẫn tới cùng một bản ghi, và lập trình viên làm việc với mô hình mạng phải tự ghi nhớ các đường truy cập khác nhau này trong đầu.

A query in CODASYL was performed by moving a **cursor** through the database by iterating over lists of records and following access paths. If a record had multiple parents (i.e., multiple incoming pointers from other records), the application code had to keep track of all the various relationships. Even CODASYL committee members admitted that this was like navigating around an n-dimensional data space [17].

Một truy vấn trong CODASYL được thực hiện bằng cách di chuyển một con trỏ (cursor) xuyên qua cơ sở dữ liệu, lặp qua các danh sách bản ghi và đi theo các đường truy cập. Nếu một bản ghi có nhiều cha (tức là nhiều con trỏ đi vào từ các bản ghi khác), thì mã ứng dụng phải tự ghi nhớ mọi quan hệ khác nhau. Ngay cả các thành viên ủy ban CODASYL cũng thừa nhận rằng điều này giống như đang lặn mò trong một không gian dữ liệu n chiều [17].

Although manual access path selection was able to make the most efficient use of the very limited hardware capabilities in the 1970s (such as tape drives, whose seeks are extremely slow), the problem was that they made the code for querying and updating the database complicated and inflexible. With both the hierarchical and the network model, if you didn't have a path to the data you wanted, you were in a difficult situation. You could change the access paths, but then you had to go through a lot of handwritten database query code and rewrite it to handle the new access paths. It was difficult to make changes to an application's data model.

Mặc dù việc tự chọn đường truy cập có thể tận dụng tối đa khả năng phần cứng vốn rất hạn chế của thập niên 1970 (chẳng hạn các ổ băng từ, có tốc độ tìm kiếm cực kỳ chậm), vấn đề là chúng khiến cho mã truy vấn và cập nhật cơ sở dữ liệu trở nên phức tạp và thiếu linh hoạt. Với cả mô hình phân cấp lẫn mô hình mạng, nếu bạn không có sẵn một đường tới dữ liệu mình cần, bạn rơi vào tình thế khó khăn. Bạn có thể đổi các đường truy cập, nhưng khi đó bạn phải rà soát một lượng lớn mã truy vấn cơ sở dữ liệu viết tay và viết lại nó để xử lý các đường truy cập mới. Việc thay đổi mô hình dữ liệu của một ứng dụng là rất khó.

The relational model — Mô hình quan hệ

What the relational model did, by contrast, was to lay out all the data in the open: a relation (table) is simply a collection of tuples (rows), and that's it. There are no labyrinthine nested structures, no complicated access paths to follow if you want to look at the data. You can read any or all of the rows in a table, selecting those that match an arbitrary condition. You can read a particular row by designating some columns as a key and matching on those. You can insert a new row into any table without worrying about foreign key relationships to and from other tables.

Trái lại, điều mà mô hình quan hệ làm là bày tất cả dữ liệu ra một cách công khai: một quan hệ (bảng) đơn giản chỉ là một tập hợp các bộ (hàng), thế thôi. Không có những cấu trúc lồng nhau như mê cung, không có những đường truy cập phức tạp phải đi theo nếu bạn muốn xem dữ liệu. Bạn có thể đọc bất kỳ hàng nào hoặc tất cả các hàng trong một bảng, chọn ra những hàng khớp với một điều kiện tùy ý. Bạn có thể đọc một hàng cụ thể bằng cách chỉ định một số cột làm khóa và khớp theo các cột đó. Bạn có thể chèn một hàng mới vào bất kỳ bảng nào mà không cần lo về các quan hệ khóa ngoại đi tới hay đi từ các bảng khác.

In a relational database, the **query optimizer** automatically decides which parts of the query to execute in which order, and which indexes to use. Those choices are effectively the “access path,” but the big difference is that they are made automatically by the query optimizer, not by the application developer, so we rarely need to think about them.

Trong một cơ sở dữ liệu quan hệ, bộ tối ưu truy vấn tự động quyết định phần nào của truy vấn được thực thi theo thứ tự nào, và dùng các chỉ mục nào. Những lựa chọn đó về cơ bản chính là “đường truy cập”, nhưng khác biệt lớn là chúng được thực hiện tự động bởi bộ tối ưu truy vấn, chứ không phải bởi lập trình viên ứng dụng, nên ta hiếm khi cần phải nghĩ về chúng.

If you want to query your data in new ways, you can just declare a new **index**, and queries will automatically use whichever indexes are most appropriate. You don't need to change your queries to take advantage of a new index. (See also “Query Languages for Data”.) The relational model thus made it much easier to add new features to applications.

Nếu muốn truy vấn dữ liệu theo những cách mới, bạn chỉ cần khai báo một chỉ mục mới, và các truy vấn sẽ tự động dùng bất kỳ chỉ mục nào phù hợp nhất. Bạn không cần thay

đổi các truy vấn của mình để tận dụng một chỉ mục mới. (Xem thêm phần “Ngôn ngữ truy vấn cho dữ liệu”.) Nhờ vậy, mô hình quan hệ đã khiến việc thêm tính năng mới vào ứng dụng trở nên dễ dàng hơn nhiều.

Query optimizers for relational databases are complicated beasts, and they have consumed many years of research and development effort [18]. But a key insight of the relational model was this: you only need to build a query optimizer once, and then all applications that use the database can benefit from it. If you don’t have a query optimizer, it’s easier to handcode the access paths for a particular query than to write a general-purpose optimizer—but the general-purpose solution wins in the long run.

Các bộ tối ưu truy vấn cho cơ sở dữ liệu quan hệ là những con quái vật phức tạp, và chúng đã ngốn nhiều năm nghiên cứu và phát triển [18]. Nhưng một nhận thức then chốt của mô hình quan hệ là: bạn chỉ cần xây dựng một bộ tối ưu truy vấn một lần, rồi mọi ứng dụng dùng cơ sở dữ liệu đó đều được hưởng lợi từ nó. Nếu bạn không có một bộ tối ưu truy vấn, thì viết tay các đường truy cập cho một truy vấn cụ thể sẽ dễ hơn là viết một bộ tối ưu đa dụng — nhưng về lâu dài, giải pháp đa dụng mới là kẻ chiến thắng.

Comparison to document databases — So sánh với cơ sở dữ liệu tài liệu

Document databases reverted back to the hierarchical model in one aspect: storing nested records (one-to-many relationships, like positions, education, and contact_info in Figure 2-1) within their parent record rather than in a separate table.

Cơ sở dữ liệu tài liệu đã quay về với mô hình phân cấp ở một khía cạnh: lưu các bản ghi lồng nhau (các quan hệ một-nhiều, như positions, education, và contact_info trong Hình 2-1) bên trong bản ghi cha của chúng thay vì trong một bảng riêng.

However, when it comes to representing many-to-one and many-to-many relationships, relational and document databases are not fundamentally different: in both cases, the related item is referenced by a unique identifier, which is called a foreign key in the relational model and a **document reference** in the document model [9]. That identifier is resolved at read time by using a join or follow-up queries. To date, document databases have not followed the path of CODASYL.

Tuy nhiên, khi nói tới việc biểu diễn các quan hệ nhiều-một và nhiều-nhiều, cơ sở dữ liệu quan hệ và cơ sở dữ liệu tài liệu về cơ bản không khác nhau: trong cả hai trường hợp, mục liên quan đều được tham chiếu bằng một định danh duy nhất, gọi là khóa ngoại trong mô hình quan hệ và tham chiếu tài liệu trong mô hình tài liệu [9]. Định danh đó được phân giải vào lúc đọc bằng cách dùng một phép join hoặc các truy vấn tiếp theo. Cho đến nay, cơ sở dữ liệu tài liệu vẫn chưa đi theo con đường của CODASYL.

Relational Versus Document Databases Today — Cơ sở dữ liệu quan hệ so với cơ sở dữ liệu tài liệu ngày nay

There are many differences to consider when comparing relational databases to document databases, including their **fault**-tolerance properties (see Chapter 5) and handling of **concurrency** (see Chapter 7). In this chapter, we will concentrate only on the differences in the data model.

Có nhiều khác biệt cần cân nhắc khi so sánh cơ sở dữ liệu quan hệ với cơ sở dữ liệu tài liệu, bao gồm các đặc tính chịu lỗi của chúng (xem Chương 5) và cách xử lý tương tranh (xem Chương 7). Trong chương này, ta sẽ chỉ tập trung vào những khác biệt ở mô hình dữ liệu.

The main arguments in favor of the document data model are schema flexibility, better performance due to locality, and that for some applications it is closer to the data structures used by the application. The relational model counters by providing better support for joins, and many-to-one and many-to-many relationships.

Các lập luận chính ủng hộ mô hình dữ liệu tài liệu là tính linh hoạt lược đồ, hiệu năng tốt hơn nhờ tính cục bộ, và với một số ứng dụng thì nó gần với các cấu trúc dữ liệu mà ứng dụng dùng hơn. Mô hình quan hệ phản pháo bằng cách hỗ trợ tốt hơn cho phép join, cùng các quan hệ nhiều-một và nhiều-nhiều.

Which data model leads to simpler application code? — Mô hình dữ liệu nào dẫn tới mã ứng dụng đơn giản hơn?

If the data in your application has a document-like structure (i.e., a tree of one-to-many relationships, where typically the entire tree is loaded at once), then it's probably a good idea to use a document model. The relational technique of **shredding**—splitting a document-like structure into multiple tables (like positions, education, and contact_info in Figure 2-1)—can lead to cumbersome schemas and unnecessarily complicated application code.

Nếu dữ liệu trong ứng dụng của bạn có cấu trúc dạng tài liệu (tức là một cây các quan hệ một-nhiều, mà thường thì cả cây được nạp cùng một lúc), thì có lẽ dùng mô hình tài liệu là một ý hay. Kỹ thuật băm nhỏ (shredding) của mô hình quan hệ — tách một cấu trúc dạng tài liệu thành nhiều bảng (như positions, education, và contact_info trong Hình 2-1) — có thể dẫn tới những lược đồ cồng kềnh và mã ứng dụng phức tạp một cách không cần thiết.

The document model has limitations: for example, you cannot refer directly to a nested item within a document, but instead you need to say something like “the second item in the list of positions for user 251” (much like an access path in the hierarchical model). However, as long as documents are not too deeply nested, that is not usually a problem.

Mô hình tài liệu có những hạn chế: ví dụ, bạn không thể tham chiếu trực tiếp tới một mục lồng bên trong một tài liệu, mà thay vào đó bạn phải nói kiểu như “mục thứ hai trong danh sách positions của người dùng 251” (rất giống một đường truy cập trong mô hình phân cấp). Tuy nhiên, miễn là các tài liệu không lồng quá sâu, đó thường không phải vấn đề.

The poor support for joins in document databases may or may not be a problem, depending on the application. For example, many-to-many relationships may never be needed in an analytics application that uses a document database to record which events occurred at which time [19].

Sự hỗ trợ kém cho phép join trong cơ sở dữ liệu tài liệu có thể là vấn đề hoặc không, tùy theo ứng dụng. Ví dụ, các quan hệ nhiều-nhiều có thể chẳng bao giờ cần đến trong một ứng dụng phân tích dùng cơ sở dữ liệu tài liệu để ghi lại sự kiện nào đã xảy ra vào thời điểm nào [19].

However, if your application does use many-to-many relationships, the document model becomes less appealing. It's possible to reduce the need for joins by denormalizing, but then the application code needs to do additional work to keep the denormalized data consistent. Joins can be emulated in application code by making multiple requests to the database, but that also moves complexity into the application and is usually slower than a join performed by specialized code inside the database. In such cases, using a document model can lead to significantly more complex application code and worse performance [15].

Tuy nhiên, nếu ứng dụng của bạn có dùng các quan hệ nhiều-nhiều, thì mô hình tài liệu

trở nên kém hấp dẫn hơn. Có thể giảm nhu cầu phép join bằng cách phi chuẩn hóa, nhưng khi đó mã ứng dụng phải làm thêm việc để giữ cho dữ liệu đã phi chuẩn hóa luôn nhất quán. Phép join có thể được mô phỏng trong mã ứng dụng bằng cách gửi nhiều yêu cầu tới cơ sở dữ liệu, nhưng điều đó cũng đẩy sự phức tạp vào ứng dụng và thường chậm hơn một phép join được thực hiện bởi mã chuyên dụng bên trong cơ sở dữ liệu. Trong những trường hợp như vậy, dùng mô hình tài liệu có thể dẫn tới mã ứng dụng phức tạp hơn đáng kể và hiệu năng tệ hơn [15].

It's not possible to say in general which data model leads to simpler application code; it depends on the kinds of relationships that exist between data items. For highly interconnected data, the document model is awkward, the relational model is acceptable, and graph models (see “Graph-Like Data Models”) are the most natural.

Không thể nói một cách tổng quát mô hình dữ liệu nào dẫn tới mã ứng dụng đơn giản hơn; điều đó phụ thuộc vào những loại quan hệ tồn tại giữa các mục dữ liệu. Với dữ liệu liên kết chặt chẽ, mô hình tài liệu thì gượng gạo, mô hình quan hệ thì chấp nhận được, còn các mô hình đồ thị (xem “Các mô hình dữ liệu dạng đồ thị”) là tự nhiên nhất.

Schema flexibility in the document model — Tính linh hoạt lược đồ trong mô hình tài liệu

Most document databases, and the JSON support in relational databases, do not enforce any schema on the data in documents. XML support in relational databases usually comes with optional schema validation. No schema means that arbitrary keys and values can be added to a document, and when reading, clients have no guarantees as to what fields the documents may contain.

Hầu hết cơ sở dữ liệu tài liệu, và sự hỗ trợ JSON trong cơ sở dữ liệu quan hệ, không áp đặt bất kỳ lược đồ nào lên dữ liệu trong các tài liệu. Sự hỗ trợ XML trong cơ sở dữ liệu quan hệ thường đi kèm tùy chọn kiểm tra hợp lệ theo lược đồ. Không có lược đồ nghĩa là có thể thêm các khóa và giá trị tùy ý vào một tài liệu, và khi đọc, client không có gì bảo đảm về việc tài liệu có thể chứa những trường nào.

Document databases are sometimes called **schemaless**, but that's misleading, as the code that reads the data usually assumes some kind of structure—i.e., there is an implicit schema, but it is not enforced by the database [20]. A more accurate term is **schema-on-read** (the structure of the data is implicit, and only interpreted when the data is read), in contrast with **schema-on-write** (the traditional approach of relational databases, where the schema is explicit and the database ensures all written data conforms to it) [21].

Cơ sở dữ liệu tài liệu đôi khi được gọi là phi lược đồ (schemaless), nhưng cách gọi đó dễ gây hiểu lầm, vì mã đọc dữ liệu thường giả định một dạng cấu trúc nào đó — tức là có một lược đồ ngầm, nhưng nó không được cơ sở dữ liệu cưỡng chế [20]. Một thuật ngữ chính xác hơn là lược đồ-lúc-đọc (schema-on-read) (cấu trúc của dữ liệu là ngầm định, và chỉ được diễn giải khi dữ liệu được đọc), trái với lược đồ-lúc-ghi (schema-on-write) (cách tiếp cận truyền thống của cơ sở dữ liệu quan hệ, trong đó lược đồ là tường minh và cơ sở dữ liệu bảo đảm mọi dữ liệu được ghi đều tuân theo nó) [21].

Schema-on-read is similar to dynamic (runtime) type checking in programming languages, whereas schema-on-write is similar to static (compile-time) type checking. Just as the advocates of static and dynamic type checking have big debates about their relative merits [22], enforcement of schemas in database is a contentious topic, and in general there's no right or wrong answer.

Lược đồ-lúc-đọc tương tự kiểm tra kiểu động (lúc chạy) trong các ngôn ngữ lập trình, còn lược đồ-lúc-ghi tương tự kiểm tra kiểu tĩnh (lúc biên dịch). Cũng như những người ủng hộ

kiểm tra kiểu tĩnh và kiểu động tranh luận âm ỉ về ưu nhược điểm tương đối của chúng [22], việc cưỡng chế lược đồ trong cơ sở dữ liệu là một chủ đề gây tranh cãi, và nhìn chung không có câu trả lời đúng hay sai.

The difference between the approaches is particularly noticeable in situations where an application wants to change the format of its data. For example, say you are currently storing each user’s full name in one field, and you instead want to store the first name and last name separately [23]. In a document database, you would just start writing new documents with the new fields and have code in the application that handles the case when old documents are read. For example:

Sự khác biệt giữa các cách tiếp cận đặc biệt rõ rệt trong những tình huống mà một ứng dụng muốn thay đổi định dạng dữ liệu của nó. Ví dụ, giả sử hiện bạn đang lưu họ tên đầy đủ của mỗi người dùng trong một trường, và thay vào đó bạn muốn lưu tên và họ riêng biệt [23]. Trong một cơ sở dữ liệu tài liệu, bạn chỉ cần bắt đầu ghi các tài liệu mới với các trường mới và có mã trong ứng dụng để xử lý trường hợp đọc các tài liệu cũ. Ví dụ:

```
if (user && user.name && !user.first_name) {  
    // Documents written before Dec 8, 2013 don't have first_name  
    user.first_name = user.name.split(" ")[0];  
}
```

On the other hand, in a “statically typed” database schema, you would typically perform a **migration** along the lines of:

Ngược lại, trong một lược đồ cơ sở dữ liệu “định kiểu tĩnh”, bạn thường sẽ thực hiện một cuộc di trú đại loại như sau:

```
ALTER TABLE users ADD COLUMN first_name text;  
UPDATE users SET first_name = split_part(name, ' ', 1);      -- PostgreSQL  
UPDATE users SET first_name = substring_index(name, ' ', 1); -- MySQL
```

Schema changes have a bad reputation of being slow and requiring **downtime**. This reputation is not entirely deserved: most relational database systems execute the ALTER TABLE statement in a few milliseconds. MySQL is a notable exception—it copies the entire table on ALTER TABLE, which can mean minutes or even hours of downtime when altering a large table—although various tools exist to work around this limitation [24, 25, 26].

Việc thay đổi lược đồ vốn bị tiếng là chậm và đòi hỏi thời gian ngừng hoạt động. Tiếng xấu này không hoàn toàn xứng đáng: hầu hết hệ cơ sở dữ liệu quan hệ thực thi câu lệnh ALTER TABLE trong vài mili giây. MySQL là một ngoại lệ đáng chú ý — nó sao chép toàn bộ bảng khi ALTER TABLE, điều này có thể nghĩa là vài phút hoặc thậm chí vài giờ ngừng hoạt động khi thay đổi một bảng lớn — mặc dù có nhiều công cụ giúp khắc phục hạn chế này [24, 25, 26].

Running the UPDATE statement on a large table is likely to be slow on any database, since every row needs to be rewritten. If that is not acceptable, the application can leave first_name set to its default of NULL and fill it in at read time, like it would with a document database.

Chạy câu lệnh UPDATE trên một bảng lớn có khả năng sẽ chậm trên bất kỳ cơ sở dữ liệu nào, vì mỗi hàng đều cần được ghi lại. Nếu điều đó không chấp nhận được, ứng dụng có thể để first_name ở giá trị mặc định NULL và điền nó vào lúc đọc, giống như cách nó làm với một cơ sở dữ liệu tài liệu.

The schema-on-read approach is advantageous if the items in the collection don’t all have the same structure for some reason (i.e., the data is **heterogeneous**)—for example, because:

Cách tiếp cận lược đồ-lúc-đọc có lợi nếu vì lý do nào đó mà các mục trong tập hợp không có cùng một cấu trúc (tức là dữ liệu không đồng nhất) — ví dụ, vì:

- There are many different types of objects, and it is not practical to put each type of object in its own table.
- The structure of the data is determined by external systems over which you have no control and which may change at any time.
 - Có nhiều loại đối tượng khác nhau, và không thực tế khi đặt mỗi loại đối tượng vào bảng riêng của nó.
 - Cấu trúc của dữ liệu được quyết định bởi các hệ thống bên ngoài mà bạn không kiểm soát được và có thể thay đổi bất cứ lúc nào.

In situations like these, a schema may hurt more than it helps, and schemaless documents can be a much more natural data model. But in cases where all records are expected to have the same structure, schemas are a useful mechanism for documenting and enforcing that structure. We will discuss schemas and **schema evolution** in more detail in Chapter 4.

Trong những tình huống như vậy, một lược đồ có thể gây hại nhiều hơn lợi, và các tài liệu phi lược đồ có thể là một mô hình dữ liệu tự nhiên hơn nhiều. Nhưng trong những trường hợp mà mọi bản ghi đều được kỳ vọng có cùng cấu trúc, lược đồ là một cơ chế hữu ích để mô tả và cưỡng chế cấu trúc đó. Ta sẽ bàn về lược đồ và tiến hóa lược đồ chi tiết hơn ở Chương 4.

Data locality for queries — Tính cục bộ dữ liệu cho truy vấn

A document is usually stored as a single continuous string, encoded as JSON, XML, or a binary variant thereof (such as MongoDB's BSON). If your application often needs to access the entire document (for example, to render it on a web page), there is a performance advantage to this storage locality. If data is split across multiple tables, like in Figure 2-1, multiple index lookups are required to retrieve it all, which may require more disk seeks and take more time.

Một tài liệu thường được lưu dưới dạng một chuỗi liên tục duy nhất, được mã hóa thành JSON, XML, hoặc một biến thể nhị phân của chúng (chẳng hạn BSON của MongoDB). Nếu ứng dụng của bạn thường cần truy cập toàn bộ tài liệu (ví dụ, để hiển thị nó trên một trang web), thì có một lợi thế về hiệu năng nhờ tính cục bộ lưu trữ này. Nếu dữ liệu bị tách ra trên nhiều bảng, như trong Hình 2-1, thì cần nhiều lần tra cứu chỉ mục để lấy hết, điều này có thể đòi hỏi nhiều lần tìm kiếm trên đĩa hơn và tốn nhiều thời gian hơn.

The locality advantage only applies if you need large parts of the document at the same time. The database typically needs to **load** the entire document, even if you access only a small portion of it, which can be wasteful on large documents. On updates to a document, the entire document usually needs to be rewritten—only modifications that don't change the encoded size of a document can easily be performed in place [19]. For these reasons, it is generally recommended that you keep documents fairly small and avoid writes that increase the size of a document [9]. These performance limitations significantly reduce the set of situations in which document databases are useful.

Lợi thế về tính cục bộ chỉ áp dụng nếu bạn cần những phần lớn của tài liệu cùng một lúc. Cơ sở dữ liệu thường cần nạp toàn bộ tài liệu, ngay cả khi bạn chỉ truy cập một phần nhỏ của nó, điều này có thể lãng phí với các tài liệu lớn. Khi cập nhật một tài liệu, thường toàn bộ tài liệu cần được ghi lại — chỉ những thay đổi không làm thay đổi kích thước đã mã hóa của tài liệu mới có thể được thực hiện tại chỗ một cách dễ dàng [19]. Vì những lý do này, người ta thường khuyến nghị bạn giữ cho các tài liệu khá nhỏ và tránh những thao

tác ghi làm tăng kích thước của một tài liệu [9]. Những hạn chế về hiệu năng này thu hẹp đáng kể tập hợp các tình huống mà cơ sở dữ liệu tài liệu tỏ ra hữu ích.

It's worth pointing out that the idea of grouping related data together for locality is not limited to the document model. For example, Google's Spanner database offers the same locality properties in a relational data model, by allowing the schema to declare that a table's rows should be interleaved (nested) within a parent table [27]. Oracle allows the same, using a feature called multi-table index cluster tables [28]. The **column-family** concept in the Bigtable data model (used in Cassandra and HBase) has a similar purpose of managing locality [29].

Cũng đáng nói rằng ý tưởng gom dữ liệu liên quan lại với nhau để có tính cục bộ không chỉ giới hạn ở mô hình tài liệu. Ví dụ, cơ sở dữ liệu Spanner của Google cung cấp cùng các đặc tính cục bộ trong một mô hình dữ liệu quan hệ, bằng cách cho phép lược đồ khai báo rằng các hàng của một bảng nên được đan xen (lồng) bên trong một bảng cha [27]. Oracle cho phép điều tương tự, dùng một tính năng gọi là bảng cụm chỉ mục đa bảng [28]. Khái niệm họ cột (column-family) trong mô hình dữ liệu Bigtable (được dùng trong Cassandra và HBase) cũng có mục đích tương tự là quản lý tính cục bộ [29].

We will also see more on locality in Chapter 3.

Ta cũng sẽ thấy thêm về tính cục bộ ở Chương 3.

Convergence of document and relational databases — Sự hội tụ của cơ sở dữ liệu tài liệu và quan hệ

Most relational database systems (other than MySQL) have supported XML since the mid-2000s. This includes functions to make local modifications to XML documents and the ability to index and query inside XML documents, which allows applications to use data models very similar to what they would do when using a document database.

Hầu hết hệ cơ sở dữ liệu quan hệ (ngoài MySQL) đã hỗ trợ XML từ giữa những năm 2000. Điều này bao gồm các hàm để thực hiện các sửa đổi cục bộ trên tài liệu XML và khả năng lập chỉ mục cùng truy vấn bên trong tài liệu XML, cho phép ứng dụng dùng những mô hình dữ liệu rất giống với những gì chúng làm khi dùng một cơ sở dữ liệu tài liệu.

PostgreSQL since version 9.3 [8], MySQL since version 5.7, and IBM DB2 since version 10.5 [30] also have a similar level of support for JSON documents. Given the popularity of JSON for web APIs, it is likely that other relational databases will follow in their footsteps and add JSON support.

PostgreSQL từ phiên bản 9.3 [8], MySQL từ phiên bản 5.7, và IBM DB2 từ phiên bản 10.5 [30] cũng có mức hỗ trợ tương tự cho tài liệu JSON. Với sự phổ biến của JSON trong các web API, nhiều khả năng các cơ sở dữ liệu quan hệ khác sẽ nối gót và bổ sung hỗ trợ JSON.

On the document database side, RethinkDB supports relational-like joins in its **query language**, and some MongoDB drivers automatically resolve database references (effectively performing a **client-side** join, although this is likely to be slower than a join performed in the database since it requires additional network round-trips and is less optimized).

Về phía cơ sở dữ liệu tài liệu, RethinkDB hỗ trợ các phép join giống quan hệ trong ngôn ngữ truy vấn của nó, và một số driver của MongoDB tự động phân giải các tham chiếu cơ sở dữ liệu (về thực chất là thực hiện một phép join phía client, mặc dù việc này có khả năng chậm hơn một phép join thực hiện trong cơ sở dữ liệu vì nó đòi hỏi thêm các vòng đi-về qua mạng và ít được tối ưu hơn).

It seems that relational and document databases are becoming more similar over time, and that is a good thing: the data models complement each other. If a database is able to handle document-like data and also perform relational queries on it, applications can use the combination of features that best fits their needs.

Có vẻ như cơ sở dữ liệu quan hệ và cơ sở dữ liệu tài liệu đang ngày càng giống nhau hơn theo thời gian, và đó là điều tốt: các mô hình dữ liệu bổ trợ cho nhau. Nếu một cơ sở dữ liệu có thể xử lý dữ liệu dạng tài liệu và cũng thực hiện được các truy vấn quan hệ trên nó, thì ứng dụng có thể dùng tổ hợp các tính năng phù hợp nhất với nhu cầu của mình.

A hybrid of the relational and document models is a good route for databases to take in the future.

Một mô hình lai giữa mô hình quan hệ và mô hình tài liệu là một hướng đi tốt cho các cơ sở dữ liệu trong tương lai.

Query Languages for Data — Ngôn ngữ truy vấn cho dữ liệu

When the relational model was introduced, it included a new way of querying data: SQL is a **declarative** query language, whereas IMS and CODASYL queried the database using **imperative** code. What does that mean?

Khi mô hình quan hệ được giới thiệu, nó kèm theo một cách truy vấn dữ liệu mới: SQL là một ngôn ngữ truy vấn khai báo (declarative), trong khi IMS và CODASYL truy vấn cơ sở dữ liệu bằng mã mệnh lệnh (imperative). Điều đó nghĩa là gì?

Many commonly used programming languages are imperative. For example, if you have a list of animal species, you might write something like this to return only the sharks in the list:

Nhiều ngôn ngữ lập trình thông dụng là mệnh lệnh. Ví dụ, nếu bạn có một danh sách các loài động vật, bạn có thể viết một thứ đại loại như sau để chỉ trả về các loài cá mập trong danh sách:

```
function getSharks() {  
  var sharks = [];  
  for (var i = 0; i < animals.length; i++) {  
    if (animals[i].family === "Sharks") {  
      sharks.push(animals[i]);  
    }  
  }  
  return sharks;  
}
```

In the **relational algebra**, you would instead write:

Trong đại số quan hệ, thay vào đó bạn sẽ viết:

$\text{sharks} = \sigma_{\text{family} = \text{"Sharks"}}(\text{animals})$

$\text{sharks} = \sigma_{\text{family} = \text{"Sharks"}}(\text{animals})$

where σ (the Greek letter sigma) is the **selection operator**, returning only those animals that match the condition $\text{family} = \text{"Sharks"}$.

trong đó σ (chữ cái Hy Lạp sigma) là toán tử chọn, chỉ trả về những động vật khớp với điều kiện $\text{family} = \text{"Sharks"}$.

When SQL was defined, it followed the structure of the relational algebra fairly closely:

Khi SQL được định nghĩa, nó đi theo cấu trúc của đại số quan hệ khá sát:

```
SELECT * FROM animals WHERE family = 'Sharks';
```

An imperative language tells the computer to perform certain operations in a certain order. You can imagine stepping through the code line by line, evaluating conditions, updating variables, and deciding whether to go around the loop one more time.

Một ngôn ngữ mệnh lệnh bảo máy tính thực hiện những thao tác nhất định theo một thứ tự nhất định. Bạn có thể hình dung việc bước qua từng dòng mã một, đánh giá các điều kiện, cập nhật các biến, và quyết định có lặp lại vòng lặp thêm một lần nữa hay không.

In a declarative query language, like SQL or relational algebra, you just specify the pattern of the data you want—what conditions the results must meet, and how you want the data to be transformed (e.g., sorted, grouped, and aggregated)—but not how to achieve that goal. It is up to the database system’s query optimizer to decide which indexes and which join methods to use, and in which order to execute various parts of the query.

Trong một ngôn ngữ truy vấn khai báo, như SQL hay đại số quan hệ, bạn chỉ cần đặc tả khuôn mẫu của dữ liệu mình muốn — kết quả phải thỏa những điều kiện gì, và bạn muốn dữ liệu được biến đổi như thế nào (ví dụ, sắp xếp, gom nhóm, và tổng hợp) — chứ không phải làm thế nào để đạt được mục tiêu đó. Việc quyết định dùng chỉ mục nào và phương pháp join nào, và thực thi các phần khác nhau của truy vấn theo thứ tự nào, là tùy ở bộ tối ưu truy vấn của hệ cơ sở dữ liệu.

A declarative query language is attractive because it is typically more concise and easier to work with than an imperative **API**. But more importantly, it also hides implementation details of the database **engine**, which makes it possible for the database system to introduce performance improvements without requiring any changes to queries.

Một ngôn ngữ truy vấn khai báo hấp dẫn vì nó thường gọn gàng hơn và dễ làm việc hơn một API mệnh lệnh. Nhưng quan trọng hơn, nó cũng che giấu các chi tiết hiện thực của engine cơ sở dữ liệu, điều này khiến hệ cơ sở dữ liệu có thể đưa vào những cải tiến hiệu năng mà không cần thay đổi bất kỳ truy vấn nào.

For example, in the imperative code shown at the beginning of this section, the list of animals appears in a particular order. If the database wants to reclaim unused disk space behind the scenes, it might need to move records around, changing the order in which the animals appear. Can the database do that safely, without breaking queries?

Ví dụ, trong đoạn mã mệnh lệnh ở đầu phần này, danh sách động vật xuất hiện theo một thứ tự cụ thể. Nếu cơ sở dữ liệu muốn thu hồi dung lượng đĩa chưa dùng đằng sau hậu trường, nó có thể cần dời các bản ghi đi chỗ khác, làm thay đổi thứ tự xuất hiện của các động vật. Liệu cơ sở dữ liệu có thể làm điều đó một cách an toàn, mà không phá vỡ các truy vấn không?

The SQL example doesn’t guarantee any particular ordering, and so it doesn’t mind if the order changes. But if the query is written as imperative code, the database can never be sure whether the code is relying on the ordering or not. The fact that SQL is more limited in functionality gives the database much more room for automatic optimizations.

Ví dụ SQL không bảo đảm bất kỳ thứ tự cụ thể nào, nên nó không bận tâm nếu thứ tự thay đổi. Nhưng nếu truy vấn được viết dưới dạng mã mệnh lệnh, thì cơ sở dữ liệu không bao giờ chắc chắn được liệu mã có đang dựa vào thứ tự hay không. Việc SQL bị hạn chế hơn về chức năng lại cho cơ sở dữ liệu nhiều dư địa hơn nhiều cho các tối ưu hóa tự động.

Finally, declarative languages often lend themselves to **parallel execution**. Today, CPUs are getting faster by adding more cores, not by running at significantly higher clock speeds than before [31].

Imperative code is very hard to parallelize across multiple cores and multiple machines, because it specifies instructions that must be performed in a particular order. Declarative languages have a better chance of getting faster in parallel execution because they specify only the pattern of the results, not the algorithm that is used to determine the results. The database is free to use a parallel implementation of the query language, if appropriate [32].

Cuối cùng, các ngôn ngữ khai báo thường thuận lợi cho việc thực thi song song. Ngày nay, CPU đang trở nên nhanh hơn bằng cách thêm nhiều nhân, chứ không phải bằng cách chạy ở xung nhịp cao hơn đáng kể so với trước [31]. Mã mệnh lệnh rất khó song song hóa trên nhiều nhân và nhiều máy, vì nó đặc tả các chỉ thị phải được thực hiện theo một thứ tự cụ thể. Các ngôn ngữ khai báo có cơ hội tốt hơn để chạy nhanh hơn nhờ thực thi song song, vì chúng chỉ đặc tả khuôn mẫu của kết quả, chứ không phải thuật toán dùng để xác định kết quả. Cơ sở dữ liệu được tự do dùng một hiện thực song song của ngôn ngữ truy vấn, nếu phù hợp [32].

Declarative Queries on the Web — Truy vấn khai báo trên Web

The advantages of declarative query languages are not limited to just databases. To illustrate the point, let's compare declarative and imperative approaches in a completely different environment: a web browser.

Các ưu điểm của ngôn ngữ truy vấn khai báo không chỉ giới hạn ở cơ sở dữ liệu. Để minh họa điểm này, hãy so sánh cách tiếp cận khai báo và mệnh lệnh trong một môi trường hoàn toàn khác: một trình duyệt web.

Say you have a website about animals in the ocean. The user is currently viewing the page on sharks, so you mark the navigation item “Sharks” as currently selected, like this:

Giả sử bạn có một trang web về các loài động vật ở đại dương. Người dùng hiện đang xem trang về cá mập, nên bạn đánh dấu mục điều hướng “Sharks” là đang được chọn, như sau:

```
<ul>
  <li class="selected">
    <p>Sharks</p>
    <ul>
      <li>Great White Shark</li>
      <li>Tiger Shark</li>
      <li>Hammerhead Shark</li>
    </ul>
  </li>
  <li>
    <p>Whales</p>
    <ul>
      <li>Blue Whale</li>
      <li>Humpback Whale</li>
      <li>Fin Whale</li>
    </ul>
  </li>
</ul>
```

The selected item is marked with the **CSS** class “selected”.

Mục đang được chọn được đánh dấu bằng lớp CSS “selected”.

Sharks

is the title of the currently selected page.

Sharks

là tiêu đề của trang đang được chọn.

Now say you want the title of the currently selected page to have a blue background, so that it is visually highlighted. This is easy, using CSS:

Giờ giả sử bạn muốn tiêu đề của trang đang được chọn có nền màu xanh dương, để nó được làm nổi bật về mặt thị giác. Điều này dễ dàng, dùng CSS:

```
li.selected > p {  
    background-color: blue;  
}
```

Here the CSS **selector** `li.selected > p` declares the pattern of elements to which we want to apply the blue style: namely, all

elements whose direct parent is an

element with a CSS class of `selected`. The element

Sharks

in the example matches this pattern, but

Whales

does not match because its

parent lacks `class="selected"`.

Ở đây bộ chọn CSS `li.selected > p` khai báo khuôn mẫu các phần tử mà ta muốn áp dụng kiểu màu xanh: cụ thể là, mọi phần tử

có cha trực tiếp là một phần tử

với lớp CSS là `selected`. Phần tử

Sharks

trong ví dụ khớp với khuôn mẫu này, nhưng

Whales

thì không khớp vì cha

của nó thiếu `class="selected"`.

If you were using XSL instead of CSS, you could do something similar:

Nếu dùng XSL thay vì CSS, bạn có thể làm điều gì đó tương tự:

```
<xsl:template match="li[@class='selected']/p">  
    <fo:block background-color="blue">  
        <xsl:apply-templates/>  
    </fo:block>  
</xsl:template>
```

Here, the XPath expression `li[@class='selected']/p` is equivalent to the CSS selector `li.selected > p` in the previous example. What CSS and XSL have in common is that they are both declarative languages for specifying the styling of a document.

Ở đây, biểu thức XPath `li[@class='selected']/p` tương đương với bộ chọn CSS `li.selected > p` trong ví dụ trước. Điểm chung của CSS và XSL là cả hai đều là ngôn ngữ khai báo để đặc tả cách định kiểu cho một tài liệu.

Imagine what life would be like if you had to use an imperative approach. In JavaScript, using the core Document Object Model (**DOM**) API, the result might look something like this:

Hãy tưởng tượng cuộc sống sẽ thế nào nếu bạn phải dùng một cách tiếp cận mệnh lệnh. Trong JavaScript, dùng API lõi Document Object Model (DOM), kết quả có thể trông như thế này:

```
var liElements = document.getElementsByTagName("li");
for (var i = 0; i < liElements.length; i++) {
  if (liElements[i].className === "selected") {
    var children = liElements[i].childNodes;
    for (var j = 0; j < children.length; j++) {
      var child = children[j];
      if (child.nodeType === Node.ELEMENT_NODE && child.tagName === "P") {
        child.setAttribute("style", "background-color: blue");
      }
    }
  }
}
```

This JavaScript imperatively sets the element

Sharks

to have a blue background, but the code is awful. Not only is it much longer and harder to understand than the CSS and XSL equivalents, but it also has some serious problems:

Đoạn JavaScript này đặt phần tử

Sharks

có nền xanh theo kiểu mệnh lệnh, nhưng đoạn mã thật kinh khủng. Không chỉ dài hơn nhiều và khó hiểu hơn nhiều so với các bản tương đương bằng CSS và XSL, nó còn có vài vấn đề nghiêm trọng:

- If the selected class is removed (e.g., because the user clicks a different page), the blue color won't be removed, even if the code is rerun—and so the item will remain highlighted until the entire page is reloaded. With CSS, the browser automatically detects when the `li.selected > p` rule no longer applies and removes the blue background as soon as the selected class is removed.
- If you want to take advantage of a new API, such as `document.getElementsByClassName("selected")` or even `document.evaluate()`—which may improve performance—you have to rewrite the code. On the other hand, browser vendors can improve the performance of CSS and XPath without breaking compatibility.
 - Nếu lớp `selected` bị gỡ bỏ (ví dụ, vì người dùng nhấp vào một trang khác), thì màu xanh sẽ không được gỡ bỏ, ngay cả khi mã được chạy lại — và như vậy mục đó sẽ vẫn được làm nổi bật cho đến khi toàn bộ trang được tải lại. Với CSS, trình duyệt tự động

phát hiện khi luật `li.selected > p` không còn áp dụng nữa và gỡ bỏ nền xanh ngay khi lớp `selected` bị gỡ bỏ.

- Nếu bạn muốn tận dụng một API mới, chẳng hạn `document.getElementsByClassName("selected")` hoặc thậm chí `document.evaluate()` — vốn có thể cải thiện hiệu năng — bạn phải viết lại mã. Mặt khác, các nhà cung cấp trình duyệt có thể cải thiện hiệu năng của CSS và XPath mà không phá vỡ tính tương thích.

In a web browser, using declarative CSS styling is much better than manipulating styles imperatively in JavaScript. Similarly, in databases, declarative query languages like SQL turned out to be much better than imperative query APIs.

Trong một trình duyệt web, dùng định kiểu CSS khai báo tốt hơn nhiều so với thao tác kiểu một cách mệnh lệnh trong JavaScript. Tương tự, trong cơ sở dữ liệu, các ngôn ngữ truy vấn khai báo như SQL hóa ra tốt hơn nhiều so với các API truy vấn mệnh lệnh.

MapReduce Querying — Truy vấn bằng MapReduce

MapReduce is a programming model for processing large amounts of data in bulk across many machines, popularized by Google [33]. A limited form of MapReduce is supported by some NoSQL datastores, including MongoDB and CouchDB, as a mechanism for performing read-only queries across many documents.

MapReduce là một mô hình lập trình để xử lý lượng lớn dữ liệu hàng loạt trên nhiều máy, được Google phổ biến [33]. Một dạng giới hạn của MapReduce được một số kho dữ liệu NoSQL hỗ trợ, bao gồm MongoDB và CouchDB, như một cơ chế để thực hiện các truy vấn chỉ-đọc trên nhiều tài liệu.

MapReduce in general is described in more detail in Chapter 10. For now, we'll just briefly discuss MongoDB's use of the model.

MapReduce nói chung được mô tả chi tiết hơn ở Chương 10. Bây giờ, ta sẽ chỉ bàn vắn tắt cách MongoDB dùng mô hình này.

MapReduce is neither a declarative query language nor a fully imperative query API, but somewhere in between: the logic of the query is expressed with snippets of code, which are called repeatedly by the processing framework. It is based on the map (also known as collect) and reduce (also known as fold or inject) functions that exist in many functional programming languages.

MapReduce không phải là một ngôn ngữ truy vấn khai báo cũng chẳng phải một API truy vấn hoàn toàn mệnh lệnh, mà ở đâu đó giữa hai thái cực: logic của truy vấn được biểu đạt bằng các mẫu mã, và những mẫu mã này được framework xử lý gọi đi gọi lại nhiều lần. Nó dựa trên các hàm `map` (còn gọi là `collect`) và `reduce` (còn gọi là `fold` hoặc `inject`) vốn tồn tại trong nhiều ngôn ngữ lập trình hàm.

To give an example, imagine you are a marine biologist, and you add an observation record to your database every time you see animals in the ocean. Now you want to generate a report saying how many sharks you have sighted per month.

Để cho một ví dụ, hãy tưởng tượng bạn là một nhà sinh vật biển, và bạn thêm một bản ghi quan sát vào cơ sở dữ liệu mỗi khi bạn thấy động vật ở đại dương. Giờ bạn muốn tạo một báo cáo cho biết mỗi tháng bạn đã trông thấy bao nhiêu con cá mập.

In PostgreSQL you might express that query like this:

Trong PostgreSQL bạn có thể biểu đạt truy vấn đó như sau:


```
SELECT date_trunc('month', observation_timestamp) AS observation_month,
       sum(num_animals) AS total_animals
FROM observations
WHERE family = 'Sharks'
GROUP BY observation_month;
```

The `date_trunc('month', timestamp)` function determines the calendar month containing timestamp, and returns another timestamp representing the beginning of that month. In other words, it rounds a timestamp down to the nearest month.

Hàm `date_trunc('month', timestamp)` xác định tháng dương lịch chứa timestamp, và trả về một timestamp khác biểu diễn thời điểm đầu tháng đó. Nói cách khác, nó làm tròn một timestamp xuống tháng gần nhất.

This query first filters the observations to only show species in the Sharks family, then groups the observations by the calendar month in which they occurred, and finally adds up the number of animals seen in all observations in that month.

Truy vấn này trước tiên lọc các quan sát để chỉ hiển thị các loài thuộc họ Sharks, rồi gom nhóm các quan sát theo tháng dương lịch mà chúng xảy ra, và cuối cùng cộng dồn số động vật trông thấy trong tất cả các quan sát trong tháng đó.

The same can be expressed with MongoDB's MapReduce feature as follows:

Điều tương tự có thể được biểu đạt bằng tính năng MapReduce của MongoDB như sau:

```
db.observations.mapReduce(
  function map() {
    var year = this.observationTimestamp.getFullYear();
    var month = this.observationTimestamp.getMonth() + 1;
    emit(year + "-" + month, this.numAnimals);
  },
  function reduce(key, values) {
    return Array.sum(values);
  },
  {
    query: { family: "Sharks" },
    out: "monthlySharkReport"
  }
);
```

The filter to consider only shark species can be specified declaratively (this is a MongoDB-specific extension to MapReduce).

Bộ lọc để chỉ xét các loài cá mập có thể được đặc tả theo lối khai báo (đây là một mở rộng riêng của MongoDB cho MapReduce).

The JavaScript function `map` is called once for every document that matches query, with this set to the document object.

Hàm JavaScript `map` được gọi một lần cho mỗi tài liệu khớp với query, với `this` được đặt thành đối tượng tài liệu.

The `map` function emits a key (a string consisting of year and month, such as “2013-12” or “2014-1”) and a value (the number of animals in that observation).

Hàm map phát ra một khóa (một chuỗi gồm năm và tháng, chẳng hạn “2013-12” hoặc “2014-1”) và một giá trị (số động vật trong quan sát đó).

The key-value pairs emitted by map are grouped by key. For all key-value pairs with the same key (i.e., the same month and year), the reduce function is called once.

Các cặp khóa-giá trị do map phát ra được gom nhóm theo khóa. Với mọi cặp khóa-giá trị có cùng khóa (tức là cùng tháng và năm), hàm reduce được gọi một lần.

The reduce function adds up the number of animals from all observations in a particular month.

Hàm reduce cộng dồn số động vật từ tất cả các quan sát trong một tháng cụ thể.

The final output is written to the collection monthlySharkReport.

Kết quả cuối cùng được ghi vào collection monthlySharkReport.

For example, say the observations collection contains these two documents:

Ví dụ, giả sử collection observations chứa hai tài liệu này:

```
{
  observationTimestamp: Date.parse("Mon, 25 Dec 1995 12:34:56 GMT"),
  family:              "Sharks",
  species:              "Carcharodon carcharias",
  numAnimals: 3
}
{
  observationTimestamp: Date.parse("Tue, 12 Dec 1995 16:17:18 GMT"),
  family:              "Sharks",
  species:              "Carcharias taurus",
  numAnimals: 4
}
```

The map function would be called once for each document, resulting in emit(“1995-12”, 3) and emit(“1995-12”, 4). Subsequently, the reduce function would be called with reduce(“1995-12”, [3, 4]), returning 7.

Hàm map sẽ được gọi một lần cho mỗi tài liệu, dẫn tới emit(“1995-12”, 3) và emit(“1995-12”, 4). Sau đó, hàm reduce sẽ được gọi với reduce(“1995-12”, [3, 4]), trả về 7.

The map and reduce functions are somewhat restricted in what they are allowed to do. They must be pure functions, which means they only use the data that is passed to them as input, they cannot perform additional database queries, and they must not have any side effects. These restrictions allow the database to run the functions anywhere, in any order, and rerun them on **failure**. However, they are nevertheless powerful: they can parse strings, call library functions, perform calculations, and more.

Các hàm map và reduce bị hạn chế phần nào về những gì chúng được phép làm. Chúng phải là hàm thuần khiết, nghĩa là chúng chỉ dùng dữ liệu được truyền vào làm đầu vào, không thể thực hiện thêm các truy vấn cơ sở dữ liệu, và không được có bất kỳ tác dụng phụ nào. Những hạn chế này cho phép cơ sở dữ liệu chạy các hàm ở bất cứ đâu, theo bất kỳ thứ tự nào, và chạy lại chúng khi gặp sự cố. Tuy nhiên, chúng vẫn rất mạnh: chúng có thể phân tích chuỗi, gọi các hàm thư viện, thực hiện tính toán, và hơn thế nữa.

MapReduce is a fairly low-level programming model for distributed execution on a cluster of machines. Higher-level query languages like SQL can be implemented as a pipeline of MapReduce operations (see Chapter 10), but there are also many distributed implementations of SQL that don’t

use MapReduce. Note there is nothing in SQL that constrains it to running on a single machine, and MapReduce doesn't have a monopoly on distributed query execution.

MapReduce là một mô hình lập trình khá ở mức thấp (low-level) cho việc thực thi phân tán trên một cụm máy. Các ngôn ngữ truy vấn ở mức cao hơn như SQL có thể được hiện thực dưới dạng một đường ống (pipeline) các thao tác MapReduce (xem Chương 10), nhưng cũng có nhiều hiện thực phân tán của SQL không dùng MapReduce. Lưu ý rằng không có gì trong SQL ràng buộc nó chỉ chạy trên một máy duy nhất, và MapReduce không độc quyền việc thực thi truy vấn phân tán.

Being able to use JavaScript code in the middle of a query is a great feature for advanced queries, but it's not limited to MapReduce—some SQL databases can be extended with JavaScript functions too [34].

Khả năng dùng mã JavaScript ngay giữa một truy vấn là một tính năng tuyệt vời cho các truy vấn nâng cao, nhưng nó không chỉ giới hạn ở MapReduce — một số cơ sở dữ liệu SQL cũng có thể được mở rộng bằng các hàm JavaScript [34].

A usability problem with MapReduce is that you have to write two carefully coordinated JavaScript functions, which is often harder than writing a single query. Moreover, a declarative query language offers more opportunities for a query optimizer to improve the performance of a query. For these reasons, MongoDB 2.2 added support for a declarative query language called the **aggregation pipeline** [9]. In this language, the same shark-counting query looks like this:

Một vấn đề về khả năng dễ dùng của MapReduce là bạn phải viết hai hàm JavaScript được phối hợp cẩn thận, điều này thường khó hơn việc viết một truy vấn đơn lẻ. Hơn nữa, một ngôn ngữ truy vấn khai báo mang lại nhiều cơ hội hơn cho một bộ tối ưu truy vấn để cải thiện hiệu năng của một truy vấn. Vì những lý do này, MongoDB 2.2 đã thêm hỗ trợ cho một ngôn ngữ truy vấn khai báo gọi là aggregation pipeline [9]. Trong ngôn ngữ này, cùng truy vấn đếm cá mập đó trông như thế này:

```
db.observations.aggregate([
  { $match: { family: "Sharks" } },
  { $group: {
    _id: {
      year: { $year: "$observationTimestamp" },
      month: { $month: "$observationTimestamp" }
    },
    totalAnimals: { $sum: "$numAnimals" }
  } }
]);
```

The aggregation pipeline language is similar in expressiveness to a subset of SQL, but it uses a JSON-based syntax rather than SQL's English-sentence-style syntax; the difference is perhaps a matter of taste. The moral of the story is that a NoSQL system may find itself accidentally reinventing SQL, albeit in disguise.

Ngôn ngữ aggregation pipeline tương đồng về sức biểu đạt với một tập con của SQL, nhưng nó dùng cú pháp dựa trên JSON thay vì cú pháp kiểu câu-tiếng-Anh của SQL; khác biệt có lẽ là vấn đề khẩu vị. Bài học rút ra là một hệ thống NoSQL có thể vô tình thấy mình đang phát minh lại SQL, dẫu là dưới một lớp ngụy trang.

Graph-Like Data Models — Các mô hình dữ liệu dạng đồ thị

We saw earlier that many-to-many relationships are an important distinguishing feature between different data models. If your application has mostly one-to-many relationships (tree-structured data) or no relationships between records, the document model is appropriate.

Trước đó ta đã thấy các quan hệ nhiều-nhiều là một đặc điểm phân biệt quan trọng giữa các mô hình dữ liệu khác nhau. Nếu ứng dụng của bạn chủ yếu có các quan hệ một-nhiều (dữ liệu cấu trúc cây) hoặc không có quan hệ nào giữa các bản ghi, thì mô hình tài liệu là phù hợp.

But what if many-to-many relationships are very common in your data? The relational model can handle simple cases of many-to-many relationships, but as the connections within your data become more complex, it becomes more natural to start modeling your data as a graph.

Nhưng sẽ ra sao nếu các quan hệ nhiều-nhiều rất phổ biến trong dữ liệu của bạn? Mô hình quan hệ có thể xử lý các trường hợp nhiều-nhiều đơn giản, nhưng khi các kết nối bên trong dữ liệu của bạn trở nên phức tạp hơn, thì việc bắt đầu mô hình hóa dữ liệu dưới dạng một đồ thị trở nên tự nhiên hơn.

A graph consists of two kinds of objects: vertices (also known as nodes or entities) and edges (also known as relationships or arcs). Many kinds of data can be modeled as a graph. Typical examples include:

Một đồ thị bao gồm hai loại đối tượng: đỉnh (vertex) (còn gọi là nút hoặc thực thể) và cạnh (edge) (còn gọi là quan hệ hoặc cung). Nhiều loại dữ liệu có thể được mô hình hóa dưới dạng một đồ thị. Các ví dụ điển hình bao gồm:

Vertices are people, and edges indicate which people know each other.

Các đỉnh là người, và các cạnh cho biết những ai quen biết nhau.

Vertices are web pages, and edges indicate HTML links to other pages.

Các đỉnh là trang web, và các cạnh cho biết các liên kết HTML tới các trang khác.

Vertices are junctions, and edges represent the roads or railway lines between them.

Các đỉnh là các nút giao, và các cạnh biểu diễn đường bộ hoặc đường sắt nối giữa chúng.

Well-known algorithms can operate on these graphs: for example, car navigation systems search for the shortest path between two points in a road network, and PageRank can be used on the web graph to determine the popularity of a web page and thus its ranking in search results.

Các thuật toán nổi tiếng có thể vận hành trên những đồ thị này: ví dụ, các hệ thống dẫn đường ô tô tìm đường ngắn nhất giữa hai điểm trong một mạng lưới đường bộ, và PageRank có thể được dùng trên đồ thị web để xác định độ phổ biến của một trang web và do đó là thứ hạng của nó trong kết quả tìm kiếm.

In the examples just given, all the vertices in a graph represent the same kind of thing (people, web pages, or road junctions, respectively). However, graphs are not limited to such homogeneous data: an equally powerful use of graphs is to provide a consistent way of storing completely different types of objects in a single **datastore**. For example, Facebook maintains a single graph with many different types of vertices and edges: vertices represent people, locations, events, checkins, and comments made by users; edges indicate which people are friends with each other, which checkin happened in which location, who commented on which post, who attended which event, and so on [35].

Trong các ví dụ vừa nêu, mọi đỉnh trong một đồ thị đều biểu diễn cùng một loại thứ (lần lượt là người, trang web, hoặc nút giao đường bộ). Tuy nhiên, đồ thị không bị giới hạn ở dữ liệu đồng nhất như vậy: một cách dùng đồ thị mạnh mẽ không kém là cung cấp một cách nhất quán để lưu trữ những loại đối tượng hoàn toàn khác nhau trong một kho dữ liệu duy nhất. Ví dụ, Facebook duy trì một đồ thị duy nhất với nhiều loại đỉnh và cạnh khác nhau: các đỉnh biểu diễn người, địa điểm, sự kiện, lượt check-in, và các bình luận mà người dùng viết; các cạnh cho biết những ai là bạn bè với nhau, lượt check-in nào xảy ra ở địa điểm nào, ai bình luận vào bài đăng nào, ai tham dự sự kiện nào, v.v. [35].

In this section we will use the example shown in Figure 2-5. It could be taken from a social network or a genealogical database: it shows two people, Lucy from Idaho and Alain from Beaune, France. They are married and living in London.

Trong phần này ta sẽ dùng ví dụ minh họa trong Hình 2-5. Nó có thể được lấy từ một mạng xã hội hoặc một cơ sở dữ liệu gia phả: nó cho thấy hai người, Lucy đến từ Idaho và Alain đến từ Beaune, Pháp. Họ đã kết hôn và đang sống ở London.

Figure 2-5. Example of graph-structured data (boxes represent vertices, arrows represent edges). — **Hình 2-5. Ví dụ về dữ liệu cấu trúc đồ thị (các ô vuông biểu diễn đỉnh, các mũi tên biểu diễn cạnh).** There are several different, but related, ways of structuring and querying data in graphs. In this section we will discuss the **property graph** model (implemented by Neo4j, Titan, and InfiniteGraph) and the **triple-store** model (implemented by Datomic, AllegroGraph, and others). We will look at three declarative query languages for graphs: **Cypher**, **SPARQL**, and **Datalog**. Besides these, there are also imperative graph query languages such as Gremlin [36] and graph processing frameworks like Pregel (see Chapter 10).

Có vài cách khác nhau, nhưng có liên hệ với nhau, để cấu trúc và truy vấn dữ liệu trong đồ thị. Trong phần này ta sẽ bàn về mô hình đồ thị thuộc tính (property graph) (được hiện thực bởi Neo4j, Titan, và InfiniteGraph) và mô hình kho bộ ba (triple-store) (được hiện thực bởi Datomic, AllegroGraph, và những hệ khác). Ta sẽ xem xét ba ngôn ngữ truy vấn khai báo cho đồ thị: Cypher, SPARQL, và Datalog. Bên cạnh đó, cũng có những ngôn ngữ truy vấn đồ thị mệnh lệnh như Gremlin [36] và các framework xử lý đồ thị như Pregel (xem Chương 10).

Property Graphs — Đồ thị thuộc tính

In the **property graph** model, each **vertex** consists of:

Trong mô hình đồ thị thuộc tính, mỗi đỉnh bao gồm:

- A unique identifier
- A set of outgoing edges
- A set of incoming edges
- A collection of properties (key-value pairs)
 - Một định danh duy nhất.
 - Một tập hợp các cạnh đi ra.
 - Một tập hợp các cạnh đi vào.
 - Một tập hợp các thuộc tính (các cặp khóa-giá trị).

Each **edge** consists of:

Mỗi cạnh bao gồm:

- A unique identifier
- The vertex at which the edge starts (the tail vertex)
- The vertex at which the edge ends (the head vertex)
- A **label** to describe the kind of relationship between the two vertices
- A collection of properties (key-value pairs)
 - Một định danh duy nhất.
 - Đỉnh mà cạnh bắt đầu (đỉnh đuôi).
 - Đỉnh mà cạnh kết thúc (đỉnh đầu).
 - Một nhãn để mô tả loại quan hệ giữa hai đỉnh.
 - Một tập hợp các thuộc tính (các cặp khóa-giá trị).

You can think of a graph store as consisting of two relational tables, one for vertices and one for edges, as shown in Example 2-2 (this schema uses the PostgreSQL json datatype to store the properties of each vertex or edge). The head and tail vertex are stored for each edge; if you want the set of incoming or outgoing edges for a vertex, you can query the edges table by head_vertex or tail_vertex, respectively.

Bạn có thể hình dung một kho đồ thị gồm hai bảng quan hệ, một cho các đỉnh và một cho các cạnh, như trong Ví dụ 2-2 (lược đồ này dùng kiểu dữ liệu json của PostgreSQL để lưu các thuộc tính của mỗi đỉnh hoặc cạnh). Đỉnh đầu và đỉnh đuôi được lưu cho mỗi cạnh; nếu bạn muốn tập hợp các cạnh đi vào hoặc đi ra của một đỉnh, bạn có thể truy vấn bảng edges lần lượt theo head_vertex hoặc tail_vertex.

Example 2-2. Representing a property graph using a relational schema — Ví dụ 2-2. Biểu diễn một đồ thị thuộc tính bằng lược đồ quan hệ

```
CREATE TABLE vertices (
    vertex_id integer PRIMARY KEY,
    properties json
);
CREATE TABLE edges (
    edge_id integer PRIMARY KEY,
    tail_vertex integer REFERENCES vertices (vertex_id),
    head_vertex integer REFERENCES vertices (vertex_id),
    label text,
    properties json
);
CREATE INDEX edges_tails ON edges (tail_vertex);
CREATE INDEX edges_heads ON edges (head_vertex);
```


Some important aspects of this model are:

Một vài khía cạnh quan trọng của mô hình này là:

- Any vertex can have an edge connecting it with any other vertex. There is no schema that restricts which kinds of things can or cannot be associated.
- Given any vertex, you can efficiently find both its incoming and its outgoing edges, and thus **traverse** the graph—i.e., follow a path through a chain of vertices—both forward and backward. (That's why Example 2-2 has indexes on both the `tail_vertex` and `head_vertex` columns.)
- By using different labels for different kinds of relationships, you can store several different kinds of information in a single graph, while still maintaining a clean data model.
 - Bất kỳ đỉnh nào cũng có thể có một cạnh nối nó với bất kỳ đỉnh nào khác. Không có lược đồ nào hạn chế những loại thứ nào có thể hoặc không thể được liên kết với nhau.
 - Cho trước bất kỳ đỉnh nào, bạn có thể tìm hiệu quả cả các cạnh đi vào lẫn các cạnh đi ra của nó, và nhờ đó duyệt đồ thị — tức là đi theo một đường xuyên qua một chuỗi các đỉnh — theo cả chiều tiến lẫn chiều lùi. (Đó là lý do Ví dụ 2-2 có các chỉ mục trên cả hai cột `tail_vertex` và `head_vertex`.)
 - Bằng cách dùng các nhãn khác nhau cho các loại quan hệ khác nhau, bạn có thể lưu vài loại thông tin khác nhau trong một đồ thị duy nhất, mà vẫn giữ được một mô hình dữ liệu gọn gàng.

Those features give graphs a great deal of flexibility for data modeling, as illustrated in Figure 2-5. The figure shows a few things that would be difficult to express in a traditional relational schema, such as different kinds of regional structures in different countries (France has départements and régions, whereas the US has counties and states), quirks of history such as a country within a country (ignoring for now the intricacies of sovereign states and nations), and varying granularity of data (Lucy's current residence is specified as a city, whereas her place of birth is specified only at the level of a state).

Những đặc điểm đó mang lại cho đồ thị sự linh hoạt rất lớn trong việc mô hình hóa dữ liệu, như minh họa trong Hình 2-5. Hình này cho thấy vài thứ sẽ khó biểu diễn trong một lược đồ quan hệ truyền thống, chẳng hạn các loại cấu trúc vùng miền khác nhau ở các quốc gia khác nhau (Pháp có départements và régions, trong khi Mỹ có counties và states), những điều kỳ quặc của lịch sử như một quốc gia nằm trong một quốc gia (tạm bỏ qua những rắc rối phức tạp giữa nhà nước có chủ quyền và quốc gia-dân tộc), và độ chi tiết khác nhau của dữ liệu (nơi cư trú hiện tại của Lucy được nêu rõ tới cấp thành phố, trong khi nơi sinh của cô chỉ được nêu tới cấp bang).

You could imagine extending the graph to also include many other facts about Lucy and Alain, or other people. For **instance**, you could use it to indicate any food allergies they have (by introducing a vertex for each allergen, and an edge between a person and an allergen to indicate an allergy), and link the allergens with a set of vertices that show which foods contain which substances. Then you could write a query to find out what is safe for each person to eat. Graphs are good for **evolvability**: as you add features to your application, a graph can easily be extended to accommodate changes in your application's data structures.

Bạn có thể hình dung việc mở rộng đồ thị để bao gồm cả nhiều sự thật khác về Lucy và Alain, hoặc về những người khác. Chẳng hạn, bạn có thể dùng nó để chỉ ra bất kỳ dị ứng thực phẩm nào họ có (bằng cách đưa vào một đỉnh cho mỗi chất gây dị ứng, và một cạnh giữa một người và một chất gây dị ứng để chỉ một sự dị ứng), và liên kết các chất gây dị ứng với một tập các đỉnh cho biết món ăn nào chứa chất nào. Rồi bạn có thể viết một truy vấn để tìm ra món nào an toàn để mỗi người có thể ăn. Đồ thị rất thuận lợi cho khả năng

tiến hóa: khi bạn thêm tính năng vào ứng dụng, một đồ thị có thể dễ dàng được mở rộng để thích nghi với những thay đổi trong các cấu trúc dữ liệu của ứng dụng.

The Cypher Query Language — Ngôn ngữ truy vấn Cypher

Cypher is a declarative query language for property graphs, created for the Neo4j graph database [37]. (It is named after a character in the movie The Matrix and is not related to ciphers in cryptography [38].)

Cypher là một ngôn ngữ truy vấn khai báo dành cho đồ thị thuộc tính, được tạo ra cho cơ sở dữ liệu đồ thị Neo4j [37]. (Nó được đặt theo tên một nhân vật trong phim The Matrix và không liên quan tới ciphers trong mật mã học [38].)

Example 2-3 shows the Cypher query to insert the lefthand portion of Figure 2-5 into a graph database. The rest of the graph can be added similarly and is omitted for readability. Each vertex is given a symbolic name like USA or Idaho, and other parts of the query can use those names to create edges between the vertices, using an arrow notation: (Idaho) -[:WITHIN]-> (USA) creates an edge labeled WITHIN, with Idaho as the tail **node** and USA as the head node.

Ví dụ 2-3 cho thấy truy vấn Cypher để chèn phần bên trái của Hình 2-5 vào một cơ sở dữ liệu đồ thị. Phần còn lại của đồ thị có thể được thêm tương tự và được lược bỏ cho dễ đọc. Mỗi đỉnh được đặt một tên ký hiệu như USA hoặc Idaho, và các phần khác của truy vấn có thể dùng những tên đó để tạo cạnh giữa các đỉnh, dùng ký pháp mũi tên: (Idaho) -[:WITHIN]-> (USA) tạo một cạnh có nhãn WITHIN, với Idaho là đỉnh đuôi và USA là đỉnh đầu.

Example 2-3. A subset of the data in Figure 2-5, represented as a Cypher query — Ví dụ 2-3. Một tập con của dữ liệu trong Hình 2-5, biểu diễn dưới dạng một truy vấn Cypher

```
CREATE
  (NAmerica:Location {name:'North America', type:'continent'}),
  (USA:Location      {name:'United States', type:'country'  }),
  (Idaho:Location    {name:'Idaho',          type:'state'    }),
  (Lucy:Person       {name:'Lucy' }),
  (Idaho) -[:WITHIN]-> (USA) -[:WITHIN]-> (NAmerica),
  (Lucy)  -[:BORN_IN]-> (Idaho)
```

When all the vertices and edges of Figure 2-5 are added to the database, we can start asking interesting questions: for example, find the names of all the people who emigrated from the United States to Europe. To be more precise, here we want to find all the vertices that have a BORN_IN edge to a location within the US, and also a LIVING_IN edge to a location within Europe, and return the name property of each of those vertices.

Khi tất cả các đỉnh và cạnh của Hình 2-5 được thêm vào cơ sở dữ liệu, ta có thể bắt đầu đặt những câu hỏi thú vị: ví dụ, tìm tên của tất cả những người đã di cư từ Hoa Kỳ sang châu Âu. Nói chính xác hơn, ở đây ta muốn tìm tất cả các đỉnh có một cạnh BORN_IN tới một địa điểm bên trong Hoa Kỳ, và cũng có một cạnh LIVING_IN tới một địa điểm bên trong châu Âu, rồi trả về thuộc tính name của mỗi đỉnh đó.

Example 2-4 shows how to express that query in Cypher. The same arrow notation is used in a MATCH clause to find patterns in the graph: (person) -[:BORN_IN]-> () matches any two vertices that are related by an edge labeled BORN_IN. The tail vertex of that edge is bound to the variable person, and the head vertex is left unnamed.

Ví dụ 2-4 cho thấy cách biểu đạt truy vấn đó trong Cypher. Cùng ký pháp mũi tên được dùng trong mệnh đề MATCH để tìm các khuôn mẫu trong đồ thị: (person) -[:BORN_IN]-> () khớp với bất kỳ hai đỉnh nào có quan hệ với nhau qua một cạnh nhãn BORN_IN. Đỉnh đuôi của cạnh đó được ràng buộc vào biến person, còn đỉnh đầu thì để không đặt tên.

Example 2-4. Cypher query to find people who emigrated from the US to Europe — Ví dụ 2-4. Truy vấn Cypher để tìm những người đã di cư từ Mỹ sang châu Âu

```
MATCH
  (person) -[:BORN_IN]-> () -[:WITHIN*0..]-> (us:Location {name:'United States'}),
  (person) -[:LIVES_IN]-> () -[:WITHIN*0..]-> (eu:Location {name:'Europe'})
RETURN person.name
```

The query can be read as follows:

Truy vấn có thể đọc như sau:

Find any vertex (call it person) that meets both of the following conditions:

Tìm bất kỳ đỉnh nào (gọi nó là person) thỏa cả hai điều kiện sau:

- person has an outgoing BORN_IN edge to some vertex. From that vertex, you can follow a chain of outgoing WITHIN edges until eventually you reach a vertex of type Location, whose name property is equal to “United States”.
- That same person vertex also has an outgoing LIVES_IN edge. Following that edge, and then a chain of outgoing WITHIN edges, you eventually reach a vertex of type Location, whose name property is equal to “Europe”.
 - person có một cạnh BORN_IN đi ra tới một đỉnh nào đó. Từ đỉnh đó, bạn có thể đi theo một chuỗi các cạnh WITHIN đi ra cho đến khi cuối cùng đạt tới một đỉnh kiểu Location, có thuộc tính name bằng “United States”.
 - Cũng chính đỉnh person đó còn có một cạnh LIVES_IN đi ra. Đi theo cạnh đó, rồi một chuỗi các cạnh WITHIN đi ra, cuối cùng bạn đạt tới một đỉnh kiểu Location, có thuộc tính name bằng “Europe”.

For each such person vertex, return the name property.

Với mỗi đỉnh person như vậy, trả về thuộc tính name.

There are several possible ways of executing the query. The description given here suggests that you start by scanning all the people in the database, examine each person’s birthplace and residence, and return only those people who meet the criteria.

Có vài cách khả dĩ để thực thi truy vấn. Mô tả nêu ở đây gợi ý rằng bạn bắt đầu bằng cách quét tất cả những người trong cơ sở dữ liệu, xem xét nơi sinh và nơi cư trú của mỗi người, và chỉ trả về những người thỏa tiêu chí.

But equivalently, you could start with the two Location vertices and work backward. If there is an index on the name property, you can probably efficiently find the two vertices representing the US and Europe. Then you can proceed to find all locations (states, regions, cities, etc.) in the US and Europe respectively by following all incoming WITHIN edges. Finally, you can look for people who can be found through an incoming BORN_IN or LIVES_IN edge at one of the location vertices.

Nhưng một cách tương đương, bạn có thể bắt đầu với hai đỉnh Location và đi ngược lại. Nếu có một chỉ mục trên thuộc tính name, bạn có lẽ có thể tìm hiệu quả hai đỉnh biểu diễn Mỹ và châu Âu. Rồi bạn có thể tiến hành tìm tất cả các địa điểm (bang, vùng, thành phố, v.v.) lần lượt ở Mỹ và châu Âu bằng cách đi theo tất cả các cạnh WITHIN đi vào. Cuối cùng,

bạn có thể tìm những người có thể được tìm thấy qua một cạnh BORN_IN hoặc LIVES_IN đi vào ở một trong các đỉnh địa điểm.

As is typical for a declarative query language, you don't need to specify such execution details when writing the query: the query optimizer automatically chooses the strategy that is predicted to be the most efficient, so you can get on with writing the rest of your application.

Như thường thấy với một ngôn ngữ truy vấn khai báo, bạn không cần đặc tả các chi tiết thực thi như vậy khi viết truy vấn: bộ tối ưu truy vấn tự động chọn chiến lược được dự đoán là hiệu quả nhất, để bạn có thể tiếp tục viết phần còn lại của ứng dụng.

Graph Queries in SQL — Truy vấn đồ thị bằng SQL

Example 2-2 suggested that graph data can be represented in a relational database. But if we put graph data in a relational structure, can we also query it using SQL?

Ví dụ 2-2 đã gợi ý rằng dữ liệu đồ thị có thể được biểu diễn trong một cơ sở dữ liệu quan hệ. Nhưng nếu ta đặt dữ liệu đồ thị vào một cấu trúc quan hệ, liệu ta có thể truy vấn nó bằng SQL không?

The answer is yes, but with some difficulty. In a relational database, you usually know in advance which joins you need in your query. In a graph query, you may need to traverse a variable number of edges before you find the vertex you're looking for—that is, the number of joins is not fixed in advance.

Câu trả lời là có, nhưng với đôi chút khó khăn. Trong một cơ sở dữ liệu quan hệ, bạn thường biết trước những phép join nào mình cần trong truy vấn. Trong một truy vấn đồ thị, bạn có thể cần duyệt một số lượng cạnh không cố định trước khi tìm thấy đỉnh mình đang tìm — tức là số phép join không được cố định trước.

In our example, that happens in the `() -[:WITHIN*0..]-> ()` rule in the Cypher query. A person's LIVES_IN edge may point at any kind of location: a street, a city, a district, a region, a state, etc. A city may be WITHIN a region, a region WITHIN a state, a state WITHIN a country, etc. The LIVES_IN edge may point directly at the location vertex you're looking for, or it may be several levels removed in the location hierarchy.

Trong ví dụ của ta, điều đó xảy ra ở luật `() -[:WITHIN*0..]-> ()` trong truy vấn Cypher. Cạnh LIVES_IN của một người có thể trở tới bất kỳ loại địa điểm nào: một con phố, một thành phố, một quận, một vùng, một bang, v.v. Một thành phố có thể WITHIN một vùng, một vùng WITHIN một bang, một bang WITHIN một quốc gia, v.v. Cạnh LIVES_IN có thể trở thẳng tới đỉnh địa điểm bạn đang tìm, hoặc nó có thể cách vài cấp trong hệ phân cấp địa điểm.

In Cypher, `:WITHIN0..` expresses that fact very concisely: it means “follow a WITHIN edge, zero or more times.” It is like the `*` operator in a regular expression.

Trong Cypher, `:WITHIN0..` biểu đạt sự thật đó một cách rất gọn: nó nghĩa là “đi theo một cạnh WITHIN, không hoặc nhiều lần.” Nó giống toán tử `*` trong một biểu thức chính quy.

Since SQL:1999, this idea of variable-length traversal paths in a query can be expressed using something called recursive common table expressions (the WITH RECURSIVE syntax). Example 2-5 shows the same query—finding the names of people who emigrated from the US to Europe—expressed in SQL using this technique (supported in PostgreSQL, IBM DB2, Oracle, and SQL Server). However, the syntax is very clumsy in comparison to Cypher.

Kể từ SQL:1999, ý tưởng về các đường duyệt có độ dài thay đổi trong một truy vấn có thể được biểu đạt bằng một thứ gọi là biểu thức bảng chung đệ quy (cú pháp WITH RECURSIVE). Ví dụ 2-5 cho thấy cùng truy vấn đó — tìm tên những người đã di cư từ Mỹ sang châu Âu — được biểu đạt trong SQL bằng kỹ thuật này (được hỗ trợ trong PostgreSQL, IBM DB2, Oracle, và SQL Server). Tuy nhiên, cú pháp rất vụng về so với Cypher.

Example 2-5. The same query as Example 2-4, expressed in SQL using recursive common table expressions — Ví dụ 2-5. Cùng truy vấn như Ví dụ 2-4, biểu đạt trong SQL bằng biểu thức bảng chung đệ quy

```
WITH RECURSIVE
-- in_usa is the set of vertex IDs of all locations within the United States
in_usa(vertex_id) AS (
    SELECT vertex_id FROM vertices WHERE properties->>'name' = 'United States'
    UNION
    SELECT edges.tail_vertex FROM edges
    JOIN in_usa ON edges.head_vertex = in_usa.vertex_id
    WHERE edges.label = 'within'
),
-- in_europe is the set of vertex IDs of all locations within Europe
in_europe(vertex_id) AS (
    SELECT vertex_id FROM vertices WHERE properties->>'name' = 'Europe'
    UNION
    SELECT edges.tail_vertex FROM edges
    JOIN in_europe ON edges.head_vertex = in_europe.vertex_id
    WHERE edges.label = 'within'
),
-- born_in_usa is the set of vertex IDs of all people born in the US
born_in_usa(vertex_id) AS (
    SELECT edges.tail_vertex FROM edges
    JOIN in_usa ON edges.head_vertex = in_usa.vertex_id
    WHERE edges.label = 'born_in'
),
-- lives_in_europe is the set of vertex IDs of all people living in Europe
lives_in_europe(vertex_id) AS (
    SELECT edges.tail_vertex FROM edges
    JOIN in_europe ON edges.head_vertex = in_europe.vertex_id
    WHERE edges.label = 'lives_in'
)
SELECT vertices.properties->>'name'
FROM vertices
-- join to find those people who were both born in the US *and* live in Europe
JOIN born_in_usa ON vertices.vertex_id = born_in_usa.vertex_id
JOIN lives_in_europe ON vertices.vertex_id = lives_in_europe.vertex_id;
```

First find the vertex whose name property has the value “United States”, and make it the first element of the set of vertices `in_usa`.

Trước tiên tìm đỉnh có thuộc tính `name` mang giá trị “United States”, và đặt nó làm phần tử đầu tiên của tập các đỉnh `in_usa`.

Follow all incoming `within` edges from vertices in the set `in_usa`, and add them to the same set, until

all incoming within edges have been visited.

Đi theo tất cả các cạnh within đi vào từ các đỉnh trong tập `in_usa`, và thêm chúng vào cùng tập đó, cho đến khi mọi cạnh within đi vào đã được duyệt qua.

Do the same starting with the vertex whose name property has the value “Europe”, and build up the set of vertices `in_europe`.

Làm tương tự, bắt đầu với đỉnh có thuộc tính name mang giá trị “Europe”, và dựng nên tập các đỉnh `in_europe`.

For each of the vertices in the set `in_usa`, follow incoming `born_in` edges to find people who were born in some place within the United States.

Với mỗi đỉnh trong tập `in_usa`, đi theo các cạnh `born_in` đi vào để tìm những người đã sinh ra ở một nơi nào đó bên trong Hoa Kỳ.

Similarly, for each of the vertices in the set `in_europe`, follow incoming `lives_in` edges to find people who live in Europe.

Tương tự, với mỗi đỉnh trong tập `in_europe`, đi theo các cạnh `lives_in` đi vào để tìm những người đang sống ở châu Âu.

Finally, intersect the set of people born in the USA with the set of people living in Europe, by joining them.

Cuối cùng, giao tập những người sinh ra ở Mỹ với tập những người đang sống ở châu Âu, bằng cách join chúng lại.

If the same query can be written in 4 lines in one query language but requires 29 lines in another, that just shows that different data models are designed to satisfy different use cases. It's important to pick a data model that is suitable for your application.

Nếu cùng một truy vấn có thể viết trong 4 dòng bằng một ngôn ngữ truy vấn này nhưng đòi hỏi 29 dòng bằng một ngôn ngữ khác, thì điều đó chỉ cho thấy các mô hình dữ liệu khác nhau được thiết kế để đáp ứng các tình huống sử dụng khác nhau. Điều quan trọng là chọn một mô hình dữ liệu phù hợp với ứng dụng của bạn.

Triple-Stores and SPARQL — Kho bộ ba và SPARQL

The **triple**-store model is mostly equivalent to the property graph model, using different words to describe the same ideas. It is nevertheless worth discussing, because there are various tools and languages for triple-stores that can be valuable additions to your toolbox for building applications.

Mô hình kho bộ ba phần lớn tương đương với mô hình đồ thị thuộc tính, dùng những từ ngữ khác nhau để mô tả cùng các ý tưởng. Tuy vậy nó vẫn đáng để bàn, vì có nhiều công cụ và ngôn ngữ cho kho bộ ba có thể là những bổ sung giá trị cho bộ đồ nghề xây dựng ứng dụng của bạn.

In a triple-store, all information is stored in the form of very simple three-part statements: (**subject**, **predicate**, object). For example, in the triple (Jim, likes, bananas), Jim is the subject, likes is the predicate (verb), and bananas is the object.

Trong một kho bộ ba, mọi thông tin được lưu dưới dạng các phát biểu ba phần rất đơn giản: (chủ thể, vị từ, đối thể). Ví dụ, trong bộ ba (Jim, likes, bananas), Jim là chủ thể, likes là vị từ (động từ), và bananas là đối thể.

The subject of a triple is equivalent to a vertex in a graph. The object is one of two things:

Chủ thể của một bộ ba tương đương với một đỉnh trong một đồ thị. Đối thể là một trong hai thứ:

- A value in a primitive datatype, such as a string or a number. In that case, the predicate and object of the triple are equivalent to the key and value of a property on the subject vertex. For example, (lucy, age, 33) is like a vertex lucy with properties {"age":33}.
- Another vertex in the graph. In that case, the predicate is an edge in the graph, the subject is the tail vertex, and the object is the head vertex. For example, in (lucy, marriedTo, alain) the subject and object lucy and alain are both vertices, and the predicate marriedTo is the label of the edge that connects them.
 - Một giá trị thuộc kiểu dữ liệu nguyên thủy, chẳng hạn một chuỗi hoặc một số. Trong trường hợp đó, vị từ và đối thể của bộ ba tương đương với khóa và giá trị của một thuộc tính trên đỉnh chủ thể. Ví dụ, (lucy, age, 33) giống một đỉnh lucy với thuộc tính {"age":33}.
 - Một đỉnh khác trong đồ thị. Trong trường hợp đó, vị từ là một cạnh trong đồ thị, chủ thể là đỉnh đuôi, và đối thể là đỉnh đầu. Ví dụ, trong (lucy, marriedTo, alain) thì chủ thể và đối thể lucy và alain đều là đỉnh, còn vị từ marriedTo là nhãn của cạnh nối chúng.

Example 2-6 shows the same data as in Example 2-3, written as triples in a format called Turtle, a subset of Notation3 (N3) [39].

Ví dụ 2-6 cho thấy cùng dữ liệu như trong Ví dụ 2-3, được viết dưới dạng các bộ ba theo một định dạng gọi là Turtle, một tập con của Notation3 (N3) [39].

Example 2-6. A subset of the data in Figure 2-5, represented as Turtle triples — Ví dụ 2-6. Một tập con của dữ liệu trong Hình 2-5, biểu diễn dưới dạng các bộ ba Turtle

```
@prefix : <urn:example:>.
_:lucy      a      :Person.
_:lucy      :name   "Lucy".
_:lucy      :bornIn _:idaho.
_:idaho     a      :Location.
_:idaho     :name   "Idaho".
_:idaho     :type   "state".
_:idaho     :within _:usa.
_:usa       a      :Location.
_:usa       :name   "United States".
_:usa       :type   "country".
_:usa       :within _:namerica.
_:namerica  a      :Location.
_:namerica  :name   "North America".
_:namerica  :type   "continent".
```

In this example, vertices of the graph are written as *:someName*. *The name doesn't mean anything outside of this file; it exists only because we otherwise wouldn't know which triples refer to the same vertex. When the predicate represents an edge, the object is a vertex, as in :idaho :within :usa. When the predicate is a property, the object is a string literal, as in :usa :name "United States".*

Trong ví dụ này, các đỉnh của đồ thị được viết là *:someName*. *Cái tên này không mang ý nghĩa gì bên ngoài file này; nó tồn tại chỉ vì nếu không thì ta sẽ không biết những bộ ba nào*

chỉ tới cùng một đỉnh. Khi vị từ biểu diễn một cạnh, thì đối thể là một đỉnh, như trong :idaho :within :usa. Khi vị từ là một thuộc tính, thì đối thể là một chuỗi nguyên văn, như trong :usa :name "United States".

It's quite repetitive to repeat the same subject over and over again, but fortunately you can use semicolons to say multiple things about the same subject. This makes the Turtle format quite nice and readable: see Example 2-7.

Khá là lặp đi lặp lại khi cứ phải nhắc lại cùng một chủ thể hết lần này tới lần khác, nhưng may mắn là bạn có thể dùng dấu chấm phẩy để nói nhiều điều về cùng một chủ thể. Điều này khiến định dạng Turtle khá đẹp và dễ đọc: xem Ví dụ 2-7.

Example 2-7. A more concise way of writing the data in Example 2-6 — Ví dụ 2-7. Một cách viết gọn hơn cho dữ liệu trong Ví dụ 2-6

```
@prefix : <urn:example:>.
_:lucy      a :Person;      :name "Lucy";          :bornIn _:idaho.
_:idaho     a :Location; :name "Idaho";          :type "state";      :within _:usa.
_:usa       a :Location; :name "United States"; :type "country"; :within _:namerica.
_:namerica  a :Location; :name "North America"; :type "continent".
```

The semantic web — Web ngữ nghĩa

If you read more about triple-stores, you may get sucked into a maelstrom of articles written about the **semantic web**. The triple-store data model is completely independent of the semantic web—for example, Datomic [40] is a triple-store that does not claim to have anything to do with it. But since the two are so closely linked in many people's minds, we should discuss them briefly.

Nếu bạn đọc thêm về kho bộ ba, bạn có thể bị cuốn vào một cơn lốc các bài viết về web ngữ nghĩa. Mô hình dữ liệu kho bộ ba hoàn toàn độc lập với web ngữ nghĩa — ví dụ, Datomic [40] là một kho bộ ba không tuyên bố có liên quan gì tới nó. Nhưng vì hai thứ này gắn bó chặt chẽ trong đầu nhiều người, ta nên bàn về chúng một cách vắn tắt.

The semantic web is fundamentally a simple and reasonable idea: websites already publish information as text and pictures for humans to read, so why don't they also publish information as machine-readable data for computers to read? The Resource Description Framework (**RDF**) [41] was intended as a mechanism for different websites to publish data in a consistent format, allowing data from different websites to be automatically combined into a web of data—a kind of internet-wide “database of everything.”

Web ngữ nghĩa về cơ bản là một ý tưởng đơn giản và hợp lý: các trang web vốn đã xuất bản thông tin dưới dạng văn bản và hình ảnh cho con người đọc, vậy tại sao chúng không xuất bản thông tin cả dưới dạng dữ liệu máy-đọc-được cho máy tính đọc? Resource Description Framework (RDF) [41] được dự định làm một cơ chế để các trang web khác nhau xuất bản dữ liệu theo một định dạng nhất quán, cho phép dữ liệu từ các trang web khác nhau được tự động kết hợp thành một mạng dữ liệu — một kiểu “cơ sở dữ liệu của mọi thứ” trên phạm vi toàn internet.

Unfortunately, the semantic web was overhyped in the early 2000s but so far hasn't shown any sign of being realized in practice, which has made many people cynical about it. It has also suffered from a dizzying plethora of acronyms, overly complex standards proposals, and hubris.

Đáng tiếc, web ngữ nghĩa bị thổi phồng quá mức vào đầu những năm 2000 nhưng cho tới nay vẫn chưa cho thấy dấu hiệu nào của việc được hiện thực trên thực tế, điều này đã

khiến nhiều người hoài nghi về nó. Nó cũng phải gánh chịu một mớ từ viết tắt rối mắt, những đề xuất chuẩn quá phức tạp, và sự kiêu ngạo.

However, if you look past those failings, there is also a lot of good work that has come out of the semantic web project. Triples can be a good internal data model for applications, even if you have no interest in publishing RDF data on the semantic web.

Tuy nhiên, nếu bạn nhìn vượt qua những thất bại đó, cũng có rất nhiều công trình tốt đã ra đời từ dự án web ngữ nghĩa. Các bộ ba có thể là một mô hình dữ liệu nội bộ tốt cho ứng dụng, ngay cả khi bạn chẳng có hứng thú gì với việc xuất bản dữ liệu RDF trên web ngữ nghĩa.

The RDF data model — Mô hình dữ liệu RDF

The Turtle language we used in Example 2-7 is a human-readable format for RDF data. Sometimes RDF is also written in an XML format, which does the same thing much more verbosely—see Example 2-8. Turtle/N3 is preferable as it is much easier on the eyes, and tools like Apache Jena [42] can automatically convert between different RDF formats if necessary.

Ngôn ngữ Turtle mà ta dùng trong Ví dụ 2-7 là một định dạng con-người-đọc-được cho dữ liệu RDF. Đôi khi RDF cũng được viết dưới dạng XML, vốn làm cùng một việc nhưng dài dòng hơn nhiều — xem Ví dụ 2-8. Turtle/N3 thì đáng ưa chuộng hơn vì nó dễ chịu hơn nhiều với đôi mắt, và các công cụ như Apache Jena [42] có thể tự động chuyển đổi giữa các định dạng RDF khác nhau nếu cần.

Example 2-8. The data of Example 2-7, expressed using RDF/XML syntax — Ví dụ 2-8. Dữ liệu của Ví dụ 2-7, biểu đạt bằng cú pháp RDF/XML

```
<rdf:RDF xmlns="urn:example:"  
  xmlns:rdf="http://www.w3.org/1999/02/22-rdf-syntax-ns#">  
  <Location rdf:nodeID="idaho">  
    <name>Idaho</name>  
    <type>state</type>  
    <within>  
      <Location rdf:nodeID="usa">  
        <name>United States</name>  
        <type>country</type>  
        <within>  
          <Location rdf:nodeID="namerica">  
            <name>North America</name>  
            <type>continent</type>  
          </Location>  
        </within>  
      </Location>  
    </within>  
  </Location>  
  <Person rdf:nodeID="lucy">  
    <name>Lucy</name>  
    <bornIn rdf:nodeID="idaho"/>  
  </Person>  
</rdf:RDF>
```

RDF has a few quirks due to the fact that it is designed for internet-wide data exchange. The subject, predicate, and object of a triple are often URIs. For example, a predicate might be an **URI** such as `http://my-company.com/**namespace**#within` or `http://my-company.com/namespace#lives_in`, rather than just `WITHIN` or `LIVES_IN`. The reasoning behind this design is that you should be able to combine your data with someone else's data, and if they attach a different meaning to the word `within` or `lives_in`, you won't get a conflict because their predicates are actually `http://other.org/foo#within` and `http://other.org/foo#lives_in`.

RDF có vài điều kỳ quặc do nó được thiết kế cho việc trao đổi dữ liệu trên phạm vi toàn internet. Chủ thể, vị từ, và đối thể của một bộ ba thường là các URI. Ví dụ, một vị từ có thể là một URI như `http://my-company.com/namespace#within` hoặc `http://my-company.com/namespace#lives_in`, thay vì chỉ là `WITHIN` hay `LIVES_IN`. Lý lẽ đằng sau thiết kế này là bạn nên có thể kết hợp dữ liệu của mình với dữ liệu của người khác, và nếu họ gán một ý nghĩa khác cho từ `within` hay `lives_in`, bạn sẽ không gặp xung đột vì các vị từ của họ thực ra là `http://other.org/foo#within` và `http://other.org/foo#lives_in`.

The URL `http://my-company.com/namespace` doesn't necessarily need to resolve to anything—from RDF's point of view, it is simply a namespace. To avoid potential confusion with `http://` URLs, the examples in this section use non-resolvable URIs such as `urn:example:within`. Fortunately, you can just specify this prefix once at the top of the file, and then forget about it.

URL `http://my-company.com/namespace` không nhất thiết phải phân giải tới thứ gì — từ góc nhìn của RDF, nó đơn giản chỉ là một không gian tên. Để tránh nhầm lẫn tiềm tàng với các URL `http://`, các ví dụ trong phần này dùng những URI không-phân-giải-được như `urn:example:within`. May mắn là bạn chỉ cần đặc tả tiền tố này một lần ở đầu file, rồi quên nó đi.

The SPARQL query language — Ngôn ngữ truy vấn SPARQL

SPARQL is a query language for triple-stores using the RDF data model [43]. (It is an acronym for SPARQL Protocol and RDF Query Language, pronounced “sparkle.”) It predates Cypher, and since Cypher's pattern matching is borrowed from SPARQL, they look quite similar [37].

SPARQL là một ngôn ngữ truy vấn cho các kho bộ ba dùng mô hình dữ liệu RDF [43]. (Nó là một từ viết tắt của SPARQL Protocol and RDF Query Language, đọc là “sparkle”.) Nó có trước Cypher, và vì việc khớp khuôn mẫu của Cypher vay mượn từ SPARQL, nên chúng trông khá giống nhau [37].

The same query as before—finding people who have moved from the US to Europe—is even more concise in SPARQL than it is in Cypher (see Example 2-9).

Cùng truy vấn như trước — tìm những người đã chuyển từ Mỹ sang châu Âu — thậm chí còn gọn hơn trong SPARQL so với trong Cypher (xem Ví dụ 2-9).

Example 2-9. The same query as Example 2-4, expressed in SPARQL — Ví dụ 2-9. Cùng truy vấn như Ví dụ 2-4, biểu đạt trong SPARQL

```
PREFIX : <urn:example:>
SELECT ?personName WHERE {
  ?person :name ?personName.
  ?person :bornIn / :within* / :name "United States".
  ?person :livesIn / :within* / :name "Europe".
}
```

The structure is very similar. The following two expressions are equivalent (variables start with a question mark in SPARQL):

Cấu trúc rất giống nhau. Hai biểu thức sau đây là tương đương (các biến bắt đầu bằng dấu hỏi trong SPARQL):

```
(person) -[:BORN_IN]-> () -[:WITHIN*0..]-> (location)    # Cypher
?person :bornIn / :within* ?location.                    # SPARQL
```

Because RDF doesn't distinguish between properties and edges but just uses predicates for both, you can use the same syntax for matching properties. In the following expression, the variable `usa` is bound to any vertex that has a name property whose value is the string "United States":

Vì RDF không phân biệt giữa thuộc tính và cạnh mà chỉ dùng các vị từ cho cả hai, nên bạn có thể dùng cùng cú pháp để khớp các thuộc tính. Trong biểu thức sau, biến `usa` được ràng buộc vào bất kỳ đỉnh nào có thuộc tính name với giá trị là chuỗi "United States":

```
(usa {name:'United States'})    # Cypher
?usa :name "United States".     # SPARQL
```

SPARQL is a nice query language—even if the semantic web never happens, it can be a powerful tool for applications to use internally.

SPARQL là một ngôn ngữ truy vấn hay — kể cả nếu web ngữ nghĩa không bao giờ thành hiện thực, nó vẫn có thể là một công cụ mạnh để ứng dụng dùng nội bộ.

Graph Databases Compared to the Network Model — Cơ sở dữ liệu đồ thị so với mô hình mạng In “Are Document Databases Repeating History?” we discussed how CODASYL and the relational model competed to solve the problem of many-to-many relationships in IMS. At first glance, CODASYL's network model looks similar to the graph model. Are graph databases the second coming of CODASYL in disguise?

Trong phần “Phải chăng cơ sở dữ liệu tài liệu đang lặp lại lịch sử?” ta đã bàn về việc CODASYL và mô hình quan hệ cạnh tranh nhau để giải quyết vấn đề các quan hệ nhiều-nhiều trong IMS. Thoạt nhìn, mô hình mạng của CODASYL trông giống mô hình đồ thị. Liệu cơ sở dữ liệu đồ thị có phải là sự tái sinh của CODASYL dưới một lớp ngụy trang?

No. They differ in several important ways:

Không. Chúng khác nhau ở vài điểm quan trọng:

- In CODASYL, a database had a schema that specified which record type could be nested within which other record type. In a graph database, there is no such restriction: any vertex can have an edge to any other vertex. This gives much greater flexibility for applications to adapt to changing requirements.
- In CODASYL, the only way to reach a particular record was to traverse one of the access paths to it. In a graph database, you can refer directly to any vertex by its unique ID, or you can use an index to find vertices with a particular value.
- In CODASYL, the children of a record were an ordered set, so the database had to maintain that ordering (which had consequences for the storage layout) and applications that inserted new records into the database had to worry about the positions of the new records in these sets. In a graph database, vertices and edges are not ordered (you can only sort the results when making a query).
- In CODASYL, all queries were imperative, difficult to write and easily broken by changes in the schema. In a graph database, you can write your traversal in imperative code if you want to,

but most graph databases also support high-level, declarative query languages such as Cypher or SPARQL.

- Trong CODASYL, một cơ sở dữ liệu có một lược đồ quy định loại bản ghi nào có thể được lồng bên trong loại bản ghi nào khác. Trong một cơ sở dữ liệu đồ thị, không có hạn chế như vậy: bất kỳ đỉnh nào cũng có thể có một cạnh tới bất kỳ đỉnh nào khác. Điều này cho ứng dụng linh hoạt lớn hơn nhiều để thích nghi với các yêu cầu thay đổi.
- Trong CODASYL, cách duy nhất để đạt tới một bản ghi cụ thể là duyệt một trong các đường truy cập tới nó. Trong một cơ sở dữ liệu đồ thị, bạn có thể tham chiếu trực tiếp tới bất kỳ đỉnh nào bằng ID duy nhất của nó, hoặc bạn có thể dùng một chỉ mục để tìm các đỉnh có một giá trị cụ thể.
- Trong CODASYL, các con của một bản ghi là một tập có thứ tự, nên cơ sở dữ liệu phải duy trì thứ tự đó (điều này có hệ quả lên cách bố trí lưu trữ) và các ứng dụng chèn bản ghi mới vào cơ sở dữ liệu phải bận tâm tới vị trí của các bản ghi mới trong các tập đó. Trong một cơ sở dữ liệu đồ thị, các đỉnh và cạnh không có thứ tự (bạn chỉ có thể sắp xếp kết quả khi thực hiện một truy vấn).
- Trong CODASYL, mọi truy vấn đều là mệnh lệnh, khó viết và dễ bị hỏng bởi những thay đổi trong lược đồ. Trong một cơ sở dữ liệu đồ thị, bạn có thể viết phép duyệt của mình bằng mã mệnh lệnh nếu muốn, nhưng hầu hết cơ sở dữ liệu đồ thị cũng hỗ trợ các ngôn ngữ truy vấn khai báo, cao tầng như Cypher hoặc SPARQL.

The Foundation: Datalog — Nền tảng: Datalog

Datalog is a much older language than SPARQL or Cypher, having been studied extensively by academics in the 1980s [44, 45, 46]. It is less well known among software engineers, but it is nevertheless important, because it provides the foundation that later query languages build upon.

Datalog là một ngôn ngữ xưa hơn nhiều so với SPARQL hay Cypher, đã được giới học thuật nghiên cứu sâu rộng trong thập niên 1980 [44, 45, 46]. Nó ít được biết đến hơn trong giới kỹ sư phần mềm, nhưng nó vẫn quan trọng, vì nó cung cấp nền tảng mà các ngôn ngữ truy vấn về sau dựng lên trên đó.

In practice, Datalog is used in a few data systems: for example, it is the query language of Datomic [40], and Cascalog [47] is a Datalog implementation for querying large datasets in Hadoop.

Trên thực tế, Datalog được dùng trong một vài hệ thống dữ liệu: ví dụ, nó là ngôn ngữ truy vấn của Datomic [40], và Cascalog [47] là một hiện thực Datalog để truy vấn các tập dữ liệu lớn trong Hadoop.

Datalog's data model is similar to the triple-store model, generalized a bit. Instead of writing a triple as (subject, predicate, object), we write it as predicate(subject, object). Example 2-10 shows how to write the data from our example in Datalog.

Mô hình dữ liệu của Datalog tương tự mô hình kho bộ ba, được tổng quát hóa đi đôi chút. Thay vì viết một bộ ba dưới dạng (chủ thể, vị từ, đối thể), ta viết nó là vị từ(chủ thể, đối thể). Ví dụ 2-10 cho thấy cách viết dữ liệu từ ví dụ của ta trong Datalog.

Example 2-10. A subset of the data in Figure 2-5, represented as Datalog facts — Ví dụ 2-10. Một tập con của dữ liệu trong Hình 2-5, biểu diễn dưới dạng các sự kiện Datalog

```
name(namerica, 'North America').  
type(namerica, continent).
```

```

name(usa, 'United States').
type(usa, country).
within(usa, namerica).
name(idaho, 'Idaho').
type(idaho, state).
within(idaho, usa).
name(lucy, 'Lucy').
born_in(lucy, idaho).

```

Now that we have defined the data, we can write the same query as before, as shown in Example 2-11. It looks a bit different from the equivalent in Cypher or SPARQL, but don't let that put you off. Datalog is a subset of **Prolog**, which you might have seen before if you've studied computer science.

Giờ khi đã định nghĩa dữ liệu, ta có thể viết cùng truy vấn như trước, như trong Ví dụ 2-11. Nó trông hơi khác so với bản tương đương trong Cypher hay SPARQL, nhưng đừng để điều đó làm bạn nản. Datalog là một tập con của Prolog, mà bạn có thể đã từng thấy nếu từng học khoa học máy tính.

Example 2-11. The same query as Example 2-4, expressed in Datalog — Ví dụ 2-11. Cùng truy vấn như Ví dụ 2-4, biểu đạt trong Datalog

```

within_recursive(Location, Name) :- name(Location, Name).      /* Rule 1 */
within_recursive(Location, Name) :- within(Location, Via),      /* Rule 2 */
                                     within_recursive(Via, Name).
migrated(Name, BornIn, LivingIn) :- name(Person, Name),        /* Rule 3 */
                                     born_in(Person, BornLoc),
                                     within_recursive(BornLoc, BornIn),
                                     lives_in(Person, LivingLoc),
                                     within_recursive(LivingLoc, LivingIn).

?- migrated(Who, 'United States', 'Europe').
/* Who = 'Lucy'. */

```

Cypher and SPARQL jump in right away with SELECT, but Datalog takes a small step at a time. We define rules that tell the database about new predicates: here, we define two new predicates, `within_recursive` and `migrated`. These predicates aren't triples stored in the database, but instead they are derived from data or from other rules. Rules can refer to other rules, just like functions can call other functions or recursively call themselves. Like this, complex queries can be built up a small piece at a time.

Cypher và SPARQL lao thẳng ngay vào với SELECT, nhưng Datalog đi từng bước nhỏ một. Ta định nghĩa các luật cho cơ sở dữ liệu biết về các vị từ mới: ở đây, ta định nghĩa hai vị từ mới, `within_recursive` và `migrated`. Các vị từ này không phải là các bộ ba được lưu trong cơ sở dữ liệu, mà thay vào đó chúng được dẫn xuất từ dữ liệu hoặc từ các luật khác. Các luật có thể tham chiếu tới các luật khác, hệt như các hàm có thể gọi các hàm khác hoặc gọi đệ quy chính chúng. Như vậy, các truy vấn phức tạp có thể được dựng lên từng mẩu nhỏ một.

In rules, words that start with an uppercase letter are variables, and predicates are matched like in Cypher and SPARQL. For example, `name(Location, Name)` matches the triple `name(namerica, 'North America')` with variable bindings `Location = namerica` and `Name = 'North America'`.

Trong các luật, các từ bắt đầu bằng chữ in hoa là biến, và các vị từ được khớp giống như trong Cypher và SPARQL. Ví dụ, `name(Location, Name)` khớp với bộ ba `name(namerica, 'North America')` với các ràng buộc biến `Location = namerica` và `Name = 'North America'`.

A rule applies if the system can find a match for all predicates on the righthand side of the :- operator. When the rule applies, it's as though the lefthand side of the :- was added to the database (with variables replaced by the values they matched).

Một luật được áp dụng nếu hệ thống có thể tìm thấy một sự khớp cho tất cả các vị từ ở vế phải của toán tử :- . Khi luật được áp dụng, thì cứ như thể vế trái của :- đã được thêm vào cơ sở dữ liệu (với các biến được thay bằng các giá trị mà chúng đã khớp).

One possible way of applying the rules is thus:

Như vậy một cách khả dĩ để áp dụng các luật là:

- name(namerica, 'North America') exists in the database, so rule 1 applies. It generates within_recursive(namerica, 'North America').
- within(usa, namerica) exists in the database and the previous step generated within_recursive(namerica, 'North America'), so rule 2 applies. It generates within_recursive(usa, 'North America').
- within(idaho, usa) exists in the database and the previous step generated within_recursive(usa, 'North America'), so rule 2 applies. It generates within_recursive(idaho, 'North America').
 - name(namerica, 'North America') tồn tại trong cơ sở dữ liệu, nên luật 1 áp dụng. Nó sinh ra within_recursive(namerica, 'North America').
 - within(usa, namerica) tồn tại trong cơ sở dữ liệu và bước trước đó đã sinh ra within_recursive(namerica, 'North America'), nên luật 2 áp dụng. Nó sinh ra within_recursive(usa, 'North America').
 - within(idaho, usa) tồn tại trong cơ sở dữ liệu và bước trước đó đã sinh ra within_recursive(usa, 'North America'), nên luật 2 áp dụng. Nó sinh ra within_recursive(idaho, 'North America').

By repeated application of rules 1 and 2, the within_recursive predicate can tell us all the locations in North America (or any other location name) contained in our database. This process is illustrated in Figure 2-6.

Bằng cách áp dụng lặp đi lặp lại luật 1 và 2, vị từ within_recursive có thể cho ta biết tất cả các địa điểm ở Bắc Mỹ (hoặc bất kỳ tên địa điểm nào khác) được chứa trong cơ sở dữ liệu của ta. Quá trình này được minh họa trong Hình 2-6.

Figure 2-6. Determining that Idaho is in North America, using the Datalog rules from Example 2-11. — Hình 2-6. Xác định rằng Idaho nằm ở Bắc Mỹ, dùng các luật Datalog từ Ví dụ 2-11. Now rule 3 can find people who were born in some location BornIn and live in some location LivingIn. By querying with BornIn = 'United States' and LivingIn = 'Europe', and leaving the person as a variable Who, we ask the Datalog system to find out which values can appear for the variable Who. So, finally we get the same answer as in the earlier Cypher and SPARQL queries.

Giờ luật 3 có thể tìm những người đã sinh ra ở một địa điểm BornIn nào đó và đang sống ở một địa điểm LivingIn nào đó. Bằng cách truy vấn với BornIn = 'United States' và LivingIn = 'Europe', và để person làm một biến Who, ta yêu cầu hệ thống Datalog tìm ra những giá trị nào có thể xuất hiện cho biến Who. Vậy là, cuối cùng ta nhận được cùng câu trả lời như trong các truy vấn Cypher và SPARQL trước đó.

The Datalog approach requires a different kind of thinking to the other query languages discussed in this chapter, but it's a very powerful approach, because rules can be combined and reused in different queries. It's less convenient for simple one-off queries, but it can cope better if your data is complex.

Cách tiếp cận Datalog đòi hỏi một kiểu tư duy khác so với các ngôn ngữ truy vấn khác được bàn trong chương này, nhưng nó là một cách tiếp cận rất mạnh, vì các luật có thể được kết hợp và tái sử dụng trong các truy vấn khác nhau. Nó kém tiện hơn cho các truy vấn đơn giản dùng một lần, nhưng nó có thể xử lý tốt hơn nếu dữ liệu của bạn phức tạp.

Summary — Tóm tắt

Data models are a huge subject, and in this chapter we have taken a quick look at a broad variety of different models. We didn't have space to go into all the details of each model, but hopefully the overview has been enough to whet your appetite to find out more about the model that best fits your application's requirements.

Mô hình dữ liệu là một chủ đề khổng lồ, và trong chương này ta đã xem lướt qua một loạt các mô hình khác nhau. Ta không có chỗ để đi vào mọi chi tiết của từng mô hình, nhưng hy vọng phần tổng quan đã đủ để khơi gợi hứng thú tìm hiểu thêm về mô hình phù hợp nhất với các yêu cầu của ứng dụng của bạn.

Historically, data started out being represented as one big tree (the hierarchical model), but that wasn't good for representing many-to-many relationships, so the relational model was invented to solve that problem. More recently, developers found that some applications don't fit well in the relational model either. New nonrelational “NoSQL” datastores have diverged in two main directions:

Về mặt lịch sử, dữ liệu khởi đầu được biểu diễn dưới dạng một cây lớn duy nhất (mô hình phân cấp), nhưng điều đó không tốt cho việc biểu diễn các quan hệ nhiều-nhiều, nên mô hình quan hệ được phát minh để giải quyết vấn đề đó. Gần đây hơn, các lập trình viên nhận thấy một số ứng dụng cũng không khớp tốt với mô hình quan hệ. Các kho dữ liệu phi quan hệ “NoSQL” mới đã rẽ theo hai hướng chính:

- Document databases target use cases where data comes in self-contained documents and relationships between one document and another are rare.
- Graph databases go in the opposite direction, targeting use cases where anything is potentially related to everything.
 - Cơ sở dữ liệu tài liệu nhắm tới các tình huống sử dụng mà dữ liệu đến dưới dạng các tài liệu tự chứa và các quan hệ giữa tài liệu này với tài liệu khác là hiếm.
 - Cơ sở dữ liệu đồ thị đi theo hướng ngược lại, nhắm tới các tình huống sử dụng mà bất cứ thứ gì cũng có thể liên quan tới mọi thứ.

All three models (document, relational, and graph) are widely used today, and each is good in its respective domain. One model can be emulated in terms of another model—for example, graph data can be represented in a relational database—but the result is often awkward. That's why we have different systems for different purposes, not a single one-size-fits-all solution.

Cả ba mô hình (tài liệu, quan hệ, và đồ thị) đều được dùng rộng rãi ngày nay, và mỗi mô hình đều tốt trong lĩnh vực tương ứng của nó. Một mô hình có thể được mô phỏng bằng một mô hình khác — ví dụ, dữ liệu đồ thị có thể được biểu diễn trong một cơ sở dữ liệu quan hệ — nhưng kết quả thường gượng gạo. Đó là lý do ta có các hệ thống khác nhau cho các mục đích khác nhau, chứ không phải một giải pháp một-cỡ-vừa-cho-tất-cả duy nhất.

One thing that document and graph databases have in common is that they typically don't enforce a schema for the data they store, which can make it easier to adapt applications to changing requirements. However, your application most likely still assumes that data has a certain structure; it's just a question of whether the schema is explicit (enforced on write) or implicit (handled on read).

Một điểm chung của cơ sở dữ liệu tài liệu và cơ sở dữ liệu đồ thị là chúng thường không cưỡng chế một lược đồ cho dữ liệu mà chúng lưu trữ, điều này có thể giúp việc thích nghi ứng dụng với các yêu cầu thay đổi dễ dàng hơn. Tuy nhiên, ứng dụng của bạn nhiều khả năng vẫn giả định rằng dữ liệu có một cấu trúc nào đó; vấn đề chỉ là lược đồ đó là tường minh (cưỡng chế lúc ghi) hay ngầm định (xử lý lúc đọc).

Each data model comes with its own query language or framework, and we discussed several examples: SQL, MapReduce, MongoDB's aggregation pipeline, Cypher, SPARQL, and Datalog. We also touched on CSS and XSL/XPath, which aren't database query languages but have interesting parallels.

Mỗi mô hình dữ liệu đi kèm ngôn ngữ truy vấn hoặc framework riêng của nó, và ta đã bàn về vài ví dụ: SQL, MapReduce, aggregation pipeline của MongoDB, Cypher, SPARQL, và Datalog. Ta cũng đã chạm tới CSS và XSL/XPath, vốn không phải là các ngôn ngữ truy vấn cơ sở dữ liệu nhưng có những điểm tương đồng thú vị.

Although we have covered a lot of ground, there are still many data models left unmentioned. To give just a few brief examples:

Mặc dù ta đã đề cập rất nhiều, vẫn còn nhiều mô hình dữ liệu chưa được nhắc tới. Chỉ nêu vài ví dụ vắn tắt:

- Researchers working with genome data often need to perform sequence-similarity searches, which means taking one very long string (representing a DNA molecule) and matching it against a large database of strings that are similar, but not identical. None of the databases described here can handle this kind of usage, which is why researchers have written specialized genome database software like GenBank [48].
- Particle physicists have been doing Big Data-style large-scale data analysis for decades, and projects like the Large Hadron Collider (LHC) now work with hundreds of petabytes! At such a scale custom solutions are required to stop the hardware cost from spiraling out of control [49].
- **Full-text search** is arguably a kind of data model that is frequently used alongside databases. Information retrieval is a large specialist subject that we won't cover in great detail in this book, but we'll touch on search indexes in Chapter 3 and Part III.
 - Các nhà nghiên cứu làm việc với dữ liệu hệ gen thường cần thực hiện các phép tìm kiếm độ tương tự chuỗi (sequence-similarity), nghĩa là lấy một chuỗi rất dài (biểu diễn một phân tử DNA) và đối khớp nó với một cơ sở dữ liệu lớn các chuỗi tương tự, nhưng không hoàn toàn giống. Không có cơ sở dữ liệu nào được mô tả ở đây có thể xử lý kiểu sử dụng này, đó là lý do các nhà nghiên cứu đã viết phần mềm cơ sở dữ liệu hệ gen chuyên biệt như GenBank [48].
 - Các nhà vật lý hạt đã làm phân tích dữ liệu quy mô lớn kiểu Big Data trong nhiều thập kỷ, và các dự án như Large Hadron Collider (LHC) giờ làm việc với hàng trăm petabyte! Ở quy mô như vậy cần các giải pháp tùy biến để ngăn chi phí phần cứng leo thang mất kiểm soát [49].
 - Tìm kiếm toàn văn được cho là một loại mô hình dữ liệu thường được dùng song song với cơ sở dữ liệu. Truy hồi thông tin là một chủ đề chuyên môn lớn mà ta sẽ không trình bày quá chi tiết trong cuốn sách này, nhưng ta sẽ chạm tới các chỉ mục tìm kiếm

ở Chương 3 và Phần III.

We have to leave it there for now. In the next chapter we will discuss some of the trade-offs that come into play when implementing the data models described in this chapter.

Ta phải dừng ở đây lúc này. Ở chương tiếp theo ta sẽ bàn về một số đánh đổi nảy sinh khi hiện thực các mô hình dữ liệu được mô tả trong chương này.

Footnotes A term borrowed from electronics. Every electric circuit has a certain impedance (resistance to alternating current) on its inputs and outputs. When you connect one circuit's output to another one's input, the power transfer across the connection is maximized if the output and input impedances of the two circuits match. An impedance mismatch can lead to signal reflections and other troubles.

Literature on the relational model distinguishes several different normal forms, but the distinctions are of little practical interest. As a rule of thumb, if you're duplicating values that could be stored in just one place, the schema is not normalized.

At the time of writing, joins are supported in RethinkDB, not supported in MongoDB, and only supported in predeclared views in CouchDB.

Foreign key constraints allow you to restrict modifications, but such constraints are not required by the relational model. Even with constraints, joins on foreign keys are performed at query time, whereas in CODASYL, the join was effectively done at insert time.

Codd's original description of the relational model [1] actually allowed something quite similar to JSON documents within a relational schema. He called it nonsimple domains. The idea was that a value in a row doesn't have to just be a primitive datatype like a number or a string, but could also be a nested relation (table)—so you can have an arbitrarily nested tree structure as a value, much like the JSON or XML support that was added to SQL over 30 years later.

IMS and CODASYL both used imperative query APIs. Applications typically used COBOL code to iterate over records in the database, one record at a time [2, 16].

Technically, Datomic uses 5-tuples rather than triples; the two additional fields are metadata for versioning.

Datomic and Cascalog use a Clojure S-expression syntax for Datalog. In the following examples we use a Prolog syntax, which is a little easier to read, but this makes no functional difference.

References [1] Edgar F. Codd: "A Relational Model of Data for Large Shared Data Banks," Communications of the ACM, volume 13, number 6, pages 377–387, June 1970. doi:10.1145/362384.362685

[2] Michael Stonebraker and Joseph M. Hellerstein: "What Goes Around Comes Around," in Readings in Database Systems, 4th edition, MIT Press, pages 2–41, 2005. ISBN: 978-0-262-69314-1

[3] Pramod J. Sadalage and Martin Fowler: NoSQL Distilled. Addison-Wesley, August 2012. ISBN: 978-0-321-82662-6

[4] Eric Evans: "NoSQL: What's in a Name?," blog.sym-link.com, October 30, 2009.

[5] James Phillips: "Surprises in Our NoSQL Adoption Survey," blog.couchbase.com, February 8, 2012.

[6] Michael Wagner: SQL/XML:2006 – Evaluierung der Standardkonformität ausgewählter Datenbanksysteme. Diplomica Verlag, Hamburg, 2010. ISBN: 978-3-836-64609-3

[7] "XML Data in SQL Server," SQL Server 2012 documentation, technet.microsoft.com, 2013.

- [8] “PostgreSQL 9.3.1 Documentation,” The PostgreSQL Global Development Group, 2013.
- [9] “The MongoDB 2.4 Manual,” MongoDB, Inc., 2013.
- [10] “RethinkDB 1.11 Documentation,” rethinkdb.com, 2013.
- [11] “Apache CouchDB 1.6 Documentation,” docs.couchdb.org, 2014.
- [12] Lin Qiao, Kapil Surlaker, Shirshanka Das, et al.: “On Brewing Fresh Espresso: LinkedIn’s Distributed Data Serving Platform,” at ACM International Conference on Management of Data (SIGMOD), June 2013.
- [13] Rick Long, Mark Harrington, Robert Hain, and Geoff Nicholls: IMS Primer. IBM Redbook SG24-5352-00, IBM International Technical Support Organization, January 2000.
- [14] Stephen D. Bartlett: “IBM’s IMS—Myths, Realities, and Opportunities,” The Clipper Group Navigator, TCG2013015LI, July 2013.
- [15] Sarah Mei: “Why You Should Never Use MongoDB,” sarahmei.com, November 11, 2013.
- [16] J. S. Knowles and D. M. R. Bell: “The CODASYL Model,” in Databases—Role and Structure: An Advanced Course, edited by P. M. Stocker, P. M. D. Gray, and M. P. Atkinson, pages 19–56, Cambridge University Press, 1984. ISBN: 978-0-521-25430-4
- [17] Charles W. Bachman: “The Programmer as Navigator,” Communications of the ACM, volume 16, number 11, pages 653–658, November 1973. doi:10.1145/355611.362534
- [18] Joseph M. Hellerstein, Michael Stonebraker, and James Hamilton: “Architecture of a Database System,” Foundations and Trends in Databases, volume 1, number 2, pages 141–259, November 2007. doi:10.1561/19000000002
- [19] Sandeep Parikh and Kelly Stirman: “Schema Design for Time Series Data in MongoDB,” blog.mongodb.org, October 30, 2013.
- [20] Martin Fowler: “Schemaless Data Structures,” martinowler.com, January 7, 2013.
- [21] Amr Awadallah: “Schema-on-Read vs. Schema-on-Write,” at Berkeley EECS RAD Lab Retreat, Santa Cruz, CA, May 2009.
- [22] Martin Odersky: “The Trouble with Types,” at Strange Loop, September 2013.
- [23] Conrad Irwin: “MongoDB—Confessions of a PostgreSQL Lover,” at HTML5DevConf, October 2013.
- [24] “Percona Toolkit Documentation: pt-online-schema-change,” Percona Ireland Ltd., 2013.
- [25] Rany Keddo, Tobias Bielohlawek, and Tobias Schmidt: “Large Hadron Migrator,” SoundCloud, 2013.
- [26] Shlomi Noach: “gh-ost: GitHub’s Online Schema Migration Tool for MySQL,” githubengineering.com, August 1, 2016.
- [27] James C. Corbett, Jeffrey Dean, Michael Epstein, et al.: “Spanner: Google’s Globally-Distributed Database,” at 10th USENIX Symposium on Operating System Design and Implementation (OSDI), October 2012.
- [28] Donald K. Burleson: “Reduce I/O with Oracle Cluster Tables,” dba-oracle.com.
- [29] Fay Chang, Jeffrey Dean, Sanjay Ghemawat, et al.: “Bigtable: A Distributed Storage System for Structured Data,” at 7th USENIX Symposium on Operating System Design and Implementation (OSDI), November 2006.

- [30] Bobbie J. Cochrane and Kathy A. McKnight: “DB2 JSON Capabilities, Part 1: Introduction to DB2 JSON,” IBM developerWorks, June 20, 2013.
- [31] Herb Sutter: “The Free Lunch Is Over: A Fundamental Turn Toward Concurrency in Software,” Dr. Dobbs’s Journal, volume 30, number 3, pages 202-210, March 2005.
- [32] Joseph M. Hellerstein: “The Declarative Imperative: Experiences and Conjectures in Distributed Logic,” Electrical Engineering and Computer Sciences, University of California at Berkeley, Tech report UCB/EECS-2010-90, June 2010.
- [33] Jeffrey Dean and Sanjay Ghemawat: “MapReduce: Simplified Data Processing on Large Clusters,” at 6th USENIX Symposium on Operating System Design and Implementation (OSDI), December 2004.
- [34] Craig Kerstiens: “JavaScript in Your Postgres,” blog.heroku.com, June 5, 2013.
- [35] Nathan Bronson, Zach Amsden, George Cabrera, et al.: “TAO: Facebook’s Distributed Data Store for the Social Graph,” at USENIX Annual Technical Conference (USENIX ATC), June 2013.
- [36] “Apache TinkerPop3.2.3 Documentation,” tinkerpop.apache.org, October 2016.
- [37] “The Neo4j Manual v2.0.0,” Neo Technology, 2013.
- [38] Emil Eifrem: Twitter correspondence, January 3, 2014.
- [39] David Beckett and Tim Berners-Lee: “Turtle – Terse RDF Triple Language,” W3C Team Submission, March 28, 2011.
- [40] “Datatomic Development Resources,” Metadata Partners, LLC, 2013.
- [41] W3C RDF Working Group: “Resource Description Framework (RDF),” w3.org, 10 February 2004.
- [42] “Apache Jena,” Apache Software Foundation.
- [43] Steve Harris, Andy Seaborne, and Eric Prud’hommeaux: “SPARQL 1.1 Query Language,” W3C Recommendation, March 2013.
- [44] Todd J. Green, Shan Shan Huang, Boon Thau Loo, and Wenchao Zhou: “Datalog and Recursive Query Processing,” Foundations and Trends in Databases, volume 5, number 2, pages 105–195, November 2013. doi:10.1561/19000000017
- [45] Stefano Ceri, Georg Gottlob, and Letizia Tanca: “What You Always Wanted to Know About Datalog (And Never Dared to Ask),” IEEE Transactions on Knowledge and Data Engineering, volume 1, number 1, pages 146–166, March 1989. doi:10.1109/69.43410
- [46] Serge Abiteboul, Richard Hull, and Victor Vianu: Foundations of Databases. Addison-Wesley, 1995. ISBN: 978-0-201-53771-0, available online at webdam.inria.fr/Alice
- [47] Nathan Marz: “Cascalog,” cascalog.org.
- [48] Dennis A. Benson, Ilene Karsch-Mizrachi, David J. Lipman, et al.: “GenBank,” Nucleic Acids Research, volume 36, Database issue, pages D25–D30, December 2007. doi:10.1093/nar/gkm929
- [49] Fons Rademakers: “ROOT for Big Data Analysis,” at Workshop on the Future of Big Data Management, London, UK, June 2013.

Chapter 3. Storage and Retrieval —

Chương 3. Lưu trữ và Truy xuất

Wer Ordnung hält, ist nur zu faul zum Suchen.

Wer Ordnung hält, ist nur zu faul zum Suchen.

(If you keep things tidily ordered, you're just too lazy to go searching.)

(Nếu bạn giữ mọi thứ ngăn nắp gọn gàng, thì chẳng qua bạn quá lười đi tìm mà thôi.)

German proverb

Ngạn ngữ Đức

On the most fundamental level, a **database** needs to do two things: when you give it some data, it should store the data, and when you ask it again later, it should give the data back to you.

Ở mức căn bản nhất, một cơ sở dữ liệu cần làm hai việc: khi ta đưa cho nó một ít dữ liệu, nó phải lưu dữ liệu lại, và khi ta hỏi lại sau này, nó phải trả dữ liệu đó về cho ta.

In Chapter 2 we discussed data models and query languages—i.e., the format in which you (the application developer) give the database your data, and the mechanism by which you can ask for it again later. In this chapter we discuss the same from the database's point of view: how we can store the data that we're given, and how we can find it again when we're asked for it.

Ở Chương 2, ta đã bàn về mô hình dữ liệu và ngôn ngữ truy vấn — tức là định dạng mà ta (lập trình viên ứng dụng) dùng để đưa dữ liệu cho cơ sở dữ liệu, và cơ chế mà ta dùng để hỏi lại dữ liệu sau này. Trong chương này, ta sẽ bàn về cùng vấn đề đó nhưng từ góc nhìn của cơ sở dữ liệu: làm thế nào để lưu dữ liệu được giao cho ta, và làm thế nào để tìm lại nó khi được yêu cầu.

Why should you, as an application developer, care how the database handles storage and retrieval internally? You're probably not going to implement your own storage **engine** from scratch, but you do need to select a storage engine that is appropriate for your application, from the many that are available. In order to tune a storage engine to perform well on your kind of workload, you need to have a rough idea of what the storage engine is doing under the hood.

Tại sao ta, với tư cách lập trình viên ứng dụng, lại phải bận tâm cách cơ sở dữ liệu xử lý việc lưu trữ và truy xuất ở bên trong? Có lẽ ta sẽ không tự hiện thực một engine lưu trữ từ con số không, nhưng ta cần chọn một engine lưu trữ phù hợp với ứng dụng của mình, trong vô số lựa chọn sẵn có. Để tinh chỉnh một engine lưu trữ hoạt động tốt với kiểu khối lượng công việc của mình, ta cần có hình dung sơ bộ về những gì engine lưu trữ đang làm bên dưới lớp vỏ.

In particular, there is a big difference between storage engines that are optimized for transactional workloads and those that are optimized for analytics. We will explore that distinction later in “**Transaction Processing** or Analytics?”, and in “**Column-Oriented Storage**” we’ll discuss a family of storage engines that is optimized for analytics.

Đặc biệt, có một khác biệt lớn giữa các engine lưu trữ được tối ưu cho khối lượng công việc giao dịch và những engine được tối ưu cho phân tích. Ta sẽ khám phá sự phân biệt này ở phần “Xử lý giao dịch hay Phân tích?”, và ở phần “Lưu trữ theo cột” ta sẽ bàn về một họ engine lưu trữ được tối ưu cho phân tích.

However, first we’ll start this chapter by talking about storage engines that are used in the kinds of databases that you’re probably familiar with: traditional relational databases, and also most so-called **NoSQL** databases. We will examine two families of storage engines: log-structured storage engines, and page-oriented storage engines such as B-trees.

Tuy nhiên, trước hết ta sẽ mở đầu chương này bằng việc nói về các engine lưu trữ được dùng trong những loại cơ sở dữ liệu mà có lẽ ta đã quen thuộc: các cơ sở dữ liệu quan hệ truyền thống, và cả phần lớn các cơ sở dữ liệu được gọi là NoSQL. Ta sẽ xem xét hai họ engine lưu trữ: engine lưu trữ cấu trúc theo log (log-structured), và engine lưu trữ hướng trang (page-oriented) như B-tree.

Data Structures That Power Your Database — Các cấu trúc dữ liệu tạo sức mạnh cho cơ sở dữ liệu của bạn

Consider the world's simplest database, implemented as two Bash functions:

Hãy xét cơ sở dữ liệu đơn giản nhất thế giới, được hiện thực bằng hai hàm Bash:

```
#!/bin/bash
db_set () {
    echo "$1,$2" >> database
}
db_get () {
    grep "^$1," database | sed -e "s/^$1,/" | tail -n 1
}
```

These two functions implement a key-value store. You can call `db_set key value`, which will store key and value in the database. The key and value can be (almost) anything you like—for example, the value could be a **JSON document**. You can then call `db_get key`, which looks up the most recent value associated with that particular key and returns it.

Hai hàm này hiện thực một kho khóa-giá trị (key-value store). Ta có thể gọi `db_set key value`, lệnh này sẽ lưu key và value vào cơ sở dữ liệu. Khóa và giá trị có thể (gần như) là bất cứ thứ gì ta muốn — ví dụ, giá trị có thể là một tài liệu JSON. Sau đó ta có thể gọi `db_get key`, lệnh này tra cứu giá trị gần đây nhất gắn với khóa cụ thể đó và trả về nó.

And it works:

Và nó hoạt động:

```
$ db_set 123456 '{"name":"London","attractions":["Big Ben","London Eye"]}'
$ db_set 42 '{"name":"San Francisco","attractions":["Golden Gate Bridge"]}'
$ db_get 42
{"name":"San Francisco","attractions":["Golden Gate Bridge"]}
```

The underlying storage format is very simple: a text file where each line contains a key-value pair, separated by a comma (roughly like a CSV file, ignoring escaping issues). Every call to `db_set` appends to the end of the file, so if you update a key several times, the old versions of the value are not overwritten—you need to look at the last occurrence of a key in a file to find the latest value (hence the `tail -n 1` in `db_get`):

Định dạng lưu trữ nền tảng rất đơn giản: một tệp văn bản trong đó mỗi dòng chứa một cặp khóa-giá trị, ngăn cách bằng dấu phẩy (đại khái giống một tệp CSV, bỏ qua các vấn đề

về escape). Mỗi lần gọi `db_set` đều ghi nối thêm (append) vào cuối tệp, nên nếu ta cập nhật một khóa nhiều lần, các phiên bản cũ của giá trị sẽ không bị ghi đè — ta cần xem lần xuất hiện cuối cùng của một khóa trong tệp để tìm giá trị mới nhất (do đó mới có tail -n 1 trong `db_get`):

```
$ db_set 42 '{"name":"San Francisco","attractions":["Exploratorium"]}'
$ db_get 42
{"name":"San Francisco","attractions":["Exploratorium"]}
$ cat database
123456,{"name":"London","attractions":["Big Ben","London Eye"]}
42,{"name":"San Francisco","attractions":["Golden Gate Bridge"]}
42,{"name":"San Francisco","attractions":["Exploratorium"]}
```

Our `db_set` function actually has pretty good performance for something that is so simple, because appending to a file is generally very efficient. Similarly to what `db_set` does, many databases internally use a log, which is an append-only data file. Real databases have more issues to deal with (such as **concurrency** control, reclaiming disk space so that the log doesn't grow forever, and handling errors and partially written records), but the basic principle is the same. Logs are incredibly useful, and we will encounter them several times in the rest of this book.

Hàm `db_set` của ta thực ra có hiệu năng khá tốt so với một thứ đơn giản đến vậy, bởi việc ghi nối vào một tệp nhìn chung rất hiệu quả. Tương tự như những gì `db_set` làm, nhiều cơ sở dữ liệu bên trong cũng dùng một log, tức một tệp dữ liệu chỉ-ghi-nối (append-only). Cơ sở dữ liệu thực tế còn phải xử lý nhiều vấn đề khác (chẳng hạn kiểm soát tương tranh, thu hồi không gian đĩa để log không phình ra mãi mãi, và xử lý lỗi cũng như các bản ghi bị ghi dở), nhưng nguyên lý cơ bản vẫn vậy. Log cực kỳ hữu ích, và ta sẽ gặp lại chúng nhiều lần trong phần còn lại của cuốn sách.

Note — Ghi chú The word log is often used to refer to application logs, where an application outputs text that describes what's happening. In this book, log is used in the more general sense: an append-only sequence of records. It doesn't have to be human-readable; it might be binary and intended only for other programs to read.

Từ log thường được dùng để chỉ log ứng dụng, nơi một ứng dụng xuất ra văn bản mô tả những gì đang diễn ra. Trong cuốn sách này, log được dùng theo nghĩa tổng quát hơn: một chuỗi bản ghi chỉ-ghi-nối. Nó không nhất thiết phải đọc được bởi con người; nó có thể ở dạng nhị phân và chỉ nhằm cho các chương trình khác đọc.

On the other hand, our `db_get` function has terrible performance if you have a large number of records in your database. Every time you want to look up a key, `db_get` has to scan the entire database file from beginning to end, looking for occurrences of the key. In algorithmic terms, the cost of a lookup is $O(n)$: if you double the number of records n in your database, a lookup takes twice as long. That's not good.

Mặt khác, hàm `db_get` của ta lại có hiệu năng tệ hại nếu ta có một số lượng lớn bản ghi trong cơ sở dữ liệu. Mỗi lần muốn tra cứu một khóa, `db_get` phải quét toàn bộ tệp cơ sở dữ liệu từ đầu đến cuối, tìm các lần xuất hiện của khóa đó. Xét về thuật toán, chi phí của một lần tra cứu là $O(n)$: nếu ta tăng gấp đôi số bản ghi n trong cơ sở dữ liệu, một lần tra cứu sẽ mất gấp đôi thời gian. Như vậy là không tốt.

In order to efficiently find the value for a particular key in the database, we need a different data structure: an **index**. In this chapter we will look at a range of indexing structures and see how they compare; the general idea behind them is to keep some additional metadata on the side, which acts

as a signpost and helps you to locate the data you want. If you want to search the same data in several different ways, you may need several different indexes on different parts of the data.

Để tìm hiệu quả giá trị ứng với một khóa cụ thể trong cơ sở dữ liệu, ta cần một cấu trúc dữ liệu khác: một chỉ mục (index). Trong chương này ta sẽ xem xét một loạt các cấu trúc lập chỉ mục và so sánh chúng; ý tưởng chung đằng sau chúng là giữ thêm một số siêu dữ liệu (metadata) bên cạnh, đóng vai trò như một biển chỉ đường và giúp ta định vị dữ liệu mình muốn. Nếu ta muốn tìm cùng một dữ liệu theo nhiều cách khác nhau, ta có thể cần nhiều chỉ mục khác nhau trên những phần khác nhau của dữ liệu.

An index is an additional structure that is derived from the primary data. Many databases allow you to add and remove indexes, and this doesn't affect the contents of the database; it only affects the performance of queries. Maintaining additional structures incurs overhead, especially on writes. For writes, it's hard to beat the performance of simply appending to a file, because that's the simplest possible write operation. Any kind of index usually slows down writes, because the index also needs to be updated every time data is written.

Một chỉ mục là một cấu trúc bổ sung được dẫn xuất từ dữ liệu gốc. Nhiều cơ sở dữ liệu cho phép ta thêm và xóa chỉ mục, và việc này không ảnh hưởng tới nội dung của cơ sở dữ liệu; nó chỉ ảnh hưởng tới hiệu năng của các truy vấn. Việc duy trì các cấu trúc bổ sung tạo ra phụ phí, đặc biệt là khi ghi. Với việc ghi, khó mà vượt qua được hiệu năng của việc chỉ đơn giản ghi nối vào một tệp, bởi đó là thao tác ghi đơn giản nhất có thể. Bất kỳ loại chỉ mục nào cũng thường làm chậm việc ghi, bởi chỉ mục cũng cần được cập nhật mỗi lần dữ liệu được ghi.

This is an important trade-off in storage systems: well-chosen indexes speed up read queries, but every index slows down writes. For this reason, databases don't usually index everything by default, but require you—the application developer or database administrator—to choose indexes manually, using your knowledge of the application's typical query patterns. You can then choose the indexes that give your application the greatest benefit, without introducing more overhead than necessary.

Đây là một đánh đổi quan trọng trong các hệ thống lưu trữ: các chỉ mục được chọn khéo léo sẽ tăng tốc các truy vấn đọc, nhưng mỗi chỉ mục lại làm chậm việc ghi. Vì lý do này, các cơ sở dữ liệu thường không lập chỉ mục mọi thứ một cách mặc định, mà yêu cầu ta — lập trình viên ứng dụng hoặc quản trị viên cơ sở dữ liệu — chọn các chỉ mục một cách thủ công, dựa trên hiểu biết của ta về các mẫu truy vấn điển hình của ứng dụng. Khi đó ta có thể chọn những chỉ mục mang lại lợi ích lớn nhất cho ứng dụng, mà không tạo ra nhiều phụ phí hơn mức cần thiết.

Hash Indexes — Chỉ mục băm

Let's start with indexes for key-value data. This is not the only kind of data you can index, but it's very common, and it's a useful **building block** for more complex indexes.

Hãy bắt đầu với các chỉ mục cho dữ liệu khóa-giá trị. Đây không phải loại dữ liệu duy nhất mà ta có thể lập chỉ mục, nhưng nó rất phổ biến, và là một khối tiêu chuẩn hữu ích cho các chỉ mục phức tạp hơn.

Key-value stores are quite similar to the dictionary type that you can find in most programming languages, and which is usually implemented as a hash map (hash **table**). Hash maps are described in many algorithms textbooks [1, 2], so we won't go into detail of how they work here. Since we already have hash maps for our in-memory data structures, why not use them to index our data on disk?

Các kho khóa-giá trị khá giống kiểu từ điển (dictionary) mà ta tìm thấy trong hầu hết các ngôn ngữ lập trình, và thường được hiện thực dưới dạng một bản đồ băm (hash map, hay bảng băm). Bản đồ băm được mô tả trong nhiều sách giáo khoa về thuật toán [1, 2], nên ta sẽ không đi sâu vào cách chúng hoạt động ở đây. Vì ta đã có sẵn bản đồ băm cho các cấu trúc dữ liệu trong bộ nhớ, tại sao không dùng chúng để lập chỉ mục dữ liệu trên đĩa?

Let's say our data storage consists only of appending to a file, as in the preceding example. Then the simplest possible indexing strategy is this: keep an in-memory hash map where every key is mapped to a byte offset in the data file—the location at which the value can be found, as illustrated in Figure 3-1. Whenever you append a new key-value pair to the file, you also update the hash map to reflect the offset of the data you just wrote (this works both for inserting new keys and for updating existing keys). When you want to look up a value, use the hash map to find the offset in the data file, seek to that location, and read the value.

Giả sử kho lưu trữ dữ liệu của ta chỉ gồm việc ghi nối vào một tệp, như trong ví dụ trước. Thế thì chiến lược lập chỉ mục đơn giản nhất có thể là: giữ một bản đồ băm trong bộ nhớ, trong đó mỗi khóa được ánh xạ tới một độ lệch byte (byte offset) trong tệp dữ liệu — vị trí mà giá trị có thể được tìm thấy, như minh họa ở Hình 3-1. Mỗi khi ghi nối một cặp khóa-giá trị mới vào tệp, ta cũng cập nhật bản đồ băm để phản ánh độ lệch của dữ liệu vừa ghi (điều này áp dụng được cả khi chèn khóa mới lẫn khi cập nhật khóa hiện có). Khi muốn tra cứu một giá trị, ta dùng bản đồ băm để tìm độ lệch trong tệp dữ liệu, nhảy (seek) tới vị trí đó, và đọc giá trị.

Figure 3-1. Storing a log of key-value pairs in a CSV-like format, indexed with an in-memory hash map. — Hình 3-1. Lưu trữ một log gồm các cặp khóa-giá trị ở định dạng giống CSV, được lập chỉ mục bằng một bản đồ băm trong bộ nhớ. This may sound simplistic, but it is a viable approach. In fact, this is essentially what Bitcask (the default storage engine in Riak) does [3]. Bitcask offers high-performance reads and writes, **subject** to the requirement that all the keys fit in the available RAM, since the hash map is kept completely in memory. The values can use more space than there is available memory, since they can be loaded from disk with just one disk seek. If that part of the data file is already in the filesystem **cache**, a read doesn't require any disk I/O at all.

Nghe có vẻ giản đơn quá mức, nhưng đây là một cách tiếp cận khả thi. Thực tế, đây về cơ bản là những gì Bitcask (engine lưu trữ mặc định trong Riak) làm [3]. Bitcask đem lại khả năng đọc và ghi hiệu năng cao, với điều kiện là toàn bộ các khóa phải vừa với dung lượng RAM khả dụng, vì bản đồ băm được giữ hoàn toàn trong bộ nhớ. Các giá trị có thể chiếm nhiều không gian hơn lượng bộ nhớ khả dụng, vì chúng có thể được nạp từ đĩa chỉ với một lần seek đĩa. Nếu phần đó của tệp dữ liệu đã nằm sẵn trong bộ nhớ đệm của hệ thống tệp, thì một lần đọc không cần bất kỳ thao tác I/O đĩa nào cả.

A storage engine like Bitcask is well suited to situations where the value for each key is updated frequently. For example, the key might be the URL of a cat video, and the value might be the number of times it has been played (incremented every time someone hits the play button). In this kind of workload, there are a lot of writes, but there are not too many distinct keys—you have a large number of writes per key, but it's feasible to keep all keys in memory.

Một engine lưu trữ như Bitcask rất phù hợp với những tình huống mà giá trị ứng với mỗi khóa được cập nhật thường xuyên. Ví dụ, khóa có thể là URL của một video về mèo, và giá trị có thể là số lần video đó đã được phát (tăng thêm mỗi khi ai đó nhấn nút phát). Trong loại khối lượng công việc này, có rất nhiều lần ghi, nhưng không có quá nhiều khóa khác biệt — ta có một số lượng lớn lần ghi trên mỗi khóa, nhưng việc giữ toàn bộ các khóa trong bộ nhớ là khả thi.

As described so far, we only ever append to a file—so how do we avoid eventually running out of disk space? A good solution is to break the log into segments of a certain size by closing a segment file when it reaches a certain size, and making subsequent writes to a new segment file. We can then perform compaction on these segments, as illustrated in Figure 3-2. Compaction means throwing away duplicate keys in the log, and keeping only the most recent update for each key.

Như đã mô tả từ đầu đến giờ, ta chỉ luôn ghi nối vào một tệp — vậy làm thế nào để tránh việc cuối cùng cạn kiệt không gian đĩa? Một giải pháp tốt là chia log thành các đoạn (segment) có kích thước nhất định, bằng cách đóng một tệp đoạn khi nó đạt một kích thước nhất định, và ghi các lần ghi tiếp theo vào một tệp đoạn mới. Sau đó ta có thể thực hiện nén (compaction) trên các đoạn này, như minh họa ở Hình 3-2. Nén nghĩa là loại bỏ các khóa trùng lặp trong log, và chỉ giữ lại bản cập nhật gần đây nhất cho mỗi khóa.

Figure 3-2. Compaction of a key-value update log (counting the number of times each cat video was played), retaining only the most recent value for each key. — Hình 3-2. Nén một log cập nhật khóa-giá trị (đếm số lần mỗi video về mèo được phát), chỉ giữ lại giá trị gần đây nhất cho mỗi khóa. Moreover, since compaction often makes segments much smaller (assuming that a key is overwritten several times on **average** within one segment), we can also merge several segments together at the same time as performing the compaction, as shown in Figure 3-3. Segments are never modified after they have been written, so the merged segment is written to a new file. The merging and compaction of frozen segments can be done in a background thread, and while it is going on, we can still continue to serve read and write requests as normal, using the old segment files. After the merging process is complete, we switch read requests to using the new merged segment instead of the old segments—and then the old segment files can simply be deleted.

Hơn nữa, vì nén thường làm các đoạn nhỏ đi đáng kể (giả sử trung bình mỗi khóa bị ghi đè vài lần trong một đoạn), ta cũng có thể gộp (merge) nhiều đoạn lại với nhau cùng lúc với việc nén, như trình bày ở Hình 3-3. Các đoạn không bao giờ bị sửa đổi sau khi đã được ghi, nên đoạn đã gộp được ghi vào một tệp mới. Việc gộp và nén các đoạn đã đóng băng có thể được thực hiện trong một luồng (thread) chạy nền, và trong khi việc đó diễn ra, ta vẫn có thể tiếp tục phục vụ các yêu cầu đọc và ghi như bình thường, dùng các tệp đoạn cũ. Sau khi quá trình gộp hoàn tất, ta chuyển các yêu cầu đọc sang dùng đoạn đã gộp mới thay vì các đoạn cũ — và rồi các tệp đoạn cũ có thể đơn giản bị xóa đi.

Figure 3-3. Performing compaction and segment merging simultaneously. — Hình 3-3. Thực hiện nén và gộp đoạn đồng thời. Each segment now has its own in-memory hash table, mapping keys to file offsets. In order to find the value for a key, we first check the most recent segment's hash map; if the key is not present we check the second-most-recent segment, and so on. The merging process keeps the number of segments small, so lookups don't need to check many hash maps.

Giờ đây mỗi đoạn có bảng băm riêng trong bộ nhớ, ánh xạ các khóa tới độ lệch trong tệp. Để tìm giá trị ứng với một khóa, trước tiên ta kiểm tra bản đồ băm của đoạn gần đây nhất; nếu khóa không có mặt, ta kiểm tra đoạn gần đây nhì, và cứ thế tiếp tục. Quá trình gộp giữ cho số lượng đoạn luôn nhỏ, nên các lần tra cứu không cần kiểm tra nhiều bản đồ băm.

Lots of detail goes into making this simple idea work in practice. Briefly, some of the issues that are important in a real implementation are:

Có rất nhiều chi tiết cần lưu tâm để biến ý tưởng đơn giản này thành hiện thực trong thực tế. Vấn đề, một số vấn đề quan trọng trong một hiện thực thực tế là:

CSV is not the best format for a log. It's faster and simpler to use a binary format that first encodes the length of a string in bytes, followed by the raw string (without need for escaping).

CSV không phải định dạng tốt nhất cho một log. Việc dùng một định dạng nhị phân vừa nhanh hơn vừa đơn giản hơn: định dạng này trước tiên mã hóa độ dài của một chuỗi tính theo byte, theo sau là chuỗi thô (không cần escape).

If you want to delete a key and its associated value, you have to append a special deletion record to the data file (sometimes called a tombstone). When log segments are merged, the tombstone tells the merging process to discard any previous values for the deleted key.

Nếu ta muốn xóa một khóa cùng giá trị gắn với nó, ta phải ghi nối một bản ghi xóa đặc biệt vào tệp dữ liệu (đôi khi gọi là một tombstone, tức “bia mộ”). Khi các đoạn log được gộp, tombstone báo cho quá trình gộp loại bỏ mọi giá trị trước đó của khóa đã bị xóa.

If the database is restarted, the in-memory hash maps are lost. In principle, you can restore each segment’s hash map by reading the entire segment file from beginning to end and noting the offset of the most recent value for every key as you go along. However, that might take a long time if the segment files are large, which would make server restarts painful. Bitcask speeds up recovery by storing a snapshot of each segment’s hash map on disk, which can be loaded into memory more quickly.

Nếu cơ sở dữ liệu được khởi động lại, các bản đồ băm trong bộ nhớ sẽ mất. Về nguyên tắc, ta có thể khôi phục bản đồ băm của từng đoạn bằng cách đọc toàn bộ tệp đoạn từ đầu đến cuối và ghi lại độ lệch của giá trị gần đây nhất cho mỗi khóa khi đọc. Tuy nhiên, việc đó có thể mất nhiều thời gian nếu các tệp đoạn lớn, khiến việc khởi động lại máy chủ trở nên đau đầu. Bitcask tăng tốc khôi phục bằng cách lưu một ảnh chụp (snapshot) bản đồ băm của mỗi đoạn trên đĩa, vốn có thể được nạp vào bộ nhớ nhanh hơn.

The database may crash at any time, including halfway through appending a record to the log. Bitcask files include checksums, allowing such corrupted parts of the log to be detected and ignored.

Cơ sở dữ liệu có thể sập (crash) bất cứ lúc nào, kể cả khi đang ghi nối dở một bản ghi vào log. Các tệp Bitcask có kèm tổng kiểm tra (checksum), cho phép phát hiện và bỏ qua các phần hỏng như vậy của log.

As writes are appended to the log in a strictly sequential order, a common implementation choice is to have only one writer thread. Data file segments are append-only and otherwise immutable, so they can be read concurrently by multiple threads.

Vì các lần ghi được ghi nối vào log theo một thứ tự tuần tự nghiêm ngặt, một lựa chọn hiện thực phổ biến là chỉ có một luồng ghi duy nhất. Các đoạn tệp dữ liệu là chỉ-ghi-nối và ngoài ra thì bất biến (immutable), nên chúng có thể được đọc tương tranh bởi nhiều luồng.

An append-only log seems wasteful at first glance: why don’t you update the file in place, overwriting the old value with the new value? But an append-only design turns out to be good for several reasons:

Một log chỉ-ghi-nối thoạt nhìn có vẻ lãng phí: tại sao ta không cập nhật tệp tại chỗ, ghi đè giá trị cũ bằng giá trị mới? Nhưng hóa ra một thiết kế chỉ-ghi-nối lại tốt vì vài lý do:

- Appending and segment merging are sequential write operations, which are generally much faster than random writes, especially on magnetic spinning-disk hard drives. To some extent sequential writes are also preferable on flash-based solid state drives (SSDs) [4]. We will discuss this issue further in “Comparing B-Trees and LSM-Trees”.
- Concurrency and crash recovery are much simpler if segment files are append-only or immutable. For example, you don’t have to worry about the case where a crash happened while a value was being overwritten, leaving you with a file containing part of the old and part of the new value spliced together.
- Merging old segments avoids the problem of data files getting fragmented over time.

- Ghi nối và gộp đoạn là các thao tác ghi tuần tự, vốn nhìn chung nhanh hơn nhiều so với ghi ngẫu nhiên, đặc biệt trên các ổ cứng từ quay. Ở một mức độ nào đó, ghi tuần tự cũng được ưa chuộng hơn trên các ổ thể rắn (SSD) dựa trên bộ nhớ flash [4]. Ta sẽ bàn thêm vấn đề này ở phần “So sánh B-tree và LSM-tree”.
- Tương tranh và khôi phục sau sự cố đơn giản hơn nhiều nếu các tệp đoạn là chỉ-ghi-nối hoặc bất biến. Ví dụ, ta không phải lo về trường hợp một sự cố xảy ra trong khi một giá trị đang được ghi đè, để lại cho ta một tệp chứa một phần giá trị cũ và một phần giá trị mới ghép vào nhau.
- Gộp các đoạn cũ tránh được vấn đề các tệp dữ liệu bị phân mảnh theo thời gian.

However, the hash table index also has limitations:

Tuy nhiên, chỉ mục bảng băm cũng có những hạn chế:

- The hash table must fit in memory, so if you have a very large number of keys, you’re out of luck. In principle, you could maintain a hash map on disk, but unfortunately it is difficult to make an on-disk hash map perform well. It requires a lot of random access I/O, it is expensive to grow when it becomes full, and hash collisions require fiddly logic [5].
- Range queries are not efficient. For example, you cannot easily scan over all keys between kitty00000 and kitty99999—you’d have to look up each key individually in the hash maps.
 - Bảng băm phải vừa với bộ nhớ, nên nếu ta có một số lượng khóa rất lớn, thì đành chịu. Về nguyên tắc, ta có thể duy trì một bản đồ băm trên đĩa, nhưng tiếc là khó mà khiến một bản đồ băm trên đĩa hoạt động tốt. Nó đòi hỏi rất nhiều thao tác I/O truy cập ngẫu nhiên, việc mở rộng khi nó đầy lên là tốn kém, và các va chạm băm (hash collision) đòi hỏi logic rắc rối [5].
 - Truy vấn theo khoảng không hiệu quả. Ví dụ, ta không thể dễ dàng quét qua tất cả các khóa nằm giữa kitty00000 và kitty99999 — ta sẽ phải tra cứu từng khóa một trong các bản đồ băm.

In the next section we will look at an indexing structure that doesn’t have those limitations.

Ở phần tiếp theo, ta sẽ xem xét một cấu trúc lập chỉ mục không vướng những hạn chế đó.

SSTables and LSM-Trees — SSTable và LSM-tree

In Figure 3-3, each log-structured storage segment is a sequence of key-value pairs. These pairs appear in the order that they were written, and values later in the log take precedence over values for the same key earlier in the log. Apart from that, the order of key-value pairs in the file does not matter.

Ở Hình 3-3, mỗi đoạn lưu trữ cấu trúc theo log là một chuỗi các cặp khóa-giá trị. Các cặp này xuất hiện theo thứ tự chúng được ghi, và các giá trị xuất hiện muộn hơn trong log được ưu tiên hơn các giá trị của cùng khóa xuất hiện sớm hơn trong log. Ngoài điều đó ra, thứ tự của các cặp khóa-giá trị trong tệp không quan trọng.

Now we can make a simple change to the format of our segment files: we require that the sequence of key-value pairs is sorted by key. At first glance, that requirement seems to break our ability to use sequential writes, but we’ll get to that in a moment.

Giờ ta có thể thực hiện một thay đổi đơn giản đối với định dạng các tệp đoạn của mình: ta yêu cầu chuỗi các cặp khóa-giá trị phải được sắp xếp theo khóa. Thoạt nhìn, yêu cầu đó có vẻ phá vỡ khả năng dùng ghi tuần tự của ta, nhưng ta sẽ bàn tới điều đó ngay sau đây.

We call this format Sorted String Table, or SSTable for short. We also require that each key only appears once within each merged segment file (the compaction process already ensures that). SSTables have several big advantages over log segments with hash indexes:

Ta gọi định dạng này là Sorted String Table, hay viết tắt là SSTable. Ta cũng yêu cầu mỗi khóa chỉ xuất hiện một lần trong mỗi tệp đoạn đã gộp (quá trình nén vốn đã đảm bảo điều này). SSTable có vài ưu điểm lớn so với các đoạn log dùng chỉ mục băm:

- Merging segments is simple and efficient, even if the files are bigger than the available memory. The approach is like the one used in the mergesort algorithm and is illustrated in Figure 3-4: you start reading the input files side by side, look at the first key in each file, copy the lowest key (according to the sort order) to the output file, and repeat. This produces a new merged segment file, also sorted by key. Figure 3-4. Merging several SSTable segments, retaining only the most recent value for each key. What if the same key appears in several input segments? Remember that each segment contains all the values written to the database during some period of time. This means that all the values in one input segment must be more recent than all the values in the other segment (assuming that we always merge adjacent segments). When multiple segments contain the same key, we can keep the value from the most recent segment and discard the values in older segments.
- In order to find a particular key in the file, you no longer need to keep an index of all the keys in memory. See Figure 3-5 for an example: say you're looking for the key handiwork, but you don't know the exact offset of that key in the segment file. However, you do know the offsets for the keys handbag and handsome, and because of the sorting you know that handiwork must appear between those two. This means you can jump to the offset for handbag and scan from there until you find handiwork (or not, if the key is not present in the file). Figure 3-5. An SSTable with an in-memory index. You still need an in-memory index to tell you the offsets for some of the keys, but it can be sparse: one key for every few kilobytes of segment file is sufficient, because a few kilobytes can be scanned very quickly.
- Since read requests need to scan over several key-value pairs in the requested range anyway, it is possible to group those records into a block and compress it before writing it to disk (indicated by the shaded area in Figure 3-5). Each entry of the sparse in-memory index then points at the start of a compressed block. Besides saving disk space, compression also reduces the I/O bandwidth use.
- Gộp các đoạn rất đơn giản và hiệu quả, ngay cả khi các tệp lớn hơn bộ nhớ khả dụng. Cách làm tương tự cách dùng trong thuật toán sắp xếp trộn (mergesort) và được minh họa ở Hình 3-4: ta bắt đầu đọc các tệp đầu vào song song cạnh nhau, nhìn khóa đầu tiên trong mỗi tệp, chép khóa nhỏ nhất (theo thứ tự sắp xếp) sang tệp đầu ra, rồi lặp lại. Điều này tạo ra một tệp đoạn đã gộp mới, cũng được sắp xếp theo khóa. Hình 3-4. Gộp nhiều đoạn SSTable, chỉ giữ lại giá trị gần đây nhất cho mỗi khóa. Sẽ thế nào nếu cùng một khóa xuất hiện trong nhiều đoạn đầu vào? Hãy nhớ rằng mỗi đoạn chứa tất cả các giá trị được ghi vào cơ sở dữ liệu trong một khoảng thời gian nào đó. Điều này nghĩa là mọi giá trị trong một đoạn đầu vào phải mới hơn mọi giá trị trong đoạn còn lại (giả sử ta luôn gộp các đoạn kề nhau). Khi nhiều đoạn chứa cùng một khóa, ta có thể giữ giá trị từ đoạn gần đây nhất và loại bỏ các giá trị trong các đoạn cũ hơn.
- Để tìm một khóa cụ thể trong tệp, ta không còn cần giữ một chỉ mục của tất cả các khóa trong bộ nhớ. Xem Hình 3-5 để có một ví dụ: giả sử ta đang tìm khóa handiwork, nhưng ta không biết độ lệch chính xác của khóa đó trong tệp đoạn. Tuy nhiên, ta biết độ lệch của các khóa handbag và handsome, và nhờ việc sắp xếp, ta biết handiwork chắc chắn phải xuất hiện giữa hai khóa đó. Điều này nghĩa là ta có thể nhảy tới độ lệch của handbag và quét từ đó cho đến khi tìm thấy handiwork (hoặc không, nếu khóa không có mặt trong tệp). Hình 3-5. Một SSTable với chỉ mục trong bộ nhớ. Ta

vẫn cần một chỉ mục trong bộ nhớ để cho biết độ lệch của một số khóa, nhưng nó có thể thưa (sparse): một khóa cho mỗi vài kilobyte của tệp đoạn là đủ, bởi vài kilobyte có thể được quét rất nhanh.

- Vì các yêu cầu đọc dù sao cũng cần quét qua vài cặp khóa-giá trị trong khoảng được yêu cầu, ta có thể nhóm các bản ghi đó thành một khối (block) và nén nó lại trước khi ghi xuống đĩa (được biểu thị bằng vùng tô bóng ở Hình 3-5). Khi đó mỗi mục của chỉ mục thưa trong bộ nhớ trở tới điểm bắt đầu của một khối đã nén. Bên cạnh việc tiết kiệm không gian đĩa, nén còn giảm việc sử dụng băng thông I/O.

Constructing and maintaining SSTables — Xây dựng và duy trì SSTable

Fine so far—but how do you get your data to be sorted by key in the first place? Our incoming writes can occur in any order.

Đến đây thì ổn rồi — nhưng làm thế nào để có được dữ liệu được sắp xếp theo khóa ngay từ đầu? Các lần ghi đến của ta có thể xảy ra theo thứ tự bất kỳ.

Maintaining a sorted structure on disk is possible (see “B-Trees”), but maintaining it in memory is much easier. There are plenty of well-known tree data structures that you can use, such as red-black trees or AVL trees [2]. With these data structures, you can insert keys in any order and read them back in sorted order.

Việc duy trì một cấu trúc đã sắp xếp trên đĩa là khả thi (xem phần “B-tree”), nhưng duy trì nó trong bộ nhớ thì dễ hơn nhiều. Có rất nhiều cấu trúc dữ liệu cây nổi tiếng mà ta có thể dùng, chẳng hạn cây đỏ-đen (red-black tree) hoặc cây AVL [2]. Với các cấu trúc dữ liệu này, ta có thể chèn các khóa theo thứ tự bất kỳ và đọc chúng ra theo thứ tự đã sắp xếp.

We can now make our storage engine work as follows:

Giờ ta có thể khiến engine lưu trữ của mình hoạt động như sau:

- When a write comes in, add it to an in-memory balanced tree data structure (for example, a red-black tree). This in-memory tree is sometimes called a memtable.
- When the memtable gets bigger than some threshold—typically a few megabytes—write it out to disk as an SSTable file. This can be done efficiently because the tree already maintains the key-value pairs sorted by key. The new SSTable file becomes the most recent segment of the database. While the SSTable is being written out to disk, writes can continue to a new memtable **instance**.
- In order to serve a read request, first try to find the key in the memtable, then in the most recent on-disk segment, then in the next-older segment, etc.
- From time to time, run a merging and compaction process in the background to combine segment files and to discard overwritten or deleted values.
 - Khi một lần ghi đến, thêm nó vào một cấu trúc dữ liệu cây cân bằng trong bộ nhớ (ví dụ, một cây đỏ-đen). Cây trong bộ nhớ này đôi khi được gọi là một memtable.
 - Khi memtable lớn hơn một ngưỡng nào đó — thường là vài megabyte — ghi nó ra đĩa thành một tệp SSTable. Việc này có thể thực hiện hiệu quả vì cây vốn đã duy trì các cặp khóa-giá trị được sắp xếp theo khóa. Tệp SSTable mới trở thành đoạn gần đây nhất của cơ sở dữ liệu. Trong khi SSTable đang được ghi ra đĩa, các lần ghi có thể tiếp tục đến một thể hiện (instance) memtable mới.
 - Để phục vụ một yêu cầu đọc, trước tiên thử tìm khóa trong memtable, rồi trong đoạn trên đĩa gần đây nhất, rồi trong đoạn cũ hơn kế tiếp, v.v.
 - Thỉnh thoảng, chạy một quá trình gộp và nén ở chế độ nền để kết hợp các tệp đoạn và loại bỏ các giá trị đã bị ghi đè hoặc đã xóa.

This scheme works very well. It only suffers from one problem: if the database crashes, the most recent writes (which are in the memtable but not yet written out to disk) are lost. In order to avoid that problem, we can keep a separate log on disk to which every write is immediately appended, just like in the previous section. That log is not in sorted order, but that doesn't matter, because its only purpose is to restore the memtable after a crash. Every time the memtable is written out to an SSTable, the corresponding log can be discarded.

Cơ chế này hoạt động rất tốt. Nó chỉ vướng một vấn đề: nếu cơ sở dữ liệu sập, các lần ghi gần đây nhất (đang ở trong memtable nhưng chưa được ghi ra đĩa) sẽ mất. Để tránh vấn đề đó, ta có thể giữ một log riêng trên đĩa mà mọi lần ghi đều được ghi nối vào ngay lập tức, giống hệt như ở phần trước. Log đó không theo thứ tự đã sắp xếp, nhưng điều đó không quan trọng, bởi mục đích duy nhất của nó là khôi phục memtable sau một sự cố. Mỗi khi memtable được ghi ra thành một SSTable, log tương ứng có thể bị loại bỏ.

Making an LSM-tree out of SSTables — Tạo một LSM-tree từ các SSTable

The algorithm described here is essentially what is used in LevelDB [6] and RocksDB [7], key-value storage engine libraries that are designed to be embedded into other applications. Among other things, LevelDB can be used in Riak as an alternative to Bitcask. Similar storage engines are used in Cassandra and HBase [8], both of which were inspired by Google's Bigtable paper [9] (which introduced the terms SSTable and memtable).

Thuật toán được mô tả ở đây về cơ bản chính là thứ được dùng trong LevelDB [6] và RocksDB [7], các thư viện engine lưu trữ khóa-giá trị được thiết kế để nhúng vào các ứng dụng khác. Bên cạnh những thứ khác, LevelDB có thể được dùng trong Riak như một lựa chọn thay thế cho Bitcask. Các engine lưu trữ tương tự được dùng trong Cassandra và HBase [8], cả hai đều lấy cảm hứng từ bài báo Bigtable của Google [9] (vốn đã giới thiệu các thuật ngữ SSTable và memtable).

Originally this indexing structure was described by Patrick O'Neil et al. under the name Log-Structured Merge-Tree (or LSM-Tree) [10], building on earlier work on log-structured filesystems [11]. Storage engines that are based on this principle of merging and compacting sorted files are often called LSM storage engines.

Ban đầu, cấu trúc lập chỉ mục này được Patrick O'Neil và cộng sự mô tả dưới cái tên Log-Structured Merge-Tree (hay LSM-Tree) [10], xây dựng dựa trên công trình trước đó về các hệ thống tệp cấu trúc theo log [11]. Các engine lưu trữ dựa trên nguyên lý gộp và nén các tệp đã sắp xếp này thường được gọi là engine lưu trữ LSM.

Lucene, an indexing engine for **full-text search** used by Elasticsearch and Solr, uses a similar method for storing its term dictionary [12, 13]. A full-text index is much more complex than a key-value index but is based on a similar idea: given a word in a search query, find all the documents (web pages, product descriptions, etc.) that mention the word. This is implemented with a key-value structure where the key is a word (a term) and the value is the list of IDs of all the documents that contain the word (the postings list). In Lucene, this mapping from term to postings list is kept in SSTable-like sorted files, which are merged in the background as needed [14].

Lucene, một engine lập chỉ mục cho tìm kiếm toàn văn được Elasticsearch và Solr sử dụng, dùng một phương pháp tương tự để lưu từ điển thuật ngữ (term dictionary) của nó [12, 13]. Một chỉ mục toàn văn phức tạp hơn nhiều so với một chỉ mục khóa-giá trị nhưng dựa trên một ý tưởng tương tự: cho một từ trong truy vấn tìm kiếm, tìm tất cả các tài liệu (trang web, mô tả sản phẩm, v.v.) có nhắc tới từ đó. Điều này được hiện thực bằng một cấu trúc khóa-giá trị, trong đó khóa là một từ (một thuật ngữ, term) và giá trị là danh sách các ID

của tất cả các tài liệu chứa từ đó (danh sách đăng tin, postings list). Trong Lucene, ánh xạ từ thuật ngữ tới postings list này được giữ trong các tệp đã sắp xếp kiểu SSTable, vốn được gộp ở chế độ nền khi cần [14].

Performance optimizations — Tối ưu hiệu năng

As always, a lot of detail goes into making a storage engine perform well in practice. For example, the LSM-tree algorithm can be slow when looking up keys that do not exist in the database: you have to check the memtable, then the segments all the way back to the oldest (possibly having to read from disk for each one) before you can be sure that the key does not exist. In order to optimize this kind of access, storage engines often use additional Bloom filters [15]. (A Bloom filter is a memory-efficient data structure for approximating the contents of a set. It can tell you if a key does not appear in the database, and thus saves many unnecessary disk reads for nonexistent keys.)

Như mọi khi, có rất nhiều chi tiết cần lưu tâm để khiến một engine lưu trữ hoạt động tốt trong thực tế. Ví dụ, thuật toán LSM-tree có thể chậm khi tra cứu các khóa không tồn tại trong cơ sở dữ liệu: ta phải kiểm tra memtable, rồi các đoạn cho đến tận đoạn cũ nhất (có thể phải đọc từ đĩa cho mỗi đoạn) trước khi có thể chắc chắn rằng khóa không tồn tại. Để tối ưu kiểu truy cập này, các engine lưu trữ thường dùng thêm các bộ lọc Bloom (Bloom filter) [15]. (Một bộ lọc Bloom là một cấu trúc dữ liệu tiết kiệm bộ nhớ để xấp xỉ nội dung của một tập hợp. Nó có thể cho ta biết liệu một khóa có không xuất hiện trong cơ sở dữ liệu hay không, và nhờ đó tiết kiệm nhiều lần đọc đĩa không cần thiết cho các khóa không tồn tại.)

There are also different strategies to determine the order and timing of how SSTables are compacted and merged. The most common options are size-tiered and leveled compaction. LevelDB and RocksDB use leveled compaction (hence the name of LevelDB), HBase uses size-tiered, and Cassandra supports both [16]. In size-tiered compaction, newer and smaller SSTables are successively merged into older and larger SSTables. In leveled compaction, the key range is split up into smaller SSTables and older data is moved into separate “levels,” which allows the compaction to proceed more incrementally and use less disk space.

Cũng có những chiến lược khác nhau để quyết định thứ tự và thời điểm các SSTable được nén và gộp. Hai lựa chọn phổ biến nhất là nén theo bậc kích thước (size-tiered) và nén theo cấp (leveled). LevelDB và RocksDB dùng nén theo cấp (do đó mới có tên LevelDB), HBase dùng theo bậc kích thước, và Cassandra hỗ trợ cả hai [16]. Trong nén theo bậc kích thước, các SSTable mới hơn và nhỏ hơn được gộp dần vào các SSTable cũ hơn và lớn hơn. Trong nén theo cấp, khoảng khóa được chia nhỏ thành các SSTable nhỏ hơn và dữ liệu cũ hơn được chuyển vào các “cấp” riêng biệt, điều này cho phép việc nén tiến hành tăng dần hơn và dùng ít không gian đĩa hơn.

Even though there are many subtleties, the basic idea of LSM-trees—keeping a cascade of SSTables that are merged in the background—is simple and effective. Even when the dataset is much bigger than the available memory it continues to work well. Since data is stored in sorted order, you can efficiently perform range queries (scanning all keys above some minimum and up to some maximum), and because the disk writes are sequential the LSM-tree can support remarkably high write **throughput**.

Mặc dù có nhiều điểm tinh tế, ý tưởng cơ bản của LSM-tree — giữ một chuỗi nối tiếp các SSTable được gộp ở chế độ nền — vừa đơn giản vừa hiệu quả. Ngay cả khi tập dữ liệu lớn hơn nhiều so với bộ nhớ khả dụng, nó vẫn tiếp tục hoạt động tốt. Vì dữ liệu được lưu theo thứ tự đã sắp xếp, ta có thể thực hiện hiệu quả các truy vấn theo khoảng (quét tất cả các khóa nằm trên một giá trị nhỏ nhất nào đó và cho tới một giá trị lớn nhất nào đó), và vì

các lần ghi xuống đĩa là tuần tự nên LSM-tree có thể hỗ trợ thông lượng ghi cao một cách đáng kể.

B-Trees — B-tree

The log-structured indexes we have discussed so far are gaining acceptance, but they are not the most common type of index. The most widely used indexing structure is quite different: the B-tree.

Các chỉ mục cấu trúc theo log mà ta đã bàn từ đầu đến giờ đang dần được chấp nhận, nhưng chúng không phải loại chỉ mục phổ biến nhất. Cấu trúc lập chỉ mục được dùng rộng rãi nhất lại khá khác biệt: B-tree.

Introduced in 1970 [17] and called “ubiquitous” less than 10 years later [18], B-trees have stood the test of time very well. They remain the standard index implementation in almost all relational databases, and many nonrelational databases use them too.

Được giới thiệu vào năm 1970 [17] và được gọi là “có mặt khắp nơi” chưa đầy 10 năm sau đó [18], B-tree đã trụ vững rất tốt qua thử thách của thời gian. Chúng vẫn là cách hiện thực chỉ mục tiêu chuẩn trong gần như mọi cơ sở dữ liệu quan hệ, và nhiều cơ sở dữ liệu phi quan hệ cũng dùng chúng.

Like SSTables, B-trees keep key-value pairs sorted by key, which allows efficient key-value lookups and range queries. But that’s where the similarity ends: B-trees have a very different design philosophy.

Giống SSTable, B-tree giữ các cặp khóa-giá trị được sắp xếp theo khóa, điều này cho phép tra cứu khóa-giá trị và truy vấn theo khoảng một cách hiệu quả. Nhưng điểm giống nhau chỉ dừng ở đó: B-tree có một triết lý thiết kế rất khác.

The log-structured indexes we saw earlier break the database down into variable-size segments, typically several megabytes or more in size, and always write a segment sequentially. By contrast, B-trees break the database down into fixed-size blocks or pages, traditionally 4 KB in size (sometimes bigger), and read or write one page at a time. This design corresponds more closely to the underlying hardware, as disks are also arranged in fixed-size blocks.

Các chỉ mục cấu trúc theo log mà ta đã thấy trước đó chia nhỏ cơ sở dữ liệu thành các đoạn có kích thước thay đổi, thường vài megabyte trở lên, và luôn ghi một đoạn theo cách tuần tự. Trái lại, B-tree chia nhỏ cơ sở dữ liệu thành các khối (block) hoặc trang (page) có kích thước cố định, theo truyền thống là 4 KB (đôi khi lớn hơn), và đọc hoặc ghi mỗi lần một trang. Thiết kế này tương ứng sát hơn với phần cứng nền tảng, bởi đĩa cũng được sắp xếp thành các khối có kích thước cố định.

Each page can be identified using an address or location, which allows one page to refer to another—similar to a pointer, but on disk instead of in memory. We can use these page references to construct a tree of pages, as illustrated in Figure 3-6.

Mỗi trang có thể được nhận diện bằng một địa chỉ hoặc vị trí, điều này cho phép một trang tham chiếu tới một trang khác — tương tự một con trỏ, nhưng nằm trên đĩa chứ không phải trong bộ nhớ. Ta có thể dùng các tham chiếu trang này để xây dựng một cây các trang, như minh họa ở Hình 3-6.

Figure 3-6. Looking up a key using a B-tree index. — Hình 3-6. Tra cứu một khóa bằng một chỉ mục B-tree. One page is designated as the root of the B-tree; whenever you want to look up a key in the index, you start here. The page contains several keys and references to child pages. Each child

is responsible for a continuous range of keys, and the keys between the references indicate where the boundaries between those ranges lie.

Một trang được chỉ định làm gốc (root) của B-tree; mỗi khi ta muốn tra cứu một khóa trong chỉ mục, ta bắt đầu từ đây. Trang này chứa vài khóa và các tham chiếu tới các trang con. Mỗi trang con chịu trách nhiệm cho một khoảng khóa liên tục, và các khóa nằm giữa các tham chiếu cho biết ranh giới giữa các khoảng đó nằm ở đâu.

In the example in Figure 3-6, we are looking for the key 251, so we know that we need to follow the page reference between the boundaries 200 and 300. That takes us to a similar-looking page that further breaks down the 200–300 range into subranges. Eventually we get down to a page containing individual keys (a leaf page), which either contains the value for each key inline or contains references to the pages where the values can be found.

Trong ví dụ ở Hình 3-6, ta đang tìm khóa 251, nên ta biết rằng ta cần đi theo tham chiếu trang nằm giữa các ranh giới 200 và 300. Tham chiếu đó đưa ta tới một trang trông tương tự, trang này tiếp tục chia nhỏ khoảng 200–300 thành các khoảng con. Cuối cùng ta đi xuống tới một trang chứa các khóa riêng lẻ (một trang lá, leaf page), trang này hoặc chứa giá trị cho mỗi khóa ngay tại chỗ (inline), hoặc chứa các tham chiếu tới các trang nơi có thể tìm thấy các giá trị.

The number of references to child pages in one page of the B-tree is called the branching factor. For example, in Figure 3-6 the branching factor is six. In practice, the branching factor depends on the amount of space required to store the page references and the range boundaries, but typically it is several hundred.

Số lượng tham chiếu tới các trang con trong một trang của B-tree được gọi là hệ số phân nhánh (branching factor). Ví dụ, ở Hình 3-6 hệ số phân nhánh là sáu. Trong thực tế, hệ số phân nhánh phụ thuộc vào lượng không gian cần để lưu các tham chiếu trang và các ranh giới khoảng, nhưng thường là vài trăm.

If you want to update the value for an existing key in a B-tree, you search for the leaf page containing that key, change the value in that page, and write the page back to disk (any references to that page remain valid). If you want to add a new key, you need to find the page whose range encompasses the new key and add it to that page. If there isn't enough free space in the page to accommodate the new key, it is split into two half-full pages, and the parent page is updated to account for the new subdivision of key ranges—see Figure 3-7.

Nếu ta muốn cập nhật giá trị cho một khóa hiện có trong một B-tree, ta tìm trang lá chứa khóa đó, thay đổi giá trị trong trang đó, và ghi trang đó trở lại đĩa (mọi tham chiếu tới trang đó vẫn còn hiệu lực). Nếu ta muốn thêm một khóa mới, ta cần tìm trang có khoảng bao trùm khóa mới và thêm khóa vào trang đó. Nếu trong trang không đủ không gian trống để chứa khóa mới, nó được tách thành hai trang đầy một nửa, và trang cha được cập nhật để phản ánh sự phân chia mới của các khoảng khóa — xem Hình 3-7.

Figure 3-7. Growing a B-tree by splitting a page. — Hình 3-7. Mở rộng một B-tree bằng cách tách một trang. This algorithm ensures that the tree remains balanced: a B-tree with n keys always has a depth of $O(\log n)$. Most databases can fit into a B-tree that is three or four levels deep, so you don't need to follow many page references to find the page you are looking for. (A four-level tree of 4 KB pages with a branching factor of 500 can store up to 256 TB.)

Thuật toán này đảm bảo cây luôn cân bằng: một B-tree với n khóa luôn có độ sâu $O(\log n)$. Hầu hết các cơ sở dữ liệu đều vừa với một B-tree sâu ba hoặc bốn cấp, nên ta không cần

đi theo nhiều tham chiếu trang để tìm trang mình cần. (Một cây bốn cấp gồm các trang 4 KB với hệ số phân nhánh 500 có thể lưu tới 256 TB.)

Making B-trees reliable — Làm cho B-tree đáng tin cậy

The basic underlying write operation of a B-tree is to overwrite a page on disk with new data. It is assumed that the overwrite does not change the location of the page; i.e., all references to that page remain intact when the page is overwritten. This is in stark contrast to log-structured indexes such as LSM-trees, which only append to files (and eventually delete obsolete files) but never modify files in place.

Thao tác ghi nền tảng cơ bản của một B-tree là ghi đè một trang trên đĩa bằng dữ liệu mới. Giả định ở đây là việc ghi đè không làm thay đổi vị trí của trang; tức là mọi tham chiếu tới trang đó vẫn còn nguyên vẹn khi trang bị ghi đè. Điều này trái ngược hoàn toàn với các chỉ mục cấu trúc theo log như LSM-tree, vốn chỉ ghi nối vào các tệp (và cuối cùng xóa các tệp đã lỗi thời) nhưng không bao giờ sửa đổi tệp tại chỗ.

You can think of overwriting a page on disk as an actual hardware operation. On a magnetic hard drive, this means moving the disk head to the right place, waiting for the right position on the spinning platter to come around, and then overwriting the appropriate sector with new data. On SSDs, what happens is somewhat more complicated, due to the fact that an SSD must erase and rewrite fairly large blocks of a storage chip at a time [19].

Ta có thể nghĩ về việc ghi đè một trang trên đĩa như một thao tác phần cứng thực sự. Trên một ổ cứng từ, điều này nghĩa là di chuyển đầu đọc đĩa tới đúng chỗ, chờ đúng vị trí trên mặt đĩa quay tới, rồi ghi đè đúng sector bằng dữ liệu mới. Trên SSD, những gì xảy ra có phần phức tạp hơn, do thực tế là một SSD phải xóa và ghi lại từng khối khá lớn của một chip lưu trữ tại một thời điểm [19].

Moreover, some operations require several different pages to be overwritten. For example, if you split a page because an insertion caused it to be overfull, you need to write the two pages that were split, and also overwrite their parent page to update the references to the two child pages. This is a dangerous operation, because if the database crashes after only some of the pages have been written, you end up with a corrupted index (e.g., there may be an orphan page that is not a child of any parent).

Hơn nữa, một số thao tác đòi hỏi vài trang khác nhau phải bị ghi đè. Ví dụ, nếu ta tách một trang vì một lần chèn khiến nó quá đầy, ta cần ghi hai trang đã được tách, và cũng ghi đè trang cha của chúng để cập nhật các tham chiếu tới hai trang con. Đây là một thao tác nguy hiểm, bởi nếu cơ sở dữ liệu sập sau khi chỉ một số trang đã được ghi, ta sẽ có một chỉ mục bị hỏng (ví dụ, có thể có một trang mồ côi không phải là con của bất kỳ trang cha nào).

In order to make the database **resilient** to crashes, it is common for B-tree implementations to include an additional data structure on disk: a write-ahead log (WAL, also known as a redo log). This is an append-only file to which every B-tree modification must be written before it can be applied to the pages of the tree itself. When the database comes back up after a crash, this log is used to restore the B-tree back to a consistent state [5, 20].

Để khiến cơ sở dữ liệu kiên cường trước các sự cố, các hiện thực B-tree thường kèm theo một cấu trúc dữ liệu bổ sung trên đĩa: một log ghi-trước (write-ahead log, WAL, còn gọi là redo log). Đây là một tệp chỉ-ghi-nối mà mọi sửa đổi B-tree đều phải được ghi vào đó trước khi có thể được áp dụng lên các trang của chính cái cây. Khi cơ sở dữ liệu khởi động lại sau một sự cố, log này được dùng để khôi phục B-tree về trạng thái nhất quán [5, 20].

An additional complication of updating pages in place is that careful concurrency control is required if multiple threads are going to access the B-tree at the same time—otherwise a thread may see the tree in an inconsistent state. This is typically done by protecting the tree’s data structures with latches (lightweight locks). Log-structured approaches are simpler in this regard, because they do all the merging in the background without interfering with incoming queries and atomically swap old segments for new segments from time to time.

Một sự phức tạp bổ sung của việc cập nhật các trang tại chỗ là cần kiểm soát tương tranh cẩn thận nếu nhiều luồng sẽ truy cập B-tree cùng lúc — nếu không một luồng có thể thấy cây ở trạng thái bất nhất. Điều này thường được thực hiện bằng cách bảo vệ các cấu trúc dữ liệu của cây bằng các latch (khóa nhẹ). Các cách tiếp cận cấu trúc theo log đơn giản hơn về mặt này, bởi chúng thực hiện toàn bộ việc gộp ở chế độ nền mà không can thiệp vào các truy vấn đến, và thỉnh thoảng trao đổi nguyên tử (atomically) các đoạn cũ lấy các đoạn mới.

B-tree optimizations — Tối ưu B-tree

As B-trees have been around for so long, it’s not surprising that many optimizations have been developed over the years. To mention just a few:

Vì B-tree đã tồn tại lâu đến vậy, không có gì ngạc nhiên khi nhiều tối ưu đã được phát triển qua nhiều năm. Chỉ nêu ra vài cái:

- Instead of overwriting pages and maintaining a WAL for crash recovery, some databases (like LMDB) use a copy-on-write scheme [21]. A modified page is written to a different location, and a new version of the parent pages in the tree is created, pointing at the new location. This approach is also useful for concurrency control, as we shall see in “Snapshot Isolation and Repeatable Read”.
- We can save space in pages by not storing the entire key, but abbreviating it. Especially in pages on the interior of the tree, keys only need to provide enough information to act as boundaries between key ranges. Packing more keys into a page allows the tree to have a higher branching factor, and thus fewer levels.
- In general, pages can be positioned anywhere on disk; there is nothing requiring pages with nearby key ranges to be nearby on disk. If a query needs to scan over a large part of the key range in sorted order, that page-by-page layout can be inefficient, because a disk seek may be required for every page that is read. Many B-tree implementations therefore try to lay out the tree so that leaf pages appear in sequential order on disk. However, it’s difficult to maintain that order as the tree grows. By contrast, since LSM-trees rewrite large segments of the storage in one go during merging, it’s easier for them to keep sequential keys close to each other on disk.
- Additional pointers have been added to the tree. For example, each leaf page may have references to its sibling pages to the left and right, which allows scanning keys in order without jumping back to parent pages.
- B-tree variants such as fractal trees [22] borrow some log-structured ideas to reduce disk seeks (and they have nothing to do with fractals).
 - Thay vì ghi đè các trang và duy trì một WAL để khôi phục sau sự cố, một số cơ sở dữ liệu (như LMDB) dùng một cơ chế sao-chép-khi-ghi (copy-on-write) [21]. Một trang đã sửa đổi được ghi tới một vị trí khác, và một phiên bản mới của các trang cha trong cây được tạo ra, trở tới vị trí mới. Cách tiếp cận này cũng hữu ích cho việc kiểm soát tương tranh, như ta sẽ thấy ở phần “Cô lập ảnh chụp và Đọc lặp lại được”.
 - Ta có thể tiết kiệm không gian trong các trang bằng cách không lưu toàn bộ khóa, mà viết tắt nó đi. Đặc biệt trong các trang nằm bên trong cây, các khóa chỉ cần cung cấp

đủ thông tin để đóng vai trò ranh giới giữa các khoảng khóa. Việc nhồi nhiều khóa hơn vào một trang cho phép cây có hệ số phân nhánh cao hơn, và do đó ít cấp hơn.

- Nhìn chung, các trang có thể được đặt ở bất cứ đâu trên đĩa; không có gì bắt buộc các trang có khoảng khóa gần nhau phải nằm gần nhau trên đĩa. Nếu một truy vấn cần quét qua một phần lớn của khoảng khóa theo thứ tự đã sắp xếp, thì cách bố trí từng-trang-một đó có thể kém hiệu quả, bởi mỗi trang được đọc có thể đòi hỏi một lần seek đĩa. Vì vậy nhiều hiện thực B-tree cố gắng bố trí cây sao cho các trang lá xuất hiện theo thứ tự tuần tự trên đĩa. Tuy nhiên, khó mà duy trì được thứ tự đó khi cây lớn lên. Trái lại, vì LSM-tree ghi lại các đoạn lớn của kho lưu trữ trong một lần lúc gộp, nên chúng dễ dàng giữ các khóa tuần tự nằm gần nhau trên đĩa hơn.
- Các con trỏ bổ sung đã được thêm vào cây. Ví dụ, mỗi trang lá có thể có các tham chiếu tới các trang anh em của nó ở bên trái và bên phải, điều này cho phép quét các khóa theo thứ tự mà không cần nhảy ngược về các trang cha.
- Các biến thể B-tree như cây fractal [22] vay mượn một số ý tưởng cấu trúc theo log để giảm các lần seek đĩa (và chúng chẳng liên quan gì tới fractal cả).

Comparing B-Trees and LSM-Trees — So sánh B-tree và LSM-tree

Even though B-tree implementations are generally more mature than LSM-tree implementations, LSM-trees are also interesting due to their performance characteristics. As a rule of thumb, LSM-trees are typically faster for writes, whereas B-trees are thought to be faster for reads [23]. Reads are typically slower on LSM-trees because they have to check several different data structures and SSTables at different stages of compaction.

Mặc dù các hiện thực B-tree nhìn chung trưởng thành hơn các hiện thực LSM-tree, LSM-tree cũng thú vị nhờ các đặc tính hiệu năng của chúng. Theo một quy tắc kinh nghiệm, LSM-tree thường nhanh hơn khi ghi, trong khi B-tree được cho là nhanh hơn khi đọc [23]. Đọc thường chậm hơn trên LSM-tree bởi chúng phải kiểm tra vài cấu trúc dữ liệu và SSTable khác nhau ở các giai đoạn nén khác nhau.

However, benchmarks are often inconclusive and sensitive to details of the workload. You need to test systems with your particular workload in order to make a valid comparison. In this section we will briefly discuss a few things that are worth considering when measuring the performance of a storage engine.

Tuy nhiên, các phép đo chuẩn (benchmark) thường không cho kết luận dứt khoát và nhạy cảm với các chi tiết của khối lượng công việc. Ta cần kiểm thử các hệ thống với khối lượng công việc cụ thể của mình để có một so sánh hợp lệ. Trong phần này, ta sẽ bàn vắn tắt một vài điều đáng cân nhắc khi đo hiệu năng của một engine lưu trữ.

Advantages of LSM-trees — Ưu điểm của LSM-tree

A B-tree index must write every piece of data at least twice: once to the write-ahead log, and once to the tree page itself (and perhaps again as pages are split). There is also overhead from having to write an entire page at a time, even if only a few bytes in that page changed. Some storage engines even overwrite the same page twice in order to avoid ending up with a partially updated page in the event of a power **failure** [24, 25].

Một chỉ mục B-tree phải ghi mỗi mẫu dữ liệu ít nhất hai lần: một lần vào log ghi-trước, và một lần vào chính trang của cây (và có lẽ thêm một lần nữa khi các trang bị tách). Cũng có phụ phí từ việc phải ghi cả một trang mỗi lần, ngay cả khi chỉ vài byte trong trang đó thay

đổi. Một số engine lưu trữ thậm chí ghi đè cùng một trang hai lần để tránh việc cuối cùng có một trang được cập nhật dở dang trong trường hợp mất điện [24, 25].

Log-structured indexes also rewrite data multiple times due to repeated compaction and merging of SSTables. This effect—one write to the database resulting in multiple writes to the disk over the course of the database’s lifetime—is known as write amplification. It is of particular concern on SSDs, which can only overwrite blocks a limited number of times before wearing out.

Các chỉ mục cấu trúc theo log cũng ghi lại dữ liệu nhiều lần do việc nén và gộp các SSTable lặp đi lặp lại. Hiệu ứng này — một lần ghi vào cơ sở dữ liệu dẫn tới nhiều lần ghi xuống đĩa trong suốt vòng đời của cơ sở dữ liệu — được gọi là khuếch đại ghi (write amplification). Nó đặc biệt đáng lo ngại trên SSD, vốn chỉ có thể ghi đè các khối một số lần hữu hạn trước khi bị mòn.

In write-heavy applications, the performance bottleneck might be the rate at which the database can write to disk. In this case, write amplification has a direct performance cost: the more that a storage engine writes to disk, the fewer writes per second it can handle within the available disk bandwidth.

Trong các ứng dụng nặng về ghi, nút thắt cổ chai về hiệu năng có thể là tốc độ mà cơ sở dữ liệu có thể ghi xuống đĩa. Trong trường hợp này, khuếch đại ghi có một chi phí hiệu năng trực tiếp: một engine lưu trữ càng ghi nhiều xuống đĩa, thì nó càng xử lý được ít lần ghi mỗi giây trong phạm vi băng thông đĩa khả dụng.

Moreover, LSM-trees are typically able to sustain higher write throughput than B-trees, partly because they sometimes have lower write amplification (although this depends on the storage engine configuration and workload), and partly because they sequentially write compact SSTable files rather than having to overwrite several pages in the tree [26]. This difference is particularly important on magnetic hard drives, where sequential writes are much faster than random writes.

Hơn nữa, LSM-tree thường có khả năng duy trì thông lượng ghi cao hơn B-tree, một phần vì đôi khi chúng có khuếch đại ghi thấp hơn (mặc dù điều này phụ thuộc vào cấu hình engine lưu trữ và khối lượng công việc), và một phần vì chúng ghi tuần tự các tệp SSTable gọn gàng thay vì phải ghi đè vài trang trong cây [26]. Khác biệt này đặc biệt quan trọng trên các ổ cứng từ, nơi ghi tuần tự nhanh hơn nhiều so với ghi ngẫu nhiên.

LSM-trees can be compressed better, and thus often produce smaller files on disk than B-trees. B-tree storage engines leave some disk space unused due to fragmentation: when a page is split or when a **row** cannot fit into an existing page, some space in a page remains unused. Since LSM-trees are not page-oriented and periodically rewrite SSTables to remove fragmentation, they have lower storage overheads, especially when using leveled compaction [27].

LSM-tree có thể được nén tốt hơn, và do đó thường tạo ra các tệp nhỏ hơn trên đĩa so với B-tree. Các engine lưu trữ B-tree để lại một phần không gian đĩa không dùng đến do phân mảnh: khi một trang bị tách hoặc khi một hàng không vừa với một trang hiện có, một phần không gian trong trang sẽ không được dùng. Vì LSM-tree không hướng trang và định kỳ ghi lại các SSTable để loại bỏ phân mảnh, chúng có phụ phí lưu trữ thấp hơn, đặc biệt khi dùng nén theo cấp [27].

On many SSDs, the firmware internally uses a log-structured algorithm to turn random writes into sequential writes on the underlying storage chips, so the impact of the storage engine’s write pattern is less pronounced [19]. However, lower write amplification and reduced fragmentation are still advantageous on SSDs: representing data more compactly allows more read and write requests within the available I/O bandwidth.

Trên nhiều SSD, firmware bên trong dùng một thuật toán cấu trúc theo log để biến các

lần ghi ngẫu nhiên thành các lần ghi tuần tự trên các chip lưu trữ nền tảng, nên tác động của mẫu ghi của engine lưu trữ ít rõ rệt hơn [19]. Tuy nhiên, khuếch đại ghi thấp hơn và phân mảnh giảm đi vẫn có lợi trên SSD: biểu diễn dữ liệu một cách gọn gàng hơn cho phép nhiều yêu cầu đọc và ghi hơn trong phạm vi băng thông I/O khả dụng.

Downsides of LSM-trees — Nhược điểm của LSM-tree

A downside of log-structured storage is that the compaction process can sometimes interfere with the performance of ongoing reads and writes. Even though storage engines try to perform compaction incrementally and without affecting concurrent access, disks have limited resources, so it can easily happen that a request needs to wait while the disk finishes an expensive compaction operation. The impact on throughput and average **response time** is usually small, but at higher percentiles (see “Describing Performance”) the response time of queries to log-structured storage engines can sometimes be quite high, and B-trees can be more predictable [28].

Một nhược điểm của lưu trữ cấu trúc theo log là quá trình nén đôi khi có thể can thiệp vào hiệu năng của các lần đọc và ghi đang diễn ra. Mặc dù các engine lưu trữ cố gắng thực hiện việc nén theo kiểu tăng dần và không ảnh hưởng tới việc truy cập tương tranh, đĩa có tài nguyên hữu hạn, nên rất dễ xảy ra việc một yêu cầu phải chờ trong khi đĩa hoàn tất một thao tác nén tốn kém. Tác động lên thông lượng và thời gian phản hồi trung bình thường nhỏ, nhưng ở các phân vị cao hơn (xem phần “Mô tả hiệu năng”) thời gian phản hồi của các truy vấn tới các engine lưu trữ cấu trúc theo log đôi khi có thể khá cao, và B-tree có thể dễ dự đoán hơn [28].

Another issue with compaction arises at high write throughput: the disk’s finite write bandwidth needs to be shared between the initial write (logging and flushing a memtable to disk) and the compaction threads running in the background. When writing to an empty database, the full disk bandwidth can be used for the initial write, but the bigger the database gets, the more disk bandwidth is required for compaction.

Một vấn đề khác với việc nén nảy sinh khi thông lượng ghi cao: băng thông ghi hữu hạn của đĩa cần được chia sẻ giữa lần ghi ban đầu (việc ghi log và đẩy một memtable xuống đĩa) và các luồng nén đang chạy ở chế độ nền. Khi ghi vào một cơ sở dữ liệu trống, toàn bộ băng thông đĩa có thể được dùng cho lần ghi ban đầu, nhưng cơ sở dữ liệu càng lớn thì càng cần nhiều băng thông đĩa cho việc nén.

If write throughput is high and compaction is not configured carefully, it can happen that compaction cannot keep up with the rate of incoming writes. In this case, the number of unmerged segments on disk keeps growing until you run out of disk space, and reads also slow down because they need to check more segment files. Typically, SSTable-based storage engines do not throttle the rate of incoming writes, even if compaction cannot keep up, so you need explicit monitoring to detect this situation [29, 30].

Nếu thông lượng ghi cao và việc nén không được cấu hình cẩn thận, có thể xảy ra việc nén không theo kịp tốc độ của các lần ghi đến. Trong trường hợp này, số lượng đoạn chưa gộp trên đĩa cứ tăng lên cho đến khi ta cạn không gian đĩa, và việc đọc cũng chậm đi vì chúng cần kiểm tra nhiều tệp đoạn hơn. Thường thì các engine lưu trữ dựa trên SSTable không điều tiết (throttle) tốc độ của các lần ghi đến, ngay cả khi việc nén không theo kịp, nên ta cần giám sát tường minh để phát hiện tình huống này [29, 30].

An advantage of B-trees is that each key exists in exactly one place in the index, whereas a log-structured storage engine may have multiple copies of the same key in different segments. This aspect makes B-trees attractive in databases that want to offer strong transactional semantics: in

many relational databases, transaction isolation is implemented using locks on ranges of keys, and in a B-tree index, those locks can be directly attached to the tree [5]. In Chapter 7 we will discuss this point in more detail.

Một ưu điểm của B-tree là mỗi khóa tồn tại ở đúng một chỗ trong chỉ mục, trong khi một engine lưu trữ cấu trúc theo log có thể có nhiều bản sao của cùng một khóa ở các đoạn khác nhau. Khía cạnh này khiến B-tree hấp dẫn trong các cơ sở dữ liệu muốn cung cấp ngữ nghĩa giao dịch mạnh: trong nhiều cơ sở dữ liệu quan hệ, việc cô lập giao dịch được hiện thực bằng các khóa (lock) trên các khoảng khóa, và trong một chỉ mục B-tree, các khóa lock đó có thể được gắn trực tiếp vào cây [5]. Ở Chương 7, ta sẽ bàn điểm này chi tiết hơn.

B-trees are very ingrained in the architecture of databases and provide consistently good performance for many workloads, so it's unlikely that they will go away anytime soon. In new datastores, log-structured indexes are becoming increasingly popular. There is no quick and easy rule for determining which type of storage engine is better for your use case, so it is worth testing empirically.

B-tree đã ăn sâu vào kiến trúc của các cơ sở dữ liệu và cung cấp hiệu năng tốt một cách ổn định cho nhiều khối lượng công việc, nên khó có khả năng chúng sẽ biến mất sớm. Trong các kho dữ liệu mới, các chỉ mục cấu trúc theo log đang ngày càng phổ biến. Không có quy tắc nhanh gọn nào để xác định loại engine lưu trữ nào tốt hơn cho tình huống sử dụng của ta, nên việc kiểm thử bằng thực nghiệm là điều đáng làm.

Other Indexing Structures — Các cấu trúc lập chỉ mục khác

So far we have only discussed key-value indexes, which are like a **primary key** index in the **relational model**. A primary key uniquely identifies one row in a relational table, or one document in a document database, or one **vertex** in a **graph** database. Other records in the database can refer to that row/document/vertex by its primary key (or ID), and the index is used to resolve such references.

Cho đến giờ ta mới chỉ bàn về các chỉ mục khóa-giá trị, vốn giống một chỉ mục khóa chính trong mô hình quan hệ. Một khóa chính nhận diện duy nhất một hàng trong một bảng quan hệ, hoặc một tài liệu trong một cơ sở dữ liệu tài liệu, hoặc một đỉnh trong một cơ sở dữ liệu đồ thị. Các bản ghi khác trong cơ sở dữ liệu có thể tham chiếu tới hàng/tài liệu/đỉnh đó bằng khóa chính (hay ID) của nó, và chỉ mục được dùng để phân giải các tham chiếu như vậy.

It is also very common to have secondary indexes. In relational databases, you can create several secondary indexes on the same table using the CREATE INDEX command, and they are often crucial for performing joins efficiently. For example, in Figure 2-1 in Chapter 2 you would most likely have a secondary index on the user_id columns so that you can find all the rows belonging to the same user in each of the tables.

Việc có các chỉ mục phụ (secondary index) cũng rất phổ biến. Trong các cơ sở dữ liệu quan hệ, ta có thể tạo vài chỉ mục phụ trên cùng một bảng bằng lệnh CREATE INDEX, và chúng thường có vai trò then chốt trong việc thực hiện các phép join một cách hiệu quả. Ví dụ, ở Hình 2-1 trong Chương 2, rất có thể ta sẽ có một chỉ mục phụ trên các cột user_id để có thể tìm tất cả các hàng thuộc về cùng một người dùng trong mỗi bảng.

A secondary index can easily be constructed from a key-value index. The main difference is that keys are not unique; i.e., there might be many rows (documents, vertices) with the same key. This can be

solved in two ways: either by making each value in the index a list of matching row identifiers (like a postings list in a full-text index) or by making each key unique by appending a row identifier to it. Either way, both B-trees and log-structured indexes can be used as secondary indexes.

Một chỉ mục phụ có thể dễ dàng được xây dựng từ một chỉ mục khóa-giá trị. Khác biệt chính là các khóa không duy nhất; tức là có thể có nhiều hàng (tài liệu, đỉnh) cùng một khóa. Điều này có thể giải quyết theo hai cách: hoặc làm cho mỗi giá trị trong chỉ mục là một danh sách các định danh hàng khớp (giống một postings list trong một chỉ mục toàn văn), hoặc làm cho mỗi khóa trở nên duy nhất bằng cách ghép thêm một định danh hàng vào nó. Dù theo cách nào, cả B-tree lẫn các chỉ mục cấu trúc theo log đều có thể được dùng làm chỉ mục phụ.

Storing values within the index — Lưu trữ giá trị bên trong chỉ mục

The key in an index is the thing that queries search for, but the value can be one of two things: it could be the actual row (document, vertex) in question, or it could be a reference to the row stored elsewhere. In the latter case, the place where rows are stored is known as a heap file, and it stores data in no particular order (it may be append-only, or it may keep track of deleted rows in order to overwrite them with new data later). The heap file approach is common because it avoids duplicating data when multiple secondary indexes are present: each index just references a location in the heap file, and the actual data is kept in one place.

Khóa trong một chỉ mục là thứ mà các truy vấn tìm kiếm, nhưng giá trị có thể là một trong hai thứ: nó có thể là chính hàng (tài liệu, đỉnh) đang xét, hoặc nó có thể là một tham chiếu tới hàng được lưu ở nơi khác. Trong trường hợp sau, nơi các hàng được lưu được gọi là một tệp heap (heap file), và nó lưu dữ liệu không theo thứ tự cụ thể nào (nó có thể là chỉ-ghi-nối, hoặc có thể theo dõi các hàng đã xóa để ghi đè chúng bằng dữ liệu mới sau này). Cách tiếp cận tệp heap phổ biến vì nó tránh nhân bản dữ liệu khi có nhiều chỉ mục phụ: mỗi chỉ mục chỉ tham chiếu một vị trí trong tệp heap, còn dữ liệu thực tế được giữ ở một chỗ.

When updating a value without changing the key, the heap file approach can be quite efficient: the record can be overwritten in place, provided that the new value is not larger than the old value. The situation is more complicated if the new value is larger, as it probably needs to be moved to a new location in the heap where there is enough space. In that case, either all indexes need to be updated to point at the new heap location of the record, or a forwarding pointer is left behind in the old heap location [5].

Khi cập nhật một giá trị mà không thay đổi khóa, cách tiếp cận tệp heap có thể khá hiệu quả: bản ghi có thể được ghi đè tại chỗ, miễn là giá trị mới không lớn hơn giá trị cũ. Tình huống phức tạp hơn nếu giá trị mới lớn hơn, vì nó có lẽ cần được chuyển tới một vị trí mới trong heap nơi có đủ không gian. Trong trường hợp đó, hoặc tất cả các chỉ mục cần được cập nhật để trỏ tới vị trí heap mới của bản ghi, hoặc một con trỏ chuyển tiếp (forwarding pointer) được để lại ở vị trí heap cũ [5].

In some situations, the extra hop from the index to the heap file is too much of a performance penalty for reads, so it can be desirable to store the indexed row directly within an index. This is known as a clustered index. For example, in MySQL's InnoDB storage engine, the primary key of a table is always a clustered index, and secondary indexes refer to the primary key (rather than a heap file location) [31]. In **SQL** Server, you can specify one clustered index per table [32].

Trong một số tình huống, bước nhảy thêm từ chỉ mục tới tệp heap là một hình phạt hiệu năng quá lớn đối với việc đọc, nên có thể muốn lưu hàng được lập chỉ mục trực tiếp bên

trong một chỉ mục. Điều này được gọi là một chỉ mục gom cụm (clustered index). Ví dụ, trong engine lưu trữ InnoDB của MySQL, khóa chính của một bảng luôn là một chỉ mục gom cụm, và các chỉ mục phụ tham chiếu tới khóa chính (chứ không phải một vị trí tệp heap) [31]. Trong SQL Server, ta có thể chỉ định một chỉ mục gom cụm cho mỗi bảng [32].

A compromise between a clustered index (storing all row data within the index) and a nonclustered index (storing only references to the data within the index) is known as a covering index or index with included columns, which stores some of a table's columns within the index [33]. This allows some queries to be answered by using the index alone (in which case, the index is said to cover the query) [32].

Một sự thỏa hiệp giữa chỉ mục gom cụm (lưu toàn bộ dữ liệu hàng bên trong chỉ mục) và chỉ mục không gom cụm (chỉ lưu các tham chiếu tới dữ liệu bên trong chỉ mục) được gọi là một chỉ mục bao phủ (covering index) hoặc chỉ mục có các cột đính kèm, vốn lưu một số cột của bảng bên trong chỉ mục [33]. Điều này cho phép một số truy vấn được trả lời chỉ bằng cách dùng chỉ mục (trong trường hợp đó, chỉ mục được nói là bao phủ truy vấn) [32].

As with any kind of duplication of data, clustered and covering indexes can speed up reads, but they require additional storage and can add overhead on writes. Databases also need to go to additional effort to enforce transactional guarantees, because applications should not see inconsistencies due to the duplication.

Như với bất kỳ kiểu nhân bản dữ liệu nào, chỉ mục gom cụm và chỉ mục bao phủ có thể tăng tốc việc đọc, nhưng chúng đòi hỏi thêm không gian lưu trữ và có thể tạo thêm phụ phí khi ghi. Các cơ sở dữ liệu cũng cần bỏ thêm công sức để thực thi các đảm bảo giao dịch, bởi các ứng dụng không nên thấy những bất nhất do việc nhân bản gây ra.

Multi-column indexes — Chỉ mục đa cột

The indexes discussed so far only map a single key to a value. That is not sufficient if we need to query multiple columns of a table (or multiple fields in a document) simultaneously.

Các chỉ mục đã bàn từ đầu đến giờ chỉ ánh xạ một khóa đơn tới một giá trị. Điều đó không đủ nếu ta cần truy vấn nhiều cột của một bảng (hoặc nhiều trường trong một tài liệu) đồng thời.

The most common type of multi-column index is called a concatenated index, which simply combines several fields into one key by appending one column to another (the index definition specifies in which order the fields are concatenated). This is like an old-fashioned paper phone book, which provides an index from (lastname, firstname) to phone number. Due to the sort order, the index can be used to find all the people with a particular last name, or all the people with a particular lastname-firstname combination. However, the index is useless if you want to find all the people with a particular first name.

Loại chỉ mục đa cột phổ biến nhất được gọi là chỉ mục ghép (concatenated index), vốn đơn giản kết hợp vài trường thành một khóa bằng cách ghép cột này với cột khác (định nghĩa chỉ mục chỉ rõ các trường được ghép theo thứ tự nào). Điều này giống một cuốn danh bạ điện thoại bằng giấy kiểu cũ, vốn cung cấp một chỉ mục từ (họ, tên) tới số điện thoại. Nhờ thứ tự sắp xếp, chỉ mục có thể được dùng để tìm tất cả những người có một họ cụ thể, hoặc tất cả những người có một tổ hợp họ-tên cụ thể. Tuy nhiên, chỉ mục trở nên vô dụng nếu ta muốn tìm tất cả những người có một tên cụ thể.

Multi-dimensional indexes are a more general way of querying several columns at once, which is

particularly important for geospatial data. For example, a restaurant-search website may have a database containing the latitude and longitude of each restaurant. When a user is looking at the restaurants on a map, the website needs to search for all the restaurants within the rectangular map area that the user is currently viewing. This requires a two-dimensional range query like the following:

Chỉ mục đa chiều (multi-dimensional index) là một cách tổng quát hơn để truy vấn nhiều cột cùng lúc, điều này đặc biệt quan trọng với dữ liệu địa không gian. Ví dụ, một website tìm kiếm nhà hàng có thể có một cơ sở dữ liệu chứa vĩ độ và kinh độ của mỗi nhà hàng. Khi một người dùng đang xem các nhà hàng trên bản đồ, website cần tìm tất cả các nhà hàng nằm trong vùng bản đồ hình chữ nhật mà người dùng đang xem. Điều này đòi hỏi một truy vấn theo khoảng hai chiều như sau:

```
SELECT * FROM restaurants WHERE latitude > 51.4946 AND latitude < 51.5079
                                AND longitude > -0.1162 AND longitude < -0.1004;
```

A standard B-tree or LSM-tree index is not able to answer that kind of query efficiently: it can give you either all the restaurants in a range of latitudes (but at any longitude), or all the restaurants in a range of longitudes (but anywhere between the North and South poles), but not both simultaneously.

Một chỉ mục B-tree hoặc LSM-tree tiêu chuẩn không thể trả lời hiệu quả kiểu truy vấn đó: nó có thể cho ta hoặc tất cả các nhà hàng trong một khoảng vĩ độ (nhưng ở bất kỳ kinh độ nào), hoặc tất cả các nhà hàng trong một khoảng kinh độ (nhưng ở bất kỳ đâu giữa Cực Bắc và Cực Nam), chứ không phải cả hai đồng thời.

One option is to translate a two-dimensional location into a single number using a space-filling curve, and then to use a regular B-tree index [34]. More commonly, specialized spatial indexes such as R-trees are used. For example, PostGIS implements geospatial indexes as R-trees using PostgreSQL's Generalized Search Tree indexing facility [35]. We don't have space to describe R-trees in detail here, but there is plenty of literature on them.

Một lựa chọn là dịch một vị trí hai chiều thành một con số duy nhất bằng cách dùng một đường cong lấp đầy không gian (space-filling curve), rồi dùng một chỉ mục B-tree thông thường [34]. Phổ biến hơn, các chỉ mục không gian chuyên biệt như R-tree được dùng. Ví dụ, PostGIS hiện thực các chỉ mục địa không gian dưới dạng R-tree bằng tiện ích lập chỉ mục Generalized Search Tree của PostgreSQL [35]. Ta không có chỗ để mô tả R-tree chi tiết ở đây, nhưng có rất nhiều tài liệu về chúng.

An interesting idea is that multi-dimensional indexes are not just for geographic locations. For example, on an ecommerce website you could use a three-dimensional index on the dimensions (red, green, blue) to search for products in a certain range of colors, or in a database of weather observations you could have a two-dimensional index on (date, temperature) in order to efficiently search for all the observations during the year 2013 where the temperature was between 25 and 30°C. With a one-dimensional index, you would have to either scan over all the records from 2013 (regardless of temperature) and then filter them by temperature, or vice versa. A 2D index could narrow down by timestamp and temperature simultaneously. This technique is used by HyperDex [36].

Một ý tưởng thú vị là các chỉ mục đa chiều không chỉ dành cho các vị trí địa lý. Ví dụ, trên một website thương mại điện tử ta có thể dùng một chỉ mục ba chiều trên các chiều (đỏ, lục, lam) để tìm các sản phẩm trong một khoảng màu nhất định, hoặc trong một cơ sở dữ liệu các quan trắc thời tiết ta có thể có một chỉ mục hai chiều trên (ngày, nhiệt độ) để tìm hiệu quả tất cả các quan trắc trong năm 2013 mà nhiệt độ nằm giữa 25 và 30°C. Với một chỉ mục một chiều, ta sẽ phải hoặc quét qua tất cả các bản ghi của năm 2013 (bất kể nhiệt

độ) rồi lọc chúng theo nhiệt độ, hoặc ngược lại. Một chỉ mục 2D có thể thu hẹp theo dấu thời gian và nhiệt độ đồng thời. Kỹ thuật này được HyperDex sử dụng [36].

Full-text search and fuzzy indexes — Tìm kiếm toàn văn và chỉ mục mờ

All the indexes discussed so far assume that you have exact data and allow you to query for exact values of a key, or a range of values of a key with a sort order. What they don't allow you to do is search for similar keys, such as misspelled words. Such fuzzy querying requires different techniques.

Tất cả các chỉ mục đã bàn từ đầu đến giờ đều giả định rằng ta có dữ liệu chính xác và cho phép ta truy vấn các giá trị chính xác của một khóa, hoặc một khoảng giá trị của một khóa theo một thứ tự sắp xếp. Thứ chúng không cho phép ta làm là tìm kiếm các khóa tương tự, chẳng hạn các từ bị viết sai chính tả. Kiểu truy vấn mờ (fuzzy) như vậy đòi hỏi các kỹ thuật khác.

For example, full-text search engines commonly allow a search for one word to be expanded to include synonyms of the word, to ignore grammatical variations of words, and to search for occurrences of words near each other in the same document, and support various other features that depend on linguistic analysis of the text. To cope with typos in documents or queries, Lucene is able to search text for words within a certain edit distance (an edit distance of 1 means that one letter has been added, removed, or replaced) [37].

Ví dụ, các engine tìm kiếm toàn văn thường cho phép một tìm kiếm cho một từ được mở rộng để bao gồm các từ đồng nghĩa với từ đó, để bỏ qua các biến thể ngữ pháp của từ, và để tìm các lần xuất hiện của các từ gần nhau trong cùng một tài liệu, cùng nhiều tính năng khác phụ thuộc vào phân tích ngôn ngữ học của văn bản. Để đối phó với lỗi chính tả trong tài liệu hoặc truy vấn, Lucene có thể tìm kiếm trong văn bản các từ nằm trong một khoảng cách chỉnh sửa (edit distance) nhất định (khoảng cách chỉnh sửa bằng 1 nghĩa là một chữ cái đã được thêm vào, bỏ đi, hoặc thay thế) [37].

As mentioned in “Making an LSM-tree out of SSTables”, Lucene uses a SSTable-like structure for its term dictionary. This structure requires a small in-memory index that tells queries at which offset in the sorted file they need to look for a key. In LevelDB, this in-memory index is a sparse collection of some of the keys, but in Lucene, the in-memory index is a finite state automaton over the characters in the keys, similar to a trie [38]. This automaton can be transformed into a Levenshtein automaton, which supports efficient search for words within a given edit distance [39].

Như đã nhắc tới ở phần “Tạo một LSM-tree từ các SSTable”, Lucene dùng một cấu trúc kiểu SSTable cho từ điển thuật ngữ của nó. Cấu trúc này đòi hỏi một chỉ mục nhỏ trong bộ nhớ cho biết các truy vấn cần tìm một khóa ở độ lệch nào trong tệp đã sắp xếp. Trong LevelDB, chỉ mục trong bộ nhớ này là một tập hợp thưa gồm một số khóa, nhưng trong Lucene, chỉ mục trong bộ nhớ là một ô-tô-mát hữu hạn trạng thái (finite state automaton) trên các ký tự trong các khóa, tương tự một trie [38]. Ô-tô-mát này có thể được biến đổi thành một ô-tô-mát Levenshtein, vốn hỗ trợ tìm kiếm hiệu quả các từ nằm trong một khoảng cách chỉnh sửa cho trước [39].

Other fuzzy search techniques go in the direction of document classification and machine learning. See an information retrieval textbook for more detail [e.g., 40].

Các kỹ thuật tìm kiếm mờ khác đi theo hướng phân loại tài liệu và học máy. Xem một cuốn sách giáo khoa về truy hồi thông tin để biết thêm chi tiết [ví dụ, 40].

Keeping everything in memory — Giữ mọi thứ trong bộ nhớ

The data structures discussed so far in this chapter have all been answers to the limitations of disks. Compared to main memory, disks are awkward to deal with. With both magnetic disks and SSDs, data on disk needs to be laid out carefully if you want good performance on reads and writes. However, we tolerate this awkwardness because disks have two significant advantages: they are durable (their contents are not lost if the power is turned off), and they have a lower cost per gigabyte than RAM.

Các cấu trúc dữ liệu đã bàn từ đầu chương này đến giờ đều là những lời giải cho các hạn chế của đĩa. So với bộ nhớ chính, đĩa khó xử lý. Với cả đĩa từ lẫn SSD, dữ liệu trên đĩa cần được bố trí cẩn thận nếu ta muốn có hiệu năng tốt khi đọc và ghi. Tuy nhiên, ta chịu đựng sự khó xử này bởi đĩa có hai ưu điểm đáng kể: chúng có độ bền (nội dung của chúng không bị mất nếu tắt nguồn), và chúng có chi phí trên mỗi gigabyte thấp hơn RAM.

As RAM becomes cheaper, the cost-per-gigabyte argument is eroded. Many datasets are simply not that big, so it's quite feasible to keep them entirely in memory, potentially distributed across several machines. This has led to the development of in-memory databases.

Khi RAM trở nên rẻ hơn, lập luận về chi phí trên mỗi gigabyte bị xói mòn. Nhiều tập dữ liệu đơn giản là không lớn đến vậy, nên việc giữ chúng hoàn toàn trong bộ nhớ là khả thi, có thể phân tán trên vài máy. Điều này đã dẫn tới sự phát triển của các cơ sở dữ liệu trong bộ nhớ (in-memory database).

Some in-memory key-value stores, such as Memcached, are intended for caching use only, where it's acceptable for data to be lost if a machine is restarted. But other in-memory databases aim for **durability**, which can be achieved with special hardware (such as battery-powered RAM), by writing a log of changes to disk, by writing periodic snapshots to disk, or by replicating the in-memory state to other machines.

Một số kho khóa-giá trị trong bộ nhớ, chẳng hạn Memcached, chỉ nhằm dùng cho việc đệm, nơi việc mất dữ liệu khi một máy được khởi động lại là chấp nhận được. Nhưng các cơ sở dữ liệu trong bộ nhớ khác lại nhắm tới độ bền, vốn có thể đạt được bằng phần cứng đặc biệt (chẳng hạn RAM nuôi bằng pin), bằng việc ghi một log các thay đổi xuống đĩa, bằng việc ghi các ảnh chụp định kỳ xuống đĩa, hoặc bằng việc sao chép trạng thái trong bộ nhớ sang các máy khác.

When an in-memory database is restarted, it needs to reload its state, either from disk or over the network from a replica (unless special hardware is used). Despite writing to disk, it's still an in-memory database, because the disk is merely used as an append-only log for durability, and reads are served entirely from memory. Writing to disk also has operational advantages: files on disk can easily be backed up, inspected, and analyzed by external utilities.

Khi một cơ sở dữ liệu trong bộ nhớ được khởi động lại, nó cần nạp lại trạng thái của mình, hoặc từ đĩa hoặc qua mạng từ một bản sao (trừ khi dùng phần cứng đặc biệt). Dù có ghi xuống đĩa, nó vẫn là một cơ sở dữ liệu trong bộ nhớ, bởi đĩa chỉ được dùng như một log chỉ-ghi-nối cho độ bền, còn việc đọc được phục vụ hoàn toàn từ bộ nhớ. Việc ghi xuống đĩa cũng có những lợi thế vận hành: các tệp trên đĩa có thể dễ dàng được sao lưu, kiểm tra, và phân tích bởi các tiện ích bên ngoài.

Products such as VoltDB, MemSQL, and Oracle TimesTen are in-memory databases with a relational model, and the vendors claim that they can offer big performance improvements by removing all the overheads associated with managing on-disk data structures [41, 42]. RAMCloud is an open source, in-memory key-value store with durability (using a log-structured approach for the data in memory as well as the data on disk) [43]. Redis and Couchbase provide weak durability by writing to disk asynchronously.

Các sản phẩm như VoltDB, MemSQL, và Oracle TimesTen là các cơ sở dữ liệu trong bộ nhớ với một mô hình quan hệ, và các nhà cung cấp tuyên bố rằng chúng có thể đem lại những cải thiện hiệu năng lớn bằng cách loại bỏ tất cả các phụ phí gắn với việc quản lý các cấu trúc dữ liệu trên đĩa [41, 42]. RAMCloud là một kho khóa-giá trị trong bộ nhớ mã nguồn mở có độ bền (dùng một cách tiếp cận cấu trúc theo log cho cả dữ liệu trong bộ nhớ lẫn dữ liệu trên đĩa) [43]. Redis và Couchbase cung cấp độ bền yếu bằng cách ghi xuống đĩa một cách bất đồng bộ.

Counterintuitively, the performance advantage of in-memory databases is not due to the fact that they don't need to read from disk. Even a disk-based storage engine may never need to read from disk if you have enough memory, because the operating system caches recently used disk blocks in memory anyway. Rather, they can be faster because they can avoid the overheads of encoding in-memory data structures in a form that can be written to disk [44].

Một cách phản trực giác, ưu thế hiệu năng của các cơ sở dữ liệu trong bộ nhớ không phải nhờ việc chúng không cần đọc từ đĩa. Ngay cả một engine lưu trữ dựa trên đĩa cũng có thể không bao giờ cần đọc từ đĩa nếu ta có đủ bộ nhớ, bởi dù sao hệ điều hành cũng đệm các khối đĩa được dùng gần đây trong bộ nhớ. Đúng hơn, chúng có thể nhanh hơn bởi chúng có thể tránh được các phụ phí của việc mã hóa các cấu trúc dữ liệu trong bộ nhớ thành một dạng có thể ghi xuống đĩa [44].

Besides performance, another interesting area for in-memory databases is providing data models that are difficult to implement with disk-based indexes. For example, Redis offers a database-like interface to various data structures such as priority queues and sets. Because it keeps all data in memory, its implementation is comparatively simple.

Bên cạnh hiệu năng, một lĩnh vực thú vị khác đối với các cơ sở dữ liệu trong bộ nhớ là cung cấp các mô hình dữ liệu vốn khó hiện thực với các chỉ mục dựa trên đĩa. Ví dụ, Redis cung cấp một giao diện kiểu cơ sở dữ liệu cho nhiều cấu trúc dữ liệu khác nhau như hàng đợi ưu tiên và tập hợp. Bởi nó giữ toàn bộ dữ liệu trong bộ nhớ, hiện thực của nó tương đối đơn giản.

Recent research indicates that an in-memory database architecture could be extended to support datasets larger than the available memory, without bringing back the overheads of a disk-centric architecture [45]. The so-called anti-caching approach works by evicting the least recently used data from memory to disk when there is not enough memory, and loading it back into memory when it is accessed again in the future. This is similar to what operating systems do with virtual memory and swap files, but the database can manage memory more efficiently than the OS, as it can work at the granularity of individual records rather than entire memory pages. This approach still requires indexes to fit entirely in memory, though (like the Bitcask example at the beginning of the chapter).

Nghiên cứu gần đây cho thấy một kiến trúc cơ sở dữ liệu trong bộ nhớ có thể được mở rộng để hỗ trợ các tập dữ liệu lớn hơn bộ nhớ khả dụng, mà không kéo theo các phụ phí của một kiến trúc lấy đĩa làm trung tâm [45]. Cách tiếp cận gọi là chống-đệm (anti-caching) hoạt động bằng cách đẩy dữ liệu ít được dùng gần đây nhất từ bộ nhớ ra đĩa khi không đủ bộ nhớ, và nạp nó trở lại bộ nhớ khi nó được truy cập lại trong tương lai. Điều này tương tự những gì hệ điều hành làm với bộ nhớ ảo và tệp trao đổi (swap), nhưng cơ sở dữ liệu có thể quản lý bộ nhớ hiệu quả hơn hệ điều hành, bởi nó có thể làm việc ở độ chi tiết của từng bản ghi riêng lẻ thay vì cả trang bộ nhớ. Tuy vậy, cách tiếp cận này vẫn đòi hỏi các chỉ mục phải vừa hoàn toàn trong bộ nhớ (giống ví dụ Bitcask ở đầu chương).

Further changes to storage engine design will probably be needed if non-volatile memory (NVM) technologies become more widely adopted [46]. At present, this is a new area of research, but it is worth keeping an eye on in the future.

Các thay đổi tiếp theo đối với thiết kế engine lưu trữ có lẽ sẽ cần thiết nếu các công nghệ bộ nhớ không khả biến (non-volatile memory, NVM) trở nên được áp dụng rộng rãi hơn [46]. Ở thời điểm hiện tại, đây là một lĩnh vực nghiên cứu mới, nhưng đáng để mắt tới trong tương lai.

Transaction Processing or Analytics? — Xử lý giao dịch hay Phân tích?

In the early days of business data processing, a write to the database typically corresponded to a commercial transaction taking place: making a sale, placing an order with a supplier, paying an employee's salary, etc. As databases expanded into areas that didn't involve money changing hands, the term transaction nevertheless stuck, referring to a group of reads and writes that form a logical unit.

Trong những ngày đầu của xử lý dữ liệu nghiệp vụ, một lần ghi vào cơ sở dữ liệu thường tương ứng với một giao dịch thương mại đang diễn ra: thực hiện một thương vụ bán hàng, đặt một đơn hàng với nhà cung cấp, trả lương cho một nhân viên, v.v. Khi các cơ sở dữ liệu mở rộng sang những lĩnh vực không liên quan tới việc tiền bạc đổi chủ, thuật ngữ giao dịch (transaction) dù vậy vẫn được giữ lại, ám chỉ một nhóm các thao tác đọc và ghi tạo thành một đơn vị logic.

Note — Ghi chú A transaction needn't necessarily have ACID (atomicity, consistency, isolation, and durability) properties. Transaction processing just means allowing clients to make low-**latency** reads and writes—as opposed to **batch processing** jobs, which only run periodically (for example, once per day). We discuss the ACID properties in Chapter 7 and batch processing in Chapter 10.

Một giao dịch không nhất thiết phải có các thuộc tính ACID (tính nguyên tử, tính nhất quán, tính cô lập, và độ bền). Xử lý giao dịch chỉ đơn thuần có nghĩa là cho phép các client thực hiện các lần đọc và ghi với độ trễ thấp — trái ngược với các tác vụ xử lý theo lô, vốn chỉ chạy định kỳ (ví dụ, mỗi ngày một lần). Ta sẽ bàn về các thuộc tính ACID ở Chương 7 và xử lý theo lô ở Chương 10.

Even though databases started being used for many different kinds of data—comments on blog posts, actions in a game, contacts in an address book, etc.—the basic **access pattern** remained similar to processing business transactions. An application typically looks up a small number of records by some key, using an index. Records are inserted or updated based on the user's input. Because these applications are interactive, the access pattern became known as online transaction processing (OLTP).

Mặc dù các cơ sở dữ liệu bắt đầu được dùng cho nhiều loại dữ liệu khác nhau — bình luận trên các bài blog, hành động trong một trò chơi, các liên hệ trong một sổ địa chỉ, v.v. — mẫu truy cập cơ bản vẫn tương tự việc xử lý các giao dịch nghiệp vụ. Một ứng dụng thường tra cứu một số lượng nhỏ bản ghi theo một khóa nào đó, dùng một chỉ mục. Các bản ghi được chèn hoặc cập nhật dựa trên đầu vào của người dùng. Vì các ứng dụng này có tính tương

tác, mẫu truy cập đó được biết đến với tên xử lý giao dịch trực tuyến (online transaction processing, OLTP).

However, databases also started being increasingly used for data analytics, which has very different access patterns. Usually an analytic query needs to scan over a huge number of records, only reading a few columns per record, and calculates aggregate statistics (such as count, sum, or average) rather than returning the raw data to the user. For example, if your data is a table of sales transactions, then analytic queries might be:

Tuy nhiên, các cơ sở dữ liệu cũng bắt đầu ngày càng được dùng cho phân tích dữ liệu, vốn có các mẫu truy cập rất khác. Thường thì một truy vấn phân tích cần quét qua một số lượng khổng lồ bản ghi, chỉ đọc một vài cột mỗi bản ghi, và tính các thống kê tổng hợp (chẳng hạn đếm, tổng, hoặc trung bình) thay vì trả về dữ liệu thô cho người dùng. Ví dụ, nếu dữ liệu của ta là một bảng các giao dịch bán hàng, thì các truy vấn phân tích có thể là:

- What was the total revenue of each of our stores in January?
- How many more bananas than usual did we sell during our latest promotion?
- Which brand of baby food is most often purchased together with brand X diapers?
 - Tổng doanh thu của mỗi cửa hàng của chúng ta trong tháng Một là bao nhiêu?
 - Trong đợt khuyến mãi gần nhất, chúng ta đã bán nhiều hơn bình thường bao nhiêu quả chuối?
 - Thương hiệu thức ăn trẻ em nào hay được mua kèm với tã hiệu X nhất?

These queries are often written by business analysts, and feed into reports that help the management of a company make better decisions (business intelligence). In order to differentiate this pattern of using databases from transaction processing, it has been called online analytic processing (OLAP) [47]. The difference between OLTP and OLAP is not always clear-cut, but some typical characteristics are listed in Table 3-1.

Các truy vấn này thường được viết bởi các nhà phân tích nghiệp vụ, và đưa vào các báo cáo giúp ban quản lý của một công ty ra quyết định tốt hơn (trí tuệ nghiệp vụ, business intelligence). Để phân biệt mẫu sử dụng cơ sở dữ liệu này với xử lý giao dịch, nó được gọi là xử lý phân tích trực tuyến (online analytic processing, OLAP) [47]. Khác biệt giữa OLTP và OLAP không phải lúc nào cũng rạch ròi, nhưng một số đặc tính điển hình được liệt kê ở Bảng 3-1.

Property	Transaction processing systems (OLTP)	Analytic systems (OLAP)
Main read pattern	Small number of records per query, fetched by key	Aggregate over large number of records
Main write pattern	Random-access, low-latency writes from user input	Bulk import (ETL) or event stream
Primarily used by	End user/customer, via web application	Internal analyst, for decision support
What data represents	Latest state of data (current point in time)	History of events that happened over time
Dataset size	Gigabytes to terabytes	Terabytes to petabytes

At first, the same databases were used for both transaction processing and analytic queries. SQL turned out to be quite flexible in this regard: it works well for OLTP-type queries as well as OLAP-type queries. Nevertheless, in the late 1980s and early 1990s, there was a trend for companies to stop

using their OLTP systems for analytics purposes, and to run the analytics on a separate database instead. This separate database was called a data warehouse.

Lúc đầu, cùng những cơ sở dữ liệu được dùng cho cả xử lý giao dịch lẫn các truy vấn phân tích. SQL hóa ra khá linh hoạt về mặt này: nó hoạt động tốt cho các truy vấn kiểu OLTP cũng như các truy vấn kiểu OLAP. Tuy vậy, vào cuối thập niên 1980 và đầu thập niên 1990, có một xu hướng các công ty ngừng dùng các hệ thống OLTP của họ cho mục đích phân tích, và chạy phân tích trên một cơ sở dữ liệu riêng để thay thế. Cơ sở dữ liệu riêng này được gọi là một kho dữ liệu (data warehouse).

Data Warehousing — Kho dữ liệu

An enterprise may have dozens of different transaction processing systems: systems powering the customer-facing website, controlling point of sale (checkout) systems in physical stores, tracking inventory in warehouses, planning routes for vehicles, managing suppliers, administering employees, etc. Each of these systems is complex and needs a team of people to maintain it, so the systems end up operating mostly autonomously from each other.

Một doanh nghiệp có thể có hàng chục hệ thống xử lý giao dịch khác nhau: các hệ thống vận hành website hướng tới khách hàng, điều khiển các hệ thống điểm bán hàng (quầy thanh toán) ở các cửa hàng vật lý, theo dõi tồn kho trong các nhà kho, hoạch định tuyến đường cho các phương tiện, quản lý nhà cung cấp, quản trị nhân viên, v.v. Mỗi hệ thống trong số này đều phức tạp và cần một đội ngũ người để bảo trì, nên rất cuộc các hệ thống vận hành phần lớn độc lập với nhau.

These OLTP systems are usually expected to be highly available and to process transactions with low latency, since they are often critical to the operation of the business. Database administrators therefore closely guard their OLTP databases. They are usually reluctant to let business analysts run ad hoc analytic queries on an OLTP database, since those queries are often expensive, scanning large parts of the dataset, which can harm the performance of concurrently executing transactions.

Các hệ thống OLTP này thường được kỳ vọng có tính sẵn sàng cao và xử lý các giao dịch với độ trễ thấp, vì chúng thường có vai trò then chốt đối với hoạt động của doanh nghiệp. Do đó các quản trị viên cơ sở dữ liệu canh giữ chặt chẽ các cơ sở dữ liệu OLTP của họ. Họ thường ngần ngại để các nhà phân tích nghiệp vụ chạy các truy vấn phân tích tùy ý (ad hoc) trên một cơ sở dữ liệu OLTP, bởi các truy vấn đó thường tốn kém, quét những phần lớn của tập dữ liệu, điều này có thể làm hại hiệu năng của các giao dịch đang thực thi đồng thời.

A data warehouse, by contrast, is a separate database that analysts can query to their hearts' content, without affecting OLTP operations [48]. The data warehouse contains a read-only copy of the data in all the various OLTP systems in the company. Data is extracted from OLTP databases (using either a periodic data dump or a continuous stream of updates), transformed into an analysis-friendly **schema**, cleaned up, and then loaded into the data warehouse. This process of getting data into the warehouse is known as Extract–Transform–**Load** (ETL) and is illustrated in Figure 3-8.

Một kho dữ liệu, trái lại, là một cơ sở dữ liệu riêng mà các nhà phân tích có thể truy vấn thỏa thích, mà không ảnh hưởng tới các hoạt động OLTP [48]. Kho dữ liệu chứa một bản sao chỉ-đọc của dữ liệu trong tất cả các hệ thống OLTP khác nhau trong công ty. Dữ liệu được trích xuất từ các cơ sở dữ liệu OLTP (dùng hoặc một lần kết xuất dữ liệu (data dump) định kỳ hoặc một luồng cập nhật liên tục), biến đổi thành một lược đồ thân thiện với phân tích, được dọn dẹp, rồi được nạp vào kho dữ liệu. Quá trình đưa dữ liệu vào kho này được

gọi là Trích xuất–Biến đổi–Nạp (Extract–Transform–Load, ETL) và được minh họa ở Hình 3-8.

Figure 3-8. Simplified outline of ETL into a data warehouse. — Hình 3-8. Sơ đồ phác thảo đơn giản của ETL vào một kho dữ liệu. Data warehouses now exist in almost all large enterprises, but in small companies they are almost unheard of. This is probably because most small companies don't have so many different OLTP systems, and most small companies have a small amount of data—small enough that it can be queried in a conventional SQL database, or even analyzed in a spreadsheet. In a large company, a lot of heavy lifting is required to do something that is simple in a small company.

Kho dữ liệu giờ đây tồn tại ở gần như tất cả các doanh nghiệp lớn, nhưng ở các công ty nhỏ thì gần như chưa từng nghe tới. Điều này có lẽ là vì hầu hết các công ty nhỏ không có nhiều hệ thống OLTP khác nhau đến vậy, và hầu hết các công ty nhỏ có một lượng dữ liệu nhỏ — đủ nhỏ để có thể truy vấn trong một cơ sở dữ liệu SQL thông thường, hoặc thậm chí phân tích trong một bảng tính. Ở một công ty lớn, cần rất nhiều công sức nặng nhọc để làm một việc vốn đơn giản ở một công ty nhỏ.

A big advantage of using a separate data warehouse, rather than querying OLTP systems directly for analytics, is that the data warehouse can be optimized for analytic access patterns. It turns out that the indexing algorithms discussed in the first half of this chapter work well for OLTP, but are not very good at answering analytic queries. In the rest of this chapter we will look at storage engines that are optimized for analytics instead.

Một ưu điểm lớn của việc dùng một kho dữ liệu riêng, thay vì truy vấn trực tiếp các hệ thống OLTP để phân tích, là kho dữ liệu có thể được tối ưu cho các mẫu truy cập phân tích. Hóa ra các thuật toán lập chỉ mục đã bàn ở nửa đầu chương này hoạt động tốt cho OLTP, nhưng không giỏi lắm trong việc trả lời các truy vấn phân tích. Trong phần còn lại của chương này, ta sẽ thay vào đó xem xét các engine lưu trữ được tối ưu cho phân tích.

The divergence between OLTP databases and data warehouses — Sự phân kỳ giữa cơ sở dữ liệu OLTP và kho dữ liệu

The **data model** of a data warehouse is most commonly relational, because SQL is generally a good fit for analytic queries. There are many graphical data analysis tools that generate SQL queries, visualize the results, and allow analysts to explore the data (through operations such as drill-down and slicing and dicing).

Mô hình dữ liệu của một kho dữ liệu phổ biến nhất là quan hệ, bởi SQL nhìn chung phù hợp cho các truy vấn phân tích. Có nhiều công cụ phân tích dữ liệu đồ họa sinh ra các truy vấn SQL, trực quan hóa các kết quả, và cho phép các nhà phân tích khám phá dữ liệu (thông qua các thao tác như khoan xuống (drill-down) và cắt lát rồi xắt nhỏ (slicing and dicing)).

On the surface, a data warehouse and a relational OLTP database look similar, because they both have a SQL query interface. However, the internals of the systems can look quite different, because they are optimized for very different query patterns. Many database vendors now focus on supporting either transaction processing or analytics workloads, but not both.

Bề ngoài, một kho dữ liệu và một cơ sở dữ liệu OLTP quan hệ trông tương tự nhau, bởi cả hai đều có một giao diện truy vấn SQL. Tuy nhiên, bên trong của các hệ thống có thể trông khá khác nhau, bởi chúng được tối ưu cho các mẫu truy vấn rất khác nhau. Nhiều nhà cung cấp cơ sở dữ liệu giờ đây tập trung hỗ trợ hoặc khối lượng công việc xử lý giao dịch hoặc phân tích, chứ không phải cả hai.

Some databases, such as Microsoft SQL Server and SAP HANA, have support for transaction processing and data warehousing in the same product. However, they are increasingly becoming two separate storage and query engines, which happen to be accessible through a common SQL interface [49, 50, 51].

Một số cơ sở dữ liệu, chẳng hạn Microsoft SQL Server và SAP HANA, có hỗ trợ cho cả xử lý giao dịch lẫn kho dữ liệu trong cùng một sản phẩm. Tuy nhiên, chúng đang ngày càng trở thành hai engine lưu trữ và truy vấn riêng biệt, mà tình cờ có thể truy cập được qua một giao diện SQL chung [49, 50, 51].

Data warehouse vendors such as Teradata, Vertica, SAP HANA, and ParAccel typically sell their systems under expensive commercial licenses. Amazon RedShift is a hosted version of ParAccel. More recently, a plethora of open source SQL-on-Hadoop projects have emerged; they are young but aiming to compete with commercial data warehouse systems. These include Apache Hive, Spark SQL, Cloudera Impala, Facebook Presto, Apache Tajo, and Apache Drill [52, 53]. Some of them are based on ideas from Google's Dremel [54].

Các nhà cung cấp kho dữ liệu như Teradata, Vertica, SAP HANA, và ParAccel thường bán hệ thống của họ dưới các giấy phép thương mại đắt đỏ. Amazon RedShift là một phiên bản được lưu trữ (hosted) của ParAccel. Gần đây hơn, vô số dự án SQL-trên-Hadoop mã nguồn mở đã xuất hiện; chúng còn non trẻ nhưng nhắm tới cạnh tranh với các hệ thống kho dữ liệu thương mại. Những dự án này bao gồm Apache Hive, Spark SQL, Cloudera Impala, Facebook Presto, Apache Tajo, và Apache Drill [52, 53]. Một số trong số chúng dựa trên các ý tưởng từ Dremel của Google [54].

Stars and Snowflakes: Schemas for Analytics — Sao và Bông tuyết: Các lược đồ cho phân tích

As explored in Chapter 2, a wide range of different data models are used in the realm of transaction processing, depending on the needs of the application. On the other hand, in analytics, there is much less diversity of data models. Many data warehouses are used in a fairly formulaic style, known as a star schema (also known as dimensional modeling [55]).

Như đã khám phá ở Chương 2, một loạt các mô hình dữ liệu khác nhau được dùng trong địa hạt xử lý giao dịch, tùy theo nhu cầu của ứng dụng. Mặt khác, trong phân tích, có ít sự đa dạng về mô hình dữ liệu hơn nhiều. Nhiều kho dữ liệu được dùng theo một phong cách khá rập khuôn, được biết đến với tên lược đồ sao (star schema, còn gọi là mô hình hóa theo chiều, dimensional modeling [55]).

The example schema in Figure 3-9 shows a data warehouse that might be found at a grocery retailer. At the center of the schema is a so-called fact table (in this example, it is called fact_sales). Each row of the fact table represents an event that occurred at a particular time (here, each row represents a customer's purchase of a product). If we were analyzing website traffic rather than retail sales, each row might represent a page view or a click by a user.

Lược đồ ví dụ ở Hình 3-9 trình bày một kho dữ liệu có thể tìm thấy ở một nhà bán lẻ tạp hóa. Ở trung tâm của lược đồ là một thứ gọi là bảng sự kiện (fact table) (trong ví dụ này, nó được gọi là fact_sales). Mỗi hàng của bảng sự kiện biểu diễn một sự kiện đã xảy ra tại một thời điểm cụ thể (ở đây, mỗi hàng biểu diễn việc một khách hàng mua một sản phẩm). Nếu ta đang phân tích lưu lượng truy cập website thay vì doanh số bán lẻ, mỗi hàng có thể biểu diễn một lượt xem trang hoặc một cú nhấp chuột của một người dùng.

Figure 3-9. Example of a star schema for use in a data warehouse. — Hình 3-9. Ví dụ về một lược đồ sao dùng trong một kho dữ liệu. Usually, facts are captured as individual events, because this allows maximum flexibility of analysis later. However, this means that the fact table can become extremely large. A big enterprise like Apple, Walmart, or eBay may have tens of petabytes of transaction history in its data warehouse, most of which is in fact tables [56].

Thường thì các sự kiện được ghi lại dưới dạng các sự kiện riêng lẻ, bởi điều này cho phép tối đa sự linh hoạt trong phân tích về sau. Tuy nhiên, điều này nghĩa là bảng sự kiện có thể trở nên cực kỳ lớn. Một doanh nghiệp lớn như Apple, Walmart, hoặc eBay có thể có hàng chục petabyte lịch sử giao dịch trong kho dữ liệu của mình, mà phần lớn nằm trong các bảng sự kiện [56].

Some of the columns in the fact table are attributes, such as the price at which the product was sold and the cost of buying it from the supplier (allowing the profit margin to be calculated). Other columns in the fact table are **foreign key** references to other tables, called dimension tables. As each row in the fact table represents an event, the dimensions represent the who, what, where, when, how, and why of the event.

Một số cột trong bảng sự kiện là các thuộc tính, chẳng hạn giá mà sản phẩm được bán và chi phí mua nó từ nhà cung cấp (cho phép tính được biên lợi nhuận). Các cột khác trong bảng sự kiện là các tham chiếu khóa ngoại tới các bảng khác, gọi là bảng chiều (dimension table). Vì mỗi hàng trong bảng sự kiện biểu diễn một sự kiện, các chiều biểu diễn cái ai, cái gì, ở đâu, khi nào, như thế nào, và tại sao của sự kiện.

For example, in Figure 3-9, one of the dimensions is the product that was sold. Each row in the `dim_product` table represents one type of product that is for sale, including its stock-keeping unit (SKU), description, brand name, category, fat content, package size, etc. Each row in the `fact_sales` table uses a foreign key to indicate which product was sold in that particular transaction. (For **simplicity**, if the customer buys several different products at once, they are represented as separate rows in the fact table.)

Ví dụ, ở Hình 3-9, một trong các chiều là sản phẩm đã được bán. Mỗi hàng trong bảng `dim_product` biểu diễn một loại sản phẩm đang được bán, bao gồm mã quản lý kho hàng (stock-keeping unit, SKU), mô tả, tên thương hiệu, danh mục, hàm lượng chất béo, kích cỡ gói, v.v. Mỗi hàng trong bảng `fact_sales` dùng một khóa ngoại để cho biết sản phẩm nào đã được bán trong giao dịch cụ thể đó. (Cho đơn giản, nếu khách hàng mua vài sản phẩm khác nhau cùng lúc, chúng được biểu diễn thành các hàng riêng biệt trong bảng sự kiện.)

Even date and time are often represented using dimension tables, because this allows additional information about dates (such as public holidays) to be encoded, allowing queries to differentiate between sales on holidays and non-holidays.

Ngay cả ngày và giờ cũng thường được biểu diễn bằng các bảng chiều, bởi điều này cho phép mã hóa thêm thông tin về các ngày (chẳng hạn các ngày lễ công cộng), cho phép các truy vấn phân biệt giữa doanh số trong các ngày lễ và ngày không lễ.

The name “star schema” comes from the fact that when the table relationships are visualized, the fact table is in the middle, surrounded by its dimension tables; the connections to these tables are like the rays of a star.

Cái tên “lược đồ sao” xuất phát từ việc khi các quan hệ giữa các bảng được trực quan hóa, bảng sự kiện nằm ở giữa, được bao quanh bởi các bảng chiều của nó; các kết nối tới những bảng này giống như các tia của một ngôi sao.

A variation of this template is known as the snowflake schema, where dimensions are further

broken down into subdimensions. For example, there could be separate tables for brands and product categories, and each row in the `dim_product` table could reference the brand and category as foreign keys, rather than storing them as strings in the `dim_product` table. Snowflake schemas are more normalized than star schemas, but star schemas are often preferred because they are simpler for analysts to work with [55].

Một biến thể của khuôn mẫu này được biết đến với tên lược đồ bông tuyết (snowflake schema), trong đó các chiều được chia nhỏ thêm thành các chiều con. Ví dụ, có thể có các bảng riêng cho các thương hiệu và các danh mục sản phẩm, và mỗi hàng trong bảng `dim_product` có thể tham chiếu thương hiệu và danh mục dưới dạng các khóa ngoại, thay vì lưu chúng dưới dạng các chuỗi trong bảng `dim_product`. Lược đồ bông tuyết được chuẩn hóa nhiều hơn lược đồ sao, nhưng lược đồ sao thường được ưa chuộng hơn bởi chúng đơn giản hơn để các nhà phân tích làm việc cùng [55].

In a typical data warehouse, tables are often very wide: fact tables often have over 100 columns, sometimes several hundred [51]. Dimension tables can also be very wide, as they include all the metadata that may be relevant for analysis—for example, the `dim_store` table may include details of which services are offered at each store, whether it has an in-store bakery, the square footage, the date when the store was first opened, when it was last remodeled, how far it is from the nearest highway, etc.

Trong một kho dữ liệu điển hình, các bảng thường rất rộng: các bảng sự kiện thường có trên 100 cột, đôi khi vài trăm [51]. Các bảng chiều cũng có thể rất rộng, bởi chúng bao gồm tất cả siêu dữ liệu có thể liên quan tới phân tích — ví dụ, bảng `dim_store` có thể bao gồm chi tiết về những dịch vụ nào được cung cấp tại mỗi cửa hàng, liệu nó có một tiệm bánh trong cửa hàng hay không, diện tích sàn, ngày cửa hàng được mở lần đầu, lần cuối nó được tu sửa, nó cách đường cao tốc gần nhất bao xa, v.v.

Column-Oriented Storage — Lưu trữ theo cột

If you have trillions of rows and petabytes of data in your fact tables, storing and querying them efficiently becomes a challenging problem. Dimension tables are usually much smaller (millions of rows), so in this section we will concentrate primarily on storage of facts.

Nếu ta có hàng nghìn tỷ hàng và hàng petabyte dữ liệu trong các bảng sự kiện, thì việc lưu trữ và truy vấn chúng một cách hiệu quả trở thành một bài toán đầy thách thức. Các bảng chiều thường nhỏ hơn nhiều (hàng triệu hàng), nên trong phần này ta sẽ tập trung chủ yếu vào việc lưu trữ các sự kiện.

Although fact tables are often over 100 columns wide, a typical data warehouse query only accesses 4 or 5 of them at one time (“SELECT *” queries are rarely needed for analytics) [51]. Take the query in Example 3-1: it accesses a large number of rows (every occurrence of someone buying fruit or candy during the 2013 calendar year), but it only needs to access three columns of the fact_sales table: date_key, product_sk, and quantity. The query ignores all other columns.

Mặc dù các bảng sự kiện thường rộng trên 100 cột, một truy vấn kho dữ liệu điển hình chỉ truy cập 4 hoặc 5 trong số đó tại một thời điểm (các truy vấn “SELECT *” hiếm khi cần cho phân tích) [51]. Hãy lấy truy vấn ở Ví dụ 3-1: nó truy cập một số lượng lớn hàng (mọi lần ai đó mua trái cây hoặc kẹo trong năm dương lịch 2013), nhưng nó chỉ cần truy cập ba cột của bảng fact_sales: date_key, product_sk, và quantity. Truy vấn bỏ qua tất cả các cột khác.

Example 3-1. Analyzing whether people are more inclined to buy fresh fruit or candy, depending on the day of the week — Ví dụ 3-1. Phân tích xem liệu người ta có nghiêng về việc mua trái cây tươi hay kẹo nhiều hơn, tùy theo ngày trong tuần

```
SELECT
    dim_date.weekday, dim_product.category,
    SUM(fact_sales.quantity) AS quantity_sold
FROM fact_sales
    JOIN dim_date    ON fact_sales.date_key   = dim_date.date_key
    JOIN dim_product ON fact_sales.product_sk = dim_product.product_sk
WHERE
    dim_date.year = 2013 AND
    dim_product.category IN ('Fresh fruit', 'Candy')
GROUP BY
    dim_date.weekday, dim_product.category;
```

How can we execute this query efficiently?

Làm thế nào để ta thực thi truy vấn này một cách hiệu quả?

In most OLTP databases, storage is laid out in a row-oriented fashion: all the values from one row of a table are stored next to each other. Document databases are similar: an entire document is typically stored as one contiguous sequence of bytes. You can see this in the CSV example of Figure 3-1.

Trong hầu hết các cơ sở dữ liệu OLTP, kho lưu trữ được bố trí theo kiểu hướng hàng (row-oriented): tất cả các giá trị từ một hàng của một bảng được lưu cạnh nhau. Các cơ sở dữ liệu tài liệu cũng tương tự: cả một tài liệu thường được lưu thành một chuỗi byte liền mạch. Ta có thể thấy điều này trong ví dụ CSV ở Hình 3-1.

In order to process a query like Example 3-1, you may have indexes on `fact_sales.date_key` and/or `fact_sales.product_sk` that tell the storage engine where to find all the sales for a particular date or for a particular product. But then, a row-oriented storage engine still needs to load all of those rows (each consisting of over 100 attributes) from disk into memory, parse them, and filter out those that don't meet the required conditions. That can take a long time.

Để xử lý một truy vấn như Ví dụ 3-1, ta có thể có các chỉ mục trên `fact_sales.date_key` và/hoặc `fact_sales.product_sk` để cho engine lưu trữ biết tìm tất cả các giao dịch bán hàng cho một ngày cụ thể hoặc cho một sản phẩm cụ thể ở đâu. Nhưng rồi, một engine lưu trữ hướng hàng vẫn cần nạp tất cả các hàng đó (mỗi hàng gồm trên 100 thuộc tính) từ đĩa vào bộ nhớ, phân tích cú pháp chúng, và lọc bỏ những hàng không thỏa các điều kiện yêu cầu. Việc đó có thể mất nhiều thời gian.

The idea behind column-oriented storage is simple: don't store all the values from one row together, but store all the values from each column together instead. If each column is stored in a separate file, a query only needs to read and parse those columns that are used in that query, which can save a lot of work. This principle is illustrated in Figure 3-10.

Ý tưởng đằng sau lưu trữ theo cột rất đơn giản: đừng lưu tất cả các giá trị từ một hàng cùng nhau, mà thay vào đó hãy lưu tất cả các giá trị từ mỗi cột cùng nhau. Nếu mỗi cột được lưu trong một tệp riêng, thì một truy vấn chỉ cần đọc và phân tích cú pháp những cột được dùng trong truy vấn đó, điều này có thể tiết kiệm rất nhiều công sức. Nguyên lý này được minh họa ở Hình 3-10.

Note — Ghi chú Column storage is easiest to understand in a relational data model, but it applies equally to nonrelational data. For example, Parquet [57] is a columnar storage format that supports a document data model, based on Google's Dremel [54].

Lưu trữ theo cột dễ hiểu nhất trong một mô hình dữ liệu quan hệ, nhưng nó áp dụng được như nhau cho dữ liệu phi quan hệ. Ví dụ, Parquet [57] là một định dạng lưu trữ theo cột hỗ trợ một mô hình dữ liệu tài liệu, dựa trên Dremel của Google [54].

Figure 3-10. Storing relational data by column, rather than by row. — Hình 3-10. Lưu trữ dữ liệu quan hệ theo cột, thay vì theo hàng. The column-oriented storage layout relies on each column file containing the rows in the same order. Thus, if you need to reassemble an entire row, you can take the 23rd entry from each of the individual column files and put them together to form the 23rd row of the table.

Cách bố trí lưu trữ hướng cột dựa vào việc mỗi tệp cột chứa các hàng theo cùng một thứ tự. Như vậy, nếu ta cần ráp lại cả một hàng, ta có thể lấy mục thứ 23 từ mỗi tệp cột riêng lẻ và ghép chúng lại để tạo thành hàng thứ 23 của bảng.

Column Compression — Nén cột

Besides only loading those columns from disk that are required for a query, we can further reduce the demands on disk throughput by compressing data. Fortunately, column-oriented storage often lends itself very well to compression.

Bên cạnh việc chỉ nạp những cột từ đĩa mà một truy vấn cần, ta có thể giảm thêm nhu cầu về thông lượng đĩa bằng cách nén dữ liệu. May mắn thay, lưu trữ hướng cột thường rất phù hợp với việc nén.

Take a look at the sequences of values for each column in Figure 3-10: they often look quite repetitive, which is a good sign for compression. Depending on the data in the column, different compression techniques can be used. One technique that is particularly effective in data warehouses is bitmap encoding, illustrated in Figure 3-11.

Hãy nhìn vào các chuỗi giá trị cho mỗi cột ở Hình 3-10: chúng thường trông khá lặp lại, đây là một dấu hiệu tốt cho việc nén. Tùy theo dữ liệu trong cột, các kỹ thuật nén khác nhau có thể được dùng. Một kỹ thuật đặc biệt hiệu quả trong các kho dữ liệu là mã hóa bitmap (bitmap encoding), được minh họa ở Hình 3-11.

Figure 3-11. Compressed, bitmap-indexed storage of a single column. — Hình 3-11. Lưu trữ một cột đơn được nén và lập chỉ mục bitmap. Often, the number of distinct values in a column is small compared to the number of rows (for example, a retailer may have billions of sales transactions, but only 100,000 distinct products). We can now take a column with n distinct values and turn it into n separate bitmaps: one bitmap for each distinct value, with one bit for each row. The bit is 1 if the row has that value, and 0 if not.

Thường thì số lượng giá trị khác biệt trong một cột là nhỏ so với số lượng hàng (ví dụ, một nhà bán lẻ có thể có hàng tỷ giao dịch bán hàng, nhưng chỉ 100.000 sản phẩm khác biệt). Giờ ta có thể lấy một cột có n giá trị khác biệt và biến nó thành n bitmap riêng biệt: một bitmap cho mỗi giá trị khác biệt, với một bit cho mỗi hàng. Bit bằng 1 nếu hàng có giá trị đó, và bằng 0 nếu không.

If n is very small (for example, a country column may have approximately 200 distinct values), those bitmaps can be stored with one bit per row. But if n is bigger, there will be a lot of zeros in most of the bitmaps (we say that they are sparse). In that case, the bitmaps can additionally be run-length encoded, as shown at the bottom of Figure 3-11. This can make the encoding of a column remarkably compact.

Nếu n rất nhỏ (ví dụ, một cột quốc gia có thể có khoảng 200 giá trị khác biệt), thì các bitmap đó có thể được lưu với một bit cho mỗi hàng. Nhưng nếu n lớn hơn, sẽ có rất nhiều số 0 trong hầu hết các bitmap (ta nói rằng chúng thưa). Trong trường hợp đó, các bitmap có thể được mã hóa độ dài chạy (run-length encoded) bổ sung, như trình bày ở cuối Hình 3-11. Điều này có thể khiến việc mã hóa một cột trở nên gọn gàng một cách đáng kể.

Bitmap indexes such as these are very well suited for the kinds of queries that are common in a data warehouse. For example:

Các chỉ mục bitmap như thế này rất phù hợp với các loại truy vấn phổ biến trong một kho dữ liệu. Ví dụ:

Load the three bitmaps for `product_sk = 30`, `product_sk = 68`, and `product_sk = 69`, and calculate the bitwise OR of the three bitmaps, which can be done very efficiently.

Nạp ba bitmap cho `product_sk = 30`, `product_sk = 68`, và `product_sk = 69`, rồi tính phép OR bit của ba bitmap, điều này có thể thực hiện rất hiệu quả.

Load the bitmaps for `product_sk = 31` and `store_sk = 3`, and calculate the bitwise AND. This works because the columns contain the rows in the same order, so the *k*th bit in one column's bitmap corresponds to the same row as the *k*th bit in another column's bitmap.

Nạp các bitmap cho `product_sk = 31` và `store_sk = 3`, rồi tính phép AND bit. Điều này hoạt động được bởi các cột chứa các hàng theo cùng một thứ tự, nên bit thứ *k* trong bitmap của một cột tương ứng với cùng một hàng như bit thứ *k* trong bitmap của một cột khác.

There are also various other compression schemes for different kinds of data, but we won't go into them in detail—see [58] for an overview.

Cũng có nhiều cơ chế nén khác cho các loại dữ liệu khác nhau, nhưng ta sẽ không đi sâu vào chúng — xem [58] để có một cái nhìn tổng quan.

Column-oriented storage and column families — Lưu trữ hướng cột và họ cột

Cassandra and HBase have a concept of column families, which they inherited from Bigtable [9]. However, it is very misleading to call them column-oriented: within each column family, they store all columns from a row together, along with a row key, and they do not use column compression. Thus, the Bigtable model is still mostly row-oriented.

Cassandra và HBase có một khái niệm về họ cột (column-family), vốn chúng thừa hưởng từ Bigtable [9]. Tuy nhiên, gọi chúng là hướng cột thì rất dễ gây hiểu lầm: bên trong mỗi họ cột, chúng lưu tất cả các cột từ một hàng cùng nhau, cùng với một khóa hàng, và chúng không dùng nén cột. Như vậy, mô hình Bigtable vẫn chủ yếu hướng hàng.

Memory bandwidth and vectorized processing — Băng thông bộ nhớ và xử lý vector hóa

For data warehouse queries that need to scan over millions of rows, a big bottleneck is the bandwidth for getting data from disk into memory. However, that is not the only bottleneck. Developers of analytical databases also worry about efficiently using the bandwidth from main memory into the CPU cache, avoiding branch mispredictions and bubbles in the CPU instruction processing pipeline, and making use of single-instruction-multi-data (SIMD) instructions in modern CPUs [59, 60].

Với các truy vấn kho dữ liệu cần quét qua hàng triệu hàng, một nút thắt cổ chai lớn là băng thông để đưa dữ liệu từ đĩa vào bộ nhớ. Tuy nhiên, đó không phải nút thắt cổ chai duy nhất. Các nhà phát triển cơ sở dữ liệu phân tích cũng lo lắng về việc sử dụng hiệu quả băng thông từ bộ nhớ chính vào bộ nhớ đệm CPU (CPU cache), tránh các dự đoán nhánh sai (branch misprediction) và các bong bóng (bubble) trong đường ống xử lý lệnh của CPU, và tận dụng các lệnh đơn-lệnh-nhiều-dữ-liệu (single-instruction-multi-data, SIMD) trong các CPU hiện đại [59, 60].

Besides reducing the volume of data that needs to be loaded from disk, column-oriented storage layouts are also good for making efficient use of CPU cycles. For example, the query engine can take a chunk of compressed column data that fits comfortably in the CPU's L1 cache and iterate through it in a tight loop (that is, with no function calls). A CPU can execute such a loop much faster than code that requires a lot of function calls and conditions for each record that is processed. Column compression allows more rows from a column to fit in the same amount of L1 cache. Operators, such as the bitwise AND and OR described previously, can be designed to operate on such chunks of compressed column data directly. This technique is known as vectorized processing [58, 49].

Bên cạnh việc giảm khối lượng dữ liệu cần được nạp từ đĩa, các cách bố trí lưu trữ hướng cột cũng tốt cho việc sử dụng hiệu quả các chu kỳ CPU. Ví dụ, engine truy vấn có thể lấy một mẫu dữ liệu cột đã nén vừa khít trong bộ nhớ đệm L1 của CPU và lặp qua nó trong một vòng lặp chặt (tức là không có lời gọi hàm). Một CPU có thể thực thi một vòng lặp như vậy nhanh hơn nhiều so với mã đòi hỏi nhiều lời gọi hàm và điều kiện cho mỗi bản ghi được xử lý. Nén cột cho phép nhiều hàng từ một cột vừa trong cùng một lượng bộ nhớ đệm L1. Các toán tử, chẳng hạn phép AND và OR bit đã mô tả trước đó, có thể được thiết kế để thao tác trực tiếp trên những mẫu dữ liệu cột đã nén như vậy. Kỹ thuật này được biết đến với tên xử lý vector hóa (vectorized processing) [58, 49].

Sort Order in Column Storage — Thứ tự sắp xếp trong lưu trữ cột

In a column store, it doesn't necessarily matter in which order the rows are stored. It's easiest to store them in the order in which they were inserted, since then inserting a new row just means appending to each of the column files. However, we can choose to impose an order, like we did with SSTables previously, and use that as an indexing mechanism.

Trong một kho lưu trữ cột, thứ tự lưu các hàng không nhất thiết quan trọng. Cách dễ nhất là lưu chúng theo thứ tự chúng được chèn vào, bởi khi đó việc chèn một hàng mới chỉ đơn giản là ghi nối vào mỗi tệp cột. Tuy nhiên, ta có thể chọn áp đặt một thứ tự, như ta đã làm với các SSTable trước đó, và dùng thứ tự đó như một cơ chế lập chỉ mục.

Note that it wouldn't make sense to sort each column independently, because then we would no longer know which items in the columns belong to the same row. We can only reconstruct a row because we know that the *k*th item in one column belongs to the same row as the *k*th item in another column.

Lưu ý rằng sẽ vô nghĩa nếu sắp xếp từng cột một cách độc lập, bởi khi đó ta sẽ không còn biết những mục nào trong các cột thuộc về cùng một hàng. Ta chỉ có thể tái dựng một hàng vì ta biết rằng mục thứ *k* trong một cột thuộc về cùng một hàng như mục thứ *k* trong một cột khác.

Rather, the data needs to be sorted an entire row at a time, even though it is stored by column. The administrator of the database can choose the columns by which the table should be sorted, using their knowledge of common queries. For example, if queries often target date ranges, such as the last month, it might make sense to make `date_key` the first sort key. Then the **query optimizer** can scan only the rows from the last month, which will be much faster than scanning all rows.

Đúng hơn, dữ liệu cần được sắp xếp theo cả một hàng tại một thời điểm, mặc dù nó được lưu theo cột. Quản trị viên của cơ sở dữ liệu có thể chọn các cột mà bảng nên được sắp xếp theo, dựa trên hiểu biết của họ về các truy vấn phổ biến. Ví dụ, nếu các truy vấn thường nhắm tới các khoảng ngày, chẳng hạn tháng vừa qua, thì có lẽ hợp lý khi đặt `date_key` làm khóa sắp xếp đầu tiên. Khi đó bộ tối ưu truy vấn có thể chỉ quét các hàng của tháng vừa qua, điều này sẽ nhanh hơn nhiều so với quét tất cả các hàng.

A second column can determine the sort order of any rows that have the same value in the first column. For example, if `date_key` is the first sort key in Figure 3-10, it might make sense for `product_sk` to be the second sort key so that all sales for the same product on the same day are grouped together in storage. That will help queries that need to group or filter sales by product within a certain date range.

Một cột thứ hai có thể quyết định thứ tự sắp xếp của bất kỳ hàng nào có cùng giá trị ở cột thứ nhất. Ví dụ, nếu `date_key` là khóa sắp xếp đầu tiên ở Hình 3-10, thì có lẽ hợp lý khi đặt `product_sk` làm khóa sắp xếp thứ hai để tất cả các giao dịch bán hàng cho cùng một sản phẩm trong cùng một ngày được gom lại với nhau trong kho lưu trữ. Điều đó sẽ giúp các truy vấn cần nhóm hoặc lọc các giao dịch bán hàng theo sản phẩm trong một khoảng ngày nhất định.

Another advantage of sorted order is that it can help with compression of columns. If the primary sort column does not have many distinct values, then after sorting, it will have long sequences where the same value is repeated many times in a row. A simple run-length encoding, like we used for the bitmaps in Figure 3-11, could compress that column down to a few kilobytes—even if the table has billions of rows.

Một ưu điểm khác của thứ tự đã sắp xếp là nó có thể giúp ích cho việc nén các cột. Nếu cột sắp xếp chính không có nhiều giá trị khác biệt, thì sau khi sắp xếp, nó sẽ có những chuỗi dài nơi cùng một giá trị được lặp lại nhiều lần liên tiếp. Một phép mã hóa độ dài chạy đơn giản, như ta đã dùng cho các bitmap ở Hình 3-11, có thể nén cột đó xuống chỉ còn vài kilobyte — ngay cả khi bảng có hàng tỷ hàng.

That compression effect is strongest on the first sort key. The second and third sort keys will be more jumbled up, and thus not have such long runs of repeated values. Columns further down the sorting priority appear in essentially random order, so they probably won't compress as well. But having the first few columns sorted is still a win overall.

Hiệu ứng nén đó mạnh nhất ở khóa sắp xếp thứ nhất. Khóa sắp xếp thứ hai và thứ ba sẽ lộn xộn hơn, và do đó không có những chuỗi dài các giá trị lặp lại như vậy. Các cột nằm xa hơn trong thứ tự ưu tiên sắp xếp xuất hiện về cơ bản theo thứ tự ngẫu nhiên, nên chúng có lẽ sẽ không nén tốt như vậy. Nhưng việc có vài cột đầu tiên được sắp xếp vẫn là một cái lợi nhìn chung.

Several different sort orders — Vài thứ tự sắp xếp khác nhau

A clever extension of this idea was introduced in C-Store and adopted in the commercial data warehouse Vertica [61, 62]. Different queries benefit from different sort orders, so why not store the same data sorted in several different ways? Data needs to be replicated to multiple machines anyway, so that you don't lose data if one machine fails. You might as well store that redundant data sorted in different ways so that when you're processing a query, you can use the version that best fits the query pattern.

Một sự mở rộng khéo léo của ý tưởng này đã được giới thiệu trong C-Store và được áp dụng trong kho dữ liệu thương mại Vertica [61, 62]. Các truy vấn khác nhau được hưởng lợi từ các thứ tự sắp xếp khác nhau, vậy tại sao không lưu cùng một dữ liệu được sắp xếp theo nhiều cách khác nhau? Dù sao dữ liệu cũng cần được sao chép sang nhiều máy, để ta không mất dữ liệu nếu một máy hỏng. Ta cũng nên lưu dữ liệu dư thừa đó được sắp xếp theo các cách khác nhau, để khi xử lý một truy vấn, ta có thể dùng phiên bản phù hợp nhất với mẫu truy vấn.

Having multiple sort orders in a column-oriented store is a bit similar to having multiple secondary indexes in a row-oriented store. But the big difference is that the row-oriented store keeps every row in one place (in the heap file or a clustered index), and secondary indexes just contain pointers to the matching rows. In a column store, there normally aren't any pointers to data elsewhere, only columns containing values.

Việc có nhiều thứ tự sắp xếp trong một kho lưu trữ hướng cột hơi giống việc có nhiều chỉ mục phụ trong một kho lưu trữ hướng hàng. Nhưng khác biệt lớn là kho lưu trữ hướng hàng giữ mỗi hàng ở một chỗ (trong tệp heap hoặc một chỉ mục gom cụm), còn các chỉ mục phụ chỉ chứa các con trỏ tới các hàng khớp. Trong một kho lưu trữ cột, thường không có bất kỳ con trỏ nào tới dữ liệu ở nơi khác, chỉ có các cột chứa các giá trị.

Writing to Column-Oriented Storage — Ghi vào lưu trữ hướng cột

These optimizations make sense in data warehouses, because most of the load consists of large read-only queries run by analysts. Column-oriented storage, compression, and sorting all help to make those read queries faster. However, they have the downside of making writes more difficult.

Các tối ưu này hợp lý trong các kho dữ liệu, bởi phần lớn tải gồm các truy vấn lớn chỉ-đọc do các nhà phân tích chạy. Lưu trữ hướng cột, nén, và sắp xếp đều giúp khiến các truy vấn đọc đó nhanh hơn. Tuy nhiên, chúng có nhược điểm là khiến việc ghi trở nên khó khăn hơn.

An update-in-place approach, like B-trees use, is not possible with compressed columns. If you wanted to insert a row in the middle of a sorted table, you would most likely have to rewrite all the column files. As rows are identified by their position within a column, the insertion has to update all columns consistently.

Một cách tiếp cận cập-nhật-tại-chỗ, như B-tree dùng, là không khả thi với các cột đã nén. Nếu ta muốn chèn một hàng vào giữa một bảng đã sắp xếp, rất có thể ta sẽ phải ghi lại tất cả các tệp cột. Vì các hàng được nhận diện bằng vị trí của chúng trong một cột, việc chèn phải cập nhật tất cả các cột một cách nhất quán.

Fortunately, we have already seen a good solution earlier in this chapter: LSM-trees. All writes first go to an in-memory store, where they are added to a sorted structure and prepared for writing to disk. It doesn't matter whether the in-memory store is row-oriented or column-oriented. When enough writes have accumulated, they are merged with the column files on disk and written to new files in bulk. This is essentially what Vertica does [62].

May mắn thay, ta đã thấy một giải pháp tốt từ trước trong chương này: LSM-tree. Tất cả các lần ghi trước tiên đi tới một kho lưu trữ trong bộ nhớ, nơi chúng được thêm vào một cấu trúc đã sắp xếp và chuẩn bị để ghi xuống đĩa. Việc kho lưu trữ trong bộ nhớ là hướng hàng hay hướng cột không quan trọng. Khi đủ số lần ghi đã tích lũy, chúng được gộp với các tệp cột trên đĩa và được ghi thành các tệp mới theo lô lớn. Đây về cơ bản là những gì Vertica làm [62].

Queries need to examine both the column data on disk and the recent writes in memory, and combine the two. However, the query optimizer hides this distinction from the user. From an analyst's point of view, data that has been modified with inserts, updates, or deletes is immediately reflected in subsequent queries.

Các truy vấn cần kiểm tra cả dữ liệu cột trên đĩa lẫn các lần ghi gần đây trong bộ nhớ, và kết hợp cả hai. Tuy nhiên, bộ tối ưu truy vấn giấu sự phân biệt này khỏi người dùng. Từ góc nhìn của một nhà phân tích, dữ liệu đã được sửa đổi bằng các lần chèn, cập nhật, hoặc xóa được phản ánh ngay lập tức trong các truy vấn tiếp theo.

Aggregation: Data Cubes and Materialized Views — Tổng hợp: Khối dữ liệu và Khung nhìn cụ thể hóa

Not every data warehouse is necessarily a column store: traditional row-oriented databases and a few other architectures are also used. However, columnar storage can be significantly faster for ad hoc analytical queries, so it is rapidly gaining popularity [51, 63].

Không phải kho dữ liệu nào cũng nhất thiết là một kho lưu trữ cột: các cơ sở dữ liệu hướng hàng truyền thống và một vài kiến trúc khác cũng được dùng. Tuy nhiên, lưu trữ theo cột có thể nhanh hơn đáng kể cho các truy vấn phân tích tùy ý, nên nó đang nhanh chóng trở nên phổ biến [51, 63].

Another aspect of data warehouses that is worth mentioning briefly is materialized aggregates. As discussed earlier, data warehouse queries often involve an aggregate function, such as COUNT, SUM, AVG, MIN, or MAX in SQL. If the same aggregates are used by many different queries, it can be wasteful to crunch through the raw data every time. Why not cache some of the counts or sums that queries use most often?

Một khía cạnh khác của các kho dữ liệu đáng được nhắc tới vẫn tất là các tổng hợp được cụ thể hóa (materialized aggregate). Như đã bàn trước đó, các truy vấn kho dữ liệu thường liên quan tới một hàm tổng hợp, chẳng hạn COUNT, SUM, AVG, MIN, hoặc MAX trong SQL. Nếu cùng những tổng hợp đó được dùng bởi nhiều truy vấn khác nhau, thì việc cày qua dữ liệu thô mỗi lần có thể lãng phí. Tại sao không đệm một số phép đếm hoặc tổng mà các truy vấn dùng thường xuyên nhất?

One way of creating such a cache is a materialized view. In a relational data model, it is often defined like a standard (virtual) view: a table-like object whose contents are the results of some query. The difference is that a materialized view is an actual copy of the query results, written to disk, whereas a virtual view is just a shortcut for writing queries. When you read from a virtual view, the SQL engine expands it into the view's underlying query on the fly and then processes the expanded query.

Một cách để tạo một bộ đệm như vậy là một khung nhìn cụ thể hóa (materialized view). Trong một mô hình dữ liệu quan hệ, nó thường được định nghĩa giống một khung nhìn (ảo) tiêu chuẩn: một đối tượng giống bảng mà nội dung của nó là các kết quả của một truy vấn nào đó. Khác biệt là một khung nhìn cụ thể hóa là một bản sao thực sự của các kết quả truy vấn, được ghi xuống đĩa, trong khi một khung nhìn ảo chỉ là một lối tắt để viết các truy vấn. Khi ta đọc từ một khung nhìn ảo, engine SQL khai triển nó thành truy vấn nền tảng của khung nhìn ngay tại chỗ rồi xử lý truy vấn đã khai triển.

When the underlying data changes, a materialized view needs to be updated, because it is a denormalized copy of the data. The database can do that automatically, but such updates make writes more expensive, which is why materialized views are not often used in OLTP databases. In read-heavy data warehouses they can make more sense (whether or not they actually improve read performance depends on the individual case).

Khi dữ liệu nền tảng thay đổi, một khung nhìn cụ thể hóa cần được cập nhật, bởi nó là một bản sao đã phi chuẩn hóa của dữ liệu. Cơ sở dữ liệu có thể làm việc đó tự động, nhưng những lần cập nhật như vậy khiến việc ghi tốn kém hơn, đó là lý do tại sao các khung nhìn cụ thể hóa không hay được dùng trong các cơ sở dữ liệu OLTP. Trong các kho dữ liệu nặng về đọc, chúng có thể hợp lý hơn (việc chúng có thực sự cải thiện hiệu năng đọc hay không phụ thuộc vào từng trường hợp cụ thể).

A common special case of a materialized view is known as a data cube or OLAP cube [64]. It is a grid of aggregates grouped by different dimensions. Figure 3-12 shows an example.

Một trường hợp đặc biệt phổ biến của một khung nhìn cụ thể hóa được biết đến với tên một khối dữ liệu (data cube) hoặc khối OLAP (OLAP cube) [64]. Nó là một lưới các tổng hợp được nhóm theo các chiều khác nhau. Hình 3-12 trình bày một ví dụ.

Figure 3-12. Two dimensions of a data cube, aggregating data by summing. — Hình 3-12. Hai chiều của một khối dữ liệu, tổng hợp dữ liệu bằng cách cộng. Imagine for now that each fact has foreign keys to only two dimension tables—in Figure 3-12, these are date and product. You can now draw a two-dimensional table, with dates along one axis and products along the other. Each cell contains the aggregate (e.g., SUM) of an attribute (e.g., net_price) of all facts with that date-product combination. Then you can apply the same aggregate along each row or column and get a summary that has been reduced by one dimension (the sales by product regardless of date, or the sales by date regardless of product).

Hãy hình dung bây giờ rằng mỗi sự kiện có các khóa ngoại tới chỉ hai bảng chiều — ở Hình 3-12, đó là ngày và sản phẩm. Giờ ta có thể vẽ một bảng hai chiều, với các ngày dọc theo một trục và các sản phẩm dọc theo trục kia. Mỗi ô chứa tổng hợp (ví dụ, SUM) của một thuộc tính (ví dụ, net_price) của tất cả các sự kiện có tổ hợp ngày-sản phẩm đó. Sau đó ta có thể áp dụng cùng phép tổng hợp dọc theo mỗi hàng hoặc cột và thu được một bản tóm tắt đã được rút gọn đi một chiều (doanh số theo sản phẩm bất kể ngày, hoặc doanh số theo ngày bất kể sản phẩm).

In general, facts often have more than two dimensions. In Figure 3-9 there are five dimensions: date, product, store, promotion, and customer. It's a lot harder to imagine what a five-dimensional hypercube would look like, but the principle remains the same: each cell contains the sales for a particular date-product-store-promotion-customer combination. These values can then repeatedly be summarized along each of the dimensions.

Nhìn chung, các sự kiện thường có nhiều hơn hai chiều. Ở Hình 3-9 có năm chiều: ngày, sản phẩm, cửa hàng, khuyến mãi, và khách hàng. Khó hơn nhiều để hình dung một siêu khối năm chiều trông như thế nào, nhưng nguyên lý vẫn vậy: mỗi ô chứa doanh số cho một tổ hợp ngày-sản phẩm-cửa hàng-khuyến mãi-khách hàng cụ thể. Các giá trị này sau đó có thể được tóm tắt lặp đi lặp lại dọc theo mỗi chiều.

The advantage of a materialized data cube is that certain queries become very fast because they have effectively been precomputed. For example, if you want to know the total sales per store yesterday, you just need to look at the totals along the appropriate dimension—no need to scan millions of rows.

Ưu điểm của một khối dữ liệu được cụ thể hóa là một số truy vấn trở nên rất nhanh bởi chúng đã được tính toán trước một cách hiệu quả. Ví dụ, nếu ta muốn biết tổng doanh số mỗi cửa hàng của ngày hôm qua, ta chỉ cần nhìn vào các tổng dọc theo chiều thích hợp — không cần quét hàng triệu hàng.

The disadvantage is that a data cube doesn't have the same flexibility as querying the raw data. For example, there is no way of calculating which proportion of sales comes from items that cost more than \$100, because the price isn't one of the dimensions. Most data warehouses therefore try to keep as much raw data as possible, and use aggregates such as data cubes only as a performance boost for certain queries.

Nhược điểm là một khối dữ liệu không có cùng sự linh hoạt như việc truy vấn dữ liệu thô. Ví dụ, không có cách nào để tính tỷ lệ doanh số đến từ các mặt hàng có giá trên 100 đô la, bởi giá không phải một trong các chiều. Do đó hầu hết các kho dữ liệu cố gắng giữ càng nhiều dữ liệu thô càng tốt, và chỉ dùng các tổng hợp như khối dữ liệu như một sự tăng tốc hiệu năng cho một số truy vấn nhất định.

Summary — Tóm tắt

In this chapter we tried to get to the bottom of how databases handle storage and retrieval. What happens when you store data in a database, and what does the database do when you query for the data again later?

Trong chương này, ta đã cố gắng đi đến tận cùng việc các cơ sở dữ liệu xử lý lưu trữ và truy xuất như thế nào. Điều gì xảy ra khi ta lưu dữ liệu vào một cơ sở dữ liệu, và cơ sở dữ liệu làm gì khi ta truy vấn dữ liệu đó lại sau này?

On a high level, we saw that storage engines fall into two broad categories: those optimized for transaction processing (OLTP), and those optimized for analytics (OLAP). There are big differences between the access patterns in those use cases:

Ở mức cao, ta đã thấy các engine lưu trữ rơi vào hai loại lớn: những engine được tối ưu cho xử lý giao dịch (OLTP), và những engine được tối ưu cho phân tích (OLAP). Có những khác biệt lớn giữa các mẫu truy cập trong các tình huống sử dụng đó:

- OLTP systems are typically user-facing, which means that they may see a huge volume of requests. In order to handle the load, applications usually only touch a small number of records in each query. The application requests records using some kind of key, and the storage engine uses an index to find the data for the requested key. Disk seek time is often the bottleneck here.
- Data warehouses and similar analytic systems are less well known, because they are primarily used by business analysts, not by end users. They handle a much lower volume of queries than OLTP systems, but each query is typically very demanding, requiring many millions of records to be scanned in a short time. Disk bandwidth (not seek time) is often the bottleneck here, and column-oriented storage is an increasingly popular solution for this kind of workload.
 - Các hệ thống OLTP thường hướng tới người dùng, điều này nghĩa là chúng có thể thấy một khối lượng yêu cầu khổng lồ. Để xử lý tải, các ứng dụng thường chỉ chạm tới một số lượng nhỏ bản ghi trong mỗi truy vấn. Ứng dụng yêu cầu các bản ghi bằng một loại khóa nào đó, và engine lưu trữ dùng một chỉ mục để tìm dữ liệu cho khóa được yêu cầu. Thời gian seek đĩa thường là nút thắt cổ chai ở đây.
 - Các kho dữ liệu và các hệ thống phân tích tương tự ít được biết đến hơn, bởi chúng chủ yếu được dùng bởi các nhà phân tích nghiệp vụ, chứ không phải người dùng cuối. Chúng xử lý một khối lượng truy vấn thấp hơn nhiều so với các hệ thống OLTP, nhưng mỗi truy vấn thường rất nặng, đòi hỏi quét nhiều triệu bản ghi trong một thời gian ngắn. Băng thông đĩa (chứ không phải thời gian seek) thường là nút thắt cổ chai ở đây, và lưu trữ hướng cột là một giải pháp ngày càng phổ biến cho loại khối lượng công việc này.

On the OLTP side, we saw storage engines from two main schools of thought:

Về phía OLTP, ta đã thấy các engine lưu trữ từ hai trường phái tư tưởng chính:

- The log-structured school, which only permits appending to files and deleting obsolete files, but never updates a file that has been written. Bitcask, SSTables, LSM-trees, LevelDB, Cassandra, HBase, Lucene, and others belong to this group.
 - The update-in-place school, which treats the disk as a set of fixed-size pages that can be overwritten. B-trees are the biggest example of this philosophy, being used in all major relational databases and also many nonrelational ones.
- Trường phái cấu trúc theo log, vốn chỉ cho phép ghi nối vào các tệp và xóa các tệp đã lỗi thời, nhưng không bao giờ cập nhật một tệp đã được ghi. Bitcask, SSTable, LSM-tree, LevelDB, Cassandra, HBase, Lucene, và những thứ khác thuộc nhóm này.
 - Trường phái cập-nhật-tại-chỗ, vốn coi đĩa như một tập các trang có kích thước cố định có thể bị ghi đè. B-tree là ví dụ lớn nhất của triết lý này, được dùng trong tất cả các cơ sở dữ liệu quan hệ lớn và cả nhiều cơ sở dữ liệu phi quan hệ.

Log-structured storage engines are a comparatively recent development. Their key idea is that they systematically turn random-access writes into sequential writes on disk, which enables higher write throughput due to the performance characteristics of hard drives and SSDs.

Các engine lưu trữ cấu trúc theo log là một bước phát triển tương đối gần đây. Ý tưởng then chốt của chúng là chúng biến một cách có hệ thống các lần ghi truy cập ngẫu nhiên thành các lần ghi tuần tự trên đĩa, điều này cho phép thông lượng ghi cao hơn nhờ các đặc tính hiệu năng của ổ cứng và SSD.

Finishing off the OLTP side, we did a brief tour through some more complicated indexing structures, and databases that are optimized for keeping all data in memory.

Khép lại phía OLTP, ta đã dạo qua một số cấu trúc lập chỉ mục phức tạp hơn, và các cơ sở dữ liệu được tối ưu cho việc giữ toàn bộ dữ liệu trong bộ nhớ.

We then took a detour from the internals of storage engines to look at the high-level architecture of a typical data warehouse. This background illustrated why analytic workloads are so different from OLTP: when your queries require sequentially scanning across a large number of rows, indexes are much less relevant. Instead it becomes important to encode data very compactly, to minimize the amount of data that the query needs to read from disk. We discussed how column-oriented storage helps achieve this goal.

Sau đó ta rẽ ngang khỏi phần bên trong của các engine lưu trữ để nhìn vào kiến trúc ở mức cao của một kho dữ liệu điển hình. Bối cảnh này minh họa tại sao các khối lượng công việc phân tích lại khác biệt với OLTP đến vậy: khi các truy vấn của ta đòi hỏi quét tuần tự qua một số lượng lớn hàng, các chỉ mục ít liên quan hơn nhiều. Thay vào đó, việc mã hóa dữ liệu thật gọn gàng trở nên quan trọng, để giảm thiểu lượng dữ liệu mà truy vấn cần đọc từ đĩa. Ta đã bàn về việc lưu trữ hướng cột giúp đạt được mục tiêu này như thế nào.

As an application developer, if you're armed with this knowledge about the internals of storage engines, you are in a much better position to know which tool is best suited for your particular application. If you need to adjust a database's tuning parameters, this understanding allows you to imagine what effect a higher or a lower value may have.

Là một lập trình viên ứng dụng, nếu ta được trang bị kiến thức này về phần bên trong của các engine lưu trữ, ta ở vào một vị thế tốt hơn nhiều để biết công cụ nào phù hợp nhất với ứng dụng cụ thể của mình. Nếu ta cần điều chỉnh các tham số tinh chỉnh của một cơ sở dữ liệu, hiểu biết này cho phép ta hình dung được một giá trị cao hơn hay thấp hơn có thể có tác động gì.

Although this chapter couldn't make you an expert in tuning any one particular storage engine, it has hopefully equipped you with enough vocabulary and ideas that you can make sense of the documentation for the database of your choice.

Mặc dù chương này không thể biến ta thành chuyên gia trong việc tinh chỉnh bất kỳ một engine lưu trữ cụ thể nào, hy vọng nó đã trang bị cho ta đủ vốn từ vựng và ý tưởng để ta có thể hiểu được tài liệu của cơ sở dữ liệu mà ta lựa chọn.

Footnotes If all keys and values had a fixed size, you could use binary search on a segment file and avoid the in-memory index entirely. However, they are usually variable-length in practice, which makes it difficult to tell where one record ends and the next one starts if you don't have an index.

Inserting a new key into a B-tree is reasonably intuitive, but deleting one (while keeping the tree balanced) is somewhat more involved [2].

This variant is sometimes known as a B tree, although the optimization is so common that it often isn't distinguished from other B-tree variants.

The meaning of online in OLAP is unclear; it probably refers to the fact that queries are not just for predefined reports, but that analysts use the OLAP system interactively for explorative queries.

References [1] Alfred V. Aho, John E. Hopcroft, and Jeffrey D. Ullman: Data Structures and Algorithms. Addison-Wesley, 1983. ISBN: 978-0-201-00023-8

[2] Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, and Clifford Stein: Introduction to Algorithms, 3rd edition. MIT Press, 2009. ISBN: 978-0-262-53305-8

[3] Justin Sheehy and David Smith: "Bitcask: A Log-Structured Hash Table for Fast Key/Value Data," Basho Technologies, April 2010.

[4] Yinan Li, Bingsheng He, Robin Jun Yang, et al.: "Tree Indexing on Solid State Drives," Proceedings of the VLDB Endowment, volume 3, number 1, pages 1195–1206, September 2010.

[5] Goetz Graefe: "Modern B-Tree Techniques," Foundations and Trends in Databases, volume 3, number 4, pages 203–402, August 2011. doi:10.1561/19000000028

[6] Jeffrey Dean and Sanjay Ghemawat: "LevelDB Implementation Notes," leveldb.googlecode.com.

[7] Dhruba Borthakur: "The History of RocksDB," rocksdb.blogspot.com, November 24, 2013.

[8] Matteo Bertozzi: "Apache HBase I/O – HFile," blog.cloudera.com, June, 29 2012.

[9] Fay Chang, Jeffrey Dean, Sanjay Ghemawat, et al.: "Bigtable: A Distributed Storage System for Structured Data," at 7th USENIX Symposium on Operating System Design and Implementation (OSDI), November 2006.

[10] Patrick O'Neil, Edward Cheng, Dieter Gawlick, and Elizabeth O'Neil: "The Log-Structured Merge-Tree (LSM-Tree)," Acta Informatica, volume 33, number 4, pages 351–385, June 1996. doi:10.1007/s002360050048

[11] Mendel Rosenblum and John K. Ousterhout: "The Design and Implementation of a Log-Structured File System," ACM Transactions on Computer Systems, volume 10, number 1, pages 26–52, February 1992. doi:10.1145/146941.146943

[12] Adrien Grand: "What Is in a Lucene Index?," at Lucene/Solr Revolution, November 14, 2013.

[13] Deepak Kandepet: "Hacking Lucene—The Index Format," hackerlabs.org, October 1, 2011.

- [14] Michael McCandless: “Visualizing Lucene’s Segment Merges,” blog.mikemccandless.com, February 11, 2011.
- [15] Burton H. Bloom: “Space/Time Trade-offs in Hash Coding with Allowable Errors,” *Communications of the ACM*, volume 13, number 7, pages 422–426, July 1970. doi:10.1145/362686.362692
- [16] “Operating Cassandra: Compaction,” *Apache Cassandra Documentation v4.0*, 2016.
- [17] Rudolf Bayer and Edward M. McCreight: “Organization and Maintenance of Large Ordered Indices,” *Boeing Scientific Research Laboratories, Mathematical and Information Sciences Laboratory*, report no. 20, July 1970.
- [18] Douglas Comer: “The Ubiquitous B-Tree,” *ACM Computing Surveys*, volume 11, number 2, pages 121–137, June 1979. doi:10.1145/356770.356776
- [19] Emmanuel Goossaert: “Coding for SSDs,” codecapsule.com, February 12, 2014.
- [20] C. Mohan and Frank Levine: “ARIES/IM: An Efficient and High Concurrency Index Management Method Using Write-Ahead Logging,” at *ACM International Conference on Management of Data (SIGMOD)*, June 1992. doi:10.1145/130283.130338
- [21] Howard Chu: “LDAP at Lightning Speed,” at *Build Stuff ’14*, November 2014.
- [22] Bradley C. Kuszmaul: “A Comparison of Fractal Trees to Log-Structured Merge (LSM) Trees,” tokutek.com, April 22, 2014.
- [23] Manos Athanassoulis, Michael S. Kester, Lukas M. Maas, et al.: “Designing Access Methods: The RUM Conjecture,” at *19th International Conference on Extending Database Technology (EDBT)*, March 2016. doi:10.5441/002/edbt.2016.42
- [24] Peter Zaitsev: “Innodb Double Write,” percona.com, August 4, 2006.
- [25] Tomas Vondra: “On the Impact of Full-Page Writes,” blog.2ndquadrant.com, November 23, 2016.
- [26] Mark Callaghan: “The Advantages of an LSM vs a B-Tree,” smallldatum.blogspot.co.uk, January 19, 2016.
- [27] Mark Callaghan: “Choosing Between Efficiency and Performance with RocksDB,” at *Code Mesh*, November 4, 2016.
- [28] Michi Mutsuzaki: “MySQL vs. LevelDB,” github.com, August 2011.
- [29] Benjamin Coverston, Jonathan Ellis, et al.: “CASSANDRA-1608: Redesigned Compaction,” issues.apache.org, July 2011.
- [30] Igor Canadi, Siying Dong, and Mark Callaghan: “RocksDB Tuning Guide,” github.com, 2016.
- [31] *MySQL 5.7 Reference Manual*. Oracle, 2014.
- [32] *Books Online for SQL Server 2012*. Microsoft, 2012.
- [33] Joe Webb: “Using Covering Indexes to Improve Query Performance,” simple-talk.com, 29 September 2008.
- [34] Frank Ramsak, Volker Markl, Robert Fenk, et al.: “Integrating the UB-Tree into a Database System Kernel,” at *26th International Conference on Very Large Data Bases (VLDB)*, September 2000.
- [35] The PostGIS Development Group: “PostGIS 2.1.2dev Manual,” postgis.net, 2014.
- [36] Robert Escriva, Bernard Wong, and Emin Gün Sirer: “HyperDex: A Distributed, Searchable Key-Value Store,” at *ACM SIGCOMM Conference*, August 2012. doi:10.1145/2377677.2377681

- [37] Michael McCandless: “Lucene’s FuzzyQuery Is 100 Times Faster in 4.0,” blog.mikemccandless.com, March 24, 2011.
- [38] Steffen Heinz, Justin Zobel, and Hugh E. Williams: “Burst Tries: A Fast, Efficient Data Structure for String Keys,” *ACM Transactions on Information Systems*, volume 20, number 2, pages 192–223, April 2002. doi:10.1145/506309.506312
- [39] Klaus U. Schulz and Stoyan Mihov: “Fast String Correction with Levenshtein Automata,” *International Journal on Document Analysis and Recognition*, volume 5, number 1, pages 67–85, November 2002. doi:10.1007/s10032-002-0082-8
- [40] Christopher D. Manning, Prabhakar Raghavan, and Hinrich Schütze: *Introduction to Information Retrieval*. Cambridge University Press, 2008. ISBN: 978-0-521-86571-5, available online at nlp.stanford.edu/IR-book
- [41] Michael Stonebraker, Samuel Madden, Daniel J. Abadi, et al.: “The End of an Architectural Era (It’s Time for a Complete Rewrite),” at 33rd International Conference on Very Large Data Bases (VLDB), September 2007.
- [42] “VoltDB Technical Overview White Paper,” VoltDB, 2014.
- [43] Stephen M. Rumble, Ankita Kejriwal, and John K. Ousterhout: “Log-Structured Memory for DRAM-Based Storage,” at 12th USENIX Conference on File and Storage Technologies (FAST), February 2014.
- [44] Stavros Harizopoulos, Daniel J. Abadi, Samuel Madden, and Michael Stonebraker: “OLTP Through the Looking Glass, and What We Found There,” at ACM International Conference on Management of Data (SIGMOD), June 2008. doi:10.1145/1376616.1376713
- [45] Justin DeBrabant, Andrew Pavlo, Stephen Tu, et al.: “Anti-Caching: A New Approach to Database Management System Architecture,” *Proceedings of the VLDB Endowment*, volume 6, number 14, pages 1942–1953, September 2013.
- [46] Joy Arulraj, Andrew Pavlo, and Subramanya R. Dulloor: “Let’s Talk About Storage & Recovery Methods for Non-Volatile Memory Database Systems,” at ACM International Conference on Management of Data (SIGMOD), June 2015. doi:10.1145/2723372.2749441
- [47] Edgar F. Codd, S. B. Codd, and C. T. Salley: “Providing OLAP to User-Analysts: An IT Mandate,” E. F. Codd Associates, 1993.
- [48] Surajit Chaudhuri and Umeshwar Dayal: “An Overview of Data Warehousing and OLAP Technology,” *ACM SIGMOD Record*, volume 26, number 1, pages 65–74, March 1997. doi:10.1145/248603.248616
- [49] Per-Åke Larson, Cipri Clinciu, Campbell Fraser, et al.: “Enhancements to SQL Server Column Stores,” at ACM International Conference on Management of Data (SIGMOD), June 2013.
- [50] Franz Färber, Norman May, Wolfgang Lehner, et al.: “The SAP HANA Database – An Architecture Overview,” *IEEE Data Engineering Bulletin*, volume 35, number 1, pages 28–33, March 2012.
- [51] Michael Stonebraker: “The Traditional RDBMS Wisdom Is (Almost Certainly) All Wrong,” presentation at EPFL, May 2013.
- [52] Daniel J. Abadi: “Classifying the SQL-on-Hadoop Solutions,” hadapt.com, October 2, 2013.
- [53] Marcel Kornacker, Alexander Behm, Victor Bittorf, et al.: “Impala: A Modern, Open-Source SQL Engine for Hadoop,” at 7th Biennial Conference on Innovative Data Systems Research (CIDR), January 2015.

- [54] Sergey Melnik, Andrey Gubarev, Jing Jing Long, et al.: “Dremel: Interactive Analysis of Web-Scale Datasets,” at 36th International Conference on Very Large Data Bases (VLDB), pages 330–339, September 2010.
- [55] Ralph Kimball and Margy Ross: *The Data Warehouse Toolkit: The Definitive Guide to Dimensional Modeling*, 3rd edition. John Wiley & Sons, July 2013. ISBN: 978-1-118-53080-1
- [56] Derrick Harris: “Why Apple, eBay, and Walmart Have Some of the Biggest Data Warehouses You’ve Ever Seen,” gigaom.com, March 27, 2013.
- [57] Julien Le Dem: “Dremel Made Simple with Parquet,” blog.twitter.com, September 11, 2013.
- [58] Daniel J. Abadi, Peter Boncz, Stavros Harizopoulos, et al.: “The Design and Implementation of Modern Column-Oriented Database Systems,” *Foundations and Trends in Databases*, volume 5, number 3, pages 197–280, December 2013. doi:10.1561/19000000024
- [59] Peter Boncz, Marcin Zukowski, and Niels Nes: “MonetDB/X100: Hyper-Pipelining Query Execution,” at 2nd Biennial Conference on Innovative Data Systems Research (CIDR), January 2005.
- [60] Jingren Zhou and Kenneth A. Ross: “Implementing Database Operations Using SIMD Instructions,” at ACM International Conference on Management of Data (SIGMOD), pages 145–156, June 2002. doi:10.1145/564691.564709
- [61] Michael Stonebraker, Daniel J. Abadi, Adam Batkin, et al.: “C-Store: A Column-oriented DBMS,” at 31st International Conference on Very Large Data Bases (VLDB), pages 553–564, September 2005.
- [62] Andrew Lamb, Matt Fuller, Ramakrishna Varadarajan, et al.: “The Vertica Analytic Database: C-Store 7 Years Later,” *Proceedings of the VLDB Endowment*, volume 5, number 12, pages 1790–1801, August 2012.
- [63] Julien Le Dem and Nong Li: “Efficient Data Storage for Analytics with Apache Parquet 2.0,” at Hadoop Summit, San Jose, June 2014.
- [64] Jim Gray, Surajit Chaudhuri, Adam Bosworth, et al.: “Data Cube: A Relational Aggregation Operator Generalizing Group-By, Cross-Tab, and Sub-Totals,” *Data Mining and Knowledge Discovery*, volume 1, number 1, pages 29–53, March 2007. doi:10.1023/A:1009726021843

Chapter 4. Encoding and Evolution

— Chương 4. Mã hóa và Tiến hóa

Everything changes and nothing stands still.

Vạn vật đều biến đổi, không gì đứng yên.

Heraclitus of Ephesus, as quoted by Plato in Cratylus (360 BCE)

Heraclitus xứ Ephesus, theo lời thuật của Plato trong Cratylus (360 TCN)

Applications inevitably change over time. Features are added or modified as new products are launched, user requirements become better understood, or business circumstances change. In Chapter 1 we introduced the idea of **evolvability**: we should aim to build systems that make it easy to adapt to change (see “Evolvability: Making Change Easy”).

Các ứng dụng tất yếu sẽ thay đổi theo thời gian. Tính năng được thêm vào hoặc chỉnh sửa khi sản phẩm mới ra mắt, khi ta hiểu rõ hơn các yêu cầu của người dùng, hoặc khi hoàn cảnh kinh doanh thay đổi. Ở Chương 1, ta đã giới thiệu ý tưởng về khả năng tiến hóa (evolvability): ta nên hướng tới việc xây dựng những hệ thống dễ dàng thích ứng với sự thay đổi (xem “Khả năng tiến hóa: Làm cho việc thay đổi trở nên dễ dàng”).

In most cases, a change to an application’s features also requires a change to data that it stores: perhaps a new field or record type needs to be captured, or perhaps existing data needs to be presented in a new way.

Trong hầu hết trường hợp, một thay đổi ở tính năng của ứng dụng cũng đòi hỏi một thay đổi tương ứng ở dữ liệu mà nó lưu trữ: có lẽ cần ghi nhận một trường hoặc kiểu bản ghi mới, hoặc có lẽ dữ liệu hiện có cần được trình bày theo một cách mới.

The data models we discussed in Chapter 2 have different ways of coping with such change. Relational databases generally assume that all data in the **database** conforms to one **schema**: although that schema can be changed (through schema migrations; i.e., ALTER statements), there is exactly one schema in force at any one point in time. By contrast, **schema-on-read** (“**schemaless**”) databases don’t enforce a schema, so the database can contain a mixture of older and newer data formats written at different times (see “**Schema flexibility** in the **document model**”).

Các mô hình dữ liệu mà ta đã bàn ở Chương 2 có những cách khác nhau để đối phó với sự thay đổi như vậy. Cơ sở dữ liệu quan hệ nhìn chung giả định rằng mọi dữ liệu trong cơ sở dữ liệu đều tuân theo một lược đồ (schema): mặc dù lược đồ đó có thể bị thay đổi (thông qua các phép di trú lược đồ; tức là các câu lệnh ALTER), nhưng tại bất kỳ thời điểm nào cũng chỉ có đúng một lược đồ đang có hiệu lực. Ngược lại, cơ sở dữ liệu lược đồ-lúc-đọc (schema-on-read) (“phi lược đồ”) không áp đặt một lược đồ nào, nên cơ sở dữ liệu có thể

chứa hỗn hợp các định dạng dữ liệu cũ và mới được ghi vào ở những thời điểm khác nhau (xem “Tính linh hoạt lược đồ trong mô hình tài liệu”).

When a data format or schema changes, a corresponding change to **application code** often needs to happen (for example, you add a new field to a record, and the application code starts reading and writing that field). However, in a large application, code changes often cannot happen instantaneously:

Khi một định dạng dữ liệu hoặc lược đồ thay đổi, thường cần có một thay đổi tương ứng ở mã ứng dụng (chẳng hạn, bạn thêm một trường mới vào bản ghi, và mã ứng dụng bắt đầu đọc và ghi trường đó). Tuy nhiên, trong một ứng dụng lớn, các thay đổi mã thường không thể diễn ra tức thời:

- With server-side applications you may want to perform a **rolling upgrade** (also known as a staged rollout), deploying the new version to a few nodes at a time, checking whether the new version is running smoothly, and gradually working your way through all the nodes. This allows new versions to be deployed without service **downtime**, and thus encourages more frequent releases and better evolvability.
- With **client**-side applications you're at the mercy of the user, who may not install the update for some time.
 - Với các ứng dụng phía máy chủ, bạn có thể muốn thực hiện một đợt nâng cấp cuốn chiếu (rolling upgrade, còn gọi là triển khai theo từng giai đoạn), tức là triển khai phiên bản mới lên vài nút mỗi lần, kiểm tra xem phiên bản mới có chạy trơn tru không, rồi dần dần làm việc qua toàn bộ các nút. Cách này cho phép triển khai phiên bản mới mà không phải ngừng dịch vụ, qua đó khuyến khích phát hành thường xuyên hơn và khả năng tiến hóa tốt hơn.
 - Với các ứng dụng phía client, bạn phải trông chờ vào người dùng, những người có thể không cài đặt bản cập nhật trong một thời gian.

This means that old and new versions of the code, and old and new data formats, may potentially all coexist in the system at the same time. In order for the system to continue running smoothly, we need to maintain compatibility in both directions:

Điều này nghĩa là các phiên bản cũ và mới của mã, cùng các định dạng dữ liệu cũ và mới, đều có khả năng cùng tồn tại trong hệ thống tại cùng một thời điểm. Để hệ thống tiếp tục chạy trơn tru, ta cần duy trì tính tương thích theo cả hai chiều:

Newer code can read data that was written by older code.

Mã mới hơn có thể đọc dữ liệu được ghi bởi mã cũ hơn.

Older code can read data that was written by newer code.

Mã cũ hơn có thể đọc dữ liệu được ghi bởi mã mới hơn.

Backward compatibility is normally not hard to achieve: as author of the newer code, you know the format of data written by older code, and so you can explicitly handle it (if necessary by simply keeping the old code to read the old data). Forward compatibility can be trickier, because it requires older code to ignore additions made by a newer version of the code.

Tính tương thích ngược (backward compatibility) thường không khó đạt được: với tư cách là tác giả của mã mới hơn, bạn biết định dạng dữ liệu được ghi bởi mã cũ hơn, nên bạn có thể xử lý nó một cách tường minh (nếu cần thì chỉ đơn giản là giữ lại mã cũ để đọc dữ liệu cũ). Tính tương thích xuôi (forward compatibility) có thể khó hơn, vì nó đòi hỏi mã cũ hơn phải bỏ qua những phần được thêm vào bởi một phiên bản mã mới hơn.

In this chapter we will look at several formats for encoding data, including **JSON**, **XML**, Protocol Buffers, Thrift, and Avro. In particular, we will look at how they handle schema changes and how they support systems where old and new data and code need to coexist. We will then discuss how those formats are used for data storage and for communication: in web services, Representational State Transfer (REST), and remote procedure calls (RPC), as well as message-passing systems such as actors and message queues.

Trong chương này, ta sẽ xem xét một số định dạng để mã hóa dữ liệu, bao gồm JSON, XML, Protocol Buffers, Thrift và Avro. Cụ thể, ta sẽ xem chúng xử lý các thay đổi lược đồ như thế nào và hỗ trợ ra sao cho những hệ thống mà dữ liệu cùng mã cũ và mới cần cùng tồn tại. Sau đó, ta sẽ bàn về cách các định dạng đó được dùng cho việc lưu trữ dữ liệu và cho việc giao tiếp: trong các dịch vụ web, Representational State Transfer (REST), và lời gọi thủ tục từ xa (remote procedure call - RPC), cũng như các hệ thống truyền thông điệp như actor và hàng đợi thông điệp.

Formats for Encoding Data — Các định dạng mã hóa dữ liệu

Programs usually work with data in (at least) two different representations:

Các chương trình thường làm việc với dữ liệu ở (ít nhất) hai cách biểu diễn khác nhau:

- In memory, data is kept in objects, structs, lists, arrays, hash tables, trees, and so on. These data structures are optimized for efficient access and manipulation by the CPU (typically using pointers).
- When you want to write data to a file or send it over the network, you have to encode it as some kind of self-contained sequence of bytes (for example, a JSON **document**). Since a pointer wouldn't make sense to any other process, this sequence-of-bytes representation looks quite different from the data structures that are normally used in memory.
 - Trong bộ nhớ, dữ liệu được giữ trong các đối tượng, struct, danh sách, mảng, bảng băm, cây, v.v. Những cấu trúc dữ liệu này được tối ưu cho việc CPU truy cập và thao tác một cách hiệu quả (thường dùng con trỏ).
 - Khi bạn muốn ghi dữ liệu ra một tập tin hoặc gửi nó qua mạng, bạn phải mã hóa nó thành một dạng chuỗi byte tự chứa nào đó (chẳng hạn một tài liệu JSON). Vì một con trỏ sẽ chẳng có ý nghĩa gì với bất kỳ tiến trình nào khác, cách biểu diễn dạng chuỗi-byte này trông khá khác so với các cấu trúc dữ liệu thường dùng trong bộ nhớ.

Thus, we need some kind of translation between the two representations. The translation from the in-memory representation to a byte sequence is called encoding (also known as serialization or marshalling), and the reverse is called decoding (parsing, deserialization, unmarshalling).

Do đó, ta cần một dạng chuyển đổi nào đó giữa hai cách biểu diễn. Việc chuyển đổi từ cách biểu diễn trong bộ nhớ sang một chuỗi byte được gọi là mã hóa (encoding, còn gọi là tuần tự hóa hay marshalling), và chiều ngược lại được gọi là giải mã (decoding, hay phân tích cú pháp, giải tuần tự hóa, unmarshalling).

Terminology clash — Xung đột thuật ngữ

Serialization is unfortunately also used in the context of transactions (see Chapter 7), with a completely different meaning. To avoid overloading the word we'll stick with encoding in this book, even though serialization is perhaps a more common term.

Tiếc rằng tuần tự hóa (serialization) cũng được dùng trong ngữ cảnh giao dịch (xem Chương 7), với một nghĩa hoàn toàn khác. Để tránh dùng quá tải một từ, trong cuốn sách này ta sẽ nhất quán dùng từ mã hóa, dù tuần tự hóa có lẽ là một thuật ngữ phổ biến hơn.

As this is such a common problem, there are a myriad different libraries and encoding formats to choose from. Let's do a brief overview.

Vì đây là một bài toán quá phổ biến, có vô số thư viện và định dạng mã hóa khác nhau để lựa chọn. Hãy điểm qua một cách ngắn gọn.

Language-Specific Formats — Các định dạng đặc thù theo ngôn ngữ

Many programming languages come with built-in support for encoding in-memory objects into byte sequences. For example, Java has `java.io.Serializable` [1], Ruby has `Marshal` [2], Python has `pickle` [3], and so on. Many third-party libraries also exist, such as `Kryo` for Java [4].

Nhiều ngôn ngữ lập trình đi kèm hỗ trợ sẵn để mã hóa các đối tượng trong bộ nhớ thành chuỗi byte. Chẳng hạn, Java có `java.io.Serializable` [1], Ruby có `Marshal` [2], Python có `pickle` [3], và cứ thế. Cũng có nhiều thư viện của bên thứ ba, chẳng hạn `Kryo` cho Java [4].

These encoding libraries are very convenient, because they allow in-memory objects to be saved and restored with minimal additional code. However, they also have a number of deep problems:

Những thư viện mã hóa này rất tiện lợi, vì chúng cho phép lưu và khôi phục các đối tượng trong bộ nhớ với rất ít mã bổ sung. Tuy nhiên, chúng cũng có một số vấn đề sâu xa:

- The encoding is often tied to a particular programming language, and reading the data in another language is very difficult. If you store or transmit data in such an encoding, you are committing yourself to your current programming language for potentially a very long time, and precluding integrating your systems with those of other organizations (which may use different languages).
- In order to restore data in the same object types, the decoding process needs to be able to instantiate arbitrary classes. This is frequently a source of security problems [5]: if an attacker

can get your application to decode an arbitrary byte sequence, they can instantiate arbitrary classes, which in turn often allows them to do terrible things such as remotely executing arbitrary code [6, 7].

- Versioning data is often an afterthought in these libraries: as they are intended for quick and easy encoding of data, they often neglect the inconvenient problems of forward and backward compatibility.
- Efficiency (CPU time taken to encode or decode, and the size of the encoded structure) is also often an afterthought. For example, Java's built-in serialization is notorious for its bad performance and bloated encoding [8].
 - Cách mã hóa thường bị buộc chặt vào một ngôn ngữ lập trình cụ thể, và việc đọc dữ liệu trong một ngôn ngữ khác là rất khó. Nếu bạn lưu trữ hoặc truyền dữ liệu bằng một cách mã hóa như vậy, bạn đang tự ràng buộc mình vào ngôn ngữ lập trình hiện tại trong một khoảng thời gian có thể là rất dài, và loại trừ khả năng tích hợp hệ thống của bạn với hệ thống của các tổ chức khác (vốn có thể dùng các ngôn ngữ khác).
 - Để khôi phục dữ liệu về đúng các kiểu đối tượng ban đầu, quá trình giải mã cần có khả năng khởi tạo các lớp tùy ý. Đây thường là một nguồn gốc của các vấn đề bảo mật [5]: nếu một kẻ tấn công có thể khiến ứng dụng của bạn giải mã một chuỗi byte tùy ý, chúng có thể khởi tạo các lớp tùy ý, và điều này lại thường cho phép chúng làm những điều khủng khiếp như thực thi mã tùy ý từ xa [6, 7].
 - Việc gắn phiên bản cho dữ liệu thường bị xem nhẹ trong các thư viện này: vì chúng được thiết kế để mã hóa dữ liệu một cách nhanh chóng và dễ dàng, chúng thường bỏ qua những vấn đề phiên toái về tính tương thích xuôi và ngược.
 - Hiệu năng (thời gian CPU để mã hóa hoặc giải mã, và kích thước của cấu trúc đã mã hóa) cũng thường bị xem nhẹ. Chẳng hạn, cơ chế tuần tự hóa sẵn có của Java nổi tiếng vì hiệu năng tệ và phần mã hóa cồng kềnh [8].

For these reasons it's generally a bad idea to use your language's built-in encoding for anything other than very transient purposes.

Vì những lý do này, nhìn chung là một ý tưởng tồi nếu dùng cơ chế mã hóa sẵn có của ngôn ngữ cho bất kỳ mục đích nào ngoài những mục đích rất tạm thời.

JSON, XML, and Binary Variants — JSON, XML và các biến thể nhị phân

Moving to standardized encodings that can be written and read by many programming languages, JSON and XML are the obvious contenders. They are widely known, widely supported, and almost as widely disliked. XML is often criticized for being too verbose and unnecessarily complicated [9]. JSON's popularity is mainly due to its built-in support in web browsers (by virtue of being a subset of JavaScript) and **simplicity** relative to XML. CSV is another popular language-independent format, albeit less powerful.

Chuyển sang các cách mã hóa được chuẩn hóa mà nhiều ngôn ngữ lập trình có thể ghi và đọc, JSON và XML là những ứng viên hiển nhiên. Chúng được biết đến rộng rãi, được hỗ trợ rộng rãi, và cũng bị chê bai gần như rộng rãi không kém. XML thường bị phê bình là quá dài dòng và phức tạp không cần thiết [9]. Sự phổ biến của JSON chủ yếu nhờ việc nó được hỗ trợ sẵn trong các trình duyệt web (do là một tập con của JavaScript) và nhờ tính đơn giản so với XML. CSV là một định dạng độc lập với ngôn ngữ phổ biến khác, dù kém mạnh mẽ hơn.

JSON, XML, and CSV are textual formats, and thus somewhat human-readable (although the syntax is a popular topic of debate). Besides the superficial syntactic issues, they also have some subtle problems:

JSON, XML và CSV là các định dạng văn bản, do đó phần nào con người đọc được (mặc dù cú pháp là một chủ đề tranh luận phổ biến). Bên cạnh những vấn đề cú pháp bề mặt, chúng còn có một số vấn đề tinh tế:

- There is a lot of ambiguity around the encoding of numbers. In XML and CSV, you cannot distinguish between a number and a string that happens to consist of digits (except by referring to an external schema). JSON distinguishes strings and numbers, but it doesn't distinguish integers and floating-point numbers, and it doesn't specify a precision. This is a problem when dealing with large numbers; for example, integers greater than 2 cannot be exactly represented in an IEEE 754 double-precision floating-point number, so such numbers become inaccurate when parsed in a language that uses floating-point numbers (such as JavaScript). An example of numbers larger than 2 occurs on Twitter, which uses a 64-bit number to identify each tweet. The JSON returned by Twitter's **API** includes tweet IDs twice, once as a JSON number and once as a decimal string, to work around the fact that the numbers are not correctly parsed by JavaScript applications [10].
- JSON and XML have good support for Unicode character strings (i.e., human-readable text), but they don't support binary strings (sequences of bytes without a character encoding). Binary strings are a useful feature, so people get around this limitation by encoding the binary data as text using Base64. The schema is then used to indicate that the value should be interpreted as Base64-encoded. This works, but it's somewhat hacky and increases the data size by 33%.
- There is optional schema support for both XML [11] and JSON [12]. These schema languages are quite powerful, and thus quite complicated to learn and implement. Use of XML schemas is fairly widespread, but many JSON-based tools don't bother using schemas. Since the correct interpretation of data (such as numbers and binary strings) depends on information in the schema, applications that don't use XML/JSON schemas need to potentially hardcode the appropriate encoding/decoding logic instead.
- CSV does not have any schema, so it is up to the application to define the meaning of each **row** and **column**. If an application change adds a new row or column, you have to handle that change manually. CSV is also a quite vague format (what happens if a value contains a comma or a newline character?). Although its escaping rules have been formally specified [13], not all parsers implement them correctly.
 - Có rất nhiều sự mơ hồ quanh việc mã hóa các con số. Trong XML và CSV, bạn không thể phân biệt một con số với một chuỗi tình cờ chỉ gồm các chữ số (trừ khi tham chiếu tới một lược đồ bên ngoài). JSON phân biệt chuỗi và số, nhưng nó không phân biệt số nguyên với số dấu phẩy động, và nó không quy định độ chính xác. Đây là một vấn đề khi xử lý các con số lớn; chẳng hạn, các số nguyên lớn hơn 2 không thể được biểu diễn chính xác trong một số dấu phẩy động độ chính xác kép theo chuẩn IEEE 754, nên những con số như vậy trở nên không chính xác khi được phân tích trong một ngôn ngữ dùng số dấu phẩy động (chẳng hạn JavaScript). Một ví dụ về các con số lớn hơn 2 xảy ra trên Twitter, vốn dùng một số 64-bit để định danh mỗi tweet. JSON do API của Twitter trả về chứa ID của tweet hai lần, một lần dưới dạng số JSON và một lần dưới dạng chuỗi thập phân, để né tránh thực tế là các con số không được các ứng dụng JavaScript phân tích đúng [10].
 - JSON và XML hỗ trợ tốt các chuỗi ký tự Unicode (tức văn bản con người đọc được), nhưng chúng không hỗ trợ các chuỗi nhị phân (chuỗi byte không kèm bộ mã hóa ký tự). Chuỗi nhị phân là một tính năng hữu ích, nên người ta lách qua hạn chế này bằng cách mã hóa dữ liệu nhị phân thành văn bản dùng Base64. Lược đồ khi đó được dùng

để chỉ ra rằng giá trị nên được hiểu là dạng đã mã hóa Base64. Cách này hoạt động được, nhưng hơi chấp vá và làm tăng kích thước dữ liệu thêm 33%.

- Cả XML [11] và JSON [12] đều có hỗ trợ lược đồ tùy chọn. Các ngôn ngữ lược đồ này khá mạnh, và do đó khá phức tạp để học và hiện thực. Việc dùng lược đồ XML khá phổ biến, nhưng nhiều công cụ dựa trên JSON chẳng buồn dùng lược đồ. Vì cách diễn giải dữ liệu đúng đắn (chẳng hạn các con số và chuỗi nhị phân) phụ thuộc vào thông tin trong lược đồ, các ứng dụng không dùng lược đồ XML/JSON có khả năng phải mã hóa cứng logic mã hóa/giải mã phù hợp để thay thế.
- CSV không có bất kỳ lược đồ nào, nên việc định nghĩa ý nghĩa của mỗi hàng và mỗi cột là tùy thuộc vào ứng dụng. Nếu một thay đổi ở ứng dụng thêm một hàng hoặc cột mới, bạn phải xử lý thay đổi đó thủ công. CSV cũng là một định dạng khá mơ hồ (điều gì xảy ra nếu một giá trị chứa dấu phẩy hoặc ký tự xuống dòng?). Mặc dù các quy tắc thoát ký tự của nó đã được đặc tả một cách hình thức [13], không phải mọi bộ phân tích đều hiện thực chúng đúng.

Despite these flaws, JSON, XML, and CSV are good enough for many purposes. It's likely that they will remain popular, especially as data interchange formats (i.e., for sending data from one organization to another). In these situations, as long as people agree on what the format is, it often doesn't matter how pretty or efficient the format is. The difficulty of getting different organizations to agree on anything outweighs most other concerns.

Dù có những khiếm khuyết này, JSON, XML và CSV vẫn đủ tốt cho nhiều mục đích. Nhiều khả năng chúng sẽ vẫn phổ biến, đặc biệt là với vai trò các định dạng trao đổi dữ liệu (tức để gửi dữ liệu từ tổ chức này sang tổ chức khác). Trong những tình huống này, chừng nào mọi người còn thống nhất định dạng là gì, thì thường chẳng quan trọng định dạng đó đẹp để hay hiệu quả đến đâu. Cái khó của việc khiến các tổ chức khác nhau đồng thuận về bất cứ điều gì còn lớn hơn hầu hết các mối quan tâm khác.

Binary encoding — Mã hóa nhị phân

For data that is used only internally within your organization, there is less pressure to use a lowest-common-denominator encoding format. For example, you could choose a format that is more compact or faster to parse. For a small dataset, the gains are negligible, but once you get into the terabytes, the choice of data format can have a big impact.

Với dữ liệu chỉ được dùng nội bộ trong tổ chức của bạn, áp lực phải dùng một định dạng mã hóa theo “mẫu số chung nhỏ nhất” sẽ ít hơn. Chẳng hạn, bạn có thể chọn một định dạng gọn gàng hơn hoặc nhanh hơn để phân tích. Với một tập dữ liệu nhỏ, lợi ích là không đáng kể, nhưng một khi bạn bước vào quy mô terabyte, việc lựa chọn định dạng dữ liệu có thể tạo ra tác động lớn.

JSON is less verbose than XML, but both still use a lot of space compared to binary formats. This observation led to the development of a profusion of binary encodings for JSON (MessagePack, BSON, BSON, BSON, and Smile, to name a few) and for XML (WBXML and Fast Infoset, for example). These formats have been adopted in various niches, but none of them are as widely adopted as the textual versions of JSON and XML.

JSON ít dài dòng hơn XML, nhưng cả hai vẫn dùng nhiều không gian so với các định dạng nhị phân. Quan sát này đã dẫn tới sự ra đời của hàng loạt cách mã hóa nhị phân cho JSON (MessagePack, BSON, BSON, BSON, và Smile, kể vài cái tên) và cho XML (chẳng hạn WBXML và Fast Infoset). Những định dạng này đã được áp dụng trong nhiều ngách khác nhau, nhưng không cái nào được áp dụng rộng rãi bằng các phiên bản văn bản của JSON và XML.

Some of these formats extend the set of datatypes (e.g., distinguishing integers and floating-point numbers, or adding support for binary strings), but otherwise they keep the JSON/XML **data model** unchanged. In particular, since they don't prescribe a schema, they need to include all the object field names within the encoded data. That is, in a binary encoding of the JSON document in Example 4-1, they will need to include the strings `userName`, `favoriteNumber`, and `interests` somewhere.

Một số định dạng trong số này mở rộng tập kiểu dữ liệu (chẳng hạn phân biệt số nguyên với số dấu phẩy động, hoặc thêm hỗ trợ cho chuỗi nhị phân), nhưng ngoài ra chúng giữ nguyên mô hình dữ liệu JSON/XML. Cụ thể, vì chúng không quy định một lược đồ, chúng cần đưa toàn bộ tên trường của đối tượng vào trong dữ liệu đã mã hóa. Tức là, trong một cách mã hóa nhị phân của tài liệu JSON ở Ví dụ 4-1, chúng sẽ cần đưa các chuỗi `userName`, `favoriteNumber` và `interests` vào đâu đó.

Example 4-1. Example record which we will encode in several binary formats in this chapter
— **Ví dụ 4-1. Bản ghi ví dụ mà ta sẽ mã hóa bằng nhiều định dạng nhị phân trong chương này**

```
{
  "userName": "Martin",
  "favoriteNumber": 1337,
  "interests": ["daydreaming", "hacking"]
}
```

Let's look at an example of MessagePack, a binary encoding for JSON. Figure 4-1 shows the byte sequence that you get if you encode the JSON document in Example 4-1 with MessagePack [14]. The first few bytes are as follows:

Hãy xem một ví dụ về MessagePack, một cách mã hóa nhị phân cho JSON. Hình 4-1 cho thấy chuỗi byte bạn nhận được nếu mã hóa tài liệu JSON ở Ví dụ 4-1 bằng MessagePack [14]. Vài byte đầu tiên như sau:

- The first byte, 0x83, indicates that what follows is an object (top four bits = 0x80) with three fields (bottom four bits = 0x03). (In case you're wondering what happens if an object has more than 15 fields, so that the number of fields doesn't fit in four bits, it then gets a different type indicator, and the number of fields is encoded in two or four bytes.)
- The second byte, 0xa8, indicates that what follows is a string (top four bits = 0xa0) that is eight bytes long (bottom four bits = 0x08).
- The next eight bytes are the field name `userName` in ASCII. Since the length was indicated previously, there's no need for any marker to tell us where the string ends (or any escaping).
- The next seven bytes encode the six-letter string value `Martin` with a prefix 0xa6, and so on.
 - Byte đầu tiên, 0x83, cho biết phần tiếp theo là một đối tượng (bốn bit cao = 0x80) với ba trường (bốn bit thấp = 0x03). (Nếu bạn thắc mắc điều gì xảy ra khi một đối tượng có hơn 15 trường, khiến số lượng trường không vừa trong bốn bit, thì khi đó nó nhận một chỉ báo kiểu khác, và số lượng trường được mã hóa trong hai hoặc bốn byte.)
 - Byte thứ hai, 0xa8, cho biết phần tiếp theo là một chuỗi (bốn bit cao = 0xa0) dài tám byte (bốn bit thấp = 0x08).
 - Tám byte tiếp theo là tên trường `userName` ở dạng ASCII. Vì độ dài đã được chỉ ra trước đó, không cần bất kỳ dấu nào để cho ta biết chuỗi kết thúc ở đâu (hay bất kỳ ký tự thoát nào).
 - Bảy byte tiếp theo mã hóa giá trị chuỗi sáu chữ cái `Martin` với một tiền tố 0xa6, và cứ thế.

The binary encoding is 66 bytes long, which is only a little less than the 81 bytes taken by the textual JSON encoding (with whitespace removed). All the binary encodings of JSON are similar in this

regard. It's not clear whether such a small space reduction (and perhaps a speedup in parsing) is worth the loss of human-readability.

Cách mã hóa nhị phân dài 66 byte, chỉ ít hơn một chút so với 81 byte mà cách mã hóa JSON dạng văn bản chiếm (sau khi loại bỏ khoảng trắng). Mọi cách mã hóa nhị phân của JSON đều tương tự về mặt này. Không rõ liệu mức giảm không gian nhỏ như vậy (và có lẽ một chút tăng tốc khi phân tích) có đáng để đánh đổi việc mất khả năng con người đọc được hay không.

In the following sections we will see how we can do much better, and encode the same record in just 32 bytes.

Trong các phần tiếp theo, ta sẽ thấy mình có thể làm tốt hơn nhiều, và mã hóa cùng bản ghi đó chỉ trong 32 byte.

Figure 4-1. Example record (Example 4-1) encoded using MessagePack. — Hình 4-1. Bản ghi ví dụ (Ví dụ 4-1) được mã hóa bằng MessagePack.

Thrift and Protocol Buffers — Thrift và Protocol Buffers

Apache Thrift [15] and Protocol Buffers (protobuf) [16] are binary encoding libraries that are based on the same principle. Protocol Buffers was originally developed at Google, Thrift was originally developed at Facebook, and both were made open source in 2007–08 [17].

Apache Thrift [15] và Protocol Buffers (protobuf) [16] là các thư viện mã hóa nhị phân dựa trên cùng một nguyên lý. Protocol Buffers ban đầu được phát triển tại Google, Thrift ban đầu được phát triển tại Facebook, và cả hai đều được mở mã nguồn vào năm 2007–08 [17].

Both Thrift and Protocol Buffers require a schema for any data that is encoded. To encode the data in Example 4-1 in Thrift, you would describe the schema in the Thrift interface definition language (IDL) like this:

Cả Thrift và Protocol Buffers đều yêu cầu một lược đồ cho bất kỳ dữ liệu nào được mã hóa. Để mã hóa dữ liệu ở Ví dụ 4-1 trong Thrift, bạn sẽ mô tả lược đồ bằng ngôn ngữ định nghĩa giao diện (interface definition language - IDL) của Thrift như thế này:

```
struct Person {
  1: required string      userName,
  2: optional i64         favoriteNumber,
  3: optional list<string> interests
}
```

The equivalent schema definition for Protocol Buffers looks very similar:

Định nghĩa lược đồ tương đương cho Protocol Buffers trông rất giống:

```
message Person {
  required string user_name      = 1;
  optional int64  favorite_number = 2;
  repeated string interests      = 3;
}
```

Thrift and Protocol Buffers each come with a code generation tool that takes a schema definition like the ones shown here, and produces classes that implement the schema in various programming

languages [18]. Your application code can call this generated code to encode or decode records of the schema.

Thrift và Protocol Buffers mỗi cái đều đi kèm một công cụ sinh mã, nhận một định nghĩa lược đồ như những ví dụ trên đây và tạo ra các lớp hiện thực lược đồ đó trong nhiều ngôn ngữ lập trình [18]. Mã ứng dụng của bạn có thể gọi mã được sinh ra này để mã hóa hoặc giải mã các bản ghi của lược đồ.

What does data encoded with this schema look like? Confusingly, Thrift has two different binary encoding formats, called BinaryProtocol and CompactProtocol, respectively. Let's look at BinaryProtocol first. Encoding Example 4-1 in that format takes 59 bytes, as shown in Figure 4-2 [19].

Dữ liệu được mã hóa bằng lược đồ này trông như thế nào? Khá rối, Thrift có hai định dạng mã hóa nhị phân khác nhau, lần lượt gọi là BinaryProtocol và CompactProtocol. Hãy xem BinaryProtocol trước. Việc mã hóa Ví dụ 4-1 ở định dạng đó chiếm 59 byte, như trong Hình 4-2 [19].

Figure 4-2. Example record encoded using Thrift's BinaryProtocol. — Hình 4-2. Bản ghi ví dụ được mã hóa bằng BinaryProtocol của Thrift. Similarly to Figure 4-1, each field has a type annotation (to indicate whether it is a string, integer, list, etc.) and, where required, a length indication (length of a string, number of items in a list). The strings that appear in the data ("Martin", "daydreaming", "hacking") are also encoded as ASCII (or rather, UTF-8), similar to before.

Tương tự như Hình 4-1, mỗi trường có một chú thích kiểu (để cho biết nó là chuỗi, số nguyên, danh sách, v.v.) và, ở những chỗ cần, một chỉ báo độ dài (độ dài của chuỗi, số lượng phần tử trong danh sách). Các chuỗi xuất hiện trong dữ liệu ("Martin", "daydreaming", "hacking") cũng được mã hóa dưới dạng ASCII (hay đúng hơn là UTF-8), tương tự như trước.

The big difference compared to Figure 4-1 is that there are no field names (userName, favoriteNumber, interests). Instead, the encoded data contains field tags, which are numbers (1, 2, and 3). Those are the numbers that appear in the schema definition. Field tags are like aliases for fields—they are a compact way of saying what field we're talking about, without having to spell out the field name.

Khác biệt lớn so với Hình 4-1 là không có tên trường (userName, favoriteNumber, interests). Thay vào đó, dữ liệu đã mã hóa chứa các thẻ trường (field tag), vốn là các con số (1, 2, và 3). Đó là những con số xuất hiện trong định nghĩa lược đồ. Thẻ trường giống như bí danh cho các trường — chúng là một cách gọn gàng để nói ta đang đề cập tới trường nào mà không phải đánh vần ra tên trường.

The Thrift CompactProtocol encoding is semantically equivalent to BinaryProtocol, but as you can see in Figure 4-3, it packs the same information into only 34 bytes. It does this by packing the field type and tag number into a single byte, and by using variable-length integers. Rather than using a full eight bytes for the number 1337, it is encoded in two bytes, with the top bit of each byte used to indicate whether there are still more bytes to come. This means numbers between -64 and 63 are encoded in one byte, numbers between -8192 and 8191 are encoded in two bytes, etc. Bigger numbers use more bytes.

Cách mã hóa CompactProtocol của Thrift tương đương về mặt ngữ nghĩa với BinaryProtocol, nhưng như bạn có thể thấy ở Hình 4-3, nó nhồi cùng lượng thông tin đó vào chỉ 34 byte. Nó làm được điều này bằng cách nhồi kiểu trường và số thẻ vào một byte duy nhất, và bằng cách dùng số nguyên độ dài thay đổi. Thay vì dùng đủ tám byte cho con số 1337, nó được mã hóa trong hai byte, với bit cao nhất của mỗi byte được dùng để cho biết liệu còn

byte nào nữa hay không. Điều này nghĩa là các số từ -64 đến 63 được mã hóa trong một byte, các số từ -8192 đến 8191 được mã hóa trong hai byte, v.v. Các con số lớn hơn dùng nhiều byte hơn.

Figure 4-3. Example record encoded using Thrift's CompactProtocol. — Hình 4-3. Bản ghi ví dụ được mã hóa bằng CompactProtocol của Thrift. Finally, Protocol Buffers (which has only one binary encoding format) encodes the same data as shown in Figure 4-4. It does the bit packing slightly differently, but is otherwise very similar to Thrift's CompactProtocol. Protocol Buffers fits the same record in 33 bytes.

Cuối cùng, Protocol Buffers (vốn chỉ có một định dạng mã hóa nhị phân) mã hóa cùng dữ liệu như trong Hình 4-4. Nó nhồi bit theo cách hơi khác một chút, nhưng ngoài ra rất giống CompactProtocol của Thrift. Protocol Buffers nhét vừa cùng bản ghi đó trong 33 byte.

Figure 4-4. Example record encoded using Protocol Buffers. — Hình 4-4. Bản ghi ví dụ được mã hóa bằng Protocol Buffers. One detail to note: in the schemas shown earlier, each field was marked either required or optional, but this makes no difference to how the field is encoded (nothing in the binary data indicates whether a field was required). The difference is simply that required enables a runtime check that fails if the field is not set, which can be useful for catching bugs.

Một chi tiết cần lưu ý: trong các lược đồ ở trên, mỗi trường được đánh dấu hoặc là required (bắt buộc) hoặc optional (tùy chọn), nhưng điều này không tạo ra khác biệt nào trong cách trường được mã hóa (không có gì trong dữ liệu nhị phân cho biết một trường là bắt buộc hay không). Khác biệt chỉ đơn giản là required bật một phép kiểm tra lúc chạy, phép kiểm tra này sẽ thất bại nếu trường không được thiết lập, điều có thể hữu ích để bắt lỗi.

Field tags and schema evolution — Thẻ trường và tiến hóa lược đồ

We said previously that schemas inevitably need to change over time. We call this **schema evolution**. How do Thrift and Protocol Buffers handle schema changes while keeping backward and forward compatibility?

Ta đã nói trước đó rằng lược đồ tất yếu cần thay đổi theo thời gian. Ta gọi đây là tiến hóa lược đồ (schema evolution). Thrift và Protocol Buffers xử lý các thay đổi lược đồ như thế nào trong khi vẫn giữ tính tương thích ngược và xuôi?

As you can see from the examples, an encoded record is just the concatenation of its encoded fields. Each field is identified by its tag number (the numbers 1, 2, 3 in the sample schemas) and annotated with a datatype (e.g., string or integer). If a field value is not set, it is simply omitted from the encoded record. From this you can see that field tags are critical to the meaning of the encoded data. You can change the name of a field in the schema, since the encoded data never refers to field names, but you cannot change a field's tag, since that would make all existing encoded data invalid.

Như bạn có thể thấy từ các ví dụ, một bản ghi đã mã hóa chỉ là phép nối tiếp các trường đã mã hóa của nó. Mỗi trường được định danh bằng số thẻ của nó (các con số 1, 2, 3 trong các lược đồ mẫu) và được chú thích bằng một kiểu dữ liệu (chẳng hạn chuỗi hoặc số nguyên). Nếu giá trị của một trường không được thiết lập, nó đơn giản bị bỏ khỏi bản ghi đã mã hóa. Từ đó bạn có thể thấy các thẻ trường có vai trò then chốt với ý nghĩa của dữ liệu đã mã hóa. Bạn có thể đổi tên một trường trong lược đồ, vì dữ liệu đã mã hóa không bao giờ tham chiếu tới tên trường, nhưng bạn không thể đổi thẻ của một trường, vì làm vậy sẽ khiến mọi dữ liệu đã mã hóa hiện có trở nên không hợp lệ.

You can add new fields to the schema, provided that you give each field a new tag number. If old code (which doesn't know about the new tag numbers you added) tries to read data written by new code, including a new field with a tag number it doesn't recognize, it can simply ignore that field. The datatype annotation allows the parser to determine how many bytes it needs to skip. This maintains forward compatibility: old code can read records that were written by new code.

Bạn có thể thêm các trường mới vào lược đồ, miễn là bạn gán cho mỗi trường một số thẻ mới. Nếu mã cũ (vốn không biết về các số thẻ mới mà bạn đã thêm) cố đọc dữ liệu được ghi bởi mã mới, bao gồm một trường mới với số thẻ mà nó không nhận ra, nó có thể đơn giản bỏ qua trường đó. Chú thích kiểu dữ liệu cho phép bộ phân tích xác định cần bỏ qua bao nhiêu byte. Điều này duy trì tính tương thích xuôi: mã cũ có thể đọc các bản ghi được ghi bởi mã mới.

What about backward compatibility? As long as each field has a unique tag number, new code can always read old data, because the tag numbers still have the same meaning. The only detail is that if you add a new field, you cannot make it required. If you were to add a field and make it required, that check would fail if new code read data written by old code, because the old code will not have written the new field that you added. Therefore, to maintain backward compatibility, every field you add after the initial **deployment** of the schema must be optional or have a default value.

Còn tính tương thích ngược thì sao? Chừng nào mỗi trường còn có một số thẻ duy nhất, mã mới luôn có thể đọc dữ liệu cũ, vì các số thẻ vẫn mang cùng ý nghĩa. Chi tiết duy nhất là nếu bạn thêm một trường mới, bạn không thể đặt nó là required. Nếu bạn thêm một trường và đặt nó là required, phép kiểm tra đó sẽ thất bại khi mã mới đọc dữ liệu được ghi bởi mã cũ, vì mã cũ sẽ không ghi trường mới mà bạn đã thêm. Do đó, để duy trì tính tương thích ngược, mọi trường bạn thêm sau lần triển khai đầu tiên của lược đồ đều phải là optional hoặc có một giá trị mặc định.

Removing a field is just like adding a field, with backward and forward compatibility concerns reversed. That means you can only remove a field that is optional (a required field can never be removed), and you can never use the same tag number again (because you may still have data written somewhere that includes the old tag number, and that field must be ignored by new code).

Loại bỏ một trường cũng giống như thêm một trường, với các mối quan tâm về tính tương thích ngược và xuôi bị đảo ngược. Tức là bạn chỉ có thể loại bỏ một trường optional (một trường required không bao giờ có thể bị loại bỏ), và bạn không bao giờ được dùng lại cùng số thẻ đó (vì có thể bạn vẫn còn dữ liệu được ghi ở đâu đó có chứa số thẻ cũ, và trường đó phải bị mã mới bỏ qua).

Datatypes and schema evolution — Kiểu dữ liệu và tiến hóa lược đồ

What about changing the datatype of a field? That may be possible—check the documentation for details—but there is a risk that values will lose precision or get truncated. For example, say you change a 32-bit integer into a 64-bit integer. New code can easily read data written by old code, because the parser can fill in any missing bits with zeros. However, if old code reads data written by new code, the old code is still using a 32-bit variable to hold the value. If the decoded 64-bit value won't fit in 32 bits, it will be truncated.

Còn việc thay đổi kiểu dữ liệu của một trường thì sao? Điều đó có thể khả thi — hãy xem tài liệu để biết chi tiết — nhưng có rủi ro là các giá trị sẽ mất độ chính xác hoặc bị cắt cụt. Chẳng hạn, giả sử bạn đổi một số nguyên 32-bit thành một số nguyên 64-bit. Mã mới có thể dễ dàng đọc dữ liệu được ghi bởi mã cũ, vì bộ phân tích có thể điền các bit còn thiếu bằng số không. Tuy nhiên, nếu mã cũ đọc dữ liệu được ghi bởi mã mới, mã cũ vẫn đang

dùng một biến 32-bit để giữ giá trị. Nếu giá trị 64-bit đã giải mã không vừa trong 32 bit, nó sẽ bị cắt cụt.

A curious detail of Protocol Buffers is that it does not have a list or array datatype, but instead has a repeated marker for fields (which is a third option alongside required and optional). As you can see in Figure 4-4, the encoding of a repeated field is just what it says on the tin: the same field tag simply appears multiple times in the record. This has the nice effect that it's okay to change an optional (single-valued) field into a repeated (multi-valued) field. New code reading old data sees a list with zero or one elements (depending on whether the field was present); old code reading new data sees only the last element of the list.

Một chi tiết thú vị của Protocol Buffers là nó không có kiểu dữ liệu danh sách hay mảng, mà thay vào đó có một dấu repeated cho các trường (đây là lựa chọn thứ ba bên cạnh required và optional). Như bạn có thể thấy ở Hình 4-4, cách mã hóa một trường repeated đúng như tên gọi của nó: cùng một thẻ trường đơn giản xuất hiện nhiều lần trong bản ghi. Điều này có hệ quả hay là việc đổi một trường optional (đơn giá trị) thành một trường repeated (đa giá trị) là ổn. Mã mới đọc dữ liệu cũ thấy một danh sách với không hoặc một phần tử (tùy thuộc trường có hiện diện hay không); mã cũ đọc dữ liệu mới chỉ thấy phần tử cuối cùng của danh sách.

Thrift has a dedicated list datatype, which is parameterized with the datatype of the list elements. This does not allow the same evolution from single-valued to multi-valued as Protocol Buffers does, but it has the advantage of supporting nested lists.

Thrift có một kiểu dữ liệu danh sách riêng, được tham số hóa bằng kiểu dữ liệu của các phần tử trong danh sách. Điều này không cho phép kiểu tiến hóa từ đơn giá trị sang đa giá trị như Protocol Buffers, nhưng nó có ưu điểm là hỗ trợ danh sách lồng nhau.

Avro — Avro

Apache Avro [20] is another binary encoding format that is interestingly different from Protocol Buffers and Thrift. It was started in 2009 as a subproject of Hadoop, as a result of Thrift not being a good fit for Hadoop's use cases [21].

Apache Avro [20] là một định dạng mã hóa nhị phân khác, khác biệt một cách thú vị so với Protocol Buffers và Thrift. Nó được khởi xướng năm 2009 như một tiểu dự án của Hadoop, do Thrift không phù hợp với các tình huống sử dụng của Hadoop [21].

Avro also uses a schema to specify the structure of the data being encoded. It has two schema languages: one (Avro IDL) intended for human editing, and one (based on JSON) that is more easily machine-readable.

Avro cũng dùng một lược đồ để đặc tả cấu trúc của dữ liệu được mã hóa. Nó có hai ngôn ngữ lược đồ: một (Avro IDL) dành cho con người soạn thảo, và một (dựa trên JSON) dễ cho máy đọc hơn.

Our example schema, written in Avro IDL, might look like this:

Lược đồ ví dụ của ta, viết bằng Avro IDL, có thể trông như thế này:

```
record Person {  
  string          userName;  
  union { null, long } favoriteNumber = null;  
  array<string>   interests;  
}
```

The equivalent JSON representation of that schema is as follows:

Cách biểu diễn JSON tương đương của lược đồ đó như sau:

```
{
  "type": "record",
  "name": "Person",
  "fields": [
    { "name": "userName",      "type": "string" },
    { "name": "favoriteNumber", "type": ["null", "long"], "default": null },
    { "name": "interests",     "type": { "type": "array", "items": "string" } }
  ]
}
```

First of all, notice that there are no tag numbers in the schema. If we encode our example record (Example 4-1) using this schema, the Avro binary encoding is just 32 bytes long—the most compact of all the encodings we have seen. The breakdown of the encoded byte sequence is shown in Figure 4-5.

Trước hết, hãy chú ý rằng không có số thẻ nào trong lược đồ. Nếu ta mã hóa bản ghi ví dụ (Ví dụ 4-1) bằng lược đồ này, cách mã hóa nhị phân Avro chỉ dài 32 byte — gọn gàng nhất trong tất cả các cách mã hóa ta đã thấy. Phân phân tích chi tiết chuỗi byte đã mã hóa được trình bày ở Hình 4-5.

If you examine the byte sequence, you can see that there is nothing to identify fields or their datatypes. The encoding simply consists of values concatenated together. A string is just a length prefix followed by UTF-8 bytes, but there's nothing in the encoded data that tells you that it is a string. It could just as well be an integer, or something else entirely. An integer is encoded using a variable-length encoding (the same as Thrift's CompactProtocol).

Nếu bạn xem xét chuỗi byte, bạn sẽ thấy không có gì để định danh các trường hay kiểu dữ liệu của chúng. Cách mã hóa đơn giản gồm các giá trị được nối tiếp với nhau. Một chuỗi chỉ là một tiền tố độ dài theo sau là các byte UTF-8, nhưng không có gì trong dữ liệu đã mã hóa cho bạn biết rằng đó là một chuỗi. Nó cũng có thể là một số nguyên, hay một thứ gì đó hoàn toàn khác. Một số nguyên được mã hóa bằng một cách mã hóa độ dài thay đổi (giống như CompactProtocol của Thrift).

Figure 4-5. Example record encoded using Avro. — Hình 4-5. Bản ghi ví dụ được mã hóa bằng Avro. To parse the binary data, you go through the fields in the order that they appear in the schema and use the schema to tell you the datatype of each field. This means that the binary data can only be decoded correctly if the code reading the data is using the exact same schema as the code that wrote the data. Any mismatch in the schema between the reader and the writer would **mean** incorrectly decoded data.

Để phân tích dữ liệu nhị phân, bạn đi qua các trường theo thứ tự chúng xuất hiện trong lược đồ và dùng lược đồ để cho biết kiểu dữ liệu của mỗi trường. Điều này nghĩa là dữ liệu nhị phân chỉ có thể được giải mã đúng nếu mã đang đọc dữ liệu dùng đúng chính xác lược đồ mà mã đã ghi dữ liệu đã dùng. Bất kỳ sự không khớp nào trong lược đồ giữa bên đọc và bên ghi đều sẽ đồng nghĩa với dữ liệu bị giải mã sai.

So, how does Avro support schema evolution?

Vậy thì, Avro hỗ trợ tiến hóa lược đồ như thế nào?

The writer's schema and the reader's schema — Lược đồ của bên ghi và lược đồ của bên đọc

With Avro, when an application wants to encode some data (to write it to a file or database, to send it over the network, etc.), it encodes the data using whatever version of the schema it knows about—for example, that schema may be compiled into the application. This is known as the writer's schema.

Với Avro, khi một ứng dụng muốn mã hóa một số dữ liệu (để ghi ra tập tin hoặc cơ sở dữ liệu, để gửi qua mạng, v.v.), nó mã hóa dữ liệu bằng bất kỳ phiên bản lược đồ nào mà nó biết — chẳng hạn, lược đồ đó có thể được biên dịch sẵn vào ứng dụng. Đây được gọi là lược đồ của bên ghi (writer's schema).

When an application wants to decode some data (read it from a file or database, receive it from the network, etc.), it is expecting the data to be in some schema, which is known as the reader's schema. That is the schema the application code is relying on—code may have been generated from that schema during the application's build process.

Khi một ứng dụng muốn giải mã một số dữ liệu (đọc nó từ tập tin hoặc cơ sở dữ liệu, nhận nó từ mạng, v.v.), nó kỳ vọng dữ liệu tuân theo một lược đồ nào đó, được gọi là lược đồ của bên đọc (reader's schema). Đó là lược đồ mà mã ứng dụng đang dựa vào — mã có thể đã được sinh ra từ lược đồ đó trong quá trình build ứng dụng.

The key idea with Avro is that the writer's schema and the reader's schema don't have to be the same—they only need to be compatible. When data is decoded (read), the Avro library resolves the differences by looking at the writer's schema and the reader's schema side by side and translating the data from the writer's schema into the reader's schema. The Avro specification [20] defines exactly how this resolution works, and it is illustrated in Figure 4-6.

Ý tưởng then chốt của Avro là lược đồ của bên ghi và lược đồ của bên đọc không nhất thiết phải giống nhau — chúng chỉ cần tương thích. Khi dữ liệu được giải mã (đọc), thư viện Avro giải quyết các khác biệt bằng cách đặt lược đồ của bên ghi và lược đồ của bên đọc cạnh nhau và dịch dữ liệu từ lược đồ của bên ghi sang lược đồ của bên đọc. Đặc tả Avro [20] định nghĩa chính xác cách phép giải quyết này hoạt động, và nó được minh họa ở Hình 4-6.

For example, it's no problem if the writer's schema and the reader's schema have their fields in a different order, because the schema resolution matches up the fields by field name. If the code reading the data encounters a field that appears in the writer's schema but not in the reader's schema, it is ignored. If the code reading the data expects some field, but the writer's schema does not contain a field of that name, it is filled in with a default value declared in the reader's schema.

Chẳng hạn, không thành vấn đề nếu lược đồ của bên ghi và lược đồ của bên đọc có các trường ở thứ tự khác nhau, vì phép giải quyết lược đồ khớp các trường theo tên trường. Nếu mã đang đọc dữ liệu gặp một trường xuất hiện trong lược đồ của bên ghi nhưng không có trong lược đồ của bên đọc, nó bị bỏ qua. Nếu mã đang đọc dữ liệu kỳ vọng một trường nào đó, nhưng lược đồ của bên ghi không chứa một trường tên như vậy, nó được điền bằng một giá trị mặc định khai báo trong lược đồ của bên đọc.

Figure 4-6. An Avro reader resolves differences between the writer's schema and the reader's schema. — Hình 4-6. Một bên đọc Avro giải quyết các khác biệt giữa lược đồ của bên ghi và lược đồ của bên đọc.

Schema evolution rules — Các quy tắc tiến hóa lược đồ

With Avro, forward compatibility means that you can have a new version of the schema as writer and an old version of the schema as reader. Conversely, backward compatibility means that you can have a new version of the schema as reader and an old version as writer.

Với Avro, tính tương thích xuôi nghĩa là bạn có thể có một phiên bản mới của lược đồ làm bên ghi và một phiên bản cũ của lược đồ làm bên đọc. Ngược lại, tính tương thích ngược nghĩa là bạn có thể có một phiên bản mới của lược đồ làm bên đọc và một phiên bản cũ làm bên ghi.

To maintain compatibility, you may only add or remove a field that has a default value. (The field `favoriteNumber` in our Avro schema has a default value of `null`.) For example, say you add a field with a default value, so this new field exists in the new schema but not the old one. When a reader using the new schema reads a record written with the old schema, the default value is filled in for the missing field.

Để duy trì tính tương thích, bạn chỉ có thể thêm hoặc loại bỏ một trường có giá trị mặc định. (Trường `favoriteNumber` trong lược đồ Avro của ta có giá trị mặc định là `null`.) Chẳng hạn, giả sử bạn thêm một trường có giá trị mặc định, nên trường mới này tồn tại trong lược đồ mới nhưng không có trong lược đồ cũ. Khi một bên đọc dùng lược đồ mới đọc một bản ghi được ghi bằng lược đồ cũ, giá trị mặc định được điền vào cho trường còn thiếu.

If you were to add a field that has no default value, new readers wouldn't be able to read data written by old writers, so you would break backward compatibility. If you were to remove a field that has no default value, old readers wouldn't be able to read data written by new writers, so you would break forward compatibility.

Nếu bạn thêm một trường không có giá trị mặc định, các bên đọc mới sẽ không thể đọc dữ liệu được ghi bởi các bên ghi cũ, nên bạn sẽ phá vỡ tính tương thích ngược. Nếu bạn loại bỏ một trường không có giá trị mặc định, các bên đọc cũ sẽ không thể đọc dữ liệu được ghi bởi các bên ghi mới, nên bạn sẽ phá vỡ tính tương thích xuôi.

In some programming languages, `null` is an acceptable default for any variable, but this is not the case in Avro: if you want to allow a field to be `null`, you have to use a union type. For example, `union { null, long, string } field;` indicates that field can be a number, or a string, or `null`. You can only use `null` as a default value if it is one of the branches of the union. This is a little more verbose than having everything nullable by default, but it helps prevent bugs by being explicit about what can and cannot be `null` [22].

Trong một số ngôn ngữ lập trình, `null` là một giá trị mặc định chấp nhận được cho bất kỳ biến nào, nhưng điều này không đúng trong Avro: nếu bạn muốn cho phép một trường nhận giá trị `null`, bạn phải dùng một kiểu hợp (union type). Chẳng hạn, `union { null, long, string } field;` cho biết field có thể là một số, một chuỗi, hoặc `null`. Bạn chỉ có thể dùng `null` làm giá trị mặc định nếu nó là một trong các nhánh của union. Cách này hơi dài dòng hơn so với việc để mọi thứ đều có thể nhận `null` theo mặc định, nhưng nó giúp ngăn ngừa bug bằng cách tường minh về việc cái gì có thể và không thể `null` [22].

Consequently, Avro doesn't have optional and required markers in the same way as Protocol Buffers and Thrift do (it has union types and default values instead).

Hệ quả là Avro không có các dấu optional và required theo cùng cách như Protocol Buffers và Thrift (thay vào đó nó có các kiểu union và các giá trị mặc định).

Changing the datatype of a field is possible, provided that Avro can convert the type. Changing the name of a field is possible but a little tricky: the reader's schema can contain aliases for field names,

so it can match an old writer's schema field names against the aliases. This means that changing a field name is backward compatible but not forward compatible. Similarly, adding a branch to a union type is backward compatible but not forward compatible.

Việc thay đổi kiểu dữ liệu của một trường là khả thi, miễn là Avro có thể chuyển đổi kiểu đó. Việc đổi tên một trường là khả thi nhưng hơi rắc rối: lược đồ của bên đọc có thể chứa các bí danh cho tên trường, nên nó có thể khớp các tên trường của lược đồ bên ghi cũ với các bí danh. Điều này nghĩa là việc đổi tên một trường là tương thích ngược nhưng không tương thích xuôi. Tương tự, việc thêm một nhánh vào một kiểu union là tương thích ngược nhưng không tương thích xuôi.

But what is the writer's schema? — Nhưng lược đồ của bên ghi là gì?

There is an important question that we've glossed over so far: how does the reader know the writer's schema with which a particular piece of data was encoded? We can't just include the entire schema with every record, because the schema would likely be much bigger than the encoded data, making all the space savings from the binary encoding futile.

Có một câu hỏi quan trọng mà ta đã lướt qua từ đầu tới giờ: làm sao bên đọc biết được lược đồ của bên ghi mà một mẫu dữ liệu cụ thể đã được mã hóa bằng nó? Ta không thể chỉ đính kèm toàn bộ lược đồ vào mỗi bản ghi, vì lược đồ có khả năng lớn hơn nhiều so với dữ liệu đã mã hóa, khiến mọi khoản tiết kiệm không gian từ cách mã hóa nhị phân trở nên vô nghĩa.

The answer depends on the context in which Avro is being used. To give a few examples:

Câu trả lời phụ thuộc vào ngữ cảnh mà Avro đang được dùng. Nêu vài ví dụ:

A common use for Avro—especially in the context of Hadoop—is for storing a large file containing millions of records, all encoded with the same schema. (We will discuss this kind of situation in Chapter 10.) In this case, the writer of that file can just include the writer's schema once at the beginning of the file. Avro specifies a file format (object container files) to do this.

Một cách dùng phổ biến của Avro — đặc biệt trong ngữ cảnh Hadoop — là để lưu trữ một tập tin lớn chứa hàng triệu bản ghi, tất cả đều được mã hóa bằng cùng một lược đồ. (Ta sẽ bàn về kiểu tình huống này ở Chương 10.) Trong trường hợp này, bên ghi của tập tin đó chỉ cần đính kèm lược đồ của bên ghi một lần ở đầu tập tin. Avro đặc tả một định dạng tập tin (các tập tin container đối tượng) để làm điều này.

In a database, different records may be written at different points in time using different writer's schemas—you cannot assume that all the records will have the same schema. The simplest solution is to include a version number at the beginning of every encoded record, and to keep a list of schema versions in your database. A reader can fetch a record, extract the version number, and then fetch the writer's schema for that version number from the database. Using that writer's schema, it can decode the rest of the record. (Espresso [23] works this way, for example.)

Trong một cơ sở dữ liệu, các bản ghi khác nhau có thể được ghi ở những thời điểm khác nhau bằng các lược đồ bên ghi khác nhau — bạn không thể giả định rằng tất cả các bản ghi sẽ có cùng lược đồ. Giải pháp đơn giản nhất là đính kèm một số phiên bản ở đầu mỗi bản ghi đã mã hóa, và giữ một danh sách các phiên bản lược đồ trong cơ sở dữ liệu của bạn. Một bên đọc có thể lấy một bản ghi, trích xuất số phiên bản, rồi lấy lược đồ của bên ghi ứng với số phiên bản đó từ cơ sở dữ liệu. Dùng lược đồ của bên ghi đó, nó có thể giải mã phần còn lại của bản ghi. (Espresso [23] hoạt động theo cách này, chẳng hạn vậy.)

When two processes are communicating over a bidirectional network connection, they can negotiate the schema version on connection setup and then use that schema for the lifetime of the connection. The Avro RPC protocol (see “Dataflow Through Services: REST and RPC”) works like this.

Khi hai tiến trình đang giao tiếp qua một kết nối mạng hai chiều, chúng có thể thương lượng phiên bản lược đồ lúc thiết lập kết nối rồi dùng lược đồ đó trong suốt vòng đời của kết nối. Giao thức RPC của Avro (xem “Luồng dữ liệu qua dịch vụ: REST và RPC”) hoạt động như thế này.

A database of schema versions is a useful thing to have in any case, since it acts as documentation and gives you a chance to check schema compatibility [24]. As the version number, you could use a simple incrementing integer, or you could use a hash of the schema.

Một cơ sở dữ liệu các phiên bản lược đồ dù sao cũng là một thứ hữu ích nên có, vì nó đóng vai trò tài liệu và cho bạn cơ hội kiểm tra tính tương thích lược đồ [24]. Với số phiên bản, bạn có thể dùng một số nguyên tăng dần đơn giản, hoặc bạn có thể dùng một mã băm của lược đồ.

Dynamically generated schemas — Các lược đồ được sinh ra một cách động

One advantage of Avro’s approach, compared to Protocol Buffers and Thrift, is that the schema doesn’t contain any tag numbers. But why is this important? What’s the problem with keeping a couple of numbers in the schema?

Một ưu điểm trong cách tiếp cận của Avro, so với Protocol Buffers và Thrift, là lược đồ không chứa bất kỳ số thẻ nào. Nhưng tại sao điều này lại quan trọng? Việc giữ vài con số trong lược đồ thì có vấn đề gì?

The difference is that Avro is friendlier to dynamically generated schemas. For example, say you have a **relational database** whose contents you want to dump to a file, and you want to use a binary format to avoid the aforementioned problems with textual formats (JSON, CSV, **SQL**). If you use Avro, you can fairly easily generate an Avro schema (in the JSON representation we saw earlier) from the relational schema and encode the database contents using that schema, dumping it all to an Avro object container file [25]. You generate a record schema for each database **table**, and each column becomes a field in that record. The column name in the database maps to the field name in Avro.

Khác biệt nằm ở chỗ Avro thân thiện hơn với các lược đồ được sinh ra một cách động. Chẳng hạn, giả sử bạn có một cơ sở dữ liệu quan hệ mà bạn muốn đổ nội dung ra một tập tin, và bạn muốn dùng một định dạng nhị phân để tránh những vấn đề đã nói ở trên với các định dạng văn bản (JSON, CSV, SQL). Nếu bạn dùng Avro, bạn có thể khá dễ dàng sinh ra một lược đồ Avro (ở dạng biểu diễn JSON mà ta đã thấy trước đó) từ lược đồ quan hệ và mã hóa nội dung cơ sở dữ liệu bằng lược đồ đó, đổ tất cả ra một tập tin container đối tượng Avro [25]. Bạn sinh một lược đồ bản ghi cho mỗi bảng trong cơ sở dữ liệu, và mỗi cột trở thành một trường trong bản ghi đó. Tên cột trong cơ sở dữ liệu ánh xạ tới tên trường trong Avro.

Now, if the database schema changes (for example, a table has one column added and one column removed), you can just generate a new Avro schema from the updated database schema and export data in the new Avro schema. The data export process does not need to pay any attention to the schema change—it can simply do the schema conversion every time it runs. Anyone who reads the new data files will see that the fields of the record have changed, but since the fields are identified by name, the updated writer’s schema can still be matched up with the old reader’s schema.

Giờ đây, nếu lược đồ cơ sở dữ liệu thay đổi (chẳng hạn, một bảng được thêm một cột và bỏ

một cột), bạn chỉ cần sinh một lược đồ Avro mới từ lược đồ cơ sở dữ liệu đã cập nhật và xuất dữ liệu theo lược đồ Avro mới. Quá trình xuất dữ liệu không cần để tâm gì tới thay đổi lược đồ — nó có thể đơn giản thực hiện việc chuyển đổi lược đồ mỗi lần chạy. Bất kỳ ai đọc các tập tin dữ liệu mới sẽ thấy các trường của bản ghi đã thay đổi, nhưng vì các trường được định danh theo tên, lược đồ của bên ghi đã cập nhật vẫn có thể được khớp với lược đồ của bên đọc cũ.

By contrast, if you were using Thrift or Protocol Buffers for this purpose, the field tags would likely have to be assigned by hand: every time the database schema changes, an administrator would have to manually update the mapping from database column names to field tags. (It might be possible to automate this, but the schema generator would have to be very careful to not assign previously used field tags.) This kind of dynamically generated schema simply wasn't a design goal of Thrift or Protocol Buffers, whereas it was for Avro.

Ngược lại, nếu bạn dùng Thrift hoặc Protocol Buffers cho mục đích này, các thẻ trường có khả năng phải được gán bằng tay: mỗi lần lược đồ cơ sở dữ liệu thay đổi, một quản trị viên sẽ phải cập nhật thủ công ánh xạ từ tên cột cơ sở dữ liệu sang các thẻ trường. (Việc này có thể tự động hóa được, nhưng bộ sinh lược đồ sẽ phải hết sức cẩn thận để không gán những thẻ trường đã dùng trước đó.) Kiểu lược đồ được sinh ra một cách động này đơn giản không phải là một mục tiêu thiết kế của Thrift hay Protocol Buffers, trong khi nó lại là mục tiêu của Avro.

Code generation and dynamically typed languages — Sinh mã và các ngôn ngữ định kiểu động

Thrift and Protocol Buffers rely on code generation: after a schema has been defined, you can generate code that implements this schema in a programming language of your choice. This is useful in statically typed languages such as Java, C++, or C#, because it allows efficient in-memory structures to be used for decoded data, and it allows type checking and autocompletion in IDEs when writing programs that access the data structures.

Thrift và Protocol Buffers dựa vào việc sinh mã: sau khi một lược đồ được định nghĩa, bạn có thể sinh mã hiện thực lược đồ này trong một ngôn ngữ lập trình bạn chọn. Điều này hữu ích trong các ngôn ngữ định kiểu tĩnh như Java, C++ hay C#, vì nó cho phép dùng các cấu trúc trong bộ nhớ hiệu quả cho dữ liệu đã giải mã, và nó cho phép kiểm tra kiểu cùng tự động hoàn thành trong các IDE khi viết các chương trình truy cập các cấu trúc dữ liệu đó.

In dynamically typed programming languages such as JavaScript, Ruby, or Python, there is not much point in generating code, since there is no compile-time type checker to satisfy. Code generation is often frowned upon in these languages, since they otherwise avoid an explicit compilation step. Moreover, in the case of a dynamically generated schema (such as an Avro schema generated from a database table), code generation is an unnecessarily obstacle to getting to the data.

Trong các ngôn ngữ lập trình định kiểu động như JavaScript, Ruby hay Python, việc sinh mã chẳng có nhiều ý nghĩa, vì không có bộ kiểm tra kiểu lúc biên dịch nào để làm hài lòng. Việc sinh mã thường bị nhìn với con mắt không thiện cảm trong các ngôn ngữ này, vì dù sao chúng cũng tránh một bước biên dịch tường minh. Hơn nữa, trong trường hợp một lược đồ được sinh ra một cách động (chẳng hạn một lược đồ Avro được sinh từ một bảng cơ sở dữ liệu), việc sinh mã là một trở ngại không cần thiết để tiếp cận dữ liệu.

Avro provides optional code generation for statically typed programming languages, but it can be used just as well without any code generation. If you have an object container file (which embeds

the writer's schema), you can simply open it using the Avro library and look at the data in the same way as you could look at a JSON file. The file is self-describing since it includes all the necessary metadata.

Avro cung cấp việc sinh mã tùy chọn cho các ngôn ngữ lập trình định kiểu tĩnh, nhưng nó cũng có thể được dùng tốt như vậy mà chẳng cần sinh mã gì cả. Nếu bạn có một tập tin container đối tượng (vốn nhúng sẵn lược đồ của bên ghi), bạn chỉ cần mở nó bằng thư viện Avro và xem dữ liệu theo cách giống như bạn có thể xem một tập tin JSON. Tập tin này tự mô tả vì nó bao gồm mọi siêu dữ liệu cần thiết.

This **property** is especially useful in conjunction with dynamically typed data processing languages like Apache Pig [26]. In Pig, you can just open some Avro files, start analyzing them, and write derived datasets to output files in Avro format without even thinking about schemas.

Tính chất này đặc biệt hữu ích khi kết hợp với các ngôn ngữ xử lý dữ liệu định kiểu động như Apache Pig [26]. Trong Pig, bạn chỉ cần mở một số tập tin Avro, bắt đầu phân tích chúng, và ghi các tập dữ liệu dẫn xuất ra các tập tin đầu ra ở định dạng Avro mà thậm chí không cần nghĩ ngợi gì về các lược đồ.

The Merits of Schemas — Những ưu điểm của lược đồ

As we saw, Protocol Buffers, Thrift, and Avro all use a schema to describe a binary encoding format. Their schema languages are much simpler than XML Schema or JSON Schema, which support much more detailed validation rules (e.g., “the string value of this field must match this regular expression” or “the integer value of this field must be between 0 and 100”). As Protocol Buffers, Thrift, and Avro are simpler to implement and simpler to use, they have grown to support a fairly wide range of programming languages.

Như ta đã thấy, Protocol Buffers, Thrift và Avro đều dùng một lược đồ để mô tả một định dạng mã hóa nhị phân. Các ngôn ngữ lược đồ của chúng đơn giản hơn nhiều so với XML Schema hay JSON Schema, vốn hỗ trợ các quy tắc kiểm tra chi tiết hơn nhiều (chẳng hạn “giá trị chuỗi của trường này phải khớp với biểu thức chính quy này” hoặc “giá trị số nguyên của trường này phải nằm trong khoảng từ 0 đến 100”). Vì Protocol Buffers, Thrift và Avro đơn giản hơn để hiện thực và đơn giản hơn để dùng, chúng đã phát triển để hỗ trợ một dải khá rộng các ngôn ngữ lập trình.

The ideas on which these encodings are based are by no means new. For example, they have a lot in common with ASN.1, a schema definition language that was first standardized in 1984 [27]. It was used to define various network protocols, and its binary encoding (DER) is still used to encode SSL certificates (X.509), for example [28]. ASN.1 supports schema evolution using tag numbers, similar to Protocol Buffers and Thrift [29]. However, it's also very complex and badly documented, so ASN.1 is probably not a good choice for new applications.

Các ý tưởng làm nền tảng cho những cách mã hóa này hoàn toàn không phải là mới. Chẳng hạn, chúng có nhiều điểm chung với ASN.1, một ngôn ngữ định nghĩa lược đồ được chuẩn hóa lần đầu vào năm 1984 [27]. Nó được dùng để định nghĩa nhiều giao thức mạng, và cách mã hóa nhị phân của nó (DER) vẫn được dùng để mã hóa các chứng chỉ SSL (X.509), chẳng hạn vậy [28]. ASN.1 hỗ trợ tiến hóa lược đồ bằng các số thẻ, tương tự Protocol Buffers và Thrift [29]. Tuy nhiên, nó cũng rất phức tạp và được lập tài liệu tồi, nên ASN.1 có lẽ không phải là một lựa chọn tốt cho các ứng dụng mới.

Many data systems also implement some kind of proprietary binary encoding for their data. For example, most relational databases have a network protocol over which you can send queries to the

database and get back responses. Those protocols are generally specific to a particular database, and the database vendor provides a driver (e.g., using the ODBC or JDBC APIs) that decodes responses from the database's network protocol into in-memory data structures.

Nhiều hệ thống dữ liệu cũng hiện thực một dạng mã hóa nhị phân độc quyền nào đó cho dữ liệu của chúng. Chẳng hạn, hầu hết các cơ sở dữ liệu quan hệ đều có một giao thức mạng mà qua đó bạn có thể gửi truy vấn tới cơ sở dữ liệu và nhận về phản hồi. Các giao thức đó nhìn chung đặc thù cho một cơ sở dữ liệu cụ thể, và nhà cung cấp cơ sở dữ liệu cung cấp một trình điều khiển (chẳng hạn dùng API ODBC hoặc JDBC) để giải mã các phản hồi từ giao thức mạng của cơ sở dữ liệu thành các cấu trúc dữ liệu trong bộ nhớ.

So, we can see that although textual data formats such as JSON, XML, and CSV are widespread, binary encodings based on schemas are also a viable option. They have a number of nice properties:

Vậy, ta có thể thấy rằng dù các định dạng dữ liệu dạng văn bản như JSON, XML và CSV rất phổ biến, các cách mã hóa nhị phân dựa trên lược đồ cũng là một lựa chọn khả thi. Chúng có một số tính chất hay:

- They can be much more compact than the various “binary JSON” variants, since they can omit field names from the encoded data.
- The schema is a valuable form of documentation, and because the schema is required for decoding, you can be sure that it is up to date (whereas manually maintained documentation may easily diverge from reality).
- Keeping a database of schemas allows you to check forward and backward compatibility of schema changes, before anything is deployed.
- For users of statically typed programming languages, the ability to generate code from the schema is useful, since it enables type checking at compile time.
 - Chúng có thể gọn gàng hơn nhiều so với các biến thể “JSON nhị phân” khác nhau, vì chúng có thể bỏ tên trường ra khỏi dữ liệu đã mã hóa.
 - Lược đồ là một dạng tài liệu giá trị, và vì lược đồ là bắt buộc để giải mã, bạn có thể chắc chắn rằng nó được cập nhật (trong khi tài liệu duy trì thủ công có thể dễ dàng đi chệch khỏi thực tế).
 - Việc giữ một cơ sở dữ liệu các lược đồ cho phép bạn kiểm tra tính tương thích xuôi và ngược của các thay đổi lược đồ, trước khi bất cứ thứ gì được triển khai.
 - Với người dùng các ngôn ngữ lập trình định kiểu tĩnh, khả năng sinh mã từ lược đồ là hữu ích, vì nó cho phép kiểm tra kiểu lúc biên dịch.

In summary, schema evolution allows the same kind of flexibility as schemaless/schema-on-read JSON databases provide (see “Schema flexibility in the document model”), while also providing better guarantees about your data and better tooling.

Tóm lại, tiến hóa lược đồ cho phép cùng kiểu linh hoạt như các cơ sở dữ liệu phi lược đồ/lược đồ-lúc-đọc JSON cung cấp (xem “Tính linh hoạt lược đồ trong mô hình tài liệu”), đồng thời mang lại những bảo đảm tốt hơn về dữ liệu của bạn và bộ công cụ tốt hơn.

Modes of Dataflow — Các phương thức luồng dữ liệu

At the beginning of this chapter we said that whenever you want to send some data to another process with which you don't share memory—for example, whenever you want to send data over the network or write it to a file—you need to encode it as a sequence of bytes. We then discussed a variety of different encodings for doing this.

Ở đầu chương này, ta đã nói rằng mỗi khi bạn muốn gửi một số dữ liệu tới một tiến trình khác mà bạn không chia sẻ bộ nhớ chung — chẳng hạn, mỗi khi bạn muốn gửi dữ liệu qua mạng hoặc ghi nó ra một tập tin — bạn cần mã hóa nó thành một chuỗi byte. Sau đó, ta đã bàn về nhiều cách mã hóa khác nhau để làm điều này.

We talked about forward and backward compatibility, which are important for evolvability (making change easy by allowing you to upgrade different parts of your system independently, and not having to change everything at once). Compatibility is a relationship between one process that encodes the data, and another process that decodes it.

Ta đã nói về tính tương thích xuôi và ngược, vốn quan trọng với khả năng tiến hóa (làm cho việc thay đổi dễ dàng bằng cách cho phép bạn nâng cấp các phần khác nhau của hệ thống một cách độc lập, và không phải thay đổi mọi thứ cùng một lúc). Tính tương thích là một mối quan hệ giữa một tiến trình mã hóa dữ liệu và một tiến trình khác giải mã nó.

That's a fairly abstract idea—there are many ways data can flow from one process to another. Who encodes the data, and who decodes it? In the rest of this chapter we will explore some of the most common ways how data flows between processes:

Đó là một ý tưởng khá trừu tượng — có nhiều cách dữ liệu có thể chảy từ tiến trình này sang tiến trình khác. Ai mã hóa dữ liệu, và ai giải mã nó? Trong phần còn lại của chương này, ta sẽ khám phá một số cách phổ biến nhất mà dữ liệu chảy giữa các tiến trình:

- Via databases (see “Dataflow Through Databases”)
- Via service calls (see “Dataflow Through Services: REST and RPC”)
- Via asynchronous message passing (see “Message-Passing Dataflow”)
 - Qua cơ sở dữ liệu (xem “Luồng dữ liệu qua cơ sở dữ liệu”)
 - Qua các lời gọi dịch vụ (xem “Luồng dữ liệu qua dịch vụ: REST và RPC”)
 - Qua truyền thông điệp bất đồng bộ (xem “Luồng dữ liệu truyền thông điệp”)

Dataflow Through Databases — Luồng dữ liệu qua cơ sở dữ liệu

In a database, the process that writes to the database encodes the data, and the process that reads from the database decodes it. There may just be a single process accessing the database, in which case the reader is simply a later version of the same process—in that case you can think of storing something in the database as sending a message to your future self.

Trong một cơ sở dữ liệu, tiến trình ghi vào cơ sở dữ liệu mã hóa dữ liệu, và tiến trình đọc từ cơ sở dữ liệu giải mã nó. Có thể chỉ có một tiến trình duy nhất truy cập cơ sở dữ liệu, trong trường hợp đó bên đọc đơn giản là một phiên bản về sau của chính tiến trình đó — khi đó bạn có thể coi việc lưu một thứ gì đó vào cơ sở dữ liệu như việc gửi một thông điệp cho chính bạn trong tương lai.

Backward compatibility is clearly necessary here; otherwise your future self won't be able to decode what you previously wrote.

Tính tương thích ngược rõ ràng là cần thiết ở đây; nếu không, chính bạn trong tương lai sẽ không thể giải mã thứ bạn đã ghi trước đó.

In general, it's common for several different processes to be accessing a database at the same time. Those processes might be several different applications or services, or they may simply be several instances of the same service (running in parallel for **scalability** or **fault** tolerance). Either way, in an environment where the application is changing, it is likely that some processes accessing the database will be running newer code and some will be running older code—for example because a new version is currently being deployed in a rolling upgrade, so some instances have been updated while others haven't yet.

Nói chung, việc nhiều tiến trình khác nhau cùng truy cập một cơ sở dữ liệu tại cùng một thời điểm là chuyện phổ biến. Những tiến trình đó có thể là nhiều ứng dụng hoặc dịch vụ khác nhau, hoặc chúng đơn giản có thể là nhiều phiên bản (instance) của cùng một dịch vụ (chạy song song để mở rộng hoặc chịu lỗi). Dù theo cách nào, trong một môi trường mà ứng dụng đang thay đổi, nhiều khả năng là một số tiến trình truy cập cơ sở dữ liệu sẽ đang chạy mã mới hơn và một số sẽ đang chạy mã cũ hơn — chẳng hạn vì một phiên bản mới hiện đang được triển khai trong một đợt nâng cấp cuốn chiếu, nên một số instance đã được cập nhật trong khi số khác thì chưa.

This means that a value in the database may be written by a newer version of the code, and subsequently read by an older version of the code that is still running. Thus, forward compatibility is also often required for databases.

Điều này nghĩa là một giá trị trong cơ sở dữ liệu có thể được ghi bởi một phiên bản mã mới hơn, và sau đó được đọc bởi một phiên bản mã cũ hơn vẫn đang chạy. Do đó, tính tương thích xuôi cũng thường được đòi hỏi với các cơ sở dữ liệu.

However, there is an additional snag. Say you add a field to a record schema, and the newer code writes a value for that new field to the database. Subsequently, an older version of the code (which doesn't yet know about the new field) reads the record, updates it, and writes it back. In this situation, the desirable behavior is usually for the old code to keep the new field intact, even though it couldn't be interpreted.

Tuy nhiên, có thêm một trở ngại. Giả sử bạn thêm một trường vào lược đồ bản ghi, và mã mới hơn ghi một giá trị cho trường mới đó vào cơ sở dữ liệu. Sau đó, một phiên bản mã cũ hơn (vốn chưa biết về trường mới) đọc bản ghi, cập nhật nó, và ghi nó trở lại. Trong tình huống này, hành vi mong muốn thường là mã cũ phải giữ nguyên vẹn trường mới, dù nó không thể diễn giải được trường đó.

The encoding formats discussed previously support such preservation of unknown fields, but sometimes you need to take care at an application level, as illustrated in Figure 4-7. For example, if you decode a database value into model objects in the application, and later reencode those model objects, the unknown field might be lost in that translation process. Solving this is not a hard problem; you just need to be aware of it.

Các định dạng mã hóa đã bàn ở trên hỗ trợ việc bảo toàn các trường chưa biết như vậy, nhưng đôi khi bạn cần cẩn thận ở mức ứng dụng, như được minh họa ở Hình 4-7. Chẳng hạn, nếu bạn giải mã một giá trị cơ sở dữ liệu thành các đối tượng mô hình trong ứng dụng, rồi sau đó mã hóa lại các đối tượng mô hình đó, trường chưa biết có thể bị mất trong quá trình chuyển đổi đó. Giải quyết điều này không phải là một bài toán khó; bạn chỉ cần ý thức được về nó.

Figure 4-7. When an older version of the application updates data previously written by a newer version of the application, data may be lost if you're not careful. — Hình 4-7. Khi một phiên bản cũ hơn của ứng dụng cập nhật dữ liệu đã được ghi trước đó bởi một phiên bản mới hơn của ứng dụng, dữ liệu có thể bị mất nếu bạn không cẩn thận.

Different values written at different times — Các giá trị khác nhau được ghi ở những thời điểm khác nhau

A database generally allows any value to be updated at any time. This means that within a single database you may have some values that were written five milliseconds ago, and some values that were written five years ago.

Một cơ sở dữ liệu nhìn chung cho phép bất kỳ giá trị nào được cập nhật vào bất kỳ lúc nào. Điều này nghĩa là trong một cơ sở dữ liệu duy nhất, bạn có thể có một số giá trị được ghi cách đây năm mili giây, và một số giá trị được ghi cách đây năm năm.

When you deploy a new version of your application (of a server-side application, at least), you may entirely replace the old version with the new version within a few minutes. The same is not true of database contents: the five-year-old data will still be there, in the original encoding, unless you have explicitly rewritten it since then. This observation is sometimes summed up as data outlives code.

Khi bạn triển khai một phiên bản mới của ứng dụng (ít nhất là của một ứng dụng phía máy chủ), bạn có thể thay thế hoàn toàn phiên bản cũ bằng phiên bản mới trong vòng vài phút. Điều tương tự không đúng với nội dung cơ sở dữ liệu: dữ liệu năm năm tuổi sẽ vẫn còn ở đó, ở cách mã hóa ban đầu, trừ khi bạn đã ghi đè nó một cách tường minh kể từ đó. Quan sát này đôi khi được tóm gọn bằng câu dữ liệu sống lâu hơn mã.

Rewriting (migrating) data into a new schema is certainly possible, but it's an expensive thing to do on a large dataset, so most databases avoid it if possible. Most relational databases allow simple schema changes, such as adding a new column with a null default value, without rewriting existing data. When an old row is read, the database fills in nulls for any columns that are missing from the encoded data on disk. LinkedIn's document database Espresso uses Avro for storage, allowing it to use Avro's schema evolution rules [23].

Việc viết lại (di trú) dữ liệu sang một lược đồ mới chắc chắn là khả thi, nhưng đó là một việc tốn kém khi làm trên một tập dữ liệu lớn, nên hầu hết các cơ sở dữ liệu đều tránh nó nếu có thể. Hầu hết các cơ sở dữ liệu quan hệ cho phép các thay đổi lược đồ đơn giản, chẳng hạn thêm một cột mới với giá trị mặc định là null, mà không phải viết lại dữ liệu hiện có. Khi một hàng cũ được đọc, cơ sở dữ liệu điền null cho bất kỳ cột nào còn thiếu

trong dữ liệu đã mã hóa trên đĩa. Cơ sở dữ liệu tài liệu Espresso của LinkedIn dùng Avro để lưu trữ, cho phép nó dùng các quy tắc tiến hóa lược đồ của Avro [23].

Schema evolution thus allows the entire database to appear as if it was encoded with a single schema, even though the underlying storage may contain records encoded with various historical versions of the schema.

Tiến hóa lược đồ do đó cho phép toàn bộ cơ sở dữ liệu trông như thể nó được mã hóa bằng một lược đồ duy nhất, dù phần lưu trữ bên dưới có thể chứa các bản ghi được mã hóa bằng nhiều phiên bản lịch sử khác nhau của lược đồ.

Archival storage — Lưu trữ dài hạn

Perhaps you take a snapshot of your database from time to time, say for **backup** purposes or for loading into a data warehouse (see “Data Warehousing”). In this case, the data dump will typically be encoded using the latest schema, even if the original encoding in the source database contained a mixture of schema versions from different eras. Since you’re copying the data anyway, you might as well encode the copy of the data consistently.

Có lẽ thỉnh thoảng bạn chụp một ảnh chụp nhanh (snapshot) của cơ sở dữ liệu, chẳng hạn cho mục đích sao lưu hoặc để nạp vào một kho dữ liệu (xem “Kho dữ liệu”). Trong trường hợp này, bản kết xuất dữ liệu thường sẽ được mã hóa bằng lược đồ mới nhất, dù cách mã hóa ban đầu trong cơ sở dữ liệu nguồn có chứa một hỗn hợp các phiên bản lược đồ từ những thời kỳ khác nhau. Vì dù sao bạn cũng đang sao chép dữ liệu, bạn cũng nên mã hóa bản sao của dữ liệu một cách nhất quán.

As the data dump is written in one go and is thereafter immutable, formats like Avro object container files are a good fit. This is also a good opportunity to encode the data in an analytics-friendly column-oriented format such as Parquet (see “Column Compression”).

Vì bản kết xuất dữ liệu được ghi một lần và sau đó là bất biến, các định dạng như tập tin container đối tượng Avro rất phù hợp. Đây cũng là một cơ hội tốt để mã hóa dữ liệu ở một định dạng hướng cột thân thiện với phân tích như Parquet (xem “Nén cột”).

In Chapter 10 we will talk more about using data in archival storage.

Ở Chương 10, ta sẽ nói thêm về việc sử dụng dữ liệu trong lưu trữ dài hạn.

Dataflow Through Services: REST and RPC — Luồng dữ liệu qua dịch vụ: REST và RPC

When you have processes that need to communicate over a network, there are a few different ways of arranging that communication. The most common arrangement is to have two roles: clients and servers. The servers expose an API over the network, and the clients can connect to the servers to make requests to that API. The API exposed by the server is known as a service.

Khi bạn có các tiến trình cần giao tiếp qua mạng, có vài cách khác nhau để sắp đặt việc giao tiếp đó. Cách sắp đặt phổ biến nhất là có hai vai trò: client và máy chủ. Các máy chủ phơi bày một API qua mạng, và các client có thể kết nối tới các máy chủ để gửi yêu cầu tới API đó. API được máy chủ phơi bày được gọi là một dịch vụ.

The web works this way: clients (web browsers) make requests to web servers, making GET requests to download HTML, CSS, JavaScript, images, etc., and making POST requests to submit data to the

server. The API consists of a standardized set of protocols and data formats (HTTP, URLs, SSL/TLS, HTML, etc.). Because web browsers, web servers, and website authors mostly agree on these standards, you can use any web browser to access any website (at least in theory!).

Web hoạt động theo cách này: các client (trình duyệt web) gửi yêu cầu tới các máy chủ web, gửi các yêu cầu GET để tải về HTML, CSS, JavaScript, hình ảnh, v.v., và gửi các yêu cầu POST để gửi dữ liệu lên máy chủ. API gồm một tập hợp các giao thức và định dạng dữ liệu được chuẩn hóa (HTTP, URL, SSL/TLS, HTML, v.v.). Vì các trình duyệt web, máy chủ web và tác giả website hầu như đều thống nhất về các chuẩn này, bạn có thể dùng bất kỳ trình duyệt web nào để truy cập bất kỳ website nào (ít nhất là trên lý thuyết!).

Web browsers are not the only type of client. For example, a native app running on a mobile device or a desktop computer can also make network requests to a server, and a client-side JavaScript application running inside a web browser can use XMLHttpRequest to become an HTTP client (this technique is known as Ajax [30]). In this case, the server's response is typically not HTML for displaying to a human, but rather data in an encoding that is convenient for further processing by the client-side application code (such as JSON). Although HTTP may be used as the transport protocol, the API implemented on top is application-specific, and the client and server need to agree on the details of that API.

Trình duyệt web không phải là loại client duy nhất. Chẳng hạn, một ứng dụng gốc (native app) chạy trên một thiết bị di động hoặc một máy tính để bàn cũng có thể gửi các yêu cầu mạng tới một máy chủ, và một ứng dụng JavaScript phía client chạy bên trong một trình duyệt web có thể dùng XMLHttpRequest để trở thành một client HTTP (kỹ thuật này được gọi là Ajax [30]). Trong trường hợp này, phản hồi của máy chủ thường không phải là HTML để hiển thị cho con người, mà là dữ liệu ở một cách mã hóa tiện cho việc mã ứng dụng phía client xử lý tiếp (chẳng hạn JSON). Mặc dù HTTP có thể được dùng làm giao thức truyền tải, API được hiện thực bên trên nó là đặc thù cho ứng dụng, và client cùng máy chủ cần thống nhất các chi tiết của API đó.

Moreover, a server can itself be a client to another service (for example, a typical web app server acts as client to a database). This approach is often used to decompose a large application into smaller services by area of functionality, such that one service makes a request to another when it requires some functionality or data from that other service. This way of building applications has traditionally been called a service-oriented architecture (SOA), more recently refined and rebranded as microservices architecture [31, 32].

Hơn nữa, bản thân một máy chủ có thể là client của một dịch vụ khác (chẳng hạn, một máy chủ ứng dụng web điển hình đóng vai trò là client của một cơ sở dữ liệu). Cách tiếp cận này thường được dùng để phân rã một ứng dụng lớn thành các dịch vụ nhỏ hơn theo lĩnh vực chức năng, sao cho một dịch vụ gửi một yêu cầu tới một dịch vụ khác khi nó cần một chức năng hoặc dữ liệu nào đó từ dịch vụ kia. Cách xây dựng ứng dụng này theo truyền thống được gọi là kiến trúc hướng dịch vụ (service-oriented architecture - SOA), gần đây hơn được tinh chỉnh và đặt lại tên thành kiến trúc microservices [31, 32].

In some ways, services are similar to databases: they typically allow clients to submit and query data. However, while databases allow arbitrary queries using the query languages we discussed in Chapter 2, services expose an application-specific API that only allows inputs and outputs that are predetermined by the business logic (application code) of the service [33]. This restriction provides a degree of encapsulation: services can impose fine-grained restrictions on what clients can and cannot do.

Theo một số khía cạnh, các dịch vụ tương tự như các cơ sở dữ liệu: chúng thường cho phép các client gửi và truy vấn dữ liệu. Tuy nhiên, trong khi các cơ sở dữ liệu cho phép truy vấn

tùy ý bằng các ngôn ngữ truy vấn mà ta đã bàn ở Chương 2, các dịch vụ phơi bày một API đặc thù cho ứng dụng vốn chỉ cho phép các đầu vào và đầu ra được định trước bởi logic nghiệp vụ (mã ứng dụng) của dịch vụ [33]. Hạn chế này mang lại một mức độ đóng gói: các dịch vụ có thể áp đặt các giới hạn chi tiết về những gì client có thể và không thể làm.

A key design goal of a service-oriented/microservices architecture is to make the application easier to change and maintain by making services independently deployable and evolvable. For example, each service should be owned by one team, and that team should be able to release new versions of the service frequently, without having to coordinate with other teams. In other words, we should expect old and new versions of servers and clients to be running at the same time, and so the data encoding used by servers and clients must be compatible across versions of the service API—precisely what we’ve been talking about in this chapter.

Một mục tiêu thiết kế then chốt của kiến trúc hướng dịch vụ/microservices là làm cho ứng dụng dễ thay đổi và bảo trì hơn bằng cách làm cho các dịch vụ có thể được triển khai và tiến hóa một cách độc lập. Chẳng hạn, mỗi dịch vụ nên được sở hữu bởi một nhóm, và nhóm đó nên có khả năng phát hành các phiên bản mới của dịch vụ thường xuyên, mà không phải phối hợp với các nhóm khác. Nói cách khác, ta nên kỳ vọng các phiên bản máy chủ và client cũ và mới sẽ cùng chạy một lúc, và do đó cách mã hóa dữ liệu mà các máy chủ và client dùng phải tương thích qua các phiên bản của API dịch vụ — đúng chính xác điều mà ta đã bàn trong chương này.

Web services — Dịch vụ web

When HTTP is used as the underlying protocol for talking to the service, it is called a web service. This is perhaps a slight misnomer, because web services are not only used on the web, but in several different contexts. For example:

Khi HTTP được dùng làm giao thức nền tảng để nói chuyện với dịch vụ, nó được gọi là một dịch vụ web (web service). Đây có lẽ là một cách gọi hơi sai, vì các dịch vụ web không chỉ được dùng trên web, mà còn trong nhiều ngữ cảnh khác nhau. Chẳng hạn:

- A client application running on a user’s device (e.g., a native app on a mobile device, or JavaScript web app using Ajax) making requests to a service over HTTP. These requests typically go over the public internet.
- One service making requests to another service owned by the same organization, often located within the same **datacenter**, as part of a service-oriented/microservices architecture. (Software that supports this kind of use case is sometimes called middleware.)
- One service making requests to a service owned by a different organization, usually via the internet. This is used for data exchange between different organizations’ backend systems. This category includes public APIs provided by online services, such as credit card processing systems, or OAuth for shared access to user data.
 - Một ứng dụng client chạy trên thiết bị của người dùng (ví dụ một ứng dụng gốc trên thiết bị di động, hoặc một ứng dụng web JavaScript dùng Ajax) gửi yêu cầu tới một dịch vụ qua HTTP. Những yêu cầu này thường đi qua internet công cộng.
 - Một dịch vụ gửi yêu cầu tới một dịch vụ khác do cùng tổ chức sở hữu, thường nằm trong cùng một trung tâm dữ liệu, như một phần của kiến trúc hướng dịch vụ/microservices. (Phần mềm hỗ trợ kiểu tình huống sử dụng này đôi khi được gọi là middleware.)
 - Một dịch vụ gửi yêu cầu tới một dịch vụ do một tổ chức khác sở hữu, thường thông qua internet. Cách này được dùng để trao đổi dữ liệu giữa các hệ thống backend của các tổ chức khác nhau. Hạng mục này bao gồm các API công khai do các dịch vụ trực

tuyến cung cấp, chẳng hạn các hệ thống xử lý thẻ tín dụng, hoặc OAuth để chia sẻ quyền truy cập dữ liệu người dùng.

There are two popular approaches to web services: REST and SOAP. They are almost diametrically opposed in terms of philosophy, and often the **subject** of heated debate among their respective proponents.

Có hai cách tiếp cận phổ biến với các dịch vụ web: REST và SOAP. Chúng gần như đối lập hoàn toàn về mặt triết lý, và thường là chủ đề tranh luận nảy lửa giữa những người ủng hộ mỗi bên.

REST is not a protocol, but rather a design philosophy that builds upon the principles of HTTP [34, 35]. It emphasizes simple data formats, using URLs for identifying resources and using HTTP features for **cache** control, authentication, and content type negotiation. REST has been gaining popularity compared to SOAP, at least in the context of cross-organizational service integration [36], and is often associated with microservices [31]. An API designed according to the principles of REST is called RESTful.

REST không phải là một giao thức, mà là một triết lý thiết kế xây dựng trên các nguyên lý của HTTP [34, 35]. Nó nhấn mạnh các định dạng dữ liệu đơn giản, dùng URL để định danh tài nguyên và dùng các tính năng của HTTP cho việc kiểm soát bộ nhớ đệm, xác thực và thương lượng kiểu nội dung. REST đã ngày càng phổ biến so với SOAP, ít nhất là trong ngữ cảnh tích hợp dịch vụ liên tổ chức [36], và thường được gắn với microservices [31]. Một API được thiết kế theo các nguyên lý của REST được gọi là RESTful.

By contrast, SOAP is an XML-based protocol for making network API requests. Although it is most commonly used over HTTP, it aims to be independent from HTTP and avoids using most HTTP features. Instead, it comes with a sprawling and complex multitude of related standards (the web service framework, known as WS-*) that add various features [37].

Ngược lại, SOAP là một giao thức dựa trên XML để gửi các yêu cầu API qua mạng. Mặc dù nó thường được dùng nhất qua HTTP, nó hướng tới việc độc lập với HTTP và tránh dùng hầu hết các tính năng của HTTP. Thay vào đó, nó đi kèm một mớ hỗn độn rườm rà và phức tạp các chuẩn liên quan (khung dịch vụ web, được gọi là WS-*) bổ sung nhiều tính năng khác nhau [37].

The API of a SOAP web service is described using an XML-based language called the Web Services Description Language, or WSDL. WSDL enables code generation so that a client can access a remote service using local classes and method calls (which are encoded to XML messages and decoded again by the framework). This is useful in statically typed programming languages, but less so in dynamically typed ones (see “Code generation and dynamically typed languages”).

API của một dịch vụ web SOAP được mô tả bằng một ngôn ngữ dựa trên XML gọi là Ngôn ngữ mô tả dịch vụ web (Web Services Description Language), hay WSDL. WSDL cho phép sinh mã để một client có thể truy cập một dịch vụ từ xa bằng các lớp và lời gọi phương thức cục bộ (vốn được khung làm việc mã hóa thành các thông điệp XML và giải mã trở lại). Điều này hữu ích trong các ngôn ngữ lập trình định kiểu tĩnh, nhưng kém hữu ích hơn trong các ngôn ngữ định kiểu động (xem “Sinh mã và các ngôn ngữ định kiểu động”).

As WSDL is not designed to be human-readable, and as SOAP messages are often too complex to construct manually, users of SOAP rely heavily on tool support, code generation, and IDEs [38]. For users of programming languages that are not supported by SOAP vendors, integration with SOAP services is difficult.

Vì WSDL không được thiết kế để con người đọc được, và vì các thông điệp SOAP thường

quá phức tạp để dựng thủ công, người dùng SOAP phụ thuộc rất nhiều vào sự hỗ trợ của công cụ, việc sinh mã và các IDE [38]. Với người dùng các ngôn ngữ lập trình không được các nhà cung cấp SOAP hỗ trợ, việc tích hợp với các dịch vụ SOAP là khó khăn.

Even though SOAP and its various extensions are ostensibly standardized, interoperability between different vendors' implementations often causes problems [39]. For all of these reasons, although SOAP is still used in many large enterprises, it has fallen out of favor in most smaller companies.

Mặc dù SOAP và các phần mở rộng khác nhau của nó bề ngoài được chuẩn hóa, khả năng tương tác giữa các bản hiện thực của các nhà cung cấp khác nhau thường gây ra vấn đề [39]. Vì tất cả những lý do này, dù SOAP vẫn được dùng trong nhiều doanh nghiệp lớn, nó đã mất đi sự ưa chuộng ở hầu hết các công ty nhỏ hơn.

RESTful APIs tend to favor simpler approaches, typically involving less code generation and automated tooling. A definition format such as OpenAPI, also known as Swagger [40], can be used to describe RESTful APIs and produce documentation.

Các API RESTful có xu hướng ưa chuộng những cách tiếp cận đơn giản hơn, thường liên quan tới ít việc sinh mã và công cụ tự động hơn. Một định dạng định nghĩa như OpenAPI, còn gọi là Swagger [40], có thể được dùng để mô tả các API RESTful và tạo ra tài liệu.

The problems with remote procedure calls (RPCs) — Các vấn đề với lời gọi thủ tục từ xa (RPC)

Web services are merely the latest incarnation of a long line of technologies for making API requests over a network, many of which received a lot of hype but have serious problems. Enterprise JavaBeans (EJB) and Java's Remote Method Invocation (RMI) are limited to Java. The Distributed Component Object Model (DCOM) is limited to Microsoft platforms. The Common Object Request Broker Architecture (CORBA) is excessively complex, and does not provide backward or forward compatibility [41].

Các dịch vụ web chỉ là hiện thân mới nhất trong một chuỗi dài các công nghệ để gửi yêu cầu API qua mạng, nhiều công nghệ trong đó từng được tung hô rất nhiều nhưng có những vấn đề nghiêm trọng. Enterprise JavaBeans (EJB) và Remote Method Invocation (RMI) của Java bị giới hạn ở Java. Distributed Component Object Model (DCOM) bị giới hạn ở các nền tảng của Microsoft. Common Object Request Broker Architecture (CORBA) thì phức tạp quá mức, và không cung cấp tính tương thích ngược hay xuôi [41].

All of these are based on the idea of a remote procedure call (RPC), which has been around since the 1970s [42]. The RPC model tries to make a request to a remote network service look the same as calling a function or method in your programming language, within the same process (this **abstraction** is called location transparency). Although RPC seems convenient at first, the approach is fundamentally flawed [43, 44]. A network request is very different from a local function call:

Tất cả những thứ này đều dựa trên ý tưởng về một lời gọi thủ tục từ xa (remote procedure call - RPC), vốn đã tồn tại từ thập niên 1970 [42]. Mô hình RPC cố làm cho một yêu cầu tới một dịch vụ mạng từ xa trông giống như việc gọi một hàm hoặc phương thức trong ngôn ngữ lập trình của bạn, trong cùng một tiến trình (trừu tượng này được gọi là tính trong suốt về vị trí - location transparency). Mặc dù RPC thoạt nhìn có vẻ tiện lợi, cách tiếp cận này về cơ bản là sai lầm [43, 44]. Một yêu cầu mạng rất khác với một lời gọi hàm cục bộ:

- A local function call is predictable and either succeeds or fails, depending only on parameters that are under your control. A network request is unpredictable: the request or response may be lost due to a network problem, or the remote machine may be slow or unavailable, and such

problems are entirely outside of your control. Network problems are common, so you have to anticipate them, for example by retrying a failed request.

- A local function call either returns a result, or throws an exception, or never returns (because it goes into an infinite loop or the process crashes). A network request has another possible outcome: it may return without a result, due to a timeout. In that case, you simply don't know what happened: if you don't get a response from the remote service, you have no way of knowing whether the request got through or not. (We discuss this issue in more detail in Chapter 8.)
- If you retry a failed network request, it could happen that the requests are actually getting through, and only the responses are getting lost. In that case, retrying will cause the action to be performed multiple times, unless you build a mechanism for deduplication (idempotence) into the protocol. Local function calls don't have this problem. (We discuss idempotence in more detail in Chapter 11.)
- Every time you call a local function, it normally takes about the same time to execute. A network request is much slower than a function call, and its **latency** is also wildly variable: at good times it may complete in less than a millisecond, but when the network is congested or the remote service is overloaded it may take many seconds to do exactly the same thing.
- When you call a local function, you can efficiently pass it references (pointers) to objects in local memory. When you make a network request, all those parameters need to be encoded into a sequence of bytes that can be sent over the network. That's okay if the parameters are primitives like numbers or strings, but quickly becomes problematic with larger objects.
- The client and the service may be implemented in different programming languages, so the RPC framework must translate datatypes from one language into another. This can end up ugly, since not all languages have the same types—recall JavaScript's problems with numbers greater than 2, for example (see "JSON, XML, and Binary Variants"). This problem doesn't exist in a single process written in a single language.
 - Một lời gọi hàm cục bộ thì có thể dự đoán được và hoặc thành công hoặc thất bại, chỉ tùy thuộc vào các tham số nằm trong tầm kiểm soát của bạn. Một yêu cầu mạng thì không thể dự đoán được: yêu cầu hoặc phản hồi có thể bị mất do một sự cố mạng, hoặc máy ở xa có thể chậm hoặc không khả dụng, và những vấn đề như vậy hoàn toàn nằm ngoài tầm kiểm soát của bạn. Các sự cố mạng là chuyện thường gặp, nên bạn phải lường trước chúng, chẳng hạn bằng cách thử lại một yêu cầu thất bại.
 - Một lời gọi hàm cục bộ hoặc trả về một kết quả, hoặc ném ra một ngoại lệ, hoặc không bao giờ trả về (vì nó rơi vào vòng lặp vô hạn hoặc tiến trình bị sập). Một yêu cầu mạng có thêm một kết cục khả dĩ khác: nó có thể trả về mà không có kết quả, do hết thời gian chờ (timeout). Trong trường hợp đó, bạn đơn giản không biết điều gì đã xảy ra: nếu bạn không nhận được phản hồi từ dịch vụ ở xa, bạn không có cách nào biết được yêu cầu có lọt qua hay không. (Ta bàn vấn đề này chi tiết hơn ở Chương 8.)
 - Nếu bạn thử lại một yêu cầu mạng thất bại, có thể xảy ra chuyện là các yêu cầu thực ra đang lọt qua, và chỉ có các phản hồi là bị mất. Trong trường hợp đó, việc thử lại sẽ khiến hành động được thực hiện nhiều lần, trừ khi bạn xây dựng một cơ chế khử trùng lặp (tính lũy đẳng - idempotence) vào trong giao thức. Các lời gọi hàm cục bộ không có vấn đề này. (Ta bàn về tính idempotence chi tiết hơn ở Chương 11.)
 - Mỗi lần bạn gọi một hàm cục bộ, nó thường mất khoảng cùng một khoảng thời gian để thực thi. Một yêu cầu mạng thì chậm hơn nhiều so với một lời gọi hàm, và độ trễ của nó cũng biến thiên dữ dội: vào lúc thuận lợi nó có thể hoàn thành trong chưa đầy một mili giây, nhưng khi mạng tắc nghẽn hoặc dịch vụ ở xa quá tải, nó có thể mất nhiều giây để làm đúng chính xác cùng một việc.
 - Khi bạn gọi một hàm cục bộ, bạn có thể truyền cho nó các tham chiếu (con trỏ) tới các đối tượng trong bộ nhớ cục bộ một cách hiệu quả. Khi bạn gửi một yêu cầu mạng,

tất cả các tham số đó cần được mã hóa thành một chuỗi byte có thể gửi qua mạng. Điều đó ổn nếu các tham số là các kiểu nguyên thủy như số hoặc chuỗi, nhưng nhanh chóng trở nên rắc rối với các đối tượng lớn hơn.

- Client và dịch vụ có thể được hiện thực bằng các ngôn ngữ lập trình khác nhau, nên khung RPC phải dịch các kiểu dữ liệu từ ngôn ngữ này sang ngôn ngữ khác. Điều này có thể trở nên xấu xí, vì không phải mọi ngôn ngữ đều có cùng các kiểu — hãy nhớ lại các vấn đề của JavaScript với những con số lớn hơn 2, chẳng hạn vậy (xem “JSON, XML và các biến thể nhị phân”). Vấn đề này không tồn tại trong một tiến trình duy nhất viết bằng một ngôn ngữ duy nhất.

All of these factors mean that there’s no point trying to make a remote service look too much like a local object in your programming language, because it’s a fundamentally different thing. Part of the appeal of REST is that it doesn’t try to hide the fact that it’s a network protocol (although this doesn’t seem to stop people from building RPC libraries on top of REST).

Tất cả những yếu tố này nghĩa là chẳng có ích gì khi cố làm cho một dịch vụ từ xa trông quá giống một đối tượng cục bộ trong ngôn ngữ lập trình của bạn, vì về cơ bản nó là một thứ khác. Một phần sức hút của REST là nó không cố che giấu thực tế rằng nó là một giao thức mạng (mặc dù điều này dường như không ngăn được người ta xây dựng các thư viện RPC bên trên REST).

Current directions for RPC — Các hướng đi hiện tại cho RPC

Despite all these problems, RPC isn’t going away. Various RPC frameworks have been built on top of all the encodings mentioned in this chapter: for example, Thrift and Avro come with RPC support included, gRPC is an RPC implementation using Protocol Buffers, Finagle also uses Thrift, and Rest.li uses JSON over HTTP.

Bất chấp tất cả những vấn đề này, RPC sẽ không biến mất. Nhiều khung RPC đã được xây dựng bên trên tất cả các cách mã hóa được nhắc tới trong chương này: chẳng hạn, Thrift và Avro đi kèm hỗ trợ RPC sẵn có, gRPC là một bản hiện thực RPC dùng Protocol Buffers, Finagle cũng dùng Thrift, và Rest.li dùng JSON qua HTTP.

This new generation of RPC frameworks is more explicit about the fact that a remote request is different from a local function call. For example, Finagle and Rest.li use futures (promises) to encapsulate asynchronous actions that may fail. Futures also simplify situations where you need to make requests to multiple services in parallel, and combine their results [45]. gRPC supports streams, where a call consists of not just one request and one response, but a series of requests and responses over time [46].

Thế hệ khung RPC mới này tường minh hơn về thực tế rằng một yêu cầu từ xa khác với một lời gọi hàm cục bộ. Chẳng hạn, Finagle và Rest.li dùng các future (promise) để đóng gói các hành động bất đồng bộ có thể thất bại. Các future cũng đơn giản hóa những tình huống mà bạn cần gửi yêu cầu tới nhiều dịch vụ song song, và kết hợp các kết quả của chúng [45]. gRPC hỗ trợ các luồng (stream), nơi một lời gọi gồm không chỉ một yêu cầu và một phản hồi, mà một chuỗi các yêu cầu và phản hồi theo thời gian [46].

Some of these frameworks also provide service discovery—that is, allowing a client to find out at which IP address and port number it can find a particular service. We will return to this topic in “Request Routing”.

Một số khung này cũng cung cấp khám phá dịch vụ (service discovery) — tức là, cho phép một client tìm ra địa chỉ IP và số cổng nào mà nó có thể tìm thấy một dịch vụ cụ thể. Ta sẽ trở lại chủ đề này ở “Định tuyến yêu cầu”.

Custom RPC protocols with a binary encoding format can achieve better performance than something generic like JSON over REST. However, a RESTful API has other significant advantages: it is good for experimentation and debugging (you can simply make requests to it using a web browser or the command-line tool curl, without any code generation or software installation), it is supported by all mainstream programming languages and platforms, and there is a vast ecosystem of tools available (servers, caches, **load** balancers, proxies, firewalls, monitoring, debugging tools, testing tools, etc.).

Các giao thức RPC tùy biến với một định dạng mã hóa nhị phân có thể đạt hiệu năng tốt hơn so với một thứ chung chung như JSON qua REST. Tuy nhiên, một API RESTful có những ưu điểm đáng kể khác: nó tốt cho việc thử nghiệm và gỡ lỗi (bạn chỉ cần gửi yêu cầu tới nó dùng một trình duyệt web hoặc công cụ dòng lệnh curl, mà không cần sinh mã hay cài đặt phần mềm gì), nó được tất cả các ngôn ngữ lập trình và nền tảng chính dòng hỗ trợ, và có một hệ sinh thái công cụ khổng lồ sẵn có (máy chủ, bộ nhớ đệm, bộ cân bằng tải, proxy, tường lửa, công cụ giám sát, công cụ gỡ lỗi, công cụ kiểm thử, v.v.).

For these reasons, REST seems to be the predominant style for public APIs. The main focus of RPC frameworks is on requests between services owned by the same organization, typically within the same datacenter.

Vì những lý do này, REST có vẻ là phong cách chủ đạo cho các API công khai. Trọng tâm chính của các khung RPC là các yêu cầu giữa các dịch vụ do cùng một tổ chức sở hữu, thường nằm trong cùng một trung tâm dữ liệu.

Data encoding and evolution for RPC — Mã hóa dữ liệu và tiến hóa cho RPC

For evolvability, it is important that RPC clients and servers can be changed and deployed independently. Compared to data flowing through databases (as described in the last section), we can make a simplifying assumption in the case of dataflow through services: it is reasonable to assume that all the servers will be updated first, and all the clients second. Thus, you only need backward compatibility on requests, and forward compatibility on responses.

Để có khả năng tiến hóa, điều quan trọng là các client và máy chủ RPC có thể được thay đổi và triển khai một cách độc lập. So với dữ liệu chảy qua các cơ sở dữ liệu (như đã mô tả ở phần trước), ta có thể đưa ra một giả định đơn giản hóa trong trường hợp luồng dữ liệu qua dịch vụ: hợp lý khi giả định rằng tất cả các máy chủ sẽ được cập nhật trước, và tất cả các client sau. Do đó, bạn chỉ cần tính tương thích ngược trên các yêu cầu, và tính tương thích xuôi trên các phản hồi.

The backward and forward compatibility properties of an RPC scheme are inherited from whatever encoding it uses:

Các tính chất tương thích ngược và xuôi của một sơ đồ RPC được thừa hưởng từ bất kỳ cách mã hóa nào mà nó dùng:

- Thrift, gRPC (Protocol Buffers), and Avro RPC can be evolved according to the compatibility rules of the respective encoding format.
- In SOAP, requests and responses are specified with XML schemas. These can be evolved, but there are some subtle pitfalls [47].
- RESTful APIs most commonly use JSON (without a formally specified schema) for responses, and JSON or **URI**-encoded/form-encoded request parameters for requests. Adding optional request parameters and adding new fields to response objects are usually considered changes that maintain compatibility.

- Thrift, gRPC (Protocol Buffers) và Avro RPC có thể được tiến hóa theo các quy tắc tương thích của định dạng mã hóa tương ứng.
- Trong SOAP, các yêu cầu và phản hồi được đặc tả bằng các lược đồ XML. Chúng có thể được tiến hóa, nhưng có một số cạm bẫy tinh tế [47].
- Các API RESTful thường dùng nhất JSON (không có một lược đồ được đặc tả hình thức) cho các phản hồi, và JSON hoặc các tham số yêu cầu được mã hóa kiểu URI/kiểu form cho các yêu cầu. Việc thêm các tham số yêu cầu tùy chọn và thêm các trường mới vào các đối tượng phản hồi thường được xem là những thay đổi duy trì tính tương thích.

Service compatibility is made harder by the fact that RPC is often used for communication across organizational boundaries, so the provider of a service often has no control over its clients and cannot force them to upgrade. Thus, compatibility needs to be maintained for a long time, perhaps indefinitely. If a compatibility-breaking change is required, the service provider often ends up maintaining multiple versions of the service API side by side.

Tính tương thích của dịch vụ trở nên khó hơn bởi thực tế rằng RPC thường được dùng để giao tiếp xuyên biên giới tổ chức, nên bên cung cấp một dịch vụ thường không có quyền kiểm soát các client của nó và không thể buộc chúng nâng cấp. Do đó, tính tương thích cần được duy trì trong một thời gian dài, có lẽ là vô thời hạn. Nếu cần một thay đổi phá vỡ tính tương thích, bên cung cấp dịch vụ thường rất cuộc phải duy trì nhiều phiên bản của API dịch vụ song song với nhau.

There is no agreement on how API versioning should work (i.e., how a client can indicate which version of the API it wants to use [48]). For RESTful APIs, common approaches are to use a version number in the URL or in the HTTP Accept header. For services that use API keys to identify a particular client, another option is to store a client's requested API version on the server and to allow this version selection to be updated through a separate administrative interface [49].

Không có sự đồng thuận về cách việc gắn phiên bản cho API nên hoạt động (tức là, làm sao một client có thể chỉ ra phiên bản nào của API mà nó muốn dùng [48]). Với các API RESTful, các cách tiếp cận phổ biến là dùng một số phiên bản trong URL hoặc trong header HTTP Accept. Với các dịch vụ dùng khóa API để định danh một client cụ thể, một lựa chọn khác là lưu phiên bản API mà một client yêu cầu trên máy chủ và cho phép việc chọn phiên bản này được cập nhật thông qua một giao diện quản trị riêng [49].

Message-Passing Dataflow — Luồng dữ liệu truyền thông điệp

We have been looking at the different ways encoded data flows from one process to another. So far, we've discussed REST and RPC (where one process sends a request over the network to another process and expects a response as quickly as possible), and databases (where one process writes encoded data, and another process reads it again sometime in the future).

Ta đã và đang xem xét những cách khác nhau mà dữ liệu đã mã hóa chảy từ tiến trình này sang tiến trình khác. Cho đến giờ, ta đã bàn về REST và RPC (nơi một tiến trình gửi một yêu cầu qua mạng tới một tiến trình khác và kỳ vọng một phản hồi càng nhanh càng tốt), và các cơ sở dữ liệu (nơi một tiến trình ghi dữ liệu đã mã hóa, và một tiến trình khác đọc lại nó vào một lúc nào đó trong tương lai).

In this final section, we will briefly look at asynchronous message-passing systems, which are somewhere between RPC and databases. They are similar to RPC in that a client's request (usually called a message) is delivered to another process with low latency. They are similar to databases in that the message is not sent via a direct network connection, but goes via an intermediary called a

message broker (also called a **message queue** or message-oriented middleware), which stores the message temporarily.

Trong phần cuối cùng này, ta sẽ xem xét sơ qua các hệ thống truyền thông điệp bất đồng bộ, vốn nằm đâu đó giữa RPC và các cơ sở dữ liệu. Chúng tương tự RPC ở chỗ yêu cầu của một client (thường gọi là một thông điệp) được chuyển tới một tiến trình khác với độ trễ thấp. Chúng tương tự các cơ sở dữ liệu ở chỗ thông điệp không được gửi qua một kết nối mạng trực tiếp, mà đi qua một bên trung gian gọi là môi giới thông điệp (message broker, còn gọi là hàng đợi thông điệp hoặc middleware hướng thông điệp), vốn lưu thông điệp tạm thời.

Using a message broker has several advantages compared to direct RPC:

Việc dùng một môi giới thông điệp có vài ưu điểm so với RPC trực tiếp:

- It can act as a buffer if the recipient is unavailable or overloaded, and thus improve system **reliability**.
- It can automatically redeliver messages to a process that has crashed, and thus prevent messages from being lost.
- It avoids the sender needing to know the IP address and port number of the recipient (which is particularly useful in a cloud deployment where virtual machines often come and go).
- It allows one message to be sent to several recipients.
- It logically decouples the sender from the recipient (the sender just publishes messages and doesn't care who consumes them).
 - Nó có thể đóng vai trò một bộ đệm nếu bên nhận không khả dụng hoặc quá tải, qua đó cải thiện độ tin cậy của hệ thống.
 - Nó có thể tự động chuyển lại các thông điệp tới một tiến trình đã bị sập, qua đó ngăn các thông điệp khỏi bị mất.
 - Nó tránh cho bên gửi việc phải biết địa chỉ IP và số cổng của bên nhận (điều này đặc biệt hữu ích trong một triển khai trên đám mây nơi các máy ảo thường xuyên đến và đi).
 - Nó cho phép một thông điệp được gửi tới nhiều bên nhận.
 - Nó tách rời về mặt logic bên gửi khỏi bên nhận (bên gửi chỉ việc công bố thông điệp và không quan tâm ai tiêu thụ chúng).

However, a difference compared to RPC is that message-passing communication is usually one-way: a sender normally doesn't expect to receive a reply to its messages. It is possible for a process to send a response, but this would usually be done on a separate channel. This communication pattern is asynchronous: the sender doesn't wait for the message to be delivered, but simply sends it and then forgets about it.

Tuy nhiên, một khác biệt so với RPC là việc giao tiếp truyền thông điệp thường là một chiều: một bên gửi thường không kỳ vọng nhận được một hồi đáp cho các thông điệp của nó. Một tiến trình có thể gửi một phản hồi, nhưng việc này thường được thực hiện trên một kênh riêng. Mẫu giao tiếp này là bất đồng bộ: bên gửi không chờ cho thông điệp được chuyển đi, mà chỉ đơn giản gửi nó rồi quên nó đi.

Message brokers — Môi giới thông điệp

In the past, the landscape of message brokers was dominated by commercial enterprise software from companies such as TIBCO, IBM WebSphere, and webMethods. More recently, open source implementations such as RabbitMQ, ActiveMQ, HornetQ, NATS, and Apache Kafka have become popular. We will compare them in more detail in Chapter 11.

Trong quá khứ, bức tranh các môi giới thông điệp bị thống trị bởi các phần mềm doanh nghiệp thương mại từ các công ty như TIBCO, IBM WebSphere và webMethods. Gần đây hơn, các bản hiện thực mã nguồn mở như RabbitMQ, ActiveMQ, HornetQ, NATS và Apache Kafka đã trở nên phổ biến. Ta sẽ so sánh chúng chi tiết hơn ở Chương 11.

The detailed delivery semantics vary by implementation and configuration, but in general, message brokers are used as follows: one process sends a message to a named **queue** or topic, and the broker ensures that the message is delivered to one or more consumers or subscribers to that queue or topic. There can be many producers and many consumers on the same topic.

Ngữ nghĩa chuyển giao chi tiết khác nhau tùy bản hiện thực và cấu hình, nhưng nhìn chung, các môi giới thông điệp được dùng như sau: một tiến trình gửi một thông điệp tới một hàng đợi hoặc chủ đề (topic) có tên, và môi giới đảm bảo rằng thông điệp được chuyển tới một hoặc nhiều bên tiêu thụ của hoặc bên đăng ký vào hàng đợi hoặc chủ đề đó. Có thể có nhiều bên sản xuất và nhiều bên tiêu thụ trên cùng một chủ đề.

A topic provides only one-way dataflow. However, a consumer may itself publish messages to another topic (so you can chain them together, as we shall see in Chapter 11), or to a reply queue that is consumed by the sender of the original message (allowing a request/response dataflow, similar to RPC).

Một chủ đề chỉ cung cấp luồng dữ liệu một chiều. Tuy nhiên, một bên tiêu thụ bản thân nó có thể công bố các thông điệp tới một chủ đề khác (nên bạn có thể nối chúng lại với nhau, như ta sẽ thấy ở Chương 11), hoặc tới một hàng đợi hồi đáp được tiêu thụ bởi bên gửi của thông điệp ban đầu (cho phép một luồng dữ liệu yêu cầu/phản hồi, tương tự RPC).

Message brokers typically don't enforce any particular data model—a message is just a sequence of bytes with some metadata, so you can use any encoding format. If the encoding is backward and forward compatible, you have the greatest flexibility to change publishers and consumers independently and deploy them in any order.

Các môi giới thông điệp thường không áp đặt bất kỳ mô hình dữ liệu cụ thể nào — một thông điệp chỉ là một chuỗi byte với một ít siêu dữ liệu, nên bạn có thể dùng bất kỳ định dạng mã hóa nào. Nếu cách mã hóa tương thích ngược và xuôi, bạn có sự linh hoạt lớn nhất để thay đổi các bên công bố và bên tiêu thụ một cách độc lập và triển khai chúng theo bất kỳ thứ tự nào.

If a consumer republishes messages to another topic, you may need to be careful to preserve unknown fields, to prevent the issue described previously in the context of databases (Figure 4-7).

Nếu một bên tiêu thụ công bố lại các thông điệp tới một chủ đề khác, bạn có thể cần cẩn thận để bảo toàn các trường chưa biết, nhằm ngăn vấn đề đã được mô tả trước đó trong ngữ cảnh các cơ sở dữ liệu (Hình 4-7).

Distributed actor frameworks — Các khung actor phân tán

The actor model is a programming model for **concurrency** in a single process. Rather than dealing directly with threads (and the associated problems of race conditions, locking, and deadlock), logic is encapsulated in actors. Each actor typically represents one client or entity, it may have some local state (which is not shared with any other actor), and it communicates with other actors by sending and receiving asynchronous messages. Message delivery is not guaranteed: in certain error scenarios, messages will be lost. Since each actor processes only one message at a time, it doesn't need to worry about threads, and each actor can be scheduled independently by the framework.

Mô hình actor là một mô hình lập trình cho tính tương tranh trong một tiến trình duy nhất. Thay vì làm việc trực tiếp với các luồng (thread) (và các vấn đề liên quan về tranh chấp, khóa và deadlock), logic được đóng gói trong các actor. Mỗi actor thường đại diện cho một client hoặc một thực thể, nó có thể có một số trạng thái cục bộ (không được chia sẻ với bất kỳ actor nào khác), và nó giao tiếp với các actor khác bằng cách gửi và nhận các thông điệp bất đồng bộ. Việc chuyển giao thông điệp không được bảo đảm: trong một số kịch bản lỗi nhất định, các thông điệp sẽ bị mất. Vì mỗi actor chỉ xử lý một thông điệp tại một thời điểm, nó không cần lo lắng về các luồng, và mỗi actor có thể được lập lịch một cách độc lập bởi khung làm việc.

In distributed actor frameworks, this programming model is used to scale an application across multiple nodes. The same message-passing mechanism is used, no matter whether the sender and recipient are on the same **node** or different nodes. If they are on different nodes, the message is transparently encoded into a byte sequence, sent over the network, and decoded on the other side.

Trong các khung actor phân tán, mô hình lập trình này được dùng để mở rộng một ứng dụng qua nhiều nút. Cùng một cơ chế truyền thông điệp được dùng, bất kể bên gửi và bên nhận có ở trên cùng một nút hay những nút khác nhau. Nếu chúng ở những nút khác nhau, thông điệp được mã hóa một cách trong suốt thành một chuỗi byte, gửi qua mạng, và giải mã ở phía bên kia.

Location transparency works better in the actor model than in RPC, because the actor model already assumes that messages may be lost, even within a single process. Although latency over the network is likely higher than within the same process, there is less of a fundamental mismatch between local and remote communication when using the actor model.

Tính trong suốt về vị trí hoạt động tốt hơn trong mô hình actor so với trong RPC, vì mô hình actor vốn đã giả định rằng các thông điệp có thể bị mất, kể cả trong một tiến trình duy nhất. Mặc dù độ trễ qua mạng có khả năng cao hơn so với trong cùng một tiến trình, có ít sự lệch pha cơ bản hơn giữa giao tiếp cục bộ và từ xa khi dùng mô hình actor.

A distributed actor framework essentially integrates a message broker and the actor programming model into a single framework. However, if you want to perform rolling upgrades of your actor-based application, you still have to worry about forward and backward compatibility, as messages may be sent from a node running the new version to a node running the old version, and vice versa.

Một khung actor phân tán về cơ bản tích hợp một môi giới thông điệp và mô hình lập trình actor vào một khung làm việc duy nhất. Tuy nhiên, nếu bạn muốn thực hiện các đợt nâng cấp cuốn chiếu cho ứng dụng dựa trên actor của bạn, bạn vẫn phải lo lắng về tính tương thích xuôi và ngược, vì các thông điệp có thể được gửi từ một nút chạy phiên bản mới tới một nút chạy phiên bản cũ, và ngược lại.

Three popular distributed actor frameworks handle message encoding as follows:

Ba khung actor phân tán phổ biến xử lý việc mã hóa thông điệp như sau:

- Akka uses Java's built-in serialization by default, which does not provide forward or backward compatibility. However, you can replace it with something like Protocol Buffers, and thus gain the ability to do rolling upgrades [50].
- Orleans by default uses a custom data encoding format that does not support rolling upgrade deployments; to deploy a new version of your application, you need to set up a new cluster, move traffic from the old cluster to the new one, and shut down the old one [51, 52]. Like with Akka, custom serialization plug-ins can be used.
- In Erlang OTP it is surprisingly hard to make changes to record schemas (despite the system having many features designed for **high availability**); rolling upgrades are possible but need

to be planned carefully [53]. An experimental new maps datatype (a JSON-like structure, introduced in Erlang R17 in 2014) may make this easier in the future [54].

- Akka theo mặc định dùng cơ chế tuần tự hóa sẵn có của Java, vốn không cung cấp tính tương thích xuôi hay ngược. Tuy nhiên, bạn có thể thay nó bằng một thứ như Protocol Buffers, qua đó có được khả năng thực hiện các đợt nâng cấp cuốn chiếu [50].
- Orleans theo mặc định dùng một định dạng mã hóa dữ liệu tùy biến vốn không hỗ trợ các đợt triển khai nâng cấp cuốn chiếu; để triển khai một phiên bản mới của ứng dụng, bạn cần thiết lập một cụm mới, chuyển lưu lượng từ cụm cũ sang cụm mới, rồi tắt cụm cũ đi [51, 52]. Giống như với Akka, các plug-in tuần tự hóa tùy biến có thể được dùng.
- Trong Erlang OTP, việc thay đổi các lược đồ bản ghi lại khó một cách đáng ngạc nhiên (dù hệ thống có nhiều tính năng được thiết kế cho tính sẵn sàng cao); các đợt nâng cấp cuốn chiếu là khả thi nhưng cần được lên kế hoạch cẩn thận [53]. Một kiểu dữ liệu maps mới mang tính thử nghiệm (một cấu trúc giống JSON, được giới thiệu trong Erlang R17 vào năm 2014) có thể làm điều này dễ hơn trong tương lai [54].

Summary — Tóm tắt

In this chapter we looked at several ways of turning data structures into bytes on the network or bytes on disk. We saw how the details of these encodings affect not only their efficiency, but more importantly also the architecture of applications and your options for deploying them.

Trong chương này, ta đã xem xét một số cách biến các cấu trúc dữ liệu thành các byte trên mạng hoặc các byte trên đĩa. Ta đã thấy các chi tiết của những cách mã hóa này ảnh hưởng không chỉ tới hiệu quả của chúng, mà quan trọng hơn còn tới kiến trúc của các ứng dụng và các lựa chọn của bạn để triển khai chúng.

In particular, many services need to support rolling upgrades, where a new version of a service is gradually deployed to a few nodes at a time, rather than deploying to all nodes simultaneously. Rolling upgrades allow new versions of a service to be released without downtime (thus encouraging frequent small releases over rare big releases) and make deployments less risky (allowing faulty releases to be detected and rolled back before they affect a large number of users). These properties are hugely beneficial for evolvability, the ease of making changes to an application.

Cụ thể, nhiều dịch vụ cần hỗ trợ các đợt nâng cấp cuốn chiếu, nơi một phiên bản mới của một dịch vụ được triển khai dần dần lên vài nút mỗi lần, thay vì triển khai lên tất cả các nút cùng một lúc. Các đợt nâng cấp cuốn chiếu cho phép phát hành các phiên bản mới của một dịch vụ mà không phải ngừng hoạt động (qua đó khuyến khích các đợt phát hành nhỏ thường xuyên thay vì các đợt phát hành lớn hiếm hoi) và làm cho việc triển khai bớt rủi ro hơn (cho phép phát hiện và hoàn tác các đợt phát hành lỗi trước khi chúng ảnh hưởng tới một số lượng lớn người dùng). Những tính chất này cực kỳ có lợi cho khả năng tiến hóa, sự dễ dàng trong việc thực hiện các thay đổi với một ứng dụng.

During rolling upgrades, or for various other reasons, we must assume that different nodes are running the different versions of our application's code. Thus, it is important that all data flowing around the system is encoded in a way that provides backward compatibility (new code can read old data) and forward compatibility (old code can read new data).

Trong các đợt nâng cấp cuốn chiếu, hoặc vì nhiều lý do khác, ta phải giả định rằng các nút khác nhau đang chạy các phiên bản khác nhau của mã ứng dụng của ta. Do đó, điều quan trọng là mọi dữ liệu chảy quanh trong hệ thống đều được mã hóa theo một cách cung cấp tính tương thích ngược (mã mới có thể đọc dữ liệu cũ) và tính tương thích xuôi (mã cũ có thể đọc dữ liệu mới).

We discussed several data encoding formats and their compatibility properties:

Ta đã bàn về một số định dạng mã hóa dữ liệu và các tính chất tương thích của chúng:

- Programming language-specific encodings are restricted to a single programming language and often fail to provide forward and backward compatibility.

- Textual formats like JSON, XML, and CSV are widespread, and their compatibility depends on how you use them. They have optional schema languages, which are sometimes helpful and sometimes a hindrance. These formats are somewhat vague about datatypes, so you have to be careful with things like numbers and binary strings.
- Binary schema-driven formats like Thrift, Protocol Buffers, and Avro allow compact, efficient encoding with clearly defined forward and backward compatibility semantics. The schemas can be useful for documentation and code generation in statically typed languages. However, they have the downside that data needs to be decoded before it is human-readable.
 - Các cách mã hóa đặc thù theo ngôn ngữ lập trình bị giới hạn ở một ngôn ngữ lập trình duy nhất và thường thất bại trong việc cung cấp tính tương thích xuôi và ngược.
 - Các định dạng văn bản như JSON, XML và CSV rất phổ biến, và tính tương thích của chúng phụ thuộc vào cách bạn dùng chúng. Chúng có các ngôn ngữ lược đồ tùy chọn, đôi khi hữu ích và đôi khi là một trở ngại. Các định dạng này hơi mơ hồ về các kiểu dữ liệu, nên bạn phải cẩn thận với những thứ như các con số và chuỗi nhị phân.
 - Các định dạng nhị phân dựa trên lược đồ như Thrift, Protocol Buffers và Avro cho phép mã hóa gọn gàng, hiệu quả với ngữ nghĩa tương thích xuôi và ngược được định nghĩa rõ ràng. Các lược đồ có thể hữu ích cho việc lập tài liệu và sinh mã trong các ngôn ngữ định kiểu tĩnh. Tuy nhiên, chúng có nhược điểm là dữ liệu cần được giải mã trước khi con người có thể đọc được.

We also discussed several modes of dataflow, illustrating different scenarios in which data encodings are important:

Ta cũng đã bàn về một số phương thức luồng dữ liệu, minh họa những kịch bản khác nhau mà trong đó các cách mã hóa dữ liệu là quan trọng:

- Databases, where the process writing to the database encodes the data and the process reading from the database decodes it
- RPC and REST APIs, where the client encodes a request, the server decodes the request and encodes a response, and the client finally decodes the response
- Asynchronous message passing (using message brokers or actors), where nodes communicate by sending each other messages that are encoded by the sender and decoded by the recipient
 - Các cơ sở dữ liệu, nơi tiến trình ghi vào cơ sở dữ liệu mã hóa dữ liệu và tiến trình đọc từ cơ sở dữ liệu giải mã nó
 - Các API RPC và REST, nơi client mã hóa một yêu cầu, máy chủ giải mã yêu cầu và mã hóa một phản hồi, và cuối cùng client giải mã phản hồi
 - Truyền thông điệp bất đồng bộ (dùng các môi giới thông điệp hoặc actor), nơi các nút giao tiếp bằng cách gửi cho nhau các thông điệp được mã hóa bởi bên gửi và giải mã bởi bên nhận

We can conclude that with a bit of care, backward/forward compatibility and rolling upgrades are quite achievable. May your application's evolution be rapid and your deployments be frequent.

Ta có thể kết luận rằng với một chút cẩn thận, tính tương thích ngược/xuôi và các đợt nâng cấp cuốn chiếu là hoàn toàn khả thi. Chúc cho sự tiến hóa của ứng dụng của bạn diễn ra mau lẹ và các đợt triển khai của bạn diễn ra thường xuyên.

Footnotes With the exception of some special cases, such as certain memory-mapped files or when operating directly on compressed data (as described in “Column Compression”).

Note that encoding has nothing to do with encryption. We don't discuss encryption in this book.

Actually, it has three—BinaryProtocol, CompactProtocol, and DenseProtocol—although DenseProtocol is only supported by the C++ implementation, so it doesn't count as cross-language [18]. Besides those, it also has two different JSON-based encoding formats [19]. What fun!

To be precise, the default value must be of the type of the first branch of the union, although this is a specific limitation of Avro, not a general feature of union types.

Except for MySQL, which often rewrites an entire table even though it is not strictly necessary, as mentioned in "Schema flexibility in the document model".

Even within each camp there are plenty of arguments. For example, HATEOAS (hypermedia as the engine of application state), often provokes discussions [35].

Despite the similarity of acronyms, SOAP is not a requirement for SOA. SOAP is a particular technology, whereas SOA is a general approach to building systems.

References [1] "Java Object Serialization Specification," docs.oracle.com, 2010.

[2] "Ruby 2.2.0 API Documentation," ruby-doc.org, Dec 2014.

[3] "The Python 3.4.3 Standard Library Reference Manual," docs.python.org, February 2015.

[4] "EsotericSoftware/kryo," github.com, October 2014.

[5] "CWE-502: Deserialization of Untrusted Data," Common Weakness Enumeration, cwe.mitre.org, July 30, 2014.

[6] Steve Breen: "What Do WebLogic, WebSphere, JBoss, Jenkins, OpenNMS, and Your Application Have in Common? This Vulnerability," foxglovesecurity.com, November 6, 2015.

[7] Patrick McKenzie: "What the Rails Security Issue Means for Your Startup," kalzumeus.com, January 31, 2013.

[8] Eishay Smith: "jvm-serializers wiki," github.com, November 2014.

[9] "XML Is a Poor Copy of S-Expressions," c2.com wiki.

[10] Matt Harris: "Snowflake: An Update and Some Very Important Information," email to Twitter Development Talk mailing list, October 19, 2010.

[11] Shudi (Sandy) Gao, C. M. Sperberg-McQueen, and Henry S. Thompson: "XML Schema 1.1," W3C Recommendation, May 2001.

[12] Francis Galiegue, Kris Zyp, and Gary Court: "JSON Schema," IETF Internet-Draft, February 2013.

[13] Yakov Shafranovich: "RFC 4180: Common Format and MIME Type for Comma-Separated Values (CSV) Files," October 2005.

[14] "MessagePack Specification," msgpack.org.

[15] Mark Slee, Aditya Agarwal, and Marc Kwiatkowski: "Thrift: Scalable Cross-Language Services Implementation," Facebook technical report, April 2007.

[16] "Protocol Buffers Developer Guide," Google, Inc., developers.google.com.

[17] Igor Anishchenko: "Thrift vs Protocol Buffers vs Avro - Biased Comparison," slideshare.net, September 17, 2012.

[18] "A Matrix of the Features Each Individual Language Library Supports," wiki.apache.org.

[19] Martin Kleppmann: "Schema Evolution in Avro, Protocol Buffers and Thrift," martin.kleppmann.com, December 5, 2012.

- [20] “Apache Avro 1.7.7 Documentation,” avro.apache.org, July 2014.
- [21] Doug Cutting, Chad Walters, Jim Kellerman, et al.: “[PROPOSAL] New Subproject: Avro,” email thread on hadoop-general mailing list, mail-archives.apache.org, April 2009.
- [22] Tony Hoare: “Null References: The Billion Dollar Mistake,” at QCon London, March 2009.
- [23] Aditya Auradkar and Tom Quiggle: “Introducing Espresso—LinkedIn’s Hot New Distributed Document Store,” engineering.linkedin.com, January 21, 2015.
- [24] Jay Kreps: “Putting Apache Kafka to Use: A Practical Guide to Building a Stream Data Platform (Part 2),” blog.confluent.io, February 25, 2015.
- [25] Gwen Shapira: “The Problem of Managing Schemas,” radar.oreilly.com, November 4, 2014.
- [26] “Apache Pig 0.14.0 Documentation,” pig.apache.org, November 2014.
- [27] John Larmouth: *ASN.1 Complete*. Morgan Kaufmann, 1999. ISBN: 978-0-122-33435-1
- [28] Russell Housley, Warwick Ford, Tim Polk, and David Solo: “RFC 2459: Internet X.509 Public Key Infrastructure: Certificate and CRL Profile,” IETF Network Working Group, Standards Track, January 1999.
- [29] Lev Walkin: “Question: Extensibility and Dropping Fields,” lionet.info, September 21, 2010.
- [30] Jesse James Garrett: “Ajax: A New Approach to Web Applications,” adaptivepath.com, February 18, 2005.
- [31] Sam Newman: *Building Microservices*. O’Reilly Media, 2015. ISBN: 978-1-491-95035-7
- [32] Chris Richardson: “Microservices: Decomposing Applications for Deployability and Scalability,” infoq.com, May 25, 2014.
- [33] Pat Helland: “Data on the Outside Versus Data on the Inside,” at 2nd Biennial Conference on Innovative Data Systems Research (CIDR), January 2005.
- [34] Roy Thomas Fielding: “Architectural Styles and the Design of Network-Based Software Architectures,” PhD Thesis, University of California, Irvine, 2000.
- [35] Roy Thomas Fielding: “REST APIs Must Be Hypertext-Driven,” roy.gbiv.com, October 20 2008.
- [36] “REST in Peace, SOAP,” royal.pingdom.com, October 15, 2010.
- [37] “Web Services Standards as of Q1 2007,” innoc.com, February 2007.
- [38] Pete Lacey: “The S Stands for Simple,” harmful.cat-v.org, November 15, 2006.
- [39] Stefan Tilkov: “Interview: Pete Lacey Criticizes Web Services,” infoq.com, December 12, 2006.
- [40] “OpenAPI Specification (fka Swagger RESTful API Documentation Specification) Version 2.0,” swagger.io, September 8, 2014.
- [41] Michi Henning: “The Rise and Fall of CORBA,” *ACM Queue*, volume 4, number 5, pages 28–34, June 2006. doi:10.1145/1142031.1142044
- [42] Andrew D. Birrell and Bruce Jay Nelson: “Implementing Remote Procedure Calls,” *ACM Transactions on Computer Systems (TOCS)*, volume 2, number 1, pages 39–59, February 1984. doi:10.1145/2080.357392
- [43] Jim Waldo, Geoff Wyant, Ann Wollrath, and Sam Kendall: “A Note on Distributed Computing,” Sun Microsystems Laboratories, Inc., Technical Report TR-94-29, November 1994.

- [44] Steve Vinoski: “Convenience over Correctness,” IEEE Internet Computing, volume 12, number 4, pages 89–92, July 2008. doi:10.1109/MIC.2008.75
- [45] Marius Eriksen: “Your Server as a Function,” at 7th Workshop on Programming Languages and Operating Systems (PLOS), November 2013. doi:10.1145/2525528.2525538
- [46] “grpc-common Documentation,” Google, Inc., github.com, February 2015.
- [47] Aditya Narayan and Irina Singh: “Designing and Versioning Compatible Web Services,” ibm.com, March 28, 2007.
- [48] Troy Hunt: “Your API Versioning Is Wrong, Which Is Why I Decided to Do It 3 Different Wrong Ways,” troyhunt.com, February 10, 2014.
- [49] “API Upgrades,” Stripe, Inc., April 2015.
- [50] Jonas Bonér: “Upgrade in an Akka Cluster,” email to akka-user mailing list, grokbase.com, August 28, 2013.
- [51] Philip A. Bernstein, Sergey Bykov, Alan Geller, et al.: “Orleans: Distributed Virtual Actors for Programmability and Scalability,” Microsoft Research Technical Report MSR-TR-2014-41, March 2014.
- [52] “Microsoft Project Orleans Documentation,” Microsoft Research, dotnet.github.io, 2015.
- [53] David Mercer, Sean Hinde, Yinso Chen, and Richard A O’Keefe: “beginner: Updating Data Structures,” email thread on erlang-questions mailing list, erlang.com, October 29, 2007.
- [54] Fred Hebert: “Postscript: Maps,” learnyousomeerlang.com, April 9, 2014.

Chapter 5. Replication — Chương 5.

Sao chép dữ liệu

The major difference between a thing that might go wrong and a thing that cannot possibly go wrong is that when a thing that cannot possibly go wrong goes wrong it usually turns out to be impossible to get at or repair.

Khác biệt lớn giữa một thứ có thể trục trặc và một thứ không thể nào trục trặc nằm ở chỗ: khi một thứ không thể nào trục trặc lại trục trặc, ta thường thấy rằng việc tiếp cận hay sửa chữa nó là điều bất khả.

Douglas Adams, Mostly Harmless (1992)

Douglas Adams, Mostly Harmless (1992)

Replication means keeping a copy of the same data on multiple machines that are connected via a network. As discussed in the introduction to Part II, there are several reasons why you might want to replicate data:

Sao chép dữ liệu (replication) nghĩa là giữ một bản sao của cùng một dữ liệu trên nhiều máy được kết nối qua mạng. Như đã bàn ở phần giới thiệu Phần II, có vài lý do khiến ta muốn sao chép dữ liệu:

- To keep data geographically close to your users (and thus reduce **latency**)
- To allow the system to continue working even if some of its parts have failed (and thus increase availability)
- To **scale out** the number of machines that can serve read queries (and thus increase read **throughput**)
 - Để giữ dữ liệu gần với người dùng về mặt địa lý (và nhờ đó giảm độ trễ).
 - Để cho phép hệ thống tiếp tục hoạt động ngay cả khi một số bộ phận của nó đã hỏng (và nhờ đó tăng tính sẵn sàng).
 - Để mở rộng số máy có thể phục vụ truy vấn đọc (và nhờ đó tăng thông lượng đọc).

In this chapter we will assume that your dataset is so small that each machine can hold a copy of the entire dataset. In Chapter 6 we will relax that assumption and discuss partitioning (sharding) of datasets that are too big for a single machine. In later chapters we will discuss various kinds of faults that can occur in a replicated **data system**, and how to deal with them.

Trong chương này, ta sẽ giả định rằng tập dữ liệu của bạn nhỏ tới mức mỗi máy có thể chứa được một bản sao của toàn bộ tập dữ liệu. Ở Chương 6, ta sẽ nới lỏng giả định đó và bàn về việc phân mảnh (partitioning, hay sharding) những tập dữ liệu quá lớn so với một máy đơn lẻ. Trong các chương sau, ta sẽ bàn về nhiều kiểu lỗi khác nhau có thể xảy ra trong một hệ thống dữ liệu được sao chép, và cách xử lý chúng.

If the data that you’re replicating does not change over time, then replication is easy: you just need to copy the data to every **node** once, and you’re done. All of the difficulty in replication lies in handling changes to replicated data, and that’s what this chapter is about. We will discuss three popular algorithms for replicating changes between nodes: single-leader, multi-leader, and leaderless replication. Almost all distributed databases use one of these three approaches. They all have various pros and cons, which we will examine in detail.

Nếu dữ liệu mà bạn đang sao chép không thay đổi theo thời gian, thì việc sao chép rất dễ: bạn chỉ cần sao dữ liệu sang mọi nút một lần là xong. Toàn bộ cái khó trong việc sao chép nằm ở chỗ xử lý các thay đổi đối với dữ liệu được sao chép, và đó chính là nội dung của chương này. Ta sẽ bàn về ba thuật toán phổ biến để sao chép các thay đổi giữa các nút: sao chép một-leader (single-leader), nhiều-leader (multi-leader), và không-leader (leaderless). Gần như mọi cơ sở dữ liệu phân tán đều dùng một trong ba cách tiếp cận này. Cả ba đều có những ưu nhược điểm khác nhau mà ta sẽ xem xét chi tiết.

There are many trade-offs to consider with replication: for example, whether to use synchronous or asynchronous replication, and how to handle failed replicas. Those are often configuration options in databases, and although the details vary by **database**, the general principles are similar across many different implementations. We will discuss the consequences of such choices in this chapter.

Có nhiều đánh đổi cần cân nhắc với việc sao chép: ví dụ, nên dùng sao chép đồng bộ hay bất đồng bộ, và xử lý các bản sao bị hỏng ra sao. Đó thường là những tùy chọn cấu hình trong cơ sở dữ liệu, và mặc dù chi tiết khác nhau tùy từng cơ sở dữ liệu, các nguyên tắc chung lại tương tự nhau trên nhiều hiện thực khác nhau. Ta sẽ bàn về hệ quả của những lựa chọn như vậy trong chương này.

Replication of databases is an old topic—the principles haven’t changed much since they were studied in the 1970s [1], because the fundamental constraints of networks have remained the same. However, outside of research, many developers continued to assume for a long time that a database consisted of just one node. Mainstream use of distributed databases is more recent. Since many application developers are new to this area, there has been a lot of misunderstanding around issues such as eventual consistency. In “Problems with Replication Lag” we will get more precise about eventual consistency and discuss things like the read-your-writes and monotonic reads guarantees.

Sao chép cơ sở dữ liệu là một chủ đề cũ — các nguyên tắc chưa thay đổi nhiều kể từ khi chúng được nghiên cứu vào thập niên 1970 [1], bởi những ràng buộc cơ bản của mạng vẫn giữ nguyên. Tuy nhiên, ngoài giới nghiên cứu, suốt một thời gian dài nhiều lập trình viên vẫn tiếp tục mặc định rằng một cơ sở dữ liệu chỉ gồm một nút duy nhất. Việc dùng cơ sở dữ liệu phân tán theo dòng chủ lưu là chuyện gần đây hơn. Vì nhiều lập trình viên ứng dụng còn mới với lĩnh vực này, đã có rất nhiều hiểu lầm xoay quanh những vấn đề như nhất quán cuối cùng (eventual consistency). Ở phần “Các vấn đề với độ trễ sao chép”, ta sẽ nói chính xác hơn về nhất quán cuối cùng và bàn về những thứ như bảo đảm đọc-thấy-bản-ghi-của-mình và đọc đơn điệu.

Leaders and Followers — Leader và follower

Each node that stores a copy of the database is called a replica. With multiple replicas, a question inevitably arises: how do we ensure that all the data ends up on all the replicas?

Mỗi nút lưu một bản sao của cơ sở dữ liệu được gọi là một bản sao (replica). Khi có nhiều bản sao, một câu hỏi tất yếu nảy sinh: làm sao để bảo đảm rằng toàn bộ dữ liệu rất cuộc đều có mặt trên tất cả các bản sao?

Every write to the database needs to be processed by every replica; otherwise, the replicas would no longer contain the same data. The most common solution for this is called leader-based replication (also known as active/passive or master-slave replication) and is illustrated in Figure 5-1. It works as follows:

Mọi thao tác ghi vào cơ sở dữ liệu đều cần được mọi bản sao xử lý; nếu không, các bản sao sẽ không còn chứa cùng dữ liệu nữa. Giải pháp phổ biến nhất cho việc này được gọi là sao chép dựa trên leader (leader-based replication, còn gọi là sao chép active/passive hoặc master-slave) và được minh họa ở Hình 5-1. Nó hoạt động như sau:

- One of the replicas is designated the leader (also known as master or primary). When clients want to write to the database, they must send their requests to the leader, which first writes the new data to its local storage.
- The other replicas are known as followers (read replicas, slaves, secondaries, or hot standbys). Whenever the leader writes new data to its local storage, it also sends the data change to all of its followers as part of a replication log or change stream. Each **follower** takes the log from the leader and updates its local copy of the database accordingly, by applying all writes in the same order as they were processed on the leader.
- When a **client** wants to read from the database, it can query either the leader or any of the followers. However, writes are only accepted on the leader (the followers are read-only from the client's point of view).
 - Một trong các bản sao được chỉ định làm leader (còn gọi là master hoặc primary). Khi client muốn ghi vào cơ sở dữ liệu, chúng phải gửi yêu cầu tới leader, và leader trước hết ghi dữ liệu mới vào kho lưu trữ cục bộ của nó.
 - Các bản sao còn lại được gọi là follower (bản sao đọc, slave, secondary, hoặc hot standby). Mỗi khi leader ghi dữ liệu mới vào kho lưu trữ cục bộ, nó cũng gửi thay đổi dữ liệu đó tới tất cả follower của mình dưới dạng một nhật ký sao chép (replication log) hay luồng thay đổi (change stream). Mỗi follower nhận nhật ký từ leader và cập nhật bản sao cục bộ của cơ sở dữ liệu cho phù hợp, bằng cách áp dụng mọi thao tác ghi theo đúng thứ tự như chúng đã được xử lý trên leader.
 - Khi một client muốn đọc từ cơ sở dữ liệu, nó có thể truy vấn hoặc leader hoặc bất kỳ

follower nào. Tuy nhiên, các thao tác ghi chỉ được chấp nhận trên leader (các follower chỉ cho đọc, xét từ góc nhìn của client).

Figure 5-1. Leader-based (master-slave) replication. — Hình 5-1. Sao chép dựa trên leader (master-slave). This mode of replication is a built-in feature of many relational databases, such as PostgreSQL (since version 9.0), MySQL, Oracle Data Guard [2], and SQL Server's AlwaysOn Availability Groups [3]. It is also used in some nonrelational databases, including MongoDB, RethinkDB, and Espresso [4]. Finally, leader-based replication is not restricted to only databases: distributed message brokers such as Kafka [5] and RabbitMQ highly available queues [6] also use it. Some network filesystems and replicated block devices such as DRBD are similar.

Chế độ sao chép này là tính năng tích hợp sẵn của nhiều cơ sở dữ liệu quan hệ, chẳng hạn PostgreSQL (từ phiên bản 9.0), MySQL, Oracle Data Guard [2], và AlwaysOn Availability Groups của SQL Server [3]. Nó cũng được dùng trong một số cơ sở dữ liệu phi quan hệ, bao gồm MongoDB, RethinkDB, và Espresso [4]. Cuối cùng, sao chép dựa trên leader không chỉ giới hạn ở cơ sở dữ liệu: các message broker phân tán như Kafka [5] và hàng đợi sẵn sàng cao của RabbitMQ [6] cũng dùng nó. Một số hệ thống tệp mạng và thiết bị khối được sao chép như DRBD cũng tương tự.

Synchronous Versus Asynchronous Replication — Sao chép đồng bộ so với bất đồng bộ

An important detail of a replicated system is whether the replication happens synchronously or asynchronously. (In relational databases, this is often a configurable option; other systems are often hardcoded to be either one or the other.)

Một chi tiết quan trọng của một hệ thống được sao chép là việc sao chép diễn ra đồng bộ hay bất đồng bộ. (Trong cơ sở dữ liệu quan hệ, đây thường là một tùy chọn cấu hình được; các hệ thống khác thường được cố định cứng theo một trong hai cách.)

Think about what happens in Figure 5-1, where the user of a website updates their profile image. At some point in time, the client sends the update request to the leader; shortly afterward, it is received by the leader. At some point, the leader forwards the data change to the followers. Eventually, the leader notifies the client that the update was successful.

Hãy nghĩ về những gì xảy ra trong Hình 5-1, nơi người dùng của một website cập nhật ảnh hồ sơ của họ. Ở một thời điểm nào đó, client gửi yêu cầu cập nhật tới leader; ngay sau đó, leader nhận được nó. Ở một thời điểm nào đó, leader chuyển tiếp thay đổi dữ liệu tới các follower. Cuối cùng, leader thông báo cho client rằng cập nhật đã thành công.

Figure 5-2 shows the communication between various components of the system: the user's client, the leader, and two followers. Time flows from left to right. A request or response message is shown as a thick arrow.

Hình 5-2 cho thấy sự giao tiếp giữa các thành phần khác nhau của hệ thống: client của người dùng, leader, và hai follower. Thời gian trôi từ trái sang phải. Một thông điệp yêu cầu hoặc phản hồi được biểu diễn bằng một mũi tên đậm.

Figure 5-2. Leader-based replication with one synchronous and one asynchronous follower. — Hình 5-2. Sao chép dựa trên leader với một follower đồng bộ và một follower bất đồng bộ. In the example of Figure 5-2, the replication to follower 1 is synchronous: the leader waits until

follower 1 has confirmed that it received the write before reporting success to the user, and before making the write visible to other clients. The replication to follower 2 is asynchronous: the leader sends the message, but doesn't wait for a response from the follower.

Trong ví dụ ở Hình 5-2, việc sao chép tới follower 1 là đồng bộ: leader chờ cho tới khi follower 1 xác nhận đã nhận được thao tác ghi rồi mới báo thành công cho người dùng, và mới làm thao tác ghi đó hiển thị với các client khác. Việc sao chép tới follower 2 là bất đồng bộ: leader gửi thông điệp đi, nhưng không chờ phản hồi từ follower.

The diagram shows that there is a substantial delay before follower 2 processes the message. Normally, replication is quite fast: most database systems apply changes to followers in less than a second. However, there is no guarantee of how long it might take. There are circumstances when followers might fall behind the leader by several minutes or more; for example, if a follower is recovering from a **failure**, if the system is operating near maximum capacity, or if there are network problems between the nodes.

Sơ đồ cho thấy có một độ trễ đáng kể trước khi follower 2 xử lý thông điệp. Thông thường, việc sao chép khá nhanh: hầu hết hệ cơ sở dữ liệu áp dụng các thay đổi lên follower trong chưa đầy một giây. Tuy nhiên, không có gì bảo đảm về việc nó có thể mất bao lâu. Có những hoàn cảnh khiến follower có thể tụt lại sau leader vài phút hoặc hơn; ví dụ, nếu một follower đang phục hồi sau một sự cố, nếu hệ thống đang vận hành gần mức công suất tối đa, hoặc nếu có vấn đề mạng giữa các nút.

The advantage of synchronous replication is that the follower is guaranteed to have an up-to-date copy of the data that is consistent with the leader. If the leader suddenly fails, we can be sure that the data is still available on the follower. The disadvantage is that if the synchronous follower doesn't respond (because it has crashed, or there is a network **fault**, or for any other reason), the write cannot be processed. The leader must block all writes and wait until the synchronous replica is available again.

Ưu điểm của sao chép đồng bộ là follower được bảo đảm có một bản sao dữ liệu cập nhật, nhất quán với leader. Nếu leader đột ngột hỏng, ta có thể chắc chắn rằng dữ liệu vẫn còn sẵn trên follower. Nhược điểm là nếu follower đồng bộ không phản hồi (vì nó đã sập, hoặc có lỗi mạng, hoặc vì bất kỳ lý do nào khác), thao tác ghi không thể được xử lý. Leader buộc phải chặn mọi thao tác ghi và chờ cho tới khi bản sao đồng bộ sẵn sàng trở lại.

For that reason, it is impractical for all followers to be synchronous: any one node **outage** would cause the whole system to grind to a halt. In practice, if you enable synchronous replication on a database, it usually means that one of the followers is synchronous, and the others are asynchronous. If the synchronous follower becomes unavailable or slow, one of the asynchronous followers is made synchronous. This guarantees that you have an up-to-date copy of the data on at least two nodes: the leader and one synchronous follower. This configuration is sometimes also called semi-synchronous [7].

Vì lý do đó, để tất cả follower đều đồng bộ là điều bất khả thi trên thực tế: chỉ cần một nút bất kỳ gián đoạn là cả hệ thống sẽ đình trệ. Trong thực tế, nếu bạn bật sao chép đồng bộ trên một cơ sở dữ liệu, điều đó thường có nghĩa là một trong các follower là đồng bộ, còn những follower khác là bất đồng bộ. Nếu follower đồng bộ trở nên không sẵn sàng hoặc chậm, một trong các follower bất đồng bộ sẽ được chuyển thành đồng bộ. Điều này bảo đảm rằng bạn có một bản sao dữ liệu cập nhật trên ít nhất hai nút: leader và một follower đồng bộ. Cấu hình này đôi khi còn được gọi là bán đồng bộ (semi-synchronous) [7].

Often, leader-based replication is configured to be completely asynchronous. In this case, if the leader fails and is not recoverable, any writes that have not yet been replicated to followers are

lost. This means that a write is not guaranteed to be durable, even if it has been confirmed to the client. However, a fully asynchronous configuration has the advantage that the leader can continue processing writes, even if all of its followers have fallen behind.

Thông thường, sao chép dựa trên leader được cấu hình để hoàn toàn bất đồng bộ. Trong trường hợp này, nếu leader hỏng và không thể phục hồi, mọi thao tác ghi chưa được sao chép tới follower sẽ bị mất. Điều này có nghĩa là một thao tác ghi không được bảo đảm là bền, ngay cả khi nó đã được xác nhận với client. Tuy nhiên, một cấu hình hoàn toàn bất đồng bộ có ưu điểm là leader có thể tiếp tục xử lý các thao tác ghi, ngay cả khi tất cả follower của nó đã tụt lại sau.

Weakening **durability** may sound like a bad trade-off, but asynchronous replication is nevertheless widely used, especially if there are many followers or if they are geographically distributed. We will return to this issue in “Problems with Replication Lag”.

Việc làm suy yếu độ bền nghe có vẻ là một đánh đổi tệ, nhưng sao chép bất đồng bộ vẫn vậy vẫn được dùng rộng rãi, đặc biệt khi có nhiều follower hoặc khi chúng phân tán về mặt địa lý. Ta sẽ quay lại vấn đề này ở phần “Các vấn đề với độ trễ sao chép”.

Research on Replication — Nghiên cứu về sao chép It can be a serious problem for asynchronously replicated systems to lose data if the leader fails, so researchers have continued investigating replication methods that do not lose data but still provide good performance and availability. For example, chain replication [8, 9] is a variant of synchronous replication that has been successfully implemented in a few systems such as Microsoft Azure Storage [10, 11].

Việc một hệ thống sao chép bất đồng bộ mất dữ liệu khi leader hỏng có thể là một vấn đề nghiêm trọng, nên các nhà nghiên cứu đã tiếp tục tìm tòi những phương pháp sao chép không làm mất dữ liệu mà vẫn mang lại hiệu năng và tính sẵn sàng tốt. Ví dụ, sao chép chuỗi (chain replication) [8, 9] là một biến thể của sao chép đồng bộ đã được hiện thực thành công trong một vài hệ thống như Microsoft Azure Storage [10, 11].

There is a strong connection between consistency of replication and consensus (getting several nodes to agree on a value), and we will explore this area of theory in more detail in Chapter 9. In this chapter we will concentrate on the simpler forms of replication that are most commonly used in databases in practice.

Có một mối liên hệ chặt chẽ giữa tính nhất quán của việc sao chép và đồng thuận (consensus, tức việc khiến nhiều nút cùng thống nhất về một giá trị), và ta sẽ khám phá mẩu lý thuyết này kỹ hơn ở Chương 9. Trong chương này, ta sẽ tập trung vào những hình thức sao chép đơn giản hơn vốn được dùng phổ biến nhất trong cơ sở dữ liệu trên thực tế.

Setting Up New Followers — Thiết lập follower mới

From time to time, you need to set up new followers—perhaps to increase the number of replicas, or to replace failed nodes. How do you ensure that the new follower has an accurate copy of the leader’s data?

Thi thoảng, bạn cần thiết lập các follower mới — có lẽ để tăng số bản sao, hoặc để thay thế các nút đã hỏng. Làm sao bạn bảo đảm rằng follower mới có một bản sao chính xác dữ liệu của leader?

Simply copying data files from one node to another is typically not sufficient: clients are constantly writing to the database, and the data is always in flux, so a standard file copy would see different parts of the database at different points in time. The result might not make any sense.

Việc chỉ đơn thuần sao chép các tệp dữ liệu từ nút này sang nút khác thường là không đủ: client liên tục ghi vào cơ sở dữ liệu, và dữ liệu luôn biến động, nên một phép sao tệp thông thường sẽ thấy các phần khác nhau của cơ sở dữ liệu ở các thời điểm khác nhau. Kết quả thu được có thể chẳng có ý nghĩa gì.

You could make the files on disk consistent by locking the database (making it unavailable for writes), but that would go against our goal of **high availability**. Fortunately, setting up a follower can usually be done without **downtime**. Conceptually, the process looks like this:

Bạn có thể làm cho các tệp trên đĩa trở nên nhất quán bằng cách khóa cơ sở dữ liệu (khiến nó không thể ghi được), nhưng điều đó sẽ đi ngược lại mục tiêu sẵn sàng cao của ta. May thay, việc thiết lập một follower thường có thể được thực hiện mà không cần ngừng hoạt động. Về mặt khái niệm, quá trình này trông như sau:

- Take a consistent snapshot of the leader's database at some point in time—if possible, without taking a lock on the entire database. Most databases have this feature, as it is also required for backups. In some cases, third-party tools are needed, such as innobackupex for MySQL [12].
- Copy the snapshot to the new follower node.
- The follower connects to the leader and requests all the data changes that have happened since the snapshot was taken. This requires that the snapshot is associated with an exact position in the leader's replication log. That position has various names: for example, PostgreSQL calls it the log sequence number, and MySQL calls it the binlog coordinates.
- When the follower has processed the backlog of data changes since the snapshot, we say it has caught up. It can now continue to process data changes from the leader as they happen.
 - Chụp một ảnh chụp nhanh (snapshot) nhất quán của cơ sở dữ liệu của leader tại một thời điểm nào đó — nếu có thể, mà không khóa toàn bộ cơ sở dữ liệu. Hầu hết cơ sở dữ liệu đều có tính năng này, vì nó cũng cần cho việc sao lưu. Trong một số trường hợp, cần đến các công cụ của bên thứ ba, chẳng hạn innobackupex cho MySQL [12].
 - Sao ảnh chụp nhanh đó sang nút follower mới.
 - Follower kết nối tới leader và yêu cầu mọi thay đổi dữ liệu đã xảy ra kể từ khi ảnh chụp nhanh được tạo. Điều này đòi hỏi ảnh chụp nhanh phải gắn với một vị trí chính xác trong nhật ký sao chép của leader. Vị trí đó có nhiều tên gọi: ví dụ, PostgreSQL gọi nó là số thứ tự nhật ký (log sequence number), còn MySQL gọi nó là tọa độ binlog (binlog coordinates).
 - Khi follower đã xử lý xong khối thay đổi dữ liệu tồn đọng kể từ ảnh chụp nhanh, ta nói nó đã bắt kịp. Giờ đây nó có thể tiếp tục xử lý các thay đổi dữ liệu từ leader ngay khi chúng xảy ra.

The practical steps of setting up a follower vary significantly by database. In some systems the process is fully automated, whereas in others it can be a somewhat arcane multi-step workflow that needs to be manually performed by an administrator.

Các bước thực tế để thiết lập một follower khác nhau đáng kể tùy từng cơ sở dữ liệu. Ở một số hệ thống, quá trình này được tự động hóa hoàn toàn, trong khi ở những hệ khác nó có thể là một quy trình nhiều bước khá bí hiểm, cần được quản trị viên thực hiện thủ công.

Handling Node Outages — Xử lý gián đoạn nút

Any node in the system can go down, perhaps unexpectedly due to a fault, but just as likely due to planned maintenance (for example, rebooting a machine to install a kernel security patch). Being able to reboot individual nodes without downtime is a big advantage for operations and maintenance. Thus, our goal is to keep the system as a whole running despite individual node failures, and to keep the impact of a node outage as small as possible.

Bất kỳ nút nào trong hệ thống cũng có thể ngừng hoạt động, có thể là bất ngờ do một lỗi, nhưng cũng dễ xảy ra không kém do bảo trì theo kế hoạch (ví dụ, khởi động lại một máy để cài bản vá bảo mật cho nhân hệ điều hành). Khả năng khởi động lại từng nút riêng lẻ mà không cần ngừng hoạt động là một lợi thế lớn cho việc vận hành và bảo trì. Do đó, mục tiêu của ta là giữ cho cả hệ thống tiếp tục chạy bất chấp các nút riêng lẻ bị hỏng, và giữ cho tác động của một gián đoạn nút nhỏ hết mức có thể.

How do you achieve high availability with leader-based replication?

Làm sao đạt được tính sẵn sàng cao với sao chép dựa trên leader?

Follower failure: Catch-up recovery — Follower hỏng: Phục hồi bắt kịp

On its local disk, each follower keeps a log of the data changes it has received from the leader. If a follower crashes and is restarted, or if the network between the leader and the follower is temporarily interrupted, the follower can recover quite easily: from its log, it knows the last transaction that was processed before the fault occurred. Thus, the follower can connect to the leader and request all the data changes that occurred during the time when the follower was disconnected. When it has applied these changes, it has caught up to the leader and can continue receiving a stream of data changes as before.

Trên đĩa cục bộ của mình, mỗi follower giữ một nhật ký các thay đổi dữ liệu mà nó đã nhận từ leader. Nếu một follower sập rồi khởi động lại, hoặc nếu mạng giữa leader và follower tạm thời bị gián đoạn, follower có thể phục hồi khá dễ dàng: từ nhật ký của mình, nó biết giao dịch cuối cùng được xử lý trước khi lỗi xảy ra. Do đó, follower có thể kết nối tới leader và yêu cầu mọi thay đổi dữ liệu đã xảy ra trong khoảng thời gian nó bị mất kết nối. Khi đã áp dụng những thay đổi này, nó đã bắt kịp leader và có thể tiếp tục nhận một luồng thay đổi dữ liệu như trước.

Leader failure: Failover — Leader hỏng: Chuyển đổi dự phòng

Handling a failure of the leader is trickier: one of the followers needs to be promoted to be the new leader, clients need to be reconfigured to send their writes to the new leader, and the other followers need to start consuming data changes from the new leader. This process is called failover.

Xử lý việc leader hỏng thì khó nhằn hơn: một trong các follower cần được nâng lên làm leader mới, các client cần được cấu hình lại để gửi thao tác ghi tới leader mới, và các follower còn lại cần bắt đầu tiêu thụ các thay đổi dữ liệu từ leader mới. Quá trình này được gọi là chuyển đổi dự phòng (failover).

Failover can happen manually (an administrator is notified that the leader has failed and takes the necessary steps to make a new leader) or automatically. An automatic failover process usually consists of the following steps:

Chuyển đổi dự phòng có thể diễn ra thủ công (một quản trị viên được thông báo rằng leader đã hỏng và thực hiện các bước cần thiết để tạo một leader mới) hoặc tự động. Một quá trình chuyển đổi dự phòng tự động thường gồm các bước sau:

- Determining that the leader has failed. There are many things that could potentially go wrong: crashes, power outages, network issues, and more. There is no foolproof way of detecting what has gone wrong, so most systems simply use a timeout: nodes frequently bounce messages back and forth between each other, and if a node doesn't respond for some period of time—say, 30 seconds—it is assumed to be dead. (If the leader is deliberately taken down for planned maintenance, this doesn't apply.)
- Choosing a new leader. This could be done through an election process (where the leader is chosen by a majority of the remaining replicas), or a new leader could be appointed by a previously elected controller node. The best candidate for leadership is usually the replica with the most up-to-date data changes from the old leader (to minimize any data loss). Getting all the nodes to agree on a new leader is a consensus problem, discussed in detail in Chapter 9.
- Reconfiguring the system to use the new leader. Clients now need to send their write requests to the new leader (we discuss this in “Request Routing”). If the old leader comes back, it might still believe that it is the leader, not realizing that the other replicas have forced it to step down. The system needs to ensure that the old leader becomes a follower and recognizes the new leader.
 - Xác định rằng leader đã hỏng. Có nhiều thứ có thể trục trặc: sập máy, mất điện, vấn đề mạng, và hơn thế nữa. Không có cách nào chắc chắn tuyệt đối để phát hiện chuyện gì đã trục trặc, nên hầu hết hệ thống chỉ đơn giản dùng một thời gian chờ (timeout): các nút thường xuyên gửi đi gửi lại các thông điệp giữa chúng với nhau, và nếu một nút không phản hồi trong một khoảng thời gian nào đó — chẳng hạn 30 giây — nó được giả định là đã chết. (Nếu leader bị chủ động hạ xuống để bảo trì theo kế hoạch, điều này không áp dụng.)
 - Chọn một leader mới. Việc này có thể được thực hiện qua một quá trình bầu chọn (trong đó leader được chọn bởi đa số các bản sao còn lại), hoặc một leader mới có thể được chỉ định bởi một nút điều khiển (controller) đã được bầu trước đó. Ứng viên tốt nhất cho cương vị leader thường là bản sao có những thay đổi dữ liệu cập nhật nhất từ leader cũ (để giảm thiểu mất mát dữ liệu). Việc khiến tất cả các nút thống nhất về một leader mới là một bài toán đồng thuận, được bàn chi tiết ở Chương 9.
 - Cấu hình lại hệ thống để dùng leader mới. Giờ đây các client cần gửi yêu cầu ghi của chúng tới leader mới (ta bàn về điều này ở phần “Định tuyến yêu cầu”). Nếu leader cũ quay trở lại, nó vẫn có thể tin rằng mình là leader, không nhận ra rằng các bản sao khác đã buộc nó phải thoái vị. Hệ thống cần bảo đảm rằng leader cũ trở thành follower và công nhận leader mới.

Failover is fraught with things that can go wrong:

Chuyển đổi dự phòng đầy rẫy những thứ có thể trục trặc:

- If asynchronous replication is used, the new leader may not have received all the writes from the old leader before it failed. If the former leader rejoins the cluster after a new leader has been chosen, what should happen to those writes? The new leader may have received conflicting writes in the meantime. The most common solution is for the old leader's unreplicated writes to simply be discarded, which may violate clients' durability expectations.
- Discarding writes is especially dangerous if other storage systems outside of the database need to be coordinated with the database contents. For example, in one incident at GitHub [13], an out-of-date MySQL follower was promoted to leader. The database used an autoincrementing counter to assign primary keys to new rows, but because the new leader's counter lagged behind the old leader's, it reused some primary keys that were previously assigned by the old leader.

These primary keys were also used in a Redis store, so the reuse of primary keys resulted in inconsistency between MySQL and Redis, which caused some private data to be disclosed to the wrong users.

- In certain fault scenarios (see Chapter 8), it could happen that two nodes both believe that they are the leader. This situation is called split brain, and it is dangerous: if both leaders accept writes, and there is no process for resolving conflicts (see “Multi-Leader Replication”), data is likely to be lost or corrupted. As a safety catch, some systems have a mechanism to shut down one node if two leaders are detected. However, if this mechanism is not carefully designed, you can end up with both nodes being shut down [14].
- What is the right timeout before the leader is declared dead? A longer timeout means a longer time to recovery in the case where the leader fails. However, if the timeout is too short, there could be unnecessary failovers. For example, a temporary **load** spike could cause a node’s **response time** to increase above the timeout, or a network glitch could cause delayed packets. If the system is already struggling with high load or network problems, an unnecessary failover is likely to make the situation worse, not better.

- Nếu dùng sao chép bất đồng bộ, leader mới có thể chưa nhận được tất cả thao tác ghi từ leader cũ trước khi nó hỏng. Nếu leader cũ tái gia nhập cụm sau khi một leader mới đã được chọn, thì những thao tác ghi đó nên ra sao? Trong lúc ấy, leader mới có thể đã nhận các thao tác ghi xung đột. Giải pháp phổ biến nhất là chỉ đơn giản loại bỏ những thao tác ghi chưa được sao chép của leader cũ, điều này có thể vi phạm kỳ vọng về độ bền của client.
- Việc loại bỏ thao tác ghi đặc biệt nguy hiểm nếu các hệ thống lưu trữ khác bên ngoài cơ sở dữ liệu cần được phối hợp với nội dung của cơ sở dữ liệu. Ví dụ, trong một sự cố tại GitHub [13], một follower MySQL không cập nhật đã được nâng lên làm leader. Cơ sở dữ liệu dùng một bộ đếm tự tăng để gán khóa chính cho các hàng mới, nhưng vì bộ đếm của leader mới tụt lại sau bộ đếm của leader cũ, nó đã tái sử dụng một số khóa chính vốn đã được leader cũ gán trước đó. Những khóa chính này cũng được dùng trong một kho Redis, nên việc tái sử dụng khóa chính đã dẫn đến sự bất nhất giữa MySQL và Redis, khiến một số dữ liệu riêng tư bị lộ cho sai người dùng.
- Trong một số kịch bản lỗi nhất định (xem Chương 8), có thể xảy ra chuyện hai nút đều tin rằng mình là leader. Tình huống này được gọi là split brain (não bị chia), và nó nguy hiểm: nếu cả hai leader đều chấp nhận thao tác ghi, và không có quy trình nào để giải quyết xung đột (xem “Sao chép nhiều-leader”), dữ liệu nhiều khả năng sẽ bị mất hoặc hỏng. Như một van an toàn, một số hệ thống có cơ chế tắt một nút nếu phát hiện hai leader. Tuy nhiên, nếu cơ chế này không được thiết kế cẩn thận, bạn có thể rơi vào cảnh cả hai nút đều bị tắt [14].
- Đây là thời gian chờ phù hợp trước khi tuyên bố leader đã chết? Thời gian chờ dài hơn nghĩa là thời gian phục hồi lâu hơn trong trường hợp leader hỏng. Tuy nhiên, nếu thời gian chờ quá ngắn, có thể xảy ra các lần chuyển đổi dự phòng không cần thiết. Ví dụ, một đợt tăng tải tạm thời có thể khiến thời gian phản hồi của một nút vượt quá thời gian chờ, hoặc một trục trặc mạng có thể khiến các gói tin bị trễ. Nếu hệ thống vốn đã đang chật vật với tải cao hoặc vấn đề mạng, một lần chuyển đổi dự phòng không cần thiết nhiều khả năng sẽ làm tình hình tệ hơn chứ không khá hơn.

There are no easy solutions to these problems. For this reason, some operations teams prefer to perform failovers manually, even if the software supports automatic failover.

Không có giải pháp dễ dàng nào cho những vấn đề này. Vì lý do đó, một số đội ngũ vận hành thích thực hiện chuyển đổi dự phòng thủ công, ngay cả khi phần mềm có hỗ trợ chuyển đổi dự phòng tự động.

These issues—node failures; unreliable networks; and trade-offs around replica consistency, durability, availability, and latency—are in fact fundamental problems in distributed systems. In Chapter 8 and Chapter 9 we will discuss them in greater depth.

Những vấn đề này — nút hỏng; mạng không tin cậy; và các đánh đổi xoay quanh tính nhất quán, độ bền, tính sẵn sàng, và độ trễ của bản sao — thực ra là những vấn đề căn bản trong hệ thống phân tán. Ở Chương 8 và Chương 9, ta sẽ bàn về chúng sâu hơn.

Implementation of Replication Logs — Hiện thực của nhật ký sao chép

How does leader-based replication work under the hood? Several different replication methods are used in practice, so let's look at each one briefly.

Sao chép dựa trên leader hoạt động ra sao ở bên trong? Vài phương pháp sao chép khác nhau được dùng trên thực tế, nên hãy xem xét sơ qua từng phương pháp.

Statement-based replication — Sao chép dựa trên câu lệnh

In the simplest case, the leader logs every write request (statement) that it executes and sends that statement log to its followers. For a **relational database**, this means that every INSERT, UPDATE, or DELETE statement is forwarded to followers, and each follower parses and executes that SQL statement as if it had been received from a client.

Trong trường hợp đơn giản nhất, leader ghi lại mọi yêu cầu ghi (câu lệnh) mà nó thực thi và gửi nhật ký câu lệnh đó tới các follower của mình. Với một cơ sở dữ liệu quan hệ, điều này có nghĩa là mọi câu lệnh INSERT, UPDATE, hay DELETE đều được chuyển tiếp tới các follower, và mỗi follower phân tích cú pháp rồi thực thi câu lệnh SQL đó như thể nó đã được nhận từ một client.

Although this may sound reasonable, there are various ways in which this approach to replication can break down:

Mặc dù điều này nghe có vẻ hợp lý, nhưng có nhiều cách khiến cách tiếp cận sao chép này có thể đổ vỡ:

- Any statement that calls a nondeterministic function, such as NOW() to get the current date and time or RAND() to get a random number, is likely to generate a different value on each replica.
- If statements use an autoincrementing **column**, or if they depend on the existing data in the database (e.g., UPDATE ... WHERE), they must be executed in exactly the same order on each replica, or else they may have a different effect. This can be limiting when there are multiple concurrently executing transactions.
- Statements that have side effects (e.g., triggers, stored procedures, user-defined functions) may result in different side effects occurring on each replica, unless the side effects are absolutely deterministic.
 - Bất kỳ câu lệnh nào gọi một hàm phi tất định, chẳng hạn NOW() để lấy ngày giờ hiện tại hoặc RAND() để lấy một số ngẫu nhiên, nhiều khả năng sẽ sinh ra một giá trị khác nhau trên mỗi bản sao.
 - Nếu các câu lệnh dùng một cột tự tăng, hoặc nếu chúng phụ thuộc vào dữ liệu hiện có trong cơ sở dữ liệu (ví dụ, UPDATE ... WHERE), thì chúng phải được thực thi theo

đúng cùng thứ tự trên mỗi bản sao, nếu không chúng có thể gây ra hiệu ứng khác nhau. Điều này có thể là một hạn chế khi có nhiều giao dịch thực thi tương tranh.

- Các câu lệnh có tác dụng phụ (ví dụ, trigger, stored procedure, hàm do người dùng định nghĩa) có thể dẫn đến những tác dụng phụ khác nhau trên mỗi bản sao, trừ phi các tác dụng phụ ấy hoàn toàn tất định.

It is possible to work around those issues—for example, the leader can replace any nondeterministic function calls with a fixed return value when the statement is logged so that the followers all get the same value. However, because there are so many **edge** cases, other replication methods are now generally preferred.

Có thể lách qua những vấn đề đó — ví dụ, leader có thể thay mọi lời gọi hàm phi tất định bằng một giá trị trả về cố định khi câu lệnh được ghi lại, để tất cả follower đều nhận cùng một giá trị. Tuy nhiên, vì có quá nhiều trường hợp biên, các phương pháp sao chép khác giờ đây nhìn chung được ưa chuộng hơn.

Statement-based replication was used in MySQL before version 5.1. It is still sometimes used today, as it is quite compact, but by default MySQL now switches to **row**-based replication (discussed shortly) if there is any nondeterminism in a statement. VoltDB uses statement-based replication, and makes it safe by requiring transactions to be deterministic [15].

Sao chép dựa trên câu lệnh từng được dùng trong MySQL trước phiên bản 5.1. Ngày nay nó đôi khi vẫn được dùng, vì nó khá gọn, nhưng theo mặc định MySQL giờ chuyển sang sao chép dựa trên hàng (bản ngay sau đây) nếu có bất kỳ tính phi tất định nào trong một câu lệnh. VoltDB dùng sao chép dựa trên câu lệnh, và làm cho nó an toàn bằng cách yêu cầu các giao dịch phải tất định [15].

Write-ahead log (WAL) shipping — Vận chuyển nhật ký ghi trước (WAL)

In Chapter 3 we discussed how storage engines represent data on disk, and we found that usually every write is appended to a log:

Ở Chương 3, ta đã bàn về cách các engine lưu trữ biểu diễn dữ liệu trên đĩa, và ta thấy rằng thường thì mọi thao tác ghi đều được nối thêm vào một nhật ký:

- In the case of a log-structured storage **engine** (see “SSTables and LSM-Trees”), this log is the main place for storage. Log segments are compacted and garbage-collected in the background.
- In the case of a B-tree (see “B-Trees”), which overwrites individual disk blocks, every modification is first written to a write-ahead log so that the **index** can be restored to a consistent state after a crash.
 - Trong trường hợp một engine lưu trữ cấu trúc nhật ký (xem “SSTable và cây LSM”), nhật ký này chính là nơi lưu trữ chính. Các phân đoạn nhật ký được nén và thu gom rác ở chế độ nền.
 - Trong trường hợp một cây B (xem “Cây B”), vốn ghi đè lên từng khối đĩa riêng lẻ, mọi thay đổi trước hết được ghi vào một nhật ký ghi trước (write-ahead log) để chỉ mục có thể được khôi phục về trạng thái nhất quán sau khi sập.

In either case, the log is an append-only sequence of bytes containing all writes to the database. We can use the exact same log to build a replica on another node: besides writing the log to disk, the leader also sends it across the network to its followers. When the follower processes this log, it builds a copy of the exact same data structures as found on the leader.

Trong cả hai trường hợp, nhật ký là một chuỗi byte chỉ-nối-thêm chứa mọi thao tác ghi vào cơ sở dữ liệu. Ta có thể dùng chính cái nhật ký đó để dựng một bản sao trên một nút

khác: ngoài việc ghi nhật ký xuống đĩa, leader còn gửi nó qua mạng tới các follower của mình. Khi follower xử lý nhật ký này, nó dựng nên một bản sao của đúng những cấu trúc dữ liệu giống như trên leader.

This method of replication is used in PostgreSQL and Oracle, among others [16]. The main disadvantage is that the log describes the data on a very low level: a WAL contains details of which bytes were changed in which disk blocks. This makes replication closely coupled to the storage engine. If the database changes its storage format from one version to another, it is typically not possible to run different versions of the database software on the leader and the followers.

Phương pháp sao chép này được dùng trong PostgreSQL và Oracle, cùng nhiều hệ khác [16]. Nhược điểm chính là nhật ký mô tả dữ liệu ở mức rất thấp: một WAL chứa các chi tiết về việc byte nào đã được thay đổi trong khối đĩa nào. Điều này khiến việc sao chép gắn kết chặt với engine lưu trữ. Nếu cơ sở dữ liệu đổi định dạng lưu trữ từ phiên bản này sang phiên bản khác, thì thường sẽ không thể chạy các phiên bản phần mềm cơ sở dữ liệu khác nhau trên leader và các follower.

That may seem like a minor implementation detail, but it can have a big operational impact. If the replication protocol allows the follower to use a newer software version than the leader, you can perform a zero-downtime upgrade of the database software by first upgrading the followers and then performing a failover to make one of the upgraded nodes the new leader. If the replication protocol does not allow this version mismatch, as is often the case with WAL shipping, such upgrades require downtime.

Điều đó nghe như một chi tiết hiện thực nhỏ nhặt, nhưng nó có thể có tác động vận hành lớn. Nếu giao thức sao chép cho phép follower dùng một phiên bản phần mềm mới hơn leader, bạn có thể thực hiện một lần nâng cấp phần mềm cơ sở dữ liệu không-ngừng-hoạt-động bằng cách trước tiên nâng cấp các follower rồi thực hiện chuyển đổi dự phòng để biến một trong các nút đã nâng cấp thành leader mới. Nếu giao thức sao chép không cho phép sự lệch phiên bản này, như thường xảy ra với việc vận chuyển WAL, thì những lần nâng cấp như vậy đòi hỏi phải ngừng hoạt động.

Logical (row-based) log replication — Sao chép nhật ký logic (dựa trên hàng)

An alternative is to use different log formats for replication and for the storage engine, which allows the replication log to be decoupled from the storage engine internals. This kind of replication log is called a logical log, to distinguish it from the storage engine's (physical) data representation.

Một lựa chọn thay thế là dùng các định dạng nhật ký khác nhau cho việc sao chép và cho engine lưu trữ, điều này cho phép tách rời nhật ký sao chép khỏi nội bộ của engine lưu trữ. Loại nhật ký sao chép này được gọi là nhật ký logic (logical log), để phân biệt với cách biểu diễn dữ liệu (vật lý) của engine lưu trữ.

A logical log for a relational database is usually a sequence of records describing writes to database tables at the granularity of a row:

Một nhật ký logic cho cơ sở dữ liệu quan hệ thường là một chuỗi các bản ghi mô tả các thao tác ghi vào các bảng cơ sở dữ liệu ở độ chi tiết cấp hàng:

- For an inserted row, the log contains the new values of all columns.
- For a deleted row, the log contains enough information to uniquely identify the row that was deleted. Typically this would be the **primary key**, but if there is no primary key on the **table**, the old values of all columns need to be logged.

- For an updated row, the log contains enough information to uniquely identify the updated row, and the new values of all columns (or at least the new values of all columns that changed).
 - Với một hàng được chèn vào, nhật ký chứa các giá trị mới của tất cả các cột.
 - Với một hàng bị xóa, nhật ký chứa đủ thông tin để nhận diện duy nhất hàng đã bị xóa. Thường thì đó sẽ là khóa chính, nhưng nếu bảng không có khóa chính, thì cần ghi lại các giá trị cũ của tất cả các cột.
 - Với một hàng được cập nhật, nhật ký chứa đủ thông tin để nhận diện duy nhất hàng được cập nhật, và các giá trị mới của tất cả các cột (hoặc ít nhất là các giá trị mới của tất cả các cột đã thay đổi).

A transaction that modifies several rows generates several such log records, followed by a record indicating that the transaction was committed. MySQL's binlog (when configured to use row-based replication) uses this approach [17].

Một giao dịch sửa đổi vài hàng sẽ sinh ra vài bản ghi nhật ký như vậy, theo sau là một bản ghi cho biết giao dịch đã được commit. Binlog của MySQL (khi được cấu hình để dùng sao chép dựa trên hàng) dùng cách tiếp cận này [17].

Since a logical log is decoupled from the storage engine internals, it can more easily be kept backward compatible, allowing the leader and the follower to run different versions of the database software, or even different storage engines.

Vì một nhật ký logic được tách rời khỏi nội bộ của engine lưu trữ, nó có thể dễ dàng được giữ tương thích ngược hơn, cho phép leader và follower chạy các phiên bản phần mềm cơ sở dữ liệu khác nhau, hoặc thậm chí các engine lưu trữ khác nhau.

A logical log format is also easier for external applications to parse. This aspect is useful if you want to send the contents of a database to an external system, such as a data warehouse for offline analysis, or for building custom indexes and caches [18]. This technique is called change data capture, and we will return to it in Chapter 11.

Một định dạng nhật ký logic cũng dễ cho các ứng dụng bên ngoài phân tích cú pháp hơn. Khía cạnh này hữu ích nếu bạn muốn gửi nội dung của một cơ sở dữ liệu tới một hệ thống bên ngoài, chẳng hạn một kho dữ liệu (data warehouse) để phân tích ngoại tuyến, hoặc để dựng các chỉ mục và bộ nhớ đệm tùy biến [18]. Kỹ thuật này được gọi là thu thập thay đổi dữ liệu (change data capture), và ta sẽ quay lại nó ở Chương 11.

Trigger-based replication — Sao chép dựa trên trigger

The replication approaches described so far are implemented by the database system, without involving any **application code**. In many cases, that's what you want—but there are some circumstances where more flexibility is needed. For example, if you want to only replicate a subset of the data, or want to replicate from one kind of database to another, or if you need conflict resolution logic (see “Handling Write Conflicts”), then you may need to move replication up to the application layer.

Các cách tiếp cận sao chép mô tả từ nãy tới giờ đều được hiện thực bởi chính hệ cơ sở dữ liệu, mà không cần dính dáng tới bất kỳ mã ứng dụng nào. Trong nhiều trường hợp, đó là điều bạn muốn — nhưng có một số hoàn cảnh cần nhiều sự linh hoạt hơn. Ví dụ, nếu bạn chỉ muốn sao chép một tập con của dữ liệu, hoặc muốn sao chép từ một loại cơ sở dữ liệu sang loại khác, hoặc nếu bạn cần logic giải quyết xung đột (xem “Xử lý xung đột ghi”), thì bạn có thể cần đưa việc sao chép lên tầng ứng dụng.

Some tools, such as Oracle GoldenGate [19], can make data changes available to an application by reading the database log. An alternative is to use features that are available in many relational databases: triggers and stored procedures.

Một số công cụ, chẳng hạn Oracle GoldenGate [19], có thể làm cho các thay đổi dữ liệu trở nên khả dụng với một ứng dụng bằng cách đọc nhật ký cơ sở dữ liệu. Một lựa chọn khác là dùng các tính năng có sẵn trong nhiều cơ sở dữ liệu quan hệ: trigger và stored procedure.

A trigger lets you register custom application code that is automatically executed when a data change (write transaction) occurs in a database system. The trigger has the opportunity to log this change into a separate table, from which it can be read by an external process. That external process can then apply any necessary application logic and replicate the data change to another system. Databus for Oracle [20] and Bucardo for Postgres [21] work like this, for example.

Một trigger cho phép bạn đăng ký mã ứng dụng tùy biến được tự động thực thi khi một thay đổi dữ liệu (giao dịch ghi) xảy ra trong một hệ cơ sở dữ liệu. Trigger có cơ hội ghi lại thay đổi này vào một bảng riêng, từ đó một tiến trình bên ngoài có thể đọc nó. Tiến trình bên ngoài đó sau đó có thể áp dụng bất kỳ logic ứng dụng cần thiết nào và sao chép thay đổi dữ liệu sang một hệ thống khác. Ví dụ, Databus cho Oracle [20] và Bucardo cho Postgres [21] hoạt động như vậy.

Trigger-based replication typically has greater overheads than other replication methods, and is more prone to bugs and limitations than the database's built-in replication. However, it can nevertheless be useful due to its flexibility.

Sao chép dựa trên trigger thường có chi phí phụ trội lớn hơn các phương pháp sao chép khác, và dễ gặp bug cùng hạn chế hơn so với sao chép tích hợp sẵn của cơ sở dữ liệu. Tuy nhiên, dầu vậy nó vẫn có thể hữu ích nhờ tính linh hoạt của mình.

Problems with Replication Lag — Các vấn đề với độ trễ sao chép

Being able to tolerate node failures is just one reason for wanting replication. As mentioned in the introduction to Part II, other reasons are **scalability** (processing more requests than a single machine can handle) and latency (placing replicas geographically closer to users).

Khả năng chịu được các nút hỏng chỉ là một lý do để muốn sao chép. Như đã đề cập ở phần giới thiệu Phần II, những lý do khác là khả năng mở rộng (xử lý nhiều yêu cầu hơn mức một máy đơn lẻ có thể kham nổi) và độ trễ (đặt các bản sao gần người dùng hơn về mặt địa lý).

Leader-based replication requires all writes to go through a single node, but read-only queries can go to any replica. For workloads that consist of mostly reads and only a small percentage of writes (a common pattern on the web), there is an attractive option: create many followers, and distribute the read requests across those followers. This removes load from the leader and allows read requests to be served by nearby replicas.

Sao chép dựa trên leader đòi hỏi mọi thao tác ghi phải đi qua một nút duy nhất, nhưng các truy vấn chỉ-đọc có thể đi tới bất kỳ bản sao nào. Với những khối lượng công việc chủ yếu là đọc và chỉ có một tỷ lệ nhỏ là ghi (một mẫu phổ biến trên web), có một lựa chọn hấp dẫn: tạo nhiều follower, và phân phối các yêu cầu đọc trên các follower đó. Điều này gỡ tải khỏi leader và cho phép các yêu cầu đọc được phục vụ bởi các bản sao gần đó.

In this read-scaling architecture, you can increase the capacity for serving read-only requests simply by adding more followers. However, this approach only realistically works with asynchronous replication—if you tried to synchronously replicate to all followers, a single node failure or network outage would make the entire system unavailable for writing. And the more nodes you have, the likelier it is that one will be down, so a fully synchronous configuration would be very unreliable.

Trong kiến trúc mở rộng-đọc này, bạn có thể tăng năng lực phục vụ các yêu cầu chỉ-đọc đơn giản bằng cách thêm nhiều follower hơn. Tuy nhiên, cách tiếp cận này thực tế chỉ hiệu quả với sao chép bất đồng bộ — nếu bạn cố sao chép đồng bộ tới tất cả follower, thì chỉ một nút hỏng hoặc một gián đoạn mạng cũng sẽ khiến cả hệ thống không thể ghi được. Và càng có nhiều nút thì khả năng một nút trong số đó bị ngừng hoạt động càng cao, nên một cấu hình hoàn toàn đồng bộ sẽ rất kém tin cậy.

Unfortunately, if an application reads from an asynchronous follower, it may see outdated information if the follower has fallen behind. This leads to apparent inconsistencies in the database: if you run the same query on the leader and a follower at the same time, you may get different results, because not all writes have been reflected in the follower. This inconsistency is just a temporary state—if you stop writing to the database and wait a while, the followers will eventually

catch up and become consistent with the leader. For that reason, this effect is known as eventual consistency [22, 23].

Đáng tiếc, nếu một ứng dụng đọc từ một follower bất đồng bộ, nó có thể thấy thông tin lỗi thời nếu follower đã tụt lại sau. Điều này dẫn đến những bất nhất hiển hiện trong cơ sở dữ liệu: nếu bạn chạy cùng một truy vấn trên leader và một follower cùng lúc, bạn có thể nhận được kết quả khác nhau, vì không phải mọi thao tác ghi đều đã được phản ánh trên follower. Sự bất nhất này chỉ là một trạng thái tạm thời — nếu bạn ngừng ghi vào cơ sở dữ liệu và chờ một lúc, các follower rốt cuộc sẽ bắt kịp và trở nên nhất quán với leader. Vì lý do đó, hiệu ứng này được gọi là nhất quán cuối cùng (eventual consistency) [22, 23].

The term “eventually” is deliberately vague: in general, there is no limit to how far a replica can fall behind. In normal operation, the delay between a write happening on the leader and being reflected on a follower—the replication lag—may be only a fraction of a second, and not noticeable in practice. However, if the system is operating near capacity or if there is a problem in the network, the lag can easily increase to several seconds or even minutes.

Từ “cuối cùng” được dùng một cách mơ hồ có chủ ý: nói chung, không có giới hạn nào về việc một bản sao có thể tụt lại sau bao xa. Trong vận hành bình thường, độ trễ giữa lúc một thao tác ghi xảy ra trên leader và lúc nó được phản ánh trên một follower — gọi là độ trễ sao chép (replication lag) — có thể chỉ là một phần nhỏ của giây, và không đáng để ý trong thực tế. Tuy nhiên, nếu hệ thống đang vận hành gần mức công suất hoặc nếu có vấn đề trong mạng, độ trễ có thể dễ dàng tăng lên vài giây hoặc thậm chí vài phút.

When the lag is so large, the inconsistencies it introduces are not just a theoretical issue but a real problem for applications. In this section we will highlight three examples of problems that are likely to occur when there is replication lag and outline some approaches to solving them.

Khi độ trễ lớn đến vậy, những bất nhất mà nó gây ra không chỉ là một vấn đề lý thuyết mà là một vấn đề thực sự đối với các ứng dụng. Trong phần này, ta sẽ làm nổi bật ba ví dụ về những vấn đề nhiều khả năng xảy ra khi có độ trễ sao chép và phác thảo một số cách tiếp cận để giải quyết chúng.

Reading Your Own Writes — Đọc thấy bản ghi của chính mình

Many applications let the user submit some data and then view what they have submitted. This might be a record in a customer database, or a comment on a discussion thread, or something else of that sort. When new data is submitted, it must be sent to the leader, but when the user views the data, it can be read from a follower. This is especially appropriate if data is frequently viewed but only occasionally written.

Nhiều ứng dụng cho phép người dùng gửi đi một dữ liệu nào đó rồi xem lại những gì họ đã gửi. Đây có thể là một bản ghi trong cơ sở dữ liệu khách hàng, hoặc một bình luận trên một luồng thảo luận, hoặc một thứ gì đó tương tự. Khi dữ liệu mới được gửi đi, nó phải được gửi tới leader, nhưng khi người dùng xem dữ liệu, nó có thể được đọc từ một follower. Điều này đặc biệt phù hợp nếu dữ liệu thường được xem nhưng chỉ thi thoảng mới được ghi.

With asynchronous replication, there is a problem, illustrated in Figure 5-3: if the user views the data shortly after making a write, the new data may not yet have reached the replica. To the user, it looks as though the data they submitted was lost, so they will be understandably unhappy.

Với sao chép bất đồng bộ, có một vấn đề, được minh họa ở Hình 5-3: nếu người dùng xem dữ liệu ngay sau khi thực hiện một thao tác ghi, dữ liệu mới có thể chưa kịp đến được bản

sao. Đối với người dùng, có vẻ như dữ liệu họ vừa gửi đã bị mất, nên họ sẽ khó chịu cũng là điều dễ hiểu.

Figure 5-3. A user makes a write, followed by a read from a stale replica. To prevent this anomaly, we need read-after-write consistency. — Hình 5-3. Một người dùng thực hiện một thao tác ghi, theo sau là một thao tác đọc từ một bản sao lỗi thời. Để ngăn dị thường này, ta cần tính nhất quán đọc-sau-ghi. In this situation, we need read-after-write consistency, also known as read-your-writes consistency [24]. This is a guarantee that if the user reloads the page, they will always see any updates they submitted themselves. It makes no promises about other users: other users' updates may not be visible until some later time. However, it reassures the user that their own input has been saved correctly.

Trong tình huống này, ta cần tính nhất quán đọc-sau-ghi (read-after-write consistency), còn gọi là tính nhất quán đọc-thấy-bản-ghi-của-mình (read-your-writes consistency) [24]. Đây là một bảo đảm rằng nếu người dùng tải lại trang, họ sẽ luôn thấy mọi cập nhật mà chính họ đã gửi. Nó không hứa hẹn gì về những người dùng khác: cập nhật của những người dùng khác có thể chưa hiển thị cho tới một lúc nào đó về sau. Tuy nhiên, nó trấn an người dùng rằng dữ liệu nhập vào của chính họ đã được lưu đúng cách.

How can we implement read-after-write consistency in a system with leader-based replication? There are various possible techniques. To mention a few:

Làm sao ta hiện thực được tính nhất quán đọc-sau-ghi trong một hệ thống dùng sao chép dựa trên leader? Có nhiều kỹ thuật khả dĩ. Nêu ra một vài kỹ thuật:

- When reading something that the user may have modified, read it from the leader; otherwise, read it from a follower. This requires that you have some way of knowing whether something might have been modified, without actually querying it. For example, user profile information on a social network is normally only editable by the owner of the profile, not by anybody else. Thus, a simple rule is: always read the user's own profile from the leader, and any other users' profiles from a follower.
- If most things in the application are potentially editable by the user, that approach won't be effective, as most things would have to be read from the leader (negating the benefit of read scaling). In that case, other criteria may be used to decide whether to read from the leader. For example, you could track the time of the last update and, for one minute after the last update, make all reads from the leader. You could also monitor the replication lag on followers and prevent queries on any follower that is more than one minute behind the leader.
- The client can remember the timestamp of its most recent write—then the system can ensure that the replica serving any reads for that user reflects updates at least until that timestamp. If a replica is not sufficiently up to date, either the read can be handled by another replica or the query can wait until the replica has caught up. The timestamp could be a logical timestamp (something that indicates ordering of writes, such as the log sequence number) or the actual system clock (in which case clock synchronization becomes critical; see “Unreliable Clocks”).
- If your replicas are distributed across multiple datacenters (for geographical proximity to users or for availability), there is additional complexity. Any request that needs to be served by the leader must be routed to the **datacenter** that contains the leader.
- Khi đọc một thứ gì đó mà người dùng có thể đã sửa đổi, hãy đọc nó từ leader; nếu không, hãy đọc nó từ một follower. Điều này đòi hỏi bạn có cách nào đó để biết liệu một thứ có thể đã bị sửa đổi hay chưa, mà không thực sự truy vấn nó. Ví dụ, thông tin hồ sơ người dùng trên một mạng xã hội thường chỉ chủ nhân hồ sơ mới sửa được, chứ không phải bất kỳ ai khác. Do đó, một quy tắc đơn giản là: luôn đọc hồ sơ của chính người dùng từ leader, và hồ sơ của bất kỳ người dùng nào khác từ một follower.

- Nếu hầu hết mọi thứ trong ứng dụng đều có khả năng bị người dùng sửa đổi, thì cách tiếp cận đó sẽ không hiệu quả, vì hầu hết mọi thứ sẽ phải đọc từ leader (triệt tiêu lợi ích của việc mở rộng-đọc). Trong trường hợp đó, các tiêu chí khác có thể được dùng để quyết định có nên đọc từ leader hay không. Ví dụ, bạn có thể theo dõi thời điểm của lần cập nhật cuối cùng và, trong một phút sau lần cập nhật cuối, thực hiện mọi thao tác đọc từ leader. Bạn cũng có thể giám sát độ trễ sao chép trên các follower và ngăn các truy vấn trên bất kỳ follower nào tụt lại sau leader hơn một phút.
- Client có thể nhớ dấu thời gian của thao tác ghi gần nhất của nó — khi đó hệ thống có thể bảo đảm rằng bản sao phục vụ bất kỳ thao tác đọc nào cho người dùng đó phản ánh được các cập nhật ít nhất là cho tới dấu thời gian ấy. Nếu một bản sao chưa đủ cập nhật, thì hoặc thao tác đọc có thể được một bản sao khác xử lý, hoặc truy vấn có thể chờ cho tới khi bản sao bắt kịp. Dấu thời gian có thể là một dấu thời gian logic (một thứ chỉ ra thứ tự của các thao tác ghi, chẳng hạn số thứ tự nhật ký) hoặc đồng hồ hệ thống thực sự (trong trường hợp đó việc đồng bộ đồng hồ trở nên tối quan trọng; xem “Đồng hồ không tin cậy”).
- Nếu các bản sao của bạn phân tán trên nhiều trung tâm dữ liệu (để gần người dùng về mặt địa lý hoặc để bảo đảm tính sẵn sàng), thì có thêm sự phức tạp. Bất kỳ yêu cầu nào cần được leader phục vụ đều phải được định tuyến tới trung tâm dữ liệu chứa leader.

Another complication arises when the same user is accessing your service from multiple devices, for example a desktop web browser and a mobile app. In this case you may want to provide cross-device read-after-write consistency: if the user enters some information on one device and then views it on another device, they should see the information they just entered.

Một rắc rối khác nảy sinh khi cùng một người dùng đang truy cập dịch vụ của bạn từ nhiều thiết bị, ví dụ một trình duyệt web trên máy tính để bàn và một ứng dụng di động. Trong trường hợp này, bạn có thể muốn cung cấp tính nhất quán đọc-sau-ghi liên-thiết-bị: nếu người dùng nhập một thông tin nào đó trên một thiết bị rồi xem nó trên một thiết bị khác, họ phải thấy được thông tin họ vừa nhập.

In this case, there are some additional issues to consider:

Trong trường hợp này, có thêm một số vấn đề cần cân nhắc:

- Approaches that require remembering the timestamp of the user’s last update become more difficult, because the code running on one device doesn’t know what updates have happened on the other device. This metadata will need to be centralized.
- If your replicas are distributed across different datacenters, there is no guarantee that connections from different devices will be routed to the same datacenter. (For example, if the user’s desktop computer uses the home broadband connection and their mobile device uses the cellular data network, the devices’ network routes may be completely different.) If your approach requires reading from the leader, you may first need to route requests from all of a user’s devices to the same datacenter.
 - Các cách tiếp cận đòi hỏi nhớ dấu thời gian của lần cập nhật cuối của người dùng trở nên khó khăn hơn, vì mã chạy trên một thiết bị không biết những cập nhật nào đã xảy ra trên thiết bị kia. Siêu dữ liệu này sẽ cần được tập trung hóa.
 - Nếu các bản sao của bạn phân tán trên các trung tâm dữ liệu khác nhau, thì không có gì bảo đảm rằng các kết nối từ các thiết bị khác nhau sẽ được định tuyến tới cùng một trung tâm dữ liệu. (Ví dụ, nếu máy tính để bàn của người dùng dùng kết nối băng rộng tại nhà còn thiết bị di động của họ dùng mạng dữ liệu di động, thì tuyến mạng của các thiết bị có thể hoàn toàn khác nhau.) Nếu cách tiếp cận của bạn đòi hỏi đọc

từ leader, bạn có thể trước hết cần định tuyến các yêu cầu từ tất cả thiết bị của một người dùng tới cùng một trung tâm dữ liệu.

Monotonic Reads — Đọc đơn điệu

Our second example of an anomaly that can occur when reading from asynchronous followers is that it's possible for a user to see things moving backward in time.

Ví dụ thứ hai của ta về một dị thường có thể xảy ra khi đọc từ các follower bất đồng bộ là người dùng có thể thấy mọi thứ chuyển động ngược về quá khứ.

This can happen if a user makes several reads from different replicas. For example, Figure 5-4 shows user 2345 making the same query twice, first to a follower with little lag, then to a follower with greater lag. (This scenario is quite likely if the user refreshes a web page, and each request is routed to a random server.) The first query returns a comment that was recently added by user 1234, but the second query doesn't return anything because the lagging follower has not yet picked up that write. In effect, the second query is observing the system at an earlier point in time than the first query. This wouldn't be so bad if the first query hadn't returned anything, because user 2345 probably wouldn't know that user 1234 had recently added a comment. However, it's very confusing for user 2345 if they first see user 1234's comment appear, and then see it disappear again.

Điều này có thể xảy ra nếu một người dùng thực hiện nhiều thao tác đọc từ các bản sao khác nhau. Ví dụ, Hình 5-4 cho thấy người dùng 2345 thực hiện cùng một truy vấn hai lần, lần đầu tới một follower có độ trễ nhỏ, rồi tới một follower có độ trễ lớn hơn. (Kịch bản này khá dễ xảy ra nếu người dùng làm mới một trang web, và mỗi yêu cầu được định tuyến tới một máy chủ ngẫu nhiên.) Truy vấn đầu tiên trả về một bình luận vừa được người dùng 1234 thêm vào, nhưng truy vấn thứ hai không trả về gì cả vì follower tụt lại sau chưa kịp tiếp nhận thao tác ghi đó. Trên thực tế, truy vấn thứ hai đang quan sát hệ thống ở một thời điểm sớm hơn truy vấn thứ nhất. Điều này sẽ không đến nỗi tệ nếu truy vấn đầu tiên không trả về gì, vì khi đó người dùng 2345 hẳn sẽ không biết rằng người dùng 1234 vừa thêm một bình luận. Tuy nhiên, sẽ rất khó hiểu đối với người dùng 2345 nếu họ trước hết thấy bình luận của người dùng 1234 xuất hiện, rồi sau đó lại thấy nó biến mất.

Figure 5-4. A user first reads from a fresh replica, then from a stale replica. Time appears to go backward. To prevent this anomaly, we need monotonic reads. — Hình 5-4. Một người dùng trước hết đọc từ một bản sao mới, rồi từ một bản sao lỗi thời. Thời gian có vẻ trôi ngược. Để ngăn dị thường này, ta cần đọc đơn điệu. Monotonic reads [23] is a guarantee that this kind of anomaly does not happen. It's a lesser guarantee than strong consistency, but a stronger guarantee than eventual consistency. When you read data, you may see an old value; monotonic reads only means that if one user makes several reads in sequence, they will not see time go backward—i.e., they will not read older data after having previously read newer data.

Đọc đơn điệu (monotonic reads) [23] là một bảo đảm rằng kiểu dị thường này không xảy ra. Đây là một bảo đảm yếu hơn nhất quán mạnh, nhưng mạnh hơn nhất quán cuối cùng. Khi bạn đọc dữ liệu, bạn có thể thấy một giá trị cũ; đọc đơn điệu chỉ có nghĩa là nếu một người dùng thực hiện nhiều thao tác đọc liên tiếp, họ sẽ không thấy thời gian trôi ngược — tức là họ sẽ không đọc dữ liệu cũ hơn sau khi trước đó đã đọc dữ liệu mới hơn.

One way of achieving monotonic reads is to make sure that each user always makes their reads from the same replica (different users can read from different replicas). For example, the replica can be chosen based on a hash of the user ID, rather than randomly. However, if that replica fails, the user's queries will need to be rerouted to another replica.

Một cách để đạt được đọc đơn điệu là bảo đảm rằng mỗi người dùng luôn thực hiện các thao tác đọc của mình từ cùng một bản sao (những người dùng khác nhau có thể đọc từ các bản sao khác nhau). Ví dụ, bản sao có thể được chọn dựa trên một giá trị băm của ID người dùng, thay vì chọn ngẫu nhiên. Tuy nhiên, nếu bản sao đó hỏng, các truy vấn của người dùng sẽ cần được định tuyến lại tới một bản sao khác.

Consistent Prefix Reads — Đọc tiền tố nhất quán

Our third example of replication lag anomalies concerns violation of causality. Imagine the following short dialog between Mr. Poons and Mrs. Cake:

Ví dụ thứ ba của ta về các dị thường do độ trễ sao chép liên quan tới việc vi phạm tính nhân quả. Hãy hình dung đoạn đối thoại ngắn sau giữa ông Poons và bà Cake:

How far into the future can you see, Mrs. Cake?

Bà Cake này, bà có thể nhìn vào tương lai xa tới đâu?

About ten seconds usually, Mr. Poons.

Thường thì khoảng mười giây thôi, ông Poons ạ.

There is a causal dependency between those two sentences: Mrs. Cake heard Mr. Poons's question and answered it.

Có một sự phụ thuộc nhân quả giữa hai câu nói đó: bà Cake nghe câu hỏi của ông Poons rồi trả lời nó.

Now, imagine a third person is listening to this conversation through followers. The things said by Mrs. Cake go through a follower with little lag, but the things said by Mr. Poons have a longer replication lag (see Figure 5-5). This observer would hear the following:

Giờ, hãy hình dung một người thứ ba đang lắng nghe cuộc trò chuyện này thông qua các follower. Những điều bà Cake nói đi qua một follower có độ trễ nhỏ, còn những điều ông Poons nói lại có độ trễ sao chép lớn hơn (xem Hình 5-5). Người quan sát này sẽ nghe thấy như sau:

About ten seconds usually, Mr. Poons.

Thường thì khoảng mười giây thôi, ông Poons ạ.

How far into the future can you see, Mrs. Cake?

Bà Cake này, bà có thể nhìn vào tương lai xa tới đâu?

To the observer it looks as though Mrs. Cake is answering the question before Mr. Poons has even asked it. Such psychic powers are impressive, but very confusing [25].

Đối với người quan sát, có vẻ như bà Cake đang trả lời câu hỏi trước cả khi ông Poons kịp hỏi nó. Năng lực ngoại cảm như vậy thật ấn tượng, nhưng cũng rất khó hiểu [25].

Figure 5-5. If some partitions are replicated slower than others, an observer may see the answer before they see the question. — Hình 5-5. Nếu một số phân mảnh được sao chép chậm hơn những phân mảnh khác, người quan sát có thể thấy câu trả lời trước khi họ thấy câu hỏi. Preventing this kind of anomaly requires another type of guarantee: consistent prefix reads [23]. This guarantee says that if a sequence of writes happens in a certain order, then anyone reading those writes will see them appear in the same order.

Ngăn chặn kiểu dị thường này đòi hỏi một loại bảo đảm khác: đọc tiền tố nhất quán (consistent prefix reads) [23]. Bảo đảm này nói rằng nếu một chuỗi các thao tác ghi xảy ra theo một thứ tự nhất định, thì bất kỳ ai đọc những thao tác ghi đó cũng sẽ thấy chúng xuất hiện theo cùng thứ tự ấy.

This is a particular problem in partitioned (sharded) databases, which we will discuss in Chapter 6. If the database always applies writes in the same order, reads always see a consistent prefix, so this anomaly cannot happen. However, in many distributed databases, different partitions operate independently, so there is no global ordering of writes: when a user reads from the database, they may see some parts of the database in an older state and some in a newer state.

Đây là một vấn đề đặc thù trong các cơ sở dữ liệu được phân mảnh (sharded), mà ta sẽ bàn ở Chương 6. Nếu cơ sở dữ liệu luôn áp dụng các thao tác ghi theo cùng một thứ tự, thì các thao tác đọc luôn thấy một tiền tố nhất quán, nên dị thường này không thể xảy ra. Tuy nhiên, trong nhiều cơ sở dữ liệu phân tán, các phân mảnh khác nhau hoạt động độc lập, nên không có thứ tự ghi toàn cục: khi một người dùng đọc từ cơ sở dữ liệu, họ có thể thấy một số phần của cơ sở dữ liệu ở trạng thái cũ hơn và một số phần ở trạng thái mới hơn.

One solution is to make sure that any writes that are causally related to each other are written to the same partition—but in some applications that cannot be done efficiently. There are also algorithms that explicitly keep track of causal dependencies, a topic that we will return to in “The “happens-before” relationship and **concurrency**”.

Một giải pháp là bảo đảm rằng bất kỳ thao tác ghi nào có liên quan nhân quả với nhau đều được ghi vào cùng một phân mảnh — nhưng trong một số ứng dụng điều đó không thể thực hiện một cách hiệu quả. Cũng có những thuật toán theo dõi tương minh các phụ thuộc nhân quả, một chủ đề mà ta sẽ quay lại ở phần “Quan hệ”xảy-ra-trước” và tính tương tranh”.

Solutions for Replication Lag — Giải pháp cho độ trễ sao chép

When working with an eventually consistent system, it is worth thinking about how the application behaves if the replication lag increases to several minutes or even hours. If the answer is “no problem,” that’s great. However, if the result is a bad experience for users, it’s important to design the system to provide a stronger guarantee, such as read-after-write. Pretending that replication is synchronous when in fact it is asynchronous is a recipe for problems down the line.

Khi làm việc với một hệ thống nhất quán cuối cùng, đáng để nghĩ về cách ứng dụng sẽ hành xử nếu độ trễ sao chép tăng lên vài phút hoặc thậm chí vài giờ. Nếu câu trả lời là “không sao cả,” thì tuyệt vời. Tuy nhiên, nếu kết quả là một trải nghiệm tồi cho người dùng, thì điều quan trọng là thiết kế hệ thống để cung cấp một bảo đảm mạnh hơn, chẳng hạn đọc-sau-ghi. Giả vờ rằng việc sao chép là đồng bộ trong khi thực ra nó là bất đồng bộ là một công thức dẫn tới rắc rối về sau.

As discussed earlier, there are ways in which an application can provide a stronger guarantee than the underlying database—for example, by performing certain kinds of reads on the leader. However, dealing with these issues in application code is complex and easy to get wrong.

Như đã bàn trước đó, có những cách để một ứng dụng cung cấp một bảo đảm mạnh hơn cơ sở dữ liệu nền tảng — ví dụ, bằng cách thực hiện một số kiểu thao tác đọc nhất định trên leader. Tuy nhiên, việc xử lý những vấn đề này trong mã ứng dụng thì phức tạp và dễ làm sai.

It would be better if application developers didn't have to worry about subtle replication issues and could just trust their databases to “do the right thing.” This is why transactions exist: they are a way for a database to provide stronger guarantees so that the application can be simpler.

Sẽ tốt hơn nếu các lập trình viên ứng dụng không phải bận tâm về những vấn đề sao chép tinh tế và có thể chỉ việc tin tưởng cơ sở dữ liệu của họ “làm điều đúng đắn.” Đây chính là lý do giao dịch tồn tại: chúng là một cách để cơ sở dữ liệu cung cấp các bảo đảm mạnh hơn, nhờ đó ứng dụng có thể đơn giản hơn.

Single-node transactions have existed for a long time. However, in the move to distributed (replicated and partitioned) databases, many systems have abandoned them, claiming that transactions are too expensive in terms of performance and availability, and asserting that eventual consistency is inevitable in a scalable system. There is some truth in that statement, but it is overly simplistic, and we will develop a more nuanced view over the course of the rest of this book. We will return to the topic of transactions in Chapters 7 and 9, and we will discuss some alternative mechanisms in Part III.

Giao dịch một-nút đã tồn tại từ lâu. Tuy nhiên, trong quá trình chuyển sang các cơ sở dữ liệu phân tán (được sao chép và phân mảnh), nhiều hệ thống đã từ bỏ chúng, viện cớ rằng giao dịch quá đắt đỏ về mặt hiệu năng và tính sẵn sàng, và khẳng định rằng nhất quán cuối cùng là điều tất yếu trong một hệ thống có khả năng mở rộng. Có một phần sự thật trong khẳng định đó, nhưng nó quá đơn giản hóa, và ta sẽ phát triển một cái nhìn nhiều sắc thái hơn xuyên suốt phần còn lại của cuốn sách này. Ta sẽ quay lại chủ đề giao dịch ở Chương 7 và 9, và ta sẽ bàn về một số cơ chế thay thế ở Phần III.

Multi-Leader Replication — Sao chép nhiều-leader

So far in this chapter we have only considered replication architectures using a single leader. Although that is a common approach, there are interesting alternatives.

Cho tới giờ trong chương này, ta mới chỉ xem xét các kiến trúc sao chép dùng một leader duy nhất. Mặc dù đó là một cách tiếp cận phổ biến, vẫn có những lựa chọn thay thế thú vị.

Leader-based replication has one major downside: there is only one leader, and all writes must go through it. If you can't connect to the leader for any reason, for example due to a network interruption between you and the leader, you can't write to the database.

Sao chép dựa trên leader có một nhược điểm lớn: chỉ có một leader, và mọi thao tác ghi đều phải đi qua nó. Nếu vì bất kỳ lý do nào bạn không thể kết nối tới leader, chẳng hạn do một gián đoạn mạng giữa bạn và leader, thì bạn không thể ghi vào cơ sở dữ liệu.

A natural extension of the leader-based replication model is to allow more than one node to accept writes. Replication still happens in the same way: each node that processes a write must forward that data change to all the other nodes. We call this a multi-leader configuration (also known as master-master or active/active replication). In this setup, each leader simultaneously acts as a follower to the other leaders.

Một sự mở rộng tự nhiên của mô hình sao chép dựa trên leader là cho phép nhiều hơn một nút chấp nhận thao tác ghi. Việc sao chép vẫn diễn ra theo cùng cách: mỗi nút xử lý một thao tác ghi phải chuyển tiếp thay đổi dữ liệu đó tới tất cả các nút khác. Ta gọi đây là một cấu hình nhiều-leader (multi-leader, còn gọi là sao chép master-master hoặc active/active). Trong cách bố trí này, mỗi leader đồng thời đóng vai một follower đối với các leader khác.

Use Cases for Multi-Leader Replication — Các tình huống dùng sao chép nhiều-leader

It rarely makes sense to use a multi-leader setup within a single datacenter, because the benefits rarely outweigh the added complexity. However, there are some situations in which this configuration is reasonable.

Hiếm khi việc dùng một cách bố trí nhiều-leader trong phạm vi một trung tâm dữ liệu là hợp lý, vì lợi ích hiếm khi xứng với độ phức tạp tăng thêm. Tuy nhiên, có một số tình huống mà cấu hình này là hợp lý.

Multi-datacenter operation — Vận hành đa-trung-tâm-dữ-liệu

Imagine you have a database with replicas in several different datacenters (perhaps so that you can tolerate failure of an entire datacenter, or perhaps in order to be closer to your users). With a normal leader-based replication setup, the leader has to be in one of the datacenters, and all writes must go through that datacenter.

Hãy hình dung bạn có một cơ sở dữ liệu với các bản sao trải trên vài trung tâm dữ liệu khác nhau (có lẽ để bạn có thể chịu được sự hỏng hóc của cả một trung tâm dữ liệu, hoặc có lẽ để gần người dùng hơn). Với một cách bố trí sao chép dựa trên leader thông thường, leader phải nằm ở một trong các trung tâm dữ liệu, và mọi thao tác ghi đều phải đi qua trung tâm dữ liệu đó.

In a multi-leader configuration, you can have a leader in each datacenter. Figure 5-6 shows what this architecture might look like. Within each datacenter, regular leader–follower replication is used; between datacenters, each datacenter’s leader replicates its changes to the leaders in other datacenters.

Trong một cấu hình nhiều-leader, bạn có thể có một leader ở mỗi trung tâm dữ liệu. Hình 5-6 cho thấy kiến trúc này có thể trông như thế nào. Bên trong mỗi trung tâm dữ liệu, sao chép leader–follower thông thường được dùng; giữa các trung tâm dữ liệu, leader của mỗi trung tâm dữ liệu sao chép các thay đổi của nó tới các leader ở các trung tâm dữ liệu khác.

Figure 5-6. Multi-leader replication across multiple datacenters. — Hình 5-6. Sao chép nhiều-leader trên nhiều trung tâm dữ liệu. Let’s compare how the single-leader and multi-leader configurations fare in a multi-datacenter **deployment**:

Hãy so sánh xem cấu hình một-leader và nhiều-leader thể hiện ra sao trong một triển khai đa-trung-tâm-dữ-liệu:

In a single-leader configuration, every write must go over the internet to the datacenter with the leader. This can add significant latency to writes and might contravene the purpose of having multiple datacenters in the first place. In a multi-leader configuration, every write can be processed in the local datacenter and is replicated asynchronously to the other datacenters. Thus, the inter-datacenter network delay is hidden from users, which means the perceived performance may be better.

Trong một cấu hình một-leader, mọi thao tác ghi đều phải đi qua internet tới trung tâm dữ liệu chứa leader. Điều này có thể thêm độ trễ đáng kể vào các thao tác ghi và có thể đi ngược lại chính mục đích của việc có nhiều trung tâm dữ liệu ngay từ đầu. Trong một cấu hình nhiều-leader, mọi thao tác ghi đều có thể được xử lý tại trung tâm dữ liệu cục bộ và được sao chép bất đồng bộ tới các trung tâm dữ liệu khác. Do đó, độ trễ mạng liên-trung-tâm-dữ-liệu được giấu khỏi người dùng, nghĩa là hiệu năng cảm nhận được có thể tốt hơn.

In a single-leader configuration, if the datacenter with the leader fails, failover can promote a follower in another datacenter to be leader. In a multi-leader configuration, each datacenter can continue operating independently of the others, and replication catches up when the failed datacenter comes back online.

Trong một cấu hình một-leader, nếu trung tâm dữ liệu chứa leader hỏng, việc chuyển đổi dự phòng có thể nâng một follower ở một trung tâm dữ liệu khác lên làm leader. Trong một cấu hình nhiều-leader, mỗi trung tâm dữ liệu có thể tiếp tục vận hành độc lập với những trung tâm khác, và việc sao chép sẽ bắt kịp khi trung tâm dữ liệu bị hỏng trở lại trực tuyến.

Traffic between datacenters usually goes over the public internet, which may be less reliable than the local network within a datacenter. A single-leader configuration is very sensitive to problems in this inter-datacenter link, because writes are made synchronously over this link. A multi-leader configuration with asynchronous replication can usually tolerate network problems better: a temporary network interruption does not prevent writes being processed.

Lưu lượng giữa các trung tâm dữ liệu thường đi qua internet công cộng, vốn có thể kém tin cậy hơn mạng cục bộ bên trong một trung tâm dữ liệu. Một cấu hình một-leader rất nhạy cảm với các vấn đề trên liên kết liên-trung-tâm-dữ-liệu này, vì các thao tác ghi được thực hiện đồng bộ qua liên kết này. Một cấu hình nhiều-leader với sao chép bất đồng bộ thường có thể chịu đựng các vấn đề mạng tốt hơn: một gián đoạn mạng tạm thời không ngăn cản việc xử lý các thao tác ghi.

Some databases support multi-leader configurations by default, but it is also often implemented with external tools, such as Tungsten Replicator for MySQL [26], BDR for PostgreSQL [27], and GoldenGate for Oracle [19].

Một số cơ sở dữ liệu hỗ trợ các cấu hình nhiều-leader theo mặc định, nhưng nó cũng thường được hiện thực bằng các công cụ bên ngoài, chẳng hạn Tungsten Replicator cho MySQL [26], BDR cho PostgreSQL [27], và GoldenGate cho Oracle [19].

Although multi-leader replication has advantages, it also has a big downside: the same data may be concurrently modified in two different datacenters, and those write conflicts must be resolved (indicated as “conflict resolution” in Figure 5-6). We will discuss this issue in “Handling Write Conflicts”.

Mặc dù sao chép nhiều-leader có những ưu điểm, nó cũng có một nhược điểm lớn: cùng một dữ liệu có thể bị sửa đổi tương tranh ở hai trung tâm dữ liệu khác nhau, và những xung đột ghi đó phải được giải quyết (được chỉ ra là “giải quyết xung đột” trong Hình 5-6). Ta sẽ bàn về vấn đề này ở phần “Xử lý xung đột ghi”.

As multi-leader replication is a somewhat retrofitted feature in many databases, there are often subtle configuration pitfalls and surprising interactions with other database features. For example, autoincrementing keys, triggers, and integrity constraints can be problematic. For this reason, multi-leader replication is often considered dangerous territory that should be avoided if possible [28].

Vì sao chép nhiều-leader là một tính năng được gắn thêm khá chắp vá trong nhiều cơ sở dữ liệu, nên thường có những cạm bẫy cấu hình tinh tế và những tương tác bất ngờ với các tính năng cơ sở dữ liệu khác. Ví dụ, khóa tự tăng, trigger, và ràng buộc toàn vẹn có thể gây rắc rối. Vì lý do đó, sao chép nhiều-leader thường được coi là vùng đất nguy hiểm cần tránh nếu có thể [28].

Clients with offline operation — Client vận hành ngoại tuyến

Another situation in which multi-leader replication is appropriate is if you have an application that needs to continue to work while it is disconnected from the internet.

Một tình huống khác mà sao chép nhiều-leader là phù hợp là khi bạn có một ứng dụng cần tiếp tục hoạt động trong khi nó bị ngắt kết nối khỏi internet.

For example, consider the calendar apps on your mobile phone, your laptop, and other devices. You need to be able to see your meetings (make read requests) and enter new meetings (make write requests) at any time, regardless of whether your device currently has an internet connection. If you make any changes while you are offline, they need to be synced with a server and your other devices when the device is next online.

Ví dụ, hãy xét các ứng dụng lịch trên điện thoại di động, máy tính xách tay, và các thiết bị khác của bạn. Bạn cần có thể xem các cuộc họp của mình (thực hiện yêu cầu đọc) và nhập các cuộc họp mới (thực hiện yêu cầu ghi) vào bất kỳ lúc nào, bất kể thiết bị của bạn hiện có kết nối internet hay không. Nếu bạn thực hiện bất kỳ thay đổi nào trong khi ngoại tuyến, chúng cần được đồng bộ với một máy chủ và các thiết bị khác của bạn vào lần kế tiếp thiết bị trực tuyến.

In this case, every device has a local database that acts as a leader (it accepts write requests), and there is an asynchronous multi-leader replication process (sync) between the replicas of your calendar on all of your devices. The replication lag may be hours or even days, depending on when you have internet access available.

Trong trường hợp này, mỗi thiết bị có một cơ sở dữ liệu cục bộ đóng vai trò một leader (nó chấp nhận yêu cầu ghi), và có một quá trình sao chép nhiều-leader bất đồng bộ (đồng bộ hóa) giữa các bản sao của lịch của bạn trên tất cả các thiết bị. Độ trễ sao chép có thể là vài giờ hoặc thậm chí vài ngày, tùy thuộc vào khi nào bạn có sẵn truy cập internet.

From an architectural point of view, this setup is essentially the same as multi-leader replication between datacenters, taken to the extreme: each device is a “datacenter,” and the network connection between them is extremely unreliable. As the rich history of broken calendar sync implementations demonstrates, multi-leader replication is a tricky thing to get right.

Xét từ góc độ kiến trúc, cách bố trí này về cơ bản giống với sao chép nhiều-leader giữa các trung tâm dữ liệu, được đẩy tới mức cực đoan: mỗi thiết bị là một “trung tâm dữ liệu,” và kết nối mạng giữa chúng cực kỳ không tin cậy. Như lịch sử phong phú của những hiện thực đồng bộ lịch hỏng hóc đã cho thấy, sao chép nhiều-leader là một thứ khó làm cho đúng.

There are tools that aim to make this kind of multi-leader configuration easier. For example, CouchDB is designed for this mode of operation [29].

Có những công cụ nhằm tới việc làm cho kiểu cấu hình nhiều-leader này dễ dàng hơn. Ví dụ, CouchDB được thiết kế cho chế độ vận hành này [29].

Collaborative editing — Soạn thảo cộng tác

Real-time collaborative editing applications allow several people to edit a **document** simultaneously. For example, Etherpad [30] and Google Docs [31] allow multiple people to concurrently edit a text document or spreadsheet (the algorithm is briefly discussed in “Automatic Conflict Resolution”).

Các ứng dụng soạn thảo cộng tác thời gian thực cho phép nhiều người cùng chỉnh sửa một tài liệu đồng thời. Ví dụ, Etherpad [30] và Google Docs [31] cho phép nhiều người chỉnh sửa tương tranh một tài liệu văn bản hoặc bảng tính (thuật toán được bàn sơ qua ở phần “Giải quyết xung đột tự động”).

We don’t usually think of collaborative editing as a database replication problem, but it has a lot in common with the previously mentioned offline editing use case. When one user edits a document, the changes are instantly applied to their local replica (the state of the document in their web browser or client application) and asynchronously replicated to the server and any other users who are editing the same document.

Ta thường không nghĩ về soạn thảo cộng tác như một bài toán sao chép cơ sở dữ liệu, nhưng nó có nhiều điểm chung với tình huống soạn thảo ngoại tuyến đã đề cập trước đó. Khi một người dùng chỉnh sửa một tài liệu, các thay đổi lập tức được áp dụng lên bản sao cục bộ của họ (trạng thái của tài liệu trong trình duyệt web hoặc ứng dụng client của họ)

và được sao chép bất đồng bộ tới máy chủ cùng bất kỳ người dùng nào khác đang chỉnh sửa cùng tài liệu đó.

If you want to guarantee that there will be no editing conflicts, the application must obtain a lock on the document before a user can edit it. If another user wants to edit the same document, they first have to wait until the first user has committed their changes and released the lock. This collaboration model is equivalent to single-leader replication with transactions on the leader.

Nếu bạn muốn bảo đảm rằng sẽ không có xung đột chỉnh sửa, ứng dụng phải lấy một khóa trên tài liệu trước khi một người dùng có thể chỉnh sửa nó. Nếu một người dùng khác muốn chỉnh sửa cùng tài liệu, họ trước hết phải chờ cho tới khi người dùng đầu tiên đã commit các thay đổi của mình và giải phóng khóa. Mô hình cộng tác này tương đương với sao chép một-leader với các giao dịch trên leader.

However, for faster collaboration, you may want to make the unit of change very small (e.g., a single keystroke) and avoid locking. This approach allows multiple users to edit simultaneously, but it also brings all the challenges of multi-leader replication, including requiring conflict resolution [32].

Tuy nhiên, để cộng tác nhanh hơn, bạn có thể muốn làm cho đơn vị thay đổi rất nhỏ (ví dụ, một lần gõ phím) và tránh khóa. Cách tiếp cận này cho phép nhiều người dùng chỉnh sửa đồng thời, nhưng nó cũng mang lại tất cả những thách thức của sao chép nhiều-leader, bao gồm việc đòi hỏi giải quyết xung đột [32].

Handling Write Conflicts — Xử lý xung đột ghi

The biggest problem with multi-leader replication is that write conflicts can occur, which means that conflict resolution is required.

Vấn đề lớn nhất với sao chép nhiều-leader là các xung đột ghi có thể xảy ra, nghĩa là cần có việc giải quyết xung đột.

For example, consider a wiki page that is simultaneously being edited by two users, as shown in Figure 5-7. User 1 changes the title of the page from A to B, and user 2 changes the title from A to C at the same time. Each user's change is successfully applied to their local leader. However, when the changes are asynchronously replicated, a conflict is detected [33]. This problem does not occur in a single-leader database.

Ví dụ, hãy xét một trang wiki đang được hai người dùng chỉnh sửa đồng thời, như trong Hình 5-7. Người dùng 1 đổi tiêu đề trang từ A sang B, và người dùng 2 đổi tiêu đề từ A sang C cùng lúc. Thay đổi của mỗi người dùng đều được áp dụng thành công lên leader cục bộ của họ. Tuy nhiên, khi các thay đổi được sao chép bất đồng bộ, một xung đột được phát hiện [33]. Vấn đề này không xảy ra trong một cơ sở dữ liệu một-leader.

Figure 5-7. A write conflict caused by two leaders concurrently updating the same record. — Hình 5-7. Một xung đột ghi gây ra bởi hai leader cùng cập nhật tương tranh một bản ghi.

Synchronous versus asynchronous conflict detection — Phát hiện xung đột đồng bộ so với bất đồng bộ

In a single-leader database, the second writer will either block and wait for the first write to complete, or abort the second write transaction, forcing the user to retry the write. On the other hand, in a multi-leader setup, both writes are successful, and the conflict is only detected asynchronously at some later point in time. At that time, it may be too late to ask the user to resolve the conflict.

Trong một cơ sở dữ liệu một-leader, người ghi thứ hai sẽ hoặc bị chặn và chờ thao tác ghi đầu tiên hoàn tất, hoặc hủy bỏ giao dịch ghi thứ hai, buộc người dùng phải thử ghi lại. Mặt khác, trong một cách bố trí nhiều-leader, cả hai thao tác ghi đều thành công, và xung đột chỉ được phát hiện một cách bất đồng bộ tại một thời điểm nào đó về sau. Vào thời điểm ấy, có thể đã quá muộn để yêu cầu người dùng giải quyết xung đột.

In principle, you could make the conflict detection synchronous—i.e., wait for the write to be replicated to all replicas before telling the user that the write was successful. However, by doing so, you would lose the main advantage of multi-leader replication: allowing each replica to accept writes independently. If you want synchronous conflict detection, you might as well just use single-leader replication.

Về nguyên tắc, bạn có thể làm cho việc phát hiện xung đột trở nên đồng bộ — tức là chờ thao tác ghi được sao chép tới tất cả các bản sao trước khi báo cho người dùng rằng thao tác ghi đã thành công. Tuy nhiên, làm như vậy, bạn sẽ đánh mất ưu điểm chính của sao chép nhiều-leader: cho phép mỗi bản sao chấp nhận thao tác ghi một cách độc lập. Nếu bạn muốn phát hiện xung đột đồng bộ, thì cứ dùng sao chép một-leader cho rồi.

Conflict avoidance — Né tránh xung đột

The simplest strategy for dealing with conflicts is to avoid them: if the application can ensure that all writes for a particular record go through the same leader, then conflicts cannot occur. Since many implementations of multi-leader replication handle conflicts quite poorly, avoiding conflicts is a frequently recommended approach [34].

Chiến lược đơn giản nhất để xử lý xung đột là né tránh chúng: nếu ứng dụng có thể bảo đảm rằng mọi thao tác ghi cho một bản ghi cụ thể đều đi qua cùng một leader, thì xung đột không thể xảy ra. Vì nhiều hiện thực của sao chép nhiều-leader xử lý xung đột khá kém, nên né tránh xung đột là một cách tiếp cận thường được khuyến nghị [34].

For example, in an application where a user can edit their own data, you can ensure that requests from a particular user are always routed to the same datacenter and use the leader in that datacenter for reading and writing. Different users may have different “home” datacenters (perhaps picked based on geographic proximity to the user), but from any one user’s point of view the configuration is essentially single-leader.

Ví dụ, trong một ứng dụng nơi người dùng có thể chỉnh sửa dữ liệu của chính mình, bạn có thể bảo đảm rằng các yêu cầu từ một người dùng cụ thể luôn được định tuyến tới cùng một trung tâm dữ liệu và dùng leader ở trung tâm dữ liệu đó để đọc và ghi. Những người dùng khác nhau có thể có các trung tâm dữ liệu “nhà” khác nhau (có lẽ được chọn dựa trên sự gần gũi địa lý với người dùng), nhưng xét từ góc nhìn của bất kỳ một người dùng nào, cấu hình về cơ bản là một-leader.

However, sometimes you might want to change the designated leader for a record—perhaps because one datacenter has failed and you need to reroute traffic to another datacenter, or perhaps because a user has moved to a different location and is now closer to a different datacenter. In this situation, conflict avoidance breaks down, and you have to deal with the possibility of concurrent writes on different leaders.

Tuy nhiên, đôi khi bạn có thể muốn thay đổi leader được chỉ định cho một bản ghi — có lẽ vì một trung tâm dữ liệu đã hỏng và bạn cần định tuyến lại lưu lượng tới một trung tâm dữ liệu khác, hoặc có lẽ vì một người dùng đã chuyển tới một vị trí khác và giờ đây gần một trung tâm dữ liệu khác hơn. Trong tình huống này, việc né tránh xung đột đổ vỡ, và bạn phải xử lý khả năng có các thao tác ghi tương tranh trên các leader khác nhau.

Converging toward a consistent state — Hội tụ về một trạng thái nhất quán

A single-leader database applies writes in a sequential order: if there are several updates to the same field, the last write determines the final value of the field.

Một cơ sở dữ liệu một-leader áp dụng các thao tác ghi theo một thứ tự tuần tự: nếu có vài cập nhật lên cùng một trường, thao tác ghi cuối cùng quyết định giá trị sau cùng của trường đó.

In a multi-leader configuration, there is no defined ordering of writes, so it's not clear what the final value should be. In Figure 5-7, at leader 1 the title is first updated to B and then to C; at leader 2 it is first updated to C and then to B. Neither order is “more correct” than the other.

Trong một cấu hình nhiều-leader, không có thứ tự ghi nào được định nghĩa, nên không rõ giá trị sau cùng nên là gì. Trong Hình 5-7, tại leader 1 tiêu đề trước hết được cập nhật thành B rồi thành C; tại leader 2 nó trước hết được cập nhật thành C rồi thành B. Không thứ tự nào “đúng hơn” thứ tự kia.

If each replica simply applied writes in the order that it saw the writes, the database would end up in an inconsistent state: the final value would be C at leader 1 and B at leader 2. That is not acceptable—every replication scheme must ensure that the data is eventually the same in all replicas. Thus, the database must resolve the conflict in a convergent way, which means that all replicas must arrive at the same final value when all changes have been replicated.

Nếu mỗi bản sao chỉ đơn giản áp dụng các thao tác ghi theo thứ tự mà nó thấy chúng, cơ sở dữ liệu sẽ rơi vào trạng thái bất nhất: giá trị sau cùng sẽ là C tại leader 1 và B tại leader 2. Điều đó không thể chấp nhận — mọi cơ chế sao chép đều phải bảo đảm rằng dữ liệu rốt cuộc là như nhau trên tất cả các bản sao. Do đó, cơ sở dữ liệu phải giải quyết xung đột theo một cách hội tụ, nghĩa là tất cả các bản sao phải đi đến cùng một giá trị sau cùng khi mọi thay đổi đã được sao chép.

There are various ways of achieving convergent conflict resolution:

Có nhiều cách để đạt được việc giải quyết xung đột hội tụ:

- Give each write a unique ID (e.g., a timestamp, a long random number, a UUID, or a hash of the key and value), pick the write with the highest ID as the winner, and throw away the other writes. If a timestamp is used, this technique is known as last write wins (LWW). Although this approach is popular, it is dangerously prone to data loss [35]. We will discuss LWW in more detail at the end of this chapter (“Detecting Concurrent Writes”).
- Give each replica a unique ID, and let writes that originated at a higher-numbered replica always take precedence over writes that originated at a lower-numbered replica. This approach also implies data loss.
- Somehow merge the values together—e.g., order them alphabetically and then concatenate them (in Figure 5-7, the merged title might be something like “B/C”).
- Record the conflict in an explicit data structure that preserves all information, and write application code that resolves the conflict at some later time (perhaps by prompting the user).
 - Gán cho mỗi thao tác ghi một ID duy nhất (ví dụ, một dấu thời gian, một số ngẫu nhiên dài, một UUID, hoặc một giá trị băm của khóa và giá trị), chọn thao tác ghi có ID cao nhất làm bên thắng, và vứt bỏ các thao tác ghi còn lại. Nếu dùng một dấu thời gian, kỹ thuật này được gọi là người-ghi-sau-thắng (last write wins, LWW). Mặc dù cách tiếp cận này phổ biến, nó dễ dẫn đến mất dữ liệu một cách nguy hiểm [35]. Ta sẽ bàn về LWW chi tiết hơn ở cuối chương này (phần “Phát hiện các thao tác ghi tương tranh”).

- Gán cho mỗi bản sao một ID duy nhất, và để các thao tác ghi bắt nguồn từ một bản sao có số hiệu cao hơn luôn thắng các thao tác ghi bắt nguồn từ một bản sao có số hiệu thấp hơn. Cách tiếp cận này cũng hàm ý mất dữ liệu.
- Bằng cách nào đó trộn các giá trị lại với nhau — ví dụ, sắp xếp chúng theo thứ tự bảng chữ cái rồi nối lại (trong Hình 5-7, tiêu đề được trộn có thể là một thứ gì đó như “B/C”).
- Ghi lại xung đột trong một cấu trúc dữ liệu tường minh giữ lại mọi thông tin, và viết mã ứng dụng để giải quyết xung đột tại một thời điểm nào đó về sau (có lẽ bằng cách hỏi người dùng).

Custom conflict resolution logic — Logic giải quyết xung đột tùy biến

As the most appropriate way of resolving a conflict may depend on the application, most multi-leader replication tools let you write conflict resolution logic using application code. That code may be executed on write or on read:

Vì cách giải quyết một xung đột phù hợp nhất có thể phụ thuộc vào ứng dụng, hầu hết các công cụ sao chép nhiều-leader cho phép bạn viết logic giải quyết xung đột bằng mã ứng dụng. Mã đó có thể được thực thi lúc ghi hoặc lúc đọc:

As soon as the database system detects a conflict in the log of replicated changes, it calls the conflict handler. For example, Bucardo allows you to write a snippet of Perl for this purpose. This handler typically cannot prompt a user—it runs in a background process and it must execute quickly.

Ngay khi hệ cơ sở dữ liệu phát hiện một xung đột trong nhật ký các thay đổi được sao chép, nó gọi bộ xử lý xung đột (conflict handler). Ví dụ, Bucardo cho phép bạn viết một đoạn mã Perl cho mục đích này. Bộ xử lý này thường không thể hỏi người dùng — nó chạy trong một tiến trình nền và nó phải thực thi nhanh.

When a conflict is detected, all the conflicting writes are stored. The next time the data is read, these multiple versions of the data are returned to the application. The application may prompt the user or automatically resolve the conflict, and write the result back to the database. CouchDB works this way, for example.

Khi một xung đột được phát hiện, tất cả các thao tác ghi xung đột được lưu lại. Lần kế tiếp dữ liệu được đọc, nhiều phiên bản này của dữ liệu được trả về cho ứng dụng. Ứng dụng có thể hỏi người dùng hoặc tự động giải quyết xung đột, và ghi kết quả ngược lại vào cơ sở dữ liệu. Ví dụ, CouchDB hoạt động theo cách này.

Note that conflict resolution usually applies at the level of an individual row or document, not for an entire transaction [36]. Thus, if you have a transaction that atomically makes several different writes (see Chapter 7), each write is still considered separately for the purposes of conflict resolution.

Lưu ý rằng việc giải quyết xung đột thường áp dụng ở cấp độ một hàng hoặc một tài liệu riêng lẻ, chứ không phải cho cả một giao dịch [36]. Do đó, nếu bạn có một giao dịch thực hiện một cách nguyên tử vài thao tác ghi khác nhau (xem Chương 7), mỗi thao tác ghi vẫn được xét riêng cho mục đích giải quyết xung đột.

Automatic Conflict Resolution — Giải quyết xung đột tự động Conflict resolution rules can quickly become complicated, and custom code can be error-prone. Amazon is a frequently cited example of surprising effects due to a conflict resolution handler: for some time, the conflict resolution logic on the shopping cart would preserve items added to the cart, but not items removed from the cart. Thus, customers would sometimes see items reappearing in their carts even though they had previously been removed [37].

Các quy tắc giải quyết xung đột có thể nhanh chóng trở nên phức tạp, và mã tùy biến có thể dễ phát sinh lỗi. Amazon là một ví dụ thường được trích dẫn về những hiệu ứng bất ngờ do một bộ xử lý giải quyết xung đột: trong một thời gian, logic giải quyết xung đột trên giỏ hàng sẽ giữ lại các món được thêm vào giỏ, nhưng không giữ lại các món bị bỏ khỏi giỏ. Do đó, khách hàng đôi khi sẽ thấy các món tái xuất hiện trong giỏ của họ dù trước đó chúng đã bị bỏ đi [37].

There has been some interesting research into automatically resolving conflicts caused by concurrent data modifications. A few lines of research are worth mentioning:

Đã có một số nghiên cứu thú vị về việc tự động giải quyết các xung đột gây ra bởi những sửa đổi dữ liệu tương tranh. Một vài hướng nghiên cứu đáng nhắc tới:

- Conflict-free replicated datatypes (CRDTs) [32, 38] are a family of data structures for sets, maps, ordered lists, counters, etc. that can be concurrently edited by multiple users, and which automatically resolve conflicts in sensible ways. Some CRDTs have been implemented in Riak 2.0 [39, 40].
- Mergeable persistent data structures [41] track history explicitly, similarly to the Git version control system, and use a three-way merge function (whereas CRDTs use two-way merges).
- Operational transformation [42] is the conflict resolution algorithm behind collaborative editing applications such as Etherpad [30] and Google Docs [31]. It was designed particularly for concurrent editing of an ordered list of items, such as the list of characters that constitute a text document.
 - Kiểu dữ liệu được sao chép không-xung-đột (conflict-free replicated datatypes, CRDT) [32, 38] là một họ các cấu trúc dữ liệu cho tập hợp, ánh xạ, danh sách có thứ tự, bộ đếm, v.v. có thể được nhiều người dùng chỉnh sửa tương tranh, và tự động giải quyết xung đột theo những cách hợp lý. Một số CRDT đã được hiện thực trong Riak 2.0 [39, 40].
 - Cấu trúc dữ liệu bền vững có thể trộn được (mergeable persistent data structures) [41] theo dõi lịch sử một cách tường minh, tương tự như hệ thống quản lý phiên bản Git, và dùng một hàm trộn ba-chiều (trong khi CRDT dùng phép trộn hai-chiều).
 - Biến đổi thao tác (operational transformation) [42] là thuật toán giải quyết xung đột đằng sau các ứng dụng soạn thảo cộng tác như Etherpad [30] và Google Docs [31]. Nó được thiết kế đặc biệt cho việc chỉnh sửa tương tranh một danh sách các mục có thứ tự, chẳng hạn danh sách các ký tự cấu thành một tài liệu văn bản.

Implementations of these algorithms in databases are still young, but it's likely that they will be integrated into more replicated data systems in the future. Automatic conflict resolution could make multi-leader data synchronization much simpler for applications to deal with.

Các hiện thực của những thuật toán này trong cơ sở dữ liệu vẫn còn non trẻ, nhưng nhiều khả năng chúng sẽ được tích hợp vào nhiều hệ thống dữ liệu được sao chép hơn trong tương lai. Việc giải quyết xung đột tự động có thể làm cho việc đồng bộ dữ liệu nhiều-leader đơn giản hơn nhiều để các ứng dụng xử lý.

What is a conflict? — Xung đột là gì?

Some kinds of conflict are obvious. In the example in Figure 5-7, two writers concurrently modified the same field in the same record, setting it to two different values. There is little doubt that this is a conflict.

Một số kiểu xung đột thì hiển nhiên. Trong ví dụ ở Hình 5-7, hai thao tác ghi cùng sửa đổi tương tranh một trường trong cùng một bản ghi, đặt nó thành hai giá trị khác nhau. Khó

mà nghi ngờ rằng đây là một xung đột.

Other kinds of conflict can be more subtle to detect. For example, consider a meeting room booking system: it tracks which room is booked by which group of people at which time. This application needs to ensure that each room is only booked by one group of people at any one time (i.e., there must not be any overlapping bookings for the same room). In this case, a conflict may arise if two different bookings are created for the same room at the same time. Even if the application checks availability before allowing a user to make a booking, there can be a conflict if the two bookings are made on two different leaders.

Các kiểu xung đột khác có thể tinh tế hơn để phát hiện. Ví dụ, hãy xét một hệ thống đặt phòng họp: nó theo dõi phòng nào được nhóm người nào đặt vào thời gian nào. Ứng dụng này cần bảo đảm rằng mỗi phòng chỉ được một nhóm người đặt tại bất kỳ một thời điểm nào (tức là không được có bất kỳ lượt đặt nào chồng lấn cho cùng một phòng). Trong trường hợp này, một xung đột có thể nảy sinh nếu hai lượt đặt khác nhau được tạo cho cùng một phòng vào cùng một thời gian. Ngay cả khi ứng dụng kiểm tra tình trạng còn trống trước khi cho phép một người dùng đặt phòng, vẫn có thể có một xung đột nếu hai lượt đặt được thực hiện trên hai leader khác nhau.

There isn't a quick ready-made answer, but in the following chapters we will trace a path toward a good understanding of this problem. We will see some more examples of conflicts in Chapter 7, and in Chapter 12 we will discuss scalable approaches for detecting and resolving conflicts in a replicated system.

Không có một câu trả lời làm sẵn chớp nhoáng, nhưng trong các chương tiếp theo ta sẽ vạch ra một con đường hướng tới sự hiểu biết thấu đáo về vấn đề này. Ta sẽ thấy thêm vài ví dụ về xung đột ở Chương 7, và ở Chương 12 ta sẽ bàn về những cách tiếp cận có khả năng mở rộng để phát hiện và giải quyết xung đột trong một hệ thống được sao chép.

Multi-Leader Replication Topologies — Các tô-pô sao chép nhiều-leader

A replication topology describes the communication paths along which writes are propagated from one node to another. If you have two leaders, like in Figure 5-7, there is only one plausible topology: leader 1 must send all of its writes to leader 2, and vice versa. With more than two leaders, various different topologies are possible. Some examples are illustrated in Figure 5-8.

Một tô-pô sao chép (replication topology) mô tả các đường giao tiếp dọc theo đó các thao tác ghi được lan truyền từ nút này sang nút khác. Nếu bạn có hai leader, như trong Hình 5-7, chỉ có một tô-pô hợp lý: leader 1 phải gửi tất cả thao tác ghi của nó tới leader 2, và ngược lại. Với nhiều hơn hai leader, nhiều tô-pô khác nhau là khả dĩ. Một số ví dụ được minh họa ở Hình 5-8.

Figure 5-8. Three example topologies in which multi-leader replication can be set up. — Hình 5-8. Ba ví dụ tô-pô mà sao chép nhiều-leader có thể được thiết lập theo. The most general topology is all-to-all (Figure 5-8 [c]), in which every leader sends its writes to every other leader. However, more restricted topologies are also used: for example, MySQL by default supports only a circular topology [34], in which each node receives writes from one node and forwards those writes (plus any writes of its own) to one other node. Another popular topology has the shape of a star: one designated root node forwards writes to all of the other nodes. The star topology can be generalized to a tree.

Tô-pô tổng quát nhất là toàn-bộ-tới-toàn-bộ (all-to-all, Hình 5-8 [c]), trong đó mỗi leader gửi các thao tác ghi của nó tới mọi leader khác. Tuy nhiên, các tô-pô hạn chế hơn cũng được dùng: ví dụ, theo mặc định MySQL chỉ hỗ trợ một tô-pô vòng tròn [34], trong đó mỗi nút nhận thao tác ghi từ một nút và chuyển tiếp các thao tác ghi đó (cùng với bất kỳ thao tác ghi nào của chính nó) tới một nút khác. Một tô-pô phổ biến khác có hình dạng một ngôi sao: một nút gốc được chỉ định sẽ chuyển tiếp các thao tác ghi tới tất cả các nút khác. Tô-pô ngôi sao có thể được tổng quát hóa thành một cây.

In circular and star topologies, a write may need to pass through several nodes before it reaches all replicas. Therefore, nodes need to forward data changes they receive from other nodes. To prevent infinite replication loops, each node is given a unique identifier, and in the replication log, each write is tagged with the identifiers of all the nodes it has passed through [43]. When a node receives a data change that is tagged with its own identifier, that data change is ignored, because the node knows that it has already been processed.

Trong các tô-pô vòng tròn và ngôi sao, một thao tác ghi có thể cần đi qua vài nút trước khi nó đến được tất cả các bản sao. Do đó, các nút cần chuyển tiếp các thay đổi dữ liệu mà chúng nhận được từ các nút khác. Để ngăn các vòng lặp sao chép vô hạn, mỗi nút được gán một định danh duy nhất, và trong nhật ký sao chép, mỗi thao tác ghi được gắn thẻ với các định danh của tất cả các nút mà nó đã đi qua [43]. Khi một nút nhận một thay đổi dữ liệu được gắn thẻ với chính định danh của nó, thay đổi dữ liệu đó bị bỏ qua, vì nút biết rằng nó đã được xử lý rồi.

A problem with circular and star topologies is that if just one node fails, it can interrupt the flow of replication messages between other nodes, causing them to be unable to communicate until the node is fixed. The topology could be reconfigured to work around the failed node, but in most deployments such reconfiguration would have to be done manually. The fault tolerance of a more densely connected topology (such as all-to-all) is better because it allows messages to travel along different paths, avoiding a single point of failure.

Một vấn đề với các tô-pô vòng tròn và ngôi sao là nếu chỉ một nút hỏng, nó có thể làm gián đoạn dòng các thông điệp sao chép giữa các nút khác, khiến chúng không thể giao tiếp cho tới khi nút đó được sửa. Tô-pô có thể được cấu hình lại để né tránh nút bị hỏng, nhưng trong hầu hết các triển khai việc cấu hình lại như vậy sẽ phải được thực hiện thủ công. Khả năng chịu lỗi của một tô-pô được kết nối dày đặc hơn (chẳng hạn toàn-bộ-tới-toàn-bộ) thì tốt hơn vì nó cho phép các thông điệp đi dọc theo những đường khác nhau, né tránh một điểm hỏng duy nhất.

On the other hand, all-to-all topologies can have issues too. In particular, some network links may be faster than others (e.g., due to network congestion), with the result that some replication messages may “overtake” others, as illustrated in Figure 5-9.

Mặt khác, các tô-pô toàn-bộ-tới-toàn-bộ cũng có thể gặp vấn đề. Cụ thể, một số liên kết mạng có thể nhanh hơn những liên kết khác (ví dụ, do tắc nghẽn mạng), với hệ quả là một số thông điệp sao chép có thể “vượt mặt” những thông điệp khác, như minh họa ở Hình 5-9.

Figure 5-9. With multi-leader replication, writes may arrive in the wrong order at some replicas. — Hình 5-9. Với sao chép nhiều-leader, các thao tác ghi có thể đến sai thứ tự ở một số bản sao. In Figure 5-9, client A inserts a row into a table on leader 1, and client B updates that row on leader 3. However, leader 2 may receive the writes in a different order: it may first receive the update (which, from its point of view, is an update to a row that does not exist in the database) and only later receive the corresponding insert (which should have preceded the update).

Trong Hình 5-9, client A chèn một hàng vào một bảng trên leader 1, và client B cập nhật hàng đó trên leader 3. Tuy nhiên, leader 2 có thể nhận các thao tác ghi theo một thứ tự khác: nó có thể trước hết nhận cập nhật (vốn, xét từ góc nhìn của nó, là một cập nhật lên một hàng không tồn tại trong cơ sở dữ liệu) và chỉ sau đó mới nhận phép chèn tương ứng (vốn lẽ ra phải đứng trước cập nhật).

This is a problem of causality, similar to the one we saw in “Consistent Prefix Reads”: the update depends on the prior insert, so we need to make sure that all nodes process the insert first, and then the update. Simply attaching a timestamp to every write is not sufficient, because clocks cannot be trusted to be sufficiently in sync to correctly order these events at leader 2 (see Chapter 8).

Đây là một vấn đề về tính nhân quả, tương tự vấn đề ta đã thấy ở phần “Đọc tiền tố nhất quán”: cập nhật phụ thuộc vào phép chèn trước đó, nên ta cần bảo đảm rằng tất cả các nút xử lý phép chèn trước, rồi mới tới cập nhật. Chỉ đơn giản gắn một dấu thời gian vào mỗi thao tác ghi là không đủ, vì không thể tin cậy rằng các đồng hồ đồng bộ đủ tốt để sắp xếp đúng thứ tự các sự kiện này tại leader 2 (xem Chương 8).

To order these events correctly, a technique called version vectors can be used, which we will discuss later in this chapter (see “Detecting Concurrent Writes”). However, conflict detection techniques are poorly implemented in many multi-leader replication systems. For example, at the time of writing, PostgreSQL BDR does not provide causal ordering of writes [27], and Tungsten Replicator for MySQL doesn’t even try to detect conflicts [34].

Để sắp xếp đúng thứ tự các sự kiện này, một kỹ thuật gọi là vector phiên bản (version vectors) có thể được dùng, mà ta sẽ bàn ở phần sau của chương này (xem “Phát hiện các thao tác ghi tương tranh”). Tuy nhiên, các kỹ thuật phát hiện xung đột được hiện thực kém trong nhiều hệ thống sao chép nhiều-leader. Ví dụ, tại thời điểm viết, PostgreSQL BDR không cung cấp việc sắp xếp nhân quả các thao tác ghi [27], và Tungsten Replicator cho MySQL thậm chí còn không buồn cố phát hiện xung đột [34].

If you are using a system with multi-leader replication, it is worth being aware of these issues, carefully reading the documentation, and thoroughly testing your database to ensure that it really does provide the guarantees you believe it to have.

Nếu bạn đang dùng một hệ thống có sao chép nhiều-leader, thì đáng để nhận thức về những vấn đề này, đọc tài liệu một cách cẩn thận, và kiểm thử thấu đáo cơ sở dữ liệu của bạn để bảo đảm rằng nó thực sự cung cấp những bảo đảm mà bạn tin rằng nó có.

Leaderless Replication — Sao chép không-leader

The replication approaches we have discussed so far in this chapter—single-leader and multi-leader replication—are based on the idea that a client sends a write request to one node (the leader), and the database system takes care of copying that write to the other replicas. A leader determines the order in which writes should be processed, and followers apply the leader’s writes in the same order.

Các cách tiếp cận sao chép mà ta đã bàn từ nãy tới giờ trong chương này — sao chép một-leader và nhiều-leader — dựa trên ý tưởng rằng một client gửi một yêu cầu ghi tới một nút (leader), và hệ cơ sở dữ liệu lo việc sao chép thao tác ghi đó tới các bản sao khác. Một leader quyết định thứ tự mà các thao tác ghi nên được xử lý, và các follower áp dụng các thao tác ghi của leader theo cùng thứ tự đó.

Some data storage systems take a different approach, abandoning the concept of a leader and allowing any replica to directly accept writes from clients. Some of the earliest replicated data systems were leaderless [1, 44], but the idea was mostly forgotten during the era of dominance of relational databases. It once again became a fashionable architecture for databases after Amazon used it for its in-house Dynamo system [37]. Riak, Cassandra, and Voldemort are open source datastores with leaderless replication models inspired by Dynamo, so this kind of database is also known as Dynamo-style.

Một số hệ thống lưu trữ dữ liệu áp dụng một cách tiếp cận khác, từ bỏ khái niệm leader và cho phép bất kỳ bản sao nào trực tiếp chấp nhận thao tác ghi từ client. Một số hệ thống dữ liệu được sao chép sớm nhất là không-leader [1, 44], nhưng ý tưởng này phần lớn bị lãng quên trong kỷ nguyên thống trị của cơ sở dữ liệu quan hệ. Nó lại một lần nữa trở thành một kiến trúc thời thượng cho cơ sở dữ liệu sau khi Amazon dùng nó cho hệ thống Dynamo nội bộ của mình [37]. Riak, Cassandra, và Voldemort là các kho dữ liệu mã nguồn mở với các mô hình sao chép không-leader lấy cảm hứng từ Dynamo, nên loại cơ sở dữ liệu này còn được gọi là kiểu-Dynamo.

In some leaderless implementations, the client directly sends its writes to several replicas, while in others, a coordinator node does this on behalf of the client. However, unlike a leader database, that coordinator does not enforce a particular ordering of writes. As we shall see, this difference in design has profound consequences for the way the database is used.

Trong một số hiện thực không-leader, client trực tiếp gửi các thao tác ghi của nó tới vài bản sao, trong khi ở những hiện thực khác, một nút điều phối (coordinator) làm việc này thay cho client. Tuy nhiên, khác với một cơ sở dữ liệu leader, nút điều phối đó không áp đặt một thứ tự ghi cụ thể nào. Như ta sẽ thấy, sự khác biệt trong thiết kế này có những hệ quả sâu sắc tới cách cơ sở dữ liệu được sử dụng.

Writing to the Database When a Node Is Down — Ghi vào cơ sở dữ liệu khi một nút đang ngừng hoạt động

Imagine you have a database with three replicas, and one of the replicas is currently unavailable—perhaps it is being rebooted to install a system update. In a leader-based configuration, if you want to continue processing writes, you may need to perform a failover (see “Handling Node Outages”).

Hãy hình dung bạn có một cơ sở dữ liệu với ba bản sao, và một trong các bản sao hiện không sẵn sàng — có lẽ nó đang được khởi động lại để cài một bản cập nhật hệ thống. Trong một cấu hình dựa trên leader, nếu bạn muốn tiếp tục xử lý các thao tác ghi, bạn có thể cần thực hiện một lần chuyển đổi dự phòng (xem “Xử lý gián đoạn nút”).

On the other hand, in a leaderless configuration, failover does not exist. Figure 5-10 shows what happens: the client (user 1234) sends the write to all three replicas in parallel, and the two available replicas accept the write but the unavailable replica misses it. Let’s say that it’s sufficient for two out of three replicas to acknowledge the write: after user 1234 has received two ok responses, we consider the write to be successful. The client simply ignores the fact that one of the replicas missed the write.

Mặt khác, trong một cấu hình không-leader, chuyển đổi dự phòng không tồn tại. Hình 5-10 cho thấy những gì xảy ra: client (người dùng 1234) gửi thao tác ghi tới cả ba bản sao song song, và hai bản sao sẵn sàng chấp nhận thao tác ghi nhưng bản sao không sẵn sàng bỏ lỡ nó. Giả sử rằng hai trong ba bản sao xác nhận thao tác ghi là đủ: sau khi người dùng 1234 nhận được hai phản hồi ok, ta coi thao tác ghi là thành công. Client chỉ đơn giản phớt lờ việc một trong các bản sao đã bỏ lỡ thao tác ghi.

Figure 5-10. A quorum write, quorum read, and read repair after a node outage. — Hình 5-10. Một thao tác ghi theo nhóm tức số, đọc theo nhóm tức số, và sửa chữa lúc đọc sau một gián đoạn nút. Now imagine that the unavailable node comes back online, and clients start reading from it. Any writes that happened while the node was down are missing from that node. Thus, if you read from that node, you may get stale (outdated) values as responses.

Giờ hãy hình dung nút không sẵn sàng đó trở lại trực tuyến, và các client bắt đầu đọc từ nó. Mọi thao tác ghi đã xảy ra trong khi nút đó ngừng hoạt động đều bị thiếu khỏi nút đó. Do đó, nếu bạn đọc từ nút đó, bạn có thể nhận được các giá trị lỗi thời (cũ) làm phản hồi.

To solve that problem, when a client reads from the database, it doesn’t just send its request to one replica: read requests are also sent to several nodes in parallel. The client may get different responses from different nodes; i.e., the up-to-date value from one node and a stale value from another. Version numbers are used to determine which value is newer (see “Detecting Concurrent Writes”).

Để giải quyết vấn đề đó, khi một client đọc từ cơ sở dữ liệu, nó không chỉ gửi yêu cầu của mình tới một bản sao: các yêu cầu đọc cũng được gửi tới vài nút song song. Client có thể nhận các phản hồi khác nhau từ các nút khác nhau; tức là giá trị cập nhật từ một nút và một giá trị lỗi thời từ một nút khác. Các số phiên bản được dùng để xác định giá trị nào mới hơn (xem “Phát hiện các thao tác ghi tương tranh”).

Read repair and anti-entropy — Sửa chữa lúc đọc và chống-entropy

The replication scheme should ensure that eventually all the data is copied to every replica. After an unavailable node comes back online, how does it catch up on the writes that it missed?

Cơ chế sao chép cần bảo đảm rằng rốt cuộc toàn bộ dữ liệu đều được sao chép tới mọi bản sao. Sau khi một nút không sẵn sàng trở lại trực tuyến, làm sao nó bắt kịp các thao tác ghi mà nó đã bỏ lỡ?

Two mechanisms are often used in Dynamo-style datastores:

Hai cơ chế thường được dùng trong các kho dữ liệu kiểu-Dynamo:

When a client makes a read from several nodes in parallel, it can detect any stale responses. For example, in Figure 5-10, user 2345 gets a version 6 value from replica 3 and a version 7 value from replicas 1 and 2. The client sees that replica 3 has a stale value and writes the newer value back to that replica. This approach works well for values that are frequently read.

Khi một client thực hiện một thao tác đọc từ vài nút song song, nó có thể phát hiện bất kỳ phản hồi lỗi thời nào. Ví dụ, trong Hình 5-10, người dùng 2345 nhận được một giá trị phiên bản 6 từ bản sao 3 và một giá trị phiên bản 7 từ các bản sao 1 và 2. Client thấy rằng bản sao 3 có một giá trị lỗi thời và ghi giá trị mới hơn ngược lại bản sao đó. Cách tiếp cận này hoạt động tốt với các giá trị thường được đọc.

In addition, some datastores have a background process that constantly looks for differences in the data between replicas and copies any missing data from one replica to another. Unlike the replication log in leader-based replication, this anti-entropy process does not copy writes in any particular order, and there may be a significant delay before data is copied.

Ngoài ra, một số kho dữ liệu có một tiến trình nền liên tục tìm kiếm những khác biệt trong dữ liệu giữa các bản sao và sao bất kỳ dữ liệu nào còn thiếu từ bản sao này sang bản sao khác. Khác với nhật ký sao chép trong sao chép dựa trên leader, tiến trình chống-entropy (anti-entropy) này không sao chép các thao tác ghi theo bất kỳ thứ tự cụ thể nào, và có thể có một độ trễ đáng kể trước khi dữ liệu được sao chép.

Not all systems implement both of these; for example, Voldemort currently does not have an anti-entropy process. Note that without an anti-entropy process, values that are rarely read may be missing from some replicas and thus have reduced durability, because read repair is only performed when a value is read by the application.

Không phải mọi hệ thống đều hiện thực cả hai cơ chế này; ví dụ, Voldemort hiện không có tiến trình chống-entropy. Lưu ý rằng nếu không có tiến trình chống-entropy, các giá trị hiếm khi được đọc có thể bị thiếu khỏi một số bản sao và do đó có độ bền giảm đi, vì việc sửa chữa lúc đọc chỉ được thực hiện khi một giá trị được ứng dụng đọc.

Quorums for reading and writing — Nhóm túc số để đọc và ghi

In the example of Figure 5-10, we considered the write to be successful even though it was only processed on two out of three replicas. What if only one out of three replicas accepted the write? How far can we push this?

Trong ví dụ ở Hình 5-10, ta đã coi thao tác ghi là thành công dù nó chỉ được xử lý trên hai trong ba bản sao. Nếu chỉ một trong ba bản sao chấp nhận thao tác ghi thì sao? Ta có thể đẩy điều này đi xa tới đâu?

If we know that every successful write is guaranteed to be present on at least two out of three replicas, that means at most one replica can be stale. Thus, if we read from at least two replicas, we can be sure that at least one of the two is up to date. If the third replica is down or slow to respond, reads can nevertheless continue returning an up-to-date value.

Nếu ta biết rằng mọi thao tác ghi thành công đều được bảo đảm có mặt trên ít nhất hai trong ba bản sao, điều đó nghĩa là nhiều nhất một bản sao có thể lỗi thời. Do đó, nếu ta đọc từ ít nhất hai bản sao, ta có thể chắc chắn rằng ít nhất một trong hai bản sao đó là cập nhật. Nếu bản sao thứ ba ngừng hoạt động hoặc phản hồi chậm, các thao tác đọc đầu vậy vẫn có thể tiếp tục trả về một giá trị cập nhật.

More generally, if there are n replicas, every write must be confirmed by w nodes to be considered successful, and we must query at least r nodes for each read. (In our example, $n = 3$, $w = 2$, $r = 2$.) As long as $w + r > n$, we expect to get an up-to-date value when reading, because at least one of the r nodes we're reading from must be up to date. Reads and writes that obey these r and w values are called quorum reads and writes [44]. You can think of r and w as the minimum number of votes required for the read or write to be valid.

Tổng quát hơn, nếu có n bản sao, mỗi thao tác ghi phải được w nút xác nhận để được coi là thành công, và ta phải truy vấn ít nhất r nút cho mỗi thao tác đọc. (Trong ví dụ của ta, $n = 3$, $w = 2$, $r = 2$.) Miễn là $w + r > n$, ta kỳ vọng nhận được một giá trị cập nhật khi đọc, vì ít nhất một trong r nút mà ta đang đọc từ đó phải là cập nhật. Các thao tác đọc và ghi tuân theo những giá trị r và w này được gọi là đọc và ghi theo nhóm tấc số (quorum reads and writes) [44]. Bạn có thể hình dung r và w như số phiếu tối thiểu cần thiết để thao tác đọc hoặc ghi là hợp lệ.

In Dynamo-style databases, the parameters n , w , and r are typically configurable. A common choice is to make n an odd number (typically 3 or 5) and to set $w = r = (n + 1) / 2$ (rounded up). However, you can vary the numbers as you see fit. For example, a workload with few writes and many reads may benefit from setting $w = n$ and $r = 1$. This makes reads faster, but has the disadvantage that just one failed node causes all database writes to fail.

Trong các cơ sở dữ liệu kiểu-Dynamo, các tham số n , w , và r thường có thể cấu hình được. Một lựa chọn phổ biến là làm cho n là một số lẻ (thường là 3 hoặc 5) và đặt $w = r = (n + 1) / 2$ (làm tròn lên). Tuy nhiên, bạn có thể thay đổi các con số tùy ý. Ví dụ, một khối lượng công việc với ít thao tác ghi và nhiều thao tác đọc có thể hưởng lợi từ việc đặt $w = n$ và $r = 1$. Điều này làm cho các thao tác đọc nhanh hơn, nhưng có nhược điểm là chỉ một nút hỏng cũng khiến mọi thao tác ghi vào cơ sở dữ liệu thất bại.

Note — Ghi chú There may be more than n nodes in the cluster, but any given value is stored only on n nodes. This allows the dataset to be partitioned, supporting datasets that are larger than you can fit on one node. We will return to partitioning in Chapter 6.

Có thể có nhiều hơn n nút trong cụm, nhưng bất kỳ giá trị cho trước nào cũng chỉ được lưu trên n nút. Điều này cho phép tập dữ liệu được phân mảnh, hỗ trợ các tập dữ liệu lớn hơn mức bạn có thể chứa trên một nút. Ta sẽ quay lại việc phân mảnh ở Chương 6.

The quorum condition, $w + r > n$, allows the system to tolerate unavailable nodes as follows:

Điều kiện nhóm tấc số, $w + r > n$, cho phép hệ thống chịu được các nút không sẵn sàng như sau:

- If $w < n$, we can still process writes if a node is unavailable.
- If $r < n$, we can still process reads if a node is unavailable.
- With $n = 3$, $w = 2$, $r = 2$ we can tolerate one unavailable node.
- With $n = 5$, $w = 3$, $r = 3$ we can tolerate two unavailable nodes. This case is illustrated in Figure 5-11.
- Normally, reads and writes are always sent to all n replicas in parallel. The parameters w and r determine how many nodes we wait for—i.e., how many of the n nodes need to report success

before we consider the read or write to be successful.

- Nếu $w < n$, ta vẫn có thể xử lý các thao tác ghi nếu một nút không sẵn sàng.
- Nếu $r < n$, ta vẫn có thể xử lý các thao tác đọc nếu một nút không sẵn sàng.
- Với $n = 3$, $w = 2$, $r = 2$ ta có thể chịu được một nút không sẵn sàng.
- Với $n = 5$, $w = 3$, $r = 3$ ta có thể chịu được hai nút không sẵn sàng. Trường hợp này được minh họa ở Hình 5-11.
- Thông thường, các thao tác đọc và ghi luôn được gửi tới cả n bản sao song song. Các tham số w và r quyết định ta chờ bao nhiêu nút — tức là bao nhiêu trong n nút cần báo thành công trước khi ta coi thao tác đọc hoặc ghi là thành công.

Figure 5-11. If $w + r > n$, at least one of the r replicas you read from must have seen the most recent successful write. — Hình 5-11. Nếu $w + r > n$, ít nhất một trong r bản sao mà bạn đọc từ đó phải đã thấy thao tác ghi thành công gần nhất. If fewer than the required w or r nodes are available, writes or reads return an error. A node could be unavailable for many reasons: because the node is down (crashed, powered down), due to an error executing the operation (can't write because the disk is full), due to a network interruption between the client and the node, or for any number of other reasons. We only care whether the node returned a successful response and don't need to distinguish between different kinds of fault.

Nếu số nút sẵn sàng ít hơn w hoặc r như yêu cầu, các thao tác ghi hoặc đọc trả về một lỗi. Một nút có thể không sẵn sàng vì nhiều lý do: vì nút ngừng hoạt động (sập, tắt nguồn), do một lỗi khi thực thi thao tác (không thể ghi vì đĩa đầy), do một gián đoạn mạng giữa client và nút, hoặc vì vô số lý do khác. Ta chỉ quan tâm liệu nút có trả về một phản hồi thành công hay không và không cần phân biệt giữa các kiểu lỗi khác nhau.

Limitations of Quorum Consistency — Hạn chế của tính nhất quán nhóm tức số

If you have n replicas, and you choose w and r such that $w + r > n$, you can generally expect every read to return the most recent value written for a key. This is the case because the set of nodes to which you've written and the set of nodes from which you've read must overlap. That is, among the nodes you read there must be at least one node with the latest value (illustrated in Figure 5-11).

Nếu bạn có n bản sao, và bạn chọn w và r sao cho $w + r > n$, bạn nhìn chung có thể kỳ vọng mọi thao tác đọc đều trả về giá trị được ghi gần nhất cho một khóa. Đây là trường hợp đúng vì tập các nút mà bạn đã ghi vào và tập các nút mà bạn đã đọc từ đó phải chồng lấn nhau. Tức là, trong số các nút bạn đọc phải có ít nhất một nút với giá trị mới nhất (minh họa ở Hình 5-11).

Often, r and w are chosen to be a majority (more than $n/2$) of nodes, because that ensures $w + r > n$ while still tolerating up to $n/2$ node failures. But quorums are not necessarily majorities—it only matters that the sets of nodes used by the read and write operations overlap in at least one node. Other quorum assignments are possible, which allows some flexibility in the design of distributed algorithms [45].

Thường thì r và w được chọn là một đa số (nhiều hơn $n/2$) các nút, vì điều đó bảo đảm $w + r > n$ trong khi vẫn chịu được tới $n/2$ nút hỏng. Nhưng nhóm tức số không nhất thiết phải là đa số — điều duy nhất quan trọng là các tập nút được dùng bởi thao tác đọc và thao tác ghi chồng lấn nhau ở ít nhất một nút. Các cách phân bổ nhóm tức số khác cũng khả dĩ, điều này cho phép có một chút linh hoạt trong việc thiết kế các thuật toán phân tán [45].

You may also set w and r to smaller numbers, so that $w + r \leq n$ (i.e., the quorum condition is not satisfied). In this case, reads and writes will still be sent to n nodes, but a smaller number of successful responses is required for the operation to succeed.

Bạn cũng có thể đặt w và r thành các con số nhỏ hơn, sao cho $w + r \leq n$ (tức là điều kiện nhóm tức số không được thỏa mãn). Trong trường hợp này, các thao tác đọc và ghi vẫn được gửi tới n nút, nhưng số phản hồi thành công cần thiết để thao tác thành công sẽ nhỏ hơn.

With a smaller w and r you are more likely to read stale values, because it's more likely that your read didn't include the node with the latest value. On the upside, this configuration allows lower latency and higher availability: if there is a network interruption and many replicas become unreachable, there's a higher chance that you can continue processing reads and writes. Only after the number of reachable replicas falls below w or r does the database become unavailable for writing or reading, respectively.

Với w và r nhỏ hơn, bạn dễ đọc phải các giá trị lỗi thời hơn, vì nhiều khả năng thao tác đọc của bạn đã không bao gồm nút có giá trị mới nhất. Mặt tích cực là cấu hình này cho phép độ trễ thấp hơn và tính sẵn sàng cao hơn: nếu có một gián đoạn mạng và nhiều bản sao trở nên không thể với tới, thì có cơ hội cao hơn rằng bạn có thể tiếp tục xử lý các thao tác đọc và ghi. Chỉ sau khi số bản sao với tới được tụt xuống dưới w hoặc r thì cơ sở dữ liệu mới trở nên không thể ghi hoặc đọc, theo thứ tự tương ứng.

However, even with $w + r > n$, there are likely to be edge cases where stale values are returned. These depend on the implementation, but possible scenarios include:

Tuy nhiên, ngay cả với $w + r > n$, nhiều khả năng vẫn có những trường hợp biên mà các giá trị lỗi thời được trả về. Những trường hợp này phụ thuộc vào hiện thực, nhưng các kịch bản khả dĩ bao gồm:

- If a sloppy quorum is used (see “Sloppy Quorums and Hinted Handoff”), the w writes may end up on different nodes than the r reads, so there is no longer a guaranteed overlap between the r nodes and the w nodes [46].
- If two writes occur concurrently, it is not clear which one happened first. In this case, the only safe solution is to merge the concurrent writes (see “Handling Write Conflicts”). If a winner is picked based on a timestamp (last write wins), writes can be lost due to clock skew [35]. We will return to this topic in “Detecting Concurrent Writes”.
- If a write happens concurrently with a read, the write may be reflected on only some of the replicas. In this case, it's undetermined whether the read returns the old or the new value.
- If a write succeeded on some replicas but failed on others (for example because the disks on some nodes are full), and overall succeeded on fewer than w replicas, it is not rolled back on the replicas where it succeeded. This means that if a write was reported as failed, subsequent reads may or may not return the value from that write [47].
- If a node carrying a new value fails, and its data is restored from a replica carrying an old value, the number of replicas storing the new value may fall below w , breaking the quorum condition.
- Even if everything is working correctly, there are edge cases in which you can get unlucky with the timing, as we shall see in “Linearizability and quorums”.
 - Nếu một nhóm tức số luộm thuộm (sloppy quorum) được dùng (xem “Nhóm tức số luộm thuộm và bàn giao gợi ý”), thì w thao tác ghi có thể rốt cuộc nằm trên các nút khác với r thao tác đọc, nên không còn sự chồng lấn được bảo đảm giữa r nút và w nút nữa [46].
 - Nếu hai thao tác ghi xảy ra tương tranh, không rõ thao tác nào xảy ra trước. Trong trường hợp này, giải pháp an toàn duy nhất là trộn các thao tác ghi tương tranh (xem

“Xử lý xung đột ghi”). Nếu một bên thắng được chọn dựa trên một dấu thời gian (người-ghi-sau-thắng), các thao tác ghi có thể bị mất do lệch đồng hồ [35]. Ta sẽ quay lại chủ đề này ở phần “Phát hiện các thao tác ghi tương tranh”.

- Nếu một thao tác ghi xảy ra tương tranh với một thao tác đọc, thao tác ghi có thể chỉ được phản ánh trên một số bản sao. Trong trường hợp này, không xác định được liệu thao tác đọc trả về giá trị cũ hay giá trị mới.
- Nếu một thao tác ghi thành công trên một số bản sao nhưng thất bại trên những bản sao khác (ví dụ vì đĩa trên một số nút bị đầy), và xét tổng thể thành công trên ít hơn w bản sao, thì nó không được hoàn tác trên các bản sao mà nó đã thành công. Điều này nghĩa là nếu một thao tác ghi được báo là thất bại, các thao tác đọc sau đó có thể trả về hoặc không trả về giá trị từ thao tác ghi đó [47].
- Nếu một nút mang một giá trị mới hỏng, và dữ liệu của nó được khôi phục từ một bản sao mang một giá trị cũ, thì số bản sao lưu giá trị mới có thể tụt xuống dưới w , làm vỡ điều kiện nhóm tức số.
- Ngay cả khi mọi thứ đang hoạt động đúng đắn, vẫn có những trường hợp biên mà bạn có thể gặp xui xẻo về thời điểm, như ta sẽ thấy ở phần “Tính tuyến tính hóa và nhóm tức số”.

Thus, although quorums appear to guarantee that a read returns the latest written value, in practice it is not so simple. Dynamo-style databases are generally optimized for use cases that can tolerate eventual consistency. The parameters w and r allow you to adjust the probability of stale values being read, but it's wise to not take them as absolute guarantees.

Do đó, mặc dù các nhóm tức số có vẻ bảo đảm rằng một thao tác đọc trả về giá trị được ghi mới nhất, nhưng trong thực tế nó không đơn giản như vậy. Các cơ sở dữ liệu kiểu-Dynamo nhìn chung được tối ưu cho các tình huống sử dụng có thể chịu được nhất quán cuối cùng. Các tham số w và r cho phép bạn điều chỉnh xác suất các giá trị lỗi thời bị đọc phải, nhưng khôn ngoan thì không nên coi chúng là các bảo đảm tuyệt đối.

In particular, you usually do not get the guarantees discussed in “Problems with Replication Lag” (reading your writes, monotonic reads, or consistent prefix reads), so the previously mentioned anomalies can occur in applications. Stronger guarantees generally require transactions or consensus. We will return to these topics in Chapter 7 and Chapter 9.

Cụ thể, bạn thường không nhận được các bảo đảm đã bàn ở phần “Các vấn đề với độ trễ sao chép” (đọc thấy bản ghi của mình, đọc đơn điệu, hoặc đọc tiền tố nhất quán), nên các dị thường đã đề cập trước đó có thể xảy ra trong các ứng dụng. Các bảo đảm mạnh hơn nhìn chung đòi hỏi giao dịch hoặc đồng thuận. Ta sẽ quay lại những chủ đề này ở Chương 7 và Chương 9.

Monitoring staleness — Giám sát mức độ lỗi thời

From an operational perspective, it's important to monitor whether your databases are returning up-to-date results. Even if your application can tolerate stale reads, you need to be aware of the health of your replication. If it falls behind significantly, it should alert you so that you can investigate the cause (for example, a problem in the network or an overloaded node).

Xét từ góc độ vận hành, điều quan trọng là giám sát xem các cơ sở dữ liệu của bạn có đang trả về các kết quả cập nhật hay không. Ngay cả khi ứng dụng của bạn có thể chịu được các thao tác đọc lỗi thời, bạn cần nhận thức được tình trạng sức khỏe của việc sao chép. Nếu nó tụt lại sau đáng kể, nó nên cảnh báo bạn để bạn có thể điều tra nguyên nhân (ví dụ, một vấn đề trong mạng hoặc một nút bị quá tải).

For leader-based replication, the database typically exposes metrics for the replication lag, which you can feed into a monitoring system. This is possible because writes are applied to the leader and to followers in the same order, and each node has a position in the replication log (the number of writes it has applied locally). By subtracting a follower’s current position from the leader’s current position, you can measure the amount of replication lag.

Với sao chép dựa trên leader, cơ sở dữ liệu thường phơi bày các số liệu về độ trễ sao chép, mà bạn có thể đưa vào một hệ thống giám sát. Điều này khả thi vì các thao tác ghi được áp dụng lên leader và lên các follower theo cùng thứ tự, và mỗi nút có một vị trí trong nhật ký sao chép (số thao tác ghi mà nó đã áp dụng cục bộ). Bằng cách lấy vị trí hiện tại của leader trừ đi vị trí hiện tại của một follower, bạn có thể đo lường độ trễ sao chép.

However, in systems with leaderless replication, there is no fixed order in which writes are applied, which makes monitoring more difficult. Moreover, if the database only uses read repair (no anti-entropy), there is no limit to how old a value might be—if a value is only infrequently read, the value returned by a stale replica may be ancient.

Tuy nhiên, trong các hệ thống dùng sao chép không-leader, không có thứ tự cố định nào mà các thao tác ghi được áp dụng theo, điều này khiến việc giám sát khó khăn hơn. Hơn nữa, nếu cơ sở dữ liệu chỉ dùng sửa chữa lúc đọc (không có chống-entropy), thì không có giới hạn nào về việc một giá trị có thể cũ tới đâu — nếu một giá trị chỉ thi thoảng mới được đọc, thì giá trị do một bản sao lỗi thời trả về có thể cổ lỗ sĩ.

There has been some research on measuring replica staleness in databases with leaderless replication and predicting the expected percentage of stale reads depending on the parameters n , w , and r [48]. This is unfortunately not yet common practice, but it would be good to include staleness measurements in the standard set of metrics for databases. Eventual consistency is a deliberately vague guarantee, but for **operability** it’s important to be able to quantify “eventual.”

Đã có một số nghiên cứu về việc đo mức độ lỗi thời của bản sao trong các cơ sở dữ liệu dùng sao chép không-leader và dự đoán tỷ lệ phần trăm thao tác đọc lỗi thời kỳ vọng tùy theo các tham số n , w , và r [48]. Đáng tiếc đây vẫn chưa phải là thông lệ phổ biến, nhưng sẽ là điều tốt nếu đưa các phép đo mức độ lỗi thời vào tập số liệu tiêu chuẩn cho cơ sở dữ liệu. Nhất quán cuối cùng là một bảo đảm mơ hồ một cách có chủ ý, nhưng vì khả năng vận hành, điều quan trọng là phải có thể định lượng được “cuối cùng.”

Sloppy Quorums and Hinted Handoff — Nhóm túc số luộm thuộm và bàn giao gợi ý

Databases with appropriately configured quorums can tolerate the failure of individual nodes without the need for failover. They can also tolerate individual nodes going slow, because requests don’t have to wait for all n nodes to respond—they can return when w or r nodes have responded. These characteristics make databases with leaderless replication appealing for use cases that require high availability and low latency, and that can tolerate occasional stale reads.

Các cơ sở dữ liệu với nhóm túc số được cấu hình phù hợp có thể chịu được sự hỏng hóc của từng nút riêng lẻ mà không cần đến chuyển đổi dự phòng. Chúng cũng có thể chịu được việc từng nút riêng lẻ chạy chậm, vì các yêu cầu không phải chờ tất cả n nút phản hồi — chúng có thể trả về khi w hoặc r nút đã phản hồi. Những đặc điểm này khiến các cơ sở dữ liệu với sao chép không-leader trở nên hấp dẫn cho các tình huống sử dụng đòi hỏi tính sẵn sàng cao và độ trễ thấp, và có thể chịu được các thao tác đọc lỗi thời thi thoảng.

However, quorums (as described so far) are not as **fault-tolerant** as they could be. A network interruption can easily cut off a client from a large number of database nodes. Although those nodes are alive, and other clients may be able to connect to them, to a client that is cut off from the database nodes, they might as well be dead. In this situation, it's likely that fewer than w or r reachable nodes remain, so the client can no longer reach a quorum.

Tuy nhiên, các nhóm tức số (như mô tả từ nãy tới giờ) không chịu lỗi tốt như chúng có thể đạt được. Một gián đoạn mạng có thể dễ dàng cắt đứt một client khỏi một số lượng lớn các nút cơ sở dữ liệu. Mặc dù những nút đó còn sống, và các client khác có thể vẫn kết nối được tới chúng, nhưng đối với một client bị cắt đứt khỏi các nút cơ sở dữ liệu, chúng cũng coi như đã chết. Trong tình huống này, nhiều khả năng còn lại ít hơn w hoặc r nút với tới được, nên client không còn đạt được một nhóm tức số nữa.

In a large cluster (with significantly more than n nodes) it's likely that the client can connect to some database nodes during the network interruption, just not to the nodes that it needs to assemble a quorum for a particular value. In that case, database designers face a trade-off:

Trong một cụm lớn (với số nút nhiều hơn n đáng kể) nhiều khả năng client có thể kết nối tới một số nút cơ sở dữ liệu trong lúc gián đoạn mạng, chỉ là không tới được những nút mà nó cần để tập hợp một nhóm tức số cho một giá trị cụ thể. Trong trường hợp đó, các nhà thiết kế cơ sở dữ liệu đối mặt với một đánh đổi:

- Is it better to return errors to all requests for which we cannot reach a quorum of w or r nodes?
- Or should we accept writes anyway, and write them to some nodes that are reachable but aren't among the n nodes on which the value usually lives?
 - Trả về lỗi cho mọi yêu cầu mà với chúng ta không thể với tới một nhóm tức số gồm w hoặc r nút thì tốt hơn?
 - Hay ta vẫn nên chấp nhận các thao tác ghi, và ghi chúng vào một số nút với tới được nhưng không nằm trong số n nút mà giá trị thường tồn tại trên đó?

The latter is known as a sloppy quorum [37]: writes and reads still require w and r successful responses, but those may include nodes that are not among the designated n “home” nodes for a value. By analogy, if you lock yourself out of your house, you may knock on the neighbor's door and ask whether you may stay on their couch temporarily.

Cách sau được gọi là một nhóm tức số luộm thuộm (sloppy quorum) [37]: các thao tác ghi và đọc vẫn đòi hỏi w và r phản hồi thành công, nhưng những phản hồi đó có thể bao gồm các nút không nằm trong số n nút “nhà” được chỉ định cho một giá trị. Lấy một phép so sánh, nếu bạn tự nhốt mình ở ngoài nhà, bạn có thể gõ cửa nhà hàng xóm và hỏi liệu mình có thể tạm tá túc trên ghế sofa của họ không.

Once the network interruption is fixed, any writes that one node temporarily accepted on behalf of another node are sent to the appropriate “home” nodes. This is called hinted handoff. (Once you find the keys to your house again, your neighbor politely asks you to get off their couch and go home.)

Một khi gián đoạn mạng được khắc phục, bất kỳ thao tác ghi nào mà một nút đã tạm thời chấp nhận thay cho một nút khác đều được gửi tới các nút “nhà” thích hợp. Điều này được gọi là bàn giao gợi ý (hinted handoff). (Một khi bạn tìm lại được chìa khóa nhà mình, người hàng xóm lịch sự đề nghị bạn rời khỏi ghế sofa của họ và về nhà.)

Sloppy quorums are particularly useful for increasing write availability: as long as any w nodes are available, the database can accept writes. However, this means that even when $w + r > n$, you cannot be sure to read the latest value for a key, because the latest value may have been temporarily written to some nodes outside of n [47].

Các nhóm tức số luộ̣m thuộ̣m đặc biệt hữu ích cho việc tăng tính sẵn sàng ghi: miễn là có bất kỳ w nút nào sẵn sàng, cơ sở dữ liệu có thể chấp nhận các thao tác ghi. Tuy nhiên, điều này nghĩa là ngay cả khi $w + r > n$, bạn không thể chắc chắn đọc được giá trị mới nhất cho một khóa, vì giá trị mới nhất có thể đã được ghi tạm thời vào một số nút bên ngoài n [47].

Thus, a sloppy quorum actually isn't a quorum at all in the traditional sense. It's only an assurance of durability, namely that the data is stored on w nodes somewhere. There is no guarantee that a read of r nodes will see it until the hinted handoff has completed.

Do đó, một nhóm tức số luộ̣m thuộ̣m thực ra hoàn toàn không phải là một nhóm tức số theo nghĩa truyền thống. Nó chỉ là một sự bảo đảm về độ bền, cụ thể là dữ liệu được lưu trên w nút ở đâu đó. Không có gì bảo đảm rằng một thao tác đọc r nút sẽ thấy được nó cho tới khi việc bàn giao gợi ý đã hoàn tất.

Sloppy quorums are optional in all common Dynamo implementations. In Riak they are enabled by default, and in Cassandra and Voldemort they are disabled by default [46, 49, 50].

Các nhóm tức số luộ̣m thuộ̣m là tùy chọn trong mọi hiện thực Dynamo phổ biến. Trong Riak chúng được bật theo mặc định, còn trong Cassandra và Voldemort chúng bị tắt theo mặc định [46, 49, 50].

Multi-datacenter operation — Vận hành đa-trung-tâm-dữ-liệu

We previously discussed cross-datacenter replication as a use case for multi-leader replication (see “Multi-Leader Replication”). Leaderless replication is also suitable for multi-datacenter operation, since it is designed to tolerate conflicting concurrent writes, network interruptions, and latency spikes.

Trước đó ta đã bàn về sao chép liên-trung-tâm-dữ-liệu như một tình huống sử dụng cho sao chép nhiều-leader (xem “Sao chép nhiều-leader”). Sao chép không-leader cũng phù hợp cho vận hành đa-trung-tâm-dữ-liệu, vì nó được thiết kế để chịu được các thao tác ghi tương tranh xung đột, các gián đoạn mạng, và các đợt tăng vọt độ trễ.

Cassandra and Voldemort implement their multi-datacenter support within the normal leaderless model: the number of replicas n includes nodes in all datacenters, and in the configuration you can specify how many of the n replicas you want to have in each datacenter. Each write from a client is sent to all replicas, regardless of datacenter, but the client usually only waits for acknowledgment from a quorum of nodes within its local datacenter so that it is unaffected by delays and interruptions on the cross-datacenter link. The higher-latency writes to other datacenters are often configured to happen asynchronously, although there is some flexibility in the configuration [50, 51].

Cassandra và Voldemort hiện thực sự hỗ trợ đa-trung-tâm-dữ-liệu của chúng bên trong mô hình không-leader thông thường: số bản sao n bao gồm các nút ở tất cả các trung tâm dữ liệu, và trong cấu hình bạn có thể chỉ định bao nhiêu trong n bản sao mà bạn muốn có ở mỗi trung tâm dữ liệu. Mỗi thao tác ghi từ một client được gửi tới tất cả các bản sao, bất kể trung tâm dữ liệu nào, nhưng client thường chỉ chờ xác nhận từ một nhóm tức số gồm các nút bên trong trung tâm dữ liệu cục bộ của nó để nó không bị ảnh hưởng bởi các độ trễ và gián đoạn trên liên kết liên-trung-tâm-dữ-liệu. Các thao tác ghi có độ trễ cao hơn tới các trung tâm dữ liệu khác thường được cấu hình để diễn ra bất đồng bộ, mặc dù có một chút linh hoạt trong cấu hình [50, 51].

Riak keeps all communication between clients and database nodes local to one datacenter, so n describes the number of replicas within one datacenter. Cross-datacenter replication between

database clusters happens asynchronously in the background, in a style that is similar to multi-leader replication [52].

Riak giữ mọi giao tiếp giữa client và các nút cơ sở dữ liệu nằm cục bộ trong một trung tâm dữ liệu, nên nó mô tả số bản sao bên trong một trung tâm dữ liệu. Sao chép liên-trung-tâm-dữ-liệu giữa các cụm cơ sở dữ liệu diễn ra bất đồng bộ ở chế độ nền, theo một kiểu tương tự như sao chép nhiều-leader [52].

Detecting Concurrent Writes — Phát hiện các thao tác ghi tương tranh

Dynamo-style databases allow several clients to concurrently write to the same key, which means that conflicts will occur even if strict quorums are used. The situation is similar to multi-leader replication (see “Handling Write Conflicts”), although in Dynamo-style databases conflicts can also arise during read repair or hinted handoff.

Các cơ sở dữ liệu kiểu-Dynamo cho phép vài client cùng ghi tương tranh vào cùng một khóa, nghĩa là xung đột sẽ xảy ra ngay cả khi các nhóm tức số nghiêm ngặt được dùng. Tình huống này tương tự sao chép nhiều-leader (xem “Xử lý xung đột ghi”), mặc dù trong các cơ sở dữ liệu kiểu-Dynamo xung đột cũng có thể nảy sinh trong lúc sửa chữa lúc đọc hoặc bàn giao gợi ý.

The problem is that events may arrive in a different order at different nodes, due to variable network delays and partial failures. For example, Figure 5-12 shows two clients, A and B, simultaneously writing to a key X in a three-node **datastore**:

Vấn đề là các sự kiện có thể đến theo một thứ tự khác nhau ở các nút khác nhau, do các độ trễ mạng biến thiên và các lỗi cục bộ. Ví dụ, Hình 5-12 cho thấy hai client, A và B, đồng thời ghi vào một khóa X trong một kho dữ liệu ba-nút:

- Node 1 receives the write from A, but never receives the write from B due to a transient outage.
- Node 2 first receives the write from A, then the write from B.
- Node 3 first receives the write from B, then the write from A.
 - Nút 1 nhận thao tác ghi từ A, nhưng không bao giờ nhận thao tác ghi từ B do một gián đoạn thoáng qua.
 - Nút 2 trước hết nhận thao tác ghi từ A, rồi tới thao tác ghi từ B.
 - Nút 3 trước hết nhận thao tác ghi từ B, rồi tới thao tác ghi từ A.

Figure 5-12. Concurrent writes in a Dynamo-style datastore: there is no well-defined ordering. — **Hình 5-12. Các thao tác ghi tương tranh trong một kho dữ liệu kiểu-Dynamo: không có thứ tự nào được định nghĩa rõ ràng.** If each node simply overwrote the value for a key whenever it received a write request from a client, the nodes would become permanently inconsistent, as shown by the final get request in Figure 5-12: node 2 thinks that the final value of X is B, whereas the other nodes think that the value is A.

Nếu mỗi nút chỉ đơn giản ghi đè giá trị cho một khóa mỗi khi nó nhận một yêu cầu ghi từ một client, các nút sẽ trở nên bất nhất vĩnh viễn, như thể hiện qua yêu cầu get cuối cùng trong Hình 5-12: nút 2 nghĩ rằng giá trị sau cùng của X là B, trong khi các nút khác nghĩ rằng giá trị là A.

In order to become eventually consistent, the replicas should converge toward the same value. How do they do that? One might hope that replicated databases would handle this automatically,

but unfortunately most implementations are quite poor: if you want to avoid losing data, you—the application developer—need to know a lot about the internals of your database’s conflict handling.

Để trở nên nhất quán cuối cùng, các bản sao nên hội tụ về cùng một giá trị. Chúng làm điều đó ra sao? Người ta có thể hy vọng rằng các cơ sở dữ liệu được sao chép sẽ tự động xử lý việc này, nhưng đáng tiếc hầu hết các hiện thực đều khá kém: nếu bạn muốn tránh mất dữ liệu, thì bạn — lập trình viên ứng dụng — cần biết rất nhiều về nội bộ của việc xử lý xung đột của cơ sở dữ liệu của mình.

We briefly touched on some techniques for conflict resolution in “Handling Write Conflicts”. Before we wrap up this chapter, let’s explore the issue in a bit more detail.

Ta đã chạm sơ qua một số kỹ thuật giải quyết xung đột ở phần “Xử lý xung đột ghi”. Trước khi khép lại chương này, hãy khám phá vấn đề này chi tiết hơn một chút.

Last write wins (discarding concurrent writes) — Người-ghi-sau-thắng (loại bỏ các thao tác ghi tương tranh)

One approach for achieving eventual convergence is to declare that each replica need only store the most “recent” value and allow “older” values to be overwritten and discarded. Then, as long as we have some way of unambiguously determining which write is more “recent,” and every write is eventually copied to every replica, the replicas will eventually converge to the same value.

Một cách tiếp cận để đạt được sự hội tụ cuối cùng là tuyên bố rằng mỗi bản sao chỉ cần lưu giá trị “mới nhất” và cho phép các giá trị “cũ hơn” bị ghi đè và loại bỏ. Khi đó, miễn là ta có cách nào đó để xác định một cách rạch ròi thao tác ghi nào là “mới hơn,” và mọi thao tác ghi rốt cuộc đều được sao chép tới mọi bản sao, thì các bản sao rốt cuộc sẽ hội tụ về cùng một giá trị.

As indicated by the quotes around “recent,” this idea is actually quite misleading. In the example of Figure 5-12, neither client knew about the other one when it sent its write requests to the database nodes, so it’s not clear which one happened first. In fact, it doesn’t really make sense to say that either happened “first”: we say the writes are concurrent, so their order is undefined.

Như ngụ ý qua các dấu ngoặc kép quanh chữ “mới nhất,” ý tưởng này thực ra khá dễ gây hiểu lầm. Trong ví dụ ở Hình 5-12, không client nào biết về client kia khi nó gửi các yêu cầu ghi của mình tới các nút cơ sở dữ liệu, nên không rõ thao tác nào xảy ra trước. Thực ra, nói rằng thao tác nào xảy ra “trước” cũng chẳng có nhiều ý nghĩa: ta nói rằng các thao tác ghi là tương tranh, nên thứ tự của chúng là không xác định.

Even though the writes don’t have a natural ordering, we can force an arbitrary order on them. For example, we can attach a timestamp to each write, pick the biggest timestamp as the most “recent,” and discard any writes with an earlier timestamp. This conflict resolution algorithm, called last write wins (LWW), is the only supported conflict resolution method in Cassandra [53], and an optional feature in Riak [35].

Mặc dù các thao tác ghi không có một thứ tự tự nhiên, ta có thể áp đặt một thứ tự tùy ý lên chúng. Ví dụ, ta có thể gắn một dấu thời gian vào mỗi thao tác ghi, chọn dấu thời gian lớn nhất làm cái “mới nhất,” và loại bỏ bất kỳ thao tác ghi nào có dấu thời gian sớm hơn. Thuật toán giải quyết xung đột này, gọi là người-ghi-sau-thắng (last write wins, LWW), là phương pháp giải quyết xung đột duy nhất được hỗ trợ trong Cassandra [53], và là một tính năng tùy chọn trong Riak [35].

LWW achieves the goal of eventual convergence, but at the cost of durability: if there are several concurrent writes to the same key, even if they were all reported as successful to the client (because

they were written to w replicas), only one of the writes will survive and the others will be silently discarded. Moreover, LWW may even drop writes that are not concurrent, as we shall discuss in “Timestamps for ordering events”.

LWW đạt được mục tiêu hội tụ cuối cùng, nhưng phải trả giá bằng độ bền: nếu có vài thao tác ghi tương tranh vào cùng một khóa, ngay cả khi tất cả chúng đều được báo là thành công với client (vì chúng đã được ghi vào w bản sao), thì chỉ một trong các thao tác ghi sẽ sống sót còn những thao tác khác sẽ bị âm thầm loại bỏ. Hơn nữa, LWW thậm chí có thể vứt bỏ các thao tác ghi không tương tranh, như ta sẽ bàn ở phần “Dấu thời gian để sắp thứ tự các sự kiện”.

There are some situations, such as caching, in which lost writes are perhaps acceptable. If losing data is not acceptable, LWW is a poor choice for conflict resolution.

Có một số tình huống, chẳng hạn caching, mà các thao tác ghi bị mất có lẽ chấp nhận được. Nếu việc mất dữ liệu là không chấp nhận được, LWW là một lựa chọn tồi cho việc giải quyết xung đột.

The only safe way of using a database with LWW is to ensure that a key is only written once and thereafter treated as immutable, thus avoiding any concurrent updates to the same key. For example, a recommended way of using Cassandra is to use a UUID as the key, thus giving each write operation a unique key [53].

Cách an toàn duy nhất để dùng một cơ sở dữ liệu với LWW là bảo đảm rằng một khóa chỉ được ghi một lần và sau đó được coi là bất biến, qua đó tránh mọi cập nhật tương tranh lên cùng một khóa. Ví dụ, một cách được khuyến nghị để dùng Cassandra là dùng một UUID làm khóa, qua đó cho mỗi thao tác ghi một khóa duy nhất [53].

The “happens-before” relationship and concurrency — Quan hệ “xảy-ra-trước” và tính tương tranh

How do we decide whether two operations are concurrent or not? To develop an intuition, let’s look at some examples:

Làm sao ta quyết định liệu hai thao tác có tương tranh hay không? Để phát triển một trực giác, hãy xem một số ví dụ:

- In Figure 5-9, the two writes are not concurrent: A’s insert happens before B’s increment, because the value incremented by B is the value inserted by A. In other words, B’s operation builds upon A’s operation, so B’s operation must have happened later. We also say that B is causally dependent on A.
- On the other hand, the two writes in Figure 5-12 are concurrent: when each client starts the operation, it does not know that another client is also performing an operation on the same key. Thus, there is no causal dependency between the operations.
 - Trong Hình 5-9, hai thao tác ghi không tương tranh: phép chèn của A xảy ra trước phép tăng của B, vì giá trị mà B tăng lên chính là giá trị do A chèn vào. Nói cách khác, thao tác của B xây dựng trên thao tác của A, nên thao tác của B hẳn phải xảy ra sau. Ta cũng nói rằng B phụ thuộc nhân quả vào A.
 - Mặt khác, hai thao tác ghi trong Hình 5-12 là tương tranh: khi mỗi client bắt đầu thao tác, nó không biết rằng một client khác cũng đang thực hiện một thao tác trên cùng khóa. Do đó, không có sự phụ thuộc nhân quả nào giữa các thao tác.

An operation A happens before another operation B if B knows about A, or depends on A, or builds upon A in some way. Whether one operation happens before another operation is the key to defining

what concurrency means. In fact, we can simply say that two operations are concurrent if neither happens before the other (i.e., neither knows about the other) [54].

Một thao tác A xảy ra trước một thao tác B khác nếu B biết về A, hoặc phụ thuộc vào A, hoặc xây dựng trên A theo một cách nào đó. Việc một thao tác có xảy ra trước một thao tác khác hay không là mấu chốt để định nghĩa tính tương tranh nghĩa là gì. Thực ra, ta có thể chỉ đơn giản nói rằng hai thao tác là tương tranh nếu không thao tác nào xảy ra trước thao tác kia (tức là không thao tác nào biết về thao tác kia) [54].

Thus, whenever you have two operations A and B, there are three possibilities: either A happened before B, or B happened before A, or A and B are concurrent. What we need is an algorithm to tell us whether two operations are concurrent or not. If one operation happened before another, the later operation should overwrite the earlier operation, but if the operations are concurrent, we have a conflict that needs to be resolved.

Do đó, mỗi khi bạn có hai thao tác A và B, có ba khả năng: hoặc A xảy ra trước B, hoặc B xảy ra trước A, hoặc A và B là tương tranh. Cái ta cần là một thuật toán cho ta biết liệu hai thao tác có tương tranh hay không. Nếu một thao tác xảy ra trước một thao tác khác, thao tác sau nên ghi đè thao tác trước, nhưng nếu các thao tác là tương tranh, ta có một xung đột cần được giải quyết.

Concurrency, Time, and Relativity — Tính tương tranh, thời gian, và thuyết tương đối It may seem that two operations should be called concurrent if they occur “at the same time”—but in fact, it is not important whether they literally overlap in time. Because of problems with clocks in distributed systems, it is actually quite difficult to tell whether two things happened at exactly the same time—an issue we will discuss in more detail in Chapter 8.

Có vẻ như hai thao tác nên được gọi là tương tranh nếu chúng xảy ra “cùng một lúc” — nhưng thực ra việc chúng có thực sự chồng lấn về mặt thời gian hay không là không quan trọng. Vì các vấn đề với đồng hồ trong hệ thống phân tán, thực ra khá khó để biết liệu hai thứ có xảy ra vào đúng cùng một thời điểm hay không — một vấn đề ta sẽ bàn chi tiết hơn ở Chương 8.

For defining concurrency, exact time doesn’t matter: we simply call two operations concurrent if they are both unaware of each other, regardless of the physical time at which they occurred. People sometimes make a connection between this principle and the special theory of relativity in physics [54], which introduced the idea that information cannot travel faster than the speed of light. Consequently, two events that occur some distance apart cannot possibly affect each other if the time between the events is shorter than the time it takes light to travel the distance between them.

Để định nghĩa tính tương tranh, thời gian chính xác không quan trọng: ta chỉ đơn giản gọi hai thao tác là tương tranh nếu cả hai đều không hay biết về nhau, bất kể thời gian vật lý mà chúng xảy ra. Người ta đôi khi liên hệ nguyên tắc này với thuyết tương đối hẹp trong vật lý [54], vốn đưa ra ý tưởng rằng thông tin không thể di chuyển nhanh hơn tốc độ ánh sáng. Do đó, hai sự kiện xảy ra cách nhau một khoảng cách không thể nào ảnh hưởng tới nhau nếu thời gian giữa các sự kiện ngắn hơn thời gian ánh sáng cần để đi hết khoảng cách giữa chúng.

In computer systems, two operations might be concurrent even though the speed of light would in principle have allowed one operation to affect the other. For example, if the network was slow or interrupted at the time, two operations can occur some time apart and still be concurrent, because the network problems prevented one operation from being able to know about the other.

Trong các hệ thống máy tính, hai thao tác có thể là tương tranh ngay cả khi tốc độ ánh

sáng về nguyên tắc đã cho phép một thao tác ảnh hưởng tới thao tác kia. Ví dụ, nếu mạng chậm hoặc bị gián đoạn vào lúc đó, hai thao tác có thể xảy ra cách nhau một khoảng thời gian mà vẫn tương tranh, vì các vấn đề mạng đã ngăn một thao tác có thể biết về thao tác kia.

Capturing the happens-before relationship — Năm bắt quan hệ xảy-ra-trước

Let's look at an algorithm that determines whether two operations are concurrent, or whether one happened before another. To keep things simple, let's start with a database that has only one replica. Once we have worked out how to do this on a single replica, we can generalize the approach to a leaderless database with multiple replicas.

Hãy xem một thuật toán xác định liệu hai thao tác có tương tranh hay không, hoặc liệu một thao tác có xảy ra trước một thao tác khác hay không. Để mọi thứ đơn giản, hãy bắt đầu với một cơ sở dữ liệu chỉ có một bản sao. Một khi ta đã tìm ra cách làm điều này trên một bản sao đơn lẻ, ta có thể tổng quát hóa cách tiếp cận sang một cơ sở dữ liệu không-leader với nhiều bản sao.

Figure 5-13 shows two clients concurrently adding items to the same shopping cart. (If that example strikes you as too inane, imagine instead two air traffic controllers concurrently adding aircraft to the sector they are tracking.) Initially, the cart is empty. Between them, the clients make five writes to the database:

Hình 5-13 cho thấy hai client đồng thời thêm các món vào cùng một giỏ hàng. (Nếu ví dụ đó với bạn quá ngớ ngẩn, thì thay vào đó hãy hình dung hai kiểm soát viên không lưu đồng thời thêm các máy bay vào khu vực mà họ đang theo dõi.) Ban đầu, giỏ hàng trống. Giữa hai bên, các client thực hiện năm thao tác ghi vào cơ sở dữ liệu:

- Client 1 adds milk to the cart. This is the first write to that key, so the server successfully stores it and assigns it version 1. The server also echoes the value back to the client, along with the version number.
- Client 2 adds eggs to the cart, not knowing that client 1 concurrently added milk (client 2 thought that its eggs were the only item in the cart). The server assigns version 2 to this write, and stores eggs and milk as two separate values. It then returns both values to the client, along with the version number of 2.
- Client 1, oblivious to client 2's write, wants to add flour to the cart, so it thinks the current cart contents should be [milk, flour]. It sends this value to the server, along with the version number 1 that the server gave client 1 previously. The server can tell from the version number that the write of [milk, flour] supersedes the prior value of [milk] but that it is concurrent with [eggs]. Thus, the server assigns version 3 to [milk, flour], overwrites the version 1 value [milk], but keeps the version 2 value [eggs] and returns both remaining values to the client.
- Meanwhile, client 2 wants to add ham to the cart, unaware that client 1 just added flour. Client 2 received the two values [milk] and [eggs] from the server in the last response, so the client now merges those values and adds ham to form a new value, [eggs, milk, ham]. It sends that value to the server, along with the previous version number 2. The server detects that version 2 overwrites [eggs] but is concurrent with [milk, flour], so the two remaining values are [milk, flour] with version 3, and [eggs, milk, ham] with version 4.
- Finally, client 1 wants to add bacon. It previously received [milk, flour] and [eggs] from the server at version 3, so it merges those, adds bacon, and sends the final value [milk, flour, eggs, bacon] to the server, along with the version number 3. This overwrites [milk, flour] (note that [eggs] was already overwritten in the last step) but is concurrent with [eggs, milk, ham], so the server keeps those two concurrent values.

- Client 1 thêm sữa vào giỏ. Đây là thao tác ghi đầu tiên vào khóa đó, nên máy chủ lưu nó thành công và gán cho nó phiên bản 1. Máy chủ cũng gửi giá trị đó lại cho client, kèm theo số phiên bản.
- Client 2 thêm trứng vào giỏ, không biết rằng client 1 đã tương tranh thêm sữa (client 2 tưởng rằng trứng của nó là món duy nhất trong giỏ). Máy chủ gán phiên bản 2 cho thao tác ghi này, và lưu trứng cùng sữa thành hai giá trị riêng biệt. Sau đó nó trả về cả hai giá trị cho client, kèm số phiên bản là 2.
- Client 1, không hay biết về thao tác ghi của client 2, muốn thêm bột mì vào giỏ, nên nó nghĩ rằng nội dung giỏ hiện tại nên là [sữa, bột mì]. Nó gửi giá trị này tới máy chủ, kèm số phiên bản 1 mà máy chủ đã cấp cho client 1 trước đó. Máy chủ có thể biết từ số phiên bản rằng thao tác ghi [sữa, bột mì] thay thế giá trị trước đó là [sữa] nhưng nó tương tranh với [trứng]. Do đó, máy chủ gán phiên bản 3 cho [sữa, bột mì], ghi đè giá trị phiên bản 1 là [sữa], nhưng giữ lại giá trị phiên bản 2 là [trứng] và trả cả hai giá trị còn lại cho client.
- Trong khi đó, client 2 muốn thêm giảm bông vào giỏ, không hay biết rằng client 1 vừa thêm bột mì. Client 2 đã nhận hai giá trị [sữa] và [trứng] từ máy chủ trong phản hồi gần nhất, nên giờ client trộn các giá trị đó và thêm giảm bông để tạo thành một giá trị mới, [trứng, sữa, giảm bông]. Nó gửi giá trị đó tới máy chủ, kèm số phiên bản 2 trước đó. Máy chủ phát hiện rằng phiên bản 2 ghi đè [trứng] nhưng tương tranh với [sữa, bột mì], nên hai giá trị còn lại là [sữa, bột mì] với phiên bản 3, và [trứng, sữa, giảm bông] với phiên bản 4.
- Cuối cùng, client 1 muốn thêm thịt xông khói. Trước đó nó đã nhận [sữa, bột mì] và [trứng] từ máy chủ ở phiên bản 3, nên nó trộn chúng, thêm thịt xông khói, và gửi giá trị sau cùng [sữa, bột mì, trứng, thịt xông khói] tới máy chủ, kèm số phiên bản 3. Điều này ghi đè [sữa, bột mì] (lưu ý rằng [trứng] đã bị ghi đè ở bước trước) nhưng tương tranh với [trứng, sữa, giảm bông], nên máy chủ giữ lại hai giá trị tương tranh đó.

Figure 5-13. Capturing causal dependencies between two clients concurrently editing a shopping cart. — Hình 5-13. Nắm bắt các phụ thuộc nhân quả giữa hai client đang tương tranh chỉnh sửa một giỏ hàng. The dataflow between the operations in Figure 5-13 is illustrated graphically in Figure 5-14. The arrows indicate which operation happened before which other operation, in the sense that the later operation knew about or depended on the earlier one. In this example, the clients are never fully up to date with the data on the server, since there is always another operation going on concurrently. But old versions of the value do get overwritten eventually, and no writes are lost.

Dòng dữ liệu giữa các thao tác trong Hình 5-13 được minh họa bằng đồ thị ở Hình 5-14. Các mũi tên chỉ ra thao tác nào xảy ra trước thao tác nào khác, theo nghĩa thao tác sau biết về hoặc phụ thuộc vào thao tác trước. Trong ví dụ này, các client không bao giờ cập nhật hoàn toàn với dữ liệu trên máy chủ, vì luôn có một thao tác khác đang diễn ra tương tranh. Nhưng các phiên bản cũ của giá trị rốt cuộc cũng bị ghi đè, và không thao tác ghi nào bị mất.

Figure 5-14. Graph of causal dependencies in Figure 5-13. — Hình 5-14. Đồ thị các phụ thuộc nhân quả trong Hình 5-13. Note that the server can determine whether two operations are concurrent by looking at the version numbers—it does not need to interpret the value itself (so the value could be any data structure). The algorithm works as follows:

Lưu ý rằng máy chủ có thể xác định liệu hai thao tác có tương tranh hay không bằng cách nhìn vào các số phiên bản — nó không cần diễn giải bản thân giá trị (nên giá trị có thể là bất kỳ cấu trúc dữ liệu nào). Thuật toán hoạt động như sau:

- The server maintains a version number for every key, increments the version number every time that key is written, and stores the new version number along with the value written.
- When a client reads a key, the server returns all values that have not been overwritten, as well as the latest version number. A client must read a key before writing.
- When a client writes a key, it must include the version number from the prior read, and it must merge together all values that it received in the prior read. (The response from a write request can be like a read, returning all current values, which allows us to chain several writes like in the shopping cart example.)
- When the server receives a write with a particular version number, it can overwrite all values with that version number or below (since it knows that they have been merged into the new value), but it must keep all values with a higher version number (because those values are concurrent with the incoming write).
 - Máy chủ duy trì một số phiên bản cho mỗi khóa, tăng số phiên bản mỗi khi khóa đó được ghi, và lưu số phiên bản mới cùng với giá trị được ghi.
 - Khi một client đọc một khóa, máy chủ trả về tất cả các giá trị chưa bị ghi đè, cũng như số phiên bản mới nhất. Một client phải đọc một khóa trước khi ghi.
 - Khi một client ghi một khóa, nó phải bao gồm số phiên bản từ lần đọc trước đó, và nó phải trộn lại với nhau tất cả các giá trị mà nó đã nhận trong lần đọc trước. (Phản hồi từ một yêu cầu ghi có thể giống như một lần đọc, trả về tất cả các giá trị hiện tại, điều này cho phép ta nối chuỗi vài thao tác ghi như trong ví dụ giỏ hàng.)
 - Khi máy chủ nhận một thao tác ghi với một số phiên bản cụ thể, nó có thể ghi đè tất cả các giá trị có số phiên bản đó hoặc thấp hơn (vì nó biết rằng chúng đã được trộn vào giá trị mới), nhưng nó phải giữ lại tất cả các giá trị có số phiên bản cao hơn (vì các giá trị đó tương tranh với thao tác ghi đang đến).

When a write includes the version number from a prior read, that tells us which previous state the write is based on. If you make a write without including a version number, it is concurrent with all other writes, so it will not overwrite anything—it will just be returned as one of the values on subsequent reads.

Khi một thao tác ghi bao gồm số phiên bản từ một lần đọc trước, điều đó cho ta biết thao tác ghi dựa trên trạng thái trước đó nào. Nếu bạn thực hiện một thao tác ghi mà không bao gồm một số phiên bản, nó tương tranh với mọi thao tác ghi khác, nên nó sẽ không ghi đè bất cứ thứ gì — nó sẽ chỉ được trả về như một trong các giá trị ở các lần đọc sau đó.

Merging concurrently written values — Trộn các giá trị được ghi tương tranh

This algorithm ensures that no data is silently dropped, but it unfortunately requires that the clients do some extra work: if several operations happen concurrently, clients have to clean up afterward by merging the concurrently written values. Riak calls these concurrent values siblings.

Thuật toán này bảo đảm rằng không dữ liệu nào bị âm thầm vứt bỏ, nhưng đáng tiếc nó đòi hỏi các client làm thêm một chút việc: nếu vài thao tác xảy ra tương tranh, các client phải dọn dẹp về sau bằng cách trộn các giá trị được ghi tương tranh. Riak gọi các giá trị tương tranh này là các anh em (siblings).

Merging sibling values is essentially the same problem as conflict resolution in multi-leader replication, which we discussed previously (see “Handling Write Conflicts”). A simple approach is to just pick one of the values based on a version number or timestamp (last write wins), but that implies losing data. So, you may need to do something more intelligent in application code.

Việc trộn các giá trị anh em về cơ bản là cùng một vấn đề với việc giải quyết xung đột trong

sao chép nhiều-leader, mà ta đã bàn trước đó (xem “Xử lý xung đột ghi”). Một cách tiếp cận đơn giản là chỉ chọn một trong các giá trị dựa trên một số phiên bản hoặc dấu thời gian (người-ghi-sau-thắng), nhưng điều đó hàm ý mất dữ liệu. Vậy nên, bạn có thể cần làm một thứ gì đó thông minh hơn trong mã ứng dụng.

With the example of a shopping cart, a reasonable approach to merging siblings is to just take the union. In Figure 5-14, the two final siblings are [milk, flour, eggs, bacon] and [eggs, milk, ham]; note that milk and eggs appear in both, even though they were each only written once. The merged value might be something like [milk, flour, eggs, bacon, ham], without duplicates.

Với ví dụ về một giỏ hàng, một cách tiếp cận hợp lý để trộn các anh em là chỉ lấy hợp. Trong Hình 5-14, hai anh em sau cùng là [sữa, bột mì, trứng, thịt xông khói] và [trứng, sữa, giảm bông]; lưu ý rằng sữa và trứng xuất hiện trong cả hai, dù mỗi món chỉ được ghi một lần. Giá trị được trộn có thể là một thứ gì đó như [sữa, bột mì, trứng, thịt xông khói, giảm bông], không có trùng lặp.

However, if you want to allow people to also remove things from their carts, and not just add things, then taking the union of siblings may not yield the right result: if you merge two sibling carts and an item has been removed in only one of them, then the removed item will reappear in the union of the siblings [37]. To prevent this problem, an item cannot simply be deleted from the database when it is removed; instead, the system must leave a marker with an appropriate version number to indicate that the item has been removed when merging siblings. Such a deletion marker is known as a tombstone. (We previously saw tombstones in the context of log compaction in “Hash Indexes”).

Tuy nhiên, nếu bạn muốn cho phép mọi người cũng được bỏ bớt các món khỏi giỏ của họ, chứ không chỉ thêm món, thì việc lấy hợp của các anh em có thể không cho ra kết quả đúng: nếu bạn trộn hai giỏ anh em và một món đã bị bỏ ở chỉ một trong hai giỏ, thì món bị bỏ đó sẽ tái xuất hiện trong hợp của các anh em [37]. Để ngăn vấn đề này, một món không thể chỉ đơn giản bị xóa khỏi cơ sở dữ liệu khi nó bị bỏ đi; thay vào đó, hệ thống phải để lại một dấu mốc với một số phiên bản thích hợp để báo rằng món đó đã bị bỏ khi trộn các anh em. Một dấu mốc xóa như vậy được gọi là một bia mộ (tombstone). (Trước đó ta đã thấy các bia mộ trong ngữ cảnh nén nhật ký ở phần “Chỉ mục băm”).

As merging siblings in application code is complex and error-prone, there are some efforts to design data structures that can perform this merging automatically, as discussed in “Automatic Conflict Resolution”. For example, Riak’s datatype support uses a family of data structures called CRDTs [38, 39, 55] that can automatically merge siblings in sensible ways, including preserving deletions.

Vì việc trộn các anh em trong mã ứng dụng thì phức tạp và dễ phát sinh lỗi, có một số nỗ lực thiết kế các cấu trúc dữ liệu có thể thực hiện việc trộn này một cách tự động, như đã bàn ở phần “Giải quyết xung đột tự động”. Ví dụ, sự hỗ trợ kiểu dữ liệu của Riak dùng một họ các cấu trúc dữ liệu gọi là CRDT [38, 39, 55] có thể tự động trộn các anh em theo những cách hợp lý, bao gồm việc giữ lại các thao tác xóa.

Version vectors — Vector phiên bản

The example in Figure 5-13 used only a single replica. How does the algorithm change when there are multiple replicas, but no leader?

Ví dụ trong Hình 5-13 chỉ dùng một bản sao đơn lẻ. Thuật toán thay đổi ra sao khi có nhiều bản sao, nhưng không có leader?

Figure 5-13 uses a single version number to capture dependencies between operations, but that is not sufficient when there are multiple replicas accepting writes concurrently. Instead, we need to use

a version number per replica as well as per key. Each replica increments its own version number when processing a write, and also keeps track of the version numbers it has seen from each of the other replicas. This information indicates which values to overwrite and which values to keep as siblings.

Hình 5-13 dùng một số phiên bản đơn lẻ để nắm bắt các phụ thuộc giữa các thao tác, nhưng điều đó không đủ khi có nhiều bản sao cùng chấp nhận thao tác ghi tương tranh. Thay vào đó, ta cần dùng một số phiên bản cho mỗi bản sao cũng như cho mỗi khóa. Mỗi bản sao tăng số phiên bản của riêng nó khi xử lý một thao tác ghi, và cũng theo dõi các số phiên bản mà nó đã thấy từ mỗi bản sao khác. Thông tin này chỉ ra giá trị nào cần ghi đè và giá trị nào cần giữ lại làm anh em.

The collection of version numbers from all the replicas is called a version vector [56]. A few variants of this idea are in use, but the most interesting is probably the dotted version vector [57], which is used in Riak 2.0 [58, 59]. We won't go into the details, but the way it works is quite similar to what we saw in our cart example.

Tập hợp các số phiên bản từ tất cả các bản sao được gọi là một vector phiên bản (version vector) [56]. Một vài biến thể của ý tưởng này đang được dùng, nhưng thú vị nhất có lẽ là vector phiên bản chấm (dotted version vector) [57], được dùng trong Riak 2.0 [58, 59]. Ta sẽ không đi vào chi tiết, nhưng cách nó hoạt động khá tương tự những gì ta đã thấy trong ví dụ giỏ hàng của mình.

Like the version numbers in Figure 5-13, version vectors are sent from the database replicas to clients when values are read, and need to be sent back to the database when a value is subsequently written. (Riak encodes the version vector as a string that it calls causal context.) The version vector allows the database to distinguish between overwrites and concurrent writes.

Giống như các số phiên bản trong Hình 5-13, các vector phiên bản được gửi từ các bản sao cơ sở dữ liệu tới các client khi các giá trị được đọc, và cần được gửi ngược lại cơ sở dữ liệu khi một giá trị sau đó được ghi. (Riak mã hóa vector phiên bản thành một chuỗi mà nó gọi là ngữ cảnh nhân quả.) Vector phiên bản cho phép cơ sở dữ liệu phân biệt giữa các thao tác ghi đè và các thao tác ghi tương tranh.

Also, like in the single-replica example, the application may need to merge siblings. The version vector structure ensures that it is safe to read from one replica and subsequently write back to another replica. Doing so may result in siblings being created, but no data is lost as long as siblings are merged correctly.

Ngoài ra, giống như trong ví dụ một-bản-sao, ứng dụng có thể cần trộn các anh em. Cấu trúc vector phiên bản bảo đảm rằng việc đọc từ một bản sao rồi sau đó ghi ngược lại một bản sao khác là an toàn. Làm như vậy có thể dẫn đến việc các anh em được tạo ra, nhưng không dữ liệu nào bị mất miễn là các anh em được trộn đúng cách.

Version vectors and vector clocks — Vector phiên bản và đồng hồ vector

A version vector is sometimes also called a vector clock, even though they are not quite the same. The difference is subtle—please see the references for details [57, 60, 61]. In brief, when comparing the state of replicas, version vectors are the right data structure to use.

Một vector phiên bản đôi khi còn được gọi là một đồng hồ vector (vector clock), dù chúng không hoàn toàn giống nhau. Sự khác biệt thì tinh tế — xin xem phần tài liệu tham khảo để biết chi tiết [57, 60, 61]. Nói ngắn gọn, khi so sánh trạng thái của các bản sao, vector phiên bản là cấu trúc dữ liệu đúng đắn để dùng.

Summary — Tóm tắt

In this chapter we looked at the issue of replication. Replication can serve several purposes:

Trong chương này, ta đã xem xét vấn đề sao chép. Sao chép có thể phục vụ vài mục đích:

Keeping the system running, even when one machine (or several machines, or an entire datacenter) goes down

Giữ cho hệ thống tiếp tục chạy, ngay cả khi một máy (hoặc vài máy, hoặc cả một trung tâm dữ liệu) ngừng hoạt động.

Allowing an application to continue working when there is a network interruption

Cho phép một ứng dụng tiếp tục hoạt động khi có một gián đoạn mạng.

Placing data geographically close to users, so that users can interact with it faster

Đặt dữ liệu gần người dùng về mặt địa lý, để người dùng có thể tương tác với nó nhanh hơn.

Being able to handle a higher volume of reads than a single machine could handle, by performing reads on replicas

Có thể xử lý một khối lượng đọc lớn hơn mức một máy đơn lẻ có thể kham nổi, bằng cách thực hiện các thao tác đọc trên các bản sao.

Despite being a simple goal—keeping a copy of the same data on several machines—replication turns out to be a remarkably tricky problem. It requires carefully thinking about concurrency and about all the things that can go wrong, and dealing with the consequences of those faults. At a minimum, we need to deal with unavailable nodes and network interruptions (and that's not even considering the more insidious kinds of fault, such as silent data corruption due to software bugs).

Bất chấp việc là một mục tiêu đơn giản — giữ một bản sao của cùng một dữ liệu trên vài máy — việc sao chép hóa ra lại là một vấn đề khó nhằn đến đáng kinh ngạc. Nó đòi hỏi suy nghĩ cẩn thận về tính tương tranh và về tất cả những thứ có thể trục trặc, và xử lý các hệ quả của những lỗi đó. Tối thiểu, ta cần xử lý các nút không sẵn sàng và các gián đoạn mạng (và đó còn chưa xét tới những kiểu lỗi hiểm hóc hơn, chẳng hạn dữ liệu bị hỏng âm thầm do các bug phần mềm).

We discussed three main approaches to replication:

Ta đã bàn về ba cách tiếp cận chính cho việc sao chép:

Clients send all writes to a single node (the leader), which sends a stream of data change events to the other replicas (followers). Reads can be performed on any replica, but reads from followers might be stale.

Các client gửi tất cả thao tác ghi tới một nút duy nhất (leader), và nút đó gửi một luồng các sự kiện thay đổi dữ liệu tới các bản sao khác (follower). Các thao tác đọc có thể được thực hiện trên bất kỳ bản sao nào, nhưng các thao tác đọc từ follower có thể bị lỗi thời.

Clients send each write to one of several leader nodes, any of which can accept writes. The leaders send streams of data change events to each other and to any follower nodes.

Các client gửi mỗi thao tác ghi tới một trong vài nút leader; bất kỳ nút nào trong số đó cũng có thể chấp nhận thao tác ghi. Các leader gửi các luồng sự kiện thay đổi dữ liệu cho nhau và cho bất kỳ nút follower nào.

Clients send each write to several nodes, and read from several nodes in parallel in order to detect and correct nodes with stale data.

Các client gửi mỗi thao tác ghi tới vài nút, và đọc từ vài nút song song để phát hiện và sửa chữa các nút có dữ liệu lỗi thời.

Each approach has advantages and disadvantages. Single-leader replication is popular because it is fairly easy to understand and there is no conflict resolution to worry about. Multi-leader and leaderless replication can be more robust in the presence of faulty nodes, network interruptions, and latency spikes—at the cost of being harder to reason about and providing only very weak consistency guarantees.

Mỗi cách tiếp cận có những ưu nhược điểm. Sao chép một-leader phổ biến vì nó khá dễ hiểu và không có việc giải quyết xung đột nào để phải bận tâm. Sao chép nhiều-leader và không-leader có thể vững vàng hơn khi có các nút lỗi, các gián đoạn mạng, và các đợt tăng vọt độ trễ — đổi lại là khó suy luận hơn và chỉ cung cấp những bảo đảm nhất quán rất yếu.

Replication can be synchronous or asynchronous, which has a profound effect on the system behavior when there is a fault. Although asynchronous replication can be fast when the system is running smoothly, it's important to figure out what happens when replication lag increases and servers fail. If a leader fails and you promote an asynchronously updated follower to be the new leader, recently committed data may be lost.

Sao chép có thể là đồng bộ hoặc bất đồng bộ, điều này có một ảnh hưởng sâu sắc tới hành vi của hệ thống khi có một lỗi. Mặc dù sao chép bất đồng bộ có thể nhanh khi hệ thống đang chạy trơn tru, điều quan trọng là phải tìm hiểu xem chuyện gì xảy ra khi độ trễ sao chép tăng và các máy chủ hỏng. Nếu một leader hỏng và bạn nâng một follower được cập nhật bất đồng bộ lên làm leader mới, dữ liệu vừa được commit gần đây có thể bị mất.

We looked at some strange effects that can be caused by replication lag, and we discussed a few consistency models which are helpful for deciding how an application should behave under replication lag:

Ta đã xem xét một số hiệu ứng kỳ lạ có thể do độ trễ sao chép gây ra, và ta đã bàn về một vài mô hình nhất quán hữu ích cho việc quyết định một ứng dụng nên hành xử ra sao khi có độ trễ sao chép:

Users should always see data that they submitted themselves.

Người dùng nên luôn thấy dữ liệu mà chính họ đã gửi.

After users have seen the data at one point in time, they shouldn't later see the data from some earlier point in time.

Sau khi người dùng đã thấy dữ liệu ở một thời điểm, họ không nên về sau lại thấy dữ liệu từ một thời điểm sớm hơn nào đó.

Users should see the data in a state that makes causal sense: for example, seeing a question and its reply in the correct order.

Người dùng nên thấy dữ liệu ở một trạng thái có ý nghĩa nhân quả: ví dụ, thấy một câu hỏi và câu trả lời của nó theo đúng thứ tự.

Finally, we discussed the concurrency issues that are inherent in multi-leader and leaderless replication approaches: because they allow multiple writes to happen concurrently, conflicts may occur. We examined an algorithm that a database might use to determine whether one operation happened before another, or whether they happened concurrently. We also touched on methods for resolving conflicts by merging together concurrent updates.

Cuối cùng, ta đã bàn về các vấn đề tương tranh vốn có trong các cách tiếp cận sao chép nhiều-leader và không-leader: vì chúng cho phép nhiều thao tác ghi xảy ra tương tranh, các xung đột có thể xảy ra. Ta đã khảo sát một thuật toán mà một cơ sở dữ liệu có thể dùng để xác định liệu một thao tác có xảy ra trước một thao tác khác hay không, hoặc liệu chúng có xảy ra tương tranh hay không. Ta cũng đã chạm tới các phương pháp giải quyết xung đột bằng cách trộn các cập nhật tương tranh lại với nhau.

In the next chapter we will continue looking at data that is distributed across multiple machines, through the counterpart of replication: splitting a large dataset into partitions.

Ở chương tiếp theo, ta sẽ tiếp tục xem xét dữ liệu được phân tán trên nhiều máy, thông qua đối tác của sao chép: tách một tập dữ liệu lớn thành các phân mảnh.

Footnotes Different people have different definitions for hot, warm, and cold standby servers. In PostgreSQL, for example, hot standby is used to refer to a replica that accepts reads from clients, whereas a warm standby processes changes from the leader but doesn't process any queries from clients. For purposes of this book, the difference isn't important.

This approach is known as fencing or, more emphatically, Shoot The Other Node In The Head (STONITH). We will discuss fencing in more detail in “The leader and the lock”.

The term eventual consistency was coined by Douglas Terry et al. [24], popularized by Werner Vogels [22], and became the battle cry of many NoSQL projects. However, not only NoSQL databases are eventually consistent: followers in an asynchronously replicated relational database have the same characteristics.

If the database is partitioned (see Chapter 6), each partition has one leader. Different partitions may have their leaders on different nodes, but each partition must nevertheless have one leader node.

Not to be confused with a star schema (see “Stars and Snowflakes: Schemas for Analytics”), which describes the structure of a data model, not the communication topology between nodes.

Dynamo is not available to users outside of Amazon. Confusingly, AWS offers a hosted database product called DynamoDB, which uses a completely different architecture: it is based on single-leader replication.

Sometimes this kind of quorum is called a strict quorum, to contrast with sloppy quorums (discussed in “Sloppy Quorums and Hinted Handoff”).

References [1] Bruce G. Lindsay, Patricia Griffiths Selinger, C. Galtieri, et al.: “Notes on Distributed Databases,” IBM Research, Research Report RJ2571(33471), July 1979.

[2] “Oracle Active Data Guard Real-Time Data Protection and Availability,” Oracle White Paper, June 2013.

- [3] “AlwaysOn Availability Groups,” in SQL Server Books Online, Microsoft, 2012.
- [4] Lin Qiao, Kapil Surlaker, Shirshanka Das, et al.: “On Brewing Fresh Espresso: LinkedIn’s Distributed Data Serving Platform,” at ACM International Conference on Management of Data (SIGMOD), June 2013.
- [5] Jun Rao: “Intra-Cluster Replication for Apache Kafka,” at ApacheCon North America, February 2013.
- [6] “Highly Available Queues,” in RabbitMQ Server Documentation, Pivotal Software, Inc., 2014.
- [7] Yoshinori Matsunobu: “Semi-Synchronous Replication at Facebook,” yoshinorimatsunobu.blogspot.co.uk, April 1, 2014.
- [8] Robbert van Renesse and Fred B. Schneider: “Chain Replication for Supporting High Throughput and Availability,” at 6th USENIX Symposium on Operating System Design and Implementation (OSDI), December 2004.
- [9] Jeff Terrace and Michael J. Freedman: “Object Storage on CRAQ: High-Throughput Chain Replication for Read-Mostly Workloads,” at USENIX Annual Technical Conference (ATC), June 2009.
- [10] Brad Calder, Ju Wang, Aaron Ogus, et al.: “Windows Azure Storage: A Highly Available Cloud Storage Service with Strong Consistency,” at 23rd ACM Symposium on Operating Systems Principles (SOSP), October 2011.
- [11] Andrew Wang: “Windows Azure Storage,” umbrant.com, February 4, 2016.
- [12] “Percona Xtrabackup - Documentation,” Percona LLC, 2014.
- [13] Jesse Newland: “GitHub Availability This Week,” github.com, September 14, 2012.
- [14] Mark Imbriaco: “Downtime Last Saturday,” github.com, December 26, 2012.
- [15] John Hugg: “‘All in’ with Determinism for Performance and Testing in Distributed Systems,” at Strange Loop, September 2015.
- [16] Amit Kapila: “WAL Internals of PostgreSQL,” at PostgreSQL Conference (PGCon), May 2012.
- [17] MySQL Internals Manual. Oracle, 2014.
- [18] Yogeshwer Sharma, Philippe Ajoux, Petchean Ang, et al.: “Wormhole: Reliable Pub-Sub to Support Geo-Replicated Internet Services,” at 12th USENIX Symposium on Networked Systems Design and Implementation (NSDI), May 2015.
- [19] “Oracle GoldenGate 12c: Real-Time Access to Real-Time Information,” Oracle White Paper, October 2013.
- [20] Shirshanka Das, Chavdar Botev, Kapil Surlaker, et al.: “All Aboard the Databus!,” at ACM Symposium on Cloud Computing (SoCC), October 2012.
- [21] Greg Sabino Mullane: “Version 5 of Bucardo Database Replication System,” blog.endpoint.com, June 23, 2014.
- [22] Werner Vogels: “Eventually Consistent,” ACM Queue, volume 6, number 6, pages 14–19, October 2008. doi:10.1145/1466443.1466448
- [23] Douglas B. Terry: “Replicated Data Consistency Explained Through Baseball,” Microsoft Research, Technical Report MSR-TR-2011-137, October 2011.

- [24] Douglas B. Terry, Alan J. Demers, Karin Petersen, et al.: “Session Guarantees for Weakly Consistent Replicated Data,” at 3rd International Conference on Parallel and Distributed Information Systems (PDIS), September 1994. doi:10.1109/PDIS.1994.331722
- [25] Terry Pratchett: *Reaper Man: A Discworld Novel*. Victor Gollancz, 1991. ISBN: 978-0-575-04979-6
- [26] “Tungsten Replicator,” Continuent, Inc., 2014.
- [27] “BDR 0.10.0 Documentation,” The PostgreSQL Global Development Group, bdr-project.org, 2015.
- [28] Robert Hodges: “If You *Must* Deploy Multi-Master Replication, Read This First,” scale-out-blog.blogspot.co.uk, March 30, 2012.
- [29] J. Chris Anderson, Jan Lehnardt, and Noah Slater: *CouchDB: The Definitive Guide*. O’Reilly Media, 2010. ISBN: 978-0-596-15589-6
- [30] AppJet, Inc.: “Etherpad and EasySync Technical Manual,” github.com, March 26, 2011.
- [31] John Day-Richter: “What’s Different About the New Google Docs: Making Collaboration Fast,” googledrive.blogspot.com, 23 September 2010.
- [32] Martin Kleppmann and Alastair R. Beresford: “A Conflict-Free Replicated JSON Datatype,” arXiv:1608.03960, August 13, 2016.
- [33] Frazer Clement: “Eventual Consistency – Detecting Conflicts,” messagepassing.blogspot.co.uk, October 20, 2011.
- [34] Robert Hodges: “State of the Art for MySQL Multi-Master Replication,” at Percona Live: MySQL Conference & Expo, April 2013.
- [35] John Daily: “Clocks Are Bad, or, Welcome to the Wonderful World of Distributed Systems,” basho.com, November 12, 2013.
- [36] Riley Berton: “Is Bi-Directional Replication (BDR) in Postgres Transactional?,” sdf.org, January 4, 2016.
- [37] Giuseppe DeCandia, Deniz Hastorun, Madan Jampani, et al.: “Dynamo: Amazon’s Highly Available Key-Value Store,” at 21st ACM Symposium on Operating Systems Principles (SOSP), October 2007.
- [38] Marc Shapiro, Nuno Preguiça, Carlos Baquero, and Marek Zawirski: “A Comprehensive Study of Convergent and Commutative Replicated Data Types,” INRIA Research Report no. 7506, January 2011.
- [39] Sam Elliott: “CRDTs: An UPDATE (or Maybe Just a PUT),” at RICON West, October 2013.
- [40] Russell Brown: “A Bluffers Guide to CRDTs in Riak,” gist.github.com, October 28, 2013.
- [41] Benjamin Farinier, Thomas Gazagnaire, and Anil Madhavapeddy: “Mergeable Persistent Data Structures,” at 26es Journées Francophones des Langages Applicatifs (JFLA), January 2015.
- [42] Chengzheng Sun and Clarence Ellis: “Operational Transformation in Real-Time Group Editors: Issues, Algorithms, and Achievements,” at ACM Conference on Computer Supported Cooperative Work (CSCW), November 1998.
- [43] Lars Hofhansl: “HBASE-7709: Infinite Loop Possible in Master/Master Replication,” issues.apache.org, January 29, 2013.
- [44] David K. Gifford: “Weighted Voting for Replicated Data,” at 7th ACM Symposium on Operating Systems Principles (SOSP), December 1979. doi:10.1145/800215.806583

- [45] Heidi Howard, Dahlia Malkhi, and Alexander Spiegelman: “Flexible Paxos: Quorum Intersection Revisited,” arXiv:1608.06696, August 24, 2016.
- [46] Joseph Blomstedt: “Re: Absolute Consistency,” email to riak-users mailing list, lists.basho.com, January 11, 2012.
- [47] Joseph Blomstedt: “Bringing Consistency to Riak,” at RICON West, October 2012.
- [48] Peter Bailis, Shivaram Venkataraman, Michael J. Franklin, et al.: “Quantifying Eventual Consistency with PBS,” Communications of the ACM, volume 57, number 8, pages 93–102, August 2014. doi:10.1145/2632792
- [49] Jonathan Ellis: “Modern Hinted Handoff,” datastax.com, December 11, 2012.
- [50] “Project Voldemort Wiki,” github.com, 2013.
- [51] “Apache Cassandra 2.0 Documentation,” DataStax, Inc., 2014.
- [52] “Riak Enterprise: Multi-Datcenter Replication.” Technical whitepaper, Basho Technologies, Inc., September 2014.
- [53] Jonathan Ellis: “Why Cassandra Doesn’t Need Vector Clocks,” datastax.com, September 2, 2013.
- [54] Leslie Lamport: “Time, Clocks, and the Ordering of Events in a Distributed System,” Communications of the ACM, volume 21, number 7, pages 558–565, July 1978. doi:10.1145/359545.359563
- [55] Joel Jacobson: “Riak 2.0: Data Types,” blog.joeljacobsen.com, March 23, 2014.
- [56] D. Stott Parker Jr., Gerald J. Popek, Gerard Rudisin, et al.: “Detection of Mutual Inconsistency in Distributed Systems,” IEEE Transactions on Software Engineering, volume 9, number 3, pages 240–247, May 1983. doi:10.1109/TSE.1983.236733
- [57] Nuno Preguiça, Carlos Baquero, Paulo Sérgio Almeida, et al.: “Dotted Version Vectors: Logical Clocks for Optimistic Replication,” arXiv:1011.5808, November 26, 2010.
- [58] Sean Cribbs: “A Brief History of Time in Riak,” at RICON, October 2014.
- [59] Russell Brown: “Vector Clocks Revisited Part 2: Dotted Version Vectors,” basho.com, November 10, 2015.
- [60] Carlos Baquero: “Version Vectors Are Not Vector Clocks,” haslab.wordpress.com, July 8, 2011.
- [61] Reinhard Schwarz and Friedemann Mattern: “Detecting Causal Relationships in Distributed Computations: In Search of the Holy Grail,” Distributed Computing, volume 7, number 3, pages 149–174, March 1994. doi:10.1007/BF02277859

Chapter 6. Partitioning — Chương 6. Phân vùng

Clearly, we must break away from the sequential and not limit the computers. We must state definitions and provide for priorities and descriptions of data. We must state relationships, not procedures.

Rõ ràng, ta phải thoát khỏi lối tư duy tuần tự và không bó buộc các máy tính. Ta phải nêu ra các định nghĩa, đồng thời quy định những thứ tự ưu tiên cùng những mô tả về dữ liệu. Ta phải nêu ra các quan hệ, chứ không phải các thủ tục.

Grace Murray Hopper, Management and the Computer of the Future (1962)

Grace Murray Hopper, Management and the Computer of the Future (1962)

In Chapter 5 we discussed replication—that is, having multiple copies of the same data on different nodes. For very large datasets, or very high query **throughput**, that is not sufficient: we need to break the data up into partitions, also known as sharding.

Ở Chương 5 ta đã bàn về nhân bản — tức là có nhiều bản sao của cùng một dữ liệu trên các nút khác nhau. Với những tập dữ liệu cực lớn, hoặc với thông lượng truy vấn rất cao, thì như vậy là chưa đủ: ta cần chia nhỏ dữ liệu thành các phân vùng (partition), còn được gọi là sharding.

Terminological confusion — Sự rối rắm về thuật ngữ

What we call a partition here is called a shard in MongoDB, Elasticsearch, and SolrCloud; it's known as a region in HBase, a tablet in Bigtable, a vnode in Cassandra and Riak, and a vBucket in Couchbase. However, partitioning is the most established term, so we'll stick with that.

Cái mà ở đây ta gọi là phân vùng (partition) thì lại được gọi là shard trong MongoDB, Elasticsearch, và SolrCloud; nó được biết đến là region trong HBase, tablet trong Bigtable, vnode trong Cassandra và Riak, và vBucket trong Couchbase. Tuy nhiên, partitioning là thuật ngữ đã được thiết lập vững chắc nhất, nên ta sẽ dùng nó.

Normally, partitions are defined in such a way that each piece of data (each record, **row**, or **document**) belongs to exactly one partition. There are various ways of achieving this, which we discuss in depth in this chapter. In effect, each partition is a small **database** of its own, although the database may support operations that touch multiple partitions at the same time.

Thông thường, các phân vùng được định nghĩa sao cho mỗi mẫu dữ liệu (mỗi bản ghi, hàng, hoặc tài liệu) thuộc về đúng một phân vùng. Có nhiều cách khác nhau để đạt được điều này, mà ta sẽ bàn sâu trong chương này. Trên thực tế, mỗi phân vùng tự nó là một cơ sở dữ liệu nhỏ, mặc dù cơ sở dữ liệu có thể hỗ trợ các thao tác chạm tới nhiều phân vùng cùng một lúc.

The main reason for wanting to partition data is **scalability**. Different partitions can be placed on different nodes in a shared-nothing cluster (see the introduction to Part II for a definition of shared nothing). Thus, a large dataset can be distributed across many disks, and the query **load** can be distributed across many processors.

Lý do chính khiến ta muốn phân vùng dữ liệu là khả năng mở rộng (scalability). Các phân vùng khác nhau có thể được đặt trên các nút khác nhau trong một cụm kiến trúc shared-nothing (xem phần giới thiệu của Phần II để biết định nghĩa của shared-nothing). Nhờ đó, một tập dữ liệu lớn có thể được phân tán trên nhiều đĩa, và tải truy vấn có thể được phân tán trên nhiều bộ xử lý.

For queries that operate on a single partition, each **node** can independently execute the queries for its own partition, so query throughput can be scaled by adding more nodes. Large, complex queries can potentially be parallelized across many nodes, although this gets significantly harder.

Với những truy vấn chỉ thao tác trên một phân vùng duy nhất, mỗi nút có thể độc lập thực thi các truy vấn cho phân vùng của riêng nó, nên thông lượng truy vấn có thể được mở rộng bằng cách thêm nhiều nút hơn. Những truy vấn lớn, phức tạp có khả năng được song song hóa trên nhiều nút, mặc dù điều này khó hơn đáng kể.

Partitioned databases were pioneered in the 1980s by products such as Teradata and Tandem NonStop **SQL** [1], and more recently rediscovered by **NoSQL** databases and Hadoop-based data warehouses. Some systems are designed for transactional workloads, and others for analytics (see “**Transaction Processing** or Analytics?”): this difference affects how the system is tuned, but the fundamentals of partitioning apply to both kinds of workloads.

Cơ sở dữ liệu phân vùng được tiên phong vào thập niên 1980 bởi các sản phẩm như Teradata và Tandem NonStop SQL [1], và gần đây hơn được tái khám phá bởi các cơ sở dữ liệu NoSQL và các kho dữ liệu dựa trên Hadoop. Một số hệ thống được thiết kế cho các tải giao dịch, số khác cho phân tích (xem “Xử lý giao dịch hay phân tích?”): khác biệt này ảnh hưởng tới cách hệ thống được tinh chỉnh, nhưng các nguyên lý nền tảng của việc phân vùng đều áp dụng cho cả hai loại tải.

In this chapter we will first look at different approaches for partitioning large datasets and observe how the indexing of data interacts with partitioning. We’ll then talk about rebalancing, which is necessary if you want to add or remove nodes in your cluster. Finally, we’ll get an overview of how databases route requests to the right partitions and execute queries.

Trong chương này, trước tiên ta sẽ xem xét các cách tiếp cận khác nhau để phân vùng những tập dữ liệu lớn và quan sát cách việc lập chỉ mục dữ liệu tương tác với phân vùng. Sau đó ta sẽ bàn về việc tái cân bằng (rebalancing), điều cần thiết nếu bạn muốn thêm hoặc bớt nút trong cụm của mình. Cuối cùng, ta sẽ có cái nhìn tổng quan về cách các cơ sở dữ liệu định tuyến yêu cầu tới đúng phân vùng và thực thi truy vấn.

Partitioning and Replication — Phân vùng và nhân bản

Partitioning is usually combined with replication so that copies of each partition are stored on multiple nodes. This means that, even though each record belongs to exactly one partition, it may still be stored on several different nodes for **fault** tolerance.

Phân vùng thường được kết hợp với nhân bản sao cho các bản sao của mỗi phân vùng được lưu trên nhiều nút. Điều này nghĩa là, mặc dù mỗi bản ghi thuộc về đúng một phân vùng, nó vẫn có thể được lưu trên vài nút khác nhau để chịu lỗi.

A node may store more than one partition. If a leader-**follower** replication model is used, the combination of partitioning and replication can look like Figure 6-1. Each partition's leader is assigned to one node, and its followers are assigned to other nodes. Each node may be the leader for some partitions and a follower for other partitions.

Một nút có thể lưu nhiều hơn một phân vùng. Nếu dùng mô hình nhân bản leader-follower, thì sự kết hợp giữa phân vùng và nhân bản có thể trông giống Hình 6-1. Leader của mỗi phân vùng được gán cho một nút, còn các follower của nó được gán cho các nút khác. Mỗi nút có thể là leader cho một số phân vùng và là follower cho các phân vùng khác.

Everything we discussed in Chapter 5 about replication of databases applies equally to replication of partitions. The choice of partitioning scheme is mostly independent of the choice of replication scheme, so we will keep things simple and ignore replication in this chapter.

Mọi điều ta đã bàn ở Chương 5 về nhân bản cơ sở dữ liệu đều áp dụng tương tự cho nhân bản các phân vùng. Việc chọn sơ đồ phân vùng hầu như độc lập với việc chọn sơ đồ nhân bản, nên ta sẽ giữ mọi thứ đơn giản và bỏ qua nhân bản trong chương này.

Figure 6-1. Combining replication and partitioning: each node acts as leader for some partitions and follower for other partitions. — Hình 6-1. Kết hợp nhân bản và phân vùng: mỗi nút đóng vai trò leader cho một số phân vùng và follower cho các phân vùng khác.

Partitioning of Key-Value Data — Phân vùng dữ liệu khóa-giá trị

Say you have a large amount of data, and you want to partition it. How do you decide which records to store on which nodes?

Giả sử bạn có một lượng lớn dữ liệu, và bạn muốn phân vùng nó. Bạn quyết định lưu bản ghi nào trên nút nào như thế nào?

Our goal with partitioning is to spread the data and the query load evenly across nodes. If every node takes a fair share, then—in theory—10 nodes should be able to handle 10 times as much data and 10 times the read and write throughput of a single node (ignoring replication for now).

Mục tiêu của ta khi phân vùng là trải đều dữ liệu và tải truy vấn trên các nút. Nếu mỗi nút gánh một phần công bằng, thì — về lý thuyết — 10 nút sẽ có thể xử lý lượng dữ liệu gấp 10 lần và thông lượng đọc-ghi gấp 10 lần so với một nút đơn lẻ (tạm bỏ qua nhân bản lúc này).

If the partitioning is unfair, so that some partitions have more data or queries than others, we call it skewed. The presence of skew makes partitioning much less effective. In an extreme case, all the load could end up on one partition, so 9 out of 10 nodes are idle and your bottleneck is the single busy node. A partition with disproportionately high load is called a hot spot.

Nếu việc phân vùng không công bằng, sao cho một số phân vùng có nhiều dữ liệu hoặc truy vấn hơn các phân vùng khác, thì ta gọi đó là lệch (skewed). Sự hiện diện của độ lệch khiến việc phân vùng kém hiệu quả hơn nhiều. Trong trường hợp cực đoan, toàn bộ tải có thể dồn về một phân vùng, nên 9 trong 10 nút ngồi không và nút bận rộn duy nhất là nút thắt cổ chai của bạn. Một phân vùng có tải cao một cách không cân xứng được gọi là điểm nóng (hot spot).

The simplest approach for avoiding hot spots would be to assign records to nodes randomly. That would distribute the data quite evenly across the nodes, but it has a big disadvantage: when you're trying to read a particular item, you have no way of knowing which node it is on, so you have to query all nodes in parallel.

Cách đơn giản nhất để tránh điểm nóng có lẽ là gán các bản ghi cho các nút một cách ngẫu nhiên. Cách đó sẽ phân tán dữ liệu khá đều trên các nút, nhưng nó có một nhược điểm lớn: khi bạn cố đọc một mục cụ thể, bạn không có cách nào biết được nó nằm trên nút nào, nên bạn phải truy vấn song song mọi nút.

We can do better. Let's assume for now that you have a simple key-value **data model**, in which you always access a record by its **primary key**. For example, in an old-fashioned paper encyclopedia, you look up an entry by its title; since all the entries are alphabetically sorted by title, you can quickly

find the one you're looking for.

Ta có thể làm tốt hơn. Hãy tạm giả định rằng bạn có một mô hình dữ liệu khóa-giá trị đơn giản, trong đó bạn luôn truy cập một bản ghi bằng khóa chính của nó. Ví dụ, trong một cuốn bách khoa toàn thư giấy kiểu cũ, bạn tra một mục theo tiêu đề của nó; vì tất cả các mục đều được sắp xếp theo bảng chữ cái theo tiêu đề, nên bạn có thể nhanh chóng tìm thấy mục mình cần.

Partitioning by Key Range — Phân vùng theo khoảng khóa

One way of partitioning is to assign a continuous range of keys (from some minimum to some maximum) to each partition, like the volumes of a paper encyclopedia (Figure 6-2). If you know the boundaries between the ranges, you can easily determine which partition contains a given key. If you also know which partition is assigned to which node, then you can make your request directly to the appropriate node (or, in the case of the encyclopedia, pick the correct book off the shelf).

Một cách phân vùng là gán cho mỗi phân vùng một khoảng khóa liên tục (từ một giá trị nhỏ nhất nào đó tới một giá trị lớn nhất nào đó), giống như các tập của một cuốn bách khoa toàn thư giấy (Hình 6-2). Nếu bạn biết ranh giới giữa các khoảng, bạn có thể dễ dàng xác định phân vùng nào chứa một khóa cho trước. Và nếu bạn cũng biết phân vùng nào được gán cho nút nào, thì bạn có thể gửi yêu cầu trực tiếp tới đúng nút (hoặc, trong trường hợp bách khoa toàn thư, rút đúng cuốn sách khỏi kệ).

Figure 6-2. A print encyclopedia is partitioned by key range. — Hình 6-2. Một cuốn bách khoa toàn thư in được phân vùng theo khoảng khóa. The ranges of keys are not necessarily evenly spaced, because your data may not be evenly distributed. For example, in Figure 6-2, volume 1 contains words starting with A and B, but volume 12 contains words starting with T, U, V, X, Y, and Z. Simply having one volume per two letters of the alphabet would lead to some volumes being much bigger than others. In order to distribute the data evenly, the partition boundaries need to adapt to the data.

Các khoảng khóa không nhất thiết được chia đều, vì dữ liệu của bạn có thể không phân bố đều. Ví dụ, trong Hình 6-2, tập 1 chứa các từ bắt đầu bằng A và B, nhưng tập 12 chứa các từ bắt đầu bằng T, U, V, X, Y, và Z. Nếu chỉ đơn giản dành một tập cho mỗi hai chữ cái của bảng chữ cái thì sẽ khiến một số tập lớn hơn nhiều so với các tập khác. Để phân tán dữ liệu đều, các ranh giới phân vùng cần thích nghi với dữ liệu.

The partition boundaries might be chosen manually by an administrator, or the database can choose them automatically (we will discuss choices of partition boundaries in more detail in “Rebalancing Partitions”). This partitioning strategy is used by Bigtable, its open source equivalent HBase [2, 3], RethinkDB, and MongoDB before version 2.4 [4].

Các ranh giới phân vùng có thể được một quản trị viên chọn thủ công, hoặc cơ sở dữ liệu có thể tự động chọn chúng (ta sẽ bàn chi tiết hơn về các lựa chọn ranh giới phân vùng ở phần “Tái cân bằng các phân vùng”). Chiến lược phân vùng này được Bigtable, bản tương đương mã nguồn mở của nó là HBase [2, 3], RethinkDB, và MongoDB trước phiên bản 2.4 [4] sử dụng.

Within each partition, we can keep keys in sorted order (see “SSTables and LSM-Trees”). This has the advantage that range scans are easy, and you can treat the key as a concatenated **index** in order to fetch several related records in one query (see “Multi-**column** indexes”). For example, consider an application that stores data from a network of sensors, where the key is the timestamp

of the measurement (year-month-day-hour-minute-second). Range scans are very useful in this case, because they let you easily fetch, say, all the readings from a particular month.

Trong mỗi phân vùng, ta có thể giữ các khóa theo thứ tự đã sắp xếp (xem “SSTables và LSM-Trees”). Điều này có lợi thế là các phép quét khoảng (range scan) trở nên dễ dàng, và bạn có thể coi khóa như một chỉ mục ghép để lấy về vài bản ghi liên quan trong một truy vấn (xem “Chỉ mục đa cột”). Ví dụ, hãy xét một ứng dụng lưu dữ liệu từ một mạng lưới cảm biến, trong đó khóa là dấu thời gian của phép đo (năm-tháng-ngày-giờ-phút-giây). Các phép quét khoảng rất hữu ích trong trường hợp này, vì chúng cho phép bạn dễ dàng lấy về, chẳng hạn, tất cả các số đo trong một tháng cụ thể.

However, the downside of key range partitioning is that certain access patterns can lead to hot spots. If the key is a timestamp, then the partitions correspond to ranges of time—e.g., one partition per day. Unfortunately, because we write data from the sensors to the database as the measurements happen, all the writes end up going to the same partition (the one for today), so that partition can be overloaded with writes while others sit idle [5].

Tuy nhiên, nhược điểm của việc phân vùng theo khoảng khóa là một số mẫu truy cập nhất định có thể dẫn tới điểm nóng. Nếu khóa là một dấu thời gian, thì các phân vùng tương ứng với các khoảng thời gian — ví dụ, mỗi ngày một phân vùng. Đáng tiếc, vì ta ghi dữ liệu từ các cảm biến vào cơ sở dữ liệu ngay khi các phép đo diễn ra, nên tất cả các thao tác ghi đều dồn về cùng một phân vùng (phân vùng của ngày hôm nay), khiến phân vùng đó có thể bị quá tải vì ghi trong khi các phân vùng khác ngồi không [5].

To avoid this problem in the sensor database, you need to use something other than the timestamp as the first element of the key. For example, you could prefix each timestamp with the sensor name so that the partitioning is first by sensor name and then by time. Assuming you have many sensors active at the same time, the write load will end up more evenly spread across the partitions. Now, when you want to fetch the values of multiple sensors within a time range, you need to perform a separate range query for each sensor name.

Để tránh vấn đề này trong cơ sở dữ liệu cảm biến, bạn cần dùng một thứ khác thay vì dấu thời gian làm phần tử đầu tiên của khóa. Ví dụ, bạn có thể thêm tiền tố tên cảm biến vào trước mỗi dấu thời gian, sao cho việc phân vùng trước hết là theo tên cảm biến rồi sau đó mới theo thời gian. Giả sử bạn có nhiều cảm biến hoạt động cùng lúc, tải ghi rất cuộc sẽ được trải đều hơn trên các phân vùng. Bây giờ, khi bạn muốn lấy giá trị của nhiều cảm biến trong một khoảng thời gian, bạn cần thực hiện một truy vấn khoảng riêng cho mỗi tên cảm biến.

Partitioning by Hash of Key — Phân vùng theo hàm băm của khóa

Because of this risk of skew and hot spots, many distributed datastores use a hash function to determine the partition for a given key.

Vì rủi ro về độ lệch và điểm nóng này, nhiều kho dữ liệu phân tán dùng một hàm băm để xác định phân vùng cho một khóa cho trước.

A good hash function takes skewed data and makes it uniformly distributed. Say you have a 32-bit hash function that takes a string. Whenever you give it a new string, it returns a seemingly random number between 0 and $2^{32} - 1$. Even if the input strings are very similar, their hashes are evenly distributed across that range of numbers.

Một hàm băm tốt nhận dữ liệu bị lệch và làm cho nó phân bố đều. Giả sử bạn có một hàm băm 32-bit nhận vào một chuỗi. Mỗi khi bạn đưa cho nó một chuỗi mới, nó trả về một con

số dường như ngẫu nhiên giữa 0 và $2^{32} - 1$. Ngay cả khi các chuỗi đầu vào rất giống nhau, các giá trị băm của chúng vẫn được phân bố đều trên khoảng số đó.

For partitioning purposes, the hash function need not be cryptographically strong: for example, Cassandra and MongoDB use MD5, and Voldemort uses the Fowler–Noll–Vo function. Many programming languages have simple hash functions built in (as they are used for hash tables), but they may not be suitable for partitioning: for example, in Java’s `Object.hashCode()` and Ruby’s `Object#hash`, the same key may have a different hash value in different processes [6].

Với mục đích phân vùng, hàm băm không cần mạnh về mặt mật mã: ví dụ, Cassandra và MongoDB dùng MD5, còn Voldemort dùng hàm Fowler–Noll–Vo. Nhiều ngôn ngữ lập trình có sẵn các hàm băm đơn giản (vì chúng được dùng cho bảng băm), nhưng chúng có thể không phù hợp cho việc phân vùng: ví dụ, trong `Object.hashCode()` của Java và `Object#hash` của Ruby, cùng một khóa có thể có giá trị băm khác nhau trong các tiến trình khác nhau [6].

Once you have a suitable hash function for keys, you can assign each partition a range of hashes (rather than a range of keys), and every key whose hash falls within a partition’s range will be stored in that partition. This is illustrated in Figure 6-3.

Một khi bạn đã có một hàm băm phù hợp cho các khóa, bạn có thể gán cho mỗi phân vùng một khoảng các giá trị băm (thay vì một khoảng các khóa), và mọi khóa có giá trị băm rơi vào khoảng của một phân vùng sẽ được lưu trong phân vùng đó. Điều này được minh họa trong Hình 6-3.

Figure 6-3. Partitioning by hash of key. — Hình 6-3. Phân vùng theo hàm băm của khóa. This technique is good at distributing keys fairly among the partitions. The partition boundaries can be evenly spaced, or they can be chosen pseudorandomly (in which case the technique is sometimes known as consistent hashing).

Kỹ thuật này tốt ở việc phân bố các khóa một cách công bằng giữa các phân vùng. Các ranh giới phân vùng có thể được chia đều, hoặc chúng có thể được chọn theo kiểu giả ngẫu nhiên (trong trường hợp này kỹ thuật đôi khi được gọi là băm nhất quán — consistent hashing).

Consistent Hashing — Băm nhất quán Consistent hashing, as defined by Karger et al. [7], is a way of evenly distributing load across an internet-wide system of caches such as a content delivery network (CDN). It uses randomly chosen partition boundaries to avoid the need for central control or distributed consensus. Note that consistent here has nothing to do with replica consistency (see Chapter 5) or ACID consistency (see Chapter 7), but rather describes a particular approach to rebalancing.

Băm nhất quán, theo định nghĩa của Karger và cộng sự [7], là một cách phân bố tải đều trên một hệ thống bộ nhớ đệm trải rộng khắp internet, chẳng hạn như một mạng phân phối nội dung (CDN). Nó dùng các ranh giới phân vùng được chọn ngẫu nhiên để tránh nhu cầu về điều khiển tập trung hay đồng thuận phân tán. Lưu ý rằng chữ “nhất quán” (consistent) ở đây chẳng liên quan gì tới tính nhất quán của bản sao (xem Chương 5) hay tính nhất quán ACID (xem Chương 7), mà chỉ mô tả một cách tiếp cận cụ thể đối với việc tái cân bằng.

As we shall see in “Rebalancing Partitions”, this particular approach actually doesn’t work very well for databases [8], so it is rarely used in practice (the documentation of some databases still refers to consistent hashing, but it is often inaccurate). Because this is so confusing, it’s best to avoid the term consistent hashing and just call it hash partitioning instead.

Như ta sẽ thấy ở phần “Tái cân bằng các phân vùng”, cách tiếp cận cụ thể này thực ra không hoạt động tốt lắm với cơ sở dữ liệu [8], nên nó hiếm khi được dùng trong thực tế (tài liệu của một số cơ sở dữ liệu vẫn nhắc tới băm nhất quán, nhưng cách dùng này thường không chính xác). Vì điều này gây rối rắm, tốt nhất là nên tránh thuật ngữ băm nhất quán và chỉ gọi nó là phân vùng theo băm (hash partitioning) cho rồi.

Unfortunately however, by using the hash of the key for partitioning we lose a nice **property** of key-range partitioning: the ability to do efficient range queries. Keys that were once adjacent are now scattered across all the partitions, so their sort order is lost. In MongoDB, if you have enabled hash-based sharding mode, any range query has to be sent to all partitions [4]. Range queries on the primary key are not supported by Riak [9], Couchbase [10], or Voldemort.

Tuy nhiên, đáng tiếc là khi dùng giá trị băm của khóa để phân vùng, ta đánh mất một đặc tính hay ho của việc phân vùng theo khoảng khóa: khả năng thực hiện các truy vấn khoảng hiệu quả. Những khóa từng kề nhau nay bị rải khắp các phân vùng, nên thứ tự sắp xếp của chúng bị mất. Trong MongoDB, nếu bạn đã bật chế độ sharding dựa trên băm, thì bất kỳ truy vấn khoảng nào cũng phải được gửi tới tất cả các phân vùng [4]. Các truy vấn khoảng trên khóa chính không được Riak [9], Couchbase [10], hay Voldemort hỗ trợ.

Cassandra achieves a compromise between the two partitioning strategies [11, 12, 13]. A **table** in Cassandra can be declared with a compound primary key consisting of several columns. Only the first part of that key is hashed to determine the partition, but the other columns are used as a concatenated index for sorting the data in Cassandra’s SSTables. A query therefore cannot search for a range of values within the first column of a compound key, but if it specifies a fixed value for the first column, it can perform an efficient range scan over the other columns of the key.

Cassandra đạt được một sự thỏa hiệp giữa hai chiến lược phân vùng [11, 12, 13]. Một bảng trong Cassandra có thể được khai báo với một khóa chính phức hợp gồm vài cột. Chỉ phần đầu tiên của khóa đó được băm để xác định phân vùng, còn các cột khác được dùng làm một chỉ mục ghép để sắp xếp dữ liệu trong các SSTable của Cassandra. Do đó, một truy vấn không thể tìm kiếm theo một khoảng giá trị trong cột đầu tiên của một khóa phức hợp, nhưng nếu nó chỉ định một giá trị cố định cho cột đầu tiên, thì nó có thể thực hiện một phép quét khoảng hiệu quả trên các cột còn lại của khóa.

The concatenated index approach enables an elegant data model for **one-to-many** relationships. For example, on a social media site, one user may post many updates. If the primary key for updates is chosen to be (user_id, update_timestamp), then you can efficiently retrieve all updates made by a particular user within some time interval, sorted by timestamp. Different users may be stored on different partitions, but within each user, the updates are stored ordered by timestamp on a single partition.

Cách tiếp cận chỉ mục ghép cho phép một mô hình dữ liệu thanh thoát cho các quan hệ một-nhiều. Ví dụ, trên một trang mạng xã hội, một người dùng có thể đăng nhiều cập nhật. Nếu khóa chính cho các cập nhật được chọn là (user_id, update_timestamp), thì bạn có thể lấy về hiệu quả tất cả các cập nhật do một người dùng cụ thể thực hiện trong một khoảng thời gian nào đó, đã được sắp xếp theo dấu thời gian. Những người dùng khác nhau có thể được lưu trên các phân vùng khác nhau, nhưng trong phạm vi mỗi người dùng, các cập nhật được lưu theo thứ tự dấu thời gian trên một phân vùng duy nhất.

Skewed Workloads and Relieving Hot Spots — Tải bị lệch và làm giảm điểm nóng

As discussed, hashing a key to determine its partition can help reduce hot spots. However, it can't avoid them entirely: in the extreme case where all reads and writes are for the same key, you still end up with all requests being routed to the same partition.

Như đã bàn, việc băm một khóa để xác định phân vùng của nó có thể giúp giảm điểm nóng. Tuy nhiên, nó không thể tránh được hoàn toàn: trong trường hợp cực đoan khi mọi thao tác đọc và ghi đều dành cho cùng một khóa, bạn vẫn rất cuộc khiến tất cả các yêu cầu bị định tuyến tới cùng một phân vùng.

This kind of workload is perhaps unusual, but not unheard of: for example, on a social media site, a celebrity user with millions of followers may cause a storm of activity when they do something [14]. This event can result in a large volume of writes to the same key (where the key is perhaps the user ID of the celebrity, or the ID of the action that people are commenting on). Hashing the key doesn't help, as the hash of two identical IDs is still the same.

Loại tải này có lẽ là bất thường, nhưng không phải chưa từng nghe đến: ví dụ, trên một trang mạng xã hội, một người dùng nổi tiếng với hàng triệu người theo dõi có thể gây ra một cơn bão hoạt động khi họ làm điều gì đó [14]. Sự kiện này có thể dẫn tới một khối lượng lớn các thao tác ghi vào cùng một khóa (mà khóa ở đây có lẽ là ID người dùng của người nổi tiếng, hoặc ID của hành động mà mọi người đang bình luận). Việc băm khóa chẳng giúp ích gì, vì giá trị băm của hai ID giống hệt nhau thì vẫn như nhau.

Today, most data systems are not able to automatically compensate for such a highly skewed workload, so it's the responsibility of the application to reduce the skew. For example, if one key is known to be very hot, a simple technique is to add a random number to the beginning or end of the key. Just a two-digit decimal random number would split the writes to the key evenly across 100 different keys, allowing those keys to be distributed to different partitions.

Ngày nay, hầu hết các hệ thống dữ liệu không thể tự động bù trừ cho một tải lệch nghiêm trọng như vậy, nên việc giảm độ lệch là trách nhiệm của ứng dụng. Ví dụ, nếu biết một khóa rất nóng, một kỹ thuật đơn giản là thêm một con số ngẫu nhiên vào đầu hoặc cuối khóa. Chỉ cần một con số ngẫu nhiên thập phân hai chữ số là đã đủ để chia đều các thao tác ghi vào khóa đó ra thành 100 khóa khác nhau, cho phép những khóa đó được phân tán tới các phân vùng khác nhau.

However, having split the writes across different keys, any reads now have to do additional work, as they have to read the data from all 100 keys and combine it. This technique also requires additional bookkeeping: it only makes sense to append the random number for the small number of hot keys; for the vast majority of keys with low write throughput this would be unnecessary overhead. Thus, you also need some way of keeping track of which keys are being split.

Tuy nhiên, một khi đã chia các thao tác ghi ra thành các khóa khác nhau, thì giờ đây bất kỳ thao tác đọc nào cũng phải làm thêm việc, vì chúng phải đọc dữ liệu từ tất cả 100 khóa rồi kết hợp lại. Kỹ thuật này cũng đòi hỏi thêm việc ghi sổ: chỉ hợp lý khi nối thêm con số ngẫu nhiên cho số ít khóa nóng; với đại đa số các khóa có thông lượng ghi thấp thì điều này sẽ là chi phí không cần thiết. Do đó, bạn cũng cần một cách nào đó để theo dõi những khóa nào đang bị chia tách.

Perhaps in the future, data systems will be able to automatically detect and compensate for skewed workloads; but for now, you need to think through the trade-offs for your own application.

Có lẽ trong tương lai, các hệ thống dữ liệu sẽ có thể tự động phát hiện và bù trừ cho các tải bị lệch; nhưng hiện tại, bạn cần tự cân nhắc kỹ các đánh đổi cho ứng dụng của riêng mình.

Partitioning and Secondary Indexes

— Phân vùng và chỉ mục phụ

The partitioning schemes we have discussed so far rely on a key-value data model. If records are only ever accessed via their primary key, we can determine the partition from that key and use it to route read and write requests to the partition responsible for that key.

Các sơ đồ phân vùng mà ta đã bàn cho tới giờ đều dựa trên một mô hình dữ liệu khóa-giá trị. Nếu các bản ghi chỉ luôn được truy cập qua khóa chính của chúng, thì ta có thể xác định phân vùng từ khóa đó và dùng nó để định tuyến các yêu cầu đọc và ghi tới phân vùng chịu trách nhiệm cho khóa đó.

The situation becomes more complicated if secondary indexes are involved (see also “Other Indexing Structures”). A secondary index usually doesn’t identify a record uniquely but rather is a way of searching for occurrences of a particular value: find all actions by user 123, find all articles containing the word hogwash, find all cars whose color is red, and so on.

Tình huống trở nên phức tạp hơn nếu có liên quan tới các chỉ mục phụ (xem thêm “Các cấu trúc lập chỉ mục khác”). Một chỉ mục phụ thường không định danh một bản ghi một cách duy nhất, mà đúng hơn là một cách để tìm kiếm các lần xuất hiện của một giá trị cụ thể: tìm tất cả hành động của người dùng 123, tìm tất cả bài viết chứa từ hogwash, tìm tất cả xe có màu đỏ, v.v.

Secondary indexes are the bread and butter of relational databases, and they are common in document databases too. Many key-value stores (such as HBase and Voldemort) have avoided secondary indexes because of their added implementation complexity, but some (such as Riak) have started adding them because they are so useful for data modeling. And finally, secondary indexes are the *raison d’être* of search servers such as Solr and Elasticsearch.

Chỉ mục phụ là cơm bữa của các cơ sở dữ liệu quan hệ, và chúng cũng phổ biến trong các cơ sở dữ liệu tài liệu. Nhiều kho khóa-giá trị (như HBase và Voldemort) đã tránh dùng chỉ mục phụ vì độ phức tạp hiện thực mà chúng thêm vào, nhưng một số (như Riak) đã bắt đầu thêm chúng vì chúng hữu ích đến vậy cho việc mô hình hóa dữ liệu. Và cuối cùng, chỉ mục phụ chính là lý do tồn tại của các máy chủ tìm kiếm như Solr và Elasticsearch.

The problem with secondary indexes is that they don’t map neatly to partitions. There are two main approaches to partitioning a database with secondary indexes: document-based partitioning and term-based partitioning.

Vấn đề với chỉ mục phụ là chúng không ánh xạ gọn gàng vào các phân vùng. Có hai cách tiếp cận chính để phân vùng một cơ sở dữ liệu có chỉ mục phụ: phân vùng theo tài liệu và phân vùng theo thuật ngữ.

Partitioning Secondary Indexes by Document — Phân vùng chỉ mục phụ theo tài liệu

For example, imagine you are operating a website for selling used cars (illustrated in Figure 6-4). Each listing has a unique ID—call it the document ID—and you partition the database by the document ID (for example, IDs 0 to 499 in partition 0, IDs 500 to 999 in partition 1, etc.).

Ví dụ, hãy hình dung bạn đang vận hành một website bán xe cũ (minh họa trong Hình 6-4). Mỗi tin rao có một ID duy nhất — gọi nó là ID tài liệu — và bạn phân vùng cơ sở dữ liệu theo ID tài liệu (ví dụ, các ID từ 0 đến 499 ở phân vùng 0, các ID từ 500 đến 999 ở phân vùng 1, v.v.).

You want to let users search for cars, allowing them to filter by color and by make, so you need a secondary index on color and make (in a document database these would be fields; in a **relational database** they would be columns). If you have declared the index, the database can perform the indexing automatically. For example, whenever a red car is added to the database, the database partition automatically adds it to the list of document IDs for the index entry color:red.

Bạn muốn cho người dùng tìm kiếm xe, cho phép họ lọc theo màu và theo hãng, nên bạn cần một chỉ mục phụ trên màu và hãng (trong một cơ sở dữ liệu tài liệu thì đây là các trường; trong một cơ sở dữ liệu quan hệ thì chúng là các cột). Nếu bạn đã khai báo chỉ mục, cơ sở dữ liệu có thể tự động thực hiện việc lập chỉ mục. Ví dụ, mỗi khi một chiếc xe màu đỏ được thêm vào cơ sở dữ liệu, phân vùng cơ sở dữ liệu tự động thêm nó vào danh sách các ID tài liệu cho mục chỉ mục color:red.

Figure 6-4. Partitioning secondary indexes by document. — Hình 6-4. Phân vùng chỉ mục phụ theo tài liệu. In this indexing approach, each partition is completely separate: each partition maintains its own secondary indexes, covering only the documents in that partition. It doesn't care what data is stored in other partitions. Whenever you need to write to the database—to add, remove, or update a document—you only need to deal with the partition that contains the document ID that you are writing. For that reason, a document-partitioned index is also known as a local index (as opposed to a global index, described in the next section).

Trong cách lập chỉ mục này, mỗi phân vùng hoàn toàn tách biệt: mỗi phân vùng duy trì các chỉ mục phụ của riêng nó, chỉ bao phủ các tài liệu trong phân vùng đó. Nó không quan tâm dữ liệu nào được lưu trong các phân vùng khác. Mỗi khi bạn cần ghi vào cơ sở dữ liệu — để thêm, xóa, hoặc cập nhật một tài liệu — bạn chỉ cần làm việc với phân vùng chứa ID tài liệu mà bạn đang ghi. Vì lý do đó, một chỉ mục phân vùng theo tài liệu còn được gọi là chỉ mục cục bộ (local index) (trái với chỉ mục toàn cục — global index, được mô tả trong phần tiếp theo).

However, reading from a document-partitioned index requires care: unless you have done something special with the document IDs, there is no reason why all the cars with a particular color or a particular make would be in the same partition. In Figure 6-4, red cars appear in both partition 0 and partition 1. Thus, if you want to search for red cars, you need to send the query to all partitions, and combine all the results you get back.

Tuy nhiên, việc đọc từ một chỉ mục phân vùng theo tài liệu đòi hỏi sự cẩn trọng: trừ khi bạn đã làm điều gì đó đặc biệt với các ID tài liệu, không có lý do gì để tất cả các xe có một màu cụ thể hoặc một hãng cụ thể đều nằm trong cùng một phân vùng. Trong Hình 6-4, các xe màu đỏ xuất hiện ở cả phân vùng 0 lẫn phân vùng 1. Do đó, nếu bạn muốn tìm kiếm các xe màu đỏ, bạn cần gửi truy vấn tới tất cả các phân vùng, rồi kết hợp tất cả các kết quả

nhận về.

This approach to querying a partitioned database is sometimes known as scatter/gather, and it can make read queries on secondary indexes quite expensive. Even if you query the partitions in parallel, scatter/gather is prone to **tail latency** amplification**** (see “Percentiles in Practice”). Nevertheless, it is widely used: MongoDB, Riak [15], Cassandra [16], Elasticsearch [17], SolrCloud [18], and VoltDB [19] all use document-partitioned secondary indexes. Most database vendors recommend that you structure your partitioning scheme so that secondary index queries can be served from a single partition, but that is not always possible, especially when you’re using multiple secondary indexes in a single query (such as filtering cars by color and by make at the same time).

Cách tiếp cận này để truy vấn một cơ sở dữ liệu phân vùng đôi khi được gọi là phát tán/thu gom (scatter/gather), và nó có thể khiến các truy vấn đọc trên chỉ mục phụ trở nên khá đắt đỏ. Ngay cả khi bạn truy vấn các phân vùng song song, phát tán/thu gom vẫn dễ bị khuếch đại độ trễ đuôi (xem “Phân vị trong thực tế”). Tuy vậy, nó được dùng rộng rãi: MongoDB, Riak [15], Cassandra [16], Elasticsearch [17], SolrCloud [18], và VoltDB [19] đều dùng các chỉ mục phụ phân vùng theo tài liệu. Hầu hết các nhà cung cấp cơ sở dữ liệu khuyến nghị bạn cấu trúc sơ đồ phân vùng của mình sao cho các truy vấn chỉ mục phụ có thể được phục vụ từ một phân vùng duy nhất, nhưng điều đó không phải lúc nào cũng khả thi, đặc biệt khi bạn dùng nhiều chỉ mục phụ trong cùng một truy vấn (chẳng hạn lọc xe theo màu và theo hãng cùng một lúc).

Figure 6-5. Partitioning secondary indexes by term. — Hình 6-5. Phân vùng chỉ mục phụ theo thuật ngữ.

Partitioning Secondary Indexes by Term — Phân vùng chỉ mục phụ theo thuật ngữ

Rather than each partition having its own secondary index (a local index), we can construct a global index that covers data in all partitions. However, we can’t just store that index on one node, since it would likely become a bottleneck and defeat the purpose of partitioning. A global index must also be partitioned, but it can be partitioned differently from the primary key index.

Thay vì để mỗi phân vùng có chỉ mục phụ riêng của nó (một chỉ mục cục bộ), ta có thể dựng một chỉ mục toàn cục bao phủ dữ liệu trong tất cả các phân vùng. Tuy nhiên, ta không thể chỉ lưu chỉ mục đó trên một nút, vì nó nhiều khả năng sẽ trở thành một nút thắt cổ chai và phá hỏng mục đích của việc phân vùng. Một chỉ mục toàn cục cũng phải được phân vùng, nhưng nó có thể được phân vùng khác với cách phân vùng chỉ mục khóa chính.

Figure 6-5 illustrates what this could look like: red cars from all partitions appear under color:red in the index, but the index is partitioned so that colors starting with the letters a to r appear in partition 0 and colors starting with s to z appear in partition 1. The index on the make of car is partitioned similarly (with the partition boundary being between f and h).

Hình 6-5 minh họa điều này có thể trông như thế nào: các xe màu đỏ từ tất cả các phân vùng đều xuất hiện dưới color:red trong chỉ mục, nhưng chỉ mục được phân vùng sao cho các màu bắt đầu bằng các chữ cái từ a đến r xuất hiện ở phân vùng 0 và các màu bắt đầu bằng s đến z xuất hiện ở phân vùng 1. Chỉ mục trên hãng xe được phân vùng tương tự (với ranh giới phân vùng nằm giữa f và h).

We call this kind of index term-partitioned, because the term we’re looking for determines the partition of the index. Here, a term would be color:red, for example. The name term comes from

full-text indexes (a particular kind of secondary index), where the terms are all the words that occur in a document.

Ta gọi loại chỉ mục này là phân vùng theo thuật ngữ (term-partitioned), vì thuật ngữ (term) mà ta đang tìm kiếm sẽ quyết định phân vùng của chỉ mục. Ở đây, một thuật ngữ chẳng hạn sẽ là `color:red`. Cái tên thuật ngữ bắt nguồn từ các chỉ mục toàn văn (một loại chỉ mục phụ cụ thể), nơi các thuật ngữ là tất cả các từ xuất hiện trong một tài liệu.

As before, we can partition the index by the term itself, or using a hash of the term. Partitioning by the term itself can be useful for range scans (e.g., on a numeric property, such as the asking price of the car), whereas partitioning on a hash of the term gives a more even distribution of load.

Như trước, ta có thể phân vùng chỉ mục theo chính thuật ngữ, hoặc dùng một giá trị băm của thuật ngữ. Phân vùng theo chính thuật ngữ có thể hữu ích cho các phép quét khoảng (ví dụ, trên một thuộc tính dạng số, chẳng hạn giá chào bán của chiếc xe), trong khi phân vùng theo một giá trị băm của thuật ngữ cho ra một sự phân bố tải đều hơn.

The advantage of a global (term-partitioned) index over a document-partitioned index is that it can make reads more efficient: rather than doing scatter/gather over all partitions, a **client** only needs to make a request to the partition containing the term that it wants. However, the downside of a global index is that writes are slower and more complicated, because a write to a single document may now affect multiple partitions of the index (every term in the document might be on a different partition, on a different node).

Lợi thế của một chỉ mục toàn cục (phân vùng theo thuật ngữ) so với một chỉ mục phân vùng theo tài liệu là nó có thể khiến các thao tác đọc hiệu quả hơn: thay vì thực hiện phát tán/thu gom trên tất cả các phân vùng, một client chỉ cần gửi một yêu cầu tới phân vùng chứa thuật ngữ mà nó muốn. Tuy nhiên, nhược điểm của một chỉ mục toàn cục là các thao tác ghi chậm hơn và phức tạp hơn, vì một thao tác ghi vào một tài liệu duy nhất giờ đây có thể ảnh hưởng tới nhiều phân vùng của chỉ mục (mỗi thuật ngữ trong tài liệu có thể nằm trên một phân vùng khác, trên một nút khác).

In an ideal world, the index would always be up to date, and every document written to the database would immediately be reflected in the index. However, in a term-partitioned index, that would require a distributed transaction across all partitions affected by a write, which is not supported in all databases (see Chapter 7 and Chapter 9).

Trong một thế giới lý tưởng, chỉ mục sẽ luôn được cập nhật, và mọi tài liệu được ghi vào cơ sở dữ liệu sẽ ngay lập tức được phản ánh trong chỉ mục. Tuy nhiên, trong một chỉ mục phân vùng theo thuật ngữ, điều đó sẽ đòi hỏi một giao dịch phân tán trên tất cả các phân vùng bị ảnh hưởng bởi một thao tác ghi, vốn không được tất cả các cơ sở dữ liệu hỗ trợ (xem Chương 7 và Chương 9).

In practice, updates to global secondary indexes are often asynchronous (that is, if you read the index shortly after a write, the change you just made may not yet be reflected in the index). For example, Amazon DynamoDB states that its global secondary indexes are updated within a fraction of a second in normal circumstances, but may experience longer propagation delays in cases of faults in the infrastructure [20].

Trong thực tế, các cập nhật cho chỉ mục phụ toàn cục thường là bất đồng bộ (tức là, nếu bạn đọc chỉ mục ngay sau một thao tác ghi, thay đổi mà bạn vừa thực hiện có thể chưa được phản ánh trong chỉ mục). Ví dụ, Amazon DynamoDB tuyên bố rằng các chỉ mục phụ toàn cục của nó được cập nhật trong vòng một phần nhỏ của giây trong điều kiện bình thường, nhưng có thể gặp độ trễ lan truyền lâu hơn trong các trường hợp có trục trặc ở hạ tầng [20].

Other uses of global term-partitioned indexes include Riak's search feature [21] and the Oracle data warehouse, which lets you choose between local and global indexing [22]. We will return to the topic of implementing term-partitioned secondary indexes in Chapter 12.

Các cách dùng khác của chỉ mục toàn cục phân vùng theo thuật ngữ bao gồm tính năng tìm kiếm của Riak [21] và kho dữ liệu Oracle, vốn cho phép bạn chọn giữa lập chỉ mục cục bộ và toàn cục [22]. Ta sẽ quay lại chủ đề hiện thực các chỉ mục phụ phân vùng theo thuật ngữ ở Chương 12.

Rebalancing Partitions — Tái cân bằng các phân vùng

Over time, things change in a database:

Theo thời gian, mọi thứ trong một cơ sở dữ liệu đều thay đổi:

- The query throughput increases, so you want to add more CPUs to handle the load.
- The dataset size increases, so you want to add more disks and RAM to store it.
- A machine fails, and other machines need to take over the failed machine's responsibilities.
 - Thông lượng truy vấn tăng lên, nên bạn muốn thêm CPU để xử lý tải.
 - Kích thước tập dữ liệu tăng lên, nên bạn muốn thêm đĩa và RAM để lưu trữ nó.
 - Một máy bị hỏng, và các máy khác cần tiếp quản trách nhiệm của máy bị hỏng.

All of these changes call for data and requests to be moved from one node to another. The process of moving load from one node in the cluster to another is called rebalancing.

Tất cả những thay đổi này đòi hỏi dữ liệu và các yêu cầu phải được di chuyển từ nút này sang nút khác. Quá trình di chuyển tải từ một nút trong cụm sang một nút khác được gọi là tái cân bằng (rebalancing).

No matter which partitioning scheme is used, rebalancing is usually expected to meet some minimum requirements:

Bất kể dùng sơ đồ phân vùng nào, việc tái cân bằng thường được kỳ vọng đáp ứng một số yêu cầu tối thiểu:

- After rebalancing, the load (data storage, read and write requests) should be shared fairly between the nodes in the cluster.
- While rebalancing is happening, the database should continue accepting reads and writes.
- No more data than necessary should be moved between nodes, to make rebalancing fast and to minimize the network and disk I/O load.
 - Sau khi tái cân bằng, tải (lưu trữ dữ liệu, các yêu cầu đọc và ghi) nên được chia sẻ công bằng giữa các nút trong cụm.
 - Trong khi việc tái cân bằng đang diễn ra, cơ sở dữ liệu nên tiếp tục chấp nhận các thao tác đọc và ghi.
 - Không nên di chuyển nhiều dữ liệu hơn mức cần thiết giữa các nút, để việc tái cân bằng diễn ra nhanh và để giảm thiểu tải I/O lên mạng và đĩa.

Strategies for Rebalancing — Các chiến lược tái cân bằng

There are a few different ways of assigning partitions to nodes [23]. Let's briefly discuss each in turn.

Có vài cách khác nhau để gán các phân vùng cho các nút [23]. Hãy bàn vắn tắt từng cách một.

How not to do it: hash mod N — Cách không nên làm: hash mod N

When partitioning by the hash of a key, we said earlier (Figure 6-3) that it's best to divide the possible hashes into ranges and assign each range to a partition (e.g., assign key to partition 0 if $0 \leq \text{hash}(\text{key}) < b_0$, to partition 1 if $b_0 \leq \text{hash}(\text{key}) < b_1$, etc.).

Khi phân vùng theo hàm băm của một khóa, trước đó ta đã nói (Hình 6-3) rằng tốt nhất là chia khoảng các giá trị băm khả dĩ thành các khoảng và gán mỗi khoảng cho một phân vùng (ví dụ, gán khóa cho phân vùng 0 nếu $0 \leq \text{hash}(\text{key}) < b_0$, cho phân vùng 1 nếu $b_0 \leq \text{hash}(\text{key}) < b_1$, v.v.).

Perhaps you wondered why we don't just use mod (the % operator in many programming languages). For example, $\text{hash}(\text{key}) \bmod 10$ would return a number between 0 and 9 (if we write the hash as a decimal number, the hash mod 10 would be the last digit). If we have 10 nodes, numbered 0 to 9, that seems like an easy way of assigning each key to a node.

Có lẽ bạn từng thắc mắc tại sao ta không dùng luôn phép mod (toán tử % trong nhiều ngôn ngữ lập trình). Ví dụ, $\text{hash}(\text{key}) \bmod 10$ sẽ trả về một con số giữa 0 và 9 (nếu ta viết giá trị băm dưới dạng số thập phân, thì giá trị băm mod 10 sẽ là chữ số cuối cùng). Nếu ta có 10 nút, được đánh số từ 0 đến 9, thì điều đó có vẻ là một cách dễ dàng để gán mỗi khóa cho một nút.

The problem with the mod N approach is that if the number of nodes N changes, most of the keys will need to be moved from one node to another. For example, say $\text{hash}(\text{key}) = 123456$. If you initially have 10 nodes, that key starts out on node 6 (because $123456 \bmod 10 = 6$). When you grow to 11 nodes, the key needs to move to node 3 ($123456 \bmod 11 = 3$), and when you grow to 12 nodes, it needs to move to node 0 ($123456 \bmod 12 = 0$). Such frequent moves make rebalancing excessively expensive.

Vấn đề với cách tiếp cận mod N là nếu số nút N thay đổi, thì hầu hết các khóa sẽ cần được di chuyển từ nút này sang nút khác. Ví dụ, giả sử $\text{hash}(\text{key}) = 123456$. Nếu ban đầu bạn có 10 nút, thì khóa đó khởi đầu ở nút 6 (vì $123456 \bmod 10 = 6$). Khi bạn mở rộng lên 11 nút, khóa cần chuyển sang nút 3 ($123456 \bmod 11 = 3$), và khi bạn mở rộng lên 12 nút, nó cần chuyển sang nút 0 ($123456 \bmod 12 = 0$). Những lần di chuyển thường xuyên như vậy khiến việc tái cân bằng trở nên đắt đỏ quá mức.

We need an approach that doesn't move data around more than necessary.

Ta cần một cách tiếp cận không di chuyển dữ liệu lòng vòng nhiều hơn mức cần thiết.

Fixed number of partitions — Số phân vùng cố định

Fortunately, there is a fairly simple solution: create many more partitions than there are nodes, and assign several partitions to each node. For example, a database running on a cluster of 10 nodes may be split into 1,000 partitions from the outset so that approximately 100 partitions are assigned to each node.

May mắn là có một giải pháp khá đơn giản: tạo ra nhiều phân vùng hơn hẳn so với số nút, và gán vài phân vùng cho mỗi nút. Ví dụ, một cơ sở dữ liệu chạy trên một cụm 10 nút có

thể được chia thành 1.000 phân vùng ngay từ đầu, sao cho khoảng 100 phân vùng được gán cho mỗi nút.

Now, if a node is added to the cluster, the new node can steal a few partitions from every existing node until partitions are fairly distributed once again. This process is illustrated in Figure 6-6. If a node is removed from the cluster, the same happens in reverse.

Bây giờ, nếu một nút được thêm vào cụm, nút mới có thể giành lấy một ít phân vùng từ mỗi nút hiện có cho đến khi các phân vùng lại được phân bố công bằng. Quá trình này được minh họa trong Hình 6-6. Nếu một nút bị bỏ khỏi cụm, điều ngược lại sẽ diễn ra.

Only entire partitions are moved between nodes. The number of partitions does not change, nor does the assignment of keys to partitions. The only thing that changes is the assignment of partitions to nodes. This change of assignment is not immediate—it takes some time to transfer a large amount of data over the network—so the old assignment of partitions is used for any reads and writes that happen while the transfer is in progress.

Chỉ có các phân vùng nguyên vẹn được di chuyển giữa các nút. Số phân vùng không thay đổi, việc gán khóa cho phân vùng cũng không thay đổi. Thứ duy nhất thay đổi là việc gán phân vùng cho nút. Sự thay đổi gán này không tức thời — phải mất một khoảng thời gian để truyền một lượng lớn dữ liệu qua mạng — nên cách gán phân vùng cũ vẫn được dùng cho mọi thao tác đọc và ghi diễn ra trong khi quá trình truyền đang tiến hành.

Figure 6-6. Adding a new node to a database cluster with multiple partitions per node. — Hình 6-6. Thêm một nút mới vào một cụm cơ sở dữ liệu với nhiều phân vùng trên mỗi nút.

In principle, you can even account for mismatched hardware in your cluster: by assigning more partitions to nodes that are more powerful, you can force those nodes to take a greater share of the load.

Về nguyên tắc, bạn thậm chí có thể tính đến phần cứng không đồng đều trong cụm của mình: bằng cách gán nhiều phân vùng hơn cho những nút mạnh hơn, bạn có thể buộc những nút đó gánh một phần tải lớn hơn.

This approach to rebalancing is used in Riak [15], Elasticsearch [24], Couchbase [10], and Voldemort [25].

Cách tiếp cận tái cân bằng này được dùng trong Riak [15], Elasticsearch [24], Couchbase [10], và Voldemort [25].

In this configuration, the number of partitions is usually fixed when the database is first set up and not changed afterward. Although in principle it's possible to split and merge partitions (see the next section), a fixed number of partitions is operationally simpler, and so many fixed-partition databases choose not to implement partition splitting. Thus, the number of partitions configured at the outset is the maximum number of nodes you can have, so you need to choose it high enough to accommodate future growth. However, each partition also has management overhead, so it's counterproductive to choose too high a number.

Trong cấu hình này, số phân vùng thường được cố định khi cơ sở dữ liệu được thiết lập lần đầu và không thay đổi về sau. Mặc dù về nguyên tắc có thể tách và gộp các phân vùng (xem phần tiếp theo), một số phân vùng cố định thì đơn giản hơn về mặt vận hành, nên nhiều cơ sở dữ liệu phân vùng cố định chọn không hiện thực việc tách phân vùng. Do đó, số phân vùng được cấu hình ngay từ đầu chính là số nút tối đa mà bạn có thể có, nên bạn cần chọn nó đủ cao để đáp ứng tăng trưởng trong tương lai. Tuy nhiên, mỗi phân vùng cũng có chi phí quản lý, nên việc chọn một con số quá cao sẽ phản tác dụng.

Choosing the right number of partitions is difficult if the total size of the dataset is highly variable (for example, if it starts small but may grow much larger over time). Since each partition contains a fixed fraction of the total data, the size of each partition grows proportionally to the total amount of data in the cluster. If partitions are very large, rebalancing and recovery from node failures become expensive. But if partitions are too small, they incur too much overhead. The best performance is achieved when the size of partitions is “just right,” neither too big nor too small, which can be hard to achieve if the number of partitions is fixed but the dataset size varies.

Việc chọn đúng số phân vùng là khó nếu tổng kích thước của tập dữ liệu biến động mạnh (ví dụ, nếu nó khởi đầu nhỏ nhưng có thể lớn lên rất nhiều theo thời gian). Vì mỗi phân vùng chứa một phần cố định của tổng dữ liệu, nên kích thước của mỗi phân vùng tăng tỉ lệ với tổng lượng dữ liệu trong cụm. Nếu các phân vùng rất lớn, thì việc tái cân bằng và phục hồi từ các sự cố nút trở nên đắt đỏ. Nhưng nếu các phân vùng quá nhỏ, thì chúng phát sinh quá nhiều chi phí. Hiệu năng tốt nhất đạt được khi kích thước phân vùng “vừa phải,” không quá to cũng không quá nhỏ, điều có thể khó đạt được nếu số phân vùng là cố định nhưng kích thước tập dữ liệu lại biến động.

Dynamic partitioning — Phân vùng động

For databases that use key range partitioning (see “Partitioning by Key Range”), a fixed number of partitions with fixed boundaries would be very inconvenient: if you got the boundaries wrong, you could end up with all of the data in one partition and all of the other partitions empty. Reconfiguring the partition boundaries manually would be very tedious.

Với các cơ sở dữ liệu dùng phân vùng theo khoảng khóa (xem “Phân vùng theo khoảng khóa”), một số phân vùng cố định với các ranh giới cố định sẽ rất bất tiện: nếu bạn đặt sai ranh giới, bạn có thể rất cuộc có toàn bộ dữ liệu nằm trong một phân vùng và tất cả các phân vùng khác đều rỗng. Việc cấu hình lại các ranh giới phân vùng một cách thủ công sẽ rất nhọc nhằn.

For that reason, key range-partitioned databases such as HBase and RethinkDB create partitions dynamically. When a partition grows to exceed a configured size (on HBase, the default is 10 GB), it is split into two partitions so that approximately half of the data ends up on each side of the split [26]. Conversely, if lots of data is deleted and a partition shrinks below some threshold, it can be merged with an adjacent partition. This process is similar to what happens at the top level of a B-tree (see “B-Trees”).

Vì lý do đó, các cơ sở dữ liệu phân vùng theo khoảng khóa như HBase và RethinkDB tạo các phân vùng một cách động. Khi một phân vùng lớn lên vượt quá một kích thước được cấu hình (trên HBase, mặc định là 10 GB), nó được tách thành hai phân vùng sao cho khoảng một nửa dữ liệu nằm ở mỗi bên của chỗ tách [26]. Ngược lại, nếu nhiều dữ liệu bị xóa và một phân vùng co lại dưới một ngưỡng nào đó, nó có thể được gộp với một phân vùng kề bên. Quá trình này tương tự những gì xảy ra ở tầng trên cùng của một cây B-tree (xem “B-Trees”).

Each partition is assigned to one node, and each node can handle multiple partitions, like in the case of a fixed number of partitions. After a large partition has been split, one of its two halves can be transferred to another node in order to balance the load. In the case of HBase, the transfer of partition files happens through HDFS, the underlying distributed filesystem [3].

Mỗi phân vùng được gán cho một nút, và mỗi nút có thể xử lý nhiều phân vùng, giống như trong trường hợp số phân vùng cố định. Sau khi một phân vùng lớn được tách, một trong hai nửa của nó có thể được chuyển sang một nút khác để cân bằng tải. Trong trường hợp

HBase, việc chuyển các tệp phân vùng diễn ra qua HDFS, hệ thống tệp phân tán nền tảng [3].

An advantage of dynamic partitioning is that the number of partitions adapts to the total data volume. If there is only a small amount of data, a small number of partitions is sufficient, so overheads are small; if there is a huge amount of data, the size of each individual partition is limited to a configurable maximum [23].

Một lợi thế của phân vùng động là số phân vùng thích nghi với tổng dung lượng dữ liệu. Nếu chỉ có một lượng nhỏ dữ liệu, thì một số ít phân vùng là đủ, nên chi phí nhỏ; nếu có một lượng dữ liệu khổng lồ, thì kích thước của mỗi phân vùng riêng lẻ bị giới hạn ở một mức tối đa có thể cấu hình [23].

However, a caveat is that an empty database starts off with a single partition, since there is no a priori information about where to draw the partition boundaries. While the dataset is small—until it hits the point at which the first partition is split—all writes have to be processed by a single node while the other nodes sit idle. To mitigate this issue, HBase and MongoDB allow an initial set of partitions to be configured on an empty database (this is called pre-splitting). In the case of key-range partitioning, pre-splitting requires that you already know what the key distribution is going to look like [4, 26].

Tuy nhiên, một điểm cần lưu ý là một cơ sở dữ liệu rỗng khởi đầu với một phân vùng duy nhất, vì không có thông tin tiên nghiệm về việc vạch các ranh giới phân vùng ở đâu. Trong khi tập dữ liệu còn nhỏ — cho đến khi nó chạm tới điểm mà phân vùng đầu tiên bị tách — mọi thao tác ghi đều phải được xử lý bởi một nút duy nhất trong khi các nút khác ngồi không. Để giảm nhẹ vấn đề này, HBase và MongoDB cho phép cấu hình một tập phân vùng ban đầu trên một cơ sở dữ liệu rỗng (điều này gọi là tách trước — pre-splitting). Trong trường hợp phân vùng theo khoảng khóa, việc tách trước đòi hỏi bạn phải biết trước phân bố khóa sẽ trông như thế nào [4, 26].

Dynamic partitioning is not only suitable for key range-partitioned data, but can equally well be used with hash-partitioned data. MongoDB since version 2.4 supports both key-range and hash partitioning, and it splits partitions dynamically in either case.

Phân vùng động không chỉ phù hợp cho dữ liệu phân vùng theo khoảng khóa, mà cũng có thể được dùng tốt như vậy với dữ liệu phân vùng theo băm. MongoDB từ phiên bản 2.4 hỗ trợ cả phân vùng theo khoảng khóa lẫn phân vùng theo băm, và nó tách các phân vùng một cách động trong cả hai trường hợp.

Partitioning proportionally to nodes — Phân vùng tỉ lệ với số nút

With dynamic partitioning, the number of partitions is proportional to the size of the dataset, since the splitting and merging processes keep the size of each partition between some fixed minimum and maximum. On the other hand, with a fixed number of partitions, the size of each partition is proportional to the size of the dataset. In both of these cases, the number of partitions is independent of the number of nodes.

Với phân vùng động, số phân vùng tỉ lệ với kích thước của tập dữ liệu, vì các quá trình tách và gộp giữ kích thước của mỗi phân vùng nằm giữa một mức tối thiểu và tối đa cố định nào đó. Mặt khác, với một số phân vùng cố định, kích thước của mỗi phân vùng tỉ lệ với kích thước của tập dữ liệu. Trong cả hai trường hợp này, số phân vùng độc lập với số nút.

A third option, used by Cassandra and Ketama, is to make the number of partitions proportional to the number of nodes—in other words, to have a fixed number of partitions per node [23, 27, 28].

In this case, the size of each partition grows proportionally to the dataset size while the number of nodes remains unchanged, but when you increase the number of nodes, the partitions become smaller again. Since a larger data volume generally requires a larger number of nodes to store, this approach also keeps the size of each partition fairly stable.

Một lựa chọn thứ ba, được Cassandra và Ketama dùng, là làm cho số phân vùng tỉ lệ với số nút — nói cách khác, có một số phân vùng cố định trên mỗi nút [23, 27, 28]. Trong trường hợp này, kích thước của mỗi phân vùng tăng tỉ lệ với kích thước tập dữ liệu trong khi số nút giữ nguyên, nhưng khi bạn tăng số nút, các phân vùng lại trở nên nhỏ hơn. Vì một lượng dữ liệu lớn hơn nói chung đòi hỏi một số nút lớn hơn để lưu trữ, nên cách tiếp cận này cũng giữ cho kích thước của mỗi phân vùng khá ổn định.

When a new node joins the cluster, it randomly chooses a fixed number of existing partitions to split, and then takes ownership of one half of each of those split partitions while leaving the other half of each partition in place. The randomization can produce unfair splits, but when averaged over a larger number of partitions (in Cassandra, 256 partitions per node by default), the new node ends up taking a fair share of the load from the existing nodes. Cassandra 3.0 introduced an alternative rebalancing algorithm that avoids unfair splits [29].

Khi một nút mới gia nhập cụm, nó chọn ngẫu nhiên một số lượng cố định các phân vùng hiện có để tách, rồi nắm quyền sở hữu một nửa của mỗi phân vùng bị tách đó trong khi để nửa còn lại của mỗi phân vùng nguyên tại chỗ. Việc chọn ngẫu nhiên có thể tạo ra những chỗ tách không công bằng, nhưng khi lấy trung bình trên một số lượng phân vùng lớn hơn (trong Cassandra, mặc định là 256 phân vùng trên mỗi nút), nút mới rớt cuộc gánh một phần tải công bằng từ các nút hiện có. Cassandra 3.0 đã giới thiệu một thuật toán tái cân bằng thay thế tránh được những chỗ tách không công bằng [29].

Picking partition boundaries randomly requires that hash-based partitioning is used (so the boundaries can be picked from the range of numbers produced by the hash function). Indeed, this approach corresponds most closely to the original definition of consistent hashing [7] (see “Consistent Hashing”). Newer hash functions can achieve a similar effect with lower metadata overhead [8].

Việc chọn ranh giới phân vùng một cách ngẫu nhiên đòi hỏi phải dùng phân vùng dựa trên băm (để các ranh giới có thể được chọn từ khoảng số mà hàm băm sinh ra). Quả thực, cách tiếp cận này tương ứng sát nhất với định nghĩa nguyên thủy của băm nhất quán [7] (xem “Băm nhất quán”). Các hàm băm mới hơn có thể đạt được hiệu quả tương tự với chi phí siêu dữ liệu thấp hơn [8].

Operations: Automatic or Manual Rebalancing — Vận hành: Tái cân bằng tự động hay thủ công

There is one important question with regard to rebalancing that we have glossed over: does the rebalancing happen automatically or manually?

Có một câu hỏi quan trọng về việc tái cân bằng mà ta đã lướt qua: việc tái cân bằng diễn ra tự động hay thủ công?

There is a gradient between fully automatic rebalancing (the system decides automatically when to move partitions from one node to another, without any administrator interaction) and fully manual (the assignment of partitions to nodes is explicitly configured by an administrator, and only changes when the administrator explicitly reconfigures it). For example, Couchbase, Riak, and Voldemort

generate a suggested partition assignment automatically, but require an administrator to commit it before it takes effect.

Có một dải mức độ nằm giữa việc tái cân bằng hoàn toàn tự động (hệ thống tự động quyết định khi nào di chuyển các phân vùng từ nút này sang nút khác, mà không có bất kỳ sự can thiệp nào của quản trị viên) và hoàn toàn thủ công (việc gán phân vùng cho nút được quản trị viên cấu hình tường minh, và chỉ thay đổi khi quản trị viên cấu hình lại nó một cách tường minh). Ví dụ, Couchbase, Riak, và Voldemort tự động sinh ra một đề xuất gán phân vùng, nhưng đòi hỏi một quản trị viên xác nhận trước khi nó có hiệu lực.

Fully automated rebalancing can be convenient, because there is less operational work to do for normal maintenance. However, it can be unpredictable. Rebalancing is an expensive operation, because it requires rerouting requests and moving a large amount of data from one node to another. If it is not done carefully, this process can overload the network or the nodes and harm the performance of other requests while the rebalancing is in progress.

Việc tái cân bằng hoàn toàn tự động có thể tiện lợi, vì có ít việc vận hành phải làm cho bảo trì thông thường hơn. Tuy nhiên, nó có thể khó lường. Tái cân bằng là một thao tác đắt đỏ, vì nó đòi hỏi định tuyến lại các yêu cầu và di chuyển một lượng lớn dữ liệu từ nút này sang nút khác. Nếu không được làm cẩn thận, quá trình này có thể làm quá tải mạng hoặc các nút và gây hại cho hiệu năng của các yêu cầu khác trong khi việc tái cân bằng đang tiến hành.

Such automation can be dangerous in combination with automatic **failure** detection. For example, say one node is overloaded and is temporarily slow to respond to requests. The other nodes conclude that the overloaded node is dead, and automatically rebalance the cluster to move load away from it. This puts additional load on the overloaded node, other nodes, and the network—making the situation worse and potentially causing a **cascading failure**.

Sự tự động hóa như vậy có thể nguy hiểm khi kết hợp với việc tự động phát hiện sự cố. Ví dụ, giả sử một nút bị quá tải và tạm thời phản hồi yêu cầu chậm. Các nút khác kết luận rằng nút quá tải đó đã chết, và tự động tái cân bằng cụm để chuyển tải đi khỏi nó. Điều này đặt thêm tải lên nút đã quá tải, lên các nút khác, và lên mạng — khiến tình hình tồi tệ hơn và có khả năng gây ra một sự cố dây chuyền.

For that reason, it can be a good thing to have a human in the loop for rebalancing. It's slower than a fully automatic process, but it can help prevent operational surprises.

Vì lý do đó, có một con người trong vòng lặp khi tái cân bằng có thể là một điều tốt. Nó chậm hơn một quá trình hoàn toàn tự động, nhưng nó có thể giúp ngăn những bất ngờ trong vận hành.

Request Routing — Định tuyến yêu cầu

We have now partitioned our dataset across multiple nodes running on multiple machines. But there remains an open question: when a client wants to make a request, how does it know which node to connect to? As partitions are rebalanced, the assignment of partitions to nodes changes. Somebody needs to stay on top of those changes in order to answer the question: if I want to read or write the key “foo”, which IP address and port number do I need to connect to?

Giờ ta đã phân vùng tập dữ liệu của mình trên nhiều nút chạy trên nhiều máy. Nhưng vẫn còn một câu hỏi bỏ ngỏ: khi một client muốn gửi một yêu cầu, làm thế nào nó biết phải kết nối tới nút nào? Khi các phân vùng được tái cân bằng, việc gán phân vùng cho nút thay đổi. Ai đó cần nắm bắt được những thay đổi này để trả lời câu hỏi: nếu tôi muốn đọc hoặc ghi khóa “foo”, tôi cần kết nối tới địa chỉ IP và số cổng nào?

This is an **instance** of a more general problem called service discovery, which isn’t limited to just databases. Any piece of software that is accessible over a network has this problem, especially if it is aiming for **high availability** (running in a redundant configuration on multiple machines). Many companies have written their own in-house service discovery tools, and many of these have been released as open source [30].

Đây là một trường hợp của một vấn đề tổng quát hơn gọi là khám phá dịch vụ (service discovery), vốn không chỉ giới hạn ở cơ sở dữ liệu. Bất kỳ phần mềm nào truy cập được qua mạng đều có vấn đề này, đặc biệt nếu nó nhắm tới tính sẵn sàng cao (chạy trong một cấu hình dự phòng trên nhiều máy). Nhiều công ty đã viết các công cụ khám phá dịch vụ nội bộ của riêng họ, và nhiều công cụ trong số này đã được phát hành dưới dạng mã nguồn mở [30].

On a high level, there are a few different approaches to this problem (illustrated in Figure 6-7):

Ở mức cao, có vài cách tiếp cận khác nhau cho vấn đề này (minh họa trong Hình 6-7):

- Allow clients to contact any node (e.g., via a round-robin load balancer). If that node coincidentally owns the partition to which the request applies, it can handle the request directly; otherwise, it forwards the request to the appropriate node, receives the reply, and passes the reply along to the client.
- Send all requests from clients to a routing tier first, which determines the node that should handle each request and forwards it accordingly. This routing tier does not itself handle any requests; it only acts as a partition-aware load balancer.
- Require that clients be aware of the partitioning and the assignment of partitions to nodes. In this case, a client can connect directly to the appropriate node, without any intermediary.
 - Cho phép các client liên hệ với bất kỳ nút nào (ví dụ, qua một bộ cân bằng tải kiểu

round-robin). Nếu nút đó tình cờ sở hữu phân vùng mà yêu cầu áp dụng tới, nó có thể xử lý yêu cầu trực tiếp; ngược lại, nó chuyển tiếp yêu cầu tới nút thích hợp, nhận về phản hồi, và chuyển phản hồi đó lại cho client.

- Gửi tất cả các yêu cầu từ client tới một tầng định tuyến (routing tier) trước, tầng này xác định nút nào nên xử lý mỗi yêu cầu rồi chuyển tiếp tương ứng. Bản thân tầng định tuyến này không xử lý bất kỳ yêu cầu nào; nó chỉ đóng vai trò một bộ cân bằng tải có nhận biết phân vùng.
- Đòi hỏi các client phải nhận biết được việc phân vùng và việc gán phân vùng cho nút. Trong trường hợp này, một client có thể kết nối trực tiếp tới nút thích hợp, không qua bất kỳ trung gian nào.

In all cases, the key problem is: how does the component making the routing decision (which may be one of the nodes, or the routing tier, or the client) learn about changes in the assignment of partitions to nodes?

Trong mọi trường hợp, vấn đề mấu chốt là: làm thế nào thành phần đưa ra quyết định định tuyến (có thể là một trong các nút, hoặc tầng định tuyến, hoặc client) biết được về những thay đổi trong việc gán phân vùng cho nút?

Figure 6-7. Three different ways of routing a request to the right node. — Hình 6-7. Ba cách khác nhau để định tuyến một yêu cầu tới đúng nút. This is a challenging problem, because it is important that all participants agree—otherwise requests would be sent to the wrong nodes and not handled correctly. There are protocols for achieving consensus in a distributed system, but they are hard to implement correctly (see Chapter 9).

Đây là một vấn đề hóc búa, vì điều quan trọng là tất cả những bên tham gia đều phải đồng thuận — nếu không thì các yêu cầu sẽ bị gửi tới sai nút và không được xử lý đúng. Có những giao thức để đạt được đồng thuận trong một hệ thống phân tán, nhưng chúng khó hiện thực một cách đúng đắn (xem Chương 9).

Many distributed data systems rely on a separate coordination service such as ZooKeeper to keep track of this cluster metadata, as illustrated in Figure 6-8. Each node registers itself in ZooKeeper, and ZooKeeper maintains the authoritative mapping of partitions to nodes. Other actors, such as the routing tier or the partitioning-aware client, can subscribe to this information in ZooKeeper. Whenever a partition changes ownership, or a node is added or removed, ZooKeeper notifies the routing tier so that it can keep its routing information up to date.

Nhiều hệ thống dữ liệu phân tán dựa vào một dịch vụ điều phối riêng biệt như ZooKeeper để theo dõi siêu dữ liệu cụm này, như minh họa trong Hình 6-8. Mỗi nút tự đăng ký mình trong ZooKeeper, và ZooKeeper duy trì ánh xạ có thẩm quyền giữa phân vùng và nút. Các tác nhân khác, chẳng hạn tầng định tuyến hoặc client có nhận biết phân vùng, có thể đăng ký nhận thông tin này trong ZooKeeper. Mỗi khi một phân vùng đổi chủ sở hữu, hoặc một nút được thêm vào hay bị bỏ đi, ZooKeeper thông báo cho tầng định tuyến để nó có thể giữ thông tin định tuyến của mình luôn cập nhật.

Figure 6-8. Using ZooKeeper to keep track of assignment of partitions to nodes. — Hình 6-8. Dùng ZooKeeper để theo dõi việc gán phân vùng cho nút. For example, LinkedIn’s Espresso uses Helix [31] for cluster management (which in turn relies on ZooKeeper), implementing a routing tier as shown in Figure 6-8. HBase, SolrCloud, and Kafka also use ZooKeeper to track partition assignment. MongoDB has a similar architecture, but it relies on its own config server implementation and mongos daemons as the routing tier.

Ví dụ, Espresso của LinkedIn dùng Helix [31] cho việc quản lý cụm (mà Helix thì lại dựa vào ZooKeeper), hiện thực một tầng định tuyến như minh họa trong Hình 6-8. HBase, SolrCloud, và Kafka cũng dùng ZooKeeper để theo dõi việc gán phân vùng. MongoDB có một kiến trúc tương tự, nhưng nó dựa vào hiện thực config server riêng của mình và các daemon mongos làm tầng định tuyến.

Cassandra and Riak take a different approach: they use a gossip protocol among the nodes to disseminate any changes in cluster state. Requests can be sent to any node, and that node forwards them to the appropriate node for the requested partition (approach 1 in Figure 6-7). This model puts more complexity in the database nodes but avoids the dependency on an external coordination service such as ZooKeeper.

Cassandra và Riak dùng một cách tiếp cận khác: chúng dùng một giao thức gossip giữa các nút để lan truyền mọi thay đổi trong trạng thái cụm. Các yêu cầu có thể được gửi tới bất kỳ nút nào, và nút đó chuyển tiếp chúng tới nút thích hợp cho phân vùng được yêu cầu (cách tiếp cận 1 trong Hình 6-7). Mô hình này đặt nhiều độ phức tạp hơn vào các nút cơ sở dữ liệu nhưng tránh được sự phụ thuộc vào một dịch vụ điều phối bên ngoài như ZooKeeper.

Couchbase does not rebalance automatically, which simplifies the design. Normally it is configured with a routing tier called moxi, which learns about routing changes from the cluster nodes [32].

Couchbase không tái cân bằng tự động, điều này làm đơn giản hóa thiết kế. Thông thường nó được cấu hình với một tầng định tuyến gọi là moxi, vốn học về các thay đổi định tuyến từ các nút trong cụm [32].

When using a routing tier or when sending requests to a random node, clients still need to find the IP addresses to connect to. These are not as fast-changing as the assignment of partitions to nodes, so it is often sufficient to use DNS for this purpose.

Khi dùng một tầng định tuyến hoặc khi gửi các yêu cầu tới một nút ngẫu nhiên, các client vẫn cần tìm các địa chỉ IP để kết nối tới. Những địa chỉ này không thay đổi nhanh như việc gán phân vùng cho nút, nên thường dùng DNS cho mục đích này là đủ.

Parallel Query Execution — Thực thi truy vấn song song

So far we have focused on very simple queries that read or write a single key (plus scatter/gather queries in the case of document-partitioned secondary indexes). This is about the level of access supported by most NoSQL distributed datastores.

Cho tới giờ ta đã tập trung vào những truy vấn rất đơn giản đọc hoặc ghi một khóa duy nhất (cộng thêm các truy vấn phát tán/thu gom trong trường hợp chỉ mục phụ phân vùng theo tài liệu). Đây gần như là mức độ truy cập mà hầu hết các kho dữ liệu phân tán NoSQL hỗ trợ.

However, massively parallel processing (MPP) relational database products, often used for analytics, are much more sophisticated in the types of queries they support. A typical data warehouse query contains several **join**, filtering, grouping, and aggregation operations. The MPP **query optimizer** breaks this complex query into a number of execution stages and partitions, many of which can be executed in parallel on different nodes of the database cluster. Queries that involve scanning over large parts of the dataset particularly benefit from such **parallel execution**.

Tuy nhiên, các sản phẩm cơ sở dữ liệu quan hệ xử lý song song quy mô lớn (MPP), thường được dùng cho phân tích, lại tinh vi hơn nhiều về các loại truy vấn mà chúng hỗ trợ. Một truy vấn kho dữ liệu điển hình chứa nhiều thao tác join, lọc, gom nhóm, và tổng hợp. Bộ

tối ưu truy vấn MPP chia truy vấn phức tạp này thành một số giai đoạn thực thi và phân vùng, mà nhiều trong số đó có thể được thực thi song song trên các nút khác nhau của cụm cơ sở dữ liệu. Những truy vấn liên quan tới việc quét trên những phần lớn của tập dữ liệu đặc biệt được hưởng lợi từ kiểu thực thi song song như vậy.

Fast parallel execution of data warehouse queries is a specialized topic, and given the business importance of analytics, it receives a lot of commercial interest. We will discuss some techniques for parallel query execution in Chapter 10. For a more detailed overview of techniques used in parallel databases, please see the references [1, 33].

Việc thực thi nhanh và song song các truy vấn kho dữ liệu là một chủ đề chuyên môn, và do tầm quan trọng nghiệp vụ của phân tích, nó nhận được rất nhiều sự quan tâm thương mại. Ta sẽ bàn về một số kỹ thuật cho việc thực thi truy vấn song song ở Chương 10. Để có cái nhìn tổng quan chi tiết hơn về các kỹ thuật được dùng trong các cơ sở dữ liệu song song, vui lòng xem các tài liệu tham khảo [1, 33].

Summary — Tóm tắt

In this chapter we explored different ways of partitioning a large dataset into smaller subsets. Partitioning is necessary when you have so much data that storing and processing it on a single machine is no longer feasible.

Trong chương này ta đã khám phá những cách khác nhau để phân vùng một tập dữ liệu lớn thành các tập con nhỏ hơn. Phân vùng là cần thiết khi bạn có quá nhiều dữ liệu đến mức việc lưu trữ và xử lý nó trên một máy duy nhất không còn khả thi.

The goal of partitioning is to spread the data and query load evenly across multiple machines, avoiding hot spots (nodes with disproportionately high load). This requires choosing a partitioning scheme that is appropriate to your data, and rebalancing the partitions when nodes are added to or removed from the cluster.

Mục tiêu của việc phân vùng là trải đều dữ liệu và tải truy vấn trên nhiều máy, tránh các điểm nóng (những nút có tải cao một cách không cân xứng). Điều này đòi hỏi chọn một sơ đồ phân vùng phù hợp với dữ liệu của bạn, và tái cân bằng các phân vùng khi các nút được thêm vào hoặc bị bỏ khỏi cụm.

We discussed two main approaches to partitioning:

Ta đã bàn về hai cách tiếp cận chính để phân vùng:

- Key range partitioning, where keys are sorted, and a partition owns all the keys from some minimum up to some maximum. Sorting has the advantage that efficient range queries are possible, but there is a risk of hot spots if the application often accesses keys that are close together in the sorted order. In this approach, partitions are typically rebalanced dynamically by splitting the range into two subranges when a partition gets too big.
- Hash partitioning, where a hash function is applied to each key, and a partition owns a range of hashes. This method destroys the ordering of keys, making range queries inefficient, but may distribute load more evenly. When partitioning by hash, it is common to create a fixed number of partitions in advance, to assign several partitions to each node, and to move entire partitions from one node to another when nodes are added or removed. Dynamic partitioning can also be used.
- Phân vùng theo khoảng khóa, trong đó các khóa được sắp xếp, và một phân vùng sở hữu tất cả các khóa từ một giá trị nhỏ nhất nào đó cho tới một giá trị lớn nhất nào đó. Việc sắp xếp có lợi thế là cho phép các truy vấn khoảng hiệu quả, nhưng có một rủi ro về điểm nóng nếu ứng dụng thường xuyên truy cập các khóa nằm gần nhau trong thứ tự đã sắp xếp. Trong cách tiếp cận này, các phân vùng thường được tái cân bằng động bằng cách tách khoảng thành hai khoảng con khi một phân vùng trở nên quá lớn.
- Phân vùng theo băm, trong đó một hàm băm được áp dụng cho mỗi khóa, và một

phân vùng sở hữu một khoảng các giá trị băm. Phương pháp này phá hủy thứ tự của các khóa, khiến các truy vấn khoảng kém hiệu quả, nhưng có thể phân bố tải đều hơn. Khi phân vùng theo băm, thường người ta tạo trước một số phân vùng cố định, gán vài phân vùng cho mỗi nút, và di chuyển nguyên các phân vùng từ nút này sang nút khác khi các nút được thêm vào hoặc bị bỏ đi. Phân vùng động cũng có thể được dùng.

Hybrid approaches are also possible, for example with a compound key: using one part of the key to identify the partition and another part for the sort order.

Các cách tiếp cận lai cũng khả thi, ví dụ với một khóa phức hợp: dùng một phần của khóa để định danh phân vùng và một phần khác cho thứ tự sắp xếp.

We also discussed the interaction between partitioning and secondary indexes. A secondary index also needs to be partitioned, and there are two methods:

Ta cũng đã bàn về sự tương tác giữa phân vùng và chỉ mục phụ. Một chỉ mục phụ cũng cần được phân vùng, và có hai phương pháp:

- Document-partitioned indexes (local indexes), where the secondary indexes are stored in the same partition as the primary key and value. This means that only a single partition needs to be updated on write, but a read of the secondary index requires a scatter/gather across all partitions.
- Term-partitioned indexes (global indexes), where the secondary indexes are partitioned separately, using the indexed values. An entry in the secondary index may include records from all partitions of the primary key. When a document is written, several partitions of the secondary index need to be updated; however, a read can be served from a single partition.
 - Chỉ mục phân vùng theo tài liệu (chỉ mục cục bộ), trong đó các chỉ mục phụ được lưu trong cùng phân vùng với khóa chính và giá trị. Điều này nghĩa là chỉ một phân vùng duy nhất cần được cập nhật khi ghi, nhưng một thao tác đọc chỉ mục phụ đòi hỏi một phép phát tán/thu gom trên tất cả các phân vùng.
 - Chỉ mục phân vùng theo thuật ngữ (chỉ mục toàn cục), trong đó các chỉ mục phụ được phân vùng riêng, dùng các giá trị đã được lập chỉ mục. Một mục trong chỉ mục phụ có thể bao gồm các bản ghi từ tất cả các phân vùng của khóa chính. Khi một tài liệu được ghi, vài phân vùng của chỉ mục phụ cần được cập nhật; tuy nhiên, một thao tác đọc có thể được phục vụ từ một phân vùng duy nhất.

Finally, we discussed techniques for routing queries to the appropriate partition, which range from simple partition-aware load balancing to sophisticated parallel query execution engines.

Cuối cùng, ta đã bàn về các kỹ thuật để định tuyến các truy vấn tới phân vùng thích hợp, trải từ việc cân bằng tải có nhận biết phân vùng đơn giản cho tới các engine thực thi truy vấn song song tinh vi.

By design, every partition operates mostly independently—that’s what allows a partitioned database to scale to multiple machines. However, operations that need to write to several partitions can be difficult to reason about: for example, what happens if the write to one partition succeeds, but another fails? We will address that question in the following chapters.

Theo thiết kế, mỗi phân vùng vận hành phần lớn một cách độc lập — đó là điều cho phép một cơ sở dữ liệu phân vùng mở rộng ra nhiều máy. Tuy nhiên, các thao tác cần ghi vào nhiều phân vùng có thể khó suy luận về tính đúng đắn: ví dụ, điều gì xảy ra nếu thao tác ghi vào một phân vùng thành công, nhưng một phân vùng khác lại thất bại? Ta sẽ giải quyết câu hỏi đó trong các chương tiếp theo.

Footnotes Partitioning, as discussed in this chapter, is a way of intentionally breaking a large database down into smaller ones. It has nothing to do with network partitions (netsplits), a type of fault in the network between nodes. We will discuss such faults in Chapter 8.

If your database only supports a key-value model, you might be tempted to implement a secondary index yourself by creating a mapping from values to document IDs in application code. If you go down this route, you need to take great care to ensure your indexes remain consistent with the underlying data. Race conditions and intermittent write failures (where some changes were saved but others weren't) can very easily cause the data to go out of sync—see “The need for multi-object transactions”.

References [1] David J. DeWitt and Jim N. Gray: “Parallel Database Systems: The Future of High Performance Database Systems,” *Communications of the ACM*, volume 35, number 6, pages 85–98, June 1992. doi:10.1145/129888.129894

[2] Lars George: “HBase vs. BigTable Comparison,” larsgeorge.com, November 2009.

[3] “The Apache HBase Reference Guide,” Apache Software Foundation, hbase.apache.org, 2014.

[4] MongoDB, Inc.: “New Hash-Based Sharding Feature in MongoDB 2.4,” blog.mongodb.org, April 10, 2013.

[5] Ikai Lan: “App Engine Datastore Tip: Monotonically Increasing Values Are Bad,” ikaisays.com, January 25, 2011.

[6] Martin Kleppmann: “Java’s hashCode Is Not Safe for Distributed Systems,” martin.kleppmann.com, June 18, 2012.

[7] David Karger, Eric Lehman, Tom Leighton, et al.: “Consistent Hashing and Random Trees: Distributed Caching Protocols for Relieving Hot Spots on the World Wide Web,” at 29th Annual ACM Symposium on Theory of Computing (STOC), pages 654–663, 1997. doi:10.1145/258533.258660

[8] John Lamping and Eric Veach: “A Fast, Minimal Memory, Consistent Hash Algorithm,” arxiv.org, June 2014.

[9] Eric Redmond: “A Little Riak Book,” Version 1.4.0, Basho Technologies, September 2013.

[10] “Couchbase 2.5 Administrator Guide,” Couchbase, Inc., 2014.

[11] Avinash Lakshman and Prashant Malik: “Cassandra – A Decentralized Structured Storage System,” at 3rd ACM SIGOPS International Workshop on Large Scale Distributed Systems and Middleware (LADIS), October 2009.

[12] Jonathan Ellis: “Facebook’s Cassandra Paper, Annotated and Compared to Apache Cassandra 2.0,” datastax.com, September 12, 2013.

[13] “Introduction to Cassandra Query Language,” DataStax, Inc., 2014.

[14] Samuel Axon: “3% of Twitter’s Servers Dedicated to Justin Bieber,” mashable.com, September 7, 2010.

[15] “Riak 1.4.8 Docs,” Basho Technologies, Inc., 2014.

[16] Richard Low: “The Sweet Spot for Cassandra Secondary Indexing,” wentnet.com, October 21, 2013.

[17] Zachary Tong: “Customizing Your Document Routing,” elasticsearch.org, June 3, 2013.

[18] “Apache Solr Reference Guide,” Apache Software Foundation, 2014.

- [19] Andrew Pavlo: “H-Store Frequently Asked Questions,” hstore.cs.brown.edu, October 2013.
- [20] “Amazon DynamoDB Developer Guide,” Amazon Web Services, Inc., 2014.
- [21] Rusty Klopheus: “Difference Between 2I and Search,” email to riak-users mailing list, lists.basho.com, October 25, 2011.
- [22] Donald K. Burleson: “Object Partitioning in Oracle,” dba-oracle.com, November 8, 2000.
- [23] Eric Evans: “Rethinking Topology in Cassandra,” at ApacheCon Europe, November 2012.
- [24] Rafał Kuć: “Reroute API Explained,” elasticsearchserverbook.com, September 30, 2013.
- [25] “Project Voldemort Documentation,” project-voldemort.com.
- [26] Enis Soztutar: “Apache HBase Region Splitting and Merging,” hortonworks.com, February 1, 2013.
- [27] Brandon Williams: “Virtual Nodes in Cassandra 1.2,” datastax.com, December 4, 2012.
- [28] Richard Jones: “libketama: Consistent Hashing Library for Memcached Clients,” metabrew.com, April 10, 2007.
- [29] Branimir Lambov: “New Token Allocation Algorithm in Cassandra 3.0,” datastax.com, January 28, 2016.
- [30] Jason Wilder: “Open-Source Service Discovery,” jasonwilder.com, February 2014.
- [31] Kishore Gopalakrishna, Shi Lu, Zhen Zhang, et al.: “Untangling Cluster Management with Helix,” at ACM Symposium on Cloud Computing (SoCC), October 2012. doi:10.1145/2391229.2391248
- [32] “Moxi 1.8 Manual,” Couchbase, Inc., 2014.
- [33] Shivnath Babu and Herodotos Herodotou: “Massively Parallel Databases and MapReduce Systems,” Foundations and Trends in Databases, volume 5, number 1, pages 1–104, November 2013. doi:10.1561/19000000036

Chapter 7. Transactions — Chương 7. Giao dịch

Some authors have claimed that general two-phase commit is too expensive to support, because of the performance or availability problems that it brings. We believe it is better to have application programmers deal with performance problems due to overuse of transactions as bottlenecks arise, rather than always coding around the lack of transactions.

Một số tác giả đã cho rằng giao thức hai pha (two-phase commit) tổng quát quá đắt đỏ để hỗ trợ, vì những vấn đề về hiệu năng hoặc tính sẵn sàng mà nó mang lại. Chúng tôi tin rằng tốt hơn là để lập trình viên ứng dụng xử lý các vấn đề hiệu năng do lạm dụng giao dịch khi nút thắt cổ chai phát sinh, thay vì lúc nào cũng phải viết mã lách qua việc thiếu vắng giao dịch.

James Corbett et al., Spanner: Google's Globally-Distributed **Database** (2012)

James Corbett và cộng sự, Spanner: Google's Globally-Distributed Database (2012)

In the harsh reality of data systems, many things can go wrong:

Trong thực tế khắc nghiệt của các hệ thống dữ liệu, rất nhiều thứ có thể trục trặc:

- The database software or hardware may fail at any time (including in the middle of a write operation).
- The application may crash at any time (including halfway through a series of operations).
- Interruptions in the network can unexpectedly cut off the application from the database, or one database **node** from another.
- Several clients may write to the database at the same time, overwriting each other's changes.
- A **client** may read data that doesn't make sense because it has only partially been updated.
- Race conditions between clients can cause surprising bugs.
 - Phần mềm hoặc phần cứng của cơ sở dữ liệu có thể hỏng bất cứ lúc nào (kể cả ngay giữa một thao tác ghi).
 - Ứng dụng có thể sập bất cứ lúc nào (kể cả khi đang thực hiện dở một chuỗi thao tác).
 - Sự gián đoạn trên mạng có thể bất ngờ cắt đứt kết nối giữa ứng dụng với cơ sở dữ liệu, hoặc giữa một nút cơ sở dữ liệu này với nút khác.
 - Nhiều client có thể cùng ghi vào cơ sở dữ liệu một lúc, ghi đè lên thay đổi của nhau.
 - Một client có thể đọc được dữ liệu vô nghĩa vì nó mới chỉ được cập nhật một phần.
 - Các tình trạng tranh đua (race condition) giữa các client có thể gây ra những bug bất ngờ.

In order to be reliable, a system has to deal with these faults and ensure that they don't cause catastrophic **failure** of the entire system. However, implementing **fault**-tolerance mechanisms is

a lot of work. It requires a lot of careful thinking about all the things that can go wrong, and a lot of testing to ensure that the solution actually works.

Để đáng tin cậy, một hệ thống phải xử lý được những lỗi này và đảm bảo chúng không gây ra hỏng hóc thảm khốc cho toàn bộ hệ thống. Tuy nhiên, hiện thực các cơ chế chịu lỗi tốn rất nhiều công sức. Nó đòi hỏi suy nghĩ thấu đáo về mọi thứ có thể trục trặc, và rất nhiều kiểm thử để đảm bảo giải pháp thực sự hoạt động.

For decades, transactions have been the mechanism of choice for simplifying these issues. A transaction is a way for an application to group several reads and writes together into a logical unit. Conceptually, all the reads and writes in a transaction are executed as one operation: either the entire transaction succeeds (commit) or it fails (abort, rollback). If it fails, the application can safely retry. With transactions, error handling becomes much simpler for an application, because it doesn't need to worry about partial failure—i.e., the case where some operations succeed and some fail (for whatever reason).

Trong nhiều thập kỷ, giao dịch (transaction) đã là cơ chế được ưa chuộng để đơn giản hóa những vấn đề này. Giao dịch là cách để một ứng dụng nhóm nhiều thao tác đọc và ghi lại với nhau thành một đơn vị logic. Về mặt khái niệm, tất cả các thao tác đọc và ghi trong một giao dịch được thực thi như một thao tác duy nhất: hoặc là toàn bộ giao dịch thành công (commit), hoặc nó thất bại (abort, rollback). Nếu thất bại, ứng dụng có thể thử lại một cách an toàn. Với giao dịch, việc xử lý lỗi trở nên đơn giản hơn nhiều cho một ứng dụng, vì nó không cần lo lắng về thất bại một phần — tức là trường hợp một số thao tác thành công còn một số thì thất bại (vì bất kỳ lý do gì).

If you have spent years working with transactions, they may seem obvious, but we shouldn't take them for granted. Transactions are not a law of nature; they were created with a purpose, namely to simplify the programming model for applications accessing a database. By using transactions, the application is free to ignore certain potential error scenarios and **concurrency** issues, because the database takes care of them instead (we call these safety guarantees).

Nếu bạn đã làm việc với giao dịch trong nhiều năm, chúng có vẻ hiển nhiên, nhưng ta không nên coi chúng là điều đương nhiên. Giao dịch không phải là một quy luật tự nhiên; chúng được tạo ra với một mục đích, đó là đơn giản hóa mô hình lập trình cho các ứng dụng truy cập cơ sở dữ liệu. Bằng cách dùng giao dịch, ứng dụng được tự do bỏ qua một số kịch bản lỗi tiềm tàng và các vấn đề tương tranh, vì cơ sở dữ liệu sẽ lo liệu chúng thay (ta gọi đây là các bảo đảm an toàn).

Not every application needs transactions, and sometimes there are advantages to weakening transactional guarantees or abandoning them entirely (for example, to achieve higher performance or higher availability). Some safety properties can be achieved without transactions.

Không phải mọi ứng dụng đều cần giao dịch, và đôi khi có những lợi ích khi làm yếu đi các bảo đảm giao dịch hoặc từ bỏ chúng hoàn toàn (ví dụ, để đạt hiệu năng cao hơn hoặc tính sẵn sàng cao hơn). Một số thuộc tính an toàn có thể đạt được mà không cần giao dịch.

How do you figure out whether you need transactions? In order to answer that question, we first need to understand exactly what safety guarantees transactions can provide, and what costs are associated with them. Although transactions seem straightforward at first glance, there are actually many subtle but important details that come into play.

Làm sao để biết bạn có cần giao dịch hay không? Để trả lời câu hỏi đó, trước hết ta cần hiểu chính xác giao dịch có thể cung cấp những bảo đảm an toàn nào, và những chi phí gắn liền với chúng là gì. Mặc dù thoạt nhìn giao dịch có vẻ đơn giản, thực ra có rất nhiều chi tiết tinh tế nhưng quan trọng cần được tính đến.

In this chapter, we will examine many examples of things that can go wrong, and explore the algorithms that databases use to guard against those issues. We will go especially deep in the area of concurrency control, discussing various kinds of race conditions that can occur and how databases implement isolation levels such as read committed, snapshot isolation, and serializability.

Trong chương này, ta sẽ xem xét nhiều ví dụ về những thứ có thể trục trặc, và khám phá các thuật toán mà cơ sở dữ liệu dùng để phòng tránh những vấn đề đó. Ta sẽ đi đặc biệt sâu vào lĩnh vực kiểm soát tương tranh (concurrency control), bàn về nhiều loại tình trạng tranh đua có thể xảy ra và cách cơ sở dữ liệu hiện thực các mức cô lập (isolation level) như read committed, snapshot isolation, và serializability.

This chapter applies to both single-node and distributed databases; in Chapter 8 we will focus the discussion on the particular challenges that arise only in distributed systems.

Chương này áp dụng cho cả cơ sở dữ liệu một nút lẫn cơ sở dữ liệu phân tán; ở Chương 8 ta sẽ tập trung thảo luận về những thách thức đặc thù chỉ phát sinh trong các hệ phân tán.

The Slippery Concept of a Transaction — Khái niệm trơn trượt về giao dịch

Almost all relational databases today, and some nonrelational databases, support transactions. Most of them follow the style that was introduced in 1975 by IBM System R, the first **SQL** database [1, 2, 3]. Although some implementation details have changed, the general idea has remained virtually the same for 40 years: the transaction support in MySQL, PostgreSQL, Oracle, SQL Server, etc., is uncannily similar to that of System R.

Hầu hết mọi cơ sở dữ liệu quan hệ ngày nay, và một số cơ sở dữ liệu phi quan hệ, đều hỗ trợ giao dịch. Phần lớn chúng theo phong cách được IBM System R, cơ sở dữ liệu SQL đầu tiên, giới thiệu vào năm 1975 [1, 2, 3]. Mặc dù một số chi tiết hiện thực đã thay đổi, ý tưởng tổng thể vẫn gần như y nguyên suốt 40 năm: cách hỗ trợ giao dịch trong MySQL, PostgreSQL, Oracle, SQL Server, v.v., giống một cách kỳ lạ với của System R.

In the late 2000s, nonrelational (**NoSQL**) databases started gaining popularity. They aimed to improve upon the relational status quo by offering a choice of new data models (see Chapter 2), and by including replication (Chapter 5) and partitioning (Chapter 6) by default. Transactions were the main casualty of this movement: many of this new generation of databases abandoned transactions entirely, or redefined the word to describe a much weaker set of guarantees than had previously been understood [4].

Vào cuối thập niên 2000, các cơ sở dữ liệu phi quan hệ (NoSQL) bắt đầu trở nên phổ biến. Chúng nhắm tới việc cải thiện hiện trạng quan hệ bằng cách cung cấp lựa chọn các mô hình dữ liệu mới (xem Chương 2), và bằng cách đưa replication (Chương 5) và phân vùng (Chương 6) vào mặc định. Giao dịch là nạn nhân chính của làn sóng này: nhiều cơ sở dữ liệu thế hệ mới đã từ bỏ giao dịch hoàn toàn, hoặc định nghĩa lại từ này để mô tả một tập bảo đảm yếu hơn nhiều so với cách hiểu trước đây [4].

With the hype around this new crop of distributed databases, there emerged a popular belief that transactions were the antithesis of **scalability**, and that any large-scale system would have to abandon transactions in order to maintain good performance and **high availability** [5, 6]. On the other hand, transactional guarantees are sometimes presented by database vendors as an essential requirement for “serious applications” with “valuable data.” Both viewpoints are pure hyperbole.

Cùng với sự thổi phồng quanh lựa cơ sở dữ liệu phân tán mới này, đã xuất hiện một niềm tin phổ biến rằng giao dịch là phản đề của khả năng mở rộng, và rằng bất kỳ hệ thống quy mô lớn nào cũng buộc phải từ bỏ giao dịch để duy trì hiệu năng tốt và tính sẵn sàng cao [5, 6]. Mặt khác, các bảo đảm giao dịch đôi khi lại được các nhà cung cấp cơ sở dữ liệu trình

bày như một yêu cầu thiết yếu cho “các ứng dụng nghiêm túc” với “dữ liệu giá trị”. Cả hai quan điểm đều là sự cường điệu thuần túy.

The truth is not that simple: like every other technical design choice, transactions have advantages and limitations. In order to understand those trade-offs, let's go into the details of the guarantees that transactions can provide—both in normal operation and in various extreme (but realistic) circumstances.

Sự thật không đơn giản như vậy: giống như mọi lựa chọn thiết kế kỹ thuật khác, giao dịch có cả ưu điểm lẫn hạn chế. Để hiểu những đánh đổi đó, hãy đi vào chi tiết những bảo đảm mà giao dịch có thể cung cấp — cả trong vận hành bình thường lẫn trong nhiều tình huống cực đoan (nhưng thực tế).

The Meaning of ACID — Ý nghĩa của ACID

The safety guarantees provided by transactions are often described by the well-known acronym ACID, which stands for Atomicity, Consistency, Isolation, and **Durability**. It was coined in 1983 by Theo Härder and Andreas Reuter [7] in an effort to establish precise terminology for fault-tolerance mechanisms in databases.

Các bảo đảm an toàn do giao dịch cung cấp thường được mô tả bằng từ viết tắt nổi tiếng ACID, viết tắt của Atomicity (tính nguyên tử), Consistency (tính nhất quán), Isolation (tính cô lập) và Durability (độ bền). Nó được Theo Härder và Andreas Reuter đặt ra vào năm 1983 [7] trong nỗ lực thiết lập thuật ngữ chính xác cho các cơ chế chịu lỗi trong cơ sở dữ liệu.

However, in practice, one database's implementation of ACID does not equal another's implementation. For example, as we shall see, there is a lot of ambiguity around the meaning of isolation [8]. The high-level idea is sound, but the devil is in the details. Today, when a system claims to be “ACID compliant,” it's unclear what guarantees you can actually expect. ACID has unfortunately become mostly a marketing term.

Tuy nhiên, trong thực tế, cách một cơ sở dữ liệu hiện thực ACID không giống với cách một cơ sở dữ liệu khác hiện thực nó. Ví dụ, như ta sẽ thấy, có rất nhiều sự mơ hồ quanh ý nghĩa của tính cô lập [8]. Ý tưởng ở tầm cao thì hợp lý, nhưng ma quỷ nằm trong chi tiết. Ngày nay, khi một hệ thống tuyên bố “tuân thủ ACID”, chẳng rõ thực ra bạn có thể trông đợi những bảo đảm nào. Đáng tiếc, ACID đã trở thành chủ yếu là một thuật ngữ tiếp thị.

(Systems that do not meet the ACID criteria are sometimes called BASE, which stands for Basically Available, Soft state, and Eventual consistency [9]. This is even more vague than the definition of ACID. It seems that the only sensible definition of BASE is “not ACID”; i.e., it can **mean** almost anything you want.)

(Các hệ thống không đáp ứng tiêu chí ACID đôi khi được gọi là BASE, viết tắt của Basically Available (về cơ bản là sẵn sàng), Soft state (trạng thái mềm) và Eventual consistency (nhất quán dần) [9]. Cách định nghĩa này còn mơ hồ hơn cả ACID. Có vẻ định nghĩa hợp lý duy nhất của BASE là “không phải ACID”; tức là nó có thể mang gần như bất cứ nghĩa nào bạn muốn.)

Let's dig into the definitions of atomicity, consistency, isolation, and durability, as this will let us refine our idea of transactions.

Hãy đào sâu vào các định nghĩa của atomicity, consistency, isolation và durability, vì điều này sẽ giúp ta tinh chỉnh ý niệm về giao dịch.

Atomicity — Tính nguyên tử (Atomicity)

In general, atomic refers to something that cannot be broken down into smaller parts. The word means similar but subtly different things in different branches of computing. For example, in multi-threaded programming, if one thread executes an atomic operation, that means there is no way that another thread could see the half-finished result of the operation. The system can only be in the state it was before the operation or after the operation, not something in between.

Nhìn chung, nguyên tử (atomic) chỉ một thứ gì đó không thể bị chia nhỏ thành các phần nhỏ hơn. Từ này mang những nghĩa tương tự nhưng khác nhau một cách tinh tế trong các nhánh khác nhau của ngành máy tính. Ví dụ, trong lập trình đa luồng, nếu một luồng thực thi một thao tác nguyên tử, điều đó có nghĩa là không cách nào một luồng khác có thể thấy kết quả nửa vời của thao tác. Hệ thống chỉ có thể ở trạng thái trước thao tác hoặc sau thao tác, chứ không ở trạng thái lửng chừng nào đó.

By contrast, in the context of ACID, atomicity is not about concurrency. It does not describe what happens if several processes try to access the same data at the same time, because that is covered under the letter I, for isolation (see “Isolation”).

Ngược lại, trong ngữ cảnh ACID, tính nguyên tử không liên quan đến tương tranh. Nó không mô tả điều gì xảy ra nếu nhiều tiến trình cùng truy cập một dữ liệu một lúc, vì điều đó được bao quát dưới chữ I, tức isolation (xem “Tính cô lập”).

Rather, ACID atomicity describes what happens if a client wants to make several writes, but a fault occurs after some of the writes have been processed—for example, a process crashes, a network connection is interrupted, a disk becomes full, or some integrity constraint is violated. If the writes are grouped together into an atomic transaction, and the transaction cannot be completed (committed) due to a fault, then the transaction is aborted and the database must discard or undo any writes it has made so far in that transaction.

Thay vào đó, tính nguyên tử trong ACID mô tả điều gì xảy ra nếu một client muốn thực hiện nhiều thao tác ghi, nhưng một lỗi xảy ra sau khi một số thao tác ghi đã được xử lý — ví dụ, một tiến trình sập, một kết nối mạng bị gián đoạn, một đĩa bị đầy, hoặc một ràng buộc toàn vẹn nào đó bị vi phạm. Nếu các thao tác ghi được nhóm lại với nhau thành một giao dịch nguyên tử, và giao dịch không thể hoàn tất (commit) do một lỗi, thì giao dịch bị hủy bỏ (abort) và cơ sở dữ liệu phải loại bỏ hoặc hoàn tác mọi thao tác ghi mà nó đã thực hiện cho tới lúc đó trong giao dịch ấy.

Without atomicity, if an error occurs partway through making multiple changes, it's difficult to know which changes have taken effect and which haven't. The application could try again, but that risks making the same change twice, leading to duplicate or incorrect data. Atomicity simplifies this problem: if a transaction was aborted, the application can be sure that it didn't change anything, so it can safely be retried.

Không có tính nguyên tử, nếu một lỗi xảy ra giữa chừng khi đang thực hiện nhiều thay đổi, sẽ rất khó biết thay đổi nào đã có hiệu lực và thay đổi nào thì chưa. Ứng dụng có thể thử lại, nhưng điều đó có nguy cơ thực hiện cùng một thay đổi hai lần, dẫn đến dữ liệu trùng lặp hoặc sai. Tính nguyên tử đơn giản hóa vấn đề này: nếu một giao dịch bị hủy bỏ, ứng dụng có thể chắc chắn rằng nó không thay đổi gì cả, nên có thể thử lại một cách an toàn.

The ability to abort a transaction on error and have all writes from that transaction discarded is the defining feature of ACID atomicity. Perhaps abortability would have been a better term than atomicity, but we will stick with atomicity since that's the usual word.

Khả năng hủy bỏ một giao dịch khi gặp lỗi và loại bỏ mọi thao tác ghi của giao dịch

đó chính là đặc trưng định nghĩa của tính nguyên tử trong ACID. Có lẽ “khả năng hủy bỏ” (abortability) sẽ là một thuật ngữ phù hợp hơn “tính nguyên tử”, nhưng ta sẽ giữ “atomicity” vì đó là từ thông dụng.

Consistency — Tính nhất quán (Consistency)

The word consistency is terribly overloaded:

Từ nhất quán (consistency) bị gán cho quá nhiều nghĩa khác nhau:

- In Chapter 5 we discussed replica consistency and the issue of eventual consistency that arises in asynchronously replicated systems (see “Problems with Replication Lag”).
- Consistent hashing is an approach to partitioning that some systems use for rebalancing (see “Consistent Hashing”).
- In the CAP theorem (see Chapter 9), the word consistency is used to mean linearizability (see “Linearizability”).
- In the context of ACID, consistency refers to an application-specific notion of the database being in a “good state.”
 - Ở Chương 5 ta đã bàn về nhất quán bản sao (replica consistency) và vấn đề nhất quán dần (eventual consistency) phát sinh trong các hệ thống nhân bản bất đồng bộ (xem “Problems with Replication Lag”).
 - Băm nhất quán (consistent hashing) là một cách tiếp cận phân vùng mà một số hệ thống dùng để tái cân bằng (xem “Consistent Hashing”).
 - Trong định lý CAP (xem Chương 9), từ nhất quán được dùng để chỉ tính tuyến tính hóa (linearizability) (xem “Linearizability”).
 - Trong ngữ cảnh ACID, nhất quán chỉ một khái niệm đặc thù theo ứng dụng về việc cơ sở dữ liệu ở trong một “trạng thái tốt”.

It’s unfortunate that the same word is used with at least four different meanings.

Thật đáng tiếc khi cùng một từ lại được dùng với ít nhất bốn nghĩa khác nhau.

The idea of ACID consistency is that you have certain statements about your data (invariants) that must always be true—for example, in an accounting system, credits and debits across all accounts must always be balanced. If a transaction starts with a database that is valid according to these invariants, and any writes during the transaction preserve the validity, then you can be sure that the invariants are always satisfied.

Ý tưởng của nhất quán trong ACID là bạn có một số mệnh đề về dữ liệu của mình (các bất biến — invariant) luôn phải đúng — ví dụ, trong một hệ thống kế toán, các khoản có và khoản nợ trên tất cả tài khoản phải luôn cân bằng. Nếu một giao dịch bắt đầu với một cơ sở dữ liệu hợp lệ theo các bất biến này, và mọi thao tác ghi trong giao dịch đều giữ gìn tính hợp lệ, thì bạn có thể chắc chắn rằng các bất biến luôn được thỏa mãn.

However, this idea of consistency depends on the application’s notion of invariants, and it’s the application’s responsibility to define its transactions correctly so that they preserve consistency. This is not something that the database can guarantee: if you write bad data that violates your invariants, the database can’t stop you. (Some specific kinds of invariants can be checked by the database, for example using **foreign key** constraints or uniqueness constraints. However, in general, the application defines what data is valid or invalid—the database only stores it.)

Tuy nhiên, ý niệm về nhất quán này phụ thuộc vào khái niệm bất biến của ứng dụng, và trách nhiệm của ứng dụng là định nghĩa các giao dịch của mình một cách đúng đắn sao cho chúng giữ gìn sự nhất quán. Đây không phải thứ mà cơ sở dữ liệu có thể đảm bảo: nếu

bạn ghi dữ liệu xấu vi phạm các bất biến của mình, cơ sở dữ liệu không thể ngăn bạn. (Một số loại bất biến cụ thể có thể được cơ sở dữ liệu kiểm tra, ví dụ dùng ràng buộc khóa ngoại hoặc ràng buộc tính duy nhất. Tuy nhiên, nhìn chung, ứng dụng mới là nơi định nghĩa dữ liệu nào hợp lệ hay không hợp lệ — cơ sở dữ liệu chỉ lưu trữ nó.)

Atomicity, isolation, and durability are properties of the database, whereas consistency (in the ACID sense) is a **property** of the application. The application may rely on the database's atomicity and isolation properties in order to achieve consistency, but it's not up to the database alone. Thus, the letter C doesn't really belong in ACID.

Atomicity, isolation và durability là các thuộc tính của cơ sở dữ liệu, trong khi consistency (theo nghĩa ACID) là một thuộc tính của ứng dụng. Ứng dụng có thể dựa vào các thuộc tính nguyên tử và cô lập của cơ sở dữ liệu để đạt được sự nhất quán, nhưng việc đó không chỉ phụ thuộc vào riêng cơ sở dữ liệu. Do đó, chữ C thực ra không thuộc về ACID.

Isolation — Tính cô lập (Isolation)

Most databases are accessed by several clients at the same time. That is no problem if they are reading and writing different parts of the database, but if they are accessing the same database records, you can run into concurrency problems (race conditions).

Hầu hết cơ sở dữ liệu được nhiều client truy cập cùng lúc. Đó không phải vấn đề nếu chúng đọc và ghi vào những phần khác nhau của cơ sở dữ liệu, nhưng nếu chúng truy cập cùng những bản ghi, bạn có thể gặp các vấn đề tương tranh (tình trạng tranh đua).

Figure 7-1 is a simple example of this kind of problem. Say you have two clients simultaneously incrementing a counter that is stored in a database. Each client needs to read the current value, add 1, and write the new value back (assuming there is no increment operation built into the database). In Figure 7-1 the counter should have increased from 42 to 44, because two increments happened, but it actually only went to 43 because of the race condition.

Hình 7-1 là một ví dụ đơn giản về loại vấn đề này. Giả sử bạn có hai client đồng thời tăng một bộ đếm được lưu trong cơ sở dữ liệu. Mỗi client cần đọc giá trị hiện tại, cộng thêm 1, rồi ghi giá trị mới trở lại (giả định cơ sở dữ liệu không có sẵn thao tác tăng). Trong Hình 7-1 lẽ ra bộ đếm phải tăng từ 42 lên 44, vì có hai lần tăng, nhưng thực tế nó chỉ lên 43 do tình trạng tranh đua.

Isolation in the sense of ACID means that concurrently executing transactions are isolated from each other: they cannot step on each other's toes. The classic database textbooks formalize isolation as serializability, which means that each transaction can pretend that it is the only transaction running on the entire database. The database ensures that when the transactions have committed, the result is the same as if they had run serially (one after another), even though in reality they may have run concurrently [10].

Tính cô lập theo nghĩa ACID có nghĩa là các giao dịch thực thi đồng thời được cô lập khỏi nhau: chúng không thể giẫm chân lên nhau. Các sách giáo khoa cơ sở dữ liệu kinh điển hình thức hóa tính cô lập thành tính khả tuần tự hóa (serializability), nghĩa là mỗi giao dịch có thể giả vờ rằng nó là giao dịch duy nhất đang chạy trên toàn bộ cơ sở dữ liệu. Cơ sở dữ liệu đảm bảo rằng khi các giao dịch đã commit, kết quả giống như thể chúng đã chạy tuần tự (lần lượt cái này sau cái kia), dù trong thực tế chúng có thể đã chạy đồng thời [10].

Figure 7-1. A race condition between two clients concurrently incrementing a counter. — Hình 7-1. Một tình trạng tranh đua giữa hai client đồng thời tăng một bộ đếm. However, in

practice, serializable isolation is rarely used, because it carries a performance penalty. Some popular databases, such as Oracle 11g, don't even implement it. In Oracle there is an isolation level called "serializable," but it actually implements something called snapshot isolation, which is a weaker guarantee than serializability [8, 11]. We will explore snapshot isolation and other forms of isolation in "Weak Isolation Levels".

Tuy nhiên, trong thực tế, cô lập khả tuần tự hóa (serializable isolation) hiếm khi được dùng, vì nó kéo theo cái giá về hiệu năng. Một số cơ sở dữ liệu phổ biến, chẳng hạn Oracle 11g, thậm chí còn không hiện thực nó. Trong Oracle có một mức cô lập gọi là "serializable", nhưng thực ra nó hiện thực một thứ gọi là snapshot isolation, vốn là một bảo đảm yếu hơn serializability [8, 11]. Ta sẽ khám phá snapshot isolation và các dạng cô lập khác trong "Các mức cô lập yếu".

Durability — Độ bền (Durability)

The purpose of a database system is to provide a safe place where data can be stored without fear of losing it. Durability is the promise that once a transaction has committed successfully, any data it has written will not be forgotten, even if there is a hardware fault or the database crashes.

Mục đích của một hệ cơ sở dữ liệu là cung cấp một nơi an toàn để lưu trữ dữ liệu mà không sợ mất nó. Độ bền là lời hứa rằng một khi giao dịch đã commit thành công, mọi dữ liệu nó đã ghi sẽ không bị quên lãng, ngay cả khi có lỗi phần cứng hoặc cơ sở dữ liệu sập.

In a single-node database, durability typically means that the data has been written to nonvolatile storage such as a hard drive or SSD. It usually also involves a write-ahead log or similar (see "Making B-trees reliable"), which allows recovery in the event that the data structures on disk are corrupted. In a replicated database, durability may mean that the data has been successfully copied to some number of nodes. In order to provide a durability guarantee, a database must wait until these writes or replications are complete before reporting a transaction as successfully committed.

Trong một cơ sở dữ liệu một nút, độ bền thường có nghĩa là dữ liệu đã được ghi vào bộ nhớ không bay hơi như đĩa cứng hoặc SSD. Nó thường cũng bao gồm một nhật ký ghi-trước (write-ahead log) hoặc tương tự (xem "Making B-trees reliable"), cho phép phục hồi trong trường hợp các cấu trúc dữ liệu trên đĩa bị hỏng. Trong một cơ sở dữ liệu có nhân bản, độ bền có thể có nghĩa là dữ liệu đã được sao chép thành công tới một số lượng nút nào đó. Để cung cấp bảo đảm về độ bền, một cơ sở dữ liệu phải chờ cho đến khi các thao tác ghi hoặc nhân bản này hoàn tất trước khi báo cáo một giao dịch là đã commit thành công.

As discussed in "**Reliability**", perfect durability does not exist: if all your hard disks and all your backups are destroyed at the same time, there's obviously nothing your database can do to save you.

Như đã bàn trong "Reliability", độ bền hoàn hảo không tồn tại: nếu tất cả đĩa cứng và tất cả bản sao lưu của bạn bị phá hủy cùng một lúc, hiển nhiên chẳng có gì cơ sở dữ liệu của bạn có thể làm để cứu bạn.

Replication and Durability — Nhân bản và Độ bền Historically, durability meant writing to an archive tape. Then it was understood as writing to a disk or SSD. More recently, it has been adapted to mean replication. Which implementation is better?

Trong lịch sử, độ bền có nghĩa là ghi vào một cuốn băng lưu trữ. Sau đó nó được hiểu là ghi vào một đĩa hoặc SSD. Gần đây hơn, nó đã được điều chỉnh để mang nghĩa nhân bản. Cách hiện thực nào tốt hơn?

The truth is, nothing is perfect:

Sự thật là, không gì hoàn hảo cả:

- If you write to disk and the machine dies, even though your data isn't lost, it is inaccessible until you either fix the machine or transfer the disk to another machine. Replicated systems can remain available.
- A correlated fault—a power **outage** or a **bug** that crashes every node on a particular input—can knock out all replicas at once (see “Reliability”), losing any data that is only in memory. Writing to disk is therefore still relevant for in-memory databases.
- In an asynchronously replicated system, recent writes may be lost when the leader becomes unavailable (see “Handling Node Outages”).
- When the power is suddenly cut, SSDs in particular have been shown to sometimes violate the guarantees they are supposed to provide: even fsync isn't guaranteed to work correctly [12]. Disk firmware can have bugs, just like any other kind of software [13, 14].
- Subtle interactions between the storage **engine** and the filesystem implementation can lead to bugs that are hard to track down, and may cause files on disk to be corrupted after a crash [15, 16].
- Data on disk can gradually become corrupted without this being detected [17]. If data has been corrupted for some time, replicas and recent backups may also be corrupted. In this case, you will need to try to restore the data from a historical **backup**.
- One study of SSDs found that between 30% and 80% of drives develop at least one bad block during the first four years of operation [18]. Magnetic hard drives have a lower rate of bad sectors, but a higher rate of complete failure than SSDs.
- If an SSD is disconnected from power, it can start losing data within a few weeks, depending on the temperature [19].
 - Nếu bạn ghi vào đĩa và máy chết, dù dữ liệu của bạn không bị mất, nó vẫn không thể truy cập được cho đến khi bạn sửa máy hoặc chuyển đĩa sang một máy khác. Các hệ thống có nhân bản có thể vẫn sẵn sàng.
 - Một lỗi tương quan — mất điện hoặc một bug làm sập mọi nút khi gặp cùng một đầu vào — có thể đánh gục tất cả bản sao cùng một lúc (xem “Reliability”), làm mất mọi dữ liệu chỉ nằm trong bộ nhớ. Do đó việc ghi vào đĩa vẫn còn ý nghĩa với các cơ sở dữ liệu trong bộ nhớ.
 - Trong một hệ thống nhân bản bất đồng bộ, các thao tác ghi gần đây có thể bị mất khi leader trở nên không sẵn sàng (xem “Handling Node Outages”).
 - Khi nguồn điện bị cắt đột ngột, các SSD đặc biệt đã được chứng minh là đôi khi vi phạm những bảo đảm mà chúng đáng lẽ phải cung cấp: ngay cả fsync cũng không được đảm bảo hoạt động đúng [12]. Firmware của đĩa có thể có bug, giống như bất kỳ loại phần mềm nào khác [13, 14].
 - Những tương tác tinh vi giữa engine lưu trữ và cách hiện thực hệ thống tệp có thể dẫn đến những bug khó truy vết, và có thể khiến các tệp trên đĩa bị hỏng sau một lần sập [15, 16].
 - Dữ liệu trên đĩa có thể dần dần bị hỏng mà không bị phát hiện [17]. Nếu dữ liệu đã hỏng trong một thời gian, các bản sao và bản sao lưu gần đây cũng có thể bị hỏng. Trong trường hợp này, bạn sẽ cần cố khôi phục dữ liệu từ một bản sao lưu cũ trong quá khứ.
 - Một nghiên cứu về SSD cho thấy từ 30% đến 80% ổ phát triển ít nhất một khối hỏng trong bốn năm vận hành đầu tiên [18]. Đĩa cứng từ tính có tỷ lệ khối hỏng thấp hơn, nhưng tỷ lệ hỏng hoàn toàn lại cao hơn SSD.
 - Nếu một SSD bị ngắt khỏi nguồn điện, nó có thể bắt đầu mất dữ liệu trong vòng vài tuần, tùy thuộc vào nhiệt độ [19].

In practice, there is no one technique that can provide absolute guarantees. There are only various

risk-reduction techniques, including writing to disk, replicating to remote machines, and backups—and they can and should be used together. As always, it’s wise to take any theoretical “guarantees” with a healthy grain of salt.

Trong thực tế, không có kỹ thuật đơn lẻ nào có thể cung cấp các bảo đảm tuyệt đối. Chỉ có nhiều kỹ thuật giảm thiểu rủi ro khác nhau, bao gồm ghi vào đĩa, nhân bản tới máy ở xa, và sao lưu — và chúng có thể và nên được dùng kết hợp. Như mọi khi, khôn ngoan là hãy đón nhận mọi “bảo đảm” lý thuyết với một chút hoài nghi lành mạnh.

Single-Object and Multi-Object Operations — Thao tác đơn đối tượng và đa đối tượng

To recap, in ACID, atomicity and isolation describe what the database should do if a client makes several writes within the same transaction:

Tóm lại, trong ACID, tính nguyên tử và tính cô lập mô tả điều cơ sở dữ liệu nên làm nếu một client thực hiện nhiều thao tác ghi trong cùng một giao dịch:

If an error occurs halfway through a sequence of writes, the transaction should be aborted, and the writes made up to that point should be discarded. In other words, the database saves you from having to worry about partial failure, by giving an all-or-nothing guarantee.

Nếu một lỗi xảy ra giữa chừng một chuỗi thao tác ghi, giao dịch nên bị hủy bỏ, và các thao tác ghi đã thực hiện cho tới điểm đó nên bị loại bỏ. Nói cách khác, cơ sở dữ liệu cứu bạn khỏi phải lo lắng về thất bại một phần, bằng cách đưa ra một bảo đảm tất-cả-hoặc-không-gì.

Concurrently running transactions shouldn’t interfere with each other. For example, if one transaction makes several writes, then another transaction should see either all or none of those writes, but not some subset.

Các giao dịch chạy đồng thời không nên can thiệp lẫn nhau. Ví dụ, nếu một giao dịch thực hiện nhiều thao tác ghi, thì một giao dịch khác nên thấy hoặc tất cả hoặc không thao tác nào trong số đó, chứ không phải một phần.

These definitions assume that you want to modify several objects (rows, documents, records) at once. Such multi-object transactions are often needed if several pieces of data need to be kept in sync. Figure 7-2 shows an example from an email application. To display the number of unread messages for a user, you could query something like:

Những định nghĩa này giả định rằng bạn muốn sửa đổi nhiều đối tượng (hàng, tài liệu, bản ghi) cùng lúc. Những giao dịch đa đối tượng như vậy thường cần đến nếu nhiều mẫu dữ liệu cần được giữ đồng bộ với nhau. Hình 7-2 cho thấy một ví dụ từ một ứng dụng email. Để hiển thị số lượng thông điệp chưa đọc của một người dùng, bạn có thể truy vấn đại loại như:

```
SELECT COUNT(*) FROM emails WHERE recipient_id = 2 AND unread_flag = true
```

However, you might find this query to be too slow if there are many emails, and decide to store the number of unread messages in a separate field (a kind of **denormalization**). Now, whenever a new message comes in, you have to increment the unread counter as well, and whenever a message is marked as read, you also have to decrement the unread counter.

Tuy nhiên, bạn có thể thấy truy vấn này quá chậm nếu có nhiều email, và quyết định lưu số thông điệp chưa đọc vào một trường riêng (một dạng phi chuẩn hóa). Giờ đây, mỗi khi

có một thông điệp mới đến, bạn phải tăng bộ đếm chưa đọc lên, và mỗi khi một thông điệp được đánh dấu là đã đọc, bạn cũng phải giảm bộ đếm chưa đọc.

In Figure 7-2, user 2 experiences an anomaly: the mailbox listing shows an unread message, but the counter shows zero unread messages because the counter increment has not yet happened. Isolation would have prevented this issue by ensuring that user 2 sees either both the inserted email and the updated counter, or neither, but not an inconsistent halfway point.

Trong Hình 7-2, người dùng 2 gặp một dị thường: danh sách hộp thư hiển thị một thông điệp chưa đọc, nhưng bộ đếm lại hiển thị không có thông điệp chưa đọc nào vì việc tăng bộ đếm chưa diễn ra. Tính cô lập lẽ ra đã ngăn được vấn đề này bằng cách đảm bảo rằng người dùng 2 thấy hoặc cả email vừa chèn lẫn bộ đếm đã cập nhật, hoặc không thấy cái nào, chứ không phải một điểm lưng chừng không nhất quán.

Figure 7-2. Violating isolation: one transaction reads another transaction’s uncommitted writes (a “dirty read”). — **Hình 7-2. Vi phạm tính cô lập: một giao dịch đọc được các thao tác ghi chưa commit của một giao dịch khác (một “đọc bẩn” — dirty read).** Figure 7-3 illustrates the need for atomicity: if an error occurs somewhere over the course of the transaction, the contents of the mailbox and the unread counter might become out of sync. In an atomic transaction, if the update to the counter fails, the transaction is aborted and the inserted email is rolled back.

Hình 7-3 minh họa nhu cầu về tính nguyên tử: nếu một lỗi xảy ra ở đâu đó trong quá trình của giao dịch, nội dung của hộp thư và bộ đếm chưa đọc có thể trở nên không đồng bộ. Trong một giao dịch nguyên tử, nếu việc cập nhật bộ đếm thất bại, giao dịch bị hủy bỏ và email vừa chèn được hoàn tác.

Figure 7-3. Atomicity ensures that if an error occurs any prior writes from that transaction are undone, to avoid an inconsistent state. — **Hình 7-3. Tính nguyên tử đảm bảo rằng nếu một lỗi xảy ra thì mọi thao tác ghi trước đó của giao dịch ấy đều được hoàn tác, để tránh một trạng thái không nhất quán.** Multi-object transactions require some way of determining which read and write operations belong to the same transaction. In relational databases, that is typically done based on the client’s TCP connection to the database server: on any particular connection, everything between a BEGIN TRANSACTION and a COMMIT statement is considered to be part of the same transaction.

Giao dịch đa đối tượng cần một cách nào đó để xác định những thao tác đọc và ghi nào thuộc cùng một giao dịch. Trong các cơ sở dữ liệu quan hệ, điều đó thường được làm dựa trên kết nối TCP của client tới máy chủ cơ sở dữ liệu: trên bất kỳ kết nối cụ thể nào, mọi thứ nằm giữa một câu lệnh BEGIN TRANSACTION và một câu lệnh COMMIT đều được coi là một phần của cùng một giao dịch.

On the other hand, many nonrelational databases don’t have such a way of grouping operations together. Even if there is a multi-object **API** (for example, a key-value store may have a multi-put operation that updates several keys in one operation), that doesn’t necessarily mean it has transaction semantics: the command may succeed for some keys and fail for others, leaving the database in a partially updated state.

Mặt khác, nhiều cơ sở dữ liệu phi quan hệ không có một cách nào như vậy để nhóm các thao tác lại với nhau. Ngay cả khi có một API đa đối tượng (ví dụ, một kho khóa-giá trị có thể có một thao tác multi-put cập nhật nhiều khóa trong một thao tác), điều đó không nhất thiết có nghĩa là nó có ngữ nghĩa giao dịch: lệnh có thể thành công với một số khóa và thất bại với những khóa khác, để cơ sở dữ liệu ở một trạng thái được cập nhật một phần.

Single-object writes — Thao tác ghi đơn đối tượng

Atomicity and isolation also apply when a single object is being changed. For example, imagine you are writing a 20 KB **JSON document** to a database:

Tính nguyên tử và tính cô lập cũng áp dụng khi một đối tượng đơn lẻ đang được thay đổi. Ví dụ, hãy hình dung bạn đang ghi một tài liệu JSON 20 KB vào một cơ sở dữ liệu:

- If the network connection is interrupted after the first 10 KB have been sent, does the database store that unparseable 10 KB fragment of JSON?
- If the power fails while the database is in the middle of overwriting the previous value on disk, do you end up with the old and new values spliced together?
- If another client reads that document while the write is in progress, will it see a partially updated value?
 - Nếu kết nối mạng bị gián đoạn sau khi 10 KB đầu tiên đã được gửi đi, liệu cơ sở dữ liệu có lưu cái mảnh JSON 10 KB không phân tích được đó không?
 - Nếu mất điện trong lúc cơ sở dữ liệu đang ghi đè lên giá trị trước đó trên đĩa, liệu bạn có rốt cuộc nhận được giá trị cũ và mới ghép nối lẫn lộn vào nhau không?
 - Nếu một client khác đọc tài liệu đó trong khi thao tác ghi đang diễn ra, liệu nó có thấy một giá trị được cập nhật một phần không?

Those issues would be incredibly confusing, so storage engines almost universally aim to provide atomicity and isolation on the level of a single object (such as a key-value pair) on one node. Atomicity can be implemented using a log for crash recovery (see “Making B-trees reliable”), and isolation can be implemented using a lock on each object (allowing only one thread to access an object at any one time).

Những vấn đề đó sẽ cực kỳ gây bối rối, nên các engine lưu trữ gần như đồng loạt nhắm tới việc cung cấp tính nguyên tử và tính cô lập ở mức một đối tượng đơn lẻ (chẳng hạn một cặp khóa-giá trị) trên một nút. Tính nguyên tử có thể được hiện thực bằng một nhật ký để phục hồi sau khi sập (xem “Making B-trees reliable”), và tính cô lập có thể được hiện thực bằng một khóa trên mỗi đối tượng (chỉ cho phép một luồng truy cập một đối tượng tại bất kỳ thời điểm nào).

Some databases also provide more complex atomic operations, such as an increment operation, which removes the need for a read-modify-write cycle like that in Figure 7-1. Similarly popular is a compare-and-set operation, which allows a write to happen only if the value has not been concurrently changed by someone else (see “Compare-and-set”).

Một số cơ sở dữ liệu cũng cung cấp những thao tác nguyên tử phức tạp hơn, chẳng hạn một thao tác tăng, vốn loại bỏ nhu cầu về một chu trình đọc-sửa-ghi như trong Hình 7-1. Tương tự phổ biến là một thao tác so-sánh-và-gán (compare-and-set), cho phép một thao tác ghi chỉ xảy ra nếu giá trị chưa bị ai đó thay đổi đồng thời (xem “Compare-and-set”).

These single-object operations are useful, as they can prevent lost updates when several clients try to write to the same object concurrently (see “Preventing Lost Updates”). However, they are not transactions in the usual sense of the word. Compare-and-set and other single-object operations have been dubbed “lightweight transactions” or even “ACID” for marketing purposes [20, 21, 22], but that terminology is misleading. A transaction is usually understood as a mechanism for grouping multiple operations on multiple objects into one unit of execution.

Những thao tác đơn đối tượng này hữu ích, vì chúng có thể ngăn các cập nhật bị mất khi nhiều client cố đồng thời ghi vào cùng một đối tượng (xem “Ngăn chặn cập nhật bị mất”). Tuy nhiên, chúng không phải là giao dịch theo nghĩa thông thường của từ này. So-sánh-và-

gán và các thao tác đơn đối tượng khác đã được gắn mác “giao dịch nhẹ” hoặc thậm chí “ACID” vì mục đích tiếp thị [20, 21, 22], nhưng thuật ngữ đó gây hiểu lầm. Một giao dịch thường được hiểu là một cơ chế để nhóm nhiều thao tác trên nhiều đối tượng thành một đơn vị thực thi.

The need for multi-object transactions — Nhu cầu về giao dịch đa đối tượng

Many distributed datastores have abandoned multi-object transactions because they are difficult to implement across partitions, and they can get in the way in some scenarios where very high availability or performance is required. However, there is nothing that fundamentally prevents transactions in a distributed database, and we will discuss implementations of distributed transactions in Chapter 9.

Nhiều kho dữ liệu phân tán đã từ bỏ giao dịch đa đối tượng vì chúng khó hiện thực xuyên các phân vùng, và chúng có thể gây cản trở trong một số kịch bản đòi hỏi tính sẵn sàng hoặc hiệu năng rất cao. Tuy nhiên, không có gì về bản chất ngăn cản giao dịch trong một cơ sở dữ liệu phân tán, và ta sẽ bàn về các hiện thực của giao dịch phân tán ở Chương 9.

But do we need multi-object transactions at all? Would it be possible to implement any application with only a key-value **data model** and single-object operations?

Nhưng liệu ta có cần giao dịch đa đối tượng hay không? Liệu có thể hiện thực bất kỳ ứng dụng nào chỉ với một mô hình dữ liệu khóa-giá trị và các thao tác đơn đối tượng không?

There are some use cases in which single-object inserts, updates, and deletes are sufficient. However, in many other cases writes to several different objects need to be coordinated:

Có một số tình huống sử dụng mà các thao tác chèn, cập nhật và xóa đơn đối tượng là đủ. Tuy nhiên, trong nhiều trường hợp khác, các thao tác ghi vào nhiều đối tượng khác nhau cần được phối hợp:

- In a relational data model, a **row** in one **table** often has a foreign key reference to a row in another table. (Similarly, in a **graph**-like data model, a **vertex** has edges to other vertices.) Multi-object transactions allow you to ensure that these references remain valid: when inserting several records that refer to one another, the foreign keys have to be correct and up to date, or the data becomes nonsensical.
- In a document data model, the fields that need to be updated together are often within the same document, which is treated as a single object—no multi-object transactions are needed when updating a single document. However, document databases lacking **join** functionality also encourage denormalization (see “Relational Versus Document Databases Today”). When denormalized information needs to be updated, like in the example of Figure 7-2, you need to update several documents in one go. Transactions are very useful in this situation to prevent denormalized data from going out of sync.
- In databases with secondary indexes (almost everything except pure key-value stores), the indexes also need to be updated every time you change a value. These indexes are different database objects from a transaction point of view: for example, without transaction isolation, it’s possible for a record to appear in one **index** but not another, because the update to the second index hasn’t happened yet.
 - Trong một mô hình dữ liệu quan hệ, một hàng trong một bảng thường có một tham chiếu khóa ngoại tới một hàng trong một bảng khác. (Tương tự, trong một mô hình dữ liệu kiểu đồ thị, một đỉnh có các cạnh tới các đỉnh khác.) Giao dịch đa đối tượng cho phép bạn đảm bảo rằng các tham chiếu này vẫn hợp lệ: khi chèn nhiều bản ghi

tham chiếu lẫn nhau, các khóa ngoại phải đúng và được cập nhật, nếu không dữ liệu trở nên vô nghĩa.

- Trong một mô hình dữ liệu tài liệu, các trường cần được cập nhật cùng nhau thường nằm trong cùng một tài liệu, vốn được coi là một đối tượng đơn lẻ — không cần giao dịch đa đối tượng khi cập nhật một tài liệu duy nhất. Tuy nhiên, các cơ sở dữ liệu tài liệu thiếu khả năng join cũng khuyến khích phi chuẩn hóa (xem “Relational Versus Document Databases Today”). Khi thông tin đã phi chuẩn hóa cần được cập nhật, như trong ví dụ ở Hình 7-2, bạn cần cập nhật nhiều tài liệu trong một lượt. Giao dịch rất hữu ích trong tình huống này để ngăn dữ liệu phi chuẩn hóa khỏi mất đồng bộ.
- Trong các cơ sở dữ liệu có chỉ mục thứ cấp (gần như mọi thứ trừ các kho khóa-giá trị thuần túy), các chỉ mục cũng cần được cập nhật mỗi khi bạn thay đổi một giá trị. Những chỉ mục này là các đối tượng cơ sở dữ liệu khác nhau dưới góc nhìn giao dịch: ví dụ, không có tính cô lập giao dịch, có thể xảy ra việc một bản ghi xuất hiện trong một chỉ mục nhưng không có trong một chỉ mục khác, vì việc cập nhật chỉ mục thứ hai chưa diễn ra.

Such applications can still be implemented without transactions. However, error handling becomes much more complicated without atomicity, and the lack of isolation can cause concurrency problems. We will discuss those in “Weak Isolation Levels”, and explore alternative approaches in Chapter 12.

Những ứng dụng như vậy vẫn có thể được hiện thực mà không cần giao dịch. Tuy nhiên, việc xử lý lỗi trở nên phức tạp hơn nhiều khi không có tính nguyên tử, và việc thiếu tính cô lập có thể gây ra các vấn đề tương tranh. Ta sẽ bàn về những điều đó trong “Các mức cô lập yếu”, và khám phá các cách tiếp cận thay thế ở Chương 12.

Handling errors and aborts — Xử lý lỗi và hủy bỏ

A key feature of a transaction is that it can be aborted and safely retried if an error occurred. ACID databases are based on this philosophy: if the database is in danger of violating its guarantee of atomicity, isolation, or durability, it would rather abandon the transaction entirely than allow it to remain half-finished.

Một đặc điểm then chốt của giao dịch là nó có thể bị hủy bỏ và thử lại một cách an toàn nếu một lỗi xảy ra. Các cơ sở dữ liệu ACID dựa trên triết lý này: nếu cơ sở dữ liệu đang đứng trước nguy cơ vi phạm bảo đảm về tính nguyên tử, tính cô lập, hoặc độ bền, nó thà từ bỏ giao dịch hoàn toàn còn hơn để nó ở trạng thái dở dang.

Not all systems follow that philosophy, though. In particular, datastores with leaderless replication (see “Leaderless Replication”) work much more on a “best effort” basis, which could be summarized as “the database will do as much as it can, and if it runs into an error, it won’t undo something it has already done”—so it’s the application’s responsibility to recover from errors.

Tuy nhiên, không phải mọi hệ thống đều theo triết lý đó. Đặc biệt, các kho dữ liệu với nhân bản không-leader (xem “Leaderless Replication”) hoạt động nhiều hơn trên cơ sở “nỗ lực hết mình”, có thể tóm tắt là “cơ sở dữ liệu sẽ làm hết sức có thể, và nếu nó gặp một lỗi, nó sẽ không hoàn tác thứ nó đã làm” — nên trách nhiệm phục hồi sau lỗi thuộc về ứng dụng.

Errors will inevitably happen, but many software developers prefer to think only about the happy path rather than the intricacies of error handling. For example, popular object-relational mapping (**ORM**) frameworks such as Rails’s ActiveRecord and Django don’t retry aborted transactions—the error usually results in an exception bubbling up the stack, so any user input is thrown away and the user gets an error message. This is a shame, because the whole point of aborts is to enable safe retries.

Lỗi rồi sẽ chắc chắn xảy ra, nhưng nhiều lập trình viên phần mềm thích chỉ nghĩ về con đường suôn sẻ thay vì những phức tạp của việc xử lý lỗi. Ví dụ, các framework ánh xạ đối tượng–quan hệ (ORM) phổ biến như ActiveRecord của Rails và Django không thử lại các giao dịch đã bị hủy bỏ — lỗi thường dẫn đến một ngoại lệ trôi lên trên ngăn xếp, nên mọi đầu vào của người dùng bị vứt bỏ và người dùng nhận được một thông báo lỗi. Điều này thật đáng tiếc, vì toàn bộ điểm mấu chốt của việc hủy bỏ là để cho phép thử lại an toàn.

Although retrying an aborted transaction is a simple and effective error handling mechanism, it isn't perfect:

Mặc dù thử lại một giao dịch đã bị hủy bỏ là một cơ chế xử lý lỗi đơn giản và hiệu quả, nó không hoàn hảo:

- If the transaction actually succeeded, but the network failed while the server tried to acknowledge the successful commit to the client (so the client thinks it failed), then retrying the transaction causes it to be performed twice—unless you have an additional application-level deduplication mechanism in place.
- If the error is due to overload, retrying the transaction will make the problem worse, not better. To avoid such feedback cycles, you can limit the number of retries, use exponential backoff, and handle overload-related errors differently from other errors (if possible).
- It is only worth retrying after transient errors (for example due to deadlock, isolation violation, temporary network interruptions, and failover); after a permanent error (e.g., constraint violation) a retry would be pointless.
- If the transaction also has side effects outside of the database, those side effects may happen even if the transaction is aborted. For example, if you're sending an email, you wouldn't want to send the email again every time you retry the transaction. If you want to make sure that several different systems either commit or abort together, two-phase commit can help (we will discuss this in “Atomic Commit and Two-Phase Commit (2PC)”).
- If the client process fails while retrying, any data it was trying to write to the database is lost.
 - Nếu giao dịch thực ra đã thành công, nhưng mạng lại hỏng trong lúc máy chủ cố báo nhận về việc commit thành công tới client (nên client nghĩ rằng nó đã thất bại), thì việc thử lại giao dịch khiến nó được thực hiện hai lần — trừ khi bạn có một cơ chế khử trùng lặp bổ sung ở mức ứng dụng.
 - Nếu lỗi là do quá tải, thử lại giao dịch sẽ làm vấn đề tồi tệ hơn chứ không tốt hơn. Để tránh những vòng phản hồi như vậy, bạn có thể giới hạn số lần thử lại, dùng giãn cách lũy thừa (exponential backoff), và xử lý các lỗi liên quan đến quá tải khác với các lỗi khác (nếu có thể).
 - Chỉ đáng thử lại sau các lỗi nhất thời (ví dụ do bế tắc — deadlock, vi phạm tính cô lập, gián đoạn mạng tạm thời, và chuyển đổi dự phòng); còn sau một lỗi vĩnh viễn (ví dụ vi phạm ràng buộc) thì thử lại sẽ vô nghĩa.
 - Nếu giao dịch còn có những tác dụng phụ bên ngoài cơ sở dữ liệu, những tác dụng phụ đó có thể vẫn xảy ra ngay cả khi giao dịch bị hủy bỏ. Ví dụ, nếu bạn đang gửi một email, bạn sẽ không muốn gửi lại email đó mỗi lần thử lại giao dịch. Nếu bạn muốn chắc chắn rằng nhiều hệ thống khác nhau hoặc cùng commit hoặc cùng hủy bỏ, giao thức hai pha có thể giúp ích (ta sẽ bàn về điều này trong “Atomic Commit and Two-Phase Commit (2PC)”).
 - Nếu tiến trình client hỏng trong lúc đang thử lại, mọi dữ liệu nó đang cố ghi vào cơ sở dữ liệu đều bị mất.

Weak Isolation Levels — Các mức cô lập yếu

If two transactions don't touch the same data, they can safely be run in parallel, because neither depends on the other. Concurrency issues (race conditions) only come into play when one transaction reads data that is concurrently modified by another transaction, or when two transactions try to simultaneously modify the same data.

Nếu hai giao dịch không động đến cùng một dữ liệu, chúng có thể được chạy song song một cách an toàn, vì không cái nào phụ thuộc vào cái kia. Các vấn đề tương tranh (tình trạng tranh đua) chỉ nảy sinh khi một giao dịch đọc dữ liệu đang đồng thời bị một giao dịch khác sửa đổi, hoặc khi hai giao dịch cố đồng thời sửa đổi cùng một dữ liệu.

Concurrency bugs are hard to find by testing, because such bugs are only triggered when you get unlucky with the timing. Such timing issues might occur very rarely, and are usually difficult to reproduce. Concurrency is also very difficult to reason about, especially in a large application where you don't necessarily know which other pieces of code are accessing the database. Application development is difficult enough if you just have one user at a time; having many concurrent users makes it much harder still, because any piece of data could unexpectedly change at any time.

Các bug tương tranh rất khó tìm ra bằng kiểm thử, vì những bug như vậy chỉ bị kích hoạt khi bạn xui xẻo về thời điểm. Những vấn đề về thời điểm như vậy có thể xảy ra rất hiếm, và thường khó tái hiện. Tương tranh cũng rất khó suy luận, đặc biệt trong một ứng dụng lớn nơi bạn không nhất thiết biết những đoạn mã nào khác đang truy cập cơ sở dữ liệu. Phát triển ứng dụng đã đủ khó nếu bạn chỉ có một người dùng tại một thời điểm; có nhiều người dùng đồng thời khiến nó còn khó hơn nhiều, vì bất kỳ mẫu dữ liệu nào cũng có thể thay đổi bất ngờ bất cứ lúc nào.

For that reason, databases have long tried to hide concurrency issues from application developers by providing transaction isolation. In theory, isolation should make your life easier by letting you pretend that no concurrency is happening: serializable isolation means that the database guarantees that transactions have the same effect as if they ran serially (i.e., one at a time, without any concurrency).

Vì lý do đó, từ lâu cơ sở dữ liệu đã cố giấu các vấn đề tương tranh khỏi lập trình viên ứng dụng bằng cách cung cấp tính cô lập giao dịch. Về lý thuyết, tính cô lập nên làm cuộc sống của bạn dễ dàng hơn bằng cách cho bạn giả vờ rằng không có tương tranh nào đang xảy ra: cô lập khả năng tuần tự hóa nghĩa là cơ sở dữ liệu đảm bảo các giao dịch có cùng hiệu ứng như thể chúng chạy tuần tự (tức là từng cái một, không có tương tranh nào).

In practice, isolation is unfortunately not that simple. Serializable isolation has a performance cost, and many databases don't want to pay that price [8]. It's therefore common for systems to

use weaker levels of isolation, which protect against some concurrency issues, but not all. Those levels of isolation are much harder to understand, and they can lead to subtle bugs, but they are nevertheless used in practice [23].

Trong thực tế, đáng tiếc là tính cô lập không đơn giản như vậy. Cô lập khả tuần tự hóa có cái giá về hiệu năng, và nhiều cơ sở dữ liệu không muốn trả cái giá đó [8]. Do đó việc các hệ thống dùng các mức cô lập yếu hơn là chuyện thường thấy, vốn bảo vệ trước một số vấn đề tương tranh, nhưng không phải tất cả. Những mức cô lập đó khó hiểu hơn nhiều, và chúng có thể dẫn đến những bug tinh vi, nhưng dẫu vậy chúng vẫn được dùng trong thực tế [23].

Concurrency bugs caused by weak transaction isolation are not just a theoretical problem. They have caused substantial loss of money [24, 25], led to investigation by financial auditors [26], and caused customer data to be corrupted [27]. A popular comment on revelations of such problems is “Use an ACID database if you’re handling financial data!”—but that misses the point. Even many popular **relational database** systems (which are usually considered “ACID”) use weak isolation, so they wouldn’t necessarily have prevented these bugs from occurring.

Các bug tương tranh do tính cô lập giao dịch yếu gây ra không chỉ là một vấn đề lý thuyết. Chúng đã gây ra những tổn thất tiền bạc đáng kể [24, 25], dẫn đến việc bị kiểm toán viên tài chính điều tra [26], và làm dữ liệu khách hàng bị hỏng [27]. Một bình luận phổ biến trước những tiết lộ về các vấn đề như vậy là “Hãy dùng cơ sở dữ liệu ACID nếu bạn đang xử lý dữ liệu tài chính!” — nhưng điều đó đã không nắm được vấn đề cốt lõi. Ngay cả nhiều hệ cơ sở dữ liệu quan hệ phổ biến (vốn thường được coi là “ACID”) cũng dùng cô lập yếu, nên chúng không nhất thiết đã ngăn được những bug này khỏi xảy ra.

Rather than blindly relying on tools, we need to develop a good understanding of the kinds of concurrency problems that exist, and how to prevent them. Then we can build applications that are reliable and correct, using the tools at our disposal.

Thay vì mù quáng dựa vào công cụ, ta cần phát triển một sự hiểu biết tốt về các loại vấn đề tương tranh đang tồn tại, và cách ngăn chặn chúng. Khi đó ta có thể xây dựng các ứng dụng đáng tin cậy và đúng đắn, dùng những công cụ sẵn có trong tay.

In this section we will look at several weak (nonserializable) isolation levels that are used in practice, and discuss in detail what kinds of race conditions can and cannot occur, so that you can decide what level is appropriate to your application. Once we’ve done that, we will discuss serializability in detail (see “Serializability”). Our discussion of isolation levels will be informal, using examples. If you want rigorous definitions and analyses of their properties, you can find them in the academic literature [28, 29, 30].

Trong phần này ta sẽ xem xét một số mức cô lập yếu (không khả tuần tự hóa) được dùng trong thực tế, và thảo luận chi tiết những loại tình trạng tranh đua nào có thể và không thể xảy ra, để bạn có thể quyết định mức nào phù hợp với ứng dụng của mình. Sau khi làm xong điều đó, ta sẽ bàn về tính khả tuần tự hóa một cách chi tiết (xem “Tính khả tuần tự hóa”). Phần thảo luận về các mức cô lập của ta sẽ mang tính không chính thức, dùng ví dụ. Nếu bạn muốn các định nghĩa chặt chẽ và phân tích về các thuộc tính của chúng, bạn có thể tìm thấy chúng trong các tài liệu học thuật [28, 29, 30].

Read Committed — Read Committed

The most basic level of transaction isolation is read committed. It makes two guarantees:

Mức cô lập giao dịch cơ bản nhất là read committed. Nó đưa ra hai bảo đảm:

- When reading from the database, you will only see data that has been committed (no dirty reads).
- When writing to the database, you will only overwrite data that has been committed (no dirty writes).
 - Khi đọc từ cơ sở dữ liệu, bạn sẽ chỉ thấy dữ liệu đã được commit (không có đọc bẩn).
 - Khi ghi vào cơ sở dữ liệu, bạn sẽ chỉ ghi đè lên dữ liệu đã được commit (không có ghi bẩn).

Let's discuss these two guarantees in more detail.

Hãy bàn về hai bảo đảm này một cách chi tiết hơn.

No dirty reads — Không có đọc bẩn

Imagine a transaction has written some data to the database, but the transaction has not yet committed or aborted. Can another transaction see that uncommitted data? If yes, that is called a dirty read [2].

Hãy hình dung một giao dịch đã ghi một số dữ liệu vào cơ sở dữ liệu, nhưng giao dịch chưa commit hay hủy bỏ. Liệu một giao dịch khác có thể thấy dữ liệu chưa commit đó không? Nếu có, đó được gọi là một đọc bẩn (dirty read) [2].

Transactions running at the read committed isolation level must prevent dirty reads. This means that any writes by a transaction only become visible to others when that transaction commits (and then all of its writes become visible at once). This is illustrated in Figure 7-4, where user 1 has set $x = 3$, but user 2's get x still returns the old value, 2, while user 1 has not yet committed.

Các giao dịch chạy ở mức cô lập read committed phải ngăn chặn đọc bẩn. Điều này nghĩa là mọi thao tác ghi của một giao dịch chỉ trở nên hiển thị với những giao dịch khác khi giao dịch đó commit (và khi ấy tất cả các thao tác ghi của nó trở nên hiển thị cùng một lúc). Điều này được minh họa trong Hình 7-4, nơi người dùng 1 đã đặt $x = 3$, nhưng lệnh get x của người dùng 2 vẫn trả về giá trị cũ, 2, trong khi người dùng 1 chưa commit.

Figure 7-4. No dirty reads: user 2 sees the new value for x only after user 1's transaction has committed. — Hình 7-4. Không có đọc bẩn: người dùng 2 chỉ thấy giá trị mới của x sau khi giao dịch của người dùng 1 đã commit. There are a few reasons why it's useful to prevent dirty reads:

Có một vài lý do tại sao việc ngăn chặn đọc bẩn lại hữu ích:

- If a transaction needs to update several objects, a dirty read means that another transaction may see some of the updates but not others. For example, in Figure 7-2, the user sees the new unread email but not the updated counter. This is a dirty read of the email. Seeing the database in a partially updated state is confusing to users and may cause other transactions to take incorrect decisions.
- If a transaction aborts, any writes it has made need to be rolled back (like in Figure 7-3). If the database allows dirty reads, that means a transaction may see data that is later rolled back—i.e., which is never actually committed to the database. Reasoning about the consequences quickly becomes mind-bending.
 - Nếu một giao dịch cần cập nhật nhiều đối tượng, một đọc bẩn nghĩa là một giao dịch khác có thể thấy một số cập nhật nhưng không thấy những cập nhật khác. Ví dụ, trong Hình 7-2, người dùng thấy email chưa đọc mới nhưng không thấy bộ đếm đã cập nhật. Đây là một đọc bẩn của email. Việc thấy cơ sở dữ liệu ở trạng thái được cập nhật một

phần gây bối rối cho người dùng và có thể khiến các giao dịch khác đưa ra những quyết định sai.

- Nếu một giao dịch bị hủy bỏ, mọi thao tác ghi nó đã thực hiện cần được hoàn tác (như trong Hình 7-3). Nếu cơ sở dữ liệu cho phép đọc bản, điều đó nghĩa là một giao dịch có thể thấy dữ liệu mà về sau bị hoàn tác — tức là dữ liệu chưa bao giờ thực sự được commit vào cơ sở dữ liệu. Việc suy luận về các hệ quả nhanh chóng trở nên rối rắm.

No dirty writes — Không có ghi bẩn

What happens if two transactions concurrently try to update the same object in a database? We don't know in which order the writes will happen, but we normally assume that the later write overwrites the earlier write.

Điều gì xảy ra nếu hai giao dịch đồng thời cố cập nhật cùng một đối tượng trong một cơ sở dữ liệu? Ta không biết các thao tác ghi sẽ xảy ra theo thứ tự nào, nhưng ta thường giả định rằng thao tác ghi đến sau ghi đè lên thao tác ghi đến trước.

However, what happens if the earlier write is part of a transaction that has not yet committed, so the later write overwrites an uncommitted value? This is called a dirty write [28]. Transactions running at the read committed isolation level must prevent dirty writes, usually by delaying the second write until the first write's transaction has committed or aborted.

Tuy nhiên, điều gì xảy ra nếu thao tác ghi đến trước là một phần của một giao dịch chưa commit, nên thao tác ghi đến sau ghi đè lên một giá trị chưa commit? Đây được gọi là một ghi bẩn (dirty write) [28]. Các giao dịch chạy ở mức cô lập read committed phải ngăn chặn ghi bẩn, thường bằng cách trì hoãn thao tác ghi thứ hai cho đến khi giao dịch của thao tác ghi thứ nhất đã commit hoặc hủy bỏ.

By preventing dirty writes, this isolation level avoids some kinds of concurrency problems:

Bằng cách ngăn chặn ghi bẩn, mức cô lập này tránh được một số loại vấn đề tương tranh:

- If transactions update multiple objects, dirty writes can lead to a bad outcome. For example, consider Figure 7-5, which illustrates a used car sales website on which two people, Alice and Bob, are simultaneously trying to buy the same car. Buying a car requires two database writes: the listing on the website needs to be updated to reflect the buyer, and the sales invoice needs to be sent to the buyer. In the case of Figure 7-5, the sale is awarded to Bob (because he performs the winning update to the listings table), but the invoice is sent to Alice (because she performs the winning update to the invoices table). Read committed prevents such mishaps.
- However, read committed does not prevent the race condition between two counter increments in Figure 7-1. In this case, the second write happens after the first transaction has committed, so it's not a dirty write. It's still incorrect, but for a different reason—in “Preventing Lost Updates” we will discuss how to make such counter increments safe.
- Nếu các giao dịch cập nhật nhiều đối tượng, ghi bẩn có thể dẫn đến một kết cục tệ hại. Ví dụ, hãy xem Hình 7-5, minh họa một website bán xe hơi cũ trên đó hai người, Alice và Bob, đang đồng thời cố mua cùng một chiếc xe. Mua một chiếc xe đòi hỏi hai thao tác ghi cơ sở dữ liệu: tin đăng trên website cần được cập nhật để phản ánh người mua, và hóa đơn bán hàng cần được gửi cho người mua. Trong trường hợp ở Hình 7-5, vụ bán được trao cho Bob (vì anh ta thực hiện thao tác cập nhật thắng vào bảng tin đăng), nhưng hóa đơn lại được gửi cho Alice (vì cô ấy thực hiện thao tác cập nhật thắng vào bảng hóa đơn). Read committed ngăn được những sự cố như vậy.
- Tuy nhiên, read committed không ngăn được tình trạng tranh đua giữa hai lần tăng bộ đếm trong Hình 7-1. Trong trường hợp này, thao tác ghi thứ hai xảy ra sau khi giao

dịch thứ nhất đã commit, nên nó không phải một ghi bẩn. Nó vẫn sai, nhưng vì một lý do khác — trong “Ngăn chặn cập nhật bị mất” ta sẽ bàn cách làm cho những lần tăng bộ đếm như vậy an toàn.

Figure 7-5. With dirty writes, conflicting writes from different transactions can be mixed up.
— **Hình 7-5. Với ghi bẩn, các thao tác ghi xung đột từ những giao dịch khác nhau có thể bị trộn lẫn lung tung.**

Implementing read committed — Hiện thực read committed

Read committed is a very popular isolation level. It is the default setting in Oracle 11g, PostgreSQL, SQL Server 2012, MemSQL, and many other databases [8].

Read committed là một mức cô lập rất phổ biến. Nó là thiết lập mặc định trong Oracle 11g, PostgreSQL, SQL Server 2012, MemSQL, và nhiều cơ sở dữ liệu khác [8].

Most commonly, databases prevent dirty writes by using row-level locks: when a transaction wants to modify a particular object (row or document), it must first acquire a lock on that object. It must then hold that lock until the transaction is committed or aborted. Only one transaction can hold the lock for any given object; if another transaction wants to write to the same object, it must wait until the first transaction is committed or aborted before it can acquire the lock and continue. This locking is done automatically by databases in read committed mode (or stronger isolation levels).

Phổ biến nhất, cơ sở dữ liệu ngăn chặn ghi bẩn bằng cách dùng khóa ở mức hàng (row-level lock): khi một giao dịch muốn sửa đổi một đối tượng cụ thể (hàng hoặc tài liệu), trước tiên nó phải lấy một khóa trên đối tượng đó. Sau đó nó phải giữ khóa đó cho đến khi giao dịch được commit hoặc hủy bỏ. Chỉ một giao dịch có thể giữ khóa cho bất kỳ đối tượng cho trước nào; nếu một giao dịch khác muốn ghi vào cùng đối tượng đó, nó phải chờ cho đến khi giao dịch thứ nhất được commit hoặc hủy bỏ trước khi nó có thể lấy khóa và tiếp tục. Việc khóa này được cơ sở dữ liệu thực hiện tự động trong chế độ read committed (hoặc các mức cô lập mạnh hơn).

How do we prevent dirty reads? One option would be to use the same lock, and to require any transaction that wants to read an object to briefly acquire the lock and then release it again immediately after reading. This would ensure that a read couldn't happen while an object has a dirty, uncommitted value (because during that time the lock would be held by the transaction that has made the write).

Làm sao ta ngăn chặn đọc bẩn? Một lựa chọn là dùng cùng khóa đó, và yêu cầu bất kỳ giao dịch nào muốn đọc một đối tượng phải lấy khóa trong chốc lát rồi thả nó ra ngay lập tức sau khi đọc. Điều này sẽ đảm bảo rằng một thao tác đọc không thể xảy ra trong khi một đối tượng đang có một giá trị bẩn, chưa commit (vì trong khoảng thời gian đó khóa sẽ do giao dịch đã thực hiện thao tác ghi giữ).

However, the approach of requiring read locks does not work well in practice, because one long-running write transaction can force many read-only transactions to wait until the long-running transaction has completed. This harms the **response time** of read-only transactions and is bad for **operability**: a slowdown in one part of an application can have a knock-on effect in a completely different part of the application, due to waiting for locks.

Tuy nhiên, cách tiếp cận yêu cầu khóa đọc không hoạt động tốt trong thực tế, vì một giao dịch ghi chạy lâu có thể buộc nhiều giao dịch chỉ-đọc phải chờ cho đến khi giao dịch chạy lâu đó hoàn tất. Điều này gây hại cho thời gian phản hồi của các giao dịch chỉ-đọc và bất

lợi cho khả năng vận hành: một sự chậm đi ở một phần của ứng dụng có thể gây ra hiệu ứng dây chuyền sang một phần hoàn toàn khác của ứng dụng, do phải chờ khóa.

For that reason, most databases prevent dirty reads using the approach illustrated in Figure 7-4: for every object that is written, the database remembers both the old committed value and the new value set by the transaction that currently holds the write lock. While the transaction is ongoing, any other transactions that read the object are simply given the old value. Only when the new value is committed do transactions switch over to reading the new value.

Vì lý do đó, hầu hết cơ sở dữ liệu ngăn chặn đọc bẩn bằng cách tiếp cận được minh họa trong Hình 7-4: với mỗi đối tượng được ghi, cơ sở dữ liệu ghi nhớ cả giá trị cũ đã commit lẫn giá trị mới được đặt bởi giao dịch hiện đang giữ khóa ghi. Trong khi giao dịch còn đang diễn ra, bất kỳ giao dịch nào khác đọc đối tượng đó đơn giản được trao giá trị cũ. Chỉ khi giá trị mới được commit thì các giao dịch mới chuyển sang đọc giá trị mới.

Snapshot Isolation and Repeatable Read — Snapshot Isolation và Repeatable Read

If you look superficially at read committed isolation, you could be forgiven for thinking that it does everything that a transaction needs to do: it allows aborts (required for atomicity), it prevents reading the incomplete results of transactions, and it prevents concurrent writes from getting intermingled. Indeed, those are useful features, and much stronger guarantees than you can get from a system that has no transactions.

Nếu bạn nhìn hời hợt vào cô lập read committed, có thể tha thứ được nếu bạn nghĩ rằng nó làm mọi thứ mà một giao dịch cần làm: nó cho phép hủy bỏ (cần thiết cho tính nguyên tử), nó ngăn việc đọc các kết quả dở dang của các giao dịch, và nó ngăn các thao tác ghi đồng thời khỏi bị trộn lẫn. Quả thực, đó là những tính năng hữu ích, và là những bảo đảm mạnh hơn nhiều so với những gì bạn có thể nhận được từ một hệ thống không có giao dịch.

However, there are still plenty of ways in which you can have concurrency bugs when using this isolation level. For example, Figure 7-6 illustrates a problem that can occur with read committed.

Tuy nhiên, vẫn còn vô số cách mà bạn có thể gặp bug tương tranh khi dùng mức cô lập này. Ví dụ, Hình 7-6 minh họa một vấn đề có thể xảy ra với read committed.

Figure 7-6. Read skew: Alice observes the database in an inconsistent state. — Hình 7-6. Lỗi khi đọc (read skew): Alice quan sát cơ sở dữ liệu ở một trạng thái không nhất quán. Say Alice has \$1,000 of savings at a bank, split across two accounts with \$500 each. Now a transaction transfers \$100 from one of her accounts to the other. If she is unlucky enough to look at her list of account balances in the same moment as that transaction is being processed, she may see one account balance at a time before the incoming payment has arrived (with a balance of \$500), and the other account after the outgoing transfer has been made (the new balance being \$400). To Alice it now appears as though she only has a total of \$900 in her accounts—it seems that \$100 has vanished into thin air.

Giả sử Alice có 1.000 đô la tiền tiết kiệm tại một ngân hàng, chia ra hai tài khoản mỗi tài khoản 500 đô la. Giờ một giao dịch chuyển 100 đô la từ một trong các tài khoản của cô sang tài khoản kia. Nếu cô đủ xui xẻo để nhìn vào danh sách số dư tài khoản của mình đúng vào khoảnh khắc giao dịch đó đang được xử lý, cô có thể thấy một tài khoản ở thời điểm trước khi khoản tiền chuyển đến kịp tới (với số dư 500 đô la), và tài khoản kia ở thời

điểm sau khi khoản chuyển đi đã được thực hiện (số dư mới là 400 đô la). Với Alice giờ có vẻ như cô chỉ có tổng cộng 900 đô la trong các tài khoản của mình — có vẻ như 100 đô la đã bốc hơi vào không khí.

This anomaly is called a nonrepeatable read or read skew: if Alice were to read the balance of account 1 again at the end of the transaction, she would see a different value (\$600) than she saw in her previous query. Read skew is considered acceptable under read committed isolation: the account balances that Alice saw were indeed committed at the time when she read them.

Dị thường này được gọi là đọc không lặp lại được (nonrepeatable read) hoặc lệch khi đọc (read skew): nếu Alice đọc số dư của tài khoản 1 lần nữa vào cuối giao dịch, cô sẽ thấy một giá trị khác (600 đô la) so với giá trị cô đã thấy trong truy vấn trước đó. Lệch khi đọc được coi là chấp nhận được dưới cô lập read committed: các số dư tài khoản mà Alice đã thấy quả thực đã được commit tại thời điểm cô đọc chúng.

Note — Ghi chú The term skew is unfortunately overloaded: we previously used it in the sense of an unbalanced workload with hot spots (see “Skewed Workloads and Relieving Hot Spots”), whereas here it means timing anomaly.

Thuật ngữ lệch (skew) đáng tiếc lại bị gán cho nhiều nghĩa: trước đây ta đã dùng nó theo nghĩa một khối lượng công việc mất cân bằng có các điểm nóng (xem “Skewed Workloads and Relieving Hot Spots”), trong khi ở đây nó có nghĩa là một dị thường về thời điểm.

In Alice’s case, this is not a lasting problem, because she will most likely see consistent account balances if she reloads the online banking website a few seconds later. However, some situations cannot tolerate such temporary inconsistency:

Trong trường hợp của Alice, đây không phải là một vấn đề kéo dài, vì rất có khả năng cô sẽ thấy các số dư tài khoản nhất quán nếu cô tải lại website ngân hàng trực tuyến vài giây sau đó. Tuy nhiên, một số tình huống không thể chịu đựng được sự không nhất quán tạm thời như vậy:

Taking a backup requires making a copy of the entire database, which may take hours on a large database. During the time that the backup process is running, writes will continue to be made to the database. Thus, you could end up with some parts of the backup containing an older version of the data, and other parts containing a newer version. If you need to restore from such a backup, the inconsistencies (such as disappearing money) become permanent.

Việc tạo một bản sao lưu đòi hỏi tạo một bản chép của toàn bộ cơ sở dữ liệu, mà có thể mất hàng giờ trên một cơ sở dữ liệu lớn. Trong khoảng thời gian tiến trình sao lưu đang chạy, các thao tác ghi sẽ tiếp tục được thực hiện vào cơ sở dữ liệu. Do đó, bạn có thể rất cuộc có một bản sao lưu mà một số phần chứa một phiên bản dữ liệu cũ hơn, và những phần khác chứa một phiên bản mới hơn. Nếu bạn cần khôi phục từ một bản sao lưu như vậy, những sự không nhất quán (chẳng hạn tiền biến mất) trở nên vĩnh viễn.

Sometimes, you may want to run a query that scans over large parts of the database. Such queries are common in analytics (see “**Transaction Processing** or Analytics?”), or may be part of a periodic integrity check that everything is in order (monitoring for data corruption). These queries are likely to return nonsensical results if they observe parts of the database at different points in time.

Đôi khi, bạn có thể muốn chạy một truy vấn quét qua những phần lớn của cơ sở dữ liệu. Những truy vấn như vậy phổ biến trong phân tích (xem “Transaction Processing or Analytics?”), hoặc có thể là một phần của một lần kiểm tra toàn vẹn định kỳ rằng mọi thứ đều ổn (giám sát hỏng hóc dữ liệu). Những truy vấn này có khả năng trả về các kết quả vô nghĩa nếu chúng quan sát các phần của cơ sở dữ liệu ở những thời điểm khác nhau.

Snapshot isolation [28] is the most common solution to this problem. The idea is that each transaction reads from a consistent snapshot of the database—that is, the transaction sees all the data that was committed in the database at the start of the transaction. Even if the data is subsequently changed by another transaction, each transaction sees only the old data from that particular point in time.

Snapshot isolation [28] là giải pháp phổ biến nhất cho vấn đề này. Ý tưởng là mỗi giao dịch đọc từ một ảnh chụp nhất quán (consistent snapshot) của cơ sở dữ liệu — tức là giao dịch thấy tất cả dữ liệu đã được commit trong cơ sở dữ liệu tại thời điểm bắt đầu giao dịch. Ngay cả khi dữ liệu sau đó bị một giao dịch khác thay đổi, mỗi giao dịch chỉ thấy dữ liệu cũ từ đúng thời điểm cụ thể đó.

Snapshot isolation is a boon for long-running, read-only queries such as backups and analytics. It is very hard to reason about the meaning of a query if the data on which it operates is changing at the same time as the query is executing. When a transaction can see a consistent snapshot of the database, frozen at a particular point in time, it is much easier to understand.

Snapshot isolation là một điều phúc lành cho các truy vấn chỉ-đọc chạy lâu như sao lưu và phân tích. Rất khó suy luận về ý nghĩa của một truy vấn nếu dữ liệu mà nó thao tác đang thay đổi cùng lúc truy vấn đang thực thi. Khi một giao dịch có thể thấy một ảnh chụp nhất quán của cơ sở dữ liệu, đóng băng tại một thời điểm cụ thể, nó dễ hiểu hơn nhiều.

Snapshot isolation is a popular feature: it is supported by PostgreSQL, MySQL with the InnoDB storage engine, Oracle, SQL Server, and others [23, 31, 32].

Snapshot isolation là một tính năng phổ biến: nó được PostgreSQL, MySQL với engine lưu trữ InnoDB, Oracle, SQL Server, và những hệ khác hỗ trợ [23, 31, 32].

Implementing snapshot isolation — Hiện thực snapshot isolation

Like read committed isolation, implementations of snapshot isolation typically use write locks to prevent dirty writes (see “Implementing read committed”), which means that a transaction that makes a write can block the progress of another transaction that writes to the same object. However, reads do not require any locks. From a performance point of view, a key principle of snapshot isolation is readers never block writers, and writers never block readers. This allows a database to handle long-running read queries on a consistent snapshot at the same time as processing writes normally, without any lock contention between the two.

Giống như cơ chế read committed, các hiện thực snapshot isolation thường dùng khóa ghi để ngăn chặn ghi bẩn (xem “Hiện thực read committed”), nghĩa là một giao dịch thực hiện một thao tác ghi có thể chặn tiến độ của một giao dịch khác đang ghi vào cùng đối tượng. Tuy nhiên, các thao tác đọc không cần bất kỳ khóa nào. Dưới góc nhìn hiệu năng, một nguyên tắc then chốt của snapshot isolation là người đọc không bao giờ chặn người ghi, và người ghi không bao giờ chặn người đọc. Điều này cho phép một cơ sở dữ liệu xử lý các truy vấn đọc chạy lâu trên một ảnh chụp nhất quán đồng thời với việc xử lý các thao tác ghi như bình thường, mà không có bất kỳ tranh chấp khóa nào giữa hai bên.

To implement snapshot isolation, databases use a generalization of the mechanism we saw for preventing dirty reads in Figure 7-4. The database must potentially keep several different committed versions of an object, because various in-progress transactions may need to see the state of the database at different points in time. Because it maintains several versions of an object side by side, this technique is known as multi-version concurrency control (MVCC).

Để hiện thực snapshot isolation, cơ sở dữ liệu dùng một dạng tổng quát hóa của cơ chế mà ta đã thấy để ngăn chặn đọc bẩn trong Hình 7-4. Cơ sở dữ liệu có thể phải giữ vài phiên

bản đã commit khác nhau của một đối tượng, vì các giao dịch đang dở dang khác nhau có thể cần thấy trạng thái của cơ sở dữ liệu ở những thời điểm khác nhau. Vì nó duy trì vài phiên bản của một đối tượng cạnh nhau, kỹ thuật này được gọi là kiểm soát tương tranh đa phiên bản (multi-version concurrency control, MVCC).

If a database only needed to provide read committed isolation, but not snapshot isolation, it would be sufficient to keep two versions of an object: the committed version and the overwritten-but-not-yet-committed version. However, storage engines that support snapshot isolation typically use MVCC for their read committed isolation level as well. A typical approach is that read committed uses a separate snapshot for each query, while snapshot isolation uses the same snapshot for an entire transaction.

Nếu một cơ sở dữ liệu chỉ cần cung cấp cô lập read committed, chứ không phải snapshot isolation, thì giữ hai phiên bản của một đối tượng là đủ: phiên bản đã commit và phiên bản đã-bị-ghi-đề-nhưng-chưa-commit. Tuy nhiên, các engine lưu trữ hỗ trợ snapshot isolation thường dùng MVCC cho cả mức cô lập read committed của chúng nữa. Một cách tiếp cận điển hình là read committed dùng một ảnh chụp riêng cho mỗi truy vấn, trong khi snapshot isolation dùng cùng một ảnh chụp cho cả một giao dịch.

Figure 7-7 illustrates how MVCC-based snapshot isolation is implemented in PostgreSQL [31] (other implementations are similar). When a transaction is started, it is given a unique, always-increasing transaction ID (txid). Whenever a transaction writes anything to the database, the data it writes is tagged with the transaction ID of the writer.

Hình 7-7 minh họa cách snapshot isolation dựa trên MVCC được hiện thực trong PostgreSQL [31] (các hiện thực khác cũng tương tự). Khi một giao dịch được khởi động, nó được trao một định danh giao dịch (txid) duy nhất, luôn tăng. Mỗi khi một giao dịch ghi bất cứ thứ gì vào cơ sở dữ liệu, dữ liệu nó ghi được gắn thẻ bằng định danh giao dịch của người ghi.

Figure 7-7. Implementing snapshot isolation using multi-version objects. — Hình 7-7. Hiện thực snapshot isolation dùng các đối tượng đa phiên bản. Each row in a table has a `created_by` field, containing the ID of the transaction that inserted this row into the table. Moreover, each row has a `deleted_by` field, which is initially empty. If a transaction deletes a row, the row isn't actually deleted from the database, but it is marked for deletion by setting the `deleted_by` field to the ID of the transaction that requested the deletion. At some later time, when it is certain that no transaction can any longer access the deleted data, a **garbage collection** process in the database removes any rows marked for deletion and frees their space.

Mỗi hàng trong một bảng có một trường `created_by`, chứa định danh của giao dịch đã chèn hàng này vào bảng. Hơn nữa, mỗi hàng có một trường `deleted_by`, ban đầu để trống. Nếu một giao dịch xóa một hàng, hàng đó không thực sự bị xóa khỏi cơ sở dữ liệu, mà nó được đánh dấu để xóa bằng cách đặt trường `deleted_by` thành định danh của giao dịch đã yêu cầu việc xóa. Vào một thời điểm sau đó, khi chắc chắn rằng không còn giao dịch nào có thể truy cập dữ liệu đã xóa nữa, một tiến trình thu gom rác trong cơ sở dữ liệu loại bỏ mọi hàng được đánh dấu để xóa và giải phóng không gian của chúng.

An update is internally translated into a delete and a create. For example, in Figure 7-7, transaction 13 deducts \$100 from account 2, changing the balance from \$500 to \$400. The accounts table now actually contains two rows for account 2: a row with a balance of \$500 which was marked as deleted by transaction 13, and a row with a balance of \$400 which was created by transaction 13.

Một thao tác cập nhật được dịch nội bộ thành một thao tác xóa và một thao tác tạo. Ví dụ, trong Hình 7-7, giao dịch 13 trừ 100 đô la khỏi tài khoản 2, thay đổi số dư từ 500 đô la

xuống 400 đô la. Bảng accounts giờ thực ra chứa hai hàng cho tài khoản 2: một hàng có số dư 500 đô la được đánh dấu là đã xóa bởi giao dịch 13, và một hàng có số dư 400 đô la được tạo bởi giao dịch 13.

Visibility rules for observing a consistent snapshot — Quy tắc hiển thị để quan sát một ảnh chụp nhất quán

When a transaction reads from the database, transaction IDs are used to decide which objects it can see and which are invisible. By carefully defining **visibility** rules, the database can present a consistent snapshot of the database to the application. This works as follows:

Khi một giao dịch đọc từ cơ sở dữ liệu, các định danh giao dịch được dùng để quyết định những đối tượng nào nó có thể thấy và những đối tượng nào không hiển thị. Bằng cách định nghĩa cẩn thận các quy tắc hiển thị, cơ sở dữ liệu có thể trình ra một ảnh chụp nhất quán của cơ sở dữ liệu cho ứng dụng. Cơ chế này hoạt động như sau:

- At the start of each transaction, the database makes a list of all the other transactions that are in progress (not yet committed or aborted) at that time. Any writes that those transactions have made are ignored, even if the transactions subsequently commit.
- Any writes made by aborted transactions are ignored.
- Any writes made by transactions with a later transaction ID (i.e., which started after the current transaction started) are ignored, regardless of whether those transactions have committed.
- All other writes are visible to the application's queries.
 - Vào lúc bắt đầu mỗi giao dịch, cơ sở dữ liệu lập một danh sách tất cả các giao dịch khác đang dở dang (chưa commit hay hủy bỏ) tại thời điểm đó. Mọi thao tác ghi mà những giao dịch đó đã thực hiện đều bị bỏ qua, ngay cả khi các giao dịch sau đó commit.
 - Mọi thao tác ghi do các giao dịch đã bị hủy bỏ thực hiện đều bị bỏ qua.
 - Mọi thao tác ghi do các giao dịch có định danh giao dịch muộn hơn (tức là những giao dịch bắt đầu sau khi giao dịch hiện tại bắt đầu) thực hiện đều bị bỏ qua, bất kể những giao dịch đó đã commit hay chưa.
 - Mọi thao tác ghi khác đều hiển thị với các truy vấn của ứng dụng.

These rules apply to both creation and deletion of objects. In Figure 7-7, when transaction 12 reads from account 2, it sees a balance of \$500 because the deletion of the \$500 balance was made by transaction 13 (according to rule 3, transaction 12 cannot see a deletion made by transaction 13), and the creation of the \$400 balance is not yet visible (by the same rule).

Những quy tắc này áp dụng cho cả việc tạo lẫn xóa đối tượng. Trong Hình 7-7, khi giao dịch 12 đọc từ tài khoản 2, nó thấy số dư 500 đô la vì việc xóa số dư 500 đô la được thực hiện bởi giao dịch 13 (theo quy tắc 3, giao dịch 12 không thể thấy một thao tác xóa do giao dịch 13 thực hiện), và việc tạo số dư 400 đô la chưa hiển thị (theo cùng quy tắc đó).

Put another way, an object is visible if both of the following conditions are true:

Nói cách khác, một đối tượng hiển thị nếu cả hai điều kiện sau đều đúng:

- At the time when the reader's transaction started, the transaction that created the object had already committed.
- The object is not marked for deletion, or if it is, the transaction that requested deletion had not yet committed at the time when the reader's transaction started.
 - Vào thời điểm giao dịch của người đọc bắt đầu, giao dịch đã tạo ra đối tượng đó đã commit rồi.

- Đối tượng không bị đánh dấu để xóa, hoặc nếu có, giao dịch đã yêu cầu việc xóa chưa commit vào thời điểm giao dịch của người đọc bắt đầu.

A long-running transaction may continue using a snapshot for a long time, continuing to read values that (from other transactions' point of view) have long been overwritten or deleted. By never updating values in place but instead creating a new version every time a value is changed, the database can provide a consistent snapshot while incurring only a small overhead.

Một giao dịch chạy lâu có thể tiếp tục dùng một ảnh chụp trong một thời gian dài, tiếp tục đọc các giá trị mà (dưới góc nhìn của các giao dịch khác) từ lâu đã bị ghi đè hoặc xóa. Bằng cách không bao giờ cập nhật giá trị tại chỗ mà thay vào đó tạo một phiên bản mới mỗi khi một giá trị được thay đổi, cơ sở dữ liệu có thể cung cấp một ảnh chụp nhất quán trong khi chỉ tốn một chi phí nhỏ.

Indexes and snapshot isolation — Chỉ mục và snapshot isolation

How do indexes work in a multi-version database? One option is to have the index simply point to all versions of an object and require an index query to filter out any object versions that are not visible to the current transaction. When garbage collection removes old object versions that are no longer visible to any transaction, the corresponding index entries can also be removed.

Chỉ mục hoạt động như thế nào trong một cơ sở dữ liệu đa phiên bản? Một lựa chọn là để chỉ mục đơn giản trở tới tất cả các phiên bản của một đối tượng và yêu cầu một truy vấn chỉ mục lọc bỏ mọi phiên bản đối tượng không hiển thị với giao dịch hiện tại. Khi việc thu gom rác loại bỏ các phiên bản đối tượng cũ không còn hiển thị với bất kỳ giao dịch nào, các mục chỉ mục tương ứng cũng có thể bị loại bỏ.

In practice, many implementation details determine the performance of multi-version concurrency control. For example, PostgreSQL has optimizations for avoiding index updates if different versions of the same object can fit on the same page [31].

Trong thực tế, nhiều chi tiết hiện thực quyết định hiệu năng của kiểm soát tương tranh đa phiên bản. Ví dụ, PostgreSQL có các tối ưu hóa để tránh cập nhật chỉ mục nếu các phiên bản khác nhau của cùng một đối tượng có thể nằm vừa trên cùng một trang [31].

Another approach is used in CouchDB, Datomic, and LMDB. Although they also use B-trees (see “B-Trees”), they use an append-only/copy-on-write variant that does not overwrite pages of the tree when they are updated, but instead creates a new copy of each modified page. Parent pages, up to the root of the tree, are copied and updated to point to the new versions of their child pages. Any pages that are not affected by a write do not need to be copied, and remain immutable [33, 34, 35].

Một cách tiếp cận khác được dùng trong CouchDB, Datomic, và LMDB. Mặc dù chúng cũng dùng cây B (xem “B-Trees”), chúng dùng một biến thể chỉ-ghi-thêm/sao-chép-khi-ghi (append-only/copy-on-write) không ghi đè lên các trang của cây khi chúng được cập nhật, mà thay vào đó tạo một bản chép mới của mỗi trang bị sửa đổi. Các trang cha, cho tới gốc của cây, được sao chép và cập nhật để trở tới các phiên bản mới của các trang con của chúng. Mọi trang không bị ảnh hưởng bởi một thao tác ghi đều không cần được sao chép, và vẫn bất biến [33, 34, 35].

With append-only B-trees, every write transaction (or batch of transactions) creates a new B-tree root, and a particular root is a consistent snapshot of the database at the point in time when it was created. There is no need to filter out objects based on transaction IDs because subsequent writes cannot modify an existing B-tree; they can only create new tree roots. However, this approach also requires a background process for compaction and garbage collection.

Với các cây B chỉ-ghi-thêm, mỗi giao dịch ghi (hoặc lô giao dịch) tạo một gốc cây B mới, và một gốc cụ thể là một ảnh chụp nhất quán của cơ sở dữ liệu tại thời điểm nó được tạo. Không cần lọc bỏ các đối tượng dựa trên định danh giao dịch vì các thao tác ghi sau này không thể sửa đổi một cây B đang tồn tại; chúng chỉ có thể tạo các gốc cây mới. Tuy nhiên, cách tiếp cận này cũng đòi hỏi một tiến trình nền để nén và thu gom rác.

Repeatable read and naming confusion — Repeatable read và sự lẫn lộn tên gọi

Snapshot isolation is a useful isolation level, especially for read-only transactions. However, many databases that implement it call it by different names. In Oracle it is called serializable, and in PostgreSQL and MySQL it is called repeatable read [23].

Snapshot isolation là một mức cô lập hữu ích, đặc biệt cho các giao dịch chỉ-đọc. Tuy nhiên, nhiều cơ sở dữ liệu hiện thực nó lại gọi nó bằng những tên khác nhau. Trong Oracle nó được gọi là serializable, còn trong PostgreSQL và MySQL nó được gọi là repeatable read [23].

The reason for this naming confusion is that the SQL standard doesn't have the concept of snapshot isolation, because the standard is based on System R's 1975 definition of isolation levels [2] and snapshot isolation hadn't yet been invented then. Instead, it defines repeatable read, which looks superficially similar to snapshot isolation. PostgreSQL and MySQL call their snapshot isolation level repeatable read because it meets the requirements of the standard, and so they can claim standards compliance.

Lý do của sự lẫn lộn tên gọi này là chuẩn SQL không có khái niệm snapshot isolation, vì chuẩn này dựa trên định nghĩa các mức cô lập năm 1975 của System R [2] và snapshot isolation khi đó chưa được phát minh. Thay vào đó, chuẩn định nghĩa repeatable read, vốn nhìn bề ngoài tương tự snapshot isolation. PostgreSQL và MySQL gọi mức cô lập snapshot isolation của chúng là repeatable read vì nó đáp ứng các yêu cầu của chuẩn, và do đó chúng có thể tuyên bố tuân thủ chuẩn.

Unfortunately, the SQL standard's definition of isolation levels is flawed—it is ambiguous, imprecise, and not as implementation-independent as a standard should be [28]. Even though several databases implement repeatable read, there are big differences in the guarantees they actually provide, despite being ostensibly standardized [23]. There has been a formal definition of repeatable read in the research literature [29, 30], but most implementations don't satisfy that formal definition. And to top it off, IBM DB2 uses “repeatable read” to refer to serializability [8].

Đáng tiếc, định nghĩa các mức cô lập của chuẩn SQL có khiếm khuyết — nó mơ hồ, thiếu chính xác, và không độc lập với hiện thực như một chuẩn lẽ ra phải vậy [28]. Mặc dù nhiều cơ sở dữ liệu hiện thực repeatable read, có những khác biệt lớn trong những bảo đảm mà chúng thực sự cung cấp, dù bề ngoài đều được chuẩn hóa [23]. Đã có một định nghĩa hình thức về repeatable read trong tài liệu nghiên cứu [29, 30], nhưng hầu hết hiện thực không thỏa mãn định nghĩa hình thức đó. Và để hoàn tất bức tranh, IBM DB2 dùng “repeatable read” để chỉ tính khả tuần tự hóa [8].

As a result, nobody really knows what repeatable read means.

Kết quả là, chẳng ai thực sự biết repeatable read có nghĩa là gì.

Preventing Lost Updates — Ngăn chặn cập nhật bị mất

The read committed and snapshot isolation levels we’ve discussed so far have been primarily about the guarantees of what a read-only transaction can see in the presence of concurrent writes. We have mostly ignored the issue of two transactions writing concurrently—we have only discussed dirty writes (see “No dirty writes”), one particular type of write-write conflict that can occur.

Các mức cô lập read committed và snapshot isolation mà ta đã bàn cho đến nay chủ yếu nói về những bảo đảm về việc một giao dịch chỉ-đọc có thể thấy gì khi có các thao tác ghi đồng thời. Ta hầu như đã bỏ qua vấn đề hai giao dịch ghi đồng thời — ta mới chỉ bàn về ghi bẩn (xem “Không có ghi bẩn”), một loại xung đột ghi-ghi cụ thể có thể xảy ra.

There are several other interesting kinds of conflicts that can occur between concurrently writing transactions. The best known of these is the lost update problem, illustrated in Figure 7-1 with the example of two concurrent counter increments.

Có vài loại xung đột thú vị khác có thể xảy ra giữa các giao dịch đang ghi đồng thời. Loại nổi tiếng nhất trong số đó là vấn đề cập nhật bị mất (lost update), được minh họa trong Hình 7-1 với ví dụ về hai lần tăng bộ đếm đồng thời.

The lost update problem can occur if an application reads some value from the database, modifies it, and writes back the modified value (a read-modify-write cycle). If two transactions do this concurrently, one of the modifications can be lost, because the second write does not include the first modification. (We sometimes say that the later write clobbers the earlier write.) This pattern occurs in various different scenarios:

Vấn đề cập nhật bị mất có thể xảy ra nếu một ứng dụng đọc một giá trị nào đó từ cơ sở dữ liệu, sửa đổi nó, rồi ghi giá trị đã sửa đổi trở lại (một chu trình đọc-sửa-ghi). Nếu hai giao dịch làm điều này đồng thời, một trong các sửa đổi có thể bị mất, vì thao tác ghi thứ hai không bao gồm sửa đổi thứ nhất. (Ta đôi khi nói rằng thao tác ghi đến sau đè bẹp thao tác ghi đến trước.) Mẫu này xảy ra trong nhiều kịch bản khác nhau:

- Incrementing a counter or updating an account balance (requires reading the current value, calculating the new value, and writing back the updated value)
- Making a local change to a complex value, e.g., adding an element to a list within a JSON document (requires parsing the document, making the change, and writing back the modified document)
- Two users editing a wiki page at the same time, where each user saves their changes by sending the entire page contents to the server, overwriting whatever is currently in the database
 - Tăng một bộ đếm hoặc cập nhật số dư tài khoản (đòi hỏi đọc giá trị hiện tại, tính giá trị mới, và ghi giá trị đã cập nhật trở lại)
 - Thực hiện một thay đổi cục bộ trên một giá trị phức tạp, ví dụ thêm một phần tử vào một danh sách bên trong một tài liệu JSON (đòi hỏi phân tích tài liệu, thực hiện thay đổi, và ghi tài liệu đã sửa đổi trở lại)
 - Hai người dùng cùng chỉnh sửa một trang wiki một lúc, trong đó mỗi người lưu thay đổi của mình bằng cách gửi toàn bộ nội dung trang lên máy chủ, ghi đè lên bất cứ thứ gì hiện đang ở trong cơ sở dữ liệu

Because this is such a common problem, a variety of solutions have been developed.

Vì đây là một vấn đề rất phổ biến, đã có nhiều giải pháp khác nhau được phát triển.

Atomic write operations — Thao tác ghi nguyên tử

Many databases provide atomic update operations, which remove the need to implement read-modify-write cycles in **application code**. They are usually the best solution if your code can be expressed in terms of those operations. For example, the following instruction is concurrency-safe in most relational databases:

Nhiều cơ sở dữ liệu cung cấp các thao tác cập nhật nguyên tử, vốn loại bỏ nhu cầu hiện thực các chu trình đọc-sửa-ghi trong mã ứng dụng. Chúng thường là giải pháp tốt nhất nếu mã của bạn có thể được diễn đạt qua các thao tác đó. Ví dụ, câu lệnh sau là an toàn về tương tranh trong hầu hết các cơ sở dữ liệu quan hệ:

```
UPDATE counters SET value = value + 1 WHERE key = 'foo';
```

Similarly, document databases such as MongoDB provide atomic operations for making local modifications to a part of a JSON document, and Redis provides atomic operations for modifying data structures such as priority queues. Not all writes can easily be expressed in terms of atomic operations—for example, updates to a wiki page involve arbitrary text editing—but in situations where atomic operations can be used, they are usually the best choice.

Tương tự, các cơ sở dữ liệu tài liệu như MongoDB cung cấp các thao tác nguyên tử để thực hiện những sửa đổi cục bộ vào một phần của một tài liệu JSON, và Redis cung cấp các thao tác nguyên tử để sửa đổi các cấu trúc dữ liệu như hàng đợi ưu tiên. Không phải mọi thao tác ghi đều có thể dễ dàng diễn đạt qua các thao tác nguyên tử — ví dụ, các cập nhật vào một trang wiki liên quan đến việc chỉnh sửa văn bản tùy ý — nhưng trong những tình huống mà các thao tác nguyên tử có thể được dùng, chúng thường là lựa chọn tốt nhất.

Atomic operations are usually implemented by taking an exclusive lock on the object when it is read so that no other transaction can read it until the update has been applied. This technique is sometimes known as **cursor** stability [36, 37]. Another option is to simply force all atomic operations to be executed on a single thread.

Các thao tác nguyên tử thường được hiện thực bằng cách lấy một khóa độc quyền trên đối tượng khi nó được đọc sao cho không giao dịch nào khác có thể đọc nó cho đến khi cập nhật đã được áp dụng. Kỹ thuật này đôi khi được gọi là tính ổn định con trỏ (cursor stability) [36, 37]. Một lựa chọn khác là đơn giản buộc tất cả các thao tác nguyên tử được thực thi trên một luồng duy nhất.

Unfortunately, object-relational mapping frameworks make it easy to accidentally write code that performs unsafe read-modify-write cycles instead of using atomic operations provided by the database [38]. That's not a problem if you know what you are doing, but it is potentially a source of subtle bugs that are difficult to find by testing.

Đáng tiếc, các framework ánh xạ đối tượng-quan hệ khiến việc vô tình viết mã thực hiện các chu trình đọc-sửa-ghi không an toàn thay vì dùng các thao tác nguyên tử do cơ sở dữ liệu cung cấp trở nên dễ dàng [38]. Đó không phải vấn đề nếu bạn biết mình đang làm gì, nhưng nó có thể là một nguồn của những bug tinh vi khó tìm ra bằng kiểm thử.

Explicit locking — Khóa tường minh

Another option for preventing lost updates, if the database's built-in atomic operations don't provide the necessary functionality, is for the application to explicitly lock objects that are going to be updated. Then the application can perform a read-modify-write cycle, and if any other transaction tries to concurrently read the same object, it is forced to wait until the first read-modify-write cycle has completed.

Một lựa chọn khác để ngăn chặn cập nhật bị mất, nếu các thao tác nguyên tử có sẵn của cơ sở dữ liệu không cung cấp chức năng cần thiết, là để ứng dụng khóa một cách tường minh các đối tượng sắp được cập nhật. Khi đó ứng dụng có thể thực hiện một chu trình đọc-sửa-ghi, và nếu bất kỳ giao dịch nào khác cố đồng thời đọc cùng đối tượng đó, nó bị buộc phải chờ cho đến khi chu trình đọc-sửa-ghi thứ nhất đã hoàn tất.

For example, consider a multiplayer game in which several players can move the same figure concurrently. In this case, an atomic operation may not be sufficient, because the application also needs to ensure that a player's move abides by the rules of the game, which involves some logic that you cannot sensibly implement as a database query. Instead, you may use a lock to prevent two players from concurrently moving the same piece, as illustrated in Example 7-1.

Ví dụ, hãy xem một trò chơi nhiều người chơi trong đó nhiều người chơi có thể đồng thời di chuyển cùng một quân cờ. Trong trường hợp này, một thao tác nguyên tử có thể không đủ, vì ứng dụng còn cần đảm bảo rằng nước đi của một người chơi tuân thủ luật chơi, vốn liên quan đến một số logic mà bạn không thể hiện thực một cách hợp lý dưới dạng một truy vấn cơ sở dữ liệu. Thay vào đó, bạn có thể dùng một khóa để ngăn hai người chơi đồng thời di chuyển cùng một quân, như được minh họa trong Ví dụ 7-1.

Example 7-1. Explicitly locking rows to prevent lost updates — Ví dụ 7-1. Khóa tường minh các hàng để ngăn chặn cập nhật bị mất

```
BEGIN TRANSACTION;
SELECT * FROM figures
  WHERE name = 'robot' AND game_id = 222
  FOR UPDATE;
-- Check whether move is valid, then update the position
-- of the piece that was returned by the previous SELECT.
UPDATE figures SET position = 'c4' WHERE id = 1234;
COMMIT;
```

The FOR UPDATE clause indicates that the database should take a lock on all rows returned by this query.

Mệnh đề FOR UPDATE chỉ thị rằng cơ sở dữ liệu nên lấy một khóa trên tất cả các hàng do truy vấn này trả về.

This works, but to get it right, you need to carefully think about your application logic. It's easy to forget to add a necessary lock somewhere in the code, and thus introduce a race condition.

Cách này hoạt động, nhưng để làm cho đúng, bạn cần suy nghĩ cẩn thận về logic ứng dụng của mình. Rất dễ quên thêm một khóa cần thiết ở đâu đó trong mã, và do đó đưa vào một tình trạng tranh đua.

Automatically detecting lost updates — Tự động phát hiện cập nhật bị mất

Atomic operations and locks are ways of preventing lost updates by forcing the read-modify-write cycles to happen sequentially. An alternative is to allow them to execute in parallel and, if the transaction manager detects a lost update, abort the transaction and force it to retry its read-modify-write cycle.

Các thao tác nguyên tử và khóa là những cách ngăn chặn cập nhật bị mất bằng cách buộc các chu trình đọc-sửa-ghi xảy ra tuần tự. Một cách thay thế là cho phép chúng thực thi

song song và, nếu bộ quản lý giao dịch phát hiện một cập nhật bị mất, hủy bỏ giao dịch và buộc nó thử lại chu trình đọc-sửa-ghi của mình.

An advantage of this approach is that databases can perform this check efficiently in conjunction with snapshot isolation. Indeed, PostgreSQL’s repeatable read, Oracle’s serializable, and SQL Server’s snapshot isolation levels automatically detect when a lost update has occurred and abort the offending transaction. However, MySQL/InnoDB’s repeatable read does not detect lost updates [23]. Some authors [28, 30] argue that a database must prevent lost updates in order to qualify as providing snapshot isolation, so MySQL does not provide snapshot isolation under this definition.

Một lợi thế của cách tiếp cận này là cơ sở dữ liệu có thể thực hiện phép kiểm tra này một cách hiệu quả kết hợp với snapshot isolation. Quả thực, các mức cô lập repeatable read của PostgreSQL, serializable của Oracle, và snapshot isolation của SQL Server tự động phát hiện khi một cập nhật bị mất đã xảy ra và hủy bỏ giao dịch gây lỗi. Tuy nhiên, repeatable read của MySQL/InnoDB không phát hiện cập nhật bị mất [23]. Một số tác giả [28, 30] lập luận rằng một cơ sở dữ liệu phải ngăn chặn cập nhật bị mất để đủ điều kiện cung cấp snapshot isolation, nên theo định nghĩa này MySQL không cung cấp snapshot isolation.

Lost update detection is a great feature, because it doesn’t require application code to use any special database features—you may forget to use a lock or an atomic operation and thus introduce a bug, but lost update detection happens automatically and is thus less error-prone.

Phát hiện cập nhật bị mất là một tính năng tuyệt vời, vì nó không đòi hỏi mã ứng dụng phải dùng bất kỳ tính năng cơ sở dữ liệu đặc biệt nào — bạn có thể quên dùng một khóa hoặc một thao tác nguyên tử và do đó đưa vào một bug, nhưng việc phát hiện cập nhật bị mất xảy ra tự động và do đó ít gây lỗi hơn.

Compare-and-set — So-sánh-và-gán

In databases that don’t provide transactions, you sometimes find an atomic compare-and-set operation (previously mentioned in “Single-object writes”). The purpose of this operation is to avoid lost updates by allowing an update to happen only if the value has not changed since you last read it. If the current value does not match what you previously read, the update has no effect, and the read-modify-write cycle must be retried.

Trong các cơ sở dữ liệu không cung cấp giao dịch, đôi khi bạn tìm thấy một thao tác so-sánh-và-gán (compare-and-set) nguyên tử (đã được nhắc đến trước đây trong “Thao tác ghi đơn đối tượng”). Mục đích của thao tác này là tránh cập nhật bị mất bằng cách cho phép một cập nhật chỉ xảy ra nếu giá trị chưa thay đổi kể từ lần cuối bạn đọc nó. Nếu giá trị hiện tại không khớp với những gì bạn đã đọc trước đó, thao tác cập nhật không có hiệu lực, và chu trình đọc-sửa-ghi phải được thử lại.

For example, to prevent two users concurrently updating the same wiki page, you might try something like this, expecting the update to occur only if the content of the page hasn’t changed since the user started editing it:

Ví dụ, để ngăn hai người dùng đồng thời cập nhật cùng một trang wiki, bạn có thể thử một thứ đại loại như sau, mong rằng thao tác cập nhật chỉ xảy ra nếu nội dung của trang chưa thay đổi kể từ khi người dùng bắt đầu chỉnh sửa nó:

```
-- This may or may not be safe, depending on the database implementation
UPDATE wiki_pages SET content = 'new content'
WHERE id = 1234 AND content = 'old content';
```

If the content has changed and no longer matches ‘old content’, this update will have no effect, so you need to check whether the update took effect and retry if necessary. However, if the database allows the WHERE clause to read from an old snapshot, this statement may not prevent lost updates, because the condition may be true even though another concurrent write is occurring. Check whether your database’s compare-and-set operation is safe before relying on it.

Nếu nội dung đã thay đổi và không còn khớp với ‘old content’, thao tác cập nhật này sẽ không có hiệu lực, nên bạn cần kiểm tra xem cập nhật có hiệu lực hay không và thử lại nếu cần. Tuy nhiên, nếu cơ sở dữ liệu cho phép mệnh đề WHERE đọc từ một ảnh chụp cũ, câu lệnh này có thể không ngăn được cập nhật bị mất, vì điều kiện có thể đúng ngay cả khi một thao tác ghi đồng thời khác đang xảy ra. Hãy kiểm tra xem thao tác so-sánh-và-gán của cơ sở dữ liệu của bạn có an toàn không trước khi dựa vào nó.

Conflict resolution and replication — Giải quyết xung đột và nhân bản

In replicated databases (see Chapter 5), preventing lost updates takes on another dimension: since they have copies of the data on multiple nodes, and the data can potentially be modified concurrently on different nodes, some additional steps need to be taken to prevent lost updates.

Trong các cơ sở dữ liệu có nhân bản (xem Chương 5), việc ngăn chặn cập nhật bị mất mang thêm một chiều kích khác: vì chúng có các bản chép của dữ liệu trên nhiều nút, và dữ liệu có thể bị sửa đổi đồng thời trên các nút khác nhau, nên cần thực hiện một số bước bổ sung để ngăn chặn cập nhật bị mất.

Locks and compare-and-set operations assume that there is a single up-to-date copy of the data. However, databases with multi-leader or leaderless replication usually allow several writes to happen concurrently and replicate them asynchronously, so they cannot guarantee that there is a single up-to-date copy of the data. Thus, techniques based on locks or compare-and-set do not apply in this context. (We will revisit this issue in more detail in “Linearizability”.)

Các khóa và thao tác so-sánh-và-gán giả định rằng có một bản chép dữ liệu duy nhất, cập nhật. Tuy nhiên, các cơ sở dữ liệu với nhân bản đa-leader hoặc không-leader thường cho phép vài thao tác ghi xảy ra đồng thời và nhân bản chúng một cách bất đồng bộ, nên chúng không thể đảm bảo rằng có một bản chép dữ liệu duy nhất, cập nhật. Do đó, các kỹ thuật dựa trên khóa hoặc so-sánh-và-gán không áp dụng được trong ngữ cảnh này. (Ta sẽ trở lại vấn đề này một cách chi tiết hơn trong “Linearizability”.)

Instead, as discussed in “Detecting Concurrent Writes”, a common approach in such replicated databases is to allow concurrent writes to create several conflicting versions of a value (also known as siblings), and to use application code or special data structures to resolve and merge these versions after the fact.

Thay vào đó, như đã bàn trong “Detecting Concurrent Writes”, một cách tiếp cận phổ biến trong các cơ sở dữ liệu có nhân bản như vậy là cho phép các thao tác ghi đồng thời tạo ra vài phiên bản xung đột của một giá trị (còn gọi là các bản anh em — siblings), và dùng mã ứng dụng hoặc các cấu trúc dữ liệu đặc biệt để giải quyết và hợp nhất các phiên bản này sau đó.

Atomic operations can work well in a replicated context, especially if they are commutative (i.e., you can apply them in a different order on different replicas, and still get the same result). For example, incrementing a counter or adding an element to a set are commutative operations. That is the idea behind Riak 2.0 datatypes, which prevent lost updates across replicas. When a value is concurrently updated by different clients, Riak automatically merges together the updates in such a way that no updates are lost [39].

Các thao tác nguyên tử có thể hoạt động tốt trong một ngữ cảnh có nhân bản, đặc biệt nếu chúng có tính giao hoán (tức là bạn có thể áp dụng chúng theo thứ tự khác nhau trên các bản sao khác nhau, mà vẫn nhận được cùng kết quả). Ví dụ, tăng một bộ đếm hoặc thêm một phần tử vào một tập hợp là các thao tác giao hoán. Đó là ý tưởng đằng sau các kiểu dữ liệu của Riak 2.0, vốn ngăn chặn cập nhật bị mất xuyên các bản sao. Khi một giá trị bị các client khác nhau cập nhật đồng thời, Riak tự động hợp nhất các cập nhật lại với nhau theo cách sao cho không có cập nhật nào bị mất [39].

On the other hand, the last write wins (LWW) conflict resolution method is prone to lost updates, as discussed in “Last write wins (discarding concurrent writes)”. Unfortunately, LWW is the default in many replicated databases.

Mặt khác, phương pháp giải quyết xung đột “ghi sau thắng” (last write wins, LWW) dễ gây cập nhật bị mất, như đã bàn trong “Last write wins (discarding concurrent writes)”. Đáng tiếc, LWW là mặc định trong nhiều cơ sở dữ liệu có nhân bản.

Write Skew and Phantoms — Lịch khi ghi và Bóng ma

In the previous sections we saw dirty writes and lost updates, two kinds of race conditions that can occur when different transactions concurrently try to write to the same objects. In order to avoid data corruption, those race conditions need to be prevented—either automatically by the database, or by manual safeguards such as using locks or atomic write operations.

Trong các phần trước ta đã thấy ghi bẩn và cập nhật bị mất, hai loại tình trạng tranh đua có thể xảy ra khi các giao dịch khác nhau đồng thời cố ghi vào cùng các đối tượng. Để tránh hỏng dữ liệu, những tình trạng tranh đua đó cần được ngăn chặn — hoặc tự động bởi cơ sở dữ liệu, hoặc bằng các biện pháp bảo vệ thủ công như dùng khóa hoặc các thao tác ghi nguyên tử.

However, that is not the end of the list of potential race conditions that can occur between concurrent writes. In this section we will see some subtler examples of conflicts.

Tuy nhiên, đó không phải là điểm cuối của danh sách những tình trạng tranh đua tiềm tàng có thể xảy ra giữa các thao tác ghi đồng thời. Trong phần này ta sẽ thấy một vài ví dụ tinh vi hơn về xung đột.

To begin, imagine this example: you are writing an application for doctors to manage their on-call shifts at a hospital. The hospital usually tries to have several doctors on call at any one time, but it absolutely must have at least one doctor on call. Doctors can give up their shifts (e.g., if they are sick themselves), provided that at least one colleague remains on call in that shift [40, 41].

Để bắt đầu, hãy hình dung ví dụ này: bạn đang viết một ứng dụng cho các bác sĩ quản lý các ca trực của họ tại một bệnh viện. Bệnh viện thường cố có vài bác sĩ trực tại bất kỳ thời điểm nào, nhưng nó tuyệt đối phải có ít nhất một bác sĩ trực. Các bác sĩ có thể từ bỏ ca của mình (ví dụ, nếu bản thân họ bị ốm), miễn là còn ít nhất một đồng nghiệp trực trong ca đó [40, 41].

Now imagine that Alice and Bob are the two on-call doctors for a particular shift. Both are feeling unwell, so they both decide to request leave. Unfortunately, they happen to click the button to go off call at approximately the same time. What happens next is illustrated in Figure 7-8.

Giờ hãy hình dung rằng Alice và Bob là hai bác sĩ trực cho một ca cụ thể. Cả hai đều thấy không khỏe, nên cả hai đều quyết định xin nghỉ. Đáng tiếc, họ tình cờ nhấn nút để rời ca

trực vào khoảng cùng một thời điểm. Điều gì xảy ra tiếp theo được minh họa trong Hình 7-8.

Figure 7-8. Example of write skew causing an application bug. — Hình 7-8. Ví dụ về lệch khi ghi gây ra một bug ứng dụng. In each transaction, your application first checks that two or more doctors are currently on call; if yes, it assumes it's safe for one doctor to go off call. Since the database is using snapshot isolation, both checks return 2, so both transactions proceed to the next stage. Alice updates her own record to take herself off call, and Bob updates his own record likewise. Both transactions commit, and now no doctor is on call. Your requirement of having at least one doctor on call has been violated.

Trong mỗi giao dịch, ứng dụng của bạn trước tiên kiểm tra rằng hiện có hai bác sĩ trở lên đang trực; nếu có, nó giả định là an toàn để một bác sĩ rời ca trực. Vì cơ sở dữ liệu đang dùng snapshot isolation, cả hai phép kiểm tra đều trả về 2, nên cả hai giao dịch đều tiến tới giai đoạn tiếp theo. Alice cập nhật bản ghi của chính mình để tự rời ca trực, và Bob cũng cập nhật bản ghi của chính anh ta tương tự. Cả hai giao dịch commit, và giờ không có bác sĩ nào đang trực. Yêu cầu của bạn về việc có ít nhất một bác sĩ trực đã bị vi phạm.

Characterizing write skew — Đặc trưng hóa lệch khi ghi

This anomaly is called write skew [28]. It is neither a dirty write nor a lost update, because the two transactions are updating two different objects (Alice's and Bob's on-call records, respectively). It is less obvious that a conflict occurred here, but it's definitely a race condition: if the two transactions had run one after another, the second doctor would have been prevented from going off call. The anomalous behavior was only possible because the transactions ran concurrently.

Dị thường này được gọi là lệch khi ghi (write skew) [28]. Nó không phải là một ghi bản cũng chẳng phải một cập nhật bị mất, vì hai giao dịch đang cập nhật hai đối tượng khác nhau (bản ghi trực ca của Alice và của Bob, lần lượt). Ít rõ ràng hơn rằng một xung đột đã xảy ra ở đây, nhưng nó chắc chắn là một tình trạng tranh đua: nếu hai giao dịch đã chạy lần lượt cái này sau cái kia, bác sĩ thứ hai sẽ bị ngăn không cho rời ca trực. Hành vi dị thường này chỉ có thể xảy ra vì các giao dịch đã chạy đồng thời.

You can think of write skew as a generalization of the lost update problem. Write skew can occur if two transactions read the same objects, and then update some of those objects (different transactions may update different objects). In the special case where different transactions update the same object, you get a dirty write or lost update anomaly (depending on the timing).

Bạn có thể nghĩ về lệch khi ghi như một sự tổng quát hóa của vấn đề cập nhật bị mất. Lệch khi ghi có thể xảy ra nếu hai giao dịch đọc cùng các đối tượng, rồi cập nhật một số trong những đối tượng đó (các giao dịch khác nhau có thể cập nhật các đối tượng khác nhau). Trong trường hợp đặc biệt mà các giao dịch khác nhau cập nhật cùng một đối tượng, bạn nhận được một dị thường ghi bản hoặc cập nhật bị mất (tùy vào thời điểm).

We saw that there are various different ways of preventing lost updates. With write skew, our options are more restricted:

Ta đã thấy có nhiều cách khác nhau để ngăn chặn cập nhật bị mất. Với lệch khi ghi, các lựa chọn của ta bị hạn chế hơn:

- Atomic single-object operations don't help, as multiple objects are involved.
- The automatic detection of lost updates that you find in some implementations of snapshot isolation unfortunately doesn't help either: write skew is not automatically detected in

PostgreSQL's repeatable read, MySQL/InnoDB's repeatable read, Oracle's serializable, or SQL Server's snapshot isolation level [23]. Automatically preventing write skew requires true serializable isolation (see “Serializability”).

- Some databases allow you to configure constraints, which are then enforced by the database (e.g., uniqueness, foreign key constraints, or restrictions on a particular value). However, in order to specify that at least one doctor must be on call, you would need a constraint that involves multiple objects. Most databases do not have built-in support for such constraints, but you may be able to implement them with triggers or materialized views, depending on the database [42].
- If you can't use a serializable isolation level, the second-best option in this case is probably to explicitly lock the rows that the transaction depends on. In the doctors example, you could write something like the following: `BEGIN TRANSACTION; SELECT * FROM doctors WHERE on_call = true AND shift_id = 1234 FOR UPDATE; UPDATE doctors SET on_call = false WHERE name = 'Alice' AND shift_id = 1234; COMMIT;` As before, `FOR UPDATE` tells the database to lock all rows returned by this query.

- Các thao tác nguyên tử đơn đối tượng không giúp ích, vì có nhiều đối tượng liên quan.
- Việc tự động phát hiện cập nhật bị mất mà bạn thấy trong một số hiện thực của snapshot isolation đáng tiếc cũng không giúp ích được: lệch khi ghi không được tự động phát hiện trong repeatable read của PostgreSQL, repeatable read của MySQL/InnoDB, serializable của Oracle, hoặc mức snapshot isolation của SQL Server [23]. Việc tự động ngăn chặn lệch khi ghi đòi hỏi cô lập khả năng tuần tự hóa thực thụ (xem “Tính khả năng tuần tự hóa”).
- Một số cơ sở dữ liệu cho phép bạn cấu hình các ràng buộc, sau đó được cơ sở dữ liệu thực thi (ví dụ, tính duy nhất, ràng buộc khóa ngoại, hoặc các hạn chế trên một giá trị cụ thể). Tuy nhiên, để chỉ định rằng phải có ít nhất một bác sĩ trực, bạn sẽ cần một ràng buộc liên quan đến nhiều đối tượng. Hầu hết cơ sở dữ liệu không có hỗ trợ tích hợp cho những ràng buộc như vậy, nhưng bạn có thể hiện thực được chúng với trigger hoặc khung nhìn vật chất hóa (materialized view), tùy vào cơ sở dữ liệu [42].
- Nếu bạn không thể dùng một mức cô lập khả năng tuần tự hóa, lựa chọn tốt thứ hai trong trường hợp này có lẽ là khóa một cách tường minh các hàng mà giao dịch phụ thuộc vào. Trong ví dụ về các bác sĩ, bạn có thể viết một thứ đại loại như sau: `BEGIN TRANSACTION; SELECT * FROM doctors WHERE on_call = true AND shift_id = 1234 FOR UPDATE; UPDATE doctors SET on_call = false WHERE name = 'Alice' AND shift_id = 1234; COMMIT;` Như trước đây, `FOR UPDATE` bảo cơ sở dữ liệu khóa tất cả các hàng do truy vấn này trả về.

More examples of write skew — Thêm các ví dụ về lệch khi ghi

Write skew may seem like an esoteric issue at first, but once you're aware of it, you may notice more situations in which it can occur. Here are some more examples:

Lệch khi ghi thoạt đầu có vẻ là một vấn đề khó hiểu, nhưng một khi bạn đã nhận thức được nó, bạn có thể nhận ra thêm nhiều tình huống mà nó có thể xảy ra. Sau đây là một số ví dụ khác:

Say you want to enforce that there cannot be two bookings for the same meeting room at the same time [43]. When someone wants to make a booking, you first check for any conflicting bookings (i.e., bookings for the same room with an overlapping time range), and if none are found, you create the meeting (see Example 7-2).

Giả sử bạn muốn thực thi rằng không thể có hai lượt đặt cho cùng một phòng họp vào

cùng một lúc [43]. Khi ai đó muốn tạo một lượt đặt, trước tiên bạn kiểm tra mọi lượt đặt xung đột (tức là các lượt đặt cho cùng phòng với một khoảng thời gian chồng lấp), và nếu không tìm thấy lượt nào, bạn tạo cuộc họp (xem Ví dụ 7-2).

Example 7-2. A meeting room booking system tries to avoid double-booking (not safe under snapshot isolation) — Ví dụ 7-2. Một hệ thống đặt phòng họp cố tránh đặt trùng (không an toàn dưới snapshot isolation)

```
BEGIN TRANSACTION;
-- Check for any existing bookings that overlap with the period of noon-1pm
SELECT COUNT(*) FROM bookings
  WHERE room_id = 123 AND
         end_time > '2015-01-01 12:00' AND start_time < '2015-01-01 13:00';
-- If the previous query returned zero:
INSERT INTO bookings
  (room_id, start_time, end_time, user_id)
  VALUES (123, '2015-01-01 12:00', '2015-01-01 13:00', 666);
COMMIT;
```

Unfortunately, snapshot isolation does not prevent another user from concurrently inserting a conflicting meeting. In order to guarantee you won't get scheduling conflicts, you once again need serializable isolation.

Đáng tiếc, snapshot isolation không ngăn được một người dùng khác đồng thời chèn một cuộc họp xung đột. Để đảm bảo bạn sẽ không gặp xung đột về lịch, một lần nữa bạn cần cô lập khả năng tuần tự hóa.

In Example 7-1, we used a lock to prevent lost updates (that is, making sure that two players can't move the same figure at the same time). However, the lock doesn't prevent players from moving two different figures to the same position on the board or potentially making some other move that violates the rules of the game. Depending on the kind of rule you are enforcing, you might be able to use a unique constraint, but otherwise you're vulnerable to write skew.

Trong Ví dụ 7-1, ta đã dùng một khóa để ngăn chặn cập nhật bị mất (tức là đảm bảo rằng hai người chơi không thể di chuyển cùng một quân cờ một lúc). Tuy nhiên, khóa đó không ngăn các người chơi khỏi việc di chuyển hai quân cờ khác nhau tới cùng một vị trí trên bàn cờ hoặc có khả năng thực hiện một nước đi khác vi phạm luật chơi. Tùy vào loại luật bạn đang thực thi, bạn có thể dùng được một ràng buộc duy nhất, nhưng ngoài ra thì bạn dễ bị tổn thương trước lệch khi ghi.

On a website where each user has a unique username, two users may try to create accounts with the same username at the same time. You may use a transaction to check whether a name is taken and, if not, create an account with that name. However, like in the previous examples, that is not safe under snapshot isolation. Fortunately, a unique constraint is a simple solution here (the second transaction that tries to register the username will be aborted due to violating the constraint).

Trên một website nơi mỗi người dùng có một tên người dùng duy nhất, hai người dùng có thể cùng cố tạo tài khoản với cùng một tên người dùng một lúc. Bạn có thể dùng một giao dịch để kiểm tra xem một tên đã có người dùng chưa và, nếu chưa, tạo một tài khoản với tên đó. Tuy nhiên, như trong các ví dụ trước, điều đó không an toàn dưới snapshot isolation. May thay, một ràng buộc duy nhất là một giải pháp đơn giản ở đây (giao dịch thứ hai cố đăng ký tên người dùng sẽ bị hủy bỏ do vi phạm ràng buộc).

A service that allows users to spend money or points needs to check that a user doesn't spend more

than they have. You might implement this by inserting a tentative spending item into a user's account, listing all the items in the account, and checking that the sum is positive [44]. With write skew, it could happen that two spending items are inserted concurrently that together cause the balance to go negative, but that neither transaction notices the other.

Một dịch vụ cho phép người dùng tiêu tiền hoặc điểm cần kiểm tra rằng một người dùng không tiêu nhiều hơn số họ có. Bạn có thể hiện thực điều này bằng cách chèn một mục chi tiêu dự kiến vào tài khoản của một người dùng, liệt kê tất cả các mục trong tài khoản, và kiểm tra rằng tổng là dương [44]. Với lệch khi ghi, có thể xảy ra việc hai mục chi tiêu được chèn đồng thời mà cùng nhau khiến số dư âm, nhưng không giao dịch nào nhận ra giao dịch kia.

Phantoms causing write skew — Bóng ma gây ra lệch khi ghi

All of these examples follow a similar pattern:

Tất cả các ví dụ này đều theo một mẫu tương tự:

- A SELECT query checks whether some requirement is satisfied by searching for rows that match some search condition (there are at least two doctors on call, there are no existing bookings for that room at that time, the position on the board doesn't already have another figure on it, the username isn't already taken, there is still money in the account).
- Depending on the result of the first query, the application code decides how to continue (perhaps to go ahead with the operation, or perhaps to report an error to the user and abort).
- If the application decides to go ahead, it makes a write (INSERT, UPDATE, or DELETE) to the database and commits the transaction. The effect of this write changes the precondition of the decision of step 2. In other words, if you were to repeat the SELECT query from step 1 after committing the write, you would get a different result, because the write changed the set of rows matching the search condition (there is now one fewer doctor on call, the meeting room is now booked for that time, the position on the board is now taken by the figure that was moved, the username is now taken, there is now less money in the account).
- Một truy vấn SELECT kiểm tra xem một yêu cầu nào đó có được thỏa mãn không bằng cách tìm các hàng khớp với một điều kiện tìm kiếm (có ít nhất hai bác sĩ trực, không có lượt đặt nào hiện hữu cho phòng đó vào lúc đó, vị trí trên bàn cờ chưa có một quân cờ khác nào trên đó, tên người dùng chưa bị lấy, vẫn còn tiền trong tài khoản).
- Tùy vào kết quả của truy vấn thứ nhất, mã ứng dụng quyết định cách tiếp tục (có lẽ tiến hành thao tác, hoặc có lẽ báo lỗi cho người dùng và hủy bỏ).
- Nếu ứng dụng quyết định tiến hành, nó thực hiện một thao tác ghi (INSERT, UPDATE, hoặc DELETE) vào cơ sở dữ liệu và commit giao dịch. Hiệu ứng của thao tác ghi này thay đổi tiền đề của quyết định ở bước 2. Nói cách khác, nếu bạn lặp lại truy vấn SELECT từ bước 1 sau khi commit thao tác ghi, bạn sẽ nhận được một kết quả khác, vì thao tác ghi đã thay đổi tập các hàng khớp với điều kiện tìm kiếm (giờ có ít hơn một bác sĩ trực, phòng họp giờ đã được đặt vào lúc đó, vị trí trên bàn cờ giờ đã bị chiếm bởi quân cờ vừa được di chuyển, tên người dùng giờ đã bị lấy, giờ còn ít tiền hơn trong tài khoản).

The steps may occur in a different order. For example, you could first make the write, then the SELECT query, and finally decide whether to abort or commit based on the result of the query.

Các bước có thể xảy ra theo một thứ tự khác. Ví dụ, bạn có thể trước tiên thực hiện thao tác ghi, rồi truy vấn SELECT, và cuối cùng quyết định hủy bỏ hay commit dựa trên kết quả của truy vấn.

In the case of the doctor on call example, the row being modified in step 3 was one of the rows returned in step 1, so we could make the transaction safe and avoid write skew by locking the rows in step 1 (SELECT FOR UPDATE). However, the other four examples are different: they check for the absence of rows matching some search condition, and the write adds a row matching the same condition. If the query in step 1 doesn't return any rows, SELECT FOR UPDATE can't attach locks to anything.

Trong trường hợp ví dụ bác sĩ trực, hàng bị sửa đổi ở bước 3 là một trong các hàng được trả về ở bước 1, nên ta có thể làm cho giao dịch an toàn và tránh lệch khi ghi bằng cách khóa các hàng ở bước 1 (SELECT FOR UPDATE). Tuy nhiên, bốn ví dụ còn lại thì khác: chúng kiểm tra sự vắng mặt của các hàng khớp với một điều kiện tìm kiếm nào đó, và thao tác ghi thêm một hàng khớp với cùng điều kiện đó. Nếu truy vấn ở bước 1 không trả về hàng nào, SELECT FOR UPDATE không thể gắn khóa vào bất cứ thứ gì.

This effect, where a write in one transaction changes the result of a search query in another transaction, is called a phantom [3]. Snapshot isolation avoids phantoms in read-only queries, but in read-write transactions like the examples we discussed, phantoms can lead to particularly tricky cases of write skew.

Hiệu ứng này, trong đó một thao tác ghi trong một giao dịch thay đổi kết quả của một truy vấn tìm kiếm trong một giao dịch khác, được gọi là một bóng ma (phantom) [3]. Snapshot isolation tránh được bóng ma trong các truy vấn chỉ-đọc, nhưng trong các giao dịch đọc-ghi như các ví dụ ta đã bàn, bóng ma có thể dẫn đến những trường hợp lệch khi ghi đặc biệt hóc búa.

Materializing conflicts — Vật chất hóa xung đột

If the problem of phantoms is that there is no object to which we can attach the locks, perhaps we can artificially introduce a lock object into the database?

Nếu vấn đề của bóng ma là không có đối tượng nào để ta gắn khóa vào, thì có lẽ ta có thể nhân tạo đưa một đối tượng khóa vào cơ sở dữ liệu?

For example, in the meeting room booking case you could imagine creating a table of time slots and rooms. Each row in this table corresponds to a particular room for a particular time period (say, 15 minutes). You create rows for all possible combinations of rooms and time periods ahead of time, e.g. for the next six months.

Ví dụ, trong trường hợp đặt phòng họp bạn có thể hình dung việc tạo một bảng các khe thời gian và các phòng. Mỗi hàng trong bảng này ứng với một phòng cụ thể cho một khoảng thời gian cụ thể (giả sử, 15 phút). Bạn tạo trước các hàng cho tất cả các tổ hợp khả dĩ của phòng và khoảng thời gian, ví dụ cho sáu tháng tới.

Now a transaction that wants to create a booking can lock (SELECT FOR UPDATE) the rows in the table that correspond to the desired room and time period. After it has acquired the locks, it can check for overlapping bookings and insert a new booking as before. Note that the additional table isn't used to store information about the booking—it's purely a collection of locks which is used to prevent bookings on the same room and time range from being modified concurrently.

Giờ một giao dịch muốn tạo một lượt đặt có thể khóa (SELECT FOR UPDATE) các hàng trong bảng ứng với phòng và khoảng thời gian mong muốn. Sau khi nó đã lấy các khóa, nó có thể kiểm tra các lượt đặt chồng lấp và chèn một lượt đặt mới như trước đây. Lưu ý rằng bảng bổ sung không được dùng để lưu thông tin về lượt đặt — nó thuần túy là một tập hợp

các khóa được dùng để ngăn các lượt đặt cho cùng một phòng và khoảng thời gian khởi bị sửa đổi đồng thời.

This approach is called materializing conflicts, because it takes a phantom and turns it into a lock conflict on a concrete set of rows that exist in the database [11]. Unfortunately, it can be hard and error-prone to figure out how to materialize conflicts, and it's ugly to let a concurrency control mechanism leak into the application data model. For those reasons, materializing conflicts should be considered a last resort if no alternative is possible. A serializable isolation level is much preferable in most cases.

Cách tiếp cận này được gọi là vật chất hóa xung đột (materializing conflicts), vì nó lấy một bóng ma và biến nó thành một xung đột khóa trên một tập cụ thể các hàng tồn tại trong cơ sở dữ liệu [11]. Đáng tiếc, có thể rất khó và dễ sai khi tìm ra cách vật chất hóa xung đột, và thật xấu xí khi để một cơ chế kiểm soát tương tranh rò rỉ vào mô hình dữ liệu ứng dụng. Vì những lý do đó, vật chất hóa xung đột nên được coi là phương án cuối cùng nếu không có cách nào khác khả dĩ. Một mức cô lập khả tuần tự hóa là lựa chọn đáng ưu tiên hơn nhiều trong hầu hết các trường hợp.

Serializability — Tính khả tuần tự hóa

In this chapter we have seen several examples of transactions that are prone to race conditions. Some race conditions are prevented by the read committed and snapshot isolation levels, but others are not. We encountered some particularly tricky examples with write skew and phantoms. It's a sad situation:

Trong chương này ta đã thấy vài ví dụ về các giao dịch dễ gặp tình trạng tranh đua. Một số tình trạng tranh đua được ngăn chặn bởi các mức cô lập read committed và snapshot isolation, nhưng số khác thì không. Ta đã gặp một vài ví dụ đặc biệt hóc búa với lệch khi ghi và bóng ma. Đây là một tình cảnh đáng buồn:

- Isolation levels are hard to understand, and inconsistently implemented in different databases (e.g., the meaning of “repeatable read” varies significantly).
- If you look at your application code, it's difficult to tell whether it is safe to run at a particular isolation level—especially in a large application, where you might not be aware of all the things that may be happening concurrently.
- There are no good tools to help us detect race conditions. In principle, static analysis may help [26], but research techniques have not yet found their way into practical use. Testing for concurrency issues is hard, because they are usually nondeterministic—problems only occur if you get unlucky with the timing.
 - Các mức cô lập khó hiểu, và được hiện thực không nhất quán trong các cơ sở dữ liệu khác nhau (ví dụ, ý nghĩa của “repeatable read” thay đổi đáng kể).
 - Nếu bạn nhìn vào mã ứng dụng của mình, rất khó để nói liệu nó có an toàn để chạy ở một mức cô lập cụ thể hay không — đặc biệt trong một ứng dụng lớn, nơi bạn có thể không nhận thức được tất cả những thứ có thể đang xảy ra đồng thời.
 - Không có công cụ tốt nào giúp ta phát hiện tình trạng tranh đua. Về nguyên tắc, phân tích tĩnh có thể giúp ích [26], nhưng các kỹ thuật nghiên cứu vẫn chưa tìm được đường đến việc dùng thực tế. Kiểm thử cho các vấn đề tương tranh rất khó, vì chúng thường không tất định — các vấn đề chỉ xảy ra nếu bạn xui xẻo về thời điểm.

This is not a new problem—it has been like this since the 1970s, when weak isolation levels were first introduced [2]. All along, the answer from researchers has been simple: use serializable isolation!

Đây không phải là một vấn đề mới — nó đã như vậy kể từ thập niên 1970, khi các mức cô lập yếu lần đầu được giới thiệu [2]. Suốt từ đó, câu trả lời từ các nhà nghiên cứu vẫn đơn giản: hãy dùng cô lập khả tuần tự hóa!

Serializable isolation is usually regarded as the strongest isolation level. It guarantees that even though transactions may execute in parallel, the end result is the same as if they had executed one

at a time, serially, without any concurrency. Thus, the database guarantees that if the transactions behave correctly when run individually, they continue to be correct when run concurrently—in other words, the database prevents all possible race conditions.

Cô lập khả tuần tự hóa thường được coi là mức cô lập mạnh nhất. Nó đảm bảo rằng mặc dù các giao dịch có thể thực thi song song, kết quả cuối cùng giống như thể chúng đã thực thi từng cái một, tuần tự, không có tương tranh nào. Do đó, cơ sở dữ liệu đảm bảo rằng nếu các giao dịch hoạt động đúng khi chạy riêng lẻ, chúng tiếp tục đúng khi chạy đồng thời — nói cách khác, cơ sở dữ liệu ngăn chặn mọi tình trạng tranh đua khả dĩ.

But if serializable isolation is so much better than the mess of weak isolation levels, then why isn't everyone using it? To answer this question, we need to look at the options for implementing serializability, and how they perform. Most databases that provide serializability today use one of three techniques, which we will explore in the rest of this chapter:

Nhưng nếu cô lập khả tuần tự hóa tốt hơn nhiều so với mớ hỗn độn các mức cô lập yếu, thì tại sao không phải ai cũng dùng nó? Để trả lời câu hỏi này, ta cần nhìn vào các lựa chọn để hiện thực tính khả tuần tự hóa, và chúng có hiệu năng ra sao. Hầu hết các cơ sở dữ liệu cung cấp tính khả tuần tự hóa ngày nay dùng một trong ba kỹ thuật, mà ta sẽ khám phá trong phần còn lại của chương này:

- Literally executing transactions in a serial order (see “Actual Serial Execution”)
- Two-phase locking (see “Two-Phase Locking (2PL)”), which for several decades was the only viable option
- Optimistic concurrency control techniques such as serializable snapshot isolation (see “Serializable Snapshot Isolation (SSI)”)
 - Thực thi giao dịch theo đúng nghĩa đen trong một thứ tự tuần tự (xem “Thực thi tuần tự thực sự”)
 - Khóa hai pha (xem “Khóa hai pha (2PL)”), vốn trong vài thập kỷ là lựa chọn khả thi duy nhất
 - Các kỹ thuật kiểm soát tương tranh lạc quan như cô lập ảnh chụp khả tuần tự hóa (xem “Cô lập ảnh chụp khả tuần tự hóa (SSI)”)

For now, we will discuss these techniques primarily in the context of single-node databases; in Chapter 9 we will examine how they can be generalized to transactions that involve multiple nodes in a distributed system.

Hiện tại, ta sẽ bàn về những kỹ thuật này chủ yếu trong ngữ cảnh các cơ sở dữ liệu một nút; ở Chương 9 ta sẽ xem xét cách chúng có thể được tổng quát hóa cho các giao dịch liên quan đến nhiều nút trong một hệ phân tán.

Actual Serial Execution — Thực thi tuần tự thực sự

The simplest way of avoiding concurrency problems is to remove the concurrency entirely: to execute only one transaction at a time, in serial order, on a single thread. By doing so, we completely sidestep the problem of detecting and preventing conflicts between transactions: the resulting isolation is by definition serializable.

Cách đơn giản nhất để tránh các vấn đề tương tranh là loại bỏ tương tranh hoàn toàn: chỉ thực thi một giao dịch tại một thời điểm, theo thứ tự tuần tự, trên một luồng duy nhất. Bằng cách làm vậy, ta hoàn toàn né được vấn đề phát hiện và ngăn chặn xung đột giữa các giao dịch: sự cô lập thu được, theo định nghĩa, là khả tuần tự hóa.

Even though this seems like an obvious idea, database designers only fairly recently—around 2007—decided that a single-threaded loop for executing transactions was feasible [45]. If multi-threaded concurrency was considered essential for getting good performance during the previous 30 years, what changed to make single-threaded execution possible?

Mặc dù điều này có vẻ là một ý tưởng hiển nhiên, các nhà thiết kế cơ sở dữ liệu chỉ mới gần đây — khoảng năm 2007 — mới quyết định rằng một vòng lặp đơn luồng để thực thi giao dịch là khả thi [45]. Nếu tương tranh đa luồng được coi là thiết yếu để có hiệu năng tốt trong 30 năm trước đó, thì điều gì đã thay đổi để khiến việc thực thi đơn luồng trở nên khả thi?

Two developments caused this rethink:

Hai bước phát triển đã gây ra sự suy nghĩ lại này:

- RAM became cheap enough that for many use cases is now feasible to keep the entire active dataset in memory (see “Keeping everything in memory”). When all data that a transaction needs to access is in memory, transactions can execute much faster than if they have to wait for data to be loaded from disk.
- Database designers realized that OLTP transactions are usually short and only make a small number of reads and writes (see “Transaction Processing or Analytics?”). By contrast, long-running analytic queries are typically read-only, so they can be run on a consistent snapshot (using snapshot isolation) outside of the serial execution loop.
 - RAM trở nên đủ rẻ đến mức với nhiều tình huống sử dụng giờ là khả thi để giữ toàn bộ tập dữ liệu đang hoạt động trong bộ nhớ (xem “Keeping everything in memory”). Khi tất cả dữ liệu mà một giao dịch cần truy cập đều ở trong bộ nhớ, các giao dịch có thể thực thi nhanh hơn nhiều so với khi chúng phải chờ dữ liệu được nạp từ đĩa.
 - Các nhà thiết kế cơ sở dữ liệu nhận ra rằng các giao dịch OLTP thường ngắn và chỉ thực hiện một số ít thao tác đọc và ghi (xem “Transaction Processing or Analytics?”). Ngược lại, các truy vấn phân tích chạy lâu thường chỉ-đọc, nên chúng có thể được chạy trên một ảnh chụp nhất quán (dùng snapshot isolation) bên ngoài vòng lặp thực thi tuần tự.

The approach of executing transactions serially is implemented in VoltDB/H-Store, Redis, and Datomic [46, 47, 48]. A system designed for single-threaded execution can sometimes perform better than a system that supports concurrency, because it can avoid the coordination overhead of locking. However, its **throughput** is limited to that of a single CPU core. In order to make the most of that single thread, transactions need to be structured differently from their traditional form.

Cách tiếp cận thực thi giao dịch một cách tuần tự được hiện thực trong VoltDB/H-Store, Redis, và Datomic [46, 47, 48]. Một hệ thống được thiết kế cho thực thi đơn luồng đôi khi có thể hoạt động tốt hơn một hệ thống hỗ trợ tương tranh, vì nó có thể tránh được chi phí phối hợp của việc khóa. Tuy nhiên, thông lượng của nó bị giới hạn ở mức của một lõi CPU duy nhất. Để tận dụng tối đa luồng duy nhất đó, các giao dịch cần được cấu trúc khác đi so với dạng truyền thống của chúng.

Encapsulating transactions in stored procedures — Đóng gói giao dịch trong các thủ tục lưu trữ

In the early days of databases, the intention was that a database transaction could encompass an entire flow of user activity. For example, booking an airline ticket is a multi-stage process (searching for routes, fares, and available seats; deciding on an itinerary; booking seats on each of the flights

of the itinerary; entering passenger details; making payment). Database designers thought that it would be neat if that entire process was one transaction so that it could be committed atomically.

Trong những ngày đầu của cơ sở dữ liệu, ý định là một giao dịch cơ sở dữ liệu có thể bao trùm cả một luồng hoạt động của người dùng. Ví dụ, đặt một vé máy bay là một quá trình nhiều giai đoạn (tìm kiếm các tuyến bay, giá vé, và ghế trống; quyết định một hành trình; đặt ghế trên mỗi chuyến bay của hành trình; nhập thông tin hành khách; thanh toán). Các nhà thiết kế cơ sở dữ liệu nghĩ rằng sẽ gọn gàng nếu cả quá trình đó là một giao dịch sao cho nó có thể được commit một cách nguyên tử.

Unfortunately, humans are very slow to make up their minds and respond. If a database transaction needs to wait for input from a user, the database needs to support a potentially huge number of concurrent transactions, most of them idle. Most databases cannot do that efficiently, and so almost all OLTP applications keep transactions short by avoiding interactively waiting for a user within a transaction. On the web, this means that a transaction is committed within the same HTTP request—a transaction does not span multiple requests. A new HTTP request starts a new transaction.

Đáng tiếc, con người rất chậm trong việc quyết định và phản hồi. Nếu một giao dịch cơ sở dữ liệu cần chờ đầu vào từ một người dùng, cơ sở dữ liệu cần hỗ trợ một số lượng giao dịch đồng thời có thể cực lớn, hầu hết trong số đó ở trạng thái nhàn rỗi. Hầu hết cơ sở dữ liệu không thể làm điều đó một cách hiệu quả, và do đó gần như mọi ứng dụng OLTP đều giữ các giao dịch ngắn bằng cách tránh chờ tương tác từ một người dùng bên trong một giao dịch. Trên web, điều này có nghĩa là một giao dịch được commit trong cùng một yêu cầu HTTP — một giao dịch không trải dài qua nhiều yêu cầu. Một yêu cầu HTTP mới khởi động một giao dịch mới.

Even though the human has been taken out of the critical path, transactions have continued to be executed in an interactive client/server style, one statement at a time. An application makes a query, reads the result, perhaps makes another query depending on the result of the first query, and so on. The queries and results are sent back and forth between the application code (running on one machine) and the database server (on another machine).

Mặc dù con người đã được đưa ra khỏi đường đi tới hạn, các giao dịch vẫn tiếp tục được thực thi theo phong cách client/server tương tác, từng câu lệnh một. Một ứng dụng thực hiện một truy vấn, đọc kết quả, có lẽ thực hiện một truy vấn khác tùy vào kết quả của truy vấn thứ nhất, và cứ thế. Các truy vấn và kết quả được gửi đi gửi lại giữa mã ứng dụng (chạy trên một máy) và máy chủ cơ sở dữ liệu (trên một máy khác).

In this interactive style of transaction, a lot of time is spent in network communication between the application and the database. If you were to disallow concurrency in the database and only process one transaction at a time, the throughput would be dreadful because the database would spend most of its time waiting for the application to issue the next query for the current transaction. In this kind of database, it's necessary to process multiple transactions concurrently in order to get reasonable performance.

Trong phong cách giao dịch tương tác này, rất nhiều thời gian bị tiêu tốn vào giao tiếp mạng giữa ứng dụng và cơ sở dữ liệu. Nếu bạn cấm tương tranh trong cơ sở dữ liệu và chỉ xử lý một giao dịch tại một thời điểm, thông lượng sẽ tồi tệ vì cơ sở dữ liệu sẽ dành phần lớn thời gian của nó để chờ ứng dụng phát ra truy vấn tiếp theo cho giao dịch hiện tại. Trong loại cơ sở dữ liệu này, cần phải xử lý nhiều giao dịch đồng thời để có được hiệu năng hợp lý.

For this reason, systems with single-threaded serial transaction processing don't allow interactive multi-statement transactions. Instead, the application must submit the entire transaction code to

the database ahead of time, as a stored procedure. The differences between these approaches is illustrated in Figure 7-9. Provided that all data required by a transaction is in memory, the stored procedure can execute very fast, without waiting for any network or disk I/O.

Vì lý do này, các hệ thống xử lý giao dịch tuần tự đơn luồng không cho phép các giao dịch nhiều câu lệnh tương tác. Thay vào đó, ứng dụng phải nộp toàn bộ mã giao dịch cho cơ sở dữ liệu từ trước, dưới dạng một thủ tục lưu trữ (stored procedure). Sự khác biệt giữa các cách tiếp cận này được minh họa trong Hình 7-9. Miễn là tất cả dữ liệu mà một giao dịch cần đều ở trong bộ nhớ, thủ tục lưu trữ có thể thực thi rất nhanh, mà không phải chờ bất kỳ I/O mạng hay đĩa nào.

Figure 7-9. The difference between an interactive transaction and a stored procedure (using the example transaction of Figure 7-8). — Hình 7-9. Sự khác biệt giữa một giao dịch tương tác và một thủ tục lưu trữ (dùng giao dịch ví dụ của Hình 7-8).

Pros and cons of stored procedures — Ưu và nhược điểm của thủ tục lưu trữ

Stored procedures have existed for some time in relational databases, and they have been part of the SQL standard (SQL/PSM) since 1999. They have gained a somewhat bad reputation, for various reasons:

Thủ tục lưu trữ đã tồn tại được một thời gian trong các cơ sở dữ liệu quan hệ, và chúng đã là một phần của chuẩn SQL (SQL/PSM) kể từ năm 1999. Chúng đã có một tiếng tăm hơi xấu, vì nhiều lý do:

- Each database vendor has its own language for stored procedures (Oracle has PL/SQL, SQL Server has T-SQL, PostgreSQL has PL/pgSQL, etc.). These languages haven't kept up with developments in general-purpose programming languages, so they look quite ugly and archaic from today's point of view, and they lack the ecosystem of libraries that you find with most programming languages.
- Code running in a database is difficult to manage: compared to an application server, it's harder to debug, more awkward to keep in version control and deploy, trickier to test, and difficult to integrate with a metrics collection system for monitoring.
- A database is often much more performance-sensitive than an application server, because a single database **instance** is often shared by many application servers. A badly written stored procedure (e.g., using a lot of memory or CPU time) in a database can cause much more trouble than equivalent badly written code in an application server.
 - Mỗi nhà cung cấp cơ sở dữ liệu có ngôn ngữ riêng cho thủ tục lưu trữ (Oracle có PL/SQL, SQL Server có T-SQL, PostgreSQL có PL/pgSQL, v.v.). Những ngôn ngữ này đã không bắt kịp các bước phát triển trong các ngôn ngữ lập trình đa dụng, nên nhìn từ hôm nay chúng khá xấu xí và cổ lỗ, và chúng thiếu cái hệ sinh thái thư viện mà bạn tìm thấy ở hầu hết các ngôn ngữ lập trình.
 - Mã chạy trong một cơ sở dữ liệu khó quản lý: so với một máy chủ ứng dụng, nó khó gỡ lỗi hơn, vụng về hơn khi giữ trong quản lý phiên bản và triển khai, khó kiểm thử hơn, và khó tích hợp với một hệ thống thu thập số đo để giám sát.
 - Một cơ sở dữ liệu thường nhạy cảm về hiệu năng hơn nhiều so với một máy chủ ứng dụng, vì một phiên bản cơ sở dữ liệu duy nhất thường được nhiều máy chủ ứng dụng dùng chung. Một thủ tục lưu trữ được viết tệ (ví dụ, dùng nhiều bộ nhớ hoặc thời gian CPU) trong một cơ sở dữ liệu có thể gây ra nhiều rắc rối hơn nhiều so với mã viết tệ tương đương trong một máy chủ ứng dụng.

However, those issues can be overcome. Modern implementations of stored procedures have abandoned PL/SQL and use existing general-purpose programming languages instead: VoltDB uses Java or Groovy, Datomic uses Java or Clojure, and Redis uses Lua.

Tuy nhiên, những vấn đề đó có thể được khắc phục. Các hiện thực hiện đại của thủ tục lưu trữ đã từ bỏ PL/SQL và dùng các ngôn ngữ lập trình đa dụng đang tồn tại thay thế: VoltDB dùng Java hoặc Groovy, Datomic dùng Java hoặc Clojure, và Redis dùng Lua.

With stored procedures and in-memory data, executing all transactions on a single thread becomes feasible. As they don't need to wait for I/O and they avoid the overhead of other concurrency control mechanisms, they can achieve quite good throughput on a single thread.

Với thủ tục lưu trữ và dữ liệu trong bộ nhớ, việc thực thi tất cả giao dịch trên một luồng duy nhất trở nên khả thi. Vì chúng không cần chờ I/O và chúng tránh được chi phí của các cơ chế kiểm soát tương tranh khác, chúng có thể đạt được thông lượng khá tốt trên một luồng duy nhất.

VoltDB also uses stored procedures for replication: instead of copying a transaction's writes from one node to another, it executes the same stored procedure on each replica. VoltDB therefore requires that stored procedures are deterministic (when run on different nodes, they must produce the same result). If a transaction needs to use the current date and time, for example, it must do so through special deterministic APIs.

VoltDB cũng dùng thủ tục lưu trữ cho nhân bản: thay vì sao chép các thao tác ghi của một giao dịch từ một nút sang nút khác, nó thực thi cùng một thủ tục lưu trữ trên mỗi bản sao. Do đó VoltDB đòi hỏi rằng các thủ tục lưu trữ phải có tính tất định (khi chạy trên các nút khác nhau, chúng phải tạo ra cùng một kết quả). Nếu một giao dịch cần dùng ngày và giờ hiện tại, chẳng hạn, nó phải làm vậy thông qua các API tất định đặc biệt.

Partitioning — Phân vùng

Executing all transactions serially makes concurrency control much simpler, but limits the transaction throughput of the database to the speed of a single CPU core on a single machine. Read-only transactions may execute elsewhere, using snapshot isolation, but for applications with high write throughput, the single-threaded transaction processor can become a serious bottleneck.

Thực thi tất cả giao dịch một cách tuần tự khiến việc kiểm soát tương tranh đơn giản hơn nhiều, nhưng giới hạn thông lượng giao dịch của cơ sở dữ liệu ở tốc độ của một lõi CPU duy nhất trên một máy duy nhất. Các giao dịch chỉ-đọc có thể thực thi ở nơi khác, dùng snapshot isolation, nhưng với các ứng dụng có thông lượng ghi cao, bộ xử lý giao dịch đơn luồng có thể trở thành một nút thắt cổ chai nghiêm trọng.

In order to scale to multiple CPU cores, and multiple nodes, you can potentially partition your data (see Chapter 6), which is supported in VoltDB. If you can find a way of partitioning your dataset so that each transaction only needs to read and write data within a single partition, then each partition can have its own transaction processing thread running independently from the others. In this case, you can give each CPU core its own partition, which allows your transaction throughput to scale linearly with the number of CPU cores [47].

Để mở rộng tới nhiều lõi CPU, và nhiều nút, bạn có khả năng phân vùng dữ liệu của mình (xem Chương 6), điều được hỗ trợ trong VoltDB. Nếu bạn có thể tìm ra một cách phân vùng tập dữ liệu của mình sao cho mỗi giao dịch chỉ cần đọc và ghi dữ liệu trong một phân vùng duy nhất, thì mỗi phân vùng có thể có luồng xử lý giao dịch riêng chạy độc lập với những

luồng khác. Trong trường hợp này, bạn có thể trao cho mỗi lõi CPU một phân vùng riêng, điều cho phép thông lượng giao dịch của bạn mở rộng tuyến tính với số lõi CPU [47].

However, for any transaction that needs to access multiple partitions, the database must coordinate the transaction across all the partitions that it touches. The stored procedure needs to be performed in lock-step across all partitions to ensure serializability across the whole system.

Tuy nhiên, với bất kỳ giao dịch nào cần truy cập nhiều phân vùng, cơ sở dữ liệu phải phối hợp giao dịch xuyên tất cả các phân vùng mà nó động đến. Thủ tục lưu trữ cần được thực hiện đồng bộ từng bước xuyên tất cả các phân vùng để đảm bảo tính khả tuần tự hóa trên toàn hệ thống.

Since cross-partition transactions have additional coordination overhead, they are vastly slower than single-partition transactions. VoltDB reports a throughput of about 1,000 cross-partition writes per second, which is orders of magnitude below its single-partition throughput and cannot be increased by adding more machines [49].

Vì các giao dịch xuyên phân vùng có thêm chi phí phối hợp, chúng chậm hơn nhiều so với các giao dịch một phân vùng. VoltDB báo cáo một thông lượng khoảng 1.000 thao tác ghi xuyên phân vùng mỗi giây, vốn thấp hơn nhiều bậc so với thông lượng một phân vùng của nó và không thể tăng lên bằng cách thêm máy [49].

Whether transactions can be single-partition depends very much on the structure of the data used by the application. Simple key-value data can often be partitioned very easily, but data with multiple secondary indexes is likely to require a lot of cross-partition coordination (see “Partitioning and Secondary Indexes”).

Liệu các giao dịch có thể là một phân vùng hay không phụ thuộc rất nhiều vào cấu trúc của dữ liệu mà ứng dụng dùng. Dữ liệu khóa-giá trị đơn giản thường có thể được phân vùng rất dễ dàng, nhưng dữ liệu có nhiều chỉ mục thứ cấp có khả năng đòi hỏi nhiều phối hợp xuyên phân vùng (xem “Partitioning and Secondary Indexes”).

Summary of serial execution — Tóm tắt về thực thi tuần tự

Serial execution of transactions has become a viable way of achieving serializable isolation within certain constraints:

Thực thi tuần tự các giao dịch đã trở thành một cách khả thi để đạt được cô lập khả tuần tự hóa trong một số ràng buộc nhất định:

- Every transaction must be small and fast, because it takes only one slow transaction to stall all transaction processing.
- It is limited to use cases where the active dataset can fit in memory. Rarely accessed data could potentially be moved to disk, but if it needed to be accessed in a single-threaded transaction, the system would get very slow.
- Write throughput must be low enough to be handled on a single CPU core, or else transactions need to be partitioned without requiring cross-partition coordination.
- Cross-partition transactions are possible, but there is a hard limit to the extent to which they can be used.
 - Mỗi giao dịch phải nhỏ và nhanh, vì chỉ cần một giao dịch chậm là đủ để làm đình trệ toàn bộ việc xử lý giao dịch.
 - Nó bị giới hạn ở các tình huống sử dụng mà tập dữ liệu đang hoạt động có thể nằm vừa trong bộ nhớ. Dữ liệu ít được truy cập có khả năng được chuyển ra đĩa, nhưng nếu nó cần được truy cập trong một giao dịch đơn luồng, hệ thống sẽ trở nên rất chậm.

- Thông lượng ghi phải đủ thấp để được xử lý trên một lõi CPU duy nhất, nếu không thì các giao dịch cần được phân vùng mà không đòi hỏi phối hợp xuyên phân vùng.
- Các giao dịch xuyên phân vùng là khả thi, nhưng có một giới hạn cứng về mức độ mà chúng có thể được dùng.

Two-Phase Locking (2PL) — Khóa hai pha (2PL)

For around 30 years, there was only one widely used algorithm for serializability in databases: two-phase locking (2PL).

Trong khoảng 30 năm, chỉ có một thuật toán được dùng rộng rãi cho tính khả tuần tự hóa trong cơ sở dữ liệu: khóa hai pha (two-phase locking, 2PL).

2PL is not 2PC — 2PL không phải là 2PC

Note that while two-phase locking (2PL) sounds very similar to two-phase commit (2PC), they are completely different things. We will discuss 2PC in Chapter 9.

Lưu ý rằng mặc dù khóa hai pha (2PL) nghe rất giống giao thức hai pha (2PC), chúng là những thứ hoàn toàn khác nhau. Ta sẽ bàn về 2PC ở Chương 9.

We saw previously that locks are often used to prevent dirty writes (see “No dirty writes”): if two transactions concurrently try to write to the same object, the lock ensures that the second writer must wait until the first one has finished its transaction (aborted or committed) before it may continue.

Trước đây ta đã thấy rằng các khóa thường được dùng để ngăn chặn ghi bẩn (xem “Không có ghi bẩn”): nếu hai giao dịch đồng thời cố ghi vào cùng một đối tượng, khóa đảm bảo rằng người ghi thứ hai phải chờ cho đến khi người ghi thứ nhất đã kết thúc giao dịch của mình (hủy bỏ hoặc commit) trước khi nó có thể tiếp tục.

Two-phase locking is similar, but makes the lock requirements much stronger. Several transactions are allowed to concurrently read the same object as long as nobody is writing to it. But as soon as anyone wants to write (modify or delete) an object, exclusive access is required:

Khóa hai pha thì tương tự, nhưng làm cho các yêu cầu về khóa mạnh hơn nhiều. Vài giao dịch được phép đồng thời đọc cùng một đối tượng miễn là không ai đang ghi vào nó. Nhưng ngay khi bất kỳ ai muốn ghi (sửa đổi hoặc xóa) một đối tượng, cần có quyền truy cập độc quyền:

- If transaction A has read an object and transaction B wants to write to that object, B must wait until A commits or aborts before it can continue. (This ensures that B can’t change the object unexpectedly behind A’s back.)
- If transaction A has written an object and transaction B wants to read that object, B must wait until A commits or aborts before it can continue. (Reading an old version of the object, like in Figure 7-1, is not acceptable under 2PL.)
 - Nếu giao dịch A đã đọc một đối tượng và giao dịch B muốn ghi vào đối tượng đó, B phải chờ cho đến khi A commit hoặc hủy bỏ trước khi nó có thể tiếp tục. (Điều này đảm bảo rằng B không thể thay đổi đối tượng một cách bất ngờ sau lưng A.)
 - Nếu giao dịch A đã ghi một đối tượng và giao dịch B muốn đọc đối tượng đó, B phải chờ cho đến khi A commit hoặc hủy bỏ trước khi nó có thể tiếp tục. (Việc đọc một phiên bản cũ của đối tượng, như trong Hình 7-1, là không chấp nhận được dưới 2PL.)

In 2PL, writers don’t just block other writers; they also block readers and vice versa. Snapshot isolation has the mantra readers never block writers, and writers never block readers (see “Implementing snapshot isolation”), which captures this key difference between snapshot isolation

and two-phase locking. On the other hand, because 2PL provides serializability, it protects against all the race conditions discussed earlier, including lost updates and write skew.

Trong 2PL, người ghi không chỉ chặn những người ghi khác; họ còn chặn người đọc và ngược lại. Snapshot isolation có câu thần chú người đọc không bao giờ chặn người ghi, và người ghi không bao giờ chặn người đọc (xem “Hiện thực snapshot isolation”), vốn nắm bắt được sự khác biệt then chốt này giữa snapshot isolation và khóa hai pha. Mặt khác, vì 2PL cung cấp tính khả tuần tự hóa, nó bảo vệ trước tất cả các tình trạng tranh đua đã bàn trước đây, bao gồm cập nhật bị mất và lệch khi ghi.

Implementation of two-phase locking — Hiện thực khóa hai pha

2PL is used by the serializable isolation level in MySQL (InnoDB) and SQL Server, and the repeatable read isolation level in DB2 [23, 36].

2PL được dùng bởi mức cô lập serializable trong MySQL (InnoDB) và SQL Server, và mức cô lập repeatable read trong DB2 [23, 36].

The blocking of readers and writers is implemented by having a lock on each object in the database. The lock can either be in shared mode or in exclusive mode. The lock is used as follows:

Việc chặn người đọc và người ghi được hiện thực bằng cách có một khóa trên mỗi đối tượng trong cơ sở dữ liệu. Khóa có thể ở chế độ chia sẻ (shared mode) hoặc chế độ độc quyền (exclusive mode). Khóa được dùng như sau:

- If a transaction wants to read an object, it must first acquire the lock in shared mode. Several transactions are allowed to hold the lock in shared mode simultaneously, but if another transaction already has an exclusive lock on the object, these transactions must wait.
 - If a transaction wants to write to an object, it must first acquire the lock in exclusive mode. No other transaction may hold the lock at the same time (either in shared or in exclusive mode), so if there is any existing lock on the object, the transaction must wait.
 - If a transaction first reads and then writes an object, it may upgrade its shared lock to an exclusive lock. The upgrade works the same as getting an exclusive lock directly.
 - After a transaction has acquired the lock, it must continue to hold the lock until the end of the transaction (commit or abort). This is where the name “two-phase” comes from: the first phase (while the transaction is executing) is when the locks are acquired, and the second phase (at the end of the transaction) is when all the locks are released.
- Nếu một giao dịch muốn đọc một đối tượng, trước tiên nó phải lấy khóa ở chế độ chia sẻ. Vài giao dịch được phép giữ khóa ở chế độ chia sẻ đồng thời, nhưng nếu một giao dịch khác đã có một khóa độc quyền trên đối tượng, những giao dịch này phải chờ.
 - Nếu một giao dịch muốn ghi vào một đối tượng, trước tiên nó phải lấy khóa ở chế độ độc quyền. Không giao dịch nào khác có thể giữ khóa cùng lúc (dù ở chế độ chia sẻ hay độc quyền), nên nếu có bất kỳ khóa nào đang tồn tại trên đối tượng, giao dịch phải chờ.
 - Nếu một giao dịch trước tiên đọc rồi ghi một đối tượng, nó có thể nâng cấp khóa chia sẻ của mình thành một khóa độc quyền. Việc nâng cấp hoạt động giống như lấy một khóa độc quyền trực tiếp.
 - Sau khi một giao dịch đã lấy khóa, nó phải tiếp tục giữ khóa cho đến hết giao dịch (commit hoặc hủy bỏ). Đây là nơi cái tên “hai pha” đến từ: pha thứ nhất (trong khi giao dịch đang thực thi) là khi các khóa được lấy, và pha thứ hai (vào cuối giao dịch) là khi tất cả các khóa được nhả.

Since so many locks are in use, it can happen quite easily that transaction A is stuck waiting for transaction B to release its lock, and vice versa. This situation is called deadlock. The database automatically detects deadlocks between transactions and aborts one of them so that the others can make progress. The aborted transaction needs to be retried by the application.

Vì có quá nhiều khóa đang được dùng, có thể khá dễ xảy ra việc giao dịch A bị mắc kẹt chờ giao dịch B nhả khóa của nó, và ngược lại. Tình huống này được gọi là bế tắc (deadlock). Cơ sở dữ liệu tự động phát hiện các bế tắc giữa các giao dịch và hủy bỏ một trong số chúng để những cái khác có thể tiến triển. Giao dịch bị hủy bỏ cần được ứng dụng thử lại.

Performance of two-phase locking — Hiệu năng của khóa hai pha

The big downside of two-phase locking, and the reason why it hasn't been used by everybody since the 1970s, is performance: transaction throughput and response times of queries are significantly worse under two-phase locking than under weak isolation.

Nhược điểm lớn của khóa hai pha, và lý do tại sao nó không được mọi người dùng kể từ thập niên 1970, là hiệu năng: thông lượng giao dịch và thời gian phản hồi của các truy vấn tệ hơn đáng kể dưới khóa hai pha so với dưới cô lập yếu.

This is partly due to the overhead of acquiring and releasing all those locks, but more importantly due to reduced concurrency. By design, if two concurrent transactions try to do anything that may in any way result in a race condition, one has to wait for the other to complete.

Điều này một phần do chi phí của việc lấy và nhả tất cả những khóa đó, nhưng quan trọng hơn là do tương tranh bị giảm đi. Theo thiết kế, nếu hai giao dịch đồng thời cố làm bất cứ điều gì mà bằng cách nào đó có thể dẫn đến một tình trạng tranh đua, một cái phải chờ cái kia hoàn tất.

Traditional relational databases don't limit the duration of a transaction, because they are designed for interactive applications that wait for human input. Consequently, when one transaction has to wait on another, there is no limit on how long it may have to wait. Even if you make sure that you keep all your transactions short, a **queue** may form if several transactions want to access the same object, so a transaction may have to wait for several others to complete before it can do anything.

Các cơ sở dữ liệu quan hệ truyền thống không giới hạn thời lượng của một giao dịch, vì chúng được thiết kế cho các ứng dụng tương tác chờ đầu vào của con người. Do đó, khi một giao dịch phải chờ một giao dịch khác, không có giới hạn về việc nó có thể phải chờ bao lâu. Ngay cả khi bạn đảm bảo rằng bạn giữ tất cả các giao dịch của mình ngắn, một hàng đợi có thể hình thành nếu vài giao dịch muốn truy cập cùng một đối tượng, nên một giao dịch có thể phải chờ vài giao dịch khác hoàn tất trước khi nó có thể làm bất cứ điều gì.

For this reason, databases running 2PL can have quite unstable latencies, and they can be very slow at high percentiles (see “Describing Performance”) if there is contention in the workload. It may take just one slow transaction, or one transaction that accesses a lot of data and acquires many locks, to cause the rest of the system to grind to a halt. This instability is problematic when robust operation is required.

Vì lý do này, các cơ sở dữ liệu chạy 2PL có thể có độ trễ khá bất ổn, và chúng có thể rất chậm ở các phân vị cao (xem “Describing Performance”) nếu có tranh chấp trong khối lượng công việc. Có thể chỉ cần một giao dịch chậm, hoặc một giao dịch truy cập nhiều dữ liệu và lấy nhiều khóa, là đủ để khiến phần còn lại của hệ thống đứng khựng lại. Sự bất ổn này gây vấn đề khi cần vận hành bền vững.

Although deadlocks can happen with the lock-based read committed isolation level, they occur much more frequently under 2PL serializable isolation (depending on the access patterns of your transaction). This can be an additional performance problem: when a transaction is aborted due to deadlock and is retried, it needs to do its work all over again. If deadlocks are frequent, this can mean significant wasted effort.

Mặc dù bế tắc có thể xảy ra với mức cô lập read committed dựa trên khóa, chúng xảy ra thường xuyên hơn nhiều dưới cô lập serializable 2PL (tùy vào các mẫu truy cập của giao dịch của bạn). Đây có thể là một vấn đề hiệu năng bổ sung: khi một giao dịch bị hủy bỏ do bế tắc và được thử lại, nó cần làm lại toàn bộ công việc của mình từ đầu. Nếu bế tắc xảy ra thường xuyên, điều này có thể đồng nghĩa với công sức lãng phí đáng kể.

Predicate locks — Khóa vị từ

In the preceding description of locks, we glossed over a subtle but important detail. In “Phantoms causing write skew” we discussed the problem of phantoms—that is, one transaction changing the results of another transaction’s search query. A database with serializable isolation must prevent phantoms.

Trong phần mô tả về khóa ở trên, ta đã bỏ qua một chi tiết tinh tế nhưng quan trọng. Trong “Bóng ma gây ra lệch khi ghi” ta đã bàn về vấn đề bóng ma — tức là một giao dịch thay đổi kết quả của truy vấn tìm kiếm của một giao dịch khác. Một cơ sở dữ liệu với cô lập khả tuần tự hóa phải ngăn chặn bóng ma.

In the meeting room booking example this means that if one transaction has searched for existing bookings for a room within a certain time window (see Example 7-2), another transaction is not allowed to concurrently insert or update another booking for the same room and time range. (It’s okay to concurrently insert bookings for other rooms, or for the same room at a different time that doesn’t affect the proposed booking.)

Trong ví dụ đặt phòng họp điều này nghĩa là nếu một giao dịch đã tìm kiếm các lượt đặt hiện hữu cho một phòng trong một khung thời gian nhất định (xem Ví dụ 7-2), một giao dịch khác không được phép đồng thời chèn hoặc cập nhật một lượt đặt khác cho cùng phòng và khoảng thời gian. (Việc đồng thời chèn các lượt đặt cho các phòng khác, hoặc cho cùng phòng vào một thời điểm khác không ảnh hưởng đến lượt đặt được đề xuất, thì không sao.)

How do we implement this? Conceptually, we need a **predicate** lock [3]. It works similarly to the shared/exclusive lock described earlier, but rather than belonging to a particular object (e.g., one row in a table), it belongs to all objects that match some search condition, such as:

Ta hiện thực điều này như thế nào? Về mặt khái niệm, ta cần một khóa vị từ (predicate lock) [3]. Nó hoạt động tương tự khóa chia sẻ/độc quyền đã mô tả trước đây, nhưng thay vì thuộc về một đối tượng cụ thể (ví dụ, một hàng trong một bảng), nó thuộc về tất cả các đối tượng khớp với một điều kiện tìm kiếm nào đó, chẳng hạn:

```
SELECT * FROM bookings
WHERE room_id = 123 AND
      end_time   > '2018-01-01 12:00' AND
      start_time < '2018-01-01 13:00';
```

A predicate lock restricts access as follows:

Một khóa vị từ hạn chế việc truy cập như sau:

- If transaction A wants to read objects matching some condition, like in that SELECT query, it must acquire a shared-mode predicate lock on the conditions of the query. If another transaction B currently has an exclusive lock on any object matching those conditions, A must wait until B releases its lock before it is allowed to make its query.
- If transaction A wants to insert, update, or delete any object, it must first check whether either the old or the new value matches any existing predicate lock. If there is a matching predicate lock held by transaction B, then A must wait until B has committed or aborted before it can continue.
 - Nếu giao dịch A muốn đọc các đối tượng khớp với một điều kiện nào đó, như trong truy vấn SELECT đó, nó phải lấy một khóa vị từ ở chế độ chia sẻ trên các điều kiện của truy vấn. Nếu một giao dịch B khác hiện đang có một khóa độc quyền trên bất kỳ đối tượng nào khớp với những điều kiện đó, A phải chờ cho đến khi B nhả khóa của nó trước khi nó được phép thực hiện truy vấn của mình.
 - Nếu giao dịch A muốn chèn, cập nhật, hoặc xóa bất kỳ đối tượng nào, trước tiên nó phải kiểm tra xem hoặc giá trị cũ hoặc giá trị mới có khớp với bất kỳ khóa vị từ nào đang tồn tại không. Nếu có một khóa vị từ khớp do giao dịch B giữ, thì A phải chờ cho đến khi B đã commit hoặc hủy bỏ trước khi nó có thể tiếp tục.

The key idea here is that a predicate lock applies even to objects that do not yet exist in the database, but which might be added in the future (phantoms). If two-phase locking includes predicate locks, the database prevents all forms of write skew and other race conditions, and so its isolation becomes serializable.

Ý tưởng then chốt ở đây là một khóa vị từ áp dụng ngay cả với các đối tượng chưa tồn tại trong cơ sở dữ liệu, nhưng có thể được thêm vào trong tương lai (bóng ma). Nếu khóa hai pha bao gồm khóa vị từ, cơ sở dữ liệu ngăn chặn mọi dạng lệch khi ghi và các tình trạng tranh đua khác, và do đó sự cô lập của nó trở thành khả tuần tự hóa.

Index-range locks — Khóa khoảng chỉ mục

Unfortunately, predicate locks do not perform well: if there are many locks by active transactions, checking for matching locks becomes time-consuming. For that reason, most databases with 2PL actually implement index-range locking (also known as next-key locking), which is a simplified approximation of predicate locking [41, 50].

Đáng tiếc, khóa vị từ không hoạt động tốt: nếu có nhiều khóa từ các giao dịch đang hoạt động, việc kiểm tra các khóa khớp trở nên tốn thời gian. Vì lý do đó, hầu hết các cơ sở dữ liệu với 2PL thực ra hiện thực khóa khoảng chỉ mục (index-range locking, còn gọi là next-key locking), vốn là một sự xấp xỉ đơn giản hóa của khóa vị từ [41, 50].

It's safe to simplify a predicate by making it match a greater set of objects. For example, if you have a predicate lock for bookings of room 123 between noon and 1 p.m., you can approximate it by locking bookings for room 123 at any time, or you can approximate it by locking all rooms (not just room 123) between noon and 1 p.m. This is safe, because any write that matches the original predicate will definitely also match the approximations.

Việc đơn giản hóa một vị từ bằng cách khiến nó khớp với một tập đối tượng lớn hơn là an toàn. Ví dụ, nếu bạn có một khóa vị từ cho các lượt đặt phòng 123 từ trưa đến 1 giờ chiều, bạn có thể xấp xỉ nó bằng cách khóa các lượt đặt cho phòng 123 vào bất kỳ lúc nào, hoặc bạn có thể xấp xỉ nó bằng cách khóa tất cả các phòng (không chỉ phòng 123) từ trưa đến 1 giờ chiều. Điều này an toàn, vì bất kỳ thao tác ghi nào khớp với vị từ gốc chắc chắn cũng sẽ khớp với các xấp xỉ.

In the room bookings database you would probably have an index on the `room_id` **column**, and/or indexes on `start_time` and `end_time` (otherwise the preceding query would be very slow on a large database):

Trong cơ sở dữ liệu đặt phòng bạn có lẽ sẽ có một chỉ mục trên cột `room_id`, và/hoặc các chỉ mục trên `start_time` và `end_time` (nếu không thì truy vấn ở trên sẽ rất chậm trên một cơ sở dữ liệu lớn):

- Say your index is on `room_id`, and the database uses this index to find existing bookings for room 123. Now the database can simply attach a shared lock to this index entry, indicating that a transaction has searched for bookings of room 123.
- Alternatively, if the database uses a time-based index to find existing bookings, it can attach a shared lock to a range of values in that index, indicating that a transaction has searched for bookings that overlap with the time period of noon to 1 p.m. on January 1, 2018.
 - Giả sử chỉ mục của bạn ở trên `room_id`, và cơ sở dữ liệu dùng chỉ mục này để tìm các lượt đặt hiện hữu cho phòng 123. Giờ cơ sở dữ liệu có thể đơn giản gắn một khóa chia sẻ vào mục chỉ mục này, cho biết rằng một giao dịch đã tìm kiếm các lượt đặt của phòng 123.
 - Hoặc, nếu cơ sở dữ liệu dùng một chỉ mục dựa trên thời gian để tìm các lượt đặt hiện hữu, nó có thể gắn một khóa chia sẻ vào một khoảng giá trị trong chỉ mục đó, cho biết rằng một giao dịch đã tìm kiếm các lượt đặt chồng lấp với khoảng thời gian từ trưa đến 1 giờ chiều ngày 1 tháng 1 năm 2018.

Either way, an approximation of the search condition is attached to one of the indexes. Now, if another transaction wants to insert, update, or delete a booking for the same room and/or an overlapping time period, it will have to update the same part of the index. In the process of doing so, it will encounter the shared lock, and it will be forced to wait until the lock is released.

Dù theo cách nào, một sự xấp xỉ của điều kiện tìm kiếm được gắn vào một trong các chỉ mục. Giờ, nếu một giao dịch khác muốn chèn, cập nhật, hoặc xóa một lượt đặt cho cùng phòng và/hoặc một khoảng thời gian chồng lấp, nó sẽ phải cập nhật cùng phần đó của chỉ mục. Trong quá trình làm vậy, nó sẽ gặp phải khóa chia sẻ, và nó sẽ bị buộc phải chờ cho đến khi khóa được nhả.

This provides effective protection against phantoms and write skew. Index-range locks are not as precise as predicate locks would be (they may lock a bigger range of objects than is strictly necessary to maintain serializability), but since they have much lower overheads, they are a good compromise.

Cách này cung cấp sự bảo vệ hiệu quả trước bóng ma và lệch khi ghi. Khóa khoảng chỉ mục không chính xác bằng khóa vị từ (chúng có thể khóa một khoảng đối tượng lớn hơn mức thực sự cần thiết để duy trì tính khả tuần tự hóa), nhưng vì chúng có chi phí thấp hơn nhiều, chúng là một sự thỏa hiệp tốt.

If there is no suitable index where a range lock can be attached, the database can fall back to a shared lock on the entire table. This will not be good for performance, since it will stop all other transactions writing to the table, but it's a safe fallback position.

Nếu không có chỉ mục phù hợp nào để gắn một khóa khoảng vào, cơ sở dữ liệu có thể lùi về một khóa chia sẻ trên toàn bộ bảng. Điều này sẽ không tốt cho hiệu năng, vì nó sẽ chặn tất cả các giao dịch khác ghi vào bảng, nhưng đó là một phương án dự phòng an toàn.

Serializable Snapshot Isolation (SSI) — Cô lập ảnh chụp khả tuần tự hóa (SSI)

This chapter has painted a bleak picture of concurrency control in databases. On the one hand, we have implementations of serializability that don't perform well (two-phase locking) or don't scale well (serial execution). On the other hand, we have weak isolation levels that have good performance, but are prone to various race conditions (lost updates, write skew, phantoms, etc.). Are serializable isolation and good performance fundamentally at odds with each other?

Chương này đã vẽ nên một bức tranh ảm đạm về kiểm soát tương tranh trong cơ sở dữ liệu. Một mặt, ta có các hiện thực của tính khả tuần tự hóa không hoạt động tốt (khóa hai pha) hoặc không mở rộng tốt (thực thi tuần tự). Mặt khác, ta có các mức cô lập yếu có hiệu năng tốt, nhưng dễ gặp nhiều tình trạng tranh đua khác nhau (cập nhật bị mất, lệch khi ghi, bóng ma, v.v.). Liệu cô lập khả tuần tự hóa và hiệu năng tốt có về bản chất xung khắc với nhau không?

Perhaps not: an algorithm called serializable snapshot isolation (SSI) is very promising. It provides full serializability, but has only a small performance penalty compared to snapshot isolation. SSI is fairly new: it was first described in 2008 [40] and is the **subject** of Michael Cahill's PhD thesis [51].

Có lẽ không: một thuật toán gọi là cô lập ảnh chụp khả tuần tự hóa (serializable snapshot isolation, SSI) rất hứa hẹn. Nó cung cấp tính khả tuần tự hóa đầy đủ, nhưng chỉ có một cái giá nhỏ về hiệu năng so với snapshot isolation. SSI khá mới: nó được mô tả lần đầu vào năm 2008 [40] và là chủ đề luận án tiến sĩ của Michael Cahill [51].

Today SSI is used both in single-node databases (the serializable isolation level in PostgreSQL since version 9.1 [41]) and distributed databases (FoundationDB uses a similar algorithm). As SSI is so young compared to other concurrency control mechanisms, it is still proving its performance in practice, but it has the possibility of being fast enough to become the new default in the future.

Ngày nay SSI được dùng cả trong các cơ sở dữ liệu một nút (mức cô lập serializable trong PostgreSQL kể từ phiên bản 9.1 [41]) lẫn các cơ sở dữ liệu phân tán (FoundationDB dùng một thuật toán tương tự). Vì SSI còn rất trẻ so với các cơ chế kiểm soát tương tranh khác, nó vẫn đang chứng minh hiệu năng của mình trong thực tế, nhưng nó có khả năng trở nên đủ nhanh để thành mặc định mới trong tương lai.

Pessimistic versus optimistic concurrency control — Kiểm soát tương tranh bi quan so với lạc quan

Two-phase locking is a so-called pessimistic concurrency control mechanism: it is based on the principle that if anything might possibly go wrong (as indicated by a lock held by another transaction), it's better to wait until the situation is safe again before doing anything. It is like mutual exclusion, which is used to protect data structures in multi-threaded programming.

Khóa hai pha là một cơ chế kiểm soát tương tranh được gọi là bi quan (pessimistic): nó dựa trên nguyên tắc rằng nếu bất cứ điều gì có khả năng trục trặc (như được chỉ ra bởi một khóa do một giao dịch khác giữ), tốt hơn là chờ cho đến khi tình huống lại an toàn trước khi làm bất cứ điều gì. Nó giống như loại trừ tương hỗ (mutual exclusion), vốn được dùng để bảo vệ các cấu trúc dữ liệu trong lập trình đa luồng.

Serial execution is, in a sense, pessimistic to the extreme: it is essentially equivalent to each transaction having an exclusive lock on the entire database (or one partition of the database) for

the duration of the transaction. We compensate for the pessimism by making each transaction very fast to execute, so it only needs to hold the “lock” for a short time.

Thực thi tuần tự, theo một nghĩa nào đó, là bị quan đến cực độ: về cơ bản nó tương đương với việc mỗi giao dịch có một khóa độc quyền trên toàn bộ cơ sở dữ liệu (hoặc một phân vùng của cơ sở dữ liệu) trong suốt thời lượng của giao dịch. Ta bù lại cho sự bị quan này bằng cách làm cho mỗi giao dịch thực thi rất nhanh, nên nó chỉ cần giữ “khóa” trong một thời gian ngắn.

By contrast, serializable snapshot isolation is an optimistic concurrency control technique. Optimistic in this context means that instead of blocking if something potentially dangerous happens, transactions continue anyway, in the hope that everything will turn out all right. When a transaction wants to commit, the database checks whether anything bad happened (i.e., whether isolation was violated); if so, the transaction is aborted and has to be retried. Only transactions that executed serializably are allowed to commit.

Ngược lại, cô lập ảnh chụp khả tuần tự hóa là một kỹ thuật kiểm soát tương tranh lạc quan (optimistic). Lạc quan trong ngữ cảnh này nghĩa là thay vì chặn lại nếu một điều gì đó có khả năng nguy hiểm xảy ra, các giao dịch vẫn cứ tiếp tục, với hy vọng rằng mọi thứ sẽ ổn. Khi một giao dịch muốn commit, cơ sở dữ liệu kiểm tra xem có điều gì xấu đã xảy ra không (tức là liệu tính cô lập có bị vi phạm không); nếu có, giao dịch bị hủy bỏ và phải được thử lại. Chỉ những giao dịch đã thực thi một cách khả tuần tự hóa mới được phép commit.

Optimistic concurrency control is an old idea [52], and its advantages and disadvantages have been debated for a long time [53]. It performs badly if there is high contention (many transactions trying to access the same objects), as this leads to a high proportion of transactions needing to abort. If the system is already close to its maximum throughput, the additional transaction **load** from retried transactions can make performance worse.

Kiểm soát tương tranh lạc quan là một ý tưởng cũ [52], và các ưu nhược điểm của nó đã được tranh luận từ lâu [53]. Nó hoạt động tệ nếu có tranh chấp cao (nhiều giao dịch cố truy cập cùng các đối tượng), vì điều này dẫn đến một tỷ lệ cao các giao dịch cần hủy bỏ. Nếu hệ thống đã gần thông lượng tối đa của nó, tải giao dịch bổ sung từ các giao dịch được thử lại có thể làm hiệu năng tồi tệ hơn.

However, if there is enough spare capacity, and if contention between transactions is not too high, optimistic concurrency control techniques tend to perform better than pessimistic ones. Contention can be reduced with commutative atomic operations: for example, if several transactions concurrently want to increment a counter, it doesn’t matter in which order the increments are applied (as long as the counter isn’t read in the same transaction), so the concurrent increments can all be applied without conflicting.

Tuy nhiên, nếu có đủ năng lực dư thừa, và nếu tranh chấp giữa các giao dịch không quá cao, các kỹ thuật kiểm soát tương tranh lạc quan có xu hướng hoạt động tốt hơn các kỹ thuật bi quan. Tranh chấp có thể được giảm bớt bằng các thao tác nguyên tử giao hoán: ví dụ, nếu vài giao dịch đồng thời muốn tăng một bộ đếm, không quan trọng các lần tăng được áp dụng theo thứ tự nào (miễn là bộ đếm không bị đọc trong cùng giao dịch), nên các lần tăng đồng thời đều có thể được áp dụng mà không xung đột.

As the name suggests, SSI is based on snapshot isolation—that is, all reads within a transaction are made from a consistent snapshot of the database (see “Snapshot Isolation and Repeatable Read”). This is the main difference compared to earlier optimistic concurrency control techniques. On top of snapshot isolation, SSI adds an algorithm for detecting serialization conflicts among writes and determining which transactions to abort.

Như cái tên gợi ý, SSI dựa trên snapshot isolation — tức là tất cả các thao tác đọc trong một giao dịch được thực hiện từ một ảnh chụp nhất quán của cơ sở dữ liệu (xem “Snapshot Isolation và Repeatable Read”). Đây là điểm khác biệt chính so với các kỹ thuật kiểm soát tương tranh lạc quan trước đó. Trên nền snapshot isolation, SSI thêm một thuật toán để phát hiện các xung đột tuần tự hóa giữa các thao tác ghi và xác định những giao dịch nào cần hủy bỏ.

Decisions based on an outdated premise — Quyết định dựa trên một tiền đề đã lỗi thời

When we previously discussed write skew in snapshot isolation (see “Write Skew and Phantoms”), we observed a recurring pattern: a transaction reads some data from the database, examines the result of the query, and decides to take some action (write to the database) based on the result that it saw. However, under snapshot isolation, the result from the original query may no longer be up-to-date by the time the transaction commits, because the data may have been modified in the meantime.

Khi ta bàn về lệch khi ghi trong snapshot isolation trước đây (xem “Lệch khi ghi và Bóng ma”), ta đã quan sát một mẫu lặp đi lặp lại: một giao dịch đọc một số dữ liệu từ cơ sở dữ liệu, xem xét kết quả của truy vấn, và quyết định thực hiện một hành động nào đó (ghi vào cơ sở dữ liệu) dựa trên kết quả mà nó đã thấy. Tuy nhiên, dưới snapshot isolation, kết quả từ truy vấn gốc có thể không còn cập nhật vào thời điểm giao dịch commit, vì dữ liệu có thể đã bị sửa đổi trong khoảng giữa.

Put another way, the transaction is taking an action based on a premise (a fact that was true at the beginning of the transaction, e.g., “There are currently two doctors on call”). Later, when the transaction wants to commit, the original data may have changed—the premise may no longer be true.

Nói cách khác, giao dịch đang thực hiện một hành động dựa trên một tiền đề (một sự kiện đúng vào lúc bắt đầu giao dịch, ví dụ, “Hiện có hai bác sĩ đang trực”). Về sau, khi giao dịch muốn commit, dữ liệu gốc có thể đã thay đổi — tiền đề có thể không còn đúng.

When the application makes a query (e.g., “How many doctors are currently on call?”), the database doesn’t know how the application logic uses the result of that query. To be safe, the database needs to assume that any change in the query result (the premise) means that writes in that transaction may be invalid. In other words, there may be a causal dependency between the queries and the writes in the transaction. In order to provide serializable isolation, the database must detect situations in which a transaction may have acted on an outdated premise and abort the transaction in that case.

Khi ứng dụng thực hiện một truy vấn (ví dụ, “Hiện có bao nhiêu bác sĩ đang trực?”), cơ sở dữ liệu không biết logic ứng dụng dùng kết quả của truy vấn đó như thế nào. Để an toàn, cơ sở dữ liệu cần giả định rằng bất kỳ thay đổi nào trong kết quả truy vấn (tiền đề) đều có nghĩa là các thao tác ghi trong giao dịch đó có thể không hợp lệ. Nói cách khác, có thể có một phụ thuộc nhân quả giữa các truy vấn và các thao tác ghi trong giao dịch. Để cung cấp cô lập khả năng tuần tự hóa, cơ sở dữ liệu phải phát hiện các tình huống trong đó một giao dịch có thể đã hành động trên một tiền đề lỗi thời và hủy bỏ giao dịch trong trường hợp đó.

How does the database know if a query result might have changed? There are two cases to consider:

Cơ sở dữ liệu làm sao biết liệu một kết quả truy vấn có thể đã thay đổi không? Có hai trường hợp cần xem xét:

- Detecting reads of a stale MVCC object version (uncommitted write occurred before the read)

- Detecting writes that affect prior reads (the write occurs after the read)
 - Phát hiện việc đọc một phiên bản đối tượng MVCC cũ (một thao tác ghi chưa commit đã xảy ra trước thao tác đọc)
 - Phát hiện các thao tác ghi ảnh hưởng đến các thao tác đọc trước đó (thao tác ghi xảy ra sau thao tác đọc)

Detecting stale MVCC reads — Phát hiện các thao tác đọc MVCC cũ

Recall that snapshot isolation is usually implemented by multi-version concurrency control (MVCC; see Figure 7-10). When a transaction reads from a consistent snapshot in an MVCC database, it ignores writes that were made by any other transactions that hadn't yet committed at the time when the snapshot was taken. In Figure 7-10, transaction 43 sees Alice as having `on_call = true`, because transaction 42 (which modified Alice's on-call status) is uncommitted. However, by the time transaction 43 wants to commit, transaction 42 has already committed. This means that the write that was ignored when reading from the consistent snapshot has now taken effect, and transaction 43's premise is no longer true.

Hãy nhớ lại rằng snapshot isolation thường được hiện thực bằng kiểm soát tương tranh đa phiên bản (MVCC; xem Hình 7-10). Khi một giao dịch đọc từ một ảnh chụp nhất quán trong một cơ sở dữ liệu MVCC, nó bỏ qua các thao tác ghi được thực hiện bởi bất kỳ giao dịch nào khác chưa commit vào thời điểm ảnh chụp được lấy. Trong Hình 7-10, giao dịch 43 thấy Alice có `on_call = true`, vì giao dịch 42 (vốn đã sửa đổi trạng thái trực ca của Alice) chưa commit. Tuy nhiên, vào thời điểm giao dịch 43 muốn commit, giao dịch 42 đã commit rồi. Điều này nghĩa là thao tác ghi vốn đã bị bỏ qua khi đọc từ ảnh chụp nhất quán giờ đã có hiệu lực, và tiền đề của giao dịch 43 không còn đúng.

Figure 7-10. Detecting when a transaction reads outdated values from an MVCC snapshot. — Hình 7-10. Phát hiện khi một giao dịch đọc các giá trị lỗi thời từ một ảnh chụp MVCC.

In order to prevent this anomaly, the database needs to track when a transaction ignores another transaction's writes due to MVCC visibility rules. When the transaction wants to commit, the database checks whether any of the ignored writes have now been committed. If so, the transaction must be aborted.

Để ngăn chặn dị thường này, cơ sở dữ liệu cần theo dõi khi nào một giao dịch bỏ qua các thao tác ghi của một giao dịch khác do các quy tắc hiển thị MVCC. Khi giao dịch muốn commit, cơ sở dữ liệu kiểm tra xem có bất kỳ thao tác ghi bị bỏ qua nào giờ đã được commit không. Nếu có, giao dịch phải bị hủy bỏ.

Why wait until committing? Why not abort transaction 43 immediately when the stale read is detected? Well, if transaction 43 was a read-only transaction, it wouldn't need to be aborted, because there is no risk of write skew. At the time when transaction 43 makes its read, the database doesn't yet know whether that transaction is going to later perform a write. Moreover, transaction 42 may yet abort or may still be uncommitted at the time when transaction 43 is committed, and so the read may turn out not to have been stale after all. By avoiding unnecessary aborts, SSI preserves snapshot isolation's support for long-running reads from a consistent snapshot.

Tại sao chờ đến lúc commit? Tại sao không hủy bỏ giao dịch 43 ngay lập tức khi thao tác đọc cũ được phát hiện? Ờ thì, nếu giao dịch 43 là một giao dịch chỉ-đọc, nó sẽ không cần bị hủy bỏ, vì không có nguy cơ lệch khi ghi. Vào thời điểm giao dịch 43 thực hiện thao tác đọc của nó, cơ sở dữ liệu chưa biết liệu giao dịch đó có sẽ thực hiện một thao tác ghi về sau hay không. Hơn nữa, giao dịch 42 có thể vẫn hủy bỏ hoặc có thể vẫn chưa commit vào

thời điểm giao dịch 43 được commit, và do đó thao tác đọc rút cuộc có thể hóa ra không phải là cũ. Bằng cách tránh những lần hủy bỏ không cần thiết, SSI giữ gìn sự hỗ trợ của snapshot isolation cho các thao tác đọc chạy lâu từ một ảnh chụp nhất quán.

Detecting writes that affect prior reads — Phát hiện các thao tác ghi ảnh hưởng đến các thao tác đọc trước đó

The second case to consider is when another transaction modifies data after it has been read. This case is illustrated in Figure 7-11.

Trường hợp thứ hai cần xem xét là khi một giao dịch khác sửa đổi dữ liệu sau khi nó đã được đọc. Trường hợp này được minh họa trong Hình 7-11.

Figure 7-11. In serializable snapshot isolation, detecting when one transaction modifies another transaction's reads. — Hình 7-11. Trong cơ lập ảnh chụp khả tuần tự hóa, phát hiện khi một giao dịch sửa đổi các thao tác đọc của một giao dịch khác. In the context of two-phase locking we discussed index-range locks (see “Index-range locks”), which allow the database to lock access to all rows matching some search query, such as `WHERE shift_id = 1234`. We can use a similar technique here, except that SSI locks don't block other transactions.

Trong ngữ cảnh khóa hai pha ta đã bàn về khóa khoảng chỉ mục (xem “Khóa khoảng chỉ mục”), vốn cho phép cơ sở dữ liệu khóa quyền truy cập tới tất cả các hàng khớp với một truy vấn tìm kiếm nào đó, chẳng hạn `WHERE shift_id = 1234`. Ta có thể dùng một kỹ thuật tương tự ở đây, chỉ khác là các khóa SSI không chặn các giao dịch khác.

In Figure 7-11, transactions 42 and 43 both search for on-call doctors during shift 1234. If there is an index on `shift_id`, the database can use the index entry 1234 to record the fact that transactions 42 and 43 read this data. (If there is no index, this information can be tracked at the table level.) This information only needs to be kept for a while: after a transaction has finished (committed or aborted), and all concurrent transactions have finished, the database can forget what data it read.

Trong Hình 7-11, các giao dịch 42 và 43 đều tìm kiếm các bác sĩ trực trong ca 1234. Nếu có một chỉ mục trên `shift_id`, cơ sở dữ liệu có thể dùng mục chỉ mục 1234 để ghi lại sự kiện rằng các giao dịch 42 và 43 đã đọc dữ liệu này. (Nếu không có chỉ mục, thông tin này có thể được theo dõi ở mức bảng.) Thông tin này chỉ cần được giữ trong một thời gian: sau khi một giao dịch đã kết thúc (commit hoặc hủy bỏ), và tất cả các giao dịch đồng thời đã kết thúc, cơ sở dữ liệu có thể quên đi dữ liệu mà nó đã đọc.

When a transaction writes to the database, it must look in the indexes for any other transactions that have recently read the affected data. This process is similar to acquiring a write lock on the affected key range, but rather than blocking until the readers have committed, the lock acts as a tripwire: it simply notifies the transactions that the data they read may no longer be up to date.

Khi một giao dịch ghi vào cơ sở dữ liệu, nó phải nhìn vào các chỉ mục để tìm bất kỳ giao dịch nào khác đã gần đây đọc dữ liệu bị ảnh hưởng. Quá trình này tương tự như lấy một khóa ghi trên khoảng khóa bị ảnh hưởng, nhưng thay vì chặn lại cho đến khi những người đọc đã commit, khóa hoạt động như một dây bẫy: nó đơn giản thông báo cho các giao dịch rằng dữ liệu chúng đã đọc có thể không còn cập nhật.

In Figure 7-11, transaction 43 notifies transaction 42 that its prior read is outdated, and vice versa. Transaction 42 is first to commit, and it is successful: although transaction 43's write affected 42, 43 hasn't yet committed, so the write has not yet taken effect. However, when transaction 43 wants to commit, the conflicting write from 42 has already been committed, so 43 must abort.

Trong Hình 7-11, giao dịch 43 thông báo cho giao dịch 42 rằng thao tác đọc trước đó của nó đã lỗi thời, và ngược lại. Giao dịch 42 là cái đầu tiên commit, và nó thành công: mặc dù thao tác ghi của giao dịch 43 đã ảnh hưởng đến 42, 43 chưa commit, nên thao tác ghi chưa có hiệu lực. Tuy nhiên, khi giao dịch 43 muốn commit, thao tác ghi xung đột từ 42 đã được commit rồi, nên 43 phải hủy bỏ.

Performance of serializable snapshot isolation — Hiệu năng của cô lập ảnh chụp khả tuần tự hóa

As always, many engineering details affect how well an algorithm works in practice. For example, one trade-off is the granularity at which transactions' reads and writes are tracked. If the database keeps track of each transaction's activity in great detail, it can be precise about which transactions need to abort, but the bookkeeping overhead can become significant. Less detailed tracking is faster, but may lead to more transactions being aborted than strictly necessary.

Như mọi khi, nhiều chi tiết kỹ thuật ảnh hưởng đến việc một thuật toán hoạt động tốt ra sao trong thực tế. Ví dụ, một sự đánh đổi là độ chi tiết mà ở đó các thao tác đọc và ghi của các giao dịch được theo dõi. Nếu cơ sở dữ liệu theo dõi hoạt động của mỗi giao dịch một cách rất chi tiết, nó có thể chính xác về việc những giao dịch nào cần hủy bỏ, nhưng chi phí ghi sổ có thể trở nên đáng kể. Việc theo dõi ít chi tiết hơn thì nhanh hơn, nhưng có thể dẫn đến nhiều giao dịch bị hủy bỏ hơn mức thực sự cần thiết.

In some cases, it's okay for a transaction to read information that was overwritten by another transaction: depending on what else happened, it's sometimes possible to prove that the result of the execution is nevertheless serializable. PostgreSQL uses this theory to reduce the number of unnecessary aborts [11, 41].

Trong một số trường hợp, việc một giao dịch đọc thông tin đã bị một giao dịch khác ghi đè là không sao: tùy vào những gì khác đã xảy ra, đôi khi có thể chứng minh rằng kết quả của việc thực thi đầu vậy vẫn khả tuần tự hóa. PostgreSQL dùng lý thuyết này để giảm số lần hủy bỏ không cần thiết [11, 41].

Compared to two-phase locking, the big advantage of serializable snapshot isolation is that one transaction doesn't need to block waiting for locks held by another transaction. Like under snapshot isolation, writers don't block readers, and vice versa. This design principle makes query **latency** much more predictable and less variable. In particular, read-only queries can run on a consistent snapshot without requiring any locks, which is very appealing for read-heavy workloads.

So với khóa hai pha, lợi thế lớn của cô lập ảnh chụp khả tuần tự hóa là một giao dịch không cần chặn lại để chờ các khóa do một giao dịch khác giữ. Giống như dưới snapshot isolation, người ghi không chặn người đọc, và ngược lại. Nguyên tắc thiết kế này khiến độ trễ truy vấn dễ dự đoán hơn nhiều và ít biến động hơn. Đặc biệt, các truy vấn chỉ-đọc có thể chạy trên một ảnh chụp nhất quán mà không đòi hỏi bất kỳ khóa nào, điều rất hấp dẫn cho các khối lượng công việc thiên về đọc.

Compared to serial execution, serializable snapshot isolation is not limited to the throughput of a single CPU core: FoundationDB distributes the detection of serialization conflicts across multiple machines, allowing it to scale to very high throughput. Even though data may be partitioned across multiple machines, transactions can read and write data in multiple partitions while ensuring serializable isolation [54].

So với thực thi tuần tự, cô lập ảnh chụp khả tuần tự hóa không bị giới hạn ở thông lượng của một lõi CPU duy nhất: FoundationDB phân phối việc phát hiện các xung đột tuần tự hóa xuyên nhiều máy, cho phép nó mở rộng tới thông lượng rất cao. Mặc dù dữ liệu có

thể được phân vùng xuyên nhiều máy, các giao dịch có thể đọc và ghi dữ liệu trong nhiều phân vùng trong khi vẫn đảm bảo cô lập khả năng tuần tự hóa [54].

The rate of aborts significantly affects the overall performance of SSI. For example, a transaction that reads and writes data over a long period of time is likely to run into conflicts and abort, so SSI requires that read-write transactions be fairly short (long-running read-only transactions may be okay). However, SSI is probably less sensitive to slow transactions than two-phase locking or serial execution.

Tỷ lệ hủy bỏ ảnh hưởng đáng kể đến hiệu năng tổng thể của SSI. Ví dụ, một giao dịch đọc và ghi dữ liệu trong một khoảng thời gian dài có khả năng gặp xung đột và hủy bỏ, nên SSI đòi hỏi các giao dịch đọc-ghi phải khá ngắn (các giao dịch chỉ-đọc chạy lâu có thể không sao). Tuy nhiên, SSI có lẽ ít nhạy cảm với các giao dịch chậm hơn so với khóa hai pha hoặc thực thi tuần tự.

Summary — Tóm tắt

Transactions are an **abstraction** layer that allows an application to pretend that certain concurrency problems and certain kinds of hardware and software faults don't exist. A large class of errors is reduced down to a simple transaction abort, and the application just needs to try again.

Giao dịch là một lớp trừu tượng cho phép một ứng dụng giả vờ rằng một số vấn đề tương tranh và một số loại lỗi phần cứng và phần mềm không tồn tại. Một lớp lớn các lỗi được rút gọn xuống thành một lần hủy bỏ giao dịch đơn giản, và ứng dụng chỉ cần thử lại.

In this chapter we saw many examples of problems that transactions help prevent. Not all applications are susceptible to all those problems: an application with very simple access patterns, such as reading and writing only a single record, can probably manage without transactions. However, for more complex access patterns, transactions can hugely reduce the number of potential error cases you need to think about.

Trong chương này ta đã thấy nhiều ví dụ về các vấn đề mà giao dịch giúp ngăn chặn. Không phải mọi ứng dụng đều dễ gặp tất cả những vấn đề đó: một ứng dụng có các mẫu truy cập rất đơn giản, chẳng hạn chỉ đọc và ghi một bản ghi duy nhất, có lẽ có thể xoay sở mà không cần giao dịch. Tuy nhiên, với các mẫu truy cập phức tạp hơn, giao dịch có thể giảm đi đáng kể số trường hợp lỗi tiềm tàng mà bạn cần phải nghĩ tới.

Without transactions, various error scenarios (processes crashing, network interruptions, power outages, disk full, unexpected concurrency, etc.) mean that data can become inconsistent in various ways. For example, denormalized data can easily go out of sync with the source data. Without transactions, it becomes very difficult to reason about the effects that complex interacting accesses can have on the database.

Không có giao dịch, nhiều kịch bản lỗi (tiến trình sập, gián đoạn mạng, mất điện, đĩa đầy, tương tranh bất ngờ, v.v.) có nghĩa là dữ liệu có thể trở nên không nhất quán theo nhiều cách. Ví dụ, dữ liệu phi chuẩn hóa có thể dễ dàng mất đồng bộ với dữ liệu nguồn. Không có giao dịch, sẽ trở nên rất khó để suy luận về các hiệu ứng mà những truy cập phức tạp tương tác với nhau có thể gây ra lên cơ sở dữ liệu.

In this chapter, we went particularly deep into the topic of concurrency control. We discussed several widely used isolation levels, in particular read committed, snapshot isolation (sometimes called repeatable read), and serializable. We characterized those isolation levels by discussing various examples of race conditions:

Trong chương này, ta đã đi đặc biệt sâu vào chủ đề kiểm soát tương tranh. Ta đã bàn về vài mức cô lập được dùng rộng rãi, cụ thể là read committed, snapshot isolation (đôi khi được gọi là repeatable read), và serializable. Ta đã đặc trưng hóa những mức cô lập đó bằng cách bàn về nhiều ví dụ khác nhau về tình trạng tranh đua:

One client reads another client's writes before they have been committed. The read committed isolation level and stronger levels prevent dirty reads.

Một client đọc các thao tác ghi của một client khác trước khi chúng được commit. Mức cô lập read committed và các mức mạnh hơn ngăn chặn đọc bẩn.

One client overwrites data that another client has written, but not yet committed. Almost all transaction implementations prevent dirty writes.

Một client ghi đè lên dữ liệu mà một client khác đã ghi, nhưng chưa commit. Gần như mọi hiện thực giao dịch đều ngăn chặn ghi bẩn.

A client sees different parts of the database at different points in time. This issue is most commonly prevented with snapshot isolation, which allows a transaction to read from a consistent snapshot at one point in time. It is usually implemented with multi-version concurrency control (MVCC).

Một client thấy các phần khác nhau của cơ sở dữ liệu ở những thời điểm khác nhau. Vấn đề này phổ biến nhất được ngăn chặn bằng snapshot isolation, vốn cho phép một giao dịch đọc từ một ảnh chụp nhất quán tại một thời điểm. Nó thường được hiện thực bằng kiểm soát tương tranh đa phiên bản (MVCC).

Two clients concurrently perform a read-modify-write cycle. One overwrites the other's write without incorporating its changes, so data is lost. Some implementations of snapshot isolation prevent this anomaly automatically, while others require a manual lock (SELECT FOR UPDATE).

Hai client đồng thời thực hiện một chu trình đọc-sửa-ghi. Một cái ghi đè lên thao tác ghi của cái kia mà không tích hợp các thay đổi của nó, nên dữ liệu bị mất. Một số hiện thực của snapshot isolation ngăn chặn dị thường này một cách tự động, trong khi những hiện thực khác đòi hỏi một khóa thủ công (SELECT FOR UPDATE).

A transaction reads something, makes a decision based on the value it saw, and writes the decision to the database. However, by the time the write is made, the premise of the decision is no longer true. Only serializable isolation prevents this anomaly.

Một giao dịch đọc một thứ gì đó, đưa ra một quyết định dựa trên giá trị nó đã thấy, và ghi quyết định đó vào cơ sở dữ liệu. Tuy nhiên, vào thời điểm thao tác ghi được thực hiện, tiền đề của quyết định không còn đúng. Chỉ cô lập khả tuần tự hóa mới ngăn chặn được dị thường này.

A transaction reads objects that match some search condition. Another client makes a write that affects the results of that search. Snapshot isolation prevents straightforward phantom reads, but phantoms in the context of write skew require special treatment, such as index-range locks.

Một giao dịch đọc các đối tượng khớp với một điều kiện tìm kiếm nào đó. Một client khác thực hiện một thao tác ghi ảnh hưởng đến các kết quả của lần tìm kiếm đó. Snapshot isolation ngăn chặn các lần đọc bóng ma đơn giản, nhưng bóng ma trong ngữ cảnh lệch khi ghi đòi hỏi sự xử lý đặc biệt, chẳng hạn khóa khoảng chỉ mục.

Weak isolation levels protect against some of those anomalies but leave you, the application developer, to handle others manually (e.g., using explicit locking). Only serializable isolation protects against all of these issues. We discussed three different approaches to implementing serializable transactions:

Các mức cô lập yếu bảo vệ trước một số những dị thường đó nhưng để lại cho bạn, lập trình viên ứng dụng, xử lý những cái khác một cách thủ công (ví dụ, dùng khóa tường minh). Chỉ cô lập khả tuần tự hóa mới bảo vệ trước tất cả những vấn đề này. Ta đã bàn về ba cách tiếp cận khác nhau để hiện thực các giao dịch khả tuần tự hóa:

If you can make each transaction very fast to execute, and the transaction throughput is low enough to process on a single CPU core, this is a simple and effective option.

Nếu bạn có thể làm cho mỗi giao dịch thực thi rất nhanh, và thông lượng giao dịch đủ thấp để xử lý trên một lõi CPU duy nhất, đây là một lựa chọn đơn giản và hiệu quả.

For decades this has been the standard way of implementing serializability, but many applications avoid using it because of its performance characteristics.

Trong nhiều thập kỷ đây đã là cách tiêu chuẩn để hiện thực tính khả tuần tự hóa, nhưng nhiều ứng dụng tránh dùng nó vì các đặc tính hiệu năng của nó.

A fairly new algorithm that avoids most of the downsides of the previous approaches. It uses an optimistic approach, allowing transactions to proceed without blocking. When a transaction wants to commit, it is checked, and it is aborted if the execution was not serializable.

Một thuật toán khá mới tránh được hầu hết các nhược điểm của những cách tiếp cận trước. Nó dùng một cách tiếp cận lạc quan, cho phép các giao dịch tiến triển mà không chặn lại. Khi một giao dịch muốn commit, nó được kiểm tra, và nó bị hủy bỏ nếu việc thực thi không khả tuần tự hóa.

The examples in this chapter used a relational data model. However, as discussed in “The need for multi-object transactions”, transactions are a valuable database feature, no matter which data model is used.

Các ví dụ trong chương này dùng một mô hình dữ liệu quan hệ. Tuy nhiên, như đã bàn trong “Nhu cầu về giao dịch đa đối tượng”, giao dịch là một tính năng cơ sở dữ liệu giá trị, bất kể mô hình dữ liệu nào được dùng.

In this chapter, we explored ideas and algorithms mostly in the context of a database running on a single machine. Transactions in distributed databases open a new set of difficult challenges, which we’ll discuss in the next two chapters.

Trong chương này, ta đã khám phá các ý tưởng và thuật toán chủ yếu trong ngữ cảnh một cơ sở dữ liệu chạy trên một máy duy nhất. Giao dịch trong các cơ sở dữ liệu phân tán mở ra một tập mới những thách thức khó khăn, mà ta sẽ bàn trong hai chương tiếp theo.

Footnotes Joe Hellerstein has remarked that the C in ACID was “tossed in to make the acronym work” in Härder and Reuter’s paper [7], and that it wasn’t considered important at the time.

Arguably, an incorrect counter in an email application is not a particularly critical problem. Alternatively, think of a customer account balance instead of an unread counter, and a payment transaction instead of an email.

This is not ideal. If the TCP connection is interrupted, the transaction must be aborted. If the interruption happens after the client has requested a commit but before the server acknowledges that the commit happened, the client doesn’t know whether the transaction was committed or not. To solve this issue, a transaction manager can group operations by a unique transaction identifier that is not bound to a particular TCP connection. We will return to this topic in “The End-to-End Argument for Databases”.

Strictly speaking, the term atomic increment uses the word atomic in the sense of multi-threaded programming. In the context of ACID, it should actually be called isolated or serializable increment. But that’s getting nitpicky.

Some databases support an even weaker isolation level called read uncommitted. It prevents dirty writes, but does not prevent dirty reads.

At the time of writing, the only mainstream databases that use locks for read committed isolation are IBM DB2 and Microsoft SQL Server in the `read_committed_snapshot=off` configuration [23, 36].

To be precise, transaction IDs are 32-bit integers, so they overflow after approximately 4 billion transactions. PostgreSQL's vacuum process performs cleanup which ensures that overflow does not affect the data.

It is possible, albeit fairly complicated, to express the editing of a text document as a stream of atomic mutations. See "Automatic Conflict Resolution" for some pointers.

In PostgreSQL you can do this more elegantly using range types, but they are not widely supported in other databases.

If a transaction needs to access data that's not in memory, the best solution may be to abort the transaction, asynchronously fetch the data into memory while continuing to process other transactions, and then restart the transaction when the data has been loaded. This approach is known as anti-caching, as previously mentioned in "Keeping everything in memory".

Sometimes called strong strict two-phase locking (SS2PL) to distinguish it from other variants of 2PL.

References [1] Donald D. Chamberlin, Morton M. Astrahan, Michael W. Blasgen, et al.: "A History and Evaluation of System R," *Communications of the ACM*, volume 24, number 10, pages 632–646, October 1981. doi:10.1145/358769.358784

[2] Jim N. Gray, Raymond A. Lorie, Gianfranco R. Putzolu, and Irving L. Traiger: "Granularity of Locks and Degrees of Consistency in a Shared Data Base," in *Modelling in Data Base Management Systems: Proceedings of the IFIP Working Conference on Modelling in Data Base Management Systems*, edited by G. M. Nijssen, pages 364–394, Elsevier/North Holland Publishing, 1976. Also in *Readings in Database Systems*, 4th edition, edited by Joseph M. Hellerstein and Michael Stonebraker, MIT Press, 2005. ISBN: 978-0-262-69314-1

[3] Kapali P. Eswaran, Jim N. Gray, Raymond A. Lorie, and Irving L. Traiger: "The Notions of Consistency and Predicate Locks in a Database System," *Communications of the ACM*, volume 19, number 11, pages 624–633, November 1976.

[4] "ACID Transactions Are Incredibly Helpful," FoundationDB, LLC, 2013.

[5] John D. Cook: "ACID Versus BASE for Database Transactions," johndcook.com, July 6, 2009.

[6] Gavin Clarke: "NoSQL's CAP Theorem Busters: We Don't Drop ACID," theregister.co.uk, November 22, 2012.

[7] Theo Härder and Andreas Reuter: "Principles of Transaction-Oriented Database Recovery," *ACM Computing Surveys*, volume 15, number 4, pages 287–317, December 1983. doi:10.1145/289.291

[8] Peter Bailis, Alan Fekete, Ali Ghodsi, et al.: "HAT, not CAP: Towards Highly Available Transactions," at 14th USENIX Workshop on Hot Topics in Operating Systems (HotOS), May 2013.

[9] Armando Fox, Steven D. Gribble, Yatin Chawathe, et al.: "Cluster-Based Scalable Network Services," at 16th ACM Symposium on Operating Systems Principles (SOSP), October 1997.

[10] Philip A. Bernstein, Vassos Hadzilacos, and Nathan Goodman: *Concurrency Control and Recovery in Database Systems*. Addison-Wesley, 1987. ISBN: 978-0-201-10715-9, available online at research.microsoft.com.

[11] Alan Fekete, Dimitrios Liarokapis, Elizabeth O'Neil, et al.: "Making Snapshot Isolation Serializable," *ACM Transactions on Database Systems*, volume 30, number 2, pages 492–528, June 2005. doi:10.1145/1071610.1071615

- [12] Mai Zheng, Joseph Tucek, Feng Qin, and Mark Lillibridge: “Understanding the Robustness of SSDs Under Power Fault,” at 11th USENIX Conference on File and Storage Technologies (FAST), February 2013.
- [13] Laurie Denness: “SSDs: A Gift and a Curse,” laurie.ie, June 2, 2015.
- [14] Adam Surak: “When Solid State Drives Are Not That Solid,” blog.algolia.com, June 15, 2015.
- [15] Thanumalayan Sankaranarayana Pillai, Vijay Chidambaram, Ramnathan Alagappan, et al.: “All File Systems Are Not Created Equal: On the Complexity of Crafting Crash-Consistent Applications,” at 11th USENIX Symposium on Operating Systems Design and Implementation (OSDI), October 2014.
- [16] Chris Siebenmann: “Unix’s File Durability Problem,” utcc.utoronto.ca, April 14, 2016.
- [17] Lakshmi N. Bairavasundaram, Garth R. Goodson, Bianca Schroeder, et al.: “An Analysis of Data Corruption in the Storage Stack,” at 6th USENIX Conference on File and Storage Technologies (FAST), February 2008.
- [18] Bianca Schroeder, Raghav Lagisetty, and Arif Merchant: “Flash Reliability in Production: The Expected and the Unexpected,” at 14th USENIX Conference on File and Storage Technologies (FAST), February 2016.
- [19] Don Allison: “SSD Storage – Ignorance of Technology Is No Excuse,” blog.korelogic.com, March 24, 2015.
- [20] Dave Scherer: “Those Are Not Transactions (Cassandra 2.0),” blog.foundationdb.com, September 6, 2013.
- [21] Kyle Kingsbury: “Call Me Maybe: Cassandra,” aphyr.com, September 24, 2013.
- [22] “ACID Support in Aerospike,” Aerospike, Inc., June 2014.
- [23] Martin Kleppmann: “Hermitage: Testing the ‘I’ in ACID,” martin.kleppmann.com, November 25, 2014.
- [24] Tristan D’Agosta: “BTC Stolen from Poloniex,” bitcointalk.org, March 4, 2014.
- [25] bitcointhief2: “How I Stole Roughly 100 BTC from an Exchange and How I Could Have Stolen More!,” reddit.com, February 2, 2014.
- [26] Sudhir Jorwekar, Alan Fekete, Krithi Ramamritham, and S. Sudarshan: “Automating the Detection of Snapshot Isolation Anomalies,” at 33rd International Conference on Very Large Data Bases (VLDB), September 2007.
- [27] Michael Melanson: “Transactions: The Limits of Isolation,” michaelmelanson.net, March 20, 2014.
- [28] Hal Berenson, Philip A. Bernstein, Jim N. Gray, et al.: “A Critique of ANSI SQL Isolation Levels,” at ACM International Conference on Management of Data (SIGMOD), May 1995.
- [29] Atul Adya: “Weak Consistency: A Generalized Theory and Optimistic Implementations for Distributed Transactions,” PhD Thesis, Massachusetts Institute of Technology, March 1999.
- [30] Peter Bailis, Aaron Davidson, Alan Fekete, et al.: “Highly Available Transactions: Virtues and Limitations (Extended Version),” at 40th International Conference on Very Large Data Bases (VLDB), September 2014.
- [31] Bruce Momjian: “MVCC Unmasked,” momjian.us, July 2014.
- [32] Annamalai Gurusami: “Repeatable Read Isolation Level in InnoDB – How Consistent Read View Works,” blogs.oracle.com, January 15, 2013.

- [33] Nikita Prokopov: “Unofficial Guide to Datomic Internals,” tonsky.me, May 6, 2014.
- [34] Baron Schwartz: “Immutability, MVCC, and Garbage Collection,” xaprb.com, December 28, 2013.
- [35] J. Chris Anderson, Jan Lehnardt, and Noah Slater: CouchDB: The Definitive Guide. O’Reilly Media, 2010. ISBN: 978-0-596-15589-6
- [36] Rikdeb Mukherjee: “Isolation in DB2 (Repeatable Read, Read Stability, Cursor Stability, Uncommitted Read) with Examples,” mframes.blogspot.co.uk, July 4, 2013.
- [37] Steve Hilker: “Cursor Stability (CS) – IBM DB2 Community,” toadworld.com, March 14, 2013.
- [38] Nate Wiger: “An Atomic Rant,” nateware.com, February 18, 2010.
- [39] Joel Jacobson: “Riak 2.0: Data Types,” blog.joeljacobsen.com, March 23, 2014.
- [40] Michael J. Cahill, Uwe Röhm, and Alan Fekete: “Serializable Isolation for Snapshot Databases,” at ACM International Conference on Management of Data (SIGMOD), June 2008. doi:10.1145/1376616.1376690
- [41] Dan R. K. Ports and Kevin Grittnr: “Serializable Snapshot Isolation in PostgreSQL,” at 38th International Conference on Very Large Databases (VLDB), August 2012.
- [42] Tony Andrews: “Enforcing Complex Constraints in Oracle,” tonyandrews.blogspot.co.uk, October 15, 2004.
- [43] Douglas B. Terry, Marvin M. Theimer, Karin Petersen, et al.: “Managing Update Conflicts in Bayou, a Weakly Connected Replicated Storage System,” at 15th ACM Symposium on Operating Systems Principles (SOSP), December 1995. doi:10.1145/224056.224070
- [44] Gary Fredericks: “Postgres Serializability Bug,” github.com, September 2015.
- [45] Michael Stonebraker, Samuel Madden, Daniel J. Abadi, et al.: “The End of an Architectural Era (It’s Time for a Complete Rewrite),” at 33rd International Conference on Very Large Data Bases (VLDB), September 2007.
- [46] John Hugg: “H-Store/VoltDB Architecture vs. CEP Systems and Newer Streaming Architectures,” at Data @Scale Boston, November 2014.
- [47] Robert Kallman, Hideaki Kimura, Jonathan Natkins, et al.: “H-Store: A High-Performance, Distributed Main Memory Transaction Processing System,” Proceedings of the VLDB Endowment, volume 1, number 2, pages 1496–1499, August 2008.
- [48] Rich Hickey: “The Architecture of Datomic,” infoq.com, November 2, 2012.
- [49] John Hugg: “Debunking Myths About the VoltDB In-Memory Database,” voltdb.com, May 12, 2014.
- [50] Joseph M. Hellerstein, Michael Stonebraker, and James Hamilton: “Architecture of a Database System,” Foundations and Trends in Databases, volume 1, number 2, pages 141–259, November 2007. doi:10.1561/19000000002
- [51] Michael J. Cahill: “Serializable Isolation for Snapshot Databases,” PhD Thesis, University of Sydney, July 2009.
- [52] D. Z. Badal: “Correctness of Concurrency Control and Implications in Distributed Databases,” at 3rd International IEEE Computer Software and Applications Conference (COMPSAC), November 1979.
- [53] Rakesh Agrawal, Michael J. Carey, and Miron Livny: “Concurrency Control Performance Modeling: Alternatives and Implications,” ACM Transactions on Database Systems (TODS), volume 12, number 4, pages 609–654, December 1987. doi:10.1145/32204.32220

[54] Dave Rosenthal: “Databases at 14.4MHz,” blog.foundationdb.com, December 10, 2014.

Chapter 8. The Trouble with Distributed Systems — Những rắc rối với hệ thống phân tán

Hey I just met you The network's laggy But here's my data So store it maybe

Này anh vừa mới gặp em Mạng thì cứ lag nhưng đây là dữ liệu của em Nên có lẽ anh cứ lưu lại đi

Kyle Kingsbury, Carly Rae Jepsen and the Perils of Network Partitions (2013)

— Kyle Kingsbury, Carly Rae Jepsen and the Perils of Network Partitions (2013)

A recurring theme in the last few chapters has been how systems handle things going wrong. For example, we discussed replica failover (“Handling **Node** Outages”), replication lag (“Problems with Replication Lag”), and **concurrency** control for transactions (“Weak Isolation Levels”). As we come to understand various **edge** cases that can occur in real systems, we get better at handling them.

Một chủ đề lặp đi lặp lại trong vài chương vừa qua là cách các hệ thống xử lý khi mọi thứ trở nên trục trặc. Chẳng hạn, ta đã bàn về chuyển đổi dự phòng bản sao (“Xử lý sự cố nút”), độ trễ sao chép (“Các vấn đề với độ trễ sao chép”), và kiểm soát tương tranh cho giao dịch (“Các mức cô lập yếu”). Khi dần hiểu ra nhiều trường hợp biên có thể xảy ra trong các hệ thống thực, ta sẽ xử lý chúng tốt hơn.

However, even though we have talked a lot about faults, the last few chapters have still been too optimistic. The reality is even darker. We will now turn our pessimism to the maximum and assume that anything that can go wrong will go wrong. (Experienced systems operators will tell you that is a reasonable assumption. If you ask nicely, they might tell you some frightening stories while nursing their scars of past battles.)

Tuy nhiên, dù đã nói khá nhiều về lỗi, vài chương vừa qua vẫn còn quá lạc quan. Thực tế còn u ám hơn nhiều. Giờ ta sẽ đẩy sự bi quan của mình lên mức tối đa và giả định rằng bất cứ điều gì có thể trục trặc thì sẽ trục trặc. (Những người vận hành hệ thống dày dạn sẽ bảo bạn rằng đó là một giả định hợp lý. Nếu bạn hỏi khéo, có lẽ họ sẽ kể cho bạn vài câu chuyện rùng rợn trong lúc xoa dịu những vết sẹo từ các trận chiến đã qua.)

Working with distributed systems is fundamentally different from writing software on a single computer—and the main difference is that there are lots of new and exciting ways for things to go wrong [1, 2]. In this chapter, we will get a taste of the problems that arise in practice, and an understanding of the things we can and cannot rely on.

Làm việc với hệ thống phân tán về cơ bản khác với viết phần mềm trên một máy tính đơn

lẽ — và điểm khác biệt chính là có vô số cách mới mẻ và đầy “thú vị” để mọi thứ trục trặc [1, 2]. Trong chương này, ta sẽ nếm trải những vấn đề nảy sinh trong thực tế, cùng một hiểu biết về những gì ta có thể và không thể trông cậy.

In the end, our task as engineers is to build systems that do their job (i.e., meet the guarantees that users are expecting), in spite of everything going wrong. In Chapter 9, we will look at some examples of algorithms that can provide such guarantees in a distributed system. But first, in this chapter, we must understand what challenges we are up against.

Rốt cuộc, nhiệm vụ của ta với tư cách kỹ sư là xây dựng những hệ thống làm tròn phận sự của chúng (tức là đáp ứng các bảo đảm mà người dùng kỳ vọng), bất chấp mọi thứ đang trục trặc. Ở Chương 9, ta sẽ xem xét một vài ví dụ về các thuật toán có thể cung cấp những bảo đảm như vậy trong một hệ thống phân tán. Nhưng trước tiên, trong chương này, ta phải hiểu mình đang phải đối mặt với những thách thức gì.

This chapter is a thoroughly pessimistic and depressing overview of things that may go wrong in a distributed system. We will look into problems with networks (“Unreliable Networks”); clocks and timing issues (“Unreliable Clocks”); and we’ll discuss to what degree they are avoidable. The consequences of all these issues are disorienting, so we’ll explore how to think about the state of a distributed system and how to reason about things that have happened (“Knowledge, Truth, and Lies”).

Chương này là một bản tổng quan hết sức bi quan và ảm đạm về những thứ có thể trục trặc trong một hệ thống phân tán. Ta sẽ tìm hiểu các vấn đề với mạng (“Mạng không đáng tin cậy”); các vấn đề về đồng hồ và định thời (“Đồng hồ không đáng tin cậy”); và sẽ bàn xem chúng có thể tránh được tới mức nào. Hệ quả của tất cả những vấn đề này khiến ta hoang mang, nên ta sẽ khám phá cách suy nghĩ về trạng thái của một hệ thống phân tán và cách lập luận về những gì đã xảy ra (“Tri thức, sự thật và lời dối”).

Faults and Partial Failures — Lỗi và hỏng hóc cục bộ

When you are writing a program on a single computer, it normally behaves in a fairly predictable way: either it works or it doesn't. Buggy software may give the appearance that the computer is sometimes “having a bad day” (a problem that is often fixed by a reboot), but that is mostly just a consequence of badly written software.

Khi viết một chương trình trên một máy tính đơn lẻ, nó thường hành xử theo cách khá dễ đoán: hoặc nó chạy, hoặc không. Phần mềm có bug có thể tạo cảm giác rằng máy tính đôi khi “đang có một ngày tồi tệ” (một vấn đề thường được khắc phục bằng cách khởi động lại), nhưng đó hầu như chỉ là hệ quả của phần mềm được viết tệ.

There is no fundamental reason why software on a single computer should be flaky: when the hardware is working correctly, the same operation always produces the same result (it is deterministic). If there is a hardware problem (e.g., memory corruption or a loose connector), the consequence is usually a total system **failure** (e.g., kernel panic, “blue screen of death,” failure to start up). An individual computer with good software is usually either fully functional or entirely broken, but not something in between.

Không có lý do căn bản nào khiến phần mềm trên một máy tính đơn lẻ phải chập chờn: khi phần cứng hoạt động đúng, cùng một thao tác luôn cho ra cùng một kết quả (nó mang tính tất định). Nếu có một vấn đề phần cứng (chẳng hạn bộ nhớ bị hỏng hoặc một đầu nối lỏng), hệ quả thường là cả hệ thống hỏng hoàn toàn (chẳng hạn kernel panic, “màn hình xanh chết chóc”, không khởi động được). Một máy tính riêng lẻ với phần mềm tốt thường hoặc hoạt động đầy đủ, hoặc hỏng hoàn toàn, chứ không ở trạng thái nửa vời nào đó.

This is a deliberate choice in the design of computers: if an internal **fault** occurs, we prefer a computer to crash completely rather than returning a wrong result, because wrong results are difficult and confusing to deal with. Thus, computers hide the fuzzy physical reality on which they are implemented and present an idealized system model that operates with mathematical perfection. A CPU instruction always does the same thing; if you write some data to memory or disk, that data remains intact and doesn't get randomly corrupted. This design goal of always-correct computation goes all the way back to the very first digital computer [3].

Đây là một lựa chọn có chủ đích trong thiết kế máy tính: nếu xảy ra một lỗi bên trong, ta thà để máy tính sập hoàn toàn còn hơn là trả về một kết quả sai, bởi kết quả sai rất khó xử lý và gây rối trí. Vì vậy, máy tính che giấu hiện thực vật lý mờ mịt mà chúng được dựng nên, và trình hiện một mô hình hệ thống lý tưởng hóa vận hành với sự hoàn hảo của toán học. Một lệnh CPU luôn làm cùng một việc; nếu bạn ghi dữ liệu vào bộ nhớ hay đĩa, dữ liệu đó vẫn nguyên vẹn và không bị hỏng một cách ngẫu nhiên. Mục tiêu thiết kế “tính toán luôn đúng” này có thể truy ngược về tận chiếc máy tính số đầu tiên [3].

When you are writing software that runs on several computers, connected by a network, the situation is fundamentally different. In distributed systems, we are no longer operating in an idealized system model—we have no choice but to confront the messy reality of the physical world. And in the physical world, a remarkably wide range of things can go wrong, as illustrated by this anecdote [4]:

Khi bạn viết phần mềm chạy trên nhiều máy tính, kết nối với nhau qua mạng, tình hình về cơ bản khác hẳn. Trong các hệ thống phân tán, ta không còn vận hành trong một mô hình hệ thống lý tưởng hóa nữa — ta không còn lựa chọn nào khác ngoài việc đối mặt với hiện thực hỗn độn của thế giới vật lý. Và trong thế giới vật lý, vô số thứ có thể trục trặc, như giai thoại sau minh họa [4]:

In my limited experience I've dealt with long-lived network partitions in a single data center (DC), PDU [power distribution unit] failures, switch failures, accidental power cycles of whole racks, whole-DC backbone failures, whole-DC power failures, and a hypoglycemic driver smashing his Ford pickup truck into a DC's HVAC [heating, ventilation, and air conditioning] system. And I'm not even an ops guy.

Với kinh nghiệm hạn chế của mình, tôi đã từng phải xử lý các phân vùng mạng kéo dài trong một trung tâm dữ liệu (DC) đơn lẻ, sự cố PDU [bộ phân phối điện], sự cố switch, vô tình tắt-bật nguồn cả tủ rack, sự cố đường trục của cả DC, sự cố nguồn điện của cả DC, và một tài xế bị hạ đường huyết lái chiếc xe bán tải Ford của anh ta đâm vào hệ thống HVAC [sưởi, thông gió và điều hòa không khí] của một DC. Mà tôi thậm chí còn chẳng phải dân vận hành.

Coda Hale

— Coda Hale

In a distributed system, there may well be some parts of the system that are broken in some unpredictable way, even though other parts of the system are working fine. This is known as a partial failure. The difficulty is that partial failures are nondeterministic: if you try to do anything involving multiple nodes and the network, it may sometimes work and sometimes unpredictably fail. As we shall see, you may not even know whether something succeeded or not, as the time it takes for a message to travel across a network is also nondeterministic!

Trong một hệ thống phân tán, rất có thể có những phần nào đó của hệ thống bị hỏng theo một cách không thể đoán trước, ngay cả khi những phần khác của hệ thống vẫn hoạt động ổn. Đây được gọi là hỏng hóc cục bộ (partial failure). Cái khó là hỏng hóc cục bộ mang tính bất định: nếu bạn cố làm bất cứ việc gì liên quan tới nhiều nút và mạng, đôi khi nó chạy được và đôi khi nó hỏng một cách khó lường. Như ta sẽ thấy, bạn thậm chí có thể không biết một việc gì đó có thành công hay không, vì thời gian một thông điệp đi qua mạng cũng mang tính bất định!

This nondeterminism and possibility of partial failures is what makes distributed systems hard to work with [5].

Chính tính bất định và khả năng xảy ra hỏng hóc cục bộ này khiến các hệ thống phân tán trở nên khó làm việc cùng [5].

Cloud Computing and Supercomputing — Điện toán đám mây và siêu máy tính

There is a spectrum of philosophies on how to build large-scale computing systems:

Có cả một dải các triết lý về cách xây dựng các hệ thống tính toán quy mô lớn:

- At one end of the scale is the field of high-performance computing (HPC). Supercomputers with thousands of CPUs are typically used for computationally intensive scientific computing tasks, such as weather forecasting or molecular dynamics (simulating the movement of atoms and molecules).
- At the other extreme is cloud computing, which is not very well defined [6] but is often associated with multi-tenant datacenters, commodity computers connected with an IP network (often Ethernet), **elastic**/on-demand resource allocation, and metered billing.
- Traditional enterprise datacenters lie somewhere between these extremes.
 - Ở một đầu của thang đo là lĩnh vực tính toán hiệu năng cao (HPC). Các siêu máy tính với hàng nghìn CPU thường được dùng cho các tác vụ tính toán khoa học thiên về tính toán, chẳng hạn dự báo thời tiết hoặc động lực học phân tử (mô phỏng chuyển động của các nguyên tử và phân tử).
 - Ở thái cực kia là điện toán đám mây, vốn không được định nghĩa rõ ràng cho lắm [6] nhưng thường gắn với các trung tâm dữ liệu đa tổ chức (multi-tenant), các máy tính phổ thông kết nối qua mạng IP (thường là Ethernet), cấp phát tài nguyên co giãn/theo nhu cầu, và tính tiền theo mức dùng.
 - Các trung tâm dữ liệu doanh nghiệp truyền thống nằm đâu đó giữa hai thái cực này.

With these philosophies come very different approaches to handling faults. In a supercomputer, a job typically checkpoints the state of its computation to durable storage from time to time. If one node fails, a common solution is to simply stop the entire cluster workload. After the faulty node is repaired, the computation is restarted from the last checkpoint [7, 8]. Thus, a supercomputer is more like a single-node computer than a distributed system: it deals with partial failure by letting it escalate into total failure—if any part of the system fails, just let everything crash (like a kernel panic on a single machine).

Đi kèm với những triết lý này là những cách tiếp cận rất khác nhau trong việc xử lý lỗi. Trong một siêu máy tính, một công việc thường định kỳ ghi điểm kiểm tra (checkpoint) trạng thái tính toán của nó vào kho lưu trữ bền. Nếu một nút hỏng, một giải pháp phổ biến đơn giản là dừng toàn bộ khối lượng công việc của cụm. Sau khi nút lỗi được sửa, phép tính được khởi động lại từ điểm kiểm tra gần nhất [7, 8]. Như vậy, một siêu máy tính giống một máy tính đơn nút hơn là một hệ thống phân tán: nó xử lý hỏng hóc cục bộ bằng cách để nó leo thang thành hỏng hóc toàn phần — nếu bất kỳ phần nào của hệ thống hỏng, cứ để mọi thứ sập (như một kernel panic trên một máy đơn lẻ).

In this book we focus on systems for implementing internet services, which usually look very different from supercomputers:

Trong cuốn sách này, ta tập trung vào các hệ thống dùng để hiện thực các dịch vụ internet, vốn thường trông rất khác với siêu máy tính:

- Many internet-related applications are online, in the sense that they need to be able to serve users with low **latency** at any time. Making the service unavailable—for example, stopping the cluster for repair—is not acceptable. In contrast, offline (batch) jobs like weather simulations can be stopped and restarted with fairly low impact.
- Supercomputers are typically built from specialized hardware, where each node is quite reliable, and nodes communicate through shared memory and remote direct memory access (RDMA). On the other hand, nodes in cloud services are built from commodity machines, which can provide equivalent performance at lower cost due to economies of scale, but also have higher failure rates.

- Large **datacenter** networks are often based on IP and Ethernet, arranged in Clos topologies to provide high bisection bandwidth [9]. Supercomputers often use specialized network topologies, such as multi-dimensional meshes and toruses [10], which yield better performance for HPC workloads with known communication patterns.
- The bigger a system gets, the more likely it is that one of its components is broken. Over time, broken things get fixed and new things break, but in a system with thousands of nodes, it is reasonable to assume that something is always broken [7]. When the error handling strategy consists of simply giving up, a large system can end up spending a lot of its time recovering from faults rather than doing useful work [8].
- If the system can tolerate failed nodes and still keep working as a whole, that is a very useful feature for operations and maintenance: for example, you can perform a **rolling upgrade** (see Chapter 4), restarting one node at a time, while the service continues serving users without interruption. In cloud environments, if one **virtual machine** is not performing well, you can just kill it and request a new one (hoping that the new one will be faster).
- In a geographically distributed **deployment** (keeping data geographically close to your users to reduce access latency), communication most likely goes over the internet, which is slow and unreliable compared to local networks. Supercomputers generally assume that all of their nodes are close together.
 - Nhiều ứng dụng liên quan tới internet là trực tuyến, theo nghĩa chúng cần phục vụ người dùng với độ trễ thấp vào bất cứ lúc nào. Làm cho dịch vụ không sẵn sàng — chẳng hạn dừng cụm để sửa chữa — là điều không thể chấp nhận. Ngược lại, các công việc ngoại tuyến (theo lô) như mô phỏng thời tiết có thể dừng rồi khởi động lại với mức ảnh hưởng khá thấp.
 - Siêu máy tính thường được dựng từ phần cứng chuyên dụng, trong đó mỗi nút khá đáng tin cậy, và các nút giao tiếp qua bộ nhớ chia sẻ và truy cập bộ nhớ trực tiếp từ xa (RDMA). Mặt khác, các nút trong dịch vụ đám mây được dựng từ các máy phổ thông, vốn có thể đem lại hiệu năng tương đương với chi phí thấp hơn nhờ lợi thế kinh tế theo quy mô, nhưng cũng có tỷ lệ hỏng cao hơn.
 - Các mạng trung tâm dữ liệu lớn thường dựa trên IP và Ethernet, được bố trí theo các tô-pô Clos để cung cấp băng thông cắt đôi (bisection bandwidth) cao [9]. Siêu máy tính thường dùng các tô-pô mạng chuyên dụng, chẳng hạn lưới và hình xuyên đa chiều [10], vốn cho hiệu năng tốt hơn với các khối lượng công việc HPC có mẫu giao tiếp đã biết.
 - Hệ thống càng lớn, càng có khả năng một trong các thành phần của nó bị hỏng. Theo thời gian, những thứ hỏng được sửa và những thứ mới lại hỏng, nhưng trong một hệ thống với hàng nghìn nút, hợp lý mà nói có thể giả định rằng luôn có thứ gì đó đang hỏng [7]. Khi chiến lược xử lý lỗi chỉ là đơn giản bỏ cuộc, một hệ thống lớn có thể rất cuộc dành phần lớn thời gian để phục hồi sau lỗi thay vì làm việc hữu ích [8].
 - Nếu hệ thống có thể chịu được các nút hỏng mà vẫn tiếp tục hoạt động như một tổng thể, đó là một tính năng rất hữu ích cho vận hành và bảo trì: chẳng hạn, bạn có thể thực hiện một đợt nâng cấp cuốn chiếu (xem Chương 4), khởi động lại từng nút một, trong khi dịch vụ vẫn tiếp tục phục vụ người dùng mà không gián đoạn. Trong môi trường đám mây, nếu một máy ảo không hoạt động tốt, bạn chỉ cần “giết” nó đi và yêu cầu một cái mới (với hy vọng cái mới sẽ nhanh hơn).
 - Trong một triển khai phân tán về mặt địa lý (giữ dữ liệu ở gần người dùng về mặt địa lý để giảm độ trễ truy cập), việc giao tiếp nhiều khả năng đi qua internet, vốn chậm và không đáng tin cậy so với các mạng cục bộ. Siêu máy tính nhìn chung giả định rằng tất cả các nút của chúng đều ở gần nhau.

If we want to make distributed systems work, we must accept the possibility of partial failure and

build fault-tolerance mechanisms into the software. In other words, we need to build a reliable system from unreliable components. (As discussed in “**Reliability**”, there is no such thing as perfect reliability, so we’ll need to understand the limits of what we can realistically promise.)

Nếu muốn làm cho các hệ thống phân tán hoạt động được, ta phải chấp nhận khả năng xảy ra hỏng hóc cục bộ và xây dựng các cơ chế chịu lỗi vào trong phần mềm. Nói cách khác, ta cần xây dựng một hệ thống đáng tin cậy từ những thành phần không đáng tin cậy. (Như đã bàn ở phần “Độ tin cậy”, không có thứ gì gọi là độ tin cậy hoàn hảo, nên ta sẽ cần hiểu giới hạn của những gì ta có thể hứa hẹn một cách thực tế.)

Even in smaller systems consisting of only a few nodes, it’s important to think about partial failure. In a small system, it’s quite likely that most of the components are working correctly most of the time. However, sooner or later, some part of the system will become faulty, and the software will have to somehow handle it. The fault handling must be part of the software design, and you (as operator of the software) need to know what behavior to expect from the software in the case of a fault.

Ngay cả trong các hệ thống nhỏ hơn chỉ gồm vài nút, việc nghĩ về hỏng hóc cục bộ vẫn quan trọng. Trong một hệ thống nhỏ, khá có khả năng phần lớn các thành phần đều hoạt động đúng trong phần lớn thời gian. Tuy nhiên, sớm hay muộn, một phần nào đó của hệ thống sẽ trở nên lỗi, và phần mềm sẽ phải xử lý nó bằng cách nào đó. Việc xử lý lỗi phải là một phần của thiết kế phần mềm, và bạn (với tư cách người vận hành phần mềm) cần biết phải mong đợi hành vi nào từ phần mềm khi có lỗi.

It would be unwise to assume that faults are rare and simply hope for the best. It is important to consider a wide range of possible faults—even fairly unlikely ones—and to artificially create such situations in your testing environment to see what happens. In distributed systems, suspicion, pessimism, and paranoia pay off.

Sẽ là khôn ngoan nếu không giả định rằng lỗi là hiếm và chỉ đơn giản hy vọng vào điều tốt đẹp nhất. Điều quan trọng là cân nhắc một dải rộng các lỗi có thể xảy ra — kể cả những lỗi khá khó xảy ra — và tạo ra những tình huống như vậy một cách giả tạo trong môi trường kiểm thử để xem điều gì xảy ra. Trong các hệ thống phân tán, sự nghi ngờ, bi quan và hoang tưởng đều được đền đáp.

Building a Reliable System from Unreliable Components — Xây dựng một hệ thống đáng tin cậy từ những thành phần không đáng tin cậy You may wonder whether this makes any sense—intuitively it may seem like a system can only be as reliable as its least reliable component (its weakest link). This is not the case: in fact, it is an old idea in computing to construct a more reliable system from a less reliable underlying base [11]. For example:

Bạn có thể thắc mắc liệu điều này có hợp lý không — theo trực giác, có vẻ một hệ thống chỉ có thể đáng tin cậy bằng thành phần kém tin cậy nhất của nó (mắt xích yếu nhất). Thực ra không phải vậy: trên thực tế, việc dựng một hệ thống đáng tin cậy hơn từ một nền tảng kém tin cậy hơn là một ý tưởng cũ trong ngành máy tính [11]. Ví dụ:

- Error-correcting codes allow digital data to be transmitted accurately across a communication channel that occasionally gets some bits wrong, for example due to radio interference on a wireless network [12].
- IP (the Internet Protocol) is unreliable: it may drop, delay, duplicate, or reorder packets. TCP (the Transmission Control Protocol) provides a more reliable transport layer on top of IP: it ensures that missing packets are retransmitted, duplicates are eliminated, and packets are reassembled into the order in which they were sent.

- Các mã sửa lỗi cho phép dữ liệu số được truyền chính xác qua một kênh truyền thông thỉnh thoảng làm sai một số bit, chẳng hạn do nhiễu sóng vô tuyến trên một mạng không dây [12].
- IP (Giao thức Internet) không đáng tin cậy: nó có thể làm rớt, làm trễ, nhân đôi, hoặc đảo thứ tự các gói. TCP (Giao thức Điều khiển Truyền) cung cấp một tầng truyền tải đáng tin cậy hơn nằm trên IP: nó bảo đảm rằng các gói bị mất được truyền lại, các gói trùng lặp được loại bỏ, và các gói được lắp ráp lại đúng thứ tự mà chúng đã được gửi đi.

Although the system can be more reliable than its underlying parts, there is always a limit to how much more reliable it can be. For example, error-correcting codes can deal with a small number of single-bit errors, but if your signal is swamped by interference, there is a fundamental limit to how much data you can get through your communication channel [13]. TCP can hide packet loss, duplication, and reordering from you, but it cannot magically remove delays in the network.

Mặc dù hệ thống có thể đáng tin cậy hơn các thành phần bên dưới nó, luôn có một giới hạn cho mức nó có thể đáng tin cậy hơn tới đâu. Chẳng hạn, các mã sửa lỗi có thể xử lý một số ít lỗi đơn-bit, nhưng nếu tín hiệu của bạn bị nhiễu nhấn chìm, thì có một giới hạn căn bản cho lượng dữ liệu bạn có thể truyền qua kênh truyền thông [13]. TCP có thể che giấu việc mất gói, nhân đôi, và đảo thứ tự khỏi bạn, nhưng nó không thể loại bỏ các độ trễ trong mạng một cách thần kỳ.

Although the more reliable higher-level system is not perfect, it's still useful because it takes care of some of the tricky low-level faults, and so the remaining faults are usually easier to reason about and deal with. We will explore this matter further in “The end-to-end argument”.

Mặc dù hệ thống ở tầng cao hơn, đáng tin cậy hơn, không hoàn hảo, nó vẫn hữu ích vì nó lo liệu một số lỗi hóc búa ở tầng thấp, và nhờ đó những lỗi còn lại thường dễ lập luận và xử lý hơn. Ta sẽ khám phá vấn đề này sâu hơn ở phần “Lập luận đầu-cuối”.

Unreliable Networks — Mạng không đáng tin cậy

As discussed in the introduction to Part II, the distributed systems we focus on in this book are shared-nothing systems: i.e., a bunch of machines connected by a network. The network is the only way those machines can communicate—we assume that each machine has its own memory and disk, and one machine cannot access another machine’s memory or disk (except by making requests to a service over the network).

Như đã bàn trong phần mở đầu của Phần II, các hệ thống phân tán mà ta tập trung trong cuốn sách này là các hệ thống shared-nothing: tức là một loạt máy kết nối với nhau qua mạng. Mạng là cách duy nhất để các máy đó giao tiếp — ta giả định rằng mỗi máy có bộ nhớ và đĩa riêng, và một máy không thể truy cập bộ nhớ hay đĩa của máy khác (ngoại trừ bằng cách gửi yêu cầu tới một dịch vụ qua mạng).

Shared-nothing is not the only way of building systems, but it has become the dominant approach for building internet services, for several reasons: it’s comparatively cheap because it requires no special hardware, it can make use of commoditized cloud computing services, and it can achieve high reliability through **redundancy** across multiple geographically distributed datacenters.

Shared-nothing không phải cách duy nhất để xây dựng hệ thống, nhưng nó đã trở thành cách tiếp cận chủ đạo để xây dựng các dịch vụ internet, vì vài lý do: nó tương đối rẻ vì không đòi hỏi phần cứng đặc biệt, nó có thể tận dụng các dịch vụ điện toán đám mây đã được phổ thông hóa, và nó có thể đạt độ tin cậy cao thông qua dự phòng trên nhiều trung tâm dữ liệu phân tán về mặt địa lý.

The internet and most internal networks in datacenters (often Ethernet) are asynchronous packet networks. In this kind of network, one node can send a message (a packet) to another node, but the network gives no guarantees as to when it will arrive, or whether it will arrive at all. If you send a request and expect a response, many things could go wrong (some of which are illustrated in Figure 8-1):

Internet và hầu hết các mạng nội bộ trong các trung tâm dữ liệu (thường là Ethernet) là các mạng gói bất đồng bộ. Trong loại mạng này, một nút có thể gửi một thông điệp (một gói) tới nút khác, nhưng mạng không đưa ra bảo đảm nào về việc khi nào nó sẽ tới, hay liệu nó có tới hay không. Nếu bạn gửi một yêu cầu và mong một phản hồi, nhiều thứ có thể trục trặc (một số trong đó được minh họa ở Hình 8-1):

- Your request may have been lost (perhaps someone unplugged a network cable).
- Your request may be waiting in a **queue** and will be delivered later (perhaps the network or the recipient is overloaded).
- The remote node may have failed (perhaps it crashed or it was powered down).

- The remote node may have temporarily stopped responding (perhaps it is experiencing a long **garbage collection** pause; see “Process Pauses”), but it will start responding again later.
 - The remote node may have processed your request, but the response has been lost on the network (perhaps a network switch has been misconfigured).
 - The remote node may have processed your request, but the response has been delayed and will be delivered later (perhaps the network or your own machine is overloaded).
- Yêu cầu của bạn có thể đã bị mất (có lẽ ai đó đã rút một sợi cáp mạng).
 - Yêu cầu của bạn có thể đang chờ trong một hàng đợi và sẽ được giao sau (có lẽ mạng hoặc bên nhận đang quá tải).
 - Nút từ xa có thể đã hỏng (có lẽ nó sập hoặc bị tắt nguồn).
 - Nút từ xa có thể đã tạm thời ngừng phản hồi (có lẽ nó đang trải qua một đợt thu gom rác kéo dài; xem “Tạm dừng tiến trình”), nhưng nó sẽ bắt đầu phản hồi trở lại sau đó.
 - Nút từ xa có thể đã xử lý yêu cầu của bạn, nhưng phản hồi đã bị mất trên mạng (có lẽ một switch mạng đã bị cấu hình sai).
 - Nút từ xa có thể đã xử lý yêu cầu của bạn, nhưng phản hồi đã bị trễ và sẽ được giao sau (có lẽ mạng hoặc chính máy của bạn đang quá tải).

Figure 8-1. If you send a request and don’t get a response, it’s not possible to distinguish whether (a) the request was lost, (b) the remote node is down, or (c) the response was lost.

— Hình 8-1. Nếu bạn gửi một yêu cầu và không nhận được phản hồi, không thể phân biệt được liệu (a) yêu cầu đã bị mất, (b) nút từ xa đang ngừng hoạt động, hay (c) phản hồi đã bị mất.

The sender can’t even tell whether the packet was delivered: the only option is for the recipient to send a response message, which may in turn be lost or delayed. These issues are indistinguishable in an asynchronous network: the only information you have is that you haven’t received a response yet. If you send a request to another node and don’t receive a response, it is impossible to tell why.

Bên gửi thậm chí không thể biết liệu gói có được giao hay không: lựa chọn duy nhất là bên nhận gửi một thông điệp phản hồi, mà thông điệp này đến lượt nó cũng có thể bị mất hoặc bị trễ. Các trường hợp này không thể phân biệt được trong một mạng bất đồng bộ: thông tin duy nhất bạn có là bạn chưa nhận được phản hồi. Nếu bạn gửi một yêu cầu tới một nút khác và không nhận được phản hồi, không thể nào biết được lý do.

The usual way of handling this issue is a timeout: after some time you give up waiting and assume that the response is not going to arrive. However, when a timeout occurs, you still don’t know whether the remote node got your request or not (and if the request is still queued somewhere, it may still be delivered to the recipient, even if the sender has given up on it).

Cách thông thường để xử lý vấn đề này là một thời gian chờ (timeout): sau một khoảng thời gian, bạn ngừng chờ và giả định rằng phản hồi sẽ không tới. Tuy nhiên, khi một thời gian chờ xảy ra, bạn vẫn không biết liệu nút từ xa có nhận được yêu cầu của bạn hay không (và nếu yêu cầu vẫn còn nằm trong hàng đợi ở đâu đó, nó vẫn có thể được giao tới bên nhận, ngay cả khi bên gửi đã từ bỏ nó).

Network Faults in Practice — Sự cố mạng trong thực tế

We have been building computer networks for decades—one might hope that by now we would have figured out how to make them reliable. However, it seems that we have not yet succeeded.

Ta đã xây dựng các mạng máy tính suốt nhiều thập kỷ — người ta có thể hy vọng rằng tới giờ ta đã tìm ra cách làm cho chúng đáng tin cậy. Tuy nhiên, có vẻ ta vẫn chưa thành công.

There are some systematic studies, and plenty of anecdotal evidence, showing that network problems can be surprisingly common, even in controlled environments like a datacenter operated by one company [14]. One study in a medium-sized datacenter found about 12 network faults per month, of which half disconnected a single machine, and half disconnected an entire rack [15]. Another study measured the failure rates of components like top-of-rack switches, aggregation switches, and **load** balancers [16]. It found that adding redundant networking gear doesn't reduce faults as much as you might hope, since it doesn't guard against human error (e.g., misconfigured switches), which is a major cause of outages.

Có một số nghiên cứu hệ thống, và rất nhiều bằng chứng giai thoại, cho thấy các vấn đề mạng có thể phổ biến đến mức đáng ngạc nhiên, ngay cả trong các môi trường được kiểm soát như một trung tâm dữ liệu do một công ty vận hành [14]. Một nghiên cứu trong một trung tâm dữ liệu cỡ trung tìm thấy khoảng 12 sự cố mạng mỗi tháng, trong đó một nửa ngắt kết nối một máy đơn lẻ, và một nửa ngắt kết nối cả một tủ rack [15]. Một nghiên cứu khác đo tỷ lệ hỏng của các thành phần như switch đầu tủ rack, switch tổng hợp, và bộ cân bằng tải [16]. Nó phát hiện rằng việc bổ sung thiết bị mạng dự phòng không làm giảm lỗi nhiều như bạn có thể hy vọng, bởi nó không phòng được lỗi do con người (chẳng hạn switch cấu hình sai), vốn là một nguyên nhân lớn gây gián đoạn dịch vụ.

Public cloud services such as EC2 are notorious for having frequent transient network glitches [14], and well-managed private datacenter networks can be stabler environments. Nevertheless, nobody is immune from network problems: for example, a problem during a software upgrade for a switch could trigger a network topology reconfiguration, during which network packets could be delayed for more than a minute [17]. Sharks might bite undersea cables and damage them [18]. Other surprising faults include a network interface that sometimes drops all inbound packets but sends outbound packets successfully [19]: just because a network link works in one direction doesn't guarantee it's also working in the opposite direction.

Các dịch vụ đám mây công cộng như EC2 nổi tiếng vì hay có những trục trặc mạng thoáng qua [14], còn các mạng trung tâm dữ liệu riêng được quản lý tốt có thể là môi trường ổn định hơn. Tuy vậy, không ai miễn nhiễm với các vấn đề mạng: chẳng hạn, một vấn đề trong lúc nâng cấp phần mềm cho một switch có thể kích hoạt một đợt tái cấu hình topology mạng, trong khi đó các gói mạng có thể bị trễ hơn một phút [17]. Cá mập có thể cắn các sợi cáp ngầm dưới biển và làm hỏng chúng [18]. Những lỗi đáng ngạc nhiên khác bao gồm một giao diện mạng đôi khi làm rớt tất cả các gói đến nhưng gửi đi các gói thành công [19]: chỉ vì một liên kết mạng hoạt động theo một chiều không bảo đảm rằng nó cũng hoạt động theo chiều ngược lại.

Network partitions — Phân vùng mạng

When one part of the network is cut off from the rest due to a network fault, that is sometimes called a network partition or netsplit. In this book we'll generally stick with the more general term network fault, to avoid confusion with partitions (shards) of a storage system, as discussed in Chapter 6.

Khi một phần của mạng bị cắt khỏi phần còn lại do một sự cố mạng, đôi khi điều đó được gọi là phân vùng mạng (network partition) hoặc netsplit. Trong cuốn sách này, ta nhìn chung sẽ dùng thuật ngữ tổng quát hơn là sự cố mạng, để tránh nhầm lẫn với các phân vùng (shard) của một hệ thống lưu trữ, như đã bàn ở Chương 6.

Even if network faults are rare in your environment, the fact that faults can occur means that your software needs to be able to handle them. Whenever any communication happens over a network, it may fail—there is no way around it.

Ngay cả khi các sự cố mạng là hiếm trong môi trường của bạn, việc lỗi có thể xảy ra nghĩa là phần mềm của bạn cần có khả năng xử lý chúng. Bất cứ khi nào có giao tiếp diễn ra qua một mạng, nó đều có thể hỏng — không có cách nào tránh được điều đó.

If the error handling of network faults is not defined and tested, arbitrarily bad things could happen: for example, the cluster could become deadlocked and permanently unable to serve requests, even when the network recovers [20], or it could even delete all of your data [21]. If software is put in an unanticipated situation, it may do arbitrary unexpected things.

Nếu việc xử lý lỗi cho các sự cố mạng không được định nghĩa và kiểm thử, những thứ tồi tệ tùy tiện có thể xảy ra: chẳng hạn, cụm có thể rơi vào tình trạng deadlock và vĩnh viễn không thể phục vụ yêu cầu, ngay cả khi mạng đã phục hồi [20], hoặc thậm chí nó có thể xóa toàn bộ dữ liệu của bạn [21]. Nếu phần mềm bị đặt vào một tình huống không lường trước, nó có thể làm những điều bất ngờ tùy tiện.

Handling network faults doesn't necessarily **mean** tolerating them: if your network is normally fairly reliable, a valid approach may be to simply show an error message to users while your network is experiencing problems. However, you do need to know how your software reacts to network problems and ensure that the system can recover from them. It may make sense to deliberately trigger network problems and test the system's response (this is the idea behind Chaos Monkey; see “Reliability”).

Xử lý các sự cố mạng không nhất thiết có nghĩa là chịu được chúng: nếu mạng của bạn thường khá đáng tin cậy, một cách tiếp cận hợp lệ có thể chỉ đơn giản là hiển thị một thông báo lỗi cho người dùng trong lúc mạng của bạn đang gặp vấn đề. Tuy nhiên, bạn cần biết phần mềm của mình phản ứng thế nào với các vấn đề mạng và bảo đảm rằng hệ thống có thể phục hồi từ chúng. Việc cố tình kích hoạt các vấn đề mạng và kiểm thử phản

ứng của hệ thống có thể là hợp lý (đây là ý tưởng đằng sau Chaos Monkey; xem “Độ tin cậy”).

Detecting Faults — Phát hiện lỗi

Many systems need to automatically detect faulty nodes. For example:

Nhiều hệ thống cần tự động phát hiện các nút lỗi. Ví dụ:

- A load balancer needs to stop sending requests to a node that is dead (i.e., take it out of rotation).
- In a distributed **database** with single-leader replication, if the leader fails, one of the followers needs to be promoted to be the new leader (see “Handling Node Outages”).
 - Một bộ cân bằng tải cần ngừng gửi yêu cầu tới một nút đã chết (tức là loại nó ra khỏi vòng luân phiên).
 - Trong một cơ sở dữ liệu phân tán dùng sao chép một-người-dẫn-đầu, nếu người dẫn đầu hỏng, một trong các người theo dõi cần được thăng cấp lên làm người dẫn đầu mới (xem “Xử lý sự cố nút”).

Unfortunately, the uncertainty about the network makes it difficult to tell whether a node is working or not. In some specific circumstances you might get some feedback to explicitly tell you that something is not working:

Thật không may, sự bất định về mạng khiến ta khó nói được một nút đang hoạt động hay không. Trong một số tình huống cụ thể, bạn có thể nhận được phản hồi tường minh báo rằng có gì đó không hoạt động:

- If you can reach the machine on which the node should be running, but no process is listening on the destination port (e.g., because the process crashed), the operating system will helpfully close or refuse TCP connections by sending a RST or FIN packet in reply. However, if the node crashed while it was handling your request, you have no way of knowing how much data was actually processed by the remote node [22].
- If a node process crashed (or was killed by an administrator) but the node’s operating system is still running, a script can notify other nodes about the crash so that another node can take over quickly without having to wait for a timeout to expire. For example, HBase does this [23].
- If you have access to the management interface of the network switches in your datacenter, you can query them to detect link failures at a hardware level (e.g., if the remote machine is powered down). This option is ruled out if you’re connecting via the internet, or if you’re in a shared datacenter with no access to the switches themselves, or if you can’t reach the management interface due to a network problem.
- If a router is sure that the IP address you’re trying to connect to is unreachable, it may reply to you with an ICMP Destination Unreachable packet. However, the router doesn’t have a magic failure detection capability either—it is **subject** to the same limitations as other participants of the network.
 - Nếu bạn có thể với tới máy mà nút lẽ ra đang chạy trên đó, nhưng không có tiến trình nào lắng nghe trên cổng đích (chẳng hạn vì tiến trình đã sập), thì hệ điều hành sẽ giúp đóng hoặc từ chối các kết nối TCP bằng cách gửi một gói RST hoặc FIN để đáp lại. Tuy nhiên, nếu nút sập trong lúc đang xử lý yêu cầu của bạn, bạn không có cách nào biết được nút từ xa thực sự đã xử lý bao nhiêu dữ liệu [22].
 - Nếu một tiến trình nút sập (hoặc bị quản trị viên “giết”) nhưng hệ điều hành của nút vẫn đang chạy, thì một script có thể thông báo cho các nút khác về sự cố sập đó để

một nút khác có thể tiếp quản nhanh chóng mà không phải chờ một thời gian chờ hết hạn. Ví dụ, HBase làm điều này [23].

- Nếu bạn có quyền truy cập giao diện quản lý của các switch mạng trong trung tâm dữ liệu của mình, bạn có thể truy vấn chúng để phát hiện các sự cố liên kết ở mức phần cứng (chẳng hạn nếu máy từ xa đã bị tắt nguồn). Lựa chọn này bị loại trừ nếu bạn đang kết nối qua internet, hoặc nếu bạn ở trong một trung tâm dữ liệu dùng chung mà không có quyền truy cập chính các switch, hoặc nếu bạn không thể với tới giao diện quản lý do một vấn đề mạng.
- Nếu một bộ định tuyến chắc chắn rằng địa chỉ IP mà bạn đang cố kết nối tới là không thể với tới, nó có thể đáp lại bạn bằng một gói ICMP Destination Unreachable. Tuy nhiên, bản thân bộ định tuyến cũng không có khả năng phát hiện lỗi thần kỳ nào — nó chịu cùng những giới hạn như các thành phần khác của mạng.

Rapid feedback about a remote node being down is useful, but you can't count on it. Even if TCP acknowledges that a packet was delivered, the application may have crashed before handling it. If you want to be sure that a request was successful, you need a positive response from the application itself [24].

Phản hồi nhanh về việc một nút từ xa đang ngừng hoạt động thì hữu ích, nhưng bạn không thể trông cậy vào nó. Ngay cả khi TCP báo nhận rằng một gói đã được giao, ứng dụng có thể đã sập trước khi xử lý nó. Nếu bạn muốn chắc chắn rằng một yêu cầu đã thành công, bạn cần một phản hồi khẳng định từ chính ứng dụng [24].

Conversely, if something has gone wrong, you may get an error response at some level of the stack, but in general you have to assume that you will get no response at all. You can retry a few times (TCP retries transparently, but you may also retry at the application level), wait for a timeout to elapse, and eventually declare the node dead if you don't hear back within the timeout.

Ngược lại, nếu có gì đó trục trặc, bạn có thể nhận được một phản hồi lỗi ở một mức nào đó của ngăn xếp, nhưng nhìn chung bạn phải giả định rằng bạn sẽ không nhận được phản hồi nào cả. Bạn có thể thử lại vài lần (TCP thử lại một cách trong suốt, nhưng bạn cũng có thể thử lại ở mức ứng dụng), chờ cho một thời gian chờ trôi qua, và rồi cuộc tuyên bố nút đã chết nếu bạn không nghe được hồi âm trong khoảng thời gian chờ.

Timeouts and Unbounded Delays — Thời gian chờ và độ trễ không có giới hạn

If a timeout is the only sure way of detecting a fault, then how long should the timeout be? There is unfortunately no simple answer.

Nếu một thời gian chờ là cách chắc chắn duy nhất để phát hiện một lỗi, thì thời gian chờ nên dài bao lâu? Thật không may, không có câu trả lời đơn giản.

A long timeout means a long wait until a node is declared dead (and during this time, users may have to wait or see error messages). A short timeout detects faults faster, but carries a higher risk of incorrectly declaring a node dead when in fact it has only suffered a temporary slowdown (e.g., due to a load spike on the node or the network).

Một thời gian chờ dài nghĩa là phải chờ lâu cho tới khi một nút bị tuyên bố là đã chết (và trong thời gian này, người dùng có thể phải chờ hoặc thấy các thông báo lỗi). Một thời gian chờ ngắn phát hiện lỗi nhanh hơn, nhưng mang theo rủi ro cao hơn là tuyên bố nhầm một nút đã chết trong khi thực ra nó chỉ bị chậm tạm thời (chẳng hạn do một đợt tăng tải đột biến trên nút hoặc trên mạng).

Prematurely declaring a node dead is problematic: if the node is actually alive and in the middle of performing some action (for example, sending an email), and another node takes over, the action may end up being performed twice. We will discuss this issue in more detail in “Knowledge, Truth, and Lies”, and in Chapters 9 and 11.

Tuyên bố một nút đã chết một cách quá sớm là điều rắc rối: nếu nút thực ra vẫn sống và đang giữa chừng thực hiện một hành động nào đó (chẳng hạn gửi một email), và một nút khác tiếp quản, thì hành động đó có thể rốt cuộc bị thực hiện hai lần. Ta sẽ bàn vấn đề này chi tiết hơn ở phần “Tri thức, sự thật và lời dối”, và trong các Chương 9 và 11.

When a node is declared dead, its responsibilities need to be transferred to other nodes, which places additional load on other nodes and the network. If the system is already struggling with high load, declaring nodes dead prematurely can make the problem worse. In particular, it could happen that the node actually wasn’t dead but only slow to respond due to overload; transferring its load to other nodes can cause a **cascading failure** (in the extreme case, all nodes declare each other dead, and everything stops working).

Khi một nút bị tuyên bố đã chết, các trách nhiệm của nó cần được chuyển cho các nút khác, điều này đặt thêm tải lên các nút khác và lên mạng. Nếu hệ thống đã đang chật vật với tải cao, việc tuyên bố các nút đã chết quá sớm có thể làm vấn đề tệ hơn. Cụ thể, có thể xảy ra chuyện nút thực ra không chết mà chỉ phản hồi chậm do quá tải; việc chuyển tải của nó sang các nút khác có thể gây ra một sự cố dây chuyền (trong trường hợp cực đoan, tất cả các nút tuyên bố lẫn nhau là đã chết, và mọi thứ ngừng hoạt động).

Imagine a fictitious system with a network that guaranteed a maximum delay for packets—every packet is either delivered within some time d , or it is lost, but delivery never takes longer than d . Furthermore, assume that you can guarantee that a non-failed node always handles a request within some time r . In this case, you could guarantee that every successful request receives a response within time $2d + r$ —and if you don’t receive a response within that time, you know that either the network or the remote node is not working. If this was true, $2d + r$ would be a reasonable timeout to use.

Hãy hình dung một hệ thống giả tưởng với một mạng bảo đảm một độ trễ tối đa cho các gói — mỗi gói hoặc được giao trong một khoảng thời gian d nào đó, hoặc bị mất, nhưng việc giao không bao giờ mất hơn d . Hơn nữa, giả định rằng bạn có thể bảo đảm một nút không-hỏng luôn xử lý một yêu cầu trong một khoảng thời gian r nào đó. Trong trường hợp này, bạn có thể bảo đảm rằng mỗi yêu cầu thành công nhận được phản hồi trong khoảng thời gian $2d + r$ — và nếu bạn không nhận được phản hồi trong khoảng thời gian đó, bạn biết rằng hoặc mạng hoặc nút từ xa đang không hoạt động. Nếu điều này đúng, thì $2d + r$ sẽ là một thời gian chờ hợp lý để dùng.

Unfortunately, most systems we work with have neither of those guarantees: asynchronous networks have unbounded delays (that is, they try to deliver packets as quickly as possible, but there is no upper limit on the time it may take for a packet to arrive), and most server implementations cannot guarantee that they can handle requests within some maximum time (see “**Response time** guarantees”). For failure detection, it’s not sufficient for the system to be fast most of the time: if your timeout is low, it only takes a transient spike in round-trip times to throw the system off-balance.

Thật không may, hầu hết các hệ thống mà ta làm việc cùng đều không có cả hai bảo đảm đó: các mạng bất đồng bộ có độ trễ không có giới hạn (tức là chúng cố giao các gói nhanh nhất có thể, nhưng không có giới hạn trên cho thời gian một gói có thể mất để tới nơi), và hầu hết các hiện thực máy chủ không thể bảo đảm rằng chúng có thể xử lý các yêu cầu trong một khoảng thời gian tối đa nào đó (xem “Bảo đảm thời gian phản hồi”). Để phát hiện lỗi, việc hệ thống nhanh trong phần lớn thời gian là chưa đủ: nếu thời gian chờ của

bạn thấp, chỉ cần một đợt tăng đột biến thoáng qua trong thời gian khứ hồi cũng đủ làm hệ thống mất cân bằng.

Network congestion and queueing — Tắc nghẽn mạng và xếp hàng

When driving a car, travel times on road networks often vary most due to traffic congestion. Similarly, the variability of packet delays on computer networks is most often due to queueing [25]:

Khi lái xe, thời gian di chuyển trên mạng lưới đường bộ thường biến động nhiều nhất do tắc nghẽn giao thông. Tương tự, sự biến động của độ trễ gói trên các mạng máy tính phần lớn là do việc xếp hàng [25]:

- If several different nodes simultaneously try to send packets to the same destination, the network switch must queue them up and feed them into the destination network link one by one (as illustrated in Figure 8-2). On a busy network link, a packet may have to wait a while until it can get a slot (this is called network congestion). If there is so much incoming data that the switch queue fills up, the packet is dropped, so it needs to be resent—even though the network is functioning fine.
- When a packet reaches the destination machine, if all CPU cores are currently busy, the incoming request from the network is queued by the operating system until the application is ready to handle it. Depending on the load on the machine, this may take an arbitrary length of time.
- In virtualized environments, a running operating system is often paused for tens of milliseconds while another virtual machine uses a CPU core. During this time, the VM cannot consume any data from the network, so the incoming data is queued (buffered) by the virtual machine monitor [26], further increasing the variability of network delays.
- TCP performs flow control (also known as congestion avoidance or backpressure), in which a node limits its own rate of sending in order to avoid overloading a network link or the receiving node [27]. This means additional queueing at the sender before the data even enters the network.
 - Nếu nhiều nút khác nhau đồng thời cố gửi gói tới cùng một đích, switch mạng phải xếp chúng vào hàng và đưa chúng vào liên kết mạng đích từng cái một (như minh họa ở Hình 8-2). Trên một liên kết mạng bận rộn, một gói có thể phải chờ một lúc cho tới khi nó có được một suất (điều này được gọi là tắc nghẽn mạng). Nếu có quá nhiều dữ liệu đến khiến hàng đợi của switch đầy, gói sẽ bị làm rớt, nên nó cần được gửi lại — ngay cả khi mạng đang hoạt động ổn.
 - Khi một gói tới máy đích, nếu tất cả các nhân CPU hiện đang bận, thì yêu cầu đến từ mạng sẽ được hệ điều hành xếp vào hàng cho tới khi ứng dụng sẵn sàng xử lý nó. Tùy vào tải trên máy, điều này có thể mất một khoảng thời gian bất kỳ.
 - Trong các môi trường ảo hóa, một hệ điều hành đang chạy thường bị tạm dừng trong vài chục mili-giây trong khi một máy ảo khác dùng một nhân CPU. Trong thời gian này, máy ảo không thể tiêu thụ bất kỳ dữ liệu nào từ mạng, nên dữ liệu đến được trình giám sát máy ảo xếp vào hàng (đệm lại) [26], càng làm tăng sự biến động của độ trễ mạng.
 - TCP thực hiện điều khiển luồng (còn gọi là tránh tắc nghẽn hoặc backpressure), trong đó một nút tự giới hạn tốc độ gửi của chính mình để tránh làm quá tải một liên kết mạng hoặc nút nhận [27]. Điều này nghĩa là có thêm việc xếp hàng ở phía bên gửi trước khi dữ liệu thậm chí đi vào mạng.

Figure 8-2. If several machines send network traffic to the same destination, its switch queue can fill up. Here, ports 1, 2, and 4 are all trying to send packets to port 3. — Hình 8-2. Nếu nhiều máy gửi lưu lượng mạng tới cùng một đích, hàng đợi của switch của nó có thể bị đầy. Ở đây, các cổng 1, 2, và 4 đều đang cố gửi gói tới cổng 3. Moreover, TCP considers a packet to be lost if it is not acknowledged within some timeout (which is calculated from observed round-trip times), and lost packets are automatically retransmitted. Although the application does not see the packet loss and retransmission, it does see the resulting delay (waiting for the timeout to expire, and then waiting for the retransmitted packet to be acknowledged).

Hơn nữa, TCP coi một gói là bị mất nếu nó không được báo nhận trong một thời gian chờ nào đó (vốn được tính từ các thời gian khứ hồi quan sát được), và các gói bị mất được tự động truyền lại. Mặc dù ứng dụng không thấy việc mất gói và truyền lại, nó vẫn thấy độ trễ phát sinh từ đó (chờ cho thời gian chờ hết hạn, rồi chờ cho gói được truyền lại được báo nhận).

TCP Versus UDP — TCP so với UDP Some latency-sensitive applications, such as videoconferencing and Voice over IP (VoIP), use UDP rather than TCP. It's a trade-off between reliability and variability of delays: as UDP does not perform flow control and does not retransmit lost packets, it avoids some of the reasons for variable network delays (although it is still susceptible to switch queues and scheduling delays).

Một số ứng dụng nhạy với độ trễ, chẳng hạn hội nghị truyền hình và truyền giọng nói qua IP (VoIP), dùng UDP thay vì TCP. Đó là một sự đánh đổi giữa độ tin cậy và sự biến động của độ trễ: vì UDP không thực hiện điều khiển luồng và không truyền lại các gói bị mất, nó tránh được một số nguyên nhân gây độ trễ mạng biến động (mặc dù nó vẫn dễ bị ảnh hưởng bởi hàng đợi của switch và độ trễ định thời).

UDP is a good choice in situations where delayed data is worthless. For example, in a VoIP phone call, there probably isn't enough time to retransmit a lost packet before its data is due to be played over the loudspeakers. In this case, there's no point in retransmitting the packet—the application must instead fill the missing packet's time slot with silence (causing a brief interruption in the sound) and move on in the stream. The retry happens at the human layer instead. (“Could you repeat that please? The sound just cut out for a moment.”)

UDP là một lựa chọn tốt trong các tình huống mà dữ liệu bị trễ thì vô giá trị. Chẳng hạn, trong một cuộc gọi điện thoại VoIP, có lẽ không có đủ thời gian để truyền lại một gói bị mất trước khi dữ liệu của nó đến lúc cần được phát ra loa. Trong trường hợp này, việc truyền lại gói chẳng có ích gì — thay vào đó ứng dụng phải lấp suất thời gian của gói bị thiếu bằng sự im lặng (gây ra một quãng ngắt âm thanh ngắn) và đi tiếp trong luồng. Việc thử lại diễn ra ở tầng con người thay vào đó. (“Bạn nhắc lại được không? Âm thanh vừa bị mất một lúc.”)

All of these factors contribute to the variability of network delays. Queueing delays have an especially wide range when a system is close to its maximum capacity: a system with plenty of spare capacity can easily drain queues, whereas in a highly utilized system, long queues can build up very quickly.

Tất cả những yếu tố này góp phần vào sự biến động của độ trễ mạng. Độ trễ xếp hàng có một dải biến động đặc biệt rộng khi một hệ thống ở gần dung lượng tối đa của nó: một hệ thống có nhiều dung lượng dư thừa có thể dễ dàng làm rỗng các hàng đợi, trong khi ở một hệ thống được sử dụng cao độ, các hàng đợi dài có thể tích tụ rất nhanh.

In public clouds and multi-tenant datacenters, resources are shared among many customers: the network links and switches, and even each machine's network interface and CPUs (when running on virtual machines), are shared. Batch workloads such as **MapReduce** (see Chapter 10) can easily

saturate network links. As you have no control over or insight into other customers' usage of the shared resources, network delays can be highly variable if someone near you (a noisy neighbor) is using a lot of resources [28, 29].

Trong các đám mây công cộng và các trung tâm dữ liệu đa tổ chức, tài nguyên được chia sẻ giữa nhiều khách hàng: các liên kết mạng và switch, và thậm chí cả giao diện mạng và CPU của mỗi máy (khi chạy trên máy ảo), đều được chia sẻ. Các khối lượng công việc theo lô như MapReduce (xem Chương 10) có thể dễ dàng làm bão hòa các liên kết mạng. Vì bạn không có quyền kiểm soát hay tầm nhìn vào việc các khách hàng khác sử dụng các tài nguyên chia sẻ, độ trễ mạng có thể biến động rất lớn nếu ai đó gần bạn (một “hàng xóm ồn ào”) đang dùng nhiều tài nguyên [28, 29].

In such environments, you can only choose timeouts experimentally: measure the distribution of network round-trip times over an extended period, and over many machines, to determine the expected variability of delays. Then, taking into account your application's characteristics, you can determine an appropriate trade-off between failure detection delay and risk of premature timeouts.

Trong những môi trường như vậy, bạn chỉ có thể chọn thời gian chờ một cách thực nghiệm: đo phân phối các thời gian khứ hồi của mạng trong một khoảng thời gian kéo dài, và trên nhiều máy, để xác định mức biến động độ trễ kỳ vọng. Sau đó, có tính tới các đặc tính của ứng dụng, bạn có thể xác định một sự đánh đổi thích hợp giữa độ trễ phát hiện lỗi và rủi ro của các thời gian chờ quá sớm.

Even better, rather than using configured constant timeouts, systems can continually measure response times and their variability (jitter), and automatically adjust timeouts according to the observed response time distribution. This can be done with a Phi Accrual failure detector [30], which is used for example in Akka and Cassandra [31]. TCP retransmission timeouts also work similarly [27].

Thậm chí tốt hơn, thay vì dùng các thời gian chờ hằng số được cấu hình sẵn, các hệ thống có thể liên tục đo các thời gian phản hồi và sự biến động của chúng (jitter), và tự động điều chỉnh các thời gian chờ theo phân phối thời gian phản hồi quan sát được. Điều này có thể được thực hiện bằng một bộ phát hiện lỗi Phi Accrual [30], vốn được dùng chẳng hạn trong Akka và Cassandra [31]. Các thời gian chờ truyền lại của TCP cũng hoạt động tương tự [27].

Synchronous Versus Asynchronous Networks — Mạng đồng bộ so với mạng bất đồng bộ

Distributed systems would be a lot simpler if we could rely on the network to deliver packets with some fixed maximum delay, and not to drop packets. Why can't we solve this at the hardware level and make the network reliable so that the software doesn't need to worry about it?

Các hệ thống phân tán sẽ đơn giản hơn nhiều nếu ta có thể trông cậy vào mạng để giao các gói với một độ trễ tối đa cố định nào đó, và không làm rớt các gói. Tại sao ta không thể giải quyết điều này ở mức phần cứng và làm cho mạng đáng tin cậy để phần mềm không cần lo lắng về nó?

To answer this question, it's interesting to compare datacenter networks to the traditional fixed-line telephone network (non-cellular, non-VoIP), which is extremely reliable: delayed audio frames and dropped calls are very rare. A phone call requires a constantly low end-to-end latency and enough bandwidth to transfer the audio samples of your voice. Wouldn't it be nice to have similar reliability and predictability in computer networks?

Để trả lời câu hỏi này, sẽ thú vị khi so sánh các mạng trung tâm dữ liệu với mạng điện thoại cố định truyền thống (không phải di động, không phải VoIP), vốn cực kỳ đáng tin cậy: các khung âm thanh bị trễ và các cuộc gọi bị rớt là rất hiếm. Một cuộc gọi điện thoại đòi hỏi một độ trễ đầu-cuối thấp liên tục và đủ bằng thông để truyền các mẫu âm thanh giọng nói của bạn. Chẳng phải sẽ tuyệt nếu có được độ tin cậy và khả năng dự đoán tương tự trong các mạng máy tính hay sao?

When you make a call over the telephone network, it establishes a circuit: a fixed, guaranteed amount of bandwidth is allocated for the call, along the entire route between the two callers. This circuit remains in place until the call ends [32]. For example, an ISDN network runs at a fixed rate of 4,000 frames per second. When a call is established, it is allocated 16 bits of space within each frame (in each direction). Thus, for the duration of the call, each side is guaranteed to be able to send exactly 16 bits of audio data every 250 microseconds [33, 34].

Khi bạn thực hiện một cuộc gọi qua mạng điện thoại, nó thiết lập một mạch (circuit): một lượng băng thông cố định, được bảo đảm sẵn được cấp phát cho cuộc gọi, dọc theo toàn bộ tuyến đường giữa hai người gọi. Mạch này vẫn được giữ nguyên cho tới khi cuộc gọi kết thúc [32]. Chẳng hạn, một mạng ISDN chạy ở tốc độ cố định 4.000 khung mỗi giây. Khi một cuộc gọi được thiết lập, nó được cấp phát 16 bit không gian trong mỗi khung (theo mỗi chiều). Như vậy, trong suốt thời gian của cuộc gọi, mỗi bên được bảo đảm có thể gửi đúng 16 bit dữ liệu âm thanh sau mỗi 250 micro-giây [33, 34].

This kind of network is synchronous: even as data passes through several routers, it does not suffer from queueing, because the 16 bits of space for the call have already been reserved in the next hop of the network. And because there is no queueing, the maximum end-to-end latency of the network is fixed. We call this a bounded delay.

Loại mạng này là đồng bộ: ngay cả khi dữ liệu đi qua nhiều bộ định tuyến, nó không phải chịu việc xếp hàng, bởi 16 bit không gian cho cuộc gọi đã được dành sẵn ở chặng tiếp theo của mạng. Và vì không có việc xếp hàng, độ trễ đầu-cuối tối đa của mạng là cố định. Ta gọi đây là một độ trễ có giới hạn.

Can we not simply make network delays predictable? — Chẳng lẽ ta không thể đơn giản làm cho độ trễ mạng dễ đoán hơn?

Note that a circuit in a telephone network is very different from a TCP connection: a circuit is a fixed amount of reserved bandwidth which nobody else can use while the circuit is established, whereas the packets of a TCP connection opportunistically use whatever network bandwidth is available. You can give TCP a variable-sized block of data (e.g., an email or a web page), and it will try to transfer it in the shortest time possible. While a TCP connection is idle, it doesn't use any bandwidth.

Lưu ý rằng một mạch trong mạng điện thoại rất khác với một kết nối TCP: một mạch là một lượng băng thông được dành sẵn cố định mà không ai khác có thể dùng trong khi mạch được thiết lập, trong khi các gói của một kết nối TCP tận dụng cơ hội dùng bất kỳ băng thông mạng nào sẵn có. Bạn có thể đưa cho TCP một khối dữ liệu có kích thước thay đổi (chẳng hạn một email hay một trang web), và nó sẽ cố truyền khối đó trong thời gian ngắn nhất có thể. Trong khi một kết nối TCP ở trạng thái nhàn rỗi, nó không dùng băng thông nào.

If datacenter networks and the internet were circuit-switched networks, it would be possible to establish a guaranteed maximum round-trip time when a circuit was set up. However, they are not: Ethernet and IP are packet-switched protocols, which suffer from queueing and thus unbounded delays in the network. These protocols do not have the concept of a circuit.

Nếu các mạng trung tâm dữ liệu và internet là các mạng chuyển mạch theo mạch, thì có thể thiết lập một thời gian khứ hồi tối đa được bảo đảm khi một mạch được thiết lập. Tuy nhiên, chúng không phải vậy: Ethernet và IP là các giao thức chuyển mạch theo gói, vốn phải chịu việc xếp hàng và do đó chịu độ trễ không có giới hạn trong mạng. Các giao thức này không có khái niệm về một mạch.

Why do datacenter networks and the internet use packet switching? The answer is that they are optimized for bursty traffic. A circuit is good for an audio or video call, which needs to transfer a fairly constant number of bits per second for the duration of the call. On the other hand, requesting a web page, sending an email, or transferring a file doesn't have any particular bandwidth requirement—we just want it to complete as quickly as possible.

Tại sao các mạng trung tâm dữ liệu và internet lại dùng chuyển mạch theo gói? Câu trả lời là chúng được tối ưu cho lưu lượng bùng nổ. Một mạch thì tốt cho một cuộc gọi âm thanh hay video, vốn cần truyền một số bit khá ổn định mỗi giây trong suốt thời gian của cuộc gọi. Mặt khác, việc yêu cầu một trang web, gửi một email, hay truyền một tập tin không có yêu cầu băng thông cụ thể nào — ta chỉ muốn nó hoàn thành nhanh nhất có thể.

If you wanted to transfer a file over a circuit, you would have to guess a bandwidth allocation. If you guess too low, the transfer is unnecessarily slow, leaving network capacity unused. If you guess too high, the circuit cannot be set up (because the network cannot allow a circuit to be created if its bandwidth allocation cannot be guaranteed). Thus, using circuits for bursty data transfers wastes network capacity and makes transfers unnecessarily slow. By contrast, TCP dynamically adapts the rate of data transfer to the available network capacity.

Nếu bạn muốn truyền một tập tin qua một mạch, bạn sẽ phải đoán một lượng băng thông cấp phát. Nếu bạn đoán quá thấp, việc truyền sẽ chậm một cách không cần thiết, để lại dung lượng mạng không được dùng. Nếu bạn đoán quá cao, mạch không thể được thiết lập (bởi mạng không thể cho phép tạo một mạch nếu không thể bảo đảm lượng băng thông cấp phát cho nó). Như vậy, việc dùng mạch cho các đợt truyền dữ liệu bùng nổ vừa lãng phí dung lượng mạng vừa làm việc truyền chậm một cách không cần thiết. Ngược lại, TCP điều chỉnh tốc độ truyền dữ liệu một cách động theo dung lượng mạng sẵn có.

There have been some attempts to build hybrid networks that support both circuit switching and packet switching, such as ATM. InfiniBand has some similarities [35]: it implements end-to-end flow control at the link layer, which reduces the need for queueing in the network, although it can still suffer from delays due to link congestion [36]. With careful use of quality of service (QoS, prioritization and scheduling of packets) and admission control (rate-limiting senders), it is possible to emulate circuit switching on packet networks, or provide statistically bounded delay [25, 32].

Đã có một số nỗ lực xây dựng các mạng lai hỗ trợ cả chuyển mạch theo mạch lẫn chuyển mạch theo gói, chẳng hạn ATM. InfiniBand có một số điểm tương đồng [35]: nó hiện thực điều khiển luồng đầu-cuối ở tầng liên kết, điều này làm giảm nhu cầu xếp hàng trong mạng, mặc dù nó vẫn có thể chịu độ trễ do tắc nghẽn liên kết [36]. Với việc sử dụng cẩn thận chất lượng dịch vụ (QoS, ưu tiên và định thời các gói) và kiểm soát kết nạp (giới hạn tốc độ của bên gửi), có thể mô phỏng chuyển mạch theo mạch trên các mạng gói, hoặc cung cấp một độ trễ có giới hạn về mặt thống kê [25, 32].

Latency and Resource Utilization — Độ trễ và mức tận dụng tài nguyên More generally, you can think of variable delays as a consequence of dynamic resource partitioning.

Tổng quát hơn, bạn có thể coi độ trễ biến động là một hệ quả của việc phân vùng tài nguyên một cách động.

Say you have a wire between two telephone switches that can carry up to 10,000 simultaneous calls. Each circuit that is switched over this wire occupies one of those call slots. Thus, you can think of the wire as a resource that can be shared by up to 10,000 simultaneous users. The resource is divided up in a static way: even if you're the only call on the wire right now, and all other 9,999 slots are unused, your circuit is still allocated the same fixed amount of bandwidth as when the wire is fully utilized.

Giả sử bạn có một sợi dây giữa hai tổng đài điện thoại có thể tải tới 10.000 cuộc gọi đồng thời. Mỗi mạch được chuyển qua sợi dây này chiếm một trong các suất gọi đó. Như vậy, bạn có thể coi sợi dây như một tài nguyên có thể được chia sẻ bởi tới 10.000 người dùng đồng thời. Tài nguyên được chia ra theo một cách tĩnh: ngay cả khi bạn là cuộc gọi duy nhất trên sợi dây lúc này, và tất cả 9.999 suất còn lại không được dùng, mạch của bạn vẫn được cấp phát đúng lượng băng thông cố định như khi sợi dây được sử dụng đầy.

By contrast, the internet shares network bandwidth dynamically. Senders push and jostle with each other to get their packets over the wire as quickly as possible, and the network switches decide which packet to send (i.e., the bandwidth allocation) from one moment to the next. This approach has the downside of queueing, but the advantage is that it maximizes utilization of the wire. The wire has a fixed cost, so if you utilize it better, each byte you send over the wire is cheaper.

Ngược lại, internet chia sẻ băng thông mạng một cách động. Các bên gửi xô đẩy và chen lấn nhau để đưa các gói của mình qua sợi dây nhanh nhất có thể, và các switch mạng quyết định gói nào được gửi (tức là việc cấp phát băng thông) từ khoảng khắc này sang khoảng khắc khác. Cách tiếp cận này có nhược điểm là việc xếp hàng, nhưng lợi thế là nó tối đa hóa mức tận dụng sợi dây. Sợi dây có một chi phí cố định, nên nếu bạn tận dụng nó tốt hơn, mỗi byte bạn gửi qua sợi dây sẽ rẻ hơn.

A similar situation arises with CPUs: if you share each CPU core dynamically between several threads, one thread sometimes has to wait in the operating system's run queue while another thread is running, so a thread can be paused for varying lengths of time. However, this utilizes the hardware better than if you allocated a static number of CPU cycles to each thread (see "Response time guarantees"). Better hardware utilization is also a significant motivation for using virtual machines.

Một tình huống tương tự nảy sinh với các CPU: nếu bạn chia sẻ mỗi nhân CPU một cách động giữa nhiều luồng, một luồng đôi khi phải chờ trong hàng đợi chạy của hệ điều hành trong khi một luồng khác đang chạy, nên một luồng có thể bị tạm dừng trong những khoảng thời gian dài ngắn khác nhau. Tuy nhiên, điều này tận dụng phần cứng tốt hơn so với khi bạn cấp phát một số chu kỳ CPU tĩnh cho mỗi luồng (xem "Bảo đảm thời gian phản hồi"). Mức tận dụng phần cứng tốt hơn cũng là một động lực đáng kể cho việc dùng máy ảo.

Latency guarantees are achievable in certain environments, if resources are statically partitioned (e.g., dedicated hardware and exclusive bandwidth allocations). However, it comes at the cost of reduced utilization—in other words, it is more expensive. On the other hand, multi-tenancy with dynamic resource partitioning provides better utilization, so it is cheaper, but it has the downside of variable delays.

Các bảo đảm về độ trễ là khả thi trong một số môi trường nhất định, nếu tài nguyên được phân vùng tĩnh (chẳng hạn phần cứng chuyên dụng và cấp phát băng thông độc quyền). Tuy nhiên, điều đó phải đánh đổi bằng mức tận dụng giảm đi — nói cách khác, nó đắt hơn. Mặt khác, mô hình đa tổ chức với việc phân vùng tài nguyên động cung cấp mức tận dụng tốt hơn, nên nó rẻ hơn, nhưng nó có nhược điểm là độ trễ biến động.

Variable delays in networks are not a law of nature, but simply the result of a cost/benefit trade-off.

Độ trễ biến động trong các mạng không phải là một quy luật của tự nhiên, mà đơn giản là kết quả của một sự đánh đổi chi phí/lợi ích.

However, such quality of service is currently not enabled in multi-tenant datacenters and public clouds, or when communicating via the internet. Currently deployed technology does not allow us to make any guarantees about delays or reliability of the network: we have to assume that network congestion, queueing, and unbounded delays will happen. Consequently, there's no “correct” value for timeouts—they need to be determined experimentally.

Tuy nhiên, loại chất lượng dịch vụ như vậy hiện không được bật trong các trung tâm dữ liệu đa tổ chức và các đám mây công cộng, hay khi giao tiếp qua internet. Công nghệ đang được triển khai hiện nay không cho phép ta đưa ra bất kỳ bảo đảm nào về độ trễ hay độ tin cậy của mạng: ta phải giả định rằng tắc nghẽn mạng, xếp hàng, và độ trễ không có giới hạn sẽ xảy ra. Do đó, không có giá trị “đúng” nào cho các thời gian chờ — chúng cần được xác định một cách thực nghiệm.

Unreliable Clocks — Đồng hồ không đáng tin cậy

Clocks and time are important. Applications depend on clocks in various ways to answer questions like the following:

Đồng hồ và thời gian là quan trọng. Các ứng dụng phụ thuộc vào đồng hồ theo nhiều cách để trả lời những câu hỏi như sau:

- Has this request timed out yet?
- What's the 99th **percentile** response time of this service?
- How many queries per second did this service handle on **average** in the last five minutes?
- How long did the user spend on our site?
- When was this article published?
- At what date and time should the reminder email be sent?
- When does this **cache** entry expire?
- What is the timestamp on this error message in the log file?
 - Yêu cầu này đã hết thời gian chờ chưa?
 - Thời gian phản hồi ở phân vị thứ 99 của dịch vụ này là bao nhiêu?
 - Trung bình dịch vụ này đã xử lý bao nhiêu truy vấn mỗi giây trong năm phút vừa qua?
 - Người dùng đã dành bao nhiêu thời gian trên trang của chúng ta?
 - Bài viết này được xuất bản khi nào?
 - Email nhắc nhở nên được gửi vào ngày giờ nào?
 - Mục bộ nhớ đệm này hết hạn khi nào?
 - Dấu thời gian trên thông báo lỗi này trong tập tin log là gì?

Examples 1–4 measure durations (e.g., the time interval between a request being sent and a response being received), whereas examples 5–8 describe points in time (events that occur on a particular date, at a particular time).

Các ví dụ 1–4 đo các khoảng thời lượng (chẳng hạn khoảng thời gian giữa lúc một yêu cầu được gửi đi và một phản hồi được nhận về), trong khi các ví dụ 5–8 mô tả các điểm thời gian (các sự kiện xảy ra vào một ngày cụ thể, tại một thời điểm cụ thể).

In a distributed system, time is a tricky business, because communication is not instantaneous: it takes time for a message to travel across the network from one machine to another. The time when a message is received is always later than the time when it is sent, but due to variable delays in the network, we don't know how much later. This fact sometimes makes it difficult to determine the order in which things happened when multiple machines are involved.

Trong một hệ thống phân tán, thời gian là một chuyện hóc búa, bởi việc giao tiếp không

diễn ra tức thời: cần thời gian để một thông điệp đi qua mạng từ một máy này tới máy khác. Thời điểm một thông điệp được nhận luôn muộn hơn thời điểm nó được gửi, nhưng do độ trễ biến động trong mạng, ta không biết muộn hơn bao nhiêu. Sự thật này đôi khi khiến ta khó xác định thứ tự các sự việc đã xảy ra khi có nhiều máy tham gia.

Moreover, each machine on the network has its own clock, which is an actual hardware device: usually a quartz crystal oscillator. These devices are not perfectly accurate, so each machine has its own notion of time, which may be slightly faster or slower than on other machines. It is possible to synchronize clocks to some degree: the most commonly used mechanism is the Network Time Protocol (NTP), which allows the computer clock to be adjusted according to the time reported by a group of servers [37]. The servers in turn get their time from a more accurate time source, such as a GPS receiver.

Hơn nữa, mỗi máy trên mạng có đồng hồ riêng của nó, vốn là một thiết bị phần cứng thực sự: thường là một bộ dao động tinh thể thạch anh. Các thiết bị này không hoàn toàn chính xác, nên mỗi máy có quan niệm riêng về thời gian, vốn có thể hơi nhanh hơn hoặc chậm hơn so với các máy khác. Có thể đồng bộ các đồng hồ ở một mức độ nào đó: cơ chế được dùng phổ biến nhất là Giao thức Thời gian Mạng (NTP), vốn cho phép đồng hồ máy tính được điều chỉnh theo thời gian do một nhóm máy chủ báo về [37]. Đến lượt mình, các máy chủ này lấy thời gian của chúng từ một nguồn thời gian chính xác hơn, chẳng hạn một bộ thu GPS.

Monotonic Versus Time-of-Day Clocks — Đồng hồ đơn điệu so với đồng hồ thời gian trong ngày

Modern computers have at least two different kinds of clocks: a time-of-day clock and a monotonic clock. Although they both measure time, it is important to distinguish the two, since they serve different purposes.

Các máy tính hiện đại có ít nhất hai loại đồng hồ khác nhau: một đồng hồ thời gian trong ngày (time-of-day clock) và một đồng hồ đơn điệu (monotonic clock). Mặc dù cả hai đều đo thời gian, việc phân biệt hai loại này là quan trọng, vì chúng phục vụ những mục đích khác nhau.

Time-of-day clocks — Đồng hồ thời gian trong ngày

A time-of-day clock does what you intuitively expect of a clock: it returns the current date and time according to some calendar (also known as wall-clock time). For example, `clock_gettime(CLOCK_REALTIME)` on Linux and `System.currentTimeMillis()` in Java return the number of seconds (or milliseconds) since the epoch: midnight UTC on January 1, 1970, according to the Gregorian calendar, not counting leap seconds. Some systems use other dates as their reference point.

Một đồng hồ thời gian trong ngày làm đúng những gì bạn theo trực giác mong đợi ở một chiếc đồng hồ: nó trả về ngày giờ hiện tại theo một loại lịch nào đó (còn gọi là thời gian theo đồng hồ treo tường). Chẳng hạn, `clock_gettime(CLOCK_REALTIME)` trên Linux và `System.currentTimeMillis()` trong Java trả về số giây (hoặc mili-giây) kể từ mốc epoch: nửa đêm UTC ngày 1 tháng 1 năm 1970, theo lịch Gregory, không tính các giây nhuận. Một số hệ thống dùng các ngày khác làm điểm tham chiếu của chúng.

Time-of-day clocks are usually synchronized with NTP, which means that a timestamp from one machine (ideally) means the same as a timestamp on another machine. However, time-of-day clocks

also have various oddities, as described in the next section. In particular, if the local clock is too far ahead of the NTP server, it may be forcibly reset and appear to jump back to a previous point in time. These jumps, as well as the fact that they often ignore leap seconds, make time-of-day clocks unsuitable for measuring elapsed time [38].

Các đồng hồ thời gian trong ngày thường được đồng bộ với NTP, điều này nghĩa là một dấu thời gian từ một máy (lý tưởng nhất) mang cùng ý nghĩa với một dấu thời gian trên một máy khác. Tuy nhiên, các đồng hồ thời gian trong ngày cũng có nhiều điều kỳ quặc, như được mô tả trong phần tiếp theo. Cụ thể, nếu đồng hồ cục bộ đi quá xa phía trước so với máy chủ NTP, nó có thể bị buộc đặt lại và có vẻ như nhảy lùi về một thời điểm trước đó. Những cú nhảy này, cũng như việc chúng thường bỏ qua các giây nhuận, khiến các đồng hồ thời gian trong ngày không phù hợp để đo thời gian đã trôi qua [38].

Time-of-day clocks have also historically had quite a coarse-grained resolution, e.g., moving forward in steps of 10 ms on older Windows systems [39]. On recent systems, this is less of a problem.

Các đồng hồ thời gian trong ngày trong lịch sử cũng có độ phân giải khá thô, chẳng hạn nhích về phía trước theo từng bước 10 ms trên các hệ thống Windows cũ [39]. Trên các hệ thống gần đây, điều này ít thành vấn đề hơn.

Monotonic clocks — Đồng hồ đơn điệu

A monotonic clock is suitable for measuring a duration (time interval), such as a timeout or a service's response time: `clock_gettime(CLOCK_MONOTONIC)` on Linux and `System.nanoTime()` in Java are monotonic clocks, for example. The name comes from the fact that they are guaranteed to always move forward (whereas a time-of-day clock may jump back in time).

Một đồng hồ đơn điệu phù hợp để đo một khoảng thời lượng (khoảng thời gian), chẳng hạn một thời gian chờ hoặc thời gian phản hồi của một dịch vụ: ví dụ `clock_gettime(CLOCK_MONOTONIC)` trên Linux và `System.nanoTime()` trong Java là các đồng hồ đơn điệu. Cái tên này xuất phát từ việc chúng được bảo đảm luôn tiến về phía trước (trong khi một đồng hồ thời gian trong ngày có thể nhảy lùi về thời gian trước).

You can check the value of the monotonic clock at one point in time, do something, and then check the clock again at a later time. The difference between the two values tells you how much time elapsed between the two checks. However, the absolute value of the clock is meaningless: it might be the number of nanoseconds since the computer was started, or something similarly arbitrary. In particular, it makes no sense to compare monotonic clock values from two different computers, because they don't mean the same thing.

Bạn có thể kiểm tra giá trị của đồng hồ đơn điệu tại một thời điểm, làm một việc gì đó, rồi kiểm tra lại đồng hồ tại một thời điểm sau. Hiệu giữa hai giá trị cho bạn biết bao nhiêu thời gian đã trôi qua giữa hai lần kiểm tra. Tuy nhiên, giá trị tuyệt đối của đồng hồ thì vô nghĩa: nó có thể là số nano-giây kể từ khi máy tính được khởi động, hoặc một thứ tùy tiện tương tự. Cụ thể, việc so sánh các giá trị đồng hồ đơn điệu từ hai máy tính khác nhau là vô nghĩa, bởi chúng không mang cùng ý nghĩa.

On a server with multiple CPU sockets, there may be a separate timer per CPU, which is not necessarily synchronized with other CPUs. Operating systems compensate for any discrepancy and try to present a monotonic view of the clock to application threads, even as they are scheduled across different CPUs. However, it is wise to take this guarantee of monotonicity with a pinch of salt [40].

Trên một máy chủ có nhiều socket CPU, có thể có một bộ định thời riêng cho mỗi CPU, vốn

không nhất thiết được đồng bộ với các CPU khác. Các hệ điều hành bù trừ cho bất kỳ chênh lệch nào và cố trình ra một góc nhìn đơn điệu về đồng hồ cho các luồng ứng dụng, ngay cả khi chúng được định thời chạy trên các CPU khác nhau. Tuy nhiên, việc dè dặt với bảo đảm về tính đơn điệu này là khôn ngoan [40].

NTP may adjust the frequency at which the monotonic clock moves forward (this is known as slewing the clock) if it detects that the computer's local quartz is moving faster or slower than the NTP server. By default, NTP allows the clock rate to be speeded up or slowed down by up to 0.05%, but NTP cannot cause the monotonic clock to jump forward or backward. The resolution of monotonic clocks is usually quite good: on most systems they can measure time intervals in microseconds or less.

NTP có thể điều chỉnh tần suất mà đồng hồ đơn điệu tiến về phía trước (điều này được gọi là làm lệch tốc độ — slewing — đồng hồ) nếu nó phát hiện rằng tinh thể thạch anh cục bộ của máy tính đang chạy nhanh hơn hoặc chậm hơn máy chủ NTP. Theo mặc định, NTP cho phép tốc độ đồng hồ được tăng tốc hoặc làm chậm tới mức 0,05%, nhưng NTP không thể khiến đồng hồ đơn điệu nhảy tới hoặc lùi. Độ phân giải của các đồng hồ đơn điệu thường khá tốt: trên hầu hết các hệ thống, chúng có thể đo các khoảng thời gian ở mức micro-giây hoặc nhỏ hơn.

In a distributed system, using a monotonic clock for measuring elapsed time (e.g., timeouts) is usually fine, because it doesn't assume any synchronization between different nodes' clocks and is not sensitive to slight inaccuracies of measurement.

Trong một hệ thống phân tán, việc dùng một đồng hồ đơn điệu để đo thời gian đã trôi qua (chẳng hạn các thời gian chờ) thường ổn, bởi nó không giả định bất kỳ sự đồng bộ nào giữa các đồng hồ của những nút khác nhau và không nhạy với những sai số nhỏ của phép đo.

Clock Synchronization and Accuracy — Đồng bộ đồng hồ và độ chính xác

Monotonic clocks don't need synchronization, but time-of-day clocks need to be set according to an NTP server or other external time source in order to be useful. Unfortunately, our methods for getting a clock to tell the correct time aren't nearly as reliable or accurate as you might hope—hardware clocks and NTP can be fickle beasts. To give just a few examples:

Các đồng hồ đơn điệu không cần đồng bộ, nhưng các đồng hồ thời gian trong ngày cần được đặt theo một máy chủ NTP hoặc nguồn thời gian bên ngoài khác thì mới hữu ích. Thật không may, các phương pháp của ta để làm cho một đồng hồ báo đúng giờ lại không hề đáng tin cậy hay chính xác như bạn có thể hy vọng — đồng hồ phần cứng và NTP có thể là những con thú đồng đánh. Chỉ nêu vài ví dụ:

- The quartz clock in a computer is not very accurate: it drifts (runs faster or slower than it should). Clock drift varies depending on the temperature of the machine. Google assumes a clock drift of 200 ppm (parts per million) for its servers [41], which is equivalent to 6 ms drift for a clock that is resynchronized with a server every 30 seconds, or 17 seconds drift for a clock that is resynchronized once a day. This drift limits the best possible accuracy you can achieve, even if everything is working correctly.
- If a computer's clock differs too much from an NTP server, it may refuse to synchronize, or the local clock will be forcibly reset [37]. Any applications observing the time before and after this reset may see time go backward or suddenly jump forward.
- If a node is accidentally firewalled off from NTP servers, the misconfiguration may go unnoticed for some time. Anecdotal evidence suggests that this does happen in practice.

- NTP synchronization can only be as good as the network delay, so there is a limit to its accuracy when you're on a congested network with variable packet delays. One experiment showed that a minimum error of 35 ms is achievable when synchronizing over the internet [42], though occasional spikes in network delay lead to errors of around a second. Depending on the configuration, large network delays can cause the NTP **client** to give up entirely.
- Some NTP servers are wrong or misconfigured, reporting time that is off by hours [43, 44]. NTP clients are quite robust, because they query several servers and ignore outliers. Nevertheless, it's somewhat worrying to bet the correctness of your systems on the time that you were told by a stranger on the internet.
- Leap seconds result in a minute that is 59 seconds or 61 seconds long, which messes up timing assumptions in systems that are not designed with leap seconds in mind [45]. The fact that leap seconds have crashed many large systems [38, 46] shows how easy it is for incorrect assumptions about clocks to sneak into a system. The best way of handling leap seconds may be to make NTP servers "lie," by performing the **leap second** adjustment gradually over the course of a day (this is known as smearing) [47, 48], although actual NTP server behavior varies in practice [49].
- In virtual machines, the hardware clock is virtualized, which raises additional challenges for applications that need accurate timekeeping [50]. When a CPU core is shared between virtual machines, each VM is paused for tens of milliseconds while another VM is running. From an application's point of view, this pause manifests itself as the clock suddenly jumping forward [26].
- If you run software on devices that you don't fully control (e.g., mobile or embedded devices), you probably cannot trust the device's hardware clock at all. Some users deliberately set their hardware clock to an incorrect date and time, for example to circumvent timing limitations in games. As a result, the clock might be set to a time wildly in the past or the future.
 - Đồng hồ thạch anh trong một máy tính không chính xác lắm: nó trôi (chạy nhanh hơn hoặc chậm hơn mức đáng lẽ phải có). Độ trôi đồng hồ thay đổi tùy theo nhiệt độ của máy. Google giả định độ trôi đồng hồ 200 ppm (phần triệu) cho các máy chủ của họ [41], tương đương 6 ms trôi đối với một đồng hồ được tái đồng bộ với một máy chủ sau mỗi 30 giây, hoặc 17 giây trôi đối với một đồng hồ được tái đồng bộ một lần mỗi ngày. Độ trôi này giới hạn độ chính xác tốt nhất có thể đạt được, ngay cả khi mọi thứ đang hoạt động đúng.
 - Nếu đồng hồ của một máy tính chênh quá nhiều so với một máy chủ NTP, nó có thể từ chối đồng bộ, hoặc đồng hồ cục bộ sẽ bị buộc đặt lại [37]. Bất kỳ ứng dụng nào quan sát thời gian trước và sau lần đặt lại này có thể thấy thời gian đi lùi hoặc đột ngột nhảy tới phía trước.
 - Nếu một nút vô tình bị tường lửa chặn khỏi các máy chủ NTP, việc cấu hình sai có thể không bị phát hiện trong một thời gian. Bằng chứng giai thoại gợi ý rằng điều này có xảy ra trong thực tế.
 - Việc đồng bộ NTP chỉ có thể tốt ngang với độ trễ mạng, nên có một giới hạn cho độ chính xác của nó khi bạn ở trên một mạng tắc nghẽn với độ trễ gói biến động. Một thí nghiệm cho thấy có thể đạt sai số tối thiểu 35 ms khi đồng bộ qua internet [42], dù những đợt tăng đột biến thỉnh thoảng trong độ trễ mạng dẫn tới sai số khoảng một giây. Tùy vào cấu hình, độ trễ mạng lớn có thể khiến client NTP bỏ cuộc hoàn toàn.
 - Một số máy chủ NTP bị sai hoặc cấu hình sai, báo về thời gian lệch hàng giờ [43, 44]. Các client NTP khá kiên cường, bởi chúng truy vấn nhiều máy chủ và bỏ qua các giá trị ngoại lai. Tuy vậy, việc đặt cược tính đúng đắn của các hệ thống của bạn vào thời gian mà một người lạ trên internet báo cho bạn là điều có phần đáng lo.
 - Các giây nhuận dẫn tới một phút dài 59 giây hoặc 61 giây, điều này làm rối các giả định về định thời trong các hệ thống không được thiết kế có tính tới giây nhuận [45].

Việc các giây nhuận đã làm sập nhiều hệ thống lớn [38, 46] cho thấy thật dễ dàng làm sao để những giả định sai về đồng hồ lén lút len vào một hệ thống. Cách tốt nhất để xử lý các giây nhuận có lẽ là khiến các máy chủ NTP “nói dối”, bằng cách thực hiện việc điều chỉnh giây nhuận một cách dần dần trong suốt một ngày (điều này được gọi là làm nhòe — smearing) [47, 48], mặc dù hành vi thực tế của máy chủ NTP có thay đổi trong thực tế [49].

- Trong các máy ảo, đồng hồ phần cứng được ảo hóa, điều này đặt ra những thách thức thêm cho các ứng dụng cần đo thời gian chính xác [50]. Khi một nhân CPU được chia sẻ giữa các máy ảo, mỗi máy ảo bị tạm dừng trong vài chục mili-giây trong khi một máy ảo khác đang chạy. Từ góc nhìn của một ứng dụng, sự tạm dừng này biểu hiện thành việc đồng hồ đột ngột nhảy tới phía trước [26].
- Nếu bạn chạy phần mềm trên các thiết bị mà bạn không hoàn toàn kiểm soát (chẳng hạn các thiết bị di động hoặc nhúng), thì có lẽ bạn không thể tin tưởng đồng hồ phần cứng của thiết bị chút nào. Một số người dùng cố ý đặt đồng hồ phần cứng của họ thành một ngày giờ sai, chẳng hạn để lách các giới hạn về thời gian trong trò chơi. Kết quả là đồng hồ có thể được đặt thành một thời điểm xa lắc trong quá khứ hoặc tương lai.

It is possible to achieve very good clock accuracy if you care about it sufficiently to invest significant resources. For example, the MiFID II draft European regulation for financial institutions requires all high-frequency trading funds to synchronize their clocks to within 100 microseconds of UTC, in order to help debug market anomalies such as “flash crashes” and to help detect market manipulation [51].

Có thể đạt được độ chính xác đồng hồ rất tốt nếu bạn quan tâm tới nó đủ nhiều để đầu tư nguồn lực đáng kể. Chẳng hạn, dự thảo quy định châu Âu MiFID II cho các định chế tài chính yêu cầu tất cả các quỹ giao dịch tần suất cao đồng bộ đồng hồ của họ trong vòng 100 micro-giây so với UTC, nhằm giúp gỡ lỗi các bất thường thị trường như các “đợt sụp chớp nhoáng” và giúp phát hiện thao túng thị trường [51].

Such accuracy can be achieved using GPS receivers, the Precision Time Protocol (PTP) [52], and careful deployment and monitoring. However, it requires significant effort and expertise, and there are plenty of ways clock synchronization can go wrong. If your NTP daemon is misconfigured, or a firewall is blocking NTP traffic, the clock error due to drift can quickly become large.

Độ chính xác như vậy có thể đạt được bằng các bộ thu GPS, Giao thức Thời gian Chính xác (PTP) [52], và việc triển khai cùng giám sát cẩn thận. Tuy nhiên, nó đòi hỏi nỗ lực và chuyên môn đáng kể, và có vô số cách mà việc đồng bộ đồng hồ có thể trục trặc. Nếu daemon NTP của bạn bị cấu hình sai, hoặc một tường lửa đang chặn lưu lượng NTP, thì sai số đồng hồ do trôi có thể nhanh chóng trở nên lớn.

Relying on Synchronized Clocks — Trông cậy vào các đồng hồ đã đồng bộ

The problem with clocks is that while they seem simple and easy to use, they have a surprising number of pitfalls: a day may not have exactly 86,400 seconds, time-of-day clocks may move backward in time, and the time on one node may be quite different from the time on another node.

Vấn đề với đồng hồ là dù chúng có vẻ đơn giản và dễ dùng, chúng lại có một số lượng đáng ngạc nhiên các cạm bẫy: một ngày có thể không có đúng 86.400 giây, các đồng hồ thời gian trong ngày có thể đi lùi về thời gian trước, và thời gian trên một nút có thể khác khá nhiều so với thời gian trên một nút khác.

Earlier in this chapter we discussed networks dropping and arbitrarily delaying packets. Even though networks are well behaved most of the time, software must be designed on the assumption that the network will occasionally be faulty, and the software must handle such faults gracefully. The same is true with clocks: although they work quite well most of the time, robust software needs to be prepared to deal with incorrect clocks.

Trước đó trong chương này, ta đã bàn về việc các mạng làm rớt và làm trễ các gói một cách tùy tiện. Mặc dù các mạng hành xử tốt trong phần lớn thời gian, phần mềm phải được thiết kế dựa trên giả định rằng mạng thỉnh thoảng sẽ lỗi, và phần mềm phải xử lý những lỗi như vậy một cách uyển chuyển. Điều tương tự cũng đúng với đồng hồ: mặc dù chúng hoạt động khá tốt trong phần lớn thời gian, phần mềm kiên cường cần sẵn sàng đối phó với các đồng hồ sai.

Part of the problem is that incorrect clocks easily go unnoticed. If a machine's CPU is defective or its network is misconfigured, it most likely won't work at all, so it will quickly be noticed and fixed. On the other hand, if its quartz clock is defective or its NTP client is misconfigured, most things will seem to work fine, even though its clock gradually drifts further and further away from reality. If some piece of software is relying on an accurately synchronized clock, the result is more likely to be silent and subtle data loss than a dramatic crash [53, 54].

Một phần của vấn đề là các đồng hồ sai dễ dàng không bị để ý. Nếu CPU của một máy bị lỗi hoặc mạng của nó bị cấu hình sai, nhiều khả năng nó sẽ không hoạt động chút nào, nên nó sẽ nhanh chóng bị phát hiện và sửa. Mặt khác, nếu đồng hồ thạch anh của nó bị lỗi hoặc client NTP của nó bị cấu hình sai, hầu hết mọi thứ sẽ có vẻ hoạt động ổn, ngay cả khi đồng hồ của nó dần dần trôi ngày càng xa khỏi thực tế. Nếu một phần mềm nào đó đang trông cậy vào một đồng hồ được đồng bộ chính xác, thì kết quả nhiều khả năng là sự mất dữ liệu âm thầm và tinh vi hơn là một cú sập ngoạn mục [53, 54].

Thus, if you use software that requires synchronized clocks, it is essential that you also carefully monitor the clock offsets between all the machines. Any node whose clock drifts too far from the others should be declared dead and removed from the cluster. Such monitoring ensures that you notice the broken clocks before they can cause too much damage.

Như vậy, nếu bạn dùng phần mềm đòi hỏi các đồng hồ được đồng bộ, thì điều thiết yếu là bạn cũng giám sát cẩn thận độ chênh đồng hồ giữa tất cả các máy. Bất kỳ nút nào có đồng hồ trôi quá xa khỏi các nút khác cần được tuyên bố đã chết và loại bỏ khỏi cụm. Việc giám sát như vậy bảo đảm rằng bạn để ý các đồng hồ hỏng trước khi chúng có thể gây ra quá nhiều thiệt hại.

Timestamps for ordering events — Dấu thời gian để sắp xếp thứ tự các sự kiện

Let's consider one particular situation in which it is tempting, but dangerous, to rely on clocks: ordering of events across multiple nodes. For example, if two clients write to a distributed database, who got there first? Which write is the more recent one?

Hãy xét một tình huống cụ thể trong đó việc trông cậy vào đồng hồ là hấp dẫn, nhưng nguy hiểm: sắp xếp thứ tự các sự kiện qua nhiều nút. Chẳng hạn, nếu hai client ghi vào một cơ sở dữ liệu phân tán, ai tới trước? Lần ghi nào là gần đây hơn?

Figure 8-3 illustrates a dangerous use of time-of-day clocks in a database with multi-leader replication (the example is similar to Figure 5-9). Client A writes $x = 1$ on node 1; the write is replicated to node 3; client B increments x on node 3 (we now have $x = 2$); and finally, both writes are replicated to node 2.

Hình 8-3 minh họa một cách dùng đồng hồ thời gian trong ngày nguy hiểm trong một cơ sở dữ liệu dùng sao chép nhiều-người-dẫn-đầu (ví dụ này tương tự với Hình 5-9). Client A ghi $x = 1$ trên nút 1; lần ghi được sao chép sang nút 3; client B tăng x trên nút 3 (giờ ta có $x = 2$); và cuối cùng, cả hai lần ghi được sao chép sang nút 2.

Figure 8-3. The write by client B is causally later than the write by client A, but B's write has an earlier timestamp. — Hình 8-3. Lần ghi của client B muộn hơn về mặt nhân quả so với lần ghi của client A, nhưng lần ghi của B lại có dấu thời gian sớm hơn. In Figure 8-3, when a write is replicated to other nodes, it is tagged with a timestamp according to the time-of-day clock on the node where the write originated. The clock synchronization is very good in this example: the skew between node 1 and node 3 is less than 3 ms, which is probably better than you can expect in practice.

Trong Hình 8-3, khi một lần ghi được sao chép sang các nút khác, nó được gắn một dấu thời gian theo đồng hồ thời gian trong ngày trên nút nơi lần ghi bắt nguồn. Việc đồng bộ đồng hồ trong ví dụ này rất tốt: độ chênh giữa nút 1 và nút 3 là dưới 3 ms, vốn có lẽ tốt hơn mức bạn có thể mong đợi trong thực tế.

Nevertheless, the timestamps in Figure 8-3 fail to order the events correctly: the write $x = 1$ has a timestamp of 42.004 seconds, but the write $x = 2$ has a timestamp of 42.003 seconds, even though $x = 2$ occurred unambiguously later. When node 2 receives these two events, it will incorrectly conclude that $x = 1$ is the more recent value and drop the write $x = 2$. In effect, client B's increment operation will be lost.

Tuy vậy, các dấu thời gian trong Hình 8-3 không sắp xếp đúng thứ tự các sự kiện: lần ghi $x = 1$ có dấu thời gian 42,004 giây, nhưng lần ghi $x = 2$ có dấu thời gian 42,003 giây, mặc dù $x = 2$ rõ ràng xảy ra muộn hơn. Khi nút 2 nhận được hai sự kiện này, nó sẽ kết luận sai rằng $x = 1$ là giá trị gần đây hơn và làm rớt lần ghi $x = 2$. Trên thực tế, thao tác tăng của client B sẽ bị mất.

This conflict resolution strategy is called last write wins (LWW), and it is widely used in both multi-leader replication and leaderless databases such as Cassandra [53] and Riak [54] (see “Last write wins (discarding concurrent writes)”). Some implementations generate timestamps on the client rather than the server, but this doesn't change the fundamental problems with LWW:

Chiến lược giải quyết xung đột này được gọi là lần ghi cuối thắng (last write wins, LWW), và nó được dùng rộng rãi cả trong sao chép nhiều-người-dẫn-đầu lẫn trong các cơ sở dữ liệu không-người-dẫn-đầu như Cassandra [53] và Riak [54] (xem “Lần ghi cuối thắng (loại bỏ các lần ghi tương tranh)”). Một số hiện thực sinh dấu thời gian trên client thay vì trên máy chủ, nhưng điều này không làm thay đổi các vấn đề căn bản của LWW:

- Database writes can mysteriously disappear: a node with a lagging clock is unable to overwrite values previously written by a node with a fast clock until the clock skew between the nodes has elapsed [54, 55]. This scenario can cause arbitrary amounts of data to be silently dropped without any error being reported to the application.
- LWW cannot distinguish between writes that occurred sequentially in quick succession (in Figure 8-3, client B's increment definitely occurs after client A's write) and writes that were truly concurrent (neither writer was aware of the other). Additional causality tracking mechanisms, such as version vectors, are needed in order to prevent violations of causality (see “Detecting Concurrent Writes”).
- It is possible for two nodes to independently generate writes with the same timestamp, especially when the clock only has millisecond resolution. An additional tiebreaker value

(which can simply be a large random number) is required to resolve such conflicts, but this approach can also lead to violations of causality [53].

- Các lần ghi cơ sở dữ liệu có thể biến mất một cách bí ẩn: một nút có đồng hồ chậm không thể ghi đè các giá trị đã được ghi trước đó bởi một nút có đồng hồ nhanh cho tới khi độ chênh đồng hồ giữa các nút trôi qua [54, 55]. Tình huống này có thể khiến những lượng dữ liệu tùy tiện bị âm thầm làm rớt mà không có bất kỳ lỗi nào được báo về cho ứng dụng.
- LWW không thể phân biệt giữa các lần ghi xảy ra tuần tự liên tiếp nhanh chóng (trong Hình 8-3, thao tác tăng của client B chắc chắn xảy ra sau lần ghi của client A) và các lần ghi thực sự tương tranh (không bên ghi nào biết về bên kia). Cần thêm các cơ chế theo dõi nhân quả, chẳng hạn các vector phiên bản, để ngăn các vi phạm về nhân quả (xem “Phát hiện các lần ghi tương tranh”).
- Có thể xảy ra việc hai nút độc lập sinh ra các lần ghi với cùng một dấu thời gian, đặc biệt khi đồng hồ chỉ có độ phân giải mili-giây. Cần thêm một giá trị phân định (tiebreaker) (có thể đơn giản là một số ngẫu nhiên lớn) để giải quyết những xung đột như vậy, nhưng cách này cũng có thể dẫn tới các vi phạm về nhân quả [53].

Thus, even though it is tempting to resolve conflicts by keeping the most “recent” value and discarding others, it’s important to be aware that the definition of “recent” depends on a local time-of-day clock, which may well be incorrect. Even with tightly NTP-synchronized clocks, you could send a packet at timestamp 100 ms (according to the sender’s clock) and have it arrive at timestamp 99 ms (according to the recipient’s clock)—so it appears as though the packet arrived before it was sent, which is impossible.

Như vậy, mặc dù việc giải quyết xung đột bằng cách giữ lại giá trị “gần đây” nhất và loại bỏ các giá trị khác là hấp dẫn, điều quan trọng là phải ý thức rằng định nghĩa của “gần đây” phụ thuộc vào một đồng hồ thời gian trong ngày cục bộ, vốn rất có thể sai. Ngay cả với các đồng hồ được đồng bộ NTP chặt chẽ, bạn có thể gửi một gói tại dấu thời gian 100 ms (theo đồng hồ của bên gửi) và để nó tới tại dấu thời gian 99 ms (theo đồng hồ của bên nhận) — nên có vẻ như gói đã tới trước khi nó được gửi đi, điều này là bất khả thi.

Could NTP synchronization be made accurate enough that such incorrect orderings cannot occur? Probably not, because NTP’s synchronization accuracy is itself limited by the network round-trip time, in addition to other sources of error such as quartz drift. For correct ordering, you would need the clock source to be significantly more accurate than the thing you are measuring (namely network delay).

Liệu việc đồng bộ NTP có thể được làm cho đủ chính xác để những thứ tự sai như vậy không thể xảy ra hay không? Có lẽ không, bởi bản thân độ chính xác đồng bộ của NTP bị giới hạn bởi thời gian khứ hồi của mạng, cộng thêm các nguồn sai số khác như độ trôi thạch anh. Để sắp xếp đúng thứ tự, bạn sẽ cần nguồn đồng hồ chính xác hơn đáng kể so với thứ bạn đang đo (cụ thể là độ trễ mạng).

So-called logical clocks [56, 57], which are based on incrementing counters rather than an oscillating quartz crystal, are a safer alternative for ordering events (see “Detecting Concurrent Writes”). Logical clocks do not measure the time of day or the number of seconds elapsed, only the relative ordering of events (whether one event happened before or after another). In contrast, time-of-day and monotonic clocks, which measure actual elapsed time, are also known as physical clocks. We’ll look at ordering a bit more in “Ordering Guarantees”.

Cái gọi là đồng hồ logic [56, 57], vốn dựa trên các bộ đếm tăng dần thay vì một tinh thể thạch anh dao động, là một giải pháp thay thế an toàn hơn để sắp xếp thứ tự các sự kiện (xem “Phát hiện các lần ghi tương tranh”). Các đồng hồ logic không đo thời gian trong ngày

hay số giây đã trôi qua, mà chỉ đo thứ tự tương đối của các sự kiện (liệu một sự kiện xảy ra trước hay sau một sự kiện khác). Ngược lại, các đồng hồ thời gian trong ngày và đồng hồ đơn điệu, vốn đo thời gian thực sự đã trôi qua, còn được gọi là các đồng hồ vật lý. Ta sẽ xem xét việc sắp xếp thứ tự kỹ hơn một chút ở phần “Các bảo đảm về thứ tự”.

Clock readings have a confidence interval — Số đọc đồng hồ có một khoảng tin cậy

You may be able to read a machine’s time-of-day clock with microsecond or even nanosecond resolution. But even if you can get such a fine-grained measurement, that doesn’t mean the value is actually accurate to such precision. In fact, it most likely is not—as mentioned previously, the drift in an imprecise quartz clock can easily be several milliseconds, even if you synchronize with an NTP server on the local network every minute. With an NTP server on the public internet, the best possible accuracy is probably to the tens of milliseconds, and the error may easily spike to over 100 ms when there is network congestion [57].

Bạn có thể đọc được đồng hồ thời gian trong ngày của một máy với độ phân giải micro-giây hoặc thậm chí nano-giây. Nhưng ngay cả khi bạn có được một phép đo tinh tế đến vậy, điều đó không có nghĩa là giá trị đó thực sự chính xác tới mức ấy. Trên thực tế, nhiều khả năng là không — như đã nêu trước đó, độ trôi trong một đồng hồ thạch anh thiếu chính xác có thể dễ dàng lên tới vài mili-giây, ngay cả khi bạn đồng bộ với một máy chủ NTP trên mạng cục bộ mỗi phút. Với một máy chủ NTP trên internet công cộng, độ chính xác tốt nhất có thể có lẽ ở mức vài chục mili-giây, và sai số có thể dễ dàng tăng vọt lên trên 100 ms khi có tắc nghẽn mạng [57].

Thus, it doesn’t make sense to think of a clock reading as a point in time—it is more like a range of times, within a confidence interval: for example, a system may be 95% confident that the time now is between 10.3 and 10.5 seconds past the minute, but it doesn’t know any more precisely than that [58]. If we only know the time ± 100 ms, the microsecond digits in the timestamp are essentially meaningless.

Như vậy, việc coi một số đọc đồng hồ như một điểm thời gian là vô nghĩa — nó giống một dải các thời gian hơn, nằm trong một khoảng tin cậy: chẳng hạn, một hệ thống có thể tin cậy 95% rằng thời gian lúc này nằm giữa 10,3 và 10,5 giây sau phút, nhưng nó không biết chính xác hơn thế [58]. Nếu ta chỉ biết thời gian với sai số ± 100 ms, thì các chữ số micro-giây trong dấu thời gian về cơ bản là vô nghĩa.

The uncertainty bound can be calculated based on your time source. If you have a GPS receiver or atomic (caesium) clock directly attached to your computer, the expected error range is reported by the manufacturer. If you’re getting the time from a server, the uncertainty is based on the expected quartz drift since your last sync with the server, plus the NTP server’s uncertainty, plus the network round-trip time to the server (to a first approximation, and assuming you trust the server).

Cận trên của sự bất định có thể được tính toán dựa trên nguồn thời gian của bạn. Nếu bạn có một bộ thu GPS hoặc một đồng hồ nguyên tử (cesium) gắn trực tiếp vào máy tính, thì dải sai số kỳ vọng được nhà sản xuất báo cáo. Nếu bạn lấy thời gian từ một máy chủ, thì sự bất định dựa trên độ trôi thạch anh kỳ vọng kể từ lần đồng bộ cuối với máy chủ, cộng với sự bất định của máy chủ NTP, cộng với thời gian khứ hồi mạng tới máy chủ (theo phép gần đúng bậc một, và giả định rằng bạn tin tưởng máy chủ).

Unfortunately, most systems don’t expose this uncertainty: for example, when you call `clock_gettime()`, the return value doesn’t tell you the expected error of the timestamp, so you don’t know if its confidence interval is five milliseconds or five years.

Thật không may, hầu hết các hệ thống không phơi bày sự bất định này: chẳng hạn, khi bạn gọi `clock_gettime()`, giá trị trả về không cho bạn biết sai số kỳ vọng của dấu thời gian, nên bạn không biết khoảng tin cậy của nó là năm mili-giây hay năm năm.

An interesting exception is Google’s TrueTime **API** in Spanner [41], which explicitly reports the confidence interval on the local clock. When you ask it for the current time, you get back two values: [earliest, latest], which are the earliest possible and the latest possible timestamp. Based on its uncertainty calculations, the clock knows that the actual current time is somewhere within that interval. The width of the interval depends, among other things, on how long it has been since the local quartz clock was last synchronized with a more accurate clock source.

Một ngoại lệ thú vị là API TrueTime của Google trong Spanner [41], vốn báo cáo tường minh khoảng tin cậy trên đồng hồ cục bộ. Khi bạn hỏi nó thời gian hiện tại, bạn nhận lại hai giá trị: [earliest, latest], là dấu thời gian sớm nhất có thể và muộn nhất có thể. Dựa trên các tính toán về sự bất định của nó, đồng hồ biết rằng thời gian hiện tại thực sự nằm đâu đó trong khoảng đó. Bề rộng của khoảng phụ thuộc, trong số những thứ khác, vào việc đã bao lâu kể từ khi đồng hồ thạch anh cục bộ được đồng bộ lần cuối với một nguồn đồng hồ chính xác hơn.

Synchronized clocks for global snapshots — Đồng hồ đã đồng bộ cho các ảnh chụp nhanh toàn cục

In “Snapshot Isolation and Repeatable Read” we discussed snapshot isolation, which is a very useful feature in databases that need to support both small, fast read-write transactions and large, long-running read-only transactions (e.g., for backups or analytics). It allows read-only transactions to see the database in a consistent state at a particular point in time, without locking and interfering with read-write transactions.

Ở phần “Cô lập ảnh chụp nhanh và đọc lặp lại được” ta đã bàn về cô lập ảnh chụp nhanh, vốn là một tính năng rất hữu ích trong các cơ sở dữ liệu cần hỗ trợ cả các giao dịch đọc-ghi nhỏ, nhanh lẫn các giao dịch chỉ-đọc lớn, chạy lâu (chẳng hạn để sao lưu hoặc phân tích). Nó cho phép các giao dịch chỉ-đọc thấy cơ sở dữ liệu ở một trạng thái nhất quán tại một điểm thời gian cụ thể, mà không khóa và can thiệp vào các giao dịch đọc-ghi.

The most common implementation of snapshot isolation requires a monotonically increasing transaction ID. If a write happened later than the snapshot (i.e., the write has a greater transaction ID than the snapshot), that write is invisible to the snapshot transaction. On a single-node database, a simple counter is sufficient for generating transaction IDs.

Cách hiện thực phổ biến nhất của cô lập ảnh chụp nhanh đòi hỏi một ID giao dịch tăng đơn điệu. Nếu một lần ghi xảy ra muộn hơn ảnh chụp nhanh (tức là lần ghi có ID giao dịch lớn hơn ảnh chụp nhanh), thì lần ghi đó là vô hình đối với giao dịch ảnh chụp nhanh. Trên một cơ sở dữ liệu đơn nút, một bộ đếm đơn giản là đủ để sinh các ID giao dịch.

However, when a database is distributed across many machines, potentially in multiple datacenters, a global, monotonically increasing transaction ID (across all partitions) is difficult to generate, because it requires coordination. The transaction ID must reflect causality: if transaction B reads a value that was written by transaction A, then B must have a higher transaction ID than A—otherwise, the snapshot would not be consistent. With lots of small, rapid transactions, creating transaction IDs in a distributed system becomes an untenable bottleneck.

Tuy nhiên, khi một cơ sở dữ liệu được phân tán trên nhiều máy, có thể ở nhiều trung tâm dữ liệu, một ID giao dịch tăng đơn điệu, toàn cục (trên tất cả các phân vùng) là khó sinh ra, bởi nó đòi hỏi sự phối hợp. ID giao dịch phải phản ánh nhân quả: nếu giao dịch B đọc

một giá trị được ghi bởi giao dịch A, thì B phải có một ID giao dịch cao hơn A — nếu không, ảnh chụp nhanh sẽ không nhất quán. Với rất nhiều giao dịch nhỏ, nhanh, việc tạo các ID giao dịch trong một hệ thống phân tán trở thành một nút thắt cổ chai không thể chấp nhận được.

Can we use the timestamps from synchronized time-of-day clocks as transaction IDs? If we could get the synchronization good enough, they would have the right properties: later transactions have a higher timestamp. The problem, of course, is the uncertainty about clock accuracy.

Liệu ta có thể dùng các dấu thời gian từ các đồng hồ thời gian trong ngày đã đồng bộ làm ID giao dịch hay không? Nếu ta có thể làm cho việc đồng bộ đủ tốt, thì chúng sẽ có các thuộc tính đúng đắn: các giao dịch muộn hơn có một dấu thời gian cao hơn. Vấn đề, dĩ nhiên, là sự bất định về độ chính xác của đồng hồ.

Spanner implements snapshot isolation across datacenters in this way [59, 60]. It uses the clock's confidence interval as reported by the TrueTime API, and is based on the following observation: if you have two confidence intervals, each consisting of an earliest and latest possible timestamp ($A = [A_{\text{earliest}}, A_{\text{latest}}]$ and $B = [B_{\text{earliest}}, B_{\text{latest}}]$), and those two intervals do not overlap (i.e., $A_{\text{earliest}} < A_{\text{latest}} < B_{\text{earliest}} < B_{\text{latest}}$), then B definitely happened after A—there can be no doubt. Only if the intervals overlap are we unsure in which order A and B happened.

Spanner hiện thực cô lập ảnh chụp nhanh qua các trung tâm dữ liệu theo cách này [59, 60]. Nó dùng khoảng tin cậy của đồng hồ như được API TrueTime báo cáo, và dựa trên quan sát sau: nếu bạn có hai khoảng tin cậy, mỗi khoảng gồm một dấu thời gian sớm nhất có thể và muộn nhất có thể ($A = [A_{\text{sớm nhất}}, A_{\text{muộn nhất}}]$ và $B = [B_{\text{sớm nhất}}, B_{\text{muộn nhất}}]$), và hai khoảng đó không chồng lấn (tức là $A_{\text{sớm nhất}} < A_{\text{muộn nhất}} < B_{\text{sớm nhất}} < B_{\text{muộn nhất}}$), thì B chắc chắn xảy ra sau A — không thể nghi ngờ gì. Chỉ khi các khoảng chồng lấn thì ta mới không chắc A và B xảy ra theo thứ tự nào.

In order to ensure that transaction timestamps reflect causality, Spanner deliberately waits for the length of the confidence interval before committing a read-write transaction. By doing so, it ensures that any transaction that may read the data is at a sufficiently later time, so their confidence intervals do not overlap. In order to keep the wait time as short as possible, Spanner needs to keep the clock uncertainty as small as possible; for this purpose, Google deploys a GPS receiver or atomic clock in each datacenter, allowing clocks to be synchronized to within about 7 ms [41].

Để bảo đảm rằng các dấu thời gian giao dịch phản ánh nhân quả, Spanner cố tình chờ trong khoảng thời gian bằng độ dài của khoảng tin cậy trước khi commit một giao dịch đọc-ghi. Bằng cách này, nó bảo đảm rằng bất kỳ giao dịch nào có thể đọc dữ liệu đều ở một thời điểm muộn hơn đủ nhiều, sao cho các khoảng tin cậy của chúng không chồng lấn. Để giữ thời gian chờ ngắn nhất có thể, Spanner cần giữ sự bất định đồng hồ nhỏ nhất có thể; vì mục đích này, Google triển khai một bộ thu GPS hoặc một đồng hồ nguyên tử trong mỗi trung tâm dữ liệu, cho phép các đồng hồ được đồng bộ trong vòng khoảng 7 ms [41].

Using clock synchronization for distributed transaction semantics is an area of active research [57, 61, 62]. These ideas are interesting, but they have not yet been implemented in mainstream databases outside of Google.

Việc dùng đồng bộ đồng hồ cho ngữ nghĩa giao dịch phân tán là một lĩnh vực đang được nghiên cứu tích cực [57, 61, 62]. Những ý tưởng này thú vị, nhưng chúng chưa được hiện thực trong các cơ sở dữ liệu chủ lưu ngoài Google.

Process Pauses — Tạm dừng tiến trình

Let's consider another example of dangerous clock use in a distributed system. Say you have a database with a single leader per partition. Only the leader is allowed to accept writes. How does a node know that it is still leader (that it hasn't been declared dead by the others), and that it may safely accept writes?

Hãy xét một ví dụ khác về cách dùng đồng hồ nguy hiểm trong một hệ thống phân tán. Giả sử bạn có một cơ sở dữ liệu với một người dẫn đầu duy nhất cho mỗi phân vùng. Chỉ người dẫn đầu được phép chấp nhận các lần ghi. Làm sao một nút biết rằng nó vẫn là người dẫn đầu (rằng nó chưa bị các nút khác tuyên bố là đã chết), và rằng nó có thể an toàn chấp nhận các lần ghi?

One option is for the leader to obtain a lease from the other nodes, which is similar to a lock with a timeout [63]. Only one node can hold the lease at any one time—thus, when a node obtains a lease, it knows that it is the leader for some amount of time, until the lease expires. In order to remain leader, the node must periodically renew the lease before it expires. If the node fails, it stops renewing the lease, so another node can take over when it expires.

Một lựa chọn là người dẫn đầu lấy một hợp đồng thuê (lease) từ các nút khác, vốn tương tự một khóa có thời gian chờ [63]. Chỉ một nút có thể giữ hợp đồng thuê tại bất kỳ thời điểm nào — như vậy, khi một nút lấy được một hợp đồng thuê, nó biết rằng nó là người dẫn đầu trong một khoảng thời gian nào đó, cho tới khi hợp đồng thuê hết hạn. Để vẫn là người dẫn đầu, nút phải định kỳ gia hạn hợp đồng thuê trước khi nó hết hạn. Nếu nút hỏng, nó ngừng gia hạn hợp đồng thuê, nên một nút khác có thể tiếp quản khi nó hết hạn.

You can imagine the request-handling loop looking something like this:

Bạn có thể hình dung vòng lặp xử lý yêu cầu trông giống như thế này:

```
while (true) {
    request = getIncomingRequest();
    // Ensure that the lease always has at least 10 seconds remaining
    if (lease.expiryTimeMillis - System.currentTimeMillis() < 10000) {
        lease = lease.renew();
    }
    if (lease.isValid()) {
        process(request);
    }
}
```

What's wrong with this code? Firstly, it's relying on synchronized clocks: the expiry time on the lease is set by a different machine (where the expiry may be calculated as the current time plus 30 seconds, for example), and it's being compared to the local system clock. If the clocks are out of sync by more than a few seconds, this code will start doing strange things.

Có gì sai với đoạn mã này? Thứ nhất, nó đang trông cậy vào các đồng hồ được đồng bộ: thời gian hết hạn trên hợp đồng thuê được đặt bởi một máy khác (nơi thời gian hết hạn có thể được tính chẳng hạn là thời gian hiện tại cộng 30 giây), và nó đang được so sánh với đồng hồ hệ thống cục bộ. Nếu các đồng hồ lệch nhau hơn vài giây, đoạn mã này sẽ bắt đầu làm những điều kỳ quặc.

Secondly, even if we change the protocol to only use the local monotonic clock, there is another problem: the code assumes that very little time passes between the point that it checks the time (`System.currentTimeMillis()`) and the time when the request is processed (`process(request)`).

Normally this code runs very quickly, so the 10 second buffer is more than enough to ensure that the lease doesn't expire in the middle of processing a request.

Thứ hai, ngay cả khi ta đổi giao thức để chỉ dùng đồng hồ đơn điệu cục bộ, vẫn có một vấn đề khác: đoạn mã giả định rằng rất ít thời gian trôi qua giữa thời điểm nó kiểm tra thời gian (`System.currentTimeMillis()`) và thời điểm yêu cầu được xử lý (`process(request)`). Thông thường đoạn mã này chạy rất nhanh, nên khoảng đệm 10 giây là quá đủ để bảo đảm rằng hợp đồng thuê không hết hạn giữa chừng khi đang xử lý một yêu cầu.

However, what if there is an unexpected pause in the execution of the program? For example, imagine the thread stops for 15 seconds around the line `lease.isValid()` before finally continuing. In that case, it's likely that the lease will have expired by the time the request is processed, and another node has already taken over as leader. However, there is nothing to tell this thread that it was paused for so long, so this code won't notice that the lease has expired until the next iteration of the loop—by which time it may have already done something unsafe by processing the request.

Tuy nhiên, điều gì xảy ra nếu có một sự tạm dừng bất ngờ trong quá trình thực thi của chương trình? Chẳng hạn, hãy hình dung luồng dừng lại trong 15 giây quanh dòng `lease.isValid()` trước khi cuối cùng tiếp tục. Trong trường hợp đó, nhiều khả năng hợp đồng thuê sẽ đã hết hạn vào lúc yêu cầu được xử lý, và một nút khác đã tiếp quản làm người dẫn đầu. Tuy nhiên, không có gì báo cho luồng này biết rằng nó đã bị tạm dừng lâu đến vậy, nên đoạn mã này sẽ không nhận ra rằng hợp đồng thuê đã hết hạn cho tới lần lặp tiếp theo của vòng lặp — và lúc đó nó có thể đã làm điều gì đó không an toàn khi xử lý yêu cầu.

Is it crazy to assume that a thread might be paused for so long? Unfortunately not. There are various reasons why this could happen:

Liệu có điên rồ không khi giả định rằng một luồng có thể bị tạm dừng lâu đến thế? Thật không may là không. Có nhiều lý do khiến điều này có thể xảy ra:

- Many programming language runtimes (such as the Java Virtual Machine) have a garbage collector (GC) that occasionally needs to stop all running threads. These “stop-the-world” GC pauses have sometimes been known to last for several minutes [64]! Even so-called “concurrent” garbage collectors like the HotSpot JVM's CMS cannot fully run in parallel with the **application code**—even they need to stop the world from time to time [65]. Although the pauses can often be reduced by changing allocation patterns or tuning GC settings [66], we must assume the worst if we want to offer robust guarantees.
- In virtualized environments, a virtual machine can be suspended (pausing the execution of all processes and saving the contents of memory to disk) and resumed (restoring the contents of memory and continuing execution). This pause can occur at any time in a process's execution and can last for an arbitrary length of time. This feature is sometimes used for live **migration** of virtual machines from one host to another without a reboot, in which case the length of the pause depends on the rate at which processes are writing to memory [67].
- On end-user devices such as laptops, execution may also be suspended and resumed arbitrarily, e.g., when the user closes the lid of their laptop.
- When the operating system context-switches to another thread, or when the hypervisor switches to a different virtual machine (when running in a virtual machine), the currently running thread can be paused at any arbitrary point in the code. In the case of a virtual machine, the CPU time spent in other virtual machines is known as steal time. If the machine is under heavy load—i.e., if there is a long queue of threads waiting to run—it may take some time before the paused thread gets to run again.
- If the application performs synchronous disk access, a thread may be paused waiting for a slow

disk I/O operation to complete [68]. In many languages, disk access can happen surprisingly, even if the code doesn't explicitly mention file access—for example, the Java classloader lazily loads class files when they are first used, which could happen at any time in the program execution. I/O pauses and GC pauses may even conspire to combine their delays [69]. If the disk is actually a network filesystem or network block device (such as Amazon's EBS), the I/O latency is further subject to the variability of network delays [29].

- If the operating system is configured to allow swapping to disk (paging), a simple memory access may result in a **page fault** that requires a page from disk to be loaded into memory. The thread is paused while this slow I/O operation takes place. If memory pressure is high, this may in turn require a different page to be swapped out to disk. In extreme circumstances, the operating system may spend most of its time swapping pages in and out of memory and getting little actual work done (this is known as thrashing). To avoid this problem, paging is often disabled on server machines (if you would rather kill a process to free up memory than risk thrashing).
- A Unix process can be paused by sending it the SIGSTOP signal, for example by pressing Ctrl-Z in a shell. This signal immediately stops the process from getting any more CPU cycles until it is resumed with SIGCONT, at which point it continues running where it left off. Even if your environment does not normally use SIGSTOP, it might be sent accidentally by an operations engineer.

- Nhiều runtime của ngôn ngữ lập trình (chẳng hạn Máy ảo Java) có một bộ thu gom rác (GC) mà thỉnh thoảng cần dừng tất cả các luồng đang chạy. Những đợt tạm dừng GC “dừng-cả-thế-giới” này đôi khi được biết là kéo dài tới vài phút [64]! Ngay cả những bộ thu gom rác gọi là “đồng thời” như CMS của HotSpot JVM cũng không thể chạy hoàn toàn song song với mã ứng dụng — ngay cả chúng cũng cần dừng cả thế giới một lúc nào đó [65]. Mặc dù các đợt tạm dừng thường có thể được giảm bớt bằng cách thay đổi các mẫu cấp phát hoặc tinh chỉnh các thiết lập GC [66], ta phải giả định trường hợp xấu nhất nếu muốn cung cấp các bảo đảm kiên cường.
- Trong các môi trường ảo hóa, một máy ảo có thể bị tạm treo (tạm dừng việc thực thi của tất cả các tiến trình và lưu nội dung của bộ nhớ vào đĩa) và được khôi phục (phục hồi nội dung của bộ nhớ và tiếp tục thực thi). Sự tạm dừng này có thể xảy ra vào bất kỳ lúc nào trong quá trình thực thi của một tiến trình và có thể kéo dài trong một khoảng thời gian bất kỳ. Tính năng này đôi khi được dùng để di trú trực tiếp (live migration) các máy ảo từ một máy chủ này sang máy chủ khác mà không cần khởi động lại, trong trường hợp đó độ dài của sự tạm dừng phụ thuộc vào tốc độ mà các tiến trình đang ghi vào bộ nhớ [67].
- Trên các thiết bị người dùng cuối như laptop, việc thực thi cũng có thể bị tạm treo và khôi phục một cách tùy tiện, chẳng hạn khi người dùng đóng nắp laptop của họ.
- Khi hệ điều hành chuyển ngữ cảnh sang một luồng khác, hoặc khi hypervisor chuyển sang một máy ảo khác (khi đang chạy trong một máy ảo), luồng đang chạy có thể bị tạm dừng tại một điểm tùy tiện trong mã. Trong trường hợp một máy ảo, thời gian CPU bị tiêu tốn trong các máy ảo khác được gọi là thời gian bị đánh cắp (steal time). Nếu máy đang chịu tải nặng — tức là nếu có một hàng đợi dài các luồng đang chờ chạy — thì có thể mất một lúc trước khi luồng bị tạm dừng được chạy lại.
- Nếu ứng dụng thực hiện truy cập đĩa đồng bộ, một luồng có thể bị tạm dừng để chờ một thao tác I/O đĩa chậm hoàn tất [68]. Trong nhiều ngôn ngữ, việc truy cập đĩa có thể xảy ra một cách đáng ngạc nhiên, ngay cả khi mã không nhắc tới việc truy cập tập tin một cách tường minh — chẳng hạn, bộ nạp lớp (classloader) của Java nạp các tập tin lớp một cách lười biếng khi chúng được dùng lần đầu, điều này có thể xảy ra vào bất kỳ lúc nào trong quá trình thực thi chương trình. Các đợt tạm dừng I/O và các đợt tạm dừng GC thậm chí có thể cấu kết để cộng dồn độ trễ của chúng [69]. Nếu đĩa thực ra là một hệ thống tập tin mạng hoặc một thiết bị khối mạng (chẳng hạn EBS của

Amazon), thì độ trễ I/O còn chịu thêm sự biến động của độ trễ mạng [29].

- Nếu hệ điều hành được cấu hình để cho phép trao đổi ra đĩa (phân trang), một lần truy cập bộ nhớ đơn giản có thể dẫn tới một lỗi trang đòi hỏi một trang từ đĩa phải được nạp vào bộ nhớ. Luồng bị tạm dừng trong khi thao tác I/O chậm này diễn ra. Nếu áp lực bộ nhớ cao, điều này đến lượt nó có thể đòi hỏi một trang khác bị trao ra đĩa. Trong những hoàn cảnh cực đoan, hệ điều hành có thể dành phần lớn thời gian của nó để trao các trang vào và ra khỏi bộ nhớ và làm được rất ít việc thực sự (điều này được gọi là thrashing). Để tránh vấn đề này, việc phân trang thường bị tắt trên các máy chủ (nếu bạn thả giết một tiến trình để giải phóng bộ nhớ còn hơn là mạo hiểm thrashing).
- Một tiến trình Unix có thể bị tạm dừng bằng cách gửi cho nó tín hiệu SIGSTOP, chẳng hạn bằng cách nhấn Ctrl-Z trong một shell. Tín hiệu này ngay lập tức ngăn tiến trình nhận thêm bất kỳ chu kỳ CPU nào cho tới khi nó được khôi phục bằng SIGCONT, lúc đó nó tiếp tục chạy từ nơi nó đã dừng. Ngay cả khi môi trường của bạn thông thường không dùng SIGSTOP, nó có thể bị một kỹ sư vận hành gửi đi một cách tình cờ.

All of these occurrences can preempt the running thread at any point and resume it at some later time, without the thread even noticing. The problem is similar to making multi-threaded code on a single machine thread-safe: you can't assume anything about timing, because arbitrary context switches and parallelism may occur.

Tất cả những sự việc này đều có thể chiếm dụng luồng đang chạy tại bất kỳ điểm nào và khôi phục nó vào một lúc nào đó về sau, mà luồng thậm chí không hề hay biết. Vấn đề này tương tự với việc làm cho mã đa luồng trên một máy đơn lẻ an toàn với luồng (thread-safe): bạn không thể giả định bất cứ điều gì về định thời, bởi các chuyển ngữ cảnh và sự song song tùy tiện có thể xảy ra.

When writing multi-threaded code on a single machine, we have fairly good tools for making it thread-safe: mutexes, semaphores, atomic counters, lock-free data structures, blocking queues, and so on. Unfortunately, these tools don't directly translate to distributed systems, because a distributed system has no shared memory—only messages sent over an unreliable network.

Khi viết mã đa luồng trên một máy đơn lẻ, ta có khá nhiều công cụ tốt để làm cho nó an toàn với luồng: mutex, semaphore, bộ đếm nguyên tử, các cấu trúc dữ liệu không khóa, các hàng đợi chặn, v.v. Thật không may, các công cụ này không chuyển dịch trực tiếp sang các hệ thống phân tán, bởi một hệ thống phân tán không có bộ nhớ chia sẻ — chỉ có các thông điệp được gửi qua một mạng không đáng tin cậy.

A node in a distributed system must assume that its execution can be paused for a significant length of time at any point, even in the middle of a function. During the pause, the rest of the world keeps moving and may even declare the paused node dead because it's not responding. Eventually, the paused node may continue running, without even noticing that it was asleep until it checks its clock sometime later.

Một nút trong một hệ thống phân tán phải giả định rằng việc thực thi của nó có thể bị tạm dừng trong một khoảng thời gian đáng kể tại bất kỳ điểm nào, kể cả giữa chừng một hàm. Trong lúc tạm dừng, phần còn lại của thế giới vẫn tiếp tục vận động và thậm chí có thể tuyên bố nút bị tạm dừng là đã chết vì nó không phản hồi. Cuối cùng, nút bị tạm dừng có thể tiếp tục chạy, thậm chí không hề nhận ra rằng nó đã “ngủ” cho tới khi nó kiểm tra đồng hồ của mình một lúc nào đó về sau.

Response time guarantees — Bảo đảm thời gian phản hồi

In many programming languages and operating systems, threads and processes may pause for an unbounded amount of time, as discussed. Those reasons for pausing can be eliminated if you try hard enough.

Trong nhiều ngôn ngữ lập trình và hệ điều hành, các luồng và tiến trình có thể tạm dừng trong một khoảng thời gian không có giới hạn, như đã bàn. Những lý do tạm dừng đó có thể được loại bỏ nếu bạn cố gắng đủ nhiều.

Some software runs in environments where a failure to respond within a specified time can cause serious damage: computers that control aircraft, rockets, robots, cars, and other physical objects must respond quickly and predictably to their sensor inputs. In these systems, there is a specified deadline by which the software must respond; if it doesn't meet the deadline, that may cause a failure of the entire system. These are so-called hard real-time systems.

Một số phần mềm chạy trong các môi trường mà việc không phản hồi được trong một khoảng thời gian xác định có thể gây thiệt hại nghiêm trọng: các máy tính điều khiển máy bay, tên lửa, robot, ô tô, và các vật thể vật lý khác phải phản hồi nhanh chóng và có thể dự đoán đối với các đầu vào cảm biến của chúng. Trong các hệ thống này, có một thời hạn xác định mà phần mềm phải phản hồi trước đó; nếu nó không đáp ứng được thời hạn, điều đó có thể gây ra hỏng hóc của toàn bộ hệ thống. Đây là cái gọi là các hệ thống thời gian thực cứng.

Is real-time really real? — Thời gian thực có thực sự là thực không?

In embedded systems, real-time means that a system is carefully designed and tested to meet specified timing guarantees in all circumstances. This meaning is in contrast to the more vague use of the term real-time on the web, where it describes servers pushing data to clients and **stream processing** without hard response time constraints (see Chapter 11).

Trong các hệ thống nhúng, thời gian thực nghĩa là một hệ thống được thiết kế và kiểm thử cẩn thận để đáp ứng các bảo đảm về định thời xác định trong mọi hoàn cảnh. Ý nghĩa này trái ngược với cách dùng mơ hồ hơn của thuật ngữ thời gian thực trên web, nơi nó mô tả các máy chủ đẩy dữ liệu tới các client và việc xử lý luồng mà không có các ràng buộc cứng về thời gian phản hồi (xem Chương 11).

For example, if your car’s onboard sensors detect that you are currently experiencing a crash, you wouldn’t want the release of the airbag to be delayed due to an inopportune GC pause in the airbag release system.

Chẳng hạn, nếu các cảm biến trên xe của bạn phát hiện rằng bạn hiện đang gặp một vụ va chạm, bạn sẽ không muốn việc bung túi khí bị trì hoãn do một đợt tạm dừng GC không đúng lúc trong hệ thống bung túi khí.

Providing real-time guarantees in a system requires support from all levels of the software stack: a real-time operating system (RTOS) that allows processes to be scheduled with a guaranteed allocation of CPU time in specified intervals is needed; library functions must **document** their worst-case execution times; dynamic memory allocation may be restricted or disallowed entirely (real-time garbage collectors exist, but the application must still ensure that it doesn’t give the GC too much work to do); and an enormous amount of testing and measurement must be done to ensure that guarantees are being met.

Việc cung cấp các bảo đảm thời gian thực trong một hệ thống đòi hỏi sự hỗ trợ từ mọi tầng của ngăn xếp phần mềm: cần một hệ điều hành thời gian thực (RTOS) cho phép các tiến trình được định thời với một sự cấp phát thời gian CPU được bảo đảm trong các khoảng xác định; các hàm thư viện phải tài liệu hóa thời gian thực thi trường hợp xấu nhất của chúng; việc cấp phát bộ nhớ động có thể bị hạn chế hoặc cấm hoàn toàn (có tồn tại các bộ thu gom rác thời gian thực, nhưng ứng dụng vẫn phải bảo đảm rằng nó không giao cho GC quá nhiều việc để làm); và phải làm một lượng khổng lồ việc kiểm thử và đo đạc để bảo đảm rằng các bảo đảm đang được đáp ứng.

All of this requires a large amount of additional work and severely restricts the range of programming languages, libraries, and tools that can be used (since most languages and tools do not provide real-time guarantees). For these reasons, developing real-time systems is very

expensive, and they are most commonly used in safety-critical embedded devices. Moreover, “real-time” is not the same as “high-performance”—in fact, real-time systems may have lower **throughput**, since they have to prioritize timely responses above all else (see also “Latency and Resource Utilization”).

Tất cả những điều này đòi hỏi một lượng lớn công việc thêm và hạn chế nghiêm trọng phạm vi các ngôn ngữ lập trình, thư viện, và công cụ có thể được dùng (vì hầu hết các ngôn ngữ và công cụ không cung cấp các bảo đảm thời gian thực). Vì những lý do này, việc phát triển các hệ thống thời gian thực rất tốn kém, và chúng phổ biến nhất trong các thiết bị nhúng quan trọng về an toàn. Hơn nữa, “thời gian thực” không giống với “hiệu năng cao” — trên thực tế, các hệ thống thời gian thực có thể có thông lượng thấp hơn, vì chúng phải ưu tiên các phản hồi đúng lúc lên trên tất cả mọi thứ (xem thêm “Độ trễ và mức tận dụng tài nguyên”).

For most server-side data processing systems, real-time guarantees are simply not economical or appropriate. Consequently, these systems must suffer the pauses and clock instability that come from operating in a non-real-time environment.

Đối với hầu hết các hệ thống xử lý dữ liệu phía máy chủ, các bảo đảm thời gian thực đơn giản là không kinh tế hoặc không thích hợp. Do đó, các hệ thống này phải chịu các đợt tạm dừng và sự bất ổn của đồng hồ vốn đi kèm với việc vận hành trong một môi trường không phải thời gian thực.

Limiting the impact of garbage collection — Giới hạn tác động của việc thu gom rác

The negative effects of process pauses can be mitigated without resorting to expensive real-time scheduling guarantees. Language runtimes have some flexibility around when they schedule garbage collections, because they can track the rate of object allocation and the remaining free memory over time.

Các tác động tiêu cực của việc tạm dừng tiến trình có thể được giảm thiểu mà không phải viện tới các bảo đảm định thời thời gian thực tốn kém. Các runtime của ngôn ngữ có một sự linh hoạt nhất định về thời điểm chúng định thời các đợt thu gom rác, bởi chúng có thể theo dõi tốc độ cấp phát đối tượng và lượng bộ nhớ trống còn lại theo thời gian.

An emerging idea is to treat GC pauses like brief planned outages of a node, and to let other nodes handle requests from clients while one node is collecting its garbage. If the runtime can warn the application that a node soon requires a GC pause, the application can stop sending new requests to that node, wait for it to finish processing outstanding requests, and then perform the GC while no requests are in progress. This trick hides GC pauses from clients and reduces the high percentiles of response time [70, 71]. Some latency-sensitive financial trading systems [72] use this approach.

Một ý tưởng đang nổi lên là coi các đợt tạm dừng GC giống như các đợt gián đoạn ngắn được lên kế hoạch của một nút, và để các nút khác xử lý các yêu cầu từ client trong khi một nút đang thu gom rác của nó. Nếu runtime có thể cảnh báo cho ứng dụng rằng một nút sắp cần một đợt tạm dừng GC, thì ứng dụng có thể ngừng gửi các yêu cầu mới tới nút đó, chờ nó xử lý xong các yêu cầu còn tồn đọng, rồi thực hiện GC trong khi không có yêu cầu nào đang được xử lý. Mẹo này che giấu các đợt tạm dừng GC khỏi các client và làm giảm các phân vị cao của thời gian phản hồi [70, 71]. Một số hệ thống giao dịch tài chính nhạy với độ trễ [72] dùng cách tiếp cận này.

A variant of this idea is to use the garbage collector only for short-lived objects (which are fast to collect) and to restart processes periodically, before they accumulate enough long-lived objects to

require a full GC of long-lived objects [65, 73]. One node can be restarted at a time, and traffic can be shifted away from the node before the planned restart, like in a rolling upgrade (see Chapter 4).

Một biến thể của ý tưởng này là chỉ dùng bộ thu gom rác cho các đối tượng có vòng đời ngắn (vốn nhanh để thu gom) và khởi động lại các tiến trình một cách định kỳ, trước khi chúng tích tụ đủ các đối tượng có vòng đời dài đến mức đòi hỏi một đợt GC đầy đủ cho các đối tượng có vòng đời dài [65, 73]. Có thể khởi động lại từng nút một, và lưu lượng có thể được dịch chuyển ra khỏi nút trước khi khởi động lại theo kế hoạch, giống như trong một đợt nâng cấp cuốn chiếu (xem Chương 4).

These measures cannot fully prevent garbage collection pauses, but they can usefully reduce their impact on the application.

Những biện pháp này không thể ngăn chặn hoàn toàn các đợt tạm dừng thu gom rác, nhưng chúng có thể giảm bớt một cách hữu ích tác động của các đợt này lên ứng dụng.

Knowledge, Truth, and Lies — Tri thức, sự thật và lời dối

So far in this chapter we have explored the ways in which distributed systems are different from programs running on a single computer: there is no shared memory, only message passing via an unreliable network with variable delays, and the systems may suffer from partial failures, unreliable clocks, and processing pauses.

Cho đến giờ trong chương này, ta đã khám phá những cách mà các hệ thống phân tán khác với các chương trình chạy trên một máy tính đơn lẻ: không có bộ nhớ chia sẻ, chỉ có việc truyền thông điệp qua một mạng không đáng tin cậy với độ trễ biến động, và các hệ thống có thể phải chịu các hỏng hóc cục bộ, các đồng hồ không đáng tin cậy, và các đợt tạm dừng xử lý.

The consequences of these issues are profoundly disorienting if you're not used to distributed systems. A node in the network cannot know anything for sure—it can only make guesses based on the messages it receives (or doesn't receive) via the network. A node can only find out what state another node is in (what data it has stored, whether it is correctly functioning, etc.) by exchanging messages with it. If a remote node doesn't respond, there is no way of knowing what state it is in, because problems in the network cannot reliably be distinguished from problems at a node.

Hệ quả của những vấn đề này khiến ta hết sức hoang mang nếu bạn chưa quen với các hệ thống phân tán. Một nút trong mạng không thể biết chắc chắn bất cứ điều gì — nó chỉ có thể đưa ra phỏng đoán dựa trên các thông điệp mà nó nhận được (hoặc không nhận được) qua mạng. Một nút chỉ có thể tìm ra một nút khác đang ở trạng thái nào (nó đã lưu dữ liệu gì, liệu nó có đang hoạt động đúng hay không, v.v.) bằng cách trao đổi các thông điệp với nó. Nếu một nút từ xa không phản hồi, không có cách nào biết được nó đang ở trạng thái nào, bởi các vấn đề trong mạng không thể được phân biệt một cách đáng tin cậy với các vấn đề tại một nút.

Discussions of these systems border on the philosophical: What do we know to be true or false in our system? How sure can we be of that knowledge, if the mechanisms for perception and measurement are unreliable? Should software systems obey the laws that we expect of the physical world, such as cause and effect?

Các thảo luận về những hệ thống này giáp ranh với triết học: Ta biết những gì là đúng hay sai trong hệ thống của mình? Ta có thể chắc chắn tới mức nào về tri thức đó, nếu các cơ chế tri giác và đo đạc đều không đáng tin cậy? Liệu các hệ thống phần mềm có nên tuân theo các quy luật mà ta mong đợi của thế giới vật lý, chẳng hạn nhân và quả?

Fortunately, we don't need to go as far as figuring out the meaning of life. In a distributed system, we can state the assumptions we are making about the behavior (the system model) and design the

actual system in such a way that it meets those assumptions. Algorithms can be proved to function correctly within a certain system model. This means that reliable behavior is achievable, even if the underlying system model provides very few guarantees.

May thay, ta không cần đi xa tới mức tìm ra ý nghĩa của cuộc sống. Trong một hệ thống phân tán, ta có thể phát biểu các giả định mà ta đang đặt ra về hành vi (mô hình hệ thống) và thiết kế hệ thống thực sao cho nó đáp ứng các giả định đó. Có thể chứng minh các thuật toán hoạt động đúng trong một mô hình hệ thống nhất định. Điều này nghĩa là hành vi đáng tin cậy là khả thi, ngay cả khi mô hình hệ thống bên dưới cung cấp rất ít bảo đảm.

However, although it is possible to make software well behaved in an unreliable system model, it is not straightforward to do so. In the rest of this chapter we will further explore the notions of knowledge and truth in distributed systems, which will help us think about the kinds of assumptions we can make and the guarantees we may want to provide. In Chapter 9 we will proceed to look at some examples of distributed systems, algorithms that provide particular guarantees under particular assumptions.

Tuy nhiên, mặc dù có thể làm cho phần mềm hành xử tốt trong một mô hình hệ thống không đáng tin cậy, làm được điều đó không hề dễ. Trong phần còn lại của chương này, ta sẽ khám phá thêm các khái niệm về tri thức và sự thật trong các hệ thống phân tán, điều này sẽ giúp ta nghĩ về những loại giả định ta có thể đặt ra và những bảo đảm ta có thể muốn cung cấp. Ở Chương 9, ta sẽ tiếp tục xem xét một số ví dụ về các hệ thống phân tán, các thuật toán cung cấp những bảo đảm cụ thể dưới những giả định cụ thể.

The Truth Is Defined by the Majority — Sự thật được định nghĩa bởi đa số

Imagine a network with an asymmetric fault: a node is able to receive all messages sent to it, but any outgoing messages from that node are dropped or delayed [19]. Even though that node is working perfectly well, and is receiving requests from other nodes, the other nodes cannot hear its responses. After some timeout, the other nodes declare it dead, because they haven't heard from the node. The situation unfolds like a nightmare: the semi-disconnected node is dragged to the graveyard, kicking and screaming “I'm not dead!”—but since nobody can hear its screaming, the funeral procession continues with stoic determination.

Hãy hình dung một mạng với một lỗi bất đối xứng: một nút có thể nhận tất cả các thông điệp được gửi tới nó, nhưng bất kỳ thông điệp nào đi ra từ nút đó đều bị làm rớt hoặc làm trễ [19]. Mặc dù nút đó hoạt động hoàn toàn ổn, và đang nhận các yêu cầu từ các nút khác, các nút khác không thể nghe được các phản hồi của nó. Sau một thời gian chờ nào đó, các nút khác tuyên bố nó đã chết, bởi chúng không nghe được hồi âm từ nút này. Tình huống diễn ra như một cơn ác mộng: nút bị-ngắt-kết-nối-một-nửa bị lôi ra nghĩa địa, vùng vẫy la hét “Tôi chưa chết!” — nhưng vì không ai có thể nghe được tiếng la hét của nó, đám rước tang vẫn tiếp tục với sự quyết tâm điềm tĩnh.

In a slightly less nightmarish scenario, the semi-disconnected node may notice that the messages it is sending are not being acknowledged by other nodes, and so realize that there must be a fault in the network. Nevertheless, the node is wrongly declared dead by the other nodes, and the semi-disconnected node cannot do anything about it.

Trong một kịch bản ít ác mộng hơn một chút, nút bị-ngắt-kết-nối-một-nửa có thể để ý rằng các thông điệp nó đang gửi không được các nút khác báo nhận, và do đó nhận ra rằng hẳn

có một lỗi trong mạng. Tuy vậy, nút vẫn bị các nút khác tuyên bố nhầm là đã chết, và nút bị-ngắt-kết-nối-một-nửa không thể làm gì được về điều đó.

As a third scenario, imagine a node that experiences a long stop-the-world garbage collection pause. All of the node's threads are preempted by the GC and paused for one minute, and consequently, no requests are processed and no responses are sent. The other nodes wait, retry, grow impatient, and eventually declare the node dead and load it onto the hearse. Finally, the GC finishes and the node's threads continue as if nothing had happened. The other nodes are surprised as the supposedly dead node suddenly raises its head out of the coffin, in full health, and starts cheerfully chatting with bystanders. At first, the GCing node doesn't even realize that an entire minute has passed and that it was declared dead—from its perspective, hardly any time has passed since it was last talking to the other nodes.

Như một kịch bản thứ ba, hãy hình dung một nút trải qua một đợt tạm dừng thu gom rác dừng-cả-thế-giới kéo dài. Tất cả các luồng của nút bị GC chiếm dụng và tạm dừng trong một phút, và do đó, không có yêu cầu nào được xử lý và không có phản hồi nào được gửi. Các nút khác chờ, thử lại, dần mất kiên nhẫn, và rốt cuộc tuyên bố nút đã chết và chất nó lên xe tang. Cuối cùng, GC hoàn tất và các luồng của nút tiếp tục như thể không có gì xảy ra. Các nút khác ngạc nhiên khi cái nút tưởng đã chết ấy đột nhiên ngóc đầu dậy khỏi quan tài, hoàn toàn khỏe mạnh, và bắt đầu vui vẻ tán gẫu với những người đứng quanh. Thoạt đầu, cái nút vừa GC ấy thậm chí không nhận ra rằng cả một phút đã trôi qua và rằng nó đã bị tuyên bố là đã chết — từ góc nhìn của nó, gần như không có thời gian nào trôi qua kể từ lần cuối nó nói chuyện với các nút khác.

The moral of these stories is that a node cannot necessarily trust its own judgment of a situation. A distributed system cannot exclusively rely on a single node, because a node may fail at any time, potentially leaving the system stuck and unable to recover. Instead, many distributed algorithms rely on a quorum, that is, voting among the nodes (see “Quorums for reading and writing”): decisions require some minimum number of votes from several nodes in order to reduce the dependence on any one particular node.

Bài học rút ra từ những câu chuyện này là một nút không nhất thiết có thể tin vào phán đoán của chính nó về một tình huống. Một hệ thống phân tán không thể chỉ trông cậy duy nhất vào một nút, bởi một nút có thể hỏng vào bất cứ lúc nào, có khả năng khiến hệ thống bị mắc kẹt và không thể phục hồi. Thay vào đó, nhiều thuật toán phân tán dựa vào một số đại biểu (quorum), tức là việc bỏ phiếu giữa các nút (xem “Số đại biểu để đọc và ghi”): các quyết định đòi hỏi một số phiếu tối thiểu nào đó từ một số nút để giảm bớt sự phụ thuộc vào bất kỳ một nút cụ thể nào.

That includes decisions about declaring nodes dead. If a quorum of nodes declares another node dead, then it must be considered dead, even if that node still very much feels alive. The individual node must abide by the quorum decision and step down.

Điều đó bao gồm các quyết định về việc tuyên bố các nút đã chết. Nếu một số đại biểu các nút tuyên bố một nút khác đã chết, thì nó phải được coi là đã chết, ngay cả khi nút đó vẫn cảm thấy mình hoàn toàn còn sống. Nút riêng lẻ phải tuân theo quyết định của số đại biểu và rút lui.

Most commonly, the quorum is an absolute majority of more than half the nodes (although other kinds of quorums are possible). A majority quorum allows the system to continue working if individual nodes have failed (with three nodes, one failure can be tolerated; with five nodes, two failures can be tolerated). However, it is still safe, because there can only be only one majority in the system—there cannot be two majorities with conflicting decisions at the same time. We will discuss the use of quorums in more detail when we get to consensus algorithms in Chapter 9.

Phổ biến nhất, số đại biểu là một đa số tuyệt đối gồm hơn một nửa số nút (mặc dù có thể có các loại số đại biểu khác). Một số đại biểu đa số cho phép hệ thống tiếp tục hoạt động nếu các nút riêng lẻ đã hỏng (với ba nút, có thể chịu được một nút hỏng; với năm nút, có thể chịu được hai nút hỏng). Tuy nhiên, nó vẫn an toàn, bởi chỉ có thể có một đa số duy nhất trong hệ thống — không thể có hai đa số với các quyết định xung đột cùng một lúc. Ta sẽ bàn về việc dùng số đại biểu chi tiết hơn khi tới các thuật toán đồng thuận ở Chương 9.

The leader and the lock — Người dẫn đầu và khóa

Frequently, a system requires there to be only one of some thing. For example:

Thường thì một hệ thống đòi hỏi rằng chỉ có một của một thứ gì đó. Ví dụ:

- Only one node is allowed to be the leader for a database partition, to avoid split brain (see “Handling Node Outages”).
- Only one transaction or client is allowed to hold the lock for a particular resource or object, to prevent concurrently writing to it and corrupting it.
- Only one user is allowed to register a particular username, because a username must uniquely identify a user.
 - Chỉ một nút được phép là người dẫn đầu cho một phân vùng cơ sở dữ liệu, để tránh tình trạng não phân liệt (split brain) (xem “Xử lý sự cố nút”).
 - Chỉ một giao dịch hoặc client được phép giữ khóa cho một tài nguyên hoặc đối tượng cụ thể, để ngăn việc ghi đồng thời vào nó và làm hỏng nó.
 - Chỉ một người dùng được phép đăng ký một tên người dùng cụ thể, bởi một tên người dùng phải nhận diện duy nhất một người dùng.

Implementing this in a distributed system requires care: even if a node believes that it is “the chosen one” (the leader of the partition, the holder of the lock, the request handler of the user who successfully grabbed the username), that doesn’t necessarily mean a quorum of nodes agrees! A node may have formerly been the leader, but if the other nodes declared it dead in the meantime (e.g., due to a network interruption or GC pause), it may have been demoted and another leader may have already been elected.

Việc hiện thực điều này trong một hệ thống phân tán đòi hỏi sự cẩn trọng: ngay cả khi một nút tin rằng nó là “kẻ được chọn” (người dẫn đầu của phân vùng, kẻ giữ khóa, kẻ xử lý yêu cầu của người dùng đã giành được tên người dùng thành công), điều đó không nhất thiết nghĩa là một số đại biểu các nút đồng ý! Một nút có thể từng là người dẫn đầu, nhưng nếu các nút khác đã tuyên bố nó đã chết trong thời gian đó (chẳng hạn do một sự gián đoạn mạng hoặc một đợt tạm dừng GC), thì nó có thể đã bị giáng cấp và một người dẫn đầu khác có thể đã được bầu.

If a node continues acting as the chosen one, even though the majority of nodes have declared it dead, it could cause problems in a system that is not carefully designed. Such a node could send messages to other nodes in its self-appointed capacity, and if other nodes believe it, the system as a whole may do something incorrect.

Nếu một nút tiếp tục hành xử như kẻ được chọn, ngay cả khi đa số các nút đã tuyên bố nó đã chết, nó có thể gây ra các vấn đề trong một hệ thống không được thiết kế cẩn thận. Một nút như vậy có thể gửi các thông điệp tới các nút khác trong tư cách mà nó tự phong, và nếu các nút khác tin nó, hệ thống như một tổng thể có thể làm điều gì đó sai.

For example, Figure 8-4 shows a data corruption **bug** due to an incorrect implementation of locking.

(The bug is not theoretical: HBase used to have this problem [74, 75].) Say you want to ensure that a file in a storage service can only be accessed by one client at a time, because if multiple clients tried to write to it, the file would become corrupted. You try to implement this by requiring a client to obtain a lease from a lock service before accessing the file.

Chẳng hạn, Hình 8-4 cho thấy một bug làm hỏng dữ liệu do một hiện thực khóa sai. (Bug này không phải lý thuyết: HBase từng có vấn đề này [74, 75].) Giả sử bạn muốn bảo đảm rằng một tập tin trong một dịch vụ lưu trữ chỉ có thể được truy cập bởi một client tại một thời điểm, bởi nếu nhiều client cùng cố ghi vào nó, tập tin sẽ bị hỏng. Bạn cố hiện thực điều này bằng cách yêu cầu một client lấy một hợp đồng thuê từ một dịch vụ khóa trước khi truy cập tập tin.

Figure 8-4. Incorrect implementation of a distributed lock: client 1 believes that it still has a valid lease, even though it has expired, and thus corrupts a file in storage. — Hình 8-4. Hiện thực sai một khóa phân tán: client 1 tin rằng nó vẫn còn một hợp đồng thuê hợp lệ, mặc dù nó đã hết hạn, và do đó làm hỏng một tập tin trong kho lưu trữ. The problem is an example of what we discussed in “Process Pauses”: if the client holding the lease is paused for too long, its lease expires. Another client can obtain a lease for the same file, and start writing to the file. When the paused client comes back, it believes (incorrectly) that it still has a valid lease and proceeds to also write to the file. As a result, the clients’ writes clash and corrupt the file.

Vấn đề này là một ví dụ về điều ta đã bàn ở phần “Tạm dừng tiến trình”: nếu client đang giữ hợp đồng thuê bị tạm dừng quá lâu, hợp đồng thuê của nó sẽ hết hạn. Một client khác có thể lấy một hợp đồng thuê cho cùng tập tin đó, và bắt đầu ghi vào tập tin. Khi client bị tạm dừng quay trở lại, nó tin (một cách sai lầm) rằng nó vẫn còn một hợp đồng thuê hợp lệ và tiến hành cũng ghi vào tập tin. Kết quả là các lần ghi của các client xung khắc nhau và làm hỏng tập tin.

Fencing tokens — Token rào chắn

When using a lock or lease to protect access to some resource, such as the file storage in Figure 8-4, we need to ensure that a node that is under a false belief of being “the chosen one” cannot disrupt the rest of the system. A fairly simple technique that achieves this goal is called fencing, and is illustrated in Figure 8-5.

Khi dùng một khóa hoặc hợp đồng thuê để bảo vệ việc truy cập một tài nguyên nào đó, chẳng hạn kho lưu trữ tập tin trong Hình 8-4, ta cần bảo đảm rằng một nút đang ở trong niềm tin sai lầm rằng mình là “kẻ được chọn” không thể phá rối phần còn lại của hệ thống. Một kỹ thuật khá đơn giản đạt được mục tiêu này được gọi là rào chắn (fencing), và được minh họa ở Hình 8-5.

Figure 8-5. Making access to storage safe by allowing writes only in the order of increasing fencing tokens. — Hình 8-5. Làm cho việc truy cập kho lưu trữ trở nên an toàn bằng cách chỉ cho phép các lần ghi theo thứ tự token rào chắn tăng dần. Let’s assume that every time the lock server grants a lock or lease, it also returns a fencing token, which is a number that increases every time a lock is granted (e.g., incremented by the lock service). We can then require that every time a client sends a write request to the storage service, it must include its current fencing token.

Hãy giả định rằng mỗi lần máy chủ khóa cấp một khóa hoặc hợp đồng thuê, nó cũng trả về một token rào chắn (fencing token), vốn là một con số tăng lên mỗi lần một khóa được cấp (chẳng hạn được dịch vụ khóa tăng thêm). Sau đó ta có thể yêu cầu rằng mỗi lần một

client gửi một yêu cầu ghi tới dịch vụ lưu trữ, nó phải kèm theo token rào chắn hiện tại của nó.

In Figure 8-5, client 1 acquires the lease with a token of 33, but then it goes into a long pause and the lease expires. Client 2 acquires the lease with a token of 34 (the number always increases) and then sends its write request to the storage service, including the token of 34. Later, client 1 comes back to life and sends its write to the storage service, including its token value 33. However, the storage server remembers that it has already processed a write with a higher token number (34), and so it rejects the request with token 33.

Trong Hình 8-5, client 1 lấy hợp đồng thuê với token là 33, nhưng sau đó nó rơi vào một đợt tạm dừng kéo dài và hợp đồng thuê hết hạn. Client 2 lấy hợp đồng thuê với token là 34 (con số luôn tăng) rồi gửi yêu cầu ghi của nó tới dịch vụ lưu trữ, kèm theo token 34. Về sau, client 1 sống lại và gửi lần ghi của nó tới dịch vụ lưu trữ, kèm theo giá trị token 33 của nó. Tuy nhiên, máy chủ lưu trữ nhớ rằng nó đã xử lý một lần ghi với số token cao hơn (34), nên nó từ chối yêu cầu với token 33.

If ZooKeeper is used as lock service, the transaction ID `zxid` or the node version `cversion` can be used as fencing token. Since they are guaranteed to be monotonically increasing, they have the required properties [74].

Nếu ZooKeeper được dùng làm dịch vụ khóa, thì ID giao dịch `zxid` hoặc phiên bản nút `cversion` có thể được dùng làm token rào chắn. Vì chúng được bảo đảm tăng đơn điệu, chúng có các thuộc tính cần thiết [74].

Note that this mechanism requires the resource itself to take an active role in checking tokens by rejecting any writes with an older token than one that has already been processed—it is not sufficient to rely on clients checking their lock status themselves. For resources that do not explicitly support fencing tokens, you might still be able work around the limitation (for example, in the case of a file storage service you could include the fencing token in the filename). However, some kind of check is necessary to avoid processing requests outside of the lock's protection.

Lưu ý rằng cơ chế này đòi hỏi bản thân tài nguyên phải đóng một vai trò chủ động trong việc kiểm tra các token bằng cách từ chối bất kỳ lần ghi nào có token cũ hơn một token đã được xử lý — việc trông cậy vào các client tự kiểm tra trạng thái khóa của chúng là chưa đủ. Đối với các tài nguyên không hỗ trợ token rào chắn một cách tường minh, bạn vẫn có thể tìm cách lách qua giới hạn này (chẳng hạn, trong trường hợp một dịch vụ lưu trữ tập tin, bạn có thể đưa token rào chắn vào tên tập tin). Tuy nhiên, một loại kiểm tra nào đó là cần thiết để tránh xử lý các yêu cầu nằm ngoài sự bảo vệ của khóa.

Checking a token on the server side may seem like a downside, but it is arguably a good thing: it is unwise for a service to assume that its clients will always be well behaved, because the clients are often run by people whose priorities are very different from the priorities of the people running the service [76]. Thus, it is a good idea for any service to protect itself from accidentally abusive clients.

Việc kiểm tra một token ở phía máy chủ có vẻ như là một nhược điểm, nhưng có thể lập luận rằng đó là một điều tốt: thật không khôn ngoan nếu một dịch vụ giả định rằng các client của nó sẽ luôn hành xử đàng hoàng, bởi các client thường được vận hành bởi những người có các ưu tiên rất khác với các ưu tiên của những người vận hành dịch vụ [76]. Như vậy, việc bất kỳ dịch vụ nào tự bảo vệ mình khỏi các client lạm dụng một cách vô tình là một ý hay.

Byzantine Faults — Lỗi Byzantine

Fencing tokens can detect and block a node that is inadvertently acting in error (e.g., because it hasn't yet found out that its lease has expired). However, if the node deliberately wanted to subvert the system's guarantees, it could easily do so by sending messages with a fake fencing token.

Các token rào chắn có thể phát hiện và chặn một nút đang vô tình hành xử sai (chẳng hạn vì nó chưa phát hiện ra rằng hợp đồng thuê của nó đã hết hạn). Tuy nhiên, nếu nút cố ý muốn lật đổ các bảo đảm của hệ thống, nó có thể dễ dàng làm vậy bằng cách gửi các thông điệp với một token rào chắn giả mạo.

In this book we assume that nodes are unreliable but honest: they may be slow or never respond (due to a fault), and their state may be outdated (due to a GC pause or network delays), but we assume that if a node does respond, it is telling the “truth”: to the best of its knowledge, it is playing by the rules of the protocol.

Trong cuốn sách này, ta giả định rằng các nút không đáng tin cậy nhưng trung thực: chúng có thể chậm hoặc không bao giờ phản hồi (do một lỗi), và trạng thái của chúng có thể lỗi thời (do một đợt tạm dừng GC hoặc độ trễ mạng), nhưng ta giả định rằng nếu một nút có phản hồi, nó đang nói “sự thật”: trong khả năng hiểu biết tốt nhất của nó, nó đang chơi theo các quy tắc của giao thức.

Distributed systems problems become much harder if there is a risk that nodes may “lie” (send arbitrary faulty or corrupted responses)—for example, if a node may claim to have received a particular message when in fact it didn't. Such behavior is known as a Byzantine fault, and the problem of reaching consensus in this untrusting environment is known as the Byzantine Generals Problem [77].

Các vấn đề về hệ thống phân tán trở nên khó hơn nhiều nếu có rủi ro rằng các nút có thể “nói dối” (gửi các phản hồi lỗi hoặc bị hỏng một cách tùy tiện) — chẳng hạn, nếu một nút có thể tuyên bố rằng nó đã nhận một thông điệp cụ thể trong khi thực ra nó không nhận. Hành vi như vậy được gọi là một lỗi Byzantine (Byzantine fault), và vấn đề đạt được đồng thuận trong một môi trường thiếu tin cậy này được gọi là Bài toán Các vị Tướng Byzantine [77].

The Byzantine Generals Problem — Bài toán Các vị Tướng Byzantine The Byzantine Generals Problem is a generalization of the so-called Two Generals Problem [78], which imagines a situation in which two army generals need to agree on a battle plan. As they have set up camp on two different sites, they can only communicate by messenger, and the messengers sometimes get delayed or lost (like packets in a network). We will discuss this problem of consensus in Chapter 9.

Bài toán Các vị Tướng Byzantine là một sự tổng quát hóa của cái gọi là Bài toán Hai vị Tướng [78], vốn hình dung một tình huống trong đó hai vị tướng quân đội cần thống nhất về một kế hoạch trận đánh. Vì họ đã đóng trại ở hai địa điểm khác nhau, họ chỉ có thể giao tiếp bằng người đưa tin, và những người đưa tin đôi khi bị trễ hoặc bị thất lạc (như các gói trong một mạng). Ta sẽ bàn về bài toán đồng thuận này ở Chương 9.

In the Byzantine version of the problem, there are n generals who need to agree, and their endeavor is hampered by the fact that there are some traitors in their midst. Most of the generals are loyal, and thus send truthful messages, but the traitors may try to deceive and confuse the others by sending fake or untrue messages (while trying to remain undiscovered). It is not known in advance who the traitors are.

Trong phiên bản Byzantine của bài toán, có n vị tướng cần thống nhất, và nỗ lực của họ bị cản trở bởi sự thật rằng có một số kẻ phản bội trong hàng ngũ của họ. Hầu hết các vị tướng đều trung thành, và do đó gửi các thông điệp chân thực, nhưng những kẻ phản bội có thể cố lừa dối và làm rối những người khác bằng cách gửi các thông điệp giả mạo hoặc không đúng sự thật (trong khi cố không bị phát hiện). Không thể biết trước ai là kẻ phản bội.

Byzantium was an ancient Greek city that later became Constantinople, in the place which is now Istanbul in Turkey. There isn't any historic evidence that the generals of Byzantium were any more prone to intrigue and conspiracy than those elsewhere. Rather, the name is derived from Byzantine in the sense of excessively complicated, bureaucratic, devious, which was used in politics long before computers [79]. Lamport wanted to choose a nationality that would not offend any readers, and he was advised that calling it The Albanian Generals Problem was not such a good idea [80].

Byzantium là một thành phố Hy Lạp cổ về sau trở thành Constantinople, ở nơi mà nay là Istanbul ở Thổ Nhĩ Kỳ. Không có bằng chứng lịch sử nào cho thấy các vị tướng của Byzantium dễ mưu mô và âm mưu hơn những nơi khác. Đúng hơn, cái tên này bắt nguồn từ chữ Byzantine theo nghĩa rắc rối quá mức, quan liêu, gian xảo, vốn được dùng trong chính trị từ rất lâu trước khi có máy tính [79]. Lamport muốn chọn một quốc tịch không xúc phạm bất kỳ độc giả nào, và ông được khuyên rằng việc gọi nó là Bài toán Các vị Tướng Albania không phải là một ý hay [80].

A system is Byzantine **fault-tolerant** if it continues to operate correctly even if some of the nodes are malfunctioning and not obeying the protocol, or if malicious attackers are interfering with the network. This concern is relevant in certain specific circumstances. For example:

Một hệ thống là chịu lỗi Byzantine nếu nó tiếp tục hoạt động đúng ngay cả khi một số nút đang trục trặc và không tuân theo giao thức, hoặc nếu những kẻ tấn công ác ý đang can thiệp vào mạng. Mối lo này có liên quan trong một số hoàn cảnh cụ thể. Ví dụ:

- In aerospace environments, the data in a computer's memory or CPU register could become corrupted by radiation, leading it to respond to other nodes in arbitrarily unpredictable ways. Since a system failure would be very expensive (e.g., an aircraft crashing and killing everyone on board, or a rocket colliding with the International Space Station), flight control systems must tolerate Byzantine faults [81, 82].
- In a system with multiple participating organizations, some participants may attempt to cheat or defraud others. In such circumstances, it is not safe for a node to simply trust another node's messages, since they may be sent with malicious intent. For example, peer-to-peer networks like Bitcoin and other blockchains can be considered to be a way of getting mutually untrusting parties to agree whether a transaction happened or not, without relying on a central authority [83].
- Trong các môi trường hàng không vũ trụ, dữ liệu trong bộ nhớ hoặc thanh ghi CPU của một máy tính có thể bị hỏng do bức xạ, khiến nó phản hồi với các nút khác theo những cách tùy tiện khó đoán. Vì một hỏng hóc hệ thống sẽ rất đắt giá (chẳng hạn một máy bay rơi và giết chết tất cả mọi người trên khoang, hoặc một tên lửa va chạm với Trạm Vũ trụ Quốc tế), các hệ thống điều khiển bay phải chịu được các lỗi Byzantine [81, 82].
- Trong một hệ thống có nhiều tổ chức tham gia, một số bên tham gia có thể tìm cách gian lận hoặc lừa đảo những bên khác. Trong những hoàn cảnh như vậy, việc một nút chỉ đơn giản tin vào các thông điệp của một nút khác là không an toàn, bởi chúng có thể được gửi với ý đồ ác ý. Chẳng hạn, các mạng ngang hàng như Bitcoin và các blockchain khác có thể được coi là một cách để khiến các bên không tin nhau cùng

thống nhất xem một giao dịch có xảy ra hay không, mà không trông cậy vào một thẩm quyền trung tâm [83].

However, in the kinds of systems we discuss in this book, we can usually safely assume that there are no Byzantine faults. In your datacenter, all the nodes are controlled by your organization (so they can hopefully be trusted) and radiation levels are low enough that memory corruption is not a major problem. Protocols for making systems Byzantine fault-tolerant are quite complicated [84], and fault-tolerant embedded systems rely on support from the hardware level [81]. In most server-side data systems, the cost of deploying Byzantine fault-tolerant solutions makes them impractical.

Tuy nhiên, trong các loại hệ thống mà ta bàn trong cuốn sách này, ta thường có thể an toàn giả định rằng không có lỗi Byzantine. Trong trung tâm dữ liệu của bạn, tất cả các nút đều được tổ chức của bạn kiểm soát (nên hy vọng là chúng có thể được tin tưởng) và mức bức xạ đủ thấp để việc bộ nhớ bị hỏng không phải là một vấn đề lớn. Các giao thức để làm cho hệ thống chịu lỗi Byzantine khá phức tạp [84], và các hệ thống nhúng chịu lỗi dựa vào sự hỗ trợ từ mức phần cứng [81]. Trong hầu hết các hệ thống dữ liệu phía máy chủ, chi phí triển khai các giải pháp chịu lỗi Byzantine khiến chúng trở nên thiếu thực tế.

Web applications do need to expect arbitrary and malicious behavior of clients that are under end-user control, such as web browsers. This is why input validation, sanitization, and output escaping are so important: to prevent **SQL** injection and cross-site scripting, for example. However, we typically don't use Byzantine fault-tolerant protocols here, but simply make the server the authority on deciding what client behavior is and isn't allowed. In peer-to-peer networks, where there is no such central authority, Byzantine fault tolerance is more relevant.

Các ứng dụng web thực sự cần phải lường trước hành vi tùy tiện và ác ý của các client nằm dưới sự kiểm soát của người dùng cuối, chẳng hạn các trình duyệt web. Đây là lý do vì sao việc xác thực đầu vào, làm sạch, và thoát ký tự đầu ra lại quan trọng đến vậy: để ngăn chặn, chẳng hạn, tấn công chèn SQL và kịch bản xuyên trang. Tuy nhiên, ở đây ta thường không dùng các giao thức chịu lỗi Byzantine, mà đơn giản coi máy chủ là thẩm quyền quyết định hành vi nào của client là được phép và không được phép. Trong các mạng ngang hàng, nơi không có thẩm quyền trung tâm như vậy, khả năng chịu lỗi Byzantine có liên quan hơn.

A bug in the software could be regarded as a Byzantine fault, but if you deploy the same software to all nodes, then a Byzantine fault-tolerant algorithm cannot save you. Most Byzantine fault-tolerant algorithms require a supermajority of more than two-thirds of the nodes to be functioning correctly (i.e., if you have four nodes, at most one may malfunction). To use this approach against bugs, you would have to have four independent implementations of the same software and hope that a bug only appears in one of the four implementations.

Một bug trong phần mềm có thể được coi là một lỗi Byzantine, nhưng nếu bạn triển khai cùng một phần mềm tới tất cả các nút, thì một thuật toán chịu lỗi Byzantine không thể cứu bạn. Hầu hết các thuật toán chịu lỗi Byzantine đòi hỏi một siêu đa số gồm hơn hai phần ba số nút hoạt động đúng (tức là nếu bạn có bốn nút, nhiều nhất một nút có thể trục trặc). Để dùng cách tiếp cận này chống lại các bug, bạn sẽ phải có bốn hiện thực độc lập của cùng một phần mềm và hy vọng rằng một bug chỉ xuất hiện trong một trong bốn hiện thực.

Similarly, it would be appealing if a protocol could protect us from vulnerabilities, security compromises, and malicious attacks. Unfortunately, this is not realistic either: in most systems, if an attacker can compromise one node, they can probably compromise all of them, because they are probably running the same software. Thus, traditional mechanisms (authentication, access control, encryption, firewalls, and so on) continue to be the main protection against attackers.

Tương tự, sẽ thật hấp dẫn nếu một giao thức có thể bảo vệ ta khỏi các lỗ hổng, các vụ xâm

phạm bảo mật, và các cuộc tấn công ác ý. Thật không may, điều này cũng không thực tế: trong hầu hết các hệ thống, nếu một kẻ tấn công có thể xâm phạm một nút, thì có lẽ chúng có thể xâm phạm tất cả chúng, bởi chúng có lẽ đang chạy cùng một phần mềm. Như vậy, các cơ chế truyền thống (xác thực, kiểm soát truy cập, mã hóa, tường lửa, v.v.) tiếp tục là sự bảo vệ chính chống lại những kẻ tấn công.

Weak forms of lying — Các dạng nói dối yếu

Although we assume that nodes are generally honest, it can be worth adding mechanisms to software that guard against weak forms of “lying”—for example, invalid messages due to hardware issues, software bugs, and misconfiguration. Such protection mechanisms are not full-blown Byzantine fault tolerance, as they would not withstand a determined adversary, but they are nevertheless simple and pragmatic steps toward better reliability. For example:

Mặc dù ta giả định rằng các nút nhìn chung là trung thực, có thể đáng để thêm vào phần mềm các cơ chế phòng vệ chống lại các dạng “nói dối” yếu — chẳng hạn, các thông điệp không hợp lệ do các vấn đề phần cứng, các bug phần mềm, và việc cấu hình sai. Những cơ chế bảo vệ như vậy không phải là khả năng chịu lỗi Byzantine đầy đủ, vì chúng sẽ không trụ được trước một đối thủ quyết tâm, nhưng dù sao chúng cũng là những bước đi đơn giản và thực dụng hướng tới độ tin cậy tốt hơn. Ví dụ:

- Network packets do sometimes get corrupted due to hardware issues or bugs in operating systems, drivers, routers, etc. Usually, corrupted packets are caught by the checksums built into TCP and UDP, but sometimes they evade detection [85, 86, 87]. Simple measures are usually sufficient protection against such corruption, such as checksums in the application-level protocol.
- A publicly accessible application must carefully sanitize any inputs from users, for example checking that a value is within a reasonable range and limiting the size of strings to prevent denial of service through large memory allocations. An internal service behind a firewall may be able to get away with less strict checks on inputs, but some basic sanity-checking of values (e.g., in protocol parsing [85]) is a good idea.
- NTP clients can be configured with multiple server addresses. When synchronizing, the client contacts all of them, estimates their errors, and checks that a majority of servers agree on some time range. As long as most of the servers are okay, a misconfigured NTP server that is reporting an incorrect time is detected as an **outlier** and is excluded from synchronization [37]. The use of multiple servers makes NTP more robust than if it only uses a single server.
 - Các gói mạng đôi khi quả thực bị hỏng do các vấn đề phần cứng hoặc các bug trong hệ điều hành, trình điều khiển, bộ định tuyến, v.v. Thường thì các gói bị hỏng được bắt bởi các checksum được tích hợp sẵn trong TCP và UDP, nhưng đôi khi chúng lọt qua sự phát hiện [85, 86, 87]. Các biện pháp đơn giản thường là sự bảo vệ đủ tốt chống lại loại hỏng này, chẳng hạn các checksum trong giao thức ở mức ứng dụng.
 - Một ứng dụng có thể được truy cập công khai phải làm sạch cẩn thận bất kỳ đầu vào nào từ người dùng, chẳng hạn kiểm tra rằng một giá trị nằm trong một dải hợp lý và giới hạn kích thước của các chuỗi để ngăn việc từ chối dịch vụ thông qua các đợt cấp phát bộ nhớ lớn. Một dịch vụ nội bộ nằm sau một tường lửa có thể qua được với các kiểm tra đầu vào ít nghiêm ngặt hơn, nhưng việc kiểm tra tính hợp lý cơ bản của các giá trị (chẳng hạn trong lúc phân tích cú pháp giao thức [85]) là một ý hay.
 - Các client NTP có thể được cấu hình với nhiều địa chỉ máy chủ. Khi đồng bộ, client liên hệ với tất cả chúng, ước lượng sai số của chúng, và kiểm tra rằng một đa số các máy chủ đồng ý về một dải thời gian nào đó. Miễn là phần lớn các máy chủ đều ổn, thì một máy chủ NTP bị cấu hình sai đang báo về một thời gian không đúng sẽ bị phát

hiện như một giá trị ngoại lai và bị loại khỏi việc đồng bộ [37]. Việc dùng nhiều máy chủ làm cho NTP kiên cường hơn so với khi nó chỉ dùng một máy chủ duy nhất.

System Model and Reality — Mô hình hệ thống và hiện thực

Many algorithms have been designed to solve distributed systems problems—for example, we will examine solutions for the consensus problem in Chapter 9. In order to be useful, these algorithms need to tolerate the various faults of distributed systems that we discussed in this chapter.

Nhiều thuật toán đã được thiết kế để giải quyết các vấn đề về hệ thống phân tán — chẳng hạn, ta sẽ khảo sát các giải pháp cho bài toán đồng thuận ở Chương 9. Để hữu ích, các thuật toán này cần chịu được nhiều loại lỗi của các hệ thống phân tán mà ta đã bàn trong chương này.

Algorithms need to be written in a way that does not depend too heavily on the details of the hardware and software configuration on which they are run. This in turn requires that we somehow formalize the kinds of faults that we expect to happen in a system. We do this by defining a system model, which is an **abstraction** that describes what things an algorithm may assume.

Các thuật toán cần được viết theo cách không phụ thuộc quá nặng vào các chi tiết của cấu hình phần cứng và phần mềm mà chúng chạy trên đó. Điều này đến lượt nó đòi hỏi ta phải hình thức hóa theo cách nào đó các loại lỗi mà ta dự kiến sẽ xảy ra trong một hệ thống. Ta làm điều này bằng cách định nghĩa một mô hình hệ thống, vốn là một trừu tượng hóa mô tả những thứ mà một thuật toán có thể giả định.

With regard to timing assumptions, three system models are in common use:

Về các giả định liên quan tới định thời, ba mô hình hệ thống đang được dùng phổ biến:

The synchronous model assumes bounded network delay, bounded process pauses, and bounded clock error. This does not imply exactly synchronized clocks or zero network delay; it just means you know that network delay, pauses, and clock drift will never exceed some fixed upper bound [88]. The synchronous model is not a realistic model of most practical systems, because (as discussed in this chapter) unbounded delays and pauses do occur.

Mô hình đồng bộ giả định độ trễ mạng có giới hạn, các đợt tạm dừng tiến trình có giới hạn, và sai số đồng hồ có giới hạn. Điều này không ngụ ý các đồng hồ được đồng bộ chính xác hay độ trễ mạng bằng không; nó chỉ nghĩa là bạn biết rằng độ trễ mạng, các đợt tạm dừng, và độ trôi đồng hồ sẽ không bao giờ vượt quá một cận trên cố định nào đó [88]. Mô hình đồng bộ không phải là một mô hình thực tế cho hầu hết các hệ thống thực dụng, bởi (như đã bàn trong chương này) các độ trễ và các đợt tạm dừng không có giới hạn quả thực có xảy ra.

Partial synchrony means that a system behaves like a synchronous system most of the time, but it sometimes exceeds the bounds for network delay, process pauses, and clock drift [88]. This is a realistic model of many systems: most of the time, networks and processes are quite well behaved—otherwise we would never be able to get anything done—but we have to reckon with the fact that any timing assumptions may be shattered occasionally. When this happens, network delay, pauses, and clock error may become arbitrarily large.

Đồng bộ một phần nghĩa là một hệ thống hành xử như một hệ thống đồng bộ trong phần lớn thời gian, nhưng đôi khi nó vượt quá các giới hạn về độ trễ mạng, các đợt tạm dừng tiến trình, và độ trôi đồng hồ [88]. Đây là một mô hình thực tế của nhiều hệ thống: trong phần lớn thời gian, các mạng và các tiến trình hành xử khá tốt — nếu không thì ta sẽ không

bao giờ làm được việc gì — nhưng ta phải tính tới sự thật rằng bất kỳ giả định nào về định thời cũng có thể thỉnh thoảng bị phá vỡ. Khi điều này xảy ra, độ trễ mạng, các đợt tạm dừng, và sai số đồng hồ có thể trở nên lớn một cách tùy tiện.

In this model, an algorithm is not allowed to make any timing assumptions—in fact, it does not even have a clock (so it cannot use timeouts). Some algorithms can be designed for the asynchronous model, but it is very restrictive.

Trong mô hình này, một thuật toán không được phép đưa ra bất kỳ giả định nào về định thời — trên thực tế, nó thậm chí không có đồng hồ (nên nó không thể dùng các thời gian chờ). Một số thuật toán có thể được thiết kế cho mô hình bất đồng bộ, nhưng nó rất hạn chế.

Moreover, besides timing issues, we have to consider node failures. The three most common system models for nodes are:

Hơn nữa, ngoài các vấn đề về định thời, ta phải cân nhắc các hỏng hóc của nút. Ba mô hình hệ thống phổ biến nhất cho các nút là:

In the crash-stop model, an algorithm may assume that a node can fail in only one way, namely by crashing. This means that the node may suddenly stop responding at any moment, and thereafter that node is gone forever—it never comes back.

Trong mô hình sập-rời-dừng (crash-stop), một thuật toán có thể giả định rằng một nút chỉ có thể hỏng theo một cách duy nhất, cụ thể là bằng cách sập. Điều này nghĩa là nút có thể đột ngột ngừng phản hồi vào bất cứ lúc nào, và sau đó nút đó biến mất vĩnh viễn — nó không bao giờ quay lại.

We assume that nodes may crash at any moment, and perhaps start responding again after some unknown time. In the crash-recovery model, nodes are assumed to have stable storage (i.e., nonvolatile disk storage) that is preserved across crashes, while the in-memory state is assumed to be lost.

Ta giả định rằng các nút có thể sập vào bất cứ lúc nào, và có lẽ bắt đầu phản hồi trở lại sau một khoảng thời gian chưa biết. Trong mô hình sập-rời-phục-hồi (crash-recovery), các nút được giả định có kho lưu trữ ổn định (tức là kho lưu trữ đĩa không bay hơi, không mất dữ liệu khi mất điện) vốn được bảo tồn qua các lần sập, trong khi trạng thái trong bộ nhớ được giả định là bị mất.

Nodes may do absolutely anything, including trying to trick and deceive other nodes, as described in the last section.

Các nút có thể làm bất cứ điều gì, kể cả việc cố lừa và đánh lừa các nút khác, như đã mô tả ở phần trước.

For modeling real systems, the partially synchronous model with crash-recovery faults is generally the most useful model. But how do distributed algorithms cope with that model?

Để mô hình hóa các hệ thống thực, mô hình đồng bộ một phần với các lỗi sập-rời-phục-hồi nhìn chung là mô hình hữu ích nhất. Nhưng các thuật toán phân tán đối phó với mô hình đó như thế nào?

Correctness of an algorithm — Tính đúng đắn của một thuật toán

To define what it means for an algorithm to be correct, we can describe its properties. For example, the output of a sorting algorithm has the **property** that for any two distinct elements of the output

list, the element further to the left is smaller than the element further to the right. That is simply a formal way of defining what it means for a list to be sorted.

Để định nghĩa thế nào là một thuật toán đúng đắn, ta có thể mô tả các thuộc tính của nó. Chẳng hạn, đầu ra của một thuật toán sắp xếp có thuộc tính rằng với bất kỳ hai phần tử phân biệt nào của danh sách đầu ra, phần tử nằm xa hơn về bên trái thì nhỏ hơn phần tử nằm xa hơn về bên phải. Đó đơn giản là một cách hình thức để định nghĩa thế nào là một danh sách đã được sắp xếp.

Similarly, we can write down the properties we want of a distributed algorithm to define what it means to be correct. For example, if we are generating fencing tokens for a lock (see “Fencing tokens”), we may require the algorithm to have the following properties:

Tương tự, ta có thể viết ra các thuộc tính mà ta muốn ở một thuật toán phân tán để định nghĩa thế nào là đúng đắn. Chẳng hạn, nếu ta đang sinh các token rào chắn cho một khóa (xem “Token rào chắn”), ta có thể yêu cầu thuật toán có các thuộc tính sau:

No two requests for a fencing token return the same value.

Không có hai yêu cầu nào cho một token rào chắn trả về cùng một giá trị.

If request x returned token tx , and request y returned token ty , and x completed before y began, then $tx < ty$.

Nếu yêu cầu x trả về token tx , và yêu cầu y trả về token ty , và x hoàn tất trước khi y bắt đầu, thì $tx < ty$.

A node that requests a fencing token and does not crash eventually receives a response.

Một nút yêu cầu một token rào chắn và không sập thì rốt cuộc nhận được một phản hồi.

An algorithm is correct in some system model if it always satisfies its properties in all situations that we assume may occur in that system model. But how does this make sense? If all nodes crash, or all network delays suddenly become infinitely long, then no algorithm will be able to get anything done.

Một thuật toán là đúng đắn trong một mô hình hệ thống nào đó nếu nó luôn thỏa mãn các thuộc tính của nó trong mọi tình huống mà ta giả định có thể xảy ra trong mô hình hệ thống đó. Nhưng điều này có ý nghĩa gì? Nếu tất cả các nút đều sập, hoặc tất cả các độ trễ mạng đột nhiên trở nên dài vô hạn, thì không thuật toán nào có thể làm được việc gì.

Safety and liveness — An toàn và sống động

To clarify the situation, it is worth distinguishing between two different kinds of properties: safety and liveness properties. In the example just given, uniqueness and monotonic sequence are safety properties, but availability is a liveness property.

Để làm rõ tình hình, đáng để phân biệt giữa hai loại thuộc tính khác nhau: các thuộc tính an toàn (safety) và sống động (liveness). Trong ví dụ vừa nêu, tính duy nhất và dãy đơn điệu là các thuộc tính an toàn, nhưng tính sẵn sàng là một thuộc tính sống động.

What distinguishes the two kinds of properties? A giveaway is that liveness properties often include the word “eventually” in their definition. (And yes, you guessed it—eventual consistency is a liveness property [89].)

Điều gì phân biệt hai loại thuộc tính này? Một dấu hiệu nhận biết là các thuộc tính sống động thường bao gồm từ “rốt cuộc” trong định nghĩa của chúng. (Và đúng vậy, bạn đoán đúng rồi — nhất quán rốt cuộc (eventual consistency) là một thuộc tính sống động [89].)

Safety is often informally defined as nothing bad happens, and liveness as something good eventually happens. However, it's best to not read too much into those informal definitions, because the meaning of good and bad is subjective. The actual definitions of safety and liveness are precise and mathematical [90]:

An toàn thường được định nghĩa một cách không chính thức là không có điều gì xấu xảy ra, còn sống động là một điều tốt tốt cuộc sẽ xảy ra. Tuy nhiên, tốt nhất là không nên suy diễn quá nhiều từ những định nghĩa không chính thức đó, bởi ý nghĩa của tốt và xấu mang tính chủ quan. Các định nghĩa thực sự của an toàn và sống động là chính xác và mang tính toán học [90]:

- If a safety property is violated, we can point at a particular point in time at which it was broken (for example, if the uniqueness property was violated, we can identify the particular operation in which a duplicate fencing token was returned). After a safety property has been violated, the violation cannot be undone—the damage is already done.
- A liveness property works the other way round: it may not hold at some point in time (for example, a node may have sent a request but not yet received a response), but there is always hope that it may be satisfied in the future (namely by receiving a response).
 - Nếu một thuộc tính an toàn bị vi phạm, ta có thể chỉ ra một điểm thời gian cụ thể mà tại đó nó bị phá vỡ (chẳng hạn, nếu thuộc tính duy nhất bị vi phạm, ta có thể nhận diện thao tác cụ thể trong đó một token rào chắn trùng lặp được trả về). Sau khi một thuộc tính an toàn đã bị vi phạm, sự vi phạm không thể được hoàn tác — thiệt hại đã xảy ra rồi.
 - Một thuộc tính sống động thì hoạt động theo cách ngược lại: nó có thể không thỏa mãn tại một thời điểm nào đó (chẳng hạn, một nút có thể đã gửi một yêu cầu nhưng chưa nhận được phản hồi), nhưng luôn có hy vọng rằng nó có thể được thỏa mãn trong tương lai (cụ thể là bằng việc nhận được một phản hồi).

An advantage of distinguishing between safety and liveness properties is that it helps us deal with difficult system models. For distributed algorithms, it is common to require that safety properties always hold, in all possible situations of a system model [88]. That is, even if all nodes crash, or the entire network fails, the algorithm must nevertheless ensure that it does not return a wrong result (i.e., that the safety properties remain satisfied).

Một lợi thế của việc phân biệt giữa các thuộc tính an toàn và sống động là nó giúp ta đối phó với các mô hình hệ thống khó. Đối với các thuật toán phân tán, việc yêu cầu rằng các thuộc tính an toàn luôn được giữ vững, trong mọi tình huống có thể có của một mô hình hệ thống, là điều phổ biến [88]. Tức là, ngay cả khi tất cả các nút đều sập, hoặc toàn bộ mạng hỏng, thuật toán dù sao cũng phải bảo đảm rằng nó không trả về một kết quả sai (tức là các thuộc tính an toàn vẫn được thỏa mãn).

However, with liveness properties we are allowed to make caveats: for example, we could say that a request needs to receive a response only if a majority of nodes have not crashed, and only if the network eventually recovers from an **outage**. The definition of the partially synchronous model requires that eventually the system returns to a synchronous state—that is, any period of network interruption lasts only for a finite duration and is then repaired.

Tuy nhiên, với các thuộc tính sống động, ta được phép đặt ra các điều kiện kèm theo: chẳng hạn, ta có thể nói rằng một yêu cầu chỉ cần nhận được một phản hồi nếu một đa số các nút chưa sập, và chỉ nếu mạng rốt cuộc phục hồi sau một đợt gián đoạn. Định nghĩa của mô hình đồng bộ một phần đòi hỏi rằng rốt cuộc hệ thống quay lại một trạng thái đồng bộ — tức là, bất kỳ giai đoạn gián đoạn mạng nào cũng chỉ kéo dài trong một khoảng thời gian hữu hạn rồi được sửa chữa.

Mapping system models to the real world — Ảnh xạ các mô hình hệ thống vào thế giới thực

Safety and liveness properties and system models are very useful for reasoning about the correctness of a distributed algorithm. However, when implementing an algorithm in practice, the messy facts of reality come back to bite you again, and it becomes clear that the system model is a simplified abstraction of reality.

Các thuộc tính an toàn và sống động cùng các mô hình hệ thống là rất hữu ích để lập luận về tính đúng đắn của một thuật toán phân tán. Tuy nhiên, khi hiện thực một thuật toán trong thực tế, những sự thật hỗn độn của hiện thực lại quay về cắn bạn lần nữa, và rõ ràng là mô hình hệ thống chỉ là một trừu tượng hóa đã được đơn giản hóa của hiện thực.

For example, algorithms in the crash-recovery model generally assume that data in stable storage survives crashes. However, what happens if the data on disk is corrupted, or the data is wiped out due to hardware error or misconfiguration [91]? What happens if a server has a firmware bug and fails to recognize its hard drives on reboot, even though the drives are correctly attached to the server [92]?

Chẳng hạn, các thuật toán trong mô hình sập-rời-phục-hồi nhìn chung giả định rằng dữ liệu trong kho lưu trữ ổn định sống sót qua các lần sập. Tuy nhiên, điều gì xảy ra nếu dữ liệu trên đĩa bị hỏng, hoặc dữ liệu bị xóa sạch do lỗi phần cứng hoặc cấu hình sai [91]? Điều gì xảy ra nếu một máy chủ có một bug firmware và không nhận ra các ổ cứng của nó khi khởi động lại, ngay cả khi các ổ đĩa được gắn đúng vào máy chủ [92]?

Quorum algorithms (see “Quorums for reading and writing”) rely on a node remembering the data that it claims to have stored. If a node may suffer from amnesia and forget previously stored data, that breaks the quorum condition, and thus breaks the correctness of the algorithm. Perhaps a new system model is needed, in which we assume that stable storage mostly survives crashes, but may sometimes be lost. But that model then becomes harder to reason about.

Các thuật toán số đại biểu (xem “Số đại biểu để đọc và ghi”) dựa vào việc một nút nhớ được dữ liệu mà nó tuyên bố đã lưu. Nếu một nút có thể mắc chứng quên và quên đi dữ liệu đã được lưu trước đó, điều đó phá vỡ điều kiện số đại biểu, và do đó phá vỡ tính đúng đắn của thuật toán. Có lẽ cần một mô hình hệ thống mới, trong đó ta giả định rằng kho lưu trữ ổn định hầu như sống sót qua các lần sập, nhưng đôi khi có thể bị mất. Nhưng mô hình đó khi đó lại trở nên khó lập luận hơn.

The theoretical description of an algorithm can declare that certain things are simply assumed not to happen—and in non-Byzantine systems, we do have to make some assumptions about faults that can and cannot happen. However, a real implementation may still have to include code to handle the case where something happens that was assumed to be impossible, even if that handling boils down to `printf(“Sucks to be you”)` and `exit(666)`—i.e., letting a human operator clean up the mess [93]. (This is arguably the difference between computer science and software engineering.)

Mô tả lý thuyết của một thuật toán có thể tuyên bố rằng một số thứ đơn giản được giả định là không xảy ra — và trong các hệ thống phi-Byzantine, ta quả thực phải đặt ra một số giả định về những lỗi có thể và không thể xảy ra. Tuy nhiên, một hiện thực thực sự vẫn có thể phải bao gồm mã để xử lý trường hợp một điều gì đó được giả định là không thể lại xảy ra, ngay cả khi việc xử lý đó rốt cuộc chỉ là `printf(“Sucks to be you”)` và `exit(666)` — tức là để một người vận hành dọn dẹp mớ hỗn độn [93]. (Có thể lập luận rằng đây là sự khác biệt giữa khoa học máy tính và kỹ nghệ phần mềm.)

That is not to say that theoretical, abstract system models are worthless—quite the opposite. They are incredibly helpful for distilling down the complexity of real systems to a manageable set of faults that

we can reason about, so that we can understand the problem and try to solve it systematically. We can prove algorithms correct by showing that their properties always hold in some system model.

Điều đó không có nghĩa là các mô hình hệ thống lý thuyết, trừu tượng là vô giá trị — hoàn toàn ngược lại. Chúng cực kỳ hữu ích để chứng cất sự phức tạp của các hệ thống thực xuống thành một tập các lỗi có thể quản lý được mà ta có thể lập luận, để ta có thể hiểu vấn đề và cố giải quyết nó một cách có hệ thống. Ta có thể chứng minh các thuật toán là đúng đắn bằng cách cho thấy rằng các thuộc tính của chúng luôn được giữ vững trong một mô hình hệ thống nào đó.

Proving an algorithm correct does not mean its implementation on a real system will necessarily always behave correctly. But it's a very good first step, because the theoretical analysis can uncover problems in an algorithm that might remain hidden for a long time in a real system, and that only come to bite you when your assumptions (e.g., about timing) are defeated due to unusual circumstances. Theoretical analysis and empirical testing are equally important.

Việc chứng minh một thuật toán là đúng đắn không có nghĩa là hiện thực của nó trên một hệ thống thực sẽ nhất thiết luôn hành xử đúng. Nhưng đó là một bước đầu rất tốt, bởi việc phân tích lý thuyết có thể phát hiện ra các vấn đề trong một thuật toán mà có thể vẫn ẩn giấu trong một thời gian dài trong một hệ thống thực, và chỉ cắn bạn khi các giả định của bạn (chẳng hạn về định thời) bị đánh bại do những hoàn cảnh bất thường. Phân tích lý thuyết và kiểm thử thực nghiệm đều quan trọng như nhau.

Summary — Tóm tắt

In this chapter we have discussed a wide range of problems that can occur in distributed systems, including:

Trong chương này, ta đã bàn về một dải rộng các vấn đề có thể xảy ra trong các hệ thống phân tán, bao gồm:

- Whenever you try to send a packet over the network, it may be lost or arbitrarily delayed. Likewise, the reply may be lost or delayed, so if you don't get a reply, you have no idea whether the message got through.
- A node's clock may be significantly out of sync with other nodes (despite your best efforts to set up NTP), it may suddenly jump forward or back in time, and relying on it is dangerous because you most likely don't have a good measure of your clock's error interval.
- A process may pause for a substantial amount of time at any point in its execution (perhaps due to a stop-the-world garbage collector), be declared dead by other nodes, and then come back to life again without realizing that it was paused.
 - Bất cứ khi nào bạn cố gửi một gói qua mạng, nó có thể bị mất hoặc bị trễ một cách tùy tiện. Tương tự, phản hồi cũng có thể bị mất hoặc bị trễ, nên nếu bạn không nhận được phản hồi, bạn không hề biết liệu thông điệp có đến được hay không.
 - Đồng hồ của một nút có thể lệch đáng kể so với các nút khác (bất chấp những nỗ lực tốt nhất của bạn để thiết lập NTP), nó có thể đột nhiên nhảy tới hoặc lùi về thời gian trước, và việc trông cậy vào nó là nguy hiểm bởi bạn nhiều khả năng không có một thước đo tốt cho khoảng sai số của đồng hồ của mình.
 - Một tiến trình có thể tạm dừng trong một khoảng thời gian đáng kể tại bất kỳ điểm nào trong quá trình thực thi của nó (có lẽ do một bộ thu gom rác dừng-cả-thế-giới), bị các nút khác tuyên bố là đã chết, rồi sống lại mà không nhận ra rằng nó đã bị tạm dừng.

The fact that such partial failures can occur is the defining characteristic of distributed systems. Whenever software tries to do anything involving other nodes, there is the possibility that it may occasionally fail, or randomly go slow, or not respond at all (and eventually time out). In distributed systems, we try to build tolerance of partial failures into software, so that the system as a whole may continue functioning even when some of its constituent parts are broken.

Sự thật rằng các hỏng hóc cục bộ như vậy có thể xảy ra chính là đặc điểm định nghĩa của các hệ thống phân tán. Bất cứ khi nào phần mềm cố làm bất cứ điều gì liên quan tới các nút khác, đều có khả năng nó thỉnh thoảng có thể hỏng, hoặc ngẫu nhiên chạy chậm, hoặc không phản hồi gì cả (và rồi cuộc hết thời gian chờ). Trong các hệ thống phân tán, ta cố xây dựng khả năng chịu các hỏng hóc cục bộ vào trong phần mềm, để hệ thống như một tổng thể có thể tiếp tục hoạt động ngay cả khi một số bộ phận cấu thành của nó bị hỏng.

To tolerate faults, the first step is to detect them, but even that is hard. Most systems don't have an

accurate mechanism of detecting whether a node has failed, so most distributed algorithms rely on timeouts to determine whether a remote node is still available. However, timeouts can't distinguish between network and node failures, and variable network delay sometimes causes a node to be falsely suspected of crashing. Moreover, sometimes a node can be in a degraded state: for example, a Gigabit network interface could suddenly drop to 1 Kb/s throughput due to a driver bug [94]. Such a node that is “limping” but not dead can be even more difficult to deal with than a cleanly failed node.

Để chịu được lỗi, bước đầu tiên là phát hiện chúng, nhưng ngay cả điều đó cũng khó. Hầu hết các hệ thống không có một cơ chế chính xác để phát hiện liệu một nút đã hỏng hay chưa, nên hầu hết các thuật toán phân tán dựa vào các thời gian chờ để xác định liệu một nút từ xa có còn sẵn sàng hay không. Tuy nhiên, các thời gian chờ không thể phân biệt giữa các sự cố mạng và các sự cố nút, và độ trễ mạng biến động đôi khi khiến một nút bị nghi ngờ một cách sai lầm là đã sập. Hơn nữa, đôi khi một nút có thể ở trong một trạng thái suy giảm: chẳng hạn, một giao diện mạng Gigabit có thể đột nhiên tụt xuống thông lượng 1 Kb/s do một bug trình điều khiển [94]. Một nút như vậy đang “khập khiễng” nhưng chưa chết có thể còn khó đối phó hơn cả một nút đã hỏng một cách dứt khoát.

Once a fault is detected, making a system tolerate it is not easy either: there is no global variable, no shared memory, no common knowledge or any other kind of shared state between the machines. Nodes can't even agree on what time it is, let alone on anything more profound. The only way information can flow from one node to another is by sending it over the unreliable network. Major decisions cannot be safely made by a single node, so we require protocols that enlist help from other nodes and try to get a quorum to agree.

Một khi một lỗi được phát hiện, việc làm cho một hệ thống chịu được nó cũng không hề dễ: không có biến toàn cục, không có bộ nhớ chia sẻ, không có tri thức chung hay bất kỳ loại trạng thái chia sẻ nào khác giữa các máy. Các nút thậm chí không thể thống nhất bây giờ là mấy giờ, chứ đừng nói tới những điều sâu sắc hơn. Cách duy nhất thông tin có thể chảy từ một nút này sang nút khác là bằng cách gửi nó qua mạng không đáng tin cậy. Các quyết định lớn không thể được đưa ra một cách an toàn bởi một nút đơn lẻ, nên ta cần các giao thức huy động sự giúp đỡ từ các nút khác và cố để một số đại biểu thống nhất.

If you're used to writing software in the idealized mathematical perfection of a single computer, where the same operation always deterministically returns the same result, then moving to the messy physical reality of distributed systems can be a bit of a shock. Conversely, distributed systems engineers will often regard a problem as trivial if it can be solved on a single computer [5], and indeed a single computer can do a lot nowadays [95]. If you can avoid opening Pandora's box and simply keep things on a single machine, it is generally worth doing so.

Nếu bạn đã quen với việc viết phần mềm trong sự hoàn hảo toán học lý tưởng hóa của một máy tính đơn lẻ, nơi cùng một thao tác luôn trả về cùng một kết quả một cách tất định, thì việc chuyển sang hiện thực vật lý hỗn độn của các hệ thống phân tán có thể là một cú sốc nho nhỏ. Ngược lại, các kỹ sư hệ thống phân tán thường sẽ coi một vấn đề là tầm thường nếu nó có thể được giải quyết trên một máy tính đơn lẻ [5], và quả thực một máy tính đơn lẻ có thể làm được rất nhiều việc ngày nay [95]. Nếu bạn có thể tránh việc mở chiếc hộp Pandora và đơn giản giữ mọi thứ trên một máy đơn lẻ, thì nhìn chung điều đó đáng để làm.

However, as discussed in the introduction to Part II, **scalability** is not the only reason for wanting to use a distributed system. Fault tolerance and low latency (by placing data geographically close to users) are equally important goals, and those things cannot be achieved with a single node.

Tuy nhiên, như đã bàn trong phần mở đầu của Phần II, khả năng mở rộng không phải là lý

do duy nhất để muốn dùng một hệ thống phân tán. Khả năng chịu lỗi và độ trễ thấp (bằng cách đặt dữ liệu ở gần người dùng về mặt địa lý) là những mục tiêu quan trọng không kém, và những thứ đó không thể đạt được với một nút đơn lẻ.

In this chapter we also went on some tangents to explore whether the unreliability of networks, clocks, and processes is an inevitable law of nature. We saw that it isn't: it is possible to give hard real-time response guarantees and bounded delays in networks, but doing so is very expensive and results in lower utilization of hardware resources. Most non-safety-critical systems choose cheap and unreliable over expensive and reliable.

Trong chương này ta cũng đã đi lạc đề một chút để khám phá xem liệu sự không đáng tin cậy của các mạng, các đồng hồ, và các tiến trình có phải là một quy luật không thể tránh khỏi của tự nhiên hay không. Ta đã thấy rằng không phải vậy: có thể đưa ra các bảo đảm phản hồi thời gian thực cứng và các độ trễ có giới hạn trong các mạng, nhưng làm vậy rất tốn kém và dẫn tới mức tận dụng tài nguyên phần cứng thấp hơn. Hầu hết các hệ thống không quan trọng về an toàn chọn rẻ và không đáng tin cậy thay vì đắt và đáng tin cậy.

We also touched on supercomputers, which assume reliable components and thus have to be stopped and restarted entirely when a component does fail. By contrast, distributed systems can run forever without being interrupted at the service level, because all faults and maintenance can be handled at the node level—at least in theory. (In practice, if a bad configuration change is rolled out to all nodes, that will still bring a distributed system to its knees.)

Ta cũng đã chạm tới các siêu máy tính, vốn giả định các thành phần đáng tin cậy và do đó phải bị dừng và khởi động lại hoàn toàn khi một thành phần quả thực hỏng. Ngược lại, các hệ thống phân tán có thể chạy mãi mãi mà không bị gián đoạn ở mức dịch vụ, bởi tất cả các lỗi và việc bảo trì đều có thể được xử lý ở mức nút — ít nhất là về lý thuyết. (Trong thực tế, nếu một thay đổi cấu hình tồi tệ được tung ra tới tất cả các nút, điều đó vẫn sẽ khiến một hệ thống phân tán quỵ gối.)

This chapter has been all about problems, and has given us a bleak outlook. In the next chapter we will move on to solutions, and discuss some algorithms that have been designed to cope with all the problems in distributed systems.

Chương này hoàn toàn nói về các vấn đề, và đã cho ta một viễn cảnh ảm đạm. Ở chương tiếp theo, ta sẽ chuyển sang các giải pháp, và bàn về một số thuật toán đã được thiết kế để đối phó với tất cả các vấn đề trong các hệ thống phân tán.

Footnotes With one exception: we will assume that faults are non-Byzantine (see “Byzantine Faults”).

Except perhaps for an occasional keepalive packet, if TCP keepalive is enabled.

Asynchronous Transfer Mode (ATM) was a competitor to Ethernet in the 1980s [32], but it didn't gain much adoption outside of telephone network core switches. It has nothing to do with automatic teller machines (also known as cash machines), despite sharing an acronym. Perhaps, in some parallel universe, the internet is based on something like ATM—in that universe, internet video calls are probably a lot more reliable than they are in ours, because they don't suffer from dropped and delayed packets.

Peering agreements between internet service providers and the establishment of routes through the Border Gateway Protocol (BGP), bear closer resemblance to circuit switching than IP itself. At this level, it is possible to buy dedicated bandwidth. However, internet routing operates at the level of networks, not individual connections between hosts, and at a much longer timescale.

Although the clock is called real-time, it has nothing to do with real-time operating systems, as discussed in “Response time guarantees”.

There are distributed sequence number generators, such as Twitter’s Snowflake, that generate approximately monotonically increasing unique IDs in a scalable way (e.g., by allocating blocks of the ID space to different nodes). However, they typically cannot guarantee an ordering that is consistent with causality, because the timescale at which blocks of IDs are assigned is longer than the timescale of database reads and writes. See also “Ordering Guarantees”.

References [1] Mark Cavage: “There’s Just No Getting Around It: You’re Building a Distributed System,” *ACM Queue*, volume 11, number 4, pages 80-89, April 2013. doi:10.1145/2466486.2482856

[2] Jay Kreps: “Getting Real About Distributed System Reliability,” *blog.empathybox.com*, March 19, 2012.

[3] Sydney Padua: *The Thrilling Adventures of Lovelace and Babbage: The (Mostly) True Story of the First Computer*. Particular Books, April 2015. ISBN: 978-0-141-98151-2

[4] Coda Hale: “You Can’t Sacrifice Partition Tolerance,” *codahale.com*, October 7, 2010.

[5] Jeff Hodges: “Notes on Distributed Systems for Young Bloods,” *somethingsimilar.com*, January 14, 2013.

[6] Antonio Regalado: “Who Coined ‘Cloud Computing’?,” *technologyreview.com*, October 31, 2011.

[7] Luiz André Barroso, Jimmy Clidaras, and Urs Hölzle: “The Datacenter as a Computer: An Introduction to the Design of Warehouse-Scale Machines, Second Edition,” *Synthesis Lectures on Computer Architecture*, volume 8, number 3, Morgan & Claypool Publishers, July 2013. doi:10.2200/S00516ED2V01Y201306CAC024, ISBN: 978-1-627-05010-4

[8] David Fiala, Frank Mueller, Christian Engelmann, et al.: “Detection and Correction of Silent Data Corruption for Large-Scale High-Performance Computing,” at *International Conference for High Performance Computing, Networking, Storage and Analysis (SC12)*, November 2012.

[9] Arjun Singh, Joon Ong, Amit Agarwal, et al.: “Jupiter Rising: A Decade of Clos Topologies and Centralized Control in Google’s Datacenter Network,” at *Annual Conference of the ACM Special Interest Group on Data Communication (SIGCOMM)*, August 2015. doi:10.1145/2785956.2787508

[10] Glenn K. Lockwood: “Hadoop’s Uncomfortable Fit in HPC,” *glennklockwood.blogspot.co.uk*, May 16, 2014.

[11] John von Neumann: “Probabilistic Logics and the Synthesis of Reliable Organisms from Unreliable Components,” in *Automata Studies (AM-34)*, edited by Claude E. Shannon and John McCarthy, Princeton University Press, 1956. ISBN: 978-0-691-07916-5

[12] Richard W. Hamming: *The Art of Doing Science and Engineering*. Taylor & Francis, 1997. ISBN: 978-9-056-99500-3

[13] Claude E. Shannon: “A Mathematical Theory of Communication,” *The Bell System Technical Journal*, volume 27, number 3, pages 379–423 and 623–656, July 1948.

[14] Peter Bailis and Kyle Kingsbury: “The Network Is Reliable,” *ACM Queue*, volume 12, number 7, pages 48-55, July 2014. doi:10.1145/2639988.2639988

[15] Joshua B. Leners, Trinabh Gupta, Marcos K. Aguilera, and Michael Walfish: “Taming Uncertainty in Distributed Systems with Help from the Network,” at *10th European Conference on Computer Systems (EuroSys)*, April 2015. doi:10.1145/2741948.2741976

- [16] Phillipa Gill, Navendu Jain, and Nachiappan Nagappan: “Understanding Network Failures in Data Centers: Measurement, Analysis, and Implications,” at ACM SIGCOMM Conference, August 2011. doi:10.1145/2018436.2018477
- [17] Mark Imbriaco: “Downtime Last Saturday,” github.com, December 26, 2012.
- [18] Will Oremus: “The Global Internet Is Being Attacked by Sharks, Google Confirms,” slate.com, August 15, 2014.
- [19] Marc A. Donges: “Re: bnx2 cards Intermittantly Going Offline,” Message to Linux netdev mailing list, spinics.net, September 13, 2012.
- [20] Kyle Kingsbury: “Call Me Maybe: Elasticsearch,” aphyr.com, June 15, 2014.
- [21] Salvatore Sanfilippo: “A Few Arguments About Redis Sentinel Properties and Fail Scenarios,” antirez.com, October 21, 2014.
- [22] Bert Hubert: “The Ultimate SO_LINGER Page, or: Why Is My TCP Not Reliable,” blog.netherlabs.nl, January 18, 2009.
- [23] Nicolas Liochon: “CAP: If All You Have Is a Timeout, Everything Looks Like a Partition,” blog.thislongrun.com, May 25, 2015.
- [24] Jerome H. Saltzer, David P. Reed, and David D. Clark: “End-To-End Arguments in System Design,” ACM Transactions on Computer Systems, volume 2, number 4, pages 277–288, November 1984. doi:10.1145/357401.357402
- [25] Matthew P. Grosvenor, Malte Schwarzkopf, Ionel Gog, et al.: “Queues Don’t Matter When You Can JUMP Them!,” at 12th USENIX Symposium on Networked Systems Design and Implementation (NSDI), May 2015.
- [26] Guohui Wang and T. S. Eugene Ng: “The Impact of Virtualization on Network Performance of Amazon EC2 Data Center,” at 29th IEEE International Conference on Computer Communications (INFOCOM), March 2010. doi:10.1109/INFOCOM.2010.5461931
- [27] Van Jacobson: “Congestion Avoidance and Control,” at ACM Symposium on Communications Architectures and Protocols (SIGCOMM), August 1988. doi:10.1145/52324.52356
- [28] Brandon Philips: “etcd: Distributed Locking and Service Discovery,” at Strange Loop, September 2014.
- [29] Steve Newman: “A Systematic Look at EC2 I/O,” blog.scalyr.com, October 16, 2012.
- [30] Naohiro Hayashibara, Xavier Défago, Rami Yared, and Takuya Katayama: “The ϕ Accrual Failure Detector,” Japan Advanced Institute of Science and Technology, School of Information Science, Technical Report IS-RR-2004-010, May 2004.
- [31] Jeffrey Wang: “Phi Accrual Failure Detector,” ternarysearch.blogspot.co.uk, August 11, 2013.
- [32] Srinivasan Keshav: An Engineering Approach to Computer Networking: ATM Networks, the Internet, and the Telephone Network. Addison-Wesley Professional, May 1997. ISBN: 978-0-201-63442-6
- [33] Cisco, “Integrated Services Digital Network,” docwiki.cisco.com.
- [34] Othmar Kyas: ATM Networks. International Thomson Publishing, 1995. ISBN: 978-1-850-32128-6
- [35] “InfiniBand FAQ,” Mellanox Technologies, December 22, 2014.
- [36] Jose Renato Santos, Yoshio Turner, and G. (John) Janakiraman: “End-to-End Congestion Control for InfiniBand,” at 22nd Annual Joint Conference of the IEEE Computer and Communications

- Societies (INFOCOM), April 2003. Also published by HP Laboratories Palo Alto, Tech Report HPL-2002-359. doi:10.1109/INFOCOM.2003.1208949
- [37] Ulrich Windl, David Dalton, Marc Martinec, and Dale R. Worley: “The NTP FAQ and HOWTO,” ntp.org, November 2006.
- [38] John Graham-Cumming: “How and why the leap second affected Cloudflare DNS,” blog.cloudflare.com, January 1, 2017.
- [39] David Holmes: “Inside the Hotspot VM: Clocks, Timers and Scheduling Events – Part I – Windows,” blogs.oracle.com, October 2, 2006.
- [40] Steve Loughran: “Time on Multi-Core, Multi-Socket Servers,” steveloughran.blogspot.co.uk, September 17, 2015.
- [41] James C. Corbett, Jeffrey Dean, Michael Epstein, et al.: “Spanner: Google’s Globally-Distributed Database,” at 10th USENIX Symposium on Operating System Design and Implementation (OSDI), October 2012.
- [42] M. Caporali and R. Ambrosini: “How Closely Can a Personal Computer Clock Track the UTC Timescale Via the Internet?,” European Journal of Physics, volume 23, number 4, pages L17–L21, June 2012. doi:10.1088/0143-0807/23/4/103
- [43] Nelson Minar: “A Survey of the NTP Network,” alumni.media.mit.edu, December 1999.
- [44] Viliam Holub: “Synchronizing Clocks in a Cassandra Cluster Pt. 1 – The Problem,” blog.logentries.com, March 14, 2014.
- [45] Poul-Henning Kamp: “The One-Second War (What Time Will You Die?),” ACM Queue, volume 9, number 4, pages 44–48, April 2011. doi:10.1145/1966989.1967009
- [46] Nelson Minar: “Leap Second Crashes Half the Internet,” somebits.com, July 3, 2012.
- [47] Christopher Pascoe: “Time, Technology and Leaping Seconds,” googleblog.blogspot.co.uk, September 15, 2011.
- [48] Mingxue Zhao and Jeff Barr: “Look Before You Leap – The Coming Leap Second and AWS,” aws.amazon.com, May 18, 2015.
- [49] Darryl Veitch and Kanthaiah Vijayalayan: “Network Timing and the 2015 Leap Second,” at 17th International Conference on Passive and Active Measurement (PAM), April 2016. doi:10.1007/978-3-319-30505-9_29
- [50] “Timekeeping in VMware Virtual Machines,” Information Guide, VMware, Inc., December 2011.
- [51] “MiFID II / MiFIR: Regulatory Technical and Implementing Standards – Annex I (Draft),” European Securities and Markets Authority, Report ESMA/2015/1464, September 2015.
- [52] Luke Bigum: “Solving MiFID II Clock Synchronisation With Minimum Spend (Part 1),” lmax.com, November 27, 2015.
- [53] Kyle Kingsbury: “Call Me Maybe: Cassandra,” aphyr.com, September 24, 2013.
- [54] John Daily: “Clocks Are Bad, or, Welcome to the Wonderful World of Distributed Systems,” basho.com, November 12, 2013.
- [55] Kyle Kingsbury: “The Trouble with Timestamps,” aphyr.com, October 12, 2013.
- [56] Leslie Lamport: “Time, Clocks, and the Ordering of Events in a Distributed System,” Communications of the ACM, volume 21, number 7, pages 558–565, July 1978. doi:10.1145/359545.359563

- [57] Sandeep Kulkarni, Murat Demirbas, Deepak Madeppa, et al.: “Logical Physical Clocks and Consistent Snapshots in Globally Distributed Databases,” State University of New York at Buffalo, Computer Science and Engineering Technical Report 2014-04, May 2014.
- [58] Justin Sheehy: “There Is No Now: Problems With Simultaneity in Distributed Systems,” ACM Queue, volume 13, number 3, pages 36–41, March 2015. doi:10.1145/2733108
- [59] Murat Demirbas: “Spanner: Google’s Globally-Distributed Database,” muratbuffalo.blogspot.co.uk, July 4, 2013.
- [60] Dahlia Malkhi and Jean-Philippe Martin: “Spanner’s Concurrency Control,” ACM SIGACT News, volume 44, number 3, pages 73–77, September 2013. doi:10.1145/2527748.2527767
- [61] Manuel Bravo, Nuno Diegues, Jingna Zeng, et al.: “On the Use of Clocks to Enforce Consistency in the Cloud,” IEEE Data Engineering Bulletin, volume 38, number 1, pages 18–31, March 2015.
- [62] Spencer Kimball: “Living Without Atomic Clocks,” cockroachlabs.com, February 17, 2016.
- [63] Cary G. Gray and David R. Cheriton: “Leases: An Efficient Fault-Tolerant Mechanism for Distributed File Cache Consistency,” at 12th ACM Symposium on Operating Systems Principles (SOSP), December 1989. doi:10.1145/74850.74870
- [64] Todd Lipcon: “Avoiding Full GCs in Apache HBase with MemStore-Local Allocation Buffers: Part 1,” blog.cloudera.com, February 24, 2011.
- [65] Martin Thompson: “Java Garbage Collection Distilled,” mechanical-sympathy.blogspot.co.uk, July 16, 2013.
- [66] Alexey Ragozin: “How to Tame Java GC Pauses? Surviving 16GiB Heap and Greater,” java.dzone.com, June 28, 2011.
- [67] Christopher Clark, Keir Fraser, Steven Hand, et al.: “Live Migration of Virtual Machines,” at 2nd USENIX Symposium on Symposium on Networked Systems Design & Implementation (NSDI), May 2005.
- [68] Mike Shaver: “fsyncers and Curveballs,” shaver.off.net, May 25, 2008.
- [69] Zhenyun Zhuang and Cuong Tran: “Eliminating Large JVM GC Pauses Caused by Background IO Traffic,” engineering.linkedin.com, February 10, 2016.
- [70] David Terei and Amit Levy: “Blade: A Data Center Garbage Collector,” arXiv:1504.02578, April 13, 2015.
- [71] Martin Maas, Tim Harris, Krste Asanović, and John Kubiawicz: “Trash Day: Coordinating Garbage Collection in Distributed Systems,” at 15th USENIX Workshop on Hot Topics in Operating Systems (HotOS), May 2015.
- [72] “Predictable Low Latency,” Cinnober Financial Technology AB, cinnober.com, November 24, 2013.
- [73] Martin Fowler: “The LMAX Architecture,” martinfowler.com, July 12, 2011.
- [74] Flavio P. Junqueira and Benjamin Reed: ZooKeeper: Distributed Process Coordination. O’Reilly Media, 2013. ISBN: 978-1-449-36130-3
- [75] Enis Söztutar: “HBase and HDFS: Understanding Filesystem Usage in HBase,” at HBaseCon, June 2013.
- [76] Caitie McCaffrey: “Clients Are Jerks: AKA How Halo 4 DoSed the Services at Launch & How We Survived,” caitiem.com, June 23, 2015.

- [77] Leslie Lamport, Robert Shostak, and Marshall Pease: “The Byzantine Generals Problem,” *ACM Transactions on Programming Languages and Systems (TOPLAS)*, volume 4, number 3, pages 382–401, July 1982. doi:10.1145/357172.357176
- [78] Jim N. Gray: “Notes on Data Base Operating Systems,” in *Operating Systems: An Advanced Course, Lecture Notes in Computer Science*, volume 60, edited by R. Bayer, R. M. Graham, and G. Seegmüller, pages 393–481, Springer-Verlag, 1978. ISBN: 978-3-540-08755-7
- [79] Brian Palmer: “How Complicated Was the Byzantine Empire?,” *slate.com*, October 20, 2011.
- [80] Leslie Lamport: “My Writings,” *research.microsoft.com*, December 16, 2014. This page can be found by searching the web for the 23-character string obtained by removing the hyphens from the string `allamport-spubso-ntheweb`.
- [81] John Rushby: “Bus Architectures for Safety-Critical Embedded Systems,” at 1st International Workshop on Embedded Software (EMSOFT), October 2001.
- [82] Jake Edge: “ELC: SpaceX Lessons Learned,” *lwn.net*, March 6, 2013.
- [83] Andrew Miller and Joseph J. LaViola, Jr.: “Anonymous Byzantine Consensus from Moderately-Hard Puzzles: A Model for Bitcoin,” University of Central Florida, Technical Report CS-TR-14-01, April 2014.
- [84] James Mickens: “The Saddest Moment,” *USENIX ;login: logout*, May 2013.
- [85] Evan Gilman: “The Discovery of Apache ZooKeeper’s Poison Packet,” *pagerduty.com*, May 7, 2015.
- [86] Jonathan Stone and Craig Partridge: “When the CRC and TCP Checksum Disagree,” at ACM Conference on Applications, Technologies, Architectures, and Protocols for Computer Communication (SIGCOMM), August 2000. doi:10.1145/347059.347561
- [87] Evan Jones: “How Both TCP and Ethernet Checksums Fail,” *evanjones.ca*, October 5, 2015.
- [88] Cynthia Dwork, Nancy Lynch, and Larry Stockmeyer: “Consensus in the Presence of Partial Synchrony,” *Journal of the ACM*, volume 35, number 2, pages 288–323, April 1988. doi:10.1145/42282.42283
- [89] Peter Bailis and Ali Ghodsi: “Eventual Consistency Today: Limitations, Extensions, and Beyond,” *ACM Queue*, volume 11, number 3, pages 55–63, March 2013. doi:10.1145/2460276.2462076
- [90] Bowen Alpern and Fred B. Schneider: “Defining Liveness,” *Information Processing Letters*, volume 21, number 4, pages 181–185, October 1985. doi:10.1016/0020-0190(85)90056-0
- [91] Flavio P. Junqueira: “Dude, Where’s My Metadata?,” *fpj.me*, May 28, 2015.
- [92] Scott Sanders: “January 28th Incident Report,” *github.com*, February 3, 2016.
- [93] Jay Kreps: “A Few Notes on Kafka and Jepsen,” *blog.empathybox.com*, September 25, 2013.
- [94] Thanh Do, Mingzhe Hao, Tanakorn Leesatapornwongsa, et al.: “Limplock: Understanding the Impact of Limpware on Scale-out Cloud Systems,” at 4th ACM Symposium on Cloud Computing (SoCC), October 2013. doi:10.1145/2523616.2523627
- [95] Frank McSherry, Michael Isard, and Derek G. Murray: “Scalability! But at What COST?,” at 15th USENIX Workshop on Hot Topics in Operating Systems (HotOS), May 2015.

Chapter 9. Consistency and Consensus — Chương 9. Tính nhất quán và Đồng thuận

Is it better to be alive and wrong or right and dead?

Sống mà sai thì tốt hơn, hay đúng mà chết thì tốt hơn?

Jay Kreps, A Few Notes on Kafka and Jepsen (2013)

Jay Kreps, A Few Notes on Kafka and Jepsen (2013)

Lots of things can go wrong in distributed systems, as discussed in Chapter 8. The simplest way of handling such faults is to simply let the entire service fail, and show the user an error message. If that solution is unacceptable, we need to find ways of tolerating faults—that is, of keeping the service functioning correctly, even if some internal component is faulty.

Có rất nhiều thứ có thể trục trặc trong các hệ thống phân tán, như đã bàn ở Chương 8. Cách đơn giản nhất để xử lý những lỗi (fault) như vậy là cứ để toàn bộ dịch vụ hỏng hóc và hiển thị một thông báo lỗi cho người dùng. Nếu giải pháp đó không thể chấp nhận được, ta cần tìm cách chịu lỗi (fault-tolerant) — tức là giữ cho dịch vụ vẫn hoạt động đúng đắn, ngay cả khi một thành phần nội bộ nào đó gặp trục trặc.

In this chapter, we will talk about some examples of algorithms and protocols for building **fault-tolerant** distributed systems. We will assume that all the problems from Chapter 8 can occur: packets can be lost, reordered, duplicated, or arbitrarily delayed in the network; clocks are approximate at best; and nodes can pause (e.g., due to **garbage collection**) or crash at any time.

Trong chương này, ta sẽ bàn về một số ví dụ về các thuật toán và giao thức để xây dựng những hệ thống phân tán chịu lỗi. Ta sẽ giả định rằng mọi vấn đề trong Chương 8 đều có thể xảy ra: các gói tin có thể bị mất, đảo thứ tự, nhân bản, hoặc trì hoãn tùy ý trên mạng; đồng hồ ở mức tốt nhất cũng chỉ là gần đúng; và các nút (node) có thể tạm dừng (chẳng hạn do thu gom rác) hoặc gặp sự cố bất cứ lúc nào.

The best way of building **fault-tolerant** systems is to find some general-purpose abstractions with useful guarantees, implement them once, and then let applications rely on those guarantees. This is the same approach as we used with transactions in Chapter 7: by using a transaction, the application can pretend that there are no crashes (atomicity), that nobody else is concurrently accessing the **database** (isolation), and that storage devices are perfectly reliable (**durability**). Even though crashes, race conditions, and disk failures do occur, the transaction **abstraction** hides those problems so that the application doesn't need to worry about them.

Cách tốt nhất để xây dựng các hệ thống chịu lỗi là tìm ra một vài trừu tượng hóa đa dụng kèm những bảo đảm hữu ích, hiện thực chúng một lần, rồi để các ứng dụng dựa vào những bảo đảm đó. Đây cũng chính là cách tiếp cận mà ta đã dùng với giao dịch ở Chương 7: bằng cách dùng một giao dịch, ứng dụng có thể giả định rằng không có sự cố nào xảy ra (tính nguyên tử), không ai khác đang đồng thời truy cập cơ sở dữ liệu (tính cô lập), và các thiết bị lưu trữ là hoàn toàn đáng tin cậy (độ bền). Mặc dù sự cố, tranh chấp đua (race condition), và hỏng đĩa thực sự có xảy ra, trừu tượng hóa giao dịch giấu đi những vấn đề đó để ứng dụng không phải bận tâm về chúng.

We will now continue along the same lines, and seek abstractions that can allow an application to ignore some of the problems with distributed systems. For example, one of the most important abstractions for distributed systems is consensus: that is, getting all of the nodes to agree on something. As we shall see in this chapter, reliably reaching consensus in spite of network faults and process failures is a surprisingly tricky problem.

Giờ ta sẽ tiếp tục theo mạch đó và đi tìm những trừu tượng hóa cho phép một ứng dụng bỏ qua một số vấn đề của hệ thống phân tán. Ví dụ, một trong những trừu tượng hóa quan trọng nhất của hệ thống phân tán là đồng thuận (consensus): tức là làm cho tất cả các nút cùng nhất trí về một điều gì đó. Như ta sẽ thấy trong chương này, việc đạt được đồng thuận một cách đáng tin cậy bất chấp lỗi mạng và lỗi tiến trình hóa ra lại là một bài toán khó một cách đáng ngạc nhiên.

Once you have an implementation of consensus, applications can use it for various purposes. For example, say you have a database with single-leader replication. If the leader dies and you need to fail over to another **node**, the remaining database nodes can use consensus to elect a new leader. As discussed in “Handling Node Outages”, it’s important that there is only one leader, and that all nodes agree who the leader is. If two nodes both believe that they are the leader, that situation is called split brain, and it often leads to data loss. Correct implementations of consensus help avoid such problems.

Một khi đã có một hiện thực của đồng thuận, ứng dụng có thể dùng nó cho nhiều mục đích khác nhau. Ví dụ, giả sử bạn có một cơ sở dữ liệu dùng nhân bản đơn-leader (single-leader replication). Nếu leader gặp sự cố và bạn cần chuyển đổi dự phòng (fail over) sang một nút khác, các nút cơ sở dữ liệu còn lại có thể dùng đồng thuận để bầu ra một leader mới. Như đã bàn ở “Handling Node Outages”, điều quan trọng là chỉ có một leader duy nhất, và tất cả các nút đều nhất trí về việc ai là leader. Nếu hai nút cùng tin rằng mình là leader, tình huống đó gọi là split brain (não chia đôi), và nó thường dẫn tới mất dữ liệu. Các hiện thực đồng thuận đúng đắn giúp tránh những vấn đề như vậy.

Later in this chapter, in “Distributed Transactions and Consensus”, we will look into algorithms to solve consensus and related problems. But first we first need to explore the range of guarantees and abstractions that can be provided in a distributed system.

Cuối chương này, trong phần “Distributed Transactions and Consensus”, ta sẽ đi sâu vào các thuật toán giải bài toán đồng thuận và các bài toán liên quan. Nhưng trước hết, ta cần khám phá phạm vi các bảo đảm và trừu tượng hóa mà một hệ thống phân tán có thể cung cấp.

We need to understand the scope of what can and cannot be done: in some situations, it’s possible for the system to tolerate faults and continue working; in other situations, that is not possible. The limits of what is and isn’t possible have been explored in depth, both in theoretical proofs and in practical implementations. We will get an overview of those fundamental limits in this chapter.

Ta cần hiểu rõ ranh giới của những gì làm được và không làm được: trong một số tình

huống, hệ thống có thể chịu lỗi và tiếp tục hoạt động; trong những tình huống khác thì không. Ranh giới của những gì khả thi và bất khả thi đã được khám phá rất sâu, cả trong các chứng minh lý thuyết lẫn các hiện thực thực tiễn. Trong chương này, ta sẽ có một cái nhìn tổng quan về những giới hạn cơ bản đó.

Researchers in the field of distributed systems have been studying these topics for decades, so there is a lot of material—we'll only be able to scratch the surface. In this book we don't have space to go into details of the formal models and proofs, so we will stick with informal intuitions. The literature references offer plenty of additional depth if you're interested.

Các nhà nghiên cứu trong lĩnh vực hệ thống phân tán đã nghiên cứu những chủ đề này hàng chục năm, nên có rất nhiều tài liệu — ta chỉ có thể chạm tới bề mặt. Trong cuốn sách này ta không có chỗ để đi vào chi tiết các mô hình hình thức và các chứng minh, nên ta sẽ chỉ dừng ở những trực giác phi hình thức. Các tài liệu tham khảo cung cấp rất nhiều chiều sâu bổ sung nếu bạn quan tâm.

Consistency Guarantees — Các bảo đảm về tính nhất quán

In “Problems with Replication Lag” we looked at some timing issues that occur in a replicated database. If you look at two database nodes at the same moment in time, you’re likely to see different data on the two nodes, because write requests arrive on different nodes at different times. These inconsistencies occur no matter what replication method the database uses (single-leader, multi-leader, or leaderless replication).

Trong phần “Problems with Replication Lag” ta đã xem xét một số vấn đề về thời điểm xảy ra trong một cơ sở dữ liệu được nhân bản. Nếu bạn nhìn vào hai nút cơ sở dữ liệu tại cùng một thời điểm, nhiều khả năng bạn sẽ thấy dữ liệu khác nhau trên hai nút, bởi các yêu cầu ghi đến những nút khác nhau vào những thời điểm khác nhau. Những sự bất nhất này xảy ra bất kể cơ sở dữ liệu dùng phương pháp nhân bản nào (đơn-leader, đa-leader, hay không-leader).

Most replicated databases provide at least eventual consistency, which means that if you stop writing to the database and wait for some unspecified length of time, then eventually all read requests will return the same value [1]. In other words, the inconsistency is temporary, and it eventually resolves itself (assuming that any faults in the network are also eventually repaired). A better name for eventual consistency may be convergence, as we expect all replicas to eventually converge to the same value [2].

Hầu hết các cơ sở dữ liệu được nhân bản cung cấp ít nhất tính nhất quán cuối cùng (eventual consistency), nghĩa là nếu bạn ngừng ghi vào cơ sở dữ liệu và chờ một khoảng thời gian không xác định, thì rốt cuộc tất cả các yêu cầu đọc sẽ trả về cùng một giá trị [1]. Nói cách khác, sự bất nhất chỉ là tạm thời, và nó rốt cuộc sẽ tự giải quyết (giả định rằng bất kỳ lỗi nào trên mạng rốt cuộc cũng được sửa). Một cái tên hay hơn cho tính nhất quán cuối cùng có lẽ là sự hội tụ (convergence), bởi ta kỳ vọng tất cả các bản sao rốt cuộc sẽ hội tụ về cùng một giá trị [2].

However, this is a very weak guarantee—it doesn’t say anything about when the replicas will converge. Until the time of convergence, reads could return anything or nothing [1]. For example, if you write a value and then immediately read it again, there is no guarantee that you will see the value you just wrote, because the read may be routed to a different replica (see “Reading Your Own Writes”).

Tuy nhiên, đây là một bảo đảm rất yếu — nó không nói gì về việc khi nào các bản sao sẽ hội tụ. Cho tới thời điểm hội tụ, các lần đọc có thể trả về bất cứ thứ gì hoặc không trả về gì cả [1]. Ví dụ, nếu bạn ghi một giá trị rồi đọc lại nó ngay lập tức, không có bảo đảm nào rằng bạn sẽ thấy giá trị mình vừa ghi, bởi lần đọc có thể được định tuyến tới một bản sao khác (xem “Reading Your Own Writes”).

Eventual consistency is hard for application developers because it is so different from the behavior of variables in a normal single-threaded program. If you assign a value to a variable and then read it shortly afterward, you don't expect to read back the old value, or for the read to fail. A database looks superficially like a variable that you can read and write, but in fact it has much more complicated semantics [3].

Tính nhất quán cuối cùng gây khó khăn cho các lập trình viên ứng dụng vì nó quá khác với hành vi của biến trong một chương trình đơn luồng thông thường. Nếu bạn gán một giá trị cho một biến rồi đọc nó ngay sau đó, bạn không hề kỳ vọng đọc lại giá trị cũ, hay là lần đọc thất bại. Một cơ sở dữ liệu thoát nhìn giống như một biến mà bạn có thể đọc và ghi, nhưng thực ra nó có ngữ nghĩa phức tạp hơn nhiều [3].

When working with a database that provides only weak guarantees, you need to be constantly aware of its limitations and not accidentally assume too much. Bugs are often subtle and hard to find by testing, because the application may work well most of the time. The **edge** cases of eventual consistency only become apparent when there is a fault in the system (e.g., a network interruption) or at high **concurrency**.

Khi làm việc với một cơ sở dữ liệu chỉ cung cấp các bảo đảm yếu, bạn cần luôn ý thức được những giới hạn của nó và không vô tình giả định quá nhiều. Các bug thường tinh vi và khó tìm ra bằng kiểm thử, bởi ứng dụng có thể hoạt động tốt trong phần lớn thời gian. Các trường hợp biên của tính nhất quán cuối cùng chỉ lộ ra khi có một lỗi trong hệ thống (chẳng hạn một gián đoạn mạng) hoặc ở mức tương tranh cao.

In this chapter we will explore stronger consistency models that data systems may choose to provide. They don't come for free: systems with stronger guarantees may have worse performance or be less fault-tolerant than systems with weaker guarantees. Nevertheless, stronger guarantees can be appealing because they are easier to use correctly. Once you have seen a few different consistency models, you'll be in a better position to decide which one best fits your needs.

Trong chương này ta sẽ khám phá những mô hình nhất quán mạnh hơn mà các hệ thống dữ liệu có thể chọn cung cấp. Chúng không miễn phí: các hệ thống với bảo đảm mạnh hơn có thể có hiệu năng kém hơn hoặc kém chịu lỗi hơn so với các hệ thống với bảo đảm yếu hơn. Tuy vậy, các bảo đảm mạnh hơn có thể hấp dẫn vì chúng dễ dùng đúng hơn. Một khi đã thấy qua vài mô hình nhất quán khác nhau, bạn sẽ ở vị thế tốt hơn để quyết định mô hình nào phù hợp nhất với nhu cầu của mình.

There is some similarity between distributed consistency models and the hierarchy of transaction isolation levels we discussed previously [4, 5] (see “Weak Isolation Levels”). But while there is some overlap, they are mostly independent concerns: transaction isolation is primarily about avoiding race conditions due to concurrently executing transactions, whereas distributed consistency is mostly about coordinating the state of replicas in the face of delays and faults.

Có một số nét tương đồng giữa các mô hình nhất quán phân tán và phân cấp các mức cô lập giao dịch mà ta đã bàn trước đó [4, 5] (xem “Weak Isolation Levels”). Nhưng dù có phần trùng lặp, chúng chủ yếu là những mối quan tâm độc lập: cô lập giao dịch chủ yếu là về việc tránh các tranh chấp đua do các giao dịch thực thi đồng thời, trong khi tính nhất quán phân tán chủ yếu là về việc điều phối trạng thái của các bản sao khi đối mặt với các trì hoãn và lỗi.

This chapter covers a broad range of topics, but as we shall see, these areas are in fact deeply linked:

Chương này bao quát một phạm vi chủ đề rộng, nhưng như ta sẽ thấy, những mảng này thực ra liên kết với nhau rất sâu sắc:

- We will start by looking at one of the strongest consistency models in common use, linearizability, and examine its pros and cons.
- We'll then examine the issue of ordering events in a distributed system ("Ordering Guarantees"), particularly around causality and total ordering.
- In the third section ("Distributed Transactions and Consensus") we will explore how to atomically commit a distributed transaction, which will finally lead us toward solutions for the consensus problem.
 - Ta sẽ bắt đầu bằng cách xem xét một trong những mô hình nhất quán mạnh nhất đang được dùng phổ biến, tính tuyến tính hóa (linearizability), và xét các ưu nhược điểm của nó.
 - Sau đó ta sẽ xem xét vấn đề sắp thứ tự các sự kiện trong một hệ thống phân tán ("Ordering Guarantees"), đặc biệt là xoay quanh tính nhân quả và sắp thứ tự toàn phần.
 - Trong phần thứ ba ("Distributed Transactions and Consensus") ta sẽ khám phá cách commit một giao dịch phân tán một cách nguyên tử, điều này rất cuộc sẽ dẫn ta tới các giải pháp cho bài toán đồng thuận.

Linearizability — Tính tuyến tính hóa

In an eventually consistent database, if you ask two different replicas the same question at the same time, you may get two different answers. That's confusing. Wouldn't it be a lot simpler if the database could give the illusion that there is only one replica (i.e., only one copy of the data)? Then every **client** would have the same view of the data, and you wouldn't have to worry about replication lag.

Trong một cơ sở dữ liệu nhất quán cuối cùng, nếu bạn hỏi hai bản sao khác nhau cùng một câu hỏi tại cùng một thời điểm, bạn có thể nhận được hai câu trả lời khác nhau. Điều đó thật khó hiểu. Chẳng phải sẽ đơn giản hơn nhiều nếu cơ sở dữ liệu có thể tạo ra ảo giác rằng chỉ có một bản sao duy nhất (tức chỉ có một bản dữ liệu) hay sao? Khi đó mọi client sẽ có cùng một góc nhìn về dữ liệu, và bạn sẽ không phải lo về độ trễ nhân bản.

This is the idea behind linearizability [6] (also known as atomic consistency [7], strong consistency, immediate consistency, or external consistency [8]). The exact definition of linearizability is quite subtle, and we will explore it in the rest of this section. But the basic idea is to make a system appear as if there were only one copy of the data, and all operations on it are atomic. With this guarantee, even though there may be multiple replicas in reality, the application does not need to worry about them.

Đây chính là ý tưởng đằng sau tính tuyến tính hóa [6] (còn gọi là tính nhất quán nguyên tử (atomic consistency) [7], tính nhất quán mạnh, tính nhất quán tức thời, hay tính nhất quán ngoại tại (external consistency) [8]). Định nghĩa chính xác của tính tuyến tính hóa khá tinh vi, và ta sẽ khám phá nó trong phần còn lại của mục này. Nhưng ý tưởng cơ bản là làm cho một hệ thống trông như thể chỉ có một bản dữ liệu duy nhất, và mọi thao tác trên nó đều là nguyên tử. Với bảo đảm này, dù trên thực tế có thể có nhiều bản sao, ứng dụng không cần phải lo về chúng.

In a linearizable system, as soon as one client successfully completes a write, all clients reading from the database must be able to see the value just written. Maintaining the illusion of a single copy of the data means guaranteeing that the value read is the most recent, up-to-date value, and doesn't come from a stale **cache** or replica. In other words, linearizability is a recency guarantee. To clarify this idea, let's look at an example of a system that is not linearizable.

Trong một hệ thống tuyến tính hóa, ngay khi một client hoàn tất thành công một lần ghi, tất cả các client đang đọc từ cơ sở dữ liệu phải có khả năng thấy được giá trị vừa được ghi. Việc duy trì ảo giác về một bản dữ liệu duy nhất nghĩa là bảo đảm rằng giá trị được đọc là giá trị mới nhất, cập nhật nhất, và không đến từ một bộ nhớ đệm hay bản sao cũ. Nói cách khác, tính tuyến tính hóa là một bảo đảm về tính tức thời (recency). Để làm rõ ý tưởng này, hãy xem một ví dụ về một hệ thống không tuyến tính hóa.

Figure 9-1. This system is not linearizable, causing football fans to be confused. — Hình 9-1. Hệ thống này không tuyến tính hóa, khiến các cổ động viên bóng đá bối rối. Figure 9-1 shows an example of a nonlinearizable sports website [9]. Alice and Bob are sitting in the same room, both checking their phones to see the outcome of the 2014 FIFA World Cup final. Just after the final score is announced, Alice refreshes the page, sees the winner announced, and excitedly tells Bob about it. Bob incredulously hits reload on his own phone, but his request goes to a database replica that is lagging, and so his phone shows that the game is still ongoing.

Hình 9-1 cho thấy một ví dụ về một trang web thể thao không tuyến tính hóa [9]. Alice và Bob đang ngồi trong cùng một phòng, cả hai đều kiểm tra điện thoại để xem kết quả trận chung kết World Cup FIFA 2014. Ngay sau khi tỷ số cuối cùng được công bố, Alice làm mới trang, thấy người chiến thắng được công bố, và phấn khích kể cho Bob nghe. Bob hoài nghi nhấn tải lại trên điện thoại của mình, nhưng yêu cầu của anh lại đến một bản sao cơ sở dữ liệu đang bị tụt hậu, và do đó điện thoại của anh hiển thị rằng trận đấu vẫn đang diễn ra.

If Alice and Bob had hit reload at the same time, it would have been less surprising if they had gotten two different query results, because they wouldn't know at exactly what time their respective requests were processed by the server. However, Bob knows that he hit the reload button (initiated his query) after he heard Alice exclaim the final score, and therefore he expects his query result to be at least as recent as Alice's. The fact that his query returned a stale result is a violation of linearizability.

Nếu Alice và Bob nhấn tải lại cùng một lúc, sẽ ít bất ngờ hơn nếu họ nhận được hai kết quả truy vấn khác nhau, bởi họ sẽ không biết chính xác vào thời điểm nào yêu cầu của mỗi người được máy chủ xử lý. Tuy nhiên, Bob biết rằng anh nhấn nút tải lại (khởi tạo truy vấn của mình) sau khi nghe Alice reo lên tỷ số cuối cùng, và do đó anh kỳ vọng kết quả truy vấn của mình ít nhất cũng mới bằng của Alice. Việc truy vấn của anh trả về một kết quả cũ là một vi phạm tính tuyến tính hóa.

What Makes a System Linearizable? — Điều gì làm cho một hệ thống tuyến tính hóa?

The basic idea behind linearizability is simple: to make a system appear as if there is only a single copy of the data. However, nailing down precisely what that means actually requires some care. In order to understand linearizability better, let's look at some more examples.

Ý tưởng cơ bản đằng sau tính tuyến tính hóa rất đơn giản: làm cho một hệ thống trông như thể chỉ có một bản dữ liệu duy nhất. Tuy nhiên, việc xác định chính xác điều đó nghĩa là gì thì thực sự đòi hỏi đôi chút cẩn trọng. Để hiểu tính tuyến tính hóa rõ hơn, hãy xem thêm vài ví dụ.

Figure 9-2 shows three clients concurrently reading and writing the same key *x* in a linearizable database. In the distributed systems literature, *x* is called a register—in practice, it could be one key in a key-value store, one **row** in a **relational database**, or one **document** in a document database, for example.

Hình 9-2 cho thấy ba client đồng thời đọc và ghi cùng một khóa *x* trong một cơ sở dữ liệu tuyến tính hóa. Trong các tài liệu về hệ thống phân tán, *x* được gọi là một thanh ghi (register) — trên thực tế, nó có thể là một khóa trong một kho khóa-giá trị, một hàng trong một cơ sở dữ liệu quan hệ, hoặc một tài liệu trong một cơ sở dữ liệu tài liệu, chẳng hạn vậy.

Figure 9-2. If a read request is concurrent with a write request, it may return either the old or the new value. — Hình 9-2. Nếu một yêu cầu đọc diễn ra đồng thời với một yêu cầu ghi, nó có thể trả về giá trị cũ hoặc giá trị mới. For simplicity, Figure 9-2 shows only the requests from the clients' point of view, not the internals of the database. Each bar is a request made by a client, where the start of a bar is the time when the request was sent, and the end of a bar is when the response was received by the client. Due to variable network delays, a client doesn't know exactly when the database processed its request—it only knows that it must have happened sometime between the client sending the request and receiving the response.

Để cho đơn giản, Hình 9-2 chỉ cho thấy các yêu cầu từ góc nhìn của client, chứ không phải các chi tiết bên trong cơ sở dữ liệu. Mỗi thanh ngang là một yêu cầu do một client tạo ra, trong đó điểm bắt đầu của thanh là thời điểm yêu cầu được gửi đi, và điểm kết thúc của thanh là thời điểm phản hồi được client nhận về. Do các trì hoãn mạng biến thiên, một client không biết chính xác khi nào cơ sở dữ liệu xử lý yêu cầu của nó — nó chỉ biết rằng việc đó hẳn đã xảy ra vào lúc nào đó giữa thời điểm client gửi yêu cầu và nhận phản hồi.

In this example, the register has two types of operations:

Trong ví dụ này, thanh ghi có hai loại thao tác:

- $\text{read}(x) \Rightarrow v$ means the client requested to read the value of register x , and the database returned the value v .
- $\text{write}(x, v) \Rightarrow r$ means the client requested to set the register x to value v , and the database returned response r (which could be ok or error).
 - $\text{read}(x) \Rightarrow v$ nghĩa là client yêu cầu đọc giá trị của thanh ghi x , và cơ sở dữ liệu trả về giá trị v .
 - $\text{write}(x, v) \Rightarrow r$ nghĩa là client yêu cầu đặt thanh ghi x thành giá trị v , và cơ sở dữ liệu trả về phản hồi r (có thể là ok hoặc error).

In Figure 9-2, the value of x is initially 0, and client C performs a write request to set it to 1. While this is happening, clients A and B are repeatedly polling the database to read the latest value. What are the possible responses that A and B might get for their read requests?

Trong Hình 9-2, giá trị của x ban đầu là 0, và client C thực hiện một yêu cầu ghi để đặt nó thành 1. Trong khi việc này đang diễn ra, các client A và B liên tục thăm dò cơ sở dữ liệu để đọc giá trị mới nhất. Những phản hồi nào là khả dĩ mà A và B có thể nhận được cho các yêu cầu đọc của họ?

- The first read operation by client A completes before the write begins, so it must definitely return the old value 0.
- The last read by client A begins after the write has completed, so it must definitely return the new value 1 if the database is linearizable: we know that the write must have been processed sometime between the start and end of the write operation, and the read must have been processed sometime between the start and end of the read operation. If the read started after the write ended, then the read must have been processed after the write, and therefore it must see the new value that was written.
- Any read operations that overlap in time with the write operation might return either 0 or 1, because we don't know whether or not the write has taken effect at the time when the read operation is processed. These operations are concurrent with the write.
 - Thao tác đọc đầu tiên của client A hoàn tất trước khi lần ghi bắt đầu, nên nó chắc chắn phải trả về giá trị cũ 0.
 - Lần đọc cuối cùng của client A bắt đầu sau khi lần ghi đã hoàn tất, nên nó chắc chắn phải trả về giá trị mới 1 nếu cơ sở dữ liệu là tuyến tính hóa: ta biết rằng lần ghi hẳn

đã được xử lý vào lúc nào đó giữa thời điểm bắt đầu và kết thúc của thao tác ghi, và lần đọc hẳn đã được xử lý vào lúc nào đó giữa thời điểm bắt đầu và kết thúc của thao tác đọc. Nếu lần đọc bắt đầu sau khi lần ghi kết thúc, thì lần đọc hẳn đã được xử lý sau lần ghi, và do đó nó phải thấy giá trị mới đã được ghi.

- Bất kỳ thao tác đọc nào chồng lấn về thời gian với thao tác ghi đều có thể trả về 0 hoặc 1, bởi ta không biết liệu lần ghi đã có hiệu lực hay chưa tại thời điểm thao tác đọc được xử lý. Những thao tác này diễn ra đồng thời với lần ghi.

However, that is not yet sufficient to fully describe linearizability: if reads that are concurrent with a write can return either the old or the new value, then readers could see a value flip back and forth between the old and the new value several times while a write is going on. That is not what we expect of a system that emulates a “single copy of the data.”

Tuy nhiên, ngần đó vẫn chưa đủ để mô tả đầy đủ tính tuyến tính hóa: nếu các lần đọc đồng thời với một lần ghi có thể trả về hoặc giá trị cũ hoặc giá trị mới, thì những người đọc có thể thấy một giá trị lật qua lật lại giữa giá trị cũ và giá trị mới vài lần trong khi một lần ghi đang diễn ra. Đó không phải điều ta kỳ vọng ở một hệ thống mô phỏng “một bản dữ liệu duy nhất”.

To make the system linearizable, we need to add another constraint, illustrated in Figure 9-3.

Để làm cho hệ thống tuyến tính hóa, ta cần thêm một ràng buộc nữa, được minh họa trong Hình 9-3.

Figure 9-3. After any one read has returned the new value, all following reads (on the same or other clients) must also return the new value. — Hình 9-3. Sau khi bất kỳ lần đọc nào đã trả về giá trị mới, mọi lần đọc tiếp theo (trên cùng client đó hoặc các client khác) cũng phải trả về giá trị mới. In a linearizable system we imagine that there must be some point in time (between the start and end of the write operation) at which the value of *x* atomically flips from 0 to 1. Thus, if one client’s read returns the new value 1, all subsequent reads must also return the new value, even if the write operation has not yet completed.

Trong một hệ thống tuyến tính hóa, ta hình dung rằng phải có một thời điểm nào đó (giữa lúc bắt đầu và kết thúc của thao tác ghi) mà tại đó giá trị của *x* lật một cách nguyên tử từ 0 sang 1. Như vậy, nếu lần đọc của một client trả về giá trị mới 1, thì mọi lần đọc tiếp theo cũng phải trả về giá trị mới, ngay cả khi thao tác ghi còn chưa hoàn tất.

This timing dependency is illustrated with an arrow in Figure 9-3. Client A is the first to read the new value, 1. Just after A’s read returns, B begins a new read. Since B’s read occurs strictly after A’s read, it must also return 1, even though the write by C is still ongoing. (It’s the same situation as with Alice and Bob in Figure 9-1: after Alice has read the new value, Bob also expects to read the new value.)

Sự phụ thuộc về thời điểm này được minh họa bằng một mũi tên trong Hình 9-3. Client A là client đầu tiên đọc được giá trị mới, 1. Ngay sau khi lần đọc của A trả về, B bắt đầu một lần đọc mới. Vì lần đọc của B diễn ra hoàn toàn sau lần đọc của A, nó cũng phải trả về 1, dù lần ghi của C vẫn đang diễn ra. (Đây cũng chính là tình huống của Alice và Bob trong Hình 9-1: sau khi Alice đã đọc giá trị mới, Bob cũng kỳ vọng đọc được giá trị mới.)

We can further refine this timing diagram to visualize each operation taking effect atomically at some point in time. A more complex example is shown in Figure 9-4 [10].

Ta có thể tinh chỉnh thêm biểu đồ thời điểm này để hình dung mỗi thao tác có hiệu lực một cách nguyên tử tại một thời điểm nào đó. Một ví dụ phức tạp hơn được cho trong Hình 9-4 [10].

In Figure 9-4 we add a third type of operation besides read and write:

Trong Hình 9-4 ta thêm một loại thao tác thứ ba bên cạnh read và write:

- $\text{cas}(x, \text{vold}, \text{vnew}) \Rightarrow r$ means the client requested an atomic compare-and-set operation (see “Compare-and-set”). If the current value of the register x equals vold, it should be atomically set to vnew. If $x \neq \text{vold}$ then the operation should leave the register unchanged and return an error. r is the database’s response (ok or error).
- $\text{cas}(x, \text{vold}, \text{vnew}) \Rightarrow r$ nghĩa là client yêu cầu một thao tác so-sánh-rồi-đặt (compare-and-set) nguyên tử (xem “Compare-and-set”). Nếu giá trị hiện tại của thanh ghi x bằng vold, nó sẽ được đặt một cách nguyên tử thành vnew. Nếu $x \neq \text{vold}$ thì thao tác sẽ để thanh ghi nguyên vẹn và trả về một lỗi. r là phản hồi của cơ sở dữ liệu (ok hoặc error).

Each operation in Figure 9-4 is marked with a vertical line (inside the bar for each operation) at the time when we think the operation was executed. Those markers are joined up in a sequential order, and the result must be a valid sequence of reads and writes for a register (every read must return the value set by the most recent write).

Mỗi thao tác trong Hình 9-4 được đánh dấu bằng một đường thẳng đứng (bên trong thanh của mỗi thao tác) tại thời điểm ta cho rằng thao tác được thực thi. Các dấu này được nối lại theo một thứ tự tuần tự, và kết quả phải là một chuỗi đọc và ghi hợp lệ cho một thanh ghi (mỗi lần đọc phải trả về giá trị được đặt bởi lần ghi gần nhất).

The requirement of linearizability is that the lines joining up the operation markers always move forward in time (from left to right), never backward. This requirement ensures the recency guarantee we discussed earlier: once a new value has been written or read, all subsequent reads see the value that was written, until it is overwritten again.

Yêu cầu của tính tuyến tính hóa là các đường nối các dấu thao tác luôn tiến về phía trước theo thời gian (từ trái sang phải), không bao giờ lùi lại. Yêu cầu này bảo đảm tính tức thời mà ta đã bàn ở trên: một khi một giá trị mới đã được ghi hoặc đọc, mọi lần đọc tiếp theo đều thấy giá trị đã được ghi, cho tới khi nó lại bị ghi đè.

Figure 9-4. Visualizing the points in time at which the reads and writes appear to have taken effect. The final read by B is not linearizable. — Hình 9-4. Hình dung các thời điểm mà tại đó các lần đọc và ghi có vẻ đã có hiệu lực. Lần đọc cuối cùng của B là không tuyến tính hóa.

There are a few interesting details to point out in Figure 9-4:

Có vài chi tiết thú vị cần chỉ ra trong Hình 9-4:

- First client B sent a request to read x , then client D sent a request to set x to 0, and then client A sent a request to set x to 1. Nevertheless, the value returned to B’s read is 1 (the value written by A). This is okay: it means that the database first processed D’s write, then A’s write, and finally B’s read. Although this is not the order in which the requests were sent, it’s an acceptable order, because the three requests are concurrent. Perhaps B’s read request was slightly delayed in the network, so it only reached the database after the two writes.
- Client B’s read returned 1 before client A received its response from the database, saying that the write of the value 1 was successful. This is also okay: it doesn’t **mean** the value was read before it was written, it just means the ok response from the database to client A was slightly delayed in the network.
- This model doesn’t assume any transaction isolation: another client may change a value at any time. For example, C first reads 1 and then reads 2, because the value was changed by B between the two reads. An atomic compare-and-set (cas) operation can be used to check the value hasn’t

been concurrently changed by another client: B and C's cas requests succeed, but D's cas request fails (by the time the database processes it, the value of x is no longer 0).

- The final read by client B (in a shaded bar) is not linearizable. The operation is concurrent with C's cas write, which updates x from 2 to 4. In the absence of other requests, it would be okay for B's read to return 2. However, client A has already read the new value 4 before B's read started, so B is not allowed to read an older value than A. Again, it's the same situation as with Alice and Bob in Figure 9-1.

- Trước tiên client B gửi một yêu cầu đọc x, rồi client D gửi một yêu cầu đặt x thành 0, và sau đó client A gửi một yêu cầu đặt x thành 1. Tuy nhiên, giá trị trả về cho lần đọc của B là 1 (giá trị do A ghi). Điều này là chấp nhận được: nó nghĩa là cơ sở dữ liệu trước tiên xử lý lần ghi của D, rồi lần ghi của A, và cuối cùng là lần đọc của B. Dù đây không phải thứ tự mà các yêu cầu được gửi đi, nó là một thứ tự chấp nhận được, bởi cả ba yêu cầu đều đồng thời. Có lẽ yêu cầu đọc của B bị trì hoãn đôi chút trên mạng, nên nó chỉ đến cơ sở dữ liệu sau hai lần ghi.
- Lần đọc của client B trả về 1 trước khi client A nhận được phản hồi từ cơ sở dữ liệu báo rằng lần ghi giá trị 1 đã thành công. Điều này cũng chấp nhận được: nó không có nghĩa là giá trị được đọc trước khi nó được ghi, mà chỉ nghĩa là phản hồi ok từ cơ sở dữ liệu tới client A bị trì hoãn đôi chút trên mạng.
- Mô hình này không giả định bất kỳ tính cô lập giao dịch nào: một client khác có thể thay đổi một giá trị bất cứ lúc nào. Ví dụ, C trước tiên đọc 1 rồi đọc 2, bởi giá trị đã bị B thay đổi giữa hai lần đọc. Một thao tác so-sánh-rồi-đặt (cas) nguyên tử có thể được dùng để kiểm tra rằng giá trị chưa bị một client khác thay đổi đồng thời: các yêu cầu cas của B và C thành công, nhưng yêu cầu cas của D thất bại (vào lúc cơ sở dữ liệu xử lý nó, giá trị của x đã không còn là 0).
- Lần đọc cuối cùng của client B (trong thanh tô đậm) là không tuyến tính hóa. Thao tác này diễn ra đồng thời với lần ghi cas của C, vốn cập nhật x từ 2 lên 4. Nếu không có các yêu cầu khác, việc lần đọc của B trả về 2 sẽ là chấp nhận được. Tuy nhiên, client A đã đọc giá trị mới 4 trước khi lần đọc của B bắt đầu, nên B không được phép đọc một giá trị cũ hơn A. Một lần nữa, đây cũng chính là tình huống của Alice và Bob trong Hình 9-1.

That is the intuition behind linearizability; the formal definition [6] describes it more precisely. It is possible (though computationally expensive) to test whether a system's behavior is linearizable by recording the timings of all requests and responses, and checking whether they can be arranged into a valid sequential order [11].

Đó là trực giác đằng sau tính tuyến tính hóa; định nghĩa hình thức [6] mô tả nó chính xác hơn. Có thể (dù tốn kém về mặt tính toán) kiểm tra xem hành vi của một hệ thống có tuyến tính hóa hay không bằng cách ghi lại thời điểm của tất cả các yêu cầu và phản hồi, rồi kiểm tra xem chúng có thể được sắp xếp thành một thứ tự tuần tự hợp lệ hay không [11].

Linearizability Versus Serializability — Tính tuyến tính hóa so với tính tuần tự hóa
Linearizability is easily confused with serializability (see “Serializability”), as both words seem to mean something like “can be arranged in a sequential order.” However, they are two quite different guarantees, and it is important to distinguish between them:

Tính tuyến tính hóa dễ bị nhầm với tính tuần tự hóa (serializability) (xem “Serializability”), bởi cả hai từ đều có vẻ mang nghĩa kiểu như “có thể được sắp xếp theo một thứ tự tuần tự”. Tuy nhiên, chúng là hai bảo đảm khá khác nhau, và điều quan trọng là phải phân biệt giữa chúng:

Serializability is an isolation **property** of transactions, where every transaction may read and write multiple objects (rows, documents, records)—see “Single-Object and Multi-Object Operations”. It guarantees that transactions behave the same as if they had executed in some serial order (each transaction running to completion before the next transaction starts). It is okay for that serial order to be different from the order in which transactions were actually run [12].

Tính tuần tự hóa là một thuộc tính cô lập của các giao dịch, trong đó mỗi giao dịch có thể đọc và ghi nhiều đối tượng (hàng, tài liệu, bản ghi) — xem “Single-Object and Multi-Object Operations”. Nó bảo đảm rằng các giao dịch hành xử giống như thể chúng đã được thực thi theo một thứ tự tuần tự nào đó (mỗi giao dịch chạy tới hết trước khi giao dịch tiếp theo bắt đầu). Thứ tự tuần tự đó được phép khác với thứ tự mà các giao dịch thực sự được chạy [12].

Linearizability is a recency guarantee on reads and writes of a register (an individual object). It doesn’t group operations together into transactions, so it does not prevent problems such as write skew (see “Write Skew and Phantoms”), unless you take additional measures such as materializing conflicts (see “Materializing conflicts”).

Tính tuyến tính hóa là một bảo đảm về tính tức thời đối với các lần đọc và ghi của một thanh ghi (một đối tượng riêng lẻ). Nó không gom các thao tác lại thành các giao dịch, nên nó không ngăn được các vấn đề như lệch ghi (write skew) (xem “Write Skew and Phantoms”), trừ khi bạn áp dụng thêm các biện pháp như hiện thực hóa xung đột (xem “Materializing conflicts”).

A database may provide both serializability and linearizability, and this combination is known as strict serializability or strong one-copy serializability (strong-1SR) [4, 13]. Implementations of serializability based on two-phase locking (see “Two-Phase Locking (2PL)”) or actual serial execution (see “Actual Serial Execution”) are typically linearizable.

Một cơ sở dữ liệu có thể cung cấp cả tính tuần tự hóa lẫn tính tuyến tính hóa, và sự kết hợp này được gọi là tính tuần tự hóa nghiêm ngặt (strict serializability) hoặc tính tuần tự hóa một-bản-sao mạnh (strong-1SR) [4, 13]. Các hiện thực của tính tuần tự hóa dựa trên khóa hai pha (xem “Two-Phase Locking (2PL)”) hoặc thực thi tuần tự thực sự (xem “Actual Serial Execution”) thường là tuyến tính hóa.

However, serializable snapshot isolation (see “Serializable Snapshot Isolation (SSI)”) is not linearizable: by design, it makes reads from a consistent snapshot, to avoid lock contention between readers and writers. The whole point of a consistent snapshot is that it does not include writes that are more recent than the snapshot, and thus reads from the snapshot are not linearizable.

Tuy nhiên, cô lập snapshot tuần tự hóa (xem “Serializable Snapshot Isolation (SSI)”) thì không tuyến tính hóa: theo thiết kế, nó thực hiện các lần đọc từ một snapshot nhất quán, để tránh tranh chấp khóa giữa người đọc và người ghi. Toàn bộ điểm cốt lõi của một snapshot nhất quán là nó không bao gồm các lần ghi mới hơn snapshot, và do đó các lần đọc từ snapshot là không tuyến tính hóa.

Relying on Linearizability — Dựa vào tính tuyến tính hóa

In what circumstances is linearizability useful? Viewing the final score of a sporting match is perhaps a frivolous example: a result that is outdated by a few seconds is unlikely to cause any real harm in this situation. However, there are a few areas in which linearizability is an important requirement for making a system work correctly.

Tính tuyến tính hóa hữu ích trong những hoàn cảnh nào? Việc xem tỷ số cuối cùng của một trận đấu thể thao có lẽ là một ví dụ tầm phào: một kết quả lỗi thời vài giây khó mà gây hại thực sự trong tình huống này. Tuy nhiên, có vài lĩnh vực mà tính tuyến tính hóa là một yêu cầu quan trọng để hệ thống hoạt động đúng đắn.

Locking and leader election — Khóa và bầu leader

A system that uses single-leader replication needs to ensure that there is indeed only one leader, not several (split brain). One way of electing a leader is to use a lock: every node that starts up tries to acquire the lock, and the one that succeeds becomes the leader [14]. No matter how this lock is implemented, it must be linearizable: all nodes must agree which node owns the lock; otherwise it is useless.

Một hệ thống dùng nhân bản đơn-leader cần bảo đảm rằng quả thực chỉ có một leader, chứ không phải nhiều leader (split brain). Một cách để bầu leader là dùng một khóa: mọi nút khi khởi động đều cố gắng giành được khóa, và nút nào thành công sẽ trở thành leader [14]. Bất kể khóa này được hiện thực ra sao, nó phải tuyến tính hóa: tất cả các nút phải nhất trí về việc nút nào sở hữu khóa; nếu không thì nó vô dụng.

Coordination services like Apache ZooKeeper [15] and etcd [16] are often used to implement distributed locks and leader election. They use consensus algorithms to implement linearizable operations in a fault-tolerant way (we discuss such algorithms later in this chapter, in “Fault-Tolerant Consensus”). There are still many subtle details to implementing locks and leader election correctly (see for example the fencing issue in “The leader and the lock”), and libraries like Apache Curator [17] help by providing higher-level recipes on top of ZooKeeper. However, a linearizable storage service is the basic foundation for these coordination tasks.

Các dịch vụ điều phối như Apache ZooKeeper [15] và etcd [16] thường được dùng để hiện thực các khóa phân tán và bầu leader. Chúng dùng các thuật toán đồng thuận để hiện thực các thao tác tuyến tính hóa một cách chịu lỗi (ta sẽ bàn về các thuật toán như vậy cuối chương này, trong “Fault-Tolerant Consensus”). Vẫn còn nhiều chi tiết tinh vi để hiện thực khóa và bầu leader một cách đúng đắn (xem chẳng hạn vấn đề rào chắn (fencing) trong “The leader and the lock”), và các thư viện như Apache Curator [17] giúp ích bằng cách cung cấp các công thức ở mức cao hơn xây trên ZooKeeper. Tuy vậy, một dịch vụ lưu trữ tuyến tính hóa là nền tảng cơ bản cho các tác vụ điều phối này.

Distributed locking is also used at a much more granular level in some distributed databases, such as Oracle Real Application Clusters (RAC) [18]. RAC uses a lock per disk page, with multiple nodes sharing access to the same disk storage system. Since these linearizable locks are on the critical path of transaction execution, RAC deployments usually have a dedicated cluster interconnect network for communication between database nodes.

Khóa phân tán cũng được dùng ở mức chi tiết hơn nhiều trong một số cơ sở dữ liệu phân tán, chẳng hạn Oracle Real Application Clusters (RAC) [18]. RAC dùng một khóa cho mỗi trang đĩa, với nhiều nút chia sẻ quyền truy cập tới cùng một hệ thống lưu trữ đĩa. Vì các khóa tuyến tính hóa này nằm trên đường găng của việc thực thi giao dịch, các triển khai RAC thường có một mạng liên kết cụm chuyên dụng để giao tiếp giữa các nút cơ sở dữ liệu.

Constraints and uniqueness guarantees — Các ràng buộc và bảo đảm tính duy nhất

Uniqueness constraints are common in databases: for example, a username or email address must uniquely identify one user, and in a file storage service there cannot be two files with the same path

and filename. If you want to enforce this constraint as the data is written (such that if two people try to concurrently create a user or a file with the same name, one of them will be returned an error), you need linearizability.

Các ràng buộc duy nhất là phổ biến trong các cơ sở dữ liệu: ví dụ, một tên người dùng hoặc địa chỉ email phải định danh duy nhất một người dùng, và trong một dịch vụ lưu trữ tệp không thể có hai tệp với cùng đường dẫn và tên tệp. Nếu bạn muốn áp đặt ràng buộc này ngay khi dữ liệu được ghi (sao cho nếu hai người cùng cố tạo một người dùng hoặc một tệp với cùng tên một cách đồng thời, một trong hai sẽ bị trả về lỗi), bạn cần tính tuyến tính hóa.

This situation is actually similar to a lock: when a user registers for your service, you can think of them acquiring a “lock” on their chosen username. The operation is also very similar to an atomic compare-and-set, setting the username to the ID of the user who claimed it, provided that the username is not already taken.

Tình huống này thực ra tương tự như một khóa: khi một người dùng đăng ký dịch vụ của bạn, bạn có thể coi như họ đang giành lấy một “khóa” trên tên người dùng họ chọn. Thao tác này cũng rất giống một thao tác so-sánh-rồi-đặt nguyên tử, đặt tên người dùng thành ID của người đã giành nó, với điều kiện là tên người dùng đó chưa bị chiếm.

Similar issues arise if you want to ensure that a bank account balance never goes negative, or that you don’t sell more items than you have in stock in the warehouse, or that two people don’t concurrently book the same seat on a flight or in a theater. These constraints all require there to be a single up-to-date value (the account balance, the stock level, the seat occupancy) that all nodes agree on.

Các vấn đề tương tự nảy sinh nếu bạn muốn bảo đảm rằng số dư tài khoản ngân hàng không bao giờ âm, hoặc bạn không bán nhiều mặt hàng hơn số lượng tồn trong kho, hoặc hai người không đồng thời đặt cùng một ghế trên một chuyến bay hay trong một rạp hát. Tất cả những ràng buộc này đều đòi hỏi phải có một giá trị cập nhật nhất duy nhất (số dư tài khoản, mức tồn kho, tình trạng ghế ngồi) mà tất cả các nút đều nhất trí.

In real applications, it is sometimes acceptable to treat such constraints loosely (for example, if a flight is overbooked, you can move customers to a different flight and offer them compensation for the inconvenience). In such cases, linearizability may not be needed, and we will discuss such loosely interpreted constraints in “Timeliness and Integrity”.

Trong các ứng dụng thực tế, đôi khi việc xử lý những ràng buộc như vậy một cách lỏng lẻo là chấp nhận được (ví dụ, nếu một chuyến bay bị đặt vượt số ghế, bạn có thể chuyển khách sang chuyến bay khác và bồi thường cho họ vì sự bất tiện). Trong những trường hợp như vậy, có thể không cần tính tuyến tính hóa, và ta sẽ bàn về những ràng buộc được diễn giải lỏng lẻo như vậy trong “Timeliness and Integrity”.

However, a hard uniqueness constraint, such as the one you typically find in relational databases, requires linearizability. Other kinds of constraints, such as **foreign key** or attribute constraints, can be implemented without requiring linearizability [19].

Tuy nhiên, một ràng buộc duy nhất cứng, chẳng hạn loại bạn thường thấy trong các cơ sở dữ liệu quan hệ, đòi hỏi tính tuyến tính hóa. Các loại ràng buộc khác, chẳng hạn ràng buộc khóa ngoại hoặc ràng buộc thuộc tính, có thể được hiện thực mà không cần tính tuyến tính hóa [19].

Cross-channel timing dependencies — Các phụ thuộc về thời điểm xuyên kênh

Notice a detail in Figure 9-1: if Alice hadn't exclaimed the score, Bob wouldn't have known that the result of his query was stale. He would have just refreshed the page again a few seconds later, and eventually seen the final score. The linearizability violation was only noticed because there was an additional communication channel in the system (Alice's voice to Bob's ears).

Hãy để ý một chi tiết trong Hình 9-1: nếu Alice không reo lên tỷ số, Bob sẽ không biết rằng kết quả truy vấn của mình là cũ. Anh sẽ chỉ làm mới trang lần nữa vài giây sau đó, và rốt cuộc thấy tỷ số cuối cùng. Vi phạm tính tuyến tính hóa chỉ bị nhận ra vì có thêm một kênh giao tiếp trong hệ thống (giọng nói của Alice tới tai Bob).

Similar situations can arise in computer systems. For example, say you have a website where users can upload a photo, and a background process resizes the photos to lower resolution for faster download (thumbnails). The architecture and dataflow of this system is illustrated in Figure 9-5.

Những tình huống tương tự có thể nảy sinh trong các hệ thống máy tính. Ví dụ, giả sử bạn có một trang web nơi người dùng có thể tải lên một bức ảnh, và một tiến trình nền sẽ thay đổi kích thước các bức ảnh xuống độ phân giải thấp hơn để tải xuống nhanh hơn (ảnh thu nhỏ). Kiến trúc và luồng dữ liệu của hệ thống này được minh họa trong Hình 9-5.

The image resizer needs to be explicitly instructed to perform a resizing job, and this instruction is sent from the web server to the resizer via a **message queue** (see Chapter 11). The web server doesn't place the entire photo on the **queue**, since most message brokers are designed for small messages, and a photo may be several megabytes in size. Instead, the photo is first written to a file storage service, and once the write is complete, the instruction to the resizer is placed on the queue.

Bộ thay đổi kích thước ảnh cần được chỉ thị tường minh để thực hiện một công việc thay đổi kích thước, và chỉ thị này được gửi từ máy chủ web tới bộ thay đổi kích thước qua một hàng đợi thông điệp (xem Chương 11). Máy chủ web không đặt toàn bộ bức ảnh lên hàng đợi, bởi hầu hết các message broker được thiết kế cho các thông điệp nhỏ, và một bức ảnh có thể nặng vài megabyte. Thay vào đó, bức ảnh trước tiên được ghi vào một dịch vụ lưu trữ tệp, và một khi lần ghi hoàn tất, chỉ thị tới bộ thay đổi kích thước mới được đặt lên hàng đợi.

Figure 9-5. The web server and image resizer communicate both through file storage and a message queue, opening the potential for race conditions. — Hình 9-5. Máy chủ web và bộ thay đổi kích thước ảnh giao tiếp qua cả lưu trữ tệp lẫn một hàng đợi thông điệp, mở ra khả năng xảy ra tranh chấp đua. If the file storage service is linearizable, then this system should work fine. If it is not linearizable, there is the risk of a race condition: the message queue (steps 3 and 4 in Figure 9-5) might be faster than the internal replication inside the storage service. In this case, when the resizer fetches the image (step 5), it might see an old version of the image, or nothing at all. If it processes an old version of the image, the full-size and resized images in the file storage become permanently inconsistent.

Nếu dịch vụ lưu trữ tệp là tuyến tính hóa, thì hệ thống này sẽ hoạt động ổn. Nếu nó không tuyến tính hóa, thì có nguy cơ xảy ra tranh chấp đua: hàng đợi thông điệp (bước 3 và 4 trong Hình 9-5) có thể nhanh hơn việc nhân bản nội bộ bên trong dịch vụ lưu trữ. Trong trường hợp này, khi bộ thay đổi kích thước lấy bức ảnh (bước 5), nó có thể thấy một phiên bản cũ của bức ảnh, hoặc không thấy gì cả. Nếu nó xử lý một phiên bản cũ của bức ảnh, thì ảnh kích thước đầy đủ và ảnh đã thay đổi kích thước trong lưu trữ tệp sẽ trở nên bất nhất vĩnh viễn.

This problem arises because there are two different communication channels between the web

server and the resizer: the file storage and the message queue. Without the recency guarantee of linearizability, race conditions between these two channels are possible. This situation is analogous to Figure 9-1, where there was also a race condition between two communication channels: the database replication and the real-life audio channel between Alice’s mouth and Bob’s ears.

Vấn đề này nảy sinh vì có hai kênh giao tiếp khác nhau giữa máy chủ web và bộ thay đổi kích thước: lưu trữ tệp và hàng đợi thông điệp. Không có bảo đảm tính tức thời của tính tuyến tính hóa, các tranh chấp đua giữa hai kênh này là có thể xảy ra. Tình huống này tương tự Hình 9-1, nơi cũng có một tranh chấp đua giữa hai kênh giao tiếp: việc nhân bản cơ sở dữ liệu và kênh âm thanh đời thực giữa miệng Alice và tai Bob.

Linearizability is not the only way of avoiding this race condition, but it’s the simplest to understand. If you control the additional communication channel (like in the case of the message queue, but not in the case of Alice and Bob), you can use alternative approaches similar to what we discussed in “Reading Your Own Writes”, at the cost of additional complexity.

Tính tuyến tính hóa không phải cách duy nhất để tránh tranh chấp đua này, nhưng nó là cách dễ hiểu nhất. Nếu bạn kiểm soát được kênh giao tiếp bổ sung (như trong trường hợp hàng đợi thông điệp, nhưng không phải trong trường hợp Alice và Bob), bạn có thể dùng các cách tiếp cận thay thế tương tự những gì đã bàn ở “Reading Your Own Writes”, với cái giá là độ phức tạp tăng thêm.

Implementing Linearizable Systems — Hiện thực các hệ thống tuyến tính hóa

Now that we’ve looked at a few examples in which linearizability is useful, let’s think about how we might implement a system that offers linearizable semantics.

Giờ ta đã xem qua vài ví dụ trong đó tính tuyến tính hóa hữu ích, hãy nghĩ về cách ta có thể hiện thực một hệ thống cung cấp ngữ nghĩa tuyến tính hóa.

Since linearizability essentially means “behave as though there is only a single copy of the data, and all operations on it are atomic,” the simplest answer would be to really only use a single copy of the data. However, that approach would not be able to tolerate faults: if the node holding that one copy failed, the data would be lost, or at least inaccessible until the node was brought up again.

Vì tính tuyến tính hóa về cơ bản nghĩa là “hành xử như thể chỉ có một bản dữ liệu duy nhất, và mọi thao tác trên nó đều là nguyên tử”, câu trả lời đơn giản nhất sẽ là thực sự chỉ dùng một bản dữ liệu duy nhất. Tuy nhiên, cách tiếp cận đó sẽ không thể chịu lỗi: nếu nút giữ bản dữ liệu duy nhất đó gặp sự cố, dữ liệu sẽ bị mất, hoặc ít nhất là không truy cập được cho tới khi nút đó được khởi động lại.

The most common approach to making a system fault-tolerant is to use replication. Let’s revisit the replication methods from Chapter 5, and compare whether they can be made linearizable:

Cách tiếp cận phổ biến nhất để làm cho một hệ thống chịu lỗi là dùng nhân bản. Hãy xem lại các phương pháp nhân bản từ Chương 5, và so sánh xem liệu chúng có thể được làm cho tuyến tính hóa hay không:

In a system with single-leader replication (see “Leaders and Followers”), the leader has the primary copy of the data that is used for writes, and the followers maintain **backup** copies of the data on other nodes. If you make reads from the leader, or from synchronously updated followers, they have the potential to be linearizable. However, not every single-leader database is actually linearizable, either by design (e.g., because it uses snapshot isolation) or due to concurrency bugs [10].

Trong một hệ thống nhân bản đơn-leader (xem “Leaders and Followers”), leader giữ bản dữ liệu chính được dùng cho các lần ghi, và các follower duy trì các bản sao lưu của dữ liệu trên các nút khác. Nếu bạn thực hiện các lần đọc từ leader, hoặc từ các follower được cập nhật đồng bộ, chúng có tiềm năng tuyến tính hóa. Tuy nhiên, không phải mọi cơ sở dữ liệu đơn-leader thực sự tuyến tính hóa, dù là do thiết kế (chẳng hạn vì nó dùng cô lập snapshot) hay do các bug về tương tranh [10].

Using the leader for reads relies on the assumption that you know for sure who the leader is. As discussed in “The Truth Is Defined by the Majority”, it is quite possible for a node to think that it is the leader, when in fact it is not—and if the delusional leader continues to serve requests, it is likely to violate linearizability [20]. With asynchronous replication, failover may even lose committed writes (see “Handling Node Outages”), which violates both durability and linearizability.

Việc dùng leader để đọc dựa trên giả định rằng bạn biết chắc chắn ai là leader. Như đã bàn ở “The Truth Is Defined by the Majority”, hoàn toàn có thể xảy ra việc một nút nghĩ rằng mình là leader, trong khi thực ra nó không phải — và nếu kẻ tưởng-bỏ-làm-leader đó tiếp tục phục vụ các yêu cầu, nhiều khả năng nó sẽ vi phạm tính tuyến tính hóa [20]. Với nhân bản bất đồng bộ, việc chuyển đổi dự phòng thậm chí có thể làm mất các lần ghi đã commit (xem “Handling Node Outages”), điều này vi phạm cả độ bền lẫn tính tuyến tính hóa.

Some consensus algorithms, which we will discuss later in this chapter, bear a resemblance to single-leader replication. However, consensus protocols contain measures to prevent split brain and stale replicas. Thanks to these details, consensus algorithms can implement linearizable storage safely. This is how ZooKeeper [21] and etcd [22] work, for example.

Một số thuật toán đồng thuận, mà ta sẽ bàn cuối chương này, có nét giống nhân bản đơn-leader. Tuy nhiên, các giao thức đồng thuận chứa các biện pháp để ngăn split brain và các bản sao cũ. Nhờ những chi tiết này, các thuật toán đồng thuận có thể hiện thực lưu trữ tuyến tính hóa một cách an toàn. Đây chẳng hạn là cách mà ZooKeeper [21] và etcd [22] hoạt động.

Systems with multi-leader replication are generally not linearizable, because they concurrently process writes on multiple nodes and asynchronously replicate them to other nodes. For this reason, they can produce conflicting writes that require resolution (see “Handling Write Conflicts”). Such conflicts are an artifact of the lack of a single copy of the data.

Các hệ thống dùng nhân bản đa-leader nói chung không tuyến tính hóa, bởi chúng xử lý các lần ghi đồng thời trên nhiều nút và nhân bản chúng một cách bất đồng bộ tới các nút khác. Vì lý do này, chúng có thể tạo ra các lần ghi xung đột cần được giải quyết (xem “Handling Write Conflicts”). Những xung đột như vậy là một sản phẩm phụ của việc thiếu một bản dữ liệu duy nhất.

For systems with leaderless replication (Dynamo-style; see “Leaderless Replication”), people sometimes claim that you can obtain “strong consistency” by requiring quorum reads and writes ($w + r > n$). Depending on the exact configuration of the quorums, and depending on how you define strong consistency, this is not quite true.

Với các hệ thống dùng nhân bản không-leader (kiểu Dynamo; xem “Leaderless Replication”), người ta đôi khi cho rằng bạn có thể đạt được “tính nhất quán mạnh” bằng cách yêu cầu các lần đọc và ghi theo nhóm túc số (quorum) ($w + r > n$). Tùy theo cấu hình chính xác của các nhóm túc số, và tùy theo cách bạn định nghĩa tính nhất quán mạnh, điều này không hẳn đúng.

“Last write wins” conflict resolution methods based on time-of-day clocks (e.g., in Cassandra; see “Relying on Synchronized Clocks”) are almost certainly nonlinearizable, because clock timestamps

cannot be guaranteed to be consistent with actual event ordering due to clock skew. Sloppy quorums (“Sloppy Quorums and Hinted Handoff”) also ruin any chance of linearizability. Even with strict quorums, nonlinearizable behavior is possible, as demonstrated in the next section.

Các phương pháp giải quyết xung đột kiểu “lần ghi cuối thắng” dựa trên đồng hồ thời gian-trong-ngày (chẳng hạn trong Cassandra; xem “Relying on Synchronized Clocks”) gần như chắc chắn là không tuyến tính hóa, bởi không thể bảo đảm các dấu thời gian của đồng hồ nhất quán với thứ tự sự kiện thực tế do lệch đồng hồ. Các nhóm túc số cầu thả (sloppy quorums) (“Sloppy Quorums and Hinted Handoff”) cũng phá hỏng mọi cơ hội đạt tính tuyến tính hóa. Ngay cả với các nhóm túc số nghiêm ngặt, hành vi không tuyến tính hóa vẫn có thể xảy ra, như sẽ được trình bày trong mục tiếp theo.

Linearizability and quorums — Tính tuyến tính hóa và các nhóm túc số

Intuitively, it seems as though strict quorum reads and writes should be linearizable in a Dynamo-style model. However, when we have variable network delays, it is possible to have race conditions, as demonstrated in Figure 9-6.

Theo trực giác, có vẻ như các lần đọc và ghi theo nhóm túc số nghiêm ngặt nên tuyến tính hóa trong một mô hình kiểu Dynamo. Tuy nhiên, khi ta có các trì hoãn mạng biến thiên, vẫn có thể xảy ra các tranh chấp đua, như được trình bày trong Hình 9-6.

Figure 9-6. A nonlinearizable execution, despite using a strict quorum. — Hình 9-6. Một lượt thực thi không tuyến tính hóa, dù dùng một nhóm túc số nghiêm ngặt. In Figure 9-6, the initial value of x is 0, and a writer client is updating x to 1 by sending the write to all three replicas ($n = 3$, $w = 3$). Concurrently, client A reads from a quorum of two nodes ($r = 2$) and sees the new value 1 on one of the nodes. Also concurrently with the write, client B reads from a different quorum of two nodes, and gets back the old value 0 from both.

Trong Hình 9-6, giá trị ban đầu của x là 0, và một client ghi đang cập nhật x thành 1 bằng cách gửi lần ghi tới cả ba bản sao ($n = 3$, $w = 3$). Đồng thời, client A đọc từ một nhóm túc số gồm hai nút ($r = 2$) và thấy giá trị mới 1 trên một trong các nút. Cũng đồng thời với lần ghi, client B đọc từ một nhóm túc số khác gồm hai nút, và nhận lại giá trị cũ 0 từ cả hai.

The quorum condition is met ($w + r > n$), but this execution is nevertheless not linearizable: B’s request begins after A’s request completes, but B returns the old value while A returns the new value. (It’s once again the Alice and Bob situation from Figure 9-1.)

Điều kiện nhóm túc số được thỏa mãn ($w + r > n$), nhưng lượt thực thi này dù vậy vẫn không tuyến tính hóa: yêu cầu của B bắt đầu sau khi yêu cầu của A hoàn tất, nhưng B trả về giá trị cũ trong khi A trả về giá trị mới. (Một lần nữa lại là tình huống Alice và Bob từ Hình 9-1.)

Interestingly, it is possible to make Dynamo-style quorums linearizable at the cost of reduced performance: a reader must perform read repair (see “Read repair and anti-entropy”) synchronously, before returning results to the application [23], and a writer must read the latest state of a quorum of nodes before sending its writes [24, 25]. However, Riak does not perform synchronous read repair due to the performance penalty [26]. Cassandra does wait for read repair to complete on quorum reads [27], but it loses linearizability if there are multiple concurrent writes to the same key, due to its use of last-write-wins conflict resolution.

Thú vị là, có thể làm cho các nhóm túc số kiểu Dynamo tuyến tính hóa với cái giá là hiệu năng giảm: một người đọc phải thực hiện sửa-khi-đọc (read repair) (xem “Read repair and

anti-entropy”) một cách đồng bộ, trước khi trả kết quả về cho ứng dụng [23], và một người ghi phải đọc trạng thái mới nhất của một nhóm tức số các nút trước khi gửi các lần ghi của mình [24, 25]. Tuy nhiên, Riak không thực hiện sửa-khi-đọc đồng bộ vì hình phạt về hiệu năng [26]. Cassandra thì có chờ sửa-khi-đọc hoàn tất ở các lần đọc theo nhóm tức số [27], nhưng nó mất tính tuyến tính hóa nếu có nhiều lần ghi đồng thời tới cùng một khóa, do nó dùng giải quyết xung đột kiểu lần-ghi-cuối-thắng.

Moreover, only linearizable read and write operations can be implemented in this way; a linearizable compare-and-set operation cannot, because it requires a consensus algorithm [28].

Hơn nữa, chỉ các thao tác đọc và ghi tuyến tính hóa mới có thể được hiện thực theo cách này; một thao tác so-sánh-rồi-đặt tuyến tính hóa thì không, bởi nó đòi hỏi một thuật toán đồng thuận [28].

In summary, it is safest to assume that a leaderless system with Dynamo-style replication does not provide linearizability.

Tóm lại, an toàn nhất là giả định rằng một hệ thống không-leader với nhân bản kiểu Dynamo không cung cấp tính tuyến tính hóa.

The Cost of Linearizability — Cái giá của tính tuyến tính hóa

As some replication methods can provide linearizability and others cannot, it is interesting to explore the pros and cons of linearizability in more depth.

Vì một số phương pháp nhân bản có thể cung cấp tính tuyến tính hóa còn số khác thì không, sẽ thú vị nếu khám phá sâu hơn các ưu nhược điểm của tính tuyến tính hóa.

We already discussed some use cases for different replication methods in Chapter 5; for example, we saw that multi-leader replication is often a good choice for multi-**datacenter** replication (see “Multi-datacenter operation”). An example of such a **deployment** is illustrated in Figure 9-7.

Ta đã bàn về một số tình huống sử dụng cho các phương pháp nhân bản khác nhau ở Chương 5; ví dụ, ta đã thấy rằng nhân bản đa-leader thường là lựa chọn tốt cho nhân bản đa-trung-tâm-dữ-liệu (xem “Multi-datacenter operation”). Một ví dụ về một triển khai như vậy được minh họa trong Hình 9-7.

Figure 9-7. A network interruption forcing a choice between linearizability and availability.
— **Hình 9-7. Một gián đoạn mạng buộc phải chọn giữa tính tuyến tính hóa và tính sẵn sàng.**
Consider what happens if there is a network interruption between the two datacenters. Let’s assume that the network within each datacenter is working, and clients can reach the datacenters, but the datacenters cannot connect to each other.

Hãy xem điều gì xảy ra nếu có một gián đoạn mạng giữa hai trung tâm dữ liệu. Giả sử mạng bên trong mỗi trung tâm dữ liệu vẫn hoạt động, và các client có thể kết nối tới các trung tâm dữ liệu, nhưng các trung tâm dữ liệu không thể kết nối với nhau.

With a multi-leader database, each datacenter can continue operating normally: since writes from one datacenter are asynchronously replicated to the other, the writes are simply queued up and exchanged when network connectivity is restored.

Với một cơ sở dữ liệu đa-leader, mỗi trung tâm dữ liệu có thể tiếp tục hoạt động bình thường; vì các lần ghi từ một trung tâm dữ liệu được nhân bản bất đồng bộ tới trung tâm kia, các lần ghi đơn giản được xếp hàng lại và trao đổi khi kết nối mạng được khôi phục.

On the other hand, if single-leader replication is used, then the leader must be in one of the datacenters. Any writes and any linearizable reads must be sent to the leader—thus, for any clients connected to a **follower** datacenter, those read and write requests must be sent synchronously over the network to the leader datacenter.

Mặt khác, nếu dùng nhân bản đơn-leader, thì leader phải nằm ở một trong các trung tâm dữ liệu. Mọi lần ghi và mọi lần đọc tuyến tính hóa đều phải được gửi tới leader — do đó, với bất kỳ client nào kết nối tới một trung tâm dữ liệu follower, các yêu cầu đọc và ghi đó phải được gửi đồng bộ qua mạng tới trung tâm dữ liệu leader.

If the network between datacenters is interrupted in a single-leader setup, clients connected to follower datacenters cannot contact the leader, so they cannot make any writes to the database, nor any linearizable reads. They can still make reads from the follower, but they might be stale (nonlinearizable). If the application requires linearizable reads and writes, the network interruption causes the application to become unavailable in the datacenters that cannot contact the leader.

Nếu mạng giữa các trung tâm dữ liệu bị gián đoạn trong một cấu hình đơn-leader, các client kết nối tới các trung tâm dữ liệu follower không thể liên lạc với leader, nên chúng không thể thực hiện bất kỳ lần ghi nào vào cơ sở dữ liệu, cũng không thể thực hiện bất kỳ lần đọc tuyến tính hóa nào. Chúng vẫn có thể thực hiện các lần đọc từ follower, nhưng những lần đọc đó có thể bị cũ (không tuyến tính hóa). Nếu ứng dụng đòi hỏi các lần đọc và ghi tuyến tính hóa, gián đoạn mạng khiến ứng dụng trở nên không sẵn sàng ở các trung tâm dữ liệu không thể liên lạc với leader.

If clients can connect directly to the leader datacenter, this is not a problem, since the application continues to work normally there. But clients that can only reach a follower datacenter will experience an **outage** until the network link is repaired.

Nếu các client có thể kết nối trực tiếp tới trung tâm dữ liệu leader, thì đây không phải vấn đề, bởi ứng dụng tiếp tục hoạt động bình thường ở đó. Nhưng các client chỉ có thể đến được một trung tâm dữ liệu follower sẽ trải qua một gián đoạn dịch vụ cho tới khi đường truyền mạng được sửa.

The CAP theorem — Định lý CAP

This issue is not just a consequence of single-leader and multi-leader replication: any linearizable database has this problem, no matter how it is implemented. The issue also isn't specific to multi-datacenter deployments, but can occur on any unreliable network, even within one datacenter. The trade-off is as follows:

Vấn đề này không chỉ là hệ quả của nhân bản đơn-leader và đa-leader: bất kỳ cơ sở dữ liệu tuyến tính hóa nào cũng có vấn đề này, bất kể nó được hiện thực ra sao. Vấn đề cũng không đặc thù cho các triển khai đa-trung-tâm-dữ-liệu, mà có thể xảy ra trên bất kỳ mạng không đáng tin cậy nào, ngay cả trong một trung tâm dữ liệu. Sự đánh đổi như sau:

- If your application requires linearizability, and some replicas are disconnected from the other replicas due to a network problem, then some replicas cannot process requests while they are disconnected: they must either wait until the network problem is fixed, or return an error (either way, they become unavailable).
- If your application does not require linearizability, then it can be written in a way that each replica can process requests independently, even if it is disconnected from other replicas (e.g., multi-leader). In this case, the application can remain available in the face of a network problem, but its behavior is not linearizable.

- Nếu ứng dụng của bạn đòi hỏi tính tuyến tính hóa, và một số bản sao bị ngắt kết nối với các bản sao khác do một vấn đề mạng, thì một số bản sao không thể xử lý các yêu cầu trong khi chúng bị ngắt kết nối: chúng phải hoặc chờ tới khi vấn đề mạng được khắc phục, hoặc trả về một lỗi (dù cách nào thì chúng cũng trở nên không sẵn sàng).
- Nếu ứng dụng của bạn không đòi hỏi tính tuyến tính hóa, thì nó có thể được viết theo cách mà mỗi bản sao có thể xử lý các yêu cầu một cách độc lập, ngay cả khi nó bị ngắt kết nối với các bản sao khác (chẳng hạn đa-leader). Trong trường hợp này, ứng dụng có thể vẫn sẵn sàng khi đối mặt với một vấn đề mạng, nhưng hành vi của nó không tuyến tính hóa.

Thus, applications that don't require linearizability can be more tolerant of network problems. This insight is popularly known as the CAP theorem [29, 30, 31, 32], named by Eric Brewer in 2000, although the trade-off has been known to designers of distributed databases since the 1970s [33, 34, 35, 36].

Do đó, các ứng dụng không đòi hỏi tính tuyến tính hóa có thể chịu đựng các vấn đề mạng tốt hơn. Nhận định này thường được biết đến dưới tên định lý CAP [29, 30, 31, 32], được Eric Brewer đặt tên năm 2000, dù sự đánh đổi này đã được những người thiết kế cơ sở dữ liệu phân tán biết đến từ thập niên 1970 [33, 34, 35, 36].

CAP was originally proposed as a rule of thumb, without precise definitions, with the goal of starting a discussion about trade-offs in databases. At the time, many distributed databases focused on providing linearizable semantics on a cluster of machines with shared storage [18], and CAP encouraged database engineers to explore a wider design space of distributed shared-nothing systems, which were more suitable for implementing large-scale web services [37]. CAP deserves credit for this culture shift—witness the explosion of new database technologies since the mid-2000s (known as **NoSQL**).

CAP ban đầu được đề xuất như một quy tắc kinh nghiệm, không có định nghĩa chính xác, với mục tiêu khơi mào một cuộc thảo luận về các đánh đổi trong cơ sở dữ liệu. Vào thời điểm đó, nhiều cơ sở dữ liệu phân tán tập trung vào việc cung cấp ngữ nghĩa tuyến tính hóa trên một cụm máy với lưu trữ dùng chung [18], và CAP đã khuyến khích các kỹ sư cơ sở dữ liệu khám phá một không gian thiết kế rộng hơn của các hệ thống phân tán shared-nothing, vốn phù hợp hơn cho việc hiện thực các dịch vụ web quy mô lớn [37]. CAP xứng đáng được ghi công cho sự chuyển dịch văn hóa này — minh chứng là sự bùng nổ của các công nghệ cơ sở dữ liệu mới kể từ giữa những năm 2000 (được biết đến là NoSQL).

The Unhelpful CAP Theorem — Định lý CAP vô bổ CAP is sometimes presented as Consistency, Availability, Partition tolerance: pick 2 out of 3. Unfortunately, putting it this way is misleading [32] because network partitions are a kind of fault, so they aren't something about which you have a choice: they will happen whether you like it or not [38].

CAP đôi khi được trình bày dưới dạng Consistency (Nhất quán), Availability (Sẵn sàng), Partition tolerance (Chịu phân vùng): chọn 2 trong 3. Đáng tiếc, trình bày theo kiểu này gây hiểu lầm [32] bởi các phân vùng mạng là một loại lỗi, nên chúng không phải thứ bạn có thể lựa chọn: chúng sẽ xảy ra dù bạn thích hay không [38].

At times when the network is working correctly, a system can provide both consistency (linearizability) and total availability. When a network fault occurs, you have to choose between either linearizability or total availability. Thus, a better way of phrasing CAP would be either Consistent or Available when Partitioned [39]. A more reliable network needs to make this choice less often, but at some point the choice is inevitable.

Vào những lúc mạng hoạt động đúng đắn, một hệ thống có thể cung cấp cả tính nhất quán (tuyến tính hóa) lẫn tính sẵn sàng toàn phần. Khi một lỗi mạng xảy ra, bạn phải chọn giữa hoặc tính tuyến tính hóa hoặc tính sẵn sàng toàn phần. Do đó, một cách diễn đạt CAP hay hơn sẽ là hoặc Nhất quán hoặc Sẵn sàng khi bị Phân vùng [39]. Một mạng đáng tin cậy hơn cần đưa ra lựa chọn này ít thường xuyên hơn, nhưng tới một lúc nào đó thì lựa chọn là không thể tránh khỏi.

In discussions of CAP there are several contradictory definitions of the term availability, and the formalization as a theorem [30] does not match its usual meaning [40]. Many so-called “highly available” (fault-tolerant) systems actually do not meet CAP’s idiosyncratic definition of availability. All in all, there is a lot of misunderstanding and confusion around CAP, and it does not help us understand systems better, so CAP is best avoided.

Trong các cuộc thảo luận về CAP có vài định nghĩa mâu thuẫn nhau cho thuật ngữ tính sẵn sàng, và việc hình thức hóa nó thành một định lý [30] không khớp với nghĩa thông thường của nó [40]. Nhiều hệ thống được gọi là “tính sẵn sàng cao” (chịu lỗi) thực ra không đáp ứng định nghĩa kỳ dị của CAP về tính sẵn sàng. Tựu trung lại, có rất nhiều sự hiểu lầm và lẫn lộn xoay quanh CAP, và nó không giúp ta hiểu các hệ thống tốt hơn, nên tốt nhất là tránh dùng CAP.

The CAP theorem as formally defined [30] is of very narrow scope: it only considers one consistency model (namely linearizability) and one kind of fault (network partitions, or nodes that are alive but disconnected from each other). It doesn’t say anything about network delays, dead nodes, or other trade-offs. Thus, although CAP has been historically influential, it has little practical value for designing systems [9, 40].

Định lý CAP theo định nghĩa hình thức [30] có phạm vi rất hẹp: nó chỉ xét một mô hình nhất quán (cụ thể là tính tuyến tính hóa) và một loại lỗi (phân vùng mạng, hay các nút còn sống nhưng bị ngắt kết nối với nhau). Nó không nói gì về các trì hoãn mạng, các nút chết, hay các đánh đổi khác. Do đó, dù CAP từng có ảnh hưởng về mặt lịch sử, nó có ít giá trị thực tiễn cho việc thiết kế hệ thống [9, 40].

There are many more interesting impossibility results in distributed systems [41], and CAP has now been superseded by more precise results [2, 42], so it is of mostly historical interest today.

Có nhiều kết quả bất khả thi thú vị hơn nữa trong các hệ thống phân tán [41], và CAP giờ đã bị thay thế bởi các kết quả chính xác hơn [2, 42], nên ngày nay nó chủ yếu chỉ còn giá trị về mặt lịch sử.

Linearizability and network delays — Tính tuyến tính hóa và các trì hoãn mạng

Although linearizability is a useful guarantee, surprisingly few systems are actually linearizable in practice. For example, even RAM on a modern multi-core CPU is not linearizable [43]: if a thread running on one CPU core writes to a memory address, and a thread on another CPU core reads the same address shortly afterward, it is not guaranteed to read the value written by the first thread (unless a memory barrier or fence [44] is used).

Mặc dù tính tuyến tính hóa là một bảo đảm hữu ích, đáng ngạc nhiên là rất ít hệ thống thực sự tuyến tính hóa trong thực tế. Ví dụ, ngay cả RAM trên một CPU đa nhân hiện đại cũng không tuyến tính hóa [43]: nếu một luồng chạy trên một nhân CPU ghi vào một địa chỉ bộ nhớ, và một luồng trên một nhân CPU khác đọc cùng địa chỉ đó ngay sau đó, không có bảo đảm rằng nó sẽ đọc được giá trị do luồng thứ nhất ghi (trừ khi dùng một rào chắn bộ nhớ (memory barrier) hay fence [44]).

The reason for this behavior is that every CPU core has its own memory cache and store buffer. Memory access first goes to the cache by default, and any changes are asynchronously written out to main memory. Since accessing data in the cache is much faster than going to main memory [45], this feature is essential for good performance on modern CPUs. However, there are now several copies of the data (one in main memory, and perhaps several more in various caches), and these copies are asynchronously updated, so linearizability is lost.

Lý do của hành vi này là mỗi nhân CPU có bộ nhớ đệm và bộ đệm ghi riêng của nó. Truy cập bộ nhớ trước tiên đi tới bộ nhớ đệm theo mặc định, và mọi thay đổi được ghi ra bộ nhớ chính một cách bất đồng bộ. Vì truy cập dữ liệu trong bộ nhớ đệm nhanh hơn nhiều so với đi tới bộ nhớ chính [45], tính năng này là thiết yếu để có hiệu năng tốt trên các CPU hiện đại. Tuy nhiên, giờ có nhiều bản sao của dữ liệu (một trong bộ nhớ chính, và có thể vài bản nữa trong các bộ nhớ đệm khác nhau), và các bản sao này được cập nhật bất đồng bộ, nên tính tuyến tính hóa bị mất.

Why make this trade-off? It makes no sense to use the CAP theorem to justify the multi-core memory consistency model: within one computer we usually assume reliable communication, and we don't expect one CPU core to be able to continue operating normally if it is disconnected from the rest of the computer. The reason for dropping linearizability is performance, not fault tolerance.

Tại sao lại đánh đổi như vậy? Việc dùng định lý CAP để biện minh cho mô hình nhất quán bộ nhớ đa nhân là vô nghĩa: bên trong một máy tính ta thường giả định việc giao tiếp là đáng tin cậy, và ta không kỳ vọng một nhân CPU có thể tiếp tục hoạt động bình thường nếu nó bị ngắt kết nối với phần còn lại của máy tính. Lý do từ bỏ tính tuyến tính hóa là hiệu năng, chứ không phải khả năng chịu lỗi.

The same is true of many distributed databases that choose not to provide linearizable guarantees: they do so primarily to increase performance, not so much for fault tolerance [46]. Linearizability is slow—and this is true all the time, not only during a network fault.

Điều tương tự cũng đúng với nhiều cơ sở dữ liệu phân tán chọn không cung cấp các bảo đảm tuyến tính hóa: chúng làm vậy chủ yếu để tăng hiệu năng, chứ không hẳn vì khả năng chịu lỗi [46]. Tính tuyến tính hóa thì chậm — và điều này đúng mọi lúc, không chỉ trong lúc có một lỗi mạng.

Can't we maybe find a more efficient implementation of linearizable storage? It seems the answer is no: Attiya and Welch [47] prove that if you want linearizability, the **response time** of read and write requests is at least proportional to the uncertainty of delays in the network. In a network with highly variable delays, like most computer networks (see “Timeouts and Unbounded Delays”), the response time of linearizable reads and writes is inevitably going to be high. A faster algorithm for linearizability does not exist, but weaker consistency models can be much faster, so this trade-off is important for **latency**-sensitive systems. In Chapter 12 we will discuss some approaches for avoiding linearizability without sacrificing correctness.

Liệu ta có thể tìm ra một hiện thực hiệu quả hơn cho lưu trữ tuyến tính hóa hay không? Có vẻ câu trả lời là không: Attiya và Welch [47] chứng minh rằng nếu bạn muốn tính tuyến tính hóa, thời gian phản hồi của các yêu cầu đọc và ghi ít nhất tỷ lệ thuận với độ bất định của các trì hoãn trên mạng. Trong một mạng với các trì hoãn biến thiên cao, như hầu hết các mạng máy tính (xem “Timeouts and Unbounded Delays”), thời gian phản hồi của các lần đọc và ghi tuyến tính hóa chắc chắn sẽ cao. Một thuật toán nhanh hơn cho tính tuyến tính hóa không tồn tại, nhưng các mô hình nhất quán yếu hơn có thể nhanh hơn nhiều, nên sự đánh đổi này quan trọng đối với các hệ thống nhạy cảm với độ trễ. Ở Chương 12 ta sẽ bàn về một số cách tiếp cận để tránh tính tuyến tính hóa mà không hy sinh tính đúng đắn.

Ordering Guarantees — Các bảo đảm về sắp thứ tự

We said previously that a linearizable register behaves as if there is only a single copy of the data, and that every operation appears to take effect atomically at one point in time. This definition implies that operations are executed in some well-defined order. We illustrated the ordering in Figure 9-4 by joining up the operations in the order in which they seem to have executed.

Trước đó ta đã nói rằng một thanh ghi tuyến tính hóa hành xử như thể chỉ có một bản dữ liệu duy nhất, và rằng mỗi thao tác có vẻ có hiệu lực một cách nguyên tử tại một thời điểm. Định nghĩa này hàm ý rằng các thao tác được thực thi theo một thứ tự nào đó được xác định rõ ràng. Ta đã minh họa thứ tự đó trong Hình 9-4 bằng cách nối các thao tác lại theo thứ tự mà chúng có vẻ đã được thực thi.

Ordering has been a recurring theme in this book, which suggests that it might be an important fundamental idea. Let’s briefly recap some of the other contexts in which we have discussed ordering:

Sắp thứ tự là một chủ đề lặp đi lặp lại trong cuốn sách này, điều đó gợi ý rằng nó có thể là một ý tưởng nền tảng quan trọng. Hãy điểm lại ngắn gọn một số bối cảnh khác trong đó ta đã bàn về sắp thứ tự:

- In Chapter 5 we saw that the main purpose of the leader in single-leader replication is to determine the order of writes in the replication log—that is, the order in which followers apply those writes. If there is no single leader, conflicts can occur due to concurrent operations (see “Handling Write Conflicts”).
- Serializability, which we discussed in Chapter 7, is about ensuring that transactions behave as if they were executed in some sequential order. It can be achieved by literally executing transactions in that serial order, or by allowing concurrent execution while preventing serialization conflicts (by locking or aborting).
- The use of timestamps and clocks in distributed systems that we discussed in Chapter 8 (see “Relying on Synchronized Clocks”) is another attempt to introduce order into a disorderly world, for example to determine which one of two writes happened later.
 - Ở Chương 5 ta đã thấy rằng mục đích chính của leader trong nhân bản đơn-leader là xác định thứ tự của các lần ghi trong nhật ký nhân bản — tức thứ tự mà các follower áp dụng các lần ghi đó. Nếu không có một leader duy nhất, các xung đột có thể xảy ra do các thao tác đồng thời (xem “Handling Write Conflicts”).
 - Tính tuần tự hóa, mà ta đã bàn ở Chương 7, là về việc bảo đảm rằng các giao dịch hành xử như thể chúng được thực thi theo một thứ tự tuần tự nào đó. Nó có thể đạt được bằng cách thực thi các giao dịch theo đúng thứ tự tuần tự đó, hoặc bằng cách cho phép thực thi đồng thời trong khi ngăn các xung đột tuần tự hóa (bằng cách khóa

hoặc hủy bỏ).

- Việc dùng các dấu thời gian và đồng hồ trong các hệ thống phân tán mà ta đã bàn ở Chương 8 (xem “Relying on Synchronized Clocks”) là một nỗ lực khác nhằm đưa thứ tự vào một thế giới hỗn loạn, ví dụ để xác định lần ghi nào trong hai lần ghi đã xảy ra sau.

It turns out that there are deep connections between ordering, linearizability, and consensus. Although this notion is a bit more theoretical and abstract than the rest of this book, it is very helpful for clarifying our understanding of what systems can and cannot do. We will explore this topic in the next few sections.

Hóa ra có những mối liên hệ sâu sắc giữa sắp thứ tự, tính tuyến tính hóa, và đồng thuận. Mặc dù khái niệm này có phần thiên về lý thuyết và trừu tượng hơn so với phần còn lại của cuốn sách, nó rất hữu ích để làm sáng tỏ hiểu biết của ta về những gì các hệ thống có thể và không thể làm. Ta sẽ khám phá chủ đề này trong vài mục tiếp theo.

Ordering and Causality — Sắp thứ tự và tính nhân quả

There are several reasons why ordering keeps coming up, and one of the reasons is that it helps preserve causality. We have already seen several examples over the course of this book where causality has been important:

Có một số lý do khiến sắp thứ tự cứ liên tục xuất hiện, và một trong những lý do là nó giúp bảo toàn tính nhân quả (causality). Ta đã thấy vài ví dụ xuyên suốt cuốn sách này trong đó tính nhân quả là quan trọng:

- In “Consistent Prefix Reads” (Figure 5-5) we saw an example where the observer of a conversation saw first the answer to a question, and then the question being answered. This is confusing because it violates our intuition of cause and effect: if a question is answered, then clearly the question had to be there first, because the person giving the answer must have seen the question (assuming they are not psychic and cannot see into the future). We say that there is a causal dependency between the question and the answer.
- A similar pattern appeared in Figure 5-9, where we looked at the replication between three leaders and noticed that some writes could “overtake” others due to network delays. From the perspective of one of the replicas it would look as though there was an update to a row that did not exist. Causality here means that a row must first be created before it can be updated.
- In “Detecting Concurrent Writes” we observed that if you have two operations A and B, there are three possibilities: either A happened before B, or B happened before A, or A and B are concurrent. This happened before relationship is another expression of causality: if A happened before B, that means B might have known about A, or built upon A, or depended on A. If A and B are concurrent, there is no causal link between them; in other words, we are sure that neither knew about the other.
- In the context of snapshot isolation for transactions (“Snapshot Isolation and Repeatable Read”), we said that a transaction reads from a consistent snapshot. But what does “consistent” mean in this context? It means consistent with causality: if the snapshot contains an answer, it must also contain the question being answered [48]. Observing the entire database at a single point in time makes it consistent with causality: the effects of all operations that happened causally before that point in time are visible, but no operations that happened causally afterward can be seen. Read skew (non-repeatable reads, as illustrated in Figure 7-6) means reading data in a state that violates causality.
- Our examples of write skew between transactions (see “Write Skew and Phantoms”) also

demonstrated causal dependencies: in Figure 7-8, Alice was allowed to go off call because the transaction thought that Bob was still on call, and vice versa. In this case, the action of going off call is causally dependent on the observation of who is currently on call. Serializable snapshot isolation (see “Serializable Snapshot Isolation (SSI)”) detects write skew by tracking the causal dependencies between transactions.

- In the example of Alice and Bob watching football (Figure 9-1), the fact that Bob got a stale result from the server after hearing Alice exclaim the result is a causality violation: Alice’s exclamation is causally dependent on the announcement of the score, so Bob should also be able to see the score after hearing Alice. The same pattern appeared again in “Cross-channel timing dependencies” in the guise of an image resizing service.

- Trong “Consistent Prefix Reads” (Hình 5-5) ta đã thấy một ví dụ trong đó người quan sát một cuộc trò chuyện thấy trước câu trả lời cho một câu hỏi, rồi mới thấy câu hỏi được trả lời. Điều này khó hiểu vì nó vi phạm trực giác của ta về nhân và quả: nếu một câu hỏi được trả lời, thì rõ ràng câu hỏi phải có trước, bởi người đưa ra câu trả lời hẳn đã thấy câu hỏi (giả định họ không phải nhà ngoại cảm và không thể nhìn thấy tương lai). Ta nói rằng có một phụ thuộc nhân quả giữa câu hỏi và câu trả lời.
- Một mẫu tương tự xuất hiện trong Hình 5-9, nơi ta xem xét việc nhân bản giữa ba leader và nhận thấy rằng một số lần ghi có thể “vượt mặt” các lần ghi khác do các trì hoãn mạng. Từ góc nhìn của một trong các bản sao, có vẻ như có một cập nhật tới một hàng vốn không tồn tại. Tính nhân quả ở đây nghĩa là một hàng phải được tạo ra trước rồi mới có thể được cập nhật.
- Trong “Detecting Concurrent Writes” ta đã quan sát rằng nếu bạn có hai thao tác A và B, thì có ba khả năng: hoặc A xảy ra trước B, hoặc B xảy ra trước A, hoặc A và B đồng thời. Quan hệ xảy-ra-trước (happened before) này là một cách diễn đạt khác của tính nhân quả: nếu A xảy ra trước B, điều đó nghĩa là B có thể đã biết về A, hoặc xây dựng dựa trên A, hoặc phụ thuộc vào A. Nếu A và B đồng thời, không có mối liên hệ nhân quả nào giữa chúng; nói cách khác, ta chắc chắn rằng không cái nào biết về cái kia.
- Trong bối cảnh cô lập snapshot cho các giao dịch (“Snapshot Isolation and Repeatable Read”), ta đã nói rằng một giao dịch đọc từ một snapshot nhất quán. Nhưng “nhất quán” nghĩa là gì trong bối cảnh này? Nó nghĩa là nhất quán với tính nhân quả: nếu snapshot chứa một câu trả lời, nó cũng phải chứa câu hỏi được trả lời [48]. Việc quan sát toàn bộ cơ sở dữ liệu tại một thời điểm duy nhất khiến nó nhất quán với tính nhân quả: tác động của tất cả các thao tác đã xảy ra về mặt nhân quả trước thời điểm đó đều hiển hiện, nhưng không thao tác nào xảy ra về mặt nhân quả sau đó có thể được thấy. Đọc lệch (read skew) (đọc không lặp lại được, như được minh họa trong Hình 7-6) nghĩa là đọc dữ liệu ở một trạng thái vi phạm tính nhân quả.
- Các ví dụ của ta về lệch ghi giữa các giao dịch (xem “Write Skew and Phantoms”) cũng cho thấy các phụ thuộc nhân quả: trong Hình 7-8, Alice được phép nghỉ trực vì giao dịch nghĩ rằng Bob vẫn đang trực, và ngược lại. Trong trường hợp này, hành động nghỉ trực phụ thuộc về mặt nhân quả vào sự quan sát về việc ai đang trực. Cô lập snapshot tuần tự hóa (xem “Serializable Snapshot Isolation (SSI)”) phát hiện lệch ghi bằng cách theo dõi các phụ thuộc nhân quả giữa các giao dịch.
- Trong ví dụ Alice và Bob xem bóng đá (Hình 9-1), việc Bob nhận được một kết quả cũ từ máy chủ sau khi nghe Alice reo lên kết quả là một vi phạm tính nhân quả: tiếng reo của Alice phụ thuộc về mặt nhân quả vào việc công bố tỷ số, nên Bob cũng phải có khả năng thấy tỷ số sau khi nghe Alice. Mẫu tương tự lại xuất hiện trong “Cross-channel timing dependencies” dưới lát một dịch vụ thay đổi kích thước ảnh.

Causality imposes an ordering on events: cause comes before effect; a message is sent before that message is received; the question comes before the answer. And, like in real life, one thing leads

to another: one node reads some data and then writes something as a result, another node reads the thing that was written and writes something else in turn, and so on. These chains of causally dependent operations define the causal order in the system—i.e., what happened before what.

Tính nhân quả áp đặt một thứ tự lên các sự kiện: nhân đến trước quả; một thông điệp được gửi trước khi thông điệp đó được nhận; câu hỏi đến trước câu trả lời. Và, như trong đời thực, việc này dẫn tới việc kia: một nút đọc một số dữ liệu rồi ghi ra một thứ gì đó như kết quả, một nút khác đọc thứ vừa được ghi và đến lượt nó lại ghi ra thứ khác, và cứ thế. Những chuỗi thao tác phụ thuộc nhân quả này định nghĩa thứ tự nhân quả trong hệ thống — tức cái gì xảy ra trước cái gì.

If a system obeys the ordering imposed by causality, we say that it is causally consistent. For example, snapshot isolation provides causal consistency: when you read from the database, and you see some piece of data, then you must also be able to see any data that causally precedes it (assuming it has not been deleted in the meantime).

Nếu một hệ thống tuân theo thứ tự do tính nhân quả áp đặt, ta nói rằng nó nhất quán nhân quả (causally consistent). Ví dụ, cô lập snapshot cung cấp tính nhất quán nhân quả: khi bạn đọc từ cơ sở dữ liệu, và bạn thấy một mẫu dữ liệu nào đó, thì bạn cũng phải có khả năng thấy mọi dữ liệu đi trước nó về mặt nhân quả (giả định nó chưa bị xóa trong khoảng thời gian đó).

The causal order is not a total order — Thứ tự nhân quả không phải thứ tự toàn phần

A total order allows any two elements to be compared, so if you have two elements, you can always say which one is greater and which one is smaller. For example, natural numbers are totally ordered: if I give you any two numbers, say 5 and 13, you can tell me that 13 is greater than 5.

Một thứ tự toàn phần (total order) cho phép so sánh hai phần tử bất kỳ, nên nếu bạn có hai phần tử, bạn luôn có thể nói cái nào lớn hơn và cái nào nhỏ hơn. Ví dụ, các số tự nhiên được sắp thứ tự toàn phần: nếu tôi cho bạn hai số bất kỳ, chẳng hạn 5 và 13, bạn có thể nói với tôi rằng 13 lớn hơn 5.

However, mathematical sets are not totally ordered: is $\{a, b\}$ greater than $\{b, c\}$? Well, you can't really compare them, because neither is a subset of the other. We say they are incomparable, and therefore mathematical sets are partially ordered: in some cases one set is greater than another (if one set contains all the elements of another), but in other cases they are incomparable.

Tuy nhiên, các tập hợp toán học thì không được sắp thứ tự toàn phần: liệu $\{a, b\}$ có lớn hơn $\{b, c\}$ không? Bạn thực sự không thể so sánh chúng, bởi không cái nào là tập con của cái kia. Ta nói chúng là không so sánh được, và do đó các tập hợp toán học được sắp thứ tự bộ phận (partially ordered): trong một số trường hợp, một tập lớn hơn tập khác (nếu một tập chứa tất cả các phần tử của tập kia), nhưng trong các trường hợp khác chúng không so sánh được.

The difference between a total order and a partial order is reflected in different database consistency models:

Sự khác biệt giữa một thứ tự toàn phần và một thứ tự bộ phận được phản ánh trong các mô hình nhất quán cơ sở dữ liệu khác nhau:

In a linearizable system, we have a total order of operations: if the system behaves as if there is only a single copy of the data, and every operation is atomic, this means that for any two operations we can always say which one happened first. This total ordering is illustrated as a timeline in Figure 9-4.

Trong một hệ thống tuyến tính hóa, ta có một thứ tự toàn phần của các thao tác: nếu hệ thống hành xử như thế chỉ có một bản dữ liệu duy nhất, và mọi thao tác là nguyên tử, thì với hai thao tác bất kỳ ta luôn có thể nói cái nào xảy ra trước. Thứ tự toàn phần này được minh họa dưới dạng một dòng thời gian trong Hình 9-4.

We said that two operations are concurrent if neither happened before the other (see “The “happens-before” relationship and concurrency”). Put another way, two events are ordered if they are causally related (one happened before the other), but they are incomparable if they are concurrent. This means that causality defines a partial order, not a total order: some operations are ordered with respect to each other, but some are incomparable.

Ta đã nói rằng hai thao tác là đồng thời nếu không cái nào xảy ra trước cái kia (xem “The happens-before relationship and concurrency”). Nói cách khác, hai sự kiện được sắp thứ tự nếu chúng có quan hệ nhân quả (một cái xảy ra trước cái kia), nhưng chúng không so sánh được nếu chúng đồng thời. Điều này nghĩa là tính nhân quả định nghĩa một thứ tự bộ phận, chứ không phải một thứ tự toàn phần: một số thao tác được sắp thứ tự so với nhau, nhưng một số thì không so sánh được.

Therefore, according to this definition, there are no concurrent operations in a linearizable **datastore**: there must be a single timeline along which all operations are totally ordered. There might be several requests waiting to be handled, but the datastore ensures that every request is handled atomically at a single point in time, acting on a single copy of the data, along a single timeline, without any concurrency.

Do đó, theo định nghĩa này, không có các thao tác đồng thời trong một kho dữ liệu tuyến tính hóa: phải có một dòng thời gian duy nhất mà dọc theo đó tất cả các thao tác được sắp thứ tự toàn phần. Có thể có vài yêu cầu đang chờ được xử lý, nhưng kho dữ liệu bảo đảm rằng mỗi yêu cầu được xử lý một cách nguyên tử tại một thời điểm duy nhất, tác động lên một bản dữ liệu duy nhất, dọc theo một dòng thời gian duy nhất, không có bất kỳ sự đồng thời nào.

Concurrency would mean that the timeline branches and merges again—and in this case, operations on different branches are incomparable (i.e., concurrent). We saw this phenomenon in Chapter 5: for example, Figure 5-14 is not a straight-line total order, but rather a jumble of different operations going on concurrently. The arrows in the diagram indicate causal dependencies—the partial ordering of operations.

Sự đồng thời sẽ có nghĩa là dòng thời gian rẽ nhánh rồi nhập lại — và trong trường hợp này, các thao tác trên các nhánh khác nhau là không so sánh được (tức đồng thời). Ta đã thấy hiện tượng này ở Chương 5: ví dụ, Hình 5-14 không phải một thứ tự toàn phần dạng đường thẳng, mà là một mớ lộn xộn các thao tác khác nhau đang diễn ra đồng thời. Các mũi tên trong sơ đồ chỉ ra các phụ thuộc nhân quả — thứ tự bộ phận của các thao tác.

If you are familiar with distributed version control systems such as Git, their version histories are very much like the **graph** of causal dependencies. Often one commit happens after another, in a straight line, but sometimes you get branches (when several people concurrently work on a project), and merges are created when those concurrently created commits are combined.

Nếu bạn quen thuộc với các hệ thống quản lý phiên bản phân tán như Git, lịch sử phiên bản của chúng rất giống đồ thị các phụ thuộc nhân quả. Thường thì một commit xảy ra sau một commit khác, theo một đường thẳng, nhưng đôi khi bạn có các nhánh (khi nhiều người đồng thời làm việc trên một dự án), và các phép merge được tạo ra khi những commit được tạo đồng thời đó được kết hợp lại.

Linearizability is stronger than causal consistency — Tính tuyến tính hóa mạnh hơn tính nhất quán nhân quả

So what is the relationship between the causal order and linearizability? The answer is that linearizability implies causality: any system that is linearizable will preserve causality correctly [7]. In particular, if there are multiple communication channels in a system (such as the message queue and the file storage service in Figure 9-5), linearizability ensures that causality is automatically preserved without the system having to do anything special (such as passing around timestamps between different components).

Vậy mối quan hệ giữa thứ tự nhân quả và tính tuyến tính hóa là gì? Câu trả lời là tính tuyến tính hóa hàm ý tính nhân quả: bất kỳ hệ thống nào tuyến tính hóa sẽ bảo toàn tính nhân quả một cách đúng đắn [7]. Cụ thể, nếu có nhiều kênh giao tiếp trong một hệ thống (chẳng hạn hàng đợi thông điệp và dịch vụ lưu trữ tệp trong Hình 9-5), tính tuyến tính hóa bảo đảm rằng tính nhân quả được tự động bảo toàn mà hệ thống không phải làm gì đặc biệt (chẳng hạn truyền các dấu thời gian qua lại giữa các thành phần khác nhau).

The fact that linearizability ensures causality is what makes linearizable systems simple to understand and appealing. However, as discussed in “The Cost of Linearizability”, making a system linearizable can harm its performance and availability, especially if the system has significant network delays (for example, if it’s geographically distributed). For this reason, some distributed data systems have abandoned linearizability, which allows them to achieve better performance but can make them difficult to work with.

Việc tính tuyến tính hóa bảo đảm tính nhân quả là điều làm cho các hệ thống tuyến tính hóa dễ hiểu và hấp dẫn. Tuy nhiên, như đã bàn ở “The Cost of Linearizability”, việc làm cho một hệ thống tuyến tính hóa có thể gây hại cho hiệu năng và tính sẵn sàng của nó, đặc biệt nếu hệ thống có các trì hoãn mạng đáng kể (ví dụ, nếu nó phân tán về mặt địa lý). Vì lý do này, một số hệ thống dữ liệu phân tán đã từ bỏ tính tuyến tính hóa, điều cho phép chúng đạt hiệu năng tốt hơn nhưng có thể khiến chúng khó làm việc cùng.

The good news is that a middle ground is possible. Linearizability is not the only way of preserving causality—there are other ways too. A system can be causally consistent without incurring the performance hit of making it linearizable (in particular, the CAP theorem does not apply). In fact, causal consistency is the strongest possible consistency model that does not slow down due to network delays, and remains available in the face of network failures [2, 42].

Tin tốt là có một con đường trung dung khả thi. Tính tuyến tính hóa không phải cách duy nhất để bảo toàn tính nhân quả — còn những cách khác nữa. Một hệ thống có thể nhất quán nhân quả mà không phải gánh chịu sự sụt giảm hiệu năng do làm cho nó tuyến tính hóa (cụ thể, định lý CAP không áp dụng). Thực ra, tính nhất quán nhân quả là mô hình nhất quán mạnh nhất có thể mà không bị chậm lại do các trì hoãn mạng, và vẫn sẵn sàng khi đối mặt với các sự cố mạng [2, 42].

In many cases, systems that appear to require linearizability in fact only really require causal consistency, which can be implemented more efficiently. Based on this observation, researchers are exploring new kinds of databases that preserve causality, with performance and availability characteristics that are similar to those of eventually consistent systems [49, 50, 51].

Trong nhiều trường hợp, các hệ thống có vẻ đòi hỏi tính tuyến tính hóa thực ra chỉ thực sự đòi hỏi tính nhất quán nhân quả, vốn có thể được hiện thực hiệu quả hơn. Dựa trên quan sát này, các nhà nghiên cứu đang khám phá những loại cơ sở dữ liệu mới bảo toàn tính nhân quả, với các đặc tính hiệu năng và sẵn sàng tương tự các hệ thống nhất quán cuối cùng [49, 50, 51].

As this research is quite recent, not much of it has yet made its way into **production** systems, and there are still challenges to be overcome [52, 53]. However, it is a promising direction for future systems.

Vì nghiên cứu này khá mới mẻ, chưa nhiều thành quả của nó đi vào các hệ thống production, và vẫn còn các thách thức cần vượt qua [52, 53]. Tuy nhiên, đây là một hướng đầy hứa hẹn cho các hệ thống tương lai.

Capturing causal dependencies — Nắm bắt các phụ thuộc nhân quả

We won't go into all the nitty-gritty details of how nonlinearizable systems can maintain causal consistency here, but just briefly explore some of the key ideas.

Ta sẽ không đi vào mọi chi tiết vụn vặt về cách các hệ thống không tuyến tính hóa có thể duy trì tính nhất quán nhân quả ở đây, mà chỉ khám phá ngắn gọn một số ý tưởng then chốt.

In order to maintain causality, you need to know which operation happened before which other operation. This is a partial order: concurrent operations may be processed in any order, but if one operation happened before another, then they must be processed in that order on every replica. Thus, when a replica processes an operation, it must ensure that all causally preceding operations (all operations that happened before) have already been processed; if some preceding operation is missing, the later operation must wait until the preceding operation has been processed.

Để duy trì tính nhân quả, bạn cần biết thao tác nào xảy ra trước thao tác nào. Đây là một thứ tự bộ phận: các thao tác đồng thời có thể được xử lý theo bất kỳ thứ tự nào, nhưng nếu một thao tác xảy ra trước một thao tác khác, thì chúng phải được xử lý theo thứ tự đó trên mọi bản sao. Do đó, khi một bản sao xử lý một thao tác, nó phải bảo đảm rằng tất cả các thao tác đi trước về mặt nhân quả (tất cả các thao tác xảy ra trước) đã được xử lý; nếu một thao tác đi trước nào đó bị thiếu, thì thao tác sau phải chờ tới khi thao tác đi trước được xử lý.

In order to determine causal dependencies, we need some way of describing the “knowledge” of a node in the system. If a node had already seen the value X when it issued the write Y, then X and Y may be causally related. The analysis uses the kinds of questions you would expect in a criminal investigation of fraud charges: did the CEO know about X at the time when they made decision Y?

Để xác định các phụ thuộc nhân quả, ta cần một cách nào đó để mô tả “kiến thức” của một nút trong hệ thống. Nếu một nút đã thấy giá trị X khi nó phát ra lần ghi Y, thì X và Y có thể có quan hệ nhân quả. Phép phân tích dùng đến loại câu hỏi mà bạn sẽ mong đợi trong một cuộc điều tra hình sự về các cáo buộc gian lận: liệu CEO có biết về X vào lúc họ ra quyết định Y hay không?

The techniques for determining which operation happened before which other operation are similar to what we discussed in “Detecting Concurrent Writes”. That section discussed causality in a leaderless datastore, where we need to detect concurrent writes to the same key in order to prevent lost updates. Causal consistency goes further: it needs to track causal dependencies across the entire database, not just for a single key. Version vectors can be generalized to do this [54].

Các kỹ thuật để xác định thao tác nào xảy ra trước thao tác nào tương tự những gì ta đã bàn ở “Detecting Concurrent Writes”. Mục đó bàn về tính nhân quả trong một kho dữ liệu không-leader, nơi ta cần phát hiện các lần ghi đồng thời tới cùng một khóa để ngăn mất cập nhật. Tính nhất quán nhân quả đi xa hơn: nó cần theo dõi các phụ thuộc nhân quả

trên toàn bộ cơ sở dữ liệu, chứ không chỉ cho một khóa duy nhất. Các vec-tơ phiên bản (version vectors) có thể được tổng quát hóa để làm việc này [54].

In order to determine the causal ordering, the database needs to know which version of the data was read by the application. This is why, in Figure 5-13, the version number from the prior operation is passed back to the database on a write. A similar idea appears in the conflict detection of SSI, as discussed in “Serializable Snapshot Isolation (SSI)”: when a transaction wants to commit, the database checks whether the version of the data that it read is still up to date. To this end, the database keeps track of which data has been read by which transaction.

Để xác định thứ tự nhân quả, cơ sở dữ liệu cần biết phiên bản nào của dữ liệu đã được ứng dụng đọc. Đây là lý do vì sao, trong Hình 5-13, số phiên bản từ thao tác trước được truyền ngược lại cho cơ sở dữ liệu khi ghi. Một ý tưởng tương tự xuất hiện trong việc phát hiện xung đột của SSI, như đã bàn ở “Serializable Snapshot Isolation (SSI)”: khi một giao dịch muốn commit, cơ sở dữ liệu kiểm tra xem phiên bản của dữ liệu mà nó đã đọc có còn cập nhật hay không. Để làm việc này, cơ sở dữ liệu theo dõi dữ liệu nào đã được giao dịch nào đọc.

Sequence Number Ordering — Sắp thứ tự bằng số thứ tự

Although causality is an important theoretical concept, actually keeping track of all causal dependencies can become impractical. In many applications, clients read lots of data before writing something, and then it is not clear whether the write is causally dependent on all or only some of those prior reads. Explicitly tracking all the data that has been read would mean a large overhead.

Mặc dù tính nhân quả là một khái niệm lý thuyết quan trọng, việc thực sự theo dõi tất cả các phụ thuộc nhân quả có thể trở nên bất khả thi trên thực tế. Trong nhiều ứng dụng, các client đọc rất nhiều dữ liệu trước khi ghi một thứ gì đó, và khi đó không rõ liệu lần ghi có phụ thuộc về mặt nhân quả vào tất cả hay chỉ một số trong những lần đọc trước đó hay không. Việc theo dõi tường minh tất cả dữ liệu đã được đọc sẽ là một gánh nặng lớn.

However, there is a better way: we can use sequence numbers or timestamps to order events. A timestamp need not come from a time-of-day clock (or physical clock, which have many problems, as discussed in “Unreliable Clocks”). It can instead come from a logical clock, which is an algorithm to generate a sequence of numbers to identify operations, typically using counters that are incremented for every operation.

Tuy nhiên, có một cách tốt hơn: ta có thể dùng các số thứ tự hoặc các dấu thời gian để sắp thứ tự các sự kiện. Một dấu thời gian không nhất thiết phải đến từ một đồng hồ thời gian-trong-ngày (hay đồng hồ vật lý, vốn có nhiều vấn đề, như đã bàn ở “Unreliable Clocks”). Thay vào đó, nó có thể đến từ một đồng hồ logic (logical clock), tức một thuật toán sinh ra một dãy số để định danh các thao tác, thường dùng các bộ đếm được tăng lên cho mỗi thao tác.

Such sequence numbers or timestamps are compact (only a few bytes in size), and they provide a total order: that is, every operation has a unique sequence number, and you can always compare two sequence numbers to determine which is greater (i.e., which operation happened later).

Những số thứ tự hoặc dấu thời gian như vậy gọn nhẹ (chỉ vài byte kích thước), và chúng cung cấp một thứ tự toàn phần: tức là mỗi thao tác có một số thứ tự duy nhất, và bạn luôn có thể so sánh hai số thứ tự để xác định cái nào lớn hơn (tức thao tác nào xảy ra sau).

In particular, we can create sequence numbers in a total order that is consistent with causality: we promise that if operation A causally happened before B, then A occurs before B in the total order

(A has a lower sequence number than B). Concurrent operations may be ordered arbitrarily. Such a total order captures all the causality information, but also imposes more ordering than strictly required by causality.

Cụ thể, ta có thể tạo các số thứ tự theo một thứ tự toàn phần nhất quán với tính nhân quả: ta cam kết rằng nếu thao tác A xảy ra về mặt nhân quả trước B, thì A xuất hiện trước B trong thứ tự toàn phần (A có số thứ tự nhỏ hơn B). Các thao tác đồng thời có thể được sắp thứ tự tùy ý. Một thứ tự toàn phần như vậy nắm bắt mọi thông tin nhân quả, nhưng cũng áp đặt nhiều thứ tự hơn mức mà tính nhân quả thực sự đòi hỏi.

In a database with single-leader replication (see “Leaders and Followers”), the replication log defines a total order of write operations that is consistent with causality. The leader can simply increment a counter for each operation, and thus assign a monotonically increasing sequence number to each operation in the replication log. If a follower applies the writes in the order they appear in the replication log, the state of the follower is always causally consistent (even if it is lagging behind the leader).

Trong một cơ sở dữ liệu dùng nhân bản đơn-leader (xem “Leaders and Followers”), nhật ký nhân bản định nghĩa một thứ tự toàn phần của các thao tác ghi nhất quán với tính nhân quả. Leader có thể đơn giản tăng một bộ đếm cho mỗi thao tác, và do đó gán một số thứ tự tăng đơn điệu cho mỗi thao tác trong nhật ký nhân bản. Nếu một follower áp dụng các lần ghi theo thứ tự chúng xuất hiện trong nhật ký nhân bản, thì trạng thái của follower luôn nhất quán nhân quả (ngay cả khi nó đang tụt hậu so với leader).

Noncausal sequence number generators — Các bộ sinh số thứ tự phi nhân quả

If there is not a single leader (perhaps because you are using a multi-leader or leaderless database, or because the database is partitioned), it is less clear how to generate sequence numbers for operations. Various methods are used in practice:

Nếu không có một leader duy nhất (có lẽ vì bạn đang dùng một cơ sở dữ liệu đa-leader hoặc không-leader, hoặc vì cơ sở dữ liệu được phân vùng), thì kém rõ ràng hơn về cách sinh các số thứ tự cho các thao tác. Nhiều phương pháp được dùng trên thực tế:

- Each node can generate its own independent set of sequence numbers. For example, if you have two nodes, one node can generate only odd numbers and the other only even numbers. In general, you could reserve some bits in the binary representation of the sequence number to contain a unique node identifier, and this would ensure that two different nodes can never generate the same sequence number.
- You can attach a timestamp from a time-of-day clock (physical clock) to each operation [55]. Such timestamps are not sequential, but if they have sufficiently high resolution, they might be sufficient to totally order operations. This fact is used in the last write wins conflict resolution method (see “Timestamps for ordering events”).
- You can preallocate blocks of sequence numbers. For example, node A might claim the block of sequence numbers from 1 to 1,000, and node B might claim the block from 1,001 to 2,000. Then each node can independently assign sequence numbers from its block, and allocate a new block when its supply of sequence numbers begins to run low.
 - Mỗi nút có thể sinh ra tập số thứ tự độc lập của riêng nó. Ví dụ, nếu bạn có hai nút, một nút có thể chỉ sinh số lẻ và nút kia chỉ sinh số chẵn. Nói chung, bạn có thể dành riêng một số bit trong biểu diễn nhị phân của số thứ tự để chứa một định danh nút

duy nhất, và điều này sẽ bảo đảm rằng hai nút khác nhau không bao giờ có thể sinh ra cùng một số thứ tự.

- Bạn có thể đính kèm một dấu thời gian từ một đồng hồ thời gian-trong-ngày (đồng hồ vật lý) vào mỗi thao tác [55]. Những dấu thời gian như vậy không tuần tự, nhưng nếu chúng có độ phân giải đủ cao, chúng có thể đủ để sắp thứ tự toàn phần các thao tác. Điều này được dùng trong phương pháp giải quyết xung đột lần-ghi-cuối-thắng (xem “Timestamps for ordering events”).
- Bạn có thể phân bổ trước các khối số thứ tự. Ví dụ, nút A có thể giành khối số thứ tự từ 1 tới 1.000, và nút B có thể giành khối từ 1.001 tới 2.000. Khi đó mỗi nút có thể độc lập gán các số thứ tự từ khối của nó, và phân bổ một khối mới khi nguồn cung số thứ tự của nó bắt đầu cạn.

These three options all perform better and are more scalable than pushing all operations through a single leader that increments a counter. They generate a unique, approximately increasing sequence number for each operation. However, they all have a problem: the sequence numbers they generate are not consistent with causality.

Cả ba phương án này đều có hiệu năng tốt hơn và dễ mở rộng hơn so với việc đẩy tất cả các thao tác qua một leader duy nhất tăng một bộ đếm. Chúng sinh ra một số thứ tự duy nhất, gần như tăng dần, cho mỗi thao tác. Tuy nhiên, tất cả đều có một vấn đề: các số thứ tự mà chúng sinh ra không nhất quán với tính nhân quả.

The causality problems occur because these sequence number generators do not correctly capture the ordering of operations across different nodes:

Các vấn đề nhân quả xảy ra vì những bộ sinh số thứ tự này không nắm bắt đúng thứ tự của các thao tác trên các nút khác nhau:

- Each node may process a different number of operations per second. Thus, if one node generates even numbers and the other generates odd numbers, the counter for even numbers may lag behind the counter for odd numbers, or vice versa. If you have an odd-numbered operation and an even-numbered operation, you cannot accurately tell which one causally happened first.
- Timestamps from physical clocks are **subject** to clock skew, which can make them inconsistent with causality. For example, see Figure 8-3, which shows a scenario in which an operation that happened causally later was actually assigned a lower timestamp.
- In the case of the block allocator, one operation may be given a sequence number in the range from 1,001 to 2,000, and a causally later operation may be given a number in the range from 1 to 1,000. Here, again, the sequence number is inconsistent with causality.
 - Mỗi nút có thể xử lý một số thao tác khác nhau mỗi giây. Do đó, nếu một nút sinh số chẵn và nút kia sinh số lẻ, bộ đếm cho số chẵn có thể tụt hậu so với bộ đếm cho số lẻ, hoặc ngược lại. Nếu bạn có một thao tác số lẻ và một thao tác số chẵn, bạn không thể nói chính xác cái nào xảy ra trước về mặt nhân quả.
 - Các dấu thời gian từ đồng hồ vật lý chịu ảnh hưởng của lệch đồng hồ, điều có thể khiến chúng không nhất quán với tính nhân quả. Ví dụ, xem Hình 8-3, vốn cho thấy một kịch bản trong đó một thao tác xảy ra về mặt nhân quả sau lại được gán một dấu thời gian thấp hơn.
 - Trong trường hợp bộ phân bổ khối, một thao tác có thể được gán một số thứ tự trong khoảng từ 1.001 tới 2.000, và một thao tác xảy ra về mặt nhân quả sau có thể được gán một số trong khoảng từ 1 tới 1.000. Ở đây, một lần nữa, số thứ tự không nhất quán với tính nhân quả.

Lamport timestamps — Các dấu thời gian Lamport

Although the three sequence number generators just described are inconsistent with causality, there is actually a simple method for generating sequence numbers that is consistent with causality. It is called a Lamport timestamp, proposed in 1978 by Leslie Lamport [56], in what is now one of the most-cited papers in the field of distributed systems.

Mặc dù ba bộ sinh số thứ tự vừa mô tả không nhất quán với tính nhân quả, thực ra có một phương pháp đơn giản để sinh các số thứ tự nhất quán với tính nhân quả. Nó được gọi là dấu thời gian Lamport (Lamport timestamp), do Leslie Lamport đề xuất năm 1978 [56], trong cái mà giờ là một trong những bài báo được trích dẫn nhiều nhất trong lĩnh vực hệ thống phân tán.

The use of Lamport timestamps is illustrated in Figure 9-8. Each node has a unique identifier, and each node keeps a counter of the number of operations it has processed. The Lamport timestamp is then simply a pair of (counter, node ID). Two nodes may sometimes have the same counter value, but by including the node ID in the timestamp, each timestamp is made unique.

Việc sử dụng các dấu thời gian Lamport được minh họa trong Hình 9-8. Mỗi nút có một định danh duy nhất, và mỗi nút giữ một bộ đếm số lượng thao tác mà nó đã xử lý. Dấu thời gian Lamport khi đó đơn giản là một cặp (bộ đếm, ID nút). Hai nút đôi khi có thể có cùng giá trị bộ đếm, nhưng bằng cách đưa ID nút vào dấu thời gian, mỗi dấu thời gian được làm cho duy nhất.

Figure 9-8. Lamport timestamps provide a total ordering consistent with causality. — Hình 9-8. Các dấu thời gian Lamport cung cấp một thứ tự toàn phần nhất quán với tính nhân quả.

A Lamport timestamp bears no relationship to a physical time-of-day clock, but it provides total ordering: if you have two timestamps, the one with a greater counter value is the greater timestamp; if the counter values are the same, the one with the greater node ID is the greater timestamp.

Một dấu thời gian Lamport không liên quan gì tới một đồng hồ thời gian-trong-ngày vật lý, nhưng nó cung cấp một thứ tự toàn phần: nếu bạn có hai dấu thời gian, cái có giá trị bộ đếm lớn hơn là dấu thời gian lớn hơn; nếu các giá trị bộ đếm bằng nhau, cái có ID nút lớn hơn là dấu thời gian lớn hơn.

So far this description is essentially the same as the even/odd counters described in the last section. The key idea about Lamport timestamps, which makes them consistent with causality, is the following: every node and every client keeps track of the maximum counter value it has seen so far, and includes that maximum on every request. When a node receives a request or response with a maximum counter value greater than its own counter value, it immediately increases its own counter to that maximum.

Cho tới đây, mô tả này về cơ bản giống với các bộ đếm chẵn/lẻ được mô tả trong mục trước. Ý tưởng then chốt về các dấu thời gian Lamport, điều làm cho chúng nhất quán với tính nhân quả, là như sau: mỗi nút và mỗi client theo dõi giá trị bộ đếm lớn nhất mà nó đã thấy cho tới nay, và đính kèm giá trị lớn nhất đó vào mỗi yêu cầu. Khi một nút nhận được một yêu cầu hoặc phản hồi với một giá trị bộ đếm lớn nhất lớn hơn giá trị bộ đếm của chính nó, nó lập tức tăng bộ đếm của chính nó lên giá trị lớn nhất đó.

This is shown in Figure 9-8, where client A receives a counter value of 5 from node 2, and then sends that maximum of 5 to node 1. At that time, node 1's counter was only 1, but it was immediately moved forward to 5, so the next operation had an incremented counter value of 6.

Điều này được cho thấy trong Hình 9-8, nơi client A nhận một giá trị bộ đếm là 5 từ nút 2,

rồi gửi giá trị lớn nhất 5 đó tới nút 1. Vào lúc đó, bộ đếm của nút 1 mới chỉ là 1, nhưng nó lập tức được đẩy lên 5, nên thao tác tiếp theo có một giá trị bộ đếm được tăng lên là 6.

As long as the maximum counter value is carried along with every operation, this scheme ensures that the ordering from the Lamport timestamps is consistent with causality, because every causal dependency results in an increased timestamp.

Miễn là giá trị bộ đếm lớn nhất được mang theo cùng mọi thao tác, cơ chế này bảo đảm rằng thứ tự từ các dấu thời gian Lamport nhất quán với tính nhân quả, bởi mọi phụ thuộc nhân quả đều dẫn tới một dấu thời gian được tăng lên.

Lamport timestamps are sometimes confused with version vectors, which we saw in “Detecting Concurrent Writes”. Although there are some similarities, they have a different purpose: version vectors can distinguish whether two operations are concurrent or whether one is causally dependent on the other, whereas Lamport timestamps always enforce a total ordering. From the total ordering of Lamport timestamps, you cannot tell whether two operations are concurrent or whether they are causally dependent. The advantage of Lamport timestamps over version vectors is that they are more compact.

Các dấu thời gian Lamport đôi khi bị nhầm với các vec-tơ phiên bản, mà ta đã thấy ở “Detecting Concurrent Writes”. Mặc dù có một số nét tương đồng, chúng có mục đích khác nhau: các vec-tơ phiên bản có thể phân biệt liệu hai thao tác là đồng thời hay một cái phụ thuộc về mặt nhân quả vào cái kia, trong khi các dấu thời gian Lamport luôn áp đặt một thứ tự toàn phần. Từ thứ tự toàn phần của các dấu thời gian Lamport, bạn không thể nói liệu hai thao tác là đồng thời hay chúng phụ thuộc về mặt nhân quả. Ưu điểm của các dấu thời gian Lamport so với các vec-tơ phiên bản là chúng gọn nhẹ hơn.

Timestamp ordering is not sufficient — Sắp thứ tự bằng dấu thời gian là chưa đủ

Although Lamport timestamps define a total order of operations that is consistent with causality, they are not quite sufficient to solve many common problems in distributed systems.

Mặc dù các dấu thời gian Lamport định nghĩa một thứ tự toàn phần của các thao tác nhất quán với tính nhân quả, chúng vẫn chưa hẳn đủ để giải nhiều bài toán phổ biến trong các hệ thống phân tán.

For example, consider a system that needs to ensure that a username uniquely identifies a user account. If two users concurrently try to create an account with the same username, one of the two should succeed and the other should fail. (We touched on this problem previously in “The leader and the lock”.)

Ví dụ, hãy xét một hệ thống cần bảo đảm rằng một tên người dùng định danh duy nhất một tài khoản người dùng. Nếu hai người dùng đồng thời cố tạo một tài khoản với cùng tên người dùng, một trong hai sẽ thành công và người kia sẽ thất bại. (Ta đã chạm tới bài toán này trước đó trong “The leader and the lock”.)

At first glance, it seems as though a total ordering of operations (e.g., using Lamport timestamps) should be sufficient to solve this problem: if two accounts with the same username are created, pick the one with the lower timestamp as the winner (the one who grabbed the username first), and let the one with the greater timestamp fail. Since timestamps are totally ordered, this comparison is always valid.

Thoạt nhìn, có vẻ như một thứ tự toàn phần của các thao tác (chẳng hạn dùng các dấu thời gian Lamport) là đủ để giải bài toán này: nếu hai tài khoản với cùng tên người dùng được

tạo ra, chọn cái có dấu thời gian thấp hơn làm người thắng (người giành tên người dùng trước), và để cái có dấu thời gian lớn hơn thất bại. Vì các dấu thời gian được sắp thứ tự toàn phần, phép so sánh này luôn hợp lệ.

This approach works for determining the winner after the fact: once you have collected all the username creation operations in the system, you can compare their timestamps. However, it is not sufficient when a node has just received a request from a user to create a username, and needs to decide right now whether the request should succeed or fail. At that moment, the node does not know whether another node is concurrently in the process of creating an account with the same username, and what timestamp that other node may assign to the operation.

Cách tiếp cận này hiệu quả để xác định người thắng sau khi mọi việc đã rồi: một khi bạn đã thu thập tất cả các thao tác tạo tên người dùng trong hệ thống, bạn có thể so sánh các dấu thời gian của chúng. Tuy nhiên, nó không đủ khi một nút vừa nhận được một yêu cầu từ một người dùng để tạo một tên người dùng, và cần quyết định ngay bây giờ xem yêu cầu nên thành công hay thất bại. Tại thời điểm đó, nút không biết liệu một nút khác có đang đồng thời trong quá trình tạo một tài khoản với cùng tên người dùng hay không, và nút kia có thể gán dấu thời gian nào cho thao tác đó.

In order to be sure that no other node is in the process of concurrently creating an account with the same username and a lower timestamp, you would have to check with every other node to see what it is doing [56]. If one of the other nodes has failed or cannot be reached due to a network problem, this system would grind to a halt. This is not the kind of fault-tolerant system that we need.

Để chắc chắn rằng không có nút nào khác đang trong quá trình đồng thời tạo một tài khoản với cùng tên người dùng và một dấu thời gian thấp hơn, bạn sẽ phải kiểm tra với mọi nút khác để xem nó đang làm gì [56]. Nếu một trong các nút khác đã hỏng hóc hoặc không thể liên lạc được do một vấn đề mạng, hệ thống này sẽ đình trệ. Đây không phải loại hệ thống chịu lỗi mà ta cần.

The problem here is that the total order of operations only emerges after you have collected all of the operations. If another node has generated some operations, but you don't yet know what they are, you cannot construct the final ordering of operations: the unknown operations from the other node may need to be inserted at various positions in the total order.

Vấn đề ở đây là thứ tự toàn phần của các thao tác chỉ xuất hiện sau khi bạn đã thu thập tất cả các thao tác. Nếu một nút khác đã sinh ra một số thao tác, nhưng bạn còn chưa biết chúng là gì, bạn không thể dựng ra thứ tự cuối cùng của các thao tác: các thao tác chưa biết từ nút kia có thể cần được chèn vào nhiều vị trí khác nhau trong thứ tự toàn phần.

To conclude: in order to implement something like a uniqueness constraint for usernames, it's not sufficient to have a total ordering of operations—you also need to know when that order is finalized. If you have an operation to create a username, and you are sure that no other node can insert a claim for the same username ahead of your operation in the total order, then you can safely declare the operation successful.

Tóm lại: để hiện thực một thứ gì đó như một ràng buộc duy nhất cho các tên người dùng, có một thứ tự toàn phần của các thao tác là chưa đủ — bạn cũng cần biết khi nào thứ tự đó được chốt lại. Nếu bạn có một thao tác tạo một tên người dùng, và bạn chắc chắn rằng không nút nào khác có thể chèn một yêu cầu giành cùng tên người dùng vào trước thao tác của bạn trong thứ tự toàn phần, thì bạn có thể an toàn tuyên bố thao tác thành công.

This idea of knowing when your total order is finalized is captured in the topic of total order broadcast.

Ý tưởng về việc biết khi nào thứ tự toàn phần của bạn được chốt lại này được gói gọn trong chủ đề phát toàn-thứ-tự (total order broadcast).

Total Order Broadcast — Phát toàn-thứ-tự

If your program runs only on a single CPU core, it is easy to define a total ordering of operations: it is simply the order in which they were executed by the CPU. However, in a distributed system, getting all nodes to agree on the same total ordering of operations is tricky. In the last section we discussed ordering by timestamps or sequence numbers, but found that it is not as powerful as single-leader replication (if you use timestamp ordering to implement a uniqueness constraint, you cannot tolerate any faults).

Nếu chương trình của bạn chỉ chạy trên một nhân CPU duy nhất, thì dễ định nghĩa một thứ tự toàn phần của các thao tác: nó đơn giản là thứ tự mà chúng được CPU thực thi. Tuy nhiên, trong một hệ thống phân tán, việc làm cho tất cả các nút nhất trí về cùng một thứ tự toàn phần của các thao tác là khó. Trong mục trước ta đã bàn về việc sắp thứ tự bằng các dấu thời gian hoặc số thứ tự, nhưng thấy rằng nó không mạnh bằng nhân bản đơn-leader (nếu bạn dùng sắp thứ tự bằng dấu thời gian để hiện thực một ràng buộc duy nhất, bạn không thể chịu được bất kỳ lỗi nào).

As discussed, single-leader replication determines a total order of operations by choosing one node as the leader and sequencing all operations on a single CPU core on the leader. The challenge then is how to scale the system if the **throughput** is greater than a single leader can handle, and also how to handle failover if the leader fails (see “Handling Node Outages”). In the distributed systems literature, this problem is known as total order broadcast or atomic broadcast [25, 57, 58].

Như đã bàn, nhân bản đơn-leader xác định một thứ tự toàn phần của các thao tác bằng cách chọn một nút làm leader và tuần tự hóa tất cả các thao tác trên một nhân CPU duy nhất trên leader. Thách thức khi đó là làm sao mở rộng hệ thống nếu thông lượng lớn hơn mức một leader duy nhất có thể xử lý, và cũng là làm sao xử lý chuyển đổi dự phòng nếu leader hỏng hóc (xem “Handling Node Outages”). Trong các tài liệu về hệ thống phân tán, bài toán này được biết đến là phát toàn-thứ-tự hoặc phát nguyên tử (atomic broadcast) [25, 57, 58].

Scope of ordering guarantee — Phạm vi của bảo đảm sắp thứ tự

Partitioned databases with a single leader per partition often maintain ordering only per partition, which means they cannot offer consistency guarantees (e.g., consistent snapshots, foreign key references) across partitions. Total ordering across all partitions is possible, but requires additional coordination [59].

Các cơ sở dữ liệu được phân vùng với một leader cho mỗi phân vùng thường chỉ duy trì sắp thứ tự cho từng phân vùng, điều này nghĩa là chúng không thể cung cấp các bảo đảm nhất quán (chẳng hạn các snapshot nhất quán, các tham chiếu khóa ngoại) xuyên các phân vùng. Sắp thứ tự toàn phần xuyên tất cả các phân vùng là khả thi, nhưng đòi hỏi sự điều phối bổ sung [59].

Total order broadcast is usually described as a protocol for exchanging messages between nodes. Informally, it requires that two safety properties always be satisfied:

Phát toàn-thứ-tự thường được mô tả như một giao thức để trao đổi thông điệp giữa các nút. Một cách phi hình thức, nó đòi hỏi hai thuộc tính an toàn (safety) luôn được thỏa mãn:

No messages are lost: if a message is delivered to one node, it is delivered to all nodes.

Không có thông điệp nào bị mất: nếu một thông điệp được chuyển tới một nút, nó được chuyển tới tất cả các nút.

Messages are delivered to every node in the same order.

Các thông điệp được chuyển tới mọi nút theo cùng một thứ tự.

A correct algorithm for total order broadcast must ensure that the **reliability** and ordering properties are always satisfied, even if a node or the network is faulty. Of course, messages will not be delivered while the network is interrupted, but an algorithm can keep retrying so that the messages get through when the network is eventually repaired (and then they must still be delivered in the correct order).

Một thuật toán đúng đắn cho phát toàn-thứ-tự phải bảo đảm rằng các thuộc tính độ tin cậy và sắp thứ tự luôn được thỏa mãn, ngay cả khi một nút hoặc mạng gặp trục trặc. Tất nhiên, các thông điệp sẽ không được chuyển trong khi mạng bị gián đoạn, nhưng một thuật toán có thể tiếp tục thử lại để các thông điệp đến nơi khi mạng rốt cuộc được sửa (và khi đó chúng vẫn phải được chuyển theo đúng thứ tự).

Using total order broadcast — Sử dụng phát toàn-thứ-tự

Consensus services such as ZooKeeper and etcd actually implement total order broadcast. This fact is a hint that there is a strong connection between total order broadcast and consensus, which we will explore later in this chapter.

Các dịch vụ đồng thuận như ZooKeeper và etcd thực ra hiện thực phát toàn-thứ-tự. Điều này là một gợi ý rằng có một mối liên hệ chặt chẽ giữa phát toàn-thứ-tự và đồng thuận, điều mà ta sẽ khám phá cuối chương này.

Total order broadcast is exactly what you need for database replication: if every message represents a write to the database, and every replica processes the same writes in the same order, then the replicas will remain consistent with each other (aside from any temporary replication lag). This principle is known as state machine replication [60], and we will return to it in Chapter 11.

Phát toàn-thứ-tự chính là thứ bạn cần cho nhân bản cơ sở dữ liệu: nếu mỗi thông điệp biểu diễn một lần ghi vào cơ sở dữ liệu, và mỗi bản sao xử lý cùng các lần ghi theo cùng một thứ tự, thì các bản sao sẽ giữ nhất quán với nhau (ngoại trừ bất kỳ độ trễ nhân bản tạm thời nào). Nguyên lý này được biết đến là nhân bản máy trạng thái (state machine replication) [60], và ta sẽ quay lại nó ở Chương 11.

Similarly, total order broadcast can be used to implement serializable transactions: as discussed in “Actual Serial Execution”, if every message represents a deterministic transaction to be executed as a stored procedure, and if every node processes those messages in the same order, then the partitions and replicas of the database are kept consistent with each other [61].

Tương tự, phát toàn-thứ-tự có thể được dùng để hiện thực các giao dịch tuần tự hóa: như đã bàn ở “Actual Serial Execution”, nếu mỗi thông điệp biểu diễn một giao dịch tất định cần được thực thi dưới dạng một thủ tục lưu trữ (stored procedure), và nếu mỗi nút xử lý các thông điệp đó theo cùng một thứ tự, thì các phân vùng và bản sao của cơ sở dữ liệu được giữ nhất quán với nhau [61].

An important aspect of total order broadcast is that the order is fixed at the time the messages are delivered: a node is not allowed to retroactively insert a message into an earlier position in the order if subsequent messages have already been delivered. This fact makes total order broadcast stronger than timestamp ordering.

Một khía cạnh quan trọng của phát toàn-thứ-tự là thứ tự được cố định vào lúc các thông điệp được chuyển: một nút không được phép chen ngược một thông điệp vào một vị trí sớm hơn trong thứ tự nếu các thông điệp sau đó đã được chuyển. Điều này làm cho phát toàn-thứ-tự mạnh hơn sắp thứ tự bằng dấu thời gian.

Another way of looking at total order broadcast is that it is a way of creating a log (as in a replication log, transaction log, or write-ahead log): delivering a message is like appending to the log. Since all nodes must deliver the same messages in the same order, all nodes can read the log and see the same sequence of messages.

Một cách khác để nhìn nhận phát toàn-thứ-tự là nó là một cách tạo ra một nhật ký (như trong nhật ký nhân bản, nhật ký giao dịch, hay nhật ký ghi-trước): việc chuyển một thông điệp giống như việc thêm vào (append) nhật ký. Vì tất cả các nút phải chuyển cùng các thông điệp theo cùng một thứ tự, tất cả các nút có thể đọc nhật ký và thấy cùng một chuỗi thông điệp.

Total order broadcast is also useful for implementing a lock service that provides fencing tokens (see “Fencing tokens”). Every request to acquire the lock is appended as a message to the log, and all messages are sequentially numbered in the order they appear in the log. The sequence number can

then serve as a fencing token, because it is monotonically increasing. In ZooKeeper, this sequence number is called zxid [15].

Phát toàn-thứ-tự cũng hữu ích để hiện thực một dịch vụ khóa cung cấp các thẻ rào chắn (fencing tokens) (xem “Fencing tokens”). Mọi yêu cầu giành khóa được thêm vào nhật ký dưới dạng một thông điệp, và tất cả các thông điệp được đánh số tuần tự theo thứ tự chúng xuất hiện trong nhật ký. Số thứ tự khi đó có thể đóng vai trò một thẻ rào chắn, bởi nó tăng đơn điệu. Trong ZooKeeper, số thứ tự này được gọi là zxid [15].

Implementing linearizable storage using total order broadcast — Hiện thực lưu trữ tuyến tính hóa bằng phát toàn-thứ-tự

As illustrated in Figure 9-4, in a linearizable system there is a total order of operations. Does that mean linearizability is the same as total order broadcast? Not quite, but there are close links between the two.

Như được minh họa trong Hình 9-4, trong một hệ thống tuyến tính hóa có một thứ tự toàn phần của các thao tác. Vậy điều đó có nghĩa là tính tuyến tính hóa cũng giống như phát toàn-thứ-tự không? Không hẳn, nhưng có những mối liên hệ chặt chẽ giữa hai điều này.

Total order broadcast is asynchronous: messages are guaranteed to be delivered reliably in a fixed order, but there is no guarantee about when a message will be delivered (so one recipient may lag behind the others). By contrast, linearizability is a recency guarantee: a read is guaranteed to see the latest value written.

Phát toàn-thứ-tự là bất đồng bộ: các thông điệp được bảo đảm chuyển một cách đáng tin cậy theo một thứ tự cố định, nhưng không có bảo đảm về việc khi nào một thông điệp sẽ được chuyển (nên một người nhận có thể tụt hậu so với những người khác). Ngược lại, tính tuyến tính hóa là một bảo đảm về tính tức thời: một lần đọc được bảo đảm thấy giá trị mới nhất được ghi.

However, if you have total order broadcast, you can build linearizable storage on top of it. For example, you can ensure that usernames uniquely identify user accounts.

Tuy nhiên, nếu bạn có phát toàn-thứ-tự, bạn có thể xây dựng lưu trữ tuyến tính hóa trên nền nó. Ví dụ, bạn có thể bảo đảm rằng các tên người dùng định danh duy nhất các tài khoản người dùng.

Imagine that for every possible username, you can have a linearizable register with an atomic compare-and-set operation. Every register initially has the value null (indicating that the username is not taken). When a user wants to create a username, you execute a compare-and-set operation on the register for that username, setting it to the user account ID, under the condition that the previous register value is null. If multiple users try to concurrently grab the same username, only one of the compare-and-set operations will succeed, because the others will see a value other than null (due to linearizability).

Hãy hình dung rằng với mỗi tên người dùng khả dĩ, bạn có thể có một thanh ghi tuyến tính hóa với một thao tác so-sánh-rồi-đặt nguyên tử. Mỗi thanh ghi ban đầu có giá trị null (cho biết tên người dùng chưa bị chiếm). Khi một người dùng muốn tạo một tên người dùng, bạn thực thi một thao tác so-sánh-rồi-đặt trên thanh ghi cho tên người dùng đó, đặt nó thành ID tài khoản người dùng, với điều kiện là giá trị thanh ghi trước đó là null. Nếu nhiều người dùng đồng thời cố giành cùng một tên người dùng, chỉ một trong các thao tác so-sánh-rồi-đặt sẽ thành công, bởi các thao tác khác sẽ thấy một giá trị khác null (do tính tuyến tính hóa).

You can implement such a linearizable compare-and-set operation as follows by using total order broadcast as an append-only log [62, 63]:

Bạn có thể hiện thực một thao tác so-sánh-rồi-đặt tuyến tính hóa như vậy theo cách sau bằng cách dùng phát toàn-thứ-tự làm một nhật ký chỉ-thêm (append-only) [62, 63]:

- Append a message to the log, tentatively indicating the username you want to claim.
- Read the log, and wait for the message you appended to be delivered back to you.
- Check for any messages claiming the username that you want. If the first message for your desired username is your own message, then you are successful: you can commit the username claim (perhaps by appending another message to the log) and acknowledge it to the client. If the first message for your desired username is from another user, you abort the operation.
 - Thêm một thông điệp vào nhật ký, tạm thời cho biết tên người dùng bạn muốn giành.
 - Đọc nhật ký, và chờ cho thông điệp bạn đã thêm được chuyển trở lại cho bạn.
 - Kiểm tra xem có thông điệp nào giành tên người dùng mà bạn muốn hay không. Nếu thông điệp đầu tiên cho tên người dùng bạn mong muốn là thông điệp của chính bạn, thì bạn thành công: bạn có thể commit việc giành tên người dùng (có lẽ bằng cách thêm một thông điệp khác vào nhật ký) và báo nhận cho client. Nếu thông điệp đầu tiên cho tên người dùng bạn mong muốn là từ một người dùng khác, bạn hủy bỏ thao tác.

Because log entries are delivered to all nodes in the same order, if there are several concurrent writes, all nodes will agree on which one came first. Choosing the first of the conflicting writes as the winner and aborting later ones ensures that all nodes agree on whether a write was committed or aborted. A similar approach can be used to implement serializable multi-object transactions on top of a log [62].

Vì các mục nhật ký được chuyển tới tất cả các nút theo cùng một thứ tự, nếu có vài lần ghi đồng thời, tất cả các nút sẽ nhất trí về việc cái nào đến trước. Việc chọn cái đầu tiên trong các lần ghi xung đột làm người thắng và hủy bỏ các cái sau bảo đảm rằng tất cả các nút nhất trí về việc một lần ghi đã được commit hay bị hủy bỏ. Một cách tiếp cận tương tự có thể được dùng để hiện thực các giao dịch đa-đối-tượng tuần tự hóa trên nền một nhật ký [62].

While this procedure ensures linearizable writes, it doesn't guarantee linearizable reads—if you read from a store that is asynchronously updated from the log, it may be stale. (To be precise, the procedure described here provides sequential consistency [47, 64], sometimes also known as timeline consistency [65, 66], a slightly weaker guarantee than linearizability.) To make reads linearizable, there are a few options:

Mặc dù thủ tục này bảo đảm các lần ghi tuyến tính hóa, nó không bảo đảm các lần đọc tuyến tính hóa — nếu bạn đọc từ một kho được cập nhật bất đồng bộ từ nhật ký, nó có thể bị cũ. (Nói chính xác, thủ tục được mô tả ở đây cung cấp tính nhất quán tuần tự (sequential consistency) [47, 64], đôi khi cũng được biết đến là tính nhất quán dòng thời gian (timeline consistency) [65, 66], một bảo đảm yếu hơn một chút so với tính tuyến tính hóa.) Để làm cho các lần đọc tuyến tính hóa, có vài phương án:

- You can sequence reads through the log by appending a message, reading the log, and performing the actual read when the message is delivered back to you. The message's position in the log thus defines the point in time at which the read happens. (Quorum reads in etcd work somewhat like this [16].)
- If the log allows you to fetch the position of the latest log message in a linearizable way, you can query that position, wait for all entries up to that position to be delivered to you, and then

perform the read. (This is the idea behind ZooKeeper’s sync() operation [15].)

- You can make your read from a replica that is synchronously updated on writes, and is thus sure to be up to date. (This technique is used in chain replication [63]; see also “Research on Replication”).
 - Bạn có thể tuần tự hóa các lần đọc qua nhật ký bằng cách thêm một thông điệp, đọc nhật ký, và thực hiện lần đọc thực sự khi thông điệp được chuyển trở lại cho bạn. Vị trí của thông điệp trong nhật ký do đó định nghĩa thời điểm mà lần đọc xảy ra. (Các lần đọc theo nhóm tức số trong etcd hoạt động đại loại theo kiểu này [16].)
 - Nếu nhật ký cho phép bạn lấy vị trí của thông điệp nhật ký mới nhất theo cách tuyến tính hóa, bạn có thể truy vấn vị trí đó, chờ cho tất cả các mục cho tới vị trí đó được chuyển tới bạn, rồi thực hiện lần đọc. (Đây là ý tưởng đằng sau thao tác sync() của ZooKeeper [15].)
 - Bạn có thể thực hiện lần đọc từ một bản sao được cập nhật đồng bộ khi ghi, và do đó chắc chắn là cập nhật. (Kỹ thuật này được dùng trong nhân bản chuỗi (chain replication) [63]; xem thêm “Research on Replication”).

Implementing total order broadcast using linearizable storage — Hiện thực phát toàn-thứ-tự bằng lưu trữ tuyến tính hóa

The last section showed how to build a linearizable compare-and-set operation from total order broadcast. We can also turn it around, assume that we have linearizable storage, and show how to build total order broadcast from it.

Mục trước cho thấy cách xây dựng một thao tác so-sánh-rồi-đặt tuyến tính hóa từ phát toàn-thứ-tự. Ta cũng có thể đảo ngược lại, giả định rằng ta có lưu trữ tuyến tính hóa, và cho thấy cách xây dựng phát toàn-thứ-tự từ nó.

The easiest way is to assume you have a linearizable register that stores an integer and that has an atomic increment-and-get operation [28]. Alternatively, an atomic compare-and-set operation would also do the job.

Cách dễ nhất là giả định bạn có một thanh ghi tuyến tính hóa lưu một số nguyên và có một thao tác tăng-rồi-lấy (increment-and-get) nguyên tử [28]. Ngoài ra, một thao tác so-sánh-rồi-đặt nguyên tử cũng làm được việc.

The algorithm is simple: for every message you want to send through total order broadcast, you increment-and-get the linearizable integer, and then attach the value you got from the register as a sequence number to the message. You can then send the message to all nodes (resending any lost messages), and the recipients will deliver the messages consecutively by sequence number.

Thuật toán đơn giản: với mỗi thông điệp bạn muốn gửi qua phát toàn-thứ-tự, bạn tăng-rồi-lấy số nguyên tuyến tính hóa, rồi đính kèm giá trị bạn nhận được từ thanh ghi làm số thứ tự cho thông điệp. Sau đó bạn có thể gửi thông điệp tới tất cả các nút (gửi lại bất kỳ thông điệp nào bị mất), và những người nhận sẽ chuyển các thông điệp liên tiếp theo số thứ tự.

Note that unlike Lamport timestamps, the numbers you get from incrementing the linearizable register form a sequence with no gaps. Thus, if a node has delivered message 4 and receives an incoming message with a sequence number of 6, it knows that it must wait for message 5 before it can deliver message 6. The same is not the case with Lamport timestamps—in fact, this is the key difference between total order broadcast and timestamp ordering.

Lưu ý rằng khác với các dấu thời gian Lamport, các số bạn nhận được từ việc tăng thanh ghi tuyến tính hóa tạo thành một dãy không có khoảng trống. Do đó, nếu một nút đã chuyển

thông điệp 4 và nhận một thông điệp đến với số thứ tự là 6, nó biết rằng nó phải chờ thông điệp 5 trước khi có thể chuyển thông điệp 6. Điều tương tự không đúng với các dấu thời gian Lamport — thực ra, đây chính là khác biệt then chốt giữa phát toàn-thứ-tự và sắp thứ tự bằng dấu thời gian.

How hard could it be to make a linearizable integer with an atomic increment-and-get operation? As usual, if things never failed, it would be easy: you could just keep it in a variable on one node. The problem lies in handling the situation when network connections to that node are interrupted, and restoring the value when that node fails [59]. In general, if you think hard enough about linearizable sequence number generators, you inevitably end up with a consensus algorithm.

Việc tạo ra một số nguyên tuyến tính hóa với một thao tác tăng-rồi-lấy nguyên tử thì khó đến mức nào? Như thường lệ, nếu mọi thứ không bao giờ trục trặc, thì sẽ dễ: bạn chỉ cần giữ nó trong một biến trên một nút. Vấn đề nằm ở việc xử lý tình huống khi các kết nối mạng tới nút đó bị gián đoạn, và việc khôi phục giá trị khi nút đó hỏng hóc [59]. Nói chung, nếu bạn suy nghĩ đủ kỹ về các bộ sinh số thứ tự tuyến tính hóa, bạn chắc chắn rốt cuộc sẽ đi tới một thuật toán đồng thuận.

This is no coincidence: it can be proved that a linearizable compare-and-set (or increment-and-get) register and total order broadcast are both equivalent to consensus [28, 67]. That is, if you can solve one of these problems, you can transform it into a solution for the others. This is quite a profound and surprising insight!

Đây không phải sự trùng hợp: có thể chứng minh rằng một thanh ghi so-sánh-rồi-đặt (hoặc tăng-rồi-lấy) tuyến tính hóa và phát toàn-thứ-tự đều tương đương với đồng thuận [28, 67]. Tức là, nếu bạn có thể giải một trong những bài toán này, bạn có thể biến đổi nó thành một giải pháp cho các bài toán còn lại. Đây là một nhận định khá sâu sắc và đáng ngạc nhiên!

It is time to finally tackle the consensus problem head-on, which we will do in the rest of this chapter.

Đã đến lúc cuối cùng đối mặt trực diện với bài toán đồng thuận, điều mà ta sẽ làm trong phần còn lại của chương này.

Distributed Transactions and Consensus — Giao dịch phân tán và Đồng thuận

Consensus is one of the most important and fundamental problems in distributed computing. On the surface, it seems simple: informally, the goal is simply to get several nodes to agree on something. You might think that this shouldn't be too hard. Unfortunately, many broken systems have been built in the mistaken belief that this problem is easy to solve.

Đồng thuận là một trong những bài toán quan trọng và nền tảng nhất trong tính toán phân tán. Bề ngoài, nó có vẻ đơn giản: một cách phi hình thức, mục tiêu đơn giản là làm cho vài nút nhất trí về một điều gì đó. Bạn có thể nghĩ rằng điều này hẳn không quá khó. Đáng tiếc, nhiều hệ thống lỗi đã được xây dựng dựa trên niềm tin nhầm lẫn rằng bài toán này dễ giải.

Although consensus is very important, the section about it appears late in this book because the topic is quite subtle, and appreciating the subtleties requires some prerequisite knowledge. Even in the academic research community, the understanding of consensus only gradually crystallized over the course of decades, with many misunderstandings along the way. Now that we have discussed replication (Chapter 5), transactions (Chapter 7), system models (Chapter 8), linearizability, and total order broadcast (this chapter), we are finally ready to tackle the consensus problem.

Mặc dù đồng thuận rất quan trọng, mục về nó xuất hiện muộn trong cuốn sách này bởi chủ đề khá tinh vi, và việc thấu hiểu những điều tinh vi đó đòi hỏi một số kiến thức tiên quyết. Ngay cả trong cộng đồng nghiên cứu học thuật, sự thấu hiểu về đồng thuận cũng chỉ dần dần kết tinh qua hàng chục năm, với nhiều hiểu lầm trên đường đi. Giờ ta đã bàn về nhân bản (Chương 5), giao dịch (Chương 7), các mô hình hệ thống (Chương 8), tính tuyến tính hóa, và phát toàn-thứ-tự (chương này), cuối cùng ta đã sẵn sàng đối mặt với bài toán đồng thuận.

There are a number of situations in which it is important for nodes to agree. For example:

Có một số tình huống trong đó việc các nút nhất trí là quan trọng. Ví dụ:

In a database with single-leader replication, all nodes need to agree on which node is the leader. The leadership position might become contested if some nodes can't communicate with others due to a network fault. In this case, consensus is important to avoid a bad failover, resulting in a split brain situation in which two nodes both believe themselves to be the leader (see “Handling Node Outages”). If there were two leaders, they would both accept writes and their data would diverge, leading to inconsistency and data loss.

Trong một cơ sở dữ liệu dùng nhân bản đơn-leader, tất cả các nút cần nhất trí về việc nút nào là leader. Vị trí lãnh đạo có thể trở nên tranh chấp nếu một số nút không thể giao tiếp với những nút khác do một lỗi mạng. Trong trường hợp này, đồng thuận là quan trọng để tránh một lần chuyển đổi dự phòng tồi, dẫn tới một tình huống split brain trong đó hai nút cùng tin rằng chính mình là leader (xem “Handling Node Outages”). Nếu có hai leader, cả hai sẽ chấp nhận các lần ghi và dữ liệu của chúng sẽ phân kỳ, dẫn tới sự bất nhất và mất dữ liệu.

In a database that supports transactions spanning several nodes or partitions, we have the problem that a transaction may fail on some nodes but succeed on others. If we want to maintain transaction atomicity (in the sense of ACID; see “Atomicity”), we have to get all nodes to agree on the outcome of the transaction: either they all abort/**roll back** (if anything goes wrong) or they all commit (if nothing goes wrong). This **instance** of consensus is known as the atomic commit problem.

Trong một cơ sở dữ liệu hỗ trợ các giao dịch trải rộng trên vài nút hoặc phân vùng, ta có vấn đề là một giao dịch có thể thất bại trên một số nút nhưng thành công trên những nút khác. Nếu ta muốn duy trì tính nguyên tử giao dịch (theo nghĩa ACID; xem “Atomicity”), ta phải làm cho tất cả các nút nhất trí về kết cục của giao dịch: hoặc tất cả đều hủy bỏ/hoàn tác (nếu có gì đó trục trặc) hoặc tất cả đều commit (nếu không có gì trục trặc). Trường hợp đồng thuận này được biết đến là bài toán commit nguyên tử (atomic commit).

The Impossibility of Consensus — Tính bất khả thi của đồng thuận You may have heard about the FLP result [68]—named after the authors Fischer, Lynch, and Paterson—which proves that there is no algorithm that is always able to reach consensus if there is a risk that a node may crash. In a distributed system, we must assume that nodes may crash, so reliable consensus is impossible. Yet, here we are, discussing algorithms for achieving consensus. What is going on here?

Bạn có thể đã nghe về kết quả FLP [68] — được đặt tên theo các tác giả Fischer, Lynch, và Paterson — vốn chứng minh rằng không có thuật toán nào luôn có khả năng đạt được đồng thuận nếu có nguy cơ một nút có thể gặp sự cố. Trong một hệ thống phân tán, ta phải giả định rằng các nút có thể gặp sự cố, nên đồng thuận đáng tin cậy là bất khả thi. Vậy mà, ở đây ta lại đang bàn về các thuật toán để đạt được đồng thuận. Có chuyện gì ở đây vậy?

The answer is that the FLP result is proved in the asynchronous system model (see “System Model and Reality”), a very restrictive model that assumes a deterministic algorithm that cannot use any clocks or timeouts. If the algorithm is allowed to use timeouts, or some other way of identifying suspected crashed nodes (even if the suspicion is sometimes wrong), then consensus becomes solvable [67]. Even just allowing the algorithm to use random numbers is sufficient to get around the impossibility result [69].

Câu trả lời là kết quả FLP được chứng minh trong mô hình hệ thống bất đồng bộ (xem “System Model and Reality”), một mô hình rất hạn chế giả định một thuật toán tất định không thể dùng bất kỳ đồng hồ hay timeout nào. Nếu thuật toán được phép dùng timeout, hoặc một cách nào đó khác để nhận diện các nút bị nghi là đã gặp sự cố (ngay cả khi sự nghi ngờ đó đôi khi sai), thì đồng thuận trở nên giải được [67]. Thậm chí chỉ cần cho phép thuật toán dùng các số ngẫu nhiên cũng đủ để vòng tránh kết quả bất khả thi này [69].

Thus, although the FLP result about the impossibility of consensus is of great theoretical importance, distributed systems can usually achieve consensus in practice.

Do đó, mặc dù kết quả FLP về tính bất khả thi của đồng thuận có tầm quan trọng lý thuyết lớn, các hệ thống phân tán thường có thể đạt được đồng thuận trên thực tế.

In this section we will first examine the atomic commit problem in more detail. In particular, we

will discuss the two-phase commit (2PC) algorithm, which is the most common way of solving atomic commit and which is implemented in various databases, messaging systems, and application servers. It turns out that 2PC is a kind of consensus algorithm—but not a very good one [70, 71].

Trong mục này ta sẽ trước tiên xem xét bài toán commit nguyên tử chi tiết hơn. Cụ thể, ta sẽ bàn về thuật toán commit hai pha (2PC), vốn là cách phổ biến nhất để giải commit nguyên tử và được hiện thực trong nhiều cơ sở dữ liệu, hệ thống nhắn tin, và máy chủ ứng dụng. Hóa ra 2PC là một loại thuật toán đồng thuận — nhưng không phải một thuật toán tốt lắm [70, 71].

By learning from 2PC we will then work our way toward better consensus algorithms, such as those used in ZooKeeper (Zab) and etcd (Raft).

Bằng cách học hỏi từ 2PC, sau đó ta sẽ lần tới các thuật toán đồng thuận tốt hơn, chẳng hạn những thuật toán được dùng trong ZooKeeper (Zab) và etcd (Raft).

Atomic Commit and Two-Phase Commit (2PC) — Commit nguyên tử và Commit hai pha (2PC)

In Chapter 7 we learned that the purpose of transaction atomicity is to provide simple semantics in the case where something goes wrong in the middle of making several writes. The outcome of a transaction is either a successful commit, in which case all of the transaction’s writes are made durable, or an abort, in which case all of the transaction’s writes are rolled back (i.e., undone or discarded).

Ở Chương 7 ta đã học rằng mục đích của tính nguyên tử giao dịch là cung cấp ngữ nghĩa đơn giản trong trường hợp có gì đó trục trặc giữa chừng khi đang thực hiện vài lần ghi. Kết cục của một giao dịch hoặc là một lần commit thành công, trong trường hợp đó tất cả các lần ghi của giao dịch được làm cho bền vững, hoặc là một lần hủy bỏ, trong trường hợp đó tất cả các lần ghi của giao dịch được hoàn tác (tức bị đảo ngược hoặc loại bỏ).

Atomicity prevents failed transactions from littering the database with half-finished results and half-updated state. This is especially important for multi-object transactions (see “Single-Object and Multi-Object Operations”) and databases that maintain secondary indexes. Each secondary **index** is a separate data structure from the primary data—thus, if you modify some data, the corresponding change needs to also be made in the secondary index. Atomicity ensures that the secondary index stays consistent with the primary data (if the index became inconsistent with the primary data, it would not be very useful).

Tính nguyên tử ngăn các giao dịch thất bại làm cơ sở dữ liệu vương vãi những kết quả dở dang và trạng thái cập nhật nửa vời. Điều này đặc biệt quan trọng đối với các giao dịch đa-đối-tượng (xem “Single-Object and Multi-Object Operations”) và các cơ sở dữ liệu duy trì các chỉ mục phụ. Mỗi chỉ mục phụ là một cấu trúc dữ liệu tách biệt khỏi dữ liệu chính — do đó, nếu bạn sửa đổi một số dữ liệu, thay đổi tương ứng cũng cần được thực hiện trong chỉ mục phụ. Tính nguyên tử bảo đảm rằng chỉ mục phụ giữ nhất quán với dữ liệu chính (nếu chỉ mục trở nên bất nhất với dữ liệu chính, nó sẽ không hữu ích lắm).

From single-node to distributed atomic commit — Từ commit nguyên tử đơn nút tới commit nguyên tử phân tán

For transactions that execute at a single database node, atomicity is commonly implemented by the storage **engine**. When the client asks the database node to commit the transaction, the database

makes the transaction's writes durable (typically in a write-ahead log; see "Making B-trees reliable") and then appends a commit record to the log on disk. If the database crashes in the middle of this process, the transaction is recovered from the log when the node restarts: if the commit record was successfully written to disk before the crash, the transaction is considered committed; if not, any writes from that transaction are rolled back.

Đối với các giao dịch thực thi tại một nút cơ sở dữ liệu duy nhất, tính nguyên tử thường được hiện thực bởi engine lưu trữ. Khi client yêu cầu nút cơ sở dữ liệu commit giao dịch, cơ sở dữ liệu làm cho các lần ghi của giao dịch bền vững (thường trong một nhật ký ghi-trước; xem "Making B-trees reliable") rồi thêm một bản ghi commit vào nhật ký trên đĩa. Nếu cơ sở dữ liệu gặp sự cố giữa quá trình này, giao dịch được khôi phục từ nhật ký khi nút khởi động lại: nếu bản ghi commit đã được ghi thành công vào đĩa trước khi gặp sự cố, giao dịch được coi là đã commit; nếu không, bất kỳ lần ghi nào từ giao dịch đó đều bị hoàn tác.

Thus, on a single node, transaction commitment crucially depends on the order in which data is durably written to disk: first the data, then the commit record [72]. The key deciding moment for whether the transaction commits or aborts is the moment at which the disk finishes writing the commit record: before that moment, it is still possible to abort (due to a crash), but after that moment, the transaction is committed (even if the database crashes). Thus, it is a single device (the controller of one particular disk drive, attached to one particular node) that makes the commit atomic.

Do đó, trên một nút duy nhất, việc commit giao dịch phụ thuộc cốt yếu vào thứ tự mà dữ liệu được ghi bền vững vào đĩa: trước tiên là dữ liệu, rồi tới bản ghi commit [72]. Thời điểm quyết định then chốt cho việc giao dịch commit hay hủy bỏ là thời điểm mà đĩa hoàn tất việc ghi bản ghi commit: trước thời điểm đó, vẫn có thể hủy bỏ (do một lần gặp sự cố), nhưng sau thời điểm đó, giao dịch đã được commit (ngay cả khi cơ sở dữ liệu gặp sự cố). Do đó, chính một thiết bị duy nhất (bộ điều khiển của một ổ đĩa cụ thể, gắn với một nút cụ thể) làm cho lần commit trở nên nguyên tử.

However, what if multiple nodes are involved in a transaction? For example, perhaps you have a multi-object transaction in a partitioned database, or a term-partitioned secondary index (in which the index entry may be on a different node from the primary data; see "Partitioning and Secondary Indexes"). Most "NoSQL" distributed datastores do not support such distributed transactions, but various clustered relational systems do (see "Distributed Transactions in Practice").

Tuy nhiên, sẽ thế nào nếu có nhiều nút tham gia vào một giao dịch? Ví dụ, có lẽ bạn có một giao dịch đa-đối-tượng trong một cơ sở dữ liệu được phân vùng, hoặc một chỉ mục phụ được phân vùng theo từ khóa (trong đó mục chỉ mục có thể nằm trên một nút khác với dữ liệu chính; xem "Partitioning and Secondary Indexes"). Hầu hết các kho dữ liệu phân tán "NoSQL" không hỗ trợ các giao dịch phân tán như vậy, nhưng nhiều hệ thống quan hệ phân cụm thì có (xem "Distributed Transactions in Practice").

In these cases, it is not sufficient to simply send a commit request to all of the nodes and independently commit the transaction on each one. In doing so, it could easily happen that the commit succeeds on some nodes and fails on other nodes, which would violate the atomicity guarantee:

Trong những trường hợp này, không đủ để chỉ đơn giản gửi một yêu cầu commit tới tất cả các nút và độc lập commit giao dịch trên từng nút. Làm vậy có thể dễ dàng dẫn tới việc lần commit thành công trên một số nút và thất bại trên những nút khác, điều này sẽ vi phạm bảo đảm tính nguyên tử:

- Some nodes may detect a constraint violation or conflict, making an abort necessary, while

other nodes are successfully able to commit.

- Some of the commit requests might be lost in the network, eventually aborting due to a timeout, while other commit requests get through.
 - Some nodes may crash before the commit record is fully written and roll back on recovery, while others successfully commit.
- Một số nút có thể phát hiện một vi phạm ràng buộc hoặc một xung đột, khiến việc hủy bỏ là cần thiết, trong khi các nút khác lại commit thành công.
 - Một số yêu cầu commit có thể bị mất trên mạng, rồi cuộc hủy bỏ do một timeout, trong khi các yêu cầu commit khác đến được nơi.
 - Một số nút có thể gặp sự cố trước khi bản ghi commit được ghi đầy đủ và hoàn tác khi khôi phục, trong khi những nút khác commit thành công.

If some nodes commit the transaction but others abort it, the nodes become inconsistent with each other (like in Figure 7-3). And once a transaction has been committed on one node, it cannot be retracted again if it later turns out that it was aborted on another node. For this reason, a node must only commit once it is certain that all other nodes in the transaction are also going to commit.

Nếu một số nút commit giao dịch nhưng những nút khác hủy bỏ nó, các nút trở nên bất nhất với nhau (như trong Hình 7-3). Và một khi một giao dịch đã được commit trên một nút, nó không thể bị rút lại lần nữa nếu sau đó hóa ra nó đã bị hủy bỏ trên một nút khác. Vì lý do này, một nút chỉ được commit một khi nó chắc chắn rằng tất cả các nút khác trong giao dịch cũng sẽ commit.

A transaction commit must be irrevocable—you are not allowed to change your mind and retroactively abort a transaction after it has been committed. The reason for this rule is that once data has been committed, it becomes visible to other transactions, and thus other clients may start relying on that data; this principle forms the basis of read committed isolation, discussed in “Read Committed”. If a transaction was allowed to abort after committing, any transactions that read the committed data would be based on data that was retroactively declared not to have existed—so they would have to be reverted as well.

Một lần commit giao dịch phải là không thể thu hồi — bạn không được phép đổi ý và hủy bỏ ngược một giao dịch sau khi nó đã được commit. Lý do của quy tắc này là một khi dữ liệu đã được commit, nó trở nên hiển hiện với các giao dịch khác, và do đó các client khác có thể bắt đầu dựa vào dữ liệu đó; nguyên lý này tạo nền tảng cho cô lập đọc-đã-commit, được bàn ở “Read Committed”. Nếu một giao dịch được phép hủy bỏ sau khi commit, thì bất kỳ giao dịch nào đã đọc dữ liệu đã commit sẽ dựa trên dữ liệu được tuyên bố ngược lại là chưa từng tồn tại — nên chúng cũng sẽ phải bị đảo ngược.

(It is possible for the effects of a committed transaction to later be undone by another, compensating transaction [73, 74]. However, from the database’s point of view this is a separate transaction, and thus any cross-transaction correctness requirements are the application’s problem.)

(Có thể làm cho tác động của một giao dịch đã commit sau đó bị hoàn tác bởi một giao dịch khác, giao dịch bù trừ [73, 74]. Tuy nhiên, từ góc nhìn của cơ sở dữ liệu thì đây là một giao dịch tách biệt, và do đó bất kỳ yêu cầu đúng đắn xuyên-giao-dịch nào cũng là vấn đề của ứng dụng.)

Introduction to two-phase commit — Giới thiệu về commit hai pha

Two-phase commit is an algorithm for achieving atomic transaction commit across multiple nodes—i.e., to ensure that either all nodes commit or all nodes abort. It is a classic algorithm in distributed databases [13, 35, 75]. 2PC is used internally in some databases and also made available

to applications in the form of XA transactions [76, 77] (which are supported by the Java Transaction **API**, for example) or via WS-AtomicTransaction for SOAP web services [78, 79].

Commit hai pha là một thuật toán để đạt được commit giao dịch nguyên tử trên nhiều nút — tức để bảo đảm rằng hoặc tất cả các nút commit hoặc tất cả các nút hủy bỏ. Nó là một thuật toán kinh điển trong các cơ sở dữ liệu phân tán [13, 35, 75]. 2PC được dùng bên trong một số cơ sở dữ liệu và cũng được cung cấp cho các ứng dụng dưới dạng các giao dịch XA [76, 77] (vốn được hỗ trợ bởi Java Transaction API, chẳng hạn) hoặc qua WS-AtomicTransaction cho các dịch vụ web SOAP [78, 79].

The basic flow of 2PC is illustrated in Figure 9-9. Instead of a single commit request, as with a single-node transaction, the commit/abort process in 2PC is split into two phases (hence the name).

Luồng cơ bản của 2PC được minh họa trong Hình 9-9. Thay vì một yêu cầu commit duy nhất, như với một giao dịch đơn-nút, quá trình commit/hủy bỏ trong 2PC được chia thành hai pha (do đó có cái tên này).

Figure 9-9. A successful execution of two-phase commit (2PC). — Hình 9-9. Một lượt thực thi thành công của commit hai pha (2PC).

Don't confuse 2PC and 2PL — Đừng nhầm 2PC với 2PL

Two-phase commit (2PC) and two-phase locking (see “Two-Phase Locking (2PL)”) are two very different things. 2PC provides atomic commit in a distributed database, whereas 2PL provides serializable isolation. To avoid confusion, it's best to think of them as entirely separate concepts and to ignore the unfortunate similarity in the names.

Commit hai pha (2PC) và khóa hai pha (xem “Two-Phase Locking (2PL)”) là hai thứ rất khác nhau. 2PC cung cấp commit nguyên tử trong một cơ sở dữ liệu phân tán, trong khi 2PL cung cấp cô lập tuần tự hóa. Để tránh nhầm lẫn, tốt nhất là nghĩ về chúng như những khái niệm hoàn toàn tách biệt và bỏ qua sự tương đồng đáng tiếc trong tên gọi.

2PC uses a new component that does not normally appear in single-node transactions: a coordinator (also known as transaction manager). The coordinator is often implemented as a library within the same application process that is requesting the transaction (e.g., embedded in a Java EE container), but it can also be a separate process or service. Examples of such coordinators include Narayana, JOTM, BTM, or MSDTC.

2PC dùng một thành phần mới thường không xuất hiện trong các giao dịch đơn-nút: một bộ điều phối (coordinator) (còn gọi là trình quản lý giao dịch). Bộ điều phối thường được hiện thực dưới dạng một thư viện bên trong cùng tiến trình ứng dụng đang yêu cầu giao dịch (chẳng hạn được nhúng trong một container Java EE), nhưng nó cũng có thể là một tiến trình hoặc dịch vụ tách biệt. Các ví dụ về những bộ điều phối như vậy gồm Narayana, JOTM, BTM, hoặc MSDTC.

A 2PC transaction begins with the application reading and writing data on multiple database nodes, as normal. We call these database nodes participants in the transaction. When the application is ready to commit, the coordinator begins phase 1: it sends a prepare request to each of the nodes, asking them whether they are able to commit. The coordinator then tracks the responses from the participants:

Một giao dịch 2PC bắt đầu bằng việc ứng dụng đọc và ghi dữ liệu trên nhiều nút cơ sở dữ liệu, như bình thường. Ta gọi những nút cơ sở dữ liệu này là các bên tham gia (participant) trong giao dịch. Khi ứng dụng sẵn sàng commit, bộ điều phối bắt đầu pha 1: nó gửi một yêu cầu chuẩn bị (prepare) tới mỗi nút, hỏi chúng xem chúng có thể commit hay không. Bộ điều phối sau đó theo dõi các phản hồi từ các bên tham gia:

- If all participants reply “yes,” indicating they are ready to commit, then the coordinator sends out a commit request in phase 2, and the commit actually takes place.
- If any of the participants replies “no,” the coordinator sends an abort request to all nodes in phase 2.

- Nếu tất cả các bên tham gia trả lời “có”, cho biết chúng sẵn sàng commit, thì bộ điều phối gửi đi một yêu cầu commit trong pha 2, và lần commit thực sự diễn ra.
- Nếu bất kỳ bên tham gia nào trả lời “không”, bộ điều phối gửi một yêu cầu hủy bỏ tới tất cả các nút trong pha 2.

This process is somewhat like the traditional marriage ceremony in Western cultures: the minister asks the bride and groom individually whether each wants to marry the other, and typically receives the answer “I do” from both. After receiving both acknowledgments, the minister pronounces the couple husband and wife: the transaction is committed, and the happy fact is broadcast to all attendees. If either bride or groom does not say “yes,” the ceremony is aborted [73].

Quá trình này hơi giống lễ cưới truyền thống trong các nền văn hóa phương Tây: vị mục sư hỏi riêng cô dâu và chú rể xem mỗi người có muốn kết hôn với người kia không, và thường nhận được câu trả lời “Con đồng ý” từ cả hai. Sau khi nhận được cả hai lời báo nhận, vị mục sư tuyên bố cặp đôi thành vợ chồng: giao dịch được commit, và sự kiện hạnh phúc được phát cho tất cả những người dự lễ. Nếu hoặc cô dâu hoặc chú rể không nói “đồng ý”, buổi lễ bị hủy bỏ [73].

A system of promises — Một hệ thống các lời hứa

From this short description it might not be clear why two-phase commit ensures atomicity, while one-phase commit across several nodes does not. Surely the prepare and commit requests can just as easily be lost in the two-phase case. What makes 2PC different?

Từ mô tả ngắn gọn này, có thể chưa rõ vì sao commit hai pha bảo đảm tính nguyên tử, trong khi commit một pha trên vài nút thì không. Chắc chắn các yêu cầu chuẩn bị và commit cũng có thể dễ dàng bị mất trong trường hợp hai pha không kém. Vậy điều gì làm cho 2PC khác biệt?

To understand why it works, we have to break down the process in a bit more detail:

Để hiểu vì sao nó hoạt động, ta phải phân tích quá trình chi tiết hơn một chút:

- When the application wants to begin a distributed transaction, it requests a transaction ID from the coordinator. This transaction ID is globally unique.
- The application begins a single-node transaction on each of the participants, and attaches the globally unique transaction ID to the single-node transaction. All reads and writes are done in one of these single-node transactions. If anything goes wrong at this stage (for example, a node crashes or a request times out), the coordinator or any of the participants can abort.
- When the application is ready to commit, the coordinator sends a prepare request to all participants, tagged with the global transaction ID. If any of these requests fails or times out, the coordinator sends an abort request for that transaction ID to all participants.
- When a participant receives the prepare request, it makes sure that it can definitely commit the transaction under all circumstances. This includes writing all transaction data to disk (a crash, a power **failure**, or running out of disk space is not an acceptable excuse for refusing to commit later), and checking for any conflicts or constraint violations. By replying “yes” to the coordinator, the node promises to commit the transaction without error if requested. In other words, the participant surrenders the right to abort the transaction, but without actually committing it.
- When the coordinator has received responses to all prepare requests, it makes a definitive decision on whether to commit or abort the transaction (committing only if all participants voted “yes”). The coordinator must write that decision to its transaction log on disk so that it knows which way it decided in case it subsequently crashes. This is called the commit point.

- Once the coordinator's decision has been written to disk, the commit or abort request is sent to all participants. If this request fails or times out, the coordinator must retry forever until it succeeds. There is no more going back: if the decision was to commit, that decision must be enforced, no matter how many retries it takes. If a participant has crashed in the meantime, the transaction will be committed when it recovers—since the participant voted “yes,” it cannot refuse to commit when it recovers.

- Khi ứng dụng muốn bắt đầu một giao dịch phân tán, nó yêu cầu một ID giao dịch từ bộ điều phối. ID giao dịch này là duy nhất trên toàn cục.
- Ứng dụng bắt đầu một giao dịch đơn-nút trên mỗi bên tham gia, và đính kèm ID giao dịch duy nhất toàn cục vào giao dịch đơn-nút đó. Tất cả các lần đọc và ghi được thực hiện trong một trong những giao dịch đơn-nút này. Nếu có gì đó trục trặc ở giai đoạn này (ví dụ, một nút gặp sự cố hoặc một yêu cầu hết thời gian chờ), bộ điều phối hoặc bất kỳ bên tham gia nào cũng có thể hủy bỏ.
- Khi ứng dụng sẵn sàng commit, bộ điều phối gửi một yêu cầu chuẩn bị tới tất cả các bên tham gia, được gắn thẻ với ID giao dịch toàn cục. Nếu bất kỳ yêu cầu nào trong số này thất bại hoặc hết thời gian chờ, bộ điều phối gửi một yêu cầu hủy bỏ cho ID giao dịch đó tới tất cả các bên tham gia.
- Khi một bên tham gia nhận được yêu cầu chuẩn bị, nó bảo đảm rằng nó chắc chắn có thể commit giao dịch trong mọi hoàn cảnh. Điều này bao gồm việc ghi tất cả dữ liệu giao dịch vào đĩa (một lần gặp sự cố, mất điện, hoặc hết dung lượng đĩa không phải lý do chấp nhận được để từ chối commit về sau), và kiểm tra mọi xung đột hoặc vi phạm ràng buộc. Bằng cách trả lời “có” cho bộ điều phối, nút hứa sẽ commit giao dịch không lỗi nếu được yêu cầu. Nói cách khác, bên tham gia từ bỏ quyền hủy bỏ giao dịch, nhưng không thực sự commit nó.
- Khi bộ điều phối đã nhận được phản hồi cho tất cả các yêu cầu chuẩn bị, nó đưa ra một quyết định dứt khoát về việc commit hay hủy bỏ giao dịch (chỉ commit nếu tất cả các bên tham gia đã bỏ phiếu “có”). Bộ điều phối phải ghi quyết định đó vào nhật ký giao dịch của nó trên đĩa để nó biết nó đã quyết định theo hướng nào trong trường hợp sau đó nó gặp sự cố. Điều này được gọi là điểm commit (commit point).
- Một khi quyết định của bộ điều phối đã được ghi vào đĩa, yêu cầu commit hoặc hủy bỏ được gửi tới tất cả các bên tham gia. Nếu yêu cầu này thất bại hoặc hết thời gian chờ, bộ điều phối phải thử lại mãi mãi tới khi nó thành công. Không còn đường lui: nếu quyết định là commit, quyết định đó phải được thực thi, bất kể tốn bao nhiêu lần thử lại. Nếu một bên tham gia đã gặp sự cố trong khoảng thời gian đó, giao dịch sẽ được commit khi nó khôi phục — vì bên tham gia đã bỏ phiếu “có”, nó không thể từ chối commit khi nó khôi phục.

Thus, the protocol contains two crucial “points of no return”: when a participant votes “yes,” it promises that it will definitely be able to commit later (although the coordinator may still choose to abort); and once the coordinator decides, that decision is irrevocable. Those promises ensure the atomicity of 2PC. (Single-node atomic commit lumps these two events into one: writing the commit record to the transaction log.)

Do đó, giao thức chứa hai “điểm không thể quay lại” then chốt: khi một bên tham gia bỏ phiếu “có”, nó hứa rằng nó chắc chắn sẽ có thể commit về sau (mặc dù bộ điều phối vẫn có thể chọn hủy bỏ); và một khi bộ điều phối đã quyết định, quyết định đó là không thể thu hồi. Những lời hứa đó bảo đảm tính nguyên tử của 2PC. (Commit nguyên tử đơn-nút gộp hai sự kiện này thành một: việc ghi bản ghi commit vào nhật ký giao dịch.)

Returning to the marriage analogy, before saying “I do,” you and your bride/groom have the freedom to abort the transaction by saying “No way!” (or something to that effect). However, after saying “I

do,” you cannot retract that statement. If you faint after saying “I do” and you don’t hear the minister speak the words “You are now husband and wife,” that doesn’t change the fact that the transaction was committed. When you recover consciousness later, you can find out whether you are married or not by querying the minister for the status of your global transaction ID, or you can wait for the minister’s next retry of the commit request (since the retries will have continued throughout your period of unconsciousness).

Quay lại phép so sánh hôn nhân, trước khi nói “Con đồng ý”, bạn và cô dâu/chú rể của mình có quyền tự do hủy bỏ giao dịch bằng cách nói “Không đời nào!” (hoặc đại loại vậy). Tuy nhiên, sau khi nói “Con đồng ý”, bạn không thể rút lại lời tuyên bố đó. Nếu bạn ngất đi sau khi nói “Con đồng ý” và bạn không nghe thấy vị mục sư nói “Giờ hai con đã thành vợ chồng”, điều đó không thay đổi sự thật rằng giao dịch đã được commit. Khi bạn tỉnh lại sau đó, bạn có thể tìm hiểu xem mình đã kết hôn hay chưa bằng cách hỏi vị mục sư về trạng thái của ID giao dịch toàn cục của bạn, hoặc bạn có thể chờ lần thử lại tiếp theo của vị mục sư cho yêu cầu commit (bởi các lần thử lại sẽ tiếp tục suốt khoảng thời gian bạn bất tỉnh).

Coordinator failure — Sự cố bộ điều phối

We have discussed what happens if one of the participants or the network fails during 2PC: if any of the prepare requests fail or time out, the coordinator aborts the transaction; if any of the commit or abort requests fail, the coordinator retries them indefinitely. However, it is less clear what happens if the coordinator crashes.

Ta đã bàn về điều gì xảy ra nếu một trong các bên tham gia hoặc mạng gặp sự cố trong khi 2PC đang diễn ra: nếu bất kỳ yêu cầu chuẩn bị nào thất bại hoặc hết thời gian chờ, bộ điều phối hủy bỏ giao dịch; nếu bất kỳ yêu cầu commit hoặc hủy bỏ nào thất bại, bộ điều phối thử lại chúng vô thời hạn. Tuy nhiên, kém rõ ràng hơn là điều gì xảy ra nếu bộ điều phối gặp sự cố.

If the coordinator fails before sending the prepare requests, a participant can safely abort the transaction. But once the participant has received a prepare request and voted “yes,” it can no longer abort unilaterally—it must wait to hear back from the coordinator whether the transaction was committed or aborted. If the coordinator crashes or the network fails at this point, the participant can do nothing but wait. A participant’s transaction in this state is called in doubt or uncertain.

Nếu bộ điều phối gặp sự cố trước khi gửi các yêu cầu chuẩn bị, một bên tham gia có thể an toàn hủy bỏ giao dịch. Nhưng một khi bên tham gia đã nhận được một yêu cầu chuẩn bị và bỏ phiếu “có”, nó không còn có thể hủy bỏ một cách đơn phương — nó phải chờ nghe lại từ bộ điều phối xem giao dịch đã được commit hay bị hủy bỏ. Nếu bộ điều phối gặp sự cố hoặc mạng hỏng vào thời điểm này, bên tham gia không thể làm gì khác ngoài chờ đợi. Giao dịch của một bên tham gia ở trạng thái này được gọi là còn nghi vấn (in doubt) hoặc bất định (uncertain).

The situation is illustrated in Figure 9-10. In this particular example, the coordinator actually decided to commit, and database 2 received the commit request. However, the coordinator crashed before it could send the commit request to database 1, and so database 1 does not know whether to commit or abort. Even a timeout does not help here: if database 1 unilaterally aborts after a timeout, it will end up inconsistent with database 2, which has committed. Similarly, it is not safe to unilaterally commit, because another participant may have aborted.

Tình huống này được minh họa trong Hình 9-10. Trong ví dụ cụ thể này, bộ điều phối thực

ra đã quyết định commit, và cơ sở dữ liệu 2 đã nhận được yêu cầu commit. Tuy nhiên, bộ điều phối gặp sự cố trước khi nó có thể gửi yêu cầu commit tới cơ sở dữ liệu 1, nên cơ sở dữ liệu 1 không biết nên commit hay hủy bỏ. Ngay cả một timeout cũng không giúp được gì ở đây: nếu cơ sở dữ liệu 1 đơn phương hủy bỏ sau một timeout, nó sẽ rất cuộc bất nhất với cơ sở dữ liệu 2, vốn đã commit. Tương tự, không an toàn nếu đơn phương commit, bởi một bên tham gia khác có thể đã hủy bỏ.

Figure 9-10. The coordinator crashes after participants vote “yes.” Database 1 does not know whether to commit or abort. — Hình 9-10. Bộ điều phối gặp sự cố sau khi các bên tham gia bỏ phiếu “có”. Cơ sở dữ liệu 1 không biết nên commit hay hủy bỏ. Without hearing from the coordinator, the participant has no way of knowing whether to commit or abort. In principle, the participants could communicate among themselves to find out how each participant voted and come to some agreement, but that is not part of the 2PC protocol.

Không nghe được từ bộ điều phối, bên tham gia không có cách nào biết nên commit hay hủy bỏ. Về nguyên tắc, các bên tham gia có thể giao tiếp với nhau để tìm ra mỗi bên đã bỏ phiếu thế nào và đi tới một thỏa thuận nào đó, nhưng điều đó không thuộc giao thức 2PC.

The only way 2PC can complete is by waiting for the coordinator to recover. This is why the coordinator must write its commit or abort decision to a transaction log on disk before sending commit or abort requests to participants: when the coordinator recovers, it determines the status of all in-doubt transactions by reading its transaction log. Any transactions that don't have a commit record in the coordinator's log are aborted. Thus, the commit point of 2PC comes down to a regular single-node atomic commit on the coordinator.

Cách duy nhất để 2PC có thể hoàn tất là chờ bộ điều phối khôi phục. Đây là lý do vì sao bộ điều phối phải ghi quyết định commit hoặc hủy bỏ của nó vào một nhật ký giao dịch trên đĩa trước khi gửi các yêu cầu commit hoặc hủy bỏ tới các bên tham gia: khi bộ điều phối khôi phục, nó xác định trạng thái của tất cả các giao dịch còn nghi vấn bằng cách đọc nhật ký giao dịch của nó. Bất kỳ giao dịch nào không có bản ghi commit trong nhật ký của bộ điều phối đều bị hủy bỏ. Do đó, điểm commit của 2PC quy về một lần commit nguyên tử đơn-nút thông thường trên bộ điều phối.

Three-phase commit — Commit ba pha

Two-phase commit is called a blocking atomic commit protocol due to the fact that 2PC can become stuck waiting for the coordinator to recover. In theory, it is possible to make an atomic commit protocol nonblocking, so that it does not get stuck if a node fails. However, making this work in practice is not so straightforward.

Commit hai pha được gọi là một giao thức commit nguyên tử có chặn (blocking) do thực tế là 2PC có thể bị mắc kẹt chờ bộ điều phối khôi phục. Về lý thuyết, có thể làm cho một giao thức commit nguyên tử không-chặn (nonblocking), để nó không bị mắc kẹt nếu một nút gặp sự cố. Tuy nhiên, làm cho điều này hoạt động trên thực tế thì không đơn giản như vậy.

As an alternative to 2PC, an algorithm called three-phase commit (3PC) has been proposed [13, 80]. However, 3PC assumes a network with bounded delay and nodes with bounded response times; in most practical systems with unbounded network delay and process pauses (see Chapter 8), it cannot guarantee atomicity.

Như một giải pháp thay thế cho 2PC, một thuật toán gọi là commit ba pha (3PC) đã được đề xuất [13, 80]. Tuy nhiên, 3PC giả định một mạng với trì hoãn có chặn trên và các nút

với thời gian phản hồi có chặn trên; trong hầu hết các hệ thống thực tế với trì hoãn mạng không có chặn trên và các lần tạm dừng tiến trình (xem Chương 8), nó không thể bảo đảm tính nguyên tử.

In general, nonblocking atomic commit requires a perfect failure detector [67, 71]—i.e., a reliable mechanism for telling whether a node has crashed or not. In a network with unbounded delay a timeout is not a reliable failure detector, because a request may time out due to a network problem even if no node has crashed. For this reason, 2PC continues to be used, despite the known problem with coordinator failure.

Nói chung, commit nguyên tử không-chặn đòi hỏi một bộ phát hiện sự cố hoàn hảo [67, 71] — tức một cơ chế đáng tin cậy để biết liệu một nút đã gặp sự cố hay chưa. Trong một mạng với trì hoãn không có chặn trên, một timeout không phải một bộ phát hiện sự cố đáng tin cậy, bởi một yêu cầu có thể hết thời gian chờ do một vấn đề mạng ngay cả khi không có nút nào gặp sự cố. Vì lý do này, 2PC vẫn tiếp tục được dùng, bất chấp vấn đề đã biết với sự cố bộ điều phối.

Distributed Transactions in Practice — Giao dịch phân tán trong thực tế

Distributed transactions, especially those implemented with two-phase commit, have a mixed reputation. On the one hand, they are seen as providing an important safety guarantee that would be hard to achieve otherwise; on the other hand, they are criticized for causing operational problems, killing performance, and promising more than they can deliver [81, 82, 83, 84]. Many cloud services choose not to implement distributed transactions due to the operational problems they engender [85, 86].

Các giao dịch phân tán, đặc biệt là những giao dịch được hiện thực bằng commit hai pha, mang một tiếng tăm lẫn lộn. Một mặt, chúng được xem là cung cấp một bảo đảm an toàn quan trọng vốn khó đạt được bằng cách khác; mặt khác, chúng bị chỉ trích vì gây ra các vấn đề vận hành, bóp nghẹt hiệu năng, và hứa hẹn nhiều hơn mức chúng có thể thực hiện [81, 82, 83, 84]. Nhiều dịch vụ đám mây chọn không hiện thực các giao dịch phân tán do các vấn đề vận hành mà chúng gây ra [85, 86].

Some implementations of distributed transactions carry a heavy performance penalty—for example, distributed transactions in MySQL are reported to be over 10 times slower than single-node transactions [87], so it is not surprising when people advise against using them. Much of the performance cost inherent in two-phase commit is due to the additional disk forcing (fsync) that is required for crash recovery [88], and the additional network round-trips.

Một số hiện thực của các giao dịch phân tán mang một hình phạt hiệu năng nặng nề — ví dụ, các giao dịch phân tán trong MySQL được báo cáo là chậm hơn 10 lần so với các giao dịch đơn-nút [87], nên không có gì ngạc nhiên khi người ta khuyên không nên dùng chúng. Phần lớn chi phí hiệu năng cố hữu trong commit hai pha là do việc ép ghi đĩa bổ sung (fsync) cần thiết cho việc khôi phục sau sự cố [88], và các vòng khứ hồi mạng bổ sung.

However, rather than dismissing distributed transactions outright, we should examine them in some more detail, because there are important lessons to be learned from them. To begin, we should be precise about what we mean by “distributed transactions.” Two quite different types of distributed transactions are often conflated:

Tuy nhiên, thay vì gạt bỏ hoàn toàn các giao dịch phân tán, ta nên xem xét chúng chi tiết hơn một chút, bởi có những bài học quan trọng cần rút ra từ chúng. Để bắt đầu, ta nên

chính xác về điều ta muốn nói qua từ “giao dịch phân tán”. Hai loại giao dịch phân tán khá khác nhau thường bị gộp lẫn:

Some distributed databases (i.e., databases that use replication and partitioning in their standard configuration) support internal transactions among the nodes of that database. For example, VoltDB and MySQL Cluster’s NDB storage engine have such internal transaction support. In this case, all the nodes participating in the transaction are running the same database software.

Một số cơ sở dữ liệu phân tán (tức các cơ sở dữ liệu dùng nhân bản và phân vùng trong cấu hình tiêu chuẩn của chúng) hỗ trợ các giao dịch nội bộ giữa các nút của cơ sở dữ liệu đó. Ví dụ, VoltDB và engine lưu trữ NDB của MySQL Cluster có sự hỗ trợ giao dịch nội bộ như vậy. Trong trường hợp này, tất cả các nút tham gia vào giao dịch đều đang chạy cùng một phần mềm cơ sở dữ liệu.

In a **heterogeneous** transaction, the participants are two or more different technologies: for example, two databases from different vendors, or even non-database systems such as message brokers. A distributed transaction across these systems must ensure atomic commit, even though the systems may be entirely different under the hood.

Trong một giao dịch không đồng nhất (heterogeneous transaction), các bên tham gia là hai hoặc nhiều công nghệ khác nhau: ví dụ, hai cơ sở dữ liệu từ các nhà cung cấp khác nhau, hoặc thậm chí các hệ thống phi-cơ-sở-dữ-liệu như các message broker. Một giao dịch phân tán xuyên các hệ thống này phải bảo đảm commit nguyên tử, ngay cả khi các hệ thống có thể hoàn toàn khác nhau ở bên dưới.

Database-internal transactions do not have to be compatible with any other system, so they can use any protocol and apply optimizations specific to that particular technology. For that reason, database-internal distributed transactions can often work quite well. On the other hand, transactions spanning heterogeneous technologies are a lot more challenging.

Các giao dịch nội bộ cơ sở dữ liệu không phải tương thích với bất kỳ hệ thống nào khác, nên chúng có thể dùng bất kỳ giao thức nào và áp dụng các tối ưu hóa đặc thù cho công nghệ cụ thể đó. Vì lý do đó, các giao dịch phân tán nội bộ cơ sở dữ liệu thường có thể hoạt động khá tốt. Mặt khác, các giao dịch trải rộng trên các công nghệ không đồng nhất thì khó hơn nhiều.

Exactly-once message processing — Xử lý thông điệp đúng-một-lần

Heterogeneous distributed transactions allow diverse systems to be integrated in powerful ways. For example, a message from a message queue can be acknowledged as processed if and only if the database transaction for processing the message was successfully committed. This is implemented by atomically committing the message acknowledgment and the database writes in a single transaction. With distributed transaction support, this is possible, even if the message broker and the database are two unrelated technologies running on different machines.

Các giao dịch phân tán không đồng nhất cho phép các hệ thống đa dạng được tích hợp theo những cách mạnh mẽ. Ví dụ, một thông điệp từ một hàng đợi thông điệp có thể được báo nhận là đã xử lý khi và chỉ khi giao dịch cơ sở dữ liệu để xử lý thông điệp đã được commit thành công. Điều này được hiện thực bằng cách commit một cách nguyên tử việc báo nhận thông điệp và các lần ghi cơ sở dữ liệu trong một giao dịch duy nhất. Với sự hỗ trợ giao dịch phân tán, điều này là khả thi, ngay cả khi message broker và cơ sở dữ liệu là hai công nghệ không liên quan chạy trên các máy khác nhau.

If either the message delivery or the database transaction fails, both are aborted, and so the message

broker may safely redeliver the message later. Thus, by atomically committing the message and the side effects of its processing, we can ensure that the message is effectively processed exactly once, even if it required a few retries before it succeeded. The abort discards any side effects of the partially completed transaction.

Nếu hoặc việc chuyển thông điệp hoặc giao dịch cơ sở dữ liệu thất bại, cả hai đều bị hủy bỏ, và do đó message broker có thể an toàn chuyển lại thông điệp về sau. Do đó, bằng cách commit một cách nguyên tử thông điệp và các tác dụng phụ của việc xử lý nó, ta có thể bảo đảm rằng thông điệp được xử lý hiệu quả đúng một lần, ngay cả khi nó cần vài lần thử lại trước khi thành công. Việc hủy bỏ loại bỏ mọi tác dụng phụ của giao dịch hoàn tất nửa vời.

Such a distributed transaction is only possible if all systems affected by the transaction are able to use the same atomic commit protocol, however. For example, say a **side effect** of processing a message is to send an email, and the email server does not support two-phase commit: it could happen that the email is sent two or more times if message processing fails and is retried. But if all side effects of processing a message are rolled back on transaction abort, then the processing step can safely be retried as if nothing had happened.

Một giao dịch phân tán như vậy chỉ khả thi nếu tất cả các hệ thống bị ảnh hưởng bởi giao dịch đều có thể dùng cùng một giao thức commit nguyên tử. Ví dụ, giả sử một tác dụng phụ của việc xử lý một thông điệp là gửi một email, và máy chủ email không hỗ trợ commit hai pha: có thể xảy ra việc email được gửi hai lần trở lên nếu việc xử lý thông điệp thất bại và được thử lại. Nhưng nếu tất cả các tác dụng phụ của việc xử lý một thông điệp được hoàn tác khi giao dịch hủy bỏ, thì bước xử lý có thể được an toàn thử lại như thể không có gì xảy ra.

We will return to the topic of exactly-once message processing in Chapter 11. Let's look first at the atomic commit protocol that allows such heterogeneous distributed transactions.

Ta sẽ quay lại chủ đề xử lý thông điệp đúng-một-lần ở Chương 11. Trước tiên hãy xem giao thức commit nguyên tử cho phép các giao dịch phân tán không đồng nhất như vậy.

XA transactions — Giao dịch XA

X/Open XA (short for eXtended Architecture) is a standard for implementing two-phase commit across heterogeneous technologies [76, 77]. It was introduced in 1991 and has been widely implemented: XA is supported by many traditional relational databases (including PostgreSQL, MySQL, DB2, **SQL** Server, and Oracle) and message brokers (including ActiveMQ, HornetQ, MSMQ, and IBM MQ).

X/Open XA (viết tắt của eXtended Architecture) là một chuẩn để hiện thực commit hai pha xuyên các công nghệ không đồng nhất [76, 77]. Nó được giới thiệu năm 1991 và đã được hiện thực rộng rãi: XA được hỗ trợ bởi nhiều cơ sở dữ liệu quan hệ truyền thống (bao gồm PostgreSQL, MySQL, DB2, SQL Server, và Oracle) và các message broker (bao gồm ActiveMQ, HornetQ, MSMQ, và IBM MQ).

XA is not a network protocol—it is merely a C API for interfacing with a transaction coordinator. Bindings for this API exist in other languages; for example, in the world of Java EE applications, XA transactions are implemented using the Java Transaction API (JTA), which in turn is supported by many drivers for databases using Java Database Connectivity (JDBC) and drivers for message brokers using the Java Message Service (JMS) APIs.

XA không phải một giao thức mạng — nó chỉ đơn thuần là một API C để giao tiếp với một

bộ điều phối giao dịch. Các ràng buộc (binding) cho API này tồn tại trong các ngôn ngữ khác; ví dụ, trong thế giới các ứng dụng Java EE, các giao dịch XA được hiện thực bằng Java Transaction API (JTA), vốn lại được hỗ trợ bởi nhiều driver cho các cơ sở dữ liệu dùng Java Database Connectivity (JDBC) và các driver cho các message broker dùng các API Java Message Service (JMS).

XA assumes that your application uses a network driver or client library to communicate with the participant databases or messaging services. If the driver supports XA, that means it calls the XA API to find out whether an operation should be part of a distributed transaction—and if so, it sends the necessary information to the database server. The driver also exposes callbacks through which the coordinator can ask the participant to prepare, commit, or abort.

XA giả định rằng ứng dụng của bạn dùng một driver mạng hoặc thư viện client để giao tiếp với các cơ sở dữ liệu hoặc dịch vụ nhắn tin tham gia. Nếu driver hỗ trợ XA, điều đó nghĩa là nó gọi API XA để tìm hiểu xem một thao tác có nên là một phần của một giao dịch phân tán hay không — và nếu có, nó gửi thông tin cần thiết tới máy chủ cơ sở dữ liệu. Driver cũng phơi bày các callback mà qua đó bộ điều phối có thể yêu cầu bên tham gia chuẩn bị, commit, hoặc hủy bỏ.

The transaction coordinator implements the XA API. The standard does not specify how it should be implemented, but in practice the coordinator is often simply a library that is loaded into the same process as the application issuing the transaction (not a separate service). It keeps track of the participants in a transaction, collects participants' responses after asking them to prepare (via a callback into the driver), and uses a log on the local disk to keep track of the commit/abort decision for each transaction.

Bộ điều phối giao dịch hiện thực API XA. Chuẩn này không quy định nó nên được hiện thực ra sao, nhưng trên thực tế bộ điều phối thường chỉ đơn giản là một thư viện được nạp vào cùng tiến trình với ứng dụng đang phát ra giao dịch (chứ không phải một dịch vụ tách biệt). Nó theo dõi các bên tham gia trong một giao dịch, thu thập các phản hồi của các bên tham gia sau khi yêu cầu chúng chuẩn bị (qua một callback vào driver), và dùng một nhật ký trên đĩa cục bộ để theo dõi quyết định commit/hủy bỏ cho mỗi giao dịch.

If the application process crashes, or the machine on which the application is running dies, the coordinator goes with it. Any participants with prepared but uncommitted transactions are then stuck in doubt. Since the coordinator's log is on the application server's local disk, that server must be restarted, and the coordinator library must read the log to recover the commit/abort outcome of each transaction. Only then can the coordinator use the database driver's XA callbacks to ask participants to commit or abort, as appropriate. The database server cannot contact the coordinator directly, since all communication must go via its client library.

Nếu tiến trình ứng dụng gặp sự cố, hoặc máy mà ứng dụng đang chạy trên đó chết, bộ điều phối cũng đi theo nó. Bất kỳ bên tham gia nào có các giao dịch đã chuẩn bị nhưng chưa commit sẽ mắc kẹt trong tình trạng còn nghi vấn. Vì nhật ký của bộ điều phối nằm trên đĩa cục bộ của máy chủ ứng dụng, máy chủ đó phải được khởi động lại, và thư viện bộ điều phối phải đọc nhật ký để khôi phục kết cục commit/hủy bỏ của mỗi giao dịch. Chỉ khi đó bộ điều phối mới có thể dùng các callback XA của driver cơ sở dữ liệu để yêu cầu các bên tham gia commit hoặc hủy bỏ, tùy theo trường hợp. Máy chủ cơ sở dữ liệu không thể liên lạc với bộ điều phối trực tiếp, bởi mọi giao tiếp phải đi qua thư viện client của nó.

Holding locks while in doubt — Giữ khóa trong khi còn nghi vấn

Why do we care so much about a transaction being stuck in doubt? Can't the rest of the system just get on with its work, and ignore the in-doubt transaction that will be cleaned up eventually?

Vì sao ta quan tâm nhiều đến vậy về việc một giao dịch mắc kẹt trong tình trạng còn nghi vấn? Chẳng phải phần còn lại của hệ thống cứ tiếp tục công việc của mình, và bỏ qua giao dịch còn nghi vấn vốn rồi cuộc sẽ được dọn dẹp hay sao?

The problem is with locking. As discussed in “Read Committed”, database transactions usually take a row-level exclusive lock on any rows they modify, to prevent dirty writes. In addition, if you want serializable isolation, a database using two-phase locking would also have to take a shared lock on any rows read by the transaction (see “Two-Phase Locking (2PL)”).

Vấn đề nằm ở việc khóa. Như đã bàn ở “Read Committed”, các giao dịch cơ sở dữ liệu thường lấy một khóa độc quyền mức-hàng trên bất kỳ hàng nào chúng sửa đổi, để ngăn các lần ghi bẩn. Ngoài ra, nếu bạn muốn cô lập tuần tự hóa, một cơ sở dữ liệu dùng khóa hai pha cũng sẽ phải lấy một khóa chia sẻ trên bất kỳ hàng nào được giao dịch đọc (xem “Two-Phase Locking (2PL)”).

The database cannot release those locks until the transaction commits or aborts (illustrated as a shaded area in Figure 9-9). Therefore, when using two-phase commit, a transaction must hold onto the locks throughout the time it is in doubt. If the coordinator has crashed and takes 20 minutes to start up again, those locks will be held for 20 minutes. If the coordinator's log is entirely lost for some reason, those locks will be held forever—or at least until the situation is manually resolved by an administrator.

Cơ sở dữ liệu không thể giải phóng những khóa đó tới khi giao dịch commit hoặc hủy bỏ (được minh họa dưới dạng một vùng tô đậm trong Hình 9-9). Do đó, khi dùng commit hai pha, một giao dịch phải giữ chặt các khóa suốt thời gian nó còn nghi vấn. Nếu bộ điều phối đã gặp sự cố và mất 20 phút để khởi động lại, những khóa đó sẽ bị giữ trong 20 phút. Nếu nhật ký của bộ điều phối hoàn toàn bị mất vì lý do nào đó, những khóa đó sẽ bị giữ mãi mãi — hoặc ít nhất tới khi tình huống được giải quyết thủ công bởi một quản trị viên.

While those locks are held, no other transaction can modify those rows. Depending on the database, other transactions may even be blocked from reading those rows. Thus, other transactions cannot simply continue with their business—if they want to access that same data, they will be blocked. This can cause large parts of your application to become unavailable until the in-doubt transaction is resolved.

Trong khi những khóa đó bị giữ, không giao dịch nào khác có thể sửa đổi những hàng đó. Tùy theo cơ sở dữ liệu, các giao dịch khác thậm chí có thể bị chặn không cho đọc những hàng đó. Do đó, các giao dịch khác không thể đơn giản tiếp tục công việc của mình — nếu chúng muốn truy cập cùng dữ liệu đó, chúng sẽ bị chặn. Điều này có thể khiến những phần lớn của ứng dụng trở nên không sẵn sàng cho tới khi giao dịch còn nghi vấn được giải quyết.

Recovering from coordinator failure — Khôi phục từ sự cố bộ điều phối

In theory, if the coordinator crashes and is restarted, it should cleanly recover its state from the log and resolve any in-doubt transactions. However, in practice, orphaned in-doubt transactions do occur [89, 90]—that is, transactions for which the coordinator cannot decide the outcome for whatever reason (e.g., because the transaction log has been lost or corrupted due to a software **bug**).

These transactions cannot be resolved automatically, so they sit forever in the database, holding locks and blocking other transactions.

Về lý thuyết, nếu bộ điều phối gặp sự cố và được khởi động lại, nó sẽ khôi phục trạng thái của mình một cách sạch sẽ từ nhật ký và giải quyết bất kỳ giao dịch còn nghi vấn nào. Tuy nhiên, trên thực tế, các giao dịch còn nghi vấn mờ mờ ảo ảo thực có xảy ra [89, 90] — tức các giao dịch mà bộ điều phối không thể quyết định kết cục vì lý do nào đó (chẳng hạn vì nhật ký giao dịch đã bị mất hoặc hỏng do một bug phần mềm). Những giao dịch này không thể được giải quyết một cách tự động, nên chúng ngồi đó mãi mãi trong cơ sở dữ liệu, giữ các khóa và chặn các giao dịch khác.

Even rebooting your database servers will not fix this problem, since a correct implementation of 2PC must preserve the locks of an in-doubt transaction even across restarts (otherwise it would risk violating the atomicity guarantee). It's a sticky situation.

Ngay cả việc khởi động lại các máy chủ cơ sở dữ liệu của bạn cũng sẽ không khắc phục được vấn đề này, bởi một hiện thực đúng đắn của 2PC phải bảo toàn các khóa của một giao dịch còn nghi vấn ngay cả qua các lần khởi động lại (nếu không nó sẽ có nguy cơ vi phạm bảo đảm tính nguyên tử). Đây là một tình huống nan giải.

The only way out is for an administrator to manually decide whether to commit or roll back the transactions. The administrator must examine the participants of each in-doubt transaction, determine whether any participant has committed or aborted already, and then apply the same outcome to the other participants. Resolving the problem potentially requires a lot of manual effort, and most likely needs to be done under high stress and time pressure during a serious production outage (otherwise, why would the coordinator be in such a bad state?).

Lối thoát duy nhất là để một quản trị viên quyết định thủ công xem nên commit hay hoàn tác các giao dịch. Quản trị viên phải xem xét các bên tham gia của mỗi giao dịch còn nghi vấn, xác định xem có bên tham gia nào đã commit hoặc hủy bỏ chưa, rồi áp dụng cùng kết cục đó cho các bên tham gia khác. Việc giải quyết vấn đề có thể đòi hỏi rất nhiều công sức thủ công, và nhiều khả năng cần được thực hiện dưới áp lực cao và sức ép thời gian trong một lần gián đoạn dịch vụ production nghiêm trọng (nếu không, vì sao bộ điều phối lại ở trong tình trạng tồi tệ như vậy?).

Many XA implementations have an emergency escape hatch called heuristic decisions: allowing a participant to unilaterally decide to abort or commit an in-doubt transaction without a definitive decision from the coordinator [76, 77, 91]. To be clear, heuristic here is a euphemism for probably breaking atomicity, since it violates the system of promises in two-phase commit. Thus, heuristic decisions are intended only for getting out of catastrophic situations, and not for regular use.

Nhiều hiện thực XA có một cửa thoát hiểm khẩn cấp gọi là các quyết định mang tính kinh nghiệm (heuristic decisions): cho phép một bên tham gia đơn phương quyết định hủy bỏ hoặc commit một giao dịch còn nghi vấn mà không có một quyết định dứt khoát từ bộ điều phối [76, 77, 91]. Nói rõ ra, “mang tính kinh nghiệm” ở đây là một uyển ngữ cho việc có lẽ phá vỡ tính nguyên tử, bởi nó vi phạm hệ thống các lời hứa trong commit hai pha. Do đó, các quyết định mang tính kinh nghiệm chỉ nhằm thoát khỏi các tình huống thảm họa, chứ không phải để dùng thường xuyên.

Limitations of distributed transactions — Các hạn chế của giao dịch phân tán

XA transactions solve the real and important problem of keeping several participant data systems consistent with each other, but as we have seen, they also introduce major operational problems. In particular, the key realization is that the transaction coordinator is itself a kind of database (in which

transaction outcomes are stored), and so it needs to be approached with the same care as any other important database:

Các giao dịch XA giải quyết vấn đề thực sự và quan trọng là giữ cho vài hệ thống dữ liệu tham gia nhất quán với nhau, nhưng như ta đã thấy, chúng cũng đưa vào những vấn đề vận hành lớn. Cụ thể, nhận định then chốt là bản thân bộ điều phối giao dịch là một loại cơ sở dữ liệu (trong đó các kết cục giao dịch được lưu), và do đó nó cần được tiếp cận với sự cẩn trọng giống như bất kỳ cơ sở dữ liệu quan trọng nào khác:

- If the coordinator is not replicated but runs only on a single machine, it is a single point of failure for the entire system (since its failure causes other application servers to block on locks held by in-doubt transactions). Surprisingly, many coordinator implementations are not highly available by default, or have only rudimentary replication support.
 - Many server-side applications are developed in a **stateless** model (as favored by HTTP), with all persistent state stored in a database, which has the advantage that application servers can be added and removed at will. However, when the coordinator is part of the application server, it changes the nature of the deployment. Suddenly, the coordinator's logs become a crucial part of the durable system state—as important as the databases themselves, since the coordinator logs are required in order to recover in-doubt transactions after a crash. Such application servers are no longer stateless.
 - Since XA needs to be compatible with a wide range of data systems, it is necessarily a lowest common denominator. For example, it cannot detect deadlocks across different systems (since that would require a standardized protocol for systems to exchange information on the locks that each transaction is waiting for), and it does not work with SSI (see “Serializable Snapshot Isolation (SSI)”), since that would require a protocol for identifying conflicts across different systems.
 - For database-internal distributed transactions (not XA), the limitations are not so great—for example, a distributed version of SSI is possible. However, there remains the problem that for 2PC to successfully commit a transaction, all participants must respond. Consequently, if any part of the system is broken, the transaction also fails. Distributed transactions thus have a tendency of amplifying failures, which runs counter to our goal of building fault-tolerant systems.
- Nếu bộ điều phối không được nhân bản mà chỉ chạy trên một máy duy nhất, nó là một điểm hỏng đơn lẻ cho toàn bộ hệ thống (bởi sự cố của nó khiến các máy chủ ứng dụng khác bị chặn trên các khóa được giữ bởi các giao dịch còn nghi vấn). Đáng ngạc nhiên là nhiều hiện thực bộ điều phối không có tính sẵn sàng cao theo mặc định, hoặc chỉ có sự hỗ trợ nhân bản sơ khai.
 - Nhiều ứng dụng phía máy chủ được phát triển theo một mô hình không trạng thái (như được HTTP ưa chuộng), với tất cả trạng thái bền vững được lưu trong một cơ sở dữ liệu, điều này có ưu điểm là các máy chủ ứng dụng có thể được thêm vào và gỡ bỏ tùy ý. Tuy nhiên, khi bộ điều phối là một phần của máy chủ ứng dụng, nó thay đổi bản chất của việc triển khai. Đột nhiên, các nhật ký của bộ điều phối trở thành một phần cốt yếu của trạng thái hệ thống bền vững — quan trọng như chính các cơ sở dữ liệu, bởi các nhật ký bộ điều phối là cần thiết để khôi phục các giao dịch còn nghi vấn sau một lần gặp sự cố. Những máy chủ ứng dụng như vậy không còn không trạng thái nữa.
 - Vì XA cần tương thích với một phạm vi rộng các hệ thống dữ liệu, nó tất yếu là một mẫu số chung nhỏ nhất. Ví dụ, nó không thể phát hiện các bế tắc (deadlock) xuyên các hệ thống khác nhau (bởi điều đó sẽ đòi hỏi một giao thức chuẩn hóa để các hệ thống trao đổi thông tin về các khóa mà mỗi giao dịch đang chờ), và nó không hoạt động với SSI (xem “Serializable Snapshot Isolation (SSI)”), bởi điều đó sẽ đòi hỏi một

- giao thức để nhận diện các xung đột xuyên các hệ thống khác nhau.
- Đối với các giao dịch phân tán nội bộ cơ sở dữ liệu (không phải XA), các hạn chế không lớn đến vậy — ví dụ, một phiên bản phân tán của SSI là khả thi. Tuy nhiên, vẫn còn vấn đề là để 2PC commit thành công một giao dịch, tất cả các bên tham gia phải phản hồi. Do đó, nếu bất kỳ phần nào của hệ thống bị trục trặc, giao dịch cũng thất bại. Các giao dịch phân tán do đó có xu hướng khuếch đại các hỏng hóc, điều này đi ngược lại mục tiêu của ta về việc xây dựng các hệ thống chịu lỗi.

Do these facts mean we should give up all hope of keeping several systems consistent with each other? Not quite—there are alternative methods that allow us to achieve the same thing without the pain of heterogeneous distributed transactions. We will return to these in Chapters 11 and 12. But first, we should wrap up the topic of consensus.

Liệu những sự kiện này có nghĩa là ta nên từ bỏ mọi hy vọng giữ cho vài hệ thống nhất quán với nhau? Không hẳn — có những phương pháp thay thế cho phép ta đạt được cùng điều đó mà không phải chịu nỗi đau của các giao dịch phân tán không đồng nhất. Ta sẽ quay lại những phương pháp này ở Chương 11 và 12. Nhưng trước tiên, ta nên kết thúc chủ đề đồng thuận.

Fault-Tolerant Consensus — Đồng thuận chịu lỗi

Informally, consensus means getting several nodes to agree on something. For example, if several people concurrently try to book the last seat on an airplane, or the same seat in a theater, or try to register an account with the same username, then a consensus algorithm could be used to determine which one of these mutually incompatible operations should be the winner.

Một cách phi hình thức, đồng thuận nghĩa là làm cho vài nút nhất trí về một điều gì đó. Ví dụ, nếu vài người đồng thời cố đặt chiếc ghế cuối cùng trên một máy bay, hoặc cùng một ghế trong một rạp hát, hoặc cố đăng ký một tài khoản với cùng một tên người dùng, thì một thuật toán đồng thuận có thể được dùng để xác định cái nào trong những thao tác xung khắc lẫn nhau này nên là người thắng.

The consensus problem is normally formalized as follows: one or more nodes may propose values, and the consensus algorithm decides on one of those values. In the seat-booking example, when several customers are concurrently trying to buy the last seat, each node handling a customer request may propose the ID of the customer it is serving, and the decision indicates which one of those customers got the seat.

Bài toán đồng thuận thường được hình thức hóa như sau: một hoặc nhiều nút có thể đề xuất các giá trị, và thuật toán đồng thuận quyết định chọn một trong những giá trị đó. Trong ví dụ đặt ghế, khi vài khách hàng đồng thời cố mua chiếc ghế cuối cùng, mỗi nút xử lý một yêu cầu khách hàng có thể đề xuất ID của khách hàng mà nó đang phục vụ, và quyết định cho biết cái nào trong những khách hàng đó đã giành được ghế.

In this formalism, a consensus algorithm must satisfy the following properties [25]:

Trong cách hình thức hóa này, một thuật toán đồng thuận phải thỏa mãn các thuộc tính sau [25]:

No two nodes decide differently.

Không có hai nút quyết định khác nhau.

No node decides twice.

Không có nút nào quyết định hai lần.

If a node decides value v , then v was proposed by some node.

Nếu một nút quyết định giá trị v , thì v đã được đề xuất bởi một nút nào đó.

Every node that does not crash eventually decides some value.

Mọi nút không gặp sự cố rốt cuộc đều quyết định một giá trị nào đó.

The uniform agreement and integrity properties define the core idea of consensus: everyone decides on the same outcome, and once you have decided, you cannot change your mind. The validity property exists mostly to rule out trivial solutions: for example, you could have an algorithm that always decides null, no matter what was proposed; this algorithm would satisfy the agreement and integrity properties, but not the validity property.

Các thuộc tính nhất trí đồng dạng (uniform agreement) và toàn vẹn (integrity) định nghĩa ý tưởng cốt lõi của đồng thuận: mọi người đều quyết định cùng một kết cục, và một khi bạn đã quyết định, bạn không thể đổi ý. Thuộc tính hợp lệ (validity) tồn tại chủ yếu để loại trừ các giải pháp tầm thường: ví dụ, bạn có thể có một thuật toán luôn quyết định null, bất kể đã đề xuất gì; thuật toán này sẽ thỏa mãn các thuộc tính nhất trí và toàn vẹn, nhưng không thỏa mãn thuộc tính hợp lệ.

If you don't care about fault tolerance, then satisfying the first three properties is easy: you can just hardcode one node to be the “dictator,” and let that node make all of the decisions. However, if that one node fails, then the system can no longer make any decisions. This is, in fact, what we saw in the case of two-phase commit: if the coordinator fails, in-doubt participants cannot decide whether to commit or abort.

Nếu bạn không quan tâm đến khả năng chịu lỗi, thì việc thỏa mãn ba thuộc tính đầu tiên là dễ: bạn chỉ cần mã hóa cứng một nút làm “kẻ độc tài”, và để nút đó đưa ra tất cả các quyết định. Tuy nhiên, nếu nút đó gặp sự cố, thì hệ thống không còn có thể đưa ra bất kỳ quyết định nào. Đây thực ra chính là điều ta đã thấy trong trường hợp commit hai pha: nếu bộ điều phối gặp sự cố, các bên tham gia còn nghi vấn không thể quyết định nên commit hay hủy bỏ.

The termination property formalizes the idea of fault tolerance. It essentially says that a consensus algorithm cannot simply sit around and do nothing forever—in other words, it must make progress. Even if some nodes fail, the other nodes must still reach a decision. (Termination is a liveness property, whereas the other three are safety properties—see “Safety and liveness”).

Thuộc tính kết thúc (termination) hình thức hóa ý tưởng về khả năng chịu lỗi. Về cơ bản nó nói rằng một thuật toán đồng thuận không thể chỉ ngồi không và không làm gì mãi mãi — nói cách khác, nó phải tiến triển. Ngay cả khi một số nút gặp sự cố, các nút khác vẫn phải đi tới một quyết định. (Kết thúc là một thuộc tính sống động (liveness), trong khi ba thuộc tính kia là các thuộc tính an toàn — xem “Safety and liveness”).

The system model of consensus assumes that when a node “crashes,” it suddenly disappears and never comes back. (Instead of a software crash, imagine that there is an earthquake, and the datacenter containing your node is destroyed by a landslide. You must assume that your node is buried under 30 feet of mud and is never going to come back online.) In this system model, any algorithm that has to wait for a node to recover is not going to be able to satisfy the termination property. In particular, 2PC does not meet the requirements for termination.

Mô hình hệ thống của đồng thuận giả định rằng khi một nút “gặp sự cố”, nó đột ngột biến mất và không bao giờ quay lại. (Thay vì một sự cố phần mềm, hãy hình dung rằng có một trận động đất, và trung tâm dữ liệu chứa nút của bạn bị phá hủy bởi một vụ lở đất. Bạn

phải giả định rằng nút của bạn bị chôn vùi dưới 9 mét bùn và sẽ không bao giờ trở lại trực tuyến.) Trong mô hình hệ thống này, bất kỳ thuật toán nào phải chờ một nút khôi phục sẽ không thể thỏa mãn thuộc tính kết thúc. Cụ thể, 2PC không đáp ứng các yêu cầu về kết thúc.

Of course, if all nodes crash and none of them are running, then it is not possible for any algorithm to decide anything. There is a limit to the number of failures that an algorithm can tolerate: in fact, it can be proved that any consensus algorithm requires at least a majority of nodes to be functioning correctly in order to assure termination [67]. That majority can safely form a quorum (see “Quorums for reading and writing”).

Tất nhiên, nếu tất cả các nút đều gặp sự cố và không nút nào đang chạy, thì không thể có thuật toán nào quyết định được bất cứ điều gì. Có một giới hạn cho số lượng hỏng hóc mà một thuật toán có thể chịu được: thực ra, có thể chứng minh rằng bất kỳ thuật toán đồng thuận nào cũng đòi hỏi ít nhất một đa số các nút hoạt động đúng đắn để bảo đảm kết thúc [67]. Đa số đó có thể an toàn tạo thành một nhóm tức số (xem “Quorums for reading and writing”).

Thus, the termination property is subject to the assumption that fewer than half of the nodes are crashed or unreachable. However, most implementations of consensus ensure that the safety properties—agreement, integrity, and validity—are always met, even if a majority of nodes fail or there is a severe network problem [92]. Thus, a large-scale outage can stop the system from being able to process requests, but it cannot corrupt the consensus system by causing it to make invalid decisions.

Do đó, thuộc tính kết thúc tùy thuộc vào giả định rằng ít hơn một nửa số nút gặp sự cố hoặc không thể liên lạc được. Tuy nhiên, hầu hết các hiện thực đồng thuận bảo đảm rằng các thuộc tính an toàn — nhất trí, toàn vẹn, và hợp lệ — luôn được đáp ứng, ngay cả khi một đa số các nút gặp sự cố hoặc có một vấn đề mạng nghiêm trọng [92]. Do đó, một lần gián đoạn dịch vụ quy mô lớn có thể khiến hệ thống không thể xử lý các yêu cầu, nhưng nó không thể làm hỏng hệ thống đồng thuận bằng cách khiến nó đưa ra các quyết định không hợp lệ.

Most consensus algorithms assume that there are no Byzantine faults, as discussed in “Byzantine Faults”. That is, if a node does not correctly follow the protocol (for example, if it sends contradictory messages to different nodes), it may break the safety properties of the protocol. It is possible to make consensus robust against Byzantine faults as long as fewer than one-third of the nodes are Byzantine-faulty [25, 93], but we don’t have space to discuss those algorithms in detail in this book.

Hầu hết các thuật toán đồng thuận giả định rằng không có lỗi Byzantine, như đã bàn ở “Byzantine Faults”. Tức là, nếu một nút không tuân theo giao thức một cách đúng đắn (ví dụ, nếu nó gửi các thông điệp mâu thuẫn tới các nút khác), nó có thể phá vỡ các thuộc tính an toàn của giao thức. Có thể làm cho đồng thuận vững vàng trước các lỗi Byzantine miễn là ít hơn một phần ba số nút bị lỗi Byzantine [25, 93], nhưng ta không có chỗ để bàn về các thuật toán đó chi tiết trong cuốn sách này.

Consensus algorithms and total order broadcast — Các thuật toán đồng thuận và phát toàn-thứ-tự

The best-known fault-tolerant consensus algorithms are Viewstamped Replication (VSR) [94, 95], Paxos [96, 97, 98, 99], Raft [22, 100, 101], and Zab [15, 21, 102]. There are quite a few similarities between these algorithms, but they are not the same [103]. In this book we won’t go into full details of the different algorithms: it’s sufficient to be aware of some of the high-level ideas that they

have in common, unless you're implementing a consensus system yourself (which is probably not advisable—it's hard [98, 104]).

Các thuật toán đồng thuận chịu lỗi nổi tiếng nhất là Viewstamped Replication (VSR) [94, 95], Paxos [96, 97, 98, 99], Raft [22, 100, 101], và Zab [15, 21, 102]. Có khá nhiều nét tương đồng giữa các thuật toán này, nhưng chúng không giống nhau [103]. Trong cuốn sách này ta sẽ không đi vào chi tiết đầy đủ của các thuật toán khác nhau: chỉ cần biết một số ý tưởng ở mức cao mà chúng có chung là đủ, trừ khi bạn tự mình hiện thực một hệ thống đồng thuận (điều có lẽ không nên làm — nó khó [98, 104]).

Most of these algorithms actually don't directly use the formal model described here (proposing and deciding on a single value, while satisfying the agreement, integrity, validity, and termination properties). Instead, they decide on a sequence of values, which makes them total order broadcast algorithms, as discussed previously in this chapter (see "Total Order Broadcast").

Hầu hết các thuật toán này thực ra không trực tiếp dùng mô hình hình thức được mô tả ở đây (đề xuất và quyết định một giá trị duy nhất, trong khi thỏa mãn các thuộc tính nhất trí, toàn vẹn, hợp lệ, và kết thúc). Thay vào đó, chúng quyết định một chuỗi các giá trị, điều này làm cho chúng thành các thuật toán phát toàn-thứ-tự, như đã bàn trước đó trong chương này (xem "Total Order Broadcast").

Remember that total order broadcast requires messages to be delivered exactly once, in the same order, to all nodes. If you think about it, this is equivalent to performing several rounds of consensus: in each round, nodes propose the message that they want to send next, and then decide on the next message to be delivered in the total order [67].

Hãy nhớ rằng phát toàn-thứ-tự đòi hỏi các thông điệp được chuyển đúng một lần, theo cùng một thứ tự, tới tất cả các nút. Nếu bạn nghĩ về nó, điều này tương đương với việc thực hiện vài vòng đồng thuận: trong mỗi vòng, các nút đề xuất thông điệp mà chúng muốn gửi tiếp theo, rồi quyết định thông điệp tiếp theo cần được chuyển trong thứ tự toàn phần [67].

So, total order broadcast is equivalent to repeated rounds of consensus (each consensus decision corresponding to one message delivery):

Vậy, phát toàn-thứ-tự tương đương với các vòng đồng thuận lặp lại (mỗi quyết định đồng thuận tương ứng với một lần chuyển thông điệp):

- Due to the agreement property of consensus, all nodes decide to deliver the same messages in the same order.
- Due to the integrity property, messages are not duplicated.
- Due to the validity property, messages are not corrupted and not fabricated out of thin air.
- Due to the termination property, messages are not lost.
 - Do thuộc tính nhất trí của đồng thuận, tất cả các nút quyết định chuyển cùng các thông điệp theo cùng một thứ tự.
 - Do thuộc tính toàn vẹn, các thông điệp không bị nhân bản.
 - Do thuộc tính hợp lệ, các thông điệp không bị làm hỏng và không bị bịa ra từ hư không.
 - Do thuộc tính kết thúc, các thông điệp không bị mất.

Viewstamped Replication, Raft, and Zab implement total order broadcast directly, because that is more efficient than doing repeated rounds of one-value-at-a-time consensus. In the case of Paxos, this optimization is known as Multi-Paxos.

Viewstamped Replication, Raft, và Zab hiện thực phát toàn-thứ-tự một cách trực tiếp, bởi

làm vậy hiệu quả hơn so với việc thực hiện các vòng đồng thuận một-giá-trị-một-lần lặp lại. Trong trường hợp Paxos, tối ưu hóa này được biết đến là Multi-Paxos.

Single-leader replication and consensus — Nhân bản đơn-leader và đồng thuận

In Chapter 5 we discussed single-leader replication (see “Leaders and Followers”), which takes all the writes to the leader and applies them to the followers in the same order, thus keeping replicas up to date. Isn’t this essentially total order broadcast? How come we didn’t have to worry about consensus in Chapter 5?

Ở Chương 5 ta đã bàn về nhân bản đơn-leader (xem “Leaders and Followers”), vốn lấy tất cả các lần ghi tới leader và áp dụng chúng cho các follower theo cùng một thứ tự, do đó giữ các bản sao cập nhật. Chẳng phải về cơ bản đây là phát toàn-thứ-tự hay sao? Làm sao mà ta lại không phải lo về đồng thuận ở Chương 5?

The answer comes down to how the leader is chosen. If the leader is manually chosen and configured by the humans in your operations team, you essentially have a “consensus algorithm” of the dictatorial variety: only one node is allowed to accept writes (i.e., make decisions about the order of writes in the replication log), and if that node goes down, the system becomes unavailable for writes until the operators manually configure a different node to be the leader. Such a system can work well in practice, but it does not satisfy the termination property of consensus because it requires human intervention in order to make progress.

Câu trả lời quy về cách leader được chọn. Nếu leader được chọn và cấu hình một cách thủ công bởi những con người trong đội vận hành của bạn, thì về cơ bản bạn có một “thuật toán đồng thuận” thuộc loại độc tài: chỉ một nút được phép chấp nhận các lần ghi (tức đưa ra các quyết định về thứ tự của các lần ghi trong nhật ký nhân bản), và nếu nút đó gặp sự cố, hệ thống trở nên không sẵn sàng cho các lần ghi tới khi các nhà vận hành cấu hình thủ công một nút khác làm leader. Một hệ thống như vậy có thể hoạt động tốt trên thực tế, nhưng nó không thỏa mãn thuộc tính kết thúc của đồng thuận bởi nó đòi hỏi sự can thiệp của con người để tiến triển.

Some databases perform automatic leader election and failover, promoting a follower to be the new leader if the old leader fails (see “Handling Node Outages”). This brings us closer to fault-tolerant total order broadcast, and thus to solving consensus.

Một số cơ sở dữ liệu thực hiện bầu leader và chuyển đổi dự phòng tự động, thăng một follower lên làm leader mới nếu leader cũ gặp sự cố (xem “Handling Node Outages”). Điều này đưa ta tới gần hơn với phát toàn-thứ-tự chịu lỗi, và do đó tới việc giải đồng thuận.

However, there is a problem. We previously discussed the problem of split brain, and said that all nodes need to agree who the leader is—otherwise two different nodes could each believe themselves to be the leader, and consequently get the database into an inconsistent state. Thus, we need consensus in order to elect a leader. But if the consensus algorithms described here are actually total order broadcast algorithms, and total order broadcast is like single-leader replication, and single-leader replication requires a leader, then...

Tuy nhiên, có một vấn đề. Trước đó ta đã bàn về vấn đề split brain, và nói rằng tất cả các nút cần nhất trí về việc ai là leader — nếu không, hai nút khác nhau có thể mỗi nút tin rằng chính mình là leader, và do đó đưa cơ sở dữ liệu vào một trạng thái bất nhất. Do đó, ta cần đồng thuận để bầu một leader. Nhưng nếu các thuật toán đồng thuận được mô tả ở đây thực ra là các thuật toán phát toàn-thứ-tự, và phát toàn-thứ-tự thì giống nhân bản đơn-leader, và nhân bản đơn-leader đòi hỏi một leader, thì...

It seems that in order to elect a leader, we first need a leader. In order to solve consensus, we must first solve consensus. How do we break out of this conundrum?

Có vẻ như để bầu một leader, ta trước tiên cần một leader. Để giải đồng thuận, ta phải trước tiên giải đồng thuận. Làm sao ta thoát khỏi nan đề này?

Epoch numbering and quorums — Đánh số kỷ nguyên và các nhóm tức số

All of the consensus protocols discussed so far internally use a leader in some form or another, but they don't guarantee that the leader is unique. Instead, they can make a weaker guarantee: the protocols define an epoch number (called the ballot number in Paxos, view number in Viewstamped Replication, and term number in Raft) and guarantee that within each epoch, the leader is unique.

Tất cả các giao thức đồng thuận được bàn cho tới nay đều dùng một leader ở dạng này hay dạng khác ở bên trong, nhưng chúng không bảo đảm rằng leader là duy nhất. Thay vào đó, chúng có thể đưa ra một bảo đảm yếu hơn: các giao thức định nghĩa một số kỷ nguyên (epoch number) (gọi là số phiếu (ballot number) trong Paxos, số khung nhìn (view number) trong Viewstamped Replication, và số nhiệm kỳ (term number) trong Raft) và bảo đảm rằng trong mỗi kỷ nguyên, leader là duy nhất.

Every time the current leader is thought to be dead, a vote is started among the nodes to elect a new leader. This election is given an incremented epoch number, and thus epoch numbers are totally ordered and monotonically increasing. If there is a conflict between two different leaders in two different epochs (perhaps because the previous leader actually wasn't dead after all), then the leader with the higher epoch number prevails.

Mỗi khi leader hiện tại bị cho là đã chết, một cuộc bỏ phiếu được khởi động giữa các nút để bầu một leader mới. Cuộc bầu cử này được cho một số kỷ nguyên được tăng lên, và do đó các số kỷ nguyên được sắp thứ tự toàn phần và tăng đơn điệu. Nếu có một xung đột giữa hai leader khác nhau trong hai kỷ nguyên khác nhau (có lẽ vì leader trước đó hóa ra rốt cuộc không hề chết), thì leader với số kỷ nguyên cao hơn thắng thế.

Before a leader is allowed to decide anything, it must first check that there isn't some other leader with a higher epoch number which might take a conflicting decision. How does a leader know that it hasn't been ousted by another node? Recall “The Truth Is Defined by the Majority”: a node cannot necessarily trust its own judgment—just because a node thinks that it is the leader, that does not necessarily mean the other nodes accept it as their leader.

Trước khi một leader được phép quyết định bất cứ điều gì, nó phải trước tiên kiểm tra rằng không có một leader nào khác với số kỷ nguyên cao hơn vốn có thể đưa ra một quyết định xung đột. Làm sao một leader biết rằng nó chưa bị truất ngôi bởi một nút khác? Hãy nhớ lại “The Truth Is Defined by the Majority”: một nút không nhất thiết có thể tin vào phán đoán của chính nó — chỉ vì một nút nghĩ rằng nó là leader, điều đó không nhất thiết có nghĩa là các nút khác chấp nhận nó làm leader của chúng.

Instead, it must collect votes from a quorum of nodes (see “Quorums for reading and writing”). For every decision that a leader wants to make, it must send the proposed value to the other nodes and wait for a quorum of nodes to respond in favor of the proposal. The quorum typically, but not always, consists of a majority of nodes [105]. A node votes in favor of a proposal only if it is not aware of any other leader with a higher epoch.

Thay vào đó, nó phải thu thập các phiếu từ một nhóm tức số các nút (xem “Quorums for reading and writing”). Với mỗi quyết định mà một leader muốn đưa ra, nó phải gửi giá trị được đề xuất tới các nút khác và chờ một nhóm tức số các nút phản hồi ủng hộ đề xuất.

Nhóm tức số thường, nhưng không phải luôn luôn, bao gồm một đa số các nút [105]. Một nút chỉ bỏ phiếu ủng hộ một đề xuất nếu nó không biết về bất kỳ leader nào khác với một kỷ nguyên cao hơn.

Thus, we have two rounds of voting: once to choose a leader, and a second time to vote on a leader's proposal. The key insight is that the quorums for those two votes must overlap: if a vote on a proposal succeeds, at least one of the nodes that voted for it must have also participated in the most recent leader election [105]. Thus, if the vote on a proposal does not reveal any higher-numbered epoch, the current leader can conclude that no leader election with a higher epoch number has happened, and therefore be sure that it still holds the leadership. It can then safely decide the proposed value.

Do đó, ta có hai vòng bỏ phiếu: một lần để chọn một leader, và một lần thứ hai để bỏ phiếu cho đề xuất của một leader. Nhận định then chốt là các nhóm tức số cho hai cuộc bỏ phiếu đó phải chồng lấn: nếu một cuộc bỏ phiếu cho một đề xuất thành công, ít nhất một trong các nút đã bỏ phiếu cho nó cũng phải đã tham gia vào cuộc bầu leader gần nhất [105]. Do đó, nếu cuộc bỏ phiếu cho một đề xuất không hé lộ bất kỳ kỷ nguyên có số cao hơn nào, leader hiện tại có thể kết luận rằng không có cuộc bầu leader nào với một số kỷ nguyên cao hơn đã xảy ra, và do đó chắc chắn rằng nó vẫn nắm giữ vị trí lãnh đạo. Khi đó nó có thể an toàn quyết định giá trị được đề xuất.

This voting process looks superficially similar to two-phase commit. The biggest differences are that in 2PC the coordinator is not elected, and that fault-tolerant consensus algorithms only require votes from a majority of nodes, whereas 2PC requires a “yes” vote from every participant. Moreover, consensus algorithms define a recovery process by which nodes can get into a consistent state after a new leader is elected, ensuring that the safety properties are always met. These differences are key to the correctness and fault tolerance of a consensus algorithm.

Quá trình bỏ phiếu này thoạt nhìn giống commit hai pha. Những khác biệt lớn nhất là trong 2PC bộ điều phối không được bầu, và rằng các thuật toán đồng thuận chịu lỗi chỉ đòi hỏi các phiếu từ một đa số các nút, trong khi 2PC đòi hỏi một phiếu “có” từ mọi bên tham gia. Hơn nữa, các thuật toán đồng thuận định nghĩa một quá trình khôi phục mà qua đó các nút có thể đi tới một trạng thái nhất quán sau khi một leader mới được bầu, bảo đảm rằng các thuộc tính an toàn luôn được đáp ứng. Những khác biệt này là then chốt cho tính đúng đắn và khả năng chịu lỗi của một thuật toán đồng thuận.

Limitations of consensus — Các hạn chế của đồng thuận

Consensus algorithms are a huge breakthrough for distributed systems: they bring concrete safety properties (agreement, integrity, and validity) to systems where everything else is uncertain, and they nevertheless remain fault-tolerant (able to make progress as long as a majority of nodes are working and reachable). They provide total order broadcast, and therefore they can also implement linearizable atomic operations in a fault-tolerant way (see “Implementing linearizable storage using total order broadcast”).

Các thuật toán đồng thuận là một bước đột phá to lớn cho các hệ thống phân tán: chúng mang các thuộc tính an toàn cụ thể (nhất trí, toàn vẹn, và hợp lệ) tới các hệ thống nơi mọi thứ khác đều bất định, và dù vậy chúng vẫn chịu lỗi (có khả năng tiến triển miễn là một đa số các nút đang hoạt động và liên lạc được). Chúng cung cấp phát toàn-thứ-tự, và do đó chúng cũng có thể hiện thực các thao tác nguyên tử tuyến tính hóa một cách chịu lỗi (xem “Implementing linearizable storage using total order broadcast”).

Nevertheless, they are not used everywhere, because the benefits come at a cost.

Tuy nhiên, chúng không được dùng ở mọi nơi, bởi các lợi ích đi kèm một cái giá.

The process by which nodes vote on proposals before they are decided is a kind of synchronous replication. As discussed in “Synchronous Versus Asynchronous Replication”, databases are often configured to use asynchronous replication. In this configuration, some committed data can potentially be lost on failover—but many people choose to accept this risk for the sake of better performance.

Quá trình mà qua đó các nút bỏ phiếu cho các đề xuất trước khi chúng được quyết định là một loại nhân bản đồng bộ. Như đã bàn ở “Synchronous Versus Asynchronous Replication”, các cơ sở dữ liệu thường được cấu hình để dùng nhân bản bất đồng bộ. Trong cấu hình này, một số dữ liệu đã commit có thể bị mất khi chuyển đổi dự phòng — nhưng nhiều người chọn chấp nhận nguy cơ này để đổi lấy hiệu năng tốt hơn.

Consensus systems always require a strict majority to operate. This means you need a minimum of three nodes in order to tolerate one failure (the remaining two out of three form a majority), or a minimum of five nodes to tolerate two failures (the remaining three out of five form a majority). If a network failure cuts off some nodes from the rest, only the majority portion of the network can make progress, and the rest is blocked (see also “The Cost of Linearizability”).

Các hệ thống đồng thuận luôn đòi hỏi một đa số chặt chẽ để hoạt động. Điều này nghĩa là bạn cần tối thiểu ba nút để chịu được một lần hỏng (hai trong ba nút còn lại tạo thành một đa số), hoặc tối thiểu năm nút để chịu được hai lần hỏng (ba trong năm nút còn lại tạo thành một đa số). Nếu một lần hỏng mạng cắt một số nút khỏi phần còn lại, chỉ phần đa số của mạng có thể tiến triển, và phần còn lại bị chặn (xem thêm “The Cost of Linearizability”).

Most consensus algorithms assume a fixed set of nodes that participate in voting, which means that you can’t just add or remove nodes in the cluster. Dynamic membership extensions to consensus algorithms allow the set of nodes in the cluster to change over time, but they are much less well understood than static membership algorithms.

Hầu hết các thuật toán đồng thuận giả định một tập cố định các nút tham gia bỏ phiếu, điều này nghĩa là bạn không thể chỉ đơn giản thêm hoặc gỡ bỏ các nút trong cụm. Các mở rộng thành viên động (dynamic membership) cho các thuật toán đồng thuận cho phép tập các nút trong cụm thay đổi theo thời gian, nhưng chúng được hiểu rõ kém hơn nhiều so với các thuật toán thành viên tĩnh.

Consensus systems generally rely on timeouts to detect failed nodes. In environments with highly variable network delays, especially geographically distributed systems, it often happens that a node falsely believes the leader to have failed due to a transient network issue. Although this error does not harm the safety properties, frequent leader elections result in terrible performance because the system can end up spending more time choosing a leader than doing any useful work.

Các hệ thống đồng thuận nói chung dựa vào các timeout để phát hiện các nút đã hỏng. Trong các môi trường với các trì hoãn mạng biến thiên cao, đặc biệt là các hệ thống phân tán về mặt địa lý, thường xảy ra việc một nút sai lầm tin rằng leader đã hỏng do một vấn đề mạng thoáng qua. Mặc dù lỗi này không gây hại cho các thuộc tính an toàn, các cuộc bầu leader thường xuyên dẫn tới hiệu năng tệ hại bởi hệ thống có thể rất cuộc dành nhiều thời gian chọn một leader hơn là làm bất kỳ công việc hữu ích nào.

Sometimes, consensus algorithms are particularly sensitive to network problems. For example, Raft has been shown to have unpleasant edge cases [106]: if the entire network is working correctly except for one particular network link that is consistently unreliable, Raft can get into situations where leadership continually bounces between two nodes, or the current leader is continually forced to resign, so the system effectively never makes progress. Other consensus algorithms have similar

problems, and designing algorithms that are more robust to unreliable networks is still an open research problem.

Đôi khi, các thuật toán đồng thuận đặc biệt nhạy cảm với các vấn đề mạng. Ví dụ, Raft đã được cho thấy có các trường hợp biên khó chịu [106]: nếu toàn bộ mạng hoạt động đúng đắn ngoại trừ một đường truyền mạng cụ thể vốn liên tục không đáng tin cậy, Raft có thể rơi vào các tình huống mà vị trí lãnh đạo liên tục nhảy qua nhảy lại giữa hai nút, hoặc leader hiện tại liên tục bị buộc phải từ chức, nên hệ thống về cơ bản không bao giờ tiến triển. Các thuật toán đồng thuận khác có các vấn đề tương tự, và việc thiết kế các thuật toán vững vàng hơn trước các mạng không đáng tin cậy vẫn là một bài toán nghiên cứu mở.

Membership and Coordination Services — Các dịch vụ thành viên và điều phối

Projects like ZooKeeper or etcd are often described as “distributed key-value stores” or “coordination and configuration services.” The API of such a service looks pretty much like that of a database: you can read and write the value for a given key, and iterate over keys. So if they’re basically databases, why do they go to all the effort of implementing a consensus algorithm? What makes them different from any other kind of database?

Các dự án như ZooKeeper hoặc etcd thường được mô tả là “các kho khóa-giá trị phân tán” hoặc “các dịch vụ điều phối và cấu hình”. API của một dịch vụ như vậy trông khá giống của một cơ sở dữ liệu: bạn có thể đọc và ghi giá trị cho một khóa cho trước, và lặp qua các khóa. Vậy nếu chúng về cơ bản là các cơ sở dữ liệu, vì sao chúng lại tốn công đến vậy để hiện thực một thuật toán đồng thuận? Điều gì làm cho chúng khác với bất kỳ loại cơ sở dữ liệu nào khác?

To understand this, it is helpful to briefly explore how a service like ZooKeeper is used. As an application developer, you will rarely need to use ZooKeeper directly, because it is actually not well suited as a general-purpose database. It is more likely that you will end up relying on it indirectly via some other project: for example, HBase, Hadoop YARN, OpenStack Nova, and Kafka all rely on ZooKeeper running in the background. What is it that these projects get from it?

Để hiểu điều này, sẽ hữu ích nếu khám phá ngắn gọn cách một dịch vụ như ZooKeeper được dùng. Là một lập trình viên ứng dụng, bạn sẽ hiếm khi cần dùng ZooKeeper trực tiếp, bởi nó thực ra không phù hợp lắm với vai trò một cơ sở dữ liệu đa dụng. Nhiều khả năng bạn sẽ rất cuộc dựa vào nó một cách gián tiếp qua một dự án khác: ví dụ, HBase, Hadoop YARN, OpenStack Nova, và Kafka đều dựa vào ZooKeeper chạy ở chế độ nền. Những dự án này nhận được gì từ nó?

ZooKeeper and etcd are designed to hold small amounts of data that can fit entirely in memory (although they still write to disk for durability)—so you wouldn’t want to store all of your application’s data here. That small amount of data is replicated across all the nodes using a fault-tolerant total order broadcast algorithm. As discussed previously, total order broadcast is just what you need for database replication: if each message represents a write to the database, applying the same writes in the same order keeps replicas consistent with each other.

ZooKeeper và etcd được thiết kế để giữ những lượng dữ liệu nhỏ có thể vừa hoàn toàn trong bộ nhớ (mặc dù chúng vẫn ghi xuống đĩa để đảm bảo độ bền) — nên bạn sẽ không muốn lưu tất cả dữ liệu của ứng dụng ở đây. Lượng dữ liệu nhỏ đó được nhân bản trên tất cả các nút bằng một thuật toán phát toàn-thứ-tự chịu lỗi. Như đã bàn trước đó, phát

toàn-thứ-tự chính là thứ bạn cần cho nhân bản cơ sở dữ liệu: nếu mỗi thông điệp biểu diễn một lần ghi vào cơ sở dữ liệu, việc áp dụng cùng các lần ghi theo cùng một thứ tự giữ các bản sao nhất quán với nhau.

ZooKeeper is modeled after Google's Chubby lock service [14, 98], implementing not only total order broadcast (and hence consensus), but also an interesting set of other features that turn out to be particularly useful when building distributed systems:

ZooKeeper được mô phỏng theo dịch vụ khóa Chubby của Google [14, 98], hiện thực không chỉ phát toàn-thứ-tự (và do đó đồng thuận), mà còn một tập thú vị các tính năng khác hóa ra đặc biệt hữu ích khi xây dựng các hệ thống phân tán:

Using an atomic compare-and-set operation, you can implement a lock: if several nodes concurrently try to perform the same operation, only one of them will succeed. The consensus protocol guarantees that the operation will be atomic and linearizable, even if a node fails or the network is interrupted at any point. A distributed lock is usually implemented as a lease, which has an expiry time so that it is eventually released in case the client fails (see "Process Pauses").

Bằng cách dùng một thao tác so-sánh-rồi-đặt nguyên tử, bạn có thể hiện thực một khóa: nếu vài nút đồng thời cố thực hiện cùng một thao tác, chỉ một trong số chúng sẽ thành công. Giao thức đồng thuận bảo đảm rằng thao tác sẽ là nguyên tử và tuyến tính hóa, ngay cả khi một nút gặp sự cố hoặc mạng bị gián đoạn tại bất kỳ điểm nào. Một khóa phân tán thường được hiện thực dưới dạng một hợp đồng thuê (lease), vốn có một thời điểm hết hạn để nó rốt cuộc được giải phóng trong trường hợp client gặp sự cố (xem "Process Pauses").

As discussed in "The leader and the lock", when some resource is protected by a lock or lease, you need a fencing token to prevent clients from conflicting with each other in the case of a process pause. The fencing token is some number that monotonically increases every time the lock is acquired. ZooKeeper provides this by totally ordering all operations and giving each operation a monotonically increasing transaction ID (zxid) and version number (cversion) [15].

Như đã bàn ở "The leader and the lock", khi một tài nguyên nào đó được bảo vệ bởi một khóa hoặc hợp đồng thuê, bạn cần một thẻ rào chắn để ngăn các client xung đột với nhau trong trường hợp một lần tạm dừng tiến trình. Thẻ rào chắn là một con số nào đó tăng đơn điệu mỗi khi khóa được giành. ZooKeeper cung cấp điều này bằng cách sắp thứ tự toàn phần tất cả các thao tác và cho mỗi thao tác một ID giao dịch (zxid) và số phiên bản (cversion) tăng đơn điệu [15].

Clients maintain a long-lived session on ZooKeeper servers, and the client and server periodically exchange heartbeats to check that the other node is still alive. Even if the connection is temporarily interrupted, or a ZooKeeper node fails, the session remains active. However, if the heartbeats cease for a duration that is longer than the session timeout, ZooKeeper declares the session to be dead. Any locks held by a session can be configured to be automatically released when the session times out (ZooKeeper calls these ephemeral nodes).

Các client duy trì một phiên (session) tồn tại lâu dài trên các máy chủ ZooKeeper, và client và máy chủ định kỳ trao đổi các nhịp tim (heartbeat) để kiểm tra rằng nút kia vẫn còn sống. Ngay cả khi kết nối bị gián đoạn tạm thời, hoặc một nút ZooKeeper gặp sự cố, phiên vẫn duy trì hoạt động. Tuy nhiên, nếu các nhịp tim ngừng trong một khoảng thời gian dài hơn thời gian chờ của phiên, ZooKeeper tuyên bố phiên đã chết. Bất kỳ khóa nào được một phiên giữ có thể được cấu hình để tự động được giải phóng khi phiên hết thời gian chờ (ZooKeeper gọi những thứ này là các nút phù du (ephemeral nodes)).

Not only can one client read locks and values that were created by another client, but it can also watch them for changes. Thus, a client can find out when another client joins the cluster (based

on the value it writes to ZooKeeper), or if another client fails (because its session times out and its ephemeral nodes disappear). By subscribing to notifications, a client avoids having to frequently poll to find out about changes.

Không chỉ một client có thể đọc các khóa và giá trị được tạo bởi một client khác, mà nó còn có thể theo dõi (watch) chúng để nhận biết các thay đổi. Do đó, một client có thể tìm ra khi nào một client khác gia nhập cụm (dựa trên giá trị mà nó ghi vào ZooKeeper), hoặc nếu một client khác gặp sự cố (vì phiên của nó hết thời gian chờ và các nút phù du của nó biến mất). Bằng cách đăng ký nhận các thông báo, một client tránh phải thường xuyên thăm dò để tìm hiểu về các thay đổi.

Of these features, only the linearizable atomic operations really require consensus. However, it is the combination of these features that makes systems like ZooKeeper so useful for distributed coordination.

Trong số các tính năng này, chỉ có các thao tác nguyên tử tuyến tính hóa là thực sự đòi hỏi đồng thuận. Tuy nhiên, chính sự kết hợp của những tính năng này làm cho các hệ thống như ZooKeeper hữu ích đến vậy cho việc điều phối phân tán.

Allocating work to nodes — Phân bổ công việc cho các nút

One example in which the ZooKeeper/Chubby model works well is if you have several instances of a process or service, and one of them needs to be chosen as leader or primary. If the leader fails, one of the other nodes should take over. This is of course useful for single-leader databases, but it's also useful for job schedulers and similar **stateful** systems.

Một ví dụ trong đó mô hình ZooKeeper/Chubby hoạt động tốt là nếu bạn có vài phiên bản của một tiến trình hoặc dịch vụ, và một trong số chúng cần được chọn làm leader hoặc primary. Nếu leader gặp sự cố, một trong các nút khác nên tiếp quản. Điều này tất nhiên hữu ích cho các cơ sở dữ liệu đơn-leader, nhưng nó cũng hữu ích cho các trình lập lịch công việc và các hệ thống có trạng thái tương tự.

Another example arises when you have some partitioned resource (database, message streams, file storage, distributed actor system, etc.) and need to decide which partition to assign to which node. As new nodes **join** the cluster, some of the partitions need to be moved from existing nodes to the new nodes in order to rebalance the **load** (see “Rebalancing Partitions”). As nodes are removed or fail, other nodes need to take over the failed nodes' work.

Một ví dụ khác nảy sinh khi bạn có một tài nguyên được phân vùng nào đó (cơ sở dữ liệu, các luồng thông điệp, lưu trữ tệp, hệ thống actor phân tán, v.v.) và cần quyết định gán phân vùng nào cho nút nào. Khi các nút mới gia nhập cụm, một số phân vùng cần được chuyển từ các nút hiện có sang các nút mới để cân bằng lại tải (xem “Rebalancing Partitions”). Khi các nút bị gỡ bỏ hoặc gặp sự cố, các nút khác cần tiếp quản công việc của các nút đã hỏng.

These kinds of tasks can be achieved by judicious use of atomic operations, ephemeral nodes, and notifications in ZooKeeper. If done correctly, this approach allows the application to automatically recover from faults without human intervention. It's not easy, despite the appearance of libraries such as Apache Curator [17] that have sprung up to provide higher-level tools on top of the ZooKeeper client API—but it is still much better than attempting to implement the necessary consensus algorithms from scratch, which has a poor success record [107].

Những loại tác vụ này có thể được hoàn thành bằng cách dùng khéo léo các thao tác nguyên tử, các nút phù du, và các thông báo trong ZooKeeper. Nếu làm đúng, cách tiếp cận này cho phép ứng dụng tự động khôi phục từ các lỗi mà không cần sự can thiệp của con người.

Điều đó không dễ, bất chấp sự xuất hiện của các thư viện như Apache Curator [17] vốn đã mọc lên để cung cấp các công cụ ở mức cao hơn trên nền API client của ZooKeeper — nhưng nó vẫn tốt hơn nhiều so với việc cố hiện thực các thuật toán đồng thuận cần thiết từ đầu, vốn có một thành tích thành công nghèo nàn [107].

An application may initially run only on a single node, but eventually may grow to thousands of nodes. Trying to perform majority votes over so many nodes would be terribly inefficient. Instead, ZooKeeper runs on a fixed number of nodes (usually three or five) and performs its majority votes among those nodes while supporting a potentially large number of clients. Thus, ZooKeeper provides a way of “outsourcing” some of the work of coordinating nodes (consensus, operation ordering, and failure detection) to an external service.

Một ứng dụng ban đầu có thể chỉ chạy trên một nút duy nhất, nhưng rất cuộc có thể phát triển tới hàng nghìn nút. Việc cố thực hiện các cuộc bỏ phiếu đa số trên ngàn ấy nút sẽ kém hiệu quả khủng khiếp. Thay vào đó, ZooKeeper chạy trên một số nút cố định (thường là ba hoặc năm) và thực hiện các cuộc bỏ phiếu đa số của nó giữa những nút đó trong khi hỗ trợ một số lượng client có khả năng rất lớn. Do đó, ZooKeeper cung cấp một cách để “thuê ngoài” một phần công việc điều phối các nút (đồng thuận, sắp thứ tự thao tác, và phát hiện sự cố) cho một dịch vụ bên ngoài.

Normally, the kind of data managed by ZooKeeper is quite slow-changing: it represents information like “the node running on 10.1.1.23 is the leader for partition 7,” which may change on a timescale of minutes or hours. It is not intended for storing the runtime state of the application, which may change thousands or even millions of times per second. If application state needs to be replicated from one node to another, other tools (such as Apache BookKeeper [108]) can be used.

Thông thường, loại dữ liệu được ZooKeeper quản lý thay đổi khá chậm: nó biểu diễn thông tin như “nút chạy trên 10.1.1.23 là leader cho phân vùng 7”, vốn có thể thay đổi trên một thang thời gian là phút hoặc giờ. Nó không nhằm để lưu trạng thái lúc chạy của ứng dụng, vốn có thể thay đổi hàng nghìn hoặc thậm chí hàng triệu lần mỗi giây. Nếu trạng thái ứng dụng cần được nhân bản từ một nút sang một nút khác, các công cụ khác (chẳng hạn Apache BookKeeper [108]) có thể được dùng.

Service discovery — Khám phá dịch vụ

ZooKeeper, etcd, and Consul are also often used for service discovery—that is, to find out which IP address you need to connect to in order to reach a particular service. In cloud datacenter environments, where it is common for virtual machines to continually come and go, you often don’t know the IP addresses of your services ahead of time. Instead, you can configure your services such that when they start up they register their network endpoints in a service registry, where they can then be found by other services.

ZooKeeper, etcd, và Consul cũng thường được dùng cho khám phá dịch vụ (service discovery) — tức để tìm ra bạn cần kết nối tới địa chỉ IP nào để tiếp cận một dịch vụ cụ thể. Trong các môi trường trung tâm dữ liệu đám mây, nơi việc các máy ảo liên tục đến và đi là phổ biến, bạn thường không biết trước các địa chỉ IP của các dịch vụ của mình. Thay vào đó, bạn có thể cấu hình các dịch vụ của mình sao cho khi chúng khởi động chúng đăng ký các điểm cuối mạng của chúng trong một sổ đăng ký dịch vụ (service registry), nơi sau đó chúng có thể được các dịch vụ khác tìm thấy.

However, it is less clear whether service discovery actually requires consensus. DNS is the traditional way of looking up the IP address for a service name, and it uses multiple layers of caching to achieve good performance and availability. Reads from DNS are absolutely not linearizable, and it is usually

not considered problematic if the results from a DNS query are a little stale [109]. It is more important that DNS is reliably available and robust to network interruptions.

Tuy nhiên, kém rõ ràng hơn là liệu khám phá dịch vụ có thực sự đòi hỏi đồng thuận hay không. DNS là cách truyền thống để tra cứu địa chỉ IP cho một tên dịch vụ, và nó dùng nhiều tầng đệm để đạt hiệu năng và tính sẵn sàng tốt. Các lần đọc từ DNS hoàn toàn không tuyến tính hóa, và thường không bị coi là có vấn đề nếu các kết quả từ một truy vấn DNS hơi cũ một chút [109]. Quan trọng hơn là DNS sẵn sàng một cách đáng tin cậy và vững vàng trước các gián đoạn mạng.

Although service discovery does not require consensus, leader election does. Thus, if your consensus system already knows who the leader is, then it can make sense to also use that information to help other services discover who the leader is. For this purpose, some consensus systems support read-only caching replicas. These replicas asynchronously receive the log of all decisions of the consensus algorithm, but do not actively participate in voting. They are therefore able to serve read requests that do not need to be linearizable.

Mặc dù khám phá dịch vụ không đòi hỏi đồng thuận, bầu leader thì có. Do đó, nếu hệ thống đồng thuận của bạn đã biết ai là leader, thì có thể hợp lý nếu cũng dùng thông tin đó để giúp các dịch vụ khác khám phá ai là leader. Vì mục đích này, một số hệ thống đồng thuận hỗ trợ các bản sao đệm chỉ-đọc. Các bản sao này nhận một cách bất đồng bộ nhật ký của tất cả các quyết định của thuật toán đồng thuận, nhưng không chủ động tham gia vào việc bỏ phiếu. Do đó chúng có thể phục vụ các yêu cầu đọc vốn không cần tuyến tính hóa.

Membership services — Các dịch vụ thành viên

ZooKeeper and friends can be seen as part of a long history of research into membership services, which goes back to the 1980s and has been important for building highly reliable systems, e.g., for air traffic control [110].

ZooKeeper và những công cụ tương tự có thể được xem là một phần của một lịch sử nghiên cứu lâu dài về các dịch vụ thành viên (membership services), vốn quay ngược về thập niên 1980 và đã quan trọng cho việc xây dựng các hệ thống có độ tin cậy cao, chẳng hạn cho kiểm soát không lưu [110].

A membership service determines which nodes are currently active and live members of a cluster. As we saw throughout Chapter 8, due to unbounded network delays it's not possible to reliably detect whether another node has failed. However, if you couple failure detection with consensus, nodes can come to an agreement about which nodes should be considered alive or not.

Một dịch vụ thành viên xác định những nút nào hiện đang là thành viên hoạt động và còn sống của một cụm. Như ta đã thấy xuyên suốt Chương 8, do các trì hoãn mạng không có chặn trên, không thể phát hiện một cách đáng tin cậy liệu một nút khác đã hỏng hay chưa. Tuy nhiên, nếu bạn kết hợp phát hiện sự cố với đồng thuận, các nút có thể đi tới một thỏa thuận về việc những nút nào nên được coi là còn sống hay không.

It could still happen that a node is incorrectly declared dead by consensus, even though it is actually alive. But it is nevertheless very useful for a system to have agreement on which nodes constitute the current membership. For example, choosing a leader could mean simply choosing the lowest-numbered among the current members, but this approach would not work if different nodes have divergent opinions on who the current members are.

Vẫn có thể xảy ra việc một nút bị đồng thuận tuyên bố sai là đã chết, ngay cả khi nó thực ra còn sống. Nhưng dù vậy, việc một hệ thống có sự nhất trí về việc những nút nào tạo

nên thành viên hiện tại vẫn rất hữu ích. Ví dụ, việc chọn một leader có thể đơn giản nghĩa là chọn nút có số thấp nhất trong số các thành viên hiện tại, nhưng cách tiếp cận này sẽ không hoạt động nếu các nút khác nhau có những quan điểm phân kỳ về việc ai là các thành viên hiện tại.

Summary — Tóm tắt

In this chapter we examined the topics of consistency and consensus from several different angles. We looked in depth at linearizability, a popular consistency model: its goal is to make replicated data appear as though there were only a single copy, and to make all operations act on it atomically. Although linearizability is appealing because it is easy to understand—it makes a database behave like a variable in a single-threaded program—it has the downside of being slow, especially in environments with large network delays.

Trong chương này ta đã xem xét các chủ đề tính nhất quán và đồng thuận từ nhiều góc độ khác nhau. Ta đã đi sâu vào tính tuyến tính hóa, một mô hình nhất quán phổ biến: mục tiêu của nó là làm cho dữ liệu được nhân bản trông như thể chỉ có một bản sao duy nhất, và làm cho tất cả các thao tác tác động lên nó một cách nguyên tử. Mặc dù tính tuyến tính hóa hấp dẫn vì nó dễ hiểu — nó làm cho một cơ sở dữ liệu hành xử như một biến trong một chương trình đơn luồng — nó có nhược điểm là chậm, đặc biệt trong các môi trường với các trì hoãn mạng lớn.

We also explored causality, which imposes an ordering on events in a system (what happened before what, based on cause and effect). Unlike linearizability, which puts all operations in a single, totally ordered timeline, causality provides us with a weaker consistency model: some things can be concurrent, so the version history is like a timeline with branching and merging. Causal consistency does not have the coordination overhead of linearizability and is much less sensitive to network problems.

Ta cũng đã khám phá tính nhân quả, vốn áp đặt một thứ tự lên các sự kiện trong một hệ thống (cái gì xảy ra trước cái gì, dựa trên nhân và quả). Khác với tính tuyến tính hóa, vốn đặt tất cả các thao tác trong một dòng thời gian duy nhất, được sắp thứ tự toàn phần, tính nhân quả cung cấp cho ta một mô hình nhất quán yếu hơn: một số thứ có thể đồng thời, nên lịch sử phiên bản giống một dòng thời gian có sự rẽ nhánh và nhập lại. Tính nhất quán nhân quả không có chi phí điều phối của tính tuyến tính hóa và ít nhạy cảm hơn nhiều với các vấn đề mạng.

However, even if we capture the causal ordering (for example using Lamport timestamps), we saw that some things cannot be implemented this way: in “Timestamp ordering is not sufficient” we considered the example of ensuring that a username is unique and rejecting concurrent registrations for the same username. If one node is going to accept a registration, it needs to somehow know that another node isn’t concurrently in the process of registering the same name. This problem led us toward consensus.

Tuy nhiên, ngay cả khi ta nắm bắt thứ tự nhân quả (chẳng hạn dùng các dấu thời gian Lamport), ta đã thấy rằng một số thứ không thể được hiện thực theo cách này: trong “Timestamp ordering is not sufficient” ta đã xét ví dụ về việc bảo đảm rằng một tên người dùng là duy nhất và từ chối các lần đăng ký đồng thời cho cùng một tên người dùng. Nếu

một nút sắp chấp nhận một lần đăng ký, nó cần bằng cách nào đó biết rằng một nút khác không đang đồng thời trong quá trình đăng ký cùng một cái tên. Vấn đề này dẫn ta tới đồng thuận.

We saw that achieving consensus means deciding something in such a way that all nodes agree on what was decided, and such that the decision is irrevocable. With some digging, it turns out that a wide range of problems are actually reducible to consensus and are equivalent to each other (in the sense that if you have a solution for one of them, you can easily transform it into a solution for one of the others). Such equivalent problems include:

Ta đã thấy rằng đạt được đồng thuận nghĩa là quyết định một điều gì đó theo cách mà tất cả các nút nhất trí về điều được quyết định, và sao cho quyết định đó là không thể thu hồi. Sau một hồi đào sâu, hóa ra một phạm vi rộng các bài toán thực ra có thể quy về đồng thuận và tương đương với nhau (theo nghĩa rằng nếu bạn có một giải pháp cho một trong số chúng, bạn có thể dễ dàng biến đổi nó thành một giải pháp cho một bài toán khác). Những bài toán tương đương như vậy gồm:

The register needs to atomically decide whether to set its value, based on whether its current value equals the parameter given in the operation.

Thanh ghi cần quyết định một cách nguyên tử xem có nên đặt giá trị của nó hay không, dựa trên việc giá trị hiện tại của nó có bằng tham số được cho trong thao tác hay không.

A database must decide whether to commit or abort a distributed transaction.

Một cơ sở dữ liệu phải quyết định nên commit hay hủy bỏ một giao dịch phân tán.

The messaging system must decide on the order in which to deliver messages.

Hệ thống nhắn tin phải quyết định thứ tự để chuyển các thông điệp.

When several clients are racing to grab a lock or lease, the lock decides which one successfully acquired it.

Khi vài client đua nhau giành một khóa hoặc hợp đồng thuê, khóa quyết định cái nào đã giành được nó thành công.

Given a failure detector (e.g., timeouts), the system must decide which nodes are alive, and which should be considered dead because their sessions timed out.

Cho trước một bộ phát hiện sự cố (chẳng hạn các timeout), hệ thống phải quyết định những nút nào còn sống, và những nút nào nên được coi là đã chết vì các phiên của chúng đã hết thời gian chờ.

When several transactions concurrently try to create conflicting records with the same key, the constraint must decide which one to allow and which should fail with a constraint violation.

Khi vài giao dịch đồng thời cố tạo các bản ghi xung đột với cùng một khóa, ràng buộc phải quyết định cái nào được cho phép và cái nào nên thất bại với một vi phạm ràng buộc.

All of these are straightforward if you only have a single node, or if you are willing to assign the decision-making capability to a single node. This is what happens in a single-leader database: all the power to make decisions is vested in the leader, which is why such databases are able to provide linearizable operations, uniqueness constraints, a totally ordered replication log, and more.

Tất cả những điều này đều đơn giản nếu bạn chỉ có một nút duy nhất, hoặc nếu bạn sẵn lòng gán khả năng ra quyết định cho một nút duy nhất. Đây là điều xảy ra trong một cơ sở dữ liệu đơn-leader: tất cả quyền ra quyết định được trao cho leader, đó là lý do vì sao

những cơ sở dữ liệu như vậy có thể cung cấp các thao tác tuyến tính hóa, các ràng buộc duy nhất, một nhật ký nhân bản được sắp thứ tự toàn phần, và hơn nữa.

However, if that single leader fails, or if a network interruption makes the leader unreachable, such a system becomes unable to make any progress. There are three ways of handling that situation:

Tuy nhiên, nếu leader duy nhất đó gặp sự cố, hoặc nếu một gián đoạn mạng khiến leader không thể liên lạc được, một hệ thống như vậy trở nên không thể tiến triển. Có ba cách để xử lý tình huống đó:

- Wait for the leader to recover, and accept that the system will be blocked in the meantime. Many XA/JTA transaction coordinators choose this option. This approach does not fully solve consensus because it does not satisfy the termination property: if the leader does not recover, the system can be blocked forever.
- Manually fail over by getting humans to choose a new leader node and reconfigure the system to use it. Many relational databases take this approach. It is a kind of consensus by “act of God”—the human operator, outside of the computer system, makes the decision. The speed of failover is limited by the speed at which humans can act, which is generally slower than computers.
- Use an algorithm to automatically choose a new leader. This approach requires a consensus algorithm, and it is advisable to use a proven algorithm that correctly handles adverse network conditions [107].
 - Chờ leader khôi phục, và chấp nhận rằng hệ thống sẽ bị chặn trong khoảng thời gian đó. Nhiều bộ điều phối giao dịch XA/JTA chọn phương án này. Cách tiếp cận này không giải quyết đồng thuận một cách trọn vẹn bởi nó không thỏa mãn thuộc tính kết thúc: nếu leader không khôi phục, hệ thống có thể bị chặn mãi mãi.
 - Chuyển đổi dự phòng thủ công bằng cách để con người chọn một nút leader mới và cấu hình lại hệ thống để dùng nó. Nhiều cơ sở dữ liệu quan hệ dùng cách tiếp cận này. Nó là một loại đồng thuận theo kiểu “thiên định” — nhà vận hành là con người, bên ngoài hệ thống máy tính, đưa ra quyết định. Tốc độ của chuyển đổi dự phòng bị giới hạn bởi tốc độ mà con người có thể hành động, vốn nói chung chậm hơn máy tính.
 - Dùng một thuật toán để tự động chọn một leader mới. Cách tiếp cận này đòi hỏi một thuật toán đồng thuận, và nên dùng một thuật toán đã được kiểm chứng vốn xử lý đúng đắn các điều kiện mạng bất lợi [107].

Although a single-leader database can provide linearizability without executing a consensus algorithm on every write, it still requires consensus to maintain its leadership and for leadership changes. Thus, in some sense, having a leader only “kicks the can down the road”: consensus is still required, only in a different place, and less frequently. The good news is that fault-tolerant algorithms and systems for consensus exist, and we briefly discussed them in this chapter.

Mặc dù một cơ sở dữ liệu đơn-leader có thể cung cấp tính tuyến tính hóa mà không cần thực thi một thuật toán đồng thuận trên mỗi lần ghi, nó vẫn đòi hỏi đồng thuận để duy trì vị trí lãnh đạo của nó và cho các thay đổi lãnh đạo. Do đó, theo một nghĩa nào đó, việc có một leader chỉ “đá quả bóng đi xa hơn”: đồng thuận vẫn cần thiết, chỉ là ở một chỗ khác, và ít thường xuyên hơn. Tin tốt là các thuật toán và hệ thống chịu lỗi cho đồng thuận tồn tại, và ta đã bàn về chúng ngắn gọn trong chương này.

Tools like ZooKeeper play an important role in providing an “outsourced” consensus, failure detection, and membership service that applications can use. It’s not easy to use, but it is much better than trying to develop your own algorithms that can withstand all the problems discussed in Chapter 8. If you find yourself wanting to do one of those things that is reducible to consensus, and you want it to be fault-tolerant, then it is advisable to use something like ZooKeeper.

Các công cụ như ZooKeeper đóng một vai trò quan trọng trong việc cung cấp một dịch vụ đồng thuận, phát hiện sự cố, và thành viên được “thuê ngoài” mà các ứng dụng có thể dùng. Nó không dễ dùng, nhưng nó tốt hơn nhiều so với việc cố phát triển các thuật toán của riêng bạn vốn có thể chịu đựng tất cả các vấn đề đã bàn ở Chương 8. Nếu bạn thấy mình muốn làm một trong những điều có thể quy về đồng thuận, và bạn muốn nó chịu lỗi, thì nên dùng một thứ gì đó như ZooKeeper.

Nevertheless, not every system necessarily requires consensus: for example, leaderless and multi-leader replication systems typically do not use global consensus. The conflicts that occur in these systems (see “Handling Write Conflicts”) are a consequence of not having consensus across different leaders, but maybe that’s okay: maybe we simply need to cope without linearizability and learn to work better with data that has branching and merging version histories.

Tuy nhiên, không phải mọi hệ thống đều nhất thiết đòi hỏi đồng thuận: ví dụ, các hệ thống nhân bản không-leader và đa-leader thường không dùng đồng thuận toàn cục. Các xung đột xảy ra trong những hệ thống này (xem “Handling Write Conflicts”) là một hệ quả của việc không có đồng thuận xuyên các leader khác nhau, nhưng có lẽ điều đó không sao: có lẽ ta đơn giản cần xoay sở mà không có tính tuyến tính hóa và học cách làm việc tốt hơn với dữ liệu có lịch sử phiên bản rẽ nhánh và nhập lại.

This chapter referenced a large body of research on the theory of distributed systems. Although the theoretical papers and proofs are not always easy to understand, and sometimes make unrealistic assumptions, they are incredibly valuable for informing practical work in this field: they help us reason about what can and cannot be done, and help us find the counterintuitive ways in which distributed systems are often flawed. If you have the time, the references are well worth exploring.

Chương này đã tham chiếu một khối lượng lớn nghiên cứu về lý thuyết hệ thống phân tán. Mặc dù các bài báo và chứng minh lý thuyết không phải lúc nào cũng dễ hiểu, và đôi khi đưa ra các giả định phi thực tế, chúng vô cùng có giá trị trong việc soi sáng công việc thực tiễn trong lĩnh vực này: chúng giúp ta lập luận về những gì có thể và không thể làm, và giúp ta tìm ra những cách phản trực giác mà các hệ thống phân tán thường mắc lỗi. Nếu bạn có thời gian, các tài liệu tham khảo rất đáng để khám phá.

This brings us to the end of Part II of this book, in which we covered replication (Chapter 5), partitioning (Chapter 6), transactions (Chapter 7), distributed system failure models (Chapter 8), and finally consistency and consensus (Chapter 9). Now that we have laid a firm foundation of theory, in Part III we will turn once again to more practical systems, and discuss how to build powerful applications from heterogeneous building blocks.

Điều này đưa ta tới cuối Phần II của cuốn sách, trong đó ta đã bao quát nhân bản (Chương 5), phân vùng (Chương 6), giao dịch (Chương 7), các mô hình hỏng hóc hệ thống phân tán (Chương 8), và cuối cùng là tính nhất quán và đồng thuận (Chương 9). Giờ ta đã đặt một nền tảng lý thuyết vững chắc, ở Phần III ta sẽ lại một lần nữa quay sang các hệ thống thực tiễn hơn, và bàn về cách xây dựng các ứng dụng mạnh mẽ từ các khối dựng không đồng nhất.

Footnotes A subtle detail of this diagram is that it assumes the existence of a global clock, represented by the horizontal axis. Even though real systems typically don’t have accurate clocks (see “Unreliable Clocks”), this assumption is okay: for the purposes of analyzing a distributed algorithm, we may pretend that an accurate global clock exists, as long as the algorithm doesn’t have access to it [47]. Instead, the algorithm can only see a mangled approximation of real time, as produced by a quartz oscillator and NTP.

A register in which reads may return either the old or the new value if they are concurrent with a write is known as a regular register [7, 25].

Strictly speaking, ZooKeeper and etcd provide linearizable writes, but reads may be stale, since by default they can be served by any one of the replicas. You can optionally request a linearizable read: etcd calls this a quorum read [16], and in ZooKeeper you need to call `sync()` before the read [15]; see “Implementing linearizable storage using total order broadcast”.

Partitioning (sharding) a single-leader database, so that there is a separate leader per partition, does not affect linearizability, since it is only a single-object guarantee. Cross-partition transactions are a different matter (see “Distributed Transactions and Consensus”).

These two choices are sometimes known as CP (consistent but not available under network partitions) and AP (available but not consistent under network partitions), respectively. However, this classification scheme has several flaws [9], so it is best avoided.

As discussed in “Network Faults in Practice”, this book uses partitioning to refer to deliberately breaking down a large dataset into smaller ones (sharding; see Chapter 6). By contrast, a network partition is a particular type of network fault, which we normally don’t consider separately from other kinds of faults. However, since it’s the P in CAP, we can’t avoid the confusion in this case.

A total order that is inconsistent with causality is easy to create, but not very useful. For example, you can generate a random UUID for each operation, and compare UUIDs lexicographically to define the total ordering of operations. This is a valid total order, but the random UUIDs tell you nothing about which operation actually happened first, or whether the operations were concurrent.

It is possible to make physical clock timestamps consistent with causality: in “Synchronized clocks for global snapshots” we discussed Google’s Spanner, which estimates the expected clock skew and waits out the uncertainty interval before committing a write. This method ensures that a causally later transaction is given a greater timestamp. However, most clocks cannot provide the required uncertainty metric.

The term atomic broadcast is traditional, but it is very confusing as it’s inconsistent with other uses of the word atomic: it has nothing to do with atomicity in ACID transactions and is only indirectly related to atomic operations (in the sense of multi-threaded programming) or atomic registers (linearizable storage). The term total order multicast is another synonym.

In a formal sense, a linearizable read-write register is an “easier” problem. Total order broadcast is equivalent to consensus [67], which has no deterministic solution in the asynchronous crash-stop model [68], whereas a linearizable read-write register can be implemented in the same system model [23, 24, 25]. However, supporting atomic operations such as compare-and-set or increment-and-get in a register makes it equivalent to consensus [28]. Thus, the problems of consensus and a linearizable register are closely related.

If you don’t wait, but acknowledge the write immediately after it has been enqueued, you get something similar to the memory consistency model of multi-core x86 processors [43]. That model is neither linearizable nor sequentially consistent.

Atomic commit is formalized slightly differently from consensus: an atomic transaction can commit only if all participants vote to commit, and must abort if any participant needs to abort. Consensus is allowed to decide on any value that is proposed by one of the participants. However, atomic commit and consensus are reducible to each other [70, 71]. Nonblocking atomic commit is harder than consensus—see “Three-phase commit”.

This particular variant of consensus is called uniform consensus, which is equivalent to regular consensus in asynchronous systems with unreliable failure detectors [71]. The academic literature

usually refers to processes rather than nodes, but we use nodes here for consistency with the rest of this book.

- References**
- [1] Peter Bailis and Ali Ghodsi: “Eventual Consistency Today: Limitations, Extensions, and Beyond,” *ACM Queue*, volume 11, number 3, pages 55-63, March 2013. doi:10.1145/2460276.2462076
 - [2] Prince Mahajan, Lorenzo Alvisi, and Mike Dahlin: “Consistency, Availability, and Convergence,” University of Texas at Austin, Department of Computer Science, Tech Report UTCS TR-11-22, May 2011.
 - [3] Alex Scotti: “Adventures in Building Your Own Database,” at All Your Base, November 2015.
 - [4] Peter Bailis, Aaron Davidson, Alan Fekete, et al.: “Highly Available Transactions: Virtues and Limitations,” at 40th International Conference on Very Large Data Bases (VLDB), September 2014. Extended version published as pre-print arXiv:1302.0309 [cs.DB].
 - [5] Paolo Viotti and Marko Vukolić: “Consistency in Non-Transactional Distributed Storage Systems,” arXiv:1512.00168, 12 April 2016.
 - [6] Maurice P. Herlihy and Jeannette M. Wing: “Linearizability: A Correctness Condition for Concurrent Objects,” *ACM Transactions on Programming Languages and Systems (TOPLAS)*, volume 12, number 3, pages 463–492, July 1990. doi:10.1145/78969.78972
 - [7] Leslie Lamport: “On interprocess communication,” *Distributed Computing*, volume 1, number 2, pages 77–101, June 1986. doi:10.1007/BF01786228
 - [8] David K. Gifford: “Information Storage in a Decentralized Computer System,” Xerox Palo Alto Research Centers, CSL-81-8, June 1981.
 - [9] Martin Kleppmann: “Please Stop Calling Databases CP or AP,” martin.kleppmann.com, May 11, 2015.
 - [10] Kyle Kingsbury: “Call Me Maybe: MongoDB Stale Reads,” aphyr.com, April 20, 2015.
 - [11] Kyle Kingsbury: “Computational Techniques in Knossos,” aphyr.com, May 17, 2014.
 - [12] Peter Bailis: “Linearizability Versus Serializability,” bailis.org, September 24, 2014.
 - [13] Philip A. Bernstein, Vassos Hadzilacos, and Nathan Goodman: *Concurrency Control and Recovery in Database Systems*. Addison-Wesley, 1987. ISBN: 978-0-201-10715-9, available online at research.microsoft.com.
 - [14] Mike Burrows: “The Chubby Lock Service for Loosely-Coupled Distributed Systems,” at 7th USENIX Symposium on Operating System Design and Implementation (OSDI), November 2006.
 - [15] Flavio P. Junqueira and Benjamin Reed: *ZooKeeper: Distributed Process Coordination*. O’Reilly Media, 2013. ISBN: 978-1-449-36130-3
 - [16] “etcd 2.0.12 Documentation,” CoreOS, Inc., 2015.
 - [17] “Apache Curator,” Apache Software Foundation, curator.apache.org, 2015.
 - [18] Morali Vallath: *Oracle 10g RAC Grid, Services & Clustering*. Elsevier Digital Press, 2006. ISBN: 978-1-555-58321-7
 - [19] Peter Bailis, Alan Fekete, Michael J Franklin, et al.: “Coordination-Avoiding Database Systems,” *Proceedings of the VLDB Endowment*, volume 8, number 3, pages 185–196, November 2014.
 - [20] Kyle Kingsbury: “Call Me Maybe: etcd and Consul,” aphyr.com, June 9, 2014.

- [21] Flavio P. Junqueira, Benjamin C. Reed, and Marco Serafini: “Zab: High-Performance Broadcast for Primary-Backup Systems,” at 41st IEEE International Conference on Dependable Systems and Networks (DSN), June 2011. doi:10.1109/DSN.2011.5958223
- [22] Diego Ongaro and John K. Ousterhout: “In Search of an Understandable Consensus Algorithm (Extended Version),” at USENIX Annual Technical Conference (ATC), June 2014.
- [23] Hagit Attiya, Amotz Bar-Noy, and Danny Dolev: “Sharing Memory Robustly in Message-Passing Systems,” *Journal of the ACM*, volume 42, number 1, pages 124–142, January 1995. doi:10.1145/200836.200869
- [24] Nancy Lynch and Alex Shvartsman: “Robust Emulation of Shared Memory Using Dynamic Quorum-Acknowledged Broadcasts,” at 27th Annual International Symposium on Fault-Tolerant Computing (FTCS), June 1997. doi:10.1109/FTCS.1997.614100
- [25] Christian Cachin, Rachid Guerraoui, and Luís Rodrigues: *Introduction to Reliable and Secure Distributed Programming*, 2nd edition. Springer, 2011. ISBN: 978-3-642-15259-7, doi:10.1007/978-3-642-15260-3
- [26] Sam Elliott, Mark Allen, and Martin Kleppmann: personal communication, thread on twitter.com, October 15, 2015.
- [27] Niklas Ekström, Mikhail Panchenko, and Jonathan Ellis: “Possible Issue with Read Repair?,” email thread on cassandra-dev mailing list, October 2012.
- [28] Maurice P. Herlihy: “Wait-Free Synchronization,” *ACM Transactions on Programming Languages and Systems (TOPLAS)*, volume 13, number 1, pages 124–149, January 1991. doi:10.1145/114005.102808
- [29] Armando Fox and Eric A. Brewer: “Harvest, Yield, and Scalable Tolerant Systems,” at 7th Workshop on Hot Topics in Operating Systems (HotOS), March 1999. doi:10.1109/HOTOS.1999.798396
- [30] Seth Gilbert and Nancy Lynch: “Brewer’s Conjecture and the Feasibility of Consistent, Available, Partition-Tolerant Web Services,” *ACM SIGACT News*, volume 33, number 2, pages 51–59, June 2002. doi:10.1145/564585.564601
- [31] Seth Gilbert and Nancy Lynch: “Perspectives on the CAP Theorem,” *IEEE Computer Magazine*, volume 45, number 2, pages 30–36, February 2012. doi:10.1109/MC.2011.389
- [32] Eric A. Brewer: “CAP Twelve Years Later: How the ‘Rules’ Have Changed,” *IEEE Computer Magazine*, volume 45, number 2, pages 23–29, February 2012. doi:10.1109/MC.2012.37
- [33] Susan B. Davidson, Hector Garcia-Molina, and Dale Skeen: “Consistency in Partitioned Networks,” *ACM Computing Surveys*, volume 17, number 3, pages 341–370, September 1985. doi:10.1145/5505.5508
- [34] Paul R. Johnson and Robert H. Thomas: “RFC 677: The Maintenance of Duplicate Databases,” *Network Working Group*, January 27, 1975.
- [35] Bruce G. Lindsay, Patricia Griffiths Selinger, C. Galtieri, et al.: “Notes on Distributed Databases,” *IBM Research, Research Report RJ2571(33471)*, July 1979.
- [36] Michael J. Fischer and Alan Michael: “Sacrificing Serializability to Attain High Availability of Data in an Unreliable Network,” at 1st ACM Symposium on Principles of Database Systems (PODS), March 1982. doi:10.1145/588111.588124
- [37] Eric A. Brewer: “NoSQL: Past, Present, Future,” at QCon San Francisco, November 2012.
- [38] Henry Robinson: “CAP Confusion: Problems with ‘Partition Tolerance,’” *blog.cloudera.com*, April 26, 2010.

- [39] Adrian Cockcroft: “Migrating to Microservices,” at QCon London, March 2014.
- [40] Martin Kleppmann: “A Critique of the CAP Theorem,” arXiv:1509.05393, September 17, 2015.
- [41] Nancy A. Lynch: “A Hundred Impossibility Proofs for Distributed Computing,” at 8th ACM Symposium on Principles of Distributed Computing (PODC), August 1989. doi:10.1145/72981.72982
- [42] Hagit Attiya, Faith Ellen, and Adam Morrison: “Limitations of Highly-Available Eventually-Consistent Data Stores,” at ACM Symposium on Principles of Distributed Computing (PODC), July 2015. doi:10.1145/2767386.2767419
- [43] Peter Sewell, Susmit Sarkar, Scott Owens, et al.: “x86-TSO: A Rigorous and Usable Programmer’s Model for x86 Multiprocessors,” *Communications of the ACM*, volume 53, number 7, pages 89–97, July 2010. doi:10.1145/1785414.1785443
- [44] Martin Thompson: “Memory Barriers/Fences,” mechanical-sympathy.blogspot.co.uk, July 24, 2011.
- [45] Ulrich Drepper: “What Every Programmer Should Know About Memory,” akkadia.org, November 21, 2007.
- [46] Daniel J. Abadi: “Consistency Tradeoffs in Modern Distributed Database System Design,” *IEEE Computer Magazine*, volume 45, number 2, pages 37–42, February 2012. doi:10.1109/MC.2012.33
- [47] Hagit Attiya and Jennifer L. Welch: “Sequential Consistency Versus Linearizability,” *ACM Transactions on Computer Systems (TOCS)*, volume 12, number 2, pages 91–122, May 1994. doi:10.1145/176575.176576
- [48] Mustaque Ahamad, Gil Neiger, James E. Burns, et al.: “Causal Memory: Definitions, Implementation, and Programming,” *Distributed Computing*, volume 9, number 1, pages 37–49, March 1995. doi:10.1007/BF01784241
- [49] Wyatt Lloyd, Michael J. Freedman, Michael Kaminsky, and David G. Andersen: “Stronger Semantics for Low-Latency Geo-Replicated Storage,” at 10th USENIX Symposium on Networked Systems Design and Implementation (NSDI), April 2013.
- [50] Marek Zawirski, Annette Bieniusa, Valter Bolegas, et al.: “SwiftCloud: Fault-Tolerant Geo-Replication Integrated All the Way to the Client Machine,” INRIA Research Report 8347, August 2013.
- [51] Peter Bailis, Ali Ghodsi, Joseph M Hellerstein, and Ion Stoica: “Bolt-on Causal Consistency,” at ACM International Conference on Management of Data (SIGMOD), June 2013.
- [52] Philippe Ajoux, Nathan Bronson, Sanjeev Kumar, et al.: “Challenges to Adopting Stronger Consistency at Scale,” at 15th USENIX Workshop on Hot Topics in Operating Systems (HotOS), May 2015.
- [53] Peter Bailis: “Causality Is Expensive (and What to Do About It),” bailis.org, February 5, 2014.
- [54] Ricardo Gonçalves, Paulo Sérgio Almeida, Carlos Baquero, and Victor Fonte: “Concise Server-Wide Causality Management for Eventually Consistent Data Stores,” at 15th IFIP International Conference on Distributed Applications and Interoperable Systems (DAIS), June 2015. doi:10.1007/978-3-319-19129-4_6
- [55] Rob Conery: “A Better ID Generator for PostgreSQL,” rob.conery.io, May 29, 2014.
- [56] Leslie Lamport: “Time, Clocks, and the Ordering of Events in a Distributed System,” *Communications of the ACM*, volume 21, number 7, pages 558–565, July 1978. doi:10.1145/359545.359563

- [57] Xavier Défago, André Schiper, and Péter Urbán: “Total Order Broadcast and Multicast Algorithms: Taxonomy and Survey,” *ACM Computing Surveys*, volume 36, number 4, pages 372–421, December 2004. doi:10.1145/1041680.1041682
- [58] Hagit Attiya and Jennifer Welch: *Distributed Computing: Fundamentals, Simulations and Advanced Topics*, 2nd edition. John Wiley & Sons, 2004. ISBN: 978-0-471-45324-6, doi:10.1002/0471478210
- [59] Mahesh Balakrishnan, Dahlia Malkhi, Vijayan Prabhakaran, et al.: “CORFU: A Shared Log Design for Flash Clusters,” at 9th USENIX Symposium on Networked Systems Design and Implementation (NSDI), April 2012.
- [60] Fred B. Schneider: “Implementing Fault-Tolerant Services Using the State Machine Approach: A Tutorial,” *ACM Computing Surveys*, volume 22, number 4, pages 299–319, December 1990.
- [61] Alexander Thomson, Thaddeus Diamond, Shu-Chun Weng, et al.: “Calvin: Fast Distributed Transactions for Partitioned Database Systems,” at ACM International Conference on Management of Data (SIGMOD), May 2012.
- [62] Mahesh Balakrishnan, Dahlia Malkhi, Ted Wobber, et al.: “Tango: Distributed Data Structures over a Shared Log,” at 24th ACM Symposium on Operating Systems Principles (SOSP), November 2013. doi:10.1145/2517349.2522732
- [63] Robbert van Renesse and Fred B. Schneider: “Chain Replication for Supporting High Throughput and Availability,” at 6th USENIX Symposium on Operating System Design and Implementation (OSDI), December 2004.
- [64] Leslie Lamport: “How to Make a Multiprocessor Computer That Correctly Executes Multiprocess Programs,” *IEEE Transactions on Computers*, volume 28, number 9, pages 690–691, September 1979. doi:10.1109/TC.1979.1675439
- [65] Enis Söztutar, Devaraj Das, and Carter Shanklin: “Apache HBase High Availability at the Next Level,” *hortonworks.com*, January 22, 2015.
- [66] Brian F Cooper, Raghu Ramakrishnan, Utkarsh Srivastava, et al.: “PNUTS: Yahoo!’s Hosted Data Serving Platform,” at 34th International Conference on Very Large Data Bases (VLDB), August 2008. doi:10.14778/1454159.1454167
- [67] Tushar Deepak Chandra and Sam Toueg: “Unreliable Failure Detectors for Reliable Distributed Systems,” *Journal of the ACM*, volume 43, number 2, pages 225–267, March 1996. doi:10.1145/226643.226647
- [68] Michael J. Fischer, Nancy Lynch, and Michael S. Paterson: “Impossibility of Distributed Consensus with One Faulty Process,” *Journal of the ACM*, volume 32, number 2, pages 374–382, April 1985. doi:10.1145/3149.214121
- [69] Michael Ben-Or: “Another Advantage of Free Choice: Completely Asynchronous Agreement Protocols,” at 2nd ACM Symposium on Principles of Distributed Computing (PODC), August 1983. doi:10.1145/800221.806707
- [70] Jim N. Gray and Leslie Lamport: “Consensus on Transaction Commit,” *ACM Transactions on Database Systems (TODS)*, volume 31, number 1, pages 133–160, March 2006. doi:10.1145/1132863.1132867
- [71] Rachid Guerraoui: “Revisiting the Relationship Between Non-Blocking Atomic Commitment and Consensus,” at 9th International Workshop on Distributed Algorithms (WDAG), September 1995. doi:10.1007/BFb0022140
- [72] Thanumalayan Sankaranarayanan Pillai, Vijay Chidambaram, Ramnatthan Alagappan, et al.: “All File Systems Are Not Created Equal: On the Complexity of Crafting Crash-Consistent Applications,” at 11th USENIX Symposium on Operating Systems Design and Implementation (OSDI), October 2014.

- [73] Jim Gray: “The Transaction Concept: Virtues and Limitations,” at 7th International Conference on Very Large Data Bases (VLDB), September 1981.
- [74] Hector Garcia-Molina and Kenneth Salem: “Sagas,” at ACM International Conference on Management of Data (SIGMOD), May 1987. doi:10.1145/38713.38742
- [75] C. Mohan, Bruce G. Lindsay, and Ron Obermarck: “Transaction Management in the R* Distributed Database Management System,” ACM Transactions on Database Systems, volume 11, number 4, pages 378–396, December 1986. doi:10.1145/7239.7266
- [76] “Distributed Transaction Processing: The XA Specification,” X/Open Company Ltd., Technical Standard XO/CAE/91/300, December 1991. ISBN: 978-1-872-63024-3
- [77] Mike Spille: “XA Exposed, Part II,” jroller.com, April 3, 2004.
- [78] Ivan Silva Neto and Francisco Reverbel: “Lessons Learned from Implementing WS-Coordination and WS-AtomicTransaction,” at 7th IEEE/ACIS International Conference on Computer and Information Science (ICIS), May 2008. doi:10.1109/ICIS.2008.75
- [79] James E. Johnson, David E. Langworthy, Leslie Lamport, and Friedrich H. Vogt: “Formal Specification of a Web Services Protocol,” at 1st International Workshop on Web Services and Formal Methods (WS-FM), February 2004. doi:10.1016/j.entcs.2004.02.022
- [80] Dale Skeen: “Nonblocking Commit Protocols,” at ACM International Conference on Management of Data (SIGMOD), April 1981. doi:10.1145/582318.582339
- [81] Gregor Hohpe: “Your Coffee Shop Doesn’t Use Two-Phase Commit,” IEEE Software, volume 22, number 2, pages 64–66, March 2005. doi:10.1109/MS.2005.52
- [82] Pat Helland: “Life Beyond Distributed Transactions: An Apostate’s Opinion,” at 3rd Biennial Conference on Innovative Data Systems Research (CIDR), January 2007.
- [83] Jonathan Oliver: “My Beef with MSDTC and Two-Phase Commits,” blog.jonathanoliver.com, April 4, 2011.
- [84] Oren Eini (Ahende Rahien): “The Fallacy of Distributed Transactions,” ayende.com, July 17, 2014.
- [85] Clemens Vasters: “Transactions in Windows Azure (with Service Bus) – An Email Discussion,” vasters.com, July 30, 2012.
- [86] “Understanding Transactionality in Azure,” NServiceBus Documentation, Particular Software, 2015.
- [87] Randy Wigginton, Ryan Lowe, Marcos Albe, and Fernando Ipar: “Distributed Transactions in MySQL,” at MySQL Conference and Expo, April 2013.
- [88] Mike Spille: “XA Exposed, Part I,” jroller.com, April 3, 2004.
- [89] Ajmer Dhariwal: “Orphaned MSDTC Transactions (-2 spids),” eraofdata.com, December 12, 2008.
- [90] Paul Randal: “Real World Story of DBCC PAGE Saving the Day,” sqlskills.com, June 19, 2013.
- [91] “in-doubt xact resolution Server Configuration Option,” SQL Server 2016 documentation, Microsoft, Inc., 2016.
- [92] Cynthia Dwork, Nancy Lynch, and Larry Stockmeyer: “Consensus in the Presence of Partial Synchrony,” Journal of the ACM, volume 35, number 2, pages 288–323, April 1988. doi:10.1145/42282.42283

- [93] Miguel Castro and Barbara H. Liskov: “Practical Byzantine Fault Tolerance and Proactive Recovery,” *ACM Transactions on Computer Systems*, volume 20, number 4, pages 396–461, November 2002. doi:10.1145/571637.571640
- [94] Brian M. Oki and Barbara H. Liskov: “Viewstamped Replication: A New Primary Copy Method to Support Highly-Available Distributed Systems,” at 7th ACM Symposium on Principles of Distributed Computing (PODC), August 1988. doi:10.1145/62546.62549
- [95] Barbara H. Liskov and James Cowling: “Viewstamped Replication Revisited,” Massachusetts Institute of Technology, Tech Report MIT-CSAIL-TR-2012-021, July 2012.
- [96] Leslie Lamport: “The Part-Time Parliament,” *ACM Transactions on Computer Systems*, volume 16, number 2, pages 133–169, May 1998. doi:10.1145/279227.279229
- [97] Leslie Lamport: “Paxos Made Simple,” *ACM SIGACT News*, volume 32, number 4, pages 51–58, December 2001.
- [98] Tushar Deepak Chandra, Robert Griesemer, and Joshua Redstone: “Paxos Made Live – An Engineering Perspective,” at 26th ACM Symposium on Principles of Distributed Computing (PODC), June 2007.
- [99] Robbert van Renesse: “Paxos Made Moderately Complex,” cs.cornell.edu, March 2011.
- [100] Diego Ongaro: “Consensus: Bridging Theory and Practice,” PhD Thesis, Stanford University, August 2014.
- [101] Heidi Howard, Malte Schwarzkopf, Anil Madhavapeddy, and Jon Crowcroft: “Raft Refloated: Do We Have Consensus?,” *ACM SIGOPS Operating Systems Review*, volume 49, number 1, pages 12–21, January 2015. doi:10.1145/2723872.2723876
- [102] André Medeiros: “ZooKeeper’s Atomic Broadcast Protocol: Theory and Practice,” Aalto University School of Science, March 20, 2012.
- [103] Robbert van Renesse, Nicolas Schiper, and Fred B. Schneider: “Vive La Différence: Paxos vs. Viewstamped Replication vs. Zab,” *IEEE Transactions on Dependable and Secure Computing*, volume 12, number 4, pages 472–484, September 2014. doi:10.1109/TDSC.2014.2355848
- [104] Will Portnoy: “Lessons Learned from Implementing Paxos,” blog.willportnoy.com, June 14, 2012.
- [105] Heidi Howard, Dahlia Malkhi, and Alexander Spiegelman: “Flexible Paxos: Quorum Intersection Revisited,” arXiv:1608.06696, August 24, 2016.
- [106] Heidi Howard and Jon Crowcroft: “Coracle: Evaluating Consensus at the Internet Edge,” at Annual Conference of the ACM Special Interest Group on Data Communication (SIGCOMM), August 2015. doi:10.1145/2829988.2790010
- [107] Kyle Kingsbury: “Call Me Maybe: Elasticsearch 1.5.0,” aphyr.com, April 27, 2015.
- [108] Ivan Kelly: “BookKeeper Tutorial,” github.com, October 2014.
- [109] Camille Fournier: “Consensus Systems for the Skeptical Architect,” at Craft Conference, Budapest, Hungary, April 2015.
- [110] Kenneth P. Birman: “A History of the Virtual Synchrony Replication Model,” in *Replication: Theory and Practice*, Springer LNCS volume 5959, chapter 6, pages 91–120, 2010. ISBN: 978-3-642-11293-5, doi:10.1007/978-3-642-11294-2_6

Chapter 10. Batch Processing —

Chương 10. Xử lý theo lô

A system cannot be successful if it is too strongly influenced by a single person. Once the initial design is complete and fairly robust, the real test begins as people with many different viewpoints undertake their own experiments.

Một hệ thống không thể thành công nếu nó bị ảnh hưởng quá mạnh bởi một cá nhân duy nhất. Một khi thiết kế ban đầu đã hoàn tất và tương đối vững chãi, thì bài kiểm tra thực sự mới bắt đầu, khi những con người với nhiều góc nhìn khác nhau tiến hành các thử nghiệm của riêng họ.

Donald Knuth

Donald Knuth

In the first two parts of this book we talked a lot about requests and queries, and the corresponding responses or results. This style of data processing is assumed in many modern data systems: you ask for something, or you send an instruction, and some time later the system (hopefully) gives you an answer. Databases, caches, search indexes, web servers, and many other systems work this way.

Trong hai phần đầu của cuốn sách này, ta đã bàn rất nhiều về các yêu cầu (request) và truy vấn, cùng những phản hồi hoặc kết quả tương ứng. Phong cách xử lý dữ liệu này được giả định trong nhiều hệ thống dữ liệu hiện đại: bạn yêu cầu một thứ gì đó, hoặc gửi đi một chỉ thị, và một lúc sau hệ thống (hy vọng là) đưa cho bạn một câu trả lời. Cơ sở dữ liệu, bộ nhớ đệm, chỉ mục tìm kiếm, máy chủ web, và nhiều hệ thống khác đều hoạt động theo cách này.

In such online systems, whether it's a web browser requesting a page or a service calling a remote **API**, we generally assume that the request is triggered by a human user, and that the user is waiting for the response. They shouldn't have to wait too long, so we pay a lot of attention to the **response time** of these systems (see “Describing Performance”).

Trong những hệ thống trực tuyến như vậy, dù là một trình duyệt web yêu cầu một trang hay một dịch vụ gọi một API từ xa, ta thường giả định rằng yêu cầu được kích hoạt bởi một người dùng là con người, và rằng người dùng đó đang chờ đợi phản hồi. Họ không nên phải chờ quá lâu, nên ta dành rất nhiều sự chú ý cho thời gian phản hồi của các hệ thống này (xem “Mô tả hiệu năng”).

The web, and increasing numbers of HTTP/REST-based APIs, has made the request/response style of interaction so common that it's easy to take it for granted. But we should remember that it's not the only way of building systems, and that other approaches have their merits too. Let's distinguish three different types of systems:

Web, cùng số lượng ngày càng tăng các API dựa trên HTTP/REST, đã khiến cho phong cách tương tác yêu cầu/phản hồi trở nên phổ biến đến mức ta dễ dàng coi nó là điều hiển nhiên. Nhưng ta nên nhớ rằng đó không phải là cách duy nhất để xây dựng hệ thống, và các cách tiếp cận khác cũng có những ưu điểm riêng. Hãy phân biệt ba loại hệ thống khác nhau:

A service waits for a request or instruction from a **client** to arrive. When one is received, the service tries to handle it as quickly as possible and sends a response back. Response time is usually the primary measure of performance of a service, and availability is often very important (if the client can't reach the service, the user will probably get an error message).

Một dịch vụ chờ một yêu cầu hoặc chỉ thị từ một client đến. Khi nhận được, dịch vụ cố gắng xử lý nó nhanh nhất có thể rồi gửi lại một phản hồi. Thời gian phản hồi thường là thước đo hiệu năng chính của một dịch vụ, và tính sẵn sàng thường rất quan trọng (nếu client không thể tiếp cận dịch vụ, người dùng nhiều khả năng sẽ nhận được một thông báo lỗi).

A **batch processing** system takes a large amount of input data, runs a job to process it, and produces some output data. Jobs often take a while (from a few minutes to several days), so there normally isn't a user waiting for the job to finish. Instead, batch jobs are often scheduled to run periodically (for example, once a day). The primary performance measure of a batch job is usually **throughput** (the time it takes to crunch through an input dataset of a certain size). We discuss batch processing in this chapter.

Một hệ thống xử lý theo lô (batch processing) nhận một lượng lớn dữ liệu đầu vào, chạy một tác vụ (job) để xử lý nó, và tạo ra một số dữ liệu đầu ra. Các tác vụ thường mất một khoảng thời gian (từ vài phút đến vài ngày), nên thông thường không có người dùng nào ngồi chờ tác vụ hoàn thành. Thay vào đó, các tác vụ theo lô thường được lên lịch chạy định kỳ (ví dụ, một lần mỗi ngày). Thước đo hiệu năng chính của một tác vụ theo lô thường là thông lượng (thời gian cần để xử lý hết một tập dữ liệu đầu vào có kích thước nhất định). Ta sẽ bàn về xử lý theo lô trong chương này.

Stream processing is somewhere between online and offline/batch processing (so it is sometimes called near-real-time or nearline processing). Like a batch processing system, a stream processor consumes inputs and produces outputs (rather than responding to requests). However, a stream job operates on events shortly after they happen, whereas a batch job operates on a fixed set of input data. This difference allows stream processing systems to have lower **latency** than the equivalent batch systems. As stream processing builds upon batch processing, we discuss it in Chapter 11.

Xử lý luồng (stream processing) nằm đâu đó giữa xử lý trực tuyến và xử lý ngoại tuyến/theo lô (nên đôi khi nó được gọi là xử lý gần-thời-gian-thực hay nearline). Giống như một hệ thống xử lý theo lô, một bộ xử lý luồng tiêu thụ đầu vào và tạo ra đầu ra (chứ không phải phản hồi các yêu cầu). Tuy nhiên, một tác vụ luồng vận hành trên các sự kiện ngay sau khi chúng xảy ra, trong khi một tác vụ theo lô vận hành trên một tập dữ liệu đầu vào cố định. Sự khác biệt này cho phép các hệ thống xử lý luồng có độ trễ thấp hơn so với các hệ thống theo lô tương đương. Vì xử lý luồng được xây dựng dựa trên xử lý theo lô, ta sẽ bàn về nó ở Chương 11.

As we shall see in this chapter, batch processing is an important **building block** in our quest to build reliable, scalable, and maintainable applications. For example, **MapReduce**, a batch processing algorithm published in 2004 [1], was (perhaps over-enthusiastically) called “the algorithm that makes Google so massively scalable” [2]. It was subsequently implemented in various open source data systems, including Hadoop, CouchDB, and MongoDB.

Như ta sẽ thấy trong chương này, xử lý theo lô là một khối tiêu chuẩn quan trọng trong

hành trình xây dựng các ứng dụng tin cậy, có khả năng mở rộng và dễ bảo trì. Ví dụ, MapReduce, một thuật toán xử lý theo lô được công bố năm 2004 [1], đã (có lẽ với chút quá phần khích) được gọi là “thuật toán làm cho Google có khả năng mở rộng khổng lồ đến vậy” [2]. Sau đó nó được hiện thực trong nhiều hệ thống dữ liệu mã nguồn mở khác nhau, bao gồm Hadoop, CouchDB, và MongoDB.

MapReduce is a fairly low-level programming model compared to the parallel processing systems that were developed for data warehouses many years previously [3, 4], but it was a major step forward in terms of the scale of processing that could be achieved on commodity hardware. Although the importance of MapReduce is now declining [5], it is still worth understanding, because it provides a clear picture of why and how batch processing is useful.

MapReduce là một mô hình lập trình khá ở mức thấp so với các hệ thống xử lý song song vốn đã được phát triển cho các kho dữ liệu (data warehouse) nhiều năm trước đó [3, 4], nhưng nó là một bước tiến lớn về quy mô xử lý có thể đạt được trên phần cứng phổ thông. Mặc dù tầm quan trọng của MapReduce nay đang suy giảm [5], nó vẫn đáng để tìm hiểu, vì nó cho ta một bức tranh rõ ràng về việc tại sao và bằng cách nào xử lý theo lô lại hữu ích.

In fact, batch processing is a very old form of computing. Long before programmable digital computers were invented, punch card tabulating machines—such as the Hollerith machines used in the 1890 US Census [6]—implemented a semi-mechanized form of batch processing to compute aggregate statistics from large inputs. And MapReduce bears an uncanny resemblance to the electromechanical IBM card-sorting machines that were widely used for business data processing in the 1940s and 1950s [7]. As usual, history has a tendency of repeating itself.

Thực ra, xử lý theo lô là một hình thức tính toán rất lâu đời. Từ rất lâu trước khi máy tính số khả lập trình được phát minh, các máy lập bảng dùng thẻ đục lỗ — chẳng hạn các máy Hollerith được dùng trong cuộc Tổng điều tra dân số Hoa Kỳ năm 1890 [6] — đã hiện thực một dạng xử lý theo lô bán cơ giới để tính các thống kê tổng hợp từ những đầu vào lớn. Và MapReduce có một sự giống nhau đến kỳ lạ với các máy phân loại thẻ điện cơ của IBM, vốn được dùng rộng rãi cho xử lý dữ liệu nghiệp vụ trong thập niên 1940 và 1950 [7]. Như thường lệ, lịch sử có xu hướng lặp lại chính nó.

In this chapter, we will look at MapReduce and several other batch processing algorithms and frameworks, and explore how they are used in modern data systems. But first, to get started, we will look at data processing using standard Unix tools. Even if you are already familiar with them, a reminder about the Unix philosophy is worthwhile because the ideas and lessons from Unix carry over to large-scale, **heterogeneous** distributed data systems.

Trong chương này, ta sẽ xem xét MapReduce cùng vài thuật toán và framework xử lý theo lô khác, và khám phá cách chúng được dùng trong các hệ thống dữ liệu hiện đại. Nhưng trước tiên, để khởi động, ta sẽ xem xét việc xử lý dữ liệu bằng các công cụ Unix tiêu chuẩn. Ngay cả khi bạn đã quen thuộc với chúng, việc nhắc lại triết lý Unix vẫn đáng giá, bởi các ý tưởng và bài học từ Unix vẫn được áp dụng cho các hệ thống dữ liệu phân tán quy mô lớn, không đồng nhất.

Batch Processing with Unix Tools — Xử lý theo lô với các công cụ Unix

Let's start with a simple example. Say you have a web server that appends a line to a log file every time it serves a request. For example, using the nginx default access log format, one line of the log might look like this:

Hãy bắt đầu với một ví dụ đơn giản. Giả sử bạn có một máy chủ web ghi thêm một dòng vào một tệp nhật ký (log) mỗi khi nó phục vụ một yêu cầu. Ví dụ, dùng định dạng nhật ký truy cập mặc định của nginx, một dòng nhật ký có thể trông như thế này:

```
216.58.210.78 - - [27/Feb/2015:17:55:11 +0000] "GET /css/typography.css HTTP/1.1"
200 3377 "http://martin.kleppmann.com/" "Mozilla/5.0 (Macintosh; Intel Mac OS X
10_9_5) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/40.0.2214.115
Safari/537.36"
```

(That is actually one line; it's only broken onto multiple lines here for readability.) There's a lot of information in that line. In order to interpret it, you need to look at the definition of the log format, which is as follows:

(Đó thực ra là một dòng; ở đây nó chỉ bị ngắt thành nhiều dòng cho dễ đọc.) Có rất nhiều thông tin trong dòng đó. Để diễn giải nó, bạn cần xem định nghĩa của định dạng nhật ký, vốn như sau:

```
$remote_addr - $remote_user [$time_local] "$request"
$status $body_bytes_sent "$http_referer" "$http_user_agent"
```

So, this one line of the log indicates that on February 27, 2015, at 17:55:11 UTC, the server received a request for the file `/css/typography.css` from the client IP address 216.58.210.78. The user was not authenticated, so `$remote_user` is set to a hyphen (-). The response status was 200 (i.e., the request was successful), and the response was 3,377 bytes in size. The web browser was Chrome 40, and it loaded the file because it was referenced in the page at the URL `http://martin.kleppmann.com/`.

Vậy, một dòng nhật ký này cho thấy rằng vào ngày 27 tháng 2 năm 2015, lúc 17:55:11 UTC, máy chủ đã nhận một yêu cầu cho tệp `/css/typography.css` từ địa chỉ IP của client là 216.58.210.78. Người dùng chưa được xác thực, nên `$remote_user` được đặt thành một dấu gạch nối (-). Trạng thái phản hồi là 200 (tức là yêu cầu thành công), và phản hồi có kích thước 3.377 byte. Trình duyệt web là Chrome 40, và nó tải tệp này vì tệp được tham chiếu trong trang ở URL `http://martin.kleppmann.com/`.

Simple Log Analysis — Phân tích nhật ký đơn giản

Various tools can take these log files and produce pretty reports about your website traffic, but for the sake of exercise, let's build our own, using basic Unix tools. For example, say you want to find the five most popular pages on your website. You can do this in a Unix shell as follows:

Nhiều công cụ khác nhau có thể nhận các tệp nhật ký này và tạo ra những báo cáo đẹp đẽ về lưu lượng truy cập trang web của bạn, nhưng để luyện tập, hãy tự xây dựng công cụ của riêng mình, dùng các công cụ Unix cơ bản. Ví dụ, giả sử bạn muốn tìm năm trang phổ biến nhất trên trang web của mình. Bạn có thể làm điều này trong một shell Unix như sau:

```
cat /var/log/nginx/access.log |  
  awk '{print $7}' |  
  sort |  
  uniq -c |  
  sort -r -n |  
  head -n 5
```

Read the log file.

Đọc tệp nhật ký.

Split each line into fields by whitespace, and output only the seventh such field from each line, which happens to be the requested URL. In our example line, this request URL is `/css/typography.css`.

Tách mỗi dòng thành các trường theo khoảng trắng, và chỉ xuất ra trường thứ bảy của mỗi dòng, vốn tình cờ là URL được yêu cầu. Trong dòng ví dụ của ta, URL yêu cầu này là `/css/typography.css`.

Alphabetically sort the list of requested URLs. If some URL has been requested *n* times, then after sorting, the file contains the same URL repeated *n* times in a **row**.

Sắp xếp danh sách các URL được yêu cầu theo thứ tự bảng chữ cái. Nếu một URL nào đó đã được yêu cầu *n* lần, thì sau khi sắp xếp, tệp chứa cùng URL đó lặp lại *n* lần liên tiếp.

The `uniq` command filters out repeated lines in its input by checking whether two adjacent lines are the same. The `-c` option tells it to also output a counter: for every distinct URL, it reports how many times that URL appeared in the input.

Lệnh `uniq` lọc bỏ những dòng lặp lại trong đầu vào của nó bằng cách kiểm tra xem hai dòng liền kề có giống nhau không. Tùy chọn `-c` bảo nó cũng xuất ra một bộ đếm: với mỗi URL phân biệt, nó báo cáo URL đó xuất hiện bao nhiêu lần trong đầu vào.

The second sort sorts by the number (`-n`) at the start of each line, which is the number of times the URL was requested. It then returns the results in reverse (`-r`) order, i.e. with the largest number first.

Lệnh sort thứ hai sắp xếp theo con số (`-n`) ở đầu mỗi dòng, vốn là số lần URL được yêu cầu. Sau đó nó trả về kết quả theo thứ tự đảo ngược (`-r`), tức là với con số lớn nhất ở đầu.

Finally, `head` outputs just the first five lines (`-n 5`) of input, and discards the rest.

Cuối cùng, `head` chỉ xuất ra năm dòng đầu tiên (`-n 5`) của đầu vào, và loại bỏ phần còn lại.

The output of that series of commands looks something like this:

Đầu ra của loạt lệnh đó trông đại loại như thế này:

```
4189 /favicon.ico  
3631 /2013/05/24/improving-security-of-ssh-private-keys.html
```



```
2124 /2012/12/05/schema-evolution-in-avro-protocol-buffers-thrift.html
1369 /
915 /css/typography.css
```

Although the preceding command line likely looks a bit obscure if you're unfamiliar with Unix tools, it is incredibly powerful. It will process gigabytes of log files in a matter of seconds, and you can easily modify the analysis to suit your needs. For example, if you want to omit CSS files from the report, change the awk argument to '\$7 !~ /\.css\$/ {print \$7}'. If you want to count top client IP addresses instead of top pages, change the awk argument to '{print \$1}'. And so on.

Mặc dù dòng lệnh phía trên có lẽ trông hơi khó hiểu nếu bạn không quen với các công cụ Unix, nó cực kỳ mạnh mẽ. Nó sẽ xử lý hàng gigabyte tệp nhật ký chỉ trong vài giây, và bạn có thể dễ dàng chỉnh sửa phép phân tích cho phù hợp với nhu cầu của mình. Ví dụ, nếu muốn bỏ các tệp CSS khỏi báo cáo, hãy đổi đổi số của awk thành '\$7 !~ /\.css\$/ {print \$7}'. Nếu muốn đếm các địa chỉ IP của client xuất hiện nhiều nhất thay vì các trang xuất hiện nhiều nhất, hãy đổi đổi số của awk thành '{print \$1}'. Và cứ thế.

We don't have space in this book to explore Unix tools in detail, but they are very much worth learning about. Surprisingly many data analyses can be done in a few minutes using some combination of awk, sed, grep, sort, uniq, and xargs, and they perform surprisingly well [8].

Cuốn sách này không có chỗ để khám phá các công cụ Unix một cách chi tiết, nhưng chúng rất đáng để tìm hiểu. Đáng ngạc nhiên là rất nhiều phép phân tích dữ liệu có thể được thực hiện chỉ trong vài phút bằng cách kết hợp awk, sed, grep, sort, uniq, và xargs, và chúng cho hiệu năng tốt một cách đáng kinh ngạc [8].

Chain of commands versus custom program — Chuỗi lệnh so với chương trình tùy chỉnh

Instead of the chain of Unix commands, you could write a simple program to do the same thing. For example, in Ruby, it might look something like this:

Thay vì dùng chuỗi lệnh Unix, bạn có thể viết một chương trình đơn giản để làm cùng một việc. Ví dụ, trong Ruby, nó có thể trông đại loại như thế này:

```
counts = Hash.new(0)
File.open('/var/log/nginx/access.log') do |file|
  file.each do |line|
    url = line.split[6]
    counts[url] += 1
  end
end
top5 = counts.map{|url, count| [count, url] }.sort.reverse[0...5]
top5.each{|count, url| puts "#{count} #{url}" }
```

counts is a hash **table** that keeps a counter for the number of times we've seen each URL. A counter is zero by default.

counts là một bảng băm giữ một bộ đếm cho số lần ta đã thấy mỗi URL. Một bộ đếm mặc định là không.

From each line of the log, we take the URL to be the seventh whitespace-separated field (the array **index** here is 6 because Ruby's arrays are zero-indexed).

Từ mỗi dòng của nhật ký, ta lấy URL là trường thứ bảy được phân tách bởi khoảng trắng (chỉ số mảng ở đây là 6 vì mảng của Ruby đánh chỉ số từ không).

Increment the counter for the URL in the current line of the log.

Tăng bộ đếm cho URL trong dòng nhật ký hiện tại.

Sort the hash table contents by counter value (descending), and take the top five entries.

Sắp xếp nội dung bảng băm theo giá trị bộ đếm (giảm dần), và lấy năm mục đầu tiên.

Print out those top five entries.

In ra năm mục đầu tiên đó.

This program is not as concise as the chain of Unix pipes, but it's fairly readable, and which of the two you prefer is partly a matter of taste. However, besides the superficial syntactic differences between the two, there is a big difference in the execution flow, which becomes apparent if you run this analysis on a large file.

Chương trình này không cô đọng bằng chuỗi pipe của Unix, nhưng nó khá dễ đọc, và việc bạn thích cái nào trong hai cái phần nào là vấn đề sở thích. Tuy nhiên, ngoài những khác biệt cú pháp bề mặt giữa hai cái, có một khác biệt lớn trong luồng thực thi, điều trở nên rõ ràng nếu bạn chạy phép phân tích này trên một tệp lớn.

Sorting versus in-memory aggregation — Sắp xếp so với tổng hợp trong bộ nhớ

The Ruby script keeps an in-memory hash table of URLs, where each URL is mapped to the number of times it has been seen. The Unix pipeline example does not have such a hash table, but instead relies on sorting a list of URLs in which multiple occurrences of the same URL are simply repeated.

Script Ruby giữ một bảng băm trong bộ nhớ chứa các URL, trong đó mỗi URL được ánh xạ tới số lần nó đã được thấy. Ví dụ pipeline Unix không có một bảng băm như vậy, mà thay vào đó dựa vào việc sắp xếp một danh sách các URL trong đó nhiều lần xuất hiện của cùng một URL chỉ đơn giản được lặp lại.

Which approach is better? It depends how many different URLs you have. For most small to mid-sized websites, you can probably fit all distinct URLs, and a counter for each URL, in (say) 1 GB of memory. In this example, the working set of the job (the amount of memory to which the job needs random access) depends only on the number of distinct URLs: if there are a million log entries for a single URL, the space required in the hash table is still just one URL plus the size of the counter. If this working set is small enough, an in-memory hash table works fine—even on a laptop.

Cách tiếp cận nào tốt hơn? Điều đó phụ thuộc vào việc bạn có bao nhiêu URL khác nhau. Với hầu hết các trang web cỡ nhỏ đến trung bình, có lẽ bạn có thể nhét tất cả các URL phân biệt, cùng một bộ đếm cho mỗi URL, vào (chẳng hạn) 1 GB bộ nhớ. Trong ví dụ này, tập làm việc (working set) của tác vụ (lượng bộ nhớ mà tác vụ cần truy cập ngẫu nhiên) chỉ phụ thuộc vào số lượng URL phân biệt: nếu có một triệu mục nhật ký cho một URL duy nhất, thì không gian cần trong bảng băm vẫn chỉ là một URL cộng với kích thước của bộ đếm. Nếu tập làm việc này đủ nhỏ, một bảng băm trong bộ nhớ hoạt động tốt — kể cả trên một máy tính xách tay.

On the other hand, if the job's working set is larger than the available memory, the sorting approach has the advantage that it can make efficient use of disks. It's the same principle as we discussed in “SSTables and LSM-Trees”: chunks of data can be sorted in memory and written out to disk as

segment files, and then multiple sorted segments can be merged into a larger sorted file. Mergesort has sequential access patterns that perform well on disks. (Remember that optimizing for sequential I/O was a recurring theme in Chapter 3. The same pattern reappears here.)

Mặt khác, nếu tập làm việc của tác vụ lớn hơn bộ nhớ khả dụng, thì cách tiếp cận sắp xếp có lợi thế ở chỗ nó có thể tận dụng đĩa một cách hiệu quả. Đó là cùng nguyên lý mà ta đã bàn ở “SSTable và LSM-Tree”: các khối dữ liệu có thể được sắp xếp trong bộ nhớ và ghi ra đĩa thành các tệp phân đoạn (segment), rồi nhiều phân đoạn đã sắp xếp có thể được trộn thành một tệp đã sắp xếp lớn hơn. Mergesort có mẫu truy cập tuần tự cho hiệu năng tốt trên đĩa. (Hãy nhớ rằng việc tối ưu cho I/O tuần tự là một chủ đề lặp đi lặp lại ở Chương 3. Cùng mẫu đó lại xuất hiện ở đây.)

The sort utility in GNU Coreutils (Linux) automatically handles larger-than-memory datasets by spilling to disk, and automatically parallelizes sorting across multiple CPU cores [9]. This means that the simple chain of Unix commands we saw earlier easily scales to large datasets, without running out of memory. The bottleneck is likely to be the rate at which the input file can be read from disk.

Tiện ích sort trong GNU Coreutils (Linux) tự động xử lý các tập dữ liệu lớn hơn bộ nhớ bằng cách tràn ra đĩa, và tự động song song hóa việc sắp xếp trên nhiều lõi CPU [9]. Điều này nghĩa là chuỗi lệnh Unix đơn giản mà ta thấy trước đó dễ dàng mở rộng tới các tập dữ liệu lớn, mà không bị hết bộ nhớ. Điểm nghẽn nhiều khả năng là tốc độ mà tệp đầu vào có thể được đọc từ đĩa.

The Unix Philosophy — Triết lý Unix

It’s no coincidence that we were able to analyze a log file quite easily, using a chain of commands like in the previous example: this was in fact one of the key design ideas of Unix, and it remains astonishingly relevant today. Let’s look at it in some more depth so that we can borrow some ideas from Unix [10].

Không phải ngẫu nhiên mà ta có thể phân tích một tệp nhật ký khá dễ dàng, dùng một chuỗi lệnh như trong ví dụ trước: đây thực ra là một trong những ý tưởng thiết kế cốt lõi của Unix, và nó vẫn còn liên quan một cách đáng kinh ngạc đến tận ngày nay. Hãy xem xét nó sâu hơn một chút để ta có thể mượn vài ý tưởng từ Unix [10].

Doug McIlroy, the inventor of Unix pipes, first described them like this in 1964 [11]: “We should have some ways of connecting programs like [a] garden hose—screw in another segment when it becomes necessary to massage data in another way. This is the way of I/O also.” The plumbing analogy stuck, and the idea of connecting programs with pipes became part of what is now known as the Unix philosophy—a set of design principles that became popular among the developers and users of Unix. The philosophy was described in 1978 as follows [12, 13]:

Doug McIlroy, người phát minh ra pipe của Unix, lần đầu mô tả chúng như thế này vào năm 1964 [11]: “Ta nên có vài cách để kết nối các chương trình giống như [một] ống tưới vườn — vặn thêm một đoạn nữa vào khi cần xử lý dữ liệu theo cách khác. Đây cũng là cách của I/O.” Phép loại suy ống nước này còn đọng lại, và ý tưởng kết nối các chương trình bằng pipe trở thành một phần của thứ mà ngày nay được gọi là triết lý Unix — một tập hợp các nguyên lý thiết kế trở nên phổ biến trong giới phát triển và người dùng Unix. Triết lý này được mô tả vào năm 1978 như sau [12, 13]:

- Make each program do one thing well. To do a new job, build afresh rather than complicate old programs by adding new “features”.

- Expect the output of every program to become the input to another, as yet unknown, program. Don't clutter output with extraneous information. Avoid stringently columnar or binary input formats. Don't insist on interactive input.
 - Design and build software, even operating systems, to be tried early, ideally within weeks. Don't hesitate to throw away the clumsy parts and rebuild them.
 - Use tools in preference to unskilled help to lighten a programming task, even if you have to detour to build the tools and expect to throw some of them out after you've finished using them.
- Làm cho mỗi chương trình làm tốt một việc. Để làm một công việc mới, hãy xây dựng lại từ đầu thay vì làm phức tạp các chương trình cũ bằng cách thêm các “tính năng” mới.
 - Mong đợi đầu ra của mọi chương trình trở thành đầu vào của một chương trình khác, chưa biết trước. Đừng làm lộn xộn đầu ra bằng thông tin thừa thãi. Tránh các định dạng đầu vào dạng cột cứng nhắc hoặc dạng nhị phân. Đừng khăng khăng đòi đầu vào tương tác.
 - Thiết kế và xây dựng phần mềm, kể cả hệ điều hành, sao cho có thể được dùng thử sớm, lý tưởng là trong vòng vài tuần. Đừng ngần ngại vứt bỏ những phần vụng về và xây dựng lại chúng.
 - Hãy dùng công cụ thay vì sự trợ giúp thiếu kỹ năng để giảm gánh nặng cho một tác vụ lập trình, ngay cả khi bạn phải đi đường vòng để xây dựng các công cụ đó và dự kiến vứt bỏ một số trong số chúng sau khi đã dùng xong.

This approach—automation, rapid prototyping, incremental iteration, being friendly to experimentation, and breaking down large projects into manageable chunks—sounds remarkably like the **Agile** and DevOps movements of today. Surprisingly little has changed in four decades.

Cách tiếp cận này — tự động hóa, tạo nguyên mẫu nhanh, lặp lại tăng dần, thân thiện với việc thử nghiệm, và chia nhỏ các dự án lớn thành những phần dễ quản lý — nghe giống một cách đáng kinh ngạc với các phong trào Agile và DevOps ngày nay. Đáng ngạc nhiên là rất ít thứ đã thay đổi trong bốn thập kỷ.

The sort tool is a great example of a program that does one thing well. It is arguably a better sorting implementation than most programming languages have in their standard libraries (which do not spill to disk and do not use multiple threads, even when that would be beneficial). And yet, sort is barely useful in isolation. It only becomes powerful in combination with the other Unix tools, such as `uniq`.

Công cụ sort là một ví dụ tuyệt vời về một chương trình làm tốt một việc. Có thể nói nó là một bản hiện thực sắp xếp tốt hơn so với cái mà hầu hết các ngôn ngữ lập trình có trong thư viện chuẩn của chúng (vốn không tràn ra đĩa và không dùng nhiều luồng, ngay cả khi điều đó sẽ có lợi). Vậy mà, sort hầu như vô dụng khi đứng riêng lẻ. Nó chỉ trở nên mạnh mẽ khi kết hợp với các công cụ Unix khác, chẳng hạn `uniq`.

A Unix shell like bash lets us easily compose these small programs into surprisingly powerful data processing jobs. Even though many of these programs are written by different groups of people, they can be joined together in flexible ways. What does Unix do to enable this composability?

Một shell Unix như bash cho phép ta dễ dàng kết hợp những chương trình nhỏ này thành các tác vụ xử lý dữ liệu mạnh mẽ một cách đáng ngạc nhiên. Mặc dù nhiều chương trình trong số này được viết bởi những nhóm người khác nhau, chúng có thể được ghép lại với nhau theo những cách linh hoạt. Unix làm gì để cho phép khả năng kết hợp này?

A uniform interface — Một giao diện đồng nhất

If you expect the output of one program to become the input to another program, that means those programs must use the same data format—in other words, a compatible interface. If you want to be able to connect any program's output to any program's input, that means that all programs must use the same input/output interface.

Nếu bạn mong đợi đầu ra của một chương trình trở thành đầu vào của một chương trình khác, điều đó có nghĩa là các chương trình đó phải dùng cùng một định dạng dữ liệu — nói cách khác, một giao diện tương thích. Nếu bạn muốn có thể kết nối đầu ra của bất kỳ chương trình nào với đầu vào của bất kỳ chương trình nào, điều đó có nghĩa là tất cả các chương trình phải dùng cùng một giao diện đầu vào/đầu ra.

In Unix, that interface is a file (or, more precisely, a file descriptor). A file is just an ordered sequence of bytes. Because that is such a simple interface, many different things can be represented using the same interface: an actual file on the filesystem, a communication channel to another process (Unix socket, stdin, stdout), a device driver (say /dev/audio or /dev/lp0), a socket representing a TCP connection, and so on. It's easy to take this for granted, but it's actually quite remarkable that these very different things can share a uniform interface, so they can easily be plugged together.

Trong Unix, giao diện đó là một tệp (hay, chính xác hơn, một bộ mô tả tệp - file descriptor). Một tệp chỉ là một dãy byte có thứ tự. Vì đó là một giao diện đơn giản đến vậy, nhiều thứ rất khác nhau có thể được biểu diễn bằng cùng giao diện đó: một tệp thực sự trên hệ thống tệp, một kênh giao tiếp tới một tiến trình khác (socket Unix, stdin, stdout), một trình điều khiển thiết bị (chẳng hạn /dev/audio hoặc /dev/lp0), một socket biểu diễn một kết nối TCP, và cứ thế. Ta dễ coi điều này là hiển nhiên, nhưng thực sự khá đáng chú ý rằng những thứ rất khác nhau này có thể chia sẻ một giao diện đồng nhất, nên chúng có thể dễ dàng được ghép lại với nhau.

By convention, many (but not all) Unix programs treat this sequence of bytes as ASCII text. Our log analysis example used this fact: `awk`, `sort`, `uniq`, and `head` all treat their input file as a list of records separated by the `\n` (newline, ASCII 0x0A) character. The choice of `\n` is arbitrary—arguably, the ASCII record separator 0x1E would have been a better choice, since it's intended for this purpose [14]—but in any case, the fact that all these programs have standardized on using the same record separator allows them to interoperate.

Theo quy ước, nhiều (nhưng không phải tất cả) chương trình Unix xử lý dãy byte này như văn bản ASCII. Ví dụ phân tích nhật ký của ta đã dùng sự thật này: `awk`, `sort`, `uniq`, và `head` đều xử lý tệp đầu vào của chúng như một danh sách các bản ghi được phân tách bởi ký tự `\n` (xuống dòng, ASCII 0x0A). Việc chọn `\n` là tùy ý — có thể lập luận rằng ký tự phân tách bản ghi của ASCII là 0x1E sẽ là một lựa chọn tốt hơn, vì nó được thiết kế cho mục đích này [14] — nhưng dù sao, việc tất cả các chương trình này đã chuẩn hóa trên cùng một ký tự phân tách bản ghi cho phép chúng tương tác với nhau.

The parsing of each record (i.e., a line of input) is more vague. Unix tools commonly split a line into fields by whitespace or tab characters, but CSV (comma-separated), pipe-separated, and other encodings are also used. Even a fairly simple tool like `xargs` has half a dozen command-line options for specifying how its input should be parsed.

Việc phân tích cú pháp mỗi bản ghi (tức là một dòng đầu vào) thì mơ hồ hơn. Các công cụ Unix thường tách một dòng thành các trường theo ký tự khoảng trắng hoặc tab, nhưng CSV (phân tách bởi dấu phẩy), phân tách bởi dấu sổ đứng, và các cách mã hóa khác cũng được dùng. Ngay cả một công cụ khá đơn giản như `xargs` cũng có nửa tá tùy chọn dòng lệnh để chỉ định cách đầu vào của nó nên được phân tích.

The uniform interface of ASCII text mostly works, but it's not exactly beautiful: our log analysis example used `{print $7}` to extract the URL, which is not very readable. In an ideal world this could have perhaps been `{print $request_url}` or something of that sort. We will return to this idea later.

Giao diện đồng nhất của văn bản ASCII phần lớn hoạt động được, nhưng nó không hẳn là đẹp đẽ: ví dụ phân tích nhật ký của ta đã dùng `{print $7}` để trích xuất URL, điều không dễ đọc lắm. Trong một thế giới lý tưởng, lẽ ra cái này có thể là `{print $request_url}` hoặc đại loại thế. Ta sẽ quay lại ý tưởng này sau.

Although it's not perfect, even decades later, the uniform interface of Unix is still something remarkable. Not many pieces of software interoperate and compose as well as Unix tools do: you can't easily pipe the contents of your email account and your online shopping history through a custom analysis tool into a spreadsheet and post the results to a social network or a wiki. Today it's an exception, not the norm, to have programs that work together as smoothly as Unix tools do.

Mặc dù không hoàn hảo, kể cả nhiều thập kỷ sau, giao diện đồng nhất của Unix vẫn là một thứ đáng chú ý. Không có nhiều phần mềm tương tác và kết hợp được tốt như các công cụ Unix: bạn không thể dễ dàng đưa nội dung tài khoản email và lịch sử mua sắm trực tuyến của mình qua một công cụ phân tích tùy chỉnh vào một bảng tính rồi đăng kết quả lên một mạng xã hội hay một wiki. Ngày nay, việc có các chương trình hoạt động cùng nhau mượt mà như các công cụ Unix là một ngoại lệ, chứ không phải lệ thường.

Even databases with the same **data model** often don't make it easy to get data out of one and into the other. This lack of integration leads to Balkanization of data.

Ngay cả các cơ sở dữ liệu có cùng mô hình dữ liệu cũng thường không khiến cho việc lấy dữ liệu ra khỏi cái này và đưa vào cái khác trở nên dễ dàng. Sự thiếu tích hợp này dẫn tới sự "Balkan hóa" của dữ liệu.

Separation of logic and wiring — Tách rời logic và đấu nối

Another characteristic feature of Unix tools is their use of standard input (stdin) and standard output (stdout). If you run a program and don't specify anything else, stdin comes from the keyboard and stdout goes to the screen. However, you can also take input from a file and/or redirect output to a file. Pipes let you attach the stdout of one process to the stdin of another process (with a small in-memory buffer, and without writing the entire intermediate data stream to disk).

Một đặc trưng tiêu biểu khác của các công cụ Unix là việc chúng dùng đầu vào chuẩn (stdin) và đầu ra chuẩn (stdout). Nếu bạn chạy một chương trình và không chỉ định gì khác, stdin đến từ bàn phím và stdout đi ra màn hình. Tuy nhiên, bạn cũng có thể lấy đầu vào từ một tệp và/hoặc chuyển hướng đầu ra tới một tệp. Pipe cho phép bạn gắn stdout của một tiến trình vào stdin của một tiến trình khác (với một bộ đệm nhỏ trong bộ nhớ, và không ghi toàn bộ luồng dữ liệu trung gian ra đĩa).

A program can still read and write files directly if it needs to, but the Unix approach works best if a program doesn't worry about particular file paths and simply uses stdin and stdout. This allows a shell user to wire up the input and output in whatever way they want; the program doesn't know or care where the input is coming from and where the output is going to. (One could say this is a form of loose coupling, late binding [15], or inversion of control [16].) Separating the input/output wiring from the program logic makes it easier to compose small tools into bigger systems.

Một chương trình vẫn có thể đọc và ghi tệp trực tiếp nếu cần, nhưng cách tiếp cận Unix hoạt động tốt nhất nếu một chương trình không bận tâm tới các đường dẫn tệp cụ thể mà chỉ đơn giản dùng stdin và stdout. Điều này cho phép người dùng shell đấu nối đầu vào

và đầu ra theo bất kỳ cách nào họ muốn; chương trình không biết và không quan tâm đầu vào đến từ đâu và đầu ra đi tới đâu. (Có thể nói đây là một dạng gắn kết lỏng, ràng buộc muộn (late binding) [15], hay đảo ngược điều khiển (inversion of control) [16].) Việc tách rời phần đầu nối đầu vào/đầu ra khỏi logic chương trình khiến cho việc kết hợp các công cụ nhỏ thành các hệ thống lớn hơn trở nên dễ dàng hơn.

You can even write your own programs and combine them with the tools provided by the operating system. Your program just needs to read input from stdin and write output to stdout, and it can participate in data processing pipelines. In the log analysis example, you could write a tool that translates user-agent strings into more sensible browser identifiers, or a tool that translates IP addresses into country codes, and simply plug it into the pipeline. The sort program doesn't care whether it's communicating with another part of the operating system or with a program written by you.

Bạn thậm chí có thể viết các chương trình của riêng mình và kết hợp chúng với các công cụ do hệ điều hành cung cấp. Chương trình của bạn chỉ cần đọc đầu vào từ stdin và ghi đầu ra ra stdout, và nó có thể tham gia vào các pipeline xử lý dữ liệu. Trong ví dụ phân tích nhật ký, bạn có thể viết một công cụ chuyển các chuỗi user-agent thành những định danh trình duyệt hợp lý hơn, hoặc một công cụ chuyển các địa chỉ IP thành mã quốc gia, và chỉ đơn giản cắm nó vào pipeline. Chương trình sort không quan tâm liệu nó đang giao tiếp với một phần khác của hệ điều hành hay với một chương trình do bạn viết.

However, there are limits to what you can do with stdin and stdout. Programs that need multiple inputs or outputs are possible but tricky. You can't pipe a program's output into a network connection [17, 18]. If a program directly opens files for reading and writing, or starts another program as a subprocess, or opens a network connection, then that I/O is wired up by the program itself. It can still be configurable (through command-line options, for example), but the flexibility of wiring up inputs and outputs in a shell is reduced.

Tuy nhiên, có những giới hạn về những gì bạn có thể làm với stdin và stdout. Các chương trình cần nhiều đầu vào hoặc đầu ra là khả thi nhưng phức tạp. Bạn không thể đưa đầu ra của một chương trình vào một kết nối mạng [17, 18]. Nếu một chương trình trực tiếp mở các tệp để đọc và ghi, hoặc khởi động một chương trình khác như một tiến trình con, hoặc mở một kết nối mạng, thì phần I/O đó được tự chương trình đầu nối. Nó vẫn có thể cấu hình được (ví dụ qua các tùy chọn dòng lệnh), nhưng sự linh hoạt trong việc đầu nối đầu vào và đầu ra trong một shell bị giảm đi.

Transparency and experimentation — Tính minh bạch và khả năng thử nghiệm

Part of what makes Unix tools so successful is that they make it quite easy to see what is going on:

Một phần làm cho các công cụ Unix thành công đến vậy là chúng khiến việc thấy được điều gì đang diễn ra trở nên khá dễ dàng:

- The input files to Unix commands are normally treated as immutable. This means you can run the commands as often as you want, trying various command-line options, without damaging the input files.
- You can end the pipeline at any point, pipe the output into less, and look at it to see if it has the expected form. This ability to inspect is great for debugging.
- You can write the output of one pipeline stage to a file and use that file as input to the next stage. This allows you to restart the later stage without rerunning the entire pipeline.

- Các tệp đầu vào cho các lệnh Unix thường được coi là bất biến. Điều này nghĩa là bạn có thể chạy các lệnh bao nhiêu lần tùy thích, thử các tùy chọn dòng lệnh khác nhau, mà không làm hỏng các tệp đầu vào.
- Bạn có thể kết thúc pipeline ở bất kỳ điểm nào, đưa đầu ra vào less, và xem nó để kiểm tra xem nó có dạng như mong đợi không. Khả năng kiểm tra này tuyệt vời cho việc gỡ lỗi.
- Bạn có thể ghi đầu ra của một giai đoạn pipeline ra một tệp và dùng tệp đó làm đầu vào cho giai đoạn tiếp theo. Điều này cho phép bạn khởi động lại giai đoạn sau mà không cần chạy lại toàn bộ pipeline.

Thus, even though Unix tools are quite blunt, simple tools compared to a **query optimizer** of a **relational database**, they remain amazingly useful, especially for experimentation.

Vì vậy, dù các công cụ Unix là những công cụ khá thô sơ, đơn giản so với một bộ tối ưu truy vấn của một cơ sở dữ liệu quan hệ, chúng vẫn hữu ích một cách đáng kinh ngạc, đặc biệt cho việc thử nghiệm.

However, the biggest limitation of Unix tools is that they run only on a single machine—and that's where tools like Hadoop come in.

Tuy nhiên, hạn chế lớn nhất của các công cụ Unix là chúng chỉ chạy trên một máy duy nhất — và đó là chỗ mà các công cụ như Hadoop xuất hiện.

MapReduce and Distributed Filesystems — MapReduce và hệ thống tệp phân tán

MapReduce is a bit like Unix tools, but distributed across potentially thousands of machines. Like Unix tools, it is a fairly blunt, brute-force, but surprisingly effective tool. A single MapReduce job is comparable to a single Unix process: it takes one or more inputs and produces one or more outputs.

MapReduce hơi giống các công cụ Unix, nhưng được phân tán trên có khả năng tới hàng nghìn máy. Giống như các công cụ Unix, nó là một công cụ khá thô sơ, kiểu vũ lực, nhưng hiệu quả một cách đáng ngạc nhiên. Một tác vụ MapReduce đơn lẻ có thể so sánh với một tiến trình Unix đơn lẻ: nó nhận một hoặc nhiều đầu vào và tạo ra một hoặc nhiều đầu ra.

As with most Unix tools, running a MapReduce job normally does not modify the input and does not have any side effects other than producing the output. The output files are written once, in a sequential fashion (not modifying any existing part of a file once it has been written).

Cũng như hầu hết các công cụ Unix, việc chạy một tác vụ MapReduce thường không sửa đổi đầu vào và không có bất kỳ tác dụng phụ nào ngoài việc tạo ra đầu ra. Các tệp đầu ra được ghi một lần, theo kiểu tuần tự (không sửa đổi bất kỳ phần nào đã tồn tại của một tệp một khi nó đã được ghi).

While Unix tools use stdin and stdout as input and output, MapReduce jobs read and write files on a distributed filesystem. In Hadoop's implementation of MapReduce, that filesystem is called HDFS (Hadoop Distributed File System), an open source reimplement of the Google File System (GFS) [19].

Trong khi các công cụ Unix dùng stdin và stdout làm đầu vào và đầu ra, các tác vụ MapReduce đọc và ghi tệp trên một hệ thống tệp phân tán. Trong bản hiện thực MapReduce của Hadoop, hệ thống tệp đó được gọi là HDFS (Hadoop Distributed File System), một bản hiện thực lại mã nguồn mở của Google File System (GFS) [19].

Various other distributed filesystems besides HDFS exist, such as GlusterFS and the Quantcast File System (QFS) [20]. Object storage services such as Amazon S3, Azure Blob Storage, and OpenStack Swift [21] are similar in many ways. In this chapter we will mostly use HDFS as a running example, but the principles apply to any distributed filesystem.

Có nhiều hệ thống tệp phân tán khác ngoài HDFS, chẳng hạn GlusterFS và Quantcast File System (QFS) [20]. Các dịch vụ lưu trữ đối tượng như Amazon S3, Azure Blob Storage, và OpenStack Swift [21] tương tự về nhiều mặt. Trong chương này, ta sẽ chủ yếu dùng HDFS làm ví dụ xuyên suốt, nhưng các nguyên lý áp dụng cho bất kỳ hệ thống tệp phân tán nào.

HDFS is based on the shared-nothing principle (see the introduction to Part II), in contrast to the shared-disk approach of Network Attached Storage (NAS) and Storage Area Network (SAN) architectures. Shared-disk storage is implemented by a centralized storage appliance, often using custom hardware and special network infrastructure such as Fibre Channel. On the other hand, the shared-nothing approach requires no special hardware, only computers connected by a conventional **datacenter** network.

HDFS dựa trên nguyên lý shared-nothing (xem phần giới thiệu của Phần II), trái ngược với cách tiếp cận shared-disk của các kiến trúc Network Attached Storage (NAS) và Storage Area Network (SAN). Lưu trữ shared-disk được hiện thực bằng một thiết bị lưu trữ tập trung, thường dùng phần cứng tùy chỉnh và hạ tầng mạng đặc biệt như Fibre Channel. Mặt khác, cách tiếp cận shared-nothing không đòi hỏi phần cứng đặc biệt nào, chỉ cần các máy tính được kết nối bởi một mạng trung tâm dữ liệu thông thường.

HDFS consists of a daemon process running on each machine, exposing a network service that allows other nodes to access files stored on that machine (assuming that every general-purpose machine in a datacenter has some disks attached to it). A central server called the NameNode keeps track of which file blocks are stored on which machine. Thus, HDFS conceptually creates one big filesystem that can use the space on the disks of all machines running the daemon.

HDFS gồm một tiến trình daemon chạy trên mỗi máy, phơi bày một dịch vụ mạng cho phép các nút khác truy cập các tệp được lưu trên máy đó (giả định rằng mỗi máy đa dụng trong một trung tâm dữ liệu có một số đĩa gắn vào nó). Một máy chủ trung tâm gọi là NameNode theo dõi xem khối tệp nào được lưu trên máy nào. Như vậy, về mặt khái niệm HDFS tạo ra một hệ thống tệp lớn duy nhất có thể dùng không gian trên các đĩa của tất cả các máy đang chạy daemon.

In order to tolerate machine and disk failures, file blocks are replicated on multiple machines. Replication may **mean** simply several copies of the same data on multiple machines, as in Chapter 5, or an erasure coding scheme such as Reed–Solomon codes, which allows lost data to be recovered with lower storage overhead than full replication [20, 22]. The techniques are similar to **RAID**, which provides **redundancy** across several disks attached to the same machine; the difference is that in a distributed filesystem, file access and replication are done over a conventional datacenter network without special hardware.

Để chịu được hỏng hóc của máy và đĩa, các khối tệp được nhân bản trên nhiều máy. Việc nhân bản có thể đơn giản là vài bản sao của cùng dữ liệu trên nhiều máy, như ở Chương 5, hoặc một lược đồ mã hóa xóa (erasure coding) chẳng hạn mã Reed–Solomon, vốn cho phép khôi phục dữ liệu bị mất với chi phí lưu trữ thấp hơn so với nhân bản toàn phần [20, 22]. Các kỹ thuật này tương tự với RAID, vốn cung cấp sự dự phòng trên vài đĩa gắn vào cùng một máy; khác biệt là trong một hệ thống tệp phân tán, việc truy cập tệp và nhân bản được thực hiện qua một mạng trung tâm dữ liệu thông thường mà không cần phần cứng đặc biệt.

HDFS has scaled well: at the time of writing, the biggest HDFS deployments run on tens of thousands of machines, with combined storage capacity of hundreds of petabytes [23]. Such large scale has become viable because the cost of data storage and access on HDFS, using commodity hardware and open source software, is much lower than that of the equivalent capacity on a dedicated storage appliance [24].

HDFS đã mở rộng tốt: tại thời điểm viết, các bản triển khai HDFS lớn nhất chạy trên hàng chục nghìn máy, với tổng dung lượng lưu trữ hàng trăm petabyte [23]. Quy mô lớn như vậy đã trở nên khả thi vì chi phí lưu trữ và truy cập dữ liệu trên HDFS, dùng phần cứng phổ thông và phần mềm mã nguồn mở, thấp hơn nhiều so với chi phí của dung lượng tương

đương trên một thiết bị lưu trữ chuyên dụng [24].

MapReduce Job Execution — Thực thi tác vụ MapReduce

MapReduce is a programming framework with which you can write code to process large datasets in a distributed filesystem like HDFS. The easiest way of understanding it is by referring back to the web server log analysis example in “Simple Log Analysis”. The pattern of data processing in MapReduce is very similar to this example:

MapReduce là một framework lập trình mà với nó bạn có thể viết mã để xử lý các tập dữ liệu lớn trong một hệ thống tệp phân tán như HDFS. Cách dễ nhất để hiểu nó là quay lại tham chiếu ví dụ phân tích nhật ký máy chủ web ở “Phân tích nhật ký đơn giản”. Mẫu xử lý dữ liệu trong MapReduce rất giống với ví dụ này:

- Read a set of input files, and break it up into records. In the web server log example, each record is one line in the log (that is, `\n` is the record separator).
- Call the mapper function to extract a key and value from each input record. In the preceding example, the mapper function is `awk '{print $7}'`: it extracts the URL (`$7`) as the key, and leaves the value empty.
- Sort all of the key-value pairs by key. In the log example, this is done by the first sort command.
- Call the reducer function to iterate over the sorted key-value pairs. If there are multiple occurrences of the same key, the sorting has made them adjacent in the list, so it is easy to combine those values without having to keep a lot of state in memory. In the preceding example, the reducer is implemented by the command `uniq -c`, which counts the number of adjacent records with the same key.
 - Đọc một tập các tệp đầu vào, và tách nó thành các bản ghi. Trong ví dụ nhật ký máy chủ web, mỗi bản ghi là một dòng trong nhật ký (tức là `\n` là ký tự phân tách bản ghi).
 - Gọi hàm mapper để trích xuất một khóa và một giá trị từ mỗi bản ghi đầu vào. Trong ví dụ phía trên, hàm mapper là `awk '{print $7}'`: nó trích xuất URL (`$7`) làm khóa, và để giá trị trống.
 - Sắp xếp tất cả các cặp khóa-giá trị theo khóa. Trong ví dụ nhật ký, điều này được thực hiện bởi lệnh `sort` đầu tiên.
 - Gọi hàm reducer để duyệt qua các cặp khóa-giá trị đã sắp xếp. Nếu có nhiều lần xuất hiện của cùng một khóa, việc sắp xếp đã khiến chúng nằm liền kề nhau trong danh sách, nên dễ dàng kết hợp các giá trị đó mà không cần giữ nhiều trạng thái trong bộ nhớ. Trong ví dụ phía trên, reducer được thực hiện bởi lệnh `uniq -c`, vốn đếm số bản ghi liền kề có cùng khóa.

Those four steps can be performed by one MapReduce job. Steps 2 (map) and 4 (reduce) are where you write your custom data processing code. Step 1 (breaking files into records) is handled by the input format parser. Step 3, the sort step, is implicit in MapReduce—you don’t have to write it, because the output from the mapper is always sorted before it is given to the reducer.

Bốn bước đó có thể được thực hiện bởi một tác vụ MapReduce. Bước 2 (map) và bước 4 (reduce) là nơi bạn viết mã xử lý dữ liệu tùy chỉnh của mình. Bước 1 (tách tệp thành các bản ghi) được xử lý bởi bộ phân tích định dạng đầu vào. Bước 3, bước sắp xếp, là ngầm định trong MapReduce — bạn không phải viết nó, vì đầu ra từ mapper luôn được sắp xếp trước khi được đưa cho reducer.

To create a MapReduce job, you need to implement two callback functions, the mapper and reducer, which behave as follows (see also “MapReduce Querying”):

Để tạo một tác vụ MapReduce, bạn cần hiện thực hai hàm callback, mapper và reducer, vốn hành xử như sau (xem thêm “Truy vấn bằng MapReduce”):

The mapper is called once for every input record, and its job is to extract the key and value from the input record. For each input, it may generate any number of key-value pairs (including none). It does not keep any state from one input record to the next, so each record is handled independently.

Mapper được gọi một lần cho mỗi bản ghi đầu vào, và việc của nó là trích xuất khóa và giá trị từ bản ghi đầu vào. Với mỗi đầu vào, nó có thể sinh ra số lượng cặp khóa-giá trị bất kỳ (kể cả không cặp nào). Nó không giữ bất kỳ trạng thái nào từ bản ghi đầu vào này sang bản ghi tiếp theo, nên mỗi bản ghi được xử lý độc lập.

The MapReduce framework takes the key-value pairs produced by the mappers, collects all the values belonging to the same key, and calls the reducer with an iterator over that collection of values. The reducer can produce output records (such as the number of occurrences of the same URL).

Framework MapReduce nhận các cặp khóa-giá trị do các mapper tạo ra, gom tất cả các giá trị thuộc cùng một khóa, và gọi reducer với một bộ lặp (iterator) trên tập hợp các giá trị đó. Reducer có thể tạo ra các bản ghi đầu ra (chẳng hạn số lần xuất hiện của cùng một URL).

In the web server log example, we had a second sort command in step 5, which ranked URLs by number of requests. In MapReduce, if you need a second sorting stage, you can implement it by writing a second MapReduce job and using the output of the first job as input to the second job. Viewed like this, the role of the mapper is to prepare the data by putting it into a form that is suitable for sorting, and the role of the reducer is to process the data that has been sorted.

Trong ví dụ nhật ký máy chủ web, ta có một lệnh sort thứ hai ở bước 5, vốn xếp hạng các URL theo số lần yêu cầu. Trong MapReduce, nếu bạn cần một giai đoạn sắp xếp thứ hai, bạn có thể hiện thực nó bằng cách viết một tác vụ MapReduce thứ hai và dùng đầu ra của tác vụ thứ nhất làm đầu vào cho tác vụ thứ hai. Nhìn theo cách này, vai trò của mapper là chuẩn bị dữ liệu bằng cách đưa nó vào một dạng phù hợp để sắp xếp, còn vai trò của reducer là xử lý dữ liệu đã được sắp xếp.

Distributed execution of MapReduce — Thực thi phân tán của MapReduce

The main difference from pipelines of Unix commands is that MapReduce can parallelize a computation across many machines, without you having to write code to explicitly handle the parallelism. The mapper and reducer only operate on one record at a time; they don’t need to know where their input is coming from or their output is going to, so the framework can handle the complexities of moving data between machines.

Khác biệt chính so với các pipeline lệnh Unix là MapReduce có thể song song hóa một phép tính trên nhiều máy, mà không cần bạn viết mã để xử lý sự song song một cách tường minh. Mapper và reducer chỉ vận hành trên một bản ghi tại một thời điểm; chúng không cần biết đầu vào của chúng đến từ đâu hay đầu ra của chúng đi tới đâu, nên framework có thể xử lý sự phức tạp của việc di chuyển dữ liệu giữa các máy.

It is possible to use standard Unix tools as mappers and reducers in a distributed computation [25], but more commonly they are implemented as functions in a conventional programming language. In Hadoop MapReduce, the mapper and reducer are each a Java class that implements a particular interface. In MongoDB and CouchDB, mappers and reducers are JavaScript functions (see “MapReduce Querying”).

Có thể dùng các công cụ Unix tiêu chuẩn làm mapper và reducer trong một phép tính phân tán [25], nhưng phổ biến hơn là chúng được hiện thực thành các hàm trong một ngôn ngữ lập trình thông thường. Trong Hadoop MapReduce, mapper và reducer mỗi cái là một lớp Java hiện thực một giao diện cụ thể. Trong MongoDB và CouchDB, mapper và reducer là các hàm JavaScript (xem “Truy vấn bằng MapReduce”).

Figure 10-1 shows the dataflow in a Hadoop MapReduce job. Its parallelization is based on partitioning (see Chapter 6): the input to a job is typically a directory in HDFS, and each file or file block within the input directory is considered to be a separate partition that can be processed by a separate map task (marked by m 1, m 2, and m 3 in Figure 10-1).

Hình 10-1 cho thấy luồng dữ liệu trong một tác vụ Hadoop MapReduce. Sự song song hóa của nó dựa trên việc phân vùng (partitioning) (xem Chương 6): đầu vào cho một tác vụ thường là một thư mục trong HDFS, và mỗi tệp hoặc khối tệp trong thư mục đầu vào được coi là một phân vùng riêng biệt có thể được xử lý bởi một tác vụ map riêng biệt (được đánh dấu bằng m 1, m 2, và m 3 trong Hình 10-1).

Each input file is typically hundreds of megabytes in size. The MapReduce scheduler (not shown in the diagram) tries to run each mapper on one of the machines that stores a replica of the input file, provided that machine has enough spare RAM and CPU resources to run the map task [26]. This principle is known as putting the computation near the data [27]: it saves copying the input file over the network, reducing network **load** and increasing **locality**.

Mỗi tệp đầu vào thường có kích thước hàng trăm megabyte. Bộ lập lịch MapReduce (không hiển thị trong sơ đồ) cố gắng chạy mỗi mapper trên một trong các máy lưu một bản sao của tệp đầu vào, miễn là máy đó có đủ tài nguyên RAM và CPU dư để chạy tác vụ map [26]. Nguyên lý này được biết đến là đặt phép tính gần dữ liệu [27]: nó tiết kiệm việc sao chép tệp đầu vào qua mạng, giảm tải mạng và tăng tính cục bộ.

Figure 10-1. A MapReduce job with three mappers and three reducers. — Hình 10-1. Một tác vụ MapReduce với ba mapper và ba reducer. In most cases, the **application code** that should run in the map task is not yet present on the machine that is assigned the task of running it, so the MapReduce framework first copies the code (e.g., JAR files in the case of a Java program) to the appropriate machines. It then starts the map task and begins reading the input file, passing one record at a time to the mapper callback. The output of the mapper consists of key-value pairs.

Trong hầu hết trường hợp, mã ứng dụng cần chạy trong tác vụ map chưa có sẵn trên máy được giao nhiệm vụ chạy nó, nên framework MapReduce trước tiên sao chép mã (ví dụ, các tệp JAR trong trường hợp một chương trình Java) tới các máy phù hợp. Sau đó nó khởi động tác vụ map và bắt đầu đọc tệp đầu vào, chuyển một bản ghi mỗi lần tới callback mapper. Đầu ra của mapper gồm các cặp khóa-giá trị.

The reduce side of the computation is also partitioned. While the number of map tasks is determined by the number of input file blocks, the number of reduce tasks is configured by the job author (it can be different from the number of map tasks). To ensure that all key-value pairs with the same key end up at the same reducer, the framework uses a hash of the key to determine which reduce task should receive a particular key-value pair (see “Partitioning by Hash of Key”).

Phía reduce của phép tính cũng được phân vùng. Trong khi số lượng tác vụ map được xác định bởi số khối tệp đầu vào, thì số lượng tác vụ reduce được cấu hình bởi tác giả tác vụ (nó có thể khác với số lượng tác vụ map). Để đảm bảo tất cả các cặp khóa-giá trị có cùng khóa kết thúc tại cùng một reducer, framework dùng một hàm băm của khóa để xác định tác vụ reduce nào nên nhận một cặp khóa-giá trị cụ thể (xem “Phân vùng theo hàm băm của khóa”).

The key-value pairs must be sorted, but the dataset is likely too large to be sorted with a conventional sorting algorithm on a single machine. Instead, the sorting is performed in stages. First, each map task partitions its output by reducer, based on the hash of the key. Each of these partitions is written to a sorted file on the mapper's local disk, using a technique similar to what we discussed in “SSTables and LSM-Trees”.

Các cặp khóa-giá trị phải được sắp xếp, nhưng tập dữ liệu nhiều khả năng quá lớn để được sắp xếp bằng một thuật toán sắp xếp thông thường trên một máy duy nhất. Thay vào đó, việc sắp xếp được thực hiện theo từng giai đoạn. Trước tiên, mỗi tác vụ map phân vùng đầu ra của nó theo reducer, dựa trên hàm băm của khóa. Mỗi phân vùng này được ghi ra một tệp đã sắp xếp trên đĩa cục bộ của mapper, dùng một kỹ thuật tương tự với cái mà ta đã bàn ở “SSTable và LSM-Tree”.

Whenever a mapper finishes reading its input file and writing its sorted output files, the MapReduce scheduler notifies the reducers that they can start fetching the output files from that mapper. The reducers connect to each of the mappers and download the files of sorted key-value pairs for their partition. The process of partitioning by reducer, sorting, and copying data partitions from mappers to reducers is known as the shuffle [26] (a confusing term—unlike shuffling a deck of cards, there is no randomness in MapReduce).

Mỗi khi một mapper đọc xong tệp đầu vào của nó và ghi xong các tệp đầu ra đã sắp xếp, bộ lập lịch MapReduce thông báo cho các reducer rằng chúng có thể bắt đầu lấy các tệp đầu ra từ mapper đó. Các reducer kết nối tới mỗi mapper và tải xuống các tệp chứa các cặp khóa-giá trị đã sắp xếp cho phân vùng của chúng. Quá trình phân vùng theo reducer, sắp xếp, và sao chép các phân vùng dữ liệu từ mapper tới reducer được biết đến là shuffle [26] (một thuật ngữ gây nhầm lẫn — không giống như xáo một bộ bài, không có sự ngẫu nhiên nào trong MapReduce).

The reduce task takes the files from the mappers and merges them together, preserving the sort order. Thus, if different mappers produced records with the same key, they will be adjacent in the merged reducer input.

Tác vụ reduce nhận các tệp từ các mapper và trộn chúng lại với nhau, giữ nguyên thứ tự sắp xếp. Như vậy, nếu các mapper khác nhau tạo ra các bản ghi có cùng khóa, chúng sẽ nằm liền kề nhau trong đầu vào đã trộn của reducer.

The reducer is called with a key and an iterator that incrementally scans over all records with the same key (which may in some cases not all fit in memory). The reducer can use arbitrary logic to process these records, and can generate any number of output records. These output records are written to a file on the distributed filesystem (usually, one copy on the local disk of the machine running the reducer, with replicas on other machines).

Reducer được gọi với một khóa và một bộ lặp quét dần qua tất cả các bản ghi có cùng khóa (trong một số trường hợp có thể không nhét hết trong bộ nhớ). Reducer có thể dùng logic tùy ý để xử lý các bản ghi này, và có thể sinh ra số lượng bản ghi đầu ra bất kỳ. Các bản ghi đầu ra này được ghi ra một tệp trên hệ thống tệp phân tán (thường là một bản sao trên đĩa cục bộ của máy đang chạy reducer, với các bản sao trên các máy khác).

MapReduce workflows — Luồng công việc MapReduce

The range of problems you can solve with a single MapReduce job is limited. Referring back to the log analysis example, a single MapReduce job could determine the number of page views per URL, but not the most popular URLs, since that requires a second round of sorting.

Phạm vi các bài toán bạn có thể giải bằng một tác vụ MapReduce đơn lẻ là có giới hạn. Quay lại tham chiếu ví dụ phân tích nhật ký, một tác vụ MapReduce đơn lẻ có thể xác định số lượt xem trang cho mỗi URL, nhưng không xác định được các URL phổ biến nhất, vì điều đó đòi hỏi một vòng sắp xếp thứ hai.

Thus, it is very common for MapReduce jobs to be chained together into workflows, such that the output of one job becomes the input to the next job. The Hadoop MapReduce framework does not have any particular support for workflows, so this chaining is done implicitly by directory name: the first job must be configured to write its output to a designated directory in HDFS, and the second job must be configured to read that same directory name as its input. From the MapReduce framework's point of view, they are two independent jobs.

Vì vậy, rất phổ biến chuyển các tác vụ MapReduce được nối lại với nhau thành các luồng công việc (workflow), sao cho đầu ra của một tác vụ trở thành đầu vào của tác vụ tiếp theo. Framework Hadoop MapReduce không có bất kỳ sự hỗ trợ cụ thể nào cho các luồng công việc, nên việc nối này được thực hiện ngầm định bằng tên thư mục: tác vụ thứ nhất phải được cấu hình để ghi đầu ra của nó vào một thư mục được chỉ định trong HDFS, và tác vụ thứ hai phải được cấu hình để đọc chính tên thư mục đó làm đầu vào của nó. Từ góc nhìn của framework MapReduce, chúng là hai tác vụ độc lập.

Chained MapReduce jobs are therefore less like pipelines of Unix commands (which pass the output of one process as input to another process directly, using only a small in-memory buffer) and more like a sequence of commands where each command's output is written to a temporary file, and the next command reads from the temporary file. This design has advantages and disadvantages, which we will discuss in “Materialization of Intermediate State”.

Các tác vụ MapReduce được nối với nhau do đó ít giống các pipeline lệnh Unix hơn (vốn chuyển đầu ra của một tiến trình làm đầu vào của một tiến trình khác một cách trực tiếp, chỉ dùng một bộ đệm nhỏ trong bộ nhớ) và giống hơn với một chuỗi lệnh trong đó đầu ra của mỗi lệnh được ghi ra một tệp tạm, và lệnh tiếp theo đọc từ tệp tạm đó. Thiết kế này có ưu điểm và nhược điểm, mà ta sẽ bàn ở “Vật chất hóa trạng thái trung gian”.

A batch job's output is only considered valid when the job has completed successfully (MapReduce discards the partial output of a failed job). Therefore, one job in a workflow can only start when the prior jobs—that is, the jobs that produce its input directories—have completed successfully. To handle these dependencies between job executions, various workflow schedulers for Hadoop have been developed, including Oozie, Azkaban, Luigi, Airflow, and Pinball [28].

Đầu ra của một tác vụ theo lô chỉ được coi là hợp lệ khi tác vụ đã hoàn thành thành công (MapReduce loại bỏ đầu ra một phần của một tác vụ bị hỏng). Do đó, một tác vụ trong một luồng công việc chỉ có thể bắt đầu khi các tác vụ trước đó — tức là các tác vụ tạo ra các thư mục đầu vào của nó — đã hoàn thành thành công. Để xử lý những phụ thuộc giữa các lần thực thi tác vụ này, nhiều bộ lập lịch luồng công việc cho Hadoop đã được phát triển, bao gồm Oozie, Azkaban, Luigi, Airflow, và Pinball [28].

These schedulers also have management features that are useful when maintaining a large collection of batch jobs. Workflows consisting of 50 to 100 MapReduce jobs are common when building recommendation systems [29], and in a large organization, many different teams may be running different jobs that read each other's output. Tool support is important for managing such complex dataflows.

Các bộ lập lịch này cũng có các tính năng quản lý hữu ích khi bảo trì một tập hợp lớn các tác vụ theo lô. Các luồng công việc gồm 50 đến 100 tác vụ MapReduce là phổ biến khi xây dựng các hệ thống gợi ý [29], và trong một tổ chức lớn, nhiều nhóm khác nhau có thể đang

chạy các tác vụ khác nhau vốn đọc đầu ra của nhau. Sự hỗ trợ từ công cụ là quan trọng để quản lý những luồng dữ liệu phức tạp như vậy.

Various higher-level tools for Hadoop, such as Pig [30], Hive [31], Cascading [32], Crunch [33], and FlumeJava [34], also set up workflows of multiple MapReduce stages that are automatically wired together appropriately.

Nhiều công cụ ở mức cao hơn cho Hadoop, chẳng hạn Pig [30], Hive [31], Cascading [32], Crunch [33], và FlumeJava [34], cũng thiết lập các luồng công việc gồm nhiều giai đoạn MapReduce vốn được tự động đấu nối với nhau một cách phù hợp.

Reduce-Side Joins and Grouping — Phép join và phép gom nhóm phía reduce

We discussed joins in Chapter 2 in the context of data models and query languages, but we have not delved into how joins are actually implemented. It is time that we pick up that thread again.

Ta đã bàn về phép join ở Chương 2 trong ngữ cảnh các mô hình dữ liệu và ngôn ngữ truy vấn, nhưng ta chưa đi sâu vào việc phép join thực sự được hiện thực như thế nào. Đã đến lúc ta nối lại mạch đó.

In many datasets it is common for one record to have an association with another record: a **foreign key** in a **relational model**, a **document reference** in a **document model**, or an **edge** in a **graph model**. A **join** is necessary whenever you have some code that needs to access records on both sides of that association (both the record that holds the reference and the record being referenced). As discussed in Chapter 2, **denormalization** can reduce the need for joins but generally not remove it entirely.

Trong nhiều tập dữ liệu, việc một bản ghi có một liên kết với một bản ghi khác là phổ biến: một khóa ngoại trong mô hình quan hệ, một tham chiếu tài liệu trong mô hình tài liệu, hay một cạnh trong mô hình đồ thị. Một phép join là cần thiết bất cứ khi nào bạn có một đoạn mã cần truy cập các bản ghi ở cả hai phía của liên kết đó (cả bản ghi giữ tham chiếu lẫn bản ghi được tham chiếu). Như đã bàn ở Chương 2, phi chuẩn hóa có thể giảm nhu cầu join nhưng thường không loại bỏ nó hoàn toàn.

In a **database**, if you execute a query that involves only a small number of records, the database will typically use an index to quickly locate the records of interest (see Chapter 3). If the query involves joins, it may require multiple index lookups. However, MapReduce has no concept of indexes—at least not in the usual sense.

Trong một cơ sở dữ liệu, nếu bạn thực thi một truy vấn chỉ liên quan tới một số nhỏ bản ghi, cơ sở dữ liệu thường sẽ dùng một chỉ mục để nhanh chóng định vị các bản ghi cần quan tâm (xem Chương 3). Nếu truy vấn có liên quan tới phép join, nó có thể đòi hỏi nhiều lần tra cứu chỉ mục. Tuy nhiên, MapReduce không có khái niệm về chỉ mục — ít nhất là không theo nghĩa thông thường.

When a MapReduce job is given a set of files as input, it reads the entire content of all of those files; a database would call this operation a full table scan. If you only want to read a small number of records, a full table scan is outrageously expensive compared to an index lookup. However, in analytic queries (see “**Transaction Processing** or Analytics?”) it is common to want to calculate aggregates over a large number of records. In this case, scanning the entire input might be quite a reasonable thing to do, especially if you can parallelize the processing across multiple machines.

Khi một tác vụ MapReduce được cho một tập tệp làm đầu vào, nó đọc toàn bộ nội dung của tất cả các tệp đó; một cơ sở dữ liệu sẽ gọi thao tác này là một lần quét toàn bảng (full table scan). Nếu bạn chỉ muốn đọc một số nhỏ bản ghi, một lần quét toàn bảng đắt đỏ một cách quá đáng so với một lần tra cứu chỉ mục. Tuy nhiên, trong các truy vấn phân tích (xem “Xử lý giao dịch hay phân tích?”) thì việc muốn tính các giá trị tổng hợp trên một lượng lớn bản ghi là phổ biến. Trong trường hợp này, quét toàn bộ đầu vào có thể là một việc khá hợp lý để làm, đặc biệt nếu bạn có thể song song hóa việc xử lý trên nhiều máy.

When we talk about joins in the context of batch processing, we mean resolving all occurrences of some association within a dataset. For example, we assume that a job is processing the data for all users simultaneously, not merely looking up the data for one particular user (which would be done far more efficiently with an index).

Khi ta nói về phép join trong ngữ cảnh xử lý theo lô, ta muốn nói tới việc giải quyết tất cả các lần xuất hiện của một liên kết nào đó trong một tập dữ liệu. Ví dụ, ta giả định rằng một tác vụ đang xử lý dữ liệu cho tất cả người dùng cùng lúc, chứ không chỉ tra cứu dữ liệu cho một người dùng cụ thể nào đó (việc này sẽ được làm hiệu quả hơn nhiều bằng một chỉ mục).

Example: analysis of user activity events — Ví dụ: phân tích các sự kiện hoạt động của người dùng

A typical example of a join in a batch job is illustrated in Figure 10-2. On the left is a log of events describing the things that logged-in users did on a website (known as activity events or clickstream data), and on the right is a database of users. You can think of this example as being part of a star **schema** (see “Stars and Snowflakes: Schemas for Analytics”): the log of events is the fact table, and the user database is one of the dimensions.

Một ví dụ điển hình về một phép join trong một tác vụ theo lô được minh họa ở Hình 10-2. Bên trái là một nhật ký các sự kiện mô tả những việc mà người dùng đã đăng nhập đã làm trên một trang web (được biết đến là các sự kiện hoạt động hay dữ liệu luồng nhấp chuột - clickstream), và bên phải là một cơ sở dữ liệu người dùng. Bạn có thể nghĩ về ví dụ này như một phần của một lược đồ hình sao (star schema) (xem “Sao và Băng tuyết: Các lược đồ cho phân tích”): nhật ký các sự kiện là bảng sự kiện (fact table), còn cơ sở dữ liệu người dùng là một trong các chiều (dimension).

Figure 10-2. A join between a log of user activity events and a database of user profiles. — Hình 10-2. Một phép join giữa một nhật ký các sự kiện hoạt động của người dùng và một cơ sở dữ liệu các hồ sơ người dùng. An analytics task may need to correlate user activity with user profile information: for example, if the profile contains the user’s age or date of birth, the system could determine which pages are most popular with which age groups. However, the activity events contain only the user ID, not the full user profile information. Embedding that profile information in every single activity event would most likely be too wasteful. Therefore, the activity events need to be joined with the user profile database.

Một tác vụ phân tích có thể cần tương quan hoạt động của người dùng với thông tin hồ sơ người dùng: ví dụ, nếu hồ sơ chứa tuổi hoặc ngày sinh của người dùng, hệ thống có thể xác định những trang nào phổ biến nhất với những nhóm tuổi nào. Tuy nhiên, các sự kiện hoạt động chỉ chứa ID của người dùng, chứ không phải toàn bộ thông tin hồ sơ người dùng. Việc nhúng thông tin hồ sơ đó vào từng sự kiện hoạt động nhiều khả năng sẽ quá lãng phí. Do đó, các sự kiện hoạt động cần được join với cơ sở dữ liệu hồ sơ người dùng.

The simplest implementation of this join would go over the activity events one by one and query the user database (on a remote server) for every user ID it encounters. This is possible, but it would most likely suffer from very poor performance: the processing throughput would be limited by the round-trip time to the database server; the effectiveness of a local **cache** would depend very much on the distribution of data, and running a large number of queries in parallel could easily overwhelm the database [35].

Bản hiện thực đơn giản nhất của phép join này sẽ duyệt qua từng sự kiện hoạt động một và truy vấn cơ sở dữ liệu người dùng (trên một máy chủ từ xa) cho mỗi ID người dùng mà nó gặp. Điều này khả thi, nhưng nhiều khả năng nó sẽ chịu hiệu năng rất kém: thông lượng xử lý sẽ bị giới hạn bởi thời gian khứ hồi tới máy chủ cơ sở dữ liệu, hiệu quả của một bộ nhớ đệm cục bộ sẽ phụ thuộc rất nhiều vào sự phân bố của dữ liệu, và việc chạy một lượng lớn truy vấn song song có thể dễ dàng làm quá tải cơ sở dữ liệu [35].

In order to achieve good throughput in a batch process, the computation must be (as much as possible) local to one machine. Making random-access requests over the network for every record you want to process is too slow. Moreover, querying a remote database would mean that the batch job becomes nondeterministic, because the data in the remote database might change.

Để đạt được thông lượng tốt trong một tiến trình theo lô, phép tính phải (càng nhiều càng tốt) cục bộ trên một máy. Việc thực hiện các yêu cầu truy cập ngẫu nhiên qua mạng cho mỗi bản ghi bạn muốn xử lý là quá chậm. Hơn nữa, việc truy vấn một cơ sở dữ liệu từ xa sẽ nghĩa là tác vụ theo lô trở nên phi xác định (nondeterministic), bởi dữ liệu trong cơ sở dữ liệu từ xa có thể thay đổi.

Thus, a better approach would be to take a copy of the user database (for example, extracted from a database **backup** using an ETL process—see “Data Warehousing”) and to put it in the same distributed filesystem as the log of user activity events. You would then have the user database in one set of files in HDFS and the user activity records in another set of files, and could use MapReduce to bring together all of the relevant records in the same place and process them efficiently.

Vì vậy, một cách tiếp cận tốt hơn sẽ là lấy một bản sao của cơ sở dữ liệu người dùng (ví dụ, được trích xuất từ một bản sao lưu cơ sở dữ liệu bằng một tiến trình ETL — xem “Kho dữ liệu”) và đặt nó vào cùng hệ thống tệp phân tán với nhật ký các sự kiện hoạt động của người dùng. Khi đó bạn sẽ có cơ sở dữ liệu người dùng trong một tập tệp trong HDFS và các bản ghi hoạt động của người dùng trong một tập tệp khác, và có thể dùng MapReduce để gom tất cả các bản ghi liên quan vào cùng một chỗ và xử lý chúng một cách hiệu quả.

Sort-merge joins — Phép join sắp-xếp-trộn

Recall that the purpose of the mapper is to extract a key and value from each input record. In the case of Figure 10-2, this key would be the user ID: one set of mappers would go over the activity events (extracting the user ID as the key and the activity event as the value), while another set of mappers would go over the user database (extracting the user ID as the key and the user’s date of birth as the value). This process is illustrated in Figure 10-3.

Hãy nhớ rằng mục đích của mapper là trích xuất một khóa và một giá trị từ mỗi bản ghi đầu vào. Trong trường hợp của Hình 10-2, khóa này sẽ là ID người dùng: một tập mapper sẽ duyệt qua các sự kiện hoạt động (trích xuất ID người dùng làm khóa và sự kiện hoạt động làm giá trị), trong khi một tập mapper khác sẽ duyệt qua cơ sở dữ liệu người dùng (trích xuất ID người dùng làm khóa và ngày sinh của người dùng làm giá trị). Quá trình này được minh họa ở Hình 10-3.

Figure 10-3. A reduce-side sort-merge join on user ID. If the input datasets are partitioned into multiple files, each could be processed with multiple mappers in parallel. — Hình 10-3. Một phép join sắp-xếp-trộn phía reduce theo ID người dùng. Nếu các tập dữ liệu đầu vào được phân vùng thành nhiều tệp, mỗi tệp có thể được xử lý với nhiều mapper song song. When the MapReduce framework partitions the mapper output by key and then sorts the key-value pairs, the effect is that all the activity events and the user record with the same user ID become adjacent to each other in the reducer input. The MapReduce job can even arrange the records to be sorted such that the reducer always sees the record from the user database first, followed by the activity events in timestamp order—this technique is known as a secondary sort [26].

Khi framework MapReduce phân vùng đầu ra của mapper theo khóa rồi sắp xếp các cặp khóa-giá trị, hiệu ứng là tất cả các sự kiện hoạt động và bản ghi người dùng có cùng ID người dùng trở nên nằm liền kề nhau trong đầu vào của reducer. Tác vụ MapReduce thậm chí có thể sắp đặt các bản ghi để được sắp xếp sao cho reducer luôn thấy bản ghi từ cơ sở dữ liệu người dùng trước, theo sau là các sự kiện hoạt động theo thứ tự dấu thời gian — kỹ thuật này được biết đến là sắp xếp thứ cấp (secondary sort) [26].

The reducer can then perform the actual join logic easily: the reducer function is called once for every user ID, and thanks to the secondary sort, the first value is expected to be the date-of-birth record from the user database. The reducer stores the date of birth in a local variable and then iterates over the activity events with the same user ID, outputting pairs of viewed-url and viewer-age-in-years. Subsequent MapReduce jobs could then calculate the distribution of viewer ages for each URL, and cluster by age group.

Khi đó reducer có thể thực hiện logic join thực sự một cách dễ dàng: hàm reducer được gọi một lần cho mỗi ID người dùng, và nhờ vào sắp xếp thứ cấp, giá trị đầu tiên được kỳ vọng là bản ghi ngày sinh từ cơ sở dữ liệu người dùng. Reducer lưu ngày sinh vào một biến cục bộ rồi duyệt qua các sự kiện hoạt động có cùng ID người dùng, xuất ra các cặp url-đã-xem và tuổi-người-xem-theo-năm. Các tác vụ MapReduce tiếp theo khi đó có thể tính sự phân bố tuổi người xem cho mỗi URL, và gom cụm theo nhóm tuổi.

Since the reducer processes all of the records for a particular user ID in one go, it only needs to keep one user record in memory at any one time, and it never needs to make any requests over the network. This algorithm is known as a sort-merge join, since mapper output is sorted by key, and the reducers then merge together the sorted lists of records from both sides of the join.

Vì reducer xử lý tất cả các bản ghi cho một ID người dùng cụ thể trong một lượt, nó chỉ cần giữ một bản ghi người dùng trong bộ nhớ tại bất kỳ thời điểm nào, và nó không bao giờ cần thực hiện bất kỳ yêu cầu nào qua mạng. Thuật toán này được biết đến là một phép join sắp-xếp-trộn (sort-merge join), vì đầu ra của mapper được sắp xếp theo khóa, và sau đó các reducer trộn các danh sách bản ghi đã sắp xếp từ cả hai phía của phép join lại với nhau.

Bringing related data together in the same place — Đưa dữ liệu liên quan về cùng một chỗ

In a sort-merge join, the mappers and the sorting process make sure that all the necessary data to perform the join operation for a particular user ID is brought together in the same place: a single call to the reducer. Having lined up all the required data in advance, the reducer can be a fairly simple, single-threaded piece of code that can churn through records with high throughput and low memory overhead.

Trong một phép join sắp-xếp-trộn, các mapper và quá trình sắp xếp đảm bảo rằng tất cả

dữ liệu cần thiết để thực hiện phép join cho một ID người dùng cụ thể được đưa về cùng một chỗ: một lần gọi duy nhất tới reducer. Đã xếp sẵn tất cả dữ liệu cần thiết từ trước, reducer có thể là một đoạn mã đơn luồng khá đơn giản, có thể nghiền qua các bản ghi với thông lượng cao và chi phí bộ nhớ thấp.

One way of looking at this architecture is that mappers “send messages” to the reducers. When a mapper emits a key-value pair, the key acts like the destination address to which the value should be delivered. Even though the key is just an arbitrary string (not an actual network address like an IP address and port number), it behaves like an address: all key-value pairs with the same key will be delivered to the same destination (a call to the reducer).

Một cách nhìn vào kiến trúc này là các mapper “gửi thông điệp” cho các reducer. Khi một mapper phát ra một cặp khóa-giá trị, khóa hoạt động như địa chỉ đích mà giá trị nên được chuyển tới. Mặc dù khóa chỉ là một chuỗi tùy ý (không phải một địa chỉ mạng thực sự như một địa chỉ IP và số cổng), nó hành xử như một địa chỉ: tất cả các cặp khóa-giá trị có cùng khóa sẽ được chuyển tới cùng một đích (một lần gọi tới reducer).

Using the MapReduce programming model has separated the physical network communication aspects of the computation (getting the data to the right machine) from the application logic (processing the data once you have it). This separation contrasts with the typical use of databases, where a request to fetch data from a database often occurs somewhere deep inside a piece of application code [36]. Since MapReduce handles all network communication, it also shields the application code from having to worry about partial failures, such as the crash of another **node**: MapReduce transparently retries failed tasks without affecting the application logic.

Việc dùng mô hình lập trình MapReduce đã tách khía cạnh giao tiếp mạng vật lý của phép tính (đưa dữ liệu tới đúng máy) ra khỏi logic ứng dụng (xử lý dữ liệu một khi bạn đã có nó). Sự tách biệt này tương phản với cách dùng cơ sở dữ liệu điển hình, trong đó một yêu cầu lấy dữ liệu từ một cơ sở dữ liệu thường xảy ra đâu đó sâu bên trong một đoạn mã ứng dụng [36]. Vì MapReduce xử lý tất cả giao tiếp mạng, nó cũng che chắn cho mã ứng dụng khỏi việc phải lo lắng về các hỏng hóc cục bộ (partial failure), chẳng hạn sự sụp đổ của một nút khác: MapReduce thử lại các tác vụ hỏng một cách trong suốt mà không ảnh hưởng tới logic ứng dụng.

GROUP BY — GROUP BY

Besides joins, another common use of the “bringing related data to the same place” pattern is grouping records by some key (as in the GROUP BY clause in **SQL**). All records with the same key form a group, and the next step is often to perform some kind of aggregation within each group—for example:

Ngoài phép join, một cách dùng phổ biến khác của mẫu “đưa dữ liệu liên quan về cùng một chỗ” là gom nhóm các bản ghi theo một khóa nào đó (như trong mệnh đề GROUP BY trong SQL). Tất cả các bản ghi có cùng khóa tạo thành một nhóm, và bước tiếp theo thường là thực hiện một dạng tổng hợp nào đó trong mỗi nhóm — ví dụ:

- Counting the number of records in each group (like in our example of counting page views, which you would express as a COUNT(*) aggregation in SQL)
- Adding up the values in one particular field (SUM(fieldname)) in SQL
- Picking the top k records according to some ranking function
 - Đếm số bản ghi trong mỗi nhóm (như trong ví dụ đếm lượt xem trang của ta, mà bạn sẽ diễn đạt thành một phép tổng hợp COUNT(*) trong SQL)
 - Cộng dồn các giá trị trong một trường cụ thể (SUM(fieldname)) trong SQL

- Chọn ra k bản ghi đứng đầu theo một hàm xếp hạng nào đó

The simplest way of implementing such a grouping operation with MapReduce is to set up the mappers so that the key-value pairs they produce use the desired grouping key. The partitioning and sorting process then brings together all the records with the same key in the same reducer. Thus, grouping and joining look quite similar when implemented on top of MapReduce.

Cách đơn giản nhất để hiện thực một thao tác gom nhóm như vậy với MapReduce là thiết lập các mapper sao cho các cặp khóa-giá trị chúng tạo ra dùng khóa gom nhóm mong muốn. Quá trình phân vùng và sắp xếp khi đó đưa tất cả các bản ghi có cùng khóa về cùng một reducer. Như vậy, gom nhóm và join trông khá giống nhau khi được hiện thực trên nền MapReduce.

Another common use for grouping is collating all the activity events for a particular user session, in order to find out the sequence of actions that the user took—a process called sessionization [37]. For example, such analysis could be used to work out whether users who were shown a new version of your website are more likely to make a purchase than those who were shown the old version (A/B testing), or to calculate whether some marketing activity is worthwhile.

Một cách dùng phổ biến khác cho việc gom nhóm là tập hợp tất cả các sự kiện hoạt động cho một phiên người dùng cụ thể, nhằm tìm ra trình tự các hành động mà người dùng đã thực hiện — một quá trình gọi là phân phiên (sessionization) [37]. Ví dụ, phép phân tích như vậy có thể được dùng để xác định liệu những người dùng được cho xem một phiên bản mới của trang web của bạn có nhiều khả năng thực hiện một giao dịch mua hơn những người được cho xem phiên bản cũ không (kiểm thử A/B), hoặc để tính xem một hoạt động tiếp thị nào đó có đáng giá không.

If you have multiple web servers handling user requests, the activity events for a particular user are most likely scattered across various different servers' log files. You can implement sessionization by using a session cookie, user ID, or similar identifier as the grouping key and bringing all the activity events for a particular user together in one place, while distributing different users' events across different partitions.

Nếu bạn có nhiều máy chủ web xử lý các yêu cầu của người dùng, các sự kiện hoạt động cho một người dùng cụ thể nhiều khả năng nằm rải rác trên các tệp nhật ký của nhiều máy chủ khác nhau. Bạn có thể hiện thực việc phân phiên bằng cách dùng một cookie phiên, ID người dùng, hoặc một định danh tương tự làm khóa gom nhóm và đưa tất cả các sự kiện hoạt động cho một người dùng cụ thể về một chỗ, đồng thời phân tán các sự kiện của những người dùng khác nhau trên các phân vùng khác nhau.

Handling skew — Xử lý lệch

The pattern of “bringing all records with the same key to the same place” breaks down if there is a very large amount of data related to a single key. For example, in a social network, most users might be connected to a few hundred people, but a small number of celebrities may have many millions of followers. Such disproportionately active database records are known as linchpin objects [38] or hot keys.

Mẫu “đưa tất cả các bản ghi có cùng khóa về cùng một chỗ” sụp đổ nếu có một lượng dữ liệu rất lớn liên quan tới một khóa duy nhất. Ví dụ, trong một mạng xã hội, hầu hết người dùng có thể được kết nối với vài trăm người, nhưng một số nhỏ người nổi tiếng có thể có hàng triệu người theo dõi. Những bản ghi cơ sở dữ liệu hoạt động không cân xứng như vậy được biết đến là các đối tượng then chốt (linchpin objects) [38] hay các khóa nóng (hot keys).

Collecting all activity related to a celebrity (e.g., replies to something they posted) in a single reducer can lead to significant skew (also known as hot spots)—that is, one reducer that must process significantly more records than the others (see “Skewed Workloads and Relieving Hot Spots”). Since a MapReduce job is only complete when all of its mappers and reducers have completed, any subsequent jobs must wait for the slowest reducer to complete before they can start.

Việc gom tất cả hoạt động liên quan tới một người nổi tiếng (ví dụ, các phản hồi cho một thứ họ đã đăng) trong một reducer duy nhất có thể dẫn tới sự lệch (skew) đáng kể (cũng được biết đến là điểm nóng - hot spots) — tức là một reducer phải xử lý nhiều bản ghi hơn hẳn so với những reducer khác (xem “Tải lệch và giảm nhẹ điểm nóng”). Vì một tác vụ MapReduce chỉ hoàn thành khi tất cả các mapper và reducer của nó đã hoàn thành, nên bất kỳ tác vụ tiếp theo nào cũng phải chờ reducer chậm nhất hoàn thành trước khi chúng có thể bắt đầu.

If a join input has hot keys, there are a few algorithms you can use to compensate. For example, the skewed join method in Pig first runs a sampling job to determine which keys are hot [39]. When performing the actual join, the mappers send any records relating to a hot key to one of several reducers, chosen at random (in contrast to conventional MapReduce, which chooses a reducer deterministically based on a hash of the key). For the other input to the join, records relating to the hot key need to be replicated to all reducers handling that key [40].

Nếu một đầu vào join có các khóa nóng, có một vài thuật toán bạn có thể dùng để bù trừ. Ví dụ, phương pháp join lệch (skewed join) trong Pig trước tiên chạy một tác vụ lấy mẫu để xác định những khóa nào là nóng [39]. Khi thực hiện phép join thực sự, các mapper gửi bất kỳ bản ghi nào liên quan tới một khóa nóng tới một trong vài reducer, được chọn ngẫu nhiên (trái ngược với MapReduce thông thường, vốn chọn một reducer một cách xác định dựa trên hàm băm của khóa). Đối với đầu vào còn lại của phép join, các bản ghi liên quan tới khóa nóng cần được nhân bản tới tất cả các reducer xử lý khóa đó [40].

This technique spreads the work of handling the hot key over several reducers, which allows it to be parallelized better, at the cost of having to replicate the other join input to multiple reducers. The sharded join method in Crunch is similar, but requires the hot keys to be specified explicitly rather than using a sampling job. This technique is also very similar to one we discussed in “Skewed Workloads and Relieving Hot Spots”, using randomization to alleviate hot spots in a partitioned database.

Kỹ thuật này trải công việc xử lý khóa nóng ra trên vài reducer, điều cho phép nó được song song hóa tốt hơn, với cái giá là phải nhân bản đầu vào join còn lại tới nhiều reducer. Phương pháp join phân mảnh (sharded join) trong Crunch thì tương tự, nhưng đòi hỏi các khóa nóng phải được chỉ định một cách tường minh thay vì dùng một tác vụ lấy mẫu. Kỹ thuật này cũng rất giống một kỹ thuật mà ta đã bàn ở “Tải lệch và giảm nhẹ điểm nóng”, dùng việc ngẫu nhiên hóa để giảm nhẹ các điểm nóng trong một cơ sở dữ liệu được phân vùng.

Hive’s skewed join optimization takes an alternative approach. It requires hot keys to be specified explicitly in the table metadata, and it stores records related to those keys in separate files from the rest. When performing a join on that table, it uses a map-side join (see the next section) for the hot keys.

Tối ưu hóa join lệch của Hive đi theo một cách tiếp cận khác. Nó đòi hỏi các khóa nóng phải được chỉ định một cách tường minh trong siêu dữ liệu của bảng, và nó lưu các bản ghi liên quan tới những khóa đó vào các tệp riêng tách khỏi phần còn lại. Khi thực hiện một phép join trên bảng đó, nó dùng một phép join phía map (xem phần tiếp theo) cho các khóa nóng.

When grouping records by a hot key and aggregating them, you can perform the grouping in two stages. The first MapReduce stage sends records to a random reducer, so that each reducer performs the grouping on a subset of records for the hot key and outputs a more compact aggregated value per key. The second MapReduce job then combines the values from all of the first-stage reducers into a single value per key.

Khi gom nhóm các bản ghi theo một khóa nóng và tổng hợp chúng, bạn có thể thực hiện việc gom nhóm theo hai giai đoạn. Giai đoạn MapReduce thứ nhất gửi các bản ghi tới một reducer ngẫu nhiên, sao cho mỗi reducer thực hiện việc gom nhóm trên một tập con các bản ghi cho khóa nóng và xuất ra một giá trị tổng hợp gọn hơn cho mỗi khóa. Tác vụ MapReduce thứ hai khi đó kết hợp các giá trị từ tất cả các reducer giai đoạn-một thành một giá trị duy nhất cho mỗi khóa.

Map-Side Joins — Phép join phía map

The join algorithms described in the last section perform the actual join logic in the reducers, and are hence known as reduce-side joins. The mappers take the role of preparing the input data: extracting the key and value from each input record, assigning the key-value pairs to a reducer partition, and sorting by key.

Các thuật toán join được mô tả trong phần trước thực hiện logic join thực sự trong các reducer, và do đó được biết đến là các phép join phía reduce (reduce-side joins). Các mapper đóng vai trò chuẩn bị dữ liệu đầu vào: trích xuất khóa và giá trị từ mỗi bản ghi đầu vào, gán các cặp khóa-giá trị cho một phân vùng reducer, và sắp xếp theo khóa.

The reduce-side approach has the advantage that you do not need to make any assumptions about the input data: whatever its properties and structure, the mappers can prepare the data to be ready for joining. However, the downside is that all that sorting, copying to reducers, and merging of reducer inputs can be quite expensive. Depending on the available memory buffers, data may be written to disk several times as it passes through the stages of MapReduce [37].

Cách tiếp cận phía reduce có ưu điểm là bạn không cần đưa ra bất kỳ giả định nào về dữ liệu đầu vào: bất kể thuộc tính và cấu trúc của nó là gì, các mapper có thể chuẩn bị dữ liệu sẵn sàng cho việc join. Tuy nhiên, nhược điểm là tất cả việc sắp xếp, sao chép tới các reducer, và trộn các đầu vào reducer đó có thể khá đắt đỏ. Tùy vào các bộ đệm bộ nhớ khả dụng, dữ liệu có thể bị ghi ra đĩa vài lần khi nó đi qua các giai đoạn của MapReduce [37].

On the other hand, if you can make certain assumptions about your input data, it is possible to make joins faster by using a so-called map-side join. This approach uses a cut-down MapReduce job in which there are no reducers and no sorting. Instead, each mapper simply reads one input file block from the distributed filesystem and writes one output file to the filesystem—that is all.

Mặt khác, nếu bạn có thể đưa ra một số giả định nhất định về dữ liệu đầu vào của mình, thì có thể làm cho phép join nhanh hơn bằng cách dùng một thứ gọi là phép join phía map (map-side join). Cách tiếp cận này dùng một tác vụ MapReduce rút gọn trong đó không có reducer và không có sắp xếp. Thay vào đó, mỗi mapper chỉ đơn giản đọc một khối tệp đầu vào từ hệ thống tệp phân tán và ghi một tệp đầu ra ra hệ thống tệp — chỉ vậy thôi.

Broadcast hash joins — Phép join băm quảng bá

The simplest way of performing a map-side join applies in the case where a large dataset is joined with a small dataset. In particular, the small dataset needs to be small enough that it can be loaded

entirely into memory in each of the mappers.

Cách đơn giản nhất để thực hiện một phép join phía map áp dụng cho trường hợp một tập dữ liệu lớn được join với một tập dữ liệu nhỏ. Cụ thể, tập dữ liệu nhỏ cần đủ nhỏ để có thể được nạp toàn bộ vào bộ nhớ trong mỗi mapper.

For example, imagine in the case of Figure 10-2 that the user database is small enough to fit in memory. In this case, when a mapper starts up, it can first read the user database from the distributed filesystem into an in-memory hash table. Once this is done, the mapper can scan over the user activity events and simply look up the user ID for each event in the hash table.

Ví dụ, hãy hình dung trong trường hợp của Hình 10-2 rằng cơ sở dữ liệu người dùng đủ nhỏ để nhét trong bộ nhớ. Trong trường hợp này, khi một mapper khởi động, nó có thể trước tiên đọc cơ sở dữ liệu người dùng từ hệ thống tệp phân tán vào một bảng băm trong bộ nhớ. Một khi điều này được làm xong, mapper có thể quét qua các sự kiện hoạt động của người dùng và chỉ đơn giản tra cứu ID người dùng cho mỗi sự kiện trong bảng băm.

There can still be several map tasks: one for each file block of the large input to the join (in the example of Figure 10-2, the activity events are the large input). Each of these mappers loads the small input entirely into memory.

Vẫn có thể có vài tác vụ map: một tác vụ cho mỗi khối tệp của đầu vào lớn của phép join (trong ví dụ của Hình 10-2, các sự kiện hoạt động là đầu vào lớn). Mỗi mapper trong số này nạp toàn bộ đầu vào nhỏ vào bộ nhớ.

This simple but effective algorithm is called a broadcast hash join: the word broadcast reflects the fact that each mapper for a partition of the large input reads the entirety of the small input (so the small input is effectively “broadcast” to all partitions of the large input), and the word hash reflects its use of a hash table. This join method is supported by Pig (under the name “replicated join”), Hive (“MapJoin”), Cascading, and Crunch. It is also used in data warehouse query engines such as Impala [41].

Thuật toán đơn giản nhưng hiệu quả này được gọi là phép join băm quảng bá (broadcast hash join): từ quảng bá (broadcast) phản ánh sự thật rằng mỗi mapper cho một phân vùng của đầu vào lớn đọc toàn bộ đầu vào nhỏ (nên đầu vào nhỏ thực chất được “quảng bá” tới tất cả các phân vùng của đầu vào lớn), còn từ băm (hash) phản ánh việc nó dùng một bảng băm. Phương pháp join này được hỗ trợ bởi Pig (dưới tên “replicated join”), Hive (“MapJoin”), Cascading, và Crunch. Nó cũng được dùng trong các engine truy vấn kho dữ liệu như Impala [41].

Instead of loading the small join input into an in-memory hash table, an alternative is to store the small join input in a read-only index on the local disk [42]. The frequently used parts of this index will remain in the operating system’s page cache, so this approach can provide random-access lookups almost as fast as an in-memory hash table, but without actually requiring the dataset to fit in memory.

Thay vì nạp đầu vào join nhỏ vào một bảng băm trong bộ nhớ, một cách khác là lưu đầu vào join nhỏ trong một chỉ mục chỉ-đọc trên đĩa cục bộ [42]. Các phần thường được dùng của chỉ mục này sẽ ở lại trong page cache của hệ điều hành, nên cách tiếp cận này có thể cung cấp các lần tra cứu truy cập ngẫu nhiên gần như nhanh bằng một bảng băm trong bộ nhớ, nhưng không thực sự đòi hỏi tập dữ liệu phải nhét vừa trong bộ nhớ.

Partitioned hash joins — Phép join băm phân vùng

If the inputs to the map-side join are partitioned in the same way, then the hash join approach can be applied to each partition independently. In the case of Figure 10-2, you might arrange for the activity

events and the user database to each be partitioned based on the last decimal digit of the user ID (so there are 10 partitions on either side). For example, mapper 3 first loads all users with an ID ending in 3 into a hash table, and then scans over all the activity events for each user whose ID ends in 3.

Nếu các đầu vào cho phép join phía map được phân vùng theo cùng một cách, thì cách tiếp cận join băm có thể được áp dụng cho mỗi phân vùng một cách độc lập. Trong trường hợp của Hình 10-2, bạn có thể sắp đặt cho các sự kiện hoạt động và cơ sở dữ liệu người dùng mỗi cái được phân vùng dựa trên chữ số thập phân cuối cùng của ID người dùng (nên có 10 phân vùng ở mỗi phía). Ví dụ, mapper 3 trước tiên nạp tất cả người dùng có ID kết thúc bằng 3 vào một bảng băm, rồi quét qua tất cả các sự kiện hoạt động cho mỗi người dùng có ID kết thúc bằng 3.

If the partitioning is done correctly, you can be sure that all the records you might want to join are located in the same numbered partition, and so it is sufficient for each mapper to only read one partition from each of the input datasets. This has the advantage that each mapper can load a smaller amount of data into its hash table.

Nếu việc phân vùng được làm đúng, bạn có thể chắc chắn rằng tất cả các bản ghi bạn có thể muốn join đều nằm trong phân vùng cùng số thứ tự, nên mỗi mapper chỉ cần đọc một phân vùng từ mỗi tập dữ liệu đầu vào là đủ. Điều này có ưu điểm là mỗi mapper có thể nạp một lượng dữ liệu nhỏ hơn vào bảng băm của nó.

This approach only works if both of the join's inputs have the same number of partitions, with records assigned to partitions based on the same key and the same hash function. If the inputs are generated by prior MapReduce jobs that already perform this grouping, then this can be a reasonable assumption to make.

Cách tiếp cận này chỉ hoạt động nếu cả hai đầu vào của phép join có cùng số phân vùng, với các bản ghi được gán cho các phân vùng dựa trên cùng một khóa và cùng một hàm băm. Nếu các đầu vào được tạo ra bởi các tác vụ MapReduce trước đó vốn đã thực hiện việc gom nhóm này, thì đây có thể là một giả định hợp lý để đưa ra.

Partitioned hash joins are known as bucketed map joins in Hive [37].

Phép join băm phân vùng được biết đến là phép bucketed map join trong Hive [37].

Map-side merge joins — Phép join trộn phía map

Another variant of a map-side join applies if the input datasets are not only partitioned in the same way, but also sorted based on the same key. In this case, it does not matter whether the inputs are small enough to fit in memory, because a mapper can perform the same merging operation that would normally be done by a reducer: reading both input files incrementally, in order of ascending key, and matching records with the same key.

Một biến thể khác của phép join phía map áp dụng nếu các tập dữ liệu đầu vào không chỉ được phân vùng theo cùng một cách, mà còn được sắp xếp dựa trên cùng một khóa. Trong trường hợp này, việc các đầu vào có đủ nhỏ để nhét vừa trong bộ nhớ hay không không quan trọng, bởi một mapper có thể thực hiện cùng thao tác trộn vốn thường được làm bởi một reducer: đọc cả hai tệp đầu vào một cách tăng dần, theo thứ tự khóa tăng dần, và khớp các bản ghi có cùng khóa.

If a map-side merge join is possible, it probably means that prior MapReduce jobs brought the input datasets into this partitioned and sorted form in the first place. In principle, this join could have been performed in the reduce stage of the prior job. However, it may still be appropriate to perform the

merge join in a separate map-only job, for example if the partitioned and sorted datasets are also needed for other purposes besides this particular join.

Nếu một phép join trộn phía map là khả thi, điều đó có lẽ có nghĩa là các tác vụ MapReduce trước đó đã đưa các tập dữ liệu đầu vào về dạng được phân vùng và sắp xếp này ngay từ đầu. Về nguyên tắc, phép join này lẽ ra có thể được thực hiện trong giai đoạn reduce của tác vụ trước đó. Tuy nhiên, vẫn có thể là phù hợp khi thực hiện phép join trộn trong một tác vụ chỉ-map riêng biệt, ví dụ nếu các tập dữ liệu đã được phân vùng và sắp xếp đó cũng cần cho các mục đích khác ngoài phép join cụ thể này.

MapReduce workflows with map-side joins — Luồng công việc MapReduce với các phép join phía map

When the output of a MapReduce join is consumed by downstream jobs, the choice of map-side or reduce-side join affects the structure of the output. The output of a reduce-side join is partitioned and sorted by the join key, whereas the output of a map-side join is partitioned and sorted in the same way as the large input (since one map task is started for each file block of the join's large input, regardless of whether a partitioned or broadcast join is used).

Khi đầu ra của một phép join MapReduce được tiêu thụ bởi các tác vụ phía sau, việc lựa chọn phép join phía map hay phía reduce ảnh hưởng tới cấu trúc của đầu ra. Đầu ra của một phép join phía reduce được phân vùng và sắp xếp theo khóa join, trong khi đầu ra của một phép join phía map được phân vùng và sắp xếp theo cùng cách với đầu vào lớn (vì một tác vụ map được khởi động cho mỗi khối tệp của đầu vào lớn của phép join, bất kể là dùng phép join phân vùng hay quảng bá).

As discussed, map-side joins also make more assumptions about the size, sorting, and partitioning of their input datasets. Knowing about the physical layout of datasets in the distributed filesystem becomes important when optimizing join strategies: it is not sufficient to just know the encoding format and the name of the directory in which the data is stored; you must also know the number of partitions and the keys by which the data is partitioned and sorted.

Như đã bàn, các phép join phía map cũng đưa ra nhiều giả định hơn về kích thước, sự sắp xếp, và sự phân vùng của các tập dữ liệu đầu vào của chúng. Việc biết về bố cục vật lý của các tập dữ liệu trong hệ thống tệp phân tán trở nên quan trọng khi tối ưu các chiến lược join: chỉ biết định dạng mã hóa và tên thư mục nơi dữ liệu được lưu là chưa đủ; bạn còn phải biết số phân vùng và các khóa mà theo đó dữ liệu được phân vùng và sắp xếp.

In the Hadoop ecosystem, this kind of metadata about the partitioning of datasets is often maintained in HCatalog and the Hive metastore [37].

Trong hệ sinh thái Hadoop, loại siêu dữ liệu này về sự phân vùng của các tập dữ liệu thường được duy trì trong HCatalog và Hive metastore [37].

The Output of Batch Workflows — Đầu ra của các luồng công việc theo lô

We have talked a lot about the various algorithms for implementing workflows of MapReduce jobs, but we neglected an important question: what is the result of all of that processing, once it is done? Why are we running all these jobs in the first place?

Ta đã bàn rất nhiều về các thuật toán khác nhau để hiện thực các luồng công việc gồm các tác vụ MapReduce, nhưng ta đã bỏ quên một câu hỏi quan trọng: kết quả của tất cả việc xử lý đó là gì, một khi nó đã xong? Tại sao ngay từ đầu ta lại chạy tất cả các tác vụ này?

In the case of database queries, we distinguished transaction processing (OLTP) purposes from analytic purposes (see “Transaction Processing or Analytics?”). We saw that OLTP queries generally look up a small number of records by key, using indexes, in order to present them to a user (for example, on a web page). On the other hand, analytic queries often scan over a large number of records, performing groupings and aggregations, and the output often has the form of a report: a graph showing the change in a metric over time, or the top 10 items according to some ranking, or a breakdown of some quantity into subcategories. The consumer of such a report is often an analyst or a manager who needs to make business decisions.

Trong trường hợp các truy vấn cơ sở dữ liệu, ta đã phân biệt các mục đích xử lý giao dịch (OLTP) với các mục đích phân tích (xem “Xử lý giao dịch hay phân tích?”). Ta đã thấy rằng các truy vấn OLTP thường tra cứu một số nhỏ bản ghi theo khóa, dùng các chỉ mục, nhằm trình bày chúng cho một người dùng (ví dụ, trên một trang web). Mặt khác, các truy vấn phân tích thường quét qua một lượng lớn bản ghi, thực hiện các phép gom nhóm và tổng hợp, và đầu ra thường có dạng một báo cáo: một đồ thị cho thấy sự thay đổi của một chỉ số theo thời gian, hoặc 10 mục đứng đầu theo một xếp hạng nào đó, hoặc một bảng phân tích một lượng nào đó thành các tiểu mục. Người tiêu thụ một báo cáo như vậy thường là một nhà phân tích hoặc một quản lý cần đưa ra các quyết định kinh doanh.

Where does batch processing fit in? It is not transaction processing, nor is it analytics. It is closer to analytics, in that a batch process typically scans over large portions of an input dataset. However, a workflow of MapReduce jobs is not the same as a SQL query used for analytic purposes (see “Comparing Hadoop to Distributed Databases”). The output of a batch process is often not a report, but some other kind of structure.

Xử lý theo lô khớp vào đâu? Nó không phải xử lý giao dịch, cũng không phải phân tích. Nó gần với phân tích hơn, ở chỗ một tiến trình theo lô thường quét qua các phần lớn của một tập dữ liệu đầu vào. Tuy nhiên, một luồng công việc gồm các tác vụ MapReduce thì không giống với một truy vấn SQL được dùng cho các mục đích phân tích (xem “So sánh Hadoop với các cơ sở dữ liệu phân tán”). Đầu ra của một tiến trình theo lô thường không phải một báo cáo, mà là một dạng cấu trúc khác.

Building search indexes — Xây dựng chỉ mục tìm kiếm

Google’s original use of MapReduce was to build indexes for its search **engine**, which was implemented as a workflow of 5 to 10 MapReduce jobs [1]. Although Google later moved away from using MapReduce for this purpose [43], it helps to understand MapReduce if you look at it through the lens of building a **search index**. (Even today, Hadoop MapReduce remains a good way of building indexes for Lucene/Solr [44].)

Cách dùng MapReduce ban đầu của Google là để xây dựng các chỉ mục cho engine tìm kiếm của nó, vốn được hiện thực thành một luồng công việc gồm 5 đến 10 tác vụ MapReduce [1]. Mặc dù về sau Google đã chuyển khỏi việc dùng MapReduce cho mục đích này [43], việc nhìn nó qua lăng kính xây dựng một chỉ mục tìm kiếm sẽ giúp hiểu MapReduce. (Ngay cả ngày nay, Hadoop MapReduce vẫn là một cách tốt để xây dựng các chỉ mục cho Lucene/Solr [44].)

We saw briefly in “**Full-text search** and fuzzy indexes” how a full-text search index such as Lucene works: it is a file (the term dictionary) in which you can efficiently look up a particular keyword

and find the list of all the **document** IDs containing that keyword (the postings list). This is a very simplified view of a search index—in reality it requires various additional data, in order to rank search results by relevance, correct misspellings, resolve synonyms, and so on—but the principle holds.

Ta đã thấy sơ qua ở “Tìm kiếm toàn văn và chỉ mục mờ” cách một chỉ mục tìm kiếm toàn văn như Lucene hoạt động: nó là một tệp (từ điển thuật ngữ - term dictionary) trong đó bạn có thể hiệu quả tra cứu một từ khóa cụ thể và tìm danh sách tất cả các ID tài liệu chứa từ khóa đó (danh sách postings - postings list). Đây là một cách nhìn rất đơn giản hóa về một chỉ mục tìm kiếm — trên thực tế nó đòi hỏi nhiều dữ liệu bổ sung khác nhau, nhằm xếp hạng kết quả tìm kiếm theo độ liên quan, sửa lỗi chính tả, giải quyết từ đồng nghĩa, và cứ thế — nhưng nguyên lý vẫn đúng.

If you need to perform a full-text search over a fixed set of documents, then a batch process is a very effective way of building the indexes: the mappers partition the set of documents as needed, each reducer builds the index for its partition, and the index files are written to the distributed filesystem. Building such document-partitioned indexes (see “Partitioning and Secondary Indexes”) parallelizes very well. Since querying a search index by keyword is a read-only operation, these index files are immutable once they have been created.

Nếu bạn cần thực hiện một tìm kiếm toàn văn trên một tập tài liệu cố định, thì một tiến trình theo lô là một cách rất hiệu quả để xây dựng các chỉ mục: các mapper phân vùng tập tài liệu khi cần, mỗi reducer xây dựng chỉ mục cho phân vùng của nó, và các tệp chỉ mục được ghi ra hệ thống tệp phân tán. Việc xây dựng các chỉ mục được phân vùng theo tài liệu (document-partitioned) như vậy (xem “Phân vùng và chỉ mục thứ cấp”) song song hóa rất tốt. Vì việc truy vấn một chỉ mục tìm kiếm theo từ khóa là một thao tác chỉ-đọc, các tệp chỉ mục này là bất biến một khi chúng đã được tạo ra.

If the indexed set of documents changes, one option is to periodically rerun the entire indexing workflow for the entire set of documents, and replace the previous index files wholesale with the new index files when it is done. This approach can be computationally expensive if only a small number of documents have changed, but it has the advantage that the indexing process is very easy to reason about: documents in, indexes out.

Nếu tập tài liệu được lập chỉ mục thay đổi, một lựa chọn là định kỳ chạy lại toàn bộ luồng công việc lập chỉ mục cho toàn bộ tập tài liệu, và thay thế trọn gói các tệp chỉ mục trước đó bằng các tệp chỉ mục mới khi nó xong. Cách tiếp cận này có thể tốn kém về mặt tính toán nếu chỉ một số nhỏ tài liệu đã thay đổi, nhưng nó có ưu điểm là tiến trình lập chỉ mục rất dễ suy luận: tài liệu vào, chỉ mục ra.

Alternatively, it is possible to build indexes incrementally. As discussed in Chapter 3, if you want to add, remove, or update documents in an index, Lucene writes out new segment files and asynchronously merges and compacts segment files in the background. We will see more on such incremental processing in Chapter 11.

Một cách khác, có thể xây dựng các chỉ mục một cách tăng dần. Như đã bàn ở Chương 3, nếu bạn muốn thêm, xóa, hoặc cập nhật các tài liệu trong một chỉ mục, Lucene ghi ra các tệp phân đoạn mới và trộn cùng nén các tệp phân đoạn một cách bất đồng bộ ở chế độ nền. Ta sẽ thấy thêm về việc xử lý tăng dần như vậy ở Chương 11.

Key-value stores as batch process output — Kho khóa-giá trị làm đầu ra của tiến trình theo lô

Search indexes are just one example of the possible outputs of a batch processing workflow. Another common use for batch processing is to build machine learning systems such as classifiers (e.g., spam filters, anomaly detection, image recognition) and recommendation systems (e.g., people you may know, products you may be interested in, or related searches [29]).

Chỉ mục tìm kiếm chỉ là một ví dụ về các đầu ra khả dĩ của một luồng công việc xử lý theo lô. Một cách dùng phổ biến khác cho xử lý theo lô là xây dựng các hệ thống học máy chẳng hạn các bộ phân loại (ví dụ, bộ lọc thư rác, phát hiện bất thường, nhận dạng hình ảnh) và các hệ thống gợi ý (ví dụ, những người bạn có thể biết, những sản phẩm bạn có thể quan tâm, hay các tìm kiếm liên quan [29]).

The output of those batch jobs is often some kind of database: for example, a database that can be queried by user ID to obtain suggested friends for that user, or a database that can be queried by product ID to get a list of related products [45].

Đầu ra của những tác vụ theo lô đó thường là một dạng cơ sở dữ liệu nào đó: ví dụ, một cơ sở dữ liệu có thể được truy vấn theo ID người dùng để lấy các gợi ý bạn bè cho người dùng đó, hoặc một cơ sở dữ liệu có thể được truy vấn theo ID sản phẩm để lấy một danh sách các sản phẩm liên quan [45].

These databases need to be queried from the web application that handles user requests, which is usually separate from the Hadoop infrastructure. So how does the output from the batch process get back into a database where the web application can query it?

Các cơ sở dữ liệu này cần được truy vấn từ ứng dụng web vốn xử lý các yêu cầu của người dùng, và ứng dụng đó thường tách biệt với hạ tầng Hadoop. Vậy làm thế nào để đầu ra từ tiến trình theo lô quay trở lại vào một cơ sở dữ liệu nơi ứng dụng web có thể truy vấn nó?

The most obvious choice might be to use the client library for your favorite database directly within a mapper or reducer, and to write from the batch job directly to the database server, one record at a time. This will work (assuming your firewall rules allow direct access from your Hadoop environment to your **production** databases), but it is a bad idea for several reasons:

Lựa chọn rõ ràng nhất có lẽ là dùng thư viện client cho cơ sở dữ liệu yêu thích của bạn trực tiếp bên trong một mapper hoặc reducer, và ghi từ tác vụ theo lô trực tiếp tới máy chủ cơ sở dữ liệu, mỗi lần một bản ghi. Cách này sẽ chạy được (giả định các quy tắc tường lửa của bạn cho phép truy cập trực tiếp từ môi trường Hadoop tới các cơ sở dữ liệu production của bạn), nhưng nó là một ý tồi vì vài lý do:

- As discussed previously in the context of joins, making a network request for every single record is orders of magnitude slower than the normal throughput of a batch task. Even if the client library supports batching, performance is likely to be poor.
- MapReduce jobs often run many tasks in parallel. If all the mappers or reducers concurrently write to the same output database, with a rate expected of a batch process, that database can easily be overwhelmed, and its performance for queries is likely to suffer. This can in turn cause operational problems in other parts of the system [35].
- Normally, MapReduce provides a clean all-or-nothing guarantee for job output: if a job succeeds, the result is the output of running every task exactly once, even if some tasks failed and had to be retried along the way; if the entire job fails, no output is produced. However, writing to an external system from inside a job produces externally visible side effects that cannot be hidden in this way. Thus, you have to worry about the results from partially completed jobs

being visible to other systems, and the complexities of Hadoop task attempts and speculative execution.

- Như đã bàn trước đó trong ngữ cảnh các phép join, việc thực hiện một yêu cầu mạng cho từng bản ghi chậm hơn nhiều bậc độ lớn so với thông lượng thông thường của một tác vụ theo lô. Ngay cả khi thư viện client hỗ trợ gom lô, hiệu năng nhiều khả năng vẫn kém.
- Các tác vụ MapReduce thường chạy nhiều tác vụ song song. Nếu tất cả các mapper hoặc reducer đồng thời ghi vào cùng một cơ sở dữ liệu đầu ra, với tốc độ kỳ vọng của một tiến trình theo lô, cơ sở dữ liệu đó có thể dễ dàng bị quá tải, và hiệu năng của nó cho các truy vấn nhiều khả năng sẽ giảm sút. Điều này đến lượt nó có thể gây ra các vấn đề vận hành ở những phần khác của hệ thống [35].
- Thông thường, MapReduce cung cấp một bảo đảm tất-cả-hoặc-không-gì gọn gàng cho đầu ra của tác vụ: nếu một tác vụ thành công, kết quả là đầu ra của việc chạy mỗi tác vụ đúng một lần, ngay cả khi một số tác vụ đã hỏng và phải được thử lại dọc đường; nếu toàn bộ tác vụ hỏng, không có đầu ra nào được tạo ra. Tuy nhiên, việc ghi tới một hệ thống bên ngoài từ bên trong một tác vụ tạo ra các tác dụng phụ hiển thị ra bên ngoài vốn không thể được giấu đi theo cách này. Vì vậy, bạn phải lo lắng về việc các kết quả từ các tác vụ hoàn thành một phần bị các hệ thống khác nhìn thấy, và về sự phức tạp của các lần thử tác vụ (task attempt) cùng việc thực thi suy đoán (speculative execution) của Hadoop.

A much better solution is to build a brand-new database inside the batch job and write it as files to the job's output directory in the distributed filesystem, just like the search indexes in the last section. Those data files are then immutable once written, and can be loaded in bulk into servers that handle read-only queries. Various key-value stores support building database files in MapReduce jobs, including Voldemort [46], Terrapin [47], ElephantDB [48], and HBase bulk loading [49].

Một giải pháp tốt hơn nhiều là xây dựng một cơ sở dữ liệu hoàn toàn mới bên trong tác vụ theo lô và ghi nó thành các tệp vào thư mục đầu ra của tác vụ trên hệ thống tệp phân tán, giống hệt như các chỉ mục tìm kiếm trong phần trước. Các tệp dữ liệu đó khi đó là bất biến một khi đã được ghi, và có thể được nạp hàng loạt vào các máy chủ xử lý các truy vấn chỉ-đọc. Nhiều kho khóa-giá trị hỗ trợ việc xây dựng các tệp cơ sở dữ liệu trong các tác vụ MapReduce, bao gồm Voldemort [46], Terrapin [47], ElephantDB [48], và việc nạp hàng loạt của HBase [49].

Building these database files is a good use of MapReduce: using a mapper to extract a key and then sorting by that key is already a lot of the work required to build an index. Since most of these key-value stores are read-only (the files can only be written once by a batch job and are then immutable), the data structures are quite simple. For example, they do not require a WAL (see “Making B-trees reliable”).

Việc xây dựng các tệp cơ sở dữ liệu này là một cách dùng tốt của MapReduce: dùng một mapper để trích xuất một khóa rồi sắp xếp theo khóa đó đã là phần lớn công việc cần để xây dựng một chỉ mục. Vì hầu hết các kho khóa-giá trị này là chỉ-đọc (các tệp chỉ có thể được ghi một lần bởi một tác vụ theo lô và sau đó là bất biến), các cấu trúc dữ liệu khá đơn giản. Ví dụ, chúng không đòi hỏi một WAL (xem “Làm cho B-tree tin cậy”).

When loading data into Voldemort, the server continues serving requests to the old data files while the new data files are copied from the distributed filesystem to the server's local disk. Once the copying is complete, the server atomically switches over to querying the new files. If anything goes wrong in this process, it can easily switch back to the old files again, since they are still there and immutable [46].

Khi nạp dữ liệu vào Voldemort, máy chủ tiếp tục phục vụ các yêu cầu trên các tệp dữ liệu cũ trong khi các tệp dữ liệu mới được sao chép từ hệ thống tệp phân tán tới đĩa cục bộ của máy chủ. Một khi việc sao chép hoàn tất, máy chủ chuyển sang truy vấn các tệp mới một cách nguyên tử. Nếu có bất kỳ điều gì trục trặc trong quá trình này, nó có thể dễ dàng chuyển trở lại các tệp cũ một lần nữa, vì chúng vẫn còn ở đó và bất biến [46].

Philosophy of batch process outputs — Triết lý về các đầu ra của tiến trình theo lô

The Unix philosophy that we discussed earlier in this chapter (“The Unix Philosophy”) encourages experimentation by being very explicit about dataflow: a program reads its input and writes its output. In the process, the input is left unchanged, any previous output is completely replaced with the new output, and there are no other side effects. This means that you can rerun a command as often as you like, tweaking or debugging it, without messing up the state of your system.

Triết lý Unix mà ta đã bàn trước đó trong chương này (“Triết lý Unix”) khuyến khích việc thử nghiệm bằng cách rất tường minh về luồng dữ liệu: một chương trình đọc đầu vào của nó và ghi đầu ra của nó. Trong quá trình đó, đầu vào được giữ nguyên không đổi, bất kỳ đầu ra trước đó nào đều được thay thế hoàn toàn bằng đầu ra mới, và không có tác dụng phụ nào khác. Điều này nghĩa là bạn có thể chạy lại một lệnh bao nhiêu lần tùy thích, tinh chỉnh hoặc gỡ lỗi nó, mà không làm rối trạng thái của hệ thống.

The handling of output from MapReduce jobs follows the same philosophy. By treating inputs as immutable and avoiding side effects (such as writing to external databases), batch jobs not only achieve good performance but also become much easier to maintain:

Việc xử lý đầu ra từ các tác vụ MapReduce tuân theo cùng một triết lý. Bằng cách coi các đầu vào là bất biến và tránh các tác dụng phụ (chẳng hạn ghi tới các cơ sở dữ liệu bên ngoài), các tác vụ theo lô không chỉ đạt được hiệu năng tốt mà còn trở nên dễ bảo trì hơn nhiều:

- If you introduce a **bug** into the code and the output is wrong or corrupted, you can simply **roll back** to a previous version of the code and rerun the job, and the output will be correct again. Or, even simpler, you can keep the old output in a different directory and simply switch back to it. Databases with read-write transactions do not have this **property**: if you deploy buggy code that writes bad data to the database, then rolling back the code will do nothing to fix the data in the database. (The idea of being able to recover from buggy code has been called human **fault** tolerance [50].)
- As a consequence of this ease of rolling back, feature development can proceed more quickly than in an environment where mistakes could mean irreversible damage. This principle of minimizing irreversibility is beneficial for Agile software development [51].
- If a map or reduce task fails, the MapReduce framework automatically re-schedules it and runs it again on the same input. If the **failure** is due to a bug in the code, it will keep crashing and eventually cause the job to fail after a few attempts; but if the failure is due to a transient issue, the fault is tolerated. This automatic retry is only safe because inputs are immutable and outputs from failed tasks are discarded by the MapReduce framework.
- The same set of files can be used as input for various different jobs, including monitoring jobs that calculate metrics and evaluate whether a job’s output has the expected characteristics (for example, by comparing it to the output from the previous run and measuring discrepancies).
- Like Unix tools, MapReduce jobs separate logic from wiring (configuring the input and output directories), which provides a separation of concerns and enables potential reuse of code: one team can focus on implementing a job that does one thing well, while other teams can decide

where and when to run that job.

- Nếu bạn đưa một bug vào mã và đầu ra bị sai hoặc bị hỏng, bạn có thể chỉ đơn giản hoàn tác về một phiên bản trước của mã và chạy lại tác vụ, và đầu ra sẽ lại đúng. Hoặc, thậm chí đơn giản hơn, bạn có thể giữ đầu ra cũ trong một thư mục khác và chỉ đơn giản chuyển trở lại nó. Các cơ sở dữ liệu với các giao dịch đọc-ghi không có thuộc tính này: nếu bạn triển khai mã lỗi vốn ghi dữ liệu xấu vào cơ sở dữ liệu, thì việc hoàn tác mã sẽ chẳng làm được gì để sửa dữ liệu trong cơ sở dữ liệu. (Ý tưởng về việc có thể phục hồi khỏi mã lỗi đã được gọi là khả năng chịu lỗi con người (human fault tolerance) [50].)
- Như một hệ quả của sự dễ dàng hoàn tác này, việc phát triển tính năng có thể tiến hành nhanh hơn so với trong một môi trường nơi các sai lầm có thể nghĩa là thiệt hại không thể đảo ngược. Nguyên lý giảm thiểu tính không-thể-đảo-ngược này có lợi cho phát triển phần mềm Agile [51].
- Nếu một tác vụ map hoặc reduce hỏng, framework MapReduce tự động lên lịch lại nó và chạy nó lại trên cùng đầu vào. Nếu hỏng hóc là do một bug trong mã, nó sẽ tiếp tục sụp đổ và rồi cuộc khiến tác vụ hỏng sau vài lần thử; nhưng nếu hỏng hóc là do một vấn đề nhất thời, lỗi được dung thứ. Việc tự động thử lại này chỉ an toàn vì các đầu vào là bất biến và các đầu ra từ các tác vụ hỏng bị framework MapReduce loại bỏ.
- Cùng một tập tệp có thể được dùng làm đầu vào cho nhiều tác vụ khác nhau, bao gồm các tác vụ giám sát vốn tính các chỉ số và đánh giá xem đầu ra của một tác vụ có các đặc tính như mong đợi không (ví dụ, bằng cách so sánh nó với đầu ra từ lần chạy trước và đo các sai khác).
- Giống như các công cụ Unix, các tác vụ MapReduce tách rời logic khỏi phần đầu nối (cấu hình các thư mục đầu vào và đầu ra), điều cung cấp một sự phân tách mối quan tâm và cho phép khả năng tái sử dụng mã: một nhóm có thể tập trung vào việc hiện thực một tác vụ làm tốt một việc, trong khi các nhóm khác có thể quyết định chạy tác vụ đó ở đâu và khi nào.

In these areas, the design principles that worked well for Unix also seem to be working well for Hadoop—but Unix and Hadoop also differ in some ways. For example, because most Unix tools assume untyped text files, they have to do a lot of input parsing (our log analysis example at the beginning of the chapter used `{print $7}` to extract the URL). On Hadoop, some of those low-value syntactic conversions are eliminated by using more structured file formats: Avro (see “Avro”) and Parquet (see “**Column**-Oriented Storage”) are often used, as they provide efficient schema-based encoding and allow evolution of their schemas over time (see Chapter 4).

Trong những lĩnh vực này, các nguyên lý thiết kế vốn hoạt động tốt cho Unix dường như cũng đang hoạt động tốt cho Hadoop — nhưng Unix và Hadoop cũng khác nhau ở một số mặt. Ví dụ, vì hầu hết các công cụ Unix giả định các tệp văn bản không định kiểu, chúng phải làm rất nhiều việc phân tích cú pháp đầu vào (ví dụ phân tích nhật ký ở đầu chương của ta đã dùng `{print $7}` để trích xuất URL). Trên Hadoop, một số trong những phép chuyển đổi cú pháp giá trị thấp đó được loại bỏ bằng cách dùng các định dạng tệp có cấu trúc hơn: Avro (xem “Avro”) và Parquet (xem “Lưu trữ hướng cột”) thường được dùng, vì chúng cung cấp mã hóa hiệu quả dựa trên lược đồ và cho phép tiến hóa các lược đồ của chúng theo thời gian (xem Chương 4).

Comparing Hadoop to Distributed Databases — So sánh Hadoop với các cơ sở dữ liệu phân tán

As we have seen, Hadoop is somewhat like a distributed version of Unix, where HDFS is the filesystem and MapReduce is a quirky implementation of a Unix process (which happens to always run the sort utility between the map phase and the reduce phase). We saw how you can implement various join and grouping operations on top of these primitives.

Như ta đã thấy, Hadoop phần nào giống một phiên bản phân tán của Unix, trong đó HDFS là hệ thống tệp còn MapReduce là một bản hiện thực kỳ quặc của một tiến trình Unix (vốn tình cờ luôn chạy tiện ích sort giữa giai đoạn map và giai đoạn reduce). Ta đã thấy cách bạn có thể hiện thực nhiều thao tác join và gom nhóm khác nhau trên nền các nguyên thủy này.

When the MapReduce paper [1] was published, it was—in some sense—not at all new. All of the processing and parallel join algorithms that we discussed in the last few sections had already been implemented in so-called massively parallel processing (MPP) databases more than a decade previously [3, 40]. For example, the Gamma database machine, Teradata, and Tandem NonStop SQL were pioneers in this area [52].

Khi bài báo về MapReduce [1] được công bố, theo một nghĩa nào đó nó hoàn toàn không mới. Tất cả các thuật toán xử lý và join song song mà ta đã bàn trong vài phần trước đã được hiện thực trong cái gọi là các cơ sở dữ liệu xử lý song song khối lớn (massively parallel processing - MPP) hơn một thập kỷ trước đó [3, 40]. Ví dụ, máy cơ sở dữ liệu Gamma, Teradata, và Tandem NonStop SQL là những người tiên phong trong lĩnh vực này [52].

The biggest difference is that MPP databases focus on **parallel execution** of analytic SQL queries on a cluster of machines, while the combination of MapReduce and a distributed filesystem [19] provides something much more like a general-purpose operating system that can run arbitrary programs.

Khác biệt lớn nhất là các cơ sở dữ liệu MPP tập trung vào việc thực thi song song các truy vấn SQL phân tích trên một cụm máy, trong khi sự kết hợp của MapReduce với một hệ thống tệp phân tán [19] cung cấp một thứ giống hơn nhiều với một hệ điều hành đa dụng có thể chạy các chương trình tùy ý.

Diversity of storage — Sự đa dạng của lưu trữ

Databases require you to structure data according to a particular model (e.g., relational or documents), whereas files in a distributed filesystem are just byte sequences, which can be written using any data model and encoding. They might be collections of database records, but they can equally well be text, images, videos, sensor readings, sparse matrices, feature vectors, genome sequences, or any other kind of data.

Các cơ sở dữ liệu đòi hỏi bạn cấu trúc dữ liệu theo một mô hình cụ thể (ví dụ, quan hệ hoặc tài liệu), trong khi các tệp trong một hệ thống tệp phân tán chỉ là các dãy byte, vốn có thể được ghi dùng bất kỳ mô hình dữ liệu và cách mã hóa nào. Chúng có thể là các tập hợp các bản ghi cơ sở dữ liệu, nhưng cũng có thể là văn bản, hình ảnh, video, các số đo cảm biến, các ma trận thưa, các vector đặc trưng, các trình tự gen, hay bất kỳ loại dữ liệu nào khác.

To put it bluntly, Hadoop opened up the possibility of indiscriminately dumping data into HDFS, and only later figuring out how to process it further [53]. By contrast, MPP databases typically

require careful up-front modeling of the data and query patterns before importing the data into the database's proprietary storage format.

Nói toạc ra, Hadoop đã mở ra khả năng đổ dữ liệu vào HDFS một cách bừa bãi, và chỉ về sau mới tìm ra cách xử lý nó tiếp [53]. Trái lại, các cơ sở dữ liệu MPP thường đòi hỏi việc mô hình hóa cẩn thận từ trước về dữ liệu và các mẫu truy vấn trước khi nhập dữ liệu vào định dạng lưu trữ độc quyền của cơ sở dữ liệu.

From a purist's point of view, it may seem that this careful modeling and import is desirable, because it means users of the database have better-quality data to work with. However, in practice, it appears that simply making data available quickly—even if it is in a quirky, difficult-to-use, raw format—is often more valuable than trying to decide on the ideal data model up front [54].

Từ góc nhìn của một người cầu toàn, có vẻ như việc mô hình hóa và nhập dữ liệu cẩn thận này là điều đáng mong muốn, vì nó nghĩa là người dùng cơ sở dữ liệu có dữ liệu chất lượng tốt hơn để làm việc. Tuy nhiên, trên thực tế, có vẻ như việc chỉ đơn giản làm cho dữ liệu sẵn có nhanh chóng — kể cả khi nó ở một định dạng thô, kỳ quặc, khó dùng — thường có giá trị hơn so với việc cố quyết định mô hình dữ liệu lý tưởng từ trước [54].

The idea is similar to a data warehouse (see “Data Warehousing”): simply bringing data from various parts of a large organization together in one place is valuable, because it enables joins across datasets that were previously disparate. The careful schema design required by an MPP database slows down that centralized data collection; collecting data in its raw form, and worrying about schema design later, allows the data collection to be speeded up (a concept sometimes known as a “data lake” or “enterprise data hub” [55]).

Ý tưởng này tương tự với một kho dữ liệu (xem “Kho dữ liệu”): việc chỉ đơn giản đưa dữ liệu từ nhiều phần khác nhau của một tổ chức lớn về cùng một chỗ là có giá trị, bởi nó cho phép các phép join trên các tập dữ liệu vốn trước đây tách rời nhau. Việc thiết kế lược đồ cẩn thận mà một cơ sở dữ liệu MPP đòi hỏi làm chậm việc thu thập dữ liệu tập trung đó; việc thu thập dữ liệu ở dạng thô của nó, và lo lắng về việc thiết kế lược đồ sau, cho phép việc thu thập dữ liệu được đẩy nhanh (một khái niệm đôi khi được biết đến là “hồ dữ liệu” (data lake) hay “trung tâm dữ liệu doanh nghiệp” (enterprise data hub) [55]).

Indiscriminate data dumping shifts the burden of interpreting the data: instead of forcing the producer of a dataset to bring it into a standardized format, the interpretation of the data becomes the consumer's problem (the **schema-on-read** approach [56]; see “**Schema flexibility** in the document model”). This can be an advantage if the producer and consumers are different teams with different priorities. There may not even be one ideal data model, but rather different views onto the data that are suitable for different purposes. Simply dumping data in its raw form allows for several such transformations. This approach has been dubbed the sushi principle: “raw data is better” [57].

Việc đổ dữ liệu bừa bãi chuyển dịch gánh nặng diễn giải dữ liệu: thay vì buộc người tạo ra một tập dữ liệu phải đưa nó về một định dạng chuẩn hóa, việc diễn giải dữ liệu trở thành vấn đề của người tiêu thụ (cách tiếp cận lược đồ-lúc-đọc (schema-on-read) [56]; xem “Tính linh hoạt lược đồ trong mô hình tài liệu”). Đây có thể là một ưu điểm nếu người tạo và người tiêu thụ là những nhóm khác nhau với những ưu tiên khác nhau. Thậm chí có thể không có một mô hình dữ liệu lý tưởng duy nhất nào, mà thay vào đó là những góc nhìn khác nhau lên dữ liệu phù hợp cho những mục đích khác nhau. Việc chỉ đơn giản đổ dữ liệu ở dạng thô của nó cho phép có nhiều phép biến đổi như vậy. Cách tiếp cận này đã được mệnh danh là nguyên lý sushi: “dữ liệu thô thì tốt hơn” [57].

Thus, Hadoop has often been used for implementing ETL processes (see “Data Warehousing”): data

from transaction processing systems is dumped into the distributed filesystem in some raw form, and then MapReduce jobs are written to clean up that data, transform it into a relational form, and import it into an MPP data warehouse for analytic purposes. Data modeling still happens, but it is in a separate step, decoupled from the data collection. This decoupling is possible because a distributed filesystem supports data encoded in any format.

Vì vậy, Hadoop thường được dùng để hiện thực các tiến trình ETL (xem “Kho dữ liệu”): dữ liệu từ các hệ thống xử lý giao dịch được đổ vào hệ thống tệp phân tán ở một dạng thô nào đó, và sau đó các tác vụ MapReduce được viết để làm sạch dữ liệu đó, biến đổi nó thành một dạng quan hệ, và nhập nó vào một kho dữ liệu MPP cho các mục đích phân tích. Việc mô hình hóa dữ liệu vẫn xảy ra, nhưng nó ở trong một bước riêng biệt, tách rời khỏi việc thu thập dữ liệu. Sự tách rời này khả thi bởi một hệ thống tệp phân tán hỗ trợ dữ liệu được mã hóa ở bất kỳ định dạng nào.

Diversity of processing models — Sự đa dạng của các mô hình xử lý

MPP databases are monolithic, tightly integrated pieces of software that take care of storage layout on disk, query planning, scheduling, and execution. Since these components can all be tuned and optimized for the specific needs of the database, the system as a whole can achieve very good performance on the types of queries for which it is designed. Moreover, the SQL **query language** allows expressive queries and elegant semantics without the need to write code, making it accessible to graphical tools used by business analysts (such as Tableau).

Các cơ sở dữ liệu MPP là những phần mềm nguyên khối, tích hợp chặt chẽ vốn lo liệu bố cục lưu trữ trên đĩa, việc hoạch định truy vấn, lập lịch, và thực thi. Vì các thành phần này có thể đều được tinh chỉnh và tối ưu cho các nhu cầu cụ thể của cơ sở dữ liệu, hệ thống như một tổng thể có thể đạt được hiệu năng rất tốt trên các loại truy vấn mà nó được thiết kế cho. Hơn nữa, ngôn ngữ truy vấn SQL cho phép các truy vấn giàu sức biểu đạt và ngữ nghĩa thanh lịch mà không cần viết mã, khiến nó dễ tiếp cận với các công cụ đồ họa được các nhà phân tích kinh doanh dùng (chẳng hạn Tableau).

On the other hand, not all kinds of processing can be sensibly expressed as SQL queries. For example, if you are building machine learning and recommendation systems, or full-text search indexes with relevance ranking models, or performing image analysis, you most likely need a more general model of data processing. These kinds of processing are often very specific to a particular application (e.g., feature engineering for machine learning, natural language models for machine translation, risk estimation functions for fraud prediction), so they inevitably require writing code, not just queries.

Mặt khác, không phải mọi loại xử lý đều có thể được diễn đạt một cách hợp lý thành các truy vấn SQL. Ví dụ, nếu bạn đang xây dựng các hệ thống học máy và gợi ý, hoặc các chỉ mục tìm kiếm toàn văn với các mô hình xếp hạng độ liên quan, hoặc thực hiện phân tích hình ảnh, bạn nhiều khả năng cần một mô hình xử lý dữ liệu tổng quát hơn. Những loại xử lý này thường rất đặc thù cho một ứng dụng cụ thể (ví dụ, kỹ thuật đặc trưng cho học máy, các mô hình ngôn ngữ tự nhiên cho dịch máy, các hàm ước lượng rủi ro cho dự đoán gian lận), nên chúng tất yếu đòi hỏi việc viết mã, chứ không chỉ các truy vấn.

MapReduce gave engineers the ability to easily run their own code over large datasets. If you have HDFS and MapReduce, you can build a SQL query execution engine on top of it, and indeed this is what the Hive project did [31]. However, you can also write many other forms of batch processes that do not lend themselves to being expressed as a SQL query.

MapReduce đã cho các kỹ sư khả năng dễ dàng chạy mã của riêng họ trên các tập dữ liệu lớn. Nếu bạn có HDFS và MapReduce, bạn có thể xây dựng một engine thực thi truy vấn

SQL trên nền nó, và quả thực đây là điều mà dự án Hive đã làm [31]. Tuy nhiên, bạn cũng có thể viết nhiều dạng tiến trình theo lô khác vốn không thích hợp để được diễn đạt thành một truy vấn SQL.

Subsequently, people found that MapReduce was too limiting and performed too badly for some types of processing, so various other processing models were developed on top of Hadoop (we will see some of them in “Beyond MapReduce”). Having two processing models, SQL and MapReduce, was not enough: even more different models were needed! And due to the openness of the Hadoop platform, it was feasible to implement a whole range of approaches, which would not have been possible within the confines of a monolithic MPP database [58].

Sau đó, người ta thấy rằng MapReduce quá hạn chế và cho hiệu năng quá kém đối với một số loại xử lý, nên nhiều mô hình xử lý khác đã được phát triển trên nền Hadoop (ta sẽ thấy một số trong số chúng ở “Vượt ra ngoài MapReduce”). Có hai mô hình xử lý, SQL và MapReduce, là chưa đủ: còn cần thậm chí nhiều mô hình khác nhau hơn nữa! Và nhờ vào tính mở của nền tảng Hadoop, việc hiện thực cả một loạt cách tiếp cận là khả thi, điều lẽ ra không thể trong giới hạn của một cơ sở dữ liệu MPP nguyên khối [58].

Crucially, those various processing models can all be run on a single shared-use cluster of machines, all accessing the same files on the distributed filesystem. In the Hadoop approach, there is no need to import the data into several different specialized systems for different kinds of processing: the system is flexible enough to support a diverse set of workloads within the same cluster. Not having to move data around makes it a lot easier to derive value from the data, and a lot easier to experiment with new processing models.

Quan trọng là, các mô hình xử lý đa dạng đó có thể đều chạy trên một cụm máy dùng chung duy nhất, tất cả cùng truy cập các tệp giống nhau trên hệ thống tệp phân tán. Trong cách tiếp cận Hadoop, không cần phải nhập dữ liệu vào nhiều hệ thống chuyên biệt khác nhau cho các loại xử lý khác nhau: hệ thống đủ linh hoạt để hỗ trợ một tập hợp đa dạng các khối lượng công việc trong cùng một cụm. Việc không phải di chuyển dữ liệu khắp nơi khiến việc rút ra giá trị từ dữ liệu trở nên dễ dàng hơn nhiều, và việc thử nghiệm với các mô hình xử lý mới trở nên dễ dàng hơn nhiều.

The Hadoop ecosystem includes both random-access OLTP databases such as HBase (see “SSTables and LSM-Trees”) and MPP-style analytic databases such as Impala [41]. Neither HBase nor Impala uses MapReduce, but both use HDFS for storage. They are very different approaches to accessing and processing data, but they can nevertheless coexist and be integrated in the same system.

Hệ sinh thái Hadoop bao gồm cả các cơ sở dữ liệu OLTP truy cập ngẫu nhiên như HBase (xem “SSTable và LSM-Tree”) lẫn các cơ sở dữ liệu phân tích kiểu MPP như Impala [41]. Cả HBase lẫn Impala đều không dùng MapReduce, nhưng cả hai đều dùng HDFS để lưu trữ. Chúng là những cách tiếp cận rất khác nhau để truy cập và xử lý dữ liệu, nhưng dù vậy chúng có thể cùng tồn tại và được tích hợp trong cùng một hệ thống.

Designing for frequent faults — Thiết kế cho các lỗi thường xuyên

When comparing MapReduce to MPP databases, two more differences in design approach stand out: the handling of faults and the use of memory and disk. Batch processes are less sensitive to faults than online systems, because they do not immediately affect users if they fail and they can always be run again.

Khi so sánh MapReduce với các cơ sở dữ liệu MPP, hai khác biệt nữa trong cách tiếp cận thiết kế nổi bật lên: việc xử lý các lỗi và việc dùng bộ nhớ và đĩa. Các tiến trình theo lô ít

nhạy cảm với lỗi hơn so với các hệ thống trực tuyến, bởi chúng không ảnh hưởng ngay lập tức tới người dùng nếu chúng hỏng và chúng luôn có thể được chạy lại.

If a node crashes while a query is executing, most MPP databases abort the entire query, and either let the user resubmit the query or automatically run it again [3]. As queries normally run for a few seconds or a few minutes at most, this way of handling errors is acceptable, since the cost of retrying is not too great. MPP databases also prefer to keep as much data as possible in memory (e.g., using hash joins) to avoid the cost of reading from disk.

Nếu một nút sập đổ trong khi một truy vấn đang thực thi, hầu hết các cơ sở dữ liệu MPP hủy bỏ toàn bộ truy vấn, và hoặc để người dùng gửi lại truy vấn hoặc tự động chạy nó lại [3]. Vì các truy vấn thường chạy trong vài giây hoặc nhiều nhất là vài phút, cách xử lý lỗi này là chấp nhận được, vì cái giá của việc thử lại không quá lớn. Các cơ sở dữ liệu MPP cũng ưu tiên giữ càng nhiều dữ liệu trong bộ nhớ càng tốt (ví dụ, dùng các phép join băm) để tránh cái giá của việc đọc từ đĩa.

On the other hand, MapReduce can tolerate the failure of a map or reduce task without it affecting the job as a whole by retrying work at the granularity of an individual task. It is also very eager to write data to disk, partly for fault tolerance, and partly on the assumption that the dataset will be too big to fit in memory anyway.

Mặt khác, MapReduce có thể chịu được hỏng hóc của một tác vụ map hoặc reduce mà không làm ảnh hưởng tới tác vụ như một tổng thể, bằng cách thử lại công việc ở mức độ chi tiết của một tác vụ riêng lẻ. Nó cũng rất sốt sắng ghi dữ liệu ra đĩa, một phần vì khả năng chịu lỗi, và một phần dựa trên giả định rằng tập dữ liệu dù sao cũng sẽ quá lớn để nhét vừa trong bộ nhớ.

The MapReduce approach is more appropriate for larger jobs: jobs that process so much data and run for such a long time that they are likely to experience at least one task failure along the way. In that case, rerunning the entire job due to a single task failure would be wasteful. Even if recovery at the granularity of an individual task introduces overheads that make fault-free processing slower, it can still be a reasonable trade-off if the rate of task failures is high enough.

Cách tiếp cận MapReduce phù hợp hơn cho các tác vụ lớn hơn: các tác vụ xử lý nhiều dữ liệu đến vậy và chạy lâu đến vậy đến nỗi chúng nhiều khả năng sẽ trải qua ít nhất một lần hỏng tác vụ dọc đường. Trong trường hợp đó, việc chạy lại toàn bộ tác vụ chỉ vì một lần hỏng tác vụ duy nhất sẽ là lãng phí. Ngay cả khi việc phục hồi ở mức độ chi tiết của một tác vụ riêng lẻ đưa vào các chi phí khiến cho việc xử lý không-lỗi chậm hơn, nó vẫn có thể là một sự đánh đổi hợp lý nếu tỷ lệ hỏng tác vụ đủ cao.

But how realistic are these assumptions? In most clusters, machine failures do occur, but they are not very frequent—probably rare enough that most jobs will not experience a machine failure. Is it really worth incurring significant overheads for the sake of fault tolerance?

Nhưng những giả định này thực tế đến đâu? Trong hầu hết các cụm, hỏng hóc máy quả có xảy ra, nhưng chúng không quá thường xuyên — có lẽ hiếm đến mức hầu hết các tác vụ sẽ không trải qua một lần hỏng máy. Việc gánh chịu những chi phí đáng kể vì khả năng chịu lỗi có thực sự đáng giá không?

To understand the reasons for MapReduce's sparing use of memory and task-level recovery, it is helpful to look at the environment for which MapReduce was originally designed. Google has mixed-use datacenters, in which online production services and offline batch jobs run on the same machines. Every task has a resource allocation (CPU cores, RAM, disk space, etc.) that is enforced using containers. Every task also has a priority, and if a higher-priority task needs more resources, lower-priority tasks on the same machine can be terminated (preempted) in order to free up

resources. Priority also determines pricing of the computing resources: teams must pay for the resources they use, and higher-priority processes cost more [59].

Để hiểu các lý do cho việc MapReduce dùng bộ nhớ một cách tiết kiệm và phục hồi ở mức tác vụ, sẽ hữu ích khi nhìn vào môi trường mà MapReduce ban đầu được thiết kế cho. Google có các trung tâm dữ liệu dùng chung hỗn hợp, trong đó các dịch vụ production trực tuyến và các tác vụ theo lô ngoại tuyến chạy trên cùng các máy. Mỗi tác vụ có một sự phân bổ tài nguyên (số lõi CPU, RAM, không gian đĩa, v.v.) được thực thi bằng các container. Mỗi tác vụ cũng có một độ ưu tiên, và nếu một tác vụ có độ ưu tiên cao hơn cần nhiều tài nguyên hơn, các tác vụ có độ ưu tiên thấp hơn trên cùng máy có thể bị chấm dứt (bị trưng dụng - preempted) nhằm giải phóng tài nguyên. Độ ưu tiên cũng quyết định giá của các tài nguyên tính toán: các nhóm phải trả tiền cho các tài nguyên họ dùng, và các tiến trình có độ ưu tiên cao hơn tốn nhiều hơn [59].

This architecture allows non-production (low-priority) computing resources to be overcommitted, because the system knows that it can reclaim the resources if necessary. Overcommitting resources in turn allows better utilization of machines and greater efficiency compared to systems that segregate production and non-production tasks. However, as MapReduce jobs run at low priority, they run the risk of being preempted at any time because a higher-priority process requires their resources. Batch jobs effectively “pick up the scraps under the table,” using any computing resources that remain after the high-priority processes have taken what they need.

Kiến trúc này cho phép các tài nguyên tính toán phi-production (độ ưu tiên thấp) được cam kết vượt mức (overcommit), bởi hệ thống biết rằng nó có thể đòi lại các tài nguyên nếu cần. Việc cam kết tài nguyên vượt mức đến lượt nó cho phép tận dụng các máy tốt hơn và hiệu quả cao hơn so với các hệ thống tách biệt các tác vụ production và phi-production. Tuy nhiên, vì các tác vụ MapReduce chạy ở độ ưu tiên thấp, chúng chịu rủi ro bị trưng dụng bất cứ lúc nào bởi một tiến trình có độ ưu tiên cao hơn cần các tài nguyên của chúng. Các tác vụ theo lô thực chất “nhặt nhanh những mảnh vụn dưới gầm bàn”, dùng bất kỳ tài nguyên tính toán nào còn lại sau khi các tiến trình có độ ưu tiên cao đã lấy những gì chúng cần.

At Google, a MapReduce task that runs for an hour has an approximately 5% risk of being terminated to make space for a higher-priority process. This rate is more than an order of magnitude higher than the rate of failures due to hardware issues, machine reboot, or other reasons [59]. At this rate of preemptions, if a job has 100 tasks that each run for 10 minutes, there is a risk greater than 50% that at least one task will be terminated before it is finished.

Tại Google, một tác vụ MapReduce chạy trong một giờ có khoảng 5% rủi ro bị chấm dứt để nhường chỗ cho một tiến trình có độ ưu tiên cao hơn. Tỷ lệ này cao hơn một bậc độ lớn so với tỷ lệ hỏng hóc do các vấn đề phần cứng, khởi động lại máy, hoặc các lý do khác [59]. Với tỷ lệ trưng dụng này, nếu một tác vụ có 100 tác vụ con mỗi cái chạy trong 10 phút, thì có rủi ro lớn hơn 50% rằng ít nhất một tác vụ con sẽ bị chấm dứt trước khi nó hoàn thành.

And this is why MapReduce is designed to tolerate frequent unexpected task termination: it's not because the hardware is particularly unreliable, it's because the freedom to arbitrarily terminate processes enables better resource utilization in a computing cluster.

Và đây là lý do tại sao MapReduce được thiết kế để chịu được việc chấm dứt tác vụ bất ngờ thường xuyên: không phải vì phần cứng đặc biệt thiếu tin cậy, mà vì sự tự do chấm dứt các tiến trình một cách tùy ý cho phép tận dụng tài nguyên tốt hơn trong một cụm tính toán.

Among open source cluster schedulers, preemption is less widely used. YARN's CapacityScheduler

supports preemption for balancing the resource allocation of different queues [58], but general priority preemption is not supported in YARN, Mesos, or Kubernetes at the time of writing [60]. In an environment where tasks are not so often terminated, the design decisions of MapReduce make less sense. In the next section, we will look at some alternatives to MapReduce that make different design decisions.

Trong số các bộ lập lịch cụm mã nguồn mở, việc trưng dụng ít được dùng rộng rãi hơn. CapacityScheduler của YARN hỗ trợ việc trưng dụng để cân bằng sự phân bổ tài nguyên của các hàng đợi khác nhau [58], nhưng việc trưng dụng theo độ ưu tiên một cách tổng quát không được hỗ trợ trong YARN, Mesos, hay Kubernetes tại thời điểm viết [60]. Trong một môi trường nơi các tác vụ không bị chấm dứt thường xuyên đến vậy, các quyết định thiết kế của MapReduce ít có ý nghĩa hơn. Trong phần tiếp theo, ta sẽ xem xét một số lựa chọn thay thế cho MapReduce vốn đưa ra những quyết định thiết kế khác.

Beyond MapReduce — Vượt ra ngoài MapReduce

Although MapReduce became very popular and received a lot of hype in the late 2000s, it is just one among many possible programming models for distributed systems. Depending on the volume of data, the structure of the data, and the type of processing being done with it, other tools may be more appropriate for expressing a computation.

Mặc dù MapReduce trở nên rất phổ biến và nhận được rất nhiều sự thổi phồng vào cuối thập niên 2000, nó chỉ là một trong nhiều mô hình lập trình khả dĩ cho các hệ thống phân tán. Tùy vào khối lượng dữ liệu, cấu trúc của dữ liệu, và loại xử lý đang được thực hiện với nó, các công cụ khác có thể phù hợp hơn để diễn đạt một phép tính.

We nevertheless spent a lot of time in this chapter discussing MapReduce because it is a useful learning tool, as it is a fairly clear and simple **abstraction** on top of a distributed filesystem. That is, simple in the sense of being able to understand what it is doing, not in the sense of being easy to use. Quite the opposite: implementing a complex processing job using the raw MapReduce APIs is actually quite hard and laborious—for **instance**, you would need to implement any join algorithms from scratch [37].

Tuy vậy ta đã dành rất nhiều thời gian trong chương này bàn về MapReduce vì nó là một công cụ học tập hữu ích, bởi nó là một trừu tượng hóa khá rõ ràng và đơn giản trên nền một hệ thống tệp phân tán. Tức là, đơn giản theo nghĩa có thể hiểu được nó đang làm gì, chứ không theo nghĩa dễ dùng. Hoàn toàn ngược lại: việc hiện thực một tác vụ xử lý phức tạp dùng các API MapReduce thô thực ra khá khó và cực nhọc — chẳng hạn, bạn sẽ cần hiện thực bất kỳ thuật toán join nào từ đầu [37].

In response to the difficulty of using MapReduce directly, various higher-level programming models (Pig, Hive, Cascading, Crunch) were created as abstractions on top of MapReduce. If you understand how MapReduce works, they are fairly easy to learn, and their higher-level constructs make many common batch processing tasks significantly easier to implement.

Để đáp lại sự khó khăn của việc dùng MapReduce một cách trực tiếp, nhiều mô hình lập trình ở mức cao hơn (Pig, Hive, Cascading, Crunch) đã được tạo ra như các trừu tượng hóa trên nền MapReduce. Nếu bạn hiểu cách MapReduce hoạt động, chúng khá dễ học, và các cấu trúc ở mức cao hơn của chúng khiến nhiều tác vụ xử lý theo lô phổ biến trở nên dễ hiện thực hơn đáng kể.

However, there are also problems with the MapReduce execution model itself, which are not fixed by adding another level of abstraction and which manifest themselves as poor performance for some kinds of processing. On the one hand, MapReduce is very robust: you can use it to process almost arbitrarily large quantities of data on an unreliable multi-tenant system with frequent task

terminations, and it will still get the job done (albeit slowly). On the other hand, other tools are sometimes orders of magnitude faster for some kinds of processing.

Tuy nhiên, cũng có những vấn đề với chính mô hình thực thi MapReduce, vốn không được khắc phục bằng cách thêm một lớp trừu tượng hóa nữa và vốn biểu hiện thành hiệu năng kém đối với một số loại xử lý. Một mặt, MapReduce rất bền bỉ: bạn có thể dùng nó để xử lý các lượng dữ liệu lớn gần như tùy ý trên một hệ thống đa người thuê thiếu tin cậy với việc chấm dứt tác vụ thường xuyên, và nó vẫn sẽ làm xong việc (dù chậm). Mặt khác, các công cụ khác đôi khi nhanh hơn nhiều bậc độ lớn đối với một số loại xử lý.

In the rest of this chapter, we will look at some of those alternatives for batch processing. In Chapter 11 we will move to stream processing, which can be regarded as another way of speeding up batch processing.

Trong phần còn lại của chương này, ta sẽ xem xét một số trong những lựa chọn thay thế đó cho xử lý theo lô. Ở Chương 11 ta sẽ chuyển sang xử lý luồng, vốn có thể được coi là một cách khác để đẩy nhanh xử lý theo lô.

Materialization of Intermediate State — Vật chất hóa trạng thái trung gian

As discussed previously, every MapReduce job is independent from every other job. The main contact points of a job with the rest of the world are its input and output directories on the distributed filesystem. If you want the output of one job to become the input to a second job, you need to configure the second job's input directory to be the same as the first job's output directory, and an external workflow scheduler must start the second job only once the first job has completed.

Như đã bàn trước đó, mọi tác vụ MapReduce đều độc lập với mọi tác vụ khác. Các điểm tiếp xúc chính của một tác vụ với phần còn lại của thế giới là các thư mục đầu vào và đầu ra của nó trên hệ thống tệp phân tán. Nếu bạn muốn đầu ra của một tác vụ trở thành đầu vào của một tác vụ thứ hai, bạn cần cấu hình thư mục đầu vào của tác vụ thứ hai sao cho giống với thư mục đầu ra của tác vụ thứ nhất, và một bộ lập lịch luồng công việc bên ngoài phải khởi động tác vụ thứ hai chỉ một khi tác vụ thứ nhất đã hoàn thành.

This setup is reasonable if the output from the first job is a dataset that you want to publish widely within your organization. In that case, you need to be able to refer to it by name and reuse it as input to several different jobs (including jobs developed by other teams). Publishing data to a well-known location in the distributed filesystem allows loose coupling so that jobs don't need to know who is producing their input or consuming their output (see “Separation of logic and wiring”).

Cách thiết lập này là hợp lý nếu đầu ra từ tác vụ thứ nhất là một tập dữ liệu mà bạn muốn công bố rộng rãi trong tổ chức của mình. Trong trường hợp đó, bạn cần có khả năng tham chiếu tới nó theo tên và tái sử dụng nó làm đầu vào cho nhiều tác vụ khác nhau (bao gồm các tác vụ được phát triển bởi các nhóm khác). Việc công bố dữ liệu tới một vị trí được biết đến rộng rãi trong hệ thống tệp phân tán cho phép sự gắn kết lỏng lẻo sao cho các tác vụ không cần biết ai đang tạo ra đầu vào của chúng hay tiêu thụ đầu ra của chúng (xem “Tách rời logic và đấu nối”).

However, in many cases, you know that the output of one job is only ever used as input to one other job, which is maintained by the same team. In this case, the files on the distributed filesystem are simply intermediate state: a means of passing data from one job to the next. In the complex workflows used to build recommendation systems consisting of 50 or 100 MapReduce jobs [29], there is a lot of such intermediate state.

Tuy nhiên, trong nhiều trường hợp, bạn biết rằng đầu ra của một tác vụ chỉ từng được dùng làm đầu vào cho một tác vụ khác duy nhất, vốn được bảo trì bởi cùng một nhóm. Trong trường hợp này, các tệp trên hệ thống tệp phân tán chỉ đơn giản là trạng thái trung gian: một phương tiện để chuyển dữ liệu từ một tác vụ sang tác vụ tiếp theo. Trong các luồng công việc phức tạp được dùng để xây dựng các hệ thống gợi ý gồm 50 hoặc 100 tác vụ MapReduce [29], có rất nhiều trạng thái trung gian như vậy.

The process of writing out this intermediate state to files is called materialization. (We came across the term previously in the context of materialized views, in “Aggregation: Data Cubes and Materialized Views”. It means to eagerly compute the result of some operation and write it out, rather than computing it on demand when requested.)

Quá trình ghi ra trạng thái trung gian này thành các tệp được gọi là vật chất hóa (materialization). (Ta đã gặp thuật ngữ này trước đó trong ngữ cảnh các khung nhìn vật chất hóa - materialized views, ở “Tổng hợp: Khối dữ liệu và khung nhìn vật chất hóa”. Nó nghĩa là tính sẵn kết quả của một thao tác nào đó và ghi nó ra, thay vì tính nó theo yêu cầu khi được hỏi tới.)

By contrast, the log analysis example at the beginning of the chapter used Unix pipes to connect the output of one command with the input of another. Pipes do not fully materialize the intermediate state, but instead stream the output to the input incrementally, using only a small in-memory buffer.

Trái lại, ví dụ phân tích nhật ký ở đầu chương đã dùng pipe của Unix để kết nối đầu ra của một lệnh với đầu vào của một lệnh khác. Pipe không vật chất hóa hoàn toàn trạng thái trung gian, mà thay vào đó truyền theo dòng đầu ra tới đầu vào một cách tăng dần, chỉ dùng một bộ đệm nhỏ trong bộ nhớ.

MapReduce’s approach of fully materializing intermediate state has downsides compared to Unix pipes:

Cách tiếp cận của MapReduce là vật chất hóa hoàn toàn trạng thái trung gian có những nhược điểm so với pipe của Unix:

- A MapReduce job can only start when all tasks in the preceding jobs (that generate its inputs) have completed, whereas processes connected by a Unix pipe are started at the same time, with output being consumed as soon as it is produced. Skew or varying load on different machines means that a job often has a few straggler tasks that take much longer to complete than the others. Having to wait until all of the preceding job’s tasks have completed slows down the execution of the workflow as a whole.
- Mappers are often redundant: they just read back the same file that was just written by a reducer, and prepare it for the next stage of partitioning and sorting. In many cases, the mapper code could be part of the previous reducer: if the reducer output was partitioned and sorted in the same way as mapper output, then reducers could be chained together directly, without interleaving with mapper stages.
- Storing intermediate state in a distributed filesystem means those files are replicated across several nodes, which is often overkill for such temporary data.
- Một tác vụ MapReduce chỉ có thể bắt đầu khi tất cả các tác vụ con trong các tác vụ trước đó (vốn sinh ra các đầu vào của nó) đã hoàn thành, trong khi các tiến trình được kết nối bằng một pipe Unix được khởi động cùng lúc, với đầu ra được tiêu thụ ngay khi nó được tạo ra. Sự lệch hoặc tải biến thiên trên các máy khác nhau nghĩa là một tác vụ thường có một vài tác vụ con lè mề (straggler) mất nhiều thời gian hơn hẳn để hoàn thành so với những tác vụ con khác. Việc phải chờ tới khi tất cả các tác vụ con của tác vụ trước đó đã hoàn thành làm chậm việc thực thi của luồng công việc như

- một tổng thể.
- Các mapper thường thừa thãi: chúng chỉ đọc lại chính tệp vừa được ghi bởi một reducer, và chuẩn bị nó cho giai đoạn phân vùng và sắp xếp tiếp theo. Trong nhiều trường hợp, mã mapper có thể là một phần của reducer trước đó: nếu đầu ra của reducer được phân vùng và sắp xếp theo cùng cách với đầu ra của mapper, thì các reducer có thể được nối với nhau trực tiếp, mà không cần xen kẽ với các giai đoạn mapper.
- Việc lưu trạng thái trung gian trong một hệ thống tệp phân tán nghĩa là các tệp đó được nhân bản trên vài nút, điều thường là quá mức cần thiết đối với dữ liệu tạm thời như vậy.

Dataflow engines — Các engine luồng dữ liệu

In order to fix these problems with MapReduce, several new execution engines for distributed batch computations were developed, the most well known of which are Spark [61, 62], Tez [63, 64], and Flink [65, 66]. There are various differences in the way they are designed, but they have one thing in common: they handle an entire workflow as one job, rather than breaking it up into independent subjobs.

Nhằm khắc phục những vấn đề này với MapReduce, vài engine thực thi mới cho các phép tính theo lô phân tán đã được phát triển, nổi tiếng nhất trong số đó là Spark [61, 62], Tez [63, 64], và Flink [65, 66]. Có nhiều khác biệt trong cách chúng được thiết kế, nhưng chúng có một điểm chung: chúng xử lý toàn bộ một luồng công việc như một tác vụ duy nhất, thay vì chia nó thành các tác vụ con độc lập.

Since they explicitly model the flow of data through several processing stages, these systems are known as dataflow engines. Like MapReduce, they work by repeatedly calling a user-defined function to process one record at a time on a single thread. They parallelize work by partitioning inputs, and they copy the output of one function over the network to become the input to another function.

Vì chúng mô hình hóa một cách tường minh dòng dữ liệu chảy qua vài giai đoạn xử lý, các hệ thống này được biết đến là các engine luồng dữ liệu (dataflow engines). Giống như MapReduce, chúng hoạt động bằng cách lặp đi lặp lại gọi một hàm do người dùng định nghĩa để xử lý một bản ghi tại một thời điểm trên một luồng duy nhất. Chúng song song hóa công việc bằng cách phân vùng các đầu vào, và chúng sao chép đầu ra của một hàm qua mạng để trở thành đầu vào của một hàm khác.

Unlike in MapReduce, these functions need not take the strict roles of alternating map and reduce, but instead can be assembled in more flexible ways. We call these functions operators, and the dataflow engine provides several different options for connecting one operator's output to another's input:

Khác với trong MapReduce, các hàm này không cần đảm nhận các vai trò nghiêm ngặt là map và reduce xen kẽ, mà thay vào đó có thể được lắp ghép theo những cách linh hoạt hơn. Ta gọi những hàm này là toán tử (operators), và engine luồng dữ liệu cung cấp vài lựa chọn khác nhau để kết nối đầu ra của một toán tử với đầu vào của một toán tử khác:

- One option is to repartition and sort records by key, like in the shuffle stage of MapReduce (see “Distributed execution of MapReduce”). This feature enables sort-merge joins and grouping in the same way as in MapReduce.
- Another possibility is to take several inputs and to partition them in the same way, but skip the sorting. This saves effort on partitioned hash joins, where the partitioning of records is important but the order is irrelevant because building the hash table randomizes the order anyway.

- For broadcast hash joins, the same output from one operator can be sent to all partitions of the join operator.
 - Một lựa chọn là phân vùng lại và sắp xếp các bản ghi theo khóa, như trong giai đoạn shuffle của MapReduce (xem “Thực thi phân tán của MapReduce”). Tính năng này cho phép các phép join sắp-xếp-trộn và gom nhóm theo cùng cách với trong MapReduce.
 - Một khả năng khác là nhận vài đầu vào và phân vùng chúng theo cùng một cách, nhưng bỏ qua việc sắp xếp. Điều này tiết kiệm công sức cho các phép join băm phân vùng, nơi sự phân vùng của các bản ghi là quan trọng nhưng thứ tự không liên quan vì việc xây dựng bảng băm dù sao cũng ngẫu nhiên hóa thứ tự.
 - Đối với các phép join băm quảng bá, cùng một đầu ra từ một toán tử có thể được gửi tới tất cả các phân vùng của toán tử join.

This style of processing engine is based on research systems like Dryad [67] and Nephelê [68], and it offers several advantages compared to the MapReduce model:

Phong cách engine xử lý này dựa trên các hệ thống nghiên cứu như Dryad [67] và Nephelê [68], và nó mang lại vài ưu điểm so với mô hình MapReduce:

- Expensive work such as sorting need only be performed in places where it is actually required, rather than always happening by default between every map and reduce stage.
- There are no unnecessary map tasks, since the work done by a mapper can often be incorporated into the preceding reduce operator (because a mapper does not change the partitioning of a dataset).
- Because all joins and data dependencies in a workflow are explicitly declared, the scheduler has an overview of what data is required where, so it can make locality optimizations. For example, it can try to place the task that consumes some data on the same machine as the task that produces it, so that the data can be exchanged through a shared memory buffer rather than having to copy it over the network.
- It is usually sufficient for intermediate state between operators to be kept in memory or written to local disk, which requires less I/O than writing it to HDFS (where it must be replicated to several machines and written to disk on each replica). MapReduce already uses this optimization for mapper output, but dataflow engines generalize the idea to all intermediate state.
- Operators can start executing as soon as their input is ready; there is no need to wait for the entire preceding stage to finish before the next one starts.
- Existing Java **Virtual Machine** (JVM) processes can be reused to run new operators, reducing startup overheads compared to MapReduce (which launches a new JVM for each task).
 - Công việc đắt đỏ chẳng hạn việc sắp xếp chỉ cần được thực hiện ở những chỗ nó thực sự cần, thay vì luôn xảy ra một cách mặc định giữa mỗi giai đoạn map và reduce.
 - Không có các tác vụ map không cần thiết, vì công việc do một mapper làm thường có thể được tích hợp vào toán tử reduce trước đó (bởi một mapper không thay đổi sự phân vùng của một tập dữ liệu).
 - Vì tất cả các phép join và phụ thuộc dữ liệu trong một luồng công việc được khai báo tường minh, bộ lập lịch có một cái nhìn tổng quan về dữ liệu nào được cần ở đâu, nên nó có thể thực hiện các tối ưu về tính cục bộ. Ví dụ, nó có thể cố đặt tác vụ tiêu thụ một số dữ liệu trên cùng máy với tác vụ tạo ra nó, sao cho dữ liệu có thể được trao đổi qua một bộ đệm bộ nhớ dùng chung thay vì phải sao chép nó qua mạng.
 - Thường thì việc giữ trạng thái trung gian giữa các toán tử trong bộ nhớ hoặc ghi ra đĩa cục bộ là đủ, điều đòi hỏi ít I/O hơn so với việc ghi nó ra HDFS (nơi nó phải được nhân bản tới vài máy và ghi ra đĩa trên mỗi bản sao). MapReduce đã dùng tối ưu này cho đầu ra của mapper, nhưng các engine luồng dữ liệu tổng quát hóa ý tưởng này

cho tất cả trạng thái trung gian.

- Các toán tử có thể bắt đầu thực thi ngay khi đầu vào của chúng sẵn sàng; không cần chờ toàn bộ giai đoạn trước đó kết thúc trước khi giai đoạn tiếp theo bắt đầu.
- Các tiến trình Máy ảo Java (JVM) hiện có thể được tái sử dụng để chạy các toán tử mới, giảm chi phí khởi động so với MapReduce (vốn khởi chạy một JVM mới cho mỗi tác vụ).

You can use dataflow engines to implement the same computations as MapReduce workflows, and they usually execute significantly faster due to the optimizations described here. Since operators are a generalization of map and reduce, the same processing code can run on either execution engine: workflows implemented in Pig, Hive, or Cascading can be switched from MapReduce to Tez or Spark with a simple configuration change, without modifying code [64].

Bạn có thể dùng các engine luồng dữ liệu để hiện thực cùng các phép tính như các luồng công việc MapReduce, và chúng thường thực thi nhanh hơn đáng kể nhờ các tối ưu được mô tả ở đây. Vì các toán tử là một sự tổng quát hóa của map và reduce, cùng một mã xử lý có thể chạy trên một trong hai engine thực thi: các luồng công việc được hiện thực trong Pig, Hive, hoặc Cascading có thể được chuyển từ MapReduce sang Tez hoặc Spark bằng một thay đổi cấu hình đơn giản, mà không cần sửa đổi mã [64].

Tez is a fairly thin library that relies on the YARN shuffle service for the actual copying of data between nodes [58], whereas Spark and Flink are big frameworks that include their own network communication layer, scheduler, and user-facing APIs. We will discuss those high-level APIs shortly.

Tez là một thư viện khá mỏng vốn dựa vào dịch vụ shuffle của YARN cho việc sao chép dữ liệu thực sự giữa các nút [58], trong khi Spark và Flink là những framework lớn bao gồm tầng giao tiếp mạng, bộ lập lịch, và các API hướng người dùng của riêng chúng. Ta sẽ bàn về các API ở mức cao đó ngay sau đây.

Fault tolerance — Khả năng chịu lỗi

An advantage of fully materializing intermediate state to a distributed filesystem is that it is durable, which makes fault tolerance fairly easy in MapReduce: if a task fails, it can just be restarted on another machine and read the same input again from the filesystem.

Một ưu điểm của việc vật chất hóa hoàn toàn trạng thái trung gian ra một hệ thống tệp phân tán là nó có độ bền, điều khiến cho khả năng chịu lỗi khá dễ dàng trong MapReduce: nếu một tác vụ hỏng, nó có thể chỉ đơn giản được khởi động lại trên một máy khác và đọc lại cùng đầu vào từ hệ thống tệp.

Spark, Flink, and Tez avoid writing intermediate state to HDFS, so they take a different approach to tolerating faults: if a machine fails and the intermediate state on that machine is lost, it is recomputed from other data that is still available (a prior intermediary stage if possible, or otherwise the original input data, which is normally on HDFS).

Spark, Flink, và Tez tránh ghi trạng thái trung gian ra HDFS, nên chúng đi theo một cách tiếp cận khác để chịu lỗi: nếu một máy hỏng và trạng thái trung gian trên máy đó bị mất, nó được tính lại từ dữ liệu khác vẫn còn sẵn (một giai đoạn trung gian trước đó nếu có thể, hoặc nếu không thì dữ liệu đầu vào gốc, vốn thường ở trên HDFS).

To enable this recomputation, the framework must keep track of how a given piece of data was computed—which input partitions it used, and which operators were applied to it. Spark uses the **resilient** distributed dataset (RDD) abstraction for tracking the ancestry of data [61], while Flink

checkpoints operator state, allowing it to resume running an operator that ran into a fault during its execution [66].

Để cho phép việc tính lại này, framework phải theo dõi xem một mẫu dữ liệu nhất định đã được tính như thế nào — nó đã dùng những phân vùng đầu vào nào, và những toán tử nào đã được áp dụng lên nó. Spark dùng trừu tượng hóa tập dữ liệu phân tán kiên cường (resilient distributed dataset - RDD) để theo dõi gốc gác của dữ liệu [61], trong khi Flink tạo điểm kiểm tra (checkpoint) cho trạng thái toán tử, cho phép nó tiếp tục chạy một toán tử vốn gặp lỗi trong khi thực thi [66].

When recomputing data, it is important to know whether the computation is deterministic: that is, given the same input data, do the operators always produce the same output? This question matters if some of the lost data has already been sent to downstream operators. If the operator is restarted and the recomputed data is not the same as the original lost data, it becomes very hard for downstream operators to resolve the contradictions between the old and new data. The solution in the case of nondeterministic operators is normally to kill the downstream operators as well, and run them again on the new data.

Khi tính lại dữ liệu, điều quan trọng là phải biết liệu phép tính có tính xác định hay không: tức là, với cùng dữ liệu đầu vào, các toán tử có luôn tạo ra cùng đầu ra không? Câu hỏi này quan trọng nếu một số dữ liệu bị mất đã được gửi tới các toán tử phía sau. Nếu toán tử được khởi động lại và dữ liệu được tính lại không giống với dữ liệu bị mất ban đầu, thì các toán tử phía sau sẽ rất khó giải quyết các mâu thuẫn giữa dữ liệu cũ và mới. Giải pháp trong trường hợp các toán tử phi xác định thường là cũng kết liễu các toán tử phía sau, và chạy chúng lại trên dữ liệu mới.

In order to avoid such cascading faults, it is better to make operators deterministic. Note however that it is easy for nondeterministic behavior to accidentally creep in: for example, many programming languages do not guarantee any particular order when iterating over elements of a hash table, many probabilistic and statistical algorithms explicitly rely on using random numbers, and any use of the system clock or external data sources is nondeterministic. Such causes of nondeterminism need to be removed in order to reliably recover from faults, for example by generating pseudorandom numbers using a fixed seed.

Để tránh các lỗi dây chuyền như vậy, tốt hơn là làm cho các toán tử có tính xác định. Tuy nhiên hãy lưu ý rằng hành vi phi xác định rất dễ vô tình len lỏi vào: ví dụ, nhiều ngôn ngữ lập trình không bảo đảm bất kỳ thứ tự cụ thể nào khi duyệt qua các phần tử của một bảng băm, nhiều thuật toán xác suất và thống kê dựa một cách tường minh vào việc dùng các số ngẫu nhiên, và bất kỳ việc dùng đồng hồ hệ thống hay các nguồn dữ liệu bên ngoài nào đều phi xác định. Những nguyên nhân gây phi xác định như vậy cần được loại bỏ nhằm phục hồi khỏi lỗi một cách tin cậy, ví dụ bằng cách sinh các số giả ngẫu nhiên dùng một hạt giống (seed) cố định.

Recovering from faults by recomputing data is not always the right answer: if the intermediate data is much smaller than the source data, or if the computation is very CPU-intensive, it is probably cheaper to materialize the intermediate data to files than to recompute it.

Việc phục hồi khỏi lỗi bằng cách tính lại dữ liệu không phải lúc nào cũng là câu trả lời đúng: nếu dữ liệu trung gian nhỏ hơn nhiều so với dữ liệu nguồn, hoặc nếu phép tính rất thiên về CPU, thì có lẽ vật chất hóa dữ liệu trung gian ra các tệp sẽ rẻ hơn so với tính lại nó.

Discussion of materialization — Bàn luận về sự vật chất hóa

Returning to the Unix analogy, we saw that MapReduce is like writing the output of each command to a temporary file, whereas dataflow engines look much more like Unix pipes. Flink especially is built around the idea of pipelined execution: that is, incrementally passing the output of an operator to other operators, and not waiting for the input to be complete before starting to process it.

Quay lại phép loại suy Unix, ta đã thấy rằng MapReduce giống như việc ghi đầu ra của mỗi lệnh ra một tệp tạm, trong khi các engine luồng dữ liệu trông giống pipe của Unix hơn nhiều. Flink đặc biệt được xây dựng quanh ý tưởng thực thi theo đường ống (pipelined execution): tức là, chuyển đầu ra của một toán tử tới các toán tử khác một cách tăng dần, và không chờ đầu vào hoàn tất trước khi bắt đầu xử lý nó.

A sorting operation inevitably needs to consume its entire input before it can produce any output, because it's possible that the very last input record is the one with the lowest key and thus needs to be the very first output record. Any operator that requires sorting will thus need to accumulate state, at least temporarily. But many other parts of a workflow can be executed in a pipelined manner.

Một thao tác sắp xếp tất yếu cần tiêu thụ toàn bộ đầu vào của nó trước khi nó có thể tạo ra bất kỳ đầu ra nào, bởi có khả năng rằng bản ghi đầu vào cuối cùng chính là bản ghi có khóa nhỏ nhất và do đó cần là bản ghi đầu ra đầu tiên. Bất kỳ toán tử nào đòi hỏi sắp xếp do đó sẽ cần tích lũy trạng thái, ít nhất là tạm thời. Nhưng nhiều phần khác của một luồng công việc có thể được thực thi theo kiểu đường ống.

When the job completes, its output needs to go somewhere durable so that users can find it and use it—most likely, it is written to the distributed filesystem again. Thus, when using a dataflow engine, materialized datasets on HDFS are still usually the inputs and the final outputs of a job. Like with MapReduce, the inputs are immutable and the output is completely replaced. The improvement over MapReduce is that you save yourself writing all the intermediate state to the filesystem as well.

Khi tác vụ hoàn thành, đầu ra của nó cần đi tới đâu đó có độ bền sao cho người dùng có thể tìm thấy nó và dùng nó — nhiều khả năng nhất, nó được ghi ra hệ thống tệp phân tán một lần nữa. Vì vậy, khi dùng một engine luồng dữ liệu, các tập dữ liệu được vật chất hóa trên HDFS vẫn thường là các đầu vào và các đầu ra cuối cùng của một tác vụ. Giống như với MapReduce, các đầu vào là bất biến và đầu ra được thay thế hoàn toàn. Sự cải thiện so với MapReduce là bạn tự tiết kiệm cho mình việc ghi cả tất cả trạng thái trung gian ra hệ thống tệp nữa.

Graphs and Iterative Processing — Đồ thị và xử lý lặp

In “Graph-Like Data Models” we discussed using graphs for modeling data, and using graph query languages to **traverse** the edges and vertices in a graph. The discussion in Chapter 2 was focused around OLTP-style use: quickly executing queries to find a small number of vertices matching certain criteria.

Ở “Các mô hình dữ liệu kiểu đồ thị” ta đã bàn về việc dùng đồ thị để mô hình hóa dữ liệu, và dùng các ngôn ngữ truy vấn đồ thị để duyệt các cạnh và đỉnh trong một đồ thị. Cuộc thảo luận ở Chương 2 tập trung quanh cách dùng kiểu OLTP: nhanh chóng thực thi các truy vấn để tìm một số nhỏ đỉnh khớp với các tiêu chí nhất định.

It is also interesting to look at graphs in a batch processing context, where the goal is to perform some kind of offline processing or analysis on an entire graph. This need often arises in machine learning applications such as recommendation engines, or in ranking systems. For example, one of the most

famous graph analysis algorithms is PageRank [69], which tries to estimate the popularity of a web page based on what other web pages link to it. It is used as part of the formula that determines the order in which web search engines present their results.

Cũng thú vị khi nhìn vào đồ thị trong ngữ cảnh xử lý theo lô, nơi mục tiêu là thực hiện một dạng xử lý hoặc phân tích ngoại tuyến nào đó trên toàn bộ một đồ thị. Nhu cầu này thường nảy sinh trong các ứng dụng học máy chẳng hạn các engine gợi ý, hoặc trong các hệ thống xếp hạng. Ví dụ, một trong những thuật toán phân tích đồ thị nổi tiếng nhất là PageRank [69], vốn cố ước lượng độ phổ biến của một trang web dựa trên những trang web khác liên kết tới nó. Nó được dùng như một phần của công thức quyết định thứ tự mà các engine tìm kiếm web trình bày kết quả của chúng.

Note — Ghi chú Dataflow engines like Spark, Flink, and Tez (see “Materialization of Intermediate State”) typically arrange the operators in a job as a directed acyclic graph (DAG). This is not the same as graph processing: in dataflow engines, the flow of data from one operator to another is structured as a graph, while the data itself typically consists of relational-style tuples. In graph processing, the data itself has the form of a graph. Another unfortunate naming confusion!

Các engine luồng dữ liệu như Spark, Flink, và Tez (xem “Vật chất hóa trạng thái trung gian”) thường sắp xếp các toán tử trong một tác vụ thành một đồ thị có hướng không chu trình (directed acyclic graph - DAG). Đây không giống với xử lý đồ thị: trong các engine luồng dữ liệu, dòng dữ liệu chảy từ một toán tử tới một toán tử khác được cấu trúc thành một đồ thị, trong khi bản thân dữ liệu thường gồm các bộ (tuple) kiểu quan hệ. Trong xử lý đồ thị, bản thân dữ liệu có dạng một đồ thị. Lại một sự nhầm lẫn tên gọi đáng tiếc nữa!

Many graph algorithms are expressed by traversing one edge at a time, joining one **vertex** with an adjacent vertex in order to propagate some information, and repeating until some condition is met—for example, until there are no more edges to follow, or until some metric converges. We saw an example in Figure 2-6, which made a list of all the locations in North America contained in a database by repeatedly following edges indicating which location is within which other location (this kind of algorithm is called a transitive closure).

Nhiều thuật toán đồ thị được diễn đạt bằng cách duyệt một cạnh tại một thời điểm, join một đỉnh với một đỉnh liền kề nhằm lan truyền một thông tin nào đó, và lặp lại cho tới khi một điều kiện nào đó được thỏa mãn — ví dụ, cho tới khi không còn cạnh nào để theo nữa, hoặc cho tới khi một chỉ số nào đó hội tụ. Ta đã thấy một ví dụ ở Hình 2-6, vốn lập một danh sách tất cả các địa điểm ở Bắc Mỹ chứa trong một cơ sở dữ liệu bằng cách lặp đi lặp lại theo các cạnh chỉ ra địa điểm nào nằm trong địa điểm nào khác (loại thuật toán này được gọi là bao đóng bắc cầu - transitive closure).

It is possible to store a graph in a distributed filesystem (in files containing lists of vertices and edges), but this idea of “repeating until done” cannot be expressed in plain MapReduce, since it only performs a single pass over the data. This kind of algorithm is thus often implemented in an iterative style:

Có thể lưu một đồ thị trong một hệ thống tệp phân tán (trong các tệp chứa danh sách các đỉnh và cạnh), nhưng ý tưởng “lặp lại cho tới khi xong” này không thể được diễn đạt trong MapReduce thuần, vì nó chỉ thực hiện một lượt duy nhất qua dữ liệu. Loại thuật toán này do đó thường được hiện thực theo kiểu lặp:

- An external scheduler runs a batch process to calculate one step of the algorithm.
- When the batch process completes, the scheduler checks whether it has finished (based on the completion condition—e.g., there are no more edges to follow, or the change compared to the last iteration is below some threshold).

- If it has not yet finished, the scheduler goes back to step 1 and runs another round of the batch process.
 - Một bộ lập lịch bên ngoài chạy một tiến trình theo lô để tính một bước của thuật toán.
 - Khi tiến trình theo lô hoàn thành, bộ lập lịch kiểm tra xem nó đã xong chưa (dựa trên điều kiện hoàn thành — ví dụ, không còn cạnh nào để theo nữa, hoặc sự thay đổi so với lần lặp trước nằm dưới một ngưỡng nào đó).
 - Nếu nó chưa xong, bộ lập lịch quay lại bước 1 và chạy một vòng nữa của tiến trình theo lô.

This approach works, but implementing it with MapReduce is often very inefficient, because MapReduce does not account for the iterative nature of the algorithm: it will always read the entire input dataset and produce a completely new output dataset, even if only a small part of the graph has changed compared to the last iteration.

Cách tiếp cận này chạy được, nhưng việc hiện thực nó với MapReduce thường rất kém hiệu quả, bởi MapReduce không tính đến bản chất lặp của thuật toán: nó sẽ luôn đọc toàn bộ tập dữ liệu đầu vào và tạo ra một tập dữ liệu đầu ra hoàn toàn mới, ngay cả khi chỉ một phần nhỏ của đồ thị đã thay đổi so với lần lặp trước.

The Pregel processing model — Mô hình xử lý Pregel

As an optimization for batch processing graphs, the bulk synchronous parallel (BSP) model of computation [70] has become popular. Among others, it is implemented by Apache Giraph [37], Spark’s GraphX API, and Flink’s Gelly API [71]. It is also known as the Pregel model, as Google’s Pregel paper popularized this approach for processing graphs [72].

Như một sự tối ưu cho việc xử lý đồ thị theo lô, mô hình tính toán song song đồng bộ khối (bulk synchronous parallel - BSP) [70] đã trở nên phổ biến. Trong số những thứ khác, nó được hiện thực bởi Apache Giraph [37], API GraphX của Spark, và API Gelly của Flink [71]. Nó cũng được biết đến là mô hình Pregel, vì bài báo Pregel của Google đã phổ biến cách tiếp cận này cho việc xử lý đồ thị [72].

Recall that in MapReduce, mappers conceptually “send a message” to a particular call of the reducer because the framework collects together all the mapper outputs with the same key. A similar idea is behind Pregel: one vertex can “send a message” to another vertex, and typically those messages are sent along the edges in a graph.

Hãy nhớ rằng trong MapReduce, về mặt khái niệm các mapper “gửi một thông điệp” tới một lần gọi cụ thể của reducer vì framework gom lại với nhau tất cả các đầu ra của mapper có cùng khóa. Một ý tưởng tương tự nằm sau Pregel: một đỉnh có thể “gửi một thông điệp” tới một đỉnh khác, và thường thì những thông điệp đó được gửi dọc theo các cạnh trong một đồ thị.

In each iteration, a function is called for each vertex, passing it all the messages that were sent to it—much like a call to the reducer. The difference from MapReduce is that in the Pregel model, a vertex remembers its state in memory from one iteration to the next, so the function only needs to process new incoming messages. If no messages are being sent in some part of the graph, no work needs to be done.

Trong mỗi lần lặp, một hàm được gọi cho mỗi đỉnh, chuyển cho nó tất cả các thông điệp đã được gửi tới nó — rất giống một lần gọi tới reducer. Khác biệt so với MapReduce là trong mô hình Pregel, một đỉnh ghi nhớ trạng thái của nó trong bộ nhớ từ lần lặp này sang lần

lặp tiếp theo, nên hàm chỉ cần xử lý các thông điệp đến mới. Nếu không có thông điệp nào đang được gửi ở một phần nào đó của đồ thị, không cần làm việc gì.

It's a bit similar to the actor model (see “Distributed actor frameworks”), if you think of each vertex as an actor, except that vertex state and messages between vertices are **fault-tolerant** and durable, and communication proceeds in fixed rounds: at every iteration, the framework delivers all messages sent in the previous iteration. Actors normally have no such timing guarantee.

Nó hơi giống mô hình tác tử (actor) (xem “Các framework tác tử phân tán”), nếu bạn nghĩ về mỗi đỉnh như một tác tử, ngoại trừ việc trạng thái đỉnh và các thông điệp giữa các đỉnh là chịu lỗi và có độ bền, và việc giao tiếp tiến hành theo các vòng cố định: ở mỗi lần lặp, framework chuyển giao tất cả các thông điệp được gửi trong lần lặp trước. Các tác tử thường không có một bảo đảm về thời điểm như vậy.

Fault tolerance — Khả năng chịu lỗi

The fact that vertices can only communicate by message passing (not by querying each other directly) helps improve the performance of Pregel jobs, since messages can be batched and there is less waiting for communication. The only waiting is between iterations: since the Pregel model guarantees that all messages sent in one iteration are delivered in the next iteration, the prior iteration must completely finish, and all of its messages must be copied over the network, before the next one can start.

Việc các đỉnh chỉ có thể giao tiếp bằng cách truyền thông điệp (chứ không phải bằng cách truy vấn lẫn nhau một cách trực tiếp) giúp cải thiện hiệu năng của các tác vụ Pregel, vì các thông điệp có thể được gom lô và có ít sự chờ đợi giao tiếp hơn. Sự chờ đợi duy nhất là giữa các lần lặp: vì mô hình Pregel bảo đảm rằng tất cả các thông điệp được gửi trong một lần lặp đều được chuyển giao trong lần lặp tiếp theo, lần lặp trước phải hoàn thành hoàn toàn, và tất cả các thông điệp của nó phải được sao chép qua mạng, trước khi lần lặp tiếp theo có thể bắt đầu.

Even though the underlying network may drop, duplicate, or arbitrarily delay messages (see “Unreliable Networks”), Pregel implementations guarantee that messages are processed exactly once at their destination vertex in the following iteration. Like MapReduce, the framework transparently recovers from faults in order to simplify the programming model for algorithms on top of Pregel.

Mặc dù mạng nền tảng có thể đánh rơi, nhân đôi, hoặc làm trễ một cách tùy ý các thông điệp (xem “Mạng không tin cậy”), các bản hiện thực Pregel bảo đảm rằng các thông điệp được xử lý đúng một lần tại đỉnh đích của chúng trong lần lặp tiếp theo. Giống như MapReduce, framework phục hồi khỏi lỗi một cách trong suốt nhằm đơn giản hóa mô hình lập trình cho các thuật toán trên nền Pregel.

This fault tolerance is achieved by periodically checkpointing the state of all vertices at the end of an iteration—i.e., writing their full state to durable storage. If a node fails and its in-memory state is lost, the simplest solution is to roll back the entire graph computation to the last checkpoint and restart the computation. If the algorithm is deterministic and messages are logged, it is also possible to selectively recover only the partition that was lost (like we previously discussed for dataflow engines) [72].

Khả năng chịu lỗi này đạt được bằng cách định kỳ tạo điểm kiểm tra cho trạng thái của tất cả các đỉnh vào cuối một lần lặp — tức là, ghi toàn bộ trạng thái của chúng ra lưu trữ có độ bền. Nếu một nút hỏng và trạng thái trong bộ nhớ của nó bị mất, giải pháp đơn giản nhất là hoàn tác toàn bộ phép tính đồ thị về điểm kiểm tra gần nhất và khởi động lại phép

tính. Nếu thuật toán có tính xác định và các thông điệp được ghi nhật ký, thì cũng có thể chọn lọc phục hồi chỉ phân vùng bị mất (như ta đã bàn trước đó cho các engine luồng dữ liệu) [72].

Parallel execution — Thực thi song song

A vertex does not need to know on which physical machine it is executing; when it sends messages to other vertices, it simply sends them to a vertex ID. It is up to the framework to partition the graph—i.e., to decide which vertex runs on which machine, and how to route messages over the network so that they end up in the right place.

Một đỉnh không cần biết nó đang thực thi trên máy vật lý nào; khi nó gửi các thông điệp tới các đỉnh khác, nó chỉ đơn giản gửi chúng tới một ID đỉnh. Việc phân vùng đồ thị — tức là, quyết định đỉnh nào chạy trên máy nào, và định tuyến các thông điệp qua mạng ra sao để chúng kết thúc ở đúng chỗ — là tùy thuộc vào framework.

Because the programming model deals with just one vertex at a time (sometimes called “thinking like a vertex”), the framework may partition the graph in arbitrary ways. Ideally it would be partitioned such that vertices are colocated on the same machine if they need to communicate a lot. However, finding such an optimized partitioning is hard—in practice, the graph is often simply partitioned by an arbitrarily assigned vertex ID, making no attempt to group related vertices together.

Vì mô hình lập trình chỉ làm việc với một đỉnh tại một thời điểm (đôi khi được gọi là “suy nghĩ như một đỉnh”), framework có thể phân vùng đồ thị theo những cách tùy ý. Lý tưởng là nó sẽ được phân vùng sao cho các đỉnh được đặt cùng nhau trên cùng một máy nếu chúng cần giao tiếp nhiều. Tuy nhiên, việc tìm một sự phân vùng được tối ưu như vậy là khó — trên thực tế, đồ thị thường chỉ đơn giản được phân vùng theo một ID đỉnh được gán tùy ý, không hề cố gom các đỉnh liên quan lại với nhau.

As a result, graph algorithms often have a lot of cross-machine communication overhead, and the intermediate state (messages sent between nodes) is often bigger than the original graph. The overhead of sending messages over the network can significantly slow down distributed graph algorithms.

Kết quả là, các thuật toán đồ thị thường có rất nhiều chi phí giao tiếp liên-máy, và trạng thái trung gian (các thông điệp được gửi giữa các nút) thường lớn hơn so với đồ thị gốc. Chi phí của việc gửi các thông điệp qua mạng có thể làm chậm đáng kể các thuật toán đồ thị phân tán.

For this reason, if your graph can fit in memory on a single computer, it's quite likely that a single-machine (maybe even single-threaded) algorithm will outperform a distributed batch process [73, 74]. Even if the graph is bigger than memory, it can fit on the disks of a single computer, single-machine processing using a framework such as GraphChi is a viable option [75]. If the graph is too big to fit on a single machine, a distributed approach such as Pregel is unavoidable; efficiently parallelizing graph algorithms is an area of ongoing research [76].

Vì lý do này, nếu đồ thị của bạn có thể nhét vừa trong bộ nhớ trên một máy tính duy nhất, thì khá có khả năng rằng một thuật toán trên-một-máy (thậm chí có thể là đơn luồng) sẽ vượt trội hơn một tiến trình theo lô phân tán [73, 74]. Ngay cả khi đồ thị lớn hơn bộ nhớ, nó có thể nhét vừa trên các đĩa của một máy tính duy nhất, việc xử lý trên-một-máy dùng một framework chẳng hạn GraphChi là một lựa chọn khả thi [75]. Nếu đồ thị quá lớn để nhét vừa trên một máy duy nhất, một cách tiếp cận phân tán chẳng hạn Pregel là không thể tránh khỏi; việc song song hóa các thuật toán đồ thị một cách hiệu quả là một lĩnh vực nghiên cứu đang tiếp diễn [76].

High-Level APIs and Languages — Các API và ngôn ngữ ở mức cao

Over the years since MapReduce first became popular, the execution engines for distributed batch processing have matured. By now, the infrastructure has become robust enough to store and process many petabytes of data on clusters of over 10,000 machines. As the problem of physically operating batch processes at such scale has been considered more or less solved, attention has turned to other areas: improving the programming model, improving the efficiency of processing, and broadening the set of problems that these technologies can solve.

Qua nhiều năm kể từ khi MapReduce lần đầu trở nên phổ biến, các engine thực thi cho xử lý theo lô phân tán đã trưởng thành. Đến nay, hạ tầng đã trở nên đủ bền bỉ để lưu trữ và xử lý nhiều petabyte dữ liệu trên các cụm hơn 10.000 máy. Vì bài toán vận hành các tiến trình theo lô về mặt vật lý ở quy mô như vậy đã được coi là ít nhiều đã được giải quyết, sự chú ý đã chuyển sang các lĩnh vực khác: cải thiện mô hình lập trình, cải thiện hiệu quả của việc xử lý, và mở rộng tập các bài toán mà những công nghệ này có thể giải quyết.

As discussed previously, higher-level languages and APIs such as Hive, Pig, Cascading, and Crunch became popular because programming MapReduce jobs by hand is quite laborious. As Tez emerged, these high-level languages had the additional benefit of being able to move to the new dataflow execution engine without the need to rewrite job code. Spark and Flink also include their own high-level dataflow APIs, often taking inspiration from FlumeJava [34].

Như đã bàn trước đó, các ngôn ngữ và API ở mức cao hơn chẳng hạn Hive, Pig, Cascading, và Crunch trở nên phổ biến bởi việc lập trình các tác vụ MapReduce bằng tay khá cực nhọc. Khi Tez xuất hiện, các ngôn ngữ ở mức cao này có thêm lợi ích là có thể chuyển sang engine thực thi luồng dữ liệu mới mà không cần viết lại mã tác vụ. Spark và Flink cũng bao gồm các API luồng dữ liệu ở mức cao của riêng chúng, thường lấy cảm hứng từ FlumeJava [34].

These dataflow APIs generally use relational-style building blocks to express a computation: joining datasets on the value of some field; grouping tuples by key; filtering by some condition; and aggregating tuples by counting, summing, or other functions. Internally, these operations are implemented using the various join and grouping algorithms that we discussed earlier in this chapter.

Các API luồng dữ liệu này nói chung dùng các khối tiêu chuẩn kiểu quan hệ để diễn đạt một phép tính: join các tập dữ liệu trên giá trị của một trường nào đó; gom nhóm các bộ theo khóa; lọc theo một điều kiện nào đó; và tổng hợp các bộ bằng cách đếm, cộng dồn, hoặc các hàm khác. Bên trong, các thao tác này được hiện thực dùng các thuật toán join và gom nhóm khác nhau mà ta đã bàn trước đó trong chương này.

Besides the obvious advantage of requiring less code, these high-level interfaces also allow interactive use, in which you write analysis code incrementally in a shell and run it frequently to observe what it is doing. This style of development is very helpful when exploring a dataset and experimenting with approaches for processing it. It is also reminiscent of the Unix philosophy, which we discussed in “The Unix Philosophy”.

Ngoài ưu điểm hiển nhiên là đòi hỏi ít mã hơn, các giao diện ở mức cao này cũng cho phép việc dùng tương tác, trong đó bạn viết mã phân tích một cách tăng dần trong một shell và chạy nó thường xuyên để quan sát nó đang làm gì. Phong cách phát triển này rất hữu ích khi khám phá một tập dữ liệu và thử nghiệm các cách tiếp cận để xử lý nó. Nó cũng gợi nhớ tới triết lý Unix, mà ta đã bàn ở “Triết lý Unix”.

Moreover, these high-level interfaces not only make the humans using the system more productive, but they also improve the job execution efficiency at a machine level.

Hơn nữa, các giao diện ở mức cao này không chỉ khiến những con người dùng hệ thống năng suất hơn, mà chúng còn cải thiện hiệu quả thực thi tác vụ ở mức máy.

The move toward declarative query languages — Sự chuyển dịch về phía các ngôn ngữ truy vấn khai báo

An advantage of specifying joins as relational operators, compared to spelling out the code that performs the join, is that the framework can analyze the properties of the join inputs and automatically decide which of the aforementioned join algorithms would be most suitable for the task at hand. Hive, Spark, and Flink have cost-based query optimizers that can do this, and even change the order of joins so that the amount of intermediate state is minimized [66, 77, 78, 79].

Một ưu điểm của việc chỉ định các phép join thành các toán tử quan hệ, so với việc viết rõ mã thực hiện phép join, là framework có thể phân tích các thuộc tính của các đầu vào join và tự động quyết định thuật toán join nào trong số đã nói ở trên sẽ phù hợp nhất cho tác vụ trước mắt. Hive, Spark, và Flink có các bộ tối ưu truy vấn dựa trên chi phí (cost-based) có thể làm điều này, và thậm chí thay đổi thứ tự của các phép join sao cho lượng trạng thái trung gian được giảm thiểu [66, 77, 78, 79].

The choice of join algorithm can make a big difference to the performance of a batch job, and it is nice not to have to understand and remember all the various join algorithms we discussed in this chapter. This is possible if joins are specified in a **declarative** way: the application simply states which joins are required, and the query optimizer decides how they can best be executed. We previously came across this idea in “Query Languages for Data”.

Việc lựa chọn thuật toán join có thể tạo ra một khác biệt lớn cho hiệu năng của một tác vụ theo lô, và thật tốt khi không phải hiểu và ghi nhớ tất cả các thuật toán join khác nhau mà ta đã bàn trong chương này. Điều này khả thi nếu các phép join được chỉ định theo cách khai báo: ứng dụng chỉ đơn giản phát biểu những phép join nào được cần, và bộ tối ưu truy vấn quyết định cách chúng có thể được thực thi tốt nhất. Ta đã gặp ý tưởng này trước đó ở “Các ngôn ngữ truy vấn cho dữ liệu”.

However, in other ways, MapReduce and its dataflow successors are very different from the fully declarative query model of SQL. MapReduce was built around the idea of function callbacks: for each record or group of records, a user-defined function (the mapper or reducer) is called, and that function is free to call arbitrary code in order to decide what to output. This approach has the advantage that you can draw upon a large ecosystem of existing libraries to do things like parsing, natural language analysis, image analysis, and running numerical or statistical algorithms.

Tuy nhiên, ở những mặt khác, MapReduce và các hậu duệ luồng dữ liệu của nó rất khác với mô hình truy vấn hoàn toàn khai báo của SQL. MapReduce được xây dựng quanh ý tưởng các callback hàm: với mỗi bản ghi hoặc nhóm bản ghi, một hàm do người dùng định nghĩa (mapper hoặc reducer) được gọi, và hàm đó được tự do gọi mã tùy ý nhằm quyết định xuất ra cái gì. Cách tiếp cận này có ưu điểm là bạn có thể tận dụng một hệ sinh thái lớn các thư viện hiện có để làm những việc như phân tích cú pháp, phân tích ngôn ngữ tự nhiên, phân tích hình ảnh, và chạy các thuật toán số học hoặc thống kê.

The freedom to easily run arbitrary code is what has long distinguished batch processing systems of MapReduce heritage from MPP databases (see “Comparing Hadoop to Distributed Databases”); although databases have facilities for writing user-defined functions, they are often cumbersome to use and not well integrated with the package managers and dependency management systems that

are widely used in most programming languages (such as Maven for Java, npm for JavaScript, and Rubygems for Ruby).

Sự tự do dễ dàng chạy mã tùy ý là thứ đã từ lâu phân biệt các hệ thống xử lý theo lô có gốc gác MapReduce với các cơ sở dữ liệu MPP (xem “So sánh Hadoop với các cơ sở dữ liệu phân tán”); mặc dù các cơ sở dữ liệu có các tiện ích để viết các hàm do người dùng định nghĩa, chúng thường cồng kềnh để dùng và không được tích hợp tốt với các trình quản lý gói và các hệ thống quản lý phụ thuộc vốn được dùng rộng rãi trong hầu hết các ngôn ngữ lập trình (chẳng hạn Maven cho Java, npm cho JavaScript, và Rubygems cho Ruby).

However, dataflow engines have found that there are also advantages to incorporating more declarative features in areas besides joins. For example, if a callback function contains only a simple filtering condition, or it just selects some fields from a record, then there is significant CPU overhead in calling the function on every record. If such simple filtering and mapping operations are expressed in a declarative way, the query optimizer can take advantage of column-oriented storage layouts (see “Column-Oriented Storage”) and read only the required columns from disk. Hive, Spark DataFrames, and Impala also use vectorized execution (see “Memory bandwidth and vectorized processing”): iterating over data in a tight inner loop that is friendly to CPU caches, and avoiding function calls. Spark generates JVM bytecode [79] and Impala uses LLVM to generate native code for these inner loops [41].

Tuy nhiên, các engine luồng dữ liệu đã thấy rằng cũng có những ưu điểm trong việc kết hợp nhiều tính năng khai báo hơn ở các lĩnh vực ngoài phép join. Ví dụ, nếu một hàm callback chỉ chứa một điều kiện lọc đơn giản, hoặc nó chỉ chọn một số trường từ một bản ghi, thì có chi phí CPU đáng kể trong việc gọi hàm trên mỗi bản ghi. Nếu những thao tác lọc và ánh xạ đơn giản như vậy được diễn đạt theo cách khai báo, bộ tối ưu truy vấn có thể tận dụng các bố cục lưu trữ hướng cột (xem “Lưu trữ hướng cột”) và chỉ đọc các cột cần thiết từ đĩa. Hive, Spark DataFrames, và Impala cũng dùng thực thi vector hóa (vectorized execution) (xem “Bảng thông bộ nhớ và xử lý vector hóa”): duyệt qua dữ liệu trong một vòng lặp trong chặt chẽ vốn thân thiện với các cache của CPU, và tránh các lời gọi hàm. Spark sinh ra bytecode JVM [79] còn Impala dùng LLVM để sinh mã gốc (native code) cho các vòng lặp trong này [41].

By incorporating declarative aspects in their high-level APIs, and having query optimizers that can take advantage of them during execution, batch processing frameworks begin to look more like MPP databases (and can achieve comparable performance). At the same time, by having the **extensibility** of being able to run arbitrary code and read data in arbitrary formats, they retain their flexibility advantage.

Bằng cách kết hợp các khía cạnh khai báo trong các API ở mức cao của chúng, và có các bộ tối ưu truy vấn có thể tận dụng chúng trong khi thực thi, các framework xử lý theo lô bắt đầu trông giống các cơ sở dữ liệu MPP hơn (và có thể đạt được hiệu năng tương đương). Đồng thời, bằng cách có khả năng mở rộng để chạy mã tùy ý và đọc dữ liệu ở các định dạng tùy ý, chúng giữ lại ưu điểm về tính linh hoạt của mình.

Specialization for different domains — Sự chuyên biệt hóa cho các lĩnh vực khác nhau

While the extensibility of being able to run arbitrary code is useful, there are also many common cases where standard processing patterns keep reoccurring, and so it is worth having reusable implementations of the common building blocks. Traditionally, MPP databases have served the needs of business intelligence analysts and business reporting, but that is just one among many domains in which batch processing is used.

Trong khi khả năng mở rộng để chạy mã tùy ý là hữu ích, cũng có nhiều trường hợp phổ biến nơi các mẫu xử lý chuẩn cứ tái diễn, nên đáng có các bản hiện thực có thể tái sử dụng của các khối tiêu chuẩn phổ biến. Theo truyền thống, các cơ sở dữ liệu MPP đã phục vụ các nhu cầu của các nhà phân tích trí tuệ kinh doanh (business intelligence) và lập báo cáo kinh doanh, nhưng đó chỉ là một trong nhiều lĩnh vực mà xử lý theo lô được dùng.

Another domain of increasing importance is statistical and numerical algorithms, which are needed for machine learning applications such as classification and recommendation systems. Reusable implementations are emerging: for example, Mahout implements various algorithms for machine learning on top of MapReduce, Spark, and Flink, while MADlib implements similar functionality inside a relational MPP database (Apache HAWQ) [54].

Một lĩnh vực khác có tầm quan trọng ngày càng tăng là các thuật toán thống kê và số học, vốn cần cho các ứng dụng học máy chẳng hạn các hệ thống phân loại và gợi ý. Các bản hiện thực có thể tái sử dụng đang xuất hiện: ví dụ, Mahout hiện thực nhiều thuật toán cho học máy trên nền MapReduce, Spark, và Flink, trong khi MADlib hiện thực chức năng tương tự bên trong một cơ sở dữ liệu MPP quan hệ (Apache HAWQ) [54].

Also useful are spatial algorithms such as k-nearest neighbors [80], which searches for items that are close to a given item in some multi-dimensional space—a kind of similarity search. Approximate search is also important for genome analysis algorithms, which need to find strings that are similar but not identical [81].

Cũng hữu ích là các thuật toán không gian chẳng hạn k-láng-giềng-gần-nhất (k-nearest neighbors) [80], vốn tìm các mục gần với một mục cho trước trong một không gian đa chiều nào đó — một kiểu tìm kiếm tương tự. Tìm kiếm xấp xỉ cũng quan trọng cho các thuật toán phân tích bộ gen, vốn cần tìm các chuỗi tương tự nhưng không đồng nhất [81].

Batch processing engines are being used for distributed execution of algorithms from an increasingly wide range of domains. As batch processing systems gain built-in functionality and high-level declarative operators, and as MPP databases become more programmable and flexible, the two are beginning to look more alike: in the end, they are all just systems for storing and processing data.

Các engine xử lý theo lô đang được dùng cho việc thực thi phân tán các thuật toán từ một phạm vi ngày càng rộng các lĩnh vực. Khi các hệ thống xử lý theo lô có thêm chức năng tích hợp sẵn và các toán tử khai báo ở mức cao, và khi các cơ sở dữ liệu MPP trở nên dễ lập trình và linh hoạt hơn, hai cái bắt đầu trông giống nhau hơn: cuối cùng, chúng đều chỉ là các hệ thống để lưu trữ và xử lý dữ liệu.

Summary — Tóm tắt

In this chapter we explored the topic of batch processing. We started by looking at Unix tools such as `awk`, `grep`, and `sort`, and we saw how the design philosophy of those tools is carried forward into MapReduce and more recent dataflow engines. Some of those design principles are that inputs are immutable, outputs are intended to become the input to another (as yet unknown) program, and complex problems are solved by composing small tools that “do one thing well.”

Trong chương này ta đã khám phá chủ đề xử lý theo lô. Ta bắt đầu bằng việc nhìn vào các công cụ Unix như `awk`, `grep`, và `sort`, và ta đã thấy cách triết lý thiết kế của những công cụ đó được mang tiếp vào MapReduce và các engine luồng dữ liệu gần đây hơn. Một số trong những nguyên lý thiết kế đó là các đầu vào là bất biến, các đầu ra được dự định trở thành đầu vào của một chương trình khác (chưa biết trước), và các bài toán phức tạp được giải bằng cách kết hợp các công cụ nhỏ “làm tốt một việc”.

In the Unix world, the uniform interface that allows one program to be composed with another is files and pipes; in MapReduce, that interface is a distributed filesystem. We saw that dataflow engines add their own pipe-like data transport mechanisms to avoid materializing intermediate state to the distributed filesystem, but the initial input and final output of a job is still usually HDFS.

Trong thế giới Unix, giao diện đồng nhất cho phép một chương trình được kết hợp với một chương trình khác là các tệp và các pipe; trong MapReduce, giao diện đó là một hệ thống tệp phân tán. Ta đã thấy rằng các engine luồng dữ liệu thêm vào các cơ chế vận chuyển dữ liệu kiểu-pipe của riêng chúng nhằm tránh việc vật chất hóa trạng thái trung gian ra hệ thống tệp phân tán, nhưng đầu vào ban đầu và đầu ra cuối cùng của một tác vụ vẫn thường là HDFS.

The two main problems that distributed batch processing frameworks need to solve are:

Hai vấn đề chính mà các framework xử lý theo lô phân tán cần giải quyết là:

In MapReduce, mappers are partitioned according to input file blocks. The output of mappers is repartitioned, sorted, and merged into a configurable number of reducer partitions. The purpose of this process is to bring all the related data—e.g., all the records with the same key—together in the same place.

Trong MapReduce, các mapper được phân vùng theo các khối tệp đầu vào. Đầu ra của các mapper được phân vùng lại, sắp xếp, và trộn thành một số phân vùng reducer có thể cấu hình. Mục đích của quá trình này là đưa tất cả dữ liệu liên quan — ví dụ, tất cả các bản ghi có cùng khóa — về cùng một chỗ.

Post-MapReduce dataflow engines try to avoid sorting unless it is required, but they otherwise take a broadly similar approach to partitioning.

Các engine luồng dữ liệu hậu-MapReduce cố tránh việc sắp xếp trừ khi nó được cần, nhưng

ngoài ra chúng đi theo một cách tiếp cận đại thể tương tự đối với việc phân vùng.

MapReduce frequently writes to disk, which makes it easy to recover from an individual failed task without restarting the entire job but slows down execution in the failure-free case. Dataflow engines perform less materialization of intermediate state and keep more in memory, which means that they need to recompute more data if a node fails. Deterministic operators reduce the amount of data that needs to be recomputed.

MapReduce thường xuyên ghi ra đĩa, điều khiến cho việc phục hồi khỏi một tác vụ con hỏng riêng lẻ trở nên dễ dàng mà không cần khởi động lại toàn bộ tác vụ nhưng làm chậm việc thực thi trong trường hợp không-lỗi. Các engine luồng dữ liệu thực hiện ít sự vật chất hóa trạng thái trung gian hơn và giữ nhiều hơn trong bộ nhớ, điều nghĩa là chúng cần tính lại nhiều dữ liệu hơn nếu một nút hỏng. Các toán tử có tính xác định làm giảm lượng dữ liệu cần được tính lại.

We discussed several join algorithms for MapReduce, most of which are also internally used in MPP databases and dataflow engines. They also provide a good illustration of how partitioned algorithms work:

Ta đã bàn về vài thuật toán join cho MapReduce, hầu hết trong số đó cũng được dùng bên trong các cơ sở dữ liệu MPP và các engine luồng dữ liệu. Chúng cũng cung cấp một minh họa tốt về cách các thuật toán được phân vùng hoạt động:

Each of the inputs being joined goes through a mapper that extracts the join key. By partitioning, sorting, and merging, all the records with the same key end up going to the same call of the reducer. This function can then output the joined records.

Mỗi đầu vào đang được join đi qua một mapper vốn trích xuất khóa join. Bằng cách phân vùng, sắp xếp, và trộn, tất cả các bản ghi có cùng khóa kết thúc đi tới cùng một lần gọi của reducer. Hàm này khi đó có thể xuất ra các bản ghi đã được join.

One of the two join inputs is small, so it is not partitioned and it can be entirely loaded into a hash table. Thus, you can start a mapper for each partition of the large join input, load the hash table for the small input into each mapper, and then scan over the large input one record at a time, querying the hash table for each record.

Một trong hai đầu vào join là nhỏ, nên nó không được phân vùng và nó có thể được nạp toàn bộ vào một bảng băm. Như vậy, bạn có thể khởi động một mapper cho mỗi phân vùng của đầu vào join lớn, nạp bảng băm cho đầu vào nhỏ vào mỗi mapper, và sau đó quét qua đầu vào lớn một bản ghi tại một thời điểm, truy vấn bảng băm cho mỗi bản ghi.

If the two join inputs are partitioned in the same way (using the same key, same hash function, and same number of partitions), then the hash table approach can be used independently for each partition.

Nếu hai đầu vào join được phân vùng theo cùng một cách (dùng cùng khóa, cùng hàm băm, và cùng số phân vùng), thì cách tiếp cận bảng băm có thể được dùng một cách độc lập cho mỗi phân vùng.

Distributed batch processing engines have a deliberately restricted programming model: callback functions (such as mappers and reducers) are assumed to be **stateless** and to have no externally visible side effects besides their designated output. This restriction allows the framework to hide some of the hard distributed systems problems behind its abstraction: in the face of crashes and network issues, tasks can be retried safely, and the output from any failed tasks is discarded. If several tasks for a partition succeed, only one of them actually makes its output visible.

Các engine xử lý theo lô phân tán có một mô hình lập trình bị hạn chế một cách có chủ ý: các hàm callback (chẳng hạn mapper và reducer) được giả định là không trạng thái và không có tác dụng phụ hiển thị ra bên ngoài nào ngoài đầu ra được chỉ định của chúng. Sự hạn chế này cho phép framework giấu một số trong các vấn đề hệ thống phân tán hóa búa đăng sau trừu tượng hóa của nó: khi đối mặt với các sự sụp đổ và các vấn đề mạng, các tác vụ có thể được thử lại một cách an toàn, và đầu ra từ bất kỳ tác vụ hỏng nào đều bị loại bỏ. Nếu vài tác vụ cho một phân vùng thành công, chỉ một trong số chúng thực sự làm cho đầu ra của nó hiển thị.

Thanks to the framework, your code in a batch processing job does not need to worry about implementing fault-tolerance mechanisms: the framework can guarantee that the final output of a job is the same as if no faults had occurred, even though in reality various tasks perhaps had to be retried. These reliable semantics are much stronger than what you usually have in online services that handle user requests and that write to databases as a **side effect** of processing a request.

Nhờ vào framework, mã của bạn trong một tác vụ xử lý theo lô không cần lo lắng về việc hiện thực các cơ chế chịu lỗi: framework có thể bảo đảm rằng đầu ra cuối cùng của một tác vụ là giống như nếu không có lỗi nào xảy ra, dù trên thực tế nhiều tác vụ có lẽ đã phải được thử lại. Các ngữ nghĩa tin cậy này mạnh hơn nhiều so với những gì bạn thường có trong các dịch vụ trực tuyến vốn xử lý các yêu cầu của người dùng và ghi tới các cơ sở dữ liệu như một tác dụng phụ của việc xử lý một yêu cầu.

The distinguishing feature of a batch processing job is that it reads some input data and produces some output data, without modifying the input—in other words, the output is derived from the input. Crucially, the input data is bounded: it has a known, fixed size (for example, it consists of a set of log files at some point in time, or a snapshot of a database's contents). Because it is bounded, a job knows when it has finished reading the entire input, and so a job eventually completes when it is done.

Đặc điểm phân biệt của một tác vụ xử lý theo lô là nó đọc một số dữ liệu đầu vào và tạo ra một số dữ liệu đầu ra, mà không sửa đổi đầu vào — nói cách khác, đầu ra được dẫn xuất từ đầu vào. Quan trọng là, dữ liệu đầu vào bị giới hạn (bounded): nó có một kích thước đã biết, cố định (ví dụ, nó gồm một tập các tệp nhật ký tại một thời điểm nào đó, hoặc một ảnh chụp nội dung của một cơ sở dữ liệu). Vì nó bị giới hạn, một tác vụ biết khi nào nó đã đọc xong toàn bộ đầu vào, và do đó một tác vụ rất cuộc hoàn thành khi nó xong.

In the next chapter, we will turn to stream processing, in which the input is unbounded—that is, you still have a job, but its inputs are never-ending streams of data. In this case, a job is never complete, because at any time there may still be more work coming in. We shall see that stream and batch processing are similar in some respects, but the assumption of unbounded streams also changes a lot about how we build systems.

Trong chương tiếp theo, ta sẽ chuyển sang xử lý luồng, trong đó đầu vào là không giới hạn (unbounded) — tức là, bạn vẫn có một tác vụ, nhưng các đầu vào của nó là những luồng dữ liệu bất tận. Trong trường hợp này, một tác vụ không bao giờ hoàn thành, bởi tại bất kỳ thời điểm nào vẫn có thể có thêm công việc đang đến. Ta sẽ thấy rằng xử lý luồng và xử lý theo lô tương tự nhau ở một số mặt, nhưng giả định về các luồng không giới hạn cũng thay đổi rất nhiều về cách ta xây dựng hệ thống.

Footnotes Some people love to point out that cat is unnecessary here, as the input file could be given directly as an argument to awk. However, the linear pipeline is more apparent when written like this.

Another example of a uniform interface is URLs and HTTP, the foundations of the web. A URL identifies a particular thing (resource) on a website, and you can link to any URL from any other

website. A user with a web browser can thus seamlessly jump between websites by following links, even though the servers may be operated by entirely unrelated organizations. This principle seems obvious today, but it was a key insight in making the web the success that it is today. Prior systems were not so uniform: for example, in the era of bulletin board systems (BBSs), each system had its own phone number and baud rate configuration. A reference from one BBS to another would have to be in the form of a phone number and modem settings; the user would have to hang up, dial the other BBS, and then manually find the information they were looking for. It wasn't possible to link directly to some piece of content inside another BBS.

Except by using a separate tool, such as netcat or curl. Unix started out trying to represent everything as files, but the BSD sockets API deviated from that convention [17]. The research operating systems Plan 9 and Inferno are more consistent in their use of files: they represent a TCP connection as a file in `/net/tcp` [18].

One difference is that with HDFS, computing tasks can be scheduled to run on the machine that stores a copy of a particular file, whereas object stores usually keep storage and computation separate. Reading from a local disk has a performance advantage if network bandwidth is a bottleneck. Note however that if erasure coding is used, the locality advantage is lost, because the data from several machines must be combined in order to reconstitute the original file [20].

The joins we talk about in this book are generally equi-joins, the most common type of join, in which a record is associated with other records that have an identical value in a particular field (such as an ID). Some databases support more general types of joins, for example using a less-than operator instead of an equality operator, but we do not have space to cover them here.

This example assumes that there is exactly one entry for each key in the hash table, which is probably true with a user database (a user ID uniquely identifies a user). In general, the hash table may need to contain several entries with the same key, and the join operator will output all matches for a key.

References [1] Jeffrey Dean and Sanjay Ghemawat: "MapReduce: Simplified Data Processing on Large Clusters," at 6th USENIX Symposium on Operating System Design and Implementation (OSDI), December 2004.

[2] Joel Spolsky: "The Perils of JavaSchools," joelonsoftware.com, December 25, 2005.

[3] Shivnath Babu and Herodotos Herodotou: "Massively Parallel Databases and MapReduce Systems," *Foundations and Trends in Databases*, volume 5, number 1, pages 1–104, November 2013. doi:10.1561/19000000036

[4] David J. DeWitt and Michael Stonebraker: "MapReduce: A Major Step Backwards," originally published at databasecolumn.vertica.com, January 17, 2008.

[5] Henry Robinson: "The Elephant Was a Trojan Horse: On the Death of Map-Reduce at Google," the-paper-trail.org, June 25, 2014.

[6] "The Hollerith Machine," United States Census Bureau, census.gov.

[7] "IBM 82, 83, and 84 Sorters Reference Manual," Edition A24-1034-1, International Business Machines Corporation, July 1962.

[8] Adam Drake: "Command-Line Tools Can Be 235x Faster than Your Hadoop Cluster," aadrake.com, January 25, 2014.

[9] "GNU Coreutils 8.23 Documentation," Free Software Foundation, Inc., 2014.

[10] Martin Kleppmann: "Kafka, Samza, and the Unix Philosophy of Distributed Data," martin.kleppmann.com, August 5, 2015.

- [11] Doug McIlroy: Internal Bell Labs memo, October 1964. Cited in: Dennis M. Richie: “Advice from Doug McIlroy,” cm.bell-labs.com.
- [12] M. D. McIlroy, E. N. Pinson, and B. A. Tague: “UNIX Time-Sharing System: Foreword,” *The Bell System Technical Journal*, volume 57, number 6, pages 1899–1904, July 1978.
- [13] Eric S. Raymond: *The Art of UNIX Programming*. Addison-Wesley, 2003. ISBN: 978-0-13-142901-7
- [14] Ronald Duncan: “Text File Formats – ASCII Delimited Text – Not CSV or TAB Delimited Text,” ronaldduncan.wordpress.com, October 31, 2009.
- [15] Alan Kay: “Is ‘Software Engineering’ an Oxymoron?,” tinlizzie.org.
- [16] Martin Fowler: “InversionOfControl,” martinfowler.com, June 26, 2005.
- [17] Daniel J. Bernstein: “Two File Descriptors for Sockets,” cryp.to.
- [18] Rob Pike and Dennis M. Ritchie: “The Styx Architecture for Distributed Systems,” *Bell Labs Technical Journal*, volume 4, number 2, pages 146–152, April 1999.
- [19] Sanjay Ghemawat, Howard Gobioff, and Shun-Tak Leung: “The Google File System,” at 19th ACM Symposium on Operating Systems Principles (SOSP), October 2003. doi:10.1145/945445.945450
- [20] Michael Ovsianikov, Silviu Rus, Damian Reeves, et al.: “The Quantcast File System,” *Proceedings of the VLDB Endowment*, volume 6, number 11, pages 1092–1101, August 2013. doi:10.14778/2536222.2536234
- [21] “OpenStack Swift 2.6.1 Developer Documentation,” OpenStack Foundation, docs.openstack.org, March 2016.
- [22] Zhe Zhang, Andrew Wang, Kai Zheng, et al.: “Introduction to HDFS Erasure Coding in Apache Hadoop,” blog.cloudera.com, September 23, 2015.
- [23] Peter Cnudde: “Hadoop Turns 10,” yahoohadoop.tumblr.com, February 5, 2016.
- [24] Eric Baldeschwieler: “Thinking About the HDFS vs. Other Storage Technologies,” hortonworks.com, July 25, 2012.
- [25] Brendan Gregg: “Manta: Unix Meets Map Reduce,” dtrace.org, June 25, 2013.
- [26] Tom White: *Hadoop: The Definitive Guide*, 4th edition. O’Reilly Media, 2015. ISBN: 978-1-491-90163-2
- [27] Jim N. Gray: “Distributed Computing Economics,” Microsoft Research Tech Report MSR-TR-2003-24, March 2003.
- [28] Márton Trencsényi: “Luigi vs Airflow vs Pinball,” bytepawn.com, February 6, 2016.
- [29] Roshan Sumbaly, Jay Kreps, and Sam Shah: “The ‘Big Data’ Ecosystem at LinkedIn,” at ACM International Conference on Management of Data (SIGMOD), July 2013. doi:10.1145/2463676.2463707
- [30] Alan F. Gates, Olga Natkovich, Shubham Chopra, et al.: “Building a High-Level Dataflow System on Top of Map-Reduce: The Pig Experience,” at 35th International Conference on Very Large Data Bases (VLDB), August 2009.
- [31] Ashish Thusoo, Joydeep Sen Sarma, Namit Jain, et al.: “Hive – A Petabyte Scale Data Warehouse Using Hadoop,” at 26th IEEE International Conference on Data Engineering (ICDE), March 2010. doi:10.1109/ICDE.2010.5447738
- [32] “Cascading 3.0 User Guide,” Concurrent, Inc., docs.cascading.org, January 2016.
- [33] “Apache Crunch User Guide,” Apache Software Foundation, crunch.apache.org.

- [34] Craig Chambers, Ashish Raniwala, Frances Perry, et al.: “FlumeJava: Easy, Efficient Data-Parallel Pipelines,” at 31st ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI), June 2010. doi:10.1145/1806596.1806638
- [35] Jay Kreps: “Why Local State is a Fundamental Primitive in Stream Processing,” oreilly.com, July 31, 2014.
- [36] Martin Kleppmann: “Rethinking Caching in Web Apps,” martin.kleppmann.com, October 1, 2012.
- [37] Mark Grover, Ted Malaska, Jonathan Seidman, and Gwen Shapira: Hadoop Application Architectures. O’Reilly Media, 2015. ISBN: 978-1-491-90004-8
- [38] Philippe Ajoux, Nathan Bronson, Sanjeev Kumar, et al.: “Challenges to Adopting Stronger Consistency at Scale,” at 15th USENIX Workshop on Hot Topics in Operating Systems (HotOS), May 2015.
- [39] Sriranjana Manjunath: “Skewed Join,” wiki.apache.org, 2009.
- [40] David J. DeWitt, Jeffrey F. Naughton, Donovan A. Schneider, and S. Seshadri: “Practical Skew Handling in Parallel Joins,” at 18th International Conference on Very Large Data Bases (VLDB), August 1992.
- [41] Marcel Kornacker, Alexander Behm, Victor Bittorf, et al.: “Impala: A Modern, Open-Source SQL Engine for Hadoop,” at 7th Biennial Conference on Innovative Data Systems Research (CIDR), January 2015.
- [42] Matthieu Monsch: “Open-Sourcing PalDB, a Lightweight Companion for Storing Side Data,” engineering.linkedin.com, October 26, 2015.
- [43] Daniel Peng and Frank Dabek: “Large-Scale Incremental Processing Using Distributed Transactions and Notifications,” at 9th USENIX conference on Operating Systems Design and Implementation (OSDI), October 2010.
- [44] “Cloudera Search User Guide,” Cloudera, Inc., September 2015.
- [45] Lili Wu, Sam Shah, Sean Choi, et al.: “The Browsemaps: Collaborative Filtering at LinkedIn,” at 6th Workshop on Recommender Systems and the Social Web (RSWeb), October 2014.
- [46] Roshan Sumbaly, Jay Kreps, Lei Gao, et al.: “Serving Large-Scale Batch Computed Data with Project Voldemort,” at 10th USENIX Conference on File and Storage Technologies (FAST), February 2012.
- [47] Varun Sharma: “Open-Sourcing Terrapin: A Serving System for Batch Generated Data,” engineering.pinterest.com, September 14, 2015.
- [48] Nathan Marz: “ElephantDB,” slideshare.net, May 30, 2011.
- [49] Jean-Daniel (JD) Cryans: “How-to: Use HBase Bulk Loading, and Why,” blog.cloudera.com, September 27, 2013.
- [50] Nathan Marz: “How to Beat the CAP Theorem,” nathanmarz.com, October 13, 2011.
- [51] Molly Bartlett Dishman and Martin Fowler: “Agile Architecture,” at O’Reilly Software Architecture Conference, March 2015.
- [52] David J. DeWitt and Jim N. Gray: “Parallel Database Systems: The Future of High Performance Database Systems,” Communications of the ACM, volume 35, number 6, pages 85–98, June 1992. doi:10.1145/129888.129894

- [53] Jay Kreps: “But the multi-tenancy thing is actually really really hard,” tweetstorm, twitter.com, October 31, 2014.
- [54] Jeffrey Cohen, Brian Dolan, Mark Dunlap, et al.: “MAD Skills: New Analysis Practices for Big Data,” *Proceedings of the VLDB Endowment*, volume 2, number 2, pages 1481–1492, August 2009. doi:10.14778/1687553.1687576
- [55] Ignacio Terrizzano, Peter Schwarz, Mary Roth, and John E. Colino: “Data Wrangling: The Challenging Journey from the Wild to the Lake,” at 7th Biennial Conference on Innovative Data Systems Research (CIDR), January 2015.
- [56] Paige Roberts: “To Schema on Read or to Schema on Write, That Is the Hadoop Data Lake Question,” *adaptivesystemsinc.com*, July 2, 2015.
- [57] Bobby Johnson and Joseph Adler: “The Sushi Principle: Raw Data Is Better,” at *Strata+Hadoop World*, February 2015.
- [58] Vinod Kumar Vavilapalli, Arun C. Murthy, Chris Douglas, et al.: “Apache Hadoop YARN: Yet Another Resource Negotiator,” at 4th ACM Symposium on Cloud Computing (SoCC), October 2013. doi:10.1145/2523616.2523633
- [59] Abhishek Verma, Luis Pedrosa, Madhukar Korupolu, et al.: “Large-Scale Cluster Management at Google with Borg,” at 10th European Conference on Computer Systems (EuroSys), April 2015. doi:10.1145/2741948.2741964
- [60] Malte Schwarzkopf: “The Evolution of Cluster Scheduler Architectures,” *firmament.io*, March 9, 2016.
- [61] Matei Zaharia, Mosharaf Chowdhury, Tathagata Das, et al.: “Resilient Distributed Datasets: A Fault-Tolerant Abstraction for In-Memory Cluster Computing,” at 9th USENIX Symposium on Networked Systems Design and Implementation (NSDI), April 2012.
- [62] Holden Karau, Andy Konwinski, Patrick Wendell, and Matei Zaharia: *Learning Spark*. O’Reilly Media, 2015. ISBN: 978-1-449-35904-1
- [63] Bikas Saha and Hitesh Shah: “Apache Tez: Accelerating Hadoop Query Processing,” at *Hadoop Summit*, June 2014.
- [64] Bikas Saha, Hitesh Shah, Siddharth Seth, et al.: “Apache Tez: A Unifying Framework for Modeling and Building Data Processing Applications,” at *ACM International Conference on Management of Data (SIGMOD)*, June 2015. doi:10.1145/2723372.2742790
- [65] Kostas Tzoumas: “Apache Flink: API, Runtime, and Project Roadmap,” *slideshare.net*, January 14, 2015.
- [66] Alexander Alexandrov, Rico Bergmann, Stephan Ewen, et al.: “The Stratosphere Platform for Big Data Analytics,” *The VLDB Journal*, volume 23, number 6, pages 939–964, May 2014. doi:10.1007/s00778-014-0357-y
- [67] Michael Isard, Mihai Budea, Yuan Yu, et al.: “Dryad: Distributed Data-Parallel Programs from Sequential Building Blocks,” at *European Conference on Computer Systems (EuroSys)*, March 2007. doi:10.1145/1272996.1273005
- [68] Daniel Warneke and Odej Kao: “Nephele: Efficient Parallel Data Processing in the Cloud,” at 2nd Workshop on Many-Task Computing on Grids and Supercomputers (MTAGS), November 2009. doi:10.1145/1646468.1646476
- [69] Lawrence Page, Sergey Brin, Rajeev Motwani, and Terry Winograd: “The PageRank Citation Ranking: Bringing Order to the Web,” *Stanford InfoLab Technical Report 422*, 1999.

- [70] Leslie G. Valiant: “A Bridging Model for Parallel Computation,” *Communications of the ACM*, volume 33, number 8, pages 103–111, August 1990. doi:10.1145/79173.79181
- [71] Stephan Ewen, Kostas Tzoumas, Moritz Kaufmann, and Volker Markl: “Spinning Fast Iterative Data Flows,” *Proceedings of the VLDB Endowment*, volume 5, number 11, pages 1268–1279, July 2012. doi:10.14778/2350229.2350245
- [72] Grzegorz Malewicz, Matthew H. Austern, Aart J. C. Bik, et al.: “Pregel: A System for Large-Scale Graph Processing,” at *ACM International Conference on Management of Data (SIGMOD)*, June 2010. doi:10.1145/1807167.1807184
- [73] Frank McSherry, Michael Isard, and Derek G. Murray: “Scalability! But at What COST?,” at *15th USENIX Workshop on Hot Topics in Operating Systems (HotOS)*, May 2015.
- [74] Ionel Gog, Malte Schwarzkopf, Natacha Crooks, et al.: “Musketeer: All for One, One for All in Data Processing Systems,” at *10th European Conference on Computer Systems (EuroSys)*, April 2015. doi:10.1145/2741948.2741968
- [75] Aapo Kyrola, Guy Blelloch, and Carlos Guestrin: “GraphChi: Large-Scale Graph Computation on Just a PC,” at *10th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, October 2012.
- [76] Andrew Lenharth, Donald Nguyen, and Keshav Pingali: “Parallel Graph Analytics,” *Communications of the ACM*, volume 59, number 5, pages 78–87, May 2016. doi:10.1145/2901919
- [77] Fabian Hüske: “Peeking into Apache Flink’s Engine Room,” flink.apache.org, March 13, 2015.
- [78] Mostafa Mokhtar: “Hive 0.14 Cost Based Optimizer (CBO) Technical Overview,” hortonworks.com, March 2, 2015.
- [79] Michael Armbrust, Reynold S Xin, Cheng Lian, et al.: “Spark SQL: Relational Data Processing in Spark,” at *ACM International Conference on Management of Data (SIGMOD)*, June 2015. doi:10.1145/2723372.2742797
- [80] Daniel Blazeovski: “Planting Quadtrees for Apache Flink,” insightdataengineering.com, March 25, 2016.
- [81] Tom White: “Genome Analysis Toolkit: Now Using Apache Spark for Data Processing,” blog.cloudera.com, April 6, 2016.

Chapter 11. Stream Processing —

Chương 11. Xử lý luồng

A complex system that works is invariably found to have evolved from a simple system that works. The inverse proposition also appears to be true: A complex system designed from scratch never works and cannot be made to work.

Một hệ thống phức tạp mà hoạt động được thì bao giờ cũng tiến hóa lên từ một hệ thống đơn giản từng hoạt động được. Mệnh đề ngược lại dường như cũng đúng: một hệ thống phức tạp được thiết kế từ con số không sẽ không bao giờ hoạt động và không thể làm cho nó hoạt động được.

John Gall, Systemantics (1975)

— John Gall, Systemantics (1975)

In Chapter 10 we discussed **batch processing**—techniques that read a set of files as input and produce a new set of output files. The output is a form of **derived data**; that is, a dataset that can be recreated by running the batch process again if necessary. We saw how this simple but powerful idea can be used to create search indexes, recommendation systems, analytics, and more.

Ở Chương 10 ta đã bàn về xử lý theo lô (batch processing) — những kỹ thuật đọc một tập tệp làm đầu vào và tạo ra một tập tệp đầu ra mới. Đầu ra là một dạng dữ liệu dẫn xuất; tức là một tập dữ liệu có thể được tái tạo lại bằng cách chạy lại tiến trình lô nếu cần. Ta đã thấy ý tưởng đơn giản nhưng mạnh mẽ này có thể được dùng để tạo ra chỉ mục tìm kiếm, hệ thống gợi ý, phân tích, và nhiều thứ khác.

However, one big assumption remained throughout Chapter 10: namely, that the input is bounded—i.e., of a known and finite size—so the batch process knows when it has finished reading its input. For example, the sorting operation that is central to **MapReduce** must read its entire input before it can start producing output: it could happen that the very last input record is the one with the lowest key, and thus needs to be the very first output record, so starting the output early is not an option.

Tuy nhiên, có một giả định lớn xuyên suốt Chương 10: cụ thể là đầu vào có giới hạn (bounded) — tức là có kích thước đã biết và hữu hạn — nên tiến trình lô biết được khi nào nó đã đọc xong đầu vào. Ví dụ, thao tác sắp xếp vốn là trung tâm của MapReduce phải đọc toàn bộ đầu vào trước khi có thể bắt đầu tạo ra đầu ra: rất có thể bản ghi đầu vào cuối cùng lại chính là bản ghi có khóa nhỏ nhất, và do đó phải là bản ghi đầu ra đầu tiên, nên việc bắt đầu sinh ra đầu ra sớm là không khả thi.

In reality, a lot of data is unbounded because it arrives gradually over time: your users produced data yesterday and today, and they will continue to produce more data tomorrow. Unless you go out of business, this process never ends, and so the dataset is never “complete” in any meaningful way [1].

Thus, batch processors must artificially divide the data into chunks of fixed duration: for example, processing a day's worth of data at the end of every day, or processing an hour's worth of data at the end of every hour.

Trên thực tế, rất nhiều dữ liệu là không giới hạn (unbounded) vì nó đến dần theo thời gian: người dùng của bạn đã sinh ra dữ liệu hôm qua và hôm nay, và họ sẽ tiếp tục sinh ra thêm dữ liệu vào ngày mai. Trừ khi bạn ngừng kinh doanh, tiến trình này không bao giờ kết thúc, và vì thế tập dữ liệu không bao giờ “hoàn chỉnh” theo bất kỳ nghĩa nào có ý nghĩa [1]. Do đó, các bộ xử lý lô buộc phải chia dữ liệu một cách nhân tạo thành những phần có khoảng thời gian cố định: ví dụ, xử lý dữ liệu của một ngày vào cuối mỗi ngày, hoặc xử lý dữ liệu của một giờ vào cuối mỗi giờ.

The problem with daily batch processes is that changes in the input are only reflected in the output a day later, which is too slow for many impatient users. To reduce the delay, we can run the processing more frequently—say, processing a second's worth of data at the end of every second—or even continuously, abandoning the fixed time slices entirely and simply processing every event as it happens. That is the idea behind **stream processing**.

Vấn đề của các tiến trình lô theo ngày là những thay đổi ở đầu vào chỉ được phản ánh ở đầu ra sau đó một ngày, điều này quá chậm đối với nhiều người dùng thiếu kiên nhẫn. Để giảm độ trễ, ta có thể chạy việc xử lý thường xuyên hơn — chẳng hạn xử lý dữ liệu của một giây vào cuối mỗi giây — hoặc thậm chí liên tục, từ bỏ hoàn toàn việc chia thời gian thành lát cố định và đơn giản là xử lý từng sự kiện ngay khi nó xảy ra. Đó chính là ý tưởng đằng sau xử lý luồng (stream processing).

In general, a “stream” refers to data that is incrementally made available over time. The concept appears in many places: in the stdin and stdout of Unix, programming languages (lazy lists) [2], filesystem APIs (such as Java's `FileInputStream`), TCP connections, delivering audio and video over the internet, and so on.

Nói chung, “luồng” (stream) chỉ dữ liệu được cung cấp một cách tăng dần theo thời gian. Khái niệm này xuất hiện ở nhiều nơi: trong stdin và stdout của Unix, trong các ngôn ngữ lập trình (danh sách lười - lazy lists) [2], trong các API hệ thống tệp (như `FileInputStream` của Java), trong các kết nối TCP, trong việc truyền âm thanh và video qua internet, v.v.

In this chapter we will look at event streams as a data management mechanism: the unbounded, incrementally processed counterpart to the batch data we saw in the last chapter. We will first discuss how streams are represented, stored, and transmitted over a network. In “Databases and Streams” we will investigate the relationship between streams and databases. And finally, in “Processing Streams” we will explore approaches and tools for processing those streams continually, and ways that they can be used to build applications.

Trong chương này ta sẽ xem các luồng sự kiện (event streams) như một cơ chế quản lý dữ liệu: đối tác không giới hạn, được xử lý tăng dần của dữ liệu lô mà ta đã thấy ở chương trước. Trước tiên ta sẽ bàn về cách các luồng được biểu diễn, lưu trữ, và truyền qua mạng. Trong “Cơ sở dữ liệu và luồng” ta sẽ tìm hiểu mối quan hệ giữa các luồng và cơ sở dữ liệu. Và cuối cùng, trong “Xử lý các luồng” ta sẽ khám phá các cách tiếp cận và công cụ để xử lý các luồng đó một cách liên tục, cùng những cách dùng chúng để xây dựng ứng dụng.

Transmitting Event Streams — Truyền tải các luồng sự kiện

In the batch processing world, the inputs and outputs of a job are files (perhaps on a distributed filesystem). What does the streaming equivalent look like?

Trong thế giới xử lý lô, đầu vào và đầu ra của một tác vụ là các tệp (có thể nằm trên một hệ thống tệp phân tán). Vậy phiên bản tương đương của luồng trông như thế nào?

When the input is a file (a sequence of bytes), the first processing step is usually to parse it into a sequence of records. In a stream processing context, a record is more commonly known as an event, but it is essentially the same thing: a small, self-contained, immutable object containing the details of something that happened at some point in time. An event usually contains a timestamp indicating when it happened according to a time-of-day clock (see “Monotonic Versus Time-of-Day Clocks”).

Khi đầu vào là một tệp (một dãy byte), bước xử lý đầu tiên thường là phân tích nó thành một dãy bản ghi. Trong ngữ cảnh xử lý luồng, một bản ghi thường được gọi là một sự kiện (event), nhưng về cơ bản nó là cùng một thứ: một đối tượng nhỏ, tự chứa, bất biến chứa các chi tiết của một thứ gì đó đã xảy ra tại một thời điểm nào đó. Một sự kiện thường chứa một dấu thời gian (timestamp) cho biết nó đã xảy ra khi nào theo đồng hồ thời-gian-trong-ngày (xem “Đồng hồ đơn điệu so với đồng hồ thời-gian-trong-ngày”).

For example, the thing that happened might be an action that a user took, such as viewing a page or making a purchase. It might also originate from a machine, such as a periodic measurement from a temperature sensor, or a CPU utilization metric. In the example of “Batch Processing with Unix Tools”, each line of the web server log is an event.

Ví dụ, thứ đã xảy ra có thể là một hành động mà người dùng thực hiện, chẳng hạn xem một trang hoặc thực hiện một giao dịch mua. Nó cũng có thể bắt nguồn từ một cỗ máy, chẳng hạn một phép đo định kỳ từ cảm biến nhiệt độ, hoặc một số liệu sử dụng CPU. Trong ví dụ ở “Xử lý lô với các công cụ Unix”, mỗi dòng trong nhật ký máy chủ web là một sự kiện.

An event may be encoded as a text string, or **JSON**, or perhaps in some binary form, as discussed in Chapter 4. This encoding allows you to store an event, for example by appending it to a file, inserting it into a relational **table**, or writing it to a **document database**. It also allows you to send the event over the network to another **node** in order to process it.

Một sự kiện có thể được mã hóa thành một chuỗi văn bản, hoặc JSON, hoặc có lẽ ở một dạng nhị phân nào đó, như đã bàn ở Chương 4. Cách mã hóa này cho phép bạn lưu một sự kiện, ví dụ bằng cách nối thêm nó vào một tệp, chèn nó vào một bảng quan hệ, hoặc ghi nó vào một cơ sở dữ liệu tài liệu. Nó cũng cho phép bạn gửi sự kiện qua mạng tới một nút khác để xử lý nó.

In batch processing, a file is written once and then potentially read by multiple jobs. Analogously, in streaming terminology, an event is generated once by a producer (also known as a publisher or sender), and then potentially processed by multiple consumers (subscribers or recipients) [3]. In a filesystem, a filename identifies a set of related records; in a streaming system, related events are usually grouped together into a topic or stream.

Trong xử lý lô, một tệp được ghi một lần rồi có thể được nhiều tác vụ đọc. Một cách tương tự, trong thuật ngữ luồng, một sự kiện được sinh ra một lần bởi một nhà sản xuất (producer, còn gọi là nhà phát hành - publisher, hoặc bên gửi - sender), rồi có thể được nhiều bên tiêu thụ (consumer, tức người đăng ký - subscriber, hoặc bên nhận - recipient) xử lý [3]. Trong một hệ thống tệp, một tên tệp định danh một tập các bản ghi liên quan; trong một hệ thống luồng, các sự kiện liên quan thường được nhóm lại với nhau thành một chủ đề (topic) hoặc một luồng.

In principle, a file or database is sufficient to connect producers and consumers: a producer writes every event that it generates to the **datastore**, and each consumer periodically polls the datastore to check for events that have appeared since it last ran. This is essentially what a batch process does when it processes a day's worth of data at the end of every day.

Về nguyên tắc, một tệp hoặc một cơ sở dữ liệu là đủ để kết nối các nhà sản xuất với các bên tiêu thụ: một nhà sản xuất ghi mọi sự kiện mà nó sinh ra vào kho dữ liệu, và mỗi bên tiêu thụ định kỳ thăm dò (poll) kho dữ liệu để kiểm tra các sự kiện đã xuất hiện kể từ lần chạy gần nhất của nó. Đây về cơ bản là điều mà một tiến trình lô làm khi nó xử lý dữ liệu của một ngày vào cuối mỗi ngày.

However, when moving toward continual processing with low delays, polling becomes expensive if the datastore is not designed for this kind of usage. The more often you poll, the lower the percentage of requests that return new events, and thus the higher the overheads become. Instead, it is better for consumers to be notified when new events appear.

Tuy nhiên, khi tiến tới xử lý liên tục với độ trễ thấp, việc thăm dò trở nên tốn kém nếu kho dữ liệu không được thiết kế cho kiểu sử dụng này. Bạn thăm dò càng thường xuyên, tỷ lệ phần trăm các yêu cầu trả về sự kiện mới càng thấp, và do đó chi phí phụ trội càng cao. Thay vào đó, sẽ tốt hơn nếu các bên tiêu thụ được thông báo khi có sự kiện mới xuất hiện.

Databases have traditionally not supported this kind of notification mechanism very well: relational databases commonly have triggers, which can react to a change (e.g., a **row** being inserted into a table), but they are very limited in what they can do and have been somewhat of an afterthought in database design [4, 5]. Instead, specialized tools have been developed for the purpose of delivering event notifications.

Các cơ sở dữ liệu theo truyền thống không hỗ trợ kiểu cơ chế thông báo này tốt cho lắm: các cơ sở dữ liệu quan hệ thường có trigger, vốn có thể phản ứng với một thay đổi (ví dụ, một hàng được chèn vào một bảng), nhưng chúng rất hạn chế về những gì có thể làm và phần nào đó chỉ là thứ được nghĩ tới sau cùng trong thiết kế cơ sở dữ liệu [4, 5]. Thay vào đó, người ta đã phát triển những công cụ chuyên biệt nhằm mục đích phân phát các thông báo sự kiện.

Messaging Systems — Các hệ thống thông điệp

A common approach for notifying consumers about new events is to use a messaging system: a producer sends a message containing the event, which is then pushed to consumers. We touched on these systems previously in “Message-Passing Dataflow”, but we will now go into more detail.

Một cách tiếp cận phổ biến để thông báo cho các bên tiêu thụ về những sự kiện mới là dùng một hệ thống thông điệp (messaging system): một nhà sản xuất gửi đi một thông điệp chứa sự kiện, thông điệp đó sau đó được đẩy (push) tới các bên tiêu thụ. Ta đã đề cập tới các hệ thống này trước đó trong “Luồng dữ liệu truyền thông điệp”, nhưng giờ ta sẽ đi vào chi tiết hơn.

A direct communication channel like a Unix pipe or TCP connection between producer and consumer would be a simple way of implementing a messaging system. However, most messaging systems expand on this basic model. In particular, Unix pipes and TCP connect exactly one sender with one recipient, whereas a messaging system allows multiple producer nodes to send messages to the same topic and allows multiple consumer nodes to receive messages in a topic.

Một kênh giao tiếp trực tiếp như một đường ống Unix (Unix pipe) hoặc một kết nối TCP giữa nhà sản xuất và bên tiêu thụ sẽ là một cách đơn giản để hiện thực một hệ thống thông điệp. Tuy nhiên, hầu hết các hệ thống thông điệp mở rộng thêm trên mô hình cơ bản này. Cụ thể, các đường ống Unix và TCP kết nối đúng một bên gửi với một bên nhận, trong khi một hệ thống thông điệp cho phép nhiều nút sản xuất gửi thông điệp tới cùng một chủ đề và cho phép nhiều nút tiêu thụ nhận các thông điệp trong một chủ đề.

Within this publish/subscribe model, different systems take a wide range of approaches, and there is no one right answer for all purposes. To differentiate the systems, it is particularly helpful to ask the following two questions:

Trong mô hình phát hành/đăng ký (publish/subscribe) này, các hệ thống khác nhau áp dụng một loạt cách tiếp cận rất rộng, và không có một câu trả lời đúng duy nhất cho mọi mục đích. Để phân biệt các hệ thống, đặc biệt hữu ích khi đặt ra hai câu hỏi sau:

- What happens if the producers send messages faster than the consumers can process them? Broadly speaking, there are three options: the system can drop messages, buffer messages in a **queue**, or apply backpressure (also known as flow control; i.e., blocking the producer from sending more messages). For example, Unix pipes and TCP use backpressure: they have a small fixed-size buffer, and if it fills up, the sender is blocked until the recipient takes data out of the buffer (see “Network congestion and queueing”). If messages are buffered in a queue, it is important to understand what happens as that queue grows. Does the system crash if the queue no longer fits in memory, or does it write messages to disk? If so, how does the disk access affect the performance of the messaging system [6]?
- What happens if nodes crash or temporarily go offline—are any messages lost? As with databases, **durability** may require some combination of writing to disk and/or replication (see the sidebar “Replication and Durability”), which has a cost. If you can afford to sometimes lose messages, you can probably get higher **throughput** and lower **latency** on the same hardware.
 - Điều gì xảy ra nếu các nhà sản xuất gửi thông điệp nhanh hơn mức các bên tiêu thụ có thể xử lý chúng? Nói rộng ra, có ba lựa chọn: hệ thống có thể loại bỏ thông điệp, đệm thông điệp trong một hàng đợi, hoặc áp dụng cơ chế phản áp (backpressure, còn gọi là điều khiển luồng - flow control; tức là chặn nhà sản xuất gửi thêm thông điệp). Ví dụ, các đường ống Unix và TCP dùng cơ chế phản áp: chúng có một bộ đệm cố định kích thước nhỏ, và nếu nó đầy, bên gửi bị chặn cho tới khi bên nhận lấy dữ liệu ra khỏi bộ đệm (xem “Tắc nghẽn và xếp hàng trên mạng”). Nếu thông điệp được đệm trong một hàng đợi, điều quan trọng là phải hiểu chuyện gì xảy ra khi hàng đợi đó lớn lên. Hệ thống có sập khi hàng đợi không còn vừa trong bộ nhớ không, hay nó ghi thông điệp xuống đĩa? Nếu vậy, việc truy cập đĩa ảnh hưởng thế nào tới hiệu năng của hệ thống thông điệp [6]?
 - Điều gì xảy ra nếu các nút sập hoặc tạm thời ngoại tuyến — có thông điệp nào bị mất

không? Cũng giống như với cơ sở dữ liệu, độ bền có thể đòi hỏi một sự kết hợp nào đó giữa việc ghi xuống đĩa và/hoặc nhân bản (xem khung “Nhân bản và độ bền”), điều này có cái giá của nó. Nếu bạn có thể chấp nhận thỉnh thoảng mất thông điệp, bạn có lẽ sẽ đạt được thông lượng cao hơn và độ trễ thấp hơn trên cùng một phần cứng.

Whether message loss is acceptable depends very much on the application. For example, with sensor readings and metrics that are transmitted periodically, an occasional missing data point is perhaps not important, since an updated value will be sent a short time later anyway. However, beware that if a large number of messages are dropped, it may not be immediately apparent that the metrics are incorrect [7]. If you are counting events, it is more important that they are delivered reliably, since every lost message means incorrect counters.

Việc mất thông điệp có chấp nhận được hay không phụ thuộc rất nhiều vào ứng dụng. Ví dụ, với các giá trị đọc từ cảm biến và các số liệu được truyền định kỳ, một điểm dữ liệu thỉnh thoảng bị thiếu có lẽ không quan trọng, vì dù sao một giá trị cập nhật sẽ được gửi đi ngay sau đó không lâu. Tuy nhiên, hãy lưu ý rằng nếu một số lượng lớn thông điệp bị loại bỏ, có thể sẽ không dễ nhận ra ngay rằng các số liệu là sai [7]. Nếu bạn đang đếm các sự kiện, thì việc chúng được phân phát một cách đáng tin cậy quan trọng hơn, vì mỗi thông điệp bị mất đồng nghĩa với một bộ đếm sai.

A nice **property** of the batch processing systems we explored in Chapter 10 is that they provide a strong **reliability** guarantee: failed tasks are automatically retried, and partial output from failed tasks is automatically discarded. This means the output is the same as if no failures had occurred, which helps simplify the programming model. Later in this chapter we will examine how we can provide similar guarantees in a streaming context.

Một tính chất hay của các hệ thống xử lý lô mà ta đã khám phá ở Chương 10 là chúng cung cấp một bảo đảm độ tin cậy mạnh mẽ: các tác vụ thất bại được tự động thử lại, và đầu ra một phần từ các tác vụ thất bại được tự động loại bỏ. Điều này có nghĩa là đầu ra giống hệt như khi không có hỏng hóc nào xảy ra, giúp đơn giản hóa mô hình lập trình. Phần sau của chương này ta sẽ xem xét cách ta có thể cung cấp những bảo đảm tương tự trong ngữ cảnh luồng.

Direct messaging from producers to consumers — Truyền thông điệp trực tiếp từ nhà sản xuất tới bên tiêu thụ

A number of messaging systems use direct network communication between producers and consumers without going via intermediary nodes:

Một số hệ thống thông điệp dùng giao tiếp mạng trực tiếp giữa nhà sản xuất và bên tiêu thụ mà không đi qua các nút trung gian:

- UDP multicast is widely used in the financial industry for streams such as stock market feeds, where low latency is important [8]. Although UDP itself is unreliable, application-level protocols can recover lost packets (the producer must remember packets it has sent so that it can retransmit them on demand).
- Brokerless messaging libraries such as ZeroMQ [9] and nanomsg take a similar approach, implementing publish/subscribe messaging over TCP or IP multicast.
- StatsD [10] and Brubeck [7] use unreliable UDP messaging for collecting metrics from all machines on the network and monitoring them. (In the StatsD protocol, counter metrics are only correct if all messages are received; using UDP makes the metrics at best approximate [11]. See also “TCP Versus UDP”.)

- If the consumer exposes a service on the network, producers can make a direct HTTP or RPC request (see “Dataflow Through Services: REST and RPC”) to push messages to the consumer. This is the idea behind webhooks [12], a pattern in which a callback URL of one service is registered with another service, and it makes a request to that URL whenever an event occurs.
 - UDP multicast được dùng rộng rãi trong ngành tài chính cho các luồng như dữ liệu thị trường chứng khoán, nơi độ trễ thấp là quan trọng [8]. Mặc dù bản thân UDP là không đáng tin cậy, các giao thức ở tầng ứng dụng có thể khôi phục các gói tin bị mất (nhà sản xuất phải ghi nhớ các gói tin nó đã gửi để có thể truyền lại chúng theo yêu cầu).
 - Các thư viện thông điệp không cần broker (brokerless) như ZeroMQ [9] và nanomsg áp dụng cách tiếp cận tương tự, hiện thực việc truyền thông điệp phát hành/đăng ký qua TCP hoặc IP multicast.
 - StatsD [10] và Brubeck [7] dùng việc truyền thông điệp UDP không đáng tin cậy để thu thập số liệu từ mọi máy trên mạng và giám sát chúng. (Trong giao thức StatsD, các số liệu bộ đếm chỉ đúng nếu mọi thông điệp đều được nhận; việc dùng UDP làm cho các số liệu giới lỗi cũng chỉ là gần đúng [11]. Xem thêm “TCP so với UDP”.)
 - Nếu bên tiêu thụ phơi bày một dịch vụ trên mạng, các nhà sản xuất có thể thực hiện một yêu cầu HTTP hoặc RPC trực tiếp (xem “Luồng dữ liệu qua các dịch vụ: REST và RPC”) để đẩy thông điệp tới bên tiêu thụ. Đây là ý tưởng đằng sau webhook [12], một mẫu trong đó một URL callback của một dịch vụ được đăng ký với một dịch vụ khác, và dịch vụ kia thực hiện một yêu cầu tới URL đó mỗi khi một sự kiện xảy ra.

Although these direct messaging systems work well in the situations for which they are designed, they generally require the **application code** to be aware of the possibility of message loss. The faults they can tolerate are quite limited: even if the protocols detect and retransmit packets that are lost in the network, they generally assume that producers and consumers are constantly online.

Mặc dù những hệ thống thông điệp trực tiếp này hoạt động tốt trong các tình huống mà chúng được thiết kế cho, nói chung chúng đòi hỏi mã ứng dụng phải ý thức được khả năng mất thông điệp. Những trục trặc mà chúng có thể chịu được khá hạn chế: ngay cả khi các giao thức phát hiện và truyền lại các gói tin bị mất trên mạng, chúng nói chung giả định rằng các nhà sản xuất và bên tiêu thụ luôn trực tuyến.

If a consumer is offline, it may miss messages that were sent while it is unreachable. Some protocols allow the producer to retry failed message deliveries, but this approach may break down if the producer crashes, losing the buffer of messages that it was supposed to retry.

Nếu một bên tiêu thụ ngoại tuyến, nó có thể bỏ lỡ các thông điệp được gửi đi trong lúc nó không thể truy cập được. Một số giao thức cho phép nhà sản xuất thử lại các lần phân phát thông điệp thất bại, nhưng cách tiếp cận này có thể sụp đổ nếu nhà sản xuất bị sập, làm mất bộ đệm các thông điệp mà nó lẽ ra phải thử lại.

Message brokers — Message broker

A widely used alternative is to send messages via a message broker (also known as a **message queue**), which is essentially a kind of database that is optimized for handling message streams [13]. It runs as a server, with producers and consumers connecting to it as clients. Producers write messages to the broker, and consumers receive them by reading them from the broker.

Một lựa chọn thay thế được dùng rộng rãi là gửi thông điệp qua một message broker (còn gọi là hàng đợi thông điệp - message queue), về cơ bản là một dạng cơ sở dữ liệu được tối ưu cho việc xử lý các luồng thông điệp [13]. Nó chạy như một máy chủ, với các nhà sản

xuất và bên tiêu thụ kết nối tới nó như các client. Các nhà sản xuất ghi thông điệp vào broker, và các bên tiêu thụ nhận chúng bằng cách đọc từ broker.

By centralizing the data in the broker, these systems can more easily tolerate clients that come and go (connect, disconnect, and crash), and the question of durability is moved to the broker instead. Some message brokers only keep messages in memory, while others (depending on configuration) write them to disk so that they are not lost in case of a broker crash. Faced with slow consumers, they generally allow unbounded queueing (as opposed to dropping messages or backpressure), although this choice may also depend on the configuration.

Bằng cách tập trung dữ liệu vào broker, các hệ thống này có thể dễ dàng hơn trong việc chịu đựng các client đến rồi đi (kết nối, ngắt kết nối, và sập), và câu hỏi về độ bền được chuyển sang broker thay vì client. Một số message broker chỉ giữ thông điệp trong bộ nhớ, trong khi những broker khác (tùy cấu hình) ghi chúng xuống đĩa để chúng không bị mất trong trường hợp broker sập. Khi đối mặt với các bên tiêu thụ chậm, chúng nói chung cho phép xếp hàng không giới hạn (thay vì loại bỏ thông điệp hoặc phản áp), mặc dù lựa chọn này cũng có thể phụ thuộc vào cấu hình.

A consequence of queueing is also that consumers are generally asynchronous: when a producer sends a message, it normally only waits for the broker to confirm that it has buffered the message and does not wait for the message to be processed by consumers. The delivery to consumers will happen at some undetermined future point in time—often within a fraction of a second, but sometimes significantly later if there is a queue backlog.

Một hệ quả của việc xếp hàng là các bên tiêu thụ nói chung hoạt động bất đồng bộ: khi một nhà sản xuất gửi một thông điệp, nó thường chỉ đợi broker xác nhận rằng đã đệm xong thông điệp và không đợi thông điệp được các bên tiêu thụ xử lý. Việc phân phát tới các bên tiêu thụ sẽ xảy ra tại một thời điểm chưa xác định trong tương lai — thường là trong một phần nhỏ của giây, nhưng đôi khi muộn hơn đáng kể nếu có một lượng hàng tồn đọng trong hàng đợi.

Message brokers compared to databases — So sánh message broker với cơ sở dữ liệu

Some message brokers can even participate in two-phase commit protocols using XA or JTA (see “Distributed Transactions in Practice”). This feature makes them quite similar in nature to databases, although there are still important practical differences between message brokers and databases:

Một số message broker thậm chí có thể tham gia vào các giao thức cam kết hai pha (two-phase commit) bằng XA hoặc JTA (xem “Giao dịch phân tán trong thực tế”). Tính năng này khiến chúng khá giống cơ sở dữ liệu về bản chất, mặc dù vẫn còn những khác biệt thực tiễn quan trọng giữa message broker và cơ sở dữ liệu:

- Databases usually keep data until it is explicitly deleted, whereas most message brokers automatically delete a message when it has been successfully delivered to its consumers. Such message brokers are not suitable for long-term data storage.
- Since they quickly delete messages, most message brokers assume that their working set is fairly small—i.e., the queues are short. If the broker needs to buffer a lot of messages because the consumers are slow (perhaps spilling messages to disk if they no longer fit in memory), each individual message takes longer to process, and the overall throughput may degrade [6].
- Databases often support secondary indexes and various ways of searching for data, while message brokers often support some way of subscribing to a subset of topics matching some

pattern. The mechanisms are different, but both are essentially ways for a **client** to select the portion of the data that it wants to know about.

- When querying a database, the result is typically based on a point-in-time snapshot of the data; if another client subsequently writes something to the database that changes the query result, the first client does not find out that its prior result is now outdated (unless it repeats the query, or polls for changes). By contrast, message brokers do not support arbitrary queries, but they do notify clients when data changes (i.e., when new messages become available).
 - Cơ sở dữ liệu thường giữ dữ liệu cho tới khi nó bị xóa một cách tường minh, trong khi hầu hết message broker tự động xóa một thông điệp khi nó đã được phân phát thành công tới các bên tiêu thụ của nó. Những message broker như vậy không phù hợp để lưu trữ dữ liệu dài hạn.
 - Vì chúng xóa thông điệp nhanh chóng, hầu hết message broker giả định rằng tập làm việc (working set) của chúng khá nhỏ — tức là các hàng đợi ngắn. Nếu broker cần đệm rất nhiều thông điệp vì các bên tiêu thụ chậm (có lẽ tràn thông điệp xuống đĩa nếu chúng không còn vừa trong bộ nhớ), thì mỗi thông điệp riêng lẻ mất nhiều thời gian xử lý hơn, và thông lượng tổng thể có thể suy giảm [6].
 - Cơ sở dữ liệu thường hỗ trợ chỉ mục phụ và nhiều cách khác nhau để tìm kiếm dữ liệu, trong khi message broker thường hỗ trợ một cách nào đó để đăng ký một tập con các chủ đề khớp với một mẫu nào đó. Các cơ chế thì khác nhau, nhưng cả hai về cơ bản đều là cách để một client chọn ra phần dữ liệu mà nó muốn biết.
 - Khi truy vấn một cơ sở dữ liệu, kết quả thường dựa trên một ảnh chụp dữ liệu tại một thời điểm; nếu một client khác sau đó ghi thứ gì đó vào cơ sở dữ liệu làm thay đổi kết quả truy vấn, client đầu tiên không phát hiện ra rằng kết quả trước đó của nó giờ đã lỗi thời (trừ khi nó lặp lại truy vấn, hoặc thăm dò để tìm thay đổi). Ngược lại, message broker không hỗ trợ các truy vấn tùy ý, nhưng chúng thông báo cho client khi dữ liệu thay đổi (tức là khi có thông điệp mới khả dụng).

This is the traditional view of message brokers, which is encapsulated in standards like JMS [14] and AMQP [15] and implemented in software like RabbitMQ, ActiveMQ, HornetQ, Qpid, TIBCO Enterprise Message Service, IBM MQ, Azure Service Bus, and Google Cloud Pub/Sub [16].

Đây là cách nhìn truyền thống về message broker, được gói gọn trong các chuẩn như JMS [14] và AMQP [15] và được hiện thực trong các phần mềm như RabbitMQ, ActiveMQ, HornetQ, Qpid, TIBCO Enterprise Message Service, IBM MQ, Azure Service Bus, và Google Cloud Pub/Sub [16].

Multiple consumers — Nhiều bên tiêu thụ

When multiple consumers read messages in the same topic, two main patterns of messaging are used, as illustrated in Figure 11-1:

Khi nhiều bên tiêu thụ đọc thông điệp trong cùng một chủ đề, có hai mẫu truyền thông điệp chính được dùng, như minh họa ở Hình 11-1:

Each message is delivered to one of the consumers, so the consumers can share the work of processing the messages in the topic. The broker may assign messages to consumers arbitrarily. This pattern is useful when the messages are expensive to process, and so you want to be able to add consumers to parallelize the processing. (In AMQP, you can implement **load** balancing by having multiple clients consuming from the same queue, and in JMS it is called a shared subscription.)

Mỗi thông điệp được phân phát tới một trong các bên tiêu thụ, nhờ vậy các bên tiêu thụ có thể chia sẻ công việc xử lý các thông điệp trong chủ đề. Broker có thể gán thông điệp cho

các bên tiêu thụ một cách tùy ý. Mẫu này hữu ích khi các thông điệp tốn kém để xử lý, và do đó bạn muốn có thể thêm các bên tiêu thụ để song song hóa việc xử lý. (Trong AMQP, bạn có thể hiện thực việc cân bằng tải bằng cách cho nhiều client cùng tiêu thụ từ một hàng đợi, và trong JMS điều này được gọi là một đăng ký dùng chung - shared subscription.)

Each message is delivered to all of the consumers. **Fan-out** allows several independent consumers to each “tune in” to the same broadcast of messages, without affecting each other—the streaming equivalent of having several different batch jobs that read the same input file. (This feature is provided by topic subscriptions in JMS, and exchange bindings in AMQP.)

Mỗi thông điệp được phân phát tới tất cả các bên tiêu thụ. Fan-out cho phép nhiều bên tiêu thụ độc lập, mỗi bên “bắt” cùng một đợt phát thông điệp, mà không ảnh hưởng lẫn nhau — đây là phiên bản tương đương của luồng so với việc có nhiều tác vụ lô khác nhau cùng đọc một tệp đầu vào. (Tính năng này được cung cấp bởi các đăng ký chủ đề - topic subscriptions trong JMS, và các ràng buộc exchange - exchange bindings trong AMQP.)

Figure 11-1. (a) Load balancing: sharing the work of consuming a topic among consumers; (b) fan-out: delivering each message to multiple consumers. — Hình 11-1. (a) Cân bằng tải: chia sẻ công việc tiêu thụ một chủ đề giữa các bên tiêu thụ; (b) fan-out: phân phát mỗi thông điệp tới nhiều bên tiêu thụ. The two patterns can be combined: for example, two separate groups of consumers may each subscribe to a topic, such that each group collectively receives all messages, but within each group only one of the nodes receives each message.

Hai mẫu này có thể được kết hợp: ví dụ, hai nhóm bên tiêu thụ riêng biệt có thể mỗi nhóm đăng ký một chủ đề, sao cho mỗi nhóm cùng nhau nhận tất cả các thông điệp, nhưng trong mỗi nhóm chỉ một trong các nút nhận mỗi thông điệp.

Acknowledgments and redelivery — Xác nhận và phân phát lại

Consumers may crash at any time, so it could happen that a broker delivers a message to a consumer but the consumer never processes it, or only partially processes it before crashing. In order to ensure that the message is not lost, message brokers use acknowledgments: a client must explicitly tell the broker when it has finished processing a message so that the broker can remove it from the queue.

Các bên tiêu thụ có thể sập bất kỳ lúc nào, nên có thể xảy ra việc một broker phân phát một thông điệp tới một bên tiêu thụ nhưng bên tiêu thụ không bao giờ xử lý nó, hoặc chỉ xử lý nó một phần trước khi sập. Để bảo đảm rằng thông điệp không bị mất, các message broker dùng cơ chế xác nhận (acknowledgment): một client phải nói tường minh cho broker biết khi nó đã xử lý xong một thông điệp để broker có thể loại thông điệp đó ra khỏi hàng đợi.

If the connection to a client is closed or times out without the broker receiving an acknowledgment, it assumes that the message was not processed, and therefore it delivers the message again to another consumer. (Note that it could happen that the message actually was fully processed, but the acknowledgment was lost in the network. Handling this case requires an atomic commit protocol, as discussed in “Distributed Transactions in Practice”.)

Nếu kết nối tới một client bị đóng hoặc hết thời gian chờ mà broker chưa nhận được xác nhận, nó giả định rằng thông điệp đã không được xử lý, và do đó nó phân phát thông điệp đó lại lần nữa tới một bên tiêu thụ khác. (Lưu ý rằng có thể xảy ra việc thông điệp thực ra đã được xử lý trọn vẹn, nhưng xác nhận bị mất trên mạng. Xử lý trường hợp này đòi hỏi một giao thức cam kết nguyên tử - atomic commit, như đã bàn ở “Giao dịch phân tán trong thực tế”.)

When combined with load balancing, this redelivery behavior has an interesting effect on the ordering of messages. In Figure 11-2, the consumers generally process messages in the order they were sent by producers. However, consumer 2 crashes while processing message m3, at the same time as consumer 1 is processing message m4. The unacknowledged message m3 is subsequently redelivered to consumer 1, with the result that consumer 1 processes messages in the order m4, m3, m5. Thus, m3 and m4 are not delivered in the same order as they were sent by producer 1.

Khi kết hợp với cân bằng tải, hành vi phân phát lại này có một tác động thú vị tới thứ tự của các thông điệp. Trong Hình 11-2, các bên tiêu thụ nói chung xử lý thông điệp theo thứ tự mà các nhà sản xuất gửi chúng. Tuy nhiên, bên tiêu thụ 2 sập trong lúc xử lý thông điệp m3, đúng vào lúc bên tiêu thụ 1 đang xử lý thông điệp m4. Thông điệp m3 chưa được xác nhận sau đó được phân phát lại cho bên tiêu thụ 1, với kết quả là bên tiêu thụ 1 xử lý các thông điệp theo thứ tự m4, m3, m5. Như vậy, m3 và m4 không được phân phát theo cùng thứ tự mà nhà sản xuất 1 đã gửi chúng.

Figure 11-2. Consumer 2 crashes while processing m3, so it is redelivered to consumer 1 at a later time. — Hình 11-2. Bên tiêu thụ 2 sập trong lúc xử lý m3, nên nó được phân phát lại cho bên tiêu thụ 1 vào một thời điểm sau. Even if the message broker otherwise tries to preserve the order of messages (as required by both the JMS and AMQP standards), the combination of load balancing with redelivery inevitably leads to messages being reordered. To avoid this issue, you can use a separate queue per consumer (i.e., not use the load balancing feature). Message reordering is not a problem if messages are completely independent of each other, but it can be important if there are causal dependencies between messages, as we shall see later in the chapter.

Ngay cả khi message broker cố gắng giữ thứ tự của các thông điệp (như cả hai chuẩn JMS và AMQP đòi hỏi), sự kết hợp giữa cân bằng tải với phân phát lại chắc chắn dẫn tới việc các thông điệp bị xáo trộn thứ tự. Để tránh vấn đề này, bạn có thể dùng một hàng đợi riêng cho mỗi bên tiêu thụ (tức là không dùng tính năng cân bằng tải). Việc xáo trộn thứ tự thông điệp không thành vấn đề nếu các thông điệp hoàn toàn độc lập với nhau, nhưng nó có thể quan trọng nếu giữa các thông điệp có những phụ thuộc nhân quả, như ta sẽ thấy ở phần sau của chương.

Partitioned Logs — Nhật ký được phân mảnh

Sending a packet over a network or making a request to a network service is normally a transient operation that leaves no permanent trace. Although it is possible to record it permanently (using packet capture and logging), we normally don't think of it that way. Even message brokers that durably write messages to disk quickly delete them again after they have been delivered to consumers, because they are built around a transient messaging mindset.

Việc gửi một gói tin qua mạng hoặc thực hiện một yêu cầu tới một dịch vụ mạng thường là một thao tác thoáng qua không để lại dấu vết vĩnh viễn. Mặc dù có thể ghi lại nó vĩnh viễn (bằng cách bắt gói tin và ghi nhật ký), ta thường không nghĩ về nó theo cách đó. Ngay cả những message broker có ghi thông điệp xuống đĩa một cách bền vững cũng nhanh chóng xóa chúng đi sau khi chúng đã được phân phát tới các bên tiêu thụ, vì chúng được xây dựng quanh một tư duy về việc truyền thông điệp mang tính thoáng qua.

Databases and filesystems take the opposite approach: everything that is written to a database or file is normally expected to be permanently recorded, at least until someone explicitly chooses to delete it again.

Cơ sở dữ liệu và hệ thống tệp áp dụng cách tiếp cận ngược lại: mọi thứ được ghi vào một cơ sở dữ liệu hoặc một tệp thường được kỳ vọng sẽ được ghi lại vĩnh viễn, ít nhất là cho tới khi ai đó chủ động chọn xóa nó đi lần nữa.

This difference in mindset has a big impact on how derived data is created. A key feature of batch processes, as discussed in Chapter 10, is that you can run them repeatedly, experimenting with the processing steps, without risk of damaging the input (since the input is read-only). This is not the case with AMQP/JMS-style messaging: receiving a message is destructive if the acknowledgment causes it to be deleted from the broker, so you cannot run the same consumer again and expect to get the same result.

Sự khác biệt trong tư duy này có một tác động lớn tới cách dữ liệu dẫn xuất được tạo ra. Một tính năng then chốt của các tiến trình lô, như đã bàn ở Chương 10, là bạn có thể chạy chúng lặp đi lặp lại, thử nghiệm với các bước xử lý, mà không có rủi ro làm hỏng đầu vào (vì đầu vào là chỉ-đọc). Điều này không đúng với kiểu truyền thông điệp theo phong cách AMQP/JMS: việc nhận một thông điệp có tính phá hủy nếu việc xác nhận khiến nó bị xóa khỏi broker, nên bạn không thể chạy lại cùng một bên tiêu thụ và mong nhận được cùng kết quả.

If you add a new consumer to a messaging system, it typically only starts receiving messages sent after the time it was registered; any prior messages are already gone and cannot be recovered. Contrast this with files and databases, where you can add a new client at any time, and it can read data written arbitrarily far in the past (as long as it has not been explicitly overwritten or deleted by the application).

Nếu bạn thêm một bên tiêu thụ mới vào một hệ thống thông điệp, nó thường chỉ bắt đầu nhận các thông điệp được gửi sau thời điểm nó được đăng ký; mọi thông điệp trước đó đều đã mất và không thể khôi phục. Hãy đối chiếu điều này với tệp và cơ sở dữ liệu, nơi bạn có thể thêm một client mới vào bất kỳ lúc nào, và nó có thể đọc dữ liệu đã được ghi từ bất kỳ thời điểm xa xôi nào trong quá khứ (miễn là dữ liệu đó chưa bị ứng dụng ghi đè hoặc xóa một cách tường minh).

Why can we not have a hybrid, combining the durable storage approach of databases with the low-latency notification facilities of messaging? This is the idea behind log-based message brokers.

Tại sao ta không thể có một thứ lai (hybrid), kết hợp cách tiếp cận lưu trữ bền vững của cơ sở dữ liệu với khả năng thông báo độ trễ thấp của việc truyền thông điệp? Đây là ý tưởng đằng sau các message broker dựa trên nhật ký (log-based message brokers).

Using logs for message storage — Dùng nhật ký để lưu trữ thông điệp

A log is simply an append-only sequence of records on disk. We previously discussed logs in the context of log-structured storage engines and write-ahead logs in Chapter 3, and in the context of replication in Chapter 5.

Một nhật ký (log) đơn giản là một dãy bản ghi chỉ-nối-thêm (append-only) trên đĩa. Trước đây ta đã bàn về các nhật ký trong ngữ cảnh của các engine lưu trữ có cấu trúc nhật ký (log-structured) và nhật ký ghi-trước (write-ahead logs) ở Chương 3, cũng như trong ngữ cảnh nhân bản ở Chương 5.

The same structure can be used to implement a message broker: a producer sends a message by appending it to the end of the log, and a consumer receives messages by reading the log sequentially. If a consumer reaches the end of the log, it waits for a notification that a new message has been

appended. The Unix tool `tail -f`, which watches a file for data being appended, essentially works like this.

Cùng cấu trúc đó có thể được dùng để hiện thực một message broker: một nhà sản xuất gửi một thông điệp bằng cách nối thêm nó vào cuối nhật ký, và một bên tiêu thụ nhận thông điệp bằng cách đọc nhật ký một cách tuần tự. Nếu một bên tiêu thụ đọc tới cuối nhật ký, nó đợi một thông báo rằng đã có một thông điệp mới được nối thêm. Công cụ Unix `tail -f`, vốn theo dõi một tệp để phát hiện dữ liệu được nối thêm, về cơ bản hoạt động theo cách này.

In order to scale to higher throughput than a single disk can offer, the log can be partitioned (in the sense of Chapter 6). Different partitions can then be hosted on different machines, making each partition a separate log that can be read and written independently from other partitions. A topic can then be defined as a group of partitions that all carry messages of the same type. This approach is illustrated in Figure 11-3.

Để mở rộng lên thông lượng cao hơn mức mà một đĩa đơn có thể cung cấp, nhật ký có thể được phân mảnh (theo nghĩa ở Chương 6). Các mảnh khác nhau khi đó có thể được đặt trên các máy khác nhau, biến mỗi mảnh thành một nhật ký riêng có thể được đọc và ghi độc lập với các mảnh khác. Một chủ đề khi đó có thể được định nghĩa là một nhóm các mảnh đều mang các thông điệp cùng loại. Cách tiếp cận này được minh họa ở Hình 11-3.

Within each partition, the broker assigns a monotonically increasing sequence number, or offset, to every message (in Figure 11-3, the numbers in boxes are message offsets). Such a sequence number makes sense because a partition is append-only, so the messages within a partition are totally ordered. There is no ordering guarantee across different partitions.

Trong mỗi mảnh, broker gán một số thứ tự tăng đơn điệu, hay offset, cho mỗi thông điệp (trong Hình 11-3, các con số trong các ô là offset của thông điệp). Một số thứ tự như vậy là hợp lý vì một mảnh là chỉ-nối-thêm, nên các thông điệp trong một mảnh được sắp thứ tự toàn phần. Không có bảo đảm về thứ tự giữa các mảnh khác nhau.

Figure 11-3. Producers send messages by appending them to a topic-partition file, and consumers read these files sequentially. — Hình 11-3. Các nhà sản xuất gửi thông điệp bằng cách nối thêm chúng vào một tệp chủ-đề-mảnh, và các bên tiêu thụ đọc các tệp này một cách tuần tự. Apache Kafka [17, 18], Amazon Kinesis Streams [19], and Twitter’s DistributedLog [20, 21] are log-based message brokers that work like this. Google Cloud Pub/Sub is architecturally similar but exposes a JMS-style **API** rather than a log **abstraction** [16]. Even though these message brokers write all messages to disk, they are able to achieve throughput of millions of messages per second by partitioning across multiple machines, and **fault** tolerance by replicating messages [22, 23].

Apache Kafka [17, 18], Amazon Kinesis Streams [19], và DistributedLog của Twitter [20, 21] là những message broker dựa trên nhật ký hoạt động như vậy. Google Cloud Pub/Sub có kiến trúc tương tự nhưng phơi bày một API theo phong cách JMS thay vì một trừu tượng nhật ký [16]. Mặc dù những message broker này ghi tất cả thông điệp xuống đĩa, chúng có thể đạt thông lượng hàng triệu thông điệp mỗi giây nhờ phân mảnh trên nhiều máy, và đạt khả năng chịu lỗi nhờ nhân bản thông điệp [22, 23].

Logs compared to traditional messaging — So sánh nhật ký với việc truyền thông điệp truyền thống

The log-based approach trivially supports fan-out messaging, because several consumers can independently read the log without affecting each other—reading a message does not delete it from

the log. To achieve load balancing across a group of consumers, instead of assigning individual messages to consumer clients, the broker can assign entire partitions to nodes in the consumer group.

Cách tiếp cận dựa trên nhật ký hỗ trợ một cách hiển nhiên việc truyền thông điệp kiểu fan-out, vì nhiều bên tiêu thụ có thể độc lập đọc nhật ký mà không ảnh hưởng lẫn nhau — việc đọc một thông điệp không xóa nó khỏi nhật ký. Để đạt được cân bằng tải trên một nhóm bên tiêu thụ, thay vì gán từng thông điệp riêng lẻ cho các client tiêu thụ, broker có thể gán nguyên các mảnh cho các nút trong nhóm tiêu thụ.

Each client then consumes all the messages in the partitions it has been assigned. Typically, when a consumer has been assigned a log partition, it reads the messages in the partition sequentially, in a straightforward single-threaded manner. This coarse-grained load balancing approach has some downsides:

Mỗi client khi đó tiêu thụ tất cả các thông điệp trong các mảnh mà nó được gán. Thông thường, khi một bên tiêu thụ đã được gán một mảnh nhật ký, nó đọc các thông điệp trong mảnh đó một cách tuần tự, theo lối đơn luồng đơn giản. Cách tiếp cận cân bằng tải thô này có một số nhược điểm:

- The number of nodes sharing the work of consuming a topic can be at most the number of log partitions in that topic, because messages within the same partition are delivered to the same node.
- If a single message is slow to process, it holds up the processing of subsequent messages in that partition (a form of **head-of-line blocking**; see “Describing Performance”).
 - Số nút chia sẻ công việc tiêu thụ một chủ đề có thể nhiều nhất là bằng số mảnh nhật ký trong chủ đề đó, vì các thông điệp trong cùng một mảnh được phân phát tới cùng một nút.
 - Nếu một thông điệp đơn lẻ chậm xử lý, nó cản trở việc xử lý các thông điệp tiếp theo trong mảnh đó (một dạng tắc nghẽn đầu hàng - head-of-line blocking; xem “Mô tả hiệu năng”).

Thus, in situations where messages may be expensive to process and you want to parallelize processing on a message-by-message basis, and where message ordering is not so important, the JMS/AMQP style of message broker is preferable. On the other hand, in situations with high message throughput, where each message is fast to process and where message ordering is important, the log-based approach works very well.

Như vậy, trong các tình huống mà thông điệp có thể tốn kém để xử lý và bạn muốn song song hóa việc xử lý theo từng-thông-điệp-một, và nơi thứ tự thông điệp không quá quan trọng, thì kiểu message broker theo phong cách JMS/AMQP là thích hợp hơn. Ngược lại, trong các tình huống có thông lượng thông điệp cao, nơi mỗi thông điệp nhanh xử lý và nơi thứ tự thông điệp là quan trọng, thì cách tiếp cận dựa trên nhật ký hoạt động rất tốt.

Consumer offsets — Offset của bên tiêu thụ

Consuming a partition sequentially makes it easy to tell which messages have been processed: all messages with an offset less than a consumer’s current offset have already been processed, and all messages with a greater offset have not yet been seen. Thus, the broker does not need to track acknowledgments for every single message—it only needs to periodically record the consumer offsets. The reduced bookkeeping overhead and the opportunities for batching and pipelining in this approach help increase the throughput of log-based systems.

Việc tiêu thụ một mảnh một cách tuần tự khiến cho việc biết được những thông điệp nào đã được xử lý trở nên dễ dàng: tất cả các thông điệp có offset nhỏ hơn offset hiện tại của một bên tiêu thụ đều đã được xử lý, và tất cả các thông điệp có offset lớn hơn thì chưa được nhìn thấy. Như vậy, broker không cần theo dõi xác nhận cho từng thông điệp riêng lẻ — nó chỉ cần định kỳ ghi lại các offset của bên tiêu thụ. Việc giảm chi phí ghi sổ này, cùng với cơ hội gộp lô (batching) và đường ống hóa (pipelining) trong cách tiếp cận này, giúp tăng thông lượng của các hệ thống dựa trên nhật ký.

This offset is in fact very similar to the log sequence number that is commonly found in single-leader database replication, and which we discussed in “Setting Up New Followers”. In database replication, the log sequence number allows a **follower** to reconnect to a leader after it has become disconnected, and resume replication without skipping any writes. Exactly the same principle is used here: the message broker behaves like a leader database, and the consumer like a follower.

Offset này thực ra rất giống với số thứ tự nhật ký (log sequence number) thường thấy trong nhân bản cơ sở dữ liệu một-leader, mà ta đã bàn ở “Thiết lập các follower mới”. Trong nhân bản cơ sở dữ liệu, số thứ tự nhật ký cho phép một follower kết nối lại với một leader sau khi nó đã bị ngắt kết nối, và tiếp tục nhân bản mà không bỏ sót bất kỳ thao tác ghi nào. Đúng cùng nguyên tắc đó được dùng ở đây: message broker hành xử như một cơ sở dữ liệu leader, và bên tiêu thụ như một follower.

If a consumer node fails, another node in the consumer group is assigned the failed consumer’s partitions, and it starts consuming messages at the last recorded offset. If the consumer had processed subsequent messages but not yet recorded their offset, those messages will be processed a second time upon restart. We will discuss ways of dealing with this issue later in the chapter.

Nếu một nút tiêu thụ hỏng, một nút khác trong nhóm tiêu thụ được gán các mảnh của bên tiêu thụ đã hỏng, và nó bắt đầu tiêu thụ thông điệp tại offset được ghi lại gần nhất. Nếu bên tiêu thụ đã xử lý các thông điệp tiếp theo nhưng chưa ghi lại offset của chúng, những thông điệp đó sẽ bị xử lý lần thứ hai khi khởi động lại. Ta sẽ bàn về cách xử lý vấn đề này ở phần sau của chương.

Disk space usage — Sử dụng không gian đĩa

If you only ever append to the log, you will eventually run out of disk space. To reclaim disk space, the log is actually divided into segments, and from time to time old segments are deleted or moved to archive storage. (We’ll discuss a more sophisticated way of freeing disk space later.)

Nếu bạn chỉ luôn nối thêm vào nhật ký, cuối cùng bạn sẽ hết không gian đĩa. Để thu hồi không gian đĩa, nhật ký thực ra được chia thành các đoạn (segment), và theo thời gian các đoạn cũ được xóa đi hoặc chuyển sang kho lưu trữ. (Ta sẽ bàn về một cách giải phóng không gian đĩa tinh vi hơn ở phần sau.)

This means that if a slow consumer cannot keep up with the rate of messages, and it falls so far behind that its consumer offset points to a deleted segment, it will miss some of the messages. Effectively, the log implements a bounded-size buffer that discards old messages when it gets full, also known as a circular buffer or ring buffer. However, since that buffer is on disk, it can be quite large.

Điều này có nghĩa là nếu một bên tiêu thụ chậm không bắt kịp tốc độ thông điệp, và nó tụt lại xa tới mức offset của nó trở tới một đoạn đã bị xóa, thì nó sẽ bỏ lỡ một số thông điệp. Thực chất, nhật ký hiện thực một bộ đệm có kích thước giới hạn, loại bỏ các thông điệp cũ khi nó đầy, còn được gọi là bộ đệm vòng (circular buffer hoặc ring buffer). Tuy nhiên, vì bộ đệm đó nằm trên đĩa, nó có thể khá lớn.

Let's do a back-of-the-envelope calculation. At the time of writing, a typical large hard drive has a capacity of 6 TB and a sequential write throughput of 150 MB/s. If you are writing messages at the fastest possible rate, it takes about 11 hours to fill the drive. Thus, the disk can buffer 11 hours' worth of messages, after which it will start overwriting old messages. This ratio remains the same, even if you use many hard drives and machines. In practice, deployments rarely use the full write bandwidth of the disk, so the log can typically keep a buffer of several days' or even weeks' worth of messages.

Hãy làm một phép tính ước lượng nhanh. Tại thời điểm viết cuốn sách này, một ổ cứng lớn điển hình có dung lượng 6 TB và thông lượng ghi tuần tự 150 MB/s. Nếu bạn ghi thông điệp ở tốc độ nhanh nhất có thể, sẽ mất khoảng 11 giờ để lấp đầy ổ. Như vậy, đĩa có thể đệm lượng thông điệp của 11 giờ, sau đó nó sẽ bắt đầu ghi đè lên các thông điệp cũ. Tỷ lệ này giữ nguyên, ngay cả khi bạn dùng nhiều ổ cứng và nhiều máy. Trong thực tế, các hệ thống triển khai hiếm khi dùng hết băng thông ghi của đĩa, nên nhật ký thường có thể giữ một bộ đệm với lượng thông điệp của vài ngày hoặc thậm chí vài tuần.

Regardless of how long you retain messages, the throughput of a log remains more or less constant, since every message is written to disk anyway [18]. This behavior is in contrast to messaging systems that keep messages in memory by default and only write them to disk if the queue grows too large: such systems are fast when queues are short and become much slower when they start writing to disk, so the throughput depends on the amount of history retained.

Bất kể bạn giữ thông điệp bao lâu, thông lượng của một nhật ký vẫn gần như không đổi, vì dù sao mọi thông điệp cũng đều được ghi xuống đĩa [18]. Hành vi này tương phản với các hệ thống thông điệp vốn mặc định giữ thông điệp trong bộ nhớ và chỉ ghi chúng xuống đĩa nếu hàng đợi lớn quá: những hệ thống như vậy nhanh khi hàng đợi ngắn và trở nên chậm hơn nhiều khi chúng bắt đầu ghi xuống đĩa, nên thông lượng phụ thuộc vào lượng lịch sử được giữ lại.

When consumers cannot keep up with producers — Khi các bên tiêu thụ không bắt kịp các nhà sản xuất

At the beginning of “Messaging Systems” we discussed three choices of what to do if a consumer cannot keep up with the rate at which producers are sending messages: dropping messages, buffering, or applying backpressure. In this taxonomy, the log-based approach is a form of buffering with a large but fixed-size buffer (limited by the available disk space).

Ở đầu phần “Các hệ thống thông điệp” ta đã bàn về ba lựa chọn xử lý khi một bên tiêu thụ không bắt kịp tốc độ mà các nhà sản xuất gửi thông điệp: loại bỏ thông điệp, đệm, hoặc áp dụng phản áp. Trong phân loại này, cách tiếp cận dựa trên nhật ký là một dạng đệm với một bộ đệm lớn nhưng có kích thước cố định (bị giới hạn bởi không gian đĩa khả dụng).

If a consumer falls so far behind that the messages it requires are older than what is retained on disk, it will not be able to read those messages—so the broker effectively drops old messages that go back further than the size of the buffer can accommodate. You can monitor how far a consumer is behind the head of the log, and raise an alert if it falls behind significantly. As the buffer is large, there is enough time for a human operator to fix the slow consumer and allow it to catch up before it starts missing messages.

Nếu một bên tiêu thụ tụt lại xa tới mức các thông điệp nó cần cũ hơn những gì còn được giữ trên đĩa, nó sẽ không thể đọc được những thông điệp đó — nên broker thực chất loại bỏ các thông điệp cũ vượt quá sức chứa của bộ đệm. Bạn có thể giám sát một bên tiêu thụ đang tụt lại bao xa so với đầu nhật ký, và phát cảnh báo nếu nó tụt lại đáng kể. Vì bộ đệm

lớn, có đủ thời gian để một người vận hành sửa bên tiêu thụ chậm và để nó bắt kịp trước khi nó bắt đầu bỏ lỡ thông điệp.

Even if a consumer does fall too far behind and starts missing messages, only that consumer is affected; it does not disrupt the service for other consumers. This fact is a big operational advantage: you can experimentally consume a **production** log for development, testing, or debugging purposes, without having to worry much about disrupting production services. When a consumer is shut down or crashes, it stops consuming resources—the only thing that remains is its consumer offset.

Ngay cả khi một bên tiêu thụ thực sự tụt lại quá xa và bắt đầu bỏ lỡ thông điệp, chỉ riêng bên tiêu thụ đó bị ảnh hưởng; nó không làm gián đoạn dịch vụ cho các bên tiêu thụ khác. Sự thật này là một lợi thế vận hành lớn: bạn có thể tiêu thụ một nhật ký production để thử nghiệm cho mục đích phát triển, kiểm thử, hoặc gỡ lỗi, mà không phải lo lắng nhiều về việc làm gián đoạn các dịch vụ production. Khi một bên tiêu thụ bị tắt hoặc sập, nó ngừng tiêu tốn tài nguyên — thứ duy nhất còn lại là offset của nó.

This behavior also contrasts with traditional message brokers, where you need to be careful to delete any queues whose consumers have been shut down—otherwise they continue unnecessarily accumulating messages and taking away memory from consumers that are still active.

Hành vi này cũng tương phản với các message broker truyền thống, nơi bạn cần cẩn thận xóa đi mọi hàng đợi mà các bên tiêu thụ của chúng đã bị tắt — nếu không chúng sẽ tiếp tục tích lũy thông điệp một cách không cần thiết và lấy mất bộ nhớ của các bên tiêu thụ vẫn còn hoạt động.

Replaying old messages — Phát lại các thông điệp cũ

We noted previously that with AMQP- and JMS-style message brokers, processing and acknowledging messages is a destructive operation, since it causes the messages to be deleted on the broker. On the other hand, in a log-based message broker, consuming messages is more like reading from a file: it is a read-only operation that does not change the log.

Trước đây ta đã lưu ý rằng với các message broker theo phong cách AMQP và JMS, việc xử lý và xác nhận thông điệp là một thao tác có tính phá hủy, vì nó khiến các thông điệp bị xóa trên broker. Ngược lại, trong một message broker dựa trên nhật ký, việc tiêu thụ thông điệp giống đọc từ một tệp hơn: đó là một thao tác chỉ-đọc không làm thay đổi nhật ký.

The only **side effect** of processing, besides any output of the consumer, is that the consumer offset moves forward. But the offset is under the consumer's control, so it can easily be manipulated if necessary: for example, you can start a copy of a consumer with yesterday's offsets and write the output to a different location, in order to reprocess the last day's worth of messages. You can repeat this any number of times, varying the processing code.

Tác dụng phụ duy nhất của việc xử lý, ngoài bất kỳ đầu ra nào của bên tiêu thụ, là offset của bên tiêu thụ tiến lên phía trước. Nhưng offset nằm dưới sự kiểm soát của bên tiêu thụ, nên nó có thể dễ dàng được điều chỉnh nếu cần: ví dụ, bạn có thể khởi động một bản sao của một bên tiêu thụ với các offset của ngày hôm qua và ghi đầu ra ra một vị trí khác, để xử lý lại lượng thông điệp của ngày hôm trước. Bạn có thể lặp lại điều này bao nhiêu lần tùy thích, thay đổi mã xử lý.

This aspect makes log-based messaging more like the batch processes of the last chapter, where derived data is clearly separated from input data through a repeatable transformation process. It allows more experimentation and easier recovery from errors and bugs, making it a good tool for integrating dataflows within an organization [24].

Khía cạnh này khiến việc truyền thông điệp dựa trên nhật ký giống với các tiến trình lô của chương trước hơn, nơi dữ liệu dẫn xuất được tách bạch rõ ràng khỏi dữ liệu đầu vào thông qua một tiến trình biến đổi có thể lặp lại. Nó cho phép thử nghiệm nhiều hơn và phục hồi dễ dàng hơn từ lỗi và bug, khiến nó trở thành một công cụ tốt để tích hợp các luồng dữ liệu trong một tổ chức [24].

Databases and Streams — Cơ sở dữ liệu và luồng

We have drawn some comparisons between message brokers and databases. Even though they have traditionally been considered separate categories of tools, we saw that log-based message brokers have been successful in taking ideas from databases and applying them to messaging. We can also go in reverse: take ideas from messaging and streams, and apply them to databases.

Ta đã rút ra một số phép so sánh giữa message broker và cơ sở dữ liệu. Mặc dù theo truyền thống chúng được coi là những loại công cụ riêng biệt, ta đã thấy rằng các message broker dựa trên nhật ký đã thành công trong việc lấy ý tưởng từ cơ sở dữ liệu và áp dụng chúng vào việc truyền thông điệp. Ta cũng có thể đi theo chiều ngược lại: lấy ý tưởng từ việc truyền thông điệp và luồng, rồi áp dụng chúng vào cơ sở dữ liệu.

We said previously that an event is a record of something that happened at some point in time. The thing that happened may be a user action (e.g., typing a search query), or a sensor reading, but it may also be a write to a database. The fact that something was written to a database is an event that can be captured, stored, and processed. This observation suggests that the connection between databases and streams runs deeper than just the physical storage of logs on disk—it is quite fundamental.

Trước đây ta đã nói rằng một sự kiện là một bản ghi về một thứ gì đó đã xảy ra tại một thời điểm nào đó. Thứ đã xảy ra có thể là một hành động của người dùng (ví dụ, gõ một truy vấn tìm kiếm), hoặc một giá trị đọc từ cảm biến, nhưng nó cũng có thể là một thao tác ghi vào một cơ sở dữ liệu. Việc một thứ gì đó đã được ghi vào một cơ sở dữ liệu là một sự kiện có thể được nắm bắt, lưu trữ, và xử lý. Quan sát này gợi ý rằng mối liên hệ giữa cơ sở dữ liệu và luồng còn sâu sắc hơn việc chỉ lưu nhật ký trên đĩa về mặt vật lý — nó khá mang tính nền tảng.

In fact, a replication log (see “Implementation of Replication Logs”) is a stream of database write events, produced by the leader as it processes transactions. The followers apply that stream of writes to their own copy of the database and thus end up with an accurate copy of the same data. The events in the replication log describe the data changes that occurred.

Thực tế, một nhật ký nhân bản (xem “Hiện thực các nhật ký nhân bản”) là một luồng các sự kiện ghi vào cơ sở dữ liệu, được leader sinh ra khi nó xử lý các giao dịch. Các follower áp dụng luồng các thao tác ghi đó vào bản sao cơ sở dữ liệu của riêng chúng và do đó rốt cuộc có được một bản sao chính xác của cùng dữ liệu đó. Các sự kiện trong nhật ký nhân bản mô tả những thay đổi dữ liệu đã xảy ra.

We also came across the state machine replication principle in “Total Order Broadcast”, which states: if every event represents a write to the database, and every replica processes the same events in the same order, then the replicas will all end up in the same final state. (Processing an event is assumed

to be a deterministic operation.) It's just another case of event streams!

Ta cũng đã gặp nguyên tắc nhân bản máy trạng thái (state machine replication) ở “Phát quảng bá theo thứ tự toàn phần”, nguyên tắc này phát biểu rằng: nếu mỗi sự kiện đại diện cho một thao tác ghi vào cơ sở dữ liệu, và mỗi bản sao xử lý cùng các sự kiện theo cùng thứ tự, thì các bản sao sẽ đều rốt cuộc ở cùng một trạng thái cuối. (Việc xử lý một sự kiện được giả định là một thao tác tất định.) Đây chỉ là một trường hợp khác của luồng sự kiện!

In this section we will first look at a problem that arises in **heterogeneous** data systems, and then explore how we can solve it by bringing ideas from event streams to databases.

Trong phần này, trước tiên ta sẽ xem một vấn đề nảy sinh trong các hệ thống dữ liệu không đồng nhất, rồi khám phá cách ta có thể giải quyết nó bằng cách đưa các ý tưởng từ luồng sự kiện vào cơ sở dữ liệu.

Keeping Systems in Sync — Giữ các hệ thống đồng bộ

As we have seen throughout this book, there is no single system that can satisfy all data storage, querying, and processing needs. In practice, most nontrivial applications need to combine several different technologies in order to satisfy their requirements: for example, using an OLTP database to serve user requests, a **cache** to speed up common requests, a full-text **index** to handle search queries, and a data warehouse for analytics. Each of these has its own copy of the data, stored in its own representation that is optimized for its own purposes.

Như ta đã thấy xuyên suốt cuốn sách này, không có một hệ thống đơn lẻ nào có thể thỏa mãn mọi nhu cầu lưu trữ, truy vấn, và xử lý dữ liệu. Trong thực tế, hầu hết các ứng dụng không tầm thường cần kết hợp một số công nghệ khác nhau để thỏa mãn các yêu cầu của chúng: ví dụ, dùng một cơ sở dữ liệu OLTP để phục vụ yêu cầu người dùng, một bộ nhớ đệm để tăng tốc các yêu cầu phổ biến, một chỉ mục toàn văn để xử lý các truy vấn tìm kiếm, và một kho dữ liệu để phân tích. Mỗi thứ trong số này có bản sao dữ liệu riêng của nó, được lưu ở dạng biểu diễn riêng được tối ưu cho mục đích riêng của nó.

As the same or related data appears in several different places, they need to be kept in sync with one another: if an item is updated in the database, it also needs to be updated in the cache, search indexes, and data warehouse. With data warehouses this synchronization is usually performed by ETL processes (see “Data Warehousing”), often by taking a full copy of a database, transforming it, and bulk-loading it into the data warehouse—in other words, a batch process. Similarly, we saw in “The Output of Batch Workflows” how search indexes, recommendation systems, and other derived data systems might be created using batch processes.

Khi cùng một dữ liệu hoặc dữ liệu liên quan xuất hiện ở nhiều nơi khác nhau, chúng cần được giữ đồng bộ với nhau: nếu một mục được cập nhật trong cơ sở dữ liệu, nó cũng cần được cập nhật trong bộ nhớ đệm, các chỉ mục tìm kiếm, và kho dữ liệu. Với các kho dữ liệu, việc đồng bộ này thường được thực hiện bằng các tiến trình ETL (xem “Kho dữ liệu”), thường bằng cách lấy một bản sao đầy đủ của một cơ sở dữ liệu, biến đổi nó, và nạp hàng loạt vào kho dữ liệu — nói cách khác, một tiến trình lô. Tương tự, ta đã thấy ở “Đầu ra của các luồng công việc lô” cách các chỉ mục tìm kiếm, hệ thống gợi ý, và các hệ thống dữ liệu dẫn xuất khác có thể được tạo ra bằng các tiến trình lô.

If periodic full database dumps are too slow, an alternative that is sometimes used is dual writes, in which the application code explicitly writes to each of the systems when data changes: for example, first writing to the database, then updating the **search index**, then invalidating the cache entries (or even performing those writes concurrently).

Nếu việc đổ toàn bộ cơ sở dữ liệu định kỳ quá chậm, một lựa chọn thay thế đôi khi được dùng là ghi kép (dual writes), trong đó mã ứng dụng ghi một cách tường minh vào từng hệ thống khi dữ liệu thay đổi: ví dụ, trước tiên ghi vào cơ sở dữ liệu, rồi cập nhật chỉ mục tìm kiếm, rồi vô hiệu hóa các mục trong bộ nhớ đệm (hoặc thậm chí thực hiện các thao tác ghi đó một cách đồng thời).

However, dual writes have some serious problems, one of which is a race condition illustrated in Figure 11-4. In this example, two clients concurrently want to update an item X: client 1 wants to set the value to A, and client 2 wants to set it to B. Both clients first write the new value to the database, then write it to the search index. Due to unlucky timing, the requests are interleaved: the database first sees the write from client 1 setting the value to A, then the write from client 2 setting the value to B, so the final value in the database is B. The search index first sees the write from client 2, then client 1, so the final value in the search index is A. The two systems are now permanently inconsistent with each other, even though no error occurred.

Tuy nhiên, ghi kép có một số vấn đề nghiêm trọng, một trong số đó là điều kiện tranh đua (race condition) được minh họa ở Hình 11-4. Trong ví dụ này, hai client đồng thời muốn cập nhật một mục X: client 1 muốn đặt giá trị thành A, và client 2 muốn đặt nó thành B. Cả hai client trước tiên ghi giá trị mới vào cơ sở dữ liệu, rồi ghi nó vào chỉ mục tìm kiếm. Do thời điểm không may, các yêu cầu xen kẽ nhau: cơ sở dữ liệu trước tiên thấy thao tác ghi từ client 1 đặt giá trị thành A, rồi thao tác ghi từ client 2 đặt giá trị thành B, nên giá trị cuối cùng trong cơ sở dữ liệu là B. Chỉ mục tìm kiếm trước tiên thấy thao tác ghi từ client 2, rồi của client 1, nên giá trị cuối cùng trong chỉ mục tìm kiếm là A. Hai hệ thống giờ vĩnh viễn không nhất quán với nhau, mặc dù không có lỗi nào xảy ra.

Figure 11-4. In the database, X is first set to A and then to B, while at the search index the writes arrive in the opposite order. — Hình 11-4. Trong cơ sở dữ liệu, X trước tiên được đặt thành A rồi thành B, trong khi tại chỉ mục tìm kiếm các thao tác ghi đến theo thứ tự ngược lại.

Unless you have some additional **concurrency** detection mechanism, such as the version vectors we discussed in “Detecting Concurrent Writes”, you will not even notice that concurrent writes occurred—one value will simply silently overwrite another value.

Trừ khi bạn có thêm một cơ chế phát hiện tương tranh nào đó, chẳng hạn các vector phiên bản (version vectors) mà ta đã bàn ở “Phát hiện các thao tác ghi đồng thời”, bạn thậm chí sẽ không nhận ra rằng các thao tác ghi đồng thời đã xảy ra — một giá trị sẽ đơn giản âm thầm ghi đè lên một giá trị khác.

Another problem with dual writes is that one of the writes may fail while the other succeeds. This is a fault-tolerance problem rather than a concurrency problem, but it also has the effect of the two systems becoming inconsistent with each other. Ensuring that they either both succeed or both fail is a case of the atomic commit problem, which is expensive to solve (see “Atomic Commit and Two-Phase Commit (2PC)”).

Một vấn đề khác của ghi kép là một trong các thao tác ghi có thể thất bại trong khi cái kia thành công. Đây là một vấn đề về khả năng chịu lỗi chứ không phải vấn đề tương tranh, nhưng nó cũng có tác dụng làm cho hai hệ thống trở nên không nhất quán với nhau. Việc bảo đảm rằng chúng hoặc cùng thành công hoặc cùng thất bại là một trường hợp của vấn đề cam kết nguyên tử, vốn tốn kém để giải quyết (xem “Cam kết nguyên tử và cam kết hai pha (2PC)”).

If you only have one replicated database with a single leader, then that leader determines the order of writes, so the state machine replication approach works among replicas of the database. However, in

Figure 11-4 there isn't a single leader: the database may have a leader and the search index may have a leader, but neither follows the other, and so conflicts can occur (see "Multi-Leader Replication").

Nếu bạn chỉ có một cơ sở dữ liệu được nhân bản với một leader duy nhất, thì leader đó quyết định thứ tự của các thao tác ghi, nên cách tiếp cận nhân bản máy trạng thái hoạt động được giữ giữa các bản sao của cơ sở dữ liệu. Tuy nhiên, trong Hình 11-4 không có một leader duy nhất: cơ sở dữ liệu có thể có một leader và chỉ mục tìm kiếm có thể có một leader, nhưng không cái nào theo sau cái kia, và do đó xung đột có thể xảy ra (xem "Nhân bản nhiều-leader").

The situation would be better if there really was only one leader—for example, the database—and if we could make the search index a follower of the database. But is this possible in practice?

Tình hình sẽ tốt hơn nếu thực sự chỉ có một leader duy nhất — ví dụ, cơ sở dữ liệu — và nếu ta có thể biến chỉ mục tìm kiếm thành một follower của cơ sở dữ liệu. Nhưng điều này có khả thi trong thực tế không?

Change Data Capture — Năm bắt thay đổi dữ liệu

The problem with most databases' replication logs is that they have long been considered to be an internal implementation detail of the database, not a public API. Clients are supposed to query the database through its **data model** and **query language**, not parse the replication logs and try to extract data from them.

Vấn đề với nhật ký nhân bản của hầu hết các cơ sở dữ liệu là chúng từ lâu đã được coi là một chi tiết hiện thực nội bộ của cơ sở dữ liệu, chứ không phải một API công khai. Các client được kỳ vọng truy vấn cơ sở dữ liệu thông qua mô hình dữ liệu và ngôn ngữ truy vấn của nó, chứ không phải phân tích nhật ký nhân bản và cố gắng trích xuất dữ liệu từ chúng.

For decades, many databases simply did not have a documented way of getting the log of changes written to them. For this reason it was difficult to take all the changes made in a database and replicate them to a different storage technology such as a search index, cache, or data warehouse.

Trong nhiều thập kỷ, nhiều cơ sở dữ liệu đơn giản là không có một cách được tài liệu hóa để lấy nhật ký các thay đổi được ghi vào chúng. Vì lý do này, việc lấy tất cả các thay đổi đã thực hiện trong một cơ sở dữ liệu và nhân bản chúng sang một công nghệ lưu trữ khác như chỉ mục tìm kiếm, bộ nhớ đệm, hoặc kho dữ liệu là khó.

More recently, there has been growing interest in change data capture (CDC), which is the process of observing all data changes written to a database and extracting them in a form in which they can be replicated to other systems. CDC is especially interesting if changes are made available as a stream, immediately as they are written.

Gần đây hơn, đã có sự quan tâm ngày càng tăng tới việc năm bắt thay đổi dữ liệu (change data capture - CDC), tức là tiến trình quan sát tất cả các thay đổi dữ liệu được ghi vào một cơ sở dữ liệu và trích xuất chúng ở một dạng mà chúng có thể được nhân bản sang các hệ thống khác. CDC đặc biệt thú vị nếu các thay đổi được cung cấp ở dạng một luồng, ngay khi chúng được ghi.

For example, you can capture the changes in a database and continually apply the same changes to a search index. If the log of changes is applied in the same order, you can expect the data in the search index to match the data in the database. The search index and any other derived data systems are just consumers of the change stream, as illustrated in Figure 11-5.

Ví dụ, bạn có thể nắm bắt các thay đổi trong một cơ sở dữ liệu và liên tục áp dụng cùng các thay đổi đó vào một chỉ mục tìm kiếm. Nếu nhật ký các thay đổi được áp dụng theo cùng thứ tự, bạn có thể kỳ vọng dữ liệu trong chỉ mục tìm kiếm khớp với dữ liệu trong cơ sở dữ liệu. Chỉ mục tìm kiếm và bất kỳ hệ thống dữ liệu dẫn xuất nào khác chỉ là các bên tiêu thụ của luồng thay đổi, như minh họa ở Hình 11-5.

Figure 11-5. Taking data in the order it was written to one database, and applying the changes to other systems in the same order. — Hình 11-5. Lấy dữ liệu theo thứ tự nó được ghi vào một cơ sở dữ liệu, và áp dụng các thay đổi vào các hệ thống khác theo cùng thứ tự.

Implementing change data capture — Hiện thực việc nắm bắt thay đổi dữ liệu

We can call the log consumers derived data systems, as discussed in the introduction to Part III: the data stored in the search index and the data warehouse is just another view onto the data in the system of record. Change data capture is a mechanism for ensuring that all changes made to the system of record are also reflected in the derived data systems so that the derived systems have an accurate copy of the data.

Ta có thể gọi các bên tiêu thụ nhật ký là các hệ thống dữ liệu dẫn xuất, như đã bàn ở phần giới thiệu của Phần III: dữ liệu được lưu trong chỉ mục tìm kiếm và kho dữ liệu chỉ là một khung nhìn khác lên dữ liệu trong hệ thống bản ghi gốc (system of record). Nắm bắt thay đổi dữ liệu là một cơ chế để bảo đảm rằng tất cả các thay đổi thực hiện trong hệ thống bản ghi gốc cũng được phản ánh trong các hệ thống dữ liệu dẫn xuất để các hệ thống dẫn xuất có một bản sao chính xác của dữ liệu.

Essentially, change data capture makes one database the leader (the one from which the changes are captured), and turns the others into followers. A log-based message broker is well suited for transporting the change events from the source database, since it preserves the ordering of messages (avoiding the reordering issue of Figure 11-2).

Về cơ bản, nắm bắt thay đổi dữ liệu biến một cơ sở dữ liệu thành leader (cái mà từ đó các thay đổi được nắm bắt), và biến các cơ sở dữ liệu khác thành follower. Một message broker dựa trên nhật ký rất phù hợp để vận chuyển các sự kiện thay đổi từ cơ sở dữ liệu nguồn, vì nó giữ thứ tự của các thông điệp (tránh được vấn đề xáo trộn thứ tự ở Hình 11-2).

Database triggers can be used to implement change data capture (see “Trigger-based replication”) by registering triggers that observe all changes to data tables and add corresponding entries to a changelog table. However, they tend to be fragile and have significant performance overheads. Parsing the replication log can be a more robust approach, although it also comes with challenges, such as handling **schema** changes.

Các trigger của cơ sở dữ liệu có thể được dùng để hiện thực việc nắm bắt thay đổi dữ liệu (xem “Nhập bản dựa trên trigger”) bằng cách đăng ký các trigger quan sát tất cả các thay đổi đối với các bảng dữ liệu và thêm các mục tương ứng vào một bảng nhật ký thay đổi (changelog). Tuy nhiên, chúng có xu hướng dễ vỡ và có chi phí phụ trội đáng kể về hiệu năng. Việc phân tích nhật ký nhập bản có thể là một cách tiếp cận vững vàng hơn, mặc dù nó cũng đi kèm những thách thức, chẳng hạn việc xử lý các thay đổi lược đồ.

LinkedIn’s Databus [25], Facebook’s Wormhole [26], and Yahoo!’s Sherpa [27] use this idea at large scale. Bottled Water implements CDC for PostgreSQL using an API that decodes the write-ahead log [28], Maxwell and Debezium do something similar for MySQL by parsing the binlog [29, 30, 31], Mongoriver reads the MongoDB oplog [32, 33], and GoldenGate provides similar facilities for Oracle [34, 35].

Databus của LinkedIn [25], Wormhole của Facebook [26], và Sherpa của Yahoo! [27] dùng ý tưởng này ở quy mô lớn. Bottled Water hiện thực CDC cho PostgreSQL bằng một API giải mã nhật ký ghi-trước [28], Maxwell và Debezium làm điều tương tự cho MySQL bằng cách phân tích binlog [29, 30, 31], Mongoriver đọc oplog của MongoDB [32, 33], và GoldenGate cung cấp các tiện ích tương tự cho Oracle [34, 35].

Like message brokers, change data capture is usually asynchronous: the system of record database does not wait for the change to be applied to consumers before committing it. This design has the operational advantage that adding a slow consumer does not affect the system of record too much, but it has the downside that all the issues of replication lag apply (see “Problems with Replication Lag”).

Giống như message broker, việc nắm bắt thay đổi dữ liệu thường là bất đồng bộ: cơ sở dữ liệu hệ thống bản ghi gốc không đợi thay đổi được áp dụng vào các bên tiêu thụ trước khi cam kết nó. Thiết kế này có lợi thế vận hành là việc thêm một bên tiêu thụ chậm không ảnh hưởng tới hệ thống bản ghi gốc quá nhiều, nhưng nó có nhược điểm là mọi vấn đề của độ trễ nhân bản đều áp dụng (xem “Các vấn đề với độ trễ nhân bản”).

Initial snapshot — Ảnh chụp ban đầu

If you have the log of all changes that were ever made to a database, you can reconstruct the entire state of the database by replaying the log. However, in many cases, keeping all changes forever would require too much disk space, and replaying it would take too long, so the log needs to be truncated.

Nếu bạn có nhật ký của tất cả các thay đổi từng được thực hiện đối với một cơ sở dữ liệu, bạn có thể tái dựng toàn bộ trạng thái của cơ sở dữ liệu bằng cách phát lại nhật ký. Tuy nhiên, trong nhiều trường hợp, việc giữ tất cả các thay đổi mãi mãi sẽ đòi hỏi quá nhiều không gian đĩa, và việc phát lại nó sẽ mất quá lâu, nên nhật ký cần được cắt ngắn.

Building a new full-text index, for example, requires a full copy of the entire database—it is not sufficient to only apply a log of recent changes, since it would be missing items that were not recently updated. Thus, if you don’t have the entire log history, you need to start with a consistent snapshot, as previously discussed in “Setting Up New Followers”.

Việc xây dựng một chỉ mục toàn văn mới, chẳng hạn, đòi hỏi một bản sao đầy đủ của toàn bộ cơ sở dữ liệu — không đủ nếu chỉ áp dụng một nhật ký các thay đổi gần đây, vì như vậy sẽ thiếu các mục không được cập nhật gần đây. Do đó, nếu bạn không có toàn bộ lịch sử nhật ký, bạn cần bắt đầu từ một ảnh chụp nhất quán, như đã bàn trước đó ở “Thiết lập các follower mới”.

The snapshot of the database must correspond to a known position or offset in the change log, so that you know at which point to start applying changes after the snapshot has been processed. Some CDC tools integrate this snapshot facility, while others leave it as a manual operation.

Ảnh chụp của cơ sở dữ liệu phải tương ứng với một vị trí hoặc offset đã biết trong nhật ký thay đổi, để bạn biết bắt đầu áp dụng các thay đổi tại điểm nào sau khi ảnh chụp đã được xử lý. Một số công cụ CDC tích hợp sẵn tiện ích ảnh chụp này, trong khi những công cụ khác để nó như một thao tác thủ công.

Log compaction — Nén nhật ký

If you can only keep a limited amount of log history, you need to go through the snapshot process every time you want to add a new derived **data system**. However, log compaction provides a good alternative.

Nếu bạn chỉ có thể giữ một lượng lịch sử nhật ký hạn chế, bạn cần phải trải qua tiến trình ảnh chụp mỗi khi muốn thêm một hệ thống dữ liệu dẫn xuất mới. Tuy nhiên, việc nén nhật ký (log compaction) cung cấp một lựa chọn thay thế tốt.

We discussed log compaction previously in “Hash Indexes”, in the context of log-structured storage engines (see Figure 3-2 for an example). The principle is simple: the storage **engine** periodically looks for log records with the same key, throws away any duplicates, and keeps only the most recent update for each key. This compaction and merging process runs in the background.

Ta đã bàn về việc nén nhật ký trước đây ở “Chỉ mục băm”, trong ngữ cảnh của các engine lưu trữ có cấu trúc nhật ký (xem Hình 3-2 để có một ví dụ). Nguyên tắc thì đơn giản: engine lưu trữ định kỳ tìm các bản ghi nhật ký có cùng khóa, vứt bỏ mọi bản trùng lặp, và chỉ giữ lại bản cập nhật gần nhất cho mỗi khóa. Tiến trình nén và trộn này chạy ở chế độ nền.

In a log-structured storage engine, an update with a special null value (a tombstone) indicates that a key was deleted, and causes it to be removed during log compaction. But as long as a key is not overwritten or deleted, it stays in the log forever. The disk space required for such a compacted log depends only on the current contents of the database, not the number of writes that have ever occurred in the database. If the same key is frequently overwritten, previous values will eventually be garbage-collected, and only the latest value will be retained.

Trong một engine lưu trữ có cấu trúc nhật ký, một bản cập nhật với một giá trị null đặc biệt (một bia mộ - tombstone) cho biết một khóa đã bị xóa, và khiến nó bị loại bỏ trong quá trình nén nhật ký. Nhưng miễn là một khóa không bị ghi đè hoặc xóa, nó vẫn nằm trong nhật ký mãi mãi. Không gian đĩa cần cho một nhật ký đã nén như vậy chỉ phụ thuộc vào nội dung hiện tại của cơ sở dữ liệu, chứ không phụ thuộc vào số thao tác ghi từng xảy ra trong cơ sở dữ liệu. Nếu cùng một khóa thường xuyên bị ghi đè, các giá trị trước đó rốt cuộc sẽ bị thu gom rác, và chỉ giá trị mới nhất được giữ lại.

The same idea works in the context of log-based message brokers and change data capture. If the CDC system is set up such that every change has a **primary key**, and every update for a key replaces the previous value for that key, then it's sufficient to keep just the most recent write for a particular key.

Cùng ý tưởng đó hoạt động trong ngữ cảnh của các message broker dựa trên nhật ký và việc nắm bắt thay đổi dữ liệu. Nếu hệ thống CDC được thiết lập sao cho mỗi thay đổi có một khóa chính, và mỗi bản cập nhật cho một khóa thay thế giá trị trước đó của khóa đó, thì chỉ cần giữ lại thao tác ghi gần nhất cho một khóa cụ thể là đủ.

Now, whenever you want to rebuild a derived data system such as a search index, you can start a new consumer from offset 0 of the log-compacted topic, and sequentially scan over all messages in the log. The log is guaranteed to contain the most recent value for every key in the database (and maybe some older values)—in other words, you can use it to obtain a full copy of the database contents without having to take another snapshot of the CDC source database.

Giờ đây, mỗi khi bạn muốn xây dựng lại một hệ thống dữ liệu dẫn xuất như một chỉ mục tìm kiếm, bạn có thể khởi động một bên tiêu thụ mới từ offset 0 của chủ đề đã được nén nhật ký, và quét tuần tự qua tất cả các thông điệp trong nhật ký. Nhật ký được bảo đảm chứa giá trị mới nhất cho mọi khóa trong cơ sở dữ liệu (và có lẽ một vài giá trị cũ hơn) — nói cách khác, bạn có thể dùng nó để có được một bản sao đầy đủ của nội dung cơ sở dữ liệu mà không phải chụp lại một ảnh khác của cơ sở dữ liệu nguồn CDC.

This log compaction feature is supported by Apache Kafka. As we shall see later in this chapter, it allows the message broker to be used for durable storage, not just for transient messaging.

Tính năng nén nhật ký này được Apache Kafka hỗ trợ. Như ta sẽ thấy ở phần sau của chương này, nó cho phép message broker được dùng để lưu trữ bền vững, chứ không chỉ để truyền thông điệp thoáng qua.

API support for change streams — Hỗ trợ API cho các luồng thay đổi

Increasingly, databases are beginning to support change streams as a first-class interface, rather than the typical retrofitted and reverse-engineered CDC efforts. For example, RethinkDB allows queries to subscribe to notifications when the results of a query change [36], Firebase [37] and CouchDB [38] provide data synchronization based on a change feed that is also made available to applications, and Meteor uses the MongoDB oplog to subscribe to data changes and update the user interface [39].

Ngày càng nhiều, các cơ sở dữ liệu bắt đầu hỗ trợ các luồng thay đổi (change streams) như một giao diện hạng nhất, thay vì kiểu nỗ lực CDC chấp vá và dịch ngược điển hình. Ví dụ, RethinkDB cho phép các truy vấn đăng ký nhận thông báo khi kết quả của một truy vấn thay đổi [36], Firebase [37] và CouchDB [38] cung cấp việc đồng bộ dữ liệu dựa trên một change feed cũng được cung cấp cho các ứng dụng, và Meteor dùng oplog của MongoDB để đăng ký nhận các thay đổi dữ liệu và cập nhật giao diện người dùng [39].

VoltDB allows transactions to continuously export data from a database in the form of a stream [40]. The database represents an output stream in the relational data model as a table into which transactions can insert tuples, but which cannot be queried. The stream then consists of the log of tuples that committed transactions have written to this special table, in the order they were committed. External consumers can asynchronously consume this log and use it to update derived data systems.

VoltDB cho phép các giao dịch liên tục xuất dữ liệu ra khỏi một cơ sở dữ liệu ở dạng một luồng [40]. Cơ sở dữ liệu này biểu diễn một luồng đầu ra trong mô hình dữ liệu quan hệ dưới dạng một bảng mà các giao dịch có thể chèn các bộ vào, nhưng không thể truy vấn được. Luồng khi đó bao gồm nhật ký các bộ mà các giao dịch đã cam kết đã ghi vào bảng đặc biệt này, theo thứ tự chúng được cam kết. Các bên tiêu thụ bên ngoài có thể tiêu thụ nhật ký này một cách bất đồng bộ và dùng nó để cập nhật các hệ thống dữ liệu dẫn xuất.

Kafka Connect [41] is an effort to integrate change data capture tools for a wide range of database systems with Kafka. Once the stream of change events is in Kafka, it can be used to update derived data systems such as search indexes, and also feed into stream processing systems as discussed later in this chapter.

Kafka Connect [41] là một nỗ lực tích hợp các công cụ nắm bắt thay đổi dữ liệu cho nhiều hệ cơ sở dữ liệu khác nhau với Kafka. Một khi luồng các sự kiện thay đổi đã nằm trong Kafka, nó có thể được dùng để cập nhật các hệ thống dữ liệu dẫn xuất như chỉ mục tìm kiếm, và cũng được nạp vào các hệ thống xử lý luồng như sẽ bàn ở phần sau của chương này.

Event Sourcing — Lấy nguồn từ sự kiện

There are some parallels between the ideas we've discussed here and event sourcing, a technique that was developed in the domain-driven design (DDD) community [42, 43, 44]. We will discuss event sourcing briefly, because it incorporates some useful and relevant ideas for streaming systems.

Có một số điểm tương đồng giữa các ý tưởng ta đã bàn ở đây và việc lấy nguồn từ sự kiện (event sourcing), một kỹ thuật được phát triển trong cộng đồng thiết kế hướng miền

(domain-driven design - DDD) [42, 43, 44]. Ta sẽ bàn về việc lấy nguồn từ sự kiện một cách ngắn gọn, vì nó hàm chứa một số ý tưởng hữu ích và liên quan cho các hệ thống luồng.

Similarly to change data capture, event sourcing involves storing all changes to the application state as a log of change events. The biggest difference is that event sourcing applies the idea at a different level of abstraction:

Tương tự như nắm bắt thay đổi dữ liệu, việc lấy nguồn từ sự kiện liên quan đến việc lưu trữ tất cả các thay đổi đối với trạng thái ứng dụng dưới dạng một nhật ký các sự kiện thay đổi. Khác biệt lớn nhất là việc lấy nguồn từ sự kiện áp dụng ý tưởng này ở một mức trừu tượng khác:

- In change data capture, the application uses the database in a mutable way, updating and deleting records at will. The log of changes is extracted from the database at a low level (e.g., by parsing the replication log), which ensures that the order of writes extracted from the database matches the order in which they were actually written, avoiding the race condition in Figure 11-4. The application writing to the database does not need to be aware that CDC is occurring.
- In event sourcing, the application logic is explicitly built on the basis of immutable events that are written to an event log. In this case, the event store is append-only, and updates or deletes are discouraged or prohibited. Events are designed to reflect things that happened at the application level, rather than low-level state changes.
 - Trong nắm bắt thay đổi dữ liệu, ứng dụng dùng cơ sở dữ liệu theo cách có thể thay đổi (mutable), cập nhật và xóa các bản ghi tùy ý. Nhật ký các thay đổi được trích xuất từ cơ sở dữ liệu ở mức thấp (ví dụ, bằng cách phân tích nhật ký nhân bản), điều này bảo đảm rằng thứ tự các thao tác ghi được trích xuất từ cơ sở dữ liệu khớp với thứ tự chúng thực sự được ghi, tránh được điều kiện tranh đua ở Hình 11-4. Ứng dụng ghi vào cơ sở dữ liệu không cần phải ý thức được rằng CDC đang diễn ra.
 - Trong việc lấy nguồn từ sự kiện, logic ứng dụng được xây dựng một cách tường minh trên cơ sở các sự kiện bất biến được ghi vào một nhật ký sự kiện. Trong trường hợp này, kho sự kiện là chỉ-nối-thêm, và việc cập nhật hoặc xóa bị ngăn cản hoặc cấm. Các sự kiện được thiết kế để phản ánh những thứ đã xảy ra ở mức ứng dụng, chứ không phải các thay đổi trạng thái ở mức thấp.

Event sourcing is a powerful technique for data modeling: from an application point of view it is more meaningful to record the user's actions as immutable events, rather than recording the effect of those actions on a mutable database. Event sourcing makes it easier to evolve applications over time, helps with debugging by making it easier to understand after the fact why something happened, and guards against application bugs (see “Advantages of immutable events”).

Việc lấy nguồn từ sự kiện là một kỹ thuật mạnh mẽ để mô hình hóa dữ liệu: từ góc nhìn của ứng dụng, việc ghi lại các hành động của người dùng dưới dạng các sự kiện bất biến có ý nghĩa hơn là ghi lại tác động của những hành động đó lên một cơ sở dữ liệu có thể thay đổi. Việc lấy nguồn từ sự kiện giúp việc tiến hóa ứng dụng theo thời gian dễ dàng hơn, hỗ trợ gỡ lỗi bằng cách giúp dễ hiểu hơn về sau lý do tại sao một thứ gì đó đã xảy ra, và phòng vệ trước các bug ứng dụng (xem “Ưu điểm của các sự kiện bất biến”).

For example, storing the event “student cancelled their course enrollment” clearly expresses the intent of a single action in a neutral fashion, whereas the side effects “one entry was deleted from the enrollments table, and one cancellation reason was added to the student feedback table” embed a lot of assumptions about the way the data is later going to be used. If a new application feature is introduced—for example, “the place is offered to the next person on the waiting list”—the event sourcing approach allows that new side effect to easily be chained off the existing event.

Ví dụ, việc lưu sự kiện “sinh viên đã hủy đăng ký khóa học của họ” diễn đạt rõ ràng ý định của một hành động đơn lẻ theo một cách trung lập, trong khi các tác dụng phụ “một mục đã bị xóa khỏi bảng đăng ký, và một lý do hủy đã được thêm vào bảng phản hồi của sinh viên” nhúng vào đó rất nhiều giả định về cách dữ liệu sẽ được dùng về sau. Nếu một tính năng ứng dụng mới được giới thiệu — ví dụ, “chỗ đó được mời cho người tiếp theo trong danh sách chờ” — thì cách tiếp cận lấy nguồn từ sự kiện cho phép dễ dàng nối tác dụng phụ mới đó vào sau sự kiện đã có.

Event sourcing is similar to the chronicle data model [45], and there are also similarities between an event log and the fact table that you find in a star schema (see “Stars and Snowflakes: Schemas for Analytics”).

Việc lấy nguồn từ sự kiện tương tự như mô hình dữ liệu biên niên (chronicle data model) [45], và cũng có những điểm tương đồng giữa một nhật ký sự kiện và bảng dữ kiện (fact table) mà bạn thấy trong một lược đồ hình sao (xem “Hình sao và bông tuyết: Các lược đồ cho phân tích”).

Specialized databases such as Event Store [46] have been developed to support applications using event sourcing, but in general the approach is independent of any particular tool. A conventional database or a log-based message broker can also be used to build applications in this style.

Các cơ sở dữ liệu chuyên biệt như Event Store [46] đã được phát triển để hỗ trợ các ứng dụng dùng việc lấy nguồn từ sự kiện, nhưng nói chung cách tiếp cận này độc lập với bất kỳ công cụ cụ thể nào. Một cơ sở dữ liệu thông thường hoặc một message broker dựa trên nhật ký cũng có thể được dùng để xây dựng các ứng dụng theo phong cách này.

Deriving current state from the event log — Dẫn xuất trạng thái hiện tại từ nhật ký sự kiện

An event log by itself is not very useful, because users generally expect to see the current state of a system, not the history of modifications. For example, on a shopping website, users expect to be able to see the current contents of their cart, not an append-only list of all the changes they have ever made to their cart.

Bản thân một nhật ký sự kiện không hữu ích lắm, vì người dùng nói chung kỳ vọng được thấy trạng thái hiện tại của một hệ thống, chứ không phải lịch sử các thay đổi. Ví dụ, trên một website mua sắm, người dùng kỳ vọng có thể thấy nội dung hiện tại của giỏ hàng của họ, chứ không phải một danh sách chỉ-nối-thêm gồm tất cả các thay đổi họ từng thực hiện đối với giỏ hàng.

Thus, applications that use event sourcing need to take the log of events (representing the data written to the system) and transform it into application state that is suitable for showing to a user (the way in which data is read from the system [47]). This transformation can use arbitrary logic, but it should be deterministic so that you can run it again and derive the same application state from the event log.

Do đó, các ứng dụng dùng việc lấy nguồn từ sự kiện cần lấy nhật ký các sự kiện (đại diện cho dữ liệu được ghi vào hệ thống) và biến đổi nó thành trạng thái ứng dụng phù hợp để hiển thị cho người dùng (cách mà dữ liệu được đọc ra từ hệ thống [47]). Phép biến đổi này có thể dùng logic tùy ý, nhưng nó nên là tất định để bạn có thể chạy lại nó và dẫn xuất ra cùng một trạng thái ứng dụng từ nhật ký sự kiện.

Like with change data capture, replaying the event log allows you to reconstruct the current state of the system. However, log compaction needs to be handled differently:

Giống như với nắm bắt thay đổi dữ liệu, việc phát lại nhật ký sự kiện cho phép bạn tái dựng trạng thái hiện tại của hệ thống. Tuy nhiên, việc nén nhật ký cần được xử lý một cách khác:

- A CDC event for the update of a record typically contains the entire new version of the record, so the current value for a primary key is entirely determined by the most recent event for that primary key, and log compaction can discard previous events for the same key.
 - On the other hand, with event sourcing, events are modeled at a higher level: an event typically expresses the intent of a user action, not the mechanics of the state update that occurred as a result of the action. In this case, later events typically do not override prior events, and so you need the full history of events to reconstruct the final state. Log compaction is not possible in the same way.
- Một sự kiện CDC cho việc cập nhật một bản ghi thường chứa toàn bộ phiên bản mới của bản ghi, nên giá trị hiện tại cho một khóa chính được xác định hoàn toàn bởi sự kiện gần nhất cho khóa chính đó, và việc nén nhật ký có thể vứt bỏ các sự kiện trước đó cho cùng khóa.
 - Ngược lại, với việc lấy nguồn từ sự kiện, các sự kiện được mô hình hóa ở mức cao hơn: một sự kiện thường diễn đạt ý định của một hành động người dùng, chứ không phải cơ chế cập nhật trạng thái đã xảy ra như kết quả của hành động. Trong trường hợp này, các sự kiện sau thường không ghi đè lên các sự kiện trước, và do đó bạn cần toàn bộ lịch sử các sự kiện để tái dựng trạng thái cuối. Việc nén nhật ký không thể thực hiện theo cùng cách đó.

Applications that use event sourcing typically have some mechanism for storing snapshots of the current state that is derived from the log of events, so they don't need to repeatedly reprocess the full log. However, this is only a performance optimization to speed up reads and recovery from crashes; the intention is that the system is able to store all raw events forever and reprocess the full event log whenever required. We discuss this assumption in "Limitations of immutability".

Các ứng dụng dùng việc lấy nguồn từ sự kiện thường có một cơ chế nào đó để lưu các ảnh chụp của trạng thái hiện tại được dẫn xuất từ nhật ký các sự kiện, nhờ vậy chúng không cần xử lý lại toàn bộ nhật ký lặp đi lặp lại. Tuy nhiên, đây chỉ là một tối ưu hiệu năng để tăng tốc việc đọc và phục hồi sau khi sập; chủ ý là hệ thống có thể lưu tất cả các sự kiện thô mãi mãi và xử lý lại toàn bộ nhật ký sự kiện bất cứ khi nào cần. Ta sẽ bàn về giả định này ở "Hạn chế của tính bất biến".

Commands and events — Lệnh và sự kiện

The event sourcing philosophy is careful to distinguish between events and commands [48]. When a request from a user first arrives, it is initially a command: at this point it may still fail, for example because some integrity condition is violated. The application must first validate that it can execute the command. If the validation is successful and the command is accepted, it becomes an event, which is durable and immutable.

Triết lý lấy nguồn từ sự kiện cẩn thận phân biệt giữa các sự kiện và các lệnh (command) [48]. Khi một yêu cầu từ một người dùng lần đầu đến, ban đầu nó là một lệnh: tại thời điểm này nó vẫn có thể thất bại, ví dụ vì một điều kiện toàn vẹn nào đó bị vi phạm. Ứng dụng trước tiên phải xác thực rằng nó có thể thực thi lệnh. Nếu việc xác thực thành công và lệnh được chấp nhận, nó trở thành một sự kiện, vốn là bền vững và bất biến.

For example, if a user tries to register a particular username, or reserve a seat on an airplane or in a theater, then the application needs to check that the username or seat is not already taken. (We

previously discussed this example in “**Fault-Tolerant Consensus**”.) When that check has succeeded, the application can generate an event to indicate that a particular username was registered by a particular user ID, or that a particular seat has been reserved for a particular customer.

Ví dụ, nếu một người dùng cố đăng ký một tên người dùng cụ thể, hoặc đặt một chỗ ngồi trên một máy bay hoặc trong một rạp hát, thì ứng dụng cần kiểm tra rằng tên người dùng hoặc chỗ ngồi đó chưa bị chiếm. (Trước đây ta đã bàn về ví dụ này ở “Đồng thuận chịu lỗi”.) Khi việc kiểm tra đó thành công, ứng dụng có thể sinh ra một sự kiện để cho biết rằng một tên người dùng cụ thể đã được đăng ký bởi một ID người dùng cụ thể, hoặc rằng một chỗ ngồi cụ thể đã được đặt cho một khách hàng cụ thể.

At the point when the event is generated, it becomes a fact. Even if the customer later decides to change or cancel the reservation, the fact remains true that they formerly held a reservation for a particular seat, and the change or cancellation is a separate event that is added later.

Tại thời điểm sự kiện được sinh ra, nó trở thành một dữ kiện (fact). Ngay cả khi khách hàng sau đó quyết định thay đổi hoặc hủy đặt chỗ, dữ kiện rằng trước đó họ đã từng giữ một đặt chỗ cho một chỗ ngồi cụ thể vẫn đúng, và việc thay đổi hoặc hủy là một sự kiện riêng được thêm vào sau.

A consumer of the event stream is not allowed to reject an event: by the time the consumer sees the event, it is already an immutable part of the log, and it may have already been seen by other consumers. Thus, any validation of a command needs to happen synchronously, before it becomes an event—for example, by using a serializable transaction that atomically validates the command and publishes the event.

Một bên tiêu thụ của luồng sự kiện không được phép từ chối một sự kiện: vào lúc bên tiêu thụ thấy sự kiện, nó đã là một phần bất biến của nhật ký, và nó có thể đã được các bên tiêu thụ khác thấy rồi. Như vậy, bất kỳ việc xác thực một lệnh nào cũng cần xảy ra một cách đồng bộ, trước khi nó trở thành một sự kiện — ví dụ, bằng cách dùng một giao dịch khả tuần tự (serializable) vốn xác thực lệnh và phát hành sự kiện một cách nguyên tử.

Alternatively, the user request to reserve a seat could be split into two events: first a tentative reservation, and then a separate confirmation event once the reservation has been validated (as discussed in “Implementing linearizable storage using total order broadcast”). This split allows the validation to take place in an asynchronous process.

Một cách khác, yêu cầu của người dùng để đặt một chỗ ngồi có thể được chia thành hai sự kiện: trước tiên là một đặt chỗ tạm thời, rồi một sự kiện xác nhận riêng một khi đặt chỗ đã được xác thực (như đã bàn ở “Hiện thực lưu trữ khả tuyến tính bằng phát quảng bá theo thứ tự toàn phần”). Việc chia tách này cho phép việc xác thực diễn ra trong một tiến trình bất đồng bộ.

State, Streams, and Immutability — Trạng thái, luồng, và tính bất biến

We saw in Chapter 10 that batch processing benefits from the immutability of its input files, so you can run experimental processing jobs on existing input files without fear of damaging them. This principle of immutability is also what makes event sourcing and change data capture so powerful.

Ta đã thấy ở Chương 10 rằng xử lý lô hưởng lợi từ tính bất biến của các tệp đầu vào của nó, nhờ vậy bạn có thể chạy các tác vụ xử lý thử nghiệm trên các tệp đầu vào hiện có mà

không sợ làm hỏng chúng. Nguyên tắc bất biến này cũng chính là thứ làm cho việc lấy nguồn từ sự kiện và nắm bắt thay đổi dữ liệu mạnh mẽ đến vậy.

We normally think of databases as storing the current state of the application—this representation is optimized for reads, and it is usually the most convenient for serving queries. The nature of state is that it changes, so databases support updating and deleting data as well as inserting it. How does this fit with immutability?

Ta thường nghĩ về cơ sở dữ liệu như nơi lưu trạng thái hiện tại của ứng dụng — cách biểu diễn này được tối ưu cho việc đọc, và nó thường là tiện lợi nhất để phục vụ các truy vấn. Bản chất của trạng thái là nó thay đổi, nên cơ sở dữ liệu hỗ trợ cập nhật và xóa dữ liệu cũng như chèn nó. Điều này khớp với tính bất biến như thế nào?

Whenever you have state that changes, that state is the result of the events that mutated it over time. For example, your list of currently available seats is the result of the reservations you have processed, the current account balance is the result of the credits and debits on the account, and the **response time graph** for your web server is an aggregation of the individual response times of all web requests that have occurred.

Mỗi khi bạn có một trạng thái thay đổi, trạng thái đó là kết quả của các sự kiện đã biến đổi nó theo thời gian. Ví dụ, danh sách các chỗ ngồi hiện đang trống là kết quả của các đặt chỗ bạn đã xử lý, số dư tài khoản hiện tại là kết quả của các khoản ghi có và ghi nợ trên tài khoản, và biểu đồ thời gian phản hồi cho máy chủ web của bạn là một tổng hợp các thời gian phản hồi riêng lẻ của tất cả các yêu cầu web đã xảy ra.

No matter how the state changes, there was always a sequence of events that caused those changes. Even as things are done and undone, the fact remains true that those events occurred. The key idea is that mutable state and an append-only log of immutable events do not contradict each other: they are two sides of the same coin. The log of all changes, the changelog, represents the evolution of state over time.

Bất kể trạng thái thay đổi như thế nào, luôn có một dãy sự kiện gây ra những thay đổi đó. Ngay cả khi mọi thứ được làm rồi lại được làm ngược lại, dữ kiện rằng những sự kiện đó đã xảy ra vẫn đúng. Ý tưởng then chốt là trạng thái có thể thay đổi và một nhật ký chỉ-nối-thêm gồm các sự kiện bất biến không mâu thuẫn với nhau: chúng là hai mặt của cùng một đồng xu. Nhật ký của tất cả các thay đổi, tức nhật ký thay đổi (changelog), đại diện cho sự tiến hóa của trạng thái theo thời gian.

If you are mathematically inclined, you might say that the application state is what you get when you integrate an event stream over time, and a change stream is what you get when you differentiate the state by time, as shown in Figure 11-6 [49, 50, 51]. The analogy has limitations (for example, the second derivative of state does not seem to be meaningful), but it's a useful starting point for thinking about data.

Nếu bạn thiên về toán học, bạn có thể nói rằng trạng thái ứng dụng là cái bạn nhận được khi tích phân một luồng sự kiện theo thời gian, và một luồng thay đổi là cái bạn nhận được khi vi phân trạng thái theo thời gian, như minh họa ở Hình 11-6 [49, 50, 51]. Phép loại suy này có những hạn chế (ví dụ, đạo hàm bậc hai của trạng thái dường như không có ý nghĩa gì), nhưng nó là một điểm khởi đầu hữu ích để suy nghĩ về dữ liệu.

Figure 11-6. The relationship between the current application state and an event stream. — Hình 11-6. Mỗi quan hệ giữa trạng thái ứng dụng hiện tại và một luồng sự kiện. If you store the changelog durably, that simply has the effect of making the state reproducible. If you consider the

log of events to be your system of record, and any mutable state as being derived from it, it becomes easier to reason about the flow of data through a system. As Pat Helland puts it [52]:

Nếu bạn lưu nhật ký thay đổi một cách bền vững, điều đó đơn giản có tác dụng làm cho trạng thái có thể tái tạo. Nếu bạn coi nhật ký các sự kiện là hệ thống bản ghi gốc của mình, và coi mọi trạng thái có thể thay đổi là được dẫn xuất từ nó, thì việc lập luận về dòng chảy của dữ liệu qua một hệ thống trở nên dễ dàng hơn. Như Pat Helland diễn đạt [52]:

Transaction logs record all the changes made to the database. High-speed appends are the only way to change the log. From this perspective, the contents of the database hold a caching of the latest record values in the logs. The truth is the log. The database is a cache of a subset of the log. That cached subset happens to be the latest value of each record and index value from the log.

Các nhật ký giao dịch ghi lại tất cả các thay đổi thực hiện đối với cơ sở dữ liệu. Việc nối thêm tốc độ cao là cách duy nhất để thay đổi nhật ký. Từ góc nhìn này, nội dung của cơ sở dữ liệu giữ một bản đệm của các giá trị bản ghi mới nhất trong các nhật ký. Sự thật nằm ở nhật ký. Cơ sở dữ liệu là một bộ đệm của một tập con của nhật ký. Tập con được đệm đó tình cờ là giá trị mới nhất của mỗi bản ghi và mỗi giá trị chỉ mục từ nhật ký.

Log compaction, as discussed in “Log compaction”, is one way of bridging the distinction between log and database state: it retains only the latest version of each record, and discards overwritten versions.

Việc nén nhật ký, như đã bàn ở “Nén nhật ký”, là một cách để bắc cầu giữa sự phân biệt nhật ký với trạng thái cơ sở dữ liệu: nó chỉ giữ lại phiên bản mới nhất của mỗi bản ghi, và vứt bỏ các phiên bản đã bị ghi đè.

Advantages of immutable events — Ưu điểm của các sự kiện bất biến

Immutability in databases is an old idea. For example, accountants have been using immutability for centuries in financial bookkeeping. When a transaction occurs, it is recorded in an append-only ledger, which is essentially a log of events describing money, goods, or services that have changed hands. The accounts, such as profit and loss or the balance sheet, are derived from the transactions in the ledger by adding them up [53].

Tính bất biến trong cơ sở dữ liệu là một ý tưởng cũ. Ví dụ, các kế toán viên đã dùng tính bất biến trong nhiều thế kỷ trong việc ghi sổ tài chính. Khi một giao dịch xảy ra, nó được ghi vào một sổ cái chỉ-nối-thêm, về cơ bản là một nhật ký các sự kiện mô tả tiền, hàng hóa, hoặc dịch vụ đã chuyển tay. Các tài khoản, chẳng hạn lãi và lỗ hoặc bảng cân đối kế toán, được dẫn xuất từ các giao dịch trong sổ cái bằng cách cộng chúng lại [53].

If a mistake is made, accountants don’t erase or change the incorrect transaction in the ledger—instead, they add another transaction that compensates for the mistake, for example refunding an incorrect charge. The incorrect transaction still remains in the ledger forever, because it might be important for auditing reasons. If incorrect figures, derived from the incorrect ledger, have already been published, then the figures for the next accounting period include a correction. This process is entirely normal in accounting [54].

Nếu có một sai sót, các kế toán viên không tẩy xóa hoặc thay đổi giao dịch sai trong sổ cái — thay vào đó, họ thêm một giao dịch khác để bù trừ cho sai sót, ví dụ hoàn lại một khoản tính phí sai. Giao dịch sai vẫn nằm trong sổ cái mãi mãi, vì nó có thể quan trọng vì lý do kiểm toán. Nếu các con số sai, được dẫn xuất từ sổ cái sai, đã được công bố, thì các con số cho kỳ kế toán tiếp theo bao gồm một khoản hiệu chỉnh. Tiến trình này hoàn toàn bình thường trong kế toán [54].

Although such auditability is particularly important in financial systems, it is also beneficial for many other systems that are not **subject** to such strict regulation. As discussed in “Philosophy of batch process outputs”, if you accidentally deploy buggy code that writes bad data to a database, recovery is much harder if the code is able to destructively overwrite data. With an append-only log of immutable events, it is much easier to diagnose what happened and recover from the problem.

Mặc dù khả năng kiểm toán như vậy đặc biệt quan trọng trong các hệ thống tài chính, nó cũng có lợi cho nhiều hệ thống khác không chịu sự quản lý nghiêm ngặt như vậy. Như đã bàn ở “Triết lý về đầu ra của tiến trình lô”, nếu bạn vô tình triển khai mã có bug ghi dữ liệu xấu vào một cơ sở dữ liệu, việc phục hồi sẽ khó hơn nhiều nếu mã có khả năng ghi đè dữ liệu một cách phá hủy. Với một nhật ký chỉ-nối-thêm gồm các sự kiện bất biến, việc chẩn đoán chuyện gì đã xảy ra và phục hồi từ vấn đề sẽ dễ hơn nhiều.

Immutable events also capture more information than just the current state. For example, on a shopping website, a customer may add an item to their cart and then remove it again. Although the second event cancels out the first event from the point of view of order fulfillment, it may be useful to know for analytics purposes that the customer was considering a particular item but then decided against it. Perhaps they will choose to buy it in the future, or perhaps they found a substitute. This information is recorded in an event log, but would be lost in a database that deletes items when they are removed from the cart [42].

Các sự kiện bất biến cũng nắm bắt nhiều thông tin hơn so với chỉ trạng thái hiện tại. Ví dụ, trên một website mua sắm, một khách hàng có thể thêm một mục vào giỏ hàng của họ rồi lại xóa nó đi. Mặc dù sự kiện thứ hai triệt tiêu sự kiện thứ nhất từ góc nhìn của việc hoàn tất đơn hàng, nhưng việc biết rằng khách hàng đã từng cân nhắc một mục cụ thể rồi lại quyết định không mua có thể hữu ích cho mục đích phân tích. Có lẽ trong tương lai họ sẽ chọn mua nó, hoặc có lẽ họ đã tìm được một thứ thay thế. Thông tin này được ghi lại trong một nhật ký sự kiện, nhưng sẽ bị mất trong một cơ sở dữ liệu vốn xóa các mục khi chúng bị bỏ ra khỏi giỏ hàng [42].

Deriving several views from the same event log — Dẫn xuất nhiều khung nhìn từ cùng một nhật ký sự kiện

Moreover, by separating mutable state from the immutable event log, you can derive several different read-oriented representations from the same log of events. This works just like having multiple consumers of a stream (Figure 11-5): for example, the analytic database Druid ingests directly from Kafka using this approach [55], Pistachio is a distributed key-value store that uses Kafka as a commit log [56], and Kafka Connect sinks can export data from Kafka to various different databases and indexes [41]. It would make sense for many other storage and indexing systems, such as search servers, to similarly take their input from a distributed log (see “Keeping Systems in Sync”).

Hơn nữa, bằng cách tách trạng thái có thể thay đổi khỏi nhật ký sự kiện bất biến, bạn có thể dẫn xuất nhiều cách biểu diễn thiên về đọc khác nhau từ cùng một nhật ký các sự kiện. Điều này hoạt động giống hệt như việc có nhiều bên tiêu thụ của một luồng (Hình 11-5): ví dụ, cơ sở dữ liệu phân tích Druid nạp trực tiếp từ Kafka theo cách tiếp cận này [55], Pistachio là một kho khóa-giá trị phân tán dùng Kafka làm một nhật ký cam kết (commit log) [56], và các sink của Kafka Connect có thể xuất dữ liệu từ Kafka sang nhiều cơ sở dữ liệu và chỉ mục khác nhau [41]. Sẽ hợp lý nếu nhiều hệ thống lưu trữ và lập chỉ mục khác, chẳng hạn các máy chủ tìm kiếm, cũng tương tự lấy đầu vào của chúng từ một nhật ký phân tán (xem “Giữ các hệ thống đồng bộ”).

Having an explicit translation step from an event log to a database makes it easier to evolve your application over time: if you want to introduce a new feature that presents your existing data in some

new way, you can use the event log to build a separate read-optimized view for the new feature, and run it alongside the existing systems without having to modify them. Running old and new systems side by side is often easier than performing a complicated schema **migration** in an existing system. Once the old system is no longer needed, you can simply shut it down and reclaim its resources [47, 57].

Việc có một bước chuyển dịch tường minh từ một nhật ký sự kiện sang một cơ sở dữ liệu khiến việc tiến hóa ứng dụng theo thời gian dễ dàng hơn: nếu bạn muốn giới thiệu một tính năng mới trình bày dữ liệu hiện có của mình theo một cách mới nào đó, bạn có thể dùng nhật ký sự kiện để xây dựng một khung nhìn riêng được tối ưu cho việc đọc cho tính năng mới đó, và chạy nó song song với các hệ thống hiện có mà không phải sửa đổi chúng. Việc chạy song song các hệ thống cũ và mới thường dễ hơn việc thực hiện một cuộc di trú lược đồ phức tạp trong một hệ thống đang chạy. Một khi hệ thống cũ không còn cần nữa, bạn có thể đơn giản tắt nó đi và thu hồi tài nguyên của nó [47, 57].

Storing data is normally quite straightforward if you don't have to worry about how it is going to be queried and accessed; many of the complexities of schema design, indexing, and storage engines are the result of wanting to support certain query and access patterns (see Chapter 3). For this reason, you gain a lot of flexibility by separating the form in which data is written from the form it is read, and by allowing several different read views. This idea is sometimes known as command query responsibility segregation (CQRS) [42, 58, 59].

Việc lưu trữ dữ liệu thường khá đơn giản nếu bạn không phải lo lắng về cách nó sẽ được truy vấn và truy cập; nhiều sự phức tạp của thiết kế lược đồ, lập chỉ mục, và các engine lưu trữ là kết quả của việc muốn hỗ trợ một số mẫu truy vấn và truy cập nhất định (xem Chương 3). Vì lý do này, bạn có được rất nhiều tính linh hoạt bằng cách tách dạng mà dữ liệu được ghi vào khỏi dạng mà nó được đọc ra, và bằng cách cho phép nhiều khung nhìn đọc khác nhau. Ý tưởng này đôi khi được gọi là phân tách trách nhiệm truy vấn và lệnh (command query responsibility segregation - CQRS) [42, 58, 59].

The traditional approach to database and schema design is based on the fallacy that data must be written in the same form as it will be queried. Debates about **normalization** and **denormalization** (see “**Many-to-One** and **Many-to-Many** Relationships”) become largely irrelevant if you can translate data from a write-optimized event log to read-optimized application state: it is entirely reasonable to **denormalize** data in the read-optimized views, as the translation process gives you a mechanism for keeping it consistent with the event log.

Cách tiếp cận truyền thống đối với thiết kế cơ sở dữ liệu và lược đồ dựa trên nguy hiểm rằng dữ liệu phải được ghi ở cùng dạng mà nó sẽ được truy vấn. Các cuộc tranh luận về chuẩn hóa và phi chuẩn hóa (xem “Quan hệ nhiều-một và nhiều-nhiều”) trở nên phần lớn không còn liên quan nếu bạn có thể chuyển dịch dữ liệu từ một nhật ký sự kiện được tối ưu cho việc ghi sang trạng thái ứng dụng được tối ưu cho việc đọc: việc phi chuẩn hóa dữ liệu trong các khung nhìn được tối ưu cho việc đọc là hoàn toàn hợp lý, vì tiến trình chuyển dịch cho bạn một cơ chế để giữ nó nhất quán với nhật ký sự kiện.

In “Describing Load” we discussed Twitter’s home timelines, a cache of recently written tweets by the people a particular user is following (like a mailbox). This is another example of read-optimized state: home timelines are highly denormalized, since your tweets are duplicated in all of the timelines of the people following you. However, the fan-out service keeps this duplicated state in sync with new tweets and new following relationships, which keeps the duplication manageable.

Trong “Mô tả tải” ta đã bàn về các dòng thời gian chính của Twitter, một bộ nhớ đệm các tweet được viết gần đây bởi những người mà một người dùng cụ thể đang theo dõi (giống một hòm thư). Đây là một ví dụ khác về trạng thái được tối ưu cho việc đọc: các dòng thời

gian chính được phi chuẩn hóa cao, vì các tweet của bạn được nhân bản trong tất cả các dòng thời gian của những người theo dõi bạn. Tuy nhiên, dịch vụ fan-out giữ trạng thái nhân bản này đồng bộ với các tweet mới và các quan hệ theo dõi mới, điều này giữ cho sự nhân bản nằm trong tầm kiểm soát.

Concurrency control — Điều khiển tương tranh

The biggest downside of event sourcing and change data capture is that the consumers of the event log are usually asynchronous, so there is a possibility that a user may make a write to the log, then read from a log-derived view and find that their write has not yet been reflected in the read view. We discussed this problem and potential solutions previously in “Reading Your Own Writes”.

Nhược điểm lớn nhất của việc lấy nguồn từ sự kiện và nắm bắt thay đổi dữ liệu là các bên tiêu thụ của nhật ký sự kiện thường là bất đồng bộ, nên có khả năng một người dùng thực hiện một thao tác ghi vào nhật ký, rồi đọc từ một khung nhìn được dẫn xuất từ nhật ký và phát hiện rằng thao tác ghi của họ chưa được phản ánh trong khung nhìn đọc. Ta đã bàn về vấn đề này và các giải pháp tiềm năng trước đây ở “Đọc lại được thao tác ghi của chính mình”.

One solution would be to perform the updates of the read view synchronously with appending the event to the log. This requires a transaction to combine the writes into an atomic unit, so either you need to keep the event log and the read view in the same storage system, or you need a distributed transaction across the different systems. Alternatively, you could use the approach discussed in “Implementing linearizable storage using total order broadcast”.

Một giải pháp sẽ là thực hiện việc cập nhật khung nhìn đọc một cách đồng bộ với việc nối thêm sự kiện vào nhật ký. Điều này đòi hỏi một giao dịch để kết hợp các thao tác ghi thành một đơn vị nguyên tử, nên hoặc bạn cần giữ nhật ký sự kiện và khung nhìn đọc trong cùng một hệ thống lưu trữ, hoặc bạn cần một giao dịch phân tán xuyên qua các hệ thống khác nhau. Một cách khác, bạn có thể dùng cách tiếp cận đã bàn ở “Hiện thực lưu trữ khả tuyến tính bằng phát quảng bá theo thứ tự toàn phần”.

On the other hand, deriving the current state from an event log also simplifies some aspects of concurrency control. Much of the need for multi-object transactions (see “Single-Object and Multi-Object Operations”) stems from a single user action requiring data to be changed in several different places. With event sourcing, you can design an event such that it is a self-contained description of a user action. The user action then requires only a single write in one place—namely appending the events to the log—which is easy to make atomic.

Mặt khác, việc dẫn xuất trạng thái hiện tại từ một nhật ký sự kiện cũng đơn giản hóa một số khía cạnh của điều khiển tương tranh. Phần lớn nhu cầu về các giao dịch đa-đối-tượng (xem “Thao tác đơn-đối-tượng và đa-đối-tượng”) bắt nguồn từ việc một hành động đơn lẻ của người dùng đòi hỏi dữ liệu phải được thay đổi ở nhiều nơi khác nhau. Với việc lấy nguồn từ sự kiện, bạn có thể thiết kế một sự kiện sao cho nó là một mô tả tự chứa của một hành động người dùng. Hành động người dùng khi đó chỉ đòi hỏi một thao tác ghi duy nhất ở một nơi — cụ thể là nối thêm các sự kiện vào nhật ký — điều này dễ làm thành nguyên tử.

If the event log and the application state are partitioned in the same way (for example, processing an event for a customer in partition 3 only requires updating partition 3 of the application state), then a straightforward single-threaded log consumer needs no concurrency control for writes—by construction, it only processes a single event at a time (see also “Actual Serial Execution”). The log removes the nondeterminism of concurrency by defining a serial order of events in a partition [24].

If an event touches multiple state partitions, a bit more work is required, which we will discuss in Chapter 12.

Nếu nhật ký sự kiện và trạng thái ứng dụng được phân mảnh theo cùng một cách (ví dụ, việc xử lý một sự kiện cho một khách hàng ở mảnh 3 chỉ đòi hỏi cập nhật mảnh 3 của trạng thái ứng dụng), thì một bên tiêu thụ nhật ký đơn luồng đơn giản không cần điều khiển tương tranh cho các thao tác ghi — về mặt cấu trúc, nó chỉ xử lý một sự kiện tại một thời điểm (xem thêm “Thực thi tuần tự thực sự”). Nhật ký loại bỏ tính bất định của tương tranh bằng cách định nghĩa một thứ tự tuần tự của các sự kiện trong một mảnh [24]. Nếu một sự kiện chạm tới nhiều mảnh trạng thái, cần làm thêm một chút việc, điều ta sẽ bàn ở Chương 12.

Limitations of immutability — Hạn chế của tính bất biến

Many systems that don’t use an event-sourced model nevertheless rely on immutability: various databases internally use immutable data structures or multi-version data to support point-in-time snapshots (see “Indexes and snapshot isolation”). Version control systems such as Git, Mercurial, and Fossil also rely on immutable data to preserve version history of files.

Nhiều hệ thống không dùng mô hình lấy nguồn từ sự kiện nhưng dù vậy vẫn dựa vào tính bất biến: nhiều cơ sở dữ liệu dùng nội bộ các cấu trúc dữ liệu bất biến hoặc dữ liệu đa-phiên-bản để hỗ trợ các ảnh chụp tại một thời điểm (xem “Chỉ mục và cô lập ảnh chụp”). Các hệ thống kiểm soát phiên bản như Git, Mercurial, và Fossil cũng dựa vào dữ liệu bất biến để giữ lịch sử phiên bản của các tệp.

To what extent is it feasible to keep an immutable history of all changes forever? The answer depends on the amount of churn in the dataset. Some workloads mostly add data and rarely update or delete; they are easy to make immutable. Other workloads have a high rate of updates and deletes on a comparatively small dataset; in these cases, the immutable history may grow prohibitively large, fragmentation may become an issue, and the performance of compaction and **garbage collection** becomes crucial for operational robustness [60, 61].

Việc giữ một lịch sử bất biến của tất cả các thay đổi mãi mãi khả thi tới mức nào? Câu trả lời phụ thuộc vào lượng biến động trong tập dữ liệu. Một số khối lượng công việc chủ yếu thêm dữ liệu và hiếm khi cập nhật hoặc xóa; chúng dễ làm thành bất biến. Các khối lượng công việc khác có tỷ lệ cập nhật và xóa cao trên một tập dữ liệu tương đối nhỏ; trong những trường hợp này, lịch sử bất biến có thể lớn lên quá mức cho phép, sự phân mảnh có thể trở thành một vấn đề, và hiệu năng của việc nén và thu gom rác trở nên then chốt đối với sự vững vàng trong vận hành [60, 61].

Besides the performance reasons, there may also be circumstances in which you need data to be deleted for administrative reasons, in spite of all immutability. For example, privacy regulations may require deleting a user’s personal information after they close their account, data protection legislation may require erroneous information to be removed, or an accidental leak of sensitive information may need to be contained.

Ngoài các lý do về hiệu năng, cũng có thể có những tình huống mà bạn cần dữ liệu bị xóa vì lý do hành chính, bất chấp mọi tính bất biến. Ví dụ, các quy định về quyền riêng tư có thể đòi hỏi xóa thông tin cá nhân của một người dùng sau khi họ đóng tài khoản, luật bảo vệ dữ liệu có thể đòi hỏi loại bỏ thông tin sai, hoặc một vụ rò rỉ vô tình thông tin nhạy cảm có thể cần được khống chế.

In these circumstances, it’s not sufficient to just append another event to the log to indicate that the prior data should be considered deleted—you actually want to rewrite history and pretend that the

data was never written in the first place. For example, Datomic calls this feature excision [62], and the Fossil version control system has a similar concept called shunning [63].

Trong những tình huống này, không đủ nếu chỉ nối thêm một sự kiện khác vào nhật ký để cho biết rằng dữ liệu trước đó nên được coi là đã bị xóa — bạn thực sự muốn viết lại lịch sử và giả vờ rằng dữ liệu đó chưa bao giờ được ghi ngay từ đầu. Ví dụ, Datomic gọi tính năng này là cắt bỏ (excision) [62], và hệ thống kiểm soát phiên bản Fossil có một khái niệm tương tự gọi là xa lánh (shunning) [63].

Truly deleting data is surprisingly hard [64], since copies can live in many places: for example, storage engines, filesystems, and SSDs often write to a new location rather than overwriting in place [52], and backups are often deliberately immutable to prevent accidental deletion or corruption. Deletion is more a matter of “making it harder to retrieve the data” than actually “making it impossible to retrieve the data.” Nevertheless, you sometimes have to try, as we shall see in “Legislation and self-regulation”.

Việc thực sự xóa dữ liệu khó một cách đáng ngạc nhiên [64], vì các bản sao có thể tồn tại ở nhiều nơi: ví dụ, các engine lưu trữ, hệ thống tệp, và SSD thường ghi vào một vị trí mới thay vì ghi đè tại chỗ [52], và các bản sao lưu thường cố ý là bất biến để ngăn việc xóa hoặc hỏng dữ liệu do vô tình. Việc xóa thiên về “làm cho việc truy xuất dữ liệu khó hơn” hơn là thực sự “làm cho việc truy xuất dữ liệu trở nên bất khả thi”. Tuy nhiên, đôi khi bạn buộc phải thử, như ta sẽ thấy ở “Luật pháp và tự điều chỉnh”.

Processing Streams — Xử lý các luồng

So far in this chapter we have talked about where streams come from (user activity events, sensors, and writes to databases), and we have talked about how streams are transported (through direct messaging, via message brokers, and in event logs).

Cho đến giờ trong chương này ta đã nói về việc các luồng đến từ đâu (các sự kiện hoạt động của người dùng, các cảm biến, và các thao tác ghi vào cơ sở dữ liệu), và ta đã nói về cách các luồng được vận chuyển (qua việc truyền thông điệp trực tiếp, qua các message broker, và trong các nhật ký sự kiện).

What remains is to discuss what you can do with the stream once you have it—namely, you can process it. Broadly, there are three options:

Thứ còn lại là bàn về những gì bạn có thể làm với luồng một khi đã có nó — cụ thể, bạn có thể xử lý nó. Nói rộng ra, có ba lựa chọn:

- You can take the data in the events and write it to a database, cache, search index, or similar storage system, from where it can then be queried by other clients. As shown in Figure 11-5, this is a good way of keeping a database in sync with changes happening in other parts of the system—especially if the stream consumer is the only client writing to the database. Writing to a storage system is the streaming equivalent of what we discussed in “The Output of Batch Workflows”.
- You can push the events to users in some way, for example by sending email alerts or push notifications, or by streaming the events to a real-time dashboard where they are visualized. In this case, a human is the ultimate consumer of the stream.
- You can process one or more input streams to produce one or more output streams. Streams may go through a pipeline consisting of several such processing stages before they eventually end up at an output (option 1 or 2).
 - Bạn có thể lấy dữ liệu trong các sự kiện và ghi nó vào một cơ sở dữ liệu, bộ nhớ đệm, chỉ mục tìm kiếm, hoặc hệ thống lưu trữ tương tự, từ đó nó có thể được truy vấn bởi các client khác. Như đã trình bày ở Hình 11-5, đây là một cách tốt để giữ một cơ sở dữ liệu đồng bộ với các thay đổi đang xảy ra ở các phần khác của hệ thống — đặc biệt nếu bên tiêu thụ luồng là client duy nhất ghi vào cơ sở dữ liệu. Việc ghi vào một hệ thống lưu trữ là phiên bản tương đương của luồng so với điều ta đã bàn ở “Đầu ra của các luồng công việc lô”.
 - Bạn có thể đẩy các sự kiện tới người dùng theo một cách nào đó, ví dụ bằng cách gửi cảnh báo email hoặc thông báo đẩy (push notification), hoặc bằng cách truyền luồng các sự kiện tới một bảng điều khiển thời gian thực nơi chúng được trực quan hóa. Trong trường hợp này, một con người là bên tiêu thụ cuối cùng của luồng.

- Bạn có thể xử lý một hoặc nhiều luồng đầu vào để tạo ra một hoặc nhiều luồng đầu ra. Các luồng có thể đi qua một đường ống gồm vài giai đoạn xử lý như vậy trước khi rốt cuộc kết thúc ở một đầu ra (lựa chọn 1 hoặc 2).

In the rest of this chapter, we will discuss option 3: processing streams to produce other, derived streams. A piece of code that processes streams like this is known as an operator or a job. It is closely related to the Unix processes and MapReduce jobs we discussed in Chapter 10, and the pattern of dataflow is similar: a stream processor consumes input streams in a read-only fashion and writes its output to a different location in an append-only fashion.

Trong phần còn lại của chương này, ta sẽ bàn về lựa chọn 3: xử lý các luồng để tạo ra các luồng dẫn xuất khác. Một đoạn mã xử lý các luồng theo cách này được gọi là một toán tử (operator) hoặc một tác vụ (job). Nó liên hệ mật thiết với các tiến trình Unix và các tác vụ MapReduce mà ta đã bàn ở Chương 10, và mẫu luồng dữ liệu thì tương tự: một bộ xử lý luồng tiêu thụ các luồng đầu vào theo cách chỉ-đọc và ghi đầu ra của nó ra một vị trí khác theo cách chỉ-nối-thêm.

The patterns for partitioning and parallelization in stream processors are also very similar to those in MapReduce and the dataflow engines we saw in Chapter 10, so we won't repeat those topics here. Basic mapping operations such as transforming and filtering records also work the same.

Các mẫu cho việc phân mảnh và song song hóa trong các bộ xử lý luồng cũng rất giống với các mẫu trong MapReduce và các engine luồng dữ liệu mà ta đã thấy ở Chương 10, nên ta sẽ không lặp lại các chủ đề đó ở đây. Các thao tác ánh xạ cơ bản như biến đổi và lọc bản ghi cũng hoạt động giống như vậy.

The one crucial difference to batch jobs is that a stream never ends. This difference has many implications: as discussed at the start of this chapter, sorting does not make sense with an unbounded dataset, and so sort-merge joins (see “Reduce-Side Joins and Grouping”) cannot be used. Fault-tolerance mechanisms must also change: with a batch job that has been running for a few minutes, a failed task can simply be restarted from the beginning, but with a stream job that has been running for several years, restarting from the beginning after a crash may not be a viable option.

Khác biệt then chốt duy nhất so với các tác vụ lô là một luồng không bao giờ kết thúc. Khác biệt này có nhiều hàm ý: như đã bàn ở đầu chương này, việc sắp xếp không có ý nghĩa với một tập dữ liệu không giới hạn, và do đó các phép join trộn-sắp-xếp (sort-merge joins) (xem “Join phía reduce và gom nhóm”) không thể được dùng. Các cơ chế chịu lỗi cũng phải thay đổi: với một tác vụ lô đã chạy được vài phút, một tác vụ thất bại có thể đơn giản được khởi động lại từ đầu, nhưng với một tác vụ luồng đã chạy được vài năm, việc khởi động lại từ đầu sau khi sập có thể không phải là một lựa chọn khả thi.

Uses of Stream Processing — Các ứng dụng của xử lý luồng

Stream processing has long been used for monitoring purposes, where an organization wants to be alerted if certain things happen. For example:

Xử lý luồng từ lâu đã được dùng cho mục đích giám sát, nơi một tổ chức muốn được cảnh báo nếu một số việc nhất định xảy ra. Ví dụ:

- Fraud detection systems need to determine if the usage patterns of a credit card have unexpectedly changed, and block the card if it is likely to have been stolen.
- Trading systems need to examine price changes in a financial market and execute trades according to specified rules.

- Manufacturing systems need to monitor the status of machines in a factory, and quickly identify the problem if there is a malfunction.
- Military and intelligence systems need to track the activities of a potential aggressor, and raise the alarm if there are signs of an attack.
 - Các hệ thống phát hiện gian lận cần xác định xem các mẫu sử dụng của một thẻ tín dụng có thay đổi bất ngờ hay không, và khóa thẻ nếu nó có khả năng đã bị đánh cắp.
 - Các hệ thống giao dịch cần xem xét các thay đổi giá trên một thị trường tài chính và thực hiện các giao dịch theo các quy tắc được quy định.
 - Các hệ thống sản xuất cần giám sát trạng thái của các cỗ máy trong một nhà máy, và nhanh chóng nhận diện vấn đề nếu có một trục trặc.
 - Các hệ thống quân sự và tình báo cần theo dõi các hoạt động của một kẻ xâm lược tiềm năng, và phát báo động nếu có dấu hiệu của một cuộc tấn công.

These kinds of applications require quite sophisticated pattern matching and correlations. However, other uses of stream processing have also emerged over time. In this section we will briefly compare and contrast some of these applications.

Những kiểu ứng dụng này đòi hỏi việc so khớp mẫu và tương quan khá tinh vi. Tuy nhiên, các ứng dụng khác của xử lý luồng cũng đã nổi lên theo thời gian. Trong phần này ta sẽ so sánh và đối chiếu ngắn gọn một số trong các ứng dụng này.

Complex event processing — Xử lý sự kiện phức hợp

Complex event processing (CEP) is an approach developed in the 1990s for analyzing event streams, especially geared toward the kind of application that requires searching for certain event patterns [65, 66]. Similarly to the way that a regular expression allows you to search for certain patterns of characters in a string, CEP allows you to specify rules to search for certain patterns of events in a stream.

Xử lý sự kiện phức hợp (complex event processing - CEP) là một cách tiếp cận được phát triển vào thập niên 1990 để phân tích các luồng sự kiện, đặc biệt hướng tới loại ứng dụng đòi hỏi tìm kiếm các mẫu sự kiện nhất định [65, 66]. Tương tự cách mà một biểu thức chính quy cho phép bạn tìm kiếm các mẫu ký tự nhất định trong một chuỗi, CEP cho phép bạn chỉ định các quy tắc để tìm kiếm các mẫu sự kiện nhất định trong một luồng.

CEP systems often use a high-level **declarative** query language like **SQL**, or a graphical user interface, to describe the patterns of events that should be detected. These queries are submitted to a processing engine that consumes the input streams and internally maintains a state machine that performs the required matching. When a match is found, the engine emits a complex event (hence the name) with the details of the event pattern that was detected [67].

Các hệ thống CEP thường dùng một ngôn ngữ truy vấn khai báo ở mức cao như SQL, hoặc một giao diện người dùng đồ họa, để mô tả các mẫu sự kiện cần được phát hiện. Các truy vấn này được nộp cho một engine xử lý vốn tiêu thụ các luồng đầu vào và duy trì nội bộ một máy trạng thái thực hiện việc so khớp cần thiết. Khi một sự khớp được tìm thấy, engine phát ra một sự kiện phức hợp (do đó có cái tên) với các chi tiết của mẫu sự kiện đã được phát hiện [67].

In these systems, the relationship between queries and data is reversed compared to normal databases. Usually, a database stores data persistently and treats queries as transient: when a query comes in, the database searches for data matching the query, and then forgets about the query when it has finished. CEP engines reverse these roles: queries are stored long-term, and events from

the input streams continuously flow past them in search of a query that matches an event pattern [68].

Trong các hệ thống này, mối quan hệ giữa truy vấn và dữ liệu bị đảo ngược so với các cơ sở dữ liệu thông thường. Thông thường, một cơ sở dữ liệu lưu dữ liệu một cách bền vững và coi các truy vấn là thoáng qua: khi một truy vấn đến, cơ sở dữ liệu tìm kiếm dữ liệu khớp với truy vấn, rồi quên đi truy vấn khi nó đã xong. Các engine CEP đảo ngược các vai trò này: các truy vấn được lưu dài hạn, và các sự kiện từ các luồng đầu vào liên tục trôi qua chúng để đi tìm một truy vấn khớp với một mẫu sự kiện [68].

Implementations of CEP include Esper [69], IBM InfoSphere Streams [70], Apama, TIBCO StreamBase, and SQLstream. Distributed stream processors like Samza are also gaining SQL support for declarative queries on streams [71].

Các hiện thực của CEP bao gồm Esper [69], IBM InfoSphere Streams [70], Apama, TIBCO StreamBase, và SQLstream. Các bộ xử lý luồng phân tán như Samza cũng đang dần có hỗ trợ SQL cho các truy vấn khai báo trên các luồng [71].

Stream analytics — Phân tích luồng

Another area in which stream processing is used is for analytics on streams. The boundary between CEP and stream analytics is blurry, but as a general rule, analytics tends to be less interested in finding specific event sequences and is more oriented toward aggregations and statistical metrics over a large number of events—for example:

Một lĩnh vực khác mà xử lý luồng được dùng là cho việc phân tích trên các luồng. Ranh giới giữa CEP và phân tích luồng thì mờ nhạt, nhưng như một quy tắc chung, phân tích có xu hướng ít quan tâm tới việc tìm các chuỗi sự kiện cụ thể và thiên về các phép tổng hợp và các số liệu thống kê trên một số lượng lớn các sự kiện — ví dụ:

- Measuring the rate of some type of event (how often it occurs per time interval)
- Calculating the rolling **average** of a value over some time period
- Comparing current statistics to previous time intervals (e.g., to detect trends or to alert on metrics that are unusually high or low compared to the same time last week)
 - Đo tốc độ của một loại sự kiện nào đó (nó xảy ra bao nhiêu lần trên một khoảng thời gian)
 - Tính trung bình trượt của một giá trị trên một khoảng thời gian nào đó
 - So sánh các số liệu thống kê hiện tại với các khoảng thời gian trước đó (ví dụ, để phát hiện xu hướng hoặc để cảnh báo về các số liệu cao hoặc thấp bất thường so với cùng thời điểm tuần trước)

Such statistics are usually computed over fixed time intervals—for example, you might want to know the average number of queries per second to a service over the last 5 minutes, and their 99th **percentile** response time during that period. Averaging over a few minutes smoothes out irrelevant fluctuations from one second to the next, while still giving you a timely picture of any changes in traffic pattern. The time interval over which you aggregate is known as a window, and we will look into windowing in more detail in “Reasoning About Time”.

Các số liệu thống kê như vậy thường được tính trên các khoảng thời gian cố định — ví dụ, bạn có thể muốn biết số truy vấn trung bình mỗi giây tới một dịch vụ trong 5 phút vừa qua, và thời gian phản hồi phân vị thứ 99 của chúng trong khoảng thời gian đó. Việc lấy trung bình trên vài phút làm trơn đi các dao động không liên quan từ giây này sang giây khác, mà vẫn cho bạn một bức tranh kịp thời về bất kỳ thay đổi nào trong mẫu lưu lượng.

Khoảng thời gian mà bạn tổng hợp trên đó được gọi là một cửa sổ (window), và ta sẽ xem xét việc cửa sổ hóa kỹ hơn ở “Lập luận về thời gian”.

Stream analytics systems sometimes use probabilistic algorithms, such as Bloom filters (which we encountered in “Performance optimizations”) for set membership, HyperLogLog [72] for cardinality estimation, and various percentile estimation algorithms (see “Percentiles in Practice”). Probabilistic algorithms produce approximate results, but have the advantage of requiring significantly less memory in the stream processor than exact algorithms. This use of approximation algorithms sometimes leads people to believe that stream processing systems are always lossy and inexact, but that is wrong: there is nothing inherently approximate about stream processing, and probabilistic algorithms are merely an optimization [73].

Các hệ thống phân tích luồng đôi khi dùng các thuật toán xác suất, chẳng hạn các bộ lọc Bloom (mà ta đã gặp ở “Các tối ưu hiệu năng”) để kiểm tra tư cách thành viên tập hợp, HyperLogLog [72] để ước lượng lực lượng, và nhiều thuật toán ước lượng phân vị khác nhau (xem “Phân vị trong thực tế”). Các thuật toán xác suất tạo ra kết quả gần đúng, nhưng có lợi thế là đòi hỏi ít bộ nhớ hơn đáng kể trong bộ xử lý luồng so với các thuật toán chính xác. Việc dùng các thuật toán gần đúng này đôi khi khiến người ta tin rằng các hệ thống xử lý luồng luôn mất mát và không chính xác, nhưng điều đó là sai: không có gì vốn dĩ gần đúng về xử lý luồng, và các thuật toán xác suất chỉ đơn thuần là một sự tối ưu [73].

Many open source distributed stream processing frameworks are designed with analytics in mind: for example, Apache Storm, Spark Streaming, Flink, Concord, Samza, and Kafka Streams [74]. Hosted services include Google Cloud Dataflow and Azure Stream Analytics.

Nhiều framework xử lý luồng phân tán mã nguồn mở được thiết kế với phân tích trong đầu: ví dụ, Apache Storm, Spark Streaming, Flink, Concord, Samza, và Kafka Streams [74]. Các dịch vụ được lưu trữ (hosted) bao gồm Google Cloud Dataflow và Azure Stream Analytics.

Maintaining materialized views — Duy trì các khung nhìn cụ thể hóa

We saw in “Databases and Streams” that a stream of changes to a database can be used to keep derived data systems, such as caches, search indexes, and data warehouses, up to date with a source database. We can regard these examples as specific cases of maintaining materialized views (see “Aggregation: Data Cubes and Materialized Views”): deriving an alternative view onto some dataset so that you can query it efficiently, and updating that view whenever the underlying data changes [50].

Ta đã thấy ở “Cơ sở dữ liệu và luồng” rằng một luồng các thay đổi đối với một cơ sở dữ liệu có thể được dùng để giữ các hệ thống dữ liệu dẫn xuất, chẳng hạn các bộ nhớ đệm, chỉ mục tìm kiếm, và kho dữ liệu, được cập nhật theo một cơ sở dữ liệu nguồn. Ta có thể coi những ví dụ này là các trường hợp cụ thể của việc duy trì các khung nhìn cụ thể hóa (materialized views) (xem “Tổng hợp: Khối dữ liệu và khung nhìn cụ thể hóa”): dẫn xuất một khung nhìn thay thế lên một tập dữ liệu nào đó để bạn có thể truy vấn nó một cách hiệu quả, và cập nhật khung nhìn đó mỗi khi dữ liệu nền thay đổi [50].

Similarly, in event sourcing, application state is maintained by applying a log of events; here the application state is also a kind of materialized view. Unlike stream analytics scenarios, it is usually not sufficient to consider only events within some time window: building the materialized view potentially requires all events over an arbitrary time period, apart from any obsolete events that may be discarded by log compaction (see “Log compaction”). In effect, you need a window that stretches all the way back to the beginning of time.

Tương tự, trong việc lấy nguồn từ sự kiện, trạng thái ứng dụng được duy trì bằng cách áp dụng một nhật ký các sự kiện; ở đây trạng thái ứng dụng cũng là một kiểu khung nhìn cụ thể hóa. Khác với các kịch bản phân tích luồng, thường là không đủ nếu chỉ xét các sự kiện trong một cửa sổ thời gian nào đó: việc xây dựng khung nhìn cụ thể hóa có khả năng đòi hỏi tất cả các sự kiện trên một khoảng thời gian tùy ý, ngoại trừ bất kỳ sự kiện lỗi thời nào có thể bị vứt bỏ bởi việc nén nhật ký (xem “Nén nhật ký”). Thực chất, bạn cần một cửa sổ trải dài tận về tới khởi đầu của thời gian.

In principle, any stream processor could be used for materialized view maintenance, although the need to maintain events forever runs counter to the assumptions of some analytics-oriented frameworks that mostly operate on windows of a limited duration. Samza and Kafka Streams support this kind of usage, building upon Kafka’s support for log compaction [75].

Về nguyên tắc, bất kỳ bộ xử lý luồng nào cũng có thể được dùng để duy trì khung nhìn cụ thể hóa, mặc dù nhu cầu duy trì các sự kiện mãi mãi đi ngược lại các giả định của một số framework thiên về phân tích vốn chủ yếu hoạt động trên các cửa sổ có thời lượng hạn chế. Samza và Kafka Streams hỗ trợ kiểu sử dụng này, xây dựng trên sự hỗ trợ của Kafka dành cho việc nén nhật ký [75].

Search on streams — Tìm kiếm trên các luồng

Besides CEP, which allows searching for patterns consisting of multiple events, there is also sometimes a need to search for individual events based on complex criteria, such as **full-text search** queries.

Bên cạnh CEP, vốn cho phép tìm kiếm các mẫu bao gồm nhiều sự kiện, đôi khi cũng có nhu cầu tìm kiếm các sự kiện riêng lẻ dựa trên các tiêu chí phức tạp, chẳng hạn các truy vấn tìm kiếm toàn văn.

For example, media monitoring services subscribe to feeds of news articles and broadcasts from media outlets, and search for any news mentioning companies, products, or topics of interest. This is done by formulating a search query in advance, and then continually matching the stream of news items against this query. Similar features exist on some websites: for example, users of real estate websites can ask to be notified when a new property matching their search criteria appears on the market. The percolator feature of Elasticsearch [76] is one option for implementing this kind of stream search.

Ví dụ, các dịch vụ giám sát truyền thông đăng ký các nguồn cấp các bài báo và các bản tin phát sóng từ các hãng truyền thông, và tìm kiếm bất kỳ tin tức nào nhắc đến các công ty, sản phẩm, hoặc chủ đề mà họ quan tâm. Việc này được thực hiện bằng cách xây dựng một truy vấn tìm kiếm trước, rồi liên tục so khớp luồng các mục tin tức với truy vấn này. Các tính năng tương tự tồn tại trên một số website: ví dụ, người dùng các website bất động sản có thể yêu cầu được thông báo khi một bất động sản mới khớp với các tiêu chí tìm kiếm của họ xuất hiện trên thị trường. Tính năng percolator của Elasticsearch [76] là một lựa chọn để hiện thực kiểu tìm kiếm luồng này.

Conventional search engines first index the documents and then run queries over the index. By contrast, searching a stream turns the processing on its head: the queries are stored, and the documents run past the queries, like in CEP. In the simplest case, you can test every document against every query, although this can get slow if you have a large number of queries. To optimize the process, it is possible to index the queries as well as the documents, and thus narrow down the set of queries that may match [77].

Các engine tìm kiếm thông thường trước tiên lập chỉ mục các tài liệu rồi chạy các truy vấn trên chỉ mục. Ngược lại, việc tìm kiếm một luồng lật ngược việc xử lý: các truy vấn được lưu, và các tài liệu trôi qua các truy vấn, giống như trong CEP. Trong trường hợp đơn giản nhất, bạn có thể kiểm tra mọi tài liệu với mọi truy vấn, mặc dù việc này có thể trở nên chậm nếu bạn có một số lượng lớn truy vấn. Để tối ưu tiến trình, có thể lập chỉ mục cho cả các truy vấn lẫn các tài liệu, và do đó thu hẹp tập các truy vấn có thể khớp [77].

Message passing and RPC — Truyền thông điệp và RPC

In “Message-Passing Dataflow” we discussed message-passing systems as an alternative to RPC—i.e., as a mechanism for services to communicate, as used for example in the actor model. Although these systems are also based on messages and events, we normally don’t think of them as stream processors:

Ở “Luồng dữ liệu truyền thông điệp” ta đã bàn về các hệ thống truyền thông điệp như một lựa chọn thay thế cho RPC — tức là một cơ chế để các dịch vụ giao tiếp, như được dùng chẳng hạn trong mô hình actor. Mặc dù các hệ thống này cũng dựa trên các thông điệp và sự kiện, ta thường không nghĩ về chúng như các bộ xử lý luồng:

- Actor frameworks are primarily a mechanism for managing concurrency and distributed execution of communicating modules, whereas stream processing is primarily a data management technique.
- Communication between actors is often ephemeral and one-to-one, whereas event logs are durable and multi-subscriber.
- Actors can communicate in arbitrary ways (including cyclic request/response patterns), but stream processors are usually set up in acyclic pipelines where every stream is the output of one particular job, and derived from a well-defined set of input streams.
 - Các framework actor chủ yếu là một cơ chế để quản lý tương tranh và thực thi phân tán của các mô-đun giao tiếp với nhau, trong khi xử lý luồng chủ yếu là một kỹ thuật quản lý dữ liệu.
 - Giao tiếp giữa các actor thường là phù du và một-đối-một, trong khi các nhật ký sự kiện là bền vững và đa-người-đăng-ký.
 - Các actor có thể giao tiếp theo những cách tùy ý (bao gồm các mẫu yêu cầu/phản hồi có chu trình), nhưng các bộ xử lý luồng thường được thiết lập trong các đường ống không có chu trình nơi mỗi luồng là đầu ra của một tác vụ cụ thể nào đó, và được dẫn xuất từ một tập các luồng đầu vào được định nghĩa rõ ràng.

That said, there is some crossover area between RPC-like systems and stream processing. For example, Apache Storm has a feature called distributed RPC, which allows user queries to be farmed out to a set of nodes that also process event streams; these queries are then interleaved with events from the input streams, and results can be aggregated and sent back to the user [78]. (See also “Multi-partition data processing”.)

Dù vậy, có một số vùng giao thoa giữa các hệ thống kiểu RPC và xử lý luồng. Ví dụ, Apache Storm có một tính năng gọi là RPC phân tán, cho phép các truy vấn người dùng được phân bổ ra một tập các nút vốn cũng xử lý các luồng sự kiện; các truy vấn này sau đó được xen kẽ với các sự kiện từ các luồng đầu vào, và kết quả có thể được tổng hợp lại và gửi về cho người dùng [78]. (Xem thêm “Xử lý dữ liệu đa-mảnh”.)

It is also possible to process streams using actor frameworks. However, many such frameworks do not guarantee message delivery in the case of crashes, so the processing is not fault-tolerant unless you implement additional retry logic.

Cũng có thể xử lý các luồng bằng các framework actor. Tuy nhiên, nhiều framework như vậy không bảo đảm việc phân phát thông điệp trong trường hợp sập, nên việc xử lý không chịu lỗi trừ khi bạn hiện thực thêm logic thử lại.

Reasoning About Time — Lập luận về thời gian

Stream processors often need to deal with time, especially when used for analytics purposes, which frequently use time windows such as “the average over the last five minutes.” It might seem that the meaning of “the last five minutes” should be unambiguous and clear, but unfortunately the notion is surprisingly tricky.

Các bộ xử lý luồng thường cần đối phó với thời gian, đặc biệt khi được dùng cho mục đích phân tích, vốn thường dùng các cửa sổ thời gian như “trung bình trong năm phút vừa qua.” Có vẻ như ý nghĩa của “năm phút vừa qua” lẽ ra phải rõ ràng và không mơ hồ, nhưng không may khái niệm này lại hóc búa một cách đáng ngạc nhiên.

In a batch process, the processing tasks rapidly crunch through a large collection of historical events. If some kind of breakdown by time needs to happen, the batch process needs to look at the timestamp embedded in each event. There is no point in looking at the system clock of the machine running the batch process, because the time at which the process is run has nothing to do with the time at which the events actually occurred.

Trong một tiến trình lô, các tác vụ xử lý nghiền nhanh qua một tập lớn các sự kiện lịch sử. Nếu cần một kiểu phân tách nào đó theo thời gian, tiến trình lô cần nhìn vào dấu thời gian được nhúng trong mỗi sự kiện. Không có ích gì khi nhìn vào đồng hồ hệ thống của máy đang chạy tiến trình lô, vì thời điểm mà tiến trình được chạy chẳng liên quan gì tới thời điểm mà các sự kiện thực sự xảy ra.

A batch process may read a year’s worth of historical events within a few minutes; in most cases, the timeline of interest is the year of history, not the few minutes of processing. Moreover, using the timestamps in the events allows the processing to be deterministic: running the same process again on the same input yields the same result (see “Fault tolerance”).

Một tiến trình lô có thể đọc lượng sự kiện lịch sử của một năm trong vòng vài phút; trong hầu hết các trường hợp, dòng thời gian đáng quan tâm là một năm lịch sử, chứ không phải vài phút xử lý. Hơn nữa, việc dùng các dấu thời gian trong các sự kiện cho phép việc xử lý mang tính tất định: chạy lại cùng một tiến trình trên cùng một đầu vào cho ra cùng một kết quả (xem “Khả năng chịu lỗi”).

On the other hand, many stream processing frameworks use the local system clock on the processing machine (the processing time) to determine windowing [79]. This approach has the advantage of being simple, and it is reasonable if the delay between event creation and event processing is negligibly short. However, it breaks down if there is any significant processing lag—i.e., if the processing may happen noticeably later than the time at which the event actually occurred.

Mặt khác, nhiều framework xử lý luồng dùng đồng hồ hệ thống cục bộ trên máy xử lý (thời gian xử lý - processing time) để xác định việc cửa sổ hóa [79]. Cách tiếp cận này có lợi thế là đơn giản, và nó hợp lý nếu độ trễ giữa việc tạo sự kiện và việc xử lý sự kiện ngắn tới mức không đáng kể. Tuy nhiên, nó sụp đổ nếu có bất kỳ độ trễ xử lý đáng kể nào — tức là nếu việc xử lý có thể xảy ra muộn hơn đáng kể so với thời điểm mà sự kiện thực sự xảy ra.

Event time versus processing time — Thời gian sự kiện so với thời gian xử lý

There are many reasons why processing may be delayed: queueing, network faults (see “Unreliable Networks”), a performance issue leading to contention in the message broker or processor, a restart of the stream consumer, or reprocessing of past events (see “Replaying old messages”) while recovering from a fault or after fixing a **bug** in the code.

Có nhiều lý do khiến việc xử lý có thể bị trì hoãn: xếp hàng, các trục trặc mạng (xem “Mạng không đáng tin cậy”), một vấn đề về hiệu năng dẫn tới tranh chấp trong message broker hoặc bộ xử lý, một lần khởi động lại bên tiêu thụ luồng, hoặc việc xử lý lại các sự kiện trong quá khứ (xem “Phát lại các thông điệp cũ”) trong khi phục hồi từ một trục trặc hoặc sau khi sửa một bug trong mã.

Moreover, message delays can also lead to unpredictable ordering of messages. For example, say a user first makes one web request (which is handled by web server A), and then a second request (which is handled by server B). A and B emit events describing the requests they handled, but B’s event reaches the message broker before A’s event does. Now stream processors will first see the B event and then the A event, even though they actually occurred in the opposite order.

Hơn nữa, độ trễ thông điệp cũng có thể dẫn tới việc các thông điệp được sắp thứ tự một cách khó lường. Ví dụ, giả sử một người dùng trước tiên thực hiện một yêu cầu web (được xử lý bởi máy chủ web A), rồi một yêu cầu thứ hai (được xử lý bởi máy chủ B). A và B phát ra các sự kiện mô tả các yêu cầu chúng đã xử lý, nhưng sự kiện của B tới message broker trước sự kiện của A. Giờ các bộ xử lý luồng sẽ thấy sự kiện B trước rồi đến sự kiện A, mặc dù chúng thực sự đã xảy ra theo thứ tự ngược lại.

If it helps to have an analogy, consider the Star Wars movies: Episode IV was released in 1977, Episode V in 1980, and Episode VI in 1983, followed by Episodes I, II, and III in 1999, 2002, and 2005, respectively, and Episode VII in 2015 [80]. If you watched the movies in the order they came out, the order in which you processed the movies is inconsistent with the order of their narrative. (The episode number is like the event timestamp, and the date when you watched the movie is the processing time.) As humans, we are able to cope with such discontinuities, but stream processing algorithms need to be specifically written to accommodate such timing and ordering issues.

Nếu một phép loại suy có ích, hãy xét các bộ phim Star Wars: Tập IV được phát hành vào năm 1977, Tập V vào năm 1980, và Tập VI vào năm 1983, tiếp theo là các Tập I, II, và III lần lượt vào các năm 1999, 2002, và 2005, rồi Tập VII vào năm 2015 [80]. Nếu bạn xem các bộ phim theo thứ tự chúng ra mắt, thì thứ tự bạn xử lý các bộ phim không nhất quán với thứ tự của câu chuyện trong phim. (Số tập giống như dấu thời gian sự kiện, và ngày mà bạn xem bộ phim là thời gian xử lý.) Là con người, ta có thể xoay sở với những gián đoạn như vậy, nhưng các thuật toán xử lý luồng cần được viết một cách đặc biệt để dung nạp những vấn đề về thời điểm và thứ tự như vậy.

Confusing event time and processing time leads to bad data. For example, say you have a stream processor that measures the rate of requests (counting the number of requests per second). If you redeploy the stream processor, it may be shut down for a minute and process the backlog of events when it comes back up. If you measure the rate based on the processing time, it will look as if there was a sudden anomalous spike of requests while processing the backlog, when in fact the real rate of requests was steady (Figure 11-7).

Việc nhầm lẫn thời gian sự kiện và thời gian xử lý dẫn tới dữ liệu xấu. Ví dụ, giả sử bạn có một bộ xử lý luồng đo tốc độ các yêu cầu (đếm số yêu cầu mỗi giây). Nếu bạn triển khai lại bộ xử lý luồng, nó có thể bị tắt trong một phút và xử lý lượng sự kiện tồn đọng khi nó hoạt động trở lại. Nếu bạn đo tốc độ dựa trên thời gian xử lý, sẽ trông như thể có một đợt tăng

vợt bất thường đột ngột của các yêu cầu trong khi xử lý lượng tồn đọng, trong khi thực ra tốc độ yêu cầu thực sự là ổn định (Hình 11-7).

Figure 11-7. Windowing by processing time introduces artifacts due to variations in processing rate. — Hình 11-7. Việc cửa sổ hóa theo thời gian xử lý tạo ra các sai lệch giả tạo do các biến thiên trong tốc độ xử lý.

Knowing when you're ready — Biết khi nào bạn đã sẵn sàng

A tricky problem when defining windows in terms of event time is that you can never be sure when you have received all of the events for a particular window, or whether there are some events still to come.

Một vấn đề hóc búa khi định nghĩa các cửa sổ theo thời gian sự kiện là bạn không bao giờ có thể chắc chắn khi nào bạn đã nhận được tất cả các sự kiện cho một cửa sổ cụ thể, hay liệu có còn một số sự kiện sắp đến nữa hay không.

For example, say you're grouping events into one-minute windows so that you can count the number of requests per minute. You have counted some number of events with timestamps that fall in the 37th minute of the hour, and time has moved on; now most of the incoming events fall within the 38th and 39th minutes of the hour. When do you declare that you have finished the window for the 37th minute, and output its counter value?

Ví dụ, giả sử bạn đang gom các sự kiện thành các cửa sổ một phút để có thể đếm số yêu cầu mỗi phút. Bạn đã đếm được một số lượng sự kiện nào đó với các dấu thời gian rơi vào phút thứ 37 của giờ, và thời gian đã trôi đi; giờ hầu hết các sự kiện đến đều rơi vào phút thứ 38 và 39 của giờ. Khi nào thì bạn tuyên bố rằng bạn đã hoàn tất cửa sổ cho phút thứ 37, và xuất ra giá trị bộ đếm của nó?

You can time out and declare a window ready after you have not seen any new events for a while, but it could still happen that some events were buffered on another machine somewhere, delayed due to a network interruption. You need to be able to handle such straggler events that arrive after the window has already been declared complete. Broadly, you have two options [1]:

Bạn có thể hết thời gian chờ và tuyên bố một cửa sổ là sẵn sàng sau khi bạn không thấy bất kỳ sự kiện mới nào trong một lúc, nhưng vẫn có thể xảy ra việc một số sự kiện đã được đệm ở một máy nào đó, bị trì hoãn do một gián đoạn mạng. Bạn cần có khả năng xử lý các sự kiện lạc đàn (straggler) như vậy đến sau khi cửa sổ đã được tuyên bố là hoàn tất. Nói rộng ra, bạn có hai lựa chọn [1]:

- Ignore the straggler events, as they are probably a small percentage of events in normal circumstances. You can track the number of dropped events as a metric, and alert if you start dropping a significant amount of data.
- Publish a correction, an updated value for the window with stragglers included. You may also need to retract the previous output.
 - Bỏ qua các sự kiện lạc đàn, vì chúng có lẽ là một tỷ lệ nhỏ các sự kiện trong các tình huống bình thường. Bạn có thể theo dõi số sự kiện bị loại bỏ như một số liệu, và cảnh báo nếu bạn bắt đầu loại bỏ một lượng dữ liệu đáng kể.
 - Công bố một bản hiệu chỉnh, một giá trị cập nhật cho cửa sổ với các sự kiện lạc đàn đã được tính vào. Bạn cũng có thể cần thu hồi lại đầu ra trước đó.

In some cases it is possible to use a special message to indicate, "From now on there will be no more messages with a timestamp earlier than t," which can be used by consumers to trigger windows [81].

However, if several producers on different machines are generating events, each with their own minimum timestamp thresholds, the consumers need to keep track of each producer individually. Adding and removing producers is trickier in this case.

Trong một số trường hợp, có thể dùng một thông điệp đặc biệt để cho biết, “Từ giờ trở đi sẽ không còn thông điệp nào với dấu thời gian sớm hơn t ,” điều mà các bên tiêu thụ có thể dùng để kích hoạt các cửa sổ [81]. Tuy nhiên, nếu nhiều nhà sản xuất trên các máy khác nhau đang sinh ra các sự kiện, mỗi nhà sản xuất với ngưỡng dấu thời gian tối thiểu riêng của nó, thì các bên tiêu thụ cần theo dõi từng nhà sản xuất một cách riêng lẻ. Việc thêm và bớt các nhà sản xuất khó hơn trong trường hợp này.

Whose clock are you using, anyway? — Mà này, bạn đang dùng đồng hồ của ai?

Assigning timestamps to events is even more difficult when events can be buffered at several points in the system. For example, consider a mobile app that reports events for usage metrics to a server. The app may be used while the device is offline, in which case it will buffer events locally on the device and send them to a server when an internet connection is next available (which may be hours or even days later). To any consumers of this stream, the events will appear as extremely delayed stragglers.

Việc gán dấu thời gian cho các sự kiện thậm chí còn khó hơn khi các sự kiện có thể bị đệm ở vài điểm trong hệ thống. Ví dụ, hãy xét một ứng dụng di động báo cáo các sự kiện cho các số liệu sử dụng tới một máy chủ. Ứng dụng có thể được dùng trong khi thiết bị ngoại tuyến, trong trường hợp đó nó sẽ đệm các sự kiện cục bộ trên thiết bị và gửi chúng tới một máy chủ khi có kết nối internet vào lần tới (điều này có thể là hàng giờ hoặc thậm chí hàng ngày sau đó). Đối với bất kỳ bên tiêu thụ nào của luồng này, các sự kiện sẽ trông như những kẻ lạc đàn bị trì hoãn cực kỳ lâu.

In this context, the timestamp on the events should really be the time at which the user interaction occurred, according to the mobile device’s local clock. However, the clock on a user-controlled device often cannot be trusted, as it may be accidentally or deliberately set to the wrong time (see “Clock Synchronization and Accuracy”). The time at which the event was received by the server (according to the server’s clock) is more likely to be accurate, since the server is under your control, but less meaningful in terms of describing the user interaction.

Trong ngữ cảnh này, dấu thời gian trên các sự kiện thực sự nên là thời điểm mà tương tác của người dùng đã xảy ra, theo đồng hồ cục bộ của thiết bị di động. Tuy nhiên, đồng hồ trên một thiết bị do người dùng kiểm soát thường không thể tin cậy được, vì nó có thể bị đặt sai giờ một cách vô tình hoặc cố ý (xem “Đồng bộ đồng hồ và độ chính xác”). Thời điểm mà sự kiện được máy chủ nhận (theo đồng hồ của máy chủ) có khả năng chính xác hơn, vì máy chủ nằm dưới sự kiểm soát của bạn, nhưng lại ít ý nghĩa hơn về mặt mô tả tương tác của người dùng.

To adjust for incorrect device clocks, one approach is to log three timestamps [82]:

Để điều chỉnh cho các đồng hồ thiết bị sai, một cách tiếp cận là ghi nhật ký ba dấu thời gian [82]:

- The time at which the event occurred, according to the device clock
- The time at which the event was sent to the server, according to the device clock
- The time at which the event was received by the server, according to the server clock
 - Thời điểm mà sự kiện xảy ra, theo đồng hồ thiết bị

- Thời điểm mà sự kiện được gửi tới máy chủ, theo đồng hồ thiết bị
- Thời điểm mà sự kiện được máy chủ nhận, theo đồng hồ máy chủ

By subtracting the second timestamp from the third, you can estimate the offset between the device clock and the server clock (assuming the network delay is negligible compared to the required timestamp accuracy). You can then apply that offset to the event timestamp, and thus estimate the true time at which the event actually occurred (assuming the device clock offset did not change between the time the event occurred and the time it was sent to the server).

Bằng cách lấy dấu thời gian thứ ba trừ đi dấu thời gian thứ hai, bạn có thể ước lượng độ lệch giữa đồng hồ thiết bị và đồng hồ máy chủ (giả định độ trễ mạng là không đáng kể so với độ chính xác dấu thời gian cần có). Sau đó bạn có thể áp dụng độ lệch đó vào dấu thời gian sự kiện, và do đó ước lượng được thời điểm thực sự mà sự kiện đã thực sự xảy ra (giả định độ lệch đồng hồ thiết bị không thay đổi giữa thời điểm sự kiện xảy ra và thời điểm nó được gửi tới máy chủ).

This problem is not unique to stream processing—batch processing suffers from exactly the same issues of reasoning about time. It is just more noticeable in a streaming context, where we are more aware of the passage of time.

Vấn đề này không riêng có đối với xử lý luồng — xử lý lô chịu đựng đúng những vấn đề tương tự về việc lập luận về thời gian. Nó chỉ dễ nhận thấy hơn trong một ngữ cảnh luồng, nơi ta ý thức hơn về sự trôi đi của thời gian.

Types of windows — Các loại cửa sổ

Once you know how the timestamp of an event should be determined, the next step is to decide how windows over time periods should be defined. The window can then be used for aggregations, for example to count events, or to calculate the average of values within the window. Several types of windows are in common use [79, 83]:

Một khi bạn biết dấu thời gian của một sự kiện nên được xác định ra sao, bước tiếp theo là quyết định các cửa sổ trên các khoảng thời gian nên được định nghĩa như thế nào. Cửa sổ khi đó có thể được dùng cho các phép tổng hợp, ví dụ để đếm các sự kiện, hoặc để tính trung bình của các giá trị trong cửa sổ. Một số loại cửa sổ được dùng phổ biến [79, 83]:

A tumbling window has a fixed length, and every event belongs to exactly one window. For example, if you have a 1-minute tumbling window, all the events with timestamps between 10:03:00 and 10:03:59 are grouped into one window, events between 10:04:00 and 10:04:59 into the next window, and so on. You could implement a 1-minute tumbling window by taking each event timestamp and rounding it down to the nearest minute to determine the window that it belongs to.

Một cửa sổ lăn (tumbling window) có một độ dài cố định, và mỗi sự kiện thuộc về đúng một cửa sổ. Ví dụ, nếu bạn có một cửa sổ lăn 1 phút, tất cả các sự kiện với dấu thời gian giữa 10:03:00 và 10:03:59 được gom vào một cửa sổ, các sự kiện giữa 10:04:00 và 10:04:59 vào cửa sổ tiếp theo, và cứ thế. Bạn có thể hiện thực một cửa sổ lăn 1 phút bằng cách lấy mỗi dấu thời gian sự kiện và làm tròn xuống tới phút gần nhất để xác định cửa sổ mà nó thuộc về.

A hopping window also has a fixed length, but allows windows to overlap in order to provide some smoothing. For example, a 5-minute window with a hop size of 1 minute would contain the events between 10:03:00 and 10:07:59, then the next window would cover events between 10:04:00 and 10:08:59, and so on. You can implement this hopping window by first calculating 1-minute tumbling windows, and then aggregating over several adjacent windows.

Một cửa sổ nhảy (hopping window) cũng có một độ dài cố định, nhưng cho phép các cửa sổ chồng lẫn nhau nhằm cung cấp một chút làm trơn. Ví dụ, một cửa sổ 5 phút với kích thước bước nhảy 1 phút sẽ chứa các sự kiện giữa 10:03:00 và 10:07:59, rồi cửa sổ tiếp theo sẽ bao phủ các sự kiện giữa 10:04:00 và 10:08:59, và cứ thế. Bạn có thể hiện thực cửa sổ nhảy này bằng cách trước tiên tính các cửa sổ lần 1 phút, rồi tổng hợp trên vài cửa sổ kề nhau.

A sliding window contains all the events that occur within some interval of each other. For example, a 5-minute sliding window would cover events at 10:03:39 and 10:08:12, because they are less than 5 minutes apart (note that tumbling and hopping 5-minute windows would not have put these two events in the same window, as they use fixed boundaries). A sliding window can be implemented by keeping a buffer of events sorted by time and removing old events when they expire from the window.

Một cửa sổ trượt (sliding window) chứa tất cả các sự kiện xảy ra cách nhau trong một khoảng nào đó. Ví dụ, một cửa sổ trượt 5 phút sẽ bao phủ các sự kiện tại 10:03:39 và 10:08:12, vì chúng cách nhau ít hơn 5 phút (lưu ý rằng các cửa sổ lăn và nhảy 5 phút sẽ không đặt hai sự kiện này vào cùng một cửa sổ, vì chúng dùng các ranh giới cố định). Một cửa sổ trượt có thể được hiện thực bằng cách giữ một bộ đệm các sự kiện được sắp xếp theo thời gian và loại bỏ các sự kiện cũ khi chúng hết hạn ra khỏi cửa sổ.

Unlike the other window types, a session window has no fixed duration. Instead, it is defined by grouping together all events for the same user that occur closely together in time, and the window ends when the user has been inactive for some time (for example, if there have been no events for 30 minutes). Sessionization is a common requirement for website analytics (see “GROUP BY”).

Khác với các loại cửa sổ kia, một cửa sổ phiên (session window) không có thời lượng cố định. Thay vào đó, nó được định nghĩa bằng cách gom lại với nhau tất cả các sự kiện của cùng một người dùng xảy ra gần nhau về thời gian, và cửa sổ kết thúc khi người dùng đã không hoạt động trong một khoảng thời gian nào đó (ví dụ, nếu không có sự kiện nào trong 30 phút). Việc phân phiên (sessionization) là một yêu cầu phổ biến cho phân tích website (xem “GROUP BY”).

Stream Joins — Các phép join luồng

In Chapter 10 we discussed how batch jobs can **join** datasets by key, and how such joins form an important part of data pipelines. Since stream processing generalizes data pipelines to incremental processing of unbounded datasets, there is exactly the same need for joins on streams.

Ở Chương 10 ta đã bàn về cách các tác vụ lô có thể join các tập dữ liệu theo khóa, và cách các phép join như vậy tạo nên một phần quan trọng của các đường ống dữ liệu. Vì xử lý luồng tổng quát hóa các đường ống dữ liệu thành việc xử lý tăng dần các tập dữ liệu không giới hạn, nên có đúng cùng nhu cầu về các phép join trên các luồng.

However, the fact that new events can appear anytime on a stream makes joins on streams more challenging than in batch jobs. To understand the situation better, let’s distinguish three different types of joins: stream-stream joins, stream-table joins, and table-table joins [84]. In the following sections we’ll illustrate each by example.

Tuy nhiên, sự thật rằng các sự kiện mới có thể xuất hiện bất cứ lúc nào trên một luồng khiến các phép join trên các luồng thách thức hơn so với trong các tác vụ lô. Để hiểu tình huống rõ hơn, hãy phân biệt ba loại join khác nhau: join luồng-luồng (stream-stream), join

luồng-bảng (stream-table), và join bảng-bảng (table-table) [84]. Trong các phần sau ta sẽ minh họa từng loại bảng ví dụ.

Stream-stream join (window join) — Join luồng-luồng (join theo cửa sổ)

Say you have a search feature on your website, and you want to detect recent trends in searched-for URLs. Every time someone types a search query, you log an event containing the query and the results returned. Every time someone clicks one of the search results, you log another event recording the click. In order to calculate the click-through rate for each URL in the search results, you need to bring together the events for the search action and the click action, which are connected by having the same session ID. Similar analyses are needed in advertising systems [85].

Giả sử bạn có một tính năng tìm kiếm trên website của mình, và bạn muốn phát hiện các xu hướng gần đây về các URL được tìm kiếm. Mỗi khi ai đó gõ một truy vấn tìm kiếm, bạn ghi một sự kiện chứa truy vấn và các kết quả được trả về. Mỗi khi ai đó nhấp vào một trong các kết quả tìm kiếm, bạn ghi một sự kiện khác ghi lại cú nhấp. Để tính tỷ lệ nhấp-quá (click-through rate) cho mỗi URL trong các kết quả tìm kiếm, bạn cần đưa lại với nhau các sự kiện cho hành động tìm kiếm và hành động nhấp, vốn được kết nối bởi việc có cùng một ID phiên. Các phân tích tương tự cũng cần thiết trong các hệ thống quảng cáo [85].

The click may never come if the user abandons their search, and even if it comes, the time between the search and the click may be highly variable: in many cases it might be a few seconds, but it could be as long as days or weeks (if a user runs a search, forgets about that browser tab, and then returns to the tab and clicks a result sometime later). Due to variable network delays, the click event may even arrive before the search event. You can choose a suitable window for the join—for example, you may choose to join a click with a search if they occur at most one hour apart.

Cú nhấp có thể không bao giờ đến nếu người dùng từ bỏ việc tìm kiếm của họ, và ngay cả khi nó đến, thời gian giữa việc tìm kiếm và cú nhấp có thể biến thiên rất lớn: trong nhiều trường hợp nó có thể là vài giây, nhưng cũng có thể lâu tới hàng ngày hoặc hàng tuần (nếu một người dùng chạy một tìm kiếm, quên đi cái tab trình duyệt đó, rồi quay lại tab và nhấp vào một kết quả vào lúc nào đó sau này). Do độ trễ mạng biến thiên, sự kiện nhấp thậm chí có thể đến trước sự kiện tìm kiếm. Bạn có thể chọn một cửa sổ phù hợp cho phép join — ví dụ, bạn có thể chọn join một cú nhấp với một tìm kiếm nếu chúng xảy ra cách nhau nhiều nhất một giờ.

Note that embedding the details of the search in the click event is not equivalent to joining the events: doing so would only tell you about the cases where the user clicked a search result, not about the searches where the user did not click any of the results. In order to measure search quality, you need accurate click-through rates, for which you need both the search events and the click events.

Lưu ý rằng việc nhúng các chi tiết của tìm kiếm vào sự kiện nhấp không tương đương với việc join các sự kiện: làm như vậy chỉ cho bạn biết về các trường hợp người dùng đã nhấp một kết quả tìm kiếm, chứ không phải về các tìm kiếm mà người dùng không nhấp bất kỳ kết quả nào. Để đo chất lượng tìm kiếm, bạn cần tỷ lệ nhấp-quá chính xác, mà để có nó thì bạn cần cả các sự kiện tìm kiếm lẫn các sự kiện nhấp.

To implement this type of join, a stream processor needs to maintain state: for example, all the events that occurred in the last hour, indexed by session ID. Whenever a search event or click event occurs, it is added to the appropriate index, and the stream processor also checks the other index to see if another event for the same session ID has already arrived. If there is a matching event, you emit an event saying which search result was clicked. If the search event expires without you seeing a matching click event, you emit an event saying which search results were not clicked.

Để hiện thực loại join này, một bộ xử lý luồng cần duy trì trạng thái: ví dụ, tất cả các sự kiện đã xảy ra trong giờ vừa qua, được lập chỉ mục theo ID phiên. Mỗi khi một sự kiện tìm kiếm hoặc sự kiện nhấp xảy ra, nó được thêm vào chỉ mục thích hợp, và bộ xử lý luồng cũng kiểm tra chỉ mục kia để xem một sự kiện khác cho cùng ID phiên đã đến hay chưa. Nếu có một sự kiện khớp, bạn phát ra một sự kiện cho biết kết quả tìm kiếm nào đã được nhấp. Nếu sự kiện tìm kiếm hết hạn mà bạn không thấy một sự kiện nhấp khớp, bạn phát ra một sự kiện cho biết những kết quả tìm kiếm nào đã không được nhấp.

Stream-table join (stream enrichment) — Join luồng-bảng (làm giàu luồng)

In “Example: analysis of user activity events” (Figure 10-2) we saw an example of a batch job joining two datasets: a set of user activity events and a database of user profiles. It is natural to think of the user activity events as a stream, and to perform the same join on a continuous basis in a stream processor: the input is a stream of activity events containing a user ID, and the output is a stream of activity events in which the user ID has been augmented with profile information about the user. This process is sometimes known as enriching the activity events with information from the database.

Ở “Ví dụ: phân tích các sự kiện hoạt động của người dùng” (Hình 10-2) ta đã thấy một ví dụ về một tác vụ lô join hai tập dữ liệu: một tập các sự kiện hoạt động của người dùng và một cơ sở dữ liệu các hồ sơ người dùng. Sẽ tự nhiên khi nghĩ về các sự kiện hoạt động của người dùng như một luồng, và thực hiện cùng phép join đó trên một cơ sở liên tục trong một bộ xử lý luồng: đầu vào là một luồng các sự kiện hoạt động chứa một ID người dùng, và đầu ra là một luồng các sự kiện hoạt động trong đó ID người dùng đã được bổ sung thông tin hồ sơ về người dùng. Tiến trình này đôi khi được gọi là làm giàu (enriching) các sự kiện hoạt động bằng thông tin từ cơ sở dữ liệu.

To perform this join, the stream process needs to look at one activity event at a time, look up the event’s user ID in the database, and add the profile information to the activity event. The database lookup could be implemented by querying a remote database; however, as discussed in “Example: analysis of user activity events”, such remote queries are likely to be slow and risk overloading the database [75].

Để thực hiện phép join này, tiến trình luồng cần nhìn vào một sự kiện hoạt động tại một thời điểm, tra cứu ID người dùng của sự kiện trong cơ sở dữ liệu, và thêm thông tin hồ sơ vào sự kiện hoạt động. Việc tra cứu cơ sở dữ liệu có thể được hiện thực bằng cách truy vấn một cơ sở dữ liệu từ xa; tuy nhiên, như đã bàn ở “Ví dụ: phân tích các sự kiện hoạt động của người dùng”, các truy vấn từ xa như vậy có khả năng chậm và có nguy cơ làm quá tải cơ sở dữ liệu [75].

Another approach is to load a copy of the database into the stream processor so that it can be queried locally without a network round-trip. This technique is very similar to the hash joins we discussed in “Map-Side Joins”: the local copy of the database might be an in-memory hash table if it is small enough, or an index on the local disk.

Một cách tiếp cận khác là nạp một bản sao của cơ sở dữ liệu vào bộ xử lý luồng để nó có thể được truy vấn cục bộ mà không cần một vòng đi-về qua mạng. Kỹ thuật này rất giống với các phép join băm (hash joins) mà ta đã bàn ở “Join phía map”: bản sao cục bộ của cơ sở dữ liệu có thể là một bảng băm trong bộ nhớ nếu nó đủ nhỏ, hoặc một chỉ mục trên đĩa cục bộ.

The difference to batch jobs is that a batch job uses a point-in-time snapshot of the database as input, whereas a stream processor is long-running, and the contents of the database are likely to change over time, so the stream processor’s local copy of the database needs to be kept up to date. This issue

can be solved by change data capture: the stream processor can subscribe to a changelog of the user profile database as well as the stream of activity events. When a profile is created or modified, the stream processor updates its local copy. Thus, we obtain a join between two streams: the activity events and the profile updates.

Khác biệt so với các tác vụ lô là một tác vụ lô dùng một ảnh chụp tại một thời điểm của cơ sở dữ liệu làm đầu vào, trong khi một bộ xử lý luồng chạy lâu dài, và nội dung của cơ sở dữ liệu có khả năng thay đổi theo thời gian, nên bản sao cục bộ của cơ sở dữ liệu của bộ xử lý luồng cần được giữ cập nhật. Vấn đề này có thể được giải quyết bằng việc nắm bắt thay đổi dữ liệu: bộ xử lý luồng có thể đăng ký một nhật ký thay đổi của cơ sở dữ liệu hồ sơ người dùng cũng như luồng các sự kiện hoạt động. Khi một hồ sơ được tạo hoặc sửa đổi, bộ xử lý luồng cập nhật bản sao cục bộ của nó. Như vậy, ta có được một phép join giữa hai luồng: các sự kiện hoạt động và các cập nhật hồ sơ.

A stream-table join is actually very similar to a stream-stream join; the biggest difference is that for the table changelog stream, the join uses a window that reaches back to the “beginning of time” (a conceptually infinite window), with newer versions of records overwriting older ones. For the stream input, the join might not maintain a window at all.

Một phép join luồng-bảng thực ra rất giống một phép join luồng-luồng; khác biệt lớn nhất là đối với luồng nhật ký thay đổi của bảng, phép join dùng một cửa sổ trải ngược về tới “khởi đầu của thời gian” (một cửa sổ về mặt khái niệm là vô hạn), với các phiên bản mới hơn của các bản ghi ghi đè lên các phiên bản cũ hơn. Đối với đầu vào luồng, phép join có thể không duy trì một cửa sổ nào cả.

Table-table join (materialized view maintenance) — Join bảng-bảng (duy trì khung nhìn cụ thể hóa)

Consider the Twitter timeline example that we discussed in “Describing Load”. We said that when a user wants to view their **home timeline**, it is too expensive to iterate over all the people the user is following, find their recent tweets, and merge them.

Hãy xét ví dụ dòng thời gian Twitter mà ta đã bàn ở “Mô tả tải”. Ta đã nói rằng khi một người dùng muốn xem dòng thời gian chính của họ, việc lặp qua tất cả những người mà người dùng đang theo dõi, tìm các tweet gần đây của họ, và trộn chúng lại là quá tốn kém.

Instead, we want a timeline cache: a kind of per-user “inbox” to which tweets are written as they are sent, so that reading the timeline is a single lookup. Materializing and maintaining this cache requires the following event processing:

Thay vào đó, ta muốn một bộ nhớ đệm dòng thời gian: một kiểu “hòm thư” cho-mỗi-người-dùng mà các tweet được ghi vào ngay khi chúng được gửi đi, nhờ vậy việc đọc dòng thời gian là một lần tra cứu duy nhất. Việc cụ thể hóa và duy trì bộ nhớ đệm này đòi hỏi việc xử lý sự kiện sau:

- When user u sends a new tweet, it is added to the timeline of every user who is following u.
- When a user deletes a tweet, it is removed from all users’ timelines.
- When user u1 starts following user u2, recent tweets by u2 are added to u1’s timeline.
- When user u1 unfollows user u2, tweets by u2 are removed from u1’s timeline.
 - Khi người dùng u gửi một tweet mới, nó được thêm vào dòng thời gian của mọi người dùng đang theo dõi u.
 - Khi một người dùng xóa một tweet, nó được loại khỏi dòng thời gian của tất cả người dùng.

- Khi người dùng u1 bắt đầu theo dõi người dùng u2, các tweet gần đây của u2 được thêm vào dòng thời gian của u1.
- Khi người dùng u1 bỏ theo dõi người dùng u2, các tweet của u2 được loại khỏi dòng thời gian của u1.

To implement this cache maintenance in a stream processor, you need streams of events for tweets (sending and deleting) and for follow relationships (following and unfollowing). The stream process needs to maintain a database containing the set of followers for each user so that it knows which timelines need to be updated when a new tweet arrives [86].

Để hiện thực việc duy trì bộ nhớ đệm này trong một bộ xử lý luồng, bạn cần các luồng sự kiện cho các tweet (gửi và xóa) và cho các quan hệ theo dõi (theo dõi và bỏ theo dõi). Tiến trình luồng cần duy trì một cơ sở dữ liệu chứa tập các người theo dõi của mỗi người dùng để nó biết những dòng thời gian nào cần được cập nhật khi một tweet mới đến [86].

Another way of looking at this stream process is that it maintains a materialized view for a query that joins two tables (tweets and follows), something like the following:

Một cách khác để nhìn nhận tiến trình luồng này là nó duy trì một khung nhìn cụ thể hóa cho một truy vấn join hai bảng (tweets và follows), đại loại như sau:

```
SELECT follows.follower_id AS timeline_id,
       array_agg(tweets.* ORDER BY tweets.timestamp DESC)
FROM tweets
JOIN follows ON follows.followee_id = tweets.sender_id
GROUP BY follows.follower_id
```

The join of the streams corresponds directly to the join of the tables in that query. The timelines are effectively a cache of the result of this query, updated every time the underlying tables change.

Phép join của các luồng tương ứng trực tiếp với phép join của các bảng trong truy vấn đó. Các dòng thời gian thực chất là một bộ nhớ đệm của kết quả của truy vấn này, được cập nhật mỗi khi các bảng nền thay đổi.

Time-dependence of joins — Sự phụ thuộc vào thời gian của các phép join

The three types of joins described here (stream-stream, stream-table, and table-table) have a lot in common: they all require the stream processor to maintain some state (search and click events, user profiles, or follower list) based on one join input, and query that state on messages from the other join input.

Ba loại join được mô tả ở đây (luồng-luồng, luồng-bảng, và bảng-bảng) có nhiều điểm chung: tất cả đều đòi hỏi bộ xử lý luồng duy trì một trạng thái nào đó (các sự kiện tìm kiếm và nhấp, các hồ sơ người dùng, hoặc danh sách người theo dõi) dựa trên một đầu vào join, và truy vấn trạng thái đó trên các thông điệp từ đầu vào join kia.

The order of the events that maintain the state is important (it matters whether you first follow and then unfollow, or the other way round). In a partitioned log, the ordering of events within a single partition is preserved, but there is typically no ordering guarantee across different streams or partitions.

Thứ tự của các sự kiện duy trì trạng thái là quan trọng (việc bạn theo dõi trước rồi bỏ theo dõi, hay ngược lại, là có khác biệt). Trong một nhật ký được phân mảnh, thứ tự của các sự kiện trong một mảnh đơn lẻ được giữ nguyên, nhưng thường không có bảo đảm về thứ tự giữa các luồng hoặc các mảnh khác nhau.

This raises a question: if events on different streams happen around a similar time, in which order are they processed? In the stream-table join example, if a user updates their profile, which activity events are joined with the old profile (processed before the profile update), and which are joined with the new profile (processed after the profile update)? Put another way: if state changes over time, and you join with some state, what point in time do you use for the join [45]?

Điều này làm nảy sinh một câu hỏi: nếu các sự kiện trên các luồng khác nhau xảy ra vào khoảng thời gian tương tự nhau, chúng được xử lý theo thứ tự nào? Trong ví dụ join luồng-bảng, nếu một người dùng cập nhật hồ sơ của họ, thì những sự kiện hoạt động nào được join với hồ sơ cũ (được xử lý trước khi hồ sơ được cập nhật), và những sự kiện nào được join với hồ sơ mới (được xử lý sau khi hồ sơ được cập nhật)? Nói cách khác: nếu trạng thái thay đổi theo thời gian, và bạn join với một trạng thái nào đó, bạn dùng thời điểm nào cho phép join [45]?

Such time dependence can occur in many places. For example, if you sell things, you need to apply the right tax rate to invoices, which depends on the country or state, the type of product, and the date of sale (since tax rates change from time to time). When joining sales to a table of tax rates, you probably want to join with the tax rate at the time of the sale, which may be different from the current tax rate if you are reprocessing historical data.

Sự phụ thuộc vào thời gian như vậy có thể xảy ra ở nhiều nơi. Ví dụ, nếu bạn bán hàng, bạn cần áp dụng đúng thuế suất cho các hóa đơn, điều này phụ thuộc vào quốc gia hoặc bang, loại sản phẩm, và ngày bán (vì thuế suất thay đổi theo thời gian). Khi join các giao dịch bán với một bảng các thuế suất, bạn có lẽ muốn join với thuế suất tại thời điểm bán, vốn có thể khác với thuế suất hiện tại nếu bạn đang xử lý lại dữ liệu lịch sử.

If the ordering of events across streams is undetermined, the join becomes nondeterministic [87], which means you cannot rerun the same job on the same input and necessarily get the same result: the events on the input streams may be interleaved in a different way when you run the job again.

Nếu thứ tự của các sự kiện giữa các luồng là không xác định, phép join trở nên không tất định [87], điều này có nghĩa là bạn không thể chạy lại cùng một tác vụ trên cùng một đầu vào và nhất thiết nhận được cùng một kết quả: các sự kiện trên các luồng đầu vào có thể được xen kẽ theo một cách khác khi bạn chạy lại tác vụ.

In data warehouses, this issue is known as a slowly changing dimension (SCD), and it is often addressed by using a unique identifier for a particular version of the joined record: for example, every time the tax rate changes, it is given a new identifier; and the invoice includes the identifier for the tax rate at the time of sale [88, 89]. This change makes the join deterministic, but has the consequence that log compaction is not possible, since all versions of the records in the table need to be retained.

Trong các kho dữ liệu, vấn đề này được gọi là một chiều thay đổi chậm (slowly changing dimension - SCD), và nó thường được giải quyết bằng cách dùng một định danh duy nhất cho một phiên bản cụ thể của bản ghi được join: ví dụ, mỗi khi thuế suất thay đổi, nó được cấp một định danh mới, và hóa đơn bao gồm định danh cho thuế suất tại thời điểm bán [88, 89]. Thay đổi này làm cho phép join trở nên tất định, nhưng có hệ quả là việc nén nhật ký không thể thực hiện được, vì tất cả các phiên bản của các bản ghi trong bảng cần được giữ lại.

Fault Tolerance — Khả năng chịu lỗi

In the final section of this chapter, let's consider how stream processors can tolerate faults. We saw in Chapter 10 that batch processing frameworks can tolerate faults fairly easily: if a task in a MapReduce job fails, it can simply be started again on another machine, and the output of the failed task is discarded. This transparent retry is possible because input files are immutable, each task writes its output to a separate file on HDFS, and output is only made visible when a task completes successfully.

Trong phần cuối của chương này, hãy xét xem các bộ xử lý luồng có thể chịu lỗi như thế nào. Ta đã thấy ở Chương 10 rằng các framework xử lý lô có thể chịu lỗi khá dễ dàng: nếu một tác vụ trong một tác vụ MapReduce thất bại, nó có thể đơn giản được khởi động lại trên một máy khác, và đầu ra của tác vụ thất bại bị vứt bỏ. Việc thử lại trong suốt này khả thi vì các tệp đầu vào là bất biến, mỗi tác vụ ghi đầu ra của nó ra một tệp riêng trên HDFS, và đầu ra chỉ được làm cho hiện hữu khi một tác vụ hoàn tất thành công.

In particular, the batch approach to fault tolerance ensures that the output of the batch job is the same as if nothing had gone wrong, even if in fact some tasks did fail. It appears as though every input record was processed exactly once—no records are skipped, and none are processed twice. Although restarting tasks means that records may in fact be processed multiple times, the visible effect in the output is as if they had only been processed once. This principle is known as exactly-once semantics, although effectively-once would be a more descriptive term [90].

Cụ thể, cách tiếp cận chịu lỗi của xử lý lô bảo đảm rằng đầu ra của tác vụ lô giống hệt như khi không có gì trục trặc, ngay cả khi thực ra một số tác vụ đã thất bại. Có vẻ như thể mỗi bản ghi đầu vào được xử lý đúng một lần — không có bản ghi nào bị bỏ qua, và không có bản ghi nào bị xử lý hai lần. Mặc dù việc khởi động lại các tác vụ có nghĩa là các bản ghi thực ra có thể bị xử lý nhiều lần, hiệu ứng hiện hữu trong đầu ra thì như thể chúng chỉ được xử lý một lần. Nguyên tắc này được gọi là ngữ nghĩa đúng-một-lần (exactly-once semantics), mặc dù về thực chất một-lần (effectively-once) sẽ là một thuật ngữ mô tả chính xác hơn [90].

The same issue of fault tolerance arises in stream processing, but it is less straightforward to handle: waiting until a task is finished before making its output visible is not an option, because a stream is infinite and so you can never finish processing it.

Cùng một vấn đề về khả năng chịu lỗi nảy sinh trong xử lý luồng, nhưng nó kém đơn giản hơn để xử lý: việc đợi cho tới khi một tác vụ hoàn tất trước khi làm cho đầu ra của nó hiện hữu không phải là một lựa chọn, vì một luồng là vô hạn nên bạn không bao giờ có thể xử lý xong nó.

Microbatching and checkpointing — Gộp vi-lô và điểm kiểm tra

One solution is to break the stream into small blocks, and treat each block like a miniature batch process. This approach is called microbatching, and it is used in Spark Streaming [91]. The batch size is typically around one second, which is the result of a performance compromise: smaller batches incur greater scheduling and coordination overhead, while larger batches **mean** a longer delay before results of the stream processor become visible.

Một giải pháp là chia luồng thành các khối nhỏ, và xử lý mỗi khối như một tiến trình lô thu nhỏ. Cách tiếp cận này được gọi là gộp vi-lô (microbatching), và nó được dùng trong Spark Streaming [91]. Kích thước lô thường vào khoảng một giây, đây là kết quả của một sự đánh đổi về hiệu năng: các lô nhỏ hơn gánh chịu chi phí lập lịch và phối hợp lớn hơn,

trong khi các lô lớn hơn đồng nghĩa với một độ trễ lâu hơn trước khi các kết quả của bộ xử lý luồng trở nên hiện hữu.

Microbatching also implicitly provides a tumbling window equal to the batch size (windowed by processing time, not event timestamps); any jobs that require larger windows need to explicitly carry over state from one microbatch to the next.

Việc gộp vi-lô cũng cung cấp một cách ngầm định một cửa sổ lăn bằng với kích thước lô (được cửa sổ hóa theo thời gian xử lý, chứ không phải theo dấu thời gian sự kiện); bất kỳ tác vụ nào đòi hỏi các cửa sổ lớn hơn cần mang trạng thái một cách tường minh từ một vi-lô sang vi-lô tiếp theo.

A variant approach, used in Apache Flink, is to periodically generate rolling checkpoints of state and write them to durable storage [92, 93]. If a stream operator crashes, it can restart from its most recent checkpoint and discard any output generated between the last checkpoint and the crash. The checkpoints are triggered by barriers in the message stream, similar to the boundaries between microbatches, but without forcing a particular window size.

Một cách tiếp cận biến thể, được dùng trong Apache Flink, là định kỳ sinh ra các điểm kiểm tra cuốn chiếu (rolling checkpoints) của trạng thái và ghi chúng vào lưu trữ bền vững [92, 93]. Nếu một toán tử luồng sập, nó có thể khởi động lại từ điểm kiểm tra gần nhất của nó và vứt bỏ bất kỳ đầu ra nào được sinh ra giữa điểm kiểm tra cuối và lúc sập. Các điểm kiểm tra được kích hoạt bởi các rào chắn (barrier) trong luồng thông điệp, tương tự như các ranh giới giữa các vi-lô, nhưng không ép buộc một kích thước cửa sổ cụ thể.

Within the confines of the stream processing framework, the microbatching and checkpointing approaches provide the same exactly-once semantics as batch processing. However, as soon as output leaves the stream processor (for example, by writing to a database, sending messages to an external message broker, or sending emails), the framework is no longer able to discard the output of a failed batch. In this case, restarting a failed task causes the external side effect to happen twice, and microbatching or checkpointing alone is not sufficient to prevent this problem.

Trong phạm vi của framework xử lý luồng, các cách tiếp cận gộp vi-lô và điểm kiểm tra cung cấp cùng ngữ nghĩa đúng-một-lần như xử lý lô. Tuy nhiên, ngay khi đầu ra rời khỏi bộ xử lý luồng (ví dụ, bằng cách ghi vào một cơ sở dữ liệu, gửi thông điệp tới một message broker bên ngoài, hoặc gửi email), framework không còn có thể vứt bỏ đầu ra của một lô thất bại nữa. Trong trường hợp này, việc khởi động lại một tác vụ thất bại khiến tác dụng phụ bên ngoài xảy ra hai lần, và chỉ riêng việc gộp vi-lô hoặc điểm kiểm tra là không đủ để ngăn vấn đề này.

Atomic commit revisited — Xét lại cam kết nguyên tử

In order to give the appearance of exactly-once processing in the presence of faults, we need to ensure that all outputs and side effects of processing an event take effect if and only if the processing is successful. Those effects include any messages sent to downstream operators or external messaging systems (including email or push notifications), any database writes, any changes to operator state, and any acknowledgment of input messages (including moving the consumer offset forward in a log-based message broker).

Để tạo ra vẻ ngoài của việc xử lý đúng-một-lần khi có các trục trặc, ta cần bảo đảm rằng tất cả các đầu ra và tác dụng phụ của việc xử lý một sự kiện có hiệu lực khi và chỉ khi việc xử lý thành công. Những tác dụng đó bao gồm bất kỳ thông điệp nào được gửi tới các toán tử ở hạ nguồn hoặc các hệ thống thông điệp bên ngoài (bao gồm email hoặc thông báo đẩy), bất kỳ thao tác ghi cơ sở dữ liệu nào, bất kỳ thay đổi nào đối với trạng thái của toán tử, và

bất kỳ xác nhận nào đối với các thông điệp đầu vào (bao gồm việc dời offset của bên tiêu thụ tiến lên trong một message broker dựa trên nhật ký).

Those things either all need to happen atomically, or none of them must happen, but they should not go out of sync with each other. If this approach sounds familiar, it is because we discussed it in “Exactly-once message processing” in the context of distributed transactions and two-phase commit.

Những thứ đó hoặc tất cả đều cần xảy ra một cách nguyên tử, hoặc không cái nào được xảy ra, nhưng chúng không nên rơi vào tình trạng mất đồng bộ với nhau. Nếu cách tiếp cận này nghe có vẻ quen, đó là vì ta đã bàn về nó ở “Xử lý thông điệp đúng-một-lần” trong ngữ cảnh của các giao dịch phân tán và cam kết hai pha.

In Chapter 9 we discussed the problems in the traditional implementations of distributed transactions, such as XA. However, in more restricted environments it is possible to implement such an atomic commit facility efficiently. This approach is used in Google Cloud Dataflow [81, 92] and VoltDB [94], and there are plans to add similar features to Apache Kafka [95, 96]. Unlike XA, these implementations do not attempt to provide transactions across heterogeneous technologies, but instead keep them internal by managing both state changes and messaging within the stream processing framework. The overhead of the transaction protocol can be amortized by processing several input messages within a single transaction.

Ở Chương 9 ta đã bàn về các vấn đề trong các hiện thực truyền thống của các giao dịch phân tán, chẳng hạn XA. Tuy nhiên, trong các môi trường hạn chế hơn, có thể hiện thực một tiện ích cam kết nguyên tử như vậy một cách hiệu quả. Cách tiếp cận này được dùng trong Google Cloud Dataflow [81, 92] và VoltDB [94], và có những kế hoạch bổ sung các tính năng tương tự vào Apache Kafka [95, 96]. Khác với XA, các hiện thực này không cố cung cấp các giao dịch xuyên qua các công nghệ không đồng nhất, mà thay vào đó giữ chúng nội bộ bằng cách quản lý cả các thay đổi trạng thái lẫn việc truyền thông điệp trong phạm vi framework xử lý luồng. Chi phí phụ trội của giao thức giao dịch có thể được phân bổ dần bằng cách xử lý vài thông điệp đầu vào trong một giao dịch duy nhất.

Idempotence — Tính lũy đẳng

Our goal is to discard the partial output of any failed tasks so that they can be safely retried without taking effect twice. Distributed transactions are one way of achieving that goal, but another way is to rely on idempotence [97].

Mục tiêu của ta là vứt bỏ đầu ra một phần của bất kỳ tác vụ thất bại nào để chúng có thể được thử lại một cách an toàn mà không có hiệu lực hai lần. Các giao dịch phân tán là một cách để đạt mục tiêu đó, nhưng một cách khác là dựa vào tính lũy đẳng (idempotence) [97].

An idempotent operation is one that you can perform multiple times, and it has the same effect as if you performed it only once. For example, setting a key in a key-value store to some fixed value is idempotent (writing the value again simply overwrites the value with an identical value), whereas incrementing a counter is not idempotent (performing the increment again means the value is incremented twice).

Một thao tác lũy đẳng là một thao tác mà bạn có thể thực hiện nhiều lần, và nó có cùng hiệu ứng như khi bạn chỉ thực hiện nó một lần. Ví dụ, việc đặt một khóa trong một kho khóa-giá trị thành một giá trị cố định nào đó là lũy đẳng (việc ghi lại giá trị đó đơn giản ghi đè giá trị bằng một giá trị y hệt), trong khi việc tăng một bộ đếm thì không lũy đẳng (thực hiện việc tăng lần nữa có nghĩa là giá trị được tăng hai lần).

Even if an operation is not naturally idempotent, it can often be made idempotent with a bit of extra metadata. For example, when consuming messages from Kafka, every message has a persistent, monotonically increasing offset. When writing a value to an external database, you can include the offset of the message that triggered the last write with the value. Thus, you can tell whether an update has already been applied, and avoid performing the same update again.

Ngay cả khi một thao tác không lũy đẳng một cách tự nhiên, nó thường có thể được làm thành lũy đẳng với một chút siêu dữ liệu bổ sung. Ví dụ, khi tiêu thụ các thông điệp từ Kafka, mỗi thông điệp có một offset bền vững, tăng đơn điệu. Khi ghi một giá trị vào một cơ sở dữ liệu bên ngoài, bạn có thể bao gồm offset của thông điệp đã kích hoạt thao tác ghi cuối cùng cùng với giá trị đó. Như vậy, bạn có thể biết được một bản cập nhật đã được áp dụng hay chưa, và tránh thực hiện cùng một bản cập nhật lần nữa.

The state handling in Storm's Trident is based on a similar idea [78]. Relying on idempotence implies several assumptions: restarting a failed task must replay the same messages in the same order (a log-based message broker does this), the processing must be deterministic, and no other node may concurrently update the same value [98, 99].

Việc xử lý trạng thái trong Trident của Storm dựa trên một ý tưởng tương tự [78]. Việc dựa vào tính lũy đẳng hàm chứa một số giả định: việc khởi động lại một tác vụ thất bại phải phát lại cùng các thông điệp theo cùng thứ tự (một message broker dựa trên nhật ký làm được điều này), việc xử lý phải mang tính tất định, và không nút nào khác được đồng thời cập nhật cùng một giá trị [98, 99].

When failing over from one processing node to another, fencing may be required (see “The leader and the lock”) to prevent interference from a node that is thought to be dead but is actually alive. Despite all those caveats, idempotent operations can be an effective way of achieving exactly-once semantics with only a small overhead.

Khi chuyển đổi dự phòng từ một nút xử lý sang một nút khác, có thể cần đến cơ chế rào chắn (fencing) (xem “Leader và khóa”) để ngăn sự can thiệp từ một nút bị cho là đã chết nhưng thực ra vẫn còn sống. Bất chấp tất cả những lưu ý đó, các thao tác lũy đẳng có thể là một cách hiệu quả để đạt được ngữ nghĩa đúng-một-lần với chỉ một chi phí phụ trội nhỏ.

Rebuilding state after a failure — Xây dựng lại trạng thái sau một sự cố

Any stream process that requires state—for example, any windowed aggregations (such as counters, averages, and histograms) and any tables and indexes used for joins—must ensure that this state can be recovered after a **failure**.

Bất kỳ tiến trình luồng nào đòi hỏi trạng thái — ví dụ, bất kỳ phép tổng hợp theo cửa sổ nào (chẳng hạn các bộ đếm, các giá trị trung bình, và các biểu đồ tần suất) cùng bất kỳ bảng và chỉ mục nào được dùng cho các phép join — đều phải bảo đảm rằng trạng thái này có thể được phục hồi sau một sự cố.

One option is to keep the state in a remote datastore and replicate it, although having to query a remote database for each individual message can be slow, as discussed in “Stream-table join (stream enrichment)”. An alternative is to keep state local to the stream processor, and replicate it periodically. Then, when the stream processor is recovering from a failure, the new task can read the replicated state and resume processing without data loss.

Một lựa chọn là giữ trạng thái trong một kho dữ liệu từ xa và nhân bản nó, mặc dù việc phải truy vấn một cơ sở dữ liệu từ xa cho mỗi thông điệp riêng lẻ có thể chậm, như đã bàn ở “Join luồng-bảng (làm giàu luồng)”. Một cách khác là giữ trạng thái cục bộ với bộ xử lý

luồng, và nhân bản nó định kỳ. Sau đó, khi bộ xử lý luồng đang phục hồi từ một sự cố, tác vụ mới có thể đọc trạng thái đã được nhân bản và tiếp tục xử lý mà không mất dữ liệu.

For example, Flink periodically captures snapshots of operator state and writes them to durable storage such as HDFS [92, 93]; Samza and Kafka Streams replicate state changes by sending them to a dedicated Kafka topic with log compaction, similar to change data capture [84, 100]. VoltDB replicates state by redundantly processing each input message on several nodes (see “Actual Serial Execution”).

Ví dụ, Flink định kỳ chụp các ảnh chụp trạng thái của toán tử và ghi chúng vào lưu trữ bền vững như HDFS [92, 93]; Samza và Kafka Streams nhân bản các thay đổi trạng thái bằng cách gửi chúng tới một chủ đề Kafka chuyên dụng có nén nhật ký, tương tự như việc nắm bắt thay đổi dữ liệu [84, 100]. VoltDB nhân bản trạng thái bằng cách xử lý dư thừa mỗi thông điệp đầu vào trên vài nút (xem “Thực thi tuần tự thực sự”).

In some cases, it may not even be necessary to replicate the state, because it can be rebuilt from the input streams. For example, if the state consists of aggregations over a fairly short window, it may be fast enough to simply replay the input events corresponding to that window. If the state is a local replica of a database, maintained by change data capture, the database can also be rebuilt from the log-compacted change stream (see “Log compaction”).

Trong một số trường hợp, thậm chí có thể không cần nhân bản trạng thái, vì nó có thể được xây dựng lại từ các luồng đầu vào. Ví dụ, nếu trạng thái bao gồm các phép tổng hợp trên một cửa sổ khá ngắn, việc đơn giản phát lại các sự kiện đầu vào tương ứng với cửa sổ đó có thể đủ nhanh. Nếu trạng thái là một bản sao cục bộ của một cơ sở dữ liệu, được duy trì bằng việc nắm bắt thay đổi dữ liệu, thì cơ sở dữ liệu cũng có thể được xây dựng lại từ luồng thay đổi đã được nén nhật ký (xem “Nén nhật ký”).

However, all of these trade-offs depend on the performance characteristics of the underlying infrastructure: in some systems, network delay may be lower than disk access latency, and network bandwidth may be comparable to disk bandwidth. There is no universally ideal trade-off for all situations, and the merits of local versus remote state may also shift as storage and networking technologies evolve.

Tuy nhiên, tất cả những sự đánh đổi này đều phụ thuộc vào các đặc tính hiệu năng của hạ tầng nền: trong một số hệ thống, độ trễ mạng có thể thấp hơn độ trễ truy cập đĩa, và băng thông mạng có thể tương đương với băng thông đĩa. Không có một sự đánh đổi nào lý tưởng phổ quát cho mọi tình huống, và những ưu điểm của trạng thái cục bộ so với trạng thái từ xa cũng có thể dịch chuyển khi các công nghệ lưu trữ và mạng tiến hóa.

Summary — Tóm tắt

In this chapter we have discussed event streams, what purposes they serve, and how to process them. In some ways, stream processing is very much like the batch processing we discussed in Chapter 10, but done continuously on unbounded (never-ending) streams rather than on a fixed-size input. From this perspective, message brokers and event logs serve as the streaming equivalent of a filesystem.

Trong chương này ta đã bàn về các luồng sự kiện, chúng phục vụ những mục đích gì, và cách xử lý chúng. Theo một số cách, xử lý luồng rất giống xử lý lô mà ta đã bàn ở Chương 10, nhưng được thực hiện liên tục trên các luồng không giới hạn (không bao giờ kết thúc) thay vì trên một đầu vào có kích thước cố định. Từ góc nhìn này, các message broker và các nhật ký sự kiện đóng vai trò là phiên bản tương đương của luồng so với một hệ thống tệp.

We spent some time comparing two types of message brokers:

Ta đã dành một chút thời gian so sánh hai loại message broker:

The broker assigns individual messages to consumers, and consumers acknowledge individual messages when they have been successfully processed. Messages are deleted from the broker once they have been acknowledged. This approach is appropriate as an asynchronous form of RPC (see also “Message-Passing Dataflow”), for example in a task queue, where the exact order of message processing is not important and where there is no need to go back and read old messages again after they have been processed.

Broker gán từng thông điệp riêng lẻ cho các bên tiêu thụ, và các bên tiêu thụ xác nhận từng thông điệp riêng lẻ khi chúng đã được xử lý thành công. Các thông điệp bị xóa khỏi broker một khi chúng đã được xác nhận. Cách tiếp cận này thích hợp như một dạng RPC bất đồng bộ (xem thêm “Luồng dữ liệu truyền thông điệp”), ví dụ trong một hàng đợi tác vụ, nơi thứ tự chính xác của việc xử lý thông điệp không quan trọng và nơi không cần quay lại đọc các thông điệp cũ lần nữa sau khi chúng đã được xử lý.

The broker assigns all messages in a partition to the same consumer node, and always delivers messages in the same order. Parallelism is achieved through partitioning, and consumers track their progress by checkpointing the offset of the last message they have processed. The broker retains messages on disk, so it is possible to jump back and reread old messages if necessary.

Broker gán tất cả các thông điệp trong một mảnh cho cùng một nút tiêu thụ, và luôn phân phát các thông điệp theo cùng thứ tự. Tính song song đạt được thông qua phân mảnh, và các bên tiêu thụ theo dõi tiến độ của chúng bằng cách đặt điểm kiểm tra cho offset của thông điệp cuối cùng mà chúng đã xử lý. Broker giữ các thông điệp trên đĩa, nên có thể nhảy ngược lại và đọc lại các thông điệp cũ nếu cần.

The log-based approach has similarities to the replication logs found in databases (see Chapter 5) and log-structured storage engines (see Chapter 3). We saw that this approach is especially appropriate

for stream processing systems that consume input streams and generate derived state or derived output streams.

Cách tiếp cận dựa trên nhật ký có những điểm tương đồng với các nhật ký nhân bản thấy trong các cơ sở dữ liệu (xem Chương 5) và các engine lưu trữ có cấu trúc nhật ký (xem Chương 3). Ta đã thấy rằng cách tiếp cận này đặc biệt thích hợp cho các hệ thống xử lý luồng vốn tiêu thụ các luồng đầu vào và sinh ra trạng thái dẫn xuất hoặc các luồng đầu ra dẫn xuất.

In terms of where streams come from, we discussed several possibilities: user activity events, sensors providing periodic readings, and data feeds (e.g., market data in finance) are naturally represented as streams. We saw that it can also be useful to think of the writes to a database as a stream: we can capture the changelog—i.e., the history of all changes made to a database—either implicitly through change data capture or explicitly through event sourcing. Log compaction allows the stream to retain a full copy of the contents of a database.

Về việc các luồng đến từ đâu, ta đã bàn về một số khả năng: các sự kiện hoạt động của người dùng, các cảm biến cung cấp các giá trị đọc định kỳ, và các nguồn cấp dữ liệu (ví dụ, dữ liệu thị trường trong tài chính) được biểu diễn một cách tự nhiên thành các luồng. Ta đã thấy rằng cũng có thể hữu ích khi nghĩ về các thao tác ghi vào một cơ sở dữ liệu như một luồng: ta có thể nắm bắt nhật ký thay đổi — tức là lịch sử của tất cả các thay đổi thực hiện đối với một cơ sở dữ liệu — hoặc một cách ngầm định thông qua việc nắm bắt thay đổi dữ liệu, hoặc một cách tường minh thông qua việc lấy nguồn từ sự kiện. Việc nén nhật ký cho phép luồng giữ một bản sao đầy đủ của nội dung một cơ sở dữ liệu.

Representing databases as streams opens up powerful opportunities for integrating systems. You can keep derived data systems such as search indexes, caches, and analytics systems continually up to date by consuming the log of changes and applying them to the derived system. You can even build fresh views onto existing data by starting from scratch and consuming the log of changes from the beginning all the way to the present.

Việc biểu diễn các cơ sở dữ liệu thành các luồng mở ra những cơ hội mạnh mẽ để tích hợp các hệ thống. Bạn có thể giữ các hệ thống dữ liệu dẫn xuất như các chỉ mục tìm kiếm, các bộ nhớ đệm, và các hệ thống phân tích liên tục được cập nhật bằng cách tiêu thụ nhật ký các thay đổi và áp dụng chúng vào hệ thống dẫn xuất. Bạn thậm chí có thể xây dựng các khung nhìn mới mẽ lên dữ liệu hiện có bằng cách bắt đầu từ con số không và tiêu thụ nhật ký các thay đổi từ đầu cho tới tận hiện tại.

The facilities for maintaining state as streams and replaying messages are also the basis for the techniques that enable stream joins and fault tolerance in various stream processing frameworks. We discussed several purposes of stream processing, including searching for event patterns (complex event processing), computing windowed aggregations (stream analytics), and keeping derived data systems up to date (materialized views).

Các tiện ích để duy trì trạng thái dưới dạng các luồng và phát lại các thông điệp cũng là cơ sở cho các kỹ thuật cho phép các phép join luồng và khả năng chịu lỗi trong nhiều framework xử lý luồng. Ta đã bàn về một số mục đích của xử lý luồng, bao gồm việc tìm kiếm các mẫu sự kiện (xử lý sự kiện phức hợp), tính các phép tổng hợp theo cửa sổ (phân tích luồng), và giữ các hệ thống dữ liệu dẫn xuất được cập nhật (các khung nhìn cụ thể hóa).

We then discussed the difficulties of reasoning about time in a stream processor, including the distinction between processing time and event timestamps, and the problem of dealing with straggler events that arrive after you thought your window was complete.

Sau đó ta đã bàn về những khó khăn của việc lập luận về thời gian trong một bộ xử lý luồng, bao gồm sự phân biệt giữa thời gian xử lý và dấu thời gian sự kiện, và vấn đề đối phó với các sự kiện lạc đàn đến sau khi bạn nghĩ cửa sổ của mình đã hoàn tất.

We distinguished three types of joins that may appear in stream processes:

Ta đã phân biệt ba loại join có thể xuất hiện trong các tiến trình luồng:

Both input streams consist of activity events, and the join operator searches for related events that occur within some window of time. For example, it may match two actions taken by the same user within 30 minutes of each other. The two join inputs may in fact be the same stream (a self-join) if you want to find related events within that one stream.

Cả hai luồng đầu vào đều bao gồm các sự kiện hoạt động, và toán tử join tìm kiếm các sự kiện liên quan xảy ra trong một cửa sổ thời gian nào đó. Ví dụ, nó có thể khớp hai hành động được thực hiện bởi cùng một người dùng trong vòng 30 phút của nhau. Hai đầu vào join thực ra có thể là cùng một luồng (một phép tự-join) nếu bạn muốn tìm các sự kiện liên quan trong chính một luồng đó.

One input stream consists of activity events, while the other is a database changelog. The changelog keeps a local copy of the database up to date. For each activity event, the join operator queries the database and outputs an enriched activity event.

Một luồng đầu vào bao gồm các sự kiện hoạt động, trong khi luồng kia là một nhật ký thay đổi của cơ sở dữ liệu. Nhật ký thay đổi giữ một bản sao cục bộ của cơ sở dữ liệu luôn được cập nhật. Với mỗi sự kiện hoạt động, toán tử join truy vấn cơ sở dữ liệu và xuất ra một sự kiện hoạt động đã được làm giàu.

Both input streams are database changelogs. In this case, every change on one side is joined with the latest state of the other side. The result is a stream of changes to the materialized view of the join between the two tables.

Cả hai luồng đầu vào đều là các nhật ký thay đổi của cơ sở dữ liệu. Trong trường hợp này, mỗi thay đổi ở một phía được join với trạng thái mới nhất của phía kia. Kết quả là một luồng các thay đổi đối với khung nhìn cụ thể hóa của phép join giữa hai bảng.

Finally, we discussed techniques for achieving fault tolerance and exactly-once semantics in a stream processor. As with batch processing, we need to discard the partial output of any failed tasks. However, since a stream process is long-running and produces output continuously, we can't simply discard all output. Instead, a finer-grained recovery mechanism can be used, based on microbatching, checkpointing, transactions, or idempotent writes.

Cuối cùng, ta đã bàn về các kỹ thuật để đạt được khả năng chịu lỗi và ngữ nghĩa đúng-một-lần trong một bộ xử lý luồng. Cũng như với xử lý lô, ta cần vứt bỏ đầu ra một phần của bất kỳ tác vụ thất bại nào. Tuy nhiên, vì một tiến trình luồng chạy lâu dài và sinh ra đầu ra liên tục, ta không thể đơn giản vứt bỏ tất cả đầu ra. Thay vào đó, có thể dùng một cơ chế phục hồi ở mức tinh hơn, dựa trên việc gộp vi-lô, đặt điểm kiểm tra, các giao dịch, hoặc các thao tác ghi lũy đẳng.

Footnotes It's possible to create a load balancing scheme in which two consumers share the work of processing a partition by having both read the full set of messages, but one of them only considers messages with even-numbered offsets while the other deals with the odd-numbered offsets. Alternatively, you could spread message processing over a thread pool, but that approach complicates consumer offset management. In general, single-threaded processing of a partition is preferable, and parallelism can be increased by using more partitions.

Thank you to Kostas Kloudas from the Flink community for coming up with this analogy.

If you regard a stream as the derivative of a table, as in Figure 11-6, and regard a join as a product of two tables $u \cdot v$, something interesting happens: the stream of changes to the materialized join follows the product rule $(u \cdot v)' = u'v + uv'$. In words: any change of tweets is joined with the current followers, and any change of followers is joined with the current tweets [49, 50].

References [1] Tyler Akidau, Robert Bradshaw, Craig Chambers, et al.: “The Dataflow Model: A Practical Approach to Balancing Correctness, Latency, and Cost in Massive-Scale, Unbounded, Out-of-Order Data Processing,” *Proceedings of the VLDB Endowment*, volume 8, number 12, pages 1792–1803, August 2015. doi:10.14778/2824032.2824076

[2] Harold Abelson, Gerald Jay Sussman, and Julie Sussman: *Structure and Interpretation of Computer Programs*, 2nd edition. MIT Press, 1996. ISBN: 978-0-262-51087-5, available online at mitpress.mit.edu

[3] Patrick Th. Eugster, Pascal A. Felber, Rachid Guerraoui, and Anne-Marie Kermarrec: “The Many Faces of Publish/Subscribe,” *ACM Computing Surveys*, volume 35, number 2, pages 114–131, June 2003. doi:10.1145/857076.857078

[4] Joseph M. Hellerstein and Michael Stonebraker: *Readings in Database Systems*, 4th edition. MIT Press, 2005. ISBN: 978-0-262-69314-1, available online at redbook.cs.berkeley.edu

[5] Don Carney, Uğur Çetintemel, Mitch Cherniack, et al.: “Monitoring Streams – A New Class of Data Management Applications,” at 28th International Conference on Very Large Data Bases (VLDB), August 2002.

[6] Matthew Sackman: “Pushing Back,” lshift.net, May 5, 2016.

[7] Vicent Martí: “Brubeck, a statsd-Compatible Metrics Aggregator,” githubengineering.com, June 15, 2015.

[8] Seth Lowenberger: “MoldUDP64 Protocol Specification V 1.00,” nasdaqtrader.com, July 2009.

[9] Pieter Hintjens: *ZeroMQ – The Guide*. O’Reilly Media, 2013. ISBN: 978-1-449-33404-8

[10] Ian Malpass: “Measure Anything, Measure Everything,” codeascraft.com, February 15, 2011.

[11] Dieter Plaetinck: “25 Graphite, Grafana and statsd Gotchas,” blog.raintank.io, March 3, 2016.

[12] Jeff Lindsay: “Web Hooks to Revolutionize the Web,” progrium.com, May 3, 2007.

[13] Jim N. Gray: “Queues Are Databases,” Microsoft Research Technical Report MSR-TR-95-56, December 1995.

[14] Mark Hapner, Rich Burridge, Rahul Sharma, et al.: “JSR-343 Java Message Service (JMS) 2.0 Specification,” jms-spec.java.net, March 2013.

[15] Sanjay Aiyagari, Matthew Arrott, Mark Atwell, et al.: “AMQP: Advanced Message Queuing Protocol Specification,” Version 0-9-1, November 2008.

[16] “Google Cloud Pub/Sub: A Google-Scale Messaging Service,” cloud.google.com, 2016.

[17] “Apache Kafka 0.9 Documentation,” kafka.apache.org, November 2015.

[18] Jay Kreps, Neha Narkhede, and Jun Rao: “Kafka: A Distributed Messaging System for Log Processing,” at 6th International Workshop on Networking Meets Databases (NetDB), June 2011.

[19] “Amazon Kinesis Streams Developer Guide,” docs.aws.amazon.com, April 2016.

- [20] Leigh Stewart and Sijie Guo: “Building DistributedLog: Twitter’s High-Performance Replicated Log Service,” blog.twitter.com, September 16, 2015.
- [21] “DistributedLog Documentation,” Twitter, Inc., distributedlog.io, May 2016.
- [22] Jay Kreps: “Benchmarking Apache Kafka: 2 Million Writes Per Second (On Three Cheap Machines),” engineering.linkedin.com, April 27, 2014.
- [23] Kartik Paramasivam: “How We’re Improving and Advancing Kafka at LinkedIn,” engineering.linkedin.com, September 2, 2015.
- [24] Jay Kreps: “The Log: What Every Software Engineer Should Know About Real-Time Data’s Unifying Abstraction,” engineering.linkedin.com, December 16, 2013.
- [25] Shirshanka Das, Chavdar Botev, Kapil Surlaker, et al.: “All Aboard the Databus!,” at 3rd ACM Symposium on Cloud Computing (SoCC), October 2012.
- [26] Yogeshwer Sharma, Philippe Ajoux, Petchean Ang, et al.: “Wormhole: Reliable Pub-Sub to Support Geo-Replicated Internet Services,” at 12th USENIX Symposium on Networked Systems Design and Implementation (NSDI), May 2015.
- [27] P. P. S. Narayan: “Sherpa Update,” developer.yahoo.com, June 8, .
- [28] Martin Kleppmann: “Bottled Water: Real-Time Integration of PostgreSQL and Kafka,” martin.kleppmann.com, April 23, 2015.
- [29] Ben Osheroﬀ: “Introducing Maxwell, a mysql-to-kafka Binlog Processor,” developer.zendesk.com, August 20, 2015.
- [30] Randall Hauch: “Debezium 0.2.1 Released,” debezium.io, June 10, 2016.
- [31] Prem Santosh Udaya Shankar: “Streaming MySQL Tables in Real-Time to Kafka,” engineeringblog.yelp.com, August 1, 2016.
- [32] “Mongoriver,” Stripe, Inc., github.com, September 2014.
- [33] Dan Harvey: “Change Data Capture with Mongo + Kafka,” at Hadoop Users Group UK, August 2015.
- [34] “Oracle GoldenGate 12c: Real-Time Access to Real-Time Information,” Oracle White Paper, March 2015.
- [35] “Oracle GoldenGate Fundamentals: How Oracle GoldenGate Works,” Oracle Corporation, youtube.com, November 2012.
- [36] Slava Akhmechet: “Advancing the Realtime Web,” rethinkdb.com, January 27, 2015.
- [37] “Firebase Realtime Database Documentation,” Google, Inc., firebase.google.com, May 2016.
- [38] “Apache CouchDB 1.6 Documentation,” docs.couchdb.org, 2014.
- [39] Matt DeBergalis: “Meteor 0.7.0: Scalable Database Queries Using MongoDB Oplog Instead of Poll-and-Diff,” info.meteor.com, December 17, 2013.
- [40] “Chapter 15. Importing and Exporting Live Data,” VoltDB 6.4 User Manual, docs.voltdb.com, June 2016.
- [41] Neha Narkhede: “Announcing Kafka Connect: Building Large-Scale Low-Latency Data Pipelines,” confluent.io, February 18, 2016.
- [42] Greg Young: “CQRS and Event Sourcing,” at Code on the Beach, August 2014.

- [43] Martin Fowler: “Event Sourcing,” martinfowler.com, December 12, 2005.
- [44] Vaughn Vernon: *Implementing Domain-Driven Design*. Addison-Wesley Professional, 2013. ISBN: 978-0-321-83457-7
- [45] H. V. Jagadish, Inderpal Singh Mumick, and Abraham Silberschatz: “View Maintenance Issues for the Chronicle Data Model,” at 14th ACM SIGACT-SIGMOD-SIGART Symposium on Principles of Database Systems (PODS), May 1995. doi:10.1145/212433.220201
- [46] “Event Store 3.5.0 Documentation,” Event Store LLP, docs.geteventstore.com, February 2016.
- [47] Martin Kleppmann: *Making Sense of Stream Processing*. Report, O’Reilly Media, May 2016.
- [48] Sander Mak: “Event-Sourced Architectures with Akka,” at JavaOne, September 2014.
- [49] Julian Hyde: personal communication, June 2016.
- [50] Ashish Gupta and Inderpal Singh Mumick: *Materialized Views: Techniques, Implementations, and Applications*. MIT Press, 1999. ISBN: 978-0-262-57122-7
- [51] Timothy Griffin and Leonid Libkin: “Incremental Maintenance of Views with Duplicates,” at ACM International Conference on Management of Data (SIGMOD), May 1995. doi:10.1145/223784.223849
- [52] Pat Helland: “Immutability Changes Everything,” at 7th Biennial Conference on Innovative Data Systems Research (CIDR), January 2015.
- [53] Martin Kleppmann: “Accounting for Computer Scientists,” martin.kleppmann.com, March 7, 2011.
- [54] Pat Helland: “Accountants Don’t Use Erasers,” blogs.msdn.com, June 14, 2007.
- [55] Fangjin Yang: “Dogfooding with Druid, Samza, and Kafka: Metametrics at Metamarkets,” metamarkets.com, June 3, 2015.
- [56] Gavin Li, Jianqiu Lv, and Hang Qi: “Pistachio: Co-Locate the Data and Compute for Fastest Cloud Compute,” yahoohadoop.tumblr.com, April 13, 2015.
- [57] Kartik Paramasivam: “Stream Processing Hard Problems – Part 1: Killing Lambda,” engineering.linkedin.com, June 27, 2016.
- [58] Martin Fowler: “CQRS,” martinfowler.com, July 14, 2011.
- [59] Greg Young: “CQRS Documents,” cqrs.files.wordpress.com, November 2010.
- [60] Baron Schwartz: “Immutability, MVCC, and Garbage Collection,” xaprb.com, December 28, 2013.
- [61] Daniel Eloff, Slava Akhmechet, Jay Kreps, et al.: “Re: Turning the Database Inside-out with Apache Samza,” Hacker News discussion, news.ycombinator.com, March 4, 2015.
- [62] “Datomic Development Resources: Excision,” Cognitect, Inc., docs.datomic.com.
- [63] “Fossil Documentation: Deleting Content from Fossil,” fossil-scm.org, 2016.
- [64] Jay Kreps: “The irony of distributed systems is that data loss is really easy but deleting data is surprisingly hard,” twitter.com, March 30, 2015.
- [65] David C. Luckham: “What’s the Difference Between ESP and CEP?,” complexevents.com, August 1, 2006.
- [66] Srinath Perera: “How Is Stream Processing and Complex Event Processing (CEP) Different?,” quora.com, December 3, 2015.

- [67] Arvind Arasu, Shivnath Babu, and Jennifer Widom: “The CQL Continuous Query Language: Semantic Foundations and Query Execution,” *The VLDB Journal*, volume 15, number 2, pages 121–142, June 2006. doi:10.1007/s00778-004-0147-z
- [68] Julian Hyde: “Data in Flight: How Streaming SQL Technology Can Help Solve the Web 2.0 Data Crunch,” *ACM Queue*, volume 7, number 11, December 2009. doi:10.1145/1661785.1667562
- [69] “Esper Reference, Version 5.4.0,” EsperTech, Inc., espertech.com, April 2016.
- [70] Zubair Nabi, Eric Bouillet, Andrew Bainbridge, and Chris Thomas: “Of Streams and Storms,” IBM technical report, developer.ibm.com, April 2014.
- [71] Milinda Pathirage, Julian Hyde, Yi Pan, and Beth Plale: “SamzaSQL: Scalable Fast Data Management with Streaming SQL,” at IEEE International Workshop on High-Performance Big Data Computing (HPBDC), May 2016. doi:10.1109/IPDPSW.2016.141
- [72] Philippe Flajolet, Éric Fusy, Olivier Gandouet, and Frédéric Meunier: “HyperLogLog: The Analysis of a Near-Optimal Cardinality Estimation Algorithm,” at Conference on Analysis of Algorithms (AofA), June 2007.
- [73] Jay Kreps: “Questioning the Lambda Architecture,” oreilly.com, July 2, 2014.
- [74] Ian Hellström: “An Overview of Apache Streaming Technologies,” databaseline.wordpress.com, March 12, 2016.
- [75] Jay Kreps: “Why Local State Is a Fundamental Primitive in Stream Processing,” oreilly.com, July 31, 2014.
- [76] Shay Banon: “Percolator,” elastic.co, February 8, 2011.
- [77] Alan Woodward and Martin Kleppmann: “Real-Time Full-Text Search with Luwak and Samza,” martin.kleppmann.com, April 13, 2015.
- [78] “Apache Storm 1.0.1 Documentation,” storm.apache.org, May 2016.
- [79] Tyler Akidau: “The World Beyond Batch: Streaming 102,” oreilly.com, January 20, 2016.
- [80] Stephan Ewen: “Streaming Analytics with Apache Flink,” at Kafka Summit, April 2016.
- [81] Tyler Akidau, Alex Balikov, Kaya Bekiroğlu, et al.: “MillWheel: Fault-Tolerant Stream Processing at Internet Scale,” at 39th International Conference on Very Large Data Bases (VLDB), August 2013.
- [82] Alex Dean: “Improving Snowplow’s Understanding of Time,” snowplowanalytics.com, September 15, 2015.
- [83] “Windowing (Azure Stream Analytics),” Microsoft Azure Reference, msdn.microsoft.com, April 2016.
- [84] “State Management,” Apache Samza 0.10 Documentation, samza.apache.org, December 2015.
- [85] Rajagopal Ananthanarayanan, Venkatesh Basker, Sumit Das, et al.: “Photon: Fault-Tolerant and Scalable Joining of Continuous Data Streams,” at ACM International Conference on Management of Data (SIGMOD), June 2013. doi:10.1145/2463676.2465272
- [86] Martin Kleppmann: “Samza Newsfeed Demo,” github.com, September 2014.
- [87] Ben Kirwin: “Doing the Impossible: Exactly-Once Messaging Patterns in Kafka,” ben.kirw.in, November 28, 2014.
- [88] Pat Helland: “Data on the Outside Versus Data on the Inside,” at 2nd Biennial Conference on Innovative Data Systems Research (CIDR), January 2005.

- [89] Ralph Kimball and Margy Ross: *The Data Warehouse Toolkit: The Definitive Guide to Dimensional Modeling*, 3rd edition. John Wiley & Sons, 2013. ISBN: 978-1-118-53080-1
- [90] Viktor Klang: “I’m coining the phrase ‘effectively-once’ for message processing with at-least-once + idempotent operations,” twitter.com, October 20, 2016.
- [91] Matei Zaharia, Tathagata Das, Haoyuan Li, et al.: “Discretized Streams: An Efficient and Fault-Tolerant Model for Stream Processing on Large Clusters,” at 4th USENIX Conference in Hot Topics in Cloud Computing (HotCloud), June 2012.
- [92] Kostas Tzoumas, Stephan Ewen, and Robert Metzger: “High-Throughput, Low-Latency, and Exactly-Once Stream Processing with Apache Flink,” data-artisans.com, August 5, 2015.
- [93] Paris Carbone, Gyula Fóra, Stephan Ewen, et al.: “Lightweight Asynchronous Snapshots for Distributed Dataflows,” [arXiv:1506.08603 \[cs.DC\]](https://arxiv.org/abs/1506.08603), June 29, 2015.
- [94] Ryan Betts and John Hugg: *Fast Data: Smart and at Scale*. Report, O’Reilly Media, October 2015.
- [95] Flavio Junqueira: “Making Sense of Exactly-Once Semantics,” at Strata+Hadoop World London, June 2016.
- [96] Jason Gustafson, Flavio Junqueira, Apurva Mehta, Sriram Subramanian, and Guozhang Wang: “KIP-98 – Exactly Once Delivery and Transactional Messaging,” cwiki.apache.org, November 2016.
- [97] Pat Helland: “Idempotence Is Not a Medical Condition,” *Communications of the ACM*, volume 55, number 5, page 56, May 2012. doi:10.1145/2160718.2160734
- [98] Jay Kreps: “Re: Trying to Achieve Deterministic Behavior on Recovery/Rewind,” email to samza-dev mailing list, September 9, 2014.
- [99] E. N. (Mootaz) Elnozahy, Lorenzo Alvisi, Yi-Min Wang, and David B. Johnson: “A Survey of Rollback-Recovery Protocols in Message-Passing Systems,” *ACM Computing Surveys*, volume 34, number 3, pages 375–408, September 2002. doi:10.1145/568522.568525
- [100] Adam Warski: “Kafka Streams – How Does It Fit the Stream Processing Landscape?,” softwaremill.com, June 1, 2016.

Chapter 12. The Future of Data Systems — Chương 12. Tương lai của các hệ thống dữ liệu

If a thing be ordained to another as to its end, its last end cannot consist in the preservation of its being. Hence a captain does not intend as a last end, the preservation of the ship entrusted to him, since a ship is ordained to something else as its end, viz. to navigation.

Nếu một vật được sắp đặt hướng tới một vật khác như tới cứu cánh của nó, thì cứu cánh tối hậu của nó không thể nằm ở việc bảo toàn sự tồn tại của chính nó. Bởi vậy một thuyền trưởng không lấy việc bảo toàn con tàu được giao phó cho mình làm cứu cánh tối hậu, vì con tàu được sắp đặt hướng tới một điều khác như cứu cánh của nó, ấy là sự đi biển.

(Often quoted as: If the highest aim of a captain was the preserve his ship, he would keep it in port forever.)

(Thường được trích dẫn thành: Nếu mục tiêu cao nhất của một thuyền trưởng là giữ gìn con tàu của mình, thì hẳn ông ta sẽ giữ nó mãi trong cảng.)

St. Thomas Aquinas, Summa Theologica (1265–1274)

Thánh Thomas Aquinas, Summa Theologica (1265–1274)

So far, this book has been mostly about describing things as they are at present. In this final chapter, we will shift our perspective toward the future and discuss how things should be: I will propose some ideas and approaches that, I believe, may fundamentally improve the ways we design and build applications.

Cho đến nay, cuốn sách này chủ yếu mô tả mọi thứ như chúng đang hiện hữu. Trong chương cuối này, ta sẽ chuyển góc nhìn về phía tương lai và bàn xem mọi thứ nên như thế nào: tôi sẽ đề xuất một số ý tưởng và cách tiếp cận mà, theo tôi tin, có thể cải thiện một cách căn bản cách ta thiết kế và xây dựng ứng dụng.

Opinions and speculation about the future are of course subjective, and so I will use the first person in this chapter when writing about my personal opinions. You are welcome to disagree with them and form your own opinions, but I hope that the ideas in this chapter will at least be a starting point for a productive discussion and bring some clarity to concepts that are often confused.

Tất nhiên, những quan điểm và suy đoán về tương lai mang tính chủ quan, nên trong chương này tôi sẽ dùng ngôi thứ nhất khi viết về ý kiến cá nhân của mình. Bạn hoàn toàn được phép bất đồng với chúng và hình thành quan điểm của riêng mình, nhưng tôi hy

vọng những ý tưởng trong chương này ít nhất sẽ là điểm khởi đầu cho một cuộc thảo luận hữu ích, và mang lại đôi chút sáng tỏ cho những khái niệm thường bị nhầm lẫn.

The goal of this book was outlined in Chapter 1: to explore how to create applications and systems that are reliable, scalable, and maintainable. These themes have run through all of the chapters: for example, we discussed many **fault**-tolerance algorithms that help improve **reliability**, partitioning to improve **scalability**, and mechanisms for evolution and **abstraction** that improve **maintainability**. In this chapter we will bring all of these ideas together, and build on them to envisage the future. Our goal is to discover how to design applications that are better than the ones of today—robust, correct, evolvable, and ultimately beneficial to humanity.

Mục tiêu của cuốn sách này đã được phác ra ở Chương 1: khám phá cách tạo ra những ứng dụng và hệ thống đáng tin cậy, có khả năng mở rộng, và dễ bảo trì. Những chủ đề này đã xuyên suốt mọi chương: chẳng hạn, ta đã bàn về nhiều thuật toán chịu lỗi giúp cải thiện độ tin cậy, về phân mảnh (partitioning) để cải thiện khả năng mở rộng, và về các cơ chế cho sự tiến hóa cùng trừu tượng hóa giúp cải thiện khả năng bảo trì. Trong chương này ta sẽ tập hợp tất cả những ý tưởng đó lại, và dựa trên chúng để hình dung về tương lai. Mục tiêu của ta là khám phá cách thiết kế những ứng dụng tốt hơn so với các ứng dụng ngày nay — vững vàng, chính xác, có khả năng tiến hóa, và tốt cuộc là có ích cho nhân loại.

Data Integration — Tích hợp dữ liệu

A recurring theme in this book has been that for any given problem, there are several solutions, all of which have different pros, cons, and trade-offs. For example, when discussing storage engines in Chapter 3, we saw log-structured storage, B-trees, and **column**-oriented storage. When discussing replication in Chapter 5, we saw single-leader, multi-leader, and leaderless approaches.

Một chủ đề lặp đi lặp lại trong cuốn sách này là: với bất kỳ bài toán cho trước nào, luôn có vài lời giải, mỗi lời giải có những ưu điểm, nhược điểm và đánh đổi khác nhau. Chẳng hạn, khi bàn về các engine lưu trữ ở Chương 3, ta đã thấy lưu trữ dạng log (log-structured), cây B (B-tree), và lưu trữ hướng cột (column-oriented). Khi bàn về sao chép (replication) ở Chương 5, ta đã thấy các cách tiếp cận đơn-leader (single-leader), đa-leader (multi-leader), và không-leader (leaderless).

If you have a problem such as “I want to store some data and look it up again later,” there is no one right solution, but many different approaches that are each appropriate in different circumstances. A software implementation typically has to pick one particular approach. It’s hard enough to get one code path robust and performing well—trying to do everything in one piece of software almost guarantees that the implementation will be poor.

Nếu bạn gặp một bài toán kiểu như “tôi muốn lưu một ít dữ liệu rồi tra cứu lại sau”, thì không có một lời giải đúng duy nhất, mà có nhiều cách tiếp cận khác nhau, mỗi cách phù hợp với những hoàn cảnh khác nhau. Một bản hiện thực phần mềm thường phải chọn lấy một cách tiếp cận cụ thể. Đã đủ khó để làm cho một đường thực thi (code path) vững vàng và chạy tốt — cố làm mọi thứ trong một phần mềm gần như chắc chắn dẫn đến một bản hiện thực kém cỏi.

Thus, the most appropriate choice of software tool also depends on the circumstances. Every piece of software, even a so-called “general-purpose” **database**, is designed for a particular usage pattern.

Do đó, lựa chọn công cụ phần mềm phù hợp nhất cũng tùy thuộc vào hoàn cảnh. Mỗi phần mềm, kể cả một cơ sở dữ liệu được gọi là “đa dụng”, đều được thiết kế cho một mẫu sử dụng cụ thể.

Faced with this profusion of alternatives, the first challenge is then to figure out the mapping between the software products and the circumstances in which they are a good fit. Vendors are understandably reluctant to tell you about the kinds of workloads for which their software is poorly suited, but hopefully the previous chapters have equipped you with some questions to ask in order to read between the lines and better understand the trade-offs.

Đối diện với muôn vàn lựa chọn này, thách thức đầu tiên là tìm ra ánh xạ giữa các sản phẩm phần mềm và những hoàn cảnh mà chúng phù hợp. Dễ hiểu là các nhà cung cấp ngần ngại nói cho bạn biết những loại khối lượng công việc (workload) mà phần mềm của họ tỏ ra kém phù hợp, nhưng hy vọng các chương trước đã trang bị cho bạn vài câu hỏi

để đọc giữa những dòng chữ và hiểu rõ hơn các đánh đổi.

However, even if you perfectly understand the mapping between tools and circumstances for their use, there is another challenge: in complex applications, data is often used in several different ways. There is unlikely to be one piece of software that is suitable for all the different circumstances in which the data is used, so you inevitably end up having to cobble together several different pieces of software in order to provide your application's functionality.

Tuy nhiên, ngay cả khi bạn đã hiểu hoàn hảo ánh xạ giữa công cụ và hoàn cảnh sử dụng, vẫn còn một thách thức khác: trong các ứng dụng phức tạp, dữ liệu thường được dùng theo nhiều cách khác nhau. Khó có một phần mềm nào phù hợp với mọi hoàn cảnh khác nhau mà dữ liệu được dùng đến, nên bạn không tránh khỏi việc phải ghép vá vài phần mềm khác nhau lại với nhau để cung cấp chức năng cho ứng dụng của mình.

Combining Specialized Tools by Deriving Data — Kết hợp các công cụ chuyên biệt bằng cách dẫn xuất dữ liệu

For example, it is common to need to integrate an OLTP database with a **full-text search index** in order to handle queries for arbitrary keywords. Although some databases (such as PostgreSQL) include a full-text indexing feature, which can be sufficient for simple applications [1], more sophisticated search facilities require specialist information retrieval tools. Conversely, search indexes are generally not very suitable as a durable system of record, and so many applications need to combine two different tools in order to satisfy all of the requirements.

Chẳng hạn, ta thường cần tích hợp một cơ sở dữ liệu OLTP với một chỉ mục tìm kiếm toàn văn để xử lý các truy vấn theo từ khóa bất kỳ. Mặc dù một số cơ sở dữ liệu (như PostgreSQL) có sẵn tính năng lập chỉ mục toàn văn, đủ dùng cho các ứng dụng đơn giản [1], những tiện ích tìm kiếm tinh vi hơn lại đòi hỏi các công cụ truy hồi thông tin chuyên dụng. Ngược lại, chỉ mục tìm kiếm nói chung không mấy phù hợp để làm một hệ thống lưu hồ sơ (system of record) bền vững, nên nhiều ứng dụng cần kết hợp hai công cụ khác nhau để thỏa mãn mọi yêu cầu.

We touched on the issue of integrating data systems in “Keeping Systems in Sync”. As the number of different representations of the data increases, the integration problem becomes harder. Besides the database and the **search index**, perhaps you need to keep copies of the data in analytics systems (data warehouses, or batch and **stream processing** systems); maintain caches or denormalized versions of objects that were derived from the original data; pass the data through machine learning, classification, ranking, or recommendation systems; or send notifications based on changes to the data.

Ta đã chạm tới vấn đề tích hợp các hệ thống dữ liệu ở phần “Giữ các hệ thống đồng bộ”. Khi số lượng cách biểu diễn khác nhau của dữ liệu tăng lên, bài toán tích hợp càng khó hơn. Ngoài cơ sở dữ liệu và chỉ mục tìm kiếm, có lẽ bạn còn cần giữ các bản sao dữ liệu trong các hệ thống phân tích (kho dữ liệu, hoặc các hệ thống xử lý theo lô và xử lý luồng); duy trì các bộ nhớ đệm hoặc các phiên bản phi chuẩn hóa của những đối tượng được dẫn xuất từ dữ liệu gốc; đưa dữ liệu qua các hệ thống học máy, phân loại, xếp hạng, hoặc gợi ý; hoặc gửi thông báo dựa trên những thay đổi của dữ liệu.

Surprisingly often I see software engineers make statements like, “In my experience, 99% of people only need X” or “...don’t need X” (for various values of X). I think that such statements say more about the experience of the speaker than about the actual usefulness of a technology. The range of different things you might want to do with data is dizzyingly wide. What one person considers to be

an obscure and pointless feature may well be a central requirement for someone else. The need for data integration often only becomes apparent if you zoom out and consider the dataflows across an entire organization.

Đáng ngạc nhiên là tôi thường thấy các kỹ sư phần mềm phát biểu kiểu như “theo kinh nghiệm của tôi, 99% mọi người chỉ cần X” hoặc “...không cần X” (với đủ loại giá trị của X). Tôi cho rằng những phát biểu như vậy nói lên nhiều điều về kinh nghiệm của người nói hơn là về tính hữu dụng thực sự của một công nghệ. Phạm vi những việc khác nhau mà bạn có thể muốn làm với dữ liệu rộng đến chóng mặt. Điều mà người này coi là một tính năng tối nghĩa và vô dụng có thể chính là một yêu cầu cốt lõi đối với người khác. Nhu cầu tích hợp dữ liệu thường chỉ trở nên rõ ràng nếu bạn lùi lại và xét các luồng dữ liệu trải khắp toàn bộ một tổ chức.

Reasoning about dataflows — Suy luận về các luồng dữ liệu

When copies of the same data need to be maintained in several storage systems in order to satisfy different access patterns, you need to be very clear about the inputs and outputs: where is data written first, and which representations are derived from which sources? How do you get data into all the right places, in the right formats?

Khi cần duy trì các bản sao của cùng một dữ liệu trong vài hệ thống lưu trữ để thỏa mãn các mẫu truy cập khác nhau, bạn cần hết sức rõ ràng về đầu vào và đầu ra: dữ liệu được ghi đầu tiên ở đâu, và những cách biểu diễn nào được dẫn xuất từ những nguồn nào? Làm sao để đưa dữ liệu đến đúng mọi nơi, với đúng định dạng?

For example, you might arrange for data to first be written to a system of record database, capturing the changes made to that database (see “Change Data Capture”) and then applying the changes to the search index in the same order. If change data capture (CDC) is the only way of updating the index, you can be confident that the index is entirely derived from the system of record, and therefore consistent with it (barring bugs in the software). Writing to the database is the only way of supplying new input into this system.

Chẳng hạn, bạn có thể sắp đặt để dữ liệu được ghi đầu tiên vào một cơ sở dữ liệu lưu hồ sơ, nắm bắt những thay đổi với cơ sở dữ liệu đó (xem “Nắm bắt thay đổi dữ liệu”) rồi áp dụng các thay đổi đó vào chỉ mục tìm kiếm theo cùng một thứ tự. Nếu nắm bắt thay đổi dữ liệu (change data capture, CDC) là cách duy nhất để cập nhật chỉ mục, bạn có thể yên tâm rằng chỉ mục được dẫn xuất hoàn toàn từ hệ thống lưu hồ sơ, và do đó nhất quán với nó (trừ phi có bug trong phần mềm). Ghi vào cơ sở dữ liệu là cách duy nhất để cung cấp đầu vào mới cho hệ thống này.

Allowing the application to directly write to both the search index and the database introduces the problem shown in Figure 11-4, in which two clients concurrently send conflicting writes, and the two storage systems process them in a different order. In this case, neither the database nor the search index is “in charge” of determining the order of writes, and so they may make contradictory decisions and become permanently inconsistent with each other.

Cho phép ứng dụng ghi trực tiếp vào cả chỉ mục tìm kiếm lẫn cơ sở dữ liệu sẽ gây ra vấn đề minh họa ở Hình 11-4, trong đó hai client đồng thời gửi các lệnh ghi mâu thuẫn nhau, và hai hệ thống lưu trữ xử lý chúng theo thứ tự khác nhau. Trong trường hợp này, cả cơ sở dữ liệu lẫn chỉ mục tìm kiếm đều không “nắm quyền” quyết định thứ tự ghi, nên chúng có thể đưa ra những quyết định mâu thuẫn nhau và trở nên bất nhất với nhau vĩnh viễn.

If it is possible for you to funnel all user input through a single system that decides on an ordering for all writes, it becomes much easier to derive other representations of the data by processing the

writes in the same order. This is an application of the state machine replication approach that we saw in “Total Order Broadcast”. Whether you use change data capture or an event sourcing log is less important than simply the principle of deciding on a total order.

Nếu có thể dồn mọi đầu vào của người dùng qua một hệ thống duy nhất quyết định thứ tự cho mọi lệnh ghi, thì việc dẫn xuất các cách biểu diễn khác của dữ liệu bằng cách xử lý các lệnh ghi theo cùng thứ tự đó trở nên dễ hơn nhiều. Đây là một ứng dụng của cách tiếp cận sao chép máy trạng thái (state machine replication) mà ta đã thấy ở “Truyền tin theo thứ tự toàn phần”. Việc bạn dùng nắm bắt thay đổi dữ liệu hay một log nguồn-sự-kiện (event sourcing) ít quan trọng hơn so với chính nguyên tắc quyết định một thứ tự toàn phần.

Updating a **derived data** system based on an event log can often be made deterministic and idempotent (see “Idempotence”), making it quite easy to recover from faults.

Việc cập nhật một hệ thống dữ liệu dẫn xuất dựa trên một log sự kiện thường có thể được làm cho mang tính tất định (deterministic) và lũy đẳng (idempotent) (xem “Tính lũy đẳng”), khiến việc phục hồi sau lỗi trở nên khá dễ dàng.

Derived data versus distributed transactions — Dữ liệu dẫn xuất so với giao dịch phân tán

The classic approach for keeping different data systems consistent with each other involves distributed transactions, as discussed in “Atomic Commit and Two-Phase Commit (2PC)”. How does the approach of using derived data systems fare in comparison to distributed transactions?

Cách tiếp cận kinh điển để giữ cho các hệ thống dữ liệu khác nhau nhất quán với nhau là dùng giao dịch phân tán (distributed transaction), như đã bàn ở “Cam kết nguyên tử và Cam kết hai pha (2PC)”. Cách tiếp cận dùng các hệ thống dữ liệu dẫn xuất so sánh ra sao với giao dịch phân tán?

At an abstract level, they achieve a similar goal by different means. Distributed transactions decide on an ordering of writes by using locks for mutual exclusion (see “Two-Phase Locking (2PL)”), while CDC and event sourcing use a log for ordering. Distributed transactions use atomic commit to ensure that changes take effect exactly once, while log-based systems are often based on deterministic retry and idempotence.

Ở mức trừu tượng, chúng đạt được một mục tiêu tương tự bằng những phương tiện khác nhau. Giao dịch phân tán quyết định thứ tự ghi bằng cách dùng khóa (lock) để loại trừ lẫn nhau (xem “Khóa hai pha (2PL)”), trong khi CDC và nguồn-sự-kiện dùng một log để định thứ tự. Giao dịch phân tán dùng cam kết nguyên tử để đảm bảo các thay đổi có hiệu lực đúng một lần, trong khi các hệ thống dựa trên log thường dựa vào việc thử lại mang tính tất định và tính lũy đẳng.

The biggest difference is that transaction systems usually provide linearizability (see “Linearizability”), which implies useful guarantees such as reading your own writes (see “Reading Your Own Writes”). On the other hand, derived data systems are often updated asynchronously, and so they do not by default offer the same timing guarantees.

Khác biệt lớn nhất là các hệ thống giao dịch thường cung cấp tính tuyến tính hóa (linearizability) (xem “Tính tuyến tính hóa”), kéo theo những đảm bảo hữu ích như đọc được chính lệnh ghi của mình (xem “Đọc được chính lệnh ghi của mình”). Mặt khác, các hệ thống dữ liệu dẫn xuất thường được cập nhật bất đồng bộ, nên theo mặc định chúng không cung cấp cùng những đảm bảo về thời điểm.

Within limited environments that are willing to pay the cost of distributed transactions, they have been used successfully. However, I think that XA has poor fault tolerance and performance characteristics (see “Distributed Transactions in Practice”), which severely limit its usefulness. I believe that it might be possible to create a better protocol for distributed transactions, but getting such a protocol widely adopted and integrated with existing tools would be challenging, and unlikely to happen soon.

Trong những môi trường hạn chế sẵn lòng trả cái giá của giao dịch phân tán, chúng đã được dùng thành công. Tuy nhiên, tôi cho rằng XA có các đặc tính chịu lỗi và hiệu năng kém (xem “Giao dịch phân tán trong thực tế”), điều này hạn chế nghiêm trọng tính hữu dụng của nó. Tôi tin rằng có thể tạo ra một giao thức tốt hơn cho giao dịch phân tán, nhưng để một giao thức như vậy được áp dụng rộng rãi và tích hợp với các công cụ hiện có sẽ là một thách thức, và khó xảy ra trong tương lai gần.

In the absence of widespread support for a good distributed transaction protocol, I believe that log-based derived data is the most promising approach for integrating different data systems. However, guarantees such as reading your own writes are useful, and I don’t think that it is productive to tell everyone “eventual consistency is inevitable—suck it up and learn to deal with it” (at least not without good guidance on how to deal with it).

Trong bối cảnh chưa có sự hỗ trợ rộng rãi cho một giao thức giao dịch phân tán tốt, tôi tin rằng dữ liệu dẫn xuất dựa trên log là cách tiếp cận hứa hẹn nhất để tích hợp các hệ thống dữ liệu khác nhau. Tuy nhiên, những đảm bảo như đọc được chính lệnh ghi của mình rất hữu ích, và tôi không nghĩ rằng việc nói với mọi người “nhất quán cuối cùng (eventual consistency) là điều tất yếu — hãy chấp nhận và học cách sống với nó” là hữu ích (ít nhất là không, nếu không kèm theo hướng dẫn tốt về cách sống với nó).

In “Aiming for Correctness” we will discuss some approaches for implementing stronger guarantees on top of asynchronously derived systems, and work toward a middle ground between distributed transactions and asynchronous log-based systems.

Ở phần “Hướng tới sự đúng đắn”, ta sẽ bàn về một số cách tiếp cận để hiện thực những đảm bảo mạnh hơn trên nền các hệ thống được dẫn xuất bất đồng bộ, và hướng tới một điểm trung dung giữa giao dịch phân tán và các hệ thống bất đồng bộ dựa trên log.

The limits of total ordering — Những giới hạn của thứ tự toàn phần

With systems that are small enough, constructing a totally ordered event log is entirely feasible (as demonstrated by the popularity of databases with single-leader replication, which construct precisely such a log). However, as systems are scaled toward bigger and more complex workloads, limitations begin to emerge:

Với các hệ thống đủ nhỏ, việc dựng nên một log sự kiện có thứ tự toàn phần là hoàn toàn khả thi (như sự phổ biến của các cơ sở dữ liệu dùng sao chép đơn-leader đã minh chứng — chúng dựng nên đúng một log như vậy). Tuy nhiên, khi hệ thống được mở rộng tới những khối lượng công việc lớn hơn và phức tạp hơn, các giới hạn bắt đầu lộ ra:

- In most cases, constructing a totally ordered log requires all events to pass through a single leader **node** that decides on the ordering. If the **throughput** of events is greater than a single machine can handle, you need to partition it across multiple machines (see “Partitioned Logs”). The order of events in two different partitions is then ambiguous.
- If the servers are spread across multiple geographically distributed datacenters, for example in order to tolerate an entire **datacenter** going offline, you typically have a separate leader in each datacenter, because network delays make synchronous cross-datacenter coordination

inefficient (see “Multi-Leader Replication”). This implies an undefined ordering of events that originate in two different datacenters.

- When applications are deployed as microservices (see “Dataflow Through Services: REST and RPC”), a common design choice is to deploy each service and its durable state as an independent unit, with no durable state shared between services. When two events originate in different services, there is no defined order for those events.
- Some applications maintain **client**-side state that is updated immediately on user input (without waiting for confirmation from a server), and even continue to work offline (see “Clients with offline operation”). With such applications, clients and servers are very likely to see events in different orders.
 - Trong hầu hết các trường hợp, việc dựng nên một log có thứ tự toàn phần đòi hỏi mọi sự kiện phải đi qua một nút leader duy nhất quyết định thứ tự. Nếu thông lượng sự kiện lớn hơn mức một máy đơn lẻ có thể xử lý, bạn cần phân mảnh nó trên nhiều máy (xem “Log được phân mảnh”). Khi đó thứ tự của các sự kiện ở hai mảnh khác nhau trở nên mơ hồ.
 - Nếu các máy chủ được trải trên nhiều trung tâm dữ liệu phân tán về mặt địa lý, chẳng hạn để chịu được việc cả một trung tâm dữ liệu ngừng hoạt động, bạn thường có một leader riêng ở mỗi trung tâm dữ liệu, bởi độ trễ mạng khiến việc phối hợp đồng bộ xuyên trung tâm dữ liệu trở nên kém hiệu quả (xem “Sao chép đa-leader”). Điều này ngụ ý một thứ tự không xác định cho các sự kiện phát sinh ở hai trung tâm dữ liệu khác nhau.
 - Khi các ứng dụng được triển khai dưới dạng vi dịch vụ (microservices) (xem “Luồng dữ liệu qua dịch vụ: REST và RPC”), một lựa chọn thiết kế phổ biến là triển khai mỗi dịch vụ cùng trạng thái bền vững của nó như một đơn vị độc lập, không chia sẻ trạng thái bền vững giữa các dịch vụ. Khi hai sự kiện phát sinh ở các dịch vụ khác nhau, không có thứ tự xác định nào cho những sự kiện đó.
 - Một số ứng dụng duy trì trạng thái phía client, được cập nhật ngay lập tức khi người dùng nhập liệu (mà không chờ xác nhận từ máy chủ), và thậm chí tiếp tục hoạt động khi ngoại tuyến (xem “Client có khả năng hoạt động ngoại tuyến”). Với những ứng dụng như vậy, rất có khả năng client và máy chủ nhìn thấy các sự kiện theo những thứ tự khác nhau.

In formal terms, deciding on a total order of events is known as total order broadcast, which is equivalent to consensus (see “Consensus algorithms and total order broadcast”). Most consensus algorithms are designed for situations in which the throughput of a single node is sufficient to process the entire stream of events, and these algorithms do not provide a mechanism for multiple nodes to share the work of ordering the events. It is still an open research problem to design consensus algorithms that can scale beyond the throughput of a single node and that work well in a geographically distributed setting.

Theo thuật ngữ hình thức, việc quyết định một thứ tự toàn phần của các sự kiện được gọi là truyền tin theo thứ tự toàn phần (total order broadcast), vốn tương đương với đồng thuận (consensus) (xem “Thuật toán đồng thuận và truyền tin theo thứ tự toàn phần”). Hầu hết các thuật toán đồng thuận được thiết kế cho những tình huống mà thông lượng của một nút đơn lẻ đủ để xử lý toàn bộ luồng sự kiện, và những thuật toán này không cung cấp cơ chế cho nhiều nút cùng chia sẻ công việc định thứ tự cho các sự kiện. Việc thiết kế các thuật toán đồng thuận có thể mở rộng vượt quá thông lượng của một nút đơn lẻ và hoạt động tốt trong một bối cảnh phân tán về mặt địa lý vẫn là một bài toán nghiên cứu còn bỏ ngỏ.

Ordering events to capture causality — Định thứ tự sự kiện để nắm bắt quan hệ nhân quả

In cases where there is no causal link between events, the lack of a total order is not a big problem, since concurrent events can be ordered arbitrarily. Some other cases are easy to handle: for example, when there are multiple updates of the same object, they can be totally ordered by routing all updates for a particular object ID to the same log partition. However, causal dependencies sometimes arise in more subtle ways (see also “Ordering and Causality”).

Trong các trường hợp không có liên hệ nhân quả giữa các sự kiện, việc thiếu một thứ tự toàn phần không phải vấn đề lớn, vì các sự kiện đồng thời có thể được định thứ tự tùy ý. Một số trường hợp khác thì dễ xử lý: chẳng hạn, khi có nhiều lần cập nhật cùng một đối tượng, chúng có thể được định thứ tự toàn phần bằng cách định tuyến mọi lần cập nhật cho một ID đối tượng cụ thể tới cùng một mảnh log. Tuy nhiên, các phụ thuộc nhân quả đôi khi nảy sinh theo những cách tinh vi hơn (xem thêm “Thứ tự và quan hệ nhân quả”).

For example, consider a social networking service, and two users who were in a relationship but have just broken up. One of the users removes the other as a friend, and then sends a message to their remaining friends complaining about their ex-partner. The user’s intention is that their ex-partner should not see the rude message, since the message was sent after the friend status was revoked.

Chẳng hạn, hãy xét một dịch vụ mạng xã hội, và hai người dùng từng có quan hệ tình cảm nhưng vừa chia tay. Một trong hai người hủy kết bạn với người kia, rồi gửi một thông điệp tới những người bạn còn lại để than phiền về người yêu cũ. Ý định của người dùng là người yêu cũ không nên nhìn thấy thông điệp khiếm nhã đó, vì thông điệp được gửi sau khi trạng thái bạn bè đã bị hủy.

However, in a system that stores friendship status in one place and messages in another place, that ordering dependency between the unfriend event and the message-send event may be lost. If the causal dependency is not captured, a service that sends notifications about new messages may process the message-send event before the unfriend event, and thus incorrectly send a notification to the ex-partner.

Tuy nhiên, trong một hệ thống lưu trạng thái bạn bè ở một chỗ và lưu thông điệp ở một chỗ khác, sự phụ thuộc về thứ tự giữa sự kiện hủy kết bạn và sự kiện gửi thông điệp đó có thể bị mất. Nếu phụ thuộc nhân quả không được nắm bắt, một dịch vụ gửi thông báo về thông điệp mới có thể xử lý sự kiện gửi thông điệp trước sự kiện hủy kết bạn, và do đó gửi nhầm thông báo tới người yêu cũ.

In this example, the notifications are effectively a **join** between the messages and the friend list, making it related to the timing issues of joins that we discussed previously (see “Time-dependence of joins”). Unfortunately, there does not seem to be a simple answer to this problem [2, 3]. Starting points include:

Trong ví dụ này, các thông báo thực chất là một phép join giữa các thông điệp và danh sách bạn bè, khiến nó liên quan tới các vấn đề về thời điểm của phép join mà ta đã bàn trước đó (xem “Tính phụ thuộc thời điểm của phép join”). Đáng tiếc, dường như không có một câu trả lời đơn giản cho vấn đề này [2, 3]. Các điểm khởi đầu bao gồm:

- Logical timestamps can provide total ordering without coordination (see “Sequence Number Ordering”), so they may help in cases where total order broadcast is not feasible. However, they still require recipients to handle events that are delivered out of order, and they require additional metadata to be passed around.
- If you can log an event to record the state of the system that the user saw before making a decision, and give that event a unique identifier, then any later events can reference that event

identifier in order to record the causal dependency [4]. We will return to this idea in “Reads are events too”.

- Conflict resolution algorithms (see “Automatic Conflict Resolution”) help with processing events that are delivered in an unexpected order. They are useful for maintaining state, but they do not help if actions have external side effects (such as sending a notification to a user).
 - Dấu thời gian logic (logical timestamp) có thể cung cấp thứ tự toàn phần mà không cần phối hợp (xem “Định thứ tự bằng số tuần tự”), nên chúng có thể giúp ích trong những trường hợp truyền tin theo thứ tự toàn phần không khả thi. Tuy nhiên, chúng vẫn đòi hỏi bên nhận phải xử lý các sự kiện được giao đến không đúng thứ tự, và đòi hỏi truyền thêm siêu dữ liệu (metadata) đi kèm.
 - Nếu bạn có thể ghi log một sự kiện để lưu lại trạng thái của hệ thống mà người dùng đã thấy trước khi ra quyết định, và gán cho sự kiện đó một định danh duy nhất, thì bất kỳ sự kiện sau này nào cũng có thể tham chiếu tới định danh sự kiện đó để ghi nhận phụ thuộc nhân quả [4]. Ta sẽ quay lại ý tưởng này ở “Lệnh đọc cũng là sự kiện”.
 - Các thuật toán giải quyết xung đột (xem “Tự động giải quyết xung đột”) giúp xử lý các sự kiện được giao đến theo thứ tự không mong đợi. Chúng hữu ích cho việc duy trì trạng thái, nhưng không giúp ích nếu hành động có tác dụng phụ ra bên ngoài (chẳng hạn gửi thông báo tới người dùng).

Perhaps, over time, patterns for application development will emerge that allow causal dependencies to be captured efficiently, and derived state to be maintained correctly, without forcing all events to go through the bottleneck of total order broadcast.

Có lẽ, theo thời gian, sẽ xuất hiện những mẫu phát triển ứng dụng cho phép nắm bắt các phụ thuộc nhân quả một cách hiệu quả, và duy trì trạng thái dẫn xuất một cách đúng đắn, mà không buộc mọi sự kiện phải đi qua nút thắt cổ chai là truyền tin theo thứ tự toàn phần.

Batch and Stream Processing — Xử lý theo lô và xử lý luồng

I would say that the goal of data integration is to make sure that data ends up in the right form in all the right places. Doing so requires consuming inputs, transforming, joining, filtering, aggregating, training models, evaluating, and eventually writing to the appropriate outputs. Batch and stream processors are the tools for achieving this goal.

Tôi sẽ nói rằng mục tiêu của tích hợp dữ liệu là đảm bảo dữ liệu rốt cuộc nằm ở đúng dạng tại đúng mọi nơi. Để làm được điều đó cần tiêu thụ các đầu vào, biến đổi, join, lọc, tổng hợp, huấn luyện mô hình, đánh giá, và cuối cùng ghi ra các đầu ra phù hợp. Bộ xử lý theo lô và bộ xử lý luồng là những công cụ để đạt được mục tiêu này.

The outputs of batch and stream processes are derived datasets such as search indexes, materialized views, recommendations to show to users, aggregate metrics, and so on (see “The Output of Batch Workflows” and “Uses of Stream Processing”).

Đầu ra của các tiến trình xử lý theo lô và xử lý luồng là các tập dữ liệu dẫn xuất như chỉ mục tìm kiếm, khung nhìn vật chất hóa (materialized view), các gợi ý để hiển thị cho người dùng, các chỉ số tổng hợp, v.v. (xem “Đầu ra của các luồng công việc theo lô” và “Các ứng dụng của xử lý luồng”).

As we saw in Chapter 10 and Chapter 11, batch and stream processing have a lot of principles in common, and the main fundamental difference is that stream processors operate on unbounded datasets whereas batch process inputs are of a known, finite size. There are also many detailed

differences in the ways the processing engines are implemented, but these distinctions are beginning to blur.

Như ta đã thấy ở Chương 10 và Chương 11, xử lý theo lô và xử lý luồng có nhiều nguyên tắc chung, và khác biệt căn bản chủ yếu là bộ xử lý luồng làm việc trên các tập dữ liệu không bị chặn (unbounded), trong khi đầu vào của tiến trình theo lô có kích thước hữu hạn đã biết. Cũng có nhiều khác biệt chi tiết trong cách hiện thực các engine xử lý, nhưng những phân biệt này đang bắt đầu mờ đi.

Spark performs stream processing on top of a **batch processing engine** by breaking the stream into microbatches, whereas Apache Flink performs batch processing on top of a stream processing engine [5]. In principle, one type of processing can be emulated on top of the other, although the performance characteristics vary: for example, microbatching may perform poorly on hopping or sliding windows [6].

Spark thực hiện xử lý luồng trên nền một engine xử lý theo lô bằng cách chia luồng thành các vi-lô (microbatch), trong khi Apache Flink thực hiện xử lý theo lô trên nền một engine xử lý luồng [5]. Về nguyên tắc, một loại xử lý có thể được mô phỏng trên nền loại kia, mặc dù đặc tính hiệu năng có khác biệt: chẳng hạn, vi-lô có thể chạy kém với cửa sổ nhảy (hopping) hoặc cửa sổ trượt (sliding) [6].

Maintaining derived state — Duy trì trạng thái dẫn xuất

Batch processing has a quite strong functional flavor (even if the code is not written in a functional programming language): it encourages deterministic, pure functions whose output depends only on the input and which have no side effects other than the explicit outputs, treating inputs as immutable and outputs as append-only. Stream processing is similar, but it extends operators to allow managed, **fault-tolerant** state (see “Rebuilding state after a **failure**”).

Xử lý theo lô mang hơi hướng hàm (functional) khá mạnh (kể cả khi mã không được viết bằng một ngôn ngữ lập trình hàm): nó khuyến khích các hàm thuần khiết, tất định, có đầu ra chỉ phụ thuộc vào đầu vào và không có tác dụng phụ nào ngoài các đầu ra tường minh, coi đầu vào là bất biến và đầu ra là chỉ-thêm (append-only). Xử lý luồng cũng tương tự, nhưng nó mở rộng các toán tử để cho phép trạng thái được quản lý và chịu lỗi (xem “Tái dựng trạng thái sau lỗi”).

The principle of deterministic functions with well-defined inputs and outputs is not only good for fault tolerance (see “Idempotence”), but also simplifies reasoning about the dataflows in an organization [7]. No matter whether the derived data is a search index, a statistical model, or a **cache**, it is helpful to think in terms of data pipelines that derive one thing from another, pushing state changes in one system through functional **application code** and applying the effects to derived systems.

Nguyên tắc dùng các hàm tất định với đầu vào và đầu ra được định nghĩa rõ ràng không chỉ tốt cho khả năng chịu lỗi (xem “Tính lũy đẳng”), mà còn đơn giản hóa việc suy luận về các luồng dữ liệu trong một tổ chức [7]. Bất kể dữ liệu dẫn xuất là một chỉ mục tìm kiếm, một mô hình thống kê, hay một bộ nhớ đệm, đều hữu ích khi nghĩ theo lối các đường ống dữ liệu (data pipeline) dẫn xuất thứ này từ thứ khác, đẩy các thay đổi trạng thái trong một hệ thống đi qua mã ứng dụng mang tính hàm và áp dụng các tác động đó lên các hệ thống dẫn xuất.

In principle, derived data systems could be maintained synchronously, just like a **relational database** updates secondary indexes synchronously within the same transaction as writes to the **table** being indexed. However, asynchrony is what makes systems based on event logs robust: it

allows a fault in one part of the system to be contained locally, whereas distributed transactions abort if any one participant fails, so they tend to amplify failures by spreading them to the rest of the system (see “Limitations of distributed transactions”).

Về nguyên tắc, các hệ thống dữ liệu dẫn xuất có thể được duy trì một cách đồng bộ, giống như một cơ sở dữ liệu quan hệ cập nhật các chỉ mục phụ một cách đồng bộ trong cùng giao dịch với các lệnh ghi vào bảng đang được lập chỉ mục. Tuy nhiên, chính tính bất đồng bộ mới là thứ làm cho các hệ thống dựa trên log sự kiện trở nên vững vàng: nó cho phép một lỗi ở một phần của hệ thống được khu trú tại chỗ, trong khi giao dịch phân tán sẽ hủy bỏ (abort) nếu bất kỳ một bên tham gia nào hỏng, nên chúng có xu hướng khuếch đại sự cố bằng cách lan nó ra phần còn lại của hệ thống (xem “Những giới hạn của giao dịch phân tán”).

We saw in “Partitioning and Secondary Indexes” that secondary indexes often cross partition boundaries. A partitioned system with secondary indexes either needs to send writes to multiple partitions (if the index is term-partitioned) or send reads to all partitions (if the index is **document-partitioned**). Such cross-partition communication is also most reliable and scalable if the index is maintained asynchronously [8] (see also “Multi-partition data processing”).

Ta đã thấy ở “Phân mảnh và chỉ mục phụ” rằng các chỉ mục phụ thường vượt qua ranh giới mảnh. Một hệ thống được phân mảnh có chỉ mục phụ hoặc phải gửi các lệnh ghi tới nhiều mảnh (nếu chỉ mục được phân mảnh theo từ khóa — term-partitioned), hoặc phải gửi các lệnh đọc tới mọi mảnh (nếu chỉ mục được phân mảnh theo tài liệu — document-partitioned). Việc giao tiếp xuyên mảnh như vậy cũng đáng tin cậy và dễ mở rộng nhất nếu chỉ mục được duy trì bất đồng bộ [8] (xem thêm “Xử lý dữ liệu đa-mảnh”).

Reprocessing data for application evolution — Xử lý lại dữ liệu cho sự tiến hóa của ứng dụng

When maintaining derived data, batch and stream processing are both useful. Stream processing allows changes in the input to be reflected in derived views with low delay, whereas batch processing allows large amounts of accumulated historical data to be reprocessed in order to derive new views onto an existing dataset.

Khi duy trì dữ liệu dẫn xuất, cả xử lý theo lô lẫn xử lý luồng đều hữu ích. Xử lý luồng cho phép các thay đổi ở đầu vào được phản ánh vào các khung nhìn dẫn xuất với độ trễ thấp, trong khi xử lý theo lô cho phép xử lý lại một lượng lớn dữ liệu lịch sử đã tích lũy để dẫn xuất các khung nhìn mới trên một tập dữ liệu hiện có.

In particular, reprocessing existing data provides a good mechanism for maintaining a system, evolving it to support new features and changed requirements (see Chapter 4). Without reprocessing, **schema evolution** is limited to simple changes like adding a new optional field to a record, or adding a new type of record. This is the case both in a **schema-on-write** and in a **schema-on-read** context (see “**Schema flexibility** in the **document model**”). On the other hand, with reprocessing it is possible to restructure a dataset into a completely different model in order to better serve new requirements.

Cụ thể, việc xử lý lại dữ liệu hiện có cung cấp một cơ chế tốt để bảo trì một hệ thống, để nó tiến hóa nhằm hỗ trợ các tính năng mới và những yêu cầu thay đổi (xem Chương 4). Không có việc xử lý lại, tiến hóa lược đồ bị giới hạn ở những thay đổi đơn giản như thêm một trường tùy chọn mới vào một bản ghi, hoặc thêm một loại bản ghi mới. Điều này đúng cả trong bối cảnh lược đồ-lúc-ghi (schema-on-write) lẫn lược đồ-lúc-đọc (schema-on-read) (xem “Tính linh hoạt lược đồ trong mô hình tài liệu”). Ngược lại, với việc xử lý lại, ta có

thể tái cấu trúc một tập dữ liệu thành một mô hình hoàn toàn khác để phục vụ tốt hơn các yêu cầu mới.

Schema Migrations on Railways — Di trú lược đồ trên đường ray Large-scale “**schema migrations**” occur in noncomputer systems as well. For example, in the early days of railway building in 19th-century England there were various competing standards for the gauge (the distance between the two rails). Trains built for one gauge couldn’t run on tracks of another gauge, which restricted the possible interconnections in the train network [9].

Các cuộc “di trú lược đồ” quy mô lớn cũng xảy ra trong các hệ thống phi máy tính. Chẳng hạn, vào những ngày đầu xây dựng đường sắt ở nước Anh thế kỷ 19, có nhiều chuẩn cạnh tranh nhau về khổ đường ray (gauge — khoảng cách giữa hai thanh ray). Các đoàn tàu được chế tạo cho một khổ ray không thể chạy trên đường ray có khổ khác, điều này hạn chế khả năng kết nối trong mạng lưới đường sắt [9].

After a single standard gauge was finally decided upon in 1846, tracks with other gauges had to be converted—but how do you do this without shutting down the train line for months or years? The solution is to first convert the track to dual gauge or mixed gauge by adding a third rail. This conversion can be done gradually, and when it is done, trains of both gauges can run on the line, using two of the three rails. Eventually, once all trains have been converted to the standard gauge, the rail providing the nonstandard gauge can be removed.

Sau khi một khổ ray chuẩn duy nhất cuối cùng được chốt vào năm 1846, các đường ray có khổ khác phải được chuyển đổi — nhưng làm sao bạn làm việc này mà không phải đóng cửa tuyến đường suốt nhiều tháng hay nhiều năm? Giải pháp là trước tiên chuyển đường ray thành khổ kép hoặc khổ hỗn hợp bằng cách thêm một thanh ray thứ ba. Việc chuyển đổi này có thể được làm dần dần, và khi hoàn tất, các đoàn tàu của cả hai khổ đều có thể chạy trên tuyến, dùng hai trong ba thanh ray. Cuối cùng, một khi tất cả các đoàn tàu đã được chuyển sang khổ chuẩn, thanh ray cung cấp khổ phi chuẩn có thể được tháo bỏ.

“Reprocessing” the existing tracks in this way, and allowing the old and new versions to exist side by side, makes it possible to change the gauge gradually over the course of years. Nevertheless, it is an expensive undertaking, which is why nonstandard gauges still exist today. For example, the BART system in the San Francisco Bay Area uses a different gauge from the majority of the US.

“Xử lý lại” các đường ray hiện có theo cách này, và cho phép phiên bản cũ và mới cùng tồn tại song song, giúp có thể thay đổi khổ ray một cách dần dần qua nhiều năm. Tuy vậy, đây là một công việc tốn kém, đó là lý do các khổ ray phi chuẩn vẫn còn tồn tại đến ngày nay. Chẳng hạn, hệ thống BART ở Vùng vịnh San Francisco dùng một khổ ray khác với phần lớn nước Mỹ.

Derived views allow gradual evolution. If you want to restructure a dataset, you do not need to perform the **migration** as a sudden switch. Instead, you can maintain the old schema and the new schema side by side as two independently derived views onto the same underlying data. You can then start shifting a small number of users to the new view in order to test its performance and find any bugs, while most users continue to be routed to the old view. Gradually, you can increase the proportion of users accessing the new view, and eventually you can drop the old view [10].

Các khung nhìn dẫn xuất cho phép tiến hóa dần dần. Nếu bạn muốn tái cấu trúc một tập dữ liệu, bạn không cần thực hiện việc di trú như một cú chuyển đột ngột. Thay vào đó, bạn có thể duy trì lược đồ cũ và lược đồ mới song song với nhau như hai khung nhìn được dẫn xuất độc lập trên cùng dữ liệu nền. Sau đó bạn có thể bắt đầu chuyển một số nhỏ người dùng sang khung nhìn mới để thử nghiệm hiệu năng và tìm bug, trong khi phần lớn người

dùng vẫn được định tuyến tới khung nhìn cũ. Dần dần, bạn có thể tăng tỷ lệ người dùng truy cập khung nhìn mới, và cuối cùng bạn có thể bỏ khung nhìn cũ [10].

The beauty of such a gradual migration is that every stage of the process is easily reversible if something goes wrong: you always have a working system to go back to. By reducing the risk of irreversible damage, you can be more confident about going ahead, and thus move faster to improve your system [11].

Vẻ đẹp của một cuộc di trú dần dần như vậy là mọi giai đoạn của quá trình đều dễ dàng đảo ngược nếu có gì đó trục trặc: bạn luôn có một hệ thống đang hoạt động để quay về. Bằng cách giảm rủi ro thiệt hại không thể đảo ngược, bạn có thể tự tin hơn khi tiến tới, và do đó di chuyển nhanh hơn để cải thiện hệ thống của mình [11].

The lambda architecture — Kiến trúc lambda

If batch processing is used to reprocess historical data, and stream processing is used to process recent updates, then how do you combine the two? The lambda architecture [12] is a proposal in this area that has gained a lot of attention.

Nếu xử lý theo lô được dùng để xử lý lại dữ liệu lịch sử, còn xử lý luồng được dùng để xử lý các cập nhật gần đây, thì làm sao bạn kết hợp cả hai? Kiến trúc lambda [12] là một đề xuất trong lĩnh vực này đã thu hút rất nhiều sự chú ý.

The core idea of the lambda architecture is that incoming data should be recorded by appending immutable events to an always-growing dataset, similarly to event sourcing (see “Event Sourcing”). From these events, read-optimized views are derived. The lambda architecture proposes running two different systems in parallel: a batch processing system such as Hadoop **MapReduce**, and a separate stream-processing system such as Storm.

Ý tưởng cốt lõi của kiến trúc lambda là dữ liệu đến nên được ghi lại bằng cách thêm các sự kiện bất biến vào một tập dữ liệu luôn lớn dần, tương tự nguồn-sự-kiện (xem “Nguồn-sự-kiện”). Từ những sự kiện này, các khung nhìn được tối ưu cho việc đọc được dẫn xuất. Kiến trúc lambda đề xuất chạy hai hệ thống khác nhau song song: một hệ thống xử lý theo lô như Hadoop MapReduce, và một hệ thống xử lý luồng riêng biệt như Storm.

In the lambda approach, the stream processor consumes the events and quickly produces an approximate update to the view; the batch processor later consumes the same set of events and produces a corrected version of the derived view. The reasoning behind this design is that batch processing is simpler and thus less prone to bugs, while stream processors are thought to be less reliable and harder to make fault-tolerant (see “Fault Tolerance”). Moreover, the stream process can use fast approximate algorithms while the batch process uses slower exact algorithms.

Trong cách tiếp cận lambda, bộ xử lý luồng tiêu thụ các sự kiện và nhanh chóng tạo ra một bản cập nhật gần đúng cho khung nhìn; bộ xử lý theo lô về sau tiêu thụ cùng tập sự kiện đó và tạo ra một phiên bản đã hiệu chỉnh của khung nhìn dẫn xuất. Lý lẽ đằng sau thiết kế này là xử lý theo lô đơn giản hơn nên ít sinh bug hơn, trong khi bộ xử lý luồng được cho là kém tin cậy hơn và khó làm cho chịu lỗi hơn (xem “Khả năng chịu lỗi”). Hơn nữa, tiến trình luồng có thể dùng các thuật toán gần đúng nhanh, trong khi tiến trình theo lô dùng các thuật toán chính xác chậm hơn.

The lambda architecture was an influential idea that shaped the design of data systems for the better, particularly by popularizing the principle of deriving views onto streams of immutable events and reprocessing events when needed. However, I also think that it has a number of practical problems:

Kiến trúc lambda là một ý tưởng có sức ảnh hưởng, đã định hình thiết kế các hệ thống dữ liệu theo hướng tốt hơn, đặc biệt bằng cách phổ biến nguyên tắc dẫn xuất các khung nhìn trên các luồng sự kiện bất biến và xử lý lại sự kiện khi cần. Tuy nhiên, tôi cũng cho rằng nó có một số vấn đề thực tế:

- Having to maintain the same logic to run both in a batch and in a stream processing framework is significant additional effort. Although libraries such as Summingbird [13] provide an abstraction for computations that can be run in either a batch or a streaming context, the operational complexity of debugging, tuning, and maintaining two different systems remains [14].
- Since the stream pipeline and the batch pipeline produce separate outputs, they need to be merged in order to respond to user requests. This merge is fairly easy if the computation is a simple aggregation over a tumbling window, but it becomes significantly harder if the view is derived using more complex operations such as joins and sessionization, or if the output is not a time series.
- Although it is great to have the ability to reprocess the entire historical dataset, doing so frequently is expensive on large datasets. Thus, the batch pipeline often needs to be set up to process incremental batches (e.g., an hour's worth of data at the end of every hour) rather than reprocessing everything. This raises the problems discussed in “Reasoning About Time”, such as handling stragglers and handling windows that cross boundaries between batches. Incrementalizing a batch computation adds complexity, making it more akin to the streaming layer, which runs counter to the goal of keeping the batch layer as simple as possible.
 - Phải duy trì cùng một logic để chạy trên cả một framework xử lý theo lô lẫn một framework xử lý luồng là một nỗ lực bổ sung đáng kể. Mặc dù các thư viện như Summingbird [13] cung cấp một lớp trừu tượng cho các phép tính có thể chạy trong bối cảnh theo lô hoặc luồng, độ phức tạp vận hành của việc gỡ lỗi, tinh chỉnh, và bảo trì hai hệ thống khác nhau vẫn còn đó [14].
 - Vì đường ống luồng và đường ống theo lô tạo ra các đầu ra riêng biệt, chúng cần được trộn lại để đáp ứng các yêu cầu của người dùng. Việc trộn này khá dễ nếu phép tính là một phép tổng hợp đơn giản trên một cửa sổ nhảy theo khối (tumbling window), nhưng trở nên khó hơn đáng kể nếu khung nhìn được dẫn xuất bằng các thao tác phức tạp hơn như phép join và phân phiên (sessionization), hoặc nếu đầu ra không phải là một chuỗi thời gian.
 - Mặc dù khả năng xử lý lại toàn bộ tập dữ liệu lịch sử là rất tuyệt, nhưng làm vậy thường xuyên thì tốn kém trên các tập dữ liệu lớn. Do đó, đường ống theo lô thường cần được thiết lập để xử lý các lô tăng dần (incremental batch) (ví dụ, lượng dữ liệu của một giờ vào cuối mỗi giờ) thay vì xử lý lại mọi thứ. Điều này làm nảy sinh các vấn đề đã bàn ở “Suy luận về thời gian”, chẳng hạn xử lý các sự kiện đến trễ (straggler) và xử lý các cửa sổ vắt ngang ranh giới giữa các lô. Việc làm cho một phép tính theo lô trở nên tăng dần sẽ tăng độ phức tạp, khiến nó giống với tăng luồng hơn, điều này đi ngược lại mục tiêu giữ cho tăng theo lô càng đơn giản càng tốt.

Unifying batch and stream processing — Hợp nhất xử lý theo lô và xử lý luồng

More recent work has enabled the benefits of the lambda architecture to be enjoyed without its downsides, by allowing both batch computations (reprocessing historical data) and stream computations (processing events as they arrive) to be implemented in the same system [15].

Các công trình gần đây hơn đã giúp tận hưởng những lợi ích của kiến trúc lambda mà không gặp các nhược điểm của nó, bằng cách cho phép cả các phép tính theo lô (xử lý lại

dữ liệu lịch sử) lẫn các phép tính luồng (xử lý sự kiện ngay khi chúng đến) được hiện thực trong cùng một hệ thống [15].

Unifying batch and stream processing in one system requires the following features, which are becoming increasingly widely available:

Việc hợp nhất xử lý theo lô và xử lý luồng trong một hệ thống đòi hỏi các tính năng sau, vốn đang ngày càng phổ biến:

- The ability to replay historical events through the same processing engine that handles the stream of recent events. For example, log-based message brokers have the ability to replay messages (see “Replaying old messages”), and some stream processors can read input from a distributed filesystem like HDFS.
- Exactly-once semantics for stream processors—that is, ensuring that the output is the same as if no faults had occurred, even if faults did in fact occur (see “Fault Tolerance”). Like with batch processing, this requires discarding the partial output of any failed tasks.
- Tools for windowing by event time, not by processing time, since processing time is meaningless when reprocessing historical events (see “Reasoning About Time”). For example, Apache Beam provides an **API** for expressing such computations, which can then be run using Apache Flink or Google Cloud Dataflow.
- Khả năng phát lại các sự kiện lịch sử qua chính engine xử lý vốn lo việc xử lý luồng các sự kiện gần đây. Chẳng hạn, các message broker dựa trên log có khả năng phát lại các thông điệp (xem “Phát lại các thông điệp cũ”), và một số bộ xử lý luồng có thể đọc đầu vào từ một hệ thống tệp phân tán như HDFS.
- Ngữ nghĩa đúng-một-lần (exactly-once) cho các bộ xử lý luồng — tức là đảm bảo đầu ra giống hệt như thể không có lỗi nào xảy ra, kể cả khi lỗi thực sự đã xảy ra (xem “Khả năng chịu lỗi”). Giống như với xử lý theo lô, điều này đòi hỏi loại bỏ đầu ra bộ phận của bất kỳ tác vụ nào bị hỏng.
- Các công cụ để định cửa sổ theo thời gian sự kiện (event time), chứ không phải theo thời gian xử lý (processing time), bởi thời gian xử lý trở nên vô nghĩa khi xử lý lại các sự kiện lịch sử (xem “Suy luận về thời gian”). Chẳng hạn, Apache Beam cung cấp một API để biểu đạt những phép tính như vậy, và chúng có thể được chạy bằng Apache Flink hoặc Google Cloud Dataflow.

Unbundling Databases — Tháo gói cơ sở dữ liệu

At a most abstract level, databases, Hadoop, and operating systems all perform the same functions: they store some data, and they allow you to process and query that data [16]. A database stores data in records of some **data model** (rows in tables, documents, vertices in a **graph**, etc.) while an operating system's filesystem stores data in files—but at their core, both are “information management” systems [17]. As we saw in Chapter 10, the Hadoop ecosystem is somewhat like a distributed version of Unix.

Ở mức trừu tượng nhất, cơ sở dữ liệu, Hadoop, và các hệ điều hành đều thực hiện cùng những chức năng: chúng lưu một ít dữ liệu, và cho phép bạn xử lý và truy vấn dữ liệu đó [16]. Một cơ sở dữ liệu lưu dữ liệu trong các bản ghi của một mô hình dữ liệu nào đó (hàng trong bảng, tài liệu, đỉnh trong đồ thị, v.v.), trong khi hệ thống tệp của một hệ điều hành lưu dữ liệu trong các tệp — nhưng ở cốt lõi, cả hai đều là các hệ thống “quản lý thông tin” [17]. Như ta đã thấy ở Chương 10, hệ sinh thái Hadoop phần nào giống một phiên bản phân tán của Unix.

Of course, there are many practical differences. For example, many filesystems do not cope very well with a directory containing 10 million small files, whereas a database containing 10 million small records is completely normal and unremarkable. Nevertheless, the similarities and differences between operating systems and databases are worth exploring.

Tất nhiên, có nhiều khác biệt thực tế. Chẳng hạn, nhiều hệ thống tệp xử lý không tốt một thư mục chứa 10 triệu tệp nhỏ, trong khi một cơ sở dữ liệu chứa 10 triệu bản ghi nhỏ là hoàn toàn bình thường và chẳng có gì đáng nói. Tuy vậy, những điểm tương đồng và khác biệt giữa hệ điều hành và cơ sở dữ liệu đáng để khám phá.

Unix and relational databases have approached the information management problem with very different philosophies. Unix viewed its purpose as presenting programmers with a logical but fairly low-level hardware abstraction, whereas relational databases wanted to give application programmers a high-level abstraction that would hide the complexities of data structures on disk, **concurrency**, crash recovery, and so on. Unix developed pipes and files that are just sequences of bytes, whereas databases developed **SQL** and transactions.

Unix và các cơ sở dữ liệu quan hệ đã tiếp cận bài toán quản lý thông tin bằng những triết lý rất khác nhau. Unix coi mục đích của mình là trình bày cho lập trình viên một lớp trừu tượng phần cứng hợp lý nhưng khá ở mức thấp, trong khi các cơ sở dữ liệu quan hệ muốn cho lập trình viên ứng dụng một lớp trừu tượng ở mức cao, che giấu sự phức tạp của các cấu trúc dữ liệu trên đĩa, tính tương tranh, phục hồi sau sự cố, v.v. Unix phát triển các pipe và tệp vốn chỉ là các chuỗi byte, trong khi các cơ sở dữ liệu phát triển SQL và giao dịch.

Which approach is better? Of course, it depends what you want. Unix is “simpler” in the sense that it is a fairly thin wrapper around hardware resources; relational databases are “simpler” in the sense that a short **declarative** query can draw on a lot of powerful infrastructure (query optimization, indexes, join methods, concurrency control, replication, etc.) without the author of the query needing to understand the implementation details.

Cách tiếp cận nào tốt hơn? Tất nhiên, điều đó tùy thuộc vào điều bạn muốn. Unix “đơn giản hơn” theo nghĩa nó là một lớp bọc khá mỏng quanh tài nguyên phần cứng; các cơ sở dữ liệu quan hệ “đơn giản hơn” theo nghĩa một truy vấn khai báo ngắn gọn có thể tận dụng rất nhiều hạ tầng mạnh mẽ (tối ưu truy vấn, chỉ mục, các phương pháp join, kiểm soát tương tranh, sao chép, v.v.) mà người viết truy vấn không cần hiểu chi tiết hiện thực.

The tension between these philosophies has lasted for decades (both Unix and the **relational model** emerged in the early 1970s) and still isn’t resolved. For example, I would interpret the **NoSQL** movement as wanting to apply a Unix-esque approach of low-level abstractions to the domain of distributed OLTP data storage.

Sự căng thẳng giữa hai triết lý này đã kéo dài hàng thập kỷ (cả Unix lẫn mô hình quan hệ đều xuất hiện vào đầu thập niên 1970) và vẫn chưa được hóa giải. Chẳng hạn, tôi diễn giải phong trào NoSQL như mong muốn áp dụng một cách tiếp cận kiểu Unix với các lớp trừu tượng ở mức thấp vào lĩnh vực lưu trữ dữ liệu OLTP phân tán.

In this section I will attempt to reconcile the two philosophies, in the hope that we can combine the best of both worlds.

Trong phần này tôi sẽ cố gắng dung hòa hai triết lý, với hy vọng ta có thể kết hợp những điều tốt nhất của cả hai thế giới.

Composing Data Storage Technologies — Kết cấu các công nghệ lưu trữ dữ liệu

Over the course of this book we have discussed various features provided by databases and how they work, including:

Trong suốt cuốn sách, ta đã bàn về nhiều tính năng khác nhau mà cơ sở dữ liệu cung cấp và cách chúng hoạt động, bao gồm:

- Secondary indexes, which allow you to efficiently search for records based on the value of a field (see “Other Indexing Structures”)
- Materialized views, which are a kind of precomputed cache of query results (see “Aggregation: Data Cubes and Materialized Views”)
- Replication logs, which keep copies of the data on other nodes up to date (see “Implementation of Replication Logs”)
- Full-text search indexes, which allow keyword search in text (see “Full-text search and fuzzy indexes”) and which are built into some relational databases [1]
 - Chỉ mục phụ, vốn cho phép bạn tìm kiếm hiệu quả các bản ghi dựa trên giá trị của một trường (xem “Các cấu trúc lập chỉ mục khác”)
 - Khung nhìn vật chất hóa, vốn là một dạng bộ nhớ đệm chứa kết quả truy vấn đã được tính sẵn (xem “Tổng hợp: khối dữ liệu và khung nhìn vật chất hóa”)
 - Log sao chép, vốn giữ cho các bản sao của dữ liệu trên các nút khác luôn được cập nhật (xem “Hiện thực log sao chép”)

- Chỉ mục tìm kiếm toàn văn, vốn cho phép tìm kiếm theo từ khóa trong văn bản (xem “Tìm kiếm toàn văn và chỉ mục mờ”) và được tích hợp sẵn trong một số cơ sở dữ liệu quan hệ [1]

In Chapters 10 and 11, similar themes emerged. We talked about building full-text search indexes (see “The Output of Batch Workflows”), about materialized view maintenance (see “Maintaining materialized views”), and about replicating changes from a database to derived data systems (see “Change Data Capture”).

Ở Chương 10 và 11, những chủ đề tương tự đã xuất hiện. Ta đã nói về việc dựng các chỉ mục tìm kiếm toàn văn (xem “Đầu ra của các luồng công việc theo lô”), về việc duy trì khung nhìn vật chất hóa (xem “Duy trì khung nhìn vật chất hóa”), và về việc sao chép các thay đổi từ một cơ sở dữ liệu sang các hệ thống dữ liệu dẫn xuất (xem “Nắm bắt thay đổi dữ liệu”).

It seems that there are parallels between the features that are built into databases and the derived data systems that people are building with batch and stream processors.

Dường như có những điểm song hành giữa các tính năng được tích hợp sẵn trong cơ sở dữ liệu và các hệ thống dữ liệu dẫn xuất mà người ta đang xây dựng bằng các bộ xử lý theo lô và xử lý luồng.

Creating an index — Tạo một chỉ mục

Think about what happens when you run `CREATE INDEX` to create a new index in a relational database. The database has to scan over a consistent snapshot of a table, pick out all of the field values being indexed, sort them, and write out the index. Then it must process the backlog of writes that have been made since the consistent snapshot was taken (assuming the table was not locked while creating the index, so writes could continue). Once that is done, the database must continue to keep the index up to date whenever a transaction writes to the table.

Hãy nghĩ về điều xảy ra khi bạn chạy `CREATE INDEX` để tạo một chỉ mục mới trong một cơ sở dữ liệu quan hệ. Cơ sở dữ liệu phải quét qua một ảnh chụp (snapshot) nhất quán của một bảng, chọn ra mọi giá trị trường đang được lập chỉ mục, sắp xếp chúng, và ghi ra chỉ mục. Sau đó nó phải xử lý phần tồn đọng các lệnh ghi đã được thực hiện kể từ khi ảnh chụp nhất quán được lấy (giả sử bảng không bị khóa trong lúc tạo chỉ mục, nên các lệnh ghi có thể tiếp tục). Một khi việc đó xong, cơ sở dữ liệu phải tiếp tục giữ chỉ mục được cập nhật mỗi khi một giao dịch ghi vào bảng.

This process is remarkably similar to setting up a new **follower** replica (see “Setting Up New Followers”), and also very similar to bootstrapping change data capture in a streaming system (see “Initial snapshot”).

Quá trình này giống một cách đáng kể với việc thiết lập một bản sao người theo dõi (follower replica) mới (xem “Thiết lập người theo dõi mới”), và cũng rất giống với việc khởi tạo nắm bắt thay đổi dữ liệu trong một hệ thống luồng (xem “Ảnh chụp ban đầu”).

Whenever you run `CREATE INDEX`, the database essentially reprocesses the existing dataset (as discussed in “Reprocessing data for application evolution”) and derives the index as a new view onto the existing data. The existing data may be a snapshot of the state rather than a log of all changes that ever happened, but the two are closely related (see “State, Streams, and Immutability”).

Mỗi khi bạn chạy `CREATE INDEX`, về bản chất cơ sở dữ liệu xử lý lại tập dữ liệu hiện có (như đã bàn ở “Xử lý lại dữ liệu cho sự tiến hóa của ứng dụng”) và dẫn xuất chỉ mục như một khung nhìn mới trên dữ liệu hiện có. Dữ liệu hiện có có thể là một ảnh chụp của trạng

thái thay vì một log mọi thay đổi từng xảy ra, nhưng hai thứ này có liên hệ chặt chẽ với nhau (xem “Trạng thái, luồng, và tính bất biến”).

The meta-database of everything — Siêu-cơ-sở-dữ-liệu của vạn vật

In this light, I think that the dataflow across an entire organization starts looking like one huge database [7]. Whenever a batch, stream, or ETL process transports data from one place and form to another place and form, it is acting like the database subsystem that keeps indexes or materialized views up to date.

Dưới ánh sáng này, tôi cho rằng luồng dữ liệu trải khắp toàn bộ một tổ chức bắt đầu trông như một cơ sở dữ liệu khổng lồ [7]. Mỗi khi một tiến trình theo lô, luồng, hoặc ETL vận chuyển dữ liệu từ một nơi và một dạng này sang một nơi và một dạng khác, nó đang hành xử như phân hệ của cơ sở dữ liệu vốn giữ cho các chỉ mục hoặc khung nhìn vật chất hóa luôn được cập nhật.

Viewed like this, batch and stream processors are like elaborate implementations of triggers, stored procedures, and materialized view maintenance routines. The derived data systems they maintain are like different index types. For example, a relational database may support B-tree indexes, hash indexes, spatial indexes (see “Multi-column indexes”), and other types of indexes. In the emerging architecture of derived data systems, instead of implementing those facilities as features of a single integrated database product, they are provided by various different pieces of software, running on different machines, administered by different teams.

Nhìn theo cách này, các bộ xử lý theo lô và xử lý luồng giống như những bản hiện thực tinh xảo của trigger, thủ tục lưu trữ (stored procedure), và các thường trình duy trì khung nhìn vật chất hóa. Các hệ thống dữ liệu dẫn xuất mà chúng duy trì giống như các loại chỉ mục khác nhau. Chẳng hạn, một cơ sở dữ liệu quan hệ có thể hỗ trợ chỉ mục cây B, chỉ mục băm (hash), chỉ mục không gian (xem “Chỉ mục nhiều cột”), và các loại chỉ mục khác. Trong kiến trúc đang hình thành của các hệ thống dữ liệu dẫn xuất, thay vì hiện thực những tiện ích đó như các tính năng của một sản phẩm cơ sở dữ liệu tích hợp duy nhất, chúng được cung cấp bởi nhiều phần mềm khác nhau, chạy trên các máy khác nhau, được quản trị bởi các nhóm khác nhau.

Where will these developments take us in the future? If we start from the premise that there is no single data model or storage format that is suitable for all access patterns, I speculate that there are two avenues by which different storage and processing tools can nevertheless be composed into a cohesive system:

Những bước phát triển này sẽ đưa ta tới đâu trong tương lai? Nếu khởi đi từ tiền đề rằng không có một mô hình dữ liệu hay định dạng lưu trữ duy nhất nào phù hợp với mọi mẫu truy cập, tôi phỏng đoán có hai con đường để các công cụ lưu trữ và xử lý khác nhau dù vậy vẫn có thể được kết cấu thành một hệ thống mạch lạc:

It is possible to provide a unified query interface to a wide variety of underlying storage engines and processing methods—an approach known as a federated database or polystore [18, 19]. For example, PostgreSQL’s foreign data wrapper feature fits this pattern [20]. Applications that need a specialized data model or query interface can still access the underlying storage engines directly, while users who want to combine data from disparate places can do so easily through the federated interface.

Ta có thể cung cấp một giao diện truy vấn hợp nhất cho một loạt rất đa dạng các engine lưu trữ và phương pháp xử lý nền — một cách tiếp cận được gọi là cơ sở dữ liệu liên hợp (federated database) hoặc polystore [18, 19]. Chẳng hạn, tính năng bọc dữ liệu ngoài (foreign data wrapper) của PostgreSQL khớp với mẫu này [20]. Các ứng dụng cần một mô

hình dữ liệu hoặc giao diện truy vấn chuyên biệt vẫn có thể truy cập trực tiếp các engine lưu trữ nền, trong khi những người dùng muốn kết hợp dữ liệu từ những nơi tản mạn có thể làm vậy một cách dễ dàng qua giao diện liên hợp.

A federated query interface follows the relational tradition of a single integrated system with a high-level **query language** and elegant semantics, but a complicated implementation.

Một giao diện truy vấn liên hợp tuân theo truyền thống quan hệ của một hệ thống tích hợp duy nhất với một ngôn ngữ truy vấn mức cao và ngữ nghĩa thanh thoát, nhưng có bản hiện thực phức tạp.

While federation addresses read-only querying across several different systems, it does not have a good answer to synchronizing writes across those systems. We said that within a single database, creating a consistent index is a built-in feature. When we compose several storage systems, we similarly need to ensure that all data changes end up in all the right places, even in the face of faults. Making it easier to reliably plug together storage systems (e.g., through change data capture and event logs) is like unbundling a database's index-maintenance features in a way that can synchronize writes across disparate technologies [7, 21].

Trong khi liên hợp giải quyết việc truy vấn chỉ-đọc trên nhiều hệ thống khác nhau, nó lại không có câu trả lời tốt cho việc đồng bộ các lệnh ghi xuyên các hệ thống đó. Ta đã nói rằng trong một cơ sở dữ liệu đơn lẻ, tạo một chỉ mục nhất quán là một tính năng tích hợp sẵn. Khi kết cấu vài hệ thống lưu trữ, ta tương tự cần đảm bảo mọi thay đổi dữ liệu rất cuộc nằm ở đúng mọi nơi, ngay cả khi đối mặt với lỗi. Việc làm cho dễ dàng ghép nối các hệ thống lưu trữ một cách đáng tin cậy (ví dụ, thông qua nắm bắt thay đổi dữ liệu và log sự kiện) giống như tháo gói các tính năng duy trì chỉ mục của một cơ sở dữ liệu theo một cách có thể đồng bộ các lệnh ghi xuyên các công nghệ tản mạn [7, 21].

The unbundled approach follows the Unix tradition of small tools that do one thing well [22], that communicate through a uniform low-level API (pipes), and that can be composed using a higher-level language (the shell) [16].

Cách tiếp cận tháo gói tuân theo truyền thống Unix của những công cụ nhỏ làm tốt một việc [22], giao tiếp qua một API mức thấp đồng nhất (pipe), và có thể được kết cấu bằng một ngôn ngữ mức cao hơn (shell) [16].

Making unbundling work — Làm cho việc tháo gói khả thi

Federation and unbundling are two sides of the same coin: composing a reliable, scalable, and maintainable system out of diverse components. Federated read-only querying requires mapping one data model into another, which takes some thought but is ultimately quite a manageable problem. I think that keeping the writes to several storage systems in sync is the harder engineering problem, and so I will focus on it.

Liên hợp và tháo gói là hai mặt của cùng một đồng xu: kết cấu một hệ thống đáng tin cậy, có khả năng mở rộng, và dễ bảo trì từ những thành phần đa dạng. Việc truy vấn chỉ-đọc theo kiểu liên hợp đòi hỏi ánh xạ một mô hình dữ liệu này sang mô hình khác, vốn cần đôi chút suy nghĩ nhưng rất cuộc là một bài toán khá dễ quản. Tôi cho rằng việc giữ cho các lệnh ghi tới vài hệ thống lưu trữ luôn đồng bộ mới là bài toán kỹ thuật khó hơn, nên tôi sẽ tập trung vào nó.

The traditional approach to synchronizing writes requires distributed transactions across **heterogeneous** storage systems [18], which I think is the wrong solution (see “Derived data versus distributed transactions”). Transactions within a single storage or stream processing system

are feasible, but when data crosses the boundary between different technologies, I believe that an asynchronous event log with idempotent writes is a much more robust and practical approach.

Cách tiếp cận truyền thống để đồng bộ các lệnh ghi đòi hỏi các giao dịch phân tán xuyên các hệ thống lưu trữ không đồng nhất [18], điều mà tôi cho là lời giải sai (xem “Dữ liệu dẫn xuất so với giao dịch phân tán”). Các giao dịch trong nội bộ một hệ thống lưu trữ hoặc xử lý luồng đơn lẻ là khả thi, nhưng khi dữ liệu vượt qua ranh giới giữa các công nghệ khác nhau, tôi tin rằng một log sự kiện bất đồng bộ với các lệnh ghi lũy đẳng là một cách tiếp cận vững vàng và thực tế hơn nhiều.

For example, distributed transactions are used within some stream processors to achieve exactly-once semantics (see “Atomic commit revisited”), and this can work quite well. However, when a transaction would need to involve systems written by different groups of people (e.g., when data is written from a stream processor to a distributed key-value store or search index), the lack of a standardized transaction protocol makes integration much harder. An ordered log of events with idempotent consumers (see “Idempotence”) is a much simpler abstraction, and thus much more feasible to implement across heterogeneous systems [7].

Chẳng hạn, các giao dịch phân tán được dùng bên trong một số bộ xử lý luồng để đạt ngữ nghĩa đúng-một-lần (xem “Xét lại cam kết nguyên tử”), và điều này có thể hoạt động khá tốt. Tuy nhiên, khi một giao dịch cần dính líu tới các hệ thống được viết bởi các nhóm người khác nhau (ví dụ, khi dữ liệu được ghi từ một bộ xử lý luồng tới một kho khóa-giá-trị phân tán hoặc chỉ mục tìm kiếm), việc thiếu một giao thức giao dịch chuẩn hóa khiến việc tích hợp khó hơn nhiều. Một log sự kiện có thứ tự với các bên tiêu thụ lũy đẳng (xem “Tính lũy đẳng”) là một lớp trừu tượng đơn giản hơn nhiều, và do đó khả thi hơn nhiều để hiện thực xuyên các hệ thống không đồng nhất [7].

The big advantage of log-based integration is loose coupling between the various components, which manifests itself in two ways:

Lợi thế lớn của tích hợp dựa trên log là sự gắn kết lỏng giữa các thành phần khác nhau, biểu hiện theo hai cách:

- At a system level, asynchronous event streams make the system as a whole more robust to outages or performance degradation of individual components. If a consumer runs slow or fails, the event log can buffer messages (see “Disk space usage”), allowing the producer and any other consumers to continue running unaffected. The faulty consumer can catch up when it is fixed, so it doesn’t miss any data, and the fault is contained. By contrast, the synchronous interaction of distributed transactions tends to escalate local faults into large-scale failures (see “Limitations of distributed transactions”).
- At a human level, unbundling data systems allows different software components and services to be developed, improved, and maintained independently from each other by different teams. Specialization allows each team to focus on doing one thing well, with well-defined interfaces to other teams’ systems. Event logs provide an interface that is powerful enough to capture fairly strong consistency properties (due to **durability** and ordering of events), but also general enough to be applicable to almost any kind of data.
 - Ở cấp hệ thống, các luồng sự kiện bất đồng bộ làm cho toàn bộ hệ thống vững vàng hơn trước sự gián đoạn hoặc suy giảm hiệu năng của từng thành phần riêng lẻ. Nếu một bên tiêu thụ chạy chậm hoặc hỏng, log sự kiện có thể đệm các thông điệp (xem “Mức sử dụng dung lượng đĩa”), cho phép bên sản xuất và mọi bên tiêu thụ khác tiếp tục chạy mà không bị ảnh hưởng. Bên tiêu thụ bị lỗi có thể bắt kịp khi nó được sửa, nên nó không bỏ sót dữ liệu nào, và lỗi được khu trú. Ngược lại, sự tương tác đồng bộ của giao dịch phân tán có xu hướng leo thang các lỗi cục bộ thành các sự cố quy mô

- lớn (xem “Những giới hạn của giao dịch phân tán”).
- Ở cấp con người, việc tháo gói các hệ thống dữ liệu cho phép các thành phần phần mềm và dịch vụ khác nhau được phát triển, cải thiện, và bảo trì độc lập với nhau bởi các nhóm khác nhau. Sự chuyên môn hóa cho phép mỗi nhóm tập trung làm tốt một việc, với các giao diện được định nghĩa rõ ràng tới các hệ thống của nhóm khác. Log sự kiện cung cấp một giao diện đủ mạnh để nắm bắt các thuộc tính nhất quán khá mạnh (nhờ độ bền và thứ tự của các sự kiện), nhưng cũng đủ tổng quát để áp dụng được cho gần như bất kỳ loại dữ liệu nào.

Unbundled versus integrated systems — Hệ thống tháo gói so với hệ thống tích hợp

If unbundling does indeed become the way of the future, it will not replace databases in their current form—they will still be needed as much as ever. Databases are still required for maintaining state in stream processors, and in order to serve queries for the output of batch and stream processors (see “The Output of Batch Workflows” and “Processing Streams”). Specialized query engines will continue to be important for particular workloads: for example, query engines in MPP data warehouses are optimized for exploratory analytic queries and handle this kind of workload very well (see “Comparing Hadoop to Distributed Databases”).

Nếu tháo gói quả thực trở thành con đường của tương lai, nó sẽ không thay thế các cơ sở dữ liệu ở dạng hiện tại — chúng vẫn sẽ cần thiết như mọi khi. Cơ sở dữ liệu vẫn cần để duy trì trạng thái trong các bộ xử lý luồng, và để phục vụ các truy vấn đối với đầu ra của các bộ xử lý theo lô và xử lý luồng (xem “Đầu ra của các luồng công việc theo lô” và “Xử lý các luồng”). Các engine truy vấn chuyên biệt sẽ tiếp tục quan trọng đối với những khối lượng công việc cụ thể: chẳng hạn, các engine truy vấn trong kho dữ liệu MPP được tối ưu cho các truy vấn phân tích khám phá và xử lý loại khối lượng công việc này rất tốt (xem “So sánh Hadoop với cơ sở dữ liệu phân tán”).

The complexity of running several different pieces of infrastructure can be a problem: each piece of software has a learning curve, configuration issues, and operational quirks, and so it is worth deploying as few moving parts as possible. A single integrated software product may also be able to achieve better and more predictable performance on the kinds of workloads for which it is designed, compared to a system consisting of several tools that you have composed with application code [23]. As I said in the Preface, building for scale that you don’t need is wasted effort and may lock you into an inflexible design. In effect, it is a form of premature optimization.

Độ phức tạp của việc vận hành vài mảnh hạ tầng khác nhau có thể là một vấn đề: mỗi phần mềm có một đường cong học tập, các vấn đề cấu hình, và những điều kỳ quặc khi vận hành, nên đáng triển khai càng ít bộ phận chuyển động càng tốt. Một sản phẩm phần mềm tích hợp duy nhất cũng có thể đạt hiệu năng tốt hơn và dễ dự đoán hơn trên những loại khối lượng công việc mà nó được thiết kế cho, so với một hệ thống gồm vài công cụ mà bạn đã kết cấu lại bằng mã ứng dụng [23]. Như tôi đã nói ở Lời nói đầu, việc xây dựng cho một quy mô mà bạn không cần là nỗ lực phí phạm và có thể trói bạn vào một thiết kế thiếu linh hoạt. Trên thực tế, đó là một dạng tối ưu hóa quá sớm.

The goal of unbundling is not to compete with individual databases on performance for particular workloads; the goal is to allow you to combine several different databases in order to achieve good performance for a much wider range of workloads than is possible with a single piece of software. It’s about breadth, not depth—in the same vein as the diversity of storage and processing models that we discussed in “Comparing Hadoop to Distributed Databases”.

Mục tiêu của tháo gói không phải để cạnh tranh với từng cơ sở dữ liệu riêng lẻ về hiệu

năng cho những khối lượng công việc cụ thể; mục tiêu là cho phép bạn kết hợp vài cơ sở dữ liệu khác nhau để đạt hiệu năng tốt trên một phạm vi khối lượng công việc rộng hơn nhiều so với mức một phần mềm đơn lẻ có thể đạt được. Đó là về bề rộng, không phải bề sâu — cùng tinh thần với sự đa dạng của các mô hình lưu trữ và xử lý mà ta đã bàn ở “So sánh Hadoop với cơ sở dữ liệu phân tán”.

Thus, if there is a single technology that does everything you need, you’re most likely best off simply using that product rather than trying to reimplement it yourself from lower-level components. The advantages of unbundling and composition only come into the picture when there is no single piece of software that satisfies all your requirements.

Do đó, nếu có một công nghệ duy nhất làm được mọi thứ bạn cần, rất có khả năng bạn nên đơn giản dùng sản phẩm đó thay vì cố tự hiện thực lại nó từ các thành phần ở mức thấp hơn. Những lợi thế của tháo gói và kết cấu chỉ xuất hiện khi không có một phần mềm đơn lẻ nào thỏa mãn mọi yêu cầu của bạn.

What’s missing? — Còn thiếu điều gì?

The tools for composing data systems are getting better, but I think one major part is missing: we don’t yet have the unbundled-database equivalent of the Unix shell (i.e., a high-level language for composing storage and processing systems in a simple and declarative way).

Các công cụ để kết cấu các hệ thống dữ liệu đang ngày càng tốt hơn, nhưng tôi cho rằng còn thiếu một phần lớn: ta chưa có thứ tương đương với shell của Unix cho cơ sở dữ liệu tháo gói (tức là một ngôn ngữ mức cao để kết cấu các hệ thống lưu trữ và xử lý theo một cách đơn giản và khai báo).

For example, I would love it if we could simply declare `mysql | elasticsearch`, by analogy to Unix pipes [22], which would be the unbundled equivalent of CREATE INDEX: it would take all the documents in a MySQL database and index them in an Elasticsearch cluster. It would then continually capture all the changes made to the database and automatically apply them to the search index, without us having to write custom application code. This kind of integration should be possible with almost any kind of storage or indexing system.

Chẳng hạn, tôi sẽ rất thích nếu ta có thể đơn giản khai báo `mysql | elasticsearch`, tương tự như pipe của Unix [22], vốn sẽ là phiên bản tháo gói của CREATE INDEX: nó sẽ lấy mọi tài liệu trong một cơ sở dữ liệu MySQL và lập chỉ mục chúng trong một cụm Elasticsearch. Sau đó nó sẽ liên tục nắm bắt mọi thay đổi với cơ sở dữ liệu và tự động áp dụng chúng vào chỉ mục tìm kiếm, mà ta không phải viết mã ứng dụng tùy biến. Kiểu tích hợp này lẽ ra phải khả thi với gần như bất kỳ loại hệ thống lưu trữ hoặc lập chỉ mục nào.

Similarly, it would be great to be able to precompute and update caches more easily. Recall that a materialized view is essentially a precomputed cache, so you could imagine creating a cache by declaratively specifying materialized views for complex queries, including recursive queries on graphs (see “Graph-Like Data Models”) and application logic. There is interesting early-stage research in this area, such as differential dataflow [24, 25], and I hope that these ideas will find their way into **production** systems.

Tương tự, sẽ rất tuyệt nếu có thể tính sẵn và cập nhật các bộ nhớ đệm một cách dễ dàng hơn. Hãy nhớ rằng một khung nhìn vật chất hóa về bản chất là một bộ nhớ đệm được tính sẵn, nên bạn có thể hình dung việc tạo một bộ nhớ đệm bằng cách khai báo các khung nhìn vật chất hóa cho những truy vấn phức tạp, bao gồm các truy vấn đệ quy trên đồ thị (xem “Các mô hình dữ liệu dạng đồ thị”) và logic ứng dụng. Có những nghiên cứu giai đoạn

đầu thú vị trong lĩnh vực này, chẳng hạn luồng dữ liệu vi phân (differential dataflow) [24, 25], và tôi hy vọng những ý tưởng này sẽ tìm được đường vào các hệ thống production.

Designing Applications Around Dataflow — Thiết kế ứng dụng xoay quanh luồng dữ liệu

The approach of unbundling databases by composing specialized storage and processing systems with application code is also becoming known as the “database inside-out” approach [26], after the title of a conference talk I gave in 2014 [27]. However, calling it a “new architecture” is too grandiose. I see it more as a design pattern, a starting point for discussion, and we give it a name simply so that we can better talk about it.

Cách tiếp cận tháo gói cơ sở dữ liệu bằng cách kết cấu các hệ thống lưu trữ và xử lý chuyên biệt với mã ứng dụng cũng đang được biết đến với tên gọi cách tiếp cận “cơ sở dữ liệu lộn-từ-trong-ra” (database inside-out) [26], theo tựa đề một bài nói chuyện hội nghị mà tôi trình bày năm 2014 [27]. Tuy nhiên, gọi nó là một “kiến trúc mới” thì quá đại ngôn. Tôi xem nó nhiều hơn như một mẫu thiết kế, một điểm khởi đầu cho thảo luận, và ta đặt cho nó một cái tên chỉ đơn giản để có thể bàn về nó dễ hơn.

These ideas are not mine; they are simply an amalgamation of other people’s ideas from which I think we should learn. In particular, there is a lot of overlap with dataflow languages such as Oz [28] and Juttle [29], functional reactive programming (FRP) languages such as Elm [30, 31], and logic programming languages such as Bloom [32]. The term unbundling in this context was proposed by Jay Kreps [7].

Những ý tưởng này không phải của tôi; chúng chỉ đơn giản là sự pha trộn các ý tưởng của những người khác mà tôi cho rằng ta nên học hỏi. Cụ thể, có rất nhiều điểm trùng lặp với các ngôn ngữ luồng dữ liệu như Oz [28] và Juttle [29], các ngôn ngữ lập trình phản ứng hàm (functional reactive programming, FRP) như Elm [30, 31], và các ngôn ngữ lập trình logic như Bloom [32]. Thuật ngữ tháo gói (unbundling) trong bối cảnh này do Jay Kreps đề xuất [7].

Even spreadsheets have dataflow programming capabilities that are miles ahead of most mainstream programming languages [33]. In a spreadsheet, you can put a formula in one cell (for example, the sum of cells in another column), and whenever any input to the formula changes, the result of the formula is automatically recalculated. This is exactly what we want at a **data system** level: when a record in a database changes, we want any index for that record to be automatically updated, and any cached views or aggregations that depend on the record to be automatically refreshed. You should not have to worry about the technical details of how this refresh happens, but be able to simply trust that it works correctly.

Ngay cả bảng tính cũng có những khả năng lập trình luồng dữ liệu đi trước hầu hết các ngôn ngữ lập trình chính thống cả dậm đường [33]. Trong một bảng tính, bạn có thể đặt một công thức vào một ô (chẳng hạn, tổng các ô ở một cột khác), và mỗi khi bất kỳ đầu vào nào của công thức thay đổi, kết quả của công thức được tự động tính lại. Đây chính xác là điều ta muốn ở cấp hệ thống dữ liệu: khi một bản ghi trong cơ sở dữ liệu thay đổi, ta muốn bất kỳ chỉ mục nào cho bản ghi đó được tự động cập nhật, và bất kỳ khung nhìn hoặc phép tổng hợp được đệm nào phụ thuộc vào bản ghi đó được tự động làm mới. Bạn không nên phải bận tâm về các chi tiết kỹ thuật của việc làm mới này diễn ra ra sao, mà có thể đơn giản tin rằng nó hoạt động đúng đắn.

Thus, I think that most data systems still have something to learn from the features that VisiCalc

already had in 1979 [34]. The difference from spreadsheets is that today's data systems need to be fault-tolerant, scalable, and store data durably. They also need to be able to integrate disparate technologies written by different groups of people over time, and reuse existing libraries and services: it is unrealistic to expect all software to be developed using one particular language, framework, or tool.

Do đó, tôi cho rằng hầu hết các hệ thống dữ liệu vẫn còn điều gì đó để học từ các tính năng mà VisiCalc đã có sẵn từ năm 1979 [34]. Khác biệt so với bảng tính là các hệ thống dữ liệu ngày nay cần chịu lỗi, có khả năng mở rộng, và lưu dữ liệu một cách bền vững. Chúng cũng cần có khả năng tích hợp các công nghệ tản mạn được viết bởi các nhóm người khác nhau qua thời gian, và tái sử dụng các thư viện và dịch vụ hiện có: thật phi thực tế khi kỳ vọng mọi phần mềm đều được phát triển bằng một ngôn ngữ, framework, hay công cụ cụ thể nào đó.

In this section I will expand on these ideas and explore some ways of building applications around the ideas of unbundled databases and dataflow.

Trong phần này tôi sẽ mở rộng những ý tưởng này và khám phá một số cách xây dựng ứng dụng xoay quanh các ý tưởng về cơ sở dữ liệu tháo gói và luồng dữ liệu.

Application code as a derivation function — Mã ứng dụng như một hàm dẫn xuất

When one dataset is derived from another, it goes through some kind of transformation function. For example:

Khi một tập dữ liệu được dẫn xuất từ một tập khác, nó đi qua một dạng hàm biến đổi nào đó. Chẳng hạn:

- A secondary index is a kind of derived dataset with a straightforward transformation function: for each **row** or document in the base table, it picks out the values in the columns or fields being indexed, and sorts by those values (assuming a B-tree or SSTable index, which are sorted by key, as discussed in Chapter 3).
 - A full-text search index is created by applying various natural language processing functions such as language detection, word segmentation, stemming or lemmatization, spelling correction, and synonym identification, followed by building a data structure for efficient lookups (such as an inverted index).
 - In a machine learning system, we can consider the model as being derived from the training data by applying various feature extraction and statistical analysis functions. When the model is applied to new input data, the output of the model is derived from the input and the model (and hence, indirectly, from the training data).
 - A cache often contains an aggregation of data in the form in which it is going to be displayed in a user interface (UI). Populating the cache thus requires knowledge of what fields are referenced in the UI; changes in the UI may require updating the definition of how the cache is populated and rebuilding the cache.
- Một chỉ mục phụ là một dạng tập dữ liệu dẫn xuất với một hàm biến đổi đơn giản: với mỗi hàng hoặc tài liệu trong bảng gốc, nó chọn ra các giá trị ở những cột hoặc trường đang được lập chỉ mục, và sắp xếp theo những giá trị đó (giả sử là một chỉ mục cây B hoặc SSTable, vốn được sắp theo khóa, như đã bàn ở Chương 3).
 - Một chỉ mục tìm kiếm toàn văn được tạo bằng cách áp dụng nhiều hàm xử lý ngôn ngữ tự nhiên như nhận diện ngôn ngữ, tách từ, lấy gốc từ (stemming) hoặc đưa về dạng từ điển (lemmatization), sửa chính tả, và nhận diện từ đồng nghĩa, theo sau là

việc dựng một cấu trúc dữ liệu cho việc tra cứu hiệu quả (như một chỉ mục đảo — inverted index).

- Trong một hệ thống học máy, ta có thể coi mô hình như được dẫn xuất từ dữ liệu huấn luyện bằng cách áp dụng nhiều hàm trích xuất đặc trưng và phân tích thống kê. Khi mô hình được áp dụng cho dữ liệu đầu vào mới, đầu ra của mô hình được dẫn xuất từ đầu vào và mô hình (và do đó, gián tiếp, từ dữ liệu huấn luyện).
- Một bộ nhớ đệm thường chứa một phép tổng hợp dữ liệu ở dạng mà nó sẽ được hiển thị trong một giao diện người dùng (UI). Do đó, việc nạp dữ liệu vào bộ nhớ đệm đòi hỏi biết những trường nào được tham chiếu trong UI; các thay đổi trong UI có thể đòi hỏi cập nhật định nghĩa cách bộ nhớ đệm được nạp và dựng lại bộ nhớ đệm.

The derivation function for a secondary index is so commonly required that it is built into many databases as a core feature, and you can invoke it by merely saying `CREATE INDEX`. For full-text indexing, basic linguistic features for common languages may be built into a database, but the more sophisticated features often require domain-specific tuning. In machine learning, feature engineering is notoriously application-specific, and often has to incorporate detailed knowledge about the user interaction and **deployment** of an application [35].

Hàm dẫn xuất cho một chỉ mục phụ được cần đến phổ biến tới mức nó được tích hợp sẵn trong nhiều cơ sở dữ liệu như một tính năng cốt lõi, và bạn có thể gọi nó chỉ bằng cách nói `CREATE INDEX`. Với lập chỉ mục toàn văn, các tính năng ngôn ngữ cơ bản cho những ngôn ngữ thông dụng có thể được tích hợp sẵn trong cơ sở dữ liệu, nhưng những tính năng tinh vi hơn thường đòi hỏi tinh chỉnh đặc thù theo lĩnh vực. Trong học máy, kỹ thuật đặc trưng (feature engineering) nổi tiếng là đặc thù theo ứng dụng, và thường phải kết hợp kiến thức chi tiết về tương tác người dùng và việc triển khai một ứng dụng [35].

When the function that creates a derived dataset is not a standard cookie-cutter function like creating a secondary index, custom code is required to handle the application-specific aspects. And this custom code is where many databases struggle. Although relational databases commonly support triggers, stored procedures, and user-defined functions, which can be used to execute application code within the database, they have been somewhat of an afterthought in database design (see “Transmitting Event Streams”).

Khi hàm tạo ra một tập dữ liệu dẫn xuất không phải là một hàm khuôn mẫu tiêu chuẩn như việc tạo một chỉ mục phụ, cần có mã tùy biến để xử lý các khía cạnh đặc thù theo ứng dụng. Và chính mã tùy biến này là chỗ mà nhiều cơ sở dữ liệu chật vật. Mặc dù các cơ sở dữ liệu quan hệ thường hỗ trợ trigger, thủ tục lưu trữ, và các hàm do người dùng định nghĩa (user-defined function), vốn có thể được dùng để thực thi mã ứng dụng bên trong cơ sở dữ liệu, chúng phần nào là một thứ được nghĩ tới sau cùng trong thiết kế cơ sở dữ liệu (xem “Truyền các luồng sự kiện”).

Separation of application code and state — Tách biệt mã ứng dụng và trạng thái

In theory, databases could be deployment environments for arbitrary application code, like an operating system. However, in practice they have turned out to be poorly suited for this purpose. They do not fit well with the requirements of modern application development, such as dependency and package management, version control, rolling upgrades, **evolvability**, monitoring, metrics, calls to network services, and integration with external systems.

Về lý thuyết, cơ sở dữ liệu có thể là môi trường triển khai cho mã ứng dụng bất kỳ, giống như một hệ điều hành. Tuy nhiên, trên thực tế chúng hóa ra lại kém phù hợp cho mục đích này. Chúng không khớp tốt với các yêu cầu của việc phát triển ứng dụng hiện đại, chẳng

hạn quản lý phụ thuộc và gói, kiểm soát phiên bản, nâng cấp cuốn chiếu, khả năng tiến hóa, giám sát, đo chỉ số, gọi tới các dịch vụ mạng, và tích hợp với các hệ thống bên ngoài.

On the other hand, deployment and cluster management tools such as Mesos, YARN, Docker, Kubernetes, and others are designed specifically for the purpose of running application code. By focusing on doing one thing well, they are able to do it much better than a database that provides execution of user-defined functions as one of its many features.

Mặt khác, các công cụ triển khai và quản lý cụm như Mesos, YARN, Docker, Kubernetes, và những công cụ khác được thiết kế chuyên cho mục đích chạy mã ứng dụng. Bằng cách tập trung làm tốt một việc, chúng có thể làm việc đó tốt hơn nhiều so với một cơ sở dữ liệu vốn cung cấp việc thực thi các hàm do người dùng định nghĩa như một trong rất nhiều tính năng của nó.

I think it makes sense to have some parts of a system that specialize in durable data storage, and other parts that specialize in running application code. The two can interact while still remaining independent.

Tôi cho rằng việc có một số phần của hệ thống chuyên về lưu trữ dữ liệu bền vững, và các phần khác chuyên về chạy mã ứng dụng, là hợp lý. Hai phần này có thể tương tác trong khi vẫn giữ độc lập với nhau.

Most web applications today are deployed as **stateless** services, in which any user request can be routed to any application server, and the server forgets everything about the request once it has sent the response. This style of deployment is convenient, as servers can be added or removed at will, but the state has to go somewhere: typically, a database. The trend has been to keep stateless application logic separate from state management (databases): not putting application logic in the database and not putting persistent state in the application [36]. As people in the functional programming community like to joke, “We believe in the separation of Church and state” [37].

Hầu hết các ứng dụng web ngày nay được triển khai dưới dạng các dịch vụ không trạng thái, trong đó bất kỳ yêu cầu nào của người dùng cũng có thể được định tuyến tới bất kỳ máy chủ ứng dụng nào, và máy chủ quên hết mọi thứ về yêu cầu một khi nó đã gửi phản hồi. Phong cách triển khai này tiện lợi, vì các máy chủ có thể được thêm hoặc bớt tùy ý, nhưng trạng thái phải nằm ở đâu đó: thường là một cơ sở dữ liệu. Xu hướng là giữ cho logic ứng dụng không trạng thái tách biệt với việc quản lý trạng thái (cơ sở dữ liệu): không đặt logic ứng dụng vào cơ sở dữ liệu và không đặt trạng thái bền vững vào ứng dụng [36]. Như những người trong cộng đồng lập trình hàm thích đùa, “Chúng tôi tin vào sự tách biệt giữa Church và state” [37].

In this typical web application model, the database acts as a kind of mutable shared variable that can be accessed synchronously over the network. The application can read and update the variable, and the database takes care of making it durable, providing some concurrency control and fault tolerance.

Trong mô hình ứng dụng web điển hình này, cơ sở dữ liệu hành xử như một dạng biến chia sẻ có thể thay đổi (mutable shared variable) mà ta có thể truy cập một cách đồng bộ qua mạng. Ứng dụng có thể đọc và cập nhật biến đó, còn cơ sở dữ liệu lo việc làm cho nó bền vững, cung cấp một mức kiểm soát tương tranh và khả năng chịu lỗi nhất định.

However, in most programming languages you cannot subscribe to changes in a mutable variable—you can only read it periodically. Unlike in a spreadsheet, readers of the variable don’t get notified if the value of the variable changes. (You can implement such notifications in your own code—this is known as the observer pattern—but most languages do not have this pattern as a built-in feature.)

Tuy nhiên, trong hầu hết các ngôn ngữ lập trình, bạn không thể đăng ký theo dõi các thay đổi của một biến có thể thay đổi — bạn chỉ có thể đọc nó định kỳ. Khác với trong một bảng tính, người đọc biến không được thông báo nếu giá trị của biến thay đổi. (Bạn có thể tự hiện thực những thông báo như vậy trong mã của mình — điều này được gọi là mẫu quan sát viên (observer pattern) — nhưng hầu hết các ngôn ngữ không có sẵn mẫu này như một tính năng tích hợp.)

Databases have inherited this passive approach to mutable data: if you want to find out whether the content of the database has changed, often your only option is to poll (i.e., to repeat your query periodically). Subscribing to changes is only just beginning to emerge as a feature (see “API support for change streams”).

Cơ sở dữ liệu đã thừa hưởng cách tiếp cận thụ động này đối với dữ liệu có thể thay đổi: nếu bạn muốn biết nội dung của cơ sở dữ liệu có thay đổi hay không, thường lựa chọn duy nhất của bạn là thăm dò (poll) (tức là lặp lại truy vấn của bạn một cách định kỳ). Việc đăng ký theo dõi các thay đổi chỉ vừa mới bắt đầu xuất hiện như một tính năng (xem “Hỗ trợ API cho luồng thay đổi”).

Dataflow: Interplay between state changes and application code — Luồng dữ liệu: Sự tương tác giữa các thay đổi trạng thái và mã ứng dụng

Thinking about applications in terms of dataflow implies renegotiating the relationship between application code and state management. Instead of treating a database as a passive variable that is manipulated by the application, we think much more about the interplay and collaboration between state, state changes, and code that processes them. Application code responds to state changes in one place by triggering state changes in another place.

Việc nghĩ về ứng dụng theo lối luồng dữ liệu ngụ ý việc thương lượng lại mối quan hệ giữa mã ứng dụng và việc quản lý trạng thái. Thay vì coi một cơ sở dữ liệu như một biến thụ động bị ứng dụng thao tác, ta nghĩ nhiều hơn về sự tương tác và phối hợp giữa trạng thái, các thay đổi trạng thái, và mã xử lý chúng. Mã ứng dụng phản ứng với các thay đổi trạng thái ở một chỗ bằng cách kích hoạt các thay đổi trạng thái ở một chỗ khác.

We saw this line of thinking in “Databases and Streams”, where we discussed treating the log of changes to a database as a stream of events that we can subscribe to. Message-passing systems such as actors (see “Message-Passing Dataflow”) also have this concept of responding to events. Already in the 1980s, the **tuple** spaces model explored expressing distributed computations in terms of processes that observe state changes and react to them [38, 39].

Ta đã thấy mạch suy nghĩ này ở “Cơ sở dữ liệu và luồng”, nơi ta bàn về việc coi log các thay đổi với một cơ sở dữ liệu như một luồng sự kiện mà ta có thể đăng ký theo dõi. Các hệ thống truyền thông điệp như actor (xem “Luồng dữ liệu kiểu truyền thông điệp”) cũng có khái niệm phản ứng với sự kiện này. Ngay từ thập niên 1980, mô hình không gian bộ (tuple spaces) đã khám phá việc biểu đạt các phép tính phân tán theo lối các tiến trình quan sát các thay đổi trạng thái và phản ứng với chúng [38, 39].

As discussed, similar things happen inside a database when a trigger fires due to a data change, or when a secondary index is updated to reflect a change in the table being indexed. Unbundling the database means taking this idea and applying it to the creation of derived datasets outside of the primary database: caches, full-text search indexes, machine learning, or analytics systems. We can use stream processing and messaging systems for this purpose.

Như đã bàn, những điều tương tự xảy ra bên trong một cơ sở dữ liệu khi một trigger kích hoạt do một thay đổi dữ liệu, hoặc khi một chỉ mục phụ được cập nhật để phản ánh một

thay đổi ở bảng đang được lập chỉ mục. Việc tháo gói cơ sở dữ liệu nghĩa là lấy ý tưởng này và áp dụng nó vào việc tạo các tập dữ liệu dẫn xuất bên ngoài cơ sở dữ liệu chính: các bộ nhớ đệm, chỉ mục tìm kiếm toàn văn, hệ thống học máy, hoặc hệ thống phân tích. Ta có thể dùng các hệ thống xử lý luồng và truyền thông điệp cho mục đích này.

The important thing to keep in mind is that maintaining derived data is not the same as asynchronous job execution, for which messaging systems are traditionally designed (see “Logs compared to traditional messaging”):

Điều quan trọng cần ghi nhớ là việc duy trì dữ liệu dẫn xuất không giống với việc thực thi tác vụ bất đồng bộ, vốn là thứ mà các hệ thống truyền thông điệp được thiết kế cho theo truyền thống (xem “So sánh log với hệ thống truyền thông điệp truyền thống”):

- When maintaining derived data, the order of state changes is often important (if several views are derived from an event log, they need to process the events in the same order so that they remain consistent with each other). As discussed in “Acknowledgments and redelivery”, many message brokers do not have this **property** when redelivering unacknowledged messages. Dual writes are also ruled out (see “Keeping Systems in Sync”).
- Fault tolerance is key for derived data: losing just a single message causes the derived dataset to go permanently out of sync with its data source. Both message delivery and derived state updates must be reliable. For example, many actor systems by default maintain actor state and messages in memory, so they are lost if the machine running the actor crashes.
 - Khi duy trì dữ liệu dẫn xuất, thứ tự của các thay đổi trạng thái thường quan trọng (nếu vài khung nhìn được dẫn xuất từ một log sự kiện, chúng cần xử lý các sự kiện theo cùng thứ tự để giữ nhất quán với nhau). Như đã bàn ở “Xác nhận và giao lại”, nhiều message broker không có thuộc tính này khi giao lại các thông điệp chưa được xác nhận. Lệnh ghi kép (dual write) cũng bị loại trừ (xem “Giữ các hệ thống đồng bộ”).
 - Khả năng chịu lỗi là then chốt đối với dữ liệu dẫn xuất: chỉ cần mất một thông điệp duy nhất cũng khiến tập dữ liệu dẫn xuất trở nên lệch khỏi nguồn dữ liệu của nó vĩnh viễn. Cả việc giao thông điệp lẫn việc cập nhật trạng thái dẫn xuất đều phải đáng tin cậy. Chẳng hạn, nhiều hệ thống actor theo mặc định duy trì trạng thái actor và các thông điệp trong bộ nhớ, nên chúng sẽ mất nếu máy chạy actor bị sự cố.

Stable message ordering and fault-tolerant message processing are quite stringent demands, but they are much less expensive and more operationally robust than distributed transactions. Modern stream processors can provide these ordering and reliability guarantees at scale, and they allow application code to be run as stream operators.

Thứ tự thông điệp ổn định và việc xử lý thông điệp chịu lỗi là những đòi hỏi khá khắt khe, nhưng chúng rẻ hơn nhiều và vững vàng hơn về mặt vận hành so với giao dịch phân tán. Các bộ xử lý luồng hiện đại có thể cung cấp những đảm bảo về thứ tự và độ tin cậy này ở quy mô lớn, và chúng cho phép mã ứng dụng được chạy như các toán tử luồng.

This application code can do the arbitrary processing that built-in derivation functions in databases generally don’t provide. Like Unix tools chained by pipes, stream operators can be composed to build large systems around dataflow. Each operator takes streams of state changes as input, and produces other streams of state changes as output.

Mã ứng dụng này có thể thực hiện những xử lý bất kỳ mà các hàm dẫn xuất tích hợp sẵn trong cơ sở dữ liệu nói chung không cung cấp. Giống như các công cụ Unix được nối với nhau bằng pipe, các toán tử luồng có thể được kết cấu để xây dựng các hệ thống lớn xoay quanh luồng dữ liệu. Mỗi toán tử nhận các luồng thay đổi trạng thái làm đầu vào, và tạo ra các luồng thay đổi trạng thái khác làm đầu ra.

Stream processors and services — Bộ xử lý luồng và dịch vụ

The currently trendy style of application development involves breaking down functionality into a set of services that communicate via synchronous network requests such as REST APIs (see “Dataflow Through Services: REST and RPC”). The advantage of such a service-oriented architecture over a single monolithic application is primarily organizational scalability through loose coupling: different teams can work on different services, which reduces coordination effort between teams (as long as the services can be deployed and updated independently).

Phong cách phát triển ứng dụng đang thịnh hành hiện nay liên quan tới việc chia nhỏ chức năng thành một tập các dịch vụ giao tiếp qua các yêu cầu mạng đồng bộ như REST API (xem “Luồng dữ liệu qua dịch vụ: REST và RPC”). Lợi thế của một kiến trúc hướng dịch vụ như vậy so với một ứng dụng nguyên khối (monolithic) duy nhất chủ yếu là khả năng mở rộng về mặt tổ chức thông qua sự gắn kết lỏng: các nhóm khác nhau có thể làm việc trên các dịch vụ khác nhau, điều này giảm công sức phối hợp giữa các nhóm (miễn là các dịch vụ có thể được triển khai và cập nhật độc lập).

Composing stream operators into dataflow systems has a lot of similar characteristics to the microservices approach [40]. However, the underlying communication mechanism is very different: one-directional, asynchronous message streams rather than synchronous request/response interactions.

Việc kết cấu các toán tử luồng thành các hệ thống luồng dữ liệu có nhiều đặc tính tương tự với cách tiếp cận vi dịch vụ [40]. Tuy nhiên, cơ chế giao tiếp nền rất khác: các luồng thông điệp một chiều, bất đồng bộ, thay vì các tương tác yêu cầu/phản hồi đồng bộ.

Besides the advantages listed in “Message-Passing Dataflow”, such as better fault tolerance, dataflow systems can also achieve better performance. For example, say a customer is purchasing an item that is priced in one currency but paid for in another currency. In order to perform the currency conversion, you need to know the current exchange rate. This operation could be implemented in two ways [40, 41]:

Bên cạnh những lợi thế được liệt kê ở “Luồng dữ liệu kiểu truyền thông điệp”, chẳng hạn khả năng chịu lỗi tốt hơn, các hệ thống luồng dữ liệu còn có thể đạt hiệu năng tốt hơn. Chẳng hạn, giả sử một khách hàng đang mua một món hàng được định giá bằng một loại tiền tệ nhưng thanh toán bằng một loại tiền tệ khác. Để thực hiện việc chuyển đổi tiền tệ, bạn cần biết tỷ giá hối đoái hiện tại. Thao tác này có thể được hiện thực theo hai cách [40, 41]:

- In the microservices approach, the code that processes the purchase would probably query an exchange-rate service or database in order to obtain the current rate for a particular currency.
- In the dataflow approach, the code that processes purchases would subscribe to a stream of exchange rate updates ahead of time, and record the current rate in a local database whenever it changes. When it comes to processing the purchase, it only needs to query the local database.
 - Trong cách tiếp cận vi dịch vụ, mã xử lý giao dịch mua hàng có lẽ sẽ truy vấn một dịch vụ hoặc cơ sở dữ liệu tỷ giá hối đoái để lấy tỷ giá hiện tại cho một loại tiền tệ cụ thể.
 - Trong cách tiếp cận luồng dữ liệu, mã xử lý các giao dịch mua hàng sẽ đăng ký theo dõi một luồng các cập nhật tỷ giá hối đoái từ trước, và ghi lại tỷ giá hiện tại vào một cơ sở dữ liệu cục bộ mỗi khi nó thay đổi. Khi đến lúc xử lý giao dịch mua hàng, nó chỉ cần truy vấn cơ sở dữ liệu cục bộ.

The second approach has replaced a synchronous network request to another service with a query to a local database (which may be on the same machine, even in the same process). Not only is the

dataflow approach faster, but it is also more robust to the failure of another service. The fastest and most reliable network request is no network request at all! Instead of RPC, we now have a stream join between purchase events and exchange rate update events (see “Stream-table join (stream enrichment)”).

Cách thứ hai đã thay một yêu cầu mạng đồng bộ tới một dịch vụ khác bằng một truy vấn tới một cơ sở dữ liệu cục bộ (có thể nằm trên cùng một máy, thậm chí trong cùng một tiến trình). Cách tiếp cận luồng dữ liệu không chỉ nhanh hơn, mà còn vững vàng hơn trước sự cố của một dịch vụ khác. Yêu cầu mạng nhanh nhất và đáng tin cậy nhất là yêu cầu mạng không hề tồn tại! Thay vì RPC, giờ đây ta có một phép join luồng giữa các sự kiện mua hàng và các sự kiện cập nhật tỷ giá hối đoái (xem “Phép join luồng-bảng (làm giàu luồng)”).

The join is time-dependent: if the purchase events are reprocessed at a later point in time, the exchange rate will have changed. If you want to reconstruct the original output, you will need to obtain the historical exchange rate at the original time of purchase. No matter whether you query a service or subscribe to a stream of exchange rate updates, you will need to handle this time dependence (see “Time-dependence of joins”).

Phép join này phụ thuộc thời điểm: nếu các sự kiện mua hàng được xử lý lại vào một thời điểm sau, tỷ giá hối đoái sẽ đã thay đổi. Nếu bạn muốn tái dựng đầu ra ban đầu, bạn sẽ cần lấy tỷ giá hối đoái lịch sử tại thời điểm mua hàng ban đầu. Bất kể bạn truy vấn một dịch vụ hay đăng ký theo dõi một luồng các cập nhật tỷ giá hối đoái, bạn sẽ cần xử lý sự phụ thuộc thời điểm này (xem “Tính phụ thuộc thời điểm của phép join”).

Subscribing to a stream of changes, rather than querying the current state when needed, brings us closer to a spreadsheet-like model of computation: when some piece of data changes, any derived data that depends on it can swiftly be updated. There are still many open questions, for example around issues like time-dependent joins, but I believe that building applications around dataflow ideas is a very promising direction to go in.

Việc đăng ký theo dõi một luồng thay đổi, thay vì truy vấn trạng thái hiện tại khi cần, đưa ta tới gần hơn một mô hình tính toán kiểu bảng tính: khi một mẫu dữ liệu nào đó thay đổi, bất kỳ dữ liệu dẫn xuất nào phụ thuộc vào nó đều có thể được cập nhật nhanh chóng. Vẫn còn nhiều câu hỏi bỏ ngỏ, chẳng hạn quanh các vấn đề như phép join phụ thuộc thời điểm, nhưng tôi tin rằng việc xây dựng ứng dụng xoay quanh các ý tưởng về luồng dữ liệu là một hướng đi rất hứa hẹn.

Observing Derived State — Quan sát trạng thái dẫn xuất

At an abstract level, the dataflow systems discussed in the last section give you a process for creating derived datasets (such as search indexes, materialized views, and predictive models) and keeping them up to date. Let’s call that process the write path: whenever some piece of information is written to the system, it may go through multiple stages of batch and stream processing, and eventually every derived dataset is updated to incorporate the data that was written. Figure 12-1 shows an example of updating a search index.

Ở mức trừu tượng, các hệ thống luồng dữ liệu bàn ở phần trước cho bạn một quy trình để tạo các tập dữ liệu dẫn xuất (như chỉ mục tìm kiếm, khung nhìn vật chất hóa, và mô hình dự báo) và giữ chúng được cập nhật. Hãy gọi quy trình đó là đường ghi (write path): mỗi khi một mẫu thông tin nào đó được ghi vào hệ thống, nó có thể đi qua nhiều giai đoạn xử lý theo lô và xử lý luồng, và cuối cùng mọi tập dữ liệu dẫn xuất được cập nhật để kết nạp dữ liệu vừa được ghi. Hình 12-1 cho thấy một ví dụ về việc cập nhật một chỉ mục tìm kiếm.

Figure 12-1. In a search index, writes (document updates) meet reads (queries). — Hình 12-1. Trong một chỉ mục tìm kiếm, các lệnh ghi (cập nhật tài liệu) gặp các lệnh đọc (truy vấn). But why do you create the derived dataset in the first place? Most likely because you want to query it again at a later time. This is the read path: when serving a user request you read from the derived dataset, perhaps perform some more processing on the results, and construct the response to the user.

Nhưng tại sao bạn lại tạo tập dữ liệu dẫn xuất ngay từ đầu? Rất có khả năng vì bạn muốn truy vấn lại nó vào một lúc sau. Đây là đường đọc (read path): khi phục vụ một yêu cầu của người dùng, bạn đọc từ tập dữ liệu dẫn xuất, có lẽ thực hiện thêm chút xử lý trên kết quả, và dựng nên phản hồi cho người dùng.

Taken together, the write path and the read path encompass the whole journey of the data, from the point where it is collected to the point where it is consumed (probably by another human). The write path is the portion of the journey that is precomputed—i.e., that is done eagerly as soon as the data comes in, regardless of whether anyone has asked to see it. The read path is the portion of the journey that only happens when someone asks for it. If you are familiar with functional programming languages, you might notice that the write path is similar to eager evaluation, and the read path is similar to lazy evaluation.

Gộp lại, đường ghi và đường đọc bao trùm toàn bộ hành trình của dữ liệu, từ điểm nó được thu thập tới điểm nó được tiêu thụ (có lẽ bởi một con người khác). Đường ghi là phần của hành trình được tính sẵn — tức là được làm một cách hăm hở ngay khi dữ liệu đến, bất kể có ai đó đã hỏi để xem nó hay chưa. Đường đọc là phần của hành trình chỉ xảy ra khi có người yêu cầu. Nếu bạn quen với các ngôn ngữ lập trình hàm, bạn có thể nhận ra rằng đường ghi giống với đánh giá hăm hở (eager evaluation), còn đường đọc giống với đánh giá lười (lazy evaluation).

The derived dataset is the place where the write path and the read path meet, as illustrated in Figure 12-1. It represents a trade-off between the amount of work that needs to be done at write time and the amount that needs to be done at read time.

Tập dữ liệu dẫn xuất là nơi đường ghi và đường đọc gặp nhau, như minh họa ở Hình 12-1. Nó tượng trưng cho một đánh đổi giữa lượng công việc cần làm lúc ghi và lượng công việc cần làm lúc đọc.

Materialized views and caching — Khung nhìn vật chất hóa và việc đệm

A full-text search index is a good example: the write path updates the index, and the read path searches the index for keywords. Both reads and writes need to do some work. Writes need to update the index entries for all terms that appear in the document. Reads need to search for each of the words in the query, and apply Boolean logic to find documents that contain all of the words in the query (an AND operator), or any synonym of each of the words (an OR operator).

Một chỉ mục tìm kiếm toàn văn là một ví dụ tốt: đường ghi cập nhật chỉ mục, còn đường đọc tìm kiếm chỉ mục theo từ khóa. Cả lệnh đọc lẫn lệnh ghi đều cần làm một số việc. Lệnh ghi cần cập nhật các mục chỉ mục cho mọi từ xuất hiện trong tài liệu. Lệnh đọc cần tìm từng từ trong truy vấn, và áp dụng logic Boole để tìm các tài liệu chứa mọi từ trong truy vấn (một toán tử AND), hoặc bất kỳ từ đồng nghĩa nào của mỗi từ (một toán tử OR).

If you didn't have an index, a search query would have to scan over all documents (like `grep`), which would get very expensive if you had a large number of documents. No index means less work on the write path (no index to update), but a lot more work on the read path.

Nếu bạn không có chỉ mục, một truy vấn tìm kiếm sẽ phải quét qua mọi tài liệu (như grep), điều này sẽ trở nên rất tốn kém nếu bạn có một số lượng lớn tài liệu. Không có chỉ mục nghĩa là ít việc hơn trên đường ghi (không có chỉ mục để cập nhật), nhưng nhiều việc hơn rất nhiều trên đường đọc.

On the other hand, you could imagine precomputing the search results for all possible queries. In that case, you would have less work to do on the read path: no Boolean logic, just find the results for your query and return them. However, the write path would be a lot more expensive: the set of possible search queries that could be asked is infinite, and thus precomputing all possible search results would require infinite time and storage space. That wouldn't work so well.

Mặt khác, bạn có thể hình dung việc tính sẵn kết quả tìm kiếm cho mọi truy vấn khả dĩ. Trong trường hợp đó, bạn sẽ có ít việc phải làm hơn trên đường đọc: không có logic Boole, chỉ việc tìm kết quả cho truy vấn của bạn và trả về chúng. Tuy nhiên, đường ghi sẽ tốn kém hơn rất nhiều: tập các truy vấn tìm kiếm khả dĩ có thể được hỏi là vô hạn, nên việc tính sẵn mọi kết quả tìm kiếm khả dĩ sẽ đòi hỏi thời gian và dung lượng lưu trữ vô hạn. Điều đó sẽ không ổn cho lắm.

Another option would be to precompute the search results for only a fixed set of the most common queries, so that they can be served quickly without having to go to the index. The uncommon queries can still be served from the index. This would generally be called a cache of common queries, although we could also call it a materialized view, as it would need to be updated when new documents appear that should be included in the results of one of the common queries.

Một lựa chọn khác là chỉ tính sẵn kết quả tìm kiếm cho một tập cố định gồm những truy vấn thông dụng nhất, để chúng có thể được phục vụ nhanh chóng mà không phải đi tới chỉ mục. Các truy vấn không thông dụng vẫn có thể được phục vụ từ chỉ mục. Cái này nói chung sẽ được gọi là một bộ nhớ đệm các truy vấn thông dụng, mặc dù ta cũng có thể gọi nó là một khung nhìn vật chất hóa, vì nó cần được cập nhật khi có tài liệu mới xuất hiện mà lẽ ra phải nằm trong kết quả của một trong các truy vấn thông dụng đó.

From this example we can see that an index is not the only possible boundary between the write path and the read path. Caching of common search results is possible, and grep-like scanning without the index is also possible on a small number of documents. Viewed like this, the role of caches, indexes, and materialized views is simple: they shift the boundary between the read path and the write path. They allow us to do more work on the write path, by precomputing results, in order to save effort on the read path.

Từ ví dụ này, ta có thể thấy rằng chỉ mục không phải là ranh giới khả dĩ duy nhất giữa đường ghi và đường đọc. Việc đệm các kết quả tìm kiếm thông dụng là khả thi, và việc quét kiểu grep không cần chỉ mục cũng khả thi trên một số lượng nhỏ tài liệu. Nhìn theo cách này, vai trò của các bộ nhớ đệm, chỉ mục, và khung nhìn vật chất hóa rất đơn giản: chúng dịch chuyển ranh giới giữa đường đọc và đường ghi. Chúng cho phép ta làm nhiều việc hơn trên đường ghi, bằng cách tính sẵn kết quả, để tiết kiệm công sức trên đường đọc.

Shifting the boundary between work done on the write path and the read path was in fact the topic of the Twitter example at the beginning of this book, in “Describing **Load**”. In that example, we also saw how the boundary between write path and read path might be drawn differently for celebrities compared to ordinary users. After 500 pages we have come full circle!

Việc dịch chuyển ranh giới giữa công việc làm trên đường ghi và đường đọc thực ra chính là chủ đề của ví dụ Twitter ở đầu cuốn sách này, trong “Mô tả tải”. Trong ví dụ đó, ta cũng đã thấy ranh giới giữa đường ghi và đường đọc có thể được vạch khác đi như thế nào đối với những người nổi tiếng so với người dùng thường. Sau 500 trang, ta đã đi trọn một

vòng!

Stateful, offline-capable clients — Client có trạng thái, có khả năng hoạt động ngoại tuyến

I find the idea of a boundary between write and read paths interesting because we can discuss shifting that boundary and explore what that shift means in practical terms. Let’s look at the idea in a different context.

Tôi thấy ý tưởng về một ranh giới giữa đường ghi và đường đọc thú vị vì ta có thể bàn về việc dịch chuyển ranh giới đó và khám phá xem sự dịch chuyển ấy có ý nghĩa gì về mặt thực tế. Hãy xét ý tưởng này trong một bối cảnh khác.

The huge popularity of web applications in the last two decades has led us to certain assumptions about application development that are easy to take for granted. In particular, the client/server model—in which clients are largely stateless and servers have the authority over data—is so common that we almost forget that anything else exists. However, technology keeps moving on, and I think it is important to question the status quo from time to time.

Sự phổ biến khổng lồ của các ứng dụng web trong hai thập kỷ qua đã dẫn ta tới một số giả định về việc phát triển ứng dụng mà ta dễ coi là đương nhiên. Cụ thể, mô hình client/server — trong đó các client phần lớn là không trạng thái còn các máy chủ có thẩm quyền đối với dữ liệu — phổ biến tới mức ta gần như quên rằng còn tồn tại thứ gì khác. Tuy nhiên, công nghệ vẫn không ngừng tiến bước, và tôi cho rằng việc đặt câu hỏi về hiện trạng theo thời gian là quan trọng.

Traditionally, web browsers have been stateless clients that can only do useful things when you have an internet connection (just about the only thing you could do offline was to scroll up and down in a page that you had previously loaded while online). However, recent “single-page” JavaScript web apps have gained a lot of **stateful** capabilities, including client-side user interface interaction and persistent local storage in the web browser. Mobile apps can similarly store a lot of state on the device and don’t require a round-trip to the server for most user interactions.

Theo truyền thống, các trình duyệt web là những client không trạng thái chỉ có thể làm những việc hữu ích khi bạn có kết nối internet (gần như việc duy nhất bạn có thể làm khi ngoại tuyến là cuộn lên cuộn xuống trong một trang mà bạn đã tải trước đó khi đang trực tuyến). Tuy nhiên, các ứng dụng web JavaScript “một-trang” (single-page) gần đây đã có được rất nhiều khả năng có trạng thái, bao gồm tương tác giao diện người dùng phía client và lưu trữ cục bộ bền vững trong trình duyệt web. Các ứng dụng di động tương tự cũng có thể lưu rất nhiều trạng thái trên thiết bị và không đòi hỏi một vòng đi-về tới máy chủ cho hầu hết các tương tác của người dùng.

These changing capabilities have led to a renewed interest in offline-first applications that do as much as possible using a local database on the same device, without requiring an internet connection, and sync with remote servers in the background when a network connection is available [42]. Since mobile devices often have slow and unreliable cellular internet connections, it’s a big advantage for users if their user interface does not have to wait for synchronous network requests, and if apps mostly work offline (see “Clients with offline operation”).

Những khả năng đang thay đổi này đã dẫn tới sự quan tâm trở lại đối với các ứng dụng ưu-tiên-ngoại-tuyến (offline-first), vốn làm càng nhiều càng tốt bằng một cơ sở dữ liệu cục bộ trên cùng thiết bị, mà không đòi hỏi kết nối internet, và đồng bộ với các máy chủ từ xa ở chế độ nền khi có kết nối mạng [42]. Vì các thiết bị di động thường có kết nối internet di động chậm và không đáng tin cậy, sẽ là một lợi thế lớn cho người dùng nếu giao diện của

họ không phải chờ các yêu cầu mạng đồng bộ, và nếu các ứng dụng phần lớn hoạt động được khi ngoại tuyến (xem “Client có khả năng hoạt động ngoại tuyến”).

When we move away from the assumption of stateless clients talking to a central database and toward state that is maintained on end-user devices, a world of new opportunities opens up. In particular, we can think of the on-device state as a cache of state on the server. The pixels on the screen are a materialized view onto model objects in the client app; the model objects are a local replica of state in a remote datacenter [27].

Khi ta rời xa giả định về các client không trạng thái trò chuyện với một cơ sở dữ liệu trung tâm và hướng tới trạng thái được duy trì trên các thiết bị của người dùng cuối, một thế giới cơ hội mới mở ra. Cụ thể, ta có thể nghĩ về trạng thái trên-thiết-bị như một bộ nhớ đệm của trạng thái trên máy chủ. Các điểm ảnh trên màn hình là một khung nhìn vật chất hóa trên các đối tượng mô hình trong ứng dụng client; các đối tượng mô hình là một bản sao cục bộ của trạng thái trong một trung tâm dữ liệu từ xa [27].

Pushing state changes to clients — Đẩy các thay đổi trạng thái tới client

In a typical web page, if you load the page in a web browser and the data subsequently changes on the server, the browser does not find out about the change until you reload the page. The browser only reads the data at one point in time, assuming that it is static—it does not subscribe to updates from the server. Thus, the state on the device is a stale cache that is not updated unless you explicitly poll for changes. (HTTP-based feed subscription protocols like RSS are really just a basic form of polling.)

Trong một trang web điển hình, nếu bạn tải trang trong một trình duyệt web và dữ liệu sau đó thay đổi trên máy chủ, trình duyệt không biết về thay đổi đó cho tới khi bạn tải lại trang. Trình duyệt chỉ đọc dữ liệu tại một thời điểm, giả định rằng nó tĩnh — nó không đăng ký theo dõi các cập nhật từ máy chủ. Do đó, trạng thái trên thiết bị là một bộ nhớ đệm cũ kỹ không được cập nhật trừ phi bạn thăm dò thay đổi một cách tường minh. (Các giao thức đăng ký nguồn cấp dựa trên HTTP như RSS thực chất chỉ là một dạng thăm dò cơ bản.)

More recent protocols have moved beyond the basic request/response pattern of HTTP: server-sent events (the EventSource API) and WebSockets provide communication channels by which a web browser can keep an open TCP connection to a server, and the server can actively push messages to the browser as long as it remains connected. This provides an opportunity for the server to actively inform the end-user client about any changes to the state it has stored locally, reducing the staleness of the client-side state.

Các giao thức gần đây hơn đã vượt khỏi mẫu yêu cầu/phản hồi cơ bản của HTTP: các sự kiện do máy chủ gửi (server-sent events, qua EventSource API) và WebSockets cung cấp các kênh giao tiếp mà qua đó một trình duyệt web có thể giữ một kết nối TCP mở tới một máy chủ, và máy chủ có thể chủ động đẩy các thông điệp tới trình duyệt chừng nào nó còn kết nối. Điều này tạo ra một cơ hội để máy chủ chủ động thông báo cho client của người dùng cuối về bất kỳ thay đổi nào đối với trạng thái mà nó đã lưu cục bộ, giảm độ cũ kỹ của trạng thái phía client.

In terms of our model of write path and read path, actively pushing state changes all the way to client devices means extending the write path all the way to the end user. When a client is first initialized, it would still need to use a read path to get its initial state, but thereafter it could rely on a stream of state changes sent by the server. The ideas we discussed around stream processing and messaging are not restricted to running only in a datacenter: we can take the ideas further, and extend them all

the way to end-user devices [43].

Theo mô hình đường ghi và đường đọc của ta, việc chủ động đẩy các thay đổi trạng thái suốt chặng tới các thiết bị client nghĩa là kéo dài đường ghi suốt chặng tới người dùng cuối. Khi một client được khởi tạo lần đầu, nó vẫn cần dùng một đường đọc để lấy trạng thái ban đầu của mình, nhưng sau đó nó có thể dựa vào một luồng các thay đổi trạng thái do máy chủ gửi. Những ý tưởng ta đã bàn quanh việc xử lý luồng và truyền thông điệp không bị giới hạn ở việc chỉ chạy trong một trung tâm dữ liệu: ta có thể đẩy những ý tưởng đó đi xa hơn, và kéo dài chúng suốt chặng tới các thiết bị của người dùng cuối [43].

The devices will be offline some of the time, and unable to receive any notifications of state changes from the server during that time. But we already solved that problem: in “Consumer offsets” we discussed how a consumer of a log-based message broker can reconnect after failing or becoming disconnected, and ensure that it doesn’t miss any messages that arrived while it was disconnected. The same technique works for individual users, where each device is a small subscriber to a small stream of events.

Các thiết bị sẽ ngoại tuyến trong một phần thời gian, và không thể nhận bất kỳ thông báo nào về các thay đổi trạng thái từ máy chủ trong khoảng thời gian đó. Nhưng ta đã giải quyết vấn đề đó rồi: ở “Độ lệch của bên tiêu thụ” ta đã bàn về cách một bên tiêu thụ của một message broker dựa trên log có thể kết nối lại sau khi hỏng hoặc bị ngắt kết nối, và đảm bảo nó không bỏ sót bất kỳ thông điệp nào đã đến trong lúc nó bị ngắt kết nối. Cùng kỹ thuật đó hoạt động cho từng người dùng riêng lẻ, trong đó mỗi thiết bị là một bên đăng ký nhỏ theo dõi một luồng sự kiện nhỏ.

End-to-end event streams — Luồng sự kiện đầu-cuối

Recent tools for developing stateful clients and user interfaces, such as the Elm language [30] and Facebook’s toolchain of React, Flux, and Redux [44], already manage internal client-side state by subscribing to a stream of events representing user input or responses from a server, structured similarly to event sourcing (see “Event Sourcing”).

Các công cụ gần đây để phát triển các client và giao diện người dùng có trạng thái, chẳng hạn ngôn ngữ Elm [30] và bộ công cụ React, Flux, và Redux của Facebook [44], vốn đã quản lý trạng thái phía client nội bộ bằng cách đăng ký theo dõi một luồng sự kiện đại diện cho đầu vào của người dùng hoặc các phản hồi từ một máy chủ, được cấu trúc tương tự như nguồn-sự-kiện (xem “Nguồn-sự-kiện”).

It would be very natural to extend this programming model to also allow a server to push state-change events into this client-side event pipeline. Thus, state changes could flow through an end-to-end write path: from the interaction on one device that triggers a state change, via event logs and through several derived data systems and stream processors, all the way to the user interface of a person observing the state on another device. These state changes could be propagated with fairly low delay—say, under one second end to end.

Sẽ rất tự nhiên khi mở rộng mô hình lập trình này để cũng cho phép một máy chủ đẩy các sự kiện thay đổi trạng thái vào đường ống sự kiện phía client này. Như vậy, các thay đổi trạng thái có thể chảy qua một đường ghi đầu-cuối: từ tương tác trên một thiết bị vốn kích hoạt một thay đổi trạng thái, qua các log sự kiện và qua vài hệ thống dữ liệu dẫn xuất cùng các bộ xử lý luồng, suốt chặng tới giao diện người dùng của một người đang quan sát trạng thái trên một thiết bị khác. Những thay đổi trạng thái này có thể được lan truyền với độ trễ khá thấp — chẳng hạn, dưới một giây từ đầu tới cuối.

Some applications, such as instant messaging and online games, already have such a “real-time” architecture (in the sense of interactions with low delay, not in the sense of “**Response time** guarantees”). But why don’t we build all applications this way?

Một số ứng dụng, chẳng hạn nhắn tin tức thời và trò chơi trực tuyến, vốn đã có một kiến trúc “thời gian thực” như vậy (theo nghĩa các tương tác có độ trễ thấp, chứ không theo nghĩa “Đảm bảo thời gian phản hồi”). Nhưng tại sao ta lại không xây dựng mọi ứng dụng theo cách này?

The challenge is that the assumption of stateless clients and request/response interactions is very deeply ingrained in our databases, libraries, frameworks, and protocols. Many datastores support read and write operations where a request returns one response, but much fewer provide an ability to subscribe to changes—i.e., a request that returns a stream of responses over time (see “API support for change streams”).

Thách thức là giả định về các client không trạng thái và các tương tác yêu cầu/phản hồi đã ăn sâu rất nhiều vào các cơ sở dữ liệu, thư viện, framework, và giao thức của ta. Nhiều kho dữ liệu hỗ trợ các thao tác đọc và ghi mà một yêu cầu trả về một phản hồi, nhưng ít kho hơn nhiều cung cấp khả năng đăng ký theo dõi các thay đổi — tức là một yêu cầu trả về một luồng phản hồi theo thời gian (xem “Hỗ trợ API cho luồng thay đổi”).

In order to extend the write path all the way to the end user, we would need to fundamentally rethink the way we build many of these systems: moving away from request/response interaction and toward publish/subscribe dataflow [27]. I think that the advantages of more responsive user interfaces and better offline support would make it worth the effort. If you are designing data systems, I hope that you will keep in mind the option of subscribing to changes, not just querying the current state.

Để kéo dài đường ghi suốt chặng tới người dùng cuối, ta sẽ cần suy nghĩ lại một cách căn bản về cách ta xây dựng nhiều hệ thống này: rời xa tương tác yêu cầu/phản hồi và hướng tới luồng dữ liệu xuất bản/đăng ký (publish/subscribe) [27]. Tôi cho rằng những lợi thế của giao diện người dùng phản hồi nhạy hơn và hỗ trợ ngoại tuyến tốt hơn sẽ khiến nỗ lực này đáng giá. Nếu bạn đang thiết kế các hệ thống dữ liệu, tôi hy vọng bạn sẽ ghi nhớ trong đầu lựa chọn đăng ký theo dõi các thay đổi, chứ không chỉ truy vấn trạng thái hiện tại.

Reads are events too — Lệnh đọc cũng là sự kiện

We discussed that when a stream processor writes derived data to a store (database, cache, or index), and when user requests query that store, the store acts as the boundary between the write path and the read path. The store allows random-access read queries to the data that would otherwise require scanning the whole event log.

Ta đã bàn rằng khi một bộ xử lý luồng ghi dữ liệu dẫn xuất vào một kho (cơ sở dữ liệu, bộ nhớ đệm, hoặc chỉ mục), và khi các yêu cầu của người dùng truy vấn kho đó, thì kho hành xử như ranh giới giữa đường ghi và đường đọc. Kho cho phép các truy vấn đọc truy cập ngẫu nhiên tới dữ liệu mà nếu không có nó sẽ đòi hỏi quét toàn bộ log sự kiện.

In many cases, the data storage is separate from the streaming system. But recall that stream processors also need to maintain state to perform aggregations and joins (see “Stream Joins”). This state is normally hidden inside the stream processor, but some frameworks allow it to also be queried by outside clients [45], turning the stream processor itself into a kind of simple database.

Trong nhiều trường hợp, kho lưu trữ dữ liệu tách biệt với hệ thống luồng. Nhưng hãy nhớ rằng các bộ xử lý luồng cũng cần duy trì trạng thái để thực hiện các phép tổng hợp và join (xem “Phép join luồng”). Trạng thái này thường được giấu bên trong bộ xử lý luồng,

nhưng một số framework cho phép nó cũng được truy vấn bởi các client bên ngoài [45], biến bản thân bộ xử lý luồng thành một dạng cơ sở dữ liệu đơn giản.

I would like to take that idea further. As discussed so far, the writes to the store go through an event log, while reads are transient network requests that go directly to the nodes that store the data being queried. This is a reasonable design, but not the only possible one. It is also possible to represent read requests as streams of events, and send both the read events and the write events through a stream processor; the processor responds to read events by emitting the result of the read to an output stream [46].

Tôi muốn đẩy ý tưởng đó đi xa hơn. Như đã bàn từ đầu tới giờ, các lệnh ghi vào kho đi qua một log sự kiện, trong khi các lệnh đọc là các yêu cầu mạng thoáng qua đi thẳng tới các nút lưu dữ liệu đang được truy vấn. Đây là một thiết kế hợp lý, nhưng không phải là thiết kế khả dĩ duy nhất. Ta cũng có thể biểu diễn các yêu cầu đọc dưới dạng các luồng sự kiện, và gửi cả sự kiện đọc lẫn sự kiện ghi qua một bộ xử lý luồng; bộ xử lý phản hồi các sự kiện đọc bằng cách phát ra kết quả của lệnh đọc tới một luồng đầu ra [46].

When both the writes and the reads are represented as events, and routed to the same stream operator in order to be handled, we are in fact performing a stream-table join between the stream of read queries and the database. The read event needs to be sent to the database partition holding the data (see “Request Routing”), just like batch and stream processors need to copartition inputs on the same key when joining (see “Reduce-Side Joins and Grouping”).

Khi cả lệnh ghi lẫn lệnh đọc đều được biểu diễn dưới dạng sự kiện, và được định tuyến tới cùng một toán tử luồng để được xử lý, thì thực ra ta đang thực hiện một phép join luồng-bảng giữa luồng các truy vấn đọc và cơ sở dữ liệu. Sự kiện đọc cần được gửi tới mảnh cơ sở dữ liệu giữ dữ liệu (xem “Định tuyến yêu cầu”), giống như các bộ xử lý theo lô và xử lý luồng cần phân mảnh đồng bộ (copartition) các đầu vào theo cùng một khóa khi join (xem “Phép join phía Reduce và gom nhóm”).

This correspondence between serving requests and performing joins is quite fundamental [47]. A one-off read request just passes the request through the join operator and then immediately forgets it; a subscribe request is a persistent join with past and future events on the other side of the join.

Sự tương ứng này giữa việc phục vụ yêu cầu và việc thực hiện phép join là khá nền tảng [47]. Một yêu cầu đọc một lần chỉ đơn giản đưa yêu cầu qua toán tử join rồi ngay lập tức quên nó đi; một yêu cầu đăng ký theo dõi là một phép join bền vững với các sự kiện quá khứ và tương lai ở phía bên kia của phép join.

Recording a log of read events potentially also has benefits with regard to tracking causal dependencies and data provenance across a system: it would allow you to reconstruct what the user saw before they made a particular decision. For example, in an online shop, it is likely that the predicted shipping date and the inventory status shown to a customer affect whether they choose to buy an item [4]. To analyze this connection, you need to record the result of the user’s query of the shipping and inventory status.

Việc ghi lại một log các sự kiện đọc cũng có thể có những lợi ích về việc theo dõi các phụ thuộc nhân quả và nguồn gốc (provenance) của dữ liệu xuyên một hệ thống: nó sẽ cho phép bạn tái dựng những gì người dùng đã thấy trước khi họ ra một quyết định cụ thể. Chẳng hạn, trong một cửa hàng trực tuyến, rất có khả năng ngày giao hàng dự kiến và tình trạng tồn kho hiển thị cho khách hàng ảnh hưởng tới việc họ có chọn mua một món hàng hay không [4]. Để phân tích mối liên hệ này, bạn cần ghi lại kết quả truy vấn của người dùng về tình trạng giao hàng và tồn kho.

Writing read events to durable storage thus enables better tracking of causal dependencies (see

“Ordering events to capture causality”), but it incurs additional storage and I/O cost. Optimizing such systems to reduce the overhead is still an open research problem [2]. But if you already log read requests for operational purposes, as a **side effect** of request processing, it is not such a great change to make the log the source of the requests instead.

Việc ghi các sự kiện đọc vào kho lưu trữ bền vững do đó cho phép theo dõi tốt hơn các phụ thuộc nhân quả (xem “Định thứ tự sự kiện để nắm bắt quan hệ nhân quả”), nhưng nó phát sinh thêm chi phí lưu trữ và I/O. Việc tối ưu những hệ thống như vậy để giảm phần phụ trội vẫn là một bài toán nghiên cứu còn bỏ ngỏ [2]. Nhưng nếu bạn đã ghi log các yêu cầu đọc cho mục đích vận hành, như một tác dụng phụ của việc xử lý yêu cầu, thì việc lấy chính log đó làm nguồn của các yêu cầu không phải là một thay đổi quá lớn.

Multi-partition data processing — Xử lý dữ liệu đa-mảnh

For queries that only touch a single partition, the effort of sending queries through a stream and collecting a stream of responses is perhaps overkill. However, this idea opens the possibility of distributed execution of complex queries that need to combine data from several partitions, taking advantage of the infrastructure for message routing, partitioning, and joining that is already provided by stream processors.

Với những truy vấn chỉ chạm một mảnh duy nhất, công sức gửi các truy vấn qua một luồng và thu thập một luồng phản hồi có lẽ là thừa thãi. Tuy nhiên, ý tưởng này mở ra khả năng thực thi phân tán các truy vấn phức tạp cần kết hợp dữ liệu từ vài mảnh, tận dụng hạ tầng cho việc định tuyến thông điệp, phân mảnh, và join vốn đã được các bộ xử lý luồng cung cấp.

Storm’s distributed RPC feature supports this usage pattern (see “Message passing and RPC”). For example, it has been used to compute the number of people who have seen a URL on Twitter—i.e., the union of the follower sets of everyone who has tweeted that URL [48]. As the set of Twitter users is partitioned, this computation requires combining results from many partitions.

Tính năng RPC phân tán của Storm hỗ trợ mẫu sử dụng này (xem “Truyền thông điệp và RPC”). Chẳng hạn, nó đã được dùng để tính số người đã thấy một URL trên Twitter — tức là hợp của các tập người theo dõi của mọi người đã tweet URL đó [48]. Vì tập người dùng Twitter được phân mảnh, phép tính này đòi hỏi kết hợp kết quả từ nhiều mảnh.

Another example of this pattern occurs in fraud prevention: in order to assess the risk of whether a particular purchase event is fraudulent, you can examine the reputation scores of the user’s IP address, email address, billing address, shipping address, and so on. Each of these reputation databases is itself partitioned, and so collecting the scores for a particular purchase event requires a sequence of joins with differently partitioned datasets [49].

Một ví dụ khác của mẫu này xuất hiện trong phòng chống gian lận: để đánh giá rủi ro liệu một sự kiện mua hàng cụ thể có gian lận hay không, bạn có thể xem xét điểm uy tín của địa chỉ IP, địa chỉ email, địa chỉ thanh toán, địa chỉ giao hàng của người dùng, v.v. Mỗi cơ sở dữ liệu uy tín này bản thân nó được phân mảnh, nên việc thu thập các điểm cho một sự kiện mua hàng cụ thể đòi hỏi một chuỗi các phép join với các tập dữ liệu được phân mảnh khác nhau [49].

The internal query execution graphs of MPP databases have similar characteristics (see “Comparing Hadoop to Distributed Databases”). If you need to perform this kind of multi-partition join, it is probably simpler to use a database that provides this feature than to implement it using a stream processor. However, treating queries as streams provides an option for implementing large-scale applications that run against the limits of conventional off-the-shelf solutions.

Các đồ thị thực thi truy vấn nội bộ của các cơ sở dữ liệu MPP có những đặc tính tương tự (xem “So sánh Hadoop với cơ sở dữ liệu phân tán”). Nếu bạn cần thực hiện kiểu join đa-mảnh này, có lẽ đơn giản hơn khi dùng một cơ sở dữ liệu cung cấp tính năng này thay vì tự hiện thực nó bằng một bộ xử lý luồng. Tuy nhiên, việc coi các truy vấn như các luồng cung cấp một lựa chọn để hiện thực các ứng dụng quy mô lớn vốn chạm tới giới hạn của các giải pháp đóng-hộp thông thường.

Aiming for Correctness — Hướng tới sự đúng đắn

With stateless services that only read data, it is not a big deal if something goes wrong: you can fix the **bug** and restart the service, and everything returns to normal. Stateful systems such as databases are not so simple: they are designed to remember things forever (more or less), so if something goes wrong, the effects also potentially last forever—which means they require more careful thought [50].

Với các dịch vụ không trạng thái chỉ đọc dữ liệu, nếu có gì đó trục trặc thì cũng không phải vấn đề lớn: bạn có thể sửa bug và khởi động lại dịch vụ, và mọi thứ trở lại bình thường. Các hệ thống có trạng thái như cơ sở dữ liệu thì không đơn giản như vậy: chúng được thiết kế để nhớ mọi thứ mãi mãi (gần như vậy), nên nếu có gì đó trục trặc, các tác động cũng có thể kéo dài mãi mãi — điều này nghĩa là chúng đòi hỏi suy nghĩ cẩn thận hơn [50].

We want to build applications that are reliable and correct (i.e., programs whose semantics are well defined and understood, even in the face of various faults). For approximately four decades, the transaction properties of atomicity, isolation, and durability (Chapter 7) have been the tools of choice for building correct applications. However, those foundations are weaker than they seem: witness for example the confusion of weak isolation levels (see “Weak Isolation Levels”).

Ta muốn xây dựng những ứng dụng đáng tin cậy và đúng đắn (tức là những chương trình có ngữ nghĩa được định nghĩa và hiểu rõ, ngay cả khi đối mặt với nhiều lỗi khác nhau). Trong khoảng bốn thập kỷ, các thuộc tính giao dịch là tính nguyên tử (atomicity), tính cô lập (isolation), và độ bền (durability) (Chương 7) đã là những công cụ được chọn lựa để xây dựng các ứng dụng đúng đắn. Tuy nhiên, những nền tảng đó yếu hơn về ngoài của chúng: hãy lấy ví dụ sự lộn xộn của các mức cô lập yếu (xem “Các mức cô lập yếu”).

In some areas, transactions are being abandoned entirely and replaced with models that offer better performance and scalability, but much messier semantics (see for example “Leaderless Replication”). Consistency is often talked about, but poorly defined (see “Consistency” and Chapter 9). Some people assert that we should “embrace weak consistency” for the sake of better availability, while lacking a clear idea of what that actually means in practice.

Ở một số lĩnh vực, các giao dịch đang bị từ bỏ hoàn toàn và được thay bằng các mô hình cho hiệu năng và khả năng mở rộng tốt hơn, nhưng có ngữ nghĩa lộn xộn hơn nhiều (xem ví dụ “Sao chép không-leader”). Tính nhất quán (consistency) thường được nói đến, nhưng được định nghĩa lỏng lẻo (xem “Tính nhất quán” và Chương 9). Một số người khẳng định rằng ta nên “ôm lấy nhất quán yếu” vì tính sẵn sàng tốt hơn, trong khi lại thiếu một ý niệm rõ ràng về điều đó thực sự có nghĩa gì trong thực tế.

For a topic that is so important, our understanding and our engineering methods are surprisingly flaky. For example, it is very difficult to determine whether it is safe to run a particular application

at a particular transaction isolation level or replication configuration [51, 52]. Often simple solutions appear to work correctly when concurrency is low and there are no faults, but turn out to have many subtle bugs in more demanding circumstances.

Với một chủ đề quan trọng đến vậy, sự hiểu biết và các phương pháp kỹ thuật của ta lại lung lay một cách đáng ngạc nhiên. Chẳng hạn, rất khó xác định liệu có an toàn hay không khi chạy một ứng dụng cụ thể ở một mức cô lập giao dịch hoặc cấu hình sao chép cụ thể [51, 52]. Thường thì các lời giải đơn giản có vẻ hoạt động đúng khi mức tương tranh thấp và không có lỗi, nhưng hóa ra lại có nhiều bug tinh vi trong những hoàn cảnh khắc nghiệt hơn.

For example, Kyle Kingsbury's Jepsen experiments [53] have highlighted the stark discrepancies between some products' claimed safety guarantees and their actual behavior in the presence of network problems and crashes. Even if infrastructure products like databases were free from problems, application code would still need to correctly use the features they provide, which is error-prone if the configuration is hard to understand (which is the case with weak isolation levels, quorum configurations, and so on).

Chẳng hạn, các thí nghiệm Jepsen của Kyle Kingsbury [53] đã làm nổi bật những khác biệt rành rành giữa các đảm bảo an toàn mà một số sản phẩm tuyên bố và hành vi thực tế của chúng khi có các vấn đề về mạng và sự cố. Ngay cả khi các sản phẩm hạ tầng như cơ sở dữ liệu hoàn toàn không có vấn đề, mã ứng dụng vẫn cần dùng đúng các tính năng mà chúng cung cấp, điều này dễ sai nếu cấu hình khó hiểu (vốn là trường hợp của các mức cô lập yếu, cấu hình đa số (quorum), v.v.).

If your application can tolerate occasionally corrupting or losing data in unpredictable ways, life is a lot simpler, and you might be able to get away with simply crossing your fingers and hoping for the best. On the other hand, if you need stronger assurances of correctness, then serializability and atomic commit are established approaches, but they come at a cost: they typically only work in a single datacenter (ruling out geographically distributed architectures), and they limit the scale and fault-tolerance properties you can achieve.

Nếu ứng dụng của bạn có thể chịu được việc thi thoảng làm hỏng hoặc mất dữ liệu theo những cách không lường trước, cuộc sống đơn giản hơn nhiều, và bạn có thể qua chuyện bằng cách đơn giản bắt chéo ngón tay và hy vọng điều tốt đẹp nhất. Mặt khác, nếu bạn cần những bảo chứng mạnh hơn về sự đúng đắn, thì khả năng tuần tự hóa (serializability) và cam kết nguyên tử là những cách tiếp cận đã được kiểm chứng, nhưng chúng đi kèm cái giá: chúng thường chỉ hoạt động trong một trung tâm dữ liệu duy nhất (loại trừ các kiến trúc phân tán về mặt địa lý), và chúng giới hạn các thuộc tính về quy mô và khả năng chịu lỗi mà bạn có thể đạt được.

While the traditional transaction approach is not going away, I also believe it is not the last word in making applications correct and **resilient** to faults. In this section I will suggest some ways of thinking about correctness in the context of dataflow architectures.

Mặc dù cách tiếp cận giao dịch truyền thống không biến mất, tôi cũng tin rằng nó không phải là lời cuối cùng trong việc làm cho các ứng dụng đúng đắn và kiên cường trước lỗi. Trong phần này tôi sẽ gợi ý một số cách suy nghĩ về sự đúng đắn trong bối cảnh các kiến trúc luồng dữ liệu.

The End-to-End Argument for Databases — Lập luận đầu-cuối cho cơ sở dữ liệu

Just because an application uses a data system that provides comparatively strong safety properties, such as serializable transactions, that does not **mean** the application is guaranteed to be free from data loss or corruption. For example, if an application has a bug that causes it to write incorrect data, or delete data from a database, serializable transactions aren't going to save you.

Chỉ vì một ứng dụng dùng một hệ thống dữ liệu cung cấp các thuộc tính an toàn tương đối mạnh, chẳng hạn các giao dịch khả tuần tự hóa, điều đó không có nghĩa là ứng dụng được đảm bảo không bị mất hay hỏng dữ liệu. Chẳng hạn, nếu một ứng dụng có một bug khiến nó ghi dữ liệu sai, hoặc xóa dữ liệu khỏi một cơ sở dữ liệu, thì các giao dịch khả tuần tự hóa sẽ chẳng cứu được bạn.

This example may seem frivolous, but it is worth taking seriously: application bugs occur, and people make mistakes. I used this example in “State, Streams, and Immutability” to argue in favor of immutable and append-only data, because it is easier to recover from such mistakes if you remove the ability of faulty code to destroy good data.

Ví dụ này thoạt nghe có vẻ tầm phào, nhưng đáng để xem xét nghiêm túc: bug ứng dụng vẫn xảy ra, và con người vẫn mắc sai lầm. Tôi đã dùng ví dụ này ở “Trạng thái, luồng, và tính bất biến” để lập luận ủng hộ dữ liệu bất biến và chỉ-thêm, bởi việc phục hồi sau những sai lầm như vậy dễ hơn nếu bạn loại bỏ khả năng mã lỗi phá hủy dữ liệu tốt.

Although immutability is useful, it is not a cure-all by itself. Let's look at a more subtle example of data corruption that can occur.

Mặc dù tính bất biến hữu ích, bản thân nó không phải là thuốc chữa bách bệnh. Hãy xét một ví dụ tinh vi hơn về hỏng dữ liệu có thể xảy ra.

Exactly-once execution of an operation — Thực thi đúng-một-lần một thao tác

In “Fault Tolerance” we encountered an idea called exactly-once (or effectively-once) semantics. If something goes wrong while processing a message, you can either give up (drop the message—i.e., incur data loss) or try again. If you try again, there is the risk that it actually succeeded the first time, but you just didn't find out about the success, and so the message ends up being processed twice.

Ở “Khả năng chịu lỗi” ta đã gặp một ý tưởng gọi là ngữ nghĩa đúng-một-lần (exactly-once, hay hiệu-quả-một-lần — effectively-once). Nếu có gì đó trục trặc khi đang xử lý một thông điệp, bạn có thể hoặc bỏ cuộc (vứt thông điệp đi — tức là chịu mất dữ liệu) hoặc thử lại. Nếu bạn thử lại, có rủi ro là nó thực ra đã thành công ở lần đầu, chỉ là bạn không biết về sự thành công đó, và do đó thông điệp rốt cuộc bị xử lý hai lần.

Processing twice is a form of data corruption: it is undesirable to charge a customer twice for the same service (billing them too much) or increment a counter twice (overstating some metric). In this context, exactly-once means arranging the computation such that the final effect is the same as if no faults had occurred, even if the operation actually was retried due to some fault. We previously discussed a few approaches for achieving this goal.

Xử lý hai lần là một dạng hỏng dữ liệu: việc tính tiền khách hàng hai lần cho cùng một dịch vụ (thu của họ quá nhiều) hoặc tăng một bộ đếm hai lần (làm phóng đại một chỉ số nào đó) là điều không mong muốn. Trong bối cảnh này, đúng-một-lần nghĩa là sắp đặt phép tính sao cho tác động cuối cùng giống hệt như thể không có lỗi nào xảy ra, ngay cả khi thao tác

thực sự đã được thử lại do một lỗi nào đó. Trước đây ta đã bàn về một vài cách tiếp cận để đạt mục tiêu này.

One of the most effective approaches is to make the operation idempotent (see “Idempotence”); that is, to ensure that it has the same effect, no matter whether it is executed once or multiple times. However, taking an operation that is not naturally idempotent and making it idempotent requires some effort and care: you may need to maintain some additional metadata (such as the set of operation IDs that have updated a value), and ensure fencing when failing over from one node to another (see “The leader and the lock”).

Một trong những cách tiếp cận hiệu quả nhất là làm cho thao tác trở nên lũy đẳng (xem “Tính lũy đẳng”); tức là đảm bảo nó có cùng tác động, bất kể nó được thực thi một lần hay nhiều lần. Tuy nhiên, việc lấy một thao tác không lũy đẳng tự nhiên và làm cho nó lũy đẳng đòi hỏi đôi chút nỗ lực và cẩn trọng: bạn có thể cần duy trì một số siêu dữ liệu bổ sung (chẳng hạn tập các ID thao tác đã cập nhật một giá trị), và đảm bảo việc rào chắn (fencing) khi chuyển đổi dự phòng từ một nút sang một nút khác (xem “Leader và khóa”).

Duplicate suppression — Triệt tiêu trùng lặp

The same pattern of needing to suppress duplicates occurs in many other places besides stream processing. For example, TCP uses sequence numbers on packets to put them in the correct order at the recipient, and to determine whether any packets were lost or duplicated on the network. Any lost packets are retransmitted and any duplicates are removed by the TCP stack before it hands the data to an application.

Cùng mẫu cần triệt tiêu trùng lặp này xuất hiện ở nhiều nơi khác ngoài xử lý luồng. Chẳng hạn, TCP dùng các số tuần tự trên các gói tin để đặt chúng đúng thứ tự ở bên nhận, và để xác định liệu có gói tin nào bị mất hay nhân đôi trên mạng hay không. Mọi gói tin bị mất đều được truyền lại và mọi gói tin trùng lặp đều bị tăng TCP loại bỏ trước khi nó trao dữ liệu cho ứng dụng.

However, this duplicate suppression only works within the context of a single TCP connection. Imagine the TCP connection is a client’s connection to a database, and it is currently executing the transaction in Example 12-1. In many databases, a transaction is tied to a client connection (if the client sends several queries, the database knows that they belong to the same transaction because they are sent on the same TCP connection). If the client suffers a network interruption and connection timeout after sending the COMMIT, but before hearing back from the database server, it does not know whether the transaction has been committed or aborted (Figure 8-1).

Tuy nhiên, việc triệt tiêu trùng lặp này chỉ hoạt động trong phạm vi một kết nối TCP duy nhất. Hãy hình dung kết nối TCP là kết nối của một client tới một cơ sở dữ liệu, và nó hiện đang thực thi giao dịch trong Ví dụ 12-1. Trong nhiều cơ sở dữ liệu, một giao dịch gắn với một kết nối client (nếu client gửi vài truy vấn, cơ sở dữ liệu biết rằng chúng thuộc cùng một giao dịch vì chúng được gửi trên cùng một kết nối TCP). Nếu client bị gián đoạn mạng và hết thời gian chờ kết nối sau khi gửi COMMIT, nhưng trước khi nghe phản hồi từ máy chủ cơ sở dữ liệu, nó không biết liệu giao dịch đã được cam kết hay đã bị hủy bỏ (Hình 8-1).

Example 12-1. A nonidempotent transfer of money from one account to another — Ví dụ 12-1. Một lệnh chuyển tiền không lũy đẳng từ tài khoản này sang tài khoản khác

```
BEGIN TRANSACTION;
```

```
UPDATE accounts SET balance = balance + 11.00 WHERE account_id = 1234;
```

```
UPDATE accounts SET balance = balance - 11.00 WHERE account_id = 4321;  
COMMIT;
```

The client can reconnect to the database and retry the transaction, but now it is outside of the scope of TCP duplicate suppression. Since the transaction in Example 12-1 is not idempotent, it could happen that \$22 is transferred instead of the desired \$11. Thus, even though Example 12-1 is a standard example for transaction atomicity, it is actually not correct, and real banks do not work like this [3].

Client có thể kết nối lại tới cơ sở dữ liệu và thử lại giao dịch, nhưng giờ đây nó nằm ngoài phạm vi triệt tiêu trùng lặp của TCP. Vì giao dịch trong Ví dụ 12-1 không lũy đẳng, có thể xảy ra việc 22 đô la bị chuyển thay vì 11 đô la như mong muốn. Do đó, mặc dù Ví dụ 12-1 là một ví dụ tiêu chuẩn cho tính nguyên tử của giao dịch, nó thực ra không đúng đắn, và các ngân hàng thực không hoạt động như thế này [3].

Two-phase commit (see “Atomic Commit and Two-Phase Commit (2PC)”) protocols break the 1:1 mapping between a TCP connection and a transaction, since they must allow a transaction coordinator to reconnect to a database after a network fault, and tell it whether to commit or abort an in-doubt transaction. Is this sufficient to ensure that the transaction will only be executed once? Unfortunately not.

Các giao thức cam kết hai pha (xem “Cam kết nguyên tử và Cam kết hai pha (2PC)”) phá vỡ ánh xạ 1:1 giữa một kết nối TCP và một giao dịch, bởi chúng phải cho phép một bộ điều phối giao dịch kết nối lại tới một cơ sở dữ liệu sau một lỗi mạng, và bảo nó cam kết hay hủy bỏ một giao dịch đang ở trạng thái nghi ngờ. Liệu điều này có đủ để đảm bảo giao dịch sẽ chỉ được thực thi một lần? Đáng tiếc là không.

Even if we can suppress duplicate transactions between the database client and server, we still need to worry about the network between the end-user device and the application server. For example, if the end-user client is a web browser, it probably uses an HTTP POST request to submit an instruction to the server. Perhaps the user is on a weak cellular data connection, and they succeed in sending the POST, but the signal becomes too weak before they are able to receive the response from the server.

Ngay cả khi ta có thể triệt tiêu các giao dịch trùng lặp giữa client và máy chủ cơ sở dữ liệu, ta vẫn cần lo về mạng giữa thiết bị của người dùng cuối và máy chủ ứng dụng. Chẳng hạn, nếu client của người dùng cuối là một trình duyệt web, nó có lẽ dùng một yêu cầu HTTP POST để gửi một chỉ thị tới máy chủ. Có lẽ người dùng đang ở trên một kết nối dữ liệu di động yếu, và họ gửi thành công yêu cầu POST, nhưng tín hiệu trở nên quá yếu trước khi họ có thể nhận được phản hồi từ máy chủ.

In this case, the user will probably be shown an error message, and they may retry manually. Web browsers warn, “Are you sure you want to submit this form again?”—and the user says yes, because they wanted the operation to happen. (The Post/Redirect/Get pattern [54] avoids this warning message in normal operation, but it doesn’t help if the POST request times out.) From the web server’s point of view the retry is a separate request, and from the database’s point of view it is a separate transaction. The usual deduplication mechanisms don’t help.

Trong trường hợp này, người dùng có lẽ sẽ thấy một thông điệp lỗi, và họ có thể thử lại một cách thủ công. Các trình duyệt web cảnh báo, “Bạn có chắc muốn gửi lại biểu mẫu này không?” — và người dùng nói có, vì họ muốn thao tác đó xảy ra. (Mẫu Post/Redirect/Get [54] tránh thông điệp cảnh báo này trong hoạt động bình thường, nhưng nó không giúp ích nếu yêu cầu POST hết thời gian chờ.) Từ góc nhìn của máy chủ web, lần thử lại là một yêu cầu riêng biệt, và từ góc nhìn của cơ sở dữ liệu, nó là một giao dịch riêng biệt. Các cơ chế khử trùng lặp thông thường không giúp được gì.

Operation identifiers — Định danh thao tác

To make the operation idempotent through several hops of network communication, it is not sufficient to rely just on a transaction mechanism provided by a database—you need to consider the end-to-end flow of the request.

Để làm cho thao tác trở nên lũy đẳng qua vài chặng giao tiếp mạng, chỉ dựa vào một cơ chế giao dịch do cơ sở dữ liệu cung cấp là không đủ — bạn cần xét tới luồng đầu-cuối của yêu cầu.

For example, you could generate a unique identifier for an operation (such as a UUID) and include it as a hidden form field in the client application, or calculate a hash of all the relevant form fields to derive the operation ID [3]. If the web browser submits the POST request twice, the two requests will have the same operation ID. You can then pass that operation ID all the way through to the database and check that you only ever execute one operation with a given ID, as shown in Example 12-2.

Chẳng hạn, bạn có thể sinh một định danh duy nhất cho một thao tác (như một UUID) và đưa nó vào như một trường biểu mẫu ẩn trong ứng dụng client, hoặc tính một mã băm của mọi trường biểu mẫu liên quan để dẫn xuất ID thao tác [3]. Nếu trình duyệt web gửi yêu cầu POST hai lần, hai yêu cầu sẽ có cùng một ID thao tác. Sau đó bạn có thể truyền ID thao tác đó suốt chặng tới cơ sở dữ liệu và kiểm tra rằng bạn chỉ từng thực thi một thao tác với một ID cho trước, như minh họa ở Ví dụ 12-2.

Example 12-2. Suppressing duplicate requests using a unique ID — Ví dụ 12-2. Triệt tiêu các yêu cầu trùng lặp bằng một ID duy nhất

```
ALTER TABLE requests ADD UNIQUE (request_id);
BEGIN TRANSACTION;
INSERT INTO requests
  (request_id, from_account, to_account, amount)
VALUES ('0286FDB8-D7E1-423F-B40B-792B3608036C', 4321, 1234, 11.00);
UPDATE accounts SET balance = balance + 11.00 WHERE account_id = 1234;
UPDATE accounts SET balance = balance - 11.00 WHERE account_id = 4321;
COMMIT;
```

Example 12-2 relies on a uniqueness constraint on the `request_id` column. If a transaction attempts to insert an ID that already exists, the `INSERT` fails and the transaction is aborted, preventing it from taking effect twice. Relational databases can generally maintain a uniqueness constraint correctly, even at weak isolation levels (whereas an application-level check-then-insert may fail under nonserializable isolation, as discussed in “Write Skew and Phantoms”).

Ví dụ 12-2 dựa vào một ràng buộc tính duy nhất trên cột `request_id`. Nếu một giao dịch cố chèn một ID đã tồn tại, lệnh `INSERT` thất bại và giao dịch bị hủy bỏ, ngăn nó có hiệu lực hai lần. Các cơ sở dữ liệu quan hệ nói chung có thể duy trì một ràng buộc tính duy nhất một cách đúng đắn, ngay cả ở các mức cô lập yếu (trong khi một phép kiểm-tra-rồi-chèn ở cấp ứng dụng có thể thất bại dưới mức cô lập phi-khả-tuần-tự, như đã bàn ở “Lịch ghi và bóng ma”).

Besides suppressing duplicate requests, the `requests` table in Example 12-2 acts as a kind of event log, hinting in the direction of event sourcing (see “Event Sourcing”). The updates to the account balances don’t actually have to happen in the same transaction as the insertion of the event, since they are redundant and could be derived from the request event in a downstream consumer—as long as the event is processed exactly once, which can again be enforced using the request ID.

Bên cạnh việc triệt tiêu các yêu cầu trùng lặp, bảng requests trong Ví dụ 12-2 hành xử như một dạng log sự kiện, gợi ý theo hướng nguồn-sự-kiện (xem “Nguồn-sự-kiện”). Các cập nhật số dư tài khoản thực ra không nhất thiết phải xảy ra trong cùng giao dịch với việc chèn sự kiện, bởi chúng dư thừa và có thể được dẫn xuất từ sự kiện yêu cầu trong một bên tiêu thụ xuôi dòng — miễn là sự kiện được xử lý đúng một lần, điều này lại có thể được thực thi bằng ID yêu cầu.

The end-to-end argument — Lập luận đầu-cuối

This scenario of suppressing duplicate transactions is just one example of a more general principle called the end-to-end argument, which was articulated by Saltzer, Reed, and Clark in 1984 [55]:

Kịch bản triệt tiêu các giao dịch trùng lặp này chỉ là một ví dụ của một nguyên tắc tổng quát hơn gọi là lập luận đầu-cuối (end-to-end argument), được Saltzer, Reed, và Clark trình bày rõ vào năm 1984 [55]:

The function in question can completely and correctly be implemented only with the knowledge and help of the application standing at the endpoints of the communication system. Therefore, providing that questioned function as a feature of the communication system itself is not possible. (Sometimes an incomplete version of the function provided by the communication system may be useful as a performance enhancement.)

Chức năng đang xét chỉ có thể được hiện thực một cách đầy đủ và đúng đắn với sự hiểu biết và trợ giúp của ứng dụng nằm ở các điểm đầu cuối của hệ thống giao tiếp. Do đó, việc cung cấp chức năng đang xét đó như một tính năng của chính hệ thống giao tiếp là không khả thi. (Đôi khi một phiên bản không đầy đủ của chức năng do hệ thống giao tiếp cung cấp có thể hữu ích như một cải thiện hiệu năng.)

In our example, the function in question was duplicate suppression. We saw that TCP suppresses duplicate packets at the TCP connection level, and some stream processors provide so-called exactly-once semantics at the message processing level, but that is not enough to prevent a user from submitting a duplicate request if the first one times out. By themselves, TCP, database transactions, and stream processors cannot entirely rule out these duplicates. Solving the problem requires an end-to-end solution: a transaction identifier that is passed all the way from the end-user client to the database.

Trong ví dụ của ta, chức năng đang xét là việc triệt tiêu trùng lặp. Ta đã thấy rằng TCP triệt tiêu các gói tin trùng lặp ở cấp kết nối TCP, và một số bộ xử lý luồng cung cấp cái gọi là ngữ nghĩa đúng-một-lần ở cấp xử lý thông điệp, nhưng điều đó không đủ để ngăn một người dùng gửi một yêu cầu trùng lặp nếu yêu cầu đầu tiên hết thời gian chờ. Tự thân chúng, TCP, các giao dịch cơ sở dữ liệu, và các bộ xử lý luồng không thể hoàn toàn loại trừ những trùng lặp này. Việc giải quyết vấn đề đòi hỏi một lời giải đầu-cuối: một định danh giao dịch được truyền suốt chặng từ client của người dùng cuối tới cơ sở dữ liệu.

The end-to-end argument also applies to checking the integrity of data: checksums built into Ethernet, TCP, and TLS can detect corruption of packets in the network, but they cannot detect corruption due to bugs in the software at the sending and receiving ends of the network connection, or corruption on the disks where the data is stored. If you want to catch all possible sources of data corruption, you also need end-to-end checksums.

Lập luận đầu-cuối cũng áp dụng cho việc kiểm tra tính toàn vẹn của dữ liệu: các mã kiểm tra (checksum) tích hợp trong Ethernet, TCP, và TLS có thể phát hiện sự hỏng hóc của các gói tin trên mạng, nhưng chúng không thể phát hiện sự hỏng hóc do bug trong phần mềm ở các đầu gửi và nhận của kết nối mạng, hoặc sự hỏng hóc trên các đĩa nơi dữ liệu được

lưu. Nếu bạn muốn bắt được mọi nguồn hỏng dữ liệu khả dĩ, bạn cũng cần các checksum đầu-cuối.

A similar argument applies with encryption [55]: the password on your home WiFi network protects against people snooping your WiFi traffic, but not against attackers elsewhere on the internet; TLS/SSL between your client and the server protects against network attackers, but not against compromises of the server. Only end-to-end encryption and authentication can protect against all of these things.

Một lập luận tương tự áp dụng cho mã hóa [55]: mật khẩu trên mạng WiFi gia đình của bạn bảo vệ chống lại những người nghe lén lưu lượng WiFi của bạn, nhưng không chống lại những kẻ tấn công ở nơi khác trên internet; TLS/SSL giữa client và máy chủ của bạn bảo vệ chống lại những kẻ tấn công mạng, nhưng không chống lại việc máy chủ bị xâm phạm. Chỉ có mã hóa và xác thực đầu-cuối mới có thể bảo vệ chống lại tất cả những điều này.

Although the low-level features (TCP duplicate suppression, Ethernet checksums, WiFi encryption) cannot provide the desired end-to-end features by themselves, they are still useful, since they reduce the probability of problems at the higher levels. For example, HTTP requests would often get mangled if we didn't have TCP putting the packets back in the right order. We just need to remember that the low-level reliability features are not by themselves sufficient to ensure end-to-end correctness.

Mặc dù các tính năng mức thấp (triệt tiêu trùng lặp của TCP, checksum của Ethernet, mã hóa WiFi) tự thân chúng không thể cung cấp các tính năng đầu-cuối mong muốn, chúng vẫn hữu ích, vì chúng giảm xác suất xảy ra vấn đề ở các tầng cao hơn. Chẳng hạn, các yêu cầu HTTP sẽ thường bị xáo trộn nếu ta không có TCP đặt các gói tin lại đúng thứ tự. Ta chỉ cần nhớ rằng các tính năng độ tin cậy mức thấp tự thân chúng không đủ để đảm bảo sự đúng đắn đầu-cuối.

Applying end-to-end thinking in data systems — Áp dụng tư duy đầu-cuối trong các hệ thống dữ liệu

This brings me back to my original thesis: just because an application uses a data system that provides comparatively strong safety properties, such as serializable transactions, that does not mean the application is guaranteed to be free from data loss or corruption. The application itself needs to take end-to-end measures, such as duplicate suppression, as well.

Điều này đưa tôi trở lại luận điểm ban đầu của mình: chỉ vì một ứng dụng dùng một hệ thống dữ liệu cung cấp các thuộc tính an toàn tương đối mạnh, chẳng hạn các giao dịch khả tuần tự hóa, điều đó không có nghĩa là ứng dụng được đảm bảo không bị mất hay hỏng dữ liệu. Bản thân ứng dụng cũng cần thực hiện các biện pháp đầu-cuối, chẳng hạn triệt tiêu trùng lặp.

That is a shame, because fault-tolerance mechanisms are hard to get right. Low-level reliability mechanisms, such as those in TCP, work quite well, and so the remaining higher-level faults occur fairly rarely. It would be really nice to wrap up the remaining high-level fault-tolerance machinery in an abstraction so that application code needn't worry about it—but I fear that we have not yet found the right abstraction.

Đó là điều đáng tiếc, bởi các cơ chế chịu lỗi rất khó làm cho đúng. Các cơ chế độ tin cậy mức thấp, chẳng hạn các cơ chế trong TCP, hoạt động khá tốt, nên các lỗi mức cao còn lại xảy ra khá hiếm. Sẽ thật tuyệt nếu gói gọn được phần máy móc chịu lỗi mức cao còn lại vào một lớp trừu tượng để mã ứng dụng không cần bận tâm về nó — nhưng tôi e rằng ta vẫn chưa tìm ra lớp trừu tượng đúng đắn.

Transactions have long been seen as a good abstraction, and I do believe that they are useful. As discussed in the introduction to Chapter 7, they take a wide range of possible issues (concurrent writes, constraint violations, crashes, network interruptions, disk failures) and collapse them down to two possible outcomes: commit or abort. That is a huge simplification of the programming model, but I fear that it is not enough.

Các giao dịch từ lâu đã được xem là một lớp trừu tượng tốt, và tôi quả thực tin rằng chúng hữu ích. Như đã bàn ở phần mở đầu Chương 7, chúng lấy một loạt vấn đề khả dĩ (ghi đồng thời, vi phạm ràng buộc, sự cố, gián đoạn mạng, hỏng đĩa) và thu gọn chúng xuống còn hai kết cục khả dĩ: cam kết hoặc hủy bỏ. Đó là một sự đơn giản hóa to lớn của mô hình lập trình, nhưng tôi e rằng nó chưa đủ.

Transactions are expensive, especially when they involve heterogeneous storage technologies (see “Distributed Transactions in Practice”). When we refuse to use distributed transactions because they are too expensive, we end up having to reimplement fault-tolerance mechanisms in application code. As numerous examples throughout this book have shown, reasoning about concurrency and partial failure is difficult and counterintuitive, and so I suspect that most application-level mechanisms do not work correctly. The consequence is lost or corrupted data.

Các giao dịch tốn kém, đặc biệt khi chúng dính líu tới các công nghệ lưu trữ không đồng nhất (xem “Giao dịch phân tán trong thực tế”). Khi ta từ chối dùng giao dịch phân tán vì chúng quá tốn kém, ta rốt cuộc phải tự hiện thực lại các cơ chế chịu lỗi trong mã ứng dụng. Như vô số ví dụ xuyên suốt cuốn sách này đã cho thấy, việc suy luận về tính tương tranh và sự cố bộ phận là khó và phản trực giác, nên tôi ngờ rằng hầu hết các cơ chế ở cấp ứng dụng không hoạt động đúng đắn. Hệ quả là dữ liệu bị mất hoặc hỏng.

For these reasons, I think it is worth exploring fault-tolerance abstractions that make it easy to provide application-specific end-to-end correctness properties, but also maintain good performance and good operational characteristics in a large-scale distributed environment.

Vì những lý do này, tôi cho rằng đáng để khám phá các lớp trừu tượng chịu lỗi giúp dễ dàng cung cấp các thuộc tính đúng đắn đầu-cuối đặc thù theo ứng dụng, nhưng cũng duy trì hiệu năng tốt và các đặc tính vận hành tốt trong một môi trường phân tán quy mô lớn.

Enforcing Constraints — Thực thi các ràng buộc

Let’s think about correctness in the context of the ideas around unbundling databases (“Unbundling Databases”). We saw that end-to-end duplicate suppression can be achieved with a request ID that is passed all the way from the client to the database that records the write. What about other kinds of constraints?

Hãy nghĩ về sự đúng đắn trong bối cảnh các ý tưởng quanh việc tháo gói cơ sở dữ liệu (“Tháo gói cơ sở dữ liệu”). Ta đã thấy rằng việc triệt tiêu trùng lặp đầu-cuối có thể đạt được bằng một ID yêu cầu được truyền suốt chặng từ client tới cơ sở dữ liệu vốn ghi lại lệnh ghi. Còn các loại ràng buộc khác thì sao?

In particular, let’s focus on uniqueness constraints—such as the one we relied on in Example 12-2. In “Constraints and uniqueness guarantees” we saw several other examples of application features that need to enforce uniqueness: a username or email address must uniquely identify a user, a file storage service cannot have more than one file with the same name, and two people cannot book the same seat on a flight or in a theater.

Cụ thể, hãy tập trung vào các ràng buộc tính duy nhất — chẳng hạn ràng buộc mà ta đã dựa vào ở Ví dụ 12-2. Ở “Ràng buộc và đảm bảo tính duy nhất” ta đã thấy vài ví dụ khác về

các tính năng ứng dụng cần thực thi tính duy nhất: một tên người dùng hoặc địa chỉ email phải định danh duy nhất một người dùng, một dịch vụ lưu trữ tệp không thể có nhiều hơn một tệp cùng tên, và hai người không thể đặt cùng một ghế trên một chuyến bay hay trong một nhà hát.

Other kinds of constraints are very similar: for example, ensuring that an account balance never goes negative, that you don't sell more items than you have in stock in the warehouse, or that a meeting room does not have overlapping bookings. Techniques that enforce uniqueness can often be used for these kinds of constraints as well.

Các loại ràng buộc khác rất tương tự: chẳng hạn, đảm bảo rằng số dư tài khoản không bao giờ âm, rằng bạn không bán nhiều món hàng hơn số bạn có trong kho, hoặc rằng một phòng họp không có các lượt đặt chỗ chồng chéo. Các kỹ thuật thực thi tính duy nhất thường có thể được dùng cho những loại ràng buộc này nữa.

Uniqueness constraints require consensus — Ràng buộc tính duy nhất đòi hỏi sự đồng thuận

In Chapter 9 we saw that in a distributed setting, enforcing a uniqueness constraint requires consensus: if there are several concurrent requests with the same value, the system somehow needs to decide which one of the conflicting operations is accepted, and reject the others as violations of the constraint.

Ở Chương 9 ta đã thấy rằng trong một bối cảnh phân tán, việc thực thi một ràng buộc tính duy nhất đòi hỏi sự đồng thuận: nếu có vài yêu cầu đồng thời với cùng một giá trị, hệ thống bằng cách nào đó cần quyết định một trong các thao tác xung đột nào được chấp nhận, và từ chối các thao tác còn lại vì vi phạm ràng buộc.

The most common way of achieving this consensus is to make a single node the leader, and put it in charge of making all the decisions. That works fine as long as you don't mind funneling all requests through a single node (even if the client is on the other side of the world), and as long as that node doesn't fail. If you need to tolerate the leader failing, you're back at the consensus problem again (see "Single-leader replication and consensus").

Cách phổ biến nhất để đạt được sự đồng thuận này là biến một nút đơn lẻ thành leader, và giao cho nó nhiệm vụ đưa ra mọi quyết định. Điều đó hoạt động tốt chừng nào bạn không ngại dồn mọi yêu cầu qua một nút đơn lẻ (ngay cả khi client ở phía bên kia của thế giới), và chừng nào nút đó không hỏng. Nếu bạn cần chịu được việc leader hỏng, bạn lại trở về với bài toán đồng thuận (xem "Sao chép đơn-leader và đồng thuận").

Uniqueness checking can be scaled out by partitioning based on the value that needs to be unique. For example, if you need to ensure uniqueness by request ID, as in Example 12-2, you can ensure all requests with the same request ID are routed to the same partition (see Chapter 6). If you need usernames to be unique, you can partition by hash of username.

Việc kiểm tra tính duy nhất có thể được mở rộng theo chiều ngang bằng cách phân mảnh dựa trên giá trị cần phải duy nhất. Chẳng hạn, nếu bạn cần đảm bảo tính duy nhất theo ID yêu cầu, như trong Ví dụ 12-2, bạn có thể đảm bảo mọi yêu cầu có cùng ID yêu cầu được định tuyến tới cùng một mảnh (xem Chương 6). Nếu bạn cần tên người dùng phải duy nhất, bạn có thể phân mảnh theo mã băm của tên người dùng.

However, asynchronous multi-master replication is ruled out, because it could happen that different masters concurrently accept conflicting writes, and thus the values are no longer unique (see

“Implementing Linearizable Systems”). If you want to be able to immediately reject any writes that would violate the constraint, synchronous coordination is unavoidable [56].

Tuy nhiên, sao chép đa-chủ (multi-master) bất đồng bộ bị loại trừ, bởi có thể xảy ra việc các chủ khác nhau đồng thời chấp nhận các lệnh ghi xung đột, và do đó các giá trị không còn duy nhất nữa (xem “Hiện thực các hệ thống tuyến tính hóa”). Nếu bạn muốn có thể từ chối ngay lập tức bất kỳ lệnh ghi nào vi phạm ràng buộc, thì việc phối hợp đồng bộ là không thể tránh khỏi [56].

Uniqueness in log-based messaging — Tính duy nhất trong truyền thông điệp dựa trên log

The log ensures that all consumers see messages in the same order—a guarantee that is formally known as total order broadcast and is equivalent to consensus (see “Total Order Broadcast”). In the unbundled database approach with log-based messaging, we can use a very similar approach to enforce uniqueness constraints.

Log đảm bảo rằng mọi bên tiêu thụ thấy các thông điệp theo cùng thứ tự — một đảm bảo được biết đến một cách hình thức là truyền tin theo thứ tự toàn phần và tương đương với đồng thuận (xem “Truyền tin theo thứ tự toàn phần”). Trong cách tiếp cận cơ sở dữ liệu tháo gói với truyền thông điệp dựa trên log, ta có thể dùng một cách tiếp cận rất tương tự để thực thi các ràng buộc tính duy nhất.

A stream processor consumes all the messages in a log partition sequentially on a single thread (see “Logs compared to traditional messaging”). Thus, if the log is partitioned based on the value that needs to be unique, a stream processor can unambiguously and deterministically decide which one of several conflicting operations came first. For example, in the case of several users trying to claim the same username [57]:

Một bộ xử lý luồng tiêu thụ mọi thông điệp trong một mảnh log một cách tuần tự trên một luồng đơn (single thread) (xem “So sánh log với hệ thống truyền thông điệp truyền thống”). Do đó, nếu log được phân mảnh dựa trên giá trị cần phải duy nhất, một bộ xử lý luồng có thể quyết định một cách rõ ràng và tất định xem một trong vài thao tác xung đột nào đến trước. Chẳng hạn, trong trường hợp vài người dùng cố giành cùng một tên người dùng [57]:

- Every request for a username is encoded as a message, and appended to a partition determined by the hash of the username.
- A stream processor sequentially reads the requests in the log, using a local database to keep track of which usernames are taken. For every request for a username that is available, it records the name as taken and emits a success message to an output stream. For every request for a username that is already taken, it emits a rejection message to an output stream.
- The client that requested the username watches the output stream and waits for a success or rejection message corresponding to its request.
 - Mỗi yêu cầu cho một tên người dùng được mã hóa thành một thông điệp, và được thêm vào một mảnh được xác định bởi mã băm của tên người dùng.
 - Một bộ xử lý luồng đọc tuần tự các yêu cầu trong log, dùng một cơ sở dữ liệu cục bộ để theo dõi những tên người dùng nào đã bị chiếm. Với mỗi yêu cầu cho một tên người dùng còn trống, nó ghi nhận tên đó là đã bị chiếm và phát một thông điệp thành công tới một luồng đầu ra. Với mỗi yêu cầu cho một tên người dùng đã bị chiếm, nó phát một thông điệp từ chối tới một luồng đầu ra.

- Client đã yêu cầu tên người dùng theo dõi luồng đầu ra và chờ một thông điệp thành công hoặc từ chối tương ứng với yêu cầu của nó.

This algorithm is basically the same as in “Implementing linearizable storage using total order broadcast”. It scales easily to a large request throughput by increasing the number of partitions, as each partition can be processed independently.

Thuật toán này về cơ bản giống với ở “Hiện thực lưu trữ tuyến tính hóa bằng truyền tin theo thứ tự toàn phần”. Nó dễ dàng mở rộng tới một thông lượng yêu cầu lớn bằng cách tăng số lượng mảnh, vì mỗi mảnh có thể được xử lý độc lập.

The approach works not only for uniqueness constraints, but also for many other kinds of constraints. Its fundamental principle is that any writes that may conflict are routed to the same partition and processed sequentially. As discussed in “What is a conflict?” and “Write Skew and Phantoms”, the definition of a conflict may depend on the application, but the stream processor can use arbitrary logic to validate a request. This idea is similar to the approach pioneered by Bayou in the 1990s [58].

Cách tiếp cận này hoạt động không chỉ cho các ràng buộc tính duy nhất, mà còn cho nhiều loại ràng buộc khác. Nguyên tắc nền tảng của nó là mọi lệnh ghi có thể xung đột đều được định tuyến tới cùng một mảnh và được xử lý tuần tự. Như đã bàn ở “Xung đột là gì?” và “Lệnh ghi và bóng ma”, định nghĩa về một xung đột có thể tùy thuộc vào ứng dụng, nhưng bộ xử lý luồng có thể dùng logic bất kỳ để thẩm định một yêu cầu. Ý tưởng này tương tự với cách tiếp cận do Bayou tiên phong vào thập niên 1990 [58].

Multi-partition request processing — Xử lý yêu cầu đa-mảnh

Ensuring that an operation is executed atomically, while satisfying constraints, becomes more interesting when several partitions are involved. In Example 12-2, there are potentially three partitions: the one containing the request ID, the one containing the payee account, and the one containing the payer account. There is no reason why those three things should be in the same partition, since they are all independent from each other.

Việc đảm bảo một thao tác được thực thi một cách nguyên tử, trong khi vẫn thỏa mãn các ràng buộc, trở nên thú vị hơn khi có vài mảnh dính líu vào. Trong Ví dụ 12-2, có khả năng có ba mảnh: mảnh chứa ID yêu cầu, mảnh chứa tài khoản người nhận tiền, và mảnh chứa tài khoản người trả tiền. Không có lý do gì ba thứ đó phải nằm trong cùng một mảnh, bởi chúng đều độc lập với nhau.

In the traditional approach to databases, executing this transaction would require an atomic commit across all three partitions, which essentially forces it into a total order with respect to all other transactions on any of those partitions. Since there is now cross-partition coordination, different partitions can no longer be processed independently, so throughput is likely to suffer.

Trong cách tiếp cận truyền thống đối với cơ sở dữ liệu, việc thực thi giao dịch này sẽ đòi hỏi một cam kết nguyên tử xuyên cả ba mảnh, điều này về bản chất buộc nó vào một thứ tự toàn phần đối với mọi giao dịch khác trên bất kỳ mảnh nào trong số đó. Vì giờ đây có sự phối hợp xuyên mảnh, các mảnh khác nhau không còn có thể được xử lý độc lập, nên thông lượng có khả năng bị ảnh hưởng.

However, it turns out that equivalent correctness can be achieved with partitioned logs, and without an atomic commit:

Tuy nhiên, hóa ra sự đúng đắn tương đương có thể đạt được với các log được phân mảnh, mà không cần một cam kết nguyên tử:

- The request to transfer money from account A to account B is given a unique request ID by the client, and appended to a log partition based on the request ID.
- A stream processor reads the log of requests. For each request message it emits two messages to output streams: a debit instruction to the payer account A (partitioned by A), and a credit instruction to the payee account B (partitioned by B). The original request ID is included in those emitted messages.
- Further processors consume the streams of credit and debit instructions, deduplicate by request ID, and apply the changes to the account balances.
 - Yêu cầu chuyển tiền từ tài khoản A sang tài khoản B được client gán một ID yêu cầu duy nhất, và được thêm vào một mảnh log dựa trên ID yêu cầu.
 - Một bộ xử lý luồng đọc log các yêu cầu. Với mỗi thông điệp yêu cầu, nó phát hai thông điệp tới các luồng đầu ra: một chỉ thị ghi nợ cho tài khoản người trả tiền A (được phân mảnh theo A), và một chỉ thị ghi có cho tài khoản người nhận tiền B (được phân mảnh theo B). ID yêu cầu gốc được đưa vào trong các thông điệp được phát ra đó.
 - Các bộ xử lý tiếp theo tiêu thụ các luồng chỉ thị ghi có và ghi nợ, khử trùng lặp theo ID yêu cầu, và áp dụng các thay đổi vào số dư tài khoản.

Steps 1 and 2 are necessary because if the client directly sent the credit and debit instructions, it would require an atomic commit across those two partitions to ensure that either both or neither happen. To avoid the need for a distributed transaction, we first durably log the request as a single message, and then derive the credit and debit instructions from that first message. Single-object writes are atomic in almost all data systems (see “Single-object writes”), and so the request either appears in the log or it doesn’t, without any need for a multi-partition atomic commit.

Các bước 1 và 2 là cần thiết bởi nếu client trực tiếp gửi các chỉ thị ghi có và ghi nợ, điều đó sẽ đòi hỏi một cam kết nguyên tử xuyên hai mảnh đó để đảm bảo rằng hoặc cả hai hoặc không cái nào xảy ra. Để tránh nhu cầu về một giao dịch phân tán, trước tiên ta ghi log yêu cầu một cách bền vững dưới dạng một thông điệp đơn, rồi dẫn xuất các chỉ thị ghi có và ghi nợ từ thông điệp đầu tiên đó. Các lệnh ghi đối-tượng-đơn (single-object) là nguyên tử trong gần như mọi hệ thống dữ liệu (xem “Lệnh ghi đối-tượng-đơn”), nên yêu cầu hoặc xuất hiện trong log hoặc không, mà không cần một cam kết nguyên tử đa-mảnh nào.

If the stream processor in step 2 crashes, it resumes processing from its last checkpoint. In doing so, it does not skip any request messages, but it may process requests multiple times and produce duplicate credit and debit instructions. However, since it is deterministic, it will just produce the same instructions again, and the processors in step 3 can easily deduplicate them using the end-to-end request ID.

Nếu bộ xử lý luồng ở bước 2 bị sự cố, nó tiếp tục xử lý từ điểm kiểm tra (checkpoint) gần nhất của nó. Khi làm vậy, nó không bỏ qua bất kỳ thông điệp yêu cầu nào, nhưng nó có thể xử lý các yêu cầu nhiều lần và tạo ra các chỉ thị ghi có và ghi nợ trùng lặp. Tuy nhiên, vì nó mang tính tất định, nó sẽ chỉ tạo ra cùng các chỉ thị một lần nữa, và các bộ xử lý ở bước 3 có thể dễ dàng khử trùng lặp chúng bằng ID yêu cầu đầu-cuối.

If you want to ensure that the payer account is not overdrawn by this transfer, you can additionally have a stream processor (partitioned by payer account number) that maintains account balances and validates transactions. Only valid transactions would then be placed in the request log in step 1.

Nếu bạn muốn đảm bảo tài khoản người trả tiền không bị rút quá hạn mức bởi lần chuyển tiền này, bạn có thể bổ sung thêm một bộ xử lý luồng (được phân mảnh theo số tài khoản người trả tiền) vốn duy trì số dư tài khoản và thẩm định các giao dịch. Chỉ những giao dịch hợp lệ mới được đặt vào log yêu cầu ở bước 1.

By breaking down the multi-partition transaction into two differently partitioned stages and using the end-to-end request ID, we have achieved the same correctness property (every request is applied exactly once to both the payer and payee accounts), even in the presence of faults, and without using an atomic commit protocol. The idea of using multiple differently partitioned stages is similar to what we discussed in “Multi-partition data processing” (see also “Concurrency control”).

Bằng cách chia nhỏ giao dịch đa-mảnh thành hai giai đoạn được phân mảnh khác nhau và dùng ID yêu cầu đầu-cuối, ta đã đạt được cùng một thuộc tính đúng đắn (mọi yêu cầu được áp dụng đúng một lần cho cả tài khoản người trả tiền lẫn người nhận tiền), ngay cả khi có lỗi, và không dùng một giao thức cam kết nguyên tử nào. Ý tưởng dùng nhiều giai đoạn được phân mảnh khác nhau tương tự với điều ta đã bàn ở “Xử lý dữ liệu đa-mảnh” (xem thêm “Kiểm soát tương tranh”).

Timeliness and Integrity — Tính kịp thời và tính toàn vẹn

A convenient property of transactions is that they are typically linearizable (see “Linearizability”): that is, a writer waits until a transaction is committed, and thereafter its writes are immediately visible to all readers.

Một thuộc tính tiện lợi của giao dịch là chúng thường tuyến tính hóa (xem “Tính tuyến tính hóa”): tức là, một bên ghi chờ cho tới khi một giao dịch được cam kết, và sau đó các lệnh ghi của nó ngay lập tức hiển thị với mọi bên đọc.

This is not the case when unbundling an operation across multiple stages of stream processors: consumers of a log are asynchronous by design, so a sender does not wait until its message has been processed by consumers. However, it is possible for a client to wait for a message to appear on an output stream. This is what we did in “Uniqueness in log-based messaging” when checking whether a uniqueness constraint was satisfied.

Đây không phải là trường hợp khi tháo gói một thao tác thành nhiều giai đoạn của các bộ xử lý luồng: các bên tiêu thụ của một log là bất đồng bộ theo thiết kế, nên một bên gửi không chờ cho tới khi thông điệp của nó đã được các bên tiêu thụ xử lý. Tuy nhiên, một client có thể chờ một thông điệp xuất hiện trên một luồng đầu ra. Đây là điều ta đã làm ở “Tính duy nhất trong truyền thông điệp dựa trên log” khi kiểm tra xem một ràng buộc tính duy nhất có được thỏa mãn hay không.

In this example, the correctness of the uniqueness check does not depend on whether the sender of the message waits for the outcome. The waiting only has the purpose of synchronously informing the sender whether or not the uniqueness check succeeded, but this notification can be decoupled from the effects of processing the message.

Trong ví dụ này, sự đúng đắn của việc kiểm tra tính duy nhất không phụ thuộc vào việc bên gửi thông điệp có chờ kết cục hay không. Việc chờ chỉ có mục đích thông báo một cách đồng bộ cho bên gửi biết việc kiểm tra tính duy nhất có thành công hay không, nhưng thông báo này có thể được tách rời khỏi các tác động của việc xử lý thông điệp.

More generally, I think the term consistency conflates two different requirements that are worth considering separately:

Tổng quát hơn, tôi cho rằng thuật ngữ tính nhất quán trộn lẫn hai yêu cầu khác nhau mà đáng để xét riêng:

Timeliness means ensuring that users observe the system in an up-to-date state. We saw previously that if a user reads from a stale copy of the data, they may observe it in an inconsistent state (see

“Problems with Replication Lag”). However, that inconsistency is temporary, and will eventually be resolved simply by waiting and trying again.

Tính kịp thời (timeliness) nghĩa là đảm bảo người dùng quan sát hệ thống ở một trạng thái cập nhật. Trước đây ta đã thấy rằng nếu một người dùng đọc từ một bản sao cũ của dữ liệu, họ có thể quan sát nó ở một trạng thái bất nhất (xem “Các vấn đề với độ trễ sao chép”). Tuy nhiên, sự bất nhất đó là tạm thời, và rốt cuộc sẽ được hóa giải đơn giản bằng cách chờ và thử lại.

The CAP theorem (see “The Cost of Linearizability”) uses consistency in the sense of linearizability, which is a strong way of achieving timeliness. Weaker timeliness properties like read-after-write consistency (see “Reading Your Own Writes”) can also be useful.

Định lý CAP (xem “Cái giá của tính tuyến tính hóa”) dùng tính nhất quán theo nghĩa tính tuyến tính hóa, vốn là một cách mạnh để đạt tính kịp thời. Các thuộc tính kịp thời yếu hơn như nhất quán đọc-sau-ghi (read-after-write) (xem “Đọc được chính lệnh ghi của mình”) cũng có thể hữu ích.

Integrity means absence of corruption; i.e., no data loss, and no contradictory or false data. In particular, if some derived dataset is maintained as a view onto some underlying data (see “Deriving current state from the event log”), the derivation must be correct. For example, a database index must correctly reflect the contents of the database—an index in which some records are missing is not very useful.

Tính toàn vẹn (integrity) nghĩa là không có sự hỏng hóc; tức là, không mất dữ liệu, và không có dữ liệu mâu thuẫn hay sai lệch. Cụ thể, nếu một tập dữ liệu dẫn xuất nào đó được duy trì như một khung nhìn trên một dữ liệu nền nào đó (xem “Dẫn xuất trạng thái hiện tại từ log sự kiện”), thì việc dẫn xuất phải đúng đắn. Chẳng hạn, một chỉ mục cơ sở dữ liệu phải phản ánh đúng nội dung của cơ sở dữ liệu — một chỉ mục thiếu vắng một số bản ghi là không mấy hữu ích.

If integrity is violated, the inconsistency is permanent: waiting and trying again is not going to fix database corruption in most cases. Instead, explicit checking and repair is needed. In the context of ACID transactions (see “The Meaning of ACID”), consistency is usually understood as some kind of application-specific notion of integrity. Atomicity and durability are important tools for preserving integrity.

Nếu tính toàn vẹn bị vi phạm, sự bất nhất là vĩnh viễn: trong hầu hết các trường hợp, việc chờ và thử lại sẽ không sửa được sự hỏng hóc của cơ sở dữ liệu. Thay vào đó, cần có việc kiểm tra và sửa chữa tường minh. Trong bối cảnh các giao dịch ACID (xem “Ý nghĩa của ACID”), tính nhất quán thường được hiểu như một dạng ý niệm về tính toàn vẹn đặc thù theo ứng dụng. Tính nguyên tử và độ bền là những công cụ quan trọng để bảo toàn tính toàn vẹn.

In slogan form: violations of timeliness are “eventual consistency,” whereas violations of integrity are “perpetual inconsistency.”

Diễn đạt theo dạng khẩu hiệu: vi phạm tính kịp thời là “nhất quán cuối cùng”, trong khi vi phạm tính toàn vẹn là “bất nhất vĩnh viễn”.

I am going to assert that in most applications, integrity is much more important than timeliness. Violations of timeliness can be annoying and confusing, but violations of integrity can be catastrophic.

Tôi sẽ khẳng định rằng trong hầu hết các ứng dụng, tính toàn vẹn quan trọng hơn nhiều so với tính kịp thời. Vi phạm tính kịp thời có thể gây phiền và khó hiểu, nhưng vi phạm

tính toàn vẹn có thể là thảm họa.

For example, on your credit card statement, it is not surprising if a transaction that you made within the last 24 hours does not yet appear—it is normal that these systems have a certain lag. We know that banks reconcile and settle transactions asynchronously, and timeliness is not very important here [3]. However, it would be very bad if the statement balance was not equal to the sum of the transactions plus the previous statement balance (an error in the sums), or if a transaction was charged to you but not paid to the merchant (disappearing money). Such problems would be violations of the integrity of the system.

Chẳng hạn, trên sao kê thẻ tín dụng của bạn, không có gì đáng ngạc nhiên nếu một giao dịch bạn thực hiện trong vòng 24 giờ qua chưa xuất hiện — việc các hệ thống này có một độ trễ nhất định là bình thường. Ta biết rằng các ngân hàng đối soát và thanh toán bù trừ các giao dịch một cách bất đồng bộ, và tính kịp thời ở đây không mấy quan trọng [3]. Tuy nhiên, sẽ rất tệ nếu số dư trên sao kê không bằng tổng các giao dịch cộng với số dư sao kê trước đó (một lỗi trong phép tổng), hoặc nếu một giao dịch bị tính tiền vào bạn nhưng không được trả cho người bán (tiền biến mất). Những vấn đề như vậy sẽ là vi phạm tính toàn vẹn của hệ thống.

Correctness of dataflow systems — Sự đúng đắn của các hệ thống luồng dữ liệu

ACID transactions usually provide both timeliness (e.g., linearizability) and integrity (e.g., atomic commit) guarantees. Thus, if you approach application correctness from the point of view of ACID transactions, the distinction between timeliness and integrity is fairly inconsequential.

Các giao dịch ACID thường cung cấp cả các đảm bảo về tính kịp thời (ví dụ, tính tuyến tính hóa) lẫn tính toàn vẹn (ví dụ, cam kết nguyên tử). Do đó, nếu bạn tiếp cận sự đúng đắn của ứng dụng từ góc nhìn các giao dịch ACID, sự phân biệt giữa tính kịp thời và tính toàn vẹn khá ít quan trọng.

On the other hand, an interesting property of the event-based dataflow systems that we have discussed in this chapter is that they decouple timeliness and integrity. When processing event streams asynchronously, there is no guarantee of timeliness, unless you explicitly build consumers that wait for a message to arrive before returning. But integrity is in fact central to streaming systems.

Mặt khác, một thuộc tính thú vị của các hệ thống luồng dữ liệu dựa trên sự kiện mà ta đã bàn trong chương này là chúng tách rời tính kịp thời và tính toàn vẹn. Khi xử lý các luồng sự kiện một cách bất đồng bộ, không có đảm bảo về tính kịp thời, trừ phi bạn tưởng mình xây dựng các bên tiêu thụ chờ một thông điệp đến trước khi trả về. Nhưng tính toàn vẹn thì thực ra là trung tâm của các hệ thống luồng.

Exactly-once or effectively-once semantics (see “Fault Tolerance”) is a mechanism for preserving integrity. If an event is lost, or if an event takes effect twice, the integrity of a data system could be violated. Thus, fault-tolerant message delivery and duplicate suppression (e.g., idempotent operations) are important for maintaining the integrity of a data system in the face of faults.

Ngữ nghĩa đúng-một-lần hay hiệu-quả-một-lần (xem “Khả năng chịu lỗi”) là một cơ chế để bảo toàn tính toàn vẹn. Nếu một sự kiện bị mất, hoặc nếu một sự kiện có hiệu lực hai lần, tính toàn vẹn của một hệ thống dữ liệu có thể bị vi phạm. Do đó, việc giao thông điệp chịu lỗi và triệt tiêu trùng lặp (ví dụ, các thao tác lũy đẳng) là quan trọng để duy trì tính toàn vẹn của một hệ thống dữ liệu khi đối mặt với lỗi.

As we saw in the last section, reliable stream processing systems can preserve integrity without requiring distributed transactions and an atomic commit protocol, which means they can potentially achieve comparable correctness with much better performance and operational robustness. We achieved this integrity through a combination of mechanisms:

Như ta đã thấy ở phần trước, các hệ thống xử lý luồng đáng tin cậy có thể bảo toàn tính toàn vẹn mà không đòi hỏi các giao dịch phân tán và một giao thức cam kết nguyên tử, điều này nghĩa là chúng có khả năng đạt sự đúng đắn tương đương với hiệu năng và sự vững vàng về vận hành tốt hơn nhiều. Ta đạt được tính toàn vẹn này thông qua một sự kết hợp các cơ chế:

- Representing the content of the write operation as a single message, which can easily be written atomically—an approach that fits very well with event sourcing (see “Event Sourcing”)
- Deriving all other state updates from that single message using deterministic derivation functions, similarly to stored procedures (see “Actual Serial Execution” and “Application code as a derivation function”)
- Passing a client-generated request ID through all these levels of processing, enabling end-to-end duplicate suppression and idempotence
- Making messages immutable and allowing derived data to be reprocessed from time to time, which makes it easier to recover from bugs (see “Advantages of immutable events”)
 - Biểu diễn nội dung của thao tác ghi dưới dạng một thông điệp đơn, vốn có thể được ghi một cách nguyên tử dễ dàng — một cách tiếp cận khớp rất tốt với nguồn-sự-kiện (xem “Nguồn-sự-kiện”)
 - Dẫn xuất mọi cập nhật trạng thái khác từ thông điệp đơn đó bằng các hàm dẫn xuất tất định, tương tự như các thủ tục lưu trữ (xem “Thực thi tuần tự thực sự” và “Mã ứng dụng như một hàm dẫn xuất”)
 - Truyền một ID yêu cầu do client sinh ra qua mọi cấp xử lý này, cho phép triệt tiêu trùng lặp và tính lũy đẳng đầu-cuối
 - Làm cho các thông điệp bất biến và cho phép dữ liệu dẫn xuất được xử lý lại theo thời gian, điều này giúp dễ dàng phục hồi sau các bug (xem “Ưu điểm của sự kiện bất biến”)

This combination of mechanisms seems to me a very promising direction for building fault-tolerant applications in the future.

Sự kết hợp các cơ chế này, với tôi, dường như là một hướng đi rất hứa hẹn để xây dựng các ứng dụng chịu lỗi trong tương lai.

Loosely interpreted constraints — Ràng buộc được diễn giải lỏng lẻo

As discussed previously, enforcing a uniqueness constraint requires consensus, typically implemented by funneling all events in a particular partition through a single node. This limitation is unavoidable if we want the traditional form of uniqueness constraint, and stream processing cannot avoid it.

Như đã bàn trước đó, việc thực thi một ràng buộc tính duy nhất đòi hỏi sự đồng thuận, thường được hiện thực bằng cách dồn mọi sự kiện trong một mảnh cụ thể qua một nút đơn lẻ. Giới hạn này là không thể tránh khỏi nếu ta muốn dạng ràng buộc tính duy nhất truyền thống, và xử lý luồng không thể tránh nó.

However, another thing to realize is that many real applications can actually get away with much weaker notions of uniqueness:

Tuy nhiên, một điều khác cần nhận ra là nhiều ứng dụng thực tế thực ra có thể qua chuyện với những ý niệm về tính duy nhất yếu hơn nhiều:

- If two people concurrently register the same username or book the same seat, you can send one of them a message to apologize, and ask them to choose a different one. This kind of change to correct a mistake is called a compensating transaction [59, 60].
- If customers order more items than you have in your warehouse, you can order in more stock, apologize to customers for the delay, and offer them a discount. This is actually the same as what you'd have to do if, say, a forklift truck ran over some of the items in your warehouse, leaving you with fewer items in stock than you thought you had [61]. Thus, the apology workflow already needs to be part of your business processes anyway, and so it might be unnecessary to require a linearizable constraint on the number of items in stock.
- Similarly, many airlines overbook airplanes in the expectation that some passengers will miss their flight, and many hotels overbook rooms, expecting that some guests will cancel. In these cases, the constraint of “one person per seat” is deliberately violated for business reasons, and compensation processes (refunds, upgrades, providing a complimentary room at a neighboring hotel) are put in place to handle situations in which demand exceeds supply. Even if there was no overbooking, apology and compensation processes would be needed in order to deal with flights being cancelled due to bad weather or staff on strike—recovering from such issues is just a normal part of business [3].
- If someone withdraws more money than they have in their account, the bank can charge them an overdraft fee and ask them to pay back what they owe. By limiting the total withdrawals per day, the risk to the bank is bounded.
 - Nếu hai người đồng thời đăng ký cùng một tên người dùng hoặc đặt cùng một ghế, bạn có thể gửi cho một trong hai một thông điệp để xin lỗi, và đề nghị họ chọn một lựa chọn khác. Loại thay đổi để sửa một sai lầm này được gọi là một giao dịch bù trừ (compensating transaction) [59, 60].
 - Nếu khách hàng đặt nhiều món hàng hơn số bạn có trong kho, bạn có thể nhập thêm hàng, xin lỗi khách hàng về sự chậm trễ, và đề nghị họ một khoản giảm giá. Điều này thực ra giống với điều bạn sẽ phải làm nếu, chẳng hạn, một chiếc xe nâng cán qua một số món hàng trong kho của bạn, khiến bạn còn ít hàng trong kho hơn so với mức bạn nghĩ mình có [61]. Do đó, luồng công việc xin lỗi dù sao cũng đã cần phải là một phần trong các quy trình nghiệp vụ của bạn, nên có lẽ không cần thiết phải đòi hỏi một ràng buộc tuyến tính hóa trên số lượng món hàng trong kho.
 - Tương tự, nhiều hãng hàng không bán vé vượt số ghế trên máy bay với kỳ vọng rằng một số hành khách sẽ lỡ chuyến bay của họ, và nhiều khách sạn đặt vượt số phòng, kỳ vọng rằng một số khách sẽ hủy. Trong những trường hợp này, ràng buộc “một người một ghế” bị cố ý vi phạm vì lý do kinh doanh, và các quy trình bù trừ (hoàn tiền, nâng hạng, cung cấp một phòng miễn phí tại một khách sạn lân cận) được đưa vào để xử lý các tình huống mà câu vượt cung. Ngay cả khi không có việc bán vượt, các quy trình xin lỗi và bù trừ vẫn cần thiết để đối phó với việc các chuyến bay bị hủy do thời tiết xấu hoặc nhân viên đình công — việc phục hồi sau những vấn đề như vậy chỉ là một phần bình thường của kinh doanh [3].
 - Nếu ai đó rút nhiều tiền hơn số họ có trong tài khoản, ngân hàng có thể tính cho họ một khoản phí thiếu chi và đề nghị họ trả lại khoản nợ. Bằng cách giới hạn tổng số tiền rút mỗi ngày, rủi ro cho ngân hàng được giới hạn.

In many business contexts, it is actually acceptable to temporarily violate a constraint and fix it up later by apologizing. The cost of the apology (in terms of money or reputation) varies, but it is often quite low: you can't unsend an email, but you can send a follow-up email with a correction. If you accidentally charge a credit card twice, you can refund one of the charges, and the cost to you is just

the processing fees and perhaps a customer complaint. Once money has been paid out of an ATM, you can't directly get it back, although in principle you can send debt collectors to recover the money if the account was overdrawn and the customer won't pay it back.

Trong nhiều bối cảnh kinh doanh, việc tạm thời vi phạm một ràng buộc rồi sửa nó về sau bằng cách xin lỗi thực ra là chấp nhận được. Cái giá của lời xin lỗi (về mặt tiền bạc hay danh tiếng) có khác nhau, nhưng nó thường khá thấp: bạn không thể thu hồi một email đã gửi, nhưng bạn có thể gửi một email tiếp theo với phần đính chính. Nếu bạn vô tình tính tiền một thẻ tín dụng hai lần, bạn có thể hoàn lại một trong các khoản tính, và cái giá với bạn chỉ là các khoản phí xử lý và có lẽ một lời phàn nàn của khách hàng. Một khi tiền đã được trả ra từ một máy ATM, bạn không thể trực tiếp lấy lại nó, mặc dù về nguyên tắc bạn có thể cử người đòi nợ đi thu hồi số tiền nếu tài khoản bị thiếu chi và khách hàng không chịu trả lại.

Whether the cost of the apology is acceptable is a business decision. If it is acceptable, the traditional model of checking all constraints before even writing the data is unnecessarily restrictive, and a linearizable constraint is not needed. It may well be a reasonable choice to go ahead with a write optimistically, and to check the constraint after the fact. You can still ensure that the validation occurs before doing things that would be expensive to recover from, but that doesn't imply you must do the validation before you even write the data.

Việc cái giá của lời xin lỗi có chấp nhận được hay không là một quyết định kinh doanh. Nếu nó chấp nhận được, mô hình truyền thống kiểm tra mọi ràng buộc trước cả khi ghi dữ liệu là hạn chế một cách không cần thiết, và không cần một ràng buộc tuyến tính hóa. Hoàn toàn có thể là một lựa chọn hợp lý khi tiến tới việc ghi một cách lạc quan, và kiểm tra ràng buộc sau đó. Bạn vẫn có thể đảm bảo việc thẩm định diễn ra trước khi làm những việc mà sẽ tốn kém để phục hồi, nhưng điều đó không ngụ ý bạn phải thực hiện việc thẩm định trước cả khi bạn ghi dữ liệu.

These applications do require integrity: you would not want to lose a reservation, or have money disappear due to mismatched credits and debits. But they don't require timeliness on the enforcement of the constraint: if you have sold more items than you have in the warehouse, you can patch up the problem after the fact by apologizing. Doing so is similar to the conflict resolution approaches we discussed in “Handling Write Conflicts”.

Những ứng dụng này quả thực đòi hỏi tính toàn vẹn: bạn sẽ không muốn mất một lượt đặt chỗ, hoặc có tiền biến mất do các khoản ghi có và ghi nợ không khớp nhau. Nhưng chúng không đòi hỏi tính kịp thời trong việc thực thi ràng buộc: nếu bạn đã bán nhiều món hàng hơn số bạn có trong kho, bạn có thể vá lại vấn đề sau đó bằng cách xin lỗi. Làm vậy tương tự với các cách tiếp cận giải quyết xung đột mà ta đã bàn ở “Xử lý xung đột ghi”.

Coordination-avoiding data systems — Các hệ thống dữ liệu tránh phối hợp

We have now made two interesting observations:

Giờ đây ta đã đưa ra hai quan sát thú vị:

- Dataflow systems can maintain integrity guarantees on derived data without atomic commit, linearizability, or synchronous cross-partition coordination.
- Although strict uniqueness constraints require timeliness and coordination, many applications are actually fine with loose constraints that may be temporarily violated and fixed up later, as long as integrity is preserved throughout.
 - Các hệ thống luồng dữ liệu có thể duy trì các đảm bảo về tính toàn vẹn trên dữ liệu

dẫn xuất mà không cần cam kết nguyên tử, tính tuyến tính hóa, hoặc phối hợp xuyên mảnh đồng bộ.

- Mặc dù các ràng buộc tính duy nhất nghiêm ngặt đòi hỏi tính kịp thời và sự phối hợp, nhiều ứng dụng thực ra ổn với các ràng buộc lỏng có thể bị vi phạm tạm thời rồi sửa về sau, miễn là tính toàn vẹn được bảo toàn xuyên suốt.

Taken together, these observations mean that dataflow systems can provide the data management services for many applications without requiring coordination, while still giving strong integrity guarantees. Such coordination-avoiding data systems have a lot of appeal: they can achieve better performance and fault tolerance than systems that need to perform synchronous coordination [56].

Gộp lại, những quan sát này nghĩa là các hệ thống luồng dữ liệu có thể cung cấp các dịch vụ quản lý dữ liệu cho nhiều ứng dụng mà không đòi hỏi sự phối hợp, trong khi vẫn cho các đảm bảo mạnh về tính toàn vẹn. Những hệ thống dữ liệu tránh phối hợp như vậy có rất nhiều sức hút: chúng có thể đạt hiệu năng và khả năng chịu lỗi tốt hơn so với các hệ thống cần thực hiện phối hợp đồng bộ [56].

For example, such a system could operate distributed across multiple datacenters in a multi-leader configuration, asynchronously replicating between regions. Any one datacenter can continue operating independently from the others, because no synchronous cross-region coordination is required. Such a system would have weak timeliness guarantees—it could not be linearizable without introducing coordination—but it can still have strong integrity guarantees.

Chẳng hạn, một hệ thống như vậy có thể vận hành phân tán trên nhiều trung tâm dữ liệu trong một cấu hình đa-leader, sao chép bất đồng bộ giữa các vùng. Bất kỳ một trung tâm dữ liệu nào cũng có thể tiếp tục vận hành độc lập với các trung tâm khác, bởi không cần phối hợp xuyên vùng đồng bộ. Một hệ thống như vậy sẽ có các đảm bảo về tính kịp thời yếu — nó không thể tuyến tính hóa mà không đưa vào sự phối hợp — nhưng nó vẫn có thể có các đảm bảo mạnh về tính toàn vẹn.

In this context, serializable transactions are still useful as part of maintaining derived state, but they can be run at a small scope where they work well [8]. Heterogeneous distributed transactions such as XA transactions (see “Distributed Transactions in Practice”) are not required. Synchronous coordination can still be introduced in places where it is needed (for example, to enforce strict constraints before an operation from which recovery is not possible), but there is no need for everything to pay the cost of coordination if only a small part of an application needs it [43].

Trong bối cảnh này, các giao dịch khả tuần tự hóa vẫn hữu ích như một phần của việc duy trì trạng thái dẫn xuất, nhưng chúng có thể được chạy ở một phạm vi nhỏ nơi chúng hoạt động tốt [8]. Các giao dịch phân tán không đồng nhất như các giao dịch XA (xem “Giao dịch phân tán trong thực tế”) là không cần thiết. Việc phối hợp đồng bộ vẫn có thể được đưa vào ở những chỗ cần đến (chẳng hạn, để thực thi các ràng buộc nghiêm ngặt trước một thao tác mà từ đó việc phục hồi là bất khả thi), nhưng không cần mọi thứ đều phải trả cái giá của sự phối hợp nếu chỉ một phần nhỏ của ứng dụng cần nó [43].

Another way of looking at coordination and constraints: they reduce the number of apologies you have to make due to inconsistencies, but potentially also reduce the performance and availability of your system, and thus potentially increase the number of apologies you have to make due to outages. You cannot reduce the number of apologies to zero, but you can aim to find the best trade-off for your needs—the sweet spot where there are neither too many inconsistencies nor too many availability problems.

Một cách khác để nhìn về sự phối hợp và các ràng buộc: chúng giảm số lượng lời xin lỗi bạn phải đưa ra do các sự bất nhất, nhưng cũng có khả năng giảm hiệu năng và tính sẵn

sàng của hệ thống của bạn, và do đó có khả năng tăng số lượng lời xin lỗi bạn phải đưa ra do các sự gián đoạn. Bạn không thể giảm số lượng lời xin lỗi xuống không, nhưng bạn có thể nhắm tới việc tìm sự đánh đổi tốt nhất cho nhu cầu của mình — điểm ngọt ngào nơi không có quá nhiều sự bất nhất cũng không có quá nhiều vấn đề về tính sẵn sàng.

Trust, but Verify — Hãy tin, nhưng phải kiểm chứng

All of our discussion of correctness, integrity, and fault-tolerance has been under the assumption that certain things might go wrong, but other things won't. We call these assumptions our system model (see “Mapping system models to the real world”): for example, we should assume that processes can crash, machines can suddenly lose power, and the network can arbitrarily delay or drop messages. But we might also assume that data written to disk is not lost after fsync, that data in memory is not corrupted, and that the multiplication instruction of our CPU always returns the correct result.

Toàn bộ phần thảo luận của ta về sự đúng đắn, tính toàn vẹn, và khả năng chịu lỗi đã diễn ra dưới giả định rằng một số thứ có thể trục trặc, nhưng những thứ khác thì không. Ta gọi những giả định này là mô hình hệ thống (system model) của mình (xem “Ánh xạ mô hình hệ thống vào thế giới thực”): chẳng hạn, ta nên giả định rằng các tiến trình có thể bị sự cố, các máy có thể đột ngột mất điện, và mạng có thể trì hoãn hoặc làm rớt các thông điệp một cách tùy tiện. Nhưng ta cũng có thể giả định rằng dữ liệu được ghi xuống đĩa không bị mất sau fsync, rằng dữ liệu trong bộ nhớ không bị hỏng, và rằng lệnh nhân của CPU luôn trả về kết quả đúng.

These assumptions are quite reasonable, as they are true most of the time, and it would be difficult to get anything done if we had to constantly worry about our computers making mistakes. Traditionally, system models take a binary approach toward faults: we assume that some things can happen, and other things can never happen. In reality, it is more a question of probabilities: some things are more likely, other things less likely. The question is whether violations of our assumptions happen often enough that we may encounter them in practice.

Những giả định này khá hợp lý, bởi chúng đúng trong hầu hết thời gian, và sẽ khó làm xong bất cứ việc gì nếu ta phải liên tục lo lắng rằng máy tính của mình mắc lỗi. Theo truyền thống, các mô hình hệ thống có cách tiếp cận nhị phân đối với lỗi: ta giả định rằng một số thứ có thể xảy ra, và những thứ khác không bao giờ xảy ra. Trên thực tế, đó nhiều hơn là một câu hỏi về xác suất: một số thứ có khả năng cao hơn, những thứ khác ít khả năng hơn. Câu hỏi là liệu các vi phạm giả định của ta có xảy ra đủ thường xuyên để ta có thể gặp chúng trong thực tế hay không.

We have seen that data can become corrupted while it is sitting untouched on disks (see “Replication and Durability”), and data corruption on the network can sometimes evade the TCP checksums (see “Weak forms of lying”). Maybe this is something we should be paying more attention to?

Ta đã thấy rằng dữ liệu có thể bị hỏng trong khi nó nằm yên không ai động tới trên đĩa (xem “Sao chép và độ bền”), và sự hỏng dữ liệu trên mạng đôi khi có thể né được các checksum của TCP (xem “Các dạng nói dối yếu”). Có lẽ đây là điều ta nên chú ý nhiều hơn?

One application that I worked on in the past collected crash reports from clients, and some of the reports we received could only be explained by random bit-flips in the memory of those devices. It seems unlikely, but if you have enough devices running your software, even very unlikely things do happen. Besides random memory corruption due to hardware faults or radiation, certain pathological memory access patterns can flip bits even in memory that has no faults [62]—an effect that can be used to break security mechanisms in operating systems [63] (this technique is known as

rowhammer). Once you look closely, hardware isn't quite the perfect abstraction that it may seem.

Một ứng dụng tôi từng làm trong quá khứ thu thập các báo cáo sự cố từ các client, và một số báo cáo ta nhận được chỉ có thể được giải thích bằng việc các bit bị lật ngẫu nhiên trong bộ nhớ của những thiết bị đó. Nghe có vẻ khó xảy ra, nhưng nếu bạn có đủ nhiều thiết bị chạy phần mềm của mình, thì ngay cả những điều rất khó xảy ra cũng vẫn xảy ra. Bên cạnh việc hỏng bộ nhớ ngẫu nhiên do lỗi phần cứng hoặc bức xạ, một số mẫu truy cập bộ nhớ bệnh lý nhất định có thể lật các bit ngay cả trong bộ nhớ không có lỗi [62] — một hiệu ứng có thể được dùng để phá vỡ các cơ chế an ninh trong các hệ điều hành [63] (kỹ thuật này được biết đến với tên rowhammer). Một khi bạn nhìn kỹ, phần cứng không hẳn là lớp trừu tượng hoàn hảo như vẻ ngoài của nó.

To be clear, random bit-flips are still very rare on modern hardware [64]. I just want to point out that they are not beyond the realm of possibility, and so they deserve some attention.

Để cho rõ ràng, các bit bị lật ngẫu nhiên vẫn rất hiếm trên phần cứng hiện đại [64]. Tôi chỉ muốn chỉ ra rằng chúng không nằm ngoài phạm vi khả dĩ, nên chúng đáng được chú ý đôi chút.

Maintaining integrity in the face of software bugs — Duy trì tính toàn vẹn khi đối mặt với bug phần mềm

Besides such hardware issues, there is always the risk of software bugs, which would not be caught by lower-level network, memory, or filesystem checksums. Even widely used database software has bugs: I have personally seen cases of MySQL failing to correctly maintain a uniqueness constraint [65] and PostgreSQL's serializable isolation level exhibiting write skew anomalies [66], even though MySQL and PostgreSQL are robust and well-regarded databases that have been battle-tested by many people for many years. In less mature software, the situation is likely to be much worse.

Bên cạnh những vấn đề phần cứng như vậy, luôn có rủi ro về bug phần mềm, vốn sẽ không bị bắt bởi các checksum mạng, bộ nhớ, hoặc hệ thống tệp ở mức thấp hơn. Ngay cả phần mềm cơ sở dữ liệu được dùng rộng rãi cũng có bug: cá nhân tôi đã thấy các trường hợp MySQL không duy trì đúng một ràng buộc tính duy nhất [65] và mức cô lập khả tuân tự hóa của PostgreSQL biểu lộ các bất thường lệch ghi [66], dù cho MySQL và PostgreSQL là những cơ sở dữ liệu vững vàng và được đánh giá cao, đã được nhiều người thử lửa trong nhiều năm. Với phần mềm kém trưởng thành hơn, tình hình có khả năng tệ hơn nhiều.

Despite considerable efforts in careful design, testing, and review, bugs still creep in. Although they are rare, and they eventually get found and fixed, there is still a period during which such bugs can corrupt data.

Bất chấp những nỗ lực đáng kể trong việc thiết kế, kiểm thử, và rà soát cẩn thận, các bug vẫn lọt vào. Mặc dù chúng hiếm, và rốt cuộc sẽ được tìm ra và sửa, vẫn có một khoảng thời gian mà trong đó các bug như vậy có thể làm hỏng dữ liệu.

When it comes to application code, we have to assume many more bugs, since most applications don't receive anywhere near the amount of review and testing that database code does. Many applications don't even correctly use the features that databases offer for preserving integrity, such as **foreign key** or uniqueness constraints [36].

Khi nói tới mã ứng dụng, ta phải giả định nhiều bug hơn nữa, bởi hầu hết các ứng dụng không nhận được lượng rà soát và kiểm thử gần bằng mức mà mã cơ sở dữ liệu nhận được. Nhiều ứng dụng thậm chí không dùng đúng các tính năng mà cơ sở dữ liệu cung cấp để bảo toàn tính toàn vẹn, chẳng hạn các ràng buộc khóa ngoại hoặc tính duy nhất [36].

Consistency in the sense of ACID (see “Consistency”) is based on the idea that the database starts off in a consistent state, and a transaction transforms it from one consistent state to another consistent state. Thus, we expect the database to always be in a consistent state. However, this notion only makes sense if you assume that the transaction is free from bugs. If the application uses the database incorrectly in some way, for example using a weak isolation level unsafely, the integrity of the database cannot be guaranteed.

Tính nhất quán theo nghĩa ACID (xem “Tính nhất quán”) dựa trên ý tưởng rằng cơ sở dữ liệu khởi đầu ở một trạng thái nhất quán, và một giao dịch biến đổi nó từ một trạng thái nhất quán này sang một trạng thái nhất quán khác. Do đó, ta kỳ vọng cơ sở dữ liệu luôn ở một trạng thái nhất quán. Tuy nhiên, ý niệm này chỉ có nghĩa nếu bạn giả định rằng giao dịch không có bug. Nếu ứng dụng dùng cơ sở dữ liệu sai theo cách nào đó, chẳng hạn dùng một mức cô lập yếu một cách không an toàn, thì tính toàn vẹn của cơ sở dữ liệu không thể được đảm bảo.

Don’t just blindly trust what they promise — Đừng mù quáng tin vào những gì họ hứa hẹn

With both hardware and software not always living up to the ideal that we would like them to be, it seems that data corruption is inevitable sooner or later. Thus, we should at least have a way of finding out if data has been corrupted so that we can fix it and try to track down the source of the error. Checking the integrity of data is known as auditing.

Khi cả phần cứng lẫn phần mềm đều không phải lúc nào cũng đạt được lý tưởng mà ta mong muốn, dường như việc hỏng dữ liệu là không thể tránh khỏi sớm hay muộn. Do đó, ít nhất ta nên có một cách để phát hiện liệu dữ liệu có bị hỏng hay không, để ta có thể sửa nó và cố truy tìm nguồn gốc của lỗi. Việc kiểm tra tính toàn vẹn của dữ liệu được gọi là kiểm toán (auditing).

As discussed in “Advantages of immutable events”, auditing is not just for financial applications. However, auditability is highly important in finance precisely because everyone knows that mistakes happen, and we all recognize the need to be able to detect and fix problems.

Như đã bàn ở “Ưu điểm của sự kiện bất biến”, việc kiểm toán không chỉ dành cho các ứng dụng tài chính. Tuy nhiên, khả năng kiểm toán đặc biệt quan trọng trong tài chính chính bởi ai cũng biết rằng sai lầm vẫn xảy ra, và tất cả ta đều nhận ra nhu cầu có thể phát hiện và sửa các vấn đề.

Mature systems similarly tend to consider the possibility of unlikely things going wrong, and manage that risk. For example, large-scale storage systems such as HDFS and Amazon S3 do not fully trust disks: they run background processes that continually read back files, compare them to other replicas, and move files from one disk to another, in order to mitigate the risk of silent corruption [67].

Các hệ thống trưởng thành tương tự cũng có xu hướng xét tới khả năng những điều khó xảy ra trực trặc, và quản lý rủi ro đó. Chẳng hạn, các hệ thống lưu trữ quy mô lớn như HDFS và Amazon S3 không hoàn toàn tin tưởng các đĩa: chúng chạy các tiến trình nền liên tục đọc lại các tệp, so sánh chúng với các bản sao khác, và di chuyển các tệp từ đĩa này sang đĩa khác, để giảm thiểu rủi ro hỏng hóc thầm lặng [67].

If you want to be sure that your data is still there, you have to actually read it and check. Most of the time it will still be there, but if it isn’t, you really want to find out sooner rather than later. By the same argument, it is important to try restoring from your backups from time to time—otherwise

you may only find out that your **backup** is broken when it is too late and you have already lost data. Don't just blindly trust that it is all working.

Nếu bạn muốn chắc chắn rằng dữ liệu của mình vẫn còn đó, bạn phải thực sự đọc nó và kiểm tra. Hầu hết thời gian nó sẽ vẫn còn đó, nhưng nếu không, bạn thực sự muốn phát hiện ra sớm còn hơn muộn. Bằng cùng lập luận đó, việc thử khôi phục từ các bản sao lưu của bạn theo thời gian là quan trọng — nếu không, bạn có thể chỉ phát hiện ra rằng bản sao lưu của mình bị hỏng khi đã quá muộn và bạn đã mất dữ liệu. Đừng mù quáng tin rằng mọi thứ đều ổn.

A culture of verification — Một văn hóa kiểm chứng

Systems like HDFS and S3 still have to assume that disks work correctly most of the time—which is a reasonable assumption, but not the same as assuming that they always work correctly. However, not many systems currently have this kind of “trust, but verify” approach of continually auditing themselves. Many assume that correctness guarantees are absolute and make no provision for the possibility of rare data corruption. I hope that in the future we will see more self-validating or self-auditing systems that continually check their own integrity, rather than relying on blind trust [68].

Các hệ thống như HDFS và S3 vẫn phải giả định rằng các đĩa hoạt động đúng trong hầu hết thời gian — đó là một giả định hợp lý, nhưng không giống với việc giả định rằng chúng luôn hoạt động đúng. Tuy nhiên, hiện chưa có nhiều hệ thống có kiểu cách tiếp cận “hãy tin, nhưng phải kiểm chứng” này — tức là liên tục tự kiểm toán bản thân. Nhiều hệ thống giả định rằng các đảm bảo về sự đúng đắn là tuyệt đối và không hề dự liệu cho khả năng hỏng dữ liệu hiếm gặp. Tôi hy vọng rằng trong tương lai ta sẽ thấy nhiều hệ thống tự-thẩm-định hoặc tự-kiểm-toán hơn, vốn liên tục kiểm tra tính toàn vẹn của chính mình, thay vì dựa vào lòng tin mù quáng [68].

I fear that the culture of ACID databases has led us toward developing applications on the basis of blindly trusting technology (such as a transaction mechanism), and neglecting any sort of auditability in the process. Since the technology we trusted worked well enough most of the time, auditing mechanisms were not deemed worth the investment.

Tôi e rằng văn hóa của các cơ sở dữ liệu ACID đã dẫn dắt ta tới việc phát triển ứng dụng trên cơ sở mù quáng tin tưởng công nghệ (chẳng hạn một cơ chế giao dịch), và bỏ bê mọi dạng khả năng kiểm toán trong quá trình đó. Vì công nghệ mà ta tin tưởng đã hoạt động đủ tốt trong hầu hết thời gian, các cơ chế kiểm toán không được coi là đáng để đầu tư.

But then the database landscape changed: weaker consistency guarantees became the norm under the banner of NoSQL, and less mature storage technologies became widely used. Yet, because the audit mechanisms had not been developed, we continued building applications on the basis of blind trust, even though this approach had now become more dangerous. Let's think for a moment about designing for auditability.

Nhưng rồi bối cảnh cơ sở dữ liệu thay đổi: các đảm bảo nhất quán yếu hơn trở thành chuẩn mực dưới ngọn cờ NoSQL, và các công nghệ lưu trữ kém trưởng thành hơn trở nên được dùng rộng rãi. Tuy nhiên, vì các cơ chế kiểm toán đã không được phát triển, ta tiếp tục xây dựng ứng dụng trên cơ sở lòng tin mù quáng, mặc dù cách tiếp cận này giờ đây đã trở nên nguy hiểm hơn. Hãy dành một phút suy nghĩ về việc thiết kế cho khả năng kiểm toán.

Designing for auditability — Thiết kế cho khả năng kiểm toán

If a transaction mutates several objects in a database, it is difficult to tell after the fact what that transaction means. Even if you capture the transaction logs (see “Change Data Capture”), the insertions, updates, and deletions in various tables do not necessarily give a clear picture of why those mutations were performed. The invocation of the application logic that decided on those mutations is transient and cannot be reproduced.

Nếu một giao dịch biến đổi vài đối tượng trong một cơ sở dữ liệu, thật khó để nói sau đó rằng giao dịch đó có nghĩa là gì. Ngay cả khi bạn nắm bắt các log giao dịch (xem “Nắm bắt thay đổi dữ liệu”), các lệnh chèn, cập nhật, và xóa ở nhiều bảng khác nhau không nhất thiết cho một bức tranh rõ ràng về lý do những biến đổi đó được thực hiện. Lời gọi logic ứng dụng vốn quyết định những biến đổi đó là thoáng qua và không thể được tái tạo.

By contrast, event-based systems can provide better auditability. In the event sourcing approach, user input to the system is represented as a single immutable event, and any resulting state updates are derived from that event. The derivation can be made deterministic and repeatable, so that running the same log of events through the same version of the derivation code will result in the same state updates.

Ngược lại, các hệ thống dựa trên sự kiện có thể cung cấp khả năng kiểm toán tốt hơn. Trong cách tiếp cận nguồn-sự-kiện, đầu vào của người dùng tới hệ thống được biểu diễn dưới dạng một sự kiện bất biến đơn, và mọi cập nhật trạng thái phát sinh đều được dẫn xuất từ sự kiện đó. Việc dẫn xuất có thể được làm cho mang tính tất định và lặp lại được, sao cho việc chạy cùng một log sự kiện qua cùng một phiên bản của mã dẫn xuất sẽ cho ra cùng các cập nhật trạng thái.

Being explicit about dataflow (see “Philosophy of batch process outputs”) makes the provenance of data much clearer, which makes integrity checking much more feasible. For the event log, we can use hashes to check that the event storage has not been corrupted. For any derived state, we can rerun the batch and stream processors that derived it from the event log in order to check whether we get the same result, or even run a redundant derivation in parallel.

Việc tường minh về luồng dữ liệu (xem “Triết lý về đầu ra của tiến trình theo lô”) làm cho nguồn gốc của dữ liệu rõ ràng hơn nhiều, điều này khiến việc kiểm tra tính toàn vẹn khả thi hơn nhiều. Với log sự kiện, ta có thể dùng các mã băm để kiểm tra rằng kho lưu trữ sự kiện không bị hỏng. Với bất kỳ trạng thái dẫn xuất nào, ta có thể chạy lại các bộ xử lý theo lô và xử lý luồng vốn đã dẫn xuất nó từ log sự kiện để kiểm tra xem ta có nhận được cùng kết quả hay không, hoặc thậm chí chạy một bản dẫn xuất dư thừa song song.

A deterministic and well-defined dataflow also makes it easier to debug and trace the execution of a system in order to determine why it did something [4, 69]. If something unexpected occurred, it is valuable to have the diagnostic capability to reproduce the exact circumstances that led to the unexpected event—a kind of time-travel debugging capability.

Một luồng dữ liệu tất định và được định nghĩa rõ ràng cũng giúp dễ dàng hơn trong việc gỡ lỗi và truy vết việc thực thi của một hệ thống nhằm xác định tại sao nó làm một điều gì đó [4, 69]. Nếu một điều bất ngờ xảy ra, sẽ rất giá trị khi có khả năng chẩn đoán để tái tạo chính xác các hoàn cảnh đã dẫn tới sự kiện bất ngờ — một dạng khả năng gỡ lỗi du-hành-thời-gian.

The end-to-end argument again — Lại là lập luận đầu-cuối

If we cannot fully trust that every individual component of the system will be free from corruption—that every piece of hardware is fault-free and that every piece of software is bug-free—then we must at least periodically check the integrity of our data. If we don’t check, we won’t find out about corruption until it is too late and it has caused some downstream damage, at which point it will be much harder and more expensive to track down the problem.

Nếu ta không thể hoàn toàn tin rằng mọi thành phần riêng lẻ của hệ thống sẽ không bị hỏng — rằng mọi mảnh phần cứng đều không lỗi và mọi mảnh phần mềm đều không bug — thì ta ít nhất phải kiểm tra tính toàn vẹn của dữ liệu một cách định kỳ. Nếu ta không kiểm tra, ta sẽ không phát hiện ra sự hỏng hóc cho tới khi quá muộn và nó đã gây ra một số thiệt hại xuôi dòng, mà tại thời điểm đó việc truy tìm vấn đề sẽ khó hơn và tốn kém hơn nhiều.

Checking the integrity of data systems is best done in an end-to-end fashion (see “The End-to-End Argument for Databases”): the more systems we can include in an integrity check, the fewer opportunities there are for corruption to go unnoticed at some stage of the process. If we can check that an entire derived data pipeline is correct end to end, then any disks, networks, services, and algorithms along the path are implicitly included in the check.

Việc kiểm tra tính toàn vẹn của các hệ thống dữ liệu được làm tốt nhất theo lối đầu-cuối (xem “Lập luận đầu-cuối cho cơ sở dữ liệu”): càng nhiều hệ thống ta có thể đưa vào một phép kiểm tra tính toàn vẹn, càng ít cơ hội cho sự hỏng hóc lọt qua mà không bị phát hiện ở một giai đoạn nào đó của quá trình. Nếu ta có thể kiểm tra rằng toàn bộ một đường ống dữ liệu dẫn xuất là đúng đắn từ đầu tới cuối, thì bất kỳ đĩa, mạng, dịch vụ, và thuật toán nào dọc đường đều ngầm được đưa vào phép kiểm tra.

Having continuous end-to-end integrity checks gives you increased confidence about the correctness of your systems, which in turn allows you to move faster [70]. Like automated testing, auditing increases the chances that bugs will be found quickly, and thus reduces the risk that a change to the system or a new storage technology will cause damage. If you are not afraid of making changes, you can much better evolve an application to meet changing requirements.

Việc có các phép kiểm tra tính toàn vẹn đầu-cuối liên tục cho bạn sự tự tin gia tăng về sự đúng đắn của các hệ thống của mình, điều này đến lượt nó cho phép bạn di chuyển nhanh hơn [70]. Giống như kiểm thử tự động, kiểm toán làm tăng khả năng các bug sẽ được tìm ra nhanh chóng, và do đó giảm rủi ro rằng một thay đổi với hệ thống hoặc một công nghệ lưu trữ mới sẽ gây thiệt hại. Nếu bạn không sợ thực hiện các thay đổi, bạn có thể tiến hóa một ứng dụng để đáp ứng các yêu cầu thay đổi tốt hơn nhiều.

Tools for auditable data systems — Công cụ cho các hệ thống dữ liệu có thể kiểm toán

At present, not many data systems make auditability a top-level concern. Some applications implement their own audit mechanisms, for example by logging all changes to a separate audit table, but guaranteeing the integrity of the audit log and the database state is still difficult. A transaction log can be made tamper-proof by periodically signing it with a hardware security module, but that does not guarantee that the right transactions went into the log in the first place.

Hiện tại, không có nhiều hệ thống dữ liệu coi khả năng kiểm toán là một mối bận tâm hàng đầu. Một số ứng dụng tự hiện thực các cơ chế kiểm toán của riêng mình, chẳng hạn bằng cách ghi log mọi thay đổi vào một bảng kiểm toán riêng, nhưng việc đảm bảo tính

toàn vẹn của log kiểm toán và trạng thái cơ sở dữ liệu vẫn còn khó. Một log giao dịch có thể được làm cho chống-giả-mạo bằng cách định kỳ ký nó bằng một mô-đun an ninh phần cứng, nhưng điều đó không đảm bảo rằng các giao dịch đúng đã đi vào log ngay từ đầu.

It would be interesting to use cryptographic tools to prove the integrity of a system in a way that is robust to a wide range of hardware and software issues, and even potentially malicious actions. Cryptocurrencies, blockchains, and distributed ledger technologies such as Bitcoin, Ethereum, Ripple, Stellar, and various others [71, 72, 73] have sprung up to explore this area.

Sẽ thú vị khi dùng các công cụ mật mã để chứng minh tính toàn vẹn của một hệ thống theo một cách vững vàng trước một loạt các vấn đề phần cứng và phần mềm, và thậm chí có khả năng cả các hành động độc hại. Các loại tiền mã hóa, blockchain, và các công nghệ sổ cái phân tán như Bitcoin, Ethereum, Ripple, Stellar, và nhiều thứ khác [71, 72, 73] đã mọc lên để khám phá lĩnh vực này.

I am not qualified to comment on the merits of these technologies as currencies or mechanisms for agreeing contracts. However, from a data systems point of view they contain some interesting ideas. Essentially, they are distributed databases, with a data model and transaction mechanism, in which different replicas can be hosted by mutually untrusting organizations. The replicas continually check each other's integrity and use a consensus protocol to agree on the transactions that should be executed.

Tôi không đủ tư cách để bình luận về các giá trị của những công nghệ này với tư cách là tiền tệ hay cơ chế để thỏa thuận hợp đồng. Tuy nhiên, từ góc nhìn các hệ thống dữ liệu, chúng chứa một số ý tưởng thú vị. Về bản chất, chúng là các cơ sở dữ liệu phân tán, với một mô hình dữ liệu và một cơ chế giao dịch, trong đó các bản sao khác nhau có thể được lưu trữ bởi các tổ chức không tin tưởng lẫn nhau. Các bản sao liên tục kiểm tra tính toàn vẹn của nhau và dùng một giao thức đồng thuận để thỏa thuận về các giao dịch nên được thực thi.

I am somewhat skeptical about the Byzantine fault tolerance aspects of these technologies (see “Byzantine Faults”), and I find the technique of proof of work (e.g., Bitcoin mining) extraordinarily wasteful. The transaction throughput of Bitcoin is rather low, albeit for political and economic reasons more than for technical ones. However, the integrity checking aspects are interesting.

Tôi phần nào hoài nghi về các khía cạnh chịu lỗi Byzantine của những công nghệ này (xem “Lỗi Byzantine”), và tôi thấy kỹ thuật bằng chứng công việc (proof of work, ví dụ việc đào Bitcoin) lãng phí một cách phi thường. Thông lượng giao dịch của Bitcoin khá thấp, dù vì lý do chính trị và kinh tế nhiều hơn là vì lý do kỹ thuật. Tuy nhiên, các khía cạnh kiểm tra tính toàn vẹn thì thú vị.

Cryptographic auditing and integrity checking often relies on Merkle trees [74], which are trees of hashes that can be used to efficiently prove that a record appears in some dataset (and a few other things). Outside of the hype of cryptocurrencies, certificate transparency is a security technology that relies on Merkle trees to check the validity of TLS/SSL certificates [75, 76].

Việc kiểm toán và kiểm tra tính toàn vẹn bằng mật mã thường dựa vào cây Merkle [74], vốn là các cây mã băm có thể được dùng để chứng minh một cách hiệu quả rằng một bản ghi xuất hiện trong một tập dữ liệu nào đó (và một vài điều khác). Bên ngoài sự cường điệu của các loại tiền mã hóa, tính minh bạch chứng chỉ (certificate transparency) là một công nghệ an ninh dựa vào cây Merkle để kiểm tra tính hợp lệ của các chứng chỉ TLS/SSL [75, 76].

I could imagine integrity-checking and auditing algorithms, like those of certificate transparency and distributed ledgers, becoming more widely used in data systems in general. Some work will be

needed to make them equally scalable as systems without cryptographic auditing, and to keep the performance penalty as low as possible. But I think this is an interesting area to watch in the future.

Tôi có thể hình dung các thuật toán kiểm tra tính toàn vẹn và kiểm toán, như các thuật toán của tính minh bạch chứng chỉ và các sổ cái phân tán, trở nên được dùng rộng rãi hơn trong các hệ thống dữ liệu nói chung. Sẽ cần một số công sức để làm cho chúng có khả năng mở rộng ngang bằng các hệ thống không có kiểm toán mật mã, và để giữ phần phạt hiệu năng càng thấp càng tốt. Nhưng tôi cho rằng đây là một lĩnh vực thú vị để dõi theo trong tương lai.

Doing the Right Thing — Làm điều đúng đắn

In the final section of this book, I would like to take a step back. Throughout this book we have examined a wide range of different architectures for data systems, evaluated their pros and cons, and explored techniques for building reliable, scalable, and maintainable applications. However, we have left out an important and fundamental part of the discussion, which I would now like to fill in.

Trong phần cuối cùng của cuốn sách này, tôi muốn lùi lại một bước. Xuyên suốt cuốn sách, ta đã xem xét một loạt các kiến trúc khác nhau cho các hệ thống dữ liệu, đánh giá các ưu nhược điểm của chúng, và khám phá các kỹ thuật để xây dựng các ứng dụng đáng tin cậy, có khả năng mở rộng, và dễ bảo trì. Tuy nhiên, ta đã bỏ sót một phần quan trọng và nền tảng của cuộc thảo luận, mà giờ đây tôi muốn bổ khuyết.

Every system is built for a purpose; every action we take has both intended and unintended consequences. The purpose may be as simple as making money, but the consequences for the world may reach far beyond that original purpose. We, the engineers building these systems, have a responsibility to carefully consider those consequences and to consciously decide what kind of world we want to live in.

Mọi hệ thống được xây dựng vì một mục đích; mọi hành động ta thực hiện đều có cả những hệ quả có chủ ý lẫn không chủ ý. Mục đích có thể đơn giản như kiếm tiền, nhưng các hệ quả đối với thế giới có thể vươn xa hơn nhiều so với mục đích ban đầu đó. Chúng ta, những kỹ sư xây dựng các hệ thống này, có trách nhiệm cẩn thận xét tới những hệ quả đó và quyết định một cách có ý thức về loại thế giới mà ta muốn sống trong đó.

We talk about data as an abstract thing, but remember that many datasets are about people: their behavior, their interests, their identity. We must treat such data with humanity and respect. Users are humans too, and human dignity is paramount.

Ta nói về dữ liệu như một thứ trừu tượng, nhưng hãy nhớ rằng nhiều tập dữ liệu là về con người: hành vi của họ, sở thích của họ, danh tính của họ. Ta phải đối xử với dữ liệu như vậy bằng tình người và sự tôn trọng. Người dùng cũng là con người, và nhân phẩm là tối thượng.

Software development increasingly involves making important ethical choices. There are guidelines to help software engineers navigate these issues, such as the ACM's Software Engineering Code of Ethics and Professional Practice [77], but they are rarely discussed, applied, and enforced in practice. As a result, engineers and product managers sometimes take a very cavalier attitude to privacy and potential negative consequences of their products [78, 79, 80].

Việc phát triển phần mềm ngày càng liên quan tới việc đưa ra những lựa chọn đạo đức quan trọng. Có những hướng dẫn để giúp các kỹ sư phần mềm định hướng qua những vấn

đề này, chẳng hạn Bộ quy tắc Đạo đức và Thực hành Nghề nghiệp về Kỹ thuật Phần mềm của ACM [77], nhưng chúng hiếm khi được bàn luận, áp dụng, và thực thi trong thực tế. Kết quả là, các kỹ sư và quản lý sản phẩm đôi khi có một thái độ rất xuề xòa đối với quyền riêng tư và các hệ quả tiêu cực tiềm tàng của sản phẩm của họ [78, 79, 80].

A technology is not good or bad in itself—what matters is how it is used and how it affects people. This is true for a software system like a search engine in much the same way as it is for a weapon like a gun. I think it is not sufficient for software engineers to focus exclusively on the technology and ignore its consequences: the ethical responsibility is ours to bear also. Reasoning about ethics is difficult, but it is too important to ignore.

Một công nghệ tự thân nó không tốt cũng không xấu — điều quan trọng là nó được dùng ra sao và nó ảnh hưởng tới con người như thế nào. Điều này đúng với một hệ thống phần mềm như một công cụ tìm kiếm cũng giống như nó đúng với một vũ khí như một khẩu súng. Tôi cho rằng việc các kỹ sư phần mềm chỉ tập trung độc nhất vào công nghệ và phớt lờ các hệ quả của nó là không đủ: trách nhiệm đạo đức cũng là của ta để gánh vác. Việc suy luận về đạo đức là khó, nhưng nó quá quan trọng để phớt lờ.

Predictive Analytics — Phân tích dự báo

For example, predictive analytics is a major part of the “Big Data” hype. Using data analysis to predict the weather, or the spread of diseases, is one thing [81]; it is another matter to predict whether a convict is likely to reoffend, whether an applicant for a loan is likely to default, or whether an insurance customer is likely to make expensive claims. The latter have a direct effect on individual people’s lives.

Chẳng hạn, phân tích dự báo (predictive analytics) là một phần lớn của sự cường điệu “Dữ liệu lớn” (Big Data). Việc dùng phân tích dữ liệu để dự báo thời tiết, hay sự lây lan của dịch bệnh, là một chuyện [81]; việc dự báo liệu một phạm nhân có khả năng tái phạm hay không, liệu một người xin vay có khả năng vỡ nợ hay không, hay liệu một khách hàng bảo hiểm có khả năng đưa ra các yêu cầu bồi thường tốn kém hay không, lại là một chuyện khác. Những điều sau có tác động trực tiếp tới cuộc sống của từng cá nhân.

Naturally, payment networks want to prevent fraudulent transactions, banks want to avoid bad loans, airlines want to avoid hijackings, and companies want to avoid hiring ineffective or untrustworthy people. From their point of view, the cost of a missed business opportunity is low, but the cost of a bad loan or a problematic employee is much higher, so it is natural for organizations to want to be cautious. If in doubt, they are better off saying no.

Đương nhiên, các mạng thanh toán muốn ngăn các giao dịch gian lận, các ngân hàng muốn tránh các khoản vay xấu, các hãng hàng không muốn tránh các vụ không tặc, và các công ty muốn tránh tuyển những người kém hiệu quả hoặc không đáng tin. Từ góc nhìn của họ, cái giá của một cơ hội kinh doanh bị bỏ lỡ là thấp, nhưng cái giá của một khoản vay xấu hoặc một nhân viên có vấn đề thì cao hơn nhiều, nên việc các tổ chức muốn thận trọng là tự nhiên. Nếu còn nghi ngờ, tốt hơn là họ nói không.

However, as algorithmic decision-making becomes more widespread, someone who has (accurately or falsely) been labeled as risky by some algorithm may suffer a large number of those “no” decisions. Systematically being excluded from jobs, air travel, insurance coverage, property rental, financial services, and other key aspects of society is such a large constraint of the individual’s freedom that it has been called “algorithmic prison” [82]. In countries that respect human rights, the criminal justice system presumes innocence until proven guilty; on the other hand, automated systems can

systematically and arbitrarily exclude a person from participating in society without any proof of guilt, and with little chance of appeal.

Tuy nhiên, khi việc ra quyết định bằng thuật toán trở nên phổ biến hơn, một người đã bị (chính xác hoặc sai lệch) gán nhãn là rủi ro bởi một thuật toán nào đó có thể phải hứng chịu một số lượng lớn những quyết định “không” đó. Việc bị loại trừ một cách có hệ thống khỏi công việc, du lịch hàng không, bảo hiểm, thuê nhà, các dịch vụ tài chính, và các khía cạnh then chốt khác của xã hội là một sự kìm hãm lớn đối với tự do của cá nhân đến mức nó đã được gọi là “nhà tù thuật toán” [82]. Ở các quốc gia tôn trọng nhân quyền, hệ thống tư pháp hình sự giả định vô tội cho tới khi được chứng minh có tội; mặt khác, các hệ thống tự động có thể loại trừ một người khỏi việc tham gia xã hội một cách có hệ thống và tuyền tiện mà không có bất kỳ bằng chứng nào về tội lỗi, và với rất ít cơ hội kháng cáo.

Bias and discrimination — Thiên kiến và phân biệt đối xử

Decisions made by an algorithm are not necessarily any better or any worse than those made by a human. Every person is likely to have biases, even if they actively try to counteract them, and discriminatory practices can become culturally institutionalized. There is hope that basing decisions on data, rather than subjective and instinctive assessments by people, could be more fair and give a better chance to people who are often overlooked in the traditional system [83].

Các quyết định do một thuật toán đưa ra không nhất thiết tốt hơn hay tệ hơn so với những quyết định do một con người đưa ra. Mọi người đều có khả năng có những thiên kiến, ngay cả khi họ chủ động cố chống lại chúng, và các thực hành phân biệt đối xử có thể trở nên được thể chế hóa về mặt văn hóa. Có hy vọng rằng việc dựa các quyết định trên dữ liệu, thay vì những đánh giá chủ quan và theo bản năng của con người, có thể công bằng hơn và cho một cơ hội tốt hơn cho những người thường bị bỏ qua trong hệ thống truyền thống [83].

When we develop predictive analytics systems, we are not merely automating a human’s decision by using software to specify the rules for when to say yes or no; we are even leaving the rules themselves to be inferred from data. However, the patterns learned by these systems are opaque: even if there is some correlation in the data, we may not know why. If there is a systematic bias in the input to an algorithm, the system will most likely learn and amplify that bias in its output [84].

Khi ta phát triển các hệ thống phân tích dự báo, ta không chỉ đơn thuần tự động hóa một quyết định của con người bằng cách dùng phần mềm để chỉ định các quy tắc cho khi nào nói có hoặc không; ta thậm chí còn để cho chính các quy tắc được suy ra từ dữ liệu. Tuy nhiên, các mẫu được những hệ thống này học được lại mờ đục: ngay cả khi có một tương quan nào đó trong dữ liệu, ta có thể không biết tại sao. Nếu có một thiên kiến mang tính hệ thống trong đầu vào của một thuật toán, hệ thống rất có khả năng sẽ học và khuếch đại thiên kiến đó trong đầu ra của nó [84].

In many countries, anti-discrimination laws prohibit treating people differently depending on protected traits such as ethnicity, age, gender, sexuality, disability, or beliefs. Other features of a person’s data may be analyzed, but what happens if they are correlated with protected traits? For example, in racially segregated neighborhoods, a person’s postal code or even their IP address is a strong predictor of race. Put like this, it seems ridiculous to believe that an algorithm could somehow take biased data as input and produce fair and impartial output from it [85]. Yet this belief often seems to be implied by proponents of data-driven decision making, an attitude that has been satirized as “machine learning is like money laundering for bias” [86].

Ở nhiều quốc gia, các luật chống phân biệt đối xử cấm việc đối xử khác nhau với con

người tùy theo các đặc điểm được bảo vệ như sắc tộc, tuổi tác, giới tính, xu hướng tính dục, khuyết tật, hoặc tín ngưỡng. Các đặc điểm khác trong dữ liệu của một người có thể được phân tích, nhưng điều gì xảy ra nếu chúng tương quan với các đặc điểm được bảo vệ? Chẳng hạn, ở các khu dân cư bị phân biệt chủng tộc, mã bưu chính hoặc thậm chí địa chỉ IP của một người là một yếu tố dự báo mạnh về chủng tộc. Nói như vậy, có vẻ thật nực cười khi tin rằng một thuật toán bằng cách nào đó có thể lấy dữ liệu thiên kiến làm đầu vào và tạo ra đầu ra công bằng và không thiên vị từ đó [85]. Tuy nhiên, niềm tin này thường có vẻ được ngụ ý bởi những người ủng hộ việc ra quyết định dựa trên dữ liệu, một thái độ đã bị châm biếm là “học máy giống như rửa tiền cho thiên kiến” [86].

Predictive analytics systems merely extrapolate from the past; if the past is discriminatory, they codify that discrimination. If we want the future to be better than the past, moral imagination is required, and that’s something only humans can provide [87]. Data and models should be our tools, not our masters.

Các hệ thống phân tích dự báo chỉ đơn thuần ngoại suy từ quá khứ; nếu quá khứ là phân biệt đối xử, chúng mã hóa sự phân biệt đó. Nếu ta muốn tương lai tốt đẹp hơn quá khứ, thì cần có trí tưởng tượng đạo đức, và đó là thứ chỉ con người mới có thể cung cấp [87]. Dữ liệu và mô hình nên là công cụ của ta, không phải chủ nhân của ta.

Responsibility and accountability — Trách nhiệm và sự quy trách

Automated decision making opens the question of responsibility and accountability [87]. If a human makes a mistake, they can be held accountable, and the person affected by the decision can appeal. Algorithms make mistakes too, but who is accountable if they go wrong [88]? When a self-driving car causes an accident, who is responsible? If an automated credit scoring algorithm systematically discriminates against people of a particular race or religion, is there any recourse? If a decision by your machine learning system comes under judicial review, can you explain to the judge how the algorithm made its decision?

Việc ra quyết định tự động mở ra câu hỏi về trách nhiệm và sự quy trách [87]. Nếu một con người mắc lỗi, họ có thể bị quy trách, và người bị ảnh hưởng bởi quyết định có thể kháng cáo. Các thuật toán cũng mắc lỗi, nhưng ai chịu trách nhiệm nếu chúng đi sai [88]? Khi một chiếc xe tự lái gây ra một tai nạn, ai chịu trách nhiệm? Nếu một thuật toán chấm điểm tín dụng tự động phân biệt đối xử một cách có hệ thống với những người thuộc một chủng tộc hoặc tôn giáo cụ thể, có cách nào để đòi lại công bằng không? Nếu một quyết định bởi hệ thống học máy của bạn bị đưa ra xem xét tư pháp, bạn có thể giải thích cho thẩm phán cách thuật toán đã đưa ra quyết định của nó không?

Credit rating agencies are an old example of collecting data to make decisions about people. A bad credit score makes life difficult, but at least a credit score is normally based on relevant facts about a person’s actual borrowing history, and any errors in the record can be corrected (although the agencies normally do not make this easy). However, scoring algorithms based on machine learning typically use a much wider range of inputs and are much more opaque, making it harder to understand how a particular decision has come about and whether someone is being treated in an unfair or discriminatory way [89].

Các cơ quan xếp hạng tín dụng là một ví dụ cũ về việc thu thập dữ liệu để ra quyết định về con người. Một điểm tín dụng xấu khiến cuộc sống khó khăn, nhưng ít nhất một điểm tín dụng thường dựa trên các sự kiện liên quan về lịch sử vay mượn thực tế của một người, và bất kỳ lỗi nào trong hồ sơ đều có thể được sửa (mặc dù các cơ quan này thường không làm cho việc này dễ dàng). Tuy nhiên, các thuật toán chấm điểm dựa trên học máy thường dùng một phạm vi đầu vào rộng hơn nhiều và mờ đục hơn nhiều, khiến việc hiểu một

quyết định cụ thể đã nảy sinh như thế nào và liệu ai đó có bị đối xử theo một cách không công bằng hoặc phân biệt đối xử hay không trở nên khó hơn [89].

A credit score summarizes “How did you behave in the past?” whereas predictive analytics usually work on the basis of “Who is similar to you, and how did people like you behave in the past?” Drawing parallels to others’ behavior implies stereotyping people, for example based on where they live (a close proxy for race and socioeconomic class). What about people who get put in the wrong bucket? Furthermore, if a decision is incorrect due to erroneous data, recourse is almost impossible [87].

Một điểm tin dụng tóm tắt “Bạn đã hành xử thế nào trong quá khứ?” trong khi phân tích dự báo thường hoạt động trên cơ sở “Ai giống bạn, và những người như bạn đã hành xử thế nào trong quá khứ?” Việc vẽ ra những điểm tương đồng với hành vi của người khác ngụ ý việc dán nhãn khuôn mẫu cho con người, chẳng hạn dựa trên nơi họ sống (một đại diện gần cho chủng tộc và giai tầng kinh tế xã hội). Còn những người bị xếp vào nhầm rồi thì sao? Hơn nữa, nếu một quyết định sai do dữ liệu lỗi, việc đòi lại công bằng gần như là bất khả thi [87].

Much data is statistical in nature, which means that even if the probability distribution on the whole is correct, individual cases may well be wrong. For example, if the **average** life expectancy in your country is 80 years, that doesn’t mean you’re expected to drop dead on your 80th birthday. From the average and the probability distribution, you can’t say much about the age to which one particular person will live. Similarly, the output of a prediction system is probabilistic and may well be wrong in individual cases.

Phần lớn dữ liệu mang bản chất thống kê, điều này nghĩa là ngay cả khi phân phối xác suất trên tổng thể là đúng, các trường hợp cá nhân vẫn rất có thể sai. Chẳng hạn, nếu tuổi thọ trung bình ở quốc gia của bạn là 80 năm, điều đó không có nghĩa là bạn được kỳ vọng sẽ lần ra chết vào sinh nhật thứ 80 của mình. Từ giá trị trung bình và phân phối xác suất, bạn không thể nói được nhiều về độ tuổi mà một người cụ thể sẽ sống tới. Tương tự, đầu ra của một hệ thống dự báo mang tính xác suất và rất có thể sai trong các trường hợp cá nhân.

A blind belief in the supremacy of data for making decisions is not only delusional, it is positively dangerous. As data-driven decision making becomes more widespread, we will need to figure out how to make algorithms accountable and transparent, how to avoid reinforcing existing biases, and how to fix them when they inevitably make mistakes.

Một niềm tin mù quáng vào tính tối thượng của dữ liệu trong việc ra quyết định không chỉ là ảo tưởng, mà còn thực sự nguy hiểm. Khi việc ra quyết định dựa trên dữ liệu trở nên phổ biến hơn, ta sẽ cần tìm ra cách làm cho các thuật toán có thể bị quy trách và minh bạch, cách tránh củng cố các thiên kiến hiện hữu, và cách sửa chúng khi chúng không tránh khỏi việc mắc lỗi.

We will also need to figure out how to prevent data being used to harm people, and realize its positive potential instead. For example, analytics can reveal financial and social characteristics of people’s lives. On the one hand, this power could be used to focus aid and support to help those people who most need it. On the other hand, it is sometimes used by predatory business seeking to identify vulnerable people and sell them risky products such as high-cost loans and worthless college degrees [87, 90].

Ta cũng sẽ cần tìm ra cách ngăn dữ liệu bị dùng để gây hại cho con người, và thay vào đó hiện thực hóa tiềm năng tích cực của nó. Chẳng hạn, phân tích có thể tiết lộ các đặc điểm tài chính và xã hội trong cuộc sống của con người. Một mặt, sức mạnh này có thể được dùng để tập trung viện trợ và hỗ trợ nhằm giúp những người cần nó nhất. Mặt khác, nó

đôi khi được dùng bởi các doanh nghiệp sẵn lòng tìm cách nhận diện những người dễ tổn thương và bán cho họ các sản phẩm rủi ro như các khoản vay lãi suất cao và các tấm bằng đại học vô giá trị [87, 90].

Feedback loops — Vòng phản hồi

Even with predictive applications that have less immediately far-reaching effects on people, such as recommendation systems, there are difficult issues that we must confront. When services become good at predicting what content users want to see, they may end up showing people only opinions they already agree with, leading to echo chambers in which stereotypes, misinformation, and polarization can breed. We are already seeing the impact of social media echo chambers on election campaigns [91].

Ngay cả với các ứng dụng dự báo có các tác động ít vươn xa tức thì hơn tới con người, chẳng hạn các hệ thống gợi ý, vẫn có những vấn đề khó khăn mà ta phải đối mặt. Khi các dịch vụ trở nên giỏi dự báo nội dung mà người dùng muốn xem, chúng có thể rốt cuộc chỉ cho người ta thấy những quan điểm mà họ vốn đã đồng ý, dẫn tới các buồng vọng âm (echo chamber) nơi các khuôn mẫu, thông tin sai lệch, và sự phân cực có thể sinh sôi. Ta đã và đang thấy tác động của các buồng vọng âm trên mạng xã hội đối với các chiến dịch tranh cử [91].

When predictive analytics affect people's lives, particularly pernicious problems arise due to self-reinforcing feedback loops. For example, consider the case of employers using credit scores to evaluate potential hires. You may be a good worker with a good credit score, but suddenly find yourself in financial difficulties due to a misfortune outside of your control. As you miss payments on your bills, your credit score suffers, and you will be less likely to find work. Joblessness pushes you toward poverty, which further worsens your scores, making it even harder to find employment [87]. It's a downward spiral due to poisonous assumptions, hidden behind a camouflage of mathematical rigor and data.

Khi phân tích dự báo ảnh hưởng tới cuộc sống của con người, những vấn đề đặc biệt nguy hại nảy sinh do các vòng phản hồi tự-củng-cố. Chẳng hạn, hãy xét trường hợp các nhà tuyển dụng dùng điểm tín dụng để đánh giá những người có thể tuyển. Bạn có thể là một người lao động tốt với một điểm tín dụng tốt, nhưng đột nhiên thấy mình rơi vào khó khăn tài chính do một bất hạnh nằm ngoài tầm kiểm soát của bạn. Khi bạn lỡ các khoản thanh toán hóa đơn, điểm tín dụng của bạn xấu đi, và bạn sẽ ít có khả năng tìm được việc hơn. Sự thất nghiệp đẩy bạn về phía nghèo đói, điều này lại càng làm xấu thêm điểm số của bạn, khiến việc tìm việc làm thậm chí còn khó hơn [87]. Đó là một vòng xoáy đi xuống do những giả định độc hại, ẩn sau một lớp nguy trang của sự chặt chẽ toán học và dữ liệu.

We can't always predict when such feedback loops happen. However, many consequences can be predicted by thinking about the entire system (not just the computerized parts, but also the people interacting with it)—an approach known as systems thinking [92]. We can try to understand how a data analysis system responds to different behaviors, structures, or characteristics. Does the system reinforce and amplify existing differences between people (e.g., making the rich richer or the poor poorer), or does it try to combat injustice? And even with the best intentions, we must beware of unintended consequences.

Ta không phải lúc nào cũng có thể dự báo khi nào những vòng phản hồi như vậy xảy ra. Tuy nhiên, nhiều hệ quả có thể được dự báo bằng cách nghĩ về toàn bộ hệ thống (không chỉ các phần được tin học hóa, mà cả những con người tương tác với nó) — một cách tiếp cận được gọi là tư duy hệ thống (systems thinking) [92]. Ta có thể cố hiểu cách một hệ thống phân tích dữ liệu phản ứng với các hành vi, cấu trúc, hoặc đặc điểm khác nhau. Liệu hệ

thống có củng cố và khuếch đại các khác biệt hiện hữu giữa con người (ví dụ, làm cho người giàu giàu hơn hoặc người nghèo nghèo hơn), hay nó cố chống lại sự bất công? Và ngay cả với những ý định tốt đẹp nhất, ta phải đề phòng những hệ quả không chủ ý.

Privacy and Tracking — Quyền riêng tư và theo dõi

Besides the problems of predictive analytics—i.e., using data to make automated decisions about people—there are ethical problems with data collection itself. What is the relationship between the organizations collecting data and the people whose data is being collected?

Bên cạnh các vấn đề của phân tích dự báo — tức là dùng dữ liệu để ra các quyết định tự động về con người — còn có các vấn đề đạo đức với bản thân việc thu thập dữ liệu. Mối quan hệ giữa các tổ chức thu thập dữ liệu và những người có dữ liệu đang bị thu thập là gì?

When a system only stores data that a user has explicitly entered, because they want the system to store and process it in a certain way, the system is performing a service for the user: the user is the customer. But when a user's activity is tracked and logged as a side effect of other things they are doing, the relationship is less clear. The service no longer just does what the user tells it to do, but it takes on interests of its own, which may conflict with the user's interests.

Khi một hệ thống chỉ lưu dữ liệu mà một người dùng đã chủ động nhập vào, bởi họ muốn hệ thống lưu và xử lý nó theo một cách nhất định, thì hệ thống đang thực hiện một dịch vụ cho người dùng: người dùng là khách hàng. Nhưng khi hoạt động của một người dùng bị theo dõi và ghi log như một tác dụng phụ của những việc khác mà họ đang làm, mối quan hệ trở nên kém rõ ràng hơn. Dịch vụ không còn chỉ làm những gì người dùng bảo nó làm, mà nó còn mang lấy các lợi ích của riêng mình, vốn có thể xung đột với lợi ích của người dùng.

Tracking behavioral data has become increasingly important for user-facing features of many online services: tracking which search results are clicked helps improve the ranking of search results; recommending “people who liked X also liked Y” helps users discover interesting and useful things; A/B tests and user flow analysis can help indicate how a user interface might be improved. Those features require some amount of tracking of user behavior, and users benefit from them.

Việc theo dõi dữ liệu hành vi đã trở nên ngày càng quan trọng đối với các tính năng hướng-tới-người-dùng của nhiều dịch vụ trực tuyến: theo dõi kết quả tìm kiếm nào được nhấp vào giúp cải thiện việc xếp hạng các kết quả tìm kiếm; gợi ý “những người thích X cũng thích Y” giúp người dùng khám phá những thứ thú vị và hữu ích; các thử nghiệm A/B và phân tích luồng người dùng có thể giúp chỉ ra một giao diện người dùng có thể được cải thiện như thế nào. Những tính năng đó đòi hỏi một mức độ theo dõi hành vi người dùng nhất định, và người dùng hưởng lợi từ chúng.

However, depending on a company's business model, tracking often doesn't stop there. If the service is funded through advertising, the advertisers are the actual customers, and the users' interests take second place. Tracking data becomes more detailed, analyses become further-reaching, and data is retained for a long time in order to build up detailed profiles of each person for marketing purposes.

Tuy nhiên, tùy vào mô hình kinh doanh của một công ty, việc theo dõi thường không dừng ở đó. Nếu dịch vụ được tài trợ thông qua quảng cáo, thì các nhà quảng cáo mới là khách hàng thực sự, và lợi ích của người dùng bị xếp xuống hàng thứ yếu. Dữ liệu theo dõi trở nên chi tiết hơn, các phân tích vươn xa hơn, và dữ liệu được giữ lại trong một thời gian dài để xây dựng các hồ sơ chi tiết về từng người cho mục đích tiếp thị.

Now the relationship between the company and the user whose data is being collected starts looking quite different. The user is given a free service and is coaxed into engaging with it as much as possible. The tracking of the user serves not primarily that individual, but rather the needs of the advertisers who are funding the service. I think this relationship can be appropriately described with a word that has more sinister connotations: surveillance.

Giờ đây mối quan hệ giữa công ty và người dùng có dữ liệu đang bị thu thập bắt đầu trông khá khác. Người dùng được trao một dịch vụ miễn phí và bị dỗ dành tương tác với nó càng nhiều càng tốt. Việc theo dõi người dùng phục vụ trước hết không phải cho cá nhân đó, mà đúng hơn là cho nhu cầu của các nhà quảng cáo đang tài trợ cho dịch vụ. Tôi cho rằng mối quan hệ này có thể được mô tả một cách thích đáng bằng một từ có hàm ý đen tối hơn: giám sát.

Surveillance — Giám sát

As a thought experiment, try replacing the word data with surveillance, and observe if common phrases still sound so good [93]. How about this: “In our surveillance-driven organization we collect real-time surveillance streams and store them in our surveillance warehouse. Our surveillance scientists use advanced analytics and surveillance processing in order to derive new insights.”

Như một thí nghiệm tư duy, hãy thử thay từ dữ liệu bằng giám sát, và quan sát xem các cụm từ thông dụng có còn nghe hay ho như vậy không [93]. Thế này thì sao: “Trong tổ chức hướng-giám-sát của chúng tôi, chúng tôi thu thập các luồng giám sát thời gian thực và lưu chúng trong kho giám sát của mình. Các nhà khoa học giám sát của chúng tôi dùng phân tích nâng cao và xử lý giám sát để dẫn xuất những hiểu biết mới.”

This thought experiment is unusually polemic for this book, *Designing Surveillance-Intensive Applications*, but I think that strong words are needed to emphasize this point. In our attempts to make software “eat the world” [94], we have built the greatest mass surveillance infrastructure the world has ever seen. Rushing toward an Internet of Things, we are rapidly approaching a world in which every inhabited space contains at least one internet-connected microphone, in the form of smartphones, smart TVs, voice-controlled assistant devices, baby monitors, and even children’s toys that use cloud-based speech recognition. Many of these devices have a terrible security record [95].

Thí nghiệm tư duy này gay gắt một cách bất thường so với cuốn sách này, cuốn *Thiết kế các ứng dụng thiên-về-giám-sát*, nhưng tôi cho rằng cần những lời lẽ mạnh để nhấn mạnh điểm này. Trong các nỗ lực làm cho phần mềm “nuốt chửng thế giới” [94], ta đã xây dựng nên hạ tầng giám sát hàng loạt vĩ đại nhất mà thế giới từng thấy. Lao nhanh về phía Internet vạn vật, ta đang nhanh chóng tiến tới một thế giới mà trong đó mọi không gian có người ở đều chứa ít nhất một micro kết nối internet, dưới dạng điện thoại thông minh, TV thông minh, các thiết bị trợ lý điều khiển bằng giọng nói, máy theo dõi trẻ em, và thậm chí những món đồ chơi trẻ em dùng nhận dạng giọng nói dựa trên đám mây. Nhiều thiết bị trong số này có một hồ sơ an ninh tồi tệ [95].

Even the most totalitarian and repressive regimes could only dream of putting a microphone in every room and forcing every person to constantly carry a device capable of tracking their location and movements. Yet we apparently voluntarily, even enthusiastically, throw ourselves into this world of total surveillance. The difference is just that the data is being collected by corporations rather than government agencies [96].

Ngay cả những chế độ toàn trị và áp bức nhất cũng chỉ có thể mơ về việc đặt một micro trong mọi căn phòng và buộc mọi người liên tục mang theo một thiết bị có khả năng theo dõi vị trí và di chuyển của họ. Tuy nhiên, ta dường như tự nguyện, thậm chí hăm hở, ném

mình vào thế giới giám sát toàn diện này. Khác biệt chỉ là dữ liệu đang bị thu thập bởi các tập đoàn thay vì các cơ quan chính phủ [96].

Not all data collection necessarily qualifies as surveillance, but examining it as such can help us understand our relationship with the data collector. Why are we seemingly happy to accept surveillance by corporations? Perhaps you feel you have nothing to hide—in other words, you are totally in line with existing power structures, you are not a marginalized minority, and you needn't fear persecution [97]. Not everyone is so fortunate. Or perhaps it's because the purpose seems benign—it's not overt coercion and conformance, but merely better recommendations and more personalized marketing. However, combined with the discussion of predictive analytics from the last section, that distinction seems less clear.

Không phải mọi việc thu thập dữ liệu đều nhất thiết đủ tư cách là giám sát, nhưng việc xem xét nó như vậy có thể giúp ta hiểu mối quan hệ của mình với bên thu thập dữ liệu. Tại sao ta dường như vui lòng chấp nhận sự giám sát bởi các tập đoàn? Có lẽ bạn cảm thấy mình không có gì để giấu — nói cách khác, bạn hoàn toàn hòa hợp với các cấu trúc quyền lực hiện hữu, bạn không phải là một thiểu số bị gạt ra lề, và bạn không cần sợ bị bức hại [97]. Không phải ai cũng may mắn như vậy. Hoặc có lẽ vì mục đích có vẻ lành tính — đó không phải là sự cưỡng ép và buộc tuân thủ công khai, mà chỉ là các gợi ý tốt hơn và tiếp thị được cá nhân hóa hơn. Tuy nhiên, kết hợp với phần thảo luận về phân tích dự báo ở phần trước, sự phân biệt đó có vẻ kém rõ ràng hơn.

We are already seeing car insurance premiums linked to tracking devices in cars, and health insurance coverage that depends on people wearing a fitness tracking device. When surveillance is used to determine things that hold sway over important aspects of life, such as insurance coverage or employment, it starts to appear less benign. Moreover, data analysis can reveal surprisingly intrusive things: for example, the movement sensor in a smartwatch or fitness tracker can be used to work out what you are typing (for example, passwords) with fairly good accuracy [98]. And algorithms for analysis are only going to get better.

Ta đã và đang thấy phí bảo hiểm xe hơi gắn với các thiết bị theo dõi trong xe, và phạm vi bảo hiểm sức khỏe phụ thuộc vào việc người ta đeo một thiết bị theo dõi thể chất. Khi sự giám sát được dùng để quyết định những thứ có ảnh hưởng tới các khía cạnh quan trọng của cuộc sống, chẳng hạn phạm vi bảo hiểm hoặc việc làm, nó bắt đầu có vẻ kém lành tính hơn. Hơn nữa, phân tích dữ liệu có thể tiết lộ những điều xâm phạm đến mức đáng ngạc nhiên: chẳng hạn, cảm biến chuyển động trong một chiếc đồng hồ thông minh hoặc máy theo dõi thể chất có thể được dùng để tìm ra bạn đang gõ gì (ví dụ, mật khẩu) với độ chính xác khá tốt [98]. Và các thuật toán phân tích sẽ chỉ càng ngày càng giỏi hơn.

Consent and freedom of choice — Sự đồng thuận và tự do lựa chọn

We might assert that users voluntarily choose to use a service that tracks their activity, and they have agreed to the terms of service and privacy policy, so they consent to data collection. We might even claim that users are receiving a valuable service in return for the data they provide, and that the tracking is necessary in order to provide the service. Undoubtedly, social networks, search engines, and various other free online services are valuable to users—but there are problems with this argument.

Ta có thể khẳng định rằng người dùng tự nguyện chọn dùng một dịch vụ theo dõi hoạt động của họ, và họ đã đồng ý với các điều khoản dịch vụ và chính sách quyền riêng tư, nên họ đồng thuận với việc thu thập dữ liệu. Ta thậm chí có thể tuyên bố rằng người dùng đang nhận được một dịch vụ giá trị để đổi lại dữ liệu mà họ cung cấp, và rằng việc theo dõi là cần thiết để cung cấp dịch vụ. Không nghi ngờ gì, các mạng xã hội, công cụ tìm kiếm, và

nhiều dịch vụ trực tuyến miễn phí khác là giá trị đối với người dùng — nhưng có những vấn đề với lập luận này.

Users have little knowledge of what data they are feeding into our databases, or how it is retained and processed—and most privacy policies do more to obscure than to illuminate. Without understanding what happens to their data, users cannot give any meaningful consent. Often, data from one user also says things about other people who are not users of the service and who have not agreed to any terms. The derived datasets that we discussed in this part of the book—in which data from the entire user base may have been combined with behavioral tracking and external data sources—are precisely the kinds of data of which users cannot have any meaningful understanding.

Người dùng có rất ít hiểu biết về dữ liệu nào họ đang nạp vào các cơ sở dữ liệu của ta, hoặc nó được giữ lại và xử lý ra sao — và hầu hết các chính sách quyền riêng tư làm việc che mờ nhiều hơn là làm sáng tỏ. Không hiểu điều gì xảy ra với dữ liệu của họ, người dùng không thể đưa ra bất kỳ sự đồng thuận có ý nghĩa nào. Thường thì dữ liệu từ một người dùng cũng nói lên những điều về những người khác vốn không phải là người dùng của dịch vụ và chưa hề đồng ý với bất kỳ điều khoản nào. Các tập dữ liệu dẫn xuất mà ta đã bàn trong phần này của cuốn sách — trong đó dữ liệu từ toàn bộ cơ sở người dùng có thể đã được kết hợp với việc theo dõi hành vi và các nguồn dữ liệu bên ngoài — chính là loại dữ liệu mà người dùng không thể nào có một hiểu biết có ý nghĩa.

Moreover, data is extracted from users through a one-way process, not a relationship with true reciprocity, and not a fair value exchange. There is no dialog, no option for users to negotiate how much data they provide and what service they receive in return: the relationship between the service and the user is very asymmetric and one-sided. The terms are set by the service, not by the user [99].

Hơn nữa, dữ liệu được trích xuất từ người dùng thông qua một quá trình một chiều, không phải một mối quan hệ có sự đáp đền thực sự, và không phải một sự trao đổi giá trị công bằng. Không có đối thoại, không có lựa chọn nào để người dùng thương lượng họ cung cấp bao nhiêu dữ liệu và họ nhận lại dịch vụ gì: mối quan hệ giữa dịch vụ và người dùng rất bất đối xứng và một chiều. Các điều khoản được đặt ra bởi dịch vụ, không phải bởi người dùng [99].

For a user who does not consent to surveillance, the only real alternative is simply not to use a service. But this choice is not free either: if a service is so popular that it is “regarded by most people as essential for basic social participation” [99], then it is not reasonable to expect people to opt out of this service—using it is de facto mandatory. For example, in most Western social communities, it has become the norm to carry a smartphone, to use Facebook for socializing, and to use Google for finding information. Especially when a service has network effects, there is a social cost to people choosing not to use it.

Với một người dùng không đồng thuận với sự giám sát, lựa chọn thay thế thực sự duy nhất là đơn giản không dùng dịch vụ. Nhưng lựa chọn này cũng không hề miễn phí: nếu một dịch vụ phổ biến đến mức nó “được hầu hết mọi người coi là thiết yếu cho sự tham gia xã hội cơ bản” [99], thì việc kỳ vọng người ta từ chối dịch vụ này là không hợp lý — việc dùng nó trên thực tế là bắt buộc. Chẳng hạn, ở hầu hết các cộng đồng xã hội phương Tây, việc mang theo một điện thoại thông minh, dùng Facebook để giao tiếp xã hội, và dùng Google để tìm thông tin đã trở thành chuẩn mực. Đặc biệt khi một dịch vụ có hiệu ứng mạng, có một cái giá xã hội cho việc người ta chọn không dùng nó.

Declining to use a service due to its tracking of users is only an option for the small number of people who are privileged enough to have the time and knowledge to understand its privacy policy, and who can afford to potentially miss out on social participation or professional opportunities that may have arisen if they had participated in the service. For people in a less privileged position, there is

no meaningful freedom of choice: surveillance becomes inescapable.

Việc từ chối dùng một dịch vụ vì nó theo dõi người dùng chỉ là một lựa chọn cho một số nhỏ những người đủ đặc quyền để có thời gian và kiến thức hiểu được chính sách quyền riêng tư của nó, và những người có thể chấp nhận việc có khả năng bỏ lỡ sự tham gia xã hội hoặc các cơ hội nghề nghiệp mà lẽ ra đã nảy sinh nếu họ tham gia dịch vụ. Với những người ở vị thế kém đặc quyền hơn, không có tự do lựa chọn có ý nghĩa nào: sự giám sát trở nên không thể né tránh.

Privacy and use of data — Quyền riêng tư và việc sử dụng dữ liệu

Sometimes people claim that “privacy is dead” on the grounds that some users are willing to post all sorts of things about their lives to social media, sometimes mundane and sometimes deeply personal. However, this claim is false and rests on a misunderstanding of the word privacy.

Đôi khi người ta tuyên bố rằng “quyền riêng tư đã chết” trên cơ sở rằng một số người dùng sẵn lòng đăng đủ loại thứ về cuộc sống của họ lên mạng xã hội, đôi khi tầm thường và đôi khi sâu kín. Tuy nhiên, tuyên bố này sai và dựa trên một sự hiểu lầm về từ quyền riêng tư.

Having privacy does not mean keeping everything secret; it means having the freedom to choose which things to reveal to whom, what to make public, and what to keep secret. The right to privacy is a decision right: it enables each person to decide where they want to be on the spectrum between secrecy and transparency in each situation [99]. It is an important aspect of a person’s freedom and autonomy.

Có quyền riêng tư không có nghĩa là giữ mọi thứ bí mật; nó có nghĩa là có tự do lựa chọn những thứ nào để tiết lộ cho ai, cái gì để công khai, và cái gì để giữ bí mật. Quyền riêng tư là một quyền quyết định: nó cho phép mỗi người quyết định họ muốn ở đâu trên phổ giữa sự bí mật và sự minh bạch trong mỗi tình huống [99]. Đó là một khía cạnh quan trọng của tự do và quyền tự chủ của một người.

When data is extracted from people through surveillance infrastructure, privacy rights are not necessarily eroded, but rather transferred to the data collector. Companies that acquire data essentially say “trust us to do the right thing with your data,” which means that the right to decide what to reveal and what to keep secret is transferred from the individual to the company.

Khi dữ liệu được trích xuất từ con người thông qua hạ tầng giám sát, các quyền riêng tư không nhất thiết bị xói mòn, mà đúng hơn là bị chuyển giao cho bên thu thập dữ liệu. Các công ty thu nạp dữ liệu về bản chất nói rằng “hãy tin chúng tôi làm điều đúng đắn với dữ liệu của bạn,” điều này nghĩa là quyền quyết định cái gì để tiết lộ và cái gì để giữ bí mật bị chuyển từ cá nhân sang công ty.

The companies in turn choose to keep much of the outcome of this surveillance secret, because to reveal it would be perceived as creepy, and would harm their business model (which relies on knowing more about people than other companies do). Intimate information about users is only revealed indirectly, for example in the form of tools for targeting advertisements to specific groups of people (such as those suffering from a particular illness).

Đến lượt mình, các công ty chọn giữ phần lớn kết quả của sự giám sát này bí mật, bởi việc tiết lộ nó sẽ bị coi là rùng rợn, và sẽ làm hại mô hình kinh doanh của họ (vốn dựa vào việc biết về con người nhiều hơn so với các công ty khác). Thông tin thầm kín về người dùng chỉ được tiết lộ một cách gián tiếp, chẳng hạn dưới dạng các công cụ để nhắm quảng cáo tới các nhóm người cụ thể (chẳng hạn những người mắc một căn bệnh cụ thể).

Even if particular users cannot be personally reidentified from the bucket of people targeted by a particular ad, they have lost their agency about the disclosure of some intimate information, such as whether they suffer from some illness. It is not the user who decides what is revealed to whom on the basis of their personal preferences—it is the company that exercises the privacy right with the goal of maximizing its profit.

Ngay cả khi những người dùng cụ thể không thể bị tái nhận diện cá nhân từ rổ những người được nhắm tới bởi một quảng cáo cụ thể, họ đã mất quyền tự quyết về việc tiết lộ một số thông tin thầm kín, chẳng hạn liệu họ có mắc một căn bệnh nào đó hay không. Không phải người dùng là người quyết định cái gì được tiết lộ cho ai trên cơ sở các sở thích cá nhân của họ — mà là công ty thực thi quyền riêng tư với mục tiêu tối đa hóa lợi nhuận của nó.

Many companies have a goal of not being perceived as creepy—avoiding the question of how intrusive their data collection actually is, and instead focusing on managing user perceptions. And even these perceptions are often managed poorly: for example, something may be factually correct, but if it triggers painful memories, the user may not want to be reminded about it [100]. With any kind of data we should expect the possibility that it is wrong, undesirable, or inappropriate in some way, and we need to build mechanisms for handling those failures. Whether something is “undesirable” or “inappropriate” is of course down to human judgment; algorithms are oblivious to such notions unless we explicitly program them to respect human needs. As engineers of these systems we must be humble, accepting and planning for such failings.

Nhiều công ty có một mục tiêu là không bị coi là rùng rợn — né tránh câu hỏi về việc thu thập dữ liệu của họ thực sự xâm phạm đến mức nào, và thay vào đó tập trung vào việc quản lý nhận thức của người dùng. Và ngay cả những nhận thức này cũng thường được quản lý tồi: chẳng hạn, một điều gì đó có thể đúng về mặt thực tế, nhưng nếu nó khơi gợi những ký ức đau đớn, người dùng có thể không muốn bị nhắc về nó [100]. Với bất kỳ loại dữ liệu nào, ta nên lường trước khả năng rằng nó sai, không mong muốn, hoặc không thích hợp theo cách nào đó, và ta cần xây dựng các cơ chế để xử lý những thất bại đó. Việc một điều gì đó là “không mong muốn” hay “không thích hợp” tất nhiên tùy thuộc vào phán đoán của con người; các thuật toán không hề biết tới những ý niệm như vậy trừ phi ta tường minh lập trình cho chúng tôn trọng các nhu cầu của con người. Là những kỹ sư của các hệ thống này, ta phải khiêm tốn, chấp nhận và dự liệu cho những thiếu sót như vậy.

Privacy settings that allow a user of an online service to control which aspects of their data other users can see are a starting point for handing back some control to users. However, regardless of the setting, the service itself still has unfettered access to the data, and is free to use it in any way permitted by the privacy policy. Even if the service promises not to sell the data to third parties, it usually grants itself unrestricted rights to process and analyze the data internally, often going much further than what is overtly visible to users.

Các thiết lập quyền riêng tư cho phép một người dùng của một dịch vụ trực tuyến kiểm soát những khía cạnh nào trong dữ liệu của họ mà các người dùng khác có thể thấy là một điểm khởi đầu để trao lại một phần quyền kiểm soát cho người dùng. Tuy nhiên, bất kể thiết lập là gì, bản thân dịch vụ vẫn có quyền truy cập không bị ràng buộc tới dữ liệu, và tự do dùng nó theo bất kỳ cách nào được chính sách quyền riêng tư cho phép. Ngay cả khi dịch vụ hứa không bán dữ liệu cho các bên thứ ba, nó thường tự trao cho mình các quyền không giới hạn để xử lý và phân tích dữ liệu trong nội bộ, thường đi xa hơn nhiều so với những gì hiển hiện công khai với người dùng.

This kind of large-scale transfer of privacy rights from individuals to corporations is historically unprecedented [99]. Surveillance has always existed, but it used to be expensive and manual, not

scalable and automated. Trust relationships have always existed, for example between a patient and their doctor, or between a defendant and their attorney—but in these cases the use of data has been strictly governed by ethical, legal, and regulatory constraints. Internet services have made it much easier to amass huge amounts of sensitive information without meaningful consent, and to use it at massive scale without users understanding what is happening to their private data.

Kiểu chuyển giao quyền riêng tư quy mô lớn từ các cá nhân sang các tập đoàn này là chưa từng có tiền lệ trong lịch sử [99]. Sự giám sát đã luôn tồn tại, nhưng nó từng tốn kém và thủ công, không có khả năng mở rộng và tự động. Các mối quan hệ tin cậy đã luôn tồn tại, chẳng hạn giữa một bệnh nhân và bác sĩ của họ, hoặc giữa một bị cáo và luật sư của họ — nhưng trong những trường hợp này, việc sử dụng dữ liệu đã được quản chặt bởi các ràng buộc đạo đức, pháp lý, và quy định. Các dịch vụ internet đã làm cho việc tích lũy những lượng khổng lồ thông tin nhạy cảm mà không có sự đồng thuận có ý nghĩa trở nên dễ hơn nhiều, và việc dùng nó ở quy mô lớn mà người dùng không hiểu điều gì đang xảy ra với dữ liệu riêng tư của họ.

Data as assets and power — Dữ liệu như tài sản và quyền lực

Since behavioral data is a byproduct of users interacting with a service, it is sometimes called “data exhaust”—suggesting that the data is worthless waste material. Viewed this way, behavioral and predictive analytics can be seen as a form of recycling that extracts value from data that would have otherwise been thrown away.

Vì dữ liệu hành vi là một sản phẩm phụ của việc người dùng tương tác với một dịch vụ, nó đôi khi được gọi là “khí thải dữ liệu” (data exhaust) — gợi ý rằng dữ liệu là chất thải vô giá trị. Nhìn theo cách này, phân tích hành vi và dự báo có thể được xem như một dạng tái chế trích xuất giá trị từ dữ liệu mà lẽ ra đã bị vứt bỏ.

More correct would be to view it the other way round: from an economic point of view, if targeted advertising is what pays for a service, then behavioral data about people is the service’s core asset. In this case, the application with which the user interacts is merely a means to lure users into feeding more and more personal information into the surveillance infrastructure [99]. The delightful human creativity and social relationships that often find expression in online services are cynically exploited by the data extraction machine.

Đúng hơn sẽ là nhìn nó theo chiều ngược lại: từ góc nhìn kinh tế, nếu quảng cáo nhắm mục tiêu là thứ chi trả cho một dịch vụ, thì dữ liệu hành vi về con người chính là tài sản cốt lõi của dịch vụ. Trong trường hợp này, ứng dụng mà người dùng tương tác chỉ đơn thuần là một phương tiện để dụ người dùng nạp ngày càng nhiều thông tin cá nhân vào hạ tầng giám sát [99]. Sự sáng tạo và các mối quan hệ xã hội đáng yêu của con người vốn thường tìm thấy sự thể hiện trong các dịch vụ trực tuyến lại bị bóc lột một cách lạnh lùng bởi cỗ máy trích xuất dữ liệu.

The assertion that personal data is a valuable asset is supported by the existence of data brokers, a shady industry operating in secrecy, purchasing, aggregating, analyzing, inferring, and reselling intrusive personal data about people, mostly for marketing purposes [90]. Startups are valued by their user numbers, by “eyeballs”—i.e., by their surveillance capabilities.

Khẳng định rằng dữ liệu cá nhân là một tài sản giá trị được củng cố bởi sự tồn tại của các bên môi giới dữ liệu (data broker), một ngành công nghiệp mờ ám hoạt động trong vòng bí mật, mua, tổng hợp, phân tích, suy luận, và bán lại dữ liệu cá nhân xâm phạm về con người, chủ yếu cho mục đích tiếp thị [90]. Các startup được định giá bởi số lượng người dùng của chúng, bởi “số nhãn cầu” — tức là bởi các khả năng giám sát của chúng.

Because the data is valuable, many people want it. Of course companies want it—that’s why they collect it in the first place. But governments want to obtain it too: by means of secret deals, coercion, legal compulsion, or simply stealing it [101]. When a company goes bankrupt, the personal data it has collected is one of the assets that get sold. Moreover, the data is difficult to secure, so breaches happen disconcertingly often [102].

Vì dữ liệu là giá trị, nhiều người muốn nó. Tất nhiên các công ty muốn nó — đó là lý do họ thu thập nó ngay từ đầu. Nhưng các chính phủ cũng muốn có được nó: bằng các thỏa thuận bí mật, sự cưỡng ép, sự bắt buộc về pháp lý, hoặc đơn giản là đánh cắp nó [101]. Khi một công ty phá sản, dữ liệu cá nhân mà nó đã thu thập là một trong những tài sản bị đem bán. Hơn nữa, dữ liệu khó được bảo mật, nên các vụ rò rỉ xảy ra thường xuyên đến mức đáng lo ngại [102].

These observations have led critics to saying that data is not just an asset, but a “toxic asset” [101], or at least “hazardous material” [103]. Even if we think that we are capable of preventing abuse of data, whenever we collect data, we need to balance the benefits with the risk of it falling into the wrong hands: computer systems may be compromised by criminals or hostile foreign intelligence services, data may be leaked by insiders, the company may fall into the hands of unscrupulous management that does not share our values, or the country may be taken over by a regime that has no qualms about compelling us to hand over the data.

Những quan sát này đã khiến các nhà phê bình nói rằng dữ liệu không chỉ là một tài sản, mà là một “tài sản độc hại” [101], hoặc ít nhất là “vật liệu nguy hiểm” [103]. Ngay cả khi ta nghĩ rằng mình có khả năng ngăn việc lạm dụng dữ liệu, thì mỗi khi ta thu thập dữ liệu, ta cần cân bằng các lợi ích với rủi ro nó rơi vào tay kẻ xấu: các hệ thống máy tính có thể bị xâm phạm bởi tội phạm hoặc các cơ quan tình báo nước ngoài thù địch, dữ liệu có thể bị rò rỉ bởi người trong nội bộ, công ty có thể rơi vào tay một ban quản lý vô đạo đức không chia sẻ các giá trị của ta, hoặc đất nước có thể bị tiếp quản bởi một chế độ không hề e ngại việc buộc ta giao nộp dữ liệu.

When collecting data, we need to consider not just today’s political environment, but all possible future governments. There is no guarantee that every government elected in future will respect human rights and civil liberties, so “it is poor civic hygiene to install technologies that could someday facilitate a police state” [104].

Khi thu thập dữ liệu, ta cần xét không chỉ môi trường chính trị ngày hôm nay, mà mọi chính phủ tương lai khả dĩ. Không có gì đảm bảo rằng mọi chính phủ được bầu trong tương lai sẽ tôn trọng nhân quyền và các quyền tự do dân sự, nên “việc cài đặt các công nghệ một ngày nào đó có thể tạo điều kiện cho một nhà nước cảnh sát là một thói vệ sinh công dân tồi” [104].

“Knowledge is power,” as the old adage goes. And furthermore, “to scrutinize others while avoiding scrutiny oneself is one of the most important forms of power” [105]. This is why totalitarian governments want surveillance: it gives them the power to control the population. Although today’s technology companies are not overtly seeking political power, the data and knowledge they have accumulated nevertheless gives them a lot of power over our lives, much of which is surreptitious, outside of public oversight [106].

“Tri thức là sức mạnh,” như câu ngạn ngữ cũ vẫn nói. Và hơn nữa, “việc soi xét người khác trong khi tránh bị soi xét chính mình là một trong những hình thức quyền lực quan trọng nhất” [105]. Đây là lý do các chính phủ toàn trị muốn sự giám sát: nó cho họ quyền lực kiểm soát dân chúng. Mặc dù các công ty công nghệ ngày nay không công khai tìm kiếm quyền lực chính trị, dữ liệu và tri thức mà họ đã tích lũy dù sao cũng cho họ rất nhiều quyền lực đối với cuộc sống của ta, phần lớn trong số đó là lén lút, nằm ngoài sự giám sát

của công chúng [106].

Remembering the Industrial Revolution — Nhớ về Cách mạng Công nghiệp

Data is the defining feature of the information age. The internet, data storage, processing, and software-driven automation are having a major impact on the global economy and human society. As our daily lives and social organization have changed in the past decade, and will probably continue to radically change in the coming decades, comparisons to the Industrial Revolution come to mind [87, 96].

Dữ liệu là đặc điểm định danh của thời đại thông tin. Internet, lưu trữ dữ liệu, xử lý, và tự động hóa do phần mềm dẫn dắt đang có một tác động lớn lên nền kinh tế toàn cầu và xã hội loài người. Khi cuộc sống hằng ngày và tổ chức xã hội của ta đã thay đổi trong thập kỷ qua, và có lẽ sẽ tiếp tục thay đổi triệt để trong các thập kỷ tới, những so sánh với Cách mạng Công nghiệp hiện lên trong tâm trí [87, 96].

The Industrial Revolution came about through major technological and agricultural advances, and it brought sustained economic growth and significantly improved living standards in the long run. Yet it also came with major problems: pollution of the air (due to smoke and chemical processes) and the water (from industrial and human waste) was dreadful. Factory owners lived in splendor, while urban workers often lived in very poor housing and worked long hours in harsh conditions. Child labor was common, including dangerous and poorly paid work in mines.

Cách mạng Công nghiệp ra đời thông qua các tiến bộ lớn về công nghệ và nông nghiệp, và nó mang lại sự tăng trưởng kinh tế bền vững cùng các tiêu chuẩn sống được cải thiện đáng kể về lâu dài. Tuy nhiên, nó cũng đi kèm những vấn đề lớn: ô nhiễm không khí (do khói và các quá trình hóa học) và nước (từ chất thải công nghiệp và con người) thật kinh khủng. Các chủ nhà máy sống trong xa hoa, trong khi công nhân thành thị thường sống trong những căn nhà rất tồi tàn và làm việc nhiều giờ trong những điều kiện khắc nghiệt. Lao động trẻ em là chuyện thường, bao gồm cả công việc nguy hiểm và được trả lương kém trong các hầm mỏ.

It took a long time before safeguards were established, such as environmental protection regulations, safety protocols for workplaces, outlawing child labor, and health inspections for food. Undoubtedly the cost of doing business increased when factories could no longer dump their waste into rivers, sell tainted foods, or exploit workers. But society as a whole benefited hugely, and few of us would want to return to a time before those regulations [87].

Đã mất một thời gian dài trước khi các biện pháp bảo vệ được thiết lập, chẳng hạn các quy định bảo vệ môi trường, các quy trình an toàn cho nơi làm việc, việc đặt lao động trẻ em ra ngoài vòng pháp luật, và các cuộc thanh tra y tế đối với thực phẩm. Không nghi ngờ gì, cái giá của việc kinh doanh tăng lên khi các nhà máy không còn có thể đổ chất thải của mình xuống sông, bán thực phẩm nhiễm độc, hoặc bóc lột công nhân. Nhưng xã hội nói chung hưởng lợi to lớn, và ít ai trong chúng ta muốn quay về một thời trước khi có những quy định đó [87].

Just as the Industrial Revolution had a dark side that needed to be managed, our transition to the information age has major problems that we need to confront and solve. I believe that the collection and use of data is one of those problems. In the words of Bruce Schneier [96]:

Cũng như Cách mạng Công nghiệp có một mặt tối cần được quản lý, sự chuyển đổi của ta sang thời đại thông tin có những vấn đề lớn mà ta cần đối mặt và giải quyết. Tôi tin rằng việc thu thập và sử dụng dữ liệu là một trong những vấn đề đó. Theo lời của Bruce Schneier [96]:

Data is the pollution problem of the information age, and protecting privacy is the environmental challenge. Almost all computers produce information. It stays around, festering. How we deal with it—how we contain it and how we dispose of it—is central to the health of our information economy. Just as we look back today at the early decades of the industrial age and wonder how our ancestors could have ignored pollution in their rush to build an industrial world, our grandchildren will look back at us during these early decades of the information age and judge us on how we addressed the challenge of data collection and misuse.

Dữ liệu là vấn đề ô nhiễm của thời đại thông tin, và việc bảo vệ quyền riêng tư là thách thức môi trường. Gần như mọi máy tính đều sản sinh thông tin. Nó nằm lại đó, mưng mủ. Cách ta xử lý nó — cách ta kiềm chế nó và cách ta vứt bỏ nó — là trung tâm của sức khỏe nền kinh tế thông tin của ta. Cũng như hôm nay ta nhìn lại những thập kỷ đầu của thời đại công nghiệp và tự hỏi làm sao tổ tiên ta có thể phớt lờ ô nhiễm trong cơn vội vã xây dựng một thế giới công nghiệp, con cháu ta sẽ nhìn lại ta trong những thập kỷ đầu của thời đại thông tin này và phán xét ta về cách ta đã giải quyết thách thức của việc thu thập và lạm dụng dữ liệu.

We should try to make them proud.

Ta nên cố gắng làm cho họ tự hào.

Legislation and self-regulation — Lập pháp và tự điều tiết

Data protection laws might be able to help preserve individuals' rights. For example, the 1995 European Data Protection Directive states that personal data must be “collected for specified, explicit and legitimate purposes and not further processed in a way incompatible with those purposes,” and furthermore that data must be “adequate, relevant and not excessive in **relation** to the purposes for which they are collected” [107].

Các luật bảo vệ dữ liệu có thể giúp bảo toàn các quyền của cá nhân. Chẳng hạn, Chỉ thị Bảo vệ Dữ liệu Châu Âu năm 1995 nêu rằng dữ liệu cá nhân phải được “thu thập cho các mục đích cụ thể, tường minh và chính đáng và không được xử lý thêm theo một cách không tương thích với các mục đích đó,” và hơn nữa rằng dữ liệu phải “đầy đủ, liên quan và không quá mức so với các mục đích mà chúng được thu thập” [107].

However, it is doubtful whether this legislation is effective in today's internet context [108]. These rules run directly counter to the philosophy of Big Data, which is to maximize data collection, to combine it with other datasets, to experiment and to explore in order to generate new insights. Exploration means using data for unforeseen purposes, which is the opposite of the “specified and explicit” purposes for which the user gave their consent (if we can meaningfully speak of consent at all [109]). Updated regulations are now being developed [89].

Tuy nhiên, đáng ngờ liệu luật pháp này có hiệu quả trong bối cảnh internet ngày nay hay không [108]. Các quy tắc này đi trực tiếp ngược lại triết lý của Dữ liệu lớn, vốn là tối đa hóa việc thu thập dữ liệu, kết hợp nó với các tập dữ liệu khác, thử nghiệm và khám phá để tạo ra những hiểu biết mới. Khám phá nghĩa là dùng dữ liệu cho các mục đích không lường trước, vốn là điều ngược lại với các mục đích “cụ thể và tường minh” mà người dùng đã trao sự đồng thuận của họ (nếu ta có thể nói về sự đồng thuận một cách có ý nghĩa chút nào [109]). Các quy định cập nhật giờ đây đang được phát triển [89].

Companies that collect lots of data about people oppose regulation as being a burden and a hindrance to innovation. To some extent that opposition is justified. For example, when sharing medical data, there are clear risks to privacy, but there are also potential opportunities: how many deaths could be prevented if data analysis was able to help us achieve better diagnostics or find

better treatments [110]? Over-regulation may prevent such breakthroughs. It is difficult to balance such potential opportunities with the risks [105].

Các công ty thu thập nhiều dữ liệu về con người phản đối quy định vì coi đó là một gánh nặng và một cản trở đối với sự đổi mới. Ở một mức độ nào đó, sự phản đối đó là chính đáng. Chẳng hạn, khi chia sẻ dữ liệu y tế, có những rủi ro rõ ràng đối với quyền riêng tư, nhưng cũng có những cơ hội tiềm tàng: có thể ngăn được bao nhiêu cái chết nếu phân tích dữ liệu có thể giúp ta đạt được chẩn đoán tốt hơn hoặc tìm ra các phương pháp điều trị tốt hơn [110]? Quy định quá mức có thể ngăn cản những bước đột phá như vậy. Thật khó để cân bằng những cơ hội tiềm tàng như vậy với các rủi ro [105].

Fundamentally, I think we need a culture shift in the tech industry with regard to personal data. We should stop regarding users as metrics to be optimized, and remember that they are humans who deserve respect, dignity, and agency. We should self-regulate our data collection and processing practices in order to establish and maintain the trust of the people who depend on our software [111]. And we should take it upon ourselves to educate end users about how their data is used, rather than keeping them in the dark.

Về căn bản, tôi cho rằng ta cần một sự dịch chuyển văn hóa trong ngành công nghệ đối với dữ liệu cá nhân. Ta nên thôi coi người dùng như các chỉ số để tối ưu hóa, và nhớ rằng họ là con người xứng đáng được tôn trọng, được hưởng nhân phẩm, và quyền tự chủ. Ta nên tự điều tiết các thực hành thu thập và xử lý dữ liệu của mình để thiết lập và duy trì lòng tin của những người phụ thuộc vào phần mềm của ta [111]. Và ta nên tự nhận lấy việc giáo dục người dùng cuối về cách dữ liệu của họ được dùng, thay vì giữ họ trong bóng tối.

We should allow each individual to maintain their privacy—i.e., their control over own data—and not steal that control from them through surveillance. Our individual right to control our data is like the natural environment of a national park: if we don't explicitly protect and care for it, it will be destroyed. It will be the tragedy of the commons, and we will all be worse off for it. Ubiquitous surveillance is not inevitable—we are still able to stop it.

Ta nên cho phép mỗi cá nhân duy trì quyền riêng tư của họ — tức là quyền kiểm soát của họ đối với dữ liệu của chính mình — và không đánh cắp quyền kiểm soát đó khỏi họ thông qua sự giám sát. Quyền cá nhân của ta trong việc kiểm soát dữ liệu của mình giống như môi trường tự nhiên của một vườn quốc gia: nếu ta không tường minh bảo vệ và chăm sóc nó, nó sẽ bị hủy hoại. Đó sẽ là bi kịch của tài sản chung (tragedy of the commons), và tất cả ta sẽ tệ hơn vì điều đó. Sự giám sát phổ khắp là không phải không thể tránh khỏi — ta vẫn còn có thể ngăn nó lại.

How exactly we might achieve this is an open question. To begin with, we should not retain data forever, but purge it as soon as it is no longer needed [111, 112]. Purging data runs counter to the idea of immutability (see “Limitations of immutability”), but that issue can be solved. A promising approach I see is to enforce access control through cryptographic protocols, rather than merely by policy [113, 114]. Overall, culture and attitude changes will be necessary.

Chính xác ta có thể đạt được điều này như thế nào là một câu hỏi còn bỏ ngỏ. Để bắt đầu, ta không nên giữ dữ liệu mãi mãi, mà thanh lọc nó ngay khi nó không còn cần thiết [111, 112]. Việc thanh lọc dữ liệu đi ngược lại ý tưởng về tính bất biến (xem “Những giới hạn của tính bất biến”), nhưng vấn đề đó có thể được giải quyết. Một cách tiếp cận hứa hẹn mà tôi thấy là thực thi việc kiểm soát truy cập thông qua các giao thức mật mã, thay vì chỉ bằng chính sách [113, 114]. Nhìn chung, các thay đổi về văn hóa và thái độ sẽ là cần thiết.

Summary — Tóm tắt

In this chapter we discussed new approaches to designing data systems, and I included my personal opinions and speculations about the future. We started with the observation that there is no one single tool that can efficiently serve all possible use cases, and so applications necessarily need to compose several different pieces of software to accomplish their goals. We discussed how to solve this data integration problem by using batch processing and event streams to let data changes flow between different systems.

Trong chương này, ta đã bàn về các cách tiếp cận mới để thiết kế các hệ thống dữ liệu, và tôi đã đưa vào những ý kiến và suy đoán cá nhân của mình về tương lai. Ta đã bắt đầu với quan sát rằng không có một công cụ đơn lẻ nào có thể phục vụ hiệu quả mọi tình huống sử dụng khả dĩ, và do đó các ứng dụng nhất thiết cần kết cấu vài phần mềm khác nhau để hoàn thành các mục tiêu của chúng. Ta đã bàn về cách giải quyết bài toán tích hợp dữ liệu này bằng cách dùng xử lý theo lô và các luồng sự kiện để cho các thay đổi dữ liệu chảy giữa các hệ thống khác nhau.

In this approach, certain systems are designated as systems of record, and other data is derived from them through transformations. In this way we can maintain indexes, materialized views, machine learning models, statistical summaries, and more. By making these derivations and transformations asynchronous and loosely coupled, a problem in one area is prevented from spreading to unrelated parts of the system, increasing the robustness and fault-tolerance of the system as a whole.

Trong cách tiếp cận này, một số hệ thống nhất định được chỉ định là hệ thống lưu hồ sơ, và các dữ liệu khác được dẫn xuất từ chúng thông qua các phép biến đổi. Bằng cách này ta có thể duy trì các chỉ mục, khung nhìn vật chất hóa, các mô hình học máy, các tóm tắt thống kê, và nhiều thứ khác. Bằng cách làm cho các phép dẫn xuất và biến đổi này bất đồng bộ và gắn kết lỏng, một vấn đề ở một lĩnh vực được ngăn không lan sang các phần không liên quan của hệ thống, làm tăng sự vững vàng và khả năng chịu lỗi của toàn bộ hệ thống.

Expressing dataflows as transformations from one dataset to another also helps evolve applications: if you want to change one of the processing steps, for example to change the structure of an index or cache, you can just rerun the new transformation code on the whole input dataset in order to rederive the output. Similarly, if something goes wrong, you can fix the code and reprocess the data in order to recover.

Việc biểu đạt các luồng dữ liệu dưới dạng các phép biến đổi từ một tập dữ liệu này sang một tập khác cũng giúp tiến hóa các ứng dụng: nếu bạn muốn thay đổi một trong các bước xử lý, chẳng hạn để thay đổi cấu trúc của một chỉ mục hoặc bộ nhớ đệm, bạn chỉ cần chạy lại mã biến đổi mới trên toàn bộ tập dữ liệu đầu vào để dẫn xuất lại đầu ra. Tương tự, nếu có gì đó trục trặc, bạn có thể sửa mã và xử lý lại dữ liệu để phục hồi.

These processes are quite similar to what databases already do internally, so we recast the idea of dataflow applications as unbundling the components of a database, and building an application by

composing these loosely coupled components.

Các quá trình này khá giống với điều mà các cơ sở dữ liệu vốn đã làm trong nội bộ, nên ta đã đúc lại ý tưởng về các ứng dụng luồng dữ liệu thành việc tháo gó các thành phần của một cơ sở dữ liệu, và xây dựng một ứng dụng bằng cách kết cấu các thành phần gắn kết lỏng này.

Derived state can be updated by observing changes in the underlying data. Moreover, the derived state itself can further be observed by downstream consumers. We can even take this dataflow all the way through to the end-user device that is displaying the data, and thus build user interfaces that dynamically update to reflect data changes and continue to work offline.

Trạng thái dẫn xuất có thể được cập nhật bằng cách quan sát các thay đổi trong dữ liệu nền. Hơn nữa, bản thân trạng thái dẫn xuất có thể được quan sát thêm bởi các bên tiêu thụ xuôi dòng. Ta thậm chí có thể đưa luồng dữ liệu này đi suốt chặng tới thiết bị của người dùng cuối đang hiển thị dữ liệu, và do đó xây dựng các giao diện người dùng tự động cập nhật để phản ánh các thay đổi dữ liệu và tiếp tục hoạt động khi ngoại tuyến.

Next, we discussed how to ensure that all of this processing remains correct in the presence of faults. We saw that strong integrity guarantees can be implemented scalably with asynchronous event processing, by using end-to-end operation identifiers to make operations idempotent and by checking constraints asynchronously. Clients can either wait until the check has passed, or go ahead without waiting but risk having to apologize about a constraint violation. This approach is much more scalable and robust than the traditional approach of using distributed transactions, and fits with how many business processes work in practice.

Tiếp theo, ta đã bàn về cách đảm bảo rằng toàn bộ việc xử lý này vẫn đúng đắn khi có lỗi. Ta đã thấy rằng các đảm bảo mạnh về tính toàn vẹn có thể được hiện thực một cách có khả năng mở rộng bằng việc xử lý sự kiện bất đồng bộ, bằng cách dùng các định danh thao tác đầu-cuối để làm cho các thao tác lũy đẳng và bằng cách kiểm tra các ràng buộc một cách bất đồng bộ. Các client có thể hoặc chờ cho tới khi việc kiểm tra đã qua, hoặc tiến tới mà không chờ nhưng chấp nhận rủi ro phải xin lỗi về một vi phạm ràng buộc. Cách tiếp cận này có khả năng mở rộng và vững vàng hơn nhiều so với cách tiếp cận truyền thống dùng các giao dịch phân tán, và khớp với cách nhiều quy trình nghiệp vụ hoạt động trong thực tế.

By structuring applications around dataflow and checking constraints asynchronously, we can avoid most coordination and create systems that maintain integrity but still perform well, even in geographically distributed scenarios and in the presence of faults. We then talked a little about using audits to verify the integrity of data and detect corruption.

Bằng cách cấu trúc các ứng dụng xoay quanh luồng dữ liệu và kiểm tra các ràng buộc một cách bất đồng bộ, ta có thể tránh được hầu hết sự phối hợp và tạo ra các hệ thống vừa duy trì được tính toàn vẹn vừa vẫn chạy tốt, ngay cả trong các kịch bản phân tán về mặt địa lý và khi có lỗi. Sau đó ta đã nói đôi chút về việc dùng kiểm toán để kiểm chứng tính toàn vẹn của dữ liệu và phát hiện sự hỏng hóc.

Finally, we took a step back and examined some ethical aspects of building **data-intensive** applications. We saw that although data can be used to do good, it can also do significant harm: making justifying decisions that seriously affect people's lives and are difficult to appeal against, leading to discrimination and exploitation, normalizing surveillance, and exposing intimate information. We also run the risk of data breaches, and we may find that a well-intentioned use of data has unintended consequences.

Cuối cùng, ta đã lùi lại một bước và xem xét một số khía cạnh đạo đức của việc xây dựng

các ứng dụng thiên về dữ liệu. Ta đã thấy rằng mặc dù dữ liệu có thể được dùng để làm điều tốt, nó cũng có thể gây hại đáng kể: đưa ra những quyết định ảnh hưởng nghiêm trọng tới cuộc sống của con người và khó kháng cáo, dẫn tới sự phân biệt đối xử và bóc lột, bình thường hóa sự giám sát, và phơi bày thông tin thầm kín. Ta cũng chịu rủi ro về các vụ rò rỉ dữ liệu, và ta có thể thấy rằng một việc sử dụng dữ liệu với thiện ý lại có những hệ quả không chủ ý.

As software and data are having such a large impact on the world, we engineers must remember that we carry a responsibility to work toward the kind of world that we want to live in: a world that treats people with humanity and respect. I hope that we can work together toward that goal.

Vì phần mềm và dữ liệu đang có một tác động lớn đến vậy lên thế giới, chúng ta những kỹ sư phải nhớ rằng ta mang một trách nhiệm là làm việc hướng tới loại thế giới mà ta muốn sống trong đó: một thế giới đối xử với con người bằng tình người và sự tôn trọng. Tôi hy vọng rằng ta có thể cùng nhau làm việc hướng tới mục tiêu đó.

Footnotes Explaining a joke rarely improves it, but I don’t want anyone to feel left out. Here, Church is a reference to the mathematician Alonzo Church, who created the lambda calculus, an early form of computation that is the basis for most functional programming languages. The lambda calculus has no mutable state (i.e., no variables that can be overwritten), so one could say that mutable state is separate from Church’s work.

In the microservices approach, you could avoid the synchronous network request by caching the exchange rate locally in the service that processes the purchase. However, in order to keep that cache fresh, you would need to periodically poll for updated exchange rates, or subscribe to a stream of changes—which is exactly what happens in the dataflow approach.

Less facetiously, the set of distinct search queries with nonempty search results is finite, assuming a finite corpus. However, it would be exponential in the number of terms in the corpus, which is still pretty bad news.

References [1] Rachid Belaid: “Postgres Full-Text Search is Good Enough!,” rachibelaid.com, July 13, 2015.

[2] Philippe Ajoux, Nathan Bronson, Sanjeev Kumar, et al.: “Challenges to Adopting Stronger Consistency at Scale,” at 15th USENIX Workshop on Hot Topics in Operating Systems (HotOS), May 2015.

[3] Pat Helland and Dave Campbell: “Building on Quicksand,” at 4th Biennial Conference on Innovative Data Systems Research (CIDR), January 2009.

[4] Jessica Kerr: “Provenance and Causality in Distributed Systems,” blog.jessitron.com, September 25, 2016.

[5] Kostas Tzoumas: “Batch Is a Special Case of Streaming,” data-artisans.com, September 15, 2015.

[6] Shinji Kim and Robert Blafford: “Stream Windowing Performance Analysis: Concord and Spark Streaming,” concord.io, July 6, 2016.

[7] Jay Kreps: “The Log: What Every Software Engineer Should Know About Real-Time Data’s Unifying Abstraction,” engineering.linkedin.com, December 16, 2013.

[8] Pat Helland: “Life Beyond Distributed Transactions: An Apostate’s Opinion,” at 3rd Biennial Conference on Innovative Data Systems Research (CIDR), January 2007.

[9] “Great Western Railway (1835–1948),” Network Rail Virtual Archive, networkrail.co.uk.

- [10] Jacqueline Xu: “Online Migrations at Scale,” stripe.com, February 2, 2017.
- [11] Molly Bartlett Dishman and Martin Fowler: “Agile Architecture,” at O’Reilly Software Architecture Conference, March 2015.
- [12] Nathan Marz and James Warren: *Big Data: Principles and Best Practices of Scalable Real-Time Data Systems*. Manning, 2015. ISBN: 978-1-617-29034-3
- [13] Oscar Boykin, Sam Ritchie, Ian O’Connell, and Jimmy Lin: “Summingbird: A Framework for Integrating Batch and Online MapReduce Computations,” at 40th International Conference on Very Large Data Bases (VLDB), September 2014.
- [14] Jay Kreps: “Questioning the Lambda Architecture,” oreilly.com, July 2, 2014.
- [15] Raul Castro Fernandez, Peter Pietzuch, Jay Kreps, et al.: “Liquid: Unifying Nearline and Offline Big Data Integration,” at 7th Biennial Conference on Innovative Data Systems Research (CIDR), January 2015.
- [16] Dennis M. Ritchie and Ken Thompson: “The UNIX Time-Sharing System,” *Communications of the ACM*, volume 17, number 7, pages 365–375, July 1974. doi:10.1145/361011.361061
- [17] Eric A. Brewer and Joseph M. Hellerstein: “CS262a: Advanced Topics in Computer Systems,” lecture notes, University of California, Berkeley, cs.berkeley.edu, August 2011.
- [18] Michael Stonebraker: “The Case for Polystores,” wp.sigmod.org, July 13, 2015.
- [19] Jennie Duggan, Aaron J. Elmore, Michael Stonebraker, et al.: “The BigDAWG Polystore System,” *ACM SIGMOD Record*, volume 44, number 2, pages 11–16, June 2015. doi:10.1145/2814710.2814713
- [20] Patrycja Dybka: “Foreign Data Wrappers for PostgreSQL,” vertabelo.com, March 24, 2015.
- [21] David B. Lomet, Alan Fekete, Gerhard Weikum, and Mike Zwilling: “Unbundling Transaction Services in the Cloud,” at 4th Biennial Conference on Innovative Data Systems Research (CIDR), January 2009.
- [22] Martin Kleppmann and Jay Kreps: “Kafka, Samza and the Unix Philosophy of Distributed Data,” *IEEE Data Engineering Bulletin*, volume 38, number 4, pages 4–14, December 2015.
- [23] John Hugg: “Winning Now and in the Future: Where VoltDB Shines,” voltdb.com, March 23, 2016.
- [24] Frank McSherry, Derek G. Murray, Rebecca Isaacs, and Michael Isard: “Differential Dataflow,” at 6th Biennial Conference on Innovative Data Systems Research (CIDR), January 2013.
- [25] Derek G Murray, Frank McSherry, Rebecca Isaacs, et al.: “Naiad: A Timely Dataflow System,” at 24th ACM Symposium on Operating Systems Principles (SOSP), pages 439–455, November 2013. doi:10.1145/2517349.2522738
- [26] Gwen Shapira: “We have a bunch of customers who are implementing ‘database inside-out’ concept and they all ask ‘is anyone else doing it? are we crazy?’” twitter.com, July 28, 2016.
- [27] Martin Kleppmann: “Turning the Database Inside-out with Apache Samza,” at Strange Loop, September 2014.
- [28] Peter Van Roy and Seif Haridi: *Concepts, Techniques, and Models of Computer Programming*. MIT Press, 2004. ISBN: 978-0-262-22069-9
- [29] “Juttle Documentation,” juttle.github.io, 2016.
- [30] Evan Czaplicki and Stephen Chong: “Asynchronous Functional Reactive Programming for GUIs,” at 34th ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI), June 2013. doi:10.1145/2491956.2462161

- [31] Engineer Bainomugisha, Andoni Lombide Carreton, Tom van Cutsem, Stijn Mostinckx, and Wolfgang de Meuter: “A Survey on Reactive Programming,” *ACM Computing Surveys*, volume 45, number 4, pages 1–34, August 2013. doi:10.1145/2501654.2501666
- [32] Peter Alvaro, Neil Conway, Joseph M. Hellerstein, and William R. Marczak: “Consistency Analysis in Bloom: A CALM and Collected Approach,” at 5th Biennial Conference on Innovative Data Systems Research (CIDR), January 2011.
- [33] Felienne Hermans: “Spreadsheets Are Code,” at Code Mesh, November 2015.
- [34] Dan Bricklin and Bob Frankston: “VisiCalc: Information from Its Creators,” danbricklin.com.
- [35] D. Sculley, Gary Holt, Daniel Golovin, et al.: “Machine Learning: The High-Interest Credit Card of Technical Debt,” at NIPS Workshop on Software Engineering for Machine Learning (SE4ML), December 2014.
- [36] Peter Bailis, Alan Fekete, Michael J Franklin, et al.: “Feral Concurrency Control: An Empirical Investigation of Modern Application Integrity,” at ACM International Conference on Management of Data (SIGMOD), June 2015. doi:10.1145/2723372.2737784
- [37] Guy Steele: “Re: Need for Macros (Was Re: Icon),” email to ll1-discuss mailing list, people.csail.mit.edu, December 24, 2001.
- [38] David Gelernter: “Generative Communication in Linda,” *ACM Transactions on Programming Languages and Systems (TOPLAS)*, volume 7, number 1, pages 80–112, January 1985. doi:10.1145/2363.2433
- [39] Patrick Th. Eugster, Pascal A. Felber, Rachid Guerraoui, and Anne-Marie Kermarrec: “The Many Faces of Publish/Subscribe,” *ACM Computing Surveys*, volume 35, number 2, pages 114–131, June 2003. doi:10.1145/857076.857078
- [40] Ben Stopford: “Microservices in a Streaming World,” at QCon London, March 2016.
- [41] Christian Posta: “Why Microservices Should Be Event Driven: Autonomy vs Authority,” blog.christianposta.com, May 27, 2016.
- [42] Alex Feyerke: “Say Hello to Offline First,” hood.ie, November 5, 2013.
- [43] Sebastian Burckhardt, Daan Leijen, Jonathan Protzenko, and Manuel Fähndrich: “Global Sequence Protocol: A Robust Abstraction for Replicated Shared State,” at 29th European Conference on Object-Oriented Programming (ECOOP), July 2015. doi:10.4230/LIPIcs.ECOOP.2015.568
- [44] Mark Soper: “Clearing Up React Data Management Confusion with Flux, Redux, and Relay,” medium.com, December 3, 2015.
- [45] Eno Thereska, Damian Guy, Michael Noll, and Neha Narkhede: “Unifying Stream Processing and Interactive Queries in Apache Kafka,” confluent.io, October 26, 2016.
- [46] Frank McSherry: “Dataflow as Database,” github.com, July 17, 2016.
- [47] Peter Alvaro: “I See What You Mean,” at Strange Loop, September 2015.
- [48] Nathan Marz: “Trident: A High-Level Abstraction for Realtime Computation,” blog.twitter.com, August 2, 2012.
- [49] Edi Bice: “Low Latency Web Scale Fraud Prevention with Apache Samza, Kafka and Friends,” at Merchant Risk Council MRC Vegas Conference, March 2016.
- [50] Charity Majors: “The Accidental DBA,” charity.wtf, October 2, 2016.

- [51] Arthur J. Bernstein, Philip M. Lewis, and Shiyong Lu: “Semantic Conditions for Correctness at Different Isolation Levels,” at 16th International Conference on Data Engineering (ICDE), February 2000. doi:10.1109/ICDE.2000.839387
- [52] Sudhir Jorwekar, Alan Fekete, Krithi Ramamritham, and S. Sudarshan: “Automating the Detection of Snapshot Isolation Anomalies,” at 33rd International Conference on Very Large Data Bases (VLDB), September 2007.
- [53] Kyle Kingsbury: Jepsen blog post series, aphyr.com, 2013–2016.
- [54] Michael Jouravlev: “Redirect After Post,” theserverside.com, August 1, 2004.
- [55] Jerome H. Saltzer, David P. Reed, and David D. Clark: “End-to-End Arguments in System Design,” *ACM Transactions on Computer Systems*, volume 2, number 4, pages 277–288, November 1984. doi:10.1145/357401.357402
- [56] Peter Bailis, Alan Fekete, Michael J. Franklin, et al.: “Coordination-Avoiding Database Systems,” *Proceedings of the VLDB Endowment*, volume 8, number 3, pages 185–196, November 2014.
- [57] Alex Yarmula: “Strong Consistency in Manhattan,” blog.twitter.com, March 17, 2016.
- [58] Douglas B Terry, Marvin M Theimer, Karin Petersen, et al.: “Managing Update Conflicts in Bayou, a Weakly Connected Replicated Storage System,” at 15th ACM Symposium on Operating Systems Principles (SOSP), pages 172–182, December 1995. doi:10.1145/224056.224070
- [59] Jim Gray: “The Transaction Concept: Virtues and Limitations,” at 7th International Conference on Very Large Data Bases (VLDB), September 1981.
- [60] Hector Garcia-Molina and Kenneth Salem: “Sagas,” at ACM International Conference on Management of Data (SIGMOD), May 1987. doi:10.1145/38713.38742
- [61] Pat Helland: “Memories, Guesses, and Apologies,” blogs.msdn.com, May 15, 2007.
- [62] Yoongu Kim, Ross Daly, Jeremie Kim, et al.: “Flipping Bits in Memory Without Accessing Them: An Experimental Study of DRAM Disturbance Errors,” at 41st Annual International Symposium on Computer Architecture (ISCA), June 2014. doi:10.1145/2678373.2665726
- [63] Mark Seaborn and Thomas Dullien: “Exploiting the DRAM Rowhammer Bug to Gain Kernel Privileges,” googleprojectzero.blogspot.co.uk, March 9, 2015.
- [64] Jim N. Gray and Catharine van Ingen: “Empirical Measurements of Disk Failure Rates and Error Rates,” Microsoft Research, MSR-TR-2005-166, December 2005.
- [65] Annamalai Gurusami and Daniel Price: “Bug #73170: Duplicates in Unique Secondary Index Because of Fix of Bug#68021,” bugs.mysql.com, July 2014.
- [66] Gary Fredericks: “Postgres Serializability Bug,” github.com, September 2015.
- [67] Xiao Chen: “HDFS DataNode Scanners and Disk Checker Explained,” blog.cloudera.com, December 20, 2016.
- [68] Jay Kreps: “Getting Real About Distributed System Reliability,” blog.empathybox.com, March 19, 2012.
- [69] Martin Fowler: “The LMAX Architecture,” martinfowler.com, July 12, 2011.
- [70] Sam Stokes: “Move Fast with Confidence,” blog.samstokes.co.uk, July 11, 2016.
- [71] “Sawtooth Lake Documentation,” Intel Corporation, intelledger.github.io, 2016.

- [72] Richard Gendal Brown: “Introducing R3 Corda™: A Distributed Ledger Designed for Financial Services,” gendal.me, April 5, 2016.
- [73] Trent McConaghy, Rodolphe Marques, Andreas Müller, et al.: “BigchainDB: A Scalable Blockchain Database,” bigchaindb.com, June 8, 2016.
- [74] Ralph C. Merkle: “A Digital Signature Based on a Conventional Encryption Function,” at CRYPTO ’87, August 1987. doi:10.1007/3-540-48184-2_32
- [75] Ben Laurie: “Certificate Transparency,” ACM Queue, volume 12, number 8, pages 10-19, August 2014. doi:10.1145/2668152.2668154
- [76] Mark D. Ryan: “Enhanced Certificate Transparency and End-to-End Encrypted Mail,” at Network and Distributed System Security Symposium (NDSS), February 2014. doi:10.14722/ndss.2014.23379
- [77] “Software Engineering Code of Ethics and Professional Practice,” Association for Computing Machinery, acm.org, 1999.
- [78] François Chollet: “Software development is starting to involve important ethical choices,” twitter.com, October 30, 2016.
- [79] Igor Perisic: “Making Hard Choices: The Quest for Ethics in Machine Learning,” engineering.linkedin.com, November 2016.
- [80] John Naughton: “Algorithm Writers Need a Code of Conduct,” theguardian.com, December 6, 2015.
- [81] Logan Kugler: “What Happens When Big Data Blunders?,” Communications of the ACM, volume 59, number 6, pages 15–16, June 2016. doi:10.1145/2911975
- [82] Bill Davidow: “Welcome to Algorithmic Prison,” theatlantic.com, February 20, 2014.
- [83] Don Peck: “They’re Watching You at Work,” theatlantic.com, December 2013.
- [84] Leigh Alexander: “Is an Algorithm Any Less Racist Than a Human?” theguardian.com, August 3, 2016.
- [85] Jesse Emspak: “How a Machine Learns Prejudice,” scientificamerican.com, December 29, 2016.
- [86] Maciej Ceglowski: “The Moral Economy of Tech,” idleness.com, June 2016.
- [87] Cathy O’Neil: Weapons of Math Destruction: How Big Data Increases Inequality and Threatens Democracy. Crown Publishing, 2016. ISBN: 978-0-553-41881-1
- [88] Julia Angwin: “Make Algorithms Accountable,” nytimes.com, August 1, 2016.
- [89] Bryce Goodman and Seth Flaxman: “European Union Regulations on Algorithmic Decision-Making and a ‘Right to Explanation’,” arXiv:1606.08813, August 31, 2016.
- [90] “A Review of the Data Broker Industry: Collection, Use, and Sale of Consumer Data for Marketing Purposes,” Staff Report, United States Senate Committee on Commerce, Science, and Transportation, commerce.senate.gov, December 2013.
- [91] Olivia Solon: “Facebook’s Failure: Did Fake News and Polarized Politics Get Trump Elected?” theguardian.com, November 10, 2016.
- [92] Donella H. Meadows and Diana Wright: Thinking in Systems: A Primer. Chelsea Green Publishing, 2008. ISBN: 978-1-603-58055-7
- [93] Daniel J. Bernstein: “Listening to a ‘big data’/‘data science’ talk,” twitter.com, May 12, 2015.
- [94] Marc Andreessen: “Why Software Is Eating the World,” The Wall Street Journal, 20 August 2011.

- [95] J. M. Porup: “‘Internet of Things’ Security Is Hilariously Broken and Getting Worse,” arstechnica.com, January 23, 2016.
- [96] Bruce Schneier: *Data and Goliath: The Hidden Battles to Collect Your Data and Control Your World*. W. W. Norton, 2015. ISBN: 978-0-393-35217-7
- [97] The Grugq: “Nothing to Hide,” grugq.tumblr.com, April 15, 2016.
- [98] Tony Beltramelli: “Deep-Spying: Spying Using Smartwatch and Deep Learning,” Masters Thesis, IT University of Copenhagen, December 2015. Available at arxiv.org/abs/1512.05616
- [99] Shoshana Zuboff: “Big Other: Surveillance Capitalism and the Prospects of an Information Civilization,” *Journal of Information Technology*, volume 30, number 1, pages 75–89, April 2015. doi:10.1057/jit.2015.5
- [100] Carina C. Zona: “Consequences of an Insightful Algorithm,” at GOTO Berlin, November 2016.
- [101] Bruce Schneier: “Data Is a Toxic Asset, So Why Not Throw It Out?,” schneier.com, March 1, 2016.
- [102] John E. Dunn: “The UK’s 15 Most Infamous Data Breaches,” techworld.com, November 18, 2016.
- [103] Cory Scott: “Data is not toxic - which implies no benefit - but rather hazardous material, where we must balance need vs. want,” twitter.com, March 6, 2016.
- [104] Bruce Schneier: “Mission Creep: When Everything Is Terrorism,” schneier.com, July 16, 2013.
- [105] Lena Ulbricht and Maximilian von Grafenstein: “Big Data: Big Power Shifts?,” *Internet Policy Review*, volume 5, number 1, March 2016. doi:10.14763/2016.1.406
- [106] Ellen P. Goodman and Julia Powles: “Facebook and Google: Most Powerful and Secretive Empires We’ve Ever Known,” theguardian.com, September 28, 2016.
- [107] Directive 95/46/EC on the protection of individuals with regard to the processing of personal data and on the free movement of such data, *Official Journal of the European Communities* No. L 281/31, eur-lex.europa.eu, November 1995.
- [108] Brendan Van Alsenoy: “Regulating Data Protection: The Allocation of Responsibility and Risk Among Actors Involved in Personal Data Processing,” Thesis, KU Leuven Centre for IT and IP Law, August 2016.
- [109] Michiel Rhoen: “Beyond Consent: Improving Data Protection Through Consumer Protection Law,” *Internet Policy Review*, volume 5, number 1, March 2016. doi:10.14763/2016.1.404
- [110] Jessica Leber: “Your Data Footprint Is Affecting Your Life in Ways You Can’t Even Imagine,” fastcoexist.com, March 15, 2016.
- [111] Maciej Cegłowski: “Haunted by Data,” idlewords.com, October 2015.
- [112] Sam Thielman: “You Are Not What You Read: Librarians Purge User Data to Protect Privacy,” theguardian.com, January 13, 2016.
- [113] Conor Friedersdorf: “Edward Snowden’s Other Motive for Leaking,” theatlantic.com, May 13, 2014.
- [114] Phillip Rogaway: “The Moral Character of Cryptographic Work,” *Cryptology ePrint* 2015/1162, December 2015.